

Contents

서문	13
Neural Memory 모노그래프와의관계	13
누구를 위한책인가	13
이 책의 방법	14
v1 을 폐기하고 다시썼다	14
읽는 순서	14
목차	16
앞부분 (front matter)	16
Part I — 배경: 자를만든다	16
Part II — 계보: 판별식을 대본다	17
Part III — 판정	17
뒷부분 (back matter)	18
Part I — 배경: 자를만든다	19
ch01. 오리엔테이션 — 세 층, 세 경로, 그리고 이름의문제	20
01.1 왜 세 층으로나누는가	20
01.2 세 층: Θ , W , E	21
01.3 읽기는 하나, 쓰기는셋	21
01.4 세 경로와판별식	22
01.5 비용 4 종과상각	23
01.6 Rosetta 사전 — NM 사전 위에 얹는행	24
01.7 이 책이 답하는 질문과 답이 놓인자리	24
(state, update, cost) 정리	25
Worked micro-example — 손익분기와 비교불가능성	26
요약	27
자가 점검체크리스트	27
다음 장으로	27
ch02. training 레짐 지도 — 무엇이 언제움직이는가	29
02.1 레짐이라는 단위 — 하나의 checkpoint 가태어나기까지	29
02.2 training run 의 구성요소다섯	30
02.3 예산의 어휘 — token budget · step · batch · epoch	31
02.4 pre-training — Θ 의 처음이자 가장큰이동	32
02.5 SFT — 형식을 고정하는 짧은레짐	33
02.6 RLHF 와 RLVR — 데이터를 run 안에서 만드는데레짐	33
02.7 continual — 배포 이후에도 움직이는 Θ	34
02.8 네 칸 표 — 레짐비교	35
02.9 Part I 의 나머지 장이 채우는칸	36
02.10 “누가 이 값을 학습하는가” —판별습관	36
(state, update, cost) 정리	37

Worked micro-example — 한 sleep 라운드의 값을 손으로매긴다	38
요약	39
자가 점검체크리스트	40
다음 장으로	40
ch03. fine-tuning 과 PEFT — 가중치에 쓰는법	41
03.1 full fine-tuning — 기준선, 그리고 그것이 남기는것	41
03.2 adapter — 삽입의 대가는 depth 이고, depth 의 대가는 decode 다	42
03.3 LoRA — 변화량에 rank 를걸다	44
03.4 어디에 걸 것인가, r 은 얼마인가 —그리고 논문이 자기 표와 어긋나는자리	45
03.5 merge 와 서빙 형태 — $L_w = 0$ 은 무엇과맞바꾼것인가	47
03.6 delta 아티팩트의 크기, 그리고 dtype 이 정하는 정보밀도	48
(state, update, cost) 정리	49
Worked micro-example — delta 아티팩트의 크기를 손으로계산한다	51
요약	52
자가 점검체크리스트	52
다음 장으로	53
ch04. distillation — 한 모델의 앎을 다른 모델로옮기기	54
04.1 gradient 의 출처를 바꾼다 — teacher 와 student	54
04.2 logit distillation 과 temperature	55
04.3 feature distillation — 출력이 아니라 내부를맞춘다	56
04.4 self-distillation — teacher 가 자기 자신일때	57
04.5 upward distillation — 작은 자기에서 큰자기로	58
04.6 distillation loss 의 형태 — 세 좌표로읽는다	60
(state, update, cost) 정리	61
Worked micro-example — temperature 하나가 gradient 와 바이트에하는일	61
요약	63
자가 점검체크리스트	63
다음 장으로	63
ch05. RL for LLM — 보상으로 가중치를움직이기	65
05.1 LLM 위에서 무엇이 state 이고 무엇이 action 이고 무엇이 reward 인가	65
05.2 policy, rollout, advantage — policy gradient 의 최소골격	66
05.3 RLHF 파이프라인 — 채점기를 학습해서 사람을 대신세운다	68
05.4 DPO — reward model 을 지우고 preference 를 직접 loss 로	69
05.5 RLVR — 채점기를 프로그램으로바꾼다	70
05.6 reward hacking — 보상이 목적을 대신할때	71
05.7 이 corpus 가 실제로 쓰는 RL 루프 — 하나는 진짜, 하나는어휘만	72
05.7.1 SEAL — 보상이 “훈련 실행 한 번” 인 RL	72
05.7.2 ReasoningBank — policy 라고 쓰고, gradient 는없다	73
(state, update, cost) 정리	74
Worked micro-example — 한 위치의 확률이 어디로가는가	75
요약	76
자가 점검체크리스트	76
다음 장으로	77
ch06. 데이터를 만드는 법 — augmentation, synthetic, 그리고붕괴	78
06.1 $\mathcal{R}_k$ 는 어디서오는가	78
06.2 LLM 에서 augmentation 은 decode 다	79
06.3 자기 생성이 남의 생성을이기는가	80
06.4 데이터 품질필터링	81

06.5 model collapse — 이름 하나에 현상넷	81
06.6 accumulate vs replace — 스케줄이 수렴을정한다	82
06.7 헤드라인이 성립하지 않는자리	84
06.8 경계의 전제 — 이 결과를 SEAL 과 Dreaming 에 옮길 수있는가	85
06.9 그래서 decode 에서 무엇이바뀌는가	85
(state, update, cost) 정리	86
Worked micro-example — 1 차원 가우시안으로 세 스케줄을 손으로따라가기	87
요약	88
자가 점검체크리스트	89
다음 장으로	89
ch07. 망각과 replay — ρ 의정체	90
07.1 catastrophic forgetting — $(U-\Theta)$ 에 없는항	90
07.2 rehearsal 와 pseudo-rehearsal — 데이터를 되돌려주는 두방법	91
07.3 EWC — 경험을 곡률로압축한다	92
07.3.1 근간 논문에 결과표가 없다는사실	93
07.4 GEM — 과거를 제약으로삼는다	94
07.5 generative replay — 데이터를 생성기로바꾼다	96
07.6 ρ 의 조작적정의	97
(state, update, cost) 정리	98
Worked micro-example	99
요약	100
자가 점검체크리스트	100
다음 장으로	101
ch08. CLS 이론과 생물학적 sleep — 은유의출처	102
08.1 왜 두 시스템인가 — Complementary Learning Systems	102
08.2 sleep 의 두 상태 — SWS 와 REM	103
08.3 생물학적 replay — 데이터 없이 다시 도는것	104
08.3.1 PLOS 실험이 실제로 보인것	104
08.3.2 이 실험에 불리한 사실 — 반드시 함께 읽어야 하는것	105
08.4 dreaming 과 표현학습 — 재생이 아니라생성	106
08.4.1 한 사이클에 세 개의 outer loss	107
08.4.2 무엇이 측정되었고 무엇이손봐졌는가	108
08.5 은유의 이송한계	109
(state, update, cost) 정리	110
Worked micro-example — 밴드 정규화 점수와 상각불능	111
요약	112
자가 점검체크리스트	113
다음 장으로	113
ch09. 기억 용량 — 얼마나 쓸 수있는가	114
09.1 무엇을 재는지 먼저 정한다 — memorization 과 generalization 의분리	114
09.2 bits per parameter — 3.64 는어떻게 측정된값인가	115
09.2.1 이 수는 여섯 가지로인쇄된다	116
09.2.2 이것은 상한이 아니라하한이다	116
09.3 C_{cap} 의 조작적 정의 —용량은 아키텍처만의 성질이아니다	117
09.4 LoRA 의 실측 용량 — 저랭크 delta 는 얼마나담는가	118
09.4.1 이 실측이 뒷받침하지 않는것	119
09.5 memory layer — 용량과 FLOPs 는분리된다	120
09.5.1 이 논문은 sleep-time compute 의 사례가아니다	121
09.6 rate-distortion 관점 — 상한이 정해지면 남는 질문은 “무엇을버릴까” 다	121

09.7 정보 밀도는 dtype 이 정한다 —[X2]	123
(state, update, cost) 정리	124
Worked micro-example — 8B 모델의 $r=16$ LoRA delta 는 몇 bits 인가	124
요약	125
자가 점검체크리스트	126
다음 장으로	126
ch10. 외부 기억과 검색비용	127
10.1 E 는 무엇으로 만들어지는가 —text·vector·graph	127
10.2 RAG — 읽기 경로에 붙은 E	128
10.3 chunking — 저장 단위를 정하는결정	129
10.4 embedding 과 유사도검색	130
10.5 $\text{ret}(E, q)$ 의 비용 — 세 항목으로의분해	131
10.6 학습된 검색기가 항상 이기지않는다	133
(state, update, cost) 정리	133
Worked micro-example — 질의 하나가 지불하는 세항	134
요약	135
자가 점검체크리스트	136
다음 장으로	136
Part II — 계보: 판별식을 대본다	137
ch11. 분기 장 — 판별식과 전수분류	138
11.1 Bridge-in: 자를 만들었으니 대본다	138
11.2 판별식 — 네 조건과 각 조건이 막는것	138
11.2.1 진술	138
11.2.2 적용 절차	138
11.2.3 각 조건을 빼면 무엇이들어오는가	139
11.2.4 판별식이 판정하지 못하는것	139
11.3 전수 분류	140
11.3.1 통과군의모양	142
11.3.2 판별식은 비용을 묻지않는다	142
11.4 이름과 실체가 갈리는 다섯사례	143
11.4.1 SCM — 제목은 consolidation, 가중치는불변	143
11.4.2 Memory Caching — $B_s = 0$, 읽기식을바꾼논문	144
11.4.3 Nested Learning — 연속체는 표기의다리다	144
11.4.4 Memory Layers — 배포 후정지	145
11.4.5 Generative Adapter — 산출물의 모양과 시계가갈린다	145
11.5 경계 사례의 처리규칙	145
11.6 경로 간 침묵지도	146
11.6.1 연대가 강제한 침묵과 선택된침묵	147
11.7 같은 경로 안에서도 창시 논문이 인용되지않는다	147
11.8 이 분류가 Part II 를배치한다	148
요약	149
자가 점검체크리스트	149
다음 장으로	149
ch12. Memorizing Transformers — 외부 기억의초기형	151
12.1 Bridge-in: 전작이 남긴문제	151
12.2 문제의식	151
12.3 Core mechanism (통일표기)	152
12.3.1 표기 대응표	154

12.4 어느 층을 언제쓰는가	154
12.5 비용 4 종	156
12.6 실험과스케일	157
12.7 Systems/serving 함의	158
12.8 한계와 bridge-out	159
요약	160
자가 점검체크리스트	160
ch13. RAG 에서 MemGPT 로 — 검색에서운영체제로	162
13.1 Bridge-in: 전작이 남긴문제	162
13.2 문제의식	162
13.3 Core mechanism (통일표기)	163
13.3.1 표기 대응표	165
13.4 어느 층을 언제쓰는가	166
13.5 비용 4 종	167
13.6 실험과스케일	169
13.6.1 RAG 의 평가설계 —전수	169
13.6.2 MemGPT 의 실험	170
13.7 Systems/serving 함의	171
13.8 한계와 bridge-out	172
요약	174
자가 점검체크리스트	174
ch14. Sleep-time Compute — 이 라인의이름	175
14.1 Bridge-in: 전작이 남긴문제	175
14.2 문제의식	176
14.3 Core mechanism (통일표기)	177
14.3.1 기제의 다섯줄	177
14.3.2 정규형과 어긋나는 지점다섯	177
14.3.3 표기 대응표	178
14.3.4 이 장이 소유하는 두정의	179
14.4 어느 층을 언제쓰는가	180
14.5 비용 4 종	182
14.5.1 B_s 가 없다는것	183
14.5.2 그래서 N_q^* 를 계산할 수없다	183
14.5.3 비용모델이 생성 토큰만센다	183
14.5.4 $\kappa_{\text{cost}} = 10$ 은단언이다	184
14.5.5 네 칸의 공백이 뜻하는것	184
14.6 실험과스케일	184
14.6.1 읽기 전에 알아야 할형식	184
14.6.2 벤치마크는 저자가만들었다	184
14.6.3 논문이 보고하는수치	185
14.6.4 논문 자신이 보고하는 불리한결과	186
14.6.5 이 책이 확인한 불리한사실	187
14.6.6 스케일 상한	188
14.7 Systems/serving 함의	188
14.8 한계와 bridge-out	189
14.8.1 논문 자신이 남긴문제	189
14.8.2 경로 간 인용여부	190
14.8.3 다음 장이 받아가는것	190
요약	191
자가 점검체크리스트	191

ch15. Mem0 · Zep · ReasoningBank · SCM — 프로덕션과 그대가	193
15.1 Bridge-in: 전작이 남긴문제	193
15.2 문제의식	193
15.3 Core mechanism (통일표기)	194
15.3.1 쓰기 연산자의종류	196
15.3.2 시간을 표현하는방식	197
15.3.3 표기 대응표	197
15.4 어느 층을 언제쓰는가	199
15.5 비용 4 종	201
15.6 실험과스케일	202
15.7 Systems/serving 함의	205
15.8 한계와 bridge-out	206
요약	208
자가 점검체크리스트	209
 ch16. Generative Adapter —경첩	 210
16.1 Bridge-in: 전작이 남긴문제	210
16.2 문제의식	211
16.3 Core mechanism (통일표기)	211
16.3.1 표기 대응표	213
16.4 어느 층을 언제쓰는가	214
16.5 비용 4 종	216
16.6 실험과스케일	218
16.7 Systems/serving 함의	221
16.8 한계와 bridge-out	222
요약	223
자가 점검체크리스트	224
 ch17. Do Language Models Need Sleep? — 오프라인재귀	 225
17.1 Bridge-in: 전작이 남긴문제	225
17.2 문제의식	226
17.3 Core mechanism (통일표기)	227
17.3.1 표기 대응표	227
17.3.2 한 pass 안의쓰기	228
17.3.3 이 장이 소유하는 두정의	229
17.4 어느 층을 언제쓰는가	230
17.4.1 판별식을 정면으로 통과한다는것	232
17.5 비용 4 종	232
17.6 실험과스케일	234
17.6.1 프로토콜 — 읽기 전에 알아야 할 것셋	234
17.6.2 Rule-110 — 계산이 병목이라는논증	235
17.6.3 Depo — 수치가 없는절	235
17.6.4 GSM-Infinite —실모델 · 실수치	235
17.6.5 throughput 과 스케일상한	237
17.7 Systems/serving 함의	237
17.8 한계와 bridge-out	238
17.8.1 논문 자신이 남긴문제	238
17.8.2 경로 간 인용여부	239
17.8.3 다음 장이 받아가는것	239
요약	240
자가 점검체크리스트	240

ch18. Nested Learning 과 Memory Caching — 연속체는다리인가	242
18.1 Bridge-in: 전작이 남긴문제	242
18.2 문제의식	243
18.3 Core mechanism (통일표기)	243
18.3.1 Nested Learning — 두 갱신규칙	243
18.3.2 Memory Caching — 읽기식만바뀐다	244
18.3.3 표기 대응표	245
18.4 어느 층을 언제쓰는가	247
18.5 비용 4 종	249
18.6 실험과스케일	251
18.6.1 [NL] 이 보고하는것	251
18.6.2 캡션이 자기 표에 반증되는 자리 — [NL] Table 6	251
18.6.3 [MC] 가 보고하는것	252
18.6.4 캡션이 자기 표에 반증되는 자리 — [MC] Table 5 와그이웃들	252
18.7 Systems/serving 합의	254
18.8 한계와 bridge-out	255
18.8.1 논문 자신이 남긴문제	255
18.8.2 이 책의 판정	255
18.8.3 경로 간 인용여부	255
18.8.4 다음 장이 받아가는것	256
요약	256
자가 점검체크리스트	256
 ch19. LoRA 와 모델 편집 — 가중치에 쓰는 두방식	 258
19.1 Bridge-in: 전작이 남긴문제	258
19.2 문제의식	258
19.3 Core mechanism (통일표기)	259
19.3.1 delta 방식 — 저랭크 부분공간으로의사영	259
19.3.2 직접 편집 — 학습률이 없는쓰기	260
19.3.3 쓸 자리를 정하는계측기	261
19.3.4 표기 대응표	261
19.4 어느 층을 언제쓰는가	263
19.5 비용 4 종	265
19.6 실험과스케일	266
19.6.1 [ROME] — 사실 하나를 넣는 데 드는것	266
19.6.2 [MEMIT] — 편집 용량의 실측상한	267
19.6.3 delta 방식의 실측 — 후속 문헌이 보고한것	267
19.6.4 Θ 층 용량의 실측 공급원 —Memory Layers	268
19.7 Systems/serving 합의	269
19.8 한계와 bridge-out	270
19.8.1 논문 자신이 남긴문제	270
19.8.2 이 책의판정	270
19.8.3 경로 간 인용여부	271
19.8.4 다음 장이 받아가는것	271
요약	272
자가 점검체크리스트	272
 ch20. SEAL — 스스로 학습 데이터를만든다	 273
20.1 Bridge-in: 전작이 남긴문제	273
20.2 문제의식	273
20.3 Core mechanism (통일표기)	274
20.3.1 표기 대응표	275

20.3.2 이 장이 소유하는 두정의	276
20.4 어느 층을 언제쓰는가	277
20.5 비용 4 종	279
20.6 실험과스케일	281
20.7 Systems/serving 함의	283
20.8 한계와 bridge-out	284
요약	285
자가 점검체크리스트	285
ch21. Language Models Need Sleep — 증류와 dreaming	287
21.1 Bridge-in: 전작이 남긴문제	287
21.2 문제의식	288
21.3 Core mechanism (통일표기)	289
21.3.1 wake — 주파수 사다리와 (U-W)	289
21.3.2 sleep — 확장, seeding, 리셋	289
21.3.3 Dreaming	291
21.3.4 표준형 (U- Θ)와의 차이	291
21.3.5 표기 대응표	292
21.4 어느 층을 언제쓰는가	293
21.5 비용 4 종	294
21.6 실험과스케일	296
21.6.1 캡션이 자기 표에 반증되는자리	297
21.6.2 숫자가 없는 실험, 그리고 이름의충돌	297
21.6.3 비교 조건과 기반구조	298
21.7 Systems/serving 함의	299
21.8 한계와 bridge-out	300
21.8.1 논문 자신이 남긴문제	300
21.8.2 이 책의판정	301
21.8.3 경로 간 인용여부	301
21.8.4 다음 장이 받아가는것	302
요약	302
자가 점검체크리스트	303
ch22. 반증 축 — 가중치에 사실을 계속 넣을 수있는가	304
22.1 Bridge-in: 전작이 남긴문제	304
22.2 문제의식	304
22.3 Core mechanism (통일표기)	305
22.3.1 표기대응표	306
22.4 어느 층을 언제쓰는가	307
22.5 비용 4 종	308
22.6 실험과스케일	309
22.6.1 한 번의 쓰기는 무엇을만드는가	309
22.6.2 쓴 것은살아남는가	310
22.6.3 잊힌 사실은 어디에있는가	311
22.6.4 쓰기의 대가는 무엇으로측정되는가	311
22.6.5 이 결과가 서 있는바닥	312
22.7 Systems/serving 함의	313
22.8 한계와 bridge-out	313
22.8.1 지속 주입의 실패양상	313
22.8.2 이송된 불리 사실 — 결함의유형학	314
22.8.3 용량·망각·붕괴가 함께 거는제약	317
22.8.4 논문의 open questions 와 살아남기 위한조건	317

22.8.5 경로 간 인용여부	318
요약	318
자가 점검체크리스트	319
ch23. 생물학이 실제로 보인것	320
23.1 Bridge-in: 전작이 남긴문제	320
23.2 문제의식	321
23.3 Core mechanism (통일표기)	321
23.3.1 PLOS — (U- Θ) 를 두 시계로 두번	321
23.3.2 PAD — (U-E) 한 번 뒤 (U- Θ) 세번	322
23.3.3 표기대응표	323
23.4 어느 층을 언제쓰는가	324
23.5 비용 4 종	326
23.6 실험과스케일	327
23.6.1 PLOS — 조건전량	327
23.6.2 PAD — 절제와통제	329
23.7 Systems/serving 함의	330
23.8 한계와 bridge-out	331
요약	333
자가 점검체크리스트	333
Part III — 판정	334
ch24. 무엇이 실제로 sleep-time compute 인가	335
24.1 판정할 질문	335
24.2 증거	335
24.3 분석	340
24.3.1 간격은 우연이아니다	340
24.3.2 간격이 실무에 하는일	341
24.3.3 판별식 없이는 “유망한가” 에 답할 수없다	342
24.4 판정	343
24.5 반증 조건	344
요약	344
다음 장으로	345
ch25. 통일 비용회계	346
25.1 판정할 질문	346
25.2 증거	346
25.2.1 비용 4 종의 corpus 전수집계	346
25.2.2 손익분기 N_q^* 가 계산되는논문은 0 편이다	349
25.2.3 이 책의 통일 비용 회계 (실험 X1)	351
25.2.4 두 개의 파생량 — 임계 길이와손익분기	352
25.3 분석	353
25.3.1 빠진 항은 우연이 아니라구조다	353
25.3.2 레짐 간 순서 — 이 실험이 단정하는것	354
25.3.3 손익분기가 실제로 말하는것	355
25.3.4 κ_{cost} — 회계전체를 지탱하는 미측정상수	355
25.3.5 $5\times$ 는 반박되지않는다	355
25.3.6 같은 자를 세 경로에 대면 자가부러진다	356
25.4 판정	356
25.5 반증 조건	357
요약	358

다음 장으로	359
ch26. 상태 용량과 정보밀도	360
26.1 판정할 질문	360
26.2 증거	360
26.3 분석	362
26.3.1 세 경로의 상태를 하나의 표에놓는다	362
26.3.2 자릿수가 벌어지는 이유는 세 경로가 서로 다른 변수를타기때문이다	363
26.3.3 그 바이트에 무엇이 담기는가 — 정보 밀도는 dtype 이정한다	364
26.3.4 이 책 자신의 정정 — 무엇이 죽고 무엇이살아남았는가	365
26.3.5 사용자 수를 곱하면 어느 계층이 문제가되는가	365
26.3.6 직접 편집 대 delta — 서빙 갈림길을 여기서확정한다	366
26.4 판정	367
26.5 반증 조건	369
요약	370
다음 장으로	370
ch27. 유망한가 — 경로별판정	371
27.1 판정할 질문	371
27.2 증거	371
27.2.1 E -경로 평가 결함의 유형별목록	375
27.3 분석	380
27.3.1 Q1 — 세 경로는 같은 문제를 풀지않는다	380
27.3.2 E -경로 — 주장의 종류가 바뀌어있다	380
27.3.3 W -경로 — 표본 1 의판정	381
27.3.4 Θ -경로 — 용량은 있고 반복이없다	382
27.3.5 Q3·Q4 — 최선인가, 대안대비우월한가	383
27.3.6 Q5 — 주류가 될까: 조건으로만답한다	385
27.4 판정	386
27.5 반증 조건	387
요약	389
다음 장으로	389
ch28. scaling law — B_s 를축으로	391
28.1 판정할 질문	391
28.2 증거	391
28.2.1 B_s 는 이 corpus 에서 축이 아니라각주다	393
28.2.2 눈금을 공통 단위로 옮길 다리가없다	394
28.2.3 축 위에 점이 여럿인 논문과 그 y 축	394
28.2.4 이송된 사실넷	395
28.3 분석	395
28.3.1 (N, D) 와 B_s 는 같은 종류의축이아니다	395
28.3.2 x 축이 여섯 단위로 흩어져 있고 옮길 다리가없다	397
28.3.3 y 축도 함께 깨져있다	397
28.3.4 이름을 만든 논문에 분자가 없다 — 그리고 그것은 관행의문제다	397
28.3.5 그래도 말할 수 있는 것 — 부호와 포화의귀속	398
28.3.6 이 책 자신의 가정에 대한감사	399
28.3.7 최소 보고 요구 R1 – R9	399
28.4 판정	401
28.5 반증 조건	402
요약	403
다음 장으로	404

ch29. system·infra 와 memory-device 기회	405
29.1 판정할 질문	405
29.2 증거	405
29.3 분석	407
29.3.1 기회의 좌표를 셋으로 못 박는다 — 총량이 아니라 입도·마감·주기다	407
29.3.2 이 자리는 Neural Memory 가 짙은 자리와 다르다	409
29.3.3 shared-weight batching 은 경로별로 다른 자리에서 깨진다	409
29.3.4 delta 를 요청 지연 안에 넣는 방식은 셋이고, 셋이 서로 다른 계층을 산다	410
29.3.5 반복이 이 기회의 시간 지평을 정한다	411
29.3.6 양자화는 이 기회의 전제이고, 그 전제가 아직 측정되지 않았다	412
29.3.7 sleep 라운드 자체가 하나의 워크로드다 — 그리고 아무도 그것을 스케줄링으로 다루지 않았다	413
29.3.8 세 경로는 한 배포에서 세 계층에 동시에 요구를 건다	414
29.4 판정	415
29.5 반증 조건	416
요약	418
다음 장으로	418
ch30. 한계와 반증조건	419
30.1 판정할 질문	419
30.2 증거	419
30.3 분석	420
30.3.1 GPU 실측이 없다는 것이 실제로 무엇을 제한하는가	420
30.3.2 X2 초판의 오류 — 하한을 상한으로	421
30.3.3 판별식은 이 책의 선택이다 — 다르게 그으면 무엇이 달라지는가	421
30.3.4 corpus 는 2026-08 의 것이다 — 어떤 논문이 나오면 무엇이 바뀌는가	422
30.3.5 이 책이 지지 않은 점	423
30.4 판정	424
30.5 반증 조건	425
요약	427
다음 장으로	428
참고문헌	429
수록 원칙	429
90.1 corpus — A 군: 판별식 통과 (9 편) + 경계 사례 (1 편)	429
90.2 corpus — B 군: 판별식 불충족 (9 편)	430
90.3 corpus — C 군: 판별식의 대상이 아닌 배경·제약 문헌 (10 편)	431
90.4 이 책이 근거로 삼는 이 책 자신의 산출물	432
90.5 본문이 이름으로만 지목하는 문헌	432
90.6 담당 장역색인	433
용어집	435
읽는 법	435
91.1 기호 — 층과 상태	435
91.2 기호 — 읽기·쓰기와 절차	436
91.3 기호 — 비용 4 종과 상각	437
91.4 기호 — 첨자·차원·그 밖의 예약 기호	439
91.5 기호 충돌 — 반드시 구분해야 하는 세 쌍	440
91.6 용어 — 프레임	440
91.7 용어 — 판별과 분류	441
91.8 용어 — training 실무 (Part I)	442
91.9 용어 — 정직성장치	443

91.10 확정명칭	444
91.11 소유 장역색인	444
부록 A — 판별식 전수분류표	446
A.1 이 부록의용도	446
A.2 판별식과 표기규약	446
A.3 표 A-1 — 전수표 (29 편)	446
A.4 표 A-2 — A 군상세	449
A.5 표 A-3 — B 군상세	450
A.6 표 A-4 — C 군상세	451
A.7 표 A-5 — 집계	452
A.8 표 A-6 — 담당 장에서찾기	452
A.9 이 표의 반증조건	453

서문

이 책은 하나의 질문에서 출발했다 — **sleep-time compute** 은유망한가. 모델이 질의를 기다리는 동안 미리 계산해 두면 나중에 싸지거나 좋아진다는 발상은 서버 엔지니어에게 즉시 매력적으로 들린다. 유휴 자원은 이미 거기 있고, 비용은 밤에 옮겨 두면 되며, wake 경로의 SLA는 손대지 않아도 된다. 그런데 그 발상으로 검색되는 문헌을 모아 놓고 실제로 답하려하면 질문이 손에 잡히지 않는다. 어떤 논문은 대화 요약을 벡터 DB에 넣고, 어떤 논문은 배포 가중치에 저랭크 expert를 덧붙이며, 어떤 논문은 KV cache를 비우기 전에 recurrence를 몇 번 더 돌린다. 셋 모두 자기 절차를 “sleep”이라 부른다. 비용을 바이트로 보고한 논문은 이 책의 corpus에 없고, FLOPs로 보고한 논문도 사실상 없다. 이름은 하나인데 대상이 셋이고, 셋을 같은 자로 댈 사람이 없다.

그래서 이 책은 질문에 곧장 답하지 않는다. 답할 수 있는 형태로 질문을 바꾸는 일을 먼저 한다. 무엇이 실제로 이 범주에 속하는지 판별하고, 속하는 것들의 비용을 하나의 회계로 옮긴 다음, 그 위에서 판정한다. 그 세 단계를 마치면 원래 질문의 답은 “유망하다” 도 “아니다” 도 아니라 **경로마다 다르다**가 된다. 그 답과 답에 이르는 절차가 이 책의 내용이다.

Neural Memory 모노그래프와의 관계

이 책은 두 번째 모노그래프다. 첫 번째는 Google Research의 test-time memorization 계열 여섯 편을 하나의 연구 프로그램으로 읽은 Neural Memory 모노그래프이며, 그 책의 척추는 fast weights W 의 갱신식 하나였다. 이 책은 그 식을 **글자 그대로 승계한다**. 이 책의 (U-W)는 Neural Memory의 master equation (M)과 같은 식이고, $\Theta \cdot W \cdot S_t \cdot \eta_t \cdot \beta_t \cdot \alpha_t$ 의 의미도 그대로다. 재정의하지 않는다.

달라지는 것은 시야다. Neural Memory는 세 층 중 W 하나만 다뤘다. 이 책은 그 위 (Θ , slow weights)와 밖 (E , external store)으로 확장하고, 갱신의 시계를 wake에서 오프라인으로 옮긴다. 그래서 두 책은 겹치지 않고 이어진다 — Neural Memory가 “추론 중 weights는 변하지 않는다”는 불변식의 소멸을 다뤘다면, 이 책은 그 소멸 이후에 **누가 언제 무엇을 쓰는가**를 다룬다. 두 책의 결론을 억지로 하나로 잇지 않는다는 규율도 승계한다. 이 책이 memory-centric 논증을 펴는 자리는 ch29의 한지점뿐이고, 그 지점은 Neural Memory가 짙은 HBM read-modify-write 기회와 다른 자리다.

누구를 위한 책인가

이 책은 **Neural Memory**를 읽은 **한 명의 엔지니어**를 상정한다. 그 독자는 transformer inference와 efficient-transformer를 운영 감각으로 알고 — KV cache, paged attention, FlashAttention tiling, GEMM shape, MFU, roofline, prefill/decode 비대칭, continuous batching — 앞 책을 거쳤으므로 backward pass, optimizer state, inner/outer loop, test-time learning도 이미 안다. 그 부분은 여기서 전제하고 참조로 넘긴다 (“→ NM ch12”).

그러나 그 독자는 **training 실무**를 모른다. fine-tuning 레짐, LoRA와 PEFT, distillation, RLHF와 RLVR, synthetic data 생성, replay buffer, catastrophic forgetting 대응, 지속학습 평가 — 이 어휘는 독자의 것이 아니다. 그런데 Θ -경로의 논문들은 이 어휘 위에서 있다. Part I이 정확히 그 공백을 메운다. 그리고 이 독자의 1번 질문은 항상 같다: “그래서 그게 **decode**에서 뭘 바꾸는데?” 모든 장이 그 질문에 답하는 절을 갖는다.

이 책의 방법

네 가지 장치가 이 책을 지탱한다. 넷 다 앞 책에서 부족했거나 없었던것이다.

판별식. 어떤 절차가 sleep-time compute 인지를 네 조건 — 질의도착 전에, $B_s > 0$ 의 예산을 써서, $\Theta \cdot W \cdot E$ 중 어느 한 층의상태를 실제로 바꾸고, 그 변화가 이후 질의에 쓰이는가 — 으로판정한다. 판별식은 논문의 자기 규정도 성능도 묻지 않는다. 보는 것은갱신식과 시계의 쌍뿐이다. ch01 이 도구를 주고 ch11 이 corpus 전체에기계적으로 적용한다.

통일 회계. 모든 기여를 네 값으로 환산한다: B_s (sleep 예산)· L_w (wake 지연기여)· C_{cap} (상태용량)· ρ (망각·열화율). 상각은 언제나 식 (A) 하나로 쓰고, 손익분기 N_q^* 를 계산할 수 있으면 계산한다. 논문에값이 없으면 “**논문에 없음**” 이라고 쓴다. 추정치를 본문 수치로올리지 않는다.

정직성 규칙. 논문에 불리한 사실의 완곡화·누락을결함으로 취급한다. 자기 ablation 표가 반증하는 캡션 주장, baseline 이이기는 수치, 비용 침묵을 성능 서사로 메우는 서술 — 세 유형은 명시적으로금지되고, 해당 사실은 반드시 그 장 본문에 들어간다. 서술의 층위도 표기로가른다: **논문의 주장**은 위치와 함께 논문에 귀속하고, **이책의 해설**은 [해설] 블록, **이 책의 평가**는 [평가] 블록으로 표시한다. 독자가 어디까지가 논문이고 어디부터가 이 책인지 매문단에서 알 수 있게 하는 것이 이 규칙의 목적이다.

자기 실험의 등급화. Part III 는 네 개의 자체 실험 (X1–X4) 위에선다. 넷 다 GPU 측정이 아니다 — 둘은 결정론적 비용·용량회계이고, 하나는 corpus 보고 실태의 전수 감사이며, 하나는 고정 시드의소규모 시뮬레이션이다. 그래서 본문 단정문으로 승격되는 것은**비용·crossover 위치·순서·bound 분류**뿐이고, 절대치는 하한또는 방향성으로만 쓴다. 각 실험은 자기가 무엇을 반박하지 않는지를 스스로 적어 두었고, 본문은 그 계약을 넘지 않는다. 이등급 규칙도 Neural Memory 에서 승계했다.

v1 을 폐기하고 다시 썼다

이 책에는 폐기된 이전 판이 있다. v1 은 70 쪽 Study, 12 쪽 학회형 원고, 57 쪽 training 배경, 62 쪽 쉬운 해설, 112 장 슬라이드를 냈고 증거 수집도성실했다 — 73 항목 소스 등록부와 일곱 건의 원문 감사가 있었다. 그런데도폐기한 이유는 분량이 아니라 **논지 전개 장치의 부재**였다. 통일표기가 없어 논문마다 원문 기호를 그대로 썼고, 논문별 장이 하나도 없어게보 전체가 한 절에 눌러 있었으며, 장과 장을 잇는 bridge 가 없어 논문들이서로 독립된 나열로 놓였다. 논문 주장과 저자의 해설과 저자의 평가가표기로 구분되지 않았고, 논문에 불리한 사실이 빠져 있었으며, 결론부의전략 서술에는 근거가 붙어 있지 않았다. 요컨대 v1 은 자료였지 논지가아니었다. 고쳐 쓰는 것보다 프레임을 먼저 세우고 처음부터 다시 쓰는 편이싸다고 판단했고, 이 책의 3 층·3 경로·비용 4 종·판별식·8 절 논문장 템플릿은 전부 v1 의 그여섯 결함을 각각 막기 위해 도입된 장치다. v1 의 증거는 재사용했으나서술은 한 줄도 옮기지 않았다.

읽는 순서

책은 세 Part 다. **Part I(ch01–ch10)** 은 자를 만든다 — 세 층, 세갱신식, 비용 4 종, 그리고 Θ -경로를 읽는 데 필요한 training 실무. **Part II(ch11–ch23)** 는 판별식을 corpus 에 대고 논문을 한 편씩해부한다. 아무것도 판정하지 않는다. **Part III(ch24–ch30)** 가판정한다.

정도는 순서대로 읽는 것이다. 그러나 급한 독자에게는 경로가 있다.

- **ch01 은 건너뛸 수 없다.** 이 책의 모든기호·식·판정 도구가 여기서 나온다. 뒤의스물아홉 장은 ch01 의 어휘로 쓰였다.
- **training 실무**를 이미 아는 독자는 ch02–ch06 을 훑고 지나가도된다 (각 장 요약과 (state, update, cost) 표만 확인). 단 **ch07(망각과 replay)** 과 **ch09(기억 용량)** 는 건너뛰지 않는다 —Part III 의 ρ 와 C_{cap} 이 이 두 장 위에서 있다.
- **결론부터 보려는 독자**의 최단 경로는 ch01 → ch11(판별식 전수적용) → ch24(무엇이 실제로 이 범주인가) → ch25(통일 회계) → ch27(경로별 판정) → ch30(반증 조건) 이다. 여섯 장이 이 책의 논지골격이다.

- **경로 하나만 필요한 독자**는 Part II 를 세로로 읽는다. E -경로는 ch12·ch13·ch14·ch15, W -경로는 ch16·ch17·ch18, Θ -경로는 ch19·ch20·ch21 이다. ch16 은경첩 장이라 어느 세로 줄을 읽든 지난다.
- **ch08 과 ch23 은 짝이다.** 생물학이 은유로 무엇을 빌려주었는지가 ch08, 그 생물학 논문들이 실제로 무엇을 보였는지가 ch23 이다. 둘을 붙여읽으면 이 라인의 이름이 어디서 왔고 무엇을 담보하지 않는지가 한 번에보인다.

각 장은 목표 상자로 열고, Part I 장은 손으로 따라갈 수치 예제와 자가점검 체크리스트로 닫는다. Part II 장은 여덟 절 고정 구조를, Part III 장은 다섯 절 판정 구조를 따르며 마지막 절은 언제나 반증 조건이다. 장참조는 “ch14” , 절 참조는 “§ 14.3” , 앞 책 참조는 “→ NM ch12” 로 쓴다. 각주는 쓰지 않는다.

저자: 이승호 (Seungho “Jimmy” Lee). 판본: v2 조립본, 2026-08-06. 이 책의 통일 표기 · 용어 기준 v0.3 을 따른다 — 그기준의 요약이 용어집이다.

목차

“sleep-time compute” 이라는 이름 아래 모인 문헌을 하나의 판별식으로 걸러내고, 남은 세 경로 ($E \cdot W \cdot \Theta$) 를하나의 비용 회계로 옮긴 뒤, 그 위에서 경로별로 판정한다. 책추 논지는이름이 기제보다 넓다이며 정식 판정은 ch24 와 ch27 이 낸다. 페이지번호는 빌드가 채운다.

앞부분 (front matter)

- 서문 — 이 책이 왜 있는가, Neural Memory 모노그래프와의 관계, 누구를 위한 책인가, 이 책의 방법 네 가지 (판별식 · 통일회계 · 정직성 규칙 · 자기 실험의 등급화), v1 을폐기한 이유, 읽는 순서.
- 목차 — 이 문서.
- Part I 도입 — 자를 만드는 열 장이 어떤 순서로 놓이는가.
- Part II 도입 — 판별식을 corpus 에 대는 열세 장의 8 절 격자와경로별 서사.
- Part III 도입 — 판정 일곱 장의 전개와 증거 등급 계약.

Part I — 배경: 자를만든다

세 층 · 세 갱신식 · 비용 4 종을 세우고, Θ -경로를 읽는 데 필요한 training 실무를 독자의 inference 어휘로건설한다. 모든 개념은 (state, update, cost) 객체로 도입되고, 각 장은 worked micro-example 로 달는다.

- ch01 · 오리엔테이션 — 세 층, 세 경로, 그리고이름의 문제 — $\Theta \cdot W \cdot E$ 와 식 $(R) \cdot (U \cdot W) \cdot (U \cdot \Theta) \cdot (U \cdot E) \cdot (A)$, 판별식, 비용 4 종, Rosetta 확장분, 이 책이 답하는 여덟 질문의 지도.
- ch02 · training 레짐 지도 — pre-train · SFT · RL · continual 을 “무엇이 언제 움직이는가” 로 정의하고 $(U \cdot \Theta)$ 의 어느 인자를채우는지 대응시킨다.
- ch03 · fine-tuning 과 PEFT — full FT · adapter · LoRA · QLoRA 를 Θ 의 부분 갱신으로 도입한다. rank r 과 적용 행렬 집합 S 가 여기서 정의된다.
- ch04 · distillation — logit · feature · self-distillation 과 upward distillation. sleep 라운드의 교사가 누구인가라는 질문의 출처.
- ch05 · RL for LLM — RLHF · DPO · RLVR 과 reward hacking. 보상으로 Θ 를 옮기는 경로와 그 검증 비용.
- ch06 · 데이터를 만드는 법 — synthetic 과 붕괴 — $\mathcal{R}_k$ 를 만드는 $\text{gen}(\cdot; B_s)$ 의 실체, 자기생성 데이터의 제약과 model collapse.
- ch07 · 망각과 replay — catastrophic forgetting, rehearsal, EWC, GEM, 성능 행렬 Π 와 replay 혼합비 ω . ρ 의 정체가 여기서 정해진다.
- ch08 · CLS 이론과 생물학적 sleep — 왜 두시스템인가, 왜 오프라인인가. 이 라인이 빌려 쓴 은유의 출처 (결산은 ch23).
- ch09 · 기억 용량 — bits-per-parameter, memory layer 의 용량 실측, 저랭크 쓰기의 용량. C_{cap} 의 상한.
- ch10 · 외부 기억과 검색 비용 — RAG, vector · graph store, $\text{ret}(E, q)$ 의 비용 분해. E 층의 형태와 값.

Part II — 계보: 판별식을 대본다

각 장은 8 절 고정 구조를 따른다: bridge-in(전작이 남긴 open question) → 문제의식 → 통일 표기 core mechanism 과 대응표 → 어느 층을 언제 쓰는가 → 비용 4 종 → 실험과 스케일 → systems/serving 함의 → 한계와 bridge-out. 이 부는 아무것도 판정하지 않는다.

- **ch11 · 분기 장 — 판별식과 전수 분류** — 네 조건과다섯 단계 적용 절차, corpus 전수분류 (통과 · 불충족 · 배경), 이름과 기제의불일치, 경로 간 침묵 지도, 강제된 침묵과 선택된 침묵의 구분.
- **ch12 · Memorizing Transformers** — E 층의초기형. 학습 중 캐시 적재라는 쓰기와 그 한계.
- **ch13 · RAG 에서 MemGPT 로** — $\hat{c}$ 가항등사상인 지점에서 벗어나는 과정, 그리고 MemGPT 가 판별식 (1) 을만족하지 않는다는 사실.
- **ch14 · Letta Sleep-time Compute** — 이라인의 이름을 만든 논문. learned context, 상속 없는 bridge-in, 자체제작 벤치, 높은 예산에서의 이득 역전.
- **ch15 · Mem0·Zep·ReasoningBank·SCM** —프로덕션 E -경로 네 시스템의 wr 형태 비교, 그리고 품질 1 위가 반복적으로 “기억 없음” 이라는 사실.
- **ch16 · Generative Adapter** — 경첩 — 산출물은 Θ 모양인데 갱신식은 (U-W) 이고 시계는 wake 다. 분류가 모양이아니라 (갱신식, 시계) 의 쌍으로 이루어지는 이유.
- **ch17 · Do Language Models Need Sleep?** — W -경로의 유일한 정면 사례. KV cache 를 비우기 전 recurrence 를오프라인에 추가로 도는 절차.
- **ch18 · Nested Learning 과 Memory Caching** —연속체는 다리인가 — 갱신 주파수 연속체가 표기의 다리인지 기제의다리인지 판정하고, W 층 용량 천장을 읽는다.
- **ch19 · LoRA 와 모델 편집** — Θ 에 쓰는 두 primitive(저랭크 증분과 직접 편집) 와 각각의 상한. ROME 자신의 유보를포함한다.
- **ch20 · SEAL** — 모델이 자기 학습 데이터를 만들어 Θ 를 갱신하는 두 시간척도 절차.
- **ch21 · Language Models Need Sleep** —wake W → sleep Θ 의 순서, Knowledge Seeding 과 Dreaming. corpus 에서 bridge-in 이 교과서적으로 성립하는 거의 유일한 사례.
- **ch22 · 반증 축 — 가중치에 사실을 계속 넣을 수있는가** — Θ -경로의 falsifier 후보와 rate – distortion 제약, 그리고 Part I 에서 이송된 방법론 결함들.
- **ch23 · 생물학이 실제로 보인 것** — 은유의 담보를확인한다. sleep 실험과 dreaming 실험이 통제군 대비 실제로 보인 것과보이지 않은 것.

Part III — 판정

Part II 가 확립한 사실과 이 책의 실험 (X1 – X4) 위에서 결론을 낸다. 각 장은판정할 질문 → 증거 → 분석 → 판정 → 반증 조건의 다섯 절이며, 마지막 절은생략되지 않는다.

- **ch24 · 무엇이 실제로 sleep-time compute 인가** —3 분할의 확정과, 이름과 기제의 간격이 일시적 지체인가 구조인가에 대한판정.
- **ch25 · 통일 비용 회계** — 세 경로를 하나의 축에올린다. 상각식 (A) 에서 빠진 항, κ 민감도, 레짐 간 순서 (X1).
- **ch26 · 상태 용량과 정보 밀도** — 경로 간 자릿수, dtype 이 정하는 정보 밀도, 사용자 수에 대한 스케일링 (X2). corpus 에서바이트로 보고한 논문이 0 편인 자리.
- **ch27 · 유망한가 — 경로별 판정** — 원 질문에답한다. 경로마다 다른 답과 그 조건, 그리고 E -경로 시스템 평가결함의 묶음 표.
- **ch28 · scaling law — B_s 를 축으로** — B_s 를 독립변수로 놓은 점들을 통일 축에 재배치하고, 보고 실태 전수감사로 law 의 성립 가능성을 판정한다 (X3).
- **ch29 · system·infra 와 memory-device 기회** — 배포 형태의 변화, shared-weight batching 이 깨지는 지점, 월계층 delta 스테이징. memory-centric 논증을 펴는 유일한 자리.

- **ch30 · 한계와 반증 조건** — 이 책의 판정을 뒤집을관측을 구체적으로 적는다. 반복 consolidation의 열화율 (X4) 과 남은 공백.

뒷부분 (back matter)

- **부록 A · 판별식 전수 분류표** — corpus 29 편의통과 · 경계 · 불충족 · 배경 판정과 각판정이 걸린 조건.
- **부록 B · 표기 대응 총람** — 각 논문의 원 표기 → 이책의 기호. 원문 수식 번호 병기.
- **부록 C · 의무 caveat 총람** — 논문에 불리한 사실의전체 목록과 그것을 담는 장.
- **부록 D · 실험 X1 - X4 재현** — 설정 · 상수 출처 · 고정 시드 · 등급계약과, 각 실험이 반박하지 않는 것.
- **참고문헌** — vendored 원문 위치 기준.
- **용어 색인 (glossary)** — 각 개념의 정의 소유 장으로 연결.

Part I — 배경: 자를 만든다

Part I 이 답하는 질문은 하나다. 이 **corpus**의 논문을 읽고 “어느층을, 어느 시간척도로, 무슨 규칙으로, 얼마의 비용에 쓰는가”를 채우려면무엇을 알아야 하는가. 논문들은 그 답을 전제한다. 어떤 논문은 독자가 replay buffer와 catastrophic forgetting을 안다고 가정하고, 어떤 논문은 LoRA rank와 checkpoint 회계를 안다고 가정하며, 어떤 논문은 bits-per-parameter라는 용량 단위를 설명 없이 쓴다. 이 책의 독자는 Neural Memory를 거쳤으므로 backward pass와 inner/outer loop는 이미알지만, 그 위의 training실무는 모른다. Part I은 그 공백만 정확히 메운다—training을 처음부터 가르치지 않고, 이 **corpus**를 읽는 데 필요한만큼만 짓는다.

배치는 세 층 프레임이 결정한다. **ch01**이 자를 만든다: 상태를 $\Theta \cdot W \cdot E$ 로 가르고, 쓰기를 $(U-W) \cdot (U-\Theta) \cdot (U-E)$ 로 가르고, 비용을 $B_s \cdot L_w \cdot C_{cap} \cdot \rho$ 로 가르고, 판별식과 상각식(A)를 손에 쥐여 준다. 뒤의 스몰아홉 장은 전부 이장의 어휘로 쓰였다.

그다음 다섯 장은 $(U-\Theta)$ 의 **우변을 연다**. **ch01**은 그 식을적기만 했고 우변은 블랙박스로 남겨 두었다. **ch02**가 pre-train·SFT·RL·continual의 네레짐을 “무엇이 언제 움직이는가”로 정의해 지도를 그리고, **ch03**이 Θ 를 부분적으로 갱신하는 도구(adapter·LoRA)를, **ch04**가 교사에게서 학생으로 옮기는 도구(distillation)를, **ch05**가 보상으로 미는도구(RLHF·DPO·RLVR)를, **ch06**이 학습집합 $\mathcal{R}_k$ 자체를 만드는 절차 $\text{gen}(\cdot; B_s)$ 와 그것이 자기생성일 때의 제약을 준다. 다섯 장은 경쟁하는 대안이 아니라같은 식의 다른 칸을 채우는 부품이다.

남은 네 장은 **한계를 준다**. **ch07**은 그 부품을 반복해서돌렸을 때 무엇이 무너지는지 — 즉 ρ 의 정체를 — 성능 행렬 Π 와 replay 혼합비 ω 로 형식화한다. **ch08**은 이 라인 전체가빌려 쓴 은유의 출처인 CLS 이론과 생물학적 sleep을 놓는다(그 은유가실제로 무엇을 담보하는지의 결산은 **ch23**이 한다). **ch09**는 C_{cap} 의 상한을 준다 — 가중치 한 개에 몇 비트를 넣을 수있는가. **ch10**은 모델 밖으로 나가 E 층의 형태와 $\text{ret}(E, q)$ 의 비용을 분해하고, 정작 RAG 자체는 이 책의 주제가아니라는 말과 함께 그 판정 기준을 **ch11**에 넘긴다.

각 장은 목표 상자로 열고, 절마다 서빙 관점의 접점을 최소 한 번 통과하며, 손으로 따라갈 수치 예제와 자가 점검 체크리스트로 닫는다. Part I을 마친독자는 임의의 sleep-time 절차를 보고 **좌변이 무엇인지 지목하고, 비용 네 칸을 채우거나 “논문에 없음”이라고 쓸 수 있다**. 그 두 가지가 Part II 전체를 관통하는 동작이다.

ch01. 오리엔테이션 — 세 층, 세 경로, 그리고 이름의문제

이 장의 목표 — 이 장을 마친 독자는 다음을 할 수 있다. 1. 임의의 sleep-time 절차를 보고 그것이 $\Theta \cdot W \cdot E$ 중 어느 층을바꾸는지 지목하고, 그 갱신을 식 $(U-W)/(U-\Theta)/(U-E)$ 중 하나로 적을 수 있다. 2. 판별식 네 조건을 적용해 어떤 절차가 이 범주에 속하고 어떤 것이인접 문헌인지 1차 분류할 수 있다. 3. 비용 4 종 (B_s, L_w, C_{cap}, ρ) 을 채우고 식 (A) 로 상각 손익분기 N_q^* 를 계산할 수 있다. 4. 이 책이 답하려는 8 개 질문이 각각 어느 Part 의 어느 장에서 답해지는지 지목할 수 있다.

왜 필요한가 — Part II 의 세 장이 이 장 없이는 나란히 읽히지않는다. ch14 의 Letta Sleep-time Compute(arXiv:2504.13171) 는질의 전 계산의 산출물을 **모델 밖 텍스트** 로 적재한다 — 쓰기는 문맥블록을 새 문자열로 대체하는 연산이다 [Letta STC App. K, p.27]. ch21 의 Language Models Need Sleep(arXiv:2606.03979) 은같은 이름의 절차로 **배포 가중치**를 갱신한다 — sleep 마다 저랭크 expert 를 파라미터에 덧붙인다 [LM Need Sleep § 3.2]. ch17 의 Do Language Models Need Sleep?(arXiv:2605.26099) 은둘 중 어느 것도 아닌 세 번째 대상을 건드린다 — 그 갱신식이 쓰는 것은텍스트도 배포 가중치도 아니다 [Do LMs Need Sleep? Eq. 3]. 세 논문모두 자기 절차를 “sleep” 이라 부른다. 층을 지정하는 어휘가 없으면 이 셋은비교 대상조차 되지 못한다.

NM 과의 관계 — Neural Memory 모노그래프는 세 층 중 W 하나만다뤘다. backward pass, optimizer state, inner/outer loop, test-time learning 은 여기서 전제하고 넘긴다 (→ NM ch02, NM ch04). NM ch01 의 Rosetta 표는 **반복하지 않는다** — 이 장은 그 위에 E 층과오프라인 시계에 해당하는 행만 없는다. NM ch11 이 개념 수준에서 소개한 catastrophic forgetting·CLS·distillation 은각각 ch07·ch08·ch04 가 실무 깊이로 받는다.

01.1 왜 세 층으로나누는가

“모델이 자는 동안 뭔가를 정리해 둔다” 는 서술은 서버 엔지니어의 어떤질문에도 답하지 않는다. 네 개만 들어 보면 즉시 드러난다.

1. 이 상태를 **evict** 해도 되는가? 산출물이 모델 밖 레코드면 재계산가능한 캐시이므로 evict 가 정책 문제다. 산출물이 배포 가중치면 evict 라는 연산 자체가 없다 — 되돌리려면 이전 체크포인트로 롤백해야한다.
2. 이 사용자의 산출물이 다른 사용자와 같은 배치에 섞여도 되는가? 텍스트 레코드는 prompt 에 붙으므로 섞인다. 사용자별 가중치 delta 가생기면 shared-weight batching 의 전제가 깨진다 (→ NM ch10 의 grouped-GEMM 논의와 같은 자리).
3. **sleep** 라운드를 100 번 돌리면 무엇이 나빠지는가? 레코드가쌓이는 쪽에서는 저장량이 자라고 검색 정밀도가 떨어진다. 가중치를고치는 쪽에서는 이전에 알던 것이 손상된다 (→ ch07). **열화의정체가 층마다 다르다**.
4. 롤백 단위는 무엇인가? 레코드 하나인가, 세션 전체인가, 배포아티팩트인가.

네 질문의 답은 전부 “무엇이 바뀌었는가”에 달려 있고, 그 “무엇”은 정확히세 가지뿐이다. 그래서 이 책은 상태를 세 층으로 나누는 것에서 시작한다. 나누지 않으면 위 네 질문은 물을 수조차 없고, 물을 수 없으면 “유망한가”라는 질문도 답할 수 없다.

[해설] 이 3분할은 새로운 이론이 아니라 **회계 단위의선택**이다. 독자가 KV cache와 weights와 vector DB를 서로 다른 예산항목으로 관리하는 것과 같은 이유로, 이 책은 sleep-time 산출물을 세 예산항목으로 나눈다. 회계 단위가 없으면 합산도 비교도 불가능하다.

01.2 세 층: Θ , W , E

정의. 모델이 답을 만들 때 참조하는 상태를 세 층으로 나눈다. Θ (**slow weights**)는 pre-train과 학습 라운드에서 움직이는 파라미터, 즉 배포 아티팩트를 이루는 가중치다. W (**fast weights**)는 test-time에 토큰·chunk 단위로 움직이는 가중치 형태의 내부 상태다. E (**external store**)는 모델 **밖**에 텍스트·벡터·그래프로 저장되는 기억이다.

Θ 와 W 의 의미는 Neural Memory에서 **글자 그대로승계**한다($\rightarrow$ NM ch01 § 1.1). 재정의하지 않는다. E 만 이 책에서 신설한다.

표 1-1 — 세 층

기호	이름	갱신 시간척도	물리적 위치	서빙에서의 정체
Θ	slow weights	학습 라운드 k	가중치	배포 아티팩트. 롤백 단위 = 체크포인트
W	fast weights	토큰/chunk t	가중치 형태의 상태	요청·세션에 붙는 고정 크기 상태
E	external store	이벤트/세션 τ	모델 밖	조회 가능한 레코드. 재계산 가능

세 층의 결정적 차이는 크기도 정확도도 아니라 **누가 소유하고 언제사라지는가**이다. E 는 요청과 독립적으로 존속하고 임의 시점에 지울 수 있다. W 는 상태를 들고 있는 주체가 사라지면 함께 사라진다. Θ 는 지우는 연산이 정의되지 않으며, 바꾸려면 새 아티팩트를 만들어 배포해야 한다. 이 소유·수명 차이가 뒤에 나올 비용 4종의 값을 전부 결정한다.

01.3 읽기는 하나, 쓰기는셋

읽기는 세 층을 모두 통과한다. 이 식은 “답을 만들 때 무엇을 보는가”를 남김없이 적은 것이다.

$$\hat{y} = f(q; \Theta, W, \text{ret}(E, q)) \quad (\text{R})$$

$\text{ret}(E, q)$ 는 질의 q 에 대한 검색 연산이다. 독자의 어휘로는 vector DB 조회 한 번이고, 비용도 그렇게 회계한다.

쓰기는 층마다 다른 식을 쓴다. 첫째, W 에 쓰는 규칙이다. 이 식은 “토큰하나가 도착할 때마다 inner loss ℓ 의 gradient 방향으로 상태를 한걸음 옮긴다”는 뜻이다.

$$W_t = \alpha_t W_{t-1} + S_t, \quad S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t) \quad (\text{U-W})$$

식 (U-W)는 Neural Memory의 **master equation (M)**과 **글자 그대로같다**. 기호도 부호도 방향도 바뀌지 않았다 — α_t 는 여전히 남기는비율이고 η_t 는 여전히 inner learning rate다. 두 모노그래프는 같은척추를 공유한다. NM은 이 식 하나를 300쪽에 걸쳐 해부했고, 이 책은 그위(Θ)

와 밖 (E) 으로 확장한다. 따라서 (U-W) 의 성분 해설은 여기서 하지 않는다 ($\rightarrow$ NM ch01 § 1.4, NM ch12).

둘째, Θ 에 쓰는 규칙이다. 이 식은 “라운드 k 까지의 경험 $\mathcal{H}_k$ 로 학습 집합 $\mathcal{R}_k$ 를 만들고, 그것으로 outer loss $\mathcal{L}$ 을 줄이는 한 라운드를 돈다” 는 뜻이다.

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k), \quad \mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s) \quad (\text{U-}\Theta)$$

η_{Θ} 는 sleep learning rate 이고 η_t (inner) 와 다른 것이다. 둘을 섞어 쓰는 것은 이 책에서 결함이다. $\mathcal{R}_k$ 는 replay·distillation·합성데이터를 통칭하며, 그것을 어떻게 만드는가가 Part I 의 절반 (ch04·ch05·ch06·ch07) 을 차지한다.

셋째, E 에 쓰는 규칙이다. 이 식은 “질의를 보기 전에 문맥 c_k 를 예산 B_s 로 오프라인 처리해 learned context $\hat{c}_k$ 를 만들고, 그것을 store 에 기록한다” 는 뜻이다.

$$E_{k+1} = \text{wr}(E_k, \hat{c}_k), \quad \hat{c}_k = S(c_k; B_s) \quad (\text{U-E})$$

$S(\cdot; B_s)$ 는 인자를 괄호로 받는 **절차**이고, (U-W) 의 S_t (momentum, 항상 시간 첨자를 단다) 와 무관하다. wr 의 구체적 형태 — 덮어쓰기인가, 합집합인가, ADD/UPDATE/DELETE 를 판단하는가 — 가 E -경로 논문들을 가르치는 축이다 ($\rightarrow$ ch14, ch15).

[해설] 세 식의 차이는 좌변에 있다. 무엇이 바뀌는지를 좌변이 말해 준다. 그러므로 어떤 논문을 읽든 첫 질문은 하나다: **이 절차의 좌변은 무엇인가**. 좌변을 못 적으면 그 절차는 아직 이해되지 않은 것이다.

01.4 세 경로와 판별식

먼저 이 책이 다루는 대상을 조작적으로 못 박는다.

정의. sleep-time compute 은 질의 q 가 도착하기 전, 유휴 시간에 예산 B_s 만큼 소비하는 계산으로서, 그 산출물이 이후 질의의 wake-time 비용 B_t 를 줄이거나 정확도를 높이는 것을 말한다.

이 정의는 **갱신 대상 층을 지정하지 않는다**. 그래서 같은 이름 아래서로 다른 세 절차가 들어온다. 층을 붙여 셋으로 가른 것이 이 책의 척추다.

정의. E-경로 는 질의 전 오프라인 추론의 산출물을 모델밖에 적재하는 절차다 — (U-E) 만 실행하고 Θ 와 W 는 불변이다. **W-경로** 는 문맥을 fast weight 로 접어 넣고 원 문맥을 버리는 절차다 — (U-W) 를 wake 가 아니라 **오프라인에 추가로** 실행한다. **Θ -경로** 는 경험을 slow weights 로 통합하는 절차다 — (U- Θ) 를 실행하며 **배포 아티팩트가 바뀐다**.

표 1-2 — 세 경로

경로	실행하는 식	산출물의 물리적 형태	배포가 바뀌는가
E-경로	(U-E)	텍스트·벡터·그래프 레코드	아니다
W-경로	(U-W) 를 오프라인에	fast weight 텐서	아니다
Θ -경로	(U- Θ)	가중치 delta	그렇다

경로를 걸치는 절차는 **순서**를 밝힌다. 예: “(U-E) 로 $\hat{c}$ 를 만든 뒤 그것을 $\mathcal{R}$ 로 삼아 (U- Θ) 를 돈다.”

경로를 가르려면 소속 여부부터 판정해야 한다. 판정 도구가 판별식이다.

판별식. 절차 P 가 sleep-time compute 인가? (1) 질의 q 가 도착하기 전에, (2) 예산 $B_s > 0$ 을 써서, (3) $\Theta \cdot W \cdot E$ 중 어느 한 층의 상태를 실제로 바꾸고, (4) 그 변화가 이후 질의에 쓰이는가. 넷을 모두 만족해야 한다. 하나라도 빠지면 인접 문헌이지 이 범주가 아니다.

네 조건은 각각 다른 것을 걸러낸다. (1)은 wake 중에 일어나는 계산을, (2)는 추가 계산 없이 저장만 하는 절차를, (3)은 상태가 아니라 읽기 방식을 바꾼 설계를, (4)는 같은 질의 안에서 소비되고 끝나는 산출물을 걸러낸다.

이 책의 척추 논지가 여기서 나온다.

[평가] 이름이 기제보다 넓다. 판별식을 corpus 전체에 적용하면 “sleep-time compute”이라는 이름 아래 모인 문헌의 상당수가 어느 조건에선가 탈락한다. 이것은 논문들의 자기 규정과 어긋나는 이 책의 판단이며, 근거는 판별식의 전수 적용이다.

그러나 전수 적용의 결과와 그 판정 근거는 ch11이 낸다. 이 장은 도구를 손에 쥐여 주는 데서 멈춘다. 지금 결론을 앞당기면 ch11의 증거를 읽을 이유가 사라진다.

01.5 비용 4 종과 상각

모든 장은 자기가 다루는 절차를 아래 네 값으로 환산한다. 값이 논문에 없으면 “논문에 없음”이라고 쓴다. 추정치를 본문 수치로 쓰지 않는다.

표 1-3 — 비용 4 종

기호	이름	단위	독자의 어휘
B_s	sleep 계산 예산	FLOPs 또는 tokens	유틸 시간 배치 job의 비용
L_w	wake 지연 기여	ms, TTFT/ITL	SLA 항목
C_{cap}	상태 용량	bytes/세션 또는 bytes/사용자	저장 sizing
ρ	망각 · 열화율	라운드당 성능 손실	반복 실행의 지속가능성

용량 기호는 반드시 C_{cap} 으로 쓴다. 맨 C 는 NM에서 승계한 chunk 크기 전용이다.

B_s 가 클수록 좋다는 주장은 그 자체로는 성립하지 않는다. sleep 비용은 한 문맥을 공유하는 질의 수 N_q 에 나뉘어 상각되기 때문이다. 이 식은 “질의 하나당 실제로 내는 값 = wake 비용 + 상각된 sleep 비용”을 말한다.

$$C_{avg} = C_{wake} + \frac{C_{sleep}}{N_q} \quad (A)$$

식 (A)가 이 책에서 가장 자주 등장하는 도구다. sleep을 쓸지 말지는 C_{avg} 가 sleep 없는 baseline보다 작아지는 손익분기 N_q^* 를 넘느냐로 결정된다. 이 장의 worked micro-example에서 손으로 계산한다.

[평가] 식 (A)는 N_q 를 안다고 가정한다. 실서빙에서 한 문맥이 몇 번 재사용되는지의 분포를 보고한 논문은 이 책의 corpus에 없다. 따라서 상각 논증은 전부 조건부다 — “ N_q 가 충분히 크면”이라는 전제를 명시하지 않은 상각 주장은 이 책에서 미완성으로 취급한다. 이 자리의 서술은 예고이고, 증거와 판정은 ch14와 ch25가 낸다 — 여기서는 도구를 쓸 때 붙는 단서만 미리 못 박는다.

[평가] C_{cap} 은 corpus에서 가장 침묵당한 값이다. 상태 용량을 바이트로 보고한 논문이 0편이다. 그래서 이 책은 ch26에서 그 회계를 직접 만든다 — 이 자리는 예고이고 판정은 ch26이 한다. 비용 4종이 비어 있는 절차를 “효율적”이라고 서술하지 않는 것이 §01.5의 실천 규칙이다.

01.6 Rosetta 사전 — NM 사전 위에 얹는 행

NM ch01 의 Rosetta 표는 W 층과 decode 내부를 독자의 어휘에 접속했다. 여기서 반복하지 않는다. 이 장이 더하는 것은 E 층과 오프라인시계에 해당하는 행이다. 셋째 열이 가장 중요하다 — **동일** (같은대상의 재사용)·**유비** (정확하나 새 요소있음)·**차이가 논점** (직관을 그대로 이식하면 틀림)을 구분하지 않으면 잘못된 직관이 이식된다.

표 1-4 — Rosetta 사전 (확장분): inference-serving 어휘 ↔ sleep-time 어휘

독자의 어휘	이 책의 대응물	대응의 성격
prefix cache / prompt cache	E -경로의 $\hat{c}$	△ 차이가 논점 : cache 는 같은 계산의 재사용 , $\hat{c}$ 는 새 추론의 선행 수행
KV cache	W (fast weights)	유비 + 차이: KV 는 append-only 무손실, W 는 고정 크기 손실 압축
vector DB 조회 모델 재배포	$\text{ret}(E, q)$ ($U-\Theta$) 1 회	동일. 비용도 동일하게 회계 동일 — Θ -경로는 배포아티팩트가 바뀐다 . E/W 와의 결정적 차이
shared-weight batching	Θ -경로에서 깨짐	△ 차이가 논점 : per-user delta 가 생기면 성립하지 않는다
warm-up / 사전 컴파일	sleep 라운드	유비 + 차이: 둘 다유희 시간 활용이나 sleep 은 품질 을 바꾼다
checkpoint 크기	C_{cap}	동일 — Θ -경로 상태량은 checkpoint 회계로 직접 읽힌다
cache hit ratio / reuse distance	N_q	동일한 회계, 새 이름 — 상각의 분모
write amplification	$B_s \div$ 늘어난 C_{cap}	유비 (정확) — 저장 1 바이트를 만드는 데 드는 계산
LSM compaction / GC	E 의감쇠 · 병합 · 삭제	유비 + 차이: 이 연산의 유무 가 E -경로 설계를 가른다 (→ ch15)
카나리 배포 · 롤백	($U-\Theta$) 라운드의 되돌리기	△ 차이가 논점 : 사용자별 delta 면 롤백 대상이 사용자 수만큼 늘어난다
dtype/quantization 선택 SLA(TTFT/ITL)	C_{cap} 의 정보 밀도 L_w	동일 도구, 새 대상 (→ ch26) 동일 — 정의상 sleep 은 L_w 에 0 을 기여해야 한다

마지막 행이 정의의 시금석이다. 어떤 절차가 L_w 를 늘린다면 그것은 sleep 이 아니라 wake-time 계산이며, 판별식 (1) 에서 탈락한다.

01.7 이 책이 답하는 질문과 답이 놓인자리

이 책은 여덟 개의 질문에 답하려 한다. 출발점은 이 책에 주어진 일곱질문이다 — 유망한가 · 주류가 될까 · 무슨문제를 푸는가 · 최선인가 · 대안 대비 우월한가 · scaling law 는 어떻게 되는가 · memory device 기회는 무엇인가. 여기에 **여덟 번째 Q2(“무엇이 실제로 이 범주에속하는가”)** 를 이 책이 스스로 더했다. 대상이 확정되지 않으면 나머지일곱이 답할 수 있는 형태를 갖추지 못하

기 때문이다. 표 1-5의 여덟 행은 그일곱을 Q2 위에서 답 가능한 순서로 재배열한 것이다. 질문마다 답이 놓인자리가 다르고, 앞 질문의 답이 뒤 질문의 전제가 된다. 순서를 지키지않으면 답이 나오지 않는다.

표 1-5 — 질문 지도

#	질문	답이 놓인 자리	왜 그 자리인가
Q1	무슨 문제를 푸는가	Part I(ch02 – ch10) + Part II(ch12 – ch23)	문제정의가 논문마다 다르므로 배경과 계보를 먼저 깬다
Q2	무엇이 실제로 이 범주인가 (이 책이 추가)	ch11(판별식전수 적용) → ch24(판정)	나머지 질문 전부의 전제. 여기서 대상이확정된다
Q3	비용은 얼마이고 비교 가능한가	ch25(통일 비용 회계)	논문들의비용 보고가 서로 다른 단위다. 통일 회계를 세워야 비교가 된다
Q4	유망한가 / 최선인가 / 대안 대비우월한가	ch27(경로별 판정)	Q2와 Q3 없이는 이 질문 자체가 형식이성립하지 않는다
Q5	주류가 될까	ch27 + ch22(반증 축)	반증 후보를 통과하는 조건만남긴다
Q6	scaling law 가 있는가 (B_s 축)	ch28	B_s 를 독립변수로놓은 점들을 통일 축에 재배치한다
Q7	memory device 기 회가 있는가	ch26(용량) + ch29(system·infra)	상태 바이트가 실측되어야 device 논의가성립한다
Q8	무엇이 이 판정을 뒤집는가	ch30(반증 조건)	반증 불가능한 판정은 이 책에서 쓰지 않는다

[해설] Q4가 원래의 질문이고 나머지는 그것을 답할 수 있는형태로 바꾸기 위한 계단이다. Q4를 곧장 물으면 답이 “경우에 따라다르다”로 끝난다. 경로를 지정하고 (Q2) 회계를 세운 (Q3) 뒤에야 Q4는**경로마다 다른** 답을 가진 질문이 된다.

(state, update, cost) 정리

이 장에서 도입한 개념을 세 층 프레임 위에 배치한다. Part I의 모든 장이마지막에 이 표를 갱신한다.

표 1-6 — 이 장의 개념 배치

개념	state (무엇이 상태인가)	update (어느 식)	cost (어느 값에 잡히는가)
slow weights	Θ	(U- Θ), 라운드 k	B_s (라운드 비용), C_{cap} (delta bytes), ρ
fast weights	W	(U- W), 토큰/chunk t	L_w (decode 내 연산), C_{cap} (상태 bytes)
external store	E	(U- E), 이벤트/세션 τ	B_s (오프라인 추론), L_w (ret 지연), C_{cap}

개념	state (무엇이 상태인가)	update (어느 식)	cost (어느 값에 잡히는가)
learned context $\hat{c}$	E 의 한 레코드	$\hat{c} = S(c; B_s)$	B_s 전액. L_w 에는 0 기여
검색 $\text{ret}(E, q)$ 상각 N_q	상태 아님 (읽기) 상태 아님 (워크로드 속성)	(R)의 인자 식 (A)의 분모	L_w 에만 잡힌다 어느 비용에도 안 잡힘 — 미보고 항목

마지막 행이 이 표에서 가장 중요한 행이다. N_q 는 비용이 아니라 워크로드의 성질이고, 그래서 비용 4종의 어느 칸에도 들어가지 않으면서 모든 상각 주장의 결론을 좌우한다 — § 01.5의 첫 [평가]가 지목한자리다.

Worked micro-example — 손익분기와 비교불가능성

아래 산수는 이 책의 예시이며 특정 논문의 수치 주장이 아니다. 목적은 식 (A)를 손으로 굴러 보는 것과, 식 (A)만으로는 경로를 고를 수 없다는 것을 숫자로 보이는 것이다.

설정. 한 세션의 문맥 c 가 10,000 tokens 이고, 이 문맥을 N_q 개의 질의가 공유한다. 비용 단위는 토큰으로 잡되, 생성 토큰은 입력토큰의 κ 배로 센다. $\kappa = 10$ 으로 둔다.

baseline(sleep 없음). 질의마다 문맥 전체를 prefill 하고 300 tokens 를 생성한다.

$$C_{\text{avg}}^{\text{base}} = 10,000 \times 1 + 300 \times 10 = 13,000 \quad (N_q \text{와 무관})$$

E-경로. sleep에서 문맥을 한 번 읽고 3,000 tokens 를 생성해 $\hat{c} = 500$ tokens 를 만든다. wake에서는 $\hat{c}$ 만 prefill 하고 150 tokens 를 생성한다.

$$C_{\text{sleep}} = 10,000 + 3,000 \times 10 = 40,000, \quad C_{\text{wake}} = 500 + 150 \times 10 = 2,000$$

식 (A)에 넣으면 $C_{\text{avg}} = 2,000 + 40,000/N_q$ 이다. 손익분기는 이 값이 13,000과 같아지는 지점이다.

$$2,000 + \frac{40,000}{N_q^*} = 13,000 \implies N_q^* = \frac{40,000}{11,000} = 3.64$$

정수로는 $N_q \geq 4$ 부터 이긴다. 표로 확인한다.

표 1-7 — N_q 에 따른 C_{avg} (baseline = 13,000)

N_q	1	2	4	10	∞
C_{avg}	42,000	22,000	12,000	6,000	2,000
baseline 대비	$3.23 \times$ 손해	$1.69 \times$ 손해	$1.08 \times$ 이득	$2.17 \times$ 이득	$6.5 \times$ 이득

κ 하나가 답을 바꾼다. $\kappa = 1$ (모든 토큰을 같게셈)로 다시 풀면 baseline은 10,300, C_{sleep} 은 13,000, C_{wake} 는 650이 되어 $N_q^* = 13,000/9,650 = 1.35$, 즉 $N_q \geq 2$ 부터 이긴다. **가정 하나로 손익분기가 2.7배 움직인다.** 실제 논문들이 쓰는 κ 의 출처와 민감도 분석의 부재는 ch14와 ch25가 다룬다.

같은 예산을 Θ -경로에 쓰면. sleep에서 같은 40,000 단위를 써서 사용자별 가중치 delta를 만들고, wake에서는 문맥도 $\hat{c}$ 도 붙이지 않은 채 150 tokens 만 생성한다고 하자. $C_{\text{wake}} = 1,500$ 이고 $N_q^* = 40,000/11,500 = 3.48$ 이다. 식 (A)만 보면 두경로가 사실상 구분되지 않는다 (3.64 대 3.48).

C_{cap} 을 보면 달라진다. E-경로의 $\hat{c}$ 는 500 tokens 이고, 영문 기준 토큰당 4 bytes 로 잡으면 세션당 2 KB 다. Θ -경로에서 rank $r = 16$ 의 저랭크 adapter(정의 $\rightarrow$ ch03, Θ -경로에서의 역할 $\rightarrow$ ch19) 를 $d = 4096$ 모델의 $32 \text{ layer} \times 2 \text{ projection}$ 에 붙이면 파라미터 수는

$$2rd \times (32 \times 2) = 2 \times 16 \times 4096 \times 64 = 8,388,608$$

이고, bf16(2 bytes/elem) 기준 사용자당 **16.8 MB** 다. 2 KB 와 16.8 MB 의 비는 약 **8,400 $\times$** , 자릿수로 네 자리 차이다. 사용자 100 만 명이면 2 GB 대 16.8 TB 다.

[평가] 결론은 어느 쪽이 이겼다가 아니다. 식 (A) 는 두 경로를 같다고 말하고 C_{cap} 은 **8,400 배 다르다고 말한다** (둘다 이 책의 예시 계산). 두 값은 단위가 달라 하나의 수로 합쳐지지 않는다. 그러므로 “어느 경로가 나은가” 는 (A) 하나로 답할 수 없고, 통일 비용회계 (ch25) 와 용량 회계 (ch26) 를 따로 세워야 한다. 이 장이 비용을 **4 종**으로 도입한 이유가 여기 있다.

요약

- 상태는 세 층이다: Θ (slow weights)· W (fast weights)· E (external store). Θ 와 W 는 Neural Memory 에서 글자 그대로 승계하고 E 만 신설한다.
- 읽기는 식 (R) 하나이고 쓰기는 $(U \cdot W) \cdot (U \cdot \Theta) \cdot (U \cdot E)$ 셋이다. 어떤 절차를 이해했다는 것은 그 절차의 **좌변을 적을 수 있다**는 뜻이다.
- 식 $(U \cdot W)$ 는 Neural Memory 의 master equation (M) 과 글자 그대로 같다. 두모노그래프는 같은 척추를 공유하며, 이 책은 그 위 (Θ) 와 밖 (E) 으로 확장한다.
- sleep-time compute 의 조작적 정의는 갱신 층을 지정하지 않는다. 그래서 E-경로 · W -경로 · Θ -경로 셋이 같은 이름 아래들어오고, 셋은 비용 구조가 근본적으로 다르다.
- 판별식은 네 조건 (질의 전 · $B_s > 0$ · 실제층 갱신 · 이후 질의에 사용)이며 넷을 모두 만족해야 한다. 전수 적용의 결과는 ch11 이 낸다 — 이름이 기제보다 넓다는 것이 이 책의 척추 논지다.
- 비용은 항상 네 값이다: $B_s \cdot L_w \cdot C_{\text{cap}} \cdot \rho$. 논문에 없으면 “논문에 없음” 이라고 쓰고, 비어 있는 절차를 “효율적” 이라고 서술하지 않는다.
- 식 (A) 의 손익분기 N_q^* 는 비용 가정 하나 (κ) 로 2.7 배움직이고, N_q 분포를 보고한 논문은 corpus 에 없다.
- 식 (A) 와 C_{cap} 은 단위가 달라 합산되지 않는다 — 경로선택은 단일 지표로 결정되지 않는다.

자가 점검체크리스트

- ☐ 임의의 sleep-time 절차를 보고 그 좌변이 $\Theta \cdot W \cdot E$ 중 무엇인지 지목하고 $(U \cdot W) / ((U \cdot \Theta) / (U \cdot E))$ 중 하나로 적을 수 있다.
- ☐ 판별식 네 조건을 순서대로 말하고, 각 조건이 무엇을 걸러내는지 예로설명할 수 있다.
- ☐ η_t 와 η_Θ , S_t 와 $S(c; B_s)$, C 와 C_{cap} 의 차이를 각각 한 문장으로 말할 수 있다.
- ☐ worked micro-example 의 손익분기 계산을 다시 하고, κ 를 바꿨을 때 N_q^* 가 어느 방향으로 얼마나 움직이는지 계산할 수 있다.
- ☐ 이 책의 8 개 질문이 각각 어느 장에서 답해지는지, 그리고 왜 Q2 와 Q3 가 Q4 보다 먼저 와야 하는지 설명할 수 있다.
- ☐ 표 1-4 의 임의의 행을 골라 그 대응이동일/유비/차이 중 무엇인지 판별하고, $\hat{c}$ 와 prefix cache 의 차이를 자기 서빙 스택의 어휘로 옮길 수 있다.

다음 장으로

이 장은 $(U \cdot \Theta)$ 를 식으로 적었지만 그 우변은 아직 블랙박스다. $\mathcal{R}_k$ 를 만드는 $\text{gen}(\mathcal{H}_k; B_s)$ 이 무엇이고, 한 라운드를 “돈다” 는 것이 실무에서 어떤 job 이며, η_Θ 는 누가 정하는지 아직 아무것도 말

하지 않았다. ch02는 그블랙박스의 지도를 그린다: pre-train·SFT·RL·continual의 네레짐을 각각 “무엇이 언제 움직이는가”로 정의하고, 각 레짐이 $(U-\Theta)$ 의 어느 인자를 채우는지 대응시킨다. 그 지도가 있어야 ch03 이후의 도구들 — 부분 갱신, distillation, 보상, 합성 데이터, replay — 이 서로 경쟁하는 대안이 아니라 같은 식의 다른 칸을 채우는 부품으로 읽힌다.

ch02. training 레짐 지도 — 무엇이 언제 움직이는가

이 장의 목표 — 이 장을 마치면 독자는 다음을 할 수 있어야 한다. 1. 하나의 배포 checkpoint 가 태어나기까지 거치는 레짐 (pre-training · SFT · RLHF/RLVR · continual) 을 이름으로 구분하고, 각 레짐에서 Θ 가 무엇에 의해 얼마나 움직이는지 말할 수 있다. 2. 임의의 training run 을 다섯구성요소 (데이터 · 목적함수 · optimizer · 스케줄 · 평가) 로 분해하고, 각 구성요소가 어느 자원을 먹는지 지목할 수 있다. 3. token budget D , step 수 n_{step} , batch 크기 N_b , epoch 수 n_{ep} 의 관계식을 쓰고, $6ND$ 로 한 레짐의 FLOPs 를즉석에서 산정할 수 있다. 4. 임의의 논문에 대해 “**이 값은 누가학습하는가**” 를 네 질문으로 판정하고, 그 결과를 $(U-W)/(U-\Theta)/(U-E)$ 중 하나에 배치할 수 있다.

왜 필요한가 — Part II 의 세 장이 이 장의 어휘를 전제한다. - **ch20 [SEAL]** (Self-Adapting Language Models, arXiv:2506.10943): SEAL 이 “inner loop” 라고 부르는 것은 이 라인의 inner loop($\rightarrow$ NM ch04) 가 **아니라한 번의 SFT run 전체다** (“an outer RL loop, which optimizes the self-edit generation, and an inner update loop, which uses the generated self-edit to update the model via gradient descent” [SEAL § 3 opening, p.3]). 레짐 어휘가 없으면 이 문장이 통째로 오독된다. - **ch21 [LM Need Sleep]** (Language Models Need Sleep, arXiv:2606.03979): 한 sleep 라운드의 내부가 SFT 와 RL 과 distillation 의 조합이고, 그 하이퍼파라미터가 표로 실려 있다 (LR $5e-6$, effective batch size 32, steps 500/100/100 [LM Need Sleep Table 5, p.26]). 이 표를 읽으려면 step·batch·epoch 이무엇의 이름인지 알아야 한다. - **ch14 [Letta STC]** (Sleep-time Compute, arXiv:2504.13171): 이 논문은레짐을 **하나도 실행하지 않는다**. 가중치 공간의 상태를 쓰지않는다고 논문이 명시한다 [Letta STC § 7, p.13]. “이 라인의 이름을만든 논문이 Θ 를 건드리지 않는다” 는 사실은 § 02.10 의판별 습관으로만 즉시 확인된다.

NM 과의 관계 — NM ch02 는 training 을 한 **step 안쪽**에서해부했다: backward pass 가 계산하는 두 GEMM, $dW = \delta x^\top$ 라는 shape, optimizer 가 $\text{update}(\text{state}, g) \rightarrow (\text{state}', \Delta \Theta)$ 서명의객체라는 것, optimizer state 의 바이트 회계. 그 전부를 여기서 반복하지않는다 ($\rightarrow$ NM ch02 § 2.3, § 2.5, § 2.8). 이 장이 더하는 것은 **step 바깥의 지도다**: 그 step 들이 몇 개씩 묶여 어떤 이름의 레짐을 이루고, 레짐마다 데이터와 목적함수가 어떻게 갈리며, 배포 아티팩트가 어느경계에서 바뀌는가.

02.1 레짐이라는 단위 — 하나의 checkpoint가태어나기까지

독자가 서빙 엔진에 올리는 checkpoint 파일 하나는 여러 공정을 순서대로통과한 결과물이다. 각 공정은 목적도 데이터도 예산도 다르며, 공정사이에서 checkpoint 가 한 번씩 저장된다. 이 공정 하나를 이 책은레짐이라고 부른다.

정의. training run 이란 고정된 데이터 원천과 고정된목적함수 아래에서 식 $(U-\Theta)$ 를 정해진 횟수만큼 반복해 하나의 checkpoint 를 만들어내는 작업 단위다. **레짐 (regime)**

이란 그런 run 들의 부류로서, (i) 데이터가 어디서 오는가, (ii) 목적함수가 무엇인가, (iii) Θ 의 어느 부분집합이 움직이는가 — 이 셋의 조합으로 식별된다.

레짐이 필요한 이유는 단순하다. 식 (U- Θ) 는 모든 레짐에서 글자 그대로 같기 때문이다.

$$\Theta^{(s+1)} = \Theta^{(s)} - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{B}_s; \Theta^{(s)}) \quad (2-1)$$

이 식이 말하는 것은 “batch $\mathcal{B}_s$ 위에서 켄 outer loss $\mathcal{L}$ 의 gradient 반대 방향으로 slow weights 를 η_{Θ} 만큼 옮긴다” 이며, 이는 pre-training 이든 SFT 든 sleep 라운드든 동일하다. 여기서 s 는 step 첨자이고 η_{Θ} 는 outer/sleep learning rate 다 (NM 의 무첨자 η 가 v2 에서 η_{Θ} 다. inner rate η_t 와 섞어 쓰지 않는다).

식 (2-1) 을 전역 표준형 (U- Θ) 와 나란히 놓으면 이 장의 논지가 한눈에 보인다. (U- Θ) 는 라운드 첨자 k 와 학습 집합 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; \mathcal{B}_s)$ 를 쓰고, (2-1) 은 step 첨자 s 와 batch $\mathcal{B}_s$ 를 쓴다. 대수는 같고 다른 것은 첨자의 시간척도와 $\mathcal{B}$ 가 어디서 오는가 뿐이다.

[해설] 그러므로 Θ -경로의 “sleep 라운드” 는 새로운 종류의 연산이 아니다. **데이터가 주어지는 대신 모델이 만들어낸다는 점만 다른, 아주 짧은 training run** 이다. 이 등식이 이 책 전체의 지렛대다 — ch21 과 ch20 에서 sleep 라운드의 비용을 계산할 때 우리는 training run 의 비용 회계를 그대로 쓰게 된다.

레짐은 순서를 갖는다. 오늘 독자가 서빙하는 instruct 계열 checkpoint 하나는 최소한 아래 사슬을 통과했다. 각 화살표에서 checkpoint 가 저장되고, 앞 레짐의 산출물이 뒤 레짐의 초기값이 된다.

$$\underbrace{\Theta_{\text{rand}}}_{\text{무작위}} \xrightarrow{\text{pre-training}} \Theta_{\text{base}} \xrightarrow{\text{SFT}} \Theta_{\text{sft}} \xrightarrow{\text{RLHF / RLVR}} \Theta_{\text{deploy}} \xrightarrow{\text{continual}} \Theta_{\text{deploy}}^{(k)}$$

이 사슬에서 처음 세 화살표는 배포 **전에** 한 번씩 지나가고, 마지막 화살표만 배포 **후에** 반복된다. 이 책이 다루는 Θ -경로 sleep-time compute 은 예외 없이 마지막 화살표에 산다 — ch20 과 ch21 이 손대는 지점이 정확히 여기다. 앞 세 화살표를 이 장이 설명하는 이유는 그 자체가 목적이어서가 아니라, **마지막 화살표가 앞의 것들을 도구로 재사용하기 때문이다**. sleep 라운드 하나를 열어 보면 안에 SFT 와 RL 이 들어있다.

systems 관점에서 레짐 경계는 **아티팩트 경계**다. 레짐이 끝날 때마다 checkpoint 가 하나 나오 고, 그 checkpoint 가 다음 레짐의 초기값이 된다. 서빙 fleet 가 실제로 mmap 하는 파일은 이 사슬의 마지막 산출물이며, Rosetta 사전의 “모델 재배포 = (U- Θ) 1 회” 라는 대응이 여기서 성립한다 — Θ -경로가 $E \cdot W$ -경로와 갈라지는 결정적 지점은 **배포 아티팩트가 바뀐다**는 것 하나다.

02.2 training run 의 구성요소 다섯

임의의 training run 은 다섯 개의 슬롯으로 완전히 기술된다. 논문을 읽을 때 이 다섯 칸을 채우지 못하면 그 논문의 학습 절차를 이해하지 못한 것이다.

정의. training run 의 다섯 구성요소는 다음과 같다. 1. **데이터** — 식 (2-1) 의 $\mathcal{B}_s$ 가 어디서 오는가. 고정 코퍼스, 사람이 쓴 시연, 모델이 생성한 rollout, 이전 라운드의 경험 $\mathcal{H}_k$ 중 하나 이상. 2. **목적함수** — 스칼라 $\mathcal{L}$ 의 정의. next-token cross-entropy, 시연에 대한 조건부 cross-entropy, 보상의 기댓값, teacher 분포와의 divergence 등. 3. **optimizer** — $\text{update}(\text{state}, \mathbf{g}) \rightarrow (\text{state}', \Delta \Theta)$ 서명의 객체 ($\rightarrow$ NM ch02 § 2.5). AdamW 가 사실상의 기본값이다. 4. **스케줄** — η_{Θ} 의 시간 프로파일 (warmup-decay), batch 크기, 총 step 수, 정밀도. 5. **평가** — 이 run 을 멈출지, 되돌릴지, 배포할지를 결정하는 측정 절차.

다섯 칸의 자원 성격이 서로 완전히 다르다는 점이 systems 독자에게 중요하다. **데이터**는 storage 와 network 를 먹고 GPU 를 굶기는항이다 (입력 파이프라인이 GPU 를 못 먹이면 MFU 가 그대로 떨어진다). **목적함수**는 logits 에서 스칼라로의 reduction 이므로 비용이 사실상 0 이다 — 즉 목적 함수를 바꾸는 것은 공짜이고, 그래서 이 분야의 논문대부분이 목적함수를 바꾼다. **optimizer** 는 HBM 을 먹는다 (param 당 state 바이트, → NM ch02 § 2.8). **스케줄**은 FLOPs 를 결정한다. **평가**는 training 이 아니라 inference 워크로드이며, 별도의 fleet 또는 별도의 시간 슬롯을 요구한다.

넷째 슬롯의 첫 값인 η_Θ 는 논문을 읽을 때 가장 먼저 보아야 할 숫자다. corpus 의 두 논문을 나란히 놓으면 그 이유가 보인다. [LM Need Sleep] 은 GRPO·SFT·Sleep 세 run 을 모두 LR 5e-6 으로 돌리고 [LM Need Sleep Table 5, p.26], [SEAL] 은 outer 루프의 갱신을 lr 3e-4 로 돌린다 [SEAL § B.2, p.20]. 60 배 차이다. 이것이 표기 실수가 아니라는 것은 두 논문이 갱신 부분공간의 크기까지 똑같이 잡았다는 데서 드러난다 — 양쪽 다 LoRA rank 64, alpha 128 이다 [LM Need Sleep Table 5, p.26; SEAL § B.2, p.20]. 갱신되는부분공간은 같고 보폭만 60 배 다르다.

[해설] η_Θ 는 그 run 이 무엇을 하려는지에 대한가장 압축된 진술이다. 작으면 기존 Θ 를 보존하면서 미세하게 옮기려는 것이고, 크면 이번 데이터에 빠르게 맞추려는 것이다. 독자의감각으로 옮기면 cache 정책과 같다 — 작은 η_Θ 는 보수적 갱신, 큰 η_Θ 는 공격적 덮어쓰기이며, 후자는 이전에 들어 있던 것들을 위험을 함께 산다 (→ ch07).

[평가] 그러므로 이 책은 서로 다른 논문의 성능 수치를 “같은 Θ -경로이므로 비교 가능하다” 는 전제로 나란히 놓지 않는다. 위 60 배는 두 결과가 같은 종류의 쓰기를 재고 있다는 보장이 없음을 뜻한다. 경로가 같아도 스케줄이 다르면 비교의 근거는 따로 세워야 한다.

다섯째 슬롯이 이 책에서 특별히 중요하다. 평가가 training run **안으로 들어오는 순간** run 의 구조가 바뀌기 때문이다. 지도학습에서 평가는 run 바깥의 감시자다. 그러나 RL 에서 보상 계산은 매 step 의 데이터 생성 경로 위에 앉고 (§ 02.6), [SEAL] 에서는 “이 self-edit 을 적용한모델이 실제로 과제를 푸는가” 라는 평가 결과가 outer gradient 의 계수가된다 [SEAL § 3.1, p.4]. 평가가 목적함수의 일부가 되는 것이 최근레짐들의 공통 형태이며, 그 대가는 training 클러스터 안에 inference 클러스터가 들어온다는 것이다.

[해설] 독자의 감각으로 옮기면 이렇다. 지도학습 run 의 시간구조는 “두꺼운 GEMM(forward/backward) + bandwidth-bound elementwise pass(optimizer step)” 의 반복이고 — 즉 prefill 성격이다. RL run 과 sleep 라운드의 시간 구조는 거기에 **decode 성격의 생성단계**가 앞에 붙는다. 같은 (U- Θ) 를 돌면서도 클러스터 점유 패턴이전혀 달라진다.

02.3 예산의 어휘 — token budget · step · batch · epoch

레짐의 크기를 말하려면 단위가 필요하다. 네 단어를 여기서 못 박는다.

정의. - **step**(= iteration): 식 (2-1) 을 한 번 적용하는것. Θ 가 움직이는 최소 사건이다. 개수를 n_{step} 으로 쓴다. - **batch**: 한 step 에서 gradient 를평균하는 표본의 집합 $\mathcal{B}_s$. 시퀀스 개수를 N_b 로 쓴다. - **epoch**: 데이터셋 전체를 한 번통과하는 것. 개수를 n_{ep} 로 쓴다. 정수일 필요는 없다. - **token budget** D : 그 run 이 소비한 총 토큰 수. 레짐의 크기를재는 이 책의 기본 단위다.

이 장에서 N 은 모델 parameter 개수, D 는 token budget 을뜻한다 (장-국소 선언). L 은 시퀀스 길이이며 NM 에서 그대로 승계한다 — 총 수를 뜻하는 기호는 L_{layer} 이고, 이 책은 맨 L 을 층 수의미로 쓰지 않는다. 비용 4 종의 L_w (wake 지연) 와는 첨자로 구분된다. 네 값의 관계는 곱셈 하나다.

$$D = n_{\text{step}} \cdot N_b \cdot L, \quad n_{\text{ep}} = \frac{D}{|\mathcal{D}|} \quad (2-2)$$

이 식이 말하는 것은 “총 소비 토큰은 step 수 $\times$ step 당 토큰이고, epoch 은 그 총량을 데이터셋 크기로 나눈 비율일 뿐” 이라는 것이다. $|D|$ 는 데이터셋의 토큰 수다. epoch 이 독립 단위가 아니라 **유도량**이라는 사실이 중요하다 — 데이터셋이 예산보다 크면 $n_{ep} < 1$ 이고, 그 경우 “epoch”이라는 단어는 아무것도 설명하지 못한다.

N_b 를 읽을 때 한 가지 함정이 있다. 논문 표에 적히는 값은 대개 한 가속기가 한 번에 forward 한 개수가 아니라 **effective batch size** — 즉 “한 번의 (2-1) 적용에 gradient 가 평균되어 들어간 시퀀스의 총수” 다. 이 값은 세 인자의 곱이다: 가속기당 micro-batch $\times$ data-parallel 랭크 수 $\times$ gradient accumulation 횟수. 앞의 둘은 병렬화이고 셋째는 순차반복이다 — gradient 를 여러 micro-batch 에 걸쳐 누적해 두었다가 한 번에 optimizer step 을 밟는 것으로, HBM 이 부족할 때 N_b 를 사는 표준 수단이다.

[해설] 이 구분이 실무적으로 중요한 이유는 셋 중 어느 조합이든 식 (2-2) 와 (2-3) 의 값이 같기 때문이다. effective batch size 32 는 32 개를 한 번에 밀어 넣든 8 개씩 4 번 누적하든 같은 D , 같은 FLOPs, 같은 Θ 궤적을 낸다 (수치 오차 제외). 바뀌는 것은 벽시계 시간과 HBM 점유뿐이다. 그러므로 [LM Need Sleep Table 5, p.26] 처럼 “effective batch size 32” 만 적힌 표에서도 이 장의 회계는 그대로 성립한다 — 논문이 하드웨어를 밝히지 않아도 D 는 확정된다.

FLOPs 로의 환산은 NM ch02 가 이미 확정한 사실 하나에서 곧바로 나온다. forward 는 parameter 당 토큰당 곱셈-덧셈 한 쌍, 즉 2 FLOPs 이고 backward 는 forward 의 약 2 배다 ($\rightarrow$ NM ch02 § 2.3). 따라서

$$\text{FLOPs}_{\text{run}} \approx 6ND \quad (\text{full fine-tuning}), \quad \text{FLOPs}_{\text{fwd only}} \approx 2ND \quad (2-3)$$

이 식이 이 장의 계산 도구 전부다. $6ND$ 의 6 은 forward 2 + backward 4 이고, backward 의 4 는 다시 “입력 방향 오차 전파 2 + weight gradient 생성 2” 로 갈린다 ($\rightarrow$ NM ch02 § 2.3 의 두 GEMM). 이 분해가 실무적으로 쓸모 있는 이유는, Θ 의 일부만 갱신하는 레짐에서는 **weight gradient 생성 항이 통째로 빠져** $6ND$ 가 $4ND$ 로 내려가기 때문이다 (그 레짐의 이름과 기제는 ch03 이 소유한다). 반대로 E -경로와 W -경로의 오프라인 처리 $S(c; B_s)$ 는 생성, 즉 forward 만 돌므로 $2ND$ 쪽이다 — 경로 간 비용 비교의 첫 자릿수가 여기서 이미 갈린다.

[해설] 독자가 쓰던 회계와 완전히 같은 도구다. serving 에서 decode 한 토큰의 비용을 $2N$ 으로 잡던 그 습관에, “training 은 같은 토큰에 3 배를 쓴다” 는 인자 하나를 붙이면 끝이다. MFU 도 그대로 적용된다: $\text{FLOPs}_{\text{run}}$ 을 (가속기 peak $\times$ 대수 $\times$ 시간 $\times$ MFU) 로 나누면 GPU-hour 가 나온다.

02.4 pre-training — Θ 의 처음이자 가장큰 이동

정의. pre-training이란 무작위 초기화된 Θ 를 대규모 미분류 텍스트 코퍼스 위에서 next-token prediction 의 cross-entropy 를 목적함수로 삼아 갱신하는 레짐이다. Θ 전체가 움직이며, 한 checkpoint 의 생애에서 D 가 압도적으로 큰 유일한 레짐이다.

데이터는 주어진 것이고, 사람이 라벨을 달지 않았으며, 목적함수는 데이터 자신이다 — 다음 토큰이 곧 정답이다. 이 자기지도 성질이 D 를 코퍼스 크기까지 밀어 올릴 수 있게 만든 유일한 이유다.

pre-training 에서 n_{ep} 가 대체로 1 부근이라는 사실은 독자가 반드시 기억할 구조적 특징이다. 식 (2-2) 에서 $|D|$ 가 D 와 같은 자릿수면 $n_{ep} \approx 1$ 이 되고, 이는 “모든 토큰을 한번씩만 본다” 는 뜻이다. 즉 pre-training 의 대부분 구간에서 모델은 **처음 보는 데이터에 대해 online 에 가까운 학습**을 한다. 뒤에서 다룰 replay·forgetting 의 문제 설정 ($\rightarrow$ ch07) 이 pre-training 안에서는 잘 드러나지 않는 이유가 이것이다 — 같은 데이터를 두 번 보지 않으면 “잊었다” 를 관측할 기회 자체가 적다.

다섯째 슬롯인 평가도 이 레짐에서는 단순하다. 목적함수가 곧 지표이므로 held-out 코퍼스의 cross-entropy 를 그대로 읽으면 run 이 정상인지 판정된다. 이 성질 — **훈련 지표와 평가 지표가**

같은 양이라는 것 — 은 pre-training에만 있는 사치다. 뒤의 모든 레짐에서 $\mathcal{L}$ 이 내려가는 것과 능력이 오르는 것은 별개의 사건이 되며, § 02.5의 SFT 수치가 그 첫 사례다.

continued pre-training(CPT) 은 pre-training 과 목적함수 · 데이터 형식이 같고 초기값만 기존 checkpoint 인 레짐이다. 이 책에서 CPT 가 중요한 이유는 Θ -경로 논문들이 자기갱신을 CPT 형태로 실행하기 때문이다. [SEAL]의 지식 통합 실험에서 $n = 200 \cdot n = 2067$ 설정은 문자 그대로 continued pretraining이며 full fine-tuning 으로 돌린다 [SEAL Table 2 열 머리글: “Continued Pretraining (n=200; full-FT)”].

systems 접점은 두 가지다. 첫째, pre-training 은 이 책에 등장하는 어떤 레짐보다 여러 자릿수 비싸다 (이 장의 Worked micro-example 에서 수치로 확인한다). 둘째, 그 비용의 대부분이 **한 번만** 지불된다. 이후 모든 레짐은 그 위에 얹히는 작은 편차이며, 이 비대칭이 “sleep 라운드가 상각될 수 있는가” 라는 질문을 애초에 성립시킨다 — 큰 Θ 는 이미 지불되었고 우리가 논쟁하는 것은 그 위의 작은 증분이다.

02.5 SFT — 형식을 고정하는 짧은 레짐

정의. supervised fine-tuning(SFT) 이란 (입력, 원하는 출력) 쌍의 집합 위에서 **출력 토큰에 대해서만** 조건부 cross-entropy 를 최소화하는 레짐이다. 데이터는 사람이 쓴 시연이거나 다른 모델의 출력이며, D 는 pre-training 보다 여러 자릿수 작다.

수식 수준에서 SFT 와 pre-training 의 차이는 단 하나, **loss mask** 다. 시퀀스의 모든 토큰에 loss 를 거는 대신 입력 (prompt) 구간을 마스킹하고 출력 구간만 $\mathcal{L}$ 에 넣는다. 목적함수를 바꾸는 비용이 0 이라는 § 02.2 의 관찰이 여기서 실감된다 — 레짐 하나가 마스크 텐서 한 장으로 정의된다.

이 마스크가 얼마나 조작 가능한 대상인지는 [SEAL] 이 보여준다. few-shot 영역에서 모델이 생성하는 self-edit 에는 텍스트만이 아니라 **inner learning rate, epoch 수, loss mask 자체**가 필드로 들어간다 [SEAL Fig. 3, § 3.2, p.6]. 즉 § 02.2 의 다섯 슬롯 중 “스케줄” 과 “목적함수” 의 일부를 모델이 스스로 써 넣는다. 레짐을 구성요소로 분해해두면 그 논문이 정확히 어느 칸을 자동화했는지 한 줄로 말할 수 있다.

SFT 는 지식을 주입하는 레짐으로 설계되지 않았다. 설계 의도는 이미 Θ 안에 있는 능력을 원하는 출력 형식으로 꺼내는 것이다. 그럼에도 Θ -경로 논문들은 SFT 를 지식 주입 수단으로 쓴다 — [SEAL] 의 단일 문단 설정이 그렇고 [LM Need Sleep] 의 sleep 라운드 내부가 그렇다. 이 용도 전용이 무해하지 않다는 증거가 corpus 안에 이미 있다. Qwen3-1.7B 에서 SFT 는 base 대비 AIME-24 에서 $49.8 \rightarrow 47.3$ 으로, HMMT-25 에서 $25.7 \rightarrow 22.9$ 로 **내려간다** [LM Need Sleep Table 2, p.12].

[평가] SFT 를 돌렸다는 사실 자체는 능력이 올랐다는 증거가 아니다. 위 두 수치는 같은 표 안에서 GRPO(51.0 / 26.1) 와 Sleep(53.2 / 29.3) 이 오르는 동안 SFT 만 내려간 사례다 [LM Need Sleep Table 2, p.12]. 이 책은 이후 어느 장에서도 “fine-tuning 했으므로 지식이 들어갔다” 는 추론을 받아들이지 않는다 — 목적함수가 무엇이었는지, 평가가 무엇을 잴지를 함께 보아야 한다.

systems 접점은 크기다. SFT run 의 D 는 서버 노드 몇 대의 하루 처리량과 같은 자릿수이며, 이것이 SFT 를 “유휴 시간에 돌릴 수 있는 작업” 의 후보로 만든다. sleep-time compute 의 Θ -경로가 SFT 를 기본 도구로 삼는 이유는 성능이 아니라 **예산이 맞기 때문이다**.

02.6 RLHF 와 RLVR — 데이터를 run 안에서 만드는 레짐

정의. RLHF(RL from human feedback) 란 사람의 선호비교로 학습된 별도 모델 Θ_{rm} 이 매기는 점수의 기댓값을 최대화하도록 Θ 를 옮기는 레짐이다. **RLVR(RL with verifiable rewards)** 이란 그 점수를 학습된 모델이 아니라 **프로그램으로 검증 가능한 정답 여부**가 매기는 레짐이다.

두 레짐의 구조적 특징은 보상이 아니라 데이터다. 지도학습 레짐에서 $\mathcal{B}_s$ 는 run 바깥에서 주어진다. RL 레짐에서 $\mathcal{B}_s$ 는 **현재의 $\Theta^{(s)}$ 가 생성한 rollout** 이다. 즉

$$\mathcal{B}_s = \text{gen}(\text{prompt 집합}; \Theta^{(s)}), \quad \mathcal{L}(\mathcal{B}_s; \Theta) = -\mathbb{E}_{y \sim \mathcal{B}_s} [R(y)] + (\text{정규화 항}) \quad (2-4)$$

$R(y)$ 는 생성 y 에 매겨진 보상이다. RL 문헌은 보상을 r 로 쓰는것이 관행이지만 이 책에서 r 은 LoRA rank 로 예약되어 있고 ($\rightarrow$ ch03), 그래서 보상은 이 책 전체에서 R 로 옮겨 적는다 — 원문 기호와 대조할 때이 한 글자만 바꿔 읽으면 된다.

이 식이 말하는 것은 “데이터 슬롯이 parameter 의 함수가 되었다” 는 것이다. 이것이 식 (U- Θ) 에 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; \mathcal{B}_s)$ 라는 생성 항이 들어있는 이유이며, [SEAL] 이 자기참조적이라고 불리는 이유이기도 하다 — 그논문은 $\text{gen}(\cdot)$ 을 Θ 자신으로 놓는다 [SEAL Abstract; § 3.1]. 개별 RL 알고리즘 (PPO·GRPO·DPO) 의 내부와 reward hacking 의 기제는 ch05 가 소유한다. 이 장이 확정하는 것은 슬롯 배치뿐이다.

“얼마나 드나” 의 답이 여기서 뒤집힌다. [LM Need Sleep] 에서 RL 레짐의 step 수는 SFT 의 5 배다 (GRPO 500 vs SFT 100 steps [LM Need Sleep Table 5, p.26]). 그러나 진짜 비용은 step 수가 아니라 **step 당 생성량**이다. 생성이 붙은 레짐은 step 하나의 값이 다르다 — 같은 논문은 자기 sleep step 하나가 SFT step 하나의 4 배라고 보고한다 (“SFT is 4x more efficient than our method” [LM Need Sleep App. B.5, p.26]). 그 생성분은 식 (2-3) 의 $2ND$ 항으로 회계된다.

데이터 슬롯이 Θ 의 함수라는 사실에는 따름정리가 하나 붙는다. $\Theta^{(s)}$ 가 움직이는 순간 이전 step 에서 만든 rollout 은 **더이상 현재 정책의 표본이 아니다**. 그래서 RL 레짐은 “생성한 데이터를 몇 step 까지 재사용할 것인가” 라는, 지도학습 레짐에는 존재하지 않는 결정을 반드시 내려야 한다. 재사용하지 않으면 매 step 생성 비용을 다시 치르고, 재사용하면 데이터와 정책이 어긋난다. 이 긴장을 다루는 알고리즘 장치는 ch05 가 소유한다. 이 장에서 확정할 것은 그것이 **다섯 슬롯 중 “데이터” 의 성질**이지 optimizer 나 목적함수의 문제가 아니라는 점이다. 같은결정이 sleep 라운드에도 그대로 나타난다 — $\mathcal{R}_k$ 를 어느시점의 Θ 로 생성했는지가 라운드의 유효성을 정한다.

[해설] systems 접점은 이 장에서 가장 강한 지점이다. **RL run 은 한 job 안에서 두 roofline 레짐을 교대시킨다**. rollout 생성 구간은 batch 가 얇고 KV cache 를 순회하는 bandwidth-bound decode 이고, gradient 구간은 두꺼운 GEMM 의 compute-bound prefill 이다. 지도학습 run 은후자만 있었다. 독자가 서빙에서 익힌 “prefill/decode 혼합스케줄링” 문제가 training 클러스터 안으로 그대로 들어온 것이다. 그교대의 값은 이 절이 이미 인용한 수치에 들어 있다 — sleep step 하나가 SFT step 하나의 4 배로 보고되고 [LM Need Sleep App. B.5, p.26], 그초과분이 들어갈 회계 칸이 식 (2-3) 의 $2ND$ 항이다.

02.7 continual — 배포 이후에도 움직이는 Θ

정의. continual training 이란 배포 이후에 도착하는경험에 대해 (U- Θ) 를 반복해 도는 레짐이다. 데이터 원천이 시간에대해 열려 있고, k 번째 라운드의 학습 집합 $\mathcal{R}_k$ 가 그 때까지 누적된 경험 $\mathcal{H}_k$ 에 의존한다.

앞의 세 레짐과 다른 점이 세 가지다.

첫째, **첨자가 하나 늘어난다**. step 첨자 s 위에 라운드 첨자 k 가 얹히고, 식 (U- Θ) 의 k 가 바로 이것이다. 한 라운드안쪽은 여전히 § 02.5·§ 02.6 의 레짐 중 하나이며 — [LM Need Sleep] 의 sleep 라운드는 distillation 과 RL 의 조합이다 [LM Need Sleep § 3.3, p.8] — 새로운 것은 라운드들이 **서로의 산출물 위에쌓인다**는 사실뿐이다.

둘째, **데이터가 i.i.d. 가 아니다**. 라운드마다 분포가 옮겨 가므로이전 라운드에서 얻은 것이 다음 라운드에서 지워질 수 있다. 이 현상의이름과 대응책은 ch07 이 소유한다. 여기서 확정할 것은 그것이**레짐의 성질**이지 알고리즘의 결함이 아니라는 점이다 — pre-training 은 $n_{\text{ep}} \approx 1$ 로 한 번 흐르고 끝나므로 같은문제를 겪지 않는다.

셋째, **배포 아티팩트가 갈라진다**. 라운드가사용자별·세션별로 돌면 checkpoint가 하나가 아니라 사용자수만큼 생긴다. corpus의 Θ -경로 논문들이 이 문제에 쓰는 대표적대응은 갱신 대상을 잘라내는 것이다. [LM Need Sleep]은 backbone 전체를 얼리고 새로 확장한 parameter만 갱신한다 (“we freeze all the parameters in the student model and only updates the expanded parameters” [LM Need Sleep § 3.3, p.8]). 그 기제의 평가는 ch21 이한다.

넷째, **평가 슬롯이 열려 있다**. 앞의 세 레짐에서 평가 집합은 run 시작 전에 고정되고 훈련 데이터와 분리된다 — [SEAL]이 RL 학습 집합과평가 문단 사이에 겹침이 없음을 명시하는 것이 그 표준 관행이다 (“there is no overlap between these sets, so we can be sure that there is no data contamination of the test passages due to RL training” [SEAL § B.1, p.20]). continual 레짐에서는 그 분리가 원리적으로 어렵다. 라운드 k 의 데이터는 배포 중에 도착한 것이고, 무엇이 이미 평가 집합에 들어있는지 run 시점에 알 수 없다. 더 근본적으로, 라운드가 쌓이면**무엇에 대해 퇴행했는지를 물을 대상 자체가 커진다** — 지난달에넣은 사실이 이번 달 라운드에서 지워졌는지 확인하려면 지난달의 평가를계속 보관하고 다시 돌려야 한다.

[평가] 이 누적 평가 비용을 비용 4 종의 어느 칸에도 계상한논문을 이 corpus에서 찾지 못했다. Θ -경로 논문들은 라운드당 학습하이퍼파라미터는 표로 실지만, “지난 라운드들을 계속 재평가하는 비용”은 B_s 에도 ρ 에도 들어가 있지 않다. ch25의 통일 회계는 이 칸을비워 둔 채 시작한다.

systems 접점은 Rosetta 사전의 두 행이 동시에 걸린다는 것이다. “모델재배포 = ($U-\Theta$) 1 회”와 “checkpoint 크기 = C_{cap} ”이 continual 레짐에서 합쳐지면, 라운드마다 사용자당 새로운 상태 바이트가생기고 shared-weight batching이 깨진다. E -경로는 이 대가를 치르지않는다 — 상태가 텍스트로 남아 prefill 로만 들어가기 때문이다. **경로 사이의 가장 큰 비용 차이는 정확도가 아니라 이 배치가능성에서 나온다는** 것이 Part III의 논점 중 하나다 (→ ch26, ch29).

02.8 네 칸 표 — 레짐비교

이 장의 핵심 산출물이다. 어떤 논문을 읽든 그 논문의 학습 절차를 이 표의한 행으로 적을 수 있어야 한다.

표 2-1 — 레짐별 네 칸

레짐	무엇이 데이터인가	무엇이 목적함수인가	무엇이 갱신되는가	얼마나 드나
pre-training	주어진 대규모 미분류 코퍼스. run 바깥에서 고정	전 토큰 next-token cross-entropy	Θ 전체	D 가 코퍼스 크기와 같은자릿수. $n_{ep} \approx 1.6ND$
continued pre-training	같은 형식의 추가 코퍼스 (도메인·신규 문서)	동일	Θ 전체 또는부분 집합	$D \ll$ pre-training. [SEAL]은 $n=200 \cdot n=2067$ 문단에서 full-FT [SEAL Table 2]
SFT	(입력, 원하는 출력) 쌍. 사람 시연 또는 모델 출력	출력 구간만의조 건부 cross-entropy (loss mask)	Θ 전체 또는 부분 집합	steps 100, effective batch 32, LR 5e-6 [LM Need Sleep Table 5, p.26]; 또는 2 epochs, batch 10, lr 3e-4 [SEAL § B.2, p.20]

레짐	무엇이 데이터인가	무엇이 목적함수인가	무엇이 갱신되는가	얼마나 드나
RLHF	현재 θ 가 만든 rollout. run 안에서 생성	학습된 θ_{rm} 의 점수 기댓값 - 정규화	$\theta(\text{policy})$. 별도로 θ_{rm} 이 선행 학습됨	step 당 생성 비용이 지배. 생성분은 $2ND$
RLVR	현재 θ 가 만든 rollout	프로그램 검증 결과의 기댓값	$\theta(\text{policy})$ 만	steps 500 [LM Need Sleep Table 5, p.26]
continual / sleep 라운드	$\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s) -$ 누적 경험에서도 델이 만든다	라운드마다 다른 (SFT loss, divergence, 보상 혼합)	θ 의 지정된 부분 집합. 나머지는 동결	라운드당 B_s . corpus 에서 이를 FLOPs 로 보고한 논문은 없다

마지막 행의 마지막 칸이 이 책이 반복해서 부딪히는 공백이다. 레짐의 이름과 하이퍼파라미터는 표로 실리지만, 그것이 몇 FLOPs 인지, 라운드가 몇번 도는지는 대부분의 논문이 적지 않는다. ch25의 통일 비용 회계가 이 칸을 채우는 작업이다.

02.9 Part I의 나머지 장이 채우는칸

표 2-1은 지도이지 설명이 아니다. 각 칸을 깊이 파는 일은 Part I의 나머지장들이 나누어 맡으며, 그 분업이 § 02.2의 다섯 슬롯을 따른다. 이 대응을여기서 못 박아 두면 이후 장들을 “또 다른 배경 지식” 이 아니라 표 2-1의 특정 칸에 대한 확대로 읽을 수 있다.

- “무엇이 갱신되는가” 칸 — θ 의 어느 부분집합을 어떻게 잘라내는가. adapter와 LoRA가 그 도구이며 **ch03** 이 소유한다. 이장은 “부분 갱신이면 식 (2-3)의 weight gradient 향이 빠진다” 는 회계사실까지만 확정했다.
- “무엇이 목적함수인가” 칸 (교사가 있는 경우) — 목표가 정답라벨이 아니라 다른 모델의 출력 분포일 때. distillation이며 **ch04** 가 소유한다. sleep 라운드의 목적함수가 대개 이 형태다.
- “무엇이 목적함수인가” 칸 (보상이 있는 경우) — 식 (2-4)의 $R(\cdot)$ 과 그 최적화. PPO·GRPO·DPO의 내부와 reward hacking은 **ch05** 가 소유한다.
- “무엇이 데이터인가” 칸 (모델이 만든 경우) — $\text{gen}(\cdot; B_s)$ 의 산출물을 다시 학습에 넣을 때 생기는 붕괴위험. **ch06** 이 소유한다. 판별 습관의 Q4가 이 장으로 연결된다.
- “얼마나 드나” 칸의 시간 축 — 라운드를 반복할 때의 열화 ρ 와 그 대응. **ch07** 이 소유한다.
- “얼마나 드나” 칸의 용량 축 — θ 에 얼마나 쓸 수 있는가, 즉 C_{cap} 의 상한. **ch09** 가 소유한다.

즉 Part I의 ch03 – ch09는 표 2-1의 칸 하나씩을 확대한 것이고, ch10만이다른 층 (E) 으로 넘어간다. 이 장은 그 지도의 좌표계를 놓은 것이다.

02.10 “누가 이 값을 학습하는가” — 판별습관

레짐 지도의 목적은 분류 자체가 아니라 판정 습관이다. 논문에 등장하는 임의의 값에 대해 네 질문을 순서대로 던진다.

정의. “누가 이 값을 학습하는가” 판별은 다음 네 질문에 답하는 절차다. Q1. 그 값은 배포 아티팩트에 남는가? 남으면 θ 또는 θ 의 증분이다. 세션과 함께 사라지면 W 이거나 E 다. Q2. 그 값을 움직이는 gradient는 어느 loss의 것인가? inner loss ℓ 이면 (U-W), outer loss $\mathcal{L}$ 이면 (U- θ), gradient가 아예 없으면 (U-E) 다. Q3. 어느시계에서 움직이는가? 토큰 t / step s / 라운드 k / 세션 τ . Q4. 그 값을 만든 데이터는 주어진 것인가, 모델이 만든 것인가? 후자면 ch06의 붕괴 위험이 이 논문에 걸린다.

Q1 – Q3 만으로도 이 책의 세 경로가 갈린다. Q4 는 그 위에 얹히는 정직성점검이다. 네 질문을 [SEAL] 의 세 객체에 적용하면 다음과 같다 — 논문한 편 안에 서로 다른 답이 세 개 들어 있다는 것이 이 습관의 필요성을 그대로 보여준다.

표 2-2 — 판별 습관의 적용 예 ([SEAL] 의 세 객체; 기제 해설은 ch20)

값	Q1 배포에 남는가	Q2 어느 loss	Q3 시계	Q4 데이터 출처	배치
backbone weights (Qwen2.5-7B / Llama-3.2-1B)	남는다	단일 문단 설정에서는 움직이지 않음 (갱신이 LoRA 부분공간에 갇힘). CPT 설정에서는 full-FT 로 전체가 움직임	— / 라운드 k	논문 밖에서 상속	상속된 Θ [SEAL § 4.1, p.6; § B.1, p.20], CPT 열은 full-FT [SEAL Table 2, p.8]
self-edit 생성 정책	남는다 (출하 모델)	outer $\mathcal{L}$ (보상 필터 후 behavior cloning)	라운드 k	모델 생성	(U- Θ) [SEAL § 3.1, pp.3–4]
self-edit 이 만든 가중치 증분	배포 시 남는다	outer $\mathcal{L}$ (생성 텍스트에 대한 causal-LM loss)	문맥마다 1 회, 질의 전	모델 생성	(U- Θ) [SEAL § 3.1, p.4]

같은 절차를 [Letta STC] 에 적용하면 세 행이 전부 비고, (U-E) 만 남는다 — 그 논문은 가중치 공간의 상태를 쓰지 않는다고 스스로 밝힌다 [Letta STC § 7, p.13]. “sleep-time compute” 이라는 이름을 만든 논문과 “sleep” 을 제목에 단 Θ -경로 논문이 **같은 이름 아래 완전히 다른 층을 건드린다**는 사실이, 이 네 질문만으로 즉시 드러난다.

[해설] 이 습관은 ch11 의 판별식과 한 몸이다. ch11 은 “질의도착 전에, 예산 $B_s > 0$ 을 써서, 세 층 중 하나를 실제로 바꾸고, 그 변화가 이후 질의에 쓰이는가” 를 묻는다. 그 네 조건 중 셋째 (“어느 층이 실제로 바뀌는가”) 를 판정하는 도구가 바로 이 절의 Q1 – Q3 다. 레짐 어휘를 먼저 배우는 이유가 여기 있다 — 층을 판정하려면 그 층을 움직이는 절차의 이름을 알아야 한다.

(state, update, cost) 정리

이 장에서 도입한 개념을 세 층 프레임 위에 배치한다.

표 2-3 — 이 장의 개념과 세 층 프레임

이 장의 개념	state (어느 층)	update (어느 식)	cost (비용 4 종의 어느 칸)
pre-training / CPT	Θ 전체	(U- Θ), B_s 주어짐	배포 전 1 회 지불. 비용 4 종에 들어가지 않음 (상각 논의의 바깥)
SFT	Θ 전체 또는 부분집합	(U- Θ), loss mask 적용	sleep 라운드 안에서 실행되면 B_s 의 항

이 장의 개념	state (어느 층)	update (어느 식)	cost (비용 4 중의 어느 칸)
RLHF / RLVR	$\Theta(\text{policy}), \Theta_{\text{rm}}(\text{별도})$	(U- Θ), $\mathcal{B}_s = \text{gen}(\cdot; \Theta^{(s)})$	B_s — 생성분 $2ND$ + gradient 분 $6ND$
continual 라운드	Θ 의 지정 부분집합	(U- Θ), $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$	B_s (라운드당), ρ (라운드 누적 열화), C_{cap} (사용자당 증분 바이트)
token budget D	— (회계 단위)	식 (2-2)	$B_s \cdot B_t$ 를 tokens 단위로 적을 때의 공통 척도
판별 습관 Q1 – Q3	세 층 전부	세 갱신식 전부	— (판정 도구)

W -경로와 E -경로는 이 표에 행이 없다. 그 두 경로는 (U- Θ) 를 돌지 않기 때문이며, 그것이 정확히 이 장이 Θ -경로 독해의 전제인 이유다. W 의 갱신은 (U- W) 로 NM 이 이미다뤘고 ($\rightarrow$ NM ch12), E 의 갱신은 gradient 없는 쓰기 연산 $\text{wr}(\cdot)$ 이다 ($\rightarrow$ ch10).

Worked micro-example — 한 sleep 라운드의 값을 손으로매긴다

이 절은 §02.3의 세 식만으로 실제 논문의 설정 하나를 끝까지 계산한다. 목표는 두 개다: 레짐의 크기를 자릿수로 감각하는 것, 그리고 독자의 1번질문 (“그래서 decode에서 뭘 바꾸는데?”)에 숫자로 답하는 것.

설정. 모델은 Qwen3-1.7B, 즉 $N = 1.7 \times 10^9$ [LM Need Sleep Table 2, p.12의 backbone]. sleep 라운드 안의 SFT 설정은 원문 표 그대로 **steps 100, effective batch size 32, LR 5e-6** [LM Need Sleep Table 5, p.26]. 원문이 적지 않은 세 값은 가정으로 명시하고, 이셋은 논문 수치가 아니다: 시퀀스 길이 $L = 1024$ (가정 A), 한 질의의 wake 생성량 500 토큰(가정 B), 비교용 pre-training 예산 $D_{\text{pre}} = 10^{12}$ 토큰(가정 C). 가정값은 결과의 자릿수를 바꾸지 않으며, 바뀌는 지점은 (e)에서 따로 짚는다. (a)–(d)에서 나오는 산출값은 전부 — D , 라운드 FLOPs, 표 2-4의 상각 열, 손익분기 N_q — 논문이보고한 수치가 아니라 위 하이퍼파라미터와 가정 A–C에서 **이 책임유도한 예시 계산**이다. 논문에서 온 수치는 N 과 위 세하이퍼파라미터뿐이다.

(a) **token budget.** 식 (2-2)에 그대로 넣는다.

$$D = n_{\text{step}} \cdot N_b \cdot L = 100 \times 32 \times 1024 = 3,276,800 \approx 3.28 \times 10^6 \text{ tokens}$$

한 라운드가 소비하는 토큰이 330만 개다. 독자의 감각으로는 중형 서빙 노드한 대의 몇 시간치 처리량이다.

(b) **라운드 FLOPs.** 식 (2-3)의 $6ND$ 다.

$$6ND = 6 \times 1.7 \times 10^9 \times 3.2768 \times 10^6 = 3.34 \times 10^{16} \text{ FLOPs}$$

Θ 의 일부만 갱신해 weight gradient 항이 빠지면 $4ND = 2.23 \times 10^{16}$ 이다 — 정확히 2/3이고, **자릿수는 그대로다**. 부분 갱신의 실제 이득이 FLOPs가 아니라 optimizer state 바이트에 있다는 ch03의 논점이 이 한 줄에서 미리 보인다.

(c) **pre-training 대비.** 같은 N 에 가정 C의 $D_{\text{pre}} = 10^{12}$ 토큰을 넣으면 $6ND_{\text{pre}} = 1.02 \times 10^{22}$ FLOPs다. 비를 잡으면

$$\frac{6ND_{\text{pre}}}{6ND} = \frac{1.02 \times 10^{22}}{3.34 \times 10^{16}} \approx 3.1 \times 10^5$$

즉 가정 C 아래에서 한 sleep 라운드는 **pre-training의 약 3×10^{-6} 이다**. 이 비대칭이 Θ -경로가 논의 대상이 되는 이유 전부터다. 라운드를 하루 한 번씩 3년을 돌려도 pre-training 한 번의 0.4%에 못 미친다 ($1000 \times 3 \times 10^{-6} = 3 \times 10^{-3}$).

(d) **상각 — 식 (A)로 손익분기 N_q 찾기.** 이제 질문을 서빙쪽으로 돌린다. 가정 B에서 한 질의의 wake 비용은 생성 토큰당 $2N$ 이므로

$$C_{\text{wake}} = 2N \times 500 = 1.7 \times 10^{12} \text{ FLOPs/질의}$$

이 값은 prefill 을 세지 않은 **하한**이다. 식 (A) $C_{\text{avg}} = C_{\text{wake}} + C_{\text{sleep}}/N_q$ 에 $C_{\text{sleep}} = 3.34 \times 10^{16}$ 을 넣고 N_q 를 훑는다.

표 2-4 – sleep 라운드 비용의 질의당 상각 ($C_{\text{sleep}} = 3.34 \times 10^{16}$, $C_{\text{wake}} = 1.7 \times 10^{12}$ FLOPs). 이 책의 예시 계산이며 논문이 보고한 값이 아니다

N_q (문맥을 공유하는 질의 수)	C_{sleep}/N_q [FLOPs/질의]	C_{wake} 대비
1	3.3×10^{16}	$1.97 \times 10^4 \times$
10^3	3.3×10^{13}	$19.7 \times$
2.0×10^4	1.7×10^{12}	$1.00 \times$ (손익분기)
2.0×10^5	1.7×10^{11}	$0.10 \times$

손익분기는 $N_q \approx 2 \times 10^4$, sleep 오버헤드를 10% 이하로 누르려면 $N_q \approx 2 \times 10^5$ 가 필요하다. 하나의 문맥에 대해 질의가 **2 만 번에서 20 만 번** 들어와야 이 라운드가 회수된다.

(e) 읽기. 세 가지가 이 계산에서 나온다.

첫째, 가정의 민감도는 선형이고 자릿수를 넘지 않는다. L 을 4096 으로 올리면 (a) 와 (b) 가 4 배가 되고 손익분기도 4 배가 된다. 생성량을 2000 토큰으로 올리면 C_{wake} 가 4 배가 되어 손익분기가 1/4 로 내려간다. 어느 쪽도 N_q 의 자릿수를 $10^4 - 10^5$ 에서 빼내지 못한다.

둘째, 이 계산에서 **decode 쪽 FLOPs 변화는 정확히 0 이다**. sleep 라운드는 wake 의 토큰 수도, 토큰당 비용도 줄이지 않는다. Θ -경로가 파는 것은 decode 비용의 절감이 아니라 **같은 decode 예산에서의 정확도**다. wake 토큰 수를 실제로 줄이는 것은 E -경로의 주장이며, 그 주장의 검증은 ch14 가 한다.

셋째, 표 2-1 의 마지막 칸이 방금 값을 얻었다. 그 값을 논문이 적어 준 것이 아니라, 논문이 적은 하이퍼파라미터 표에서 이 장의 세 식으로 **복원**한 것이다. ch25 의 통일 비용 회계는 이 복원을 corpus 전체에 대해 기계적으로 수행하는 작업이며, 그때 쓰는 도구가 정확히 식 (2-2)·(2-3) 과 식 (A) 다.

[평가] 어디까지 복원할 수 있는가는 이 책의 판단이다. steps·batch·모델 크기가 표에 있으면 B_s 는 논문이 침묵해도 되살아난다 — 위 (a) – (b) 가 그 절차 전부다. 되살아나지 않는 것은 서빙 쪽 변수인 N_q 이며, 그것은 논문이 게을러서가 아니라 논문이 가질 수 없는 값이기 때문이다. 표 2-4 의 오른쪽 열이 유효하려면 그 N_q 를 알아야 한다. 그런데 이 corpus 에서 실서빙의 N_q 분포를 보고한 논문은 없다 ($\rightarrow$ ch25). 그러므로 이 절이 정직하게 산출한 것은 “sleep 라운드가 이득이다/아니다” 가 아니라 **“이 설정에서 이득이 나려면 문맥당 질의가 최소 2×10^4 번 와야 한다”** 는 조건문이다. Part III 가 답해야 할 질문의 형태가 이것이다.

요약

- 모든 레짐은 같은 식 ($U-\Theta$) 를 돈다. 레짐을 가르는 것은 대수가 아니라 데이터 슬롯 B_s 의 출처, 목적함수의 정의, 갱신되는 Θ 의 부분집합 셋뿐이다.
- 임의의 training run 은 데이터 · 목적함수 · optimizer · 스케줄 · 평가의 다섯 슬롯으로 완전히 기술되며, 다섯 슬롯이 먹는 자원은 각각 storage/0/HBM/FLOPs/inference 로 서로 다르다.
- epoch 은 독립 단위가 아니라 $n_{\text{ep}} = D/|D|$ 라는 유도량이고, 레짐의 크기를 재는 단위는 token budget D 다. full fine-tuning 의 비용은 $6ND$, forward 만 도는 생성은 $2ND$, weight gradient 향이 빠진 부분 갱신은 $4ND$ 이며 — 자릿수를 바꾸는 것은 갱신방식이 아니라 D 다.
- pre-training 은 $n_{\text{ep}} \approx 1$ 로 한 번 흐르고, 이후 모든 레짐은 그 위의 작은 증분이다. 이 비대칭이 sleep 라운드의 상각 논의를 성립시킨다.

- SFT 는 loss mask 한 장으로 정의되는 레짐이고, 돌렸다는 사실이 능력향상의 증거가 아니다 — Qwen3-1.7B 에서 SFT 는 AIME-24 를 49.8 에서 47.3 으로 낮춘다 [LM Need Sleep Table 2, p.12].
- RL 레짐의 정의적 특징은 보상이 아니라 데이터가 Θ 의 함수라는 것이며, 그 결과 한 job 안에서 decode 성격과 prefill 성격의 두 roofline 레짐이 교대한다.
- continual 레짐만이 라운드 침자 k 를 갖고, 배포 아티팩트가 사용자별로 갈라져 shared-weight batching 을 깨뜨릴 수 있는 유일한 레짐이다.
- “누가 이 값을 학습하는가” 는 네 질문 (배포에 남는가 / 어느 loss 인가 / 어느 시계인가 / 데이터는 누가 만들었나) 으로 판정되며, 이 판정이 ch11 판별식의 셋째 조건을 집행하는 도구다.

자가 점검체크리스트

- ☐ pre-training · CPT · SFT · RLHF · RLVR · continual 각각에 대해 표 2-1 의 네칸을 보지 않고 채울 수 있다.
- ☐ 임의의 논문 부록에서 “steps 100, batch 32, LR 5e-6” 을 보았을 때 식 (2-2) 와 (2-3) 으로 D 와 FLOPs 를 즉석에서 산정할 수 있다.
- ☐ $6ND$ 의 6 이 어떻게 $2+2+2$ 로 갈라지는지 말하고, 어떤 조건에서 $4ND \cdot 2ND$ 로 내려가는지 설명할 수 있다.
- ☐ Worked micro-example 의 (a) – (d) 를 재현해, 다른 L 과 다른 생성량에서 손익분기 N_q 가 어떻게 움직이는지 계산할 수 있다.
- ☐ 임의의 논문에서 학습되는 값 하나를 골라 Q1 – Q4 에 답하고, 그 값을 $(U-W)/(U-\Theta)/(U-E)$ 중 하나에 배치할 수 있다.
- ☐ η_Θ 와 η_t , C_{cap} 과 chunk 크기 C 를 혼동하지 않고, 이 장의 어느 수식에도 후자가 등장하지 않는 이유를 말할 수 있다.
- ☐ 이 장의 레짐 어휘를 inference 어휘로 옮길 수 있다: 어느 레짐이 “모델재배포” 에 해당하고, 어느 레짐이 “warm-up/사전 컴파일” 에 해당하며, 어느 레짐만이 checkpoint 크기 (C_{cap}) 회계를 사용자 수만큼 곱하게 만드는지 말할 수 있다.

다음 장으로

이 장은 표 2-1 의 셋째 칸 — “무엇이 갱신되는가” — 을 “ Θ 전체 또는 부분집합” 이라고만 적고 넘어갔다. 그러나 Worked micro-example (b) 가 보였듯 그 선택은 FLOPs 를 2/3 로 줄일 뿐이고, 진짜 차이는 다른 곳에 있다: optimizer state 바이트, checkpoint 크기, 그리고 사용자당 저장해야 하는 증분의 크기다. continual 레짐에서 배포 아티팩트가 갈라진다는 § 02.7 의 문제는 바로 이 증분의 크기가 정한다. ch03 은 그 부분집합을 자르는 방법 — full fine-tuning, adapter, LoRA 와 그 변형 — 을 (state, update, cost) 객체로 세우고, “가중치에 쓰기 위해 얼마를 저장해야 하는가” 를 바이트로 답한다. 그 답이 나와야 ch19 의 쓰기 primitive 와 ch26 의 상태 용량 회계를 읽을 수 있다.

ch03. fine-tuning과 PEFT — 가중치에 쓰는법

이 장의 목표 — 이 장을 마치면 다음 넷을 할 수 있어야 한다. 1. full fine-tuning · adapter · LoRA 를 “ Θ 의 어느 부분집합을 움직이는가” 로 구분하고, 각각이 배포 아티팩트로 무엇을 남기는지 말한다. 2. d, L_{layer}, r , 적용 행렬 집합만 주어졌을 때 LoRA delta 의 파라미터 수와 바이트를 손으로 계산한다. 3. merge 된 형태와 분리된 delta 형태가 decode 에서 무엇을 바꾸는지, 특히 shared-weight batching 이 정확히 어디서 깨지는지 설명한다. 4. delta 의 dtype 이 저장 바이트당 정보량을 어떻게 바꾸는지 계산하고, QLoRA 와 delta 양자화가 서로 다른 축임을 구분한다.

왜 필요한가 — Part II 의 Θ -경로 세 장이 이 장 없이는 읽히지 않는다. ch19 는 LoRA 를 논문 장으로 전개하고, ch20 의 SEAL 은 self-edit 의 산출물이 LoRA delta 이며, ch21 의 Language Models Need Sleep 은 CMS 의 low-rank expert $A^{(\ell),j}, B^{(\ell),j}$ 와 폭 d_{low} 를 이 장의 (A, B, r) 위에 세운다. ch09 의 용량 논의는 Understanding LoRA as Knowledge Memory(arXiv:2603.01097) 를 다루는데, 그 논문이 재는 대상이 정확히 이 장의 r 이다. Part III 의 상태 용량 회계(ch26)와 서빙 판정(ch29)은 이 장이 정의하는 delta 아티팩트의 크기 계산을 기준선으로 쓴다.

NM 과의 관계 — Neural Memory 에 대응 장이 없다. 새로 쓴다. 다만 backward pass, optimizer state, outer loop 의 의미는 전제한다 ($\rightarrow$ NM ch02, $\rightarrow$ NM ch04). 이 장이 더하는 것은 하나다: (U- Θ) 를 전부가 아니라 부분적으로 도는 방법, 그리고 그 부분이 파일로 떨어져 배포 아티팩트가 된다는 사실. NM 이 300 쪽에 걸쳐 다룬 W (fast weights) 는 이 장에 한 번도 등장하지 않는다 — 이 장의 모든 기제는 test-time 에 아무것도 움직이지 않는다.

[표기 경고 — 이 장 전용] LoRA 원문은 사전학습 weight 행렬을 W 로 쓰고, 학습되는 소수의 파라미터 집합을 Θ 로 쓴다 [lora § 4.1, lora Eq. 2]. 이 책의 예약 기호와 둘 다 정반대다. 이 책에서 W 는 fast weights(test-time 에 움직이는 상태) 이고 Θ 는 slow weights 전체다. 이 장의 모든 수식은 통일 표기로 옮긴다: 원문 $\Phi_0 \rightarrow \Theta_0$, 원문 $W_0 \rightarrow \Theta_0^{(m)}$, 원문 $\Theta \rightarrow (A, B)$, 원문 $\alpha \rightarrow \alpha_{\text{LoRA}}$, 원문 L (층 수) $\rightarrow L_{\text{layer}}$. attention 의 네 투영 행렬도 같은 규칙을 따른다: 원문 $W_q, W_k, W_v, W_o \rightarrow \Theta^{(q)}, \Theta^{(k)}, \Theta^{(v)}, \Theta^{(o)}$ — 위첨자는 식 (3-2) 의 행렬 인덱스 m 이 취하는 값이며, 시간 첨자가 아니다. 원문 기호는 이 문단과 원문 직접 인용 밖에 등장하지 않는다. 또 이장은 LoRA 를 적용한 행렬들의 집합을 S 로 쓴다 — 이 장이 도입해 이 책 전체의 예약 기호가 된 것이고 (ch09·ch19 가 그대로 쓴다), 정의는 이 장이 소유한다. 예약 기호 S_t (momentum)· $S(c; B_s)$ (오프라인 처리 절차) 어느 쪽과도 무관하다.

03.1 full fine-tuning — 기준선, 그리고 그것이 남기는 것

full fine-tuning 은 사전학습된 slow weights Θ_0 전체를 downstream 목적함수의 gradient 로 움직여 새 Θ 를 얻는 절차다. 움직임에 구조적 제약이 없다는 것이 정의의 전부다.

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{D}} \sum_{t=1}^{|y|} \log P_{\Theta}(y_t | x, y_{<t}) \quad (3-1)$$

이 식이 말하는 것은 단순하다. 학습 집합 $\mathcal{D}$ 의 모든 (입력, 정답) 쌍에 대해 정답 토큰의 로그확률을 합이 커지도록 Θ 를 옮기고, 초기값은 Θ_0 다 [lora Eq. 1]. 이것이 ch01이 세운 표준형 (U- Θ)를 실제 목적함수로 채운 가장 평범한 형태이며, 이 장의 나머지 방법은 전부 “같은 목적함수를, 더 작은 부분공간에서” 푸는 변형이다.

표준형 (U- Θ)와 대조하면 이 장 전체의 위치가 정해진다. 표준형은 $\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k)$ 이고 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 인데, 식 (3-1)에서는 $\mathcal{R}_k$ 가 사람이 미리 만들어 놓은 $\mathcal{D}$ 로 고정되어 $\text{gen}(\cdot)$ 이 항등이 된다. 또 반복 첨자가 sleep 라운드 k 가 아니라 한 번의 adaptation run 안의 SGD step s 다 — 이 장은 step 첨자로 s 만 쓰고, k 는 ch04 이후의 sleep 라운드에 남겨 둔다. 즉 이 장의 방법은 (U- Θ)의 왼쪽 절반 (무엇을, 어떻게 움직이는가)만 채우고 오른쪽 절반 (학습 집합을 어떻게 만드는가)은 비워둔다. 그 빈칸을 채우는 것이 ch04 이후의 일이다.

먼저 왜 이 절차가 필요한지부터 정한다. 문맥에 예시를 넣는 것으로는 대체되지 않기 때문이다. GPT-3 175B에서 few-shot과 fine-tuning의 격차는 MNLI-m 40.6 대 89.5, RTE 69.0 대 85.4다 [lora Table 8, lora App. A]. 40점 이상의 격차는 프롬프트 기법으로 메울 수 있는 종류가 아니다. **가중치에 쓰는 것과 문맥에 쓰는 것은 같은 일이 아니다** — 이 사실이 Θ -경로 전체의 존재 이유이며, Part III에서 E-경로와 비교할 때 다시 불러 나온다.

문제는 그 대가다. full fine-tuning의 산출물은 Θ_0 와 크기가 같은 새 checkpoint다. 논문의 표현으로 “the new model contains as many parameters as in the original model” [lora § 1]이고, 더 정확하게는 “for each downstream task, we learn a different set of parameters $\Delta\Phi$ whose dimension $|\Delta\Phi|$ equals $|\Phi_0|$ ” [lora § 2]다. GPT-3 175B의 checkpoint는 350GB이므로 [lora § 4.2], task 하나마다 350GB가 새로 생긴다.

[해설] 독자의 어휘로 옮기면 이렇다. full fine-tuning은 **모델 재배포**다 — Rosetta 사전의 “모델 재배포 $\leftrightarrow$ (U- Θ) 1 회” 행이 바로 이 절차를 가리킨다. 배포 파이프라인 · 이미징 · 로깅 전략이 전부 달려 온다. task가 n 개면 아티팩트도 n 개이고, 그 n 개는 서로 다른 GPU에 올라가야 한다. 100개 task를 독립적으로 fine-tune하면 약 35TB다 [lora § 4.2 각주 4].

쓰기 시점의 비용도 배포 비용과 별개로 존재한다. GPT-3 175B의 full fine-tuning은 VRAM 1.2TB를 쓴다 [lora § 4.2]. 이 숫자의 출처를 정확히 아는 것이 이 장의 나머지를 이해하는 열쇠다 — 파라미터 자체가 아니라 **optimizer state와 gradient**가 지배한다 [lora § 1]. Adam 계열은 파라미터당 1차 · 2차 moment 두 벌을 들고 있어야 하고, 여기에 gradient 한 벌이 더 붙는다 ($\rightarrow$ NM ch02의 optimizer-as-object). 따라서 학습되는 파라미터 수를 줄이면 메모리는 파라미터 수보다 **가파르게** 줄어든다. 이것이 PEFT의 계산적 동기이며, 뒤에 나올 “1.2TB $\rightarrow$ 350GB”라는 수치의 기제다.

03.2 adapter — 삽입의 대가는 depth이고, depth의 대가는 decode다

adapter는 열린 Θ_0 의 층 사이에 작은 학습 가능 모듈을 직렬로 삽입하고 그 모듈만 학습하는 방식이다 (Houlsby et al. 2019). 저장 효율의 관점에서는 성공적이다 — GPT-3에서 Adapter_H는 7.1M 파라미터로 WikiSQL 71.9 / MNLI-m 89.8을, 40.1M으로 73.2 / 91.5를 낸다 [lora Table 4]. 175B 대비 10^{-4} 수준의 아티팩트로 full fine-tuning(73.8 / 89.5)에 근접한다.

문제는 저장이 아니라 계산 그래프다. adapter를 삽입한다는 말은 forward의 depth가 늘어난다는 뜻이고, 늘어난 depth는 순차적으로만 처리된다. 논문은 이를 정면으로 지적한다: “large neural networks rely on hardware parallelism to keep the latency low, and adapter layers have to be processed sequentially. This makes a difference in the online inference setting where the batch size is typically as small as one.” [lora § 3]

이 주장은 측정으로 뒷받침된다. GPT-2 medium, NVIDIA Quadro RTX8000, single forward pass 100 회 평균 [lora Table 1, lora App. B]:

표 3-1. adapter 의 inference latency 오버헤드 (ms, 괄호는 Fine-Tune/LoRA 대비 증가율) [lora Table 1]

설정	Fine-Tune / LoRA	Adapter _L	Adapter _H
batch 32, seq 512, 0.5M	1449.4 $\pm$ 0.8	1482.0 $\pm$ 1.0 (+2.2%)	1492.2 $\pm$ 1.0 (+3.0%)
batch 16, seq 256, 11M	338.0 $\pm$ 0.6	354.8 $\pm$ 0.5 (+5.0%)	366.3 $\pm$ 0.5 (+8.4%)
batch 1, seq 128, 11M	19.8 $\pm$ 2.7	23.9 $\pm$ 2.1 (+20.7%)	25.8 $\pm$ 2.2 (+30.3%)

순차 depth 의 비용은 batch 가 작을수록 커진다. 이 표는 그것을 세줄로 말한다. batch 32 / seq 512 에서는 3% 이하이고, batch 1 / seq 128 에서는 30%까지 간다. 독자에게 이 패턴은 낯설지 않다 — batch 가 작은 decode 는 compute-bound 가 아니라 memory-bound 이며, 층하나를 추가할 때 실제로 지불하는 것은 FLOPs 가 아니라 kernel launch 와 그사이의 동기화다. adapter 의 bottleneck GEMM 은 FLOPs 기준으로 무시할수준이지만 launch 횟수 기준으로는 무시되지 않는다. 그리고 online serving 의 decode 는 정확히 표의 마지막 줄에 있다.

[평가] 논문은 여기서 한 걸음 더 나가 “이 문제는 모델을 shard 하면 더 나빠진다 — AllReduce 와 Broadcast 같은 동기 연산이 늘기때문” 이라고 쓴다 [lora § 3]. 방향은 옳지만 **측정 없이 단언된다.** Table 1 과 Fig. 5 는 모두 단일 GPU 실험이고, tensor-parallel 환경의 adapter 오버헤드를 켜 숫자는 이 논문에 없다. 이 책은 이 주장을 “그럴듯한 기제” 로 인용하되 근거 있는 사실로 취급하지 않는다.

삽입 대신 **문맥을 쓰는** 계열도 있다. prefix/prompt tuning 은 학습 가능한 special token 을 입력 앞에 붙이는 방식인데, 이 계열은 depth 가 아니라 다른 예산을 먹는다 — “reduce the model’s usable sequence length” [lora § 1]. 독자의 회계로 옮기면 학습 파라미터를 늘릴수록 KV cache 예산과 유효 문맥이 줄어든다는 뜻이고, 그래서 파라미터를 늘리면 성능이 단조롭게 오르지 않는다. GPT-3 에서 PrefixEmbed 는 0.4M $\rightarrow$ 6.4M 구간에서 WikiSQL 55.9 / 58.7 / 60.6 / 63.1 / 55.9 로 마지막에 무너지고, PrefixLayer 도 5.1M $\rightarrow$ 76.1M 구간에서 68.5 / 69.8 / 70.1 / 66.4 / 64.9 로 꺾인다 [lora Table 15]. 논문 자신의 관찰도 같다: “prefix tuning is difficult to optimize and ... its performance changes non-monotonically in trainable parameters” [lora § 3]. **PEFT 계열은 저마다 다른 예산을 소모하며, adapter 는 depth 를, prefix 계열은 sequence 를 판다.** LoRA 가 파는 것이 무엇인지는 § 03.5 에서 확인한다.

품질에서도 adapter 가 일방적으로 지는 것은 아니다. E2E NLG 에서 GPT-2 Large 의 CIDEr 는 Adapter_L(0.88M) 2.49 가 LoRA(0.77M) 2.47 보다 높고 [lora Table 3], WebNLG GPT-2 Large 의 BLEU-All 은 Adapter_L(23M) 57.7 이 LoRA(0.77M) 57.0 보다 높다 [lora Table 14]. 즉 adapter 가 밀리는 측은 품질이 아니라 **merge 가능성**이다.

[평가] **다만 이 비교표들의 baseline 상당수는 재실행된 값이 아니다.** [lora Table 2, lora Table 3] 의 캡션은 “* indicates numbers published in prior works” 라고 밝히고, RoBERTa base/large 의 full fine-tuning, DeBERTa XXL full fine-tuning, GPT-2 M/L full fine-tuning, FT_{Top2}, PreLayer 가 모두 그 표시가 붙은 행이다. 즉 이 장이 인용하는 “full fine-tuning 대비” 격차의 상당수는 학습프로토콜이 LoRA 행과 같다는 보장이 없는 인용 수치와의 비교다. 이 책은 그 격차를 순서 정보로만 쓰고 소수점 차이를 논거로 쓰지 않는다.

mergeability 가 1 등 요구사항으로 올라온 것은 이 관찰 때문이며, 다음 절이 그 요구사항을 만족시키는 구성을 다룬다.

03.3 LoRA — 변화량에 rank 를 걸다

LoRA가 상속한 질문은 명시적이다. 선행 연구 (Li et al. 2018a, arXiv:1804.08838; Aghajanyan et al. 2020, arXiv:2012.13255) 는 과매개화된 학습 모델이 낮은 intrinsic dimension 위에 놓인다는 것을 objective landscape 수준에서 보였다. LoRA가 받아 가는 열린 질문은 **변화량 자체로** 옮긴 형태다: “We hypothesize that the change in weights during model adaptation also has a low ‘intrinsic rank’ ” [lora § 1, lora § 4.1].

정의. LoRA(Low-Rank Adaptation) 는 slow weights 의 증분 $\Delta\Theta$ 를 두 저차원 행렬의 곱으로 제약하고 그 두 행렬만 학습하는 (U- Θ) 의 부분공간 버전이다. **rank** r 은 그 곱의 내부 폭이며, 이 방법의 유일한 용량 knob 이다.

$$\Delta\Theta^{(m)} = B^{(m)} A^{(m)}, \quad B^{(m)} \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A^{(m)} \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{out}}, d_{\text{in}}) \quad (3-2)$$

여기서 m 은 LoRA 를 적용한 개별 weight 행렬의 인덱스다 [lora § 4.1]. 층 출력은 두 경로의 합이 된다.

$$y = \Theta_0^{(m)} x + \frac{\alpha_{\text{LoRA}}}{r} B^{(m)} A^{(m)} x \quad (3-3)$$

이 식이 말하는 것은 “열린 경로 하나와 저랭크 경로 하나가 **나란히** 붙고 출력에서 더해진다” 이다. 나란히 붙는다는 것이 § 03.2 와의 결정적 차이이다 — depth 가 늘지 않으므로 순차 처리가 강제되지 않고, 두 항이 같은 입력 x 에 대한 선형사상이므로 배포 전에 하나로 합칠 수 있다.

[해설] 식 (3-3) 의 정확한 출처를 밝혀 둔다. [lora Eq. 3] 은 scaling **없이** $h = W_0 x + B A x$ 로 쓰였고, α_{LoRA}/r 배율은 그 식 직후 산문에만 나온다 (“We then scale $\Delta W x$ by $\frac{\alpha}{r}$ ”, [lora § 4.1]). 실제구현은 둘을 합친 것이므로 이 책은 합친 형태를 표준으로 삼는다. α_{LoRA} 는 $8 \cdot 16 \cdot 32$ 같은 상수이며, 이 책의 예약 기호 α_t (retention gate, 남기는 비율 $\in [0, 1]$) 와 **아무 관계가 없다**. 시간 첨자가 없다는 것으로 구분한다.

초기화는 방법의 성질을 결정한다. $A^{(m)}$ 은 Gaussian, $B^{(m)}$ 은 0 이므로 학습 시작 시점에 $\Delta\Theta = 0$ 이다 [lora § 4.1, lora Fig. 1]. 즉 쓰기가 **항등원에서 출발한다** — 쓰기 전 모델이 정확히 보존된다는 이 성질은 Θ -경로에서 반복 등장하며, ch20 의 SEAL 과 ch21 의 CMS expert 가 같은 구성을 승계한다.

학습 규칙은 (U- Θ) 를 (A, B) 로 제한한 것이다.

$$(A, B)_{s+1} = (A, B)_s - \eta_{\Theta} \nabla_{(A, B)} \mathcal{L}(\mathcal{D}; \Theta_0 + \frac{\alpha_{\text{LoRA}}}{r} B_s A_s), \quad \Theta_0 \text{ frozen} \quad (3-4)$$

표준형 (U- Θ) 와의 차이는 세 가지이고, 셋 다 이 책의 논지에 직접관계한다.

1. **부분공간 제약.** gradient 를 Θ 전체가 아니라 (A, B) 에 대해 취하므로 $\Theta_s - \Theta_0$ 가 모든 step s 에서 $\text{rank} \leq r$ manifold 에 갇힌다 [lora § 4.1].
2. B_s **knob 이 없다.** 표준형의 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 에서 $\text{gen}(\cdot)$ 이 항등이다. 학습 집합은 사람이 만든 고정 dataset (GLUE, WikiSQL, SAMSum, E2E, DART, WebNLG) 이고 [lora § 2, lora App. C], replay 도 합성 데이터도 없다. **sleep 예산이라는 축 자체가 존재하지 않는다.**
3. **첨자가 라운드가 아니다.** 식 (3-4) 의 s 는 한 번의 adaptation run 안의 평범한 SGD step 이며, sleep 라운드 첨자 k 가 **아니다** (GPT-3 은 2 epoch, GPT-2 는 5 epoch [lora App. D]). 이 구분이 이 장의 논지다 — step 을 아무리 많이 돌려도 그것은 라운드 하나 안의 일이고, 라운드가 반복되지 않으므로 누적 열화 문제가 제기되지 않는다. 라운드 첨자 k 가 실제로 1 보다 커지는 것은 ch20·ch21 에 가사다.

품질은 GPT-3 175B 에서 full fine-tuning 과 사실상 같다. LoRA 4.7M 이 WikiSQL 73.4 / MNLI-m 91.7 / SAMSum 53.8·29.8·45.9, full fine-tuning 175,255.8M 이 73.8 / 89.5 / 52.0·28.0·44.5 다 [lora Table 4]. 학습파라미터가 2.7×10^{-5} 배 ($4.7\text{M} / 175,255.8\text{M} \approx 1/37,000$, 이 책의 산술) 인데 MNLI-m 과 SAMSum 에서는 오히려 앞선다.

[평가] 그러나 완승은 아니며, 가장 인상적인 격차 하나는 공정한 비교가 아니다. 다섯 가지를 병기한다. (i) full fine-tuning 에 지는 지점이 남는다 — QQP 에서 LoRA 는 RoBERTa 두 크기 모두에서 낮다 (base 90.8 대 91.9, large 91.6 대 92.2) [lora Table 2]. 저데이터 레짐 MNLI-1k 에서도 LoRA 85.6 이 Fine-Tune 85.8 보다 낮다 [lora Table 16]. (ii) “다른 방법과 직교적으로 조합된다” 는 주장의 절반이 옳은 결과다 — LoRA+PrefixLayer(52.8M) 는 LoRA 단독 (37.7M) 보다 WikiSQL 72.9 대 73.8, MNLI 90.2 대 91.6 으로 **더 나쁘다** [lora Table 15]. (iii) 가장 널리 인용되는 격차는 초기화 이점을 포함한다 — GLUE 의 MRPC·RTE·STS-B 에서 LoRA 모듈은 “our best MNLI checkpoint” 로 warm-start 된다 [lora App. D.1]. 그 이점을 제거한 † setup 에서 RoBERTa large 의 RTE 는 87.4 에서 **85.2 로 떨어져** full fine-tuning 의 86.6 보다 낮아진다 [lora Table 2]. (iv) GPT-3 의 최고성적도 순수 LoRA 가 아니다 — WikiSQL 최고치는 LoRA+PrefixEmbed($r^{(q)} = r^{(v)} = 64$, 302.1M) 의 76.2 이고, LoRA 단독 최고치 74.0 보다 2.2 점 높다 [lora Table 15]. (v) 그리고 이 절의 GPT-3 수치는 전부 validation accuracy 다 — 별도 test split 평가가 없고 “We report the best validation performance from each run” 이며 [lora App. D.4], 표준편차도 항목별이 아니라 task 당 대표값 하나뿐이다 [lora § 5.5]. 이 책은 LoRA 를 “full fine-tuning 과 대등한 write primitive” 로 취급하되, “언제나 우월하다” 로는 취급하지 않는다.

쓰기 시점 비용은 § 03.1 에서 예고한 대로 움직인다. GPT-3 175B 에서 VRAM 은 1.2TB → 350GB, throughput 은 V100 당 32.5 → 43.1 tokens/s(논문 표현으로 25% speedup) 다 [lora § 4.2, lora § 4.2 각주 5]. 절감의 출처는 FLOPs 가 아니라 optimizer state 와 gradient 를 대부분 들고 있지 않아도 된다는 것이다 [lora § 1] — 즉 backward 의 계산량은 거의 그대로이고 **메모리만** 줄어든다. 이구분은 Θ -경로의 sleep 예산을 회계할 때 반복해서 필요하다.

[평가] 논문의 절차 서술 하나가 부록과 어긋난다. [lora § 4.1] 은 “we simply set α to the first r we try and do not tune it” 이라고 선언하지만, 보고된 값은 RoBERTa base $r = 8$, $\alpha = 8$ (일치), RoBERTa large $r = 8$, $\alpha = 16$ (불일치), DeBERTa $r = 8$, $\alpha = 8$ (일치), GPT-2 $r = 4$, $\alpha = 32$ (불일치) 다 [lora Table 9, lora Table 10, lora Table 11]. 네 설정 중 둘이 선언한 절차를 따르지 않는다. α_{LoRA} 를 “튜닝하지 않아도 되는 상수” 로 소개하는 2 차 문헌이 많으나, 원문의 근거는 그만큼 약하다.

03.4 어디에 걸 것인가, r 은 얼마인가 — 그리고 논문이 자기 표와 어긋나는 자리

식 (3-2) 는 두 개의 자유도를 남긴다. 어느 **weight 행렬에 걸 것인가** (집합 $\mathcal{S}$) 와 r 을 얼마로 둘 것인가다. 이들은 품질 knob 이면서 동시에 § 03.6 의 크기 식에 그대로 들어가는 **바이트 knob** 이므로, systems 독자에게는 하나를 고르면 다른 하나가 결정되는 관계다.

논문은 학습 파라미터를 18M 으로 고정한 채 $\mathcal{S}$ 만 바꾼다. GPT-3 175B, 96 layers [lora Table 5, lora § 7.1]:

표 3-2. 적용 행렬 집합에 따른 성능 (GPT-3 175B, 학습 파라미터 18M 고정) [lora Table 5]

$\mathcal{S}$	r	WikiSQL (± 0.5)	MultiNLI (± 0.1)
$\{\Theta^{(q)}\}$	8	70.4	91.0
$\{\Theta^{(k)}\}$	8	70.0	90.8
$\{\Theta^{(v)}\}$	8	73.0	91.0
$\{\Theta^{(o)}\}$	8	73.2	91.3
$\{\Theta^{(q)}, \Theta^{(k)}\}$	4	71.4	91.3

S	r	WikiSQL (± 0.5)	MultiNLI (± 0.1)
$\{\Theta^{(q)}, \Theta^{(v)}\}$	4	73.7	91.3
$\{\Theta^{(q)}, \Theta^{(k)}, \Theta^{(v)}, \Theta^{(o)}\}$	2	73.7	91.7

[평가] 이 표의 결론은 논문 본문의 결론과 다르다. [lora § 7.1] 은 “adapting both W_q and W_v yields the best result” / “Adapting both W_q and W_v gives the best performance overall” 이라고 쓴다. 그러나 같은 표에서 $\{\Theta^{(q)}, \Theta^{(k)}, \Theta^{(v)}, \Theta^{(o)}\} (r = 2)$ 는 WikiSQL 73.7 로 동률이고 MultiNLI 91.7 로 $\{\Theta^{(q)}, \Theta^{(v)}\}$ 의 91.3 보다 높다. 표가 실제로 보여주는 것은 “ $\{\Theta^{(q)}, \Theta^{(v)}\}$ 가 최선” 이 아니라 “같은 파라미터 예산이면 rank 를 낮추더라도 더 많은 행렬에 나누어주는 편이 열등하지 않다” 이다. 이 책은 후자를 채택한다. 2 차 문헌이 널리 인용하는 “q 와 v 에 걸러라” 라는 실무 규칙은 이 표에서 유도 되지 않는다.

r 자체의 스윙은 더 조심해서 읽어야 한다. GPT-3 $\{\Theta^{(q)}, \Theta^{(v)}\}$ 의 WikiSQL 은 $r = 1, 2, 4, 8, 64$ 에서 각각 73.4 / 73.3 / 73.7 / 73.8 / 73.5 다 [lora Table 6]. 다섯 점의 전체 폭은 0.5 이고, 같은 표의 캡션이 밝힌 변동 폭은 $\pm 0.5\%$ 다 [lora Table 6].

[평가] rank 차이를 이 데이터로는 해상할 수 없다. 스윙전체의 폭이 논문 자신이 선언한 변동 폭과 같은 크기이므로, 이 표는 rank 들사이의 순서조차 결정하지 못한다. “ $r = 1$ 이면 충분하다” 는 널리 인용되는 결론은 자기 오차막대 안에 근거를 두고 있다. 이 책은 이 스윙을 “작은 r 이 큰 r 에 크게 밀리지 않았다” 는 부재 서술로만 쓰고, 특정 r 의 우열 논거로 쓰지 않는다.

작은 모델에서는 결론이 뒤집힌다. GPT-2 Medium 의 E2E 스윙에서 저자 스스로 “Unlike on GPT-3 where $r = 1$ suffices for many tasks, here the performance peaks at $r = 16$ for validation loss and $r = 4$ for BLEU” 라고 쓴다 [lora Table 18 캡션, lora App. H.2]. 게다가 같은 캡션이 “some of our hyperparameters are tuned on $r = 4 \dots$ and thus might not be optimal for other choices of r ” 이라고 인정하므로, rank 축을 단독 변수로 읽을 수도 없다. 논문 자신의 정리가 정확하다: “the relationship between model size and the optimal rank for adaptation is still an open question” [lora App. H.2].

S 의 범위에도 선언된 한계가 있다. 이 논문의 모든 “어디에 걸것인가” 결론은 self-attention 안으로 한정된다 — “We limit our study to only adapting the attention weights for downstream tasks and freeze the MLP modules $\dots$ both for simplicity and parameter-efficiency” [lora § 4.2]. MLP 는 검증되어 배제된 것이 아니라 편의로 배제됐다. 사실 지식이 MLP 에 저장된다고 보는 모델 편집 계열 (ROME·MEMIT) 과 정면으로만나는 지점이 여기인데, 이 논문에는 그 축의 증거가 없다. 두 계열의 충돌은 ch19 가 다룬다.

그렇다면 그 delta 는 실제로 무엇을 쓰는가. 논문의 답은 “새 방향을 넣는것이 아니라 이미 있는 방향을 키운다” 이다. GPT-3 의 48 번째 층에서 $\|\Theta_0^{(q)}\|_F = 61.95$ 이고, $r = 4$ 일 때 $\|\Delta\Theta^{(q)}\|_F = 6.91$ 인 반면 $\Delta\Theta^{(q)}$ 의 top- r 부분공간으로 사영한 $\Theta_0^{(q)}$ 의 norm 은 0.32 에 불과하다 — 논문은 이비를 feature amplification factor 21.5 로 부른다 [lora Table 7, lora § 7.3]. 즉 delta 는 base 가 이미 갖고 있으나 강조하지 않던 방향을 골라증폭한다는 것이 논문의 해석이다.

[평가] 이 서사는 rank 에 따라 뒤집히고, 근거는 층 하나다. [lora § 7.3] 은 “the amplification factor is rather huge: 21.5” 를 강조하지만, [lora App. H.4] 는 $r = 64$ 에서 그 값이 “약 2” 에 불과하다고 인정한다 (3.57 / 1.90). 21.5 는 $r = 4$ 설정의 산물이다. 게다가 Table 7 과 Fig. 3·4 의 모든 수치는 96 개 층 중 48 번째층 하나에서 나온다 — “We only look at the 48th layer (out of 96) due to space constraint” [lora § 7.2]. 부록 H.1 이 층 1/32/64/96 으로 확장하지만 여전히 96 개 중 4 개이고, Table 7 의 norm 수치는 층 48 외에는 어디에도 없다. 따라서 “ Θ -경로의 쓰기는 추가가 아니라 증폭이다” 라는 명제는 이 논문에서 가설로만 성립한다. 이 질문은 ch09 (용량) 와 ch19 (모델 편집과의 대조) 가 받는다.

[해설] rank 가 늘어날 때 실제로 무엇이 늘어나는지에 대한 **용량 상한** 논의는 이 장의 소관이 아니다 (→ ch09). 이 장이 확정하는 것은 그보다 앞선 것 하나다: r 과 S 는 품질 hyperparameter 이기 이전에 **아티팩트 크기를 결정하는 두 변수**이며, 식 (3-6) 에서 정확히 선형으로 들어간다.

03.5 merge 와 서빙 형태 — $L_w = 0$ 은 무엇과 맞바꾼 것인가

정의. merge 는 학습이 끝난 뒤 delta 를 base 에 흡수해 하나의 weight 행렬로 만드는, 배포 시점의 산술 연산이다. gradient step 이 아니다.

$$\Theta^{(m)} \leftarrow \Theta_0^{(m)} + \frac{\alpha_{\text{LoRA}}}{r} B^{(m)} A^{(m)}, \quad \text{task switch: } \Theta_0^{(m)} = \Theta^{(m)} - \frac{\alpha_{\text{LoRA}}}{r} B^{(m)} A^{(m)}, \quad \Theta'^{(m)} = \Theta_0^{(m)} + \frac{\alpha_{\text{LoRA}}}{r} B'^{(m)} A'^{(m)} \quad (3-5)$$

이 한 줄이 이 장에서 가장 systems 적인 대상이다 [lora § 4.1]. merge 하면 GEMM shape 가 base 모델과 글자 그대로 같아진다 — 층이 늘지도, sequence 가 줄지도 않는다. 논문이 “we do not introduce any additional latency during inference compared to a fine-tuned model **by construction**” 이라고 쓰는 근거가 이것이다 [lora § 4.1]. 비용 4 종의 언어로: $L_w = 0$ 이다.

정의. 같은 delta 를 서빙에 올리는 방식은 두 가지이며, 이 책은 이를 **서빙 형태**라 부른다. merge 된 형태는 식 (3-5) 를 배포 전에 실행해 $\Theta_0 + \Delta\Theta$ 하나만 GPU 에 두는 것이고, **분리된 delta 형태**는 Θ_0 를 공유한 채 (A, B) 를 요청마다 별도 경로로 적용하는 것이다.

두 형태의 차이는 latency 가 아니라 누가 무엇을 공유하는가다.

merge 된 형태에서 한 forward pass 는 **정확히 하나의 task**에만 유효하다. 논문 자신이 인정한다: “it is not straightforward to batch inputs to different tasks with different A and B in a single forward pass, if one chooses to absorb A and B into W to eliminate additional inference latency” [lora § 4.2 Limitations]. Rosetta 사전의 “batch 로 weight 공유 $\leftrightarrow \Theta$ -경로에서 깨짐” 행이 가리키는 지점이 바로 여기다 — **shared-weight batching** 은 서빙의 대전제이고, merge 는 그것을 대가로 $L_w = 0$ 을 산다.

분리된 delta 형태는 그 대전제를 지킨다. Θ_0 는 배치 전체가 공유하고, 요청마다 다른 (A, B) 만 별도 경로로 태우면 된다. 대신 층마다 GEMM 두 개가 추가된다.

[해설] 이 책의 산술. 적용 행렬 하나당 base 경로는 토큰당 약 d^2 MAC 이고, LoRA 경로는 $d \times r$ 과 $r \times d$ 두 번, 즉 $2dr$ MAC 이다. 비율은 $2r/d$ 다. GPT-3 ($d = 12,288$, $r = 4$) 이면 $8/12,288 \approx 0.065\%$, $d = 4096 \cdot r = 16$ 이면 $32/4096 = 0.78\%$ 다. **FLOPs 관점에서는 무시할 수준이다.** 그러나 batch 가 작은 decode 의 병목은 FLOPs 가 아니라 kernel launch 와 메모리 왕복이며, 표 3-1 이 adapter 에서 보인 +20.7~30.3%가 경고하는 바가 정확히 그것이다. LoRA 의 두 GEMM 은 **병렬**이라 depth 를 늘리지 않으므로 adapter 만큼 나쁠 이유는 없다. 다만 이 산술은 이 책의 것이며 논문의 것이 아니다.

[평가] 그리고 논문에는 그 숫자가 하나도 없다. 논문은 “many customized models that can be swapped in and out on the fly” 를 팔면서 [lora § 4.2] 동시에 merge 하면 배치가 안 된다고 인정하고, 분리된 delta 형태의 latency 는 “가능하다” 고만 쓰고 **측정하지 않는다** [lora § 4.2 Limitations]. 나아가 latency 근거 전체의 규모가 주장의 규모와 다르다 — 표 3-1 의 모든 측정은 GPT-2 medium 단일 GPU 이고 [lora Table 1, lora App. B], 정작 배포 논거의 대상인 GPT-3 175B 에서는 어떤 latency 도 측정되지 않았다. **품질의 상한과 latency 근거의 상한이 서로 다른 모델 · 다른 하드웨어에 있다.**

이 책의 실험 [X2]는 세 경로의 서빙 구조를 네 줄로 정리한다 (Θ -경로는 delta 형태와 직접 편집으로 갈린다): E-경로는 shared-weight batching **이유지된다** (상태가 프롬프트로 들어오므로 가중치는 전부 공유), W-경로는 **부분적으로 깨진다** (세션마다 별도 fast-weight 상태가 HBM에 상주), Θ -경로의 LoRA는 **delta 단위로 깨진다** (요청마다 사용자 delta를 로드·머지해야 하며 배치 안의 사용자가 다르면 가중치가 다르다), Θ -경로의 직접 편집은 **완전히 깨진다** (개인화하려면 모델 전체 사본) [X2 Q4]. 마지막 항목이 delta 형태의 존재 이유다 — 직접 편집 ($\rightarrow$ ch19)은 공유 가중치를 제자리에서 고치므로 애초에 아티팩트가 생기지 않고, 그래서 사용자별 개인화가 구조적으로 불가능하다.

트래픽의 성격도 경로마다 다르다. Θ -경로는 **요청당 delta 로드 (수 MB) + 갱신은 sleep 라운드에만**이므로 읽기 편향이며 사용자별이다 [X2 Q4]. W-경로가 토큰마다 상태를 read-modify-write하는 쓰기 편향인 것과 대조된다 ($\rightarrow$ NM ch10).

[평가] 서빙 fleet 관점에서 이 논문이 비워 둔 칸은 정확히 네 개다. (a) 여러 task를 동시에 배칭할 때의 비용 모델, (b) delta swap 의지연과 정합성, (c) 여러 delta를 동시에 엮었을 때의 간섭, (d) 반복 write의 열화 ρ . 네 항목이 이 논문에 없다는 것은 사실 서술이고, **이 넷이 Θ -경로를 sleep-time write로 쓰려는 순간 전부 필수 항목이 된다**는 것은 이 책의 판단이다. (c)는 ch09가 다루는 후속 연구가 처음으로 측정하고, (d)는 ch20·ch21까지 가서야 부분적으로 채워진다. (a)와 (b)는 이 corpus 안에서 끝내 채워지지 않으며, 그 사실 자체를 Part III(ch29)가 결론의 재료로 쓴다.

03.6 delta 아티팩트의 크기, 그리고 dtype이 정하는 정보밀도

정의. delta 아티팩트는 $(U-\Theta)$ 의 산출물 중 Θ_0 에 더해지는 부분만 따로 저장한 파일이다. LoRA의 경우 그 크기는 정확히 다음 식으로 주어진다.

$$|\Delta\Theta| = 2 N_{\text{mod}} d r, \quad N_{\text{mod}} = L_{\text{layer}} \times |\mathcal{S}| \quad (3-6)$$

N_{mod} 는 LoRA를 적용한 weight 행렬의 총 개수이고, 계수 2는 A 와 B 두 장에서 온다 [lora § 5.1]. 이 식이 $d \cdot L_{\text{layer}} \cdot r \cdot \mathcal{S}$ 넷에만 의존하고 모델 파라미터 총수에는 의존하지 않는다는 점이 중요하다 — **delta의 크기는 모델의 크기가 아니라 모델의 폭과 깊이가 정한다**. 바이트로 옮기면 $C_{\text{cap}} = |\Delta\Theta| \times (\text{bytes/elem})$ 이며, dtype을 명시하지 않은 delta 크기 진술은 무의미하다.

논문의 실측 값이 이 식의 기준점이다. GPT-3 175B에 $r = 4$ 로 $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}$ 를 적용하면 checkpoint가 350GB $\rightarrow$ 35MB, 약 10,000배 줄어든다 [lora § 4.2]. task 100개를 엮어도 350GB + 35MB $\times$ 100 $\approx$ 354GB이고 (이 책의 예시 계산), 독립 fine-tune이면 약 35TB다 [lora § 4.2 각주 4]. Θ -경로의 **per-task 상태량은 base weights의 10^{-4} 수준이다** — 이것이 이 장이 Part III에 넘기는 첫번째 숫자다.

여기서 자주 혼동되는 두 개념을 분리한다.

정의. QLoRA는 base Θ_0 를 4-bit로 양자화해 얼려 두고 그 위에 더 높은 정밀도의 LoRA delta를 학습하는 구성이다 (Dettmers et al. 2023, arXiv:2305.14314). **양자화되는 것은 Θ_0 이지 $\Delta\Theta$ 가 아니다**. 따라서 QLoRA가 줄이는 것은 § 03.1에서 본 쓰기 시점의 GPU 메모리이며, delta 아티팩트의 크기는 바꾸지 않는다.

정의. 양자화된 delta는 그와 다른 축이다 — $\Delta\Theta$ 자체를 낮은 dtype으로 저장·전송·적재하는 것이며, 바꾸는 대상은 B_s 가 아니라 C_{cap} 이다.

이 구분이 왜 중요한지는 저장 바이트당 정보량을 계산해 보면 드러난다. 이 책의 실험 [X2 Q2]는 $d = 4096$, $L_{\text{layer}} = 32$, $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}$, $r = 16$ 인 dense-8B 앵커 (파라미터 8,388,608개)에 대해 다음을 회계한다.

표 3-3. delta 의 dtype 별 저장 바이트당 정보량 [X2 Q2]. 분자는 [lm-memorization-capacity Fig. 6 캡션] 의 3.64 bits/param 을 파라미터 수에 곱한 값이고, 이 책은 이것을 회계 용 참고값으로만 쓴다 (BPP-EXTRAPOLATED — 무작위 균등 문자열을 from-scratch 학습시킨 소형 GPT 에서 켜 값이며 LoRA delta 적용은 외삽이다). **원 논문은 이 값을 하한으로 못 박는다** — “we are only ever measuring a lower bound on model capacity” [lm-memorization-capacity § 3.2]. 따라서 이 표의 어느 칸도 용량 상한이 아니며, 실제 밀도는 이보다 높을 수 있다. 이 값의 지위와 용량 상한논의의 정본은 ch09 가 소유한다.

dtype	저장 바이트	저장 MB	저장 바이트당 bits
bf16	16,777,216	16.78	1.82
int8	8,388,608	8.39	3.64
int4	4,194,304	4.19	7.28

같은 앵커에서 E-경로의 텍스트 상태 (14K 토큰, 3:1 압축 가정) 는 2.67 bits/byte 다. 두 경로의 비는 dtype 에 따라 bf16 1.47, int8 0.73, int4 0.37 이다 [X2 Q2]. 즉 **bf16에서는 텍스트가 바이트당 정보 밀도에서 앞서고, int8에서 순서가 뒤집히며, int4에서는 Θ 가 텍스트의 2.7 배가 된다**. 순서가 뒤집히는 지점이 bf16 과 int8 사이에 있다는 것 — 이것이 이 절의 결론이다.

[해설] 이 역전의 기제는 알고리즘이 아니라 표현이다. 파라미터 하나에 담기는 정보량의 측정된 **하한**이 3.64 bits 인데 [lm-memorization-capacity Fig. 6 캡션, lm-memorization-capacity § 3.2] bf16 은 그것을 16 bits 자리에 넣는다. 낭비되는 것은 용량이 아니라 **저장 폭**이다. 그러므로 “ Θ -경로는 바이트 비효율이다” 라는 흔한 인상은 dtype 을 고정했을 때만 참이며, 알고리즘의 성질이 아니다 [X2 F3].

사용자 수로 곱하면 이 선택이 어디에 나타나는지 보인다. 같은 8B 앵커에서 사용자 10^6 명분의 웹 스토리지는 $r = 16 \cdot \text{bf16}$ 에서 16.777 TB, int4 로 내리면 4.194 TB, $r = 64 \cdot \text{bf16}$ 이면 67.109 TB 다 [X2 Q3]. 절대치는 회계이지 측정이 아니므로 자릿수와 방향으로만 읽어야 하지만, 방향은 분명하다 — **dtype 하나가 웹 계층 규모를 4 배 움직인다**.

[평가] 그런데 corpus 의 어느 논문도 delta 양자화를 논의하지 않는다 [X2 F3]. LoRA 원문에는 delta 의 load 시간도, 대역폭도, roofline 도 한 번 나오지 않는다. Θ -경로를 실제 서빙에 올리려면 delta 양자화는 선택 사항이 아니라 전제인데, 그 전제를 다룬 문헌이 이 corpus 에 없다. 이 공백은 Part III(ch26, ch29) 가 직접 메운다.

(state, update, cost) 정리

이 장의 개념을 세 층 프레임에 배치한다. 모든 행이 Θ 층이라는 것 자체가 이 장의 요약이다 — **fine-tuning 과 PEFT 는 W 도 E 도 만들지 않는다**.

표 3-4. 이 장의 개념과 세 층 프레임

개념	상태 (어느 층)	갱신 규칙	시간척도	비용 4 층에서의 자리
full fine-tuning	Θ 전체	(U- Θ), 제약 없음	adaptation run 1 회	$C_{\text{cap}} = \Theta_0 $ (GPT-3 에서 350GB)

개념	상태 (어느 층)	갱신 규칙	시간척도	비용 4 종에서의 자리
adapter	Θ 의 삽입 부분	(U- Θ), 삽입 파라미터만	adaptation run 1 회	$L_w > 0$ — merge 불가, batch 1 에서 +20.7~30.3%
LoRA (rank r)	Θ 의 rank- r 부분 공간	(U- Θ), ∇ 가 (A, B) 에만	adaptation run 1 회	$C_{\text{cap}} = 2N_{\text{mod}}dr \times \text{bytes/elem}$
merge	Θ (gradient 아님)	규칙 밖 — 배포 시점의 산술	배포 1 회	$L_w \rightarrow 0$, shared-weight batching 을 깨 뜨림
분리된 delta 형태	Θ_0 공유 + per-request (A, B)	갱신아님 — 적용 형태	요청마다	L_w 논문에 없음. FLOPs 비 $2r/d$ (이책의 산술)
QLoRA	Θ_0 의 저장 표현	갱신 아님 — 양자화	쓰기 시점	B_s 의 메모리 항
양자화된 delta	$\Delta\Theta$ 의 저장 표현	갱신 아님 — 양자화	배포 · 보관 시점	C_{cap} 의 정보 밀도 (bf16 1.82 $\rightarrow$ int4 7.28 bits/byte)
W (fast weights)	—	없다	—	이 장의 어떤 방법도 test-time 에 상태를 움직이지 않는다
E (external store)	—	없다	—	LoRA 원문에 retrieval·vector store 언급이 없다

비용 4 종을 LoRA 기준으로 채우면 다음과 같다. 빈칸을 추정으로 메우지 않는다.
표 3-5. LoRA 의 비용 4 종

기호	값	출처
B_s	논문에 없음. sleep 예산 개념이 없고 $\text{gen}(\cdot)$ 이 항등이다. 보고되는 것은 학습 VRAM 1.2TB $\rightarrow$ 350GB 와 throughput 32.5 $\rightarrow$ 43.1 tokens/s per V100 뿐이며 총 FLOPs·토큰 수·wall-clock 은 전부 없다	[lora § 4.2, lora § 4.2 각주 5]
L_w	merge 된 형태에서 0 (“by construction”). 분리된 delta 형태는 논문에 없음	[lora § 4.1, lora § 4.2 Limitations]

기호	값	출처
C_{cap}	$2N_{\text{mod}}dr$ 파라미터. GPT-3 $r = 4, \{\Theta^{(q)}, \Theta^{(v)}\}$ 에서 350GB $\rightarrow$ 35MB (약 10,000 배)	[lora § 5.1, lora § 4.2, lora § 7.1]
ρ	논문에 없음. 순차 적응 · delta 누적 · 반복 write·forgetting 측정이 하나도 없다	(부재. 원문에 forget· continual·sequential· catastrophic 어휘가 등장하 지 않음)

[평가] 마지막 줄이 이 장이 Part II 에 넘기는 가장 중요한공백이다. LoRA 는 한 task 에 한 번 적응하고 끝나며, 두 개의 $\Delta\Theta$ 를 누적하거나 같은 모델에 반복해서 쓰는 실험 이 없다. 가장 근접한 서술인 task switch(식 (3-5)) 조차 열화가 아니라 **완전되돌리기** 를 전제한다. 그러므로 “LoRA 를 sleep-time write primitive 로 쓸수 있다” 는 이 책 의 명제는 **이 논문으로는 warrant 되지 않는다**. 그것을 warrant 할 증거는 ch09(용량)·ch20(SEAL)·ch21(반복 sleep 라운드) 에서 따로 조달해야 하며, 조달되지 않으면 조달되지 않았다고 쓴다.

Worked micro-example — delta 아티팩트의 크기를 손으로계산한다

식 (3-6) 을 실제로 돌린다. 모든 단계는 암산 가능하며, 계산기가 필요한곳은 없다.

1 단계 — 네 값을 고정한다. 앵커는 dense-8B 다: $d = 4096, L_{\text{layer}} = 32$ [X2 앵커]. 적응 집합 은 표 3-2 의 관행을따라 $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}$, rank 는 $r = 16$, 저장 dtype 은 bf16 으로 둔다.

2 단계 — 적응 행렬 개수. $N_{\text{mod}} = L_{\text{layer}} \times |\mathcal{S}| = 32 \times 2 = 64$. 층마다 두 장씩, 32 개 층이므로 64 장이다.

3 단계 — 파라미터 수. 식 (3-6) 에 넣는다.

$$|\Delta\Theta| = 2 \times 64 \times 4096 \times 16$$

암산 경로는 이렇다. $2 \times 64 = 128$. $128 \times 4096 = 524,288 (= 2^{19})$. $524,288 \times 16 = 8,388,608 (= 2^{23})$. 즉 **8.39M 파라미터**다. 8B 모델 대비 약 10^{-3} 이다.

4 단계 — 바이트. dtype 을 곱한다. bf16 은 2 bytes/elem 이므로 $8,388,608 \times 2 = 16,777,216$ bytes = **16.78 MB**. int8 이면 8.39 MB, int4 면 4.19 MB 다 [X2 Q1].

교차 검증 — 논문의 값이 나오는가. 같은 식에 GPT-3 을 넣는다: $d = 12,288, L_{\text{layer}} = 96$ [lora § 1, lora § 7.1], $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}, r = 4$. $N_{\text{mod}} = 96 \times 2 = 192$. $2 \times 192 = 384$, $384 \times 12,288 = 4,718,592$, $\times 4 = 18,874,368$ — **약 18.9M** 이다. 논문이 같은 설정을 “18M parameters (roughly 35MB if stored in FP16), 96 layers” 로 적은 것과 파라미터 수에서 일치한다 [lora § 7.1]. 바이트는 $18,874,368 \times 2 = 37,748,736 = 37.7$ MB 이므로 논문의 “35MB” 는 반올림 표기이며, 논문의 “10,000 배감소” (350GB $\rightarrow$ 35MB, [lora § 4.2]) 도 그 반올림 값을 쓴 것이다. 식 (3-6) 이 준 계산값으로 다시 나누면 $350 \text{ GB} / 37.7 \text{ MB} \approx 9,300$ 배다 (이 책의예시 계산). 이 책은 계산값과 원문 표기를 이렇게 병기하고, 어느 쪽도상대를 지우지 않는다.

두 knob의 감도. 3 단계로 돌아가 r 만 $16 \rightarrow 64$ 로 올리면 4배가되어 67.11 MB, $\mathcal{S}$ 만 $\{\Theta^{(q)}, \Theta^{(v)}\} \rightarrow \{\Theta^{(q)}, \Theta^{(k)}, \Theta^{(v)}, \Theta^{(o)}\}$ 로 늘리면 2 배가 되어 33.55 MB 다 [X2 Q1]. 두 knob 은 식 (3-6) 에 **곱** 으로 들어가므로 둘 다 키우면 8 배 (134.22 MB) 다. 표 3-2 가 보인 “같은 예산이면 rank 를 낮추고 행렬을 늘려라” 는 이 곱셈을고정한 채 배분만 바꾸는 조작이다.

정보 밀도. 8,388,608 파라미터에 3.64 bits/param 을 곱하면 $30,534,533 \text{ bits} \approx 3.82 \text{ MB}$ 다 — 이 책의 예시 계산이다 (BPP-EXTRAPOLATED — 회계용 참고값이며, 3.64 는 원 논문이 하한이라고 밝힌 값이므로 이 결과도하한이다 [lm-memorization-capacity § 3.2]. 정보 서술은 ch09)

[X2 Q2]. bf16 저장 16.78 MB 로 나누면 저장 바이트당 1.82 bits, int4 저장 4.19 MB 로 나누면 7.28 bits 다. 같은 정보를 4 분의 1 바이트에 담는것이 delta 양자화가 하는 일의 전부다.

서빙 산술. 분리된 delta 형태에서 추가되는 토큰당 MAC 은 base 대비 $2r/d = 32/4096 = 1/128 \approx 0.78\%$ 다 (이 책의 산술). **사용자 스케일.** 16.78 MB 를 사용자 10^6 명에 공급하면 웹 스토리지 16.777 TB, int4 면 4.194 TB 다 [X2 Q3] — 절대치는 회계이므로 자릿수로 읽되, 4 배의 차이가 dtype 하나에서 나온다는 순서 관계는 그대로 성립한다.

요약

- 가중치에 쓰는 것은 문맥에 쓰는 것으로 대체되지 않으며 (GPT-3 few-shot 대 fine-tuning 이 MNLI-m 40.6 대 89.5 [lora Table 8]), 그 대가로 full fine-tuning 은 Θ_0 와 크기가 같은 checkpoint 를 task 마다 남긴다 — GPT-3 175B 에서 350GB 다 [lora § 2, lora § 4.2].
- adapter 가 밀리는 축은 품질이 아니라 mergeability 다. 순차 depth 의 비용은 batch 가 작을수록 커지며, batch 1 / seq 128 에서 +20.7~30.3% 다 [lora Table 1].
- LoRA 는 (U- Θ) 를 rank- r 부분공간으로 제한한 write primitive 이고, B_s knob 도 라운드 첨자도 갖지 않는다 — sleep 절차가 아니라 쓰기 도구다 [lora § 4.1, lora § 2].
- [lora § 7.1] 의 “ $\{\Theta^{(q)}, \Theta^{(v)}\}$ 가 최선” 이라는 결론은 자기 Table 5 와 어긋난다. $\{\Theta^{(q)}, \Theta^{(k)}, \Theta^{(v)}, \Theta^{(o)}\} (r = 2)$ 가 WikiSQL 73.7 로 동률, MultiNLI 91.7 로 우위다 [lora Table 5].
- merge 는 $L_w = 0$ 을 shared-weight batching 과 맞바꾼 것이며, 분리된 delta 형태의 latency 는 논문에 측정값이 없다 [lora § 4.1, lora § 4.2 Limitations].
- delta 아티팩트의 크기는 $2N_{\text{mod}}dr$ 로 정확히 주어지고, 모델 파라미터 총수가 아니라 폭 · 깊이 · rank · 적용 집합이 결정한다 [lora § 5.1].
- 저장 바이트당 정보 밀도는 bf16 1.82, int8 3.64, int4 7.28 bits/byte 이며, 텍스트 (3:1 압축 가정 2.67) 대비 순서가 bf16 과 int8 사이에서 뒤집힌다 — 3.64 bits/param 하한위에 세운 이 책의 회계다 [X2 Q2, lm-memorization-capacity § 3.2].
- LoRA 의 ρ 는 논문에 없다. 반복 write · delta 누적 · forgetting 측정이 하나도 없으므로 이 논문은 Θ -경로의 지속가능성을 warrant 하지 못한다.

자가 점검체크리스트

- ☐ full fine-tuning · adapter · LoRA 를 “ Θ 의 어느 부분집합을 어떻게 움직이는가” 와 “배포 아티팩트로 무엇을 남기는가” 두 축으로 구분해 설명할 수 있다.
- ☐ $d, L_{\text{layer}}, r, \mathcal{S}$ 만 주어졌을 때 $|\Delta\Theta|$ 와 그 바이트를 손으로 계산하고, r 과 $|\mathcal{S}|$ 가 곱으로 들어간다는 것을 보일 수 있다.
- ☐ merge 된 형태에서 $L_w = 0$ 이 성립하는 이유와, 그 대가로 무엇이 깨지는지를 GEMM shape 수준에서 설명할 수 있다.
- ☐ QLoRA 가 양자화하는 대상과 “양자화된 delta” 가 양자화하는 대상이 다르다는 것을, 각각 비용 4 종 중 무엇을 바꾸는지로 말할 수 있다.
- ☐ [lora § 7.1] 의 결론과 [lora Table 5] 의 수치가 어긋나는 지점을 지적하고, 표에서 실제로 유도되는 결론을 말할 수 있다.
- ☐ LoRA 의 비용 4 종에서 “논문에 없음” 인 칸이 어디이고 왜 그것이 Θ -경로판정에 결정적인지 설명할 수 있다.
- ☐ **Rosetta** — “batch 로 weight 공유” 라는 서빙의 대전제가 Θ -경로에서 깨지는 자리를, merge 된 형태와 분리된 delta 형태 각각에 대해 inference 어휘로 옮길 수 있다.

다음 장으로

이 장은 Θ 에 쓰는 도구를 확정했다. 그런데 식 (3-4)에서 학습 집합 $\mathcal{D}$ 는 사람이 만들어 놓은 고정 dataset 이었고, 그래서 표준형 (U- Θ)의 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 에서 $\text{gen}(\cdot)$ 이 항등으로 붕괴했다. sleep 라운드가 성립하려면이 자리가 비어 있어서는 안 된다 — 유틸 시간예 예산 B_s 를 써서 **무엇을 학습 신호로 삼을 것인가**가 정해져야 한다.

다음 장이 답할 질문은 그 자리의 가장 오래된 답이다: 모델의 출력을 다른 모델 (또는 과거의 자신)의 학습 신호로 쓰는 법, 즉 distillation 이다. ch04는 logit·feature·self-distillation과 upward distillation을 (U- Θ)의 $\mathcal{R}_k$ 생성기로 재서술하고, 그것이 ch21의 Knowledge Seeding과 ch20의 self-edit으로 이어지는 경로를 연다. 이 장의 (A, B, r) 은 그때 “무엇에 쓰는가”가 아니라 “**무엇을** 쓰는가”의 문제로 바뀐다.

ch04. distillation — 한 모델의 앎을 다른 모델로 옮기기

이 장의 목표 — 이 장을 마치면 독자는 다음을 할 수 있어야 한다. 1. teacher와 student를 파라미터의 이동 여부로 정의하고, distillation loss를 (대상, divergence, 표본 출처) 세 좌표로 분해해 쓸 수 있다. 2. temperature T 가 soft target과 gradient에 각각 무엇을 하는지 계산으로 보이고, T^2 보정이 왜 붙는지 진술할 수 있다. 3. logit distillation과 feature distillation을 저장·대역폭 비용으로 구분하고, self-distillation에서 on-policy 표본이 sleep job의 비용 형태를 training-bound에서 decode-bound로 바꾼다는 것을 설명할 수 있다. 4. **upward distillation**(작은 자기 → 큰 자기)이 통상의 방향과 반대라는 사실을 말하고, 그 절차에서 실제로 옮겨지는 것이 지식이 아니라 좌표라는 것을 [LM Need Sleep]의 식으로 논증할 수 있다.

왜 필요한가 — Part II의 세 지점이 이 장을 전제하고 쓰인다. - **ch21 [LM Need Sleep]** (Language Models Need Sleep, arXiv:2606.03979) [LM Need Sleep § 3.3]: Knowledge Seeding의 목적함수는 GKD 항과 imitation reward 항의 볼록결합이다. 그 GKD 항의 형태·표본출처·frozen 범위를 이 장이 소유한다. 같은 절의 boxed 문단제목이 그대로 “Upward Distillation (Knowledge Seeding)”이다. - **ch20 [SEAL]** (Self-Adapting LMs, arXiv:2506.10943) [SEAL § 3.1] 마지막 문단: teacher와 student를 서로 다른 모델로 분리하는 확장을 제안한다 — 제안만하고 실행하지 않는다. 그 제안이 무엇을 바꾸는 제안인지 읽으려면 teacher/student가 역할이지 개체가 아니라는 이 장의 정의가 필요하다. - ch21이 최강 baseline으로 세운 OPSD 계열, 그리고 [LM Need Sleep] App. A.4가 두 페이지에 걸쳐 열거하는 2026년 논문군 전체가 **on-policy self-distillation**이다. 그 이름이 무엇을 뜻하는지가 이 장 § 04.4다.

NM과의 관계 — Neural Memory 모노그래프에 대응 장이 없다. 이장은 NM이 세운 기계 위에서 시작한다: backward pass와 optimizer state는 → NM ch02, inner/outer loop의 이중 구조는 → NM ch04, chunk 단위 갱신은 → NM ch09로 넘긴다. 이 장이 더하는 것은 하나다 — **gradient의 출처를 label에서 다른 모델의 출력 분포로 바꾸면 무엇이 달라지는가**. NM은 W 가 무엇을 배우는지를 다뤘고, 이 장은 Θ 에 무엇을 어떻게 써넣을 수 있는지의 두 번째 도구를 다룬다 (첫번째 도구는 ch03의 PEFT다).

04.1 gradient의 출처를 바꾼다 — teacher와 student

distillation은 loss의 정답 자리에 사람이 붙인 label 대신 다른 모델의 출력 분포를 놓는 절차다. 이 치환 하나가 이 장 전체를 결정한다.

정의. **teacher**는 학습 중 파라미터가 움직이지 않고 그 출력이 학습 신호로 소비되는 모델이다. **student**는 그 신호를 재현하도록 파라미터가 움직이는 모델이다. 두 이름은 **역할**이지 개체가 아니다 — 크기·구조·정체성 어느 것도 정의에 들어가지 않는다. 이

장의 표기로 teacher 파라미터는 Θ^{te} , student 파라미터는 Θ^{st} 이며, 갱신 대상은 항상 Θ^{st} 하나다.

이 정의가 역할로만 되어 있다는 사실이 뒤의 세 절을 가능하게 한다. teacher와 student가 같은 개체일 수 있고 (§ 04.4), teacher가 student보다 작을 수도 있다 (§ 04.5). 정의에 크기가 들어 있었다면 둘 다 형용모순이 된다.

왜 label 대신 분포인가. 정답 label은 어휘 V 위의 one-hot 벡터로, 정답 토큰 하나에 대해서만 정보를 준다. teacher의 출력 분포는 같은 자리에서 $|V|$ 개 좌표 전부 값이 주어진다 — “정답은 A이고, B는 A의 1/8만큼 그럴듯하며, C는 사실상 불가능하다”는 순위와 간격까지 함께 온다. Hinton, Vinyals & Dean(2015, arXiv:1503.02531)이 dark knowledge라고 부른 것이 이 여분의 좌표들이다. 같은 토큰 하나를 소비하고도 student가 받는 신호의 차원이 1에서 $|V|$ 로 늘어난다는 것, 그것이 distillation의 자본이다.

[해설] systems 관점에서 distillation 파이프라인은 새로운 커널을 하나도 도입하지 않는다. teacher 쪽은 순수 forward — 독자가 매일 돌리는 batch prefill과 완전히 같은 연산이고, weight는 read-only이며, roofline 위치도 그대로다. student 쪽만 backward가 붙는다 (→ NM ch02 § 2.3). 새로 생기는 것은 연산이 아니라 **두 모델이 동시에 HBM에 상주한다**는 배치 제약과, teacher 출력을 student backward까지 살려두는 텐서 수명이다.

독자에게 이미 익숙한 사례가 하나 있다. speculative decoding의 draft model은 target model의 분포를 근사하도록 학습된 student이고, 그 학습절차의 표준형이 바로 distillation이다. acceptance rate는 두 분포가 얼마나 가까운지의 직접 측정치다. 즉 독자는 distillation의 산출물을 이미 serving하고 있다 — 이 장은 그 산출물이 어떻게 만들어지는지를 연다.

이 장이 이 책에서 차지하는 자리도 여기서 정해진다. Θ -경로가 sleep 라운드에 (U- Θ)를 둘러려면 두 가지가 필요하다: **어디에 쓸 것인가** (파라미터의 어느 부분공간을 움직일 것인가 — ch03의 PEFT)와 **무엇을 목표로 쓸 것인가** (gradient를 만들어 낼 신호 — 이 장). 이 corpus에서 sleep 라운드의 목표는 거의 예외 없이 사람이 붙인 label이 아니다. 모델 자신이거나, 자신의 과거이거나, 다른 모델의 분포다. 그래서 distillation은 이 경로에서 선택지가 아니라 **기본문법**이다.

04.2 logit distillation과 temperature

정의. logit distillation은 teacher와 student의 **출력 분포** (softmax 이전의 logit을 온도로 나눠 만든 분포)를 맞추는 distillation이다. 중간 표현은 보지 않고 최종 층의 분포만 본다.

한 토큰 위치에서 teacher logit을 $z^{\text{te}} \in \mathbb{R}^{|V|}$, student logit을 $z^{\text{st}} \in \mathbb{R}^{|V|}$ 라 하자. **temperature** $T > 0$ 는 softmax의 지수를 나누는 스칼라다 (이 장에서만 쓰는 장-국소 기호다. sequence 길이는 NM 표기 그대로 L 이고, T 를 길이 의미로 쓰지 않는다).

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (4-1)$$

이 식이 말하는 것은 하나다. T 를 키우면 분포가 평평해지고, 낮은 확률의 좌표들이 상대적으로 커진다. $T \rightarrow 0$ 이면 argmax의 one-hot으로 붕괴하고, $T \rightarrow \infty$ 면 균등분포로 간다. dark knowledge는 정확히 그 “낮은 확률의 좌표들”에 있으므로, T 는 **teacher의 어느 부분을 학습 신호로 삼을지 고르는 다이얼**이다. $T = 1$ 은 teacher가 실제로 serving할 때의 분포이고, $T > 1$ 은 serving에서는 결코 보지 않을 꼬리를 확대한 분포다.

정의. distillation loss의 표준형은 hard label 항과 soft target 항의 볼록결합이다.

$$\mathcal{L}_{\text{KD}} = (1 - w_{\text{KD}}) \mathcal{L}_{\text{CE}}(y, p^{\text{st},(1)}) + w_{\text{KD}} T^2 \cdot \mathcal{F}(p^{\text{te},(T)} \| p^{\text{st},(T)}) \quad (4-2)$$

$w_{\text{KD}} \in [0, 1]$ 은 두 신호의 혼합 비율, $\mathcal{F}$ 는 divergence(기본형은 forward KL), y 는 정답 label 이다. hard 항은 항상 $T = 1$ 에서 계산한다 — 정답 label에는 온도가 없기 때문이다. (w_{KD} 는 hard 대 soft의 비율이고, § 04.5에 나오는 λ_{KD} 는 [LM Need Sleep]의 divergence 대 reward 비율이다. 이름이 닮았을 뿐 다른 값이며 이 책은 둘을 섞지 않는다.)

T^2 이 왜 붙는가. soft 항의 gradient는 student logit에 대해

$$\frac{\partial}{\partial z_i^{\text{st}}} \mathcal{F}(p^{\text{te},(T)} \| p^{\text{st},(T)}) = \frac{1}{T} (p_i^{\text{st},(T)} - p_i^{\text{te},(T)}) \quad (4-3)$$

이고, 괄호 안의 확률 차이 자체도 T 가 커질수록 작아진다. 두 효과가 겹쳐 gradient의 크기가 대략 $1/T^2$ 로 줄어든다. 보정을 넣지 않으면 T 를 바꿀 때마다 w_{KD} 의 실효 의미가 함께 변해 두 항의 균형을 다시 튜닝해야 한다. T^2 은 그 결함을 끄는 정규화 상수이며, $1/T^2$ 근사가 **정확한 것은 고온 극한에서뿐이다** — 이 장의 Worked micro-example이 $T = 4$ 에서 실제 어긋남을 숫자로 보인다.

[해설] $\mathcal{F}$ 의 방향이 실무에서 갈린다. forward KL $\mathcal{F}(p^{\text{te}} \| p^{\text{st}})$ 는 teacher가 확률을 준곳을 student가 비우는 것을 강하게 벌하므로 student를 mode-covering으로 만들고, reverse KL $\mathcal{F}(p^{\text{st}} \| p^{\text{te}})$ 는 반대로 mode-seeking이다. student의 용량이 teacher보다 작을 때 forward KL은 teacher의 모든 mode를 평균하려다 어느 쪽도 아닌 분포를 만든다. [LM Need Sleep § 3.3]이 채택한 GKD(Agarwal et al. 2024)가 divergence를 고정하지 않고 $\mathcal{F}$ 를 인자로 남겨 둔 이유가 이것이다.

systems 접점은 저장이다. logit distillation을 하려면 teacher의 출력분포를 student의 backward가 소비할 때까지 들고 있어야 하는데, 그 텐서의 크기가 $L \times |\mathcal{V}|$ 다. 어휘가 십만 단위인 현대 LM에서 이것은 activation 하나가 아니라 **모델 가중치와 같은 자릿수의 텐서**다 (이장의 Worked micro-example에서 쏜다). 그래서 실무의 선택지는 둘뿐이다: teacher를 student와 같은 step에 함께 돌려 logit을 즉시 소비하고버리거나 (HBM에 두 모델 상주), teacher logit을 top- k 로 잘라 디스크에 캐시하거나 (top- k 의 k 는 남길 좌표 수를 뜻하는 장-국소 용법이며, 이 책 전역의 sleep 라운드 침자 k 와 무관하다). 전자는 sleep job의 메모리 상한을, 후자는 정확히 dark knowledge의 꼬리를 잘라낸다. 공짜선택지는 없다.

04.3 feature distillation — 출력이 아니라 내부를맞춘다

정의. feature distillation(hidden-state distillation)은 최종 분포가 아니라 **중간 층의 활성값**을 맞추는 distillation이다. teacher의 층 ℓ 출력 $h_\ell^{\text{te}} \in \mathbb{R}^{d_{\text{te}}}$ 와 student의 대응층 출력 $h_\ell^{\text{st}} \in \mathbb{R}^{d_{\text{st}}}$ 사이의 거리를 loss에 더한다.

$$\mathcal{L}_{\text{feat}} = \sum_{\ell \in \mathcal{A}} w_\ell \|P_\ell h_\ell^{\text{st}} - h_\ell^{\text{te}}\|_2^2 \quad (4-4)$$

이 식이 logit 쪽과 다른 점은 세 가지다. 첫째, 층 대응 $\mathcal{A}$ 를 사람이 정해야 한다 — teacher가 32 층이고 student가 12 층이면 어느 층을 어느 층에 붙일지가 설계 변수다. 둘째, 폭이 다른 학습 가능한 사영 $P_\ell \in \mathbb{R}^{d_{\text{te}} \times d_{\text{st}}}$ 가 필요하고, 이 P_ℓ 은 학습이 끝나면 버려지는 **일회용파라미터**다. 셋째, divergence가 확률 사이의 것이 아니라 벡터 사이의 것이므로 표현의 스케일과 회전에 민감하다 — 같은 함수를 계산하는 두 네트워크도 내부 좌표계가 다르다면 $\mathcal{L}_{\text{feat}}$ 가 크게 나온다.

같은 계열에 attention map distillation이 있다. teacher의 attention 확률행렬을 student가 재현하게 하는 것으로, 대상이 $L \times L$ 행렬이라는점만 다르고 구조는 (4-4)와 같다.

[평가] feature distillation 은 logit distillation 보다 강한가정 위에 서 있다 — **teacher** 의 내부 표현이 재현할 가치가 있다는가정이다. logit 쪽은 입출력 사상만 맞추므로 student 가 다른 내부 경로로같은 함수를 계산해도 무방하지만, feature 쪽은 경로까지 지정한다. student 의 구조가 teacher 와 다를수록 이 가정은 약해지고, 사영 P_ℓ 이 흡수해야 할 불일치가 커진다. 이 corpus 에서 sleep-time 계열논문이 전부 logit·분포 쪽만 쓰고 feature 쪽을 쓰지 않는것은 우연이 아니다 — sleep-time distillation 의 teacher 와 student 는구조가 거의 같으면서도 **파라미터 집합이 다르므로**, 내부 좌표를고정하면 새로 늘린 용량이 쓰일 자리가 없어진다 (§ 04.5).

systems 접점은 다시 저장이지만 성질이 다르다. logit 은 마지막 층 하나에서 $|V|$ 쪽으로 나오고, feature 는 $|A|$ 개 층에서 d 쪽으로 나온다. $d \ll |V|$ 이므로 층당 텐서는 작지만, 이 텐서들은 **teacher forward 의 중간 activation 이므로 checkpointing 으로 버릴 수가 없다** — 버리면 다시 계산해야 한다. 결과적으로 feature distillation 은 teacher forward 의 activation 수명을 student backward 끝까지 늘리고, 그만큼 batch 크기를 깎는다. logit 쪽이 “한 번 크게 쓰고 버리는” 트래픽이라면 feature 쪽은 “여러 곳에 오래붙잡아 두는” 점유다.

04.4 self-distillation — teacher 가 자기 자신일때

정의. self-distillation 은 teacher 와 student 가 같은모델일 때의 distillation 이다. “같다” 는 것은 세 가지 중 하나를 뜻한다: (a) 같은 파라미터의 서로 다른 시점 (라운드 k 의 스냅샷이 라운드 $k+1$ 의 teacher), (b) 같은 모델의 서로 다른 파라미터 부분집합 (일부만학습 대상), (c) 같은 파라미터 · 다른 표본 경로. 세 경우 모두 teacher 쪽은 stop-gradient 로 얼려 두므로 Θ^{te} 는상수다.

여기서 독자가 즉시 던지는 질문이 이 절의 전부다. **teacher 가 student 보다 아는 것이 없는데 어떻게 이득이 나는가.** 답은 셋이고, 셋 다 “지식이 이동해서” 가 아니다.

첫째, **표본 출처가 바뀐다.** teacher 가 만든 문장 위에서만 배운 student 는 자기가 만든 문장 위에서 처음 보는 상태에 놓인다 — 학습 분포와추론 분포의 불일치다. student 가 스스로 디코딩한 토큰 위에서 teacher 가점수를 매기면 이 불일치가 사라진다.

정의. distillation 의 표본 y 를 고정된 데이터셋 $\mathcal{D}$ 에서 뽑으면 **off-policy**, student 자신의 분포 $\text{LM}_{\Theta^{\text{st}}}(\cdot|x)$ 에서 뽑으면 **on-policy** 다. 혼합 비율을 $\lambda_{\text{on}} \in [0, 1]$ 로 둔형태가 GKD(Agarwal et al. 2024) 이며, [LM Need Sleep § 3.3] 이 그대로채택한다.

$$\mathcal{L}_{\text{GKD}} = (1 - \lambda_{\text{on}}) \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\mathcal{F}(\text{LM}_{\Theta^{\text{te}}} \| \text{LM}_{\Theta^{\text{st}}})(y|x) \right] + \lambda_{\text{on}} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \text{LM}_{\Theta^{\text{st}}}(\cdot|x)} \left[\mathcal{F}(\text{LM}_{\Theta^{\text{te}}} \| \text{LM}_{\Theta^{\text{st}}})(y|x) \right] \quad (4-5)$$

이 식이 말하는 것은 divergence 를 어디서 재느냐가 유일한 차이라는 것이다. 두 항의 $\mathcal{F}$ 는 같고, 기대값을 취하는 y 의 출처만 다르다. [LM Need Sleep § 3.3] 은 여기에 두 개의 설계 결정을 명시로 붙인다 — “we do not backpropagate through the sampling distribution of the student” (샘플링 경로로는 gradient 를 흘리지 않는다), 그리고 “we freeze all the parameters in the student model and only updates the expanded parameters” . 앞의 것은 이산 샘플링이 미분 불가능하기 때문의 표준처리이고, 뒤의 것은 § 04.5 의 핵심이다.

둘째, **목표가 label 보다 부드럽다.** one-hot label 은 정답 이외좌표를 전부 0 으로 밀지만 자기 자신의 분포는 그렇지 않다. 자기 출력을목표로 두는 것은 정칙화로 작동한다.

셋째, **teacher 가 앵커가 된다.** 라운드 k 의 자기를 목표로 두면라운드 $k+1$ 의 갱신은 옛 행동에서 멀어지는 것을 loss 로 벌받는다. 이것은 옛 데이터를 다시 넣는 것과 목적은 같고 기제는 다르다 — 데이터가아니라 함수를 붙잡는다. 옛 데이터를 다시 넣는 쪽 (replay) 은 ch07 이소유한다.

[해설] on-policy 항이 sleep job 의 비용 형태를바꾼다. $\lambda_{\text{on}} = 0$ 이면 sleep 은 평범한 training job 이고 지배 비용은 forward+backward GEMM 이다. $\lambda_{\text{on}} > 0$ 이면매 optimizer step s 앞에 student 의 autoregressive 생성이 붙는다 (한 sleep 라운드 k 안에 step s 가 여러 번 돈다 — 두 첨자는 다른시간척도다) — 즉 sleep job 의 지배 커널이 훈련의 compute-bound GEMM 이아니라 decode 의 memory-bandwidth-bound 구간이 된다. 결과적으로 sleep job 은 training 클러스터가 아니라 **wake 트래픽과 같은하드웨어 · 같은 병목을 놓고 경쟁한다**. [LM Need Sleep App. B.5] 가 보고하는 “같은 step 수에서 SFT 가 자기 방법보다 $4\times$ 효율적” 이라는비율은 이 형태 변화의 값이다 (반대로 같은 성능에 도달하려면 SFT 가 $4.3\times / 3.6\times / 4.8\times$ বেশ이제 시간을 쓴다고 같은 절이보고한다).

경계 하나를 못 박아 둔다. 모델이 **텍스트로** 자기 학습 데이터를써낸 다음 그 텍스트로 자기를 SFT 하는 절차 — [SEAL § 3.1] 의 self-edit 이 그것이다 — 도 흔히 self-distillation 이라 불리지만, 그것은데이터 공간의 절차이고 이 장의 대상이 아니다. 데이터를 만드는 법과 그위험 (자기생성 분포로 반복 학습할 때의 붕괴) 은 ch06 이 소유한다. 이 장은**분포 공간에서 divergence 를 최소화 하는 절차**만 다룬다. 두 절차는이름을 공유할 뿐 loss 의 형태도 실패 양식도 다르다.

04.5 upward distillation — 작은 자기에서 큰자기로

정의. **upward distillation** 은 teacher 가 student 보다 파라미터가 적은 distillation 이다. [LM Need Sleep § 3.3] 은 이것을 boxed 문단 제목 “Upward Distillation (Knowledge Seeding)” 으로 도입하고, teacher 와 student 가 파라미터 확장 전후의 같은모델일 때를 **self-Knowledge Seeding** 이라 부른다 [LM Need Sleep § 1 Contributions 2, p.4].

방향이 반대다. 통상의 distillation 은 큰 teacher 에서 작은 student 로 간다. 그 절차의 정당화는 한 줄로 요약된다 — teacher 가 student 보다 많이 알고, student 는 그중 답을 수 있는 만큼을 담는다. upward distillation 은 그정당화를 그대로 뒤집는다. teacher 가 덜 아는데 무엇을 옮기는가.

핵심은 옮겨지는 것이 정보량이 아니라 좌표라는 것이다. [LM Need Sleep § 3.2, p.7] 의 sleep 라운드는 먼저 receiver 블록에 low-rank expert 한 쌍을 새로 만들어 붙인다 — $A \in \mathbb{R}^{d \times d_{\text{low}}}$, $B \in \mathbb{R}^{d_{\text{low}} \times d}$, $d_{\text{low}} \ll d$. 이 확장 직후의 모델은 확장 전의 모델과다른 함수다. 새 expert 의 값이 무엇이든 그것이 forward 에더해지는 순간 출력이 흔들린다. 즉 용량을 늘리는 행위 자체가손상이다.

upward distillation 은 그 손상을 되돌리는 절차다. teacher 는 확장 전의자기이고, student 는 확장 후의 자기이며, 학습 대상은 새로 붙인 $\{A, B\}$ 뿐이다 (나머지는 전부 frozen — § 04.4 에서 인용한 그 문장이다). 목적은 큰 파라미터 공간 안에서 작은 파라미터가 계산하던 함수를재현하는 것이고, 성공하면 새 expert 는 “무해하면서 동시에 옛 행동을 자기좌표로 표현할 줄 아는” 상태가 된다. 새로 생긴 것은 지식이 아니라비어 있고 정렬된 용량이다.

[해설] 그래서 upward distillation 은 초기화 절차로 읽는 것이정확하다. 랜덤 초기화된 새 파라미터는 “아무것도 모르는” 상태가 아니라 “옛 함수를 망가뜨리는” 상태다. seeding 은 새 파라미터를 옛 함수의 값에정렬시켜, 이후 wake 에서 들어올 새 지식이 옛 지식을 덮어쓰지 않고옆에 앉을 자리를 만든다. [LM Need Sleep App. A.4 (i), p.24] 가 catastrophic forgetting 을 “분포의 문제가 아니라 용량 부족의 문제” 로재 정의하는 것이 이 설계의 전제다.

역방향이 자명하지 않은 두 번째 이유는 수렴 목표가 다르다는 데있다. 통상의 distillation 에서 student 의 상한은 teacher 다 — divergence 가 0 이면 완벽한 성공이고, 그 이상은 정의상 목표가 아니다. upward distillation 은 그럴 수 없다. divergence 를 0 으로 만든 student 는 더 큰 파라미터 공간에서 더 작은 함수를 정확히 흉내 내는 모델일 뿐이고, 늘린용량은 한 비트도 쓰이지

않은 채로 남는다. 즉 이 절차의 성공 기준은 “완전한 모방” 이 아니라 “옛 함수를 망가뜨리지 않으면서 새자유도를 열어 둔 상태”이며, 그 둘 사이의 지점이 어디인지를 정하는 값이 (4-6) 의 λ_{KD} 다. 그 값이 논문 어디에도 없다는 것이이 절 끝에서 다시 문제가 된다.

무엇이 버려지는지도 짚어야 한다. 새 expert 가 표현할 수 있는 것은 rank d_{low} 의 부분공간뿐이고, 정식화에서 논문이 d_{low} 에 대해 말하는 것은 $d_{low} \ll d$ 하나다 [LM Need Sleep § 3.2, p.7]. 실험 부록이 밝히는 것도 “차원 64 의 MLP 블록 5 개를 추가 파라미터로 쓴다” 까지이며, d 와 d_{low} 의 값 자체는 논문 어디에도 없다 [LM Need Sleep App. B, p.25]. 옛 함수전체가 그 rank 안에 들어간다는 보장은 없으므로 seeding 은 원리적으로 손실 압축이며, 무엇이 남고 무엇이 버려지는지에 대한 측정도 rate-distortion 형태의 진술도 논문에 없다. 얼마의 rank 가 얼마의 망각을 막는지 역시 정식화되어 있지 않다. 용량의 상한 자체는 ch09 가, 이 공백의 판정은 ch21 과 ch22 가 받는다.

이 절차가 복사가 아니라 전송인 이유는 마지막 단계에 있다. [LM Need Sleep § 3.3(c), p.8] 은 seeding 이 끝난 뒤 sender 블록에 그동안 쌓인 low-rank expert 들을 리셋한다 (“synaptic pruning”). 지운다는 사실이 이것을 전송으로 만든다 — 같은 내용이 빠른 블록과 느린 블록에 동시에 남아 있지 않다.

[평가] 이 절차는 이 책의 표준형 (U- Θ) 에 담기지 않는다. (U- Θ) 는 고정 차원에서 gradient 를 빼는 식인데, 여기서는 차원이 늘고 그 뒤 일부가 지워진다. 확장과 리셋에 대응하는 연산자가 표준형에 없다는 것은 프레임의 결합이 아니라 관할 밖이라는 뜻이다 — ch01 이 세운 세 층은 상태가 어디에 있는지를 정하지, 그 상태의 차원이 라운드마다 바뀌는 경우를 정하지 않는다. 이 장은 그 두 연산을 (U- Θ) 의 바깥에 붙는 전후 처리로 읽고, 차원 변동을 회계에 넣는 문제는 ch09 와 ch21 로 넘긴다.

systems 접점은 텐서 모양이다. 차원이 실제로 늘어나면 컴파일된 그래프도 kernel auto-tuning 도 sleep 마다 무효가 되므로, 논문은 확장분을 처음부터 모델에 넣어 두고 활성화 전까지 forward-backward 에서 masking 하는 구현을 제시한다 [LM Need Sleep § 3.3 Note on the Implementation, p.9]. 즉 자라는 Θ 는 서빙 쪽에 고정 모양으로 위장되고, 그 대가로 용량 상한이 배포 시점에 못 박힌다 — 그 상한의 크기는 논문에 없다.

[평가] upward distillation 의 정당화는 논리적으로 깔끔하지만, 그 기여의 크기는 논문 자신의 표에서 작다. LM Need Sleep Table 1, p.11 에서 전체 Sleep 은 79.2 / 69.0 / 46.1 이고, 확장을 뺀 “Sleep w/o Expansion” 은 78.2 / 67.9 / 44.9 다. 반대 방향으로, 확장만 baseline OPSD 에 얹은 “OPSD + Expansion” 은 77.9 / 68.2 / 45.9 로 OPSD 의 76.6 / 67.4 / 45.1 보다 +1.3 / +0.8 / +0.8 오른다. Sleep 이 OPSD 를 이기는 총 마진이 +2.6 / +1.6 / +1.0 이므로, 그 마진의 절반가량 — HMMT-25 에서는 1.0 중 0.8 — 이 Knowledge Seeding 없이 구조적 확장만으로 얻어진다. 이 표에는 시도도 신뢰구간도 없다. 판정은 ch21 로 넘긴다.

[평가] 같은 표의 다른 행은 더 직접적이다. Semantic Reward 를 제거하면 AIME-25 가 오른다 (69.2 대 전체 Sleep 의 69.0) [LM Need Sleep Table 1, p.11]. 그런데 [LM Need Sleep § 4.2, p.13] 은 “All the components contribute positively to the performance of our method” 라고 쓴다. 자기 표가 반증하는 문장이다. Imitation Learning 제거는 76.8 / 67.9 / 45.0 으로 세 열 모두 내려가므로, 이 논문에서 실제로 하중을 받는 항은 semantic 쪽이 아니라 imitation 쪽이다.

seeding 의 목적함수 자체는 § 04.6 의 좌표로 그대로 읽힌다. [LM Need Sleep § 3.3, Eq. 4 뒤의 무번호 식] 은 divergence 항과 reward 항을 섞는다.

$$\mathcal{L}_{KS} = \mathbb{E}_{x \sim \mathcal{D}} \left[(1 - \lambda_{KD}) \mathbb{E}_{y \sim \text{LM}_{\Theta^{\text{st}}}(\cdot|x)} [R(y)] - \lambda_{KD} \mathbb{E}_{y \sim \text{LM}_{\Theta^{\text{st}}}(\cdot|x)} \mathcal{F}(\text{LM}_{\Theta^{\text{te}}} \| \text{LM}_{\Theta^{\text{st}}})(y|x) \right] \quad (4-6)$$

$R(\cdot)$ 는 teacher 의 생성 $d^{(i)}$ 의 임의 접두사를 student 가 이어쓰게 하고 그 결과를 채점하는 imitation reward 이며, semantic 항과 Levenshtein 기반 절대 항의 블록결합이다 [LM Need

Sleep Eq. 3, Eq. 4]. (표기 대응: 원문은 이 보상을 $r(\cdot)$, 두 성분을 $r_{\text{sem}} \cdot r_{\text{abs}}$ 로 쓴다 [LM Need Sleep Eq. 3]. 이 책에서 r 은 LoRA rank 로 예약되어 있으므로 보상은 R 로 옮긴다.) reward 로 Θ 를 움직이는 절차 일반은 ch05 가 소유한다 — 여기서 볼 것은 그 항이 **distillation loss 와 같은 식안에 앉는다**는 사실뿐이다. 즉 (4-6) 은 “teacher 의 분포를 따라가라” (divergence 항) 와 “teacher 가 쓸 법한 문장을 실제로 써내라” (reward 항) 를 하나의 목적으로 묶는다.

[평가] 세 개의 caveat 을 여기에 붙여 둔다. 첫째, (4-6) 의 최적화 방향이 원문에 없다. reward 에서 divergence 를 뺀 형태이므로 최대화해야 하지만, 같은 페이지의 산문은 “minimize the distillation loss between teacher and student” 라고 쓴다 [LM Need Sleep § 3.3(b)]. 둘째, 원문은 같은 줄에서 $\mathcal{D}$ 를 데이터셋과 divergence 양쪽에 쓰고, 한 문단 앞 (4-5) 에서는 divergence 를 $\mathcal{F}$ 로 쓴다 — 이 책은 $\mathcal{F}$ 로 통일하고 $\mathcal{D}$ 는 데이터셋 전용으로 둔다. 셋째, λ_{on} , λ_{KD} , reward 혼합 계수, Levenshtein 임계값 어느 것도 **논문 어디에도 값이 없다** — Table 5 는 learning rate · batch · steps · LoRA rank / alpha 만 싣는다. 세 항을 합치면 판정은 하나다: **이 절차는 논문만으로 재현되지 않는다**.

기제 전체가 pre-trained Llama/Qwen 위에 얹은 graft 이며 end-to-end 로 meta-learn 된 것이 아니라는 사실은 NM ch17 에서 이미 확정되었고, ch21 이 그대로 승계한다.

한 가지 더. [SEAL § 3.1, p.4] 마지막 문단은 teacher 와 student 를 **다른 모델로** 분리하는 확장을 제안한다 — teacher 가 편집을 제안하고, student 가 그것으로 갱신되며, teacher 는 student 의 개선을 최대화하도록 학습된다는 구도다. 제안만 되어 있고 실행된 실험은 없다. 이장의 정의가 teacher/student 를 역할로만 규정한 이유가 여기서 드러난다: 같은 프레임 안에서 self-distillation, upward distillation, 그리고 이 분리형 제안이 전부 Θ^{te} 와 Θ^{st} 의 관계를 무엇으로 두느냐의 선택지로 정리된다.

04.6 distillation loss 의 형태 — 세 좌표로 읽는다

지금까지의 모든 변형은 하나의 뼈대에 붙는다.

$$\mathcal{L}_{\text{distill}} = \sum_i w_i \mathbb{E}_{y \sim \pi_i} \left[\mathcal{F}_i(g_i(\Theta^{\text{te}}; x, y) \parallel g_i(\Theta^{\text{st}}; x, y)) \right] \quad (4-7)$$

이 식이 말하는 것은 distillation 을 정하는 자유도가 정확히 셋이라는 것이다 — 무엇을 맞출 것인가 (g_i : 대상), 어떤 거리로 잴 것인가 ($\mathcal{F}_i$: divergence), 어느 분포에서 표본을 뽑을 것인가 (π_i : 표본 출처). 표 4-1 이 이 장에서 나온 것들을 그 좌표에 배치한다.

표 4-1 — distillation 의 세 좌표

변형	대상 g	divergence $\mathcal{F}$	표본 출처 π
logit distillation (§ 04.2)	$ \mathcal{V} $ 차원 출력 분포, 온도 T	forward/reverse KL	고정 데이터셋
feature distillation (§ 04.3)	층 ℓ 의 hidden h_ℓ (사영 P_ℓ 경유)	제곱 ℓ_2	고정 데이터셋
attention distillation (§ 04.3)	$L \times L$ attention 확률	KL 또는 제곱 ℓ_2	고정 데이터셋
off-policy self-distill (§ 04.4)	출력 분포	KL	고정 데이터셋
on-policy self-distill / GKD (§ 04.4)	출력 분포	KL	student 자신, 비율 λ_{on}

변형	대상 g	divergence $\mathcal{F}$	표본 출처 π
Knowledge Seeding (§ 04.5)	출력 분포 + 생성 접두사 완성	$\mathcal{F} + \text{imitation reward}$, 비율 λ_{KD}	student 자신 [LM Need Sleep Eq. 3]

마지막 행이 이 corpus 가 실제로 쓰는 좌표다. 대상이 둘이고 (분포와 생성), 표본이 student 쪽이며, teacher 는 자기 자신의 확장 이전 상태다.

[해설] 이 세 좌표는 비용과 1:1 로 대응한다. **대상**은 저장량을 정한다 (logit 이면 $|V|$ 폭, feature 면 층 수 $\times d$). **divergence** 는 사실상 공짜다 — elementwise 연산이다. **표본 출처**는 지배 커널을 정한다 (고정 데이터셋이면 GEMM, student 자신이면 decode). 실무에서 distillation 파이프라인의 비용을 추정할 때 먼저 봐야 할 칸은 세 번째다.

(state, update, cost) 정리

이 장의 개념을 세 층 프레임에 놓으면 표 4-2 가 된다. 요점은 단순하다 — **distillation** 은 전부 Θ 층의 도구다. W 도 E 도 건드리지 않는다.

표 4-2 — 이 장의 개념과 세 층 프레임

개념	어느 층	갱신식	비용 4 종에서 어디에 뜨는가
teacher forward	어느 층도 갱신하지 않음 (Θ^{te} read-only)	없음	B_s — batch prefill 과 같은 커널
logit distillation	Θ^{st}	(U- Θ), $\mathcal{R}_k$ = (입력, teacher 분포) 쌍	B_s + teacher logit 저장
feature distillation	Θ^{st} + 일회용 사영 P_ℓ	(U- Θ)	B_s + teacher activation 점유
self-distillation (off-policy)	Θ^{st} , teacher = Θ_k 스냅샷	(U- Θ)	B_s ; 두 파라미터 사본이 동시상주
on-policy self-distill ($\lambda_{\text{on}} > 0$)	위와 같음	(U- Θ), $\mathcal{R}_k$ 가 매 step 재샘플	B_s 가 decode 에지 배됨
upward distillation	$\Theta_{k+1} \supset \Theta_k$ (차원 증가)	(U- Θ) + 확장 + sender 리셋	C_{cap} — consolidation 당 $2d d_{\text{low}}$ 파라미터 [LM Need Sleep § 3.2]

L_w (wake 지연) 는 이 장의 어떤 절차에서도 직접 오르지 않는다. distillation 은 전부 질의 도착 전에 끝나고, wake 에 남는 것은 바뀐 배포아티팩트뿐이다. ρ (망각률) 는 반대로 **이 장의 절차들이 정면으로 겨냥하는 값**이지만, 그것을 측정할 수치는 이 장의 근거 논문 어디에도 없다.

Worked micro-example — temperature 하나가 gradient 와 바이트에 하는 일

이 절의 수치는 전부 **이 책의 예시 계산**이다. 논문에서 가져온 값이 하나도 없고, 손으로 계산할 수 있도록 로짓 · 어휘크기 · 시퀀스 길이를 이 책이 골랐다.

어휘가 세 개인 장난감 모델을 쓴다. 한 토큰 위치에서 teacher logit 이 $z^{\text{te}} = (4, 2, 0)$, student logit 이 $z^{\text{st}} = (2, 2, 2)$ 라 하자. student 는 아직 아무것도 모르는균등분포다.

(1) **soft target.** (4-1) 로 $T = 1$ 과 $T = 4$ 의 teacher 분포를만든다. $T = 4$ 먼 지수의 인자가 (1.0, 0.5, 0) 이 되어 $e^1 = 2.71828$, $e^{0.5} = 1.64872$, $e^0 = 1$ 이므로 합이 5.36700 이다.

T	$p^{\text{te},(T)}$
1	(0.86681, 0.11731, 0.01588)
4	(0.50648, 0.30720, 0.18632)

세 번째 좌표가 0.0159 에서 0.1863 으로 **11.7 배** 커진다. 이것이 dark knowledge 를 학습 신호로 끌어올린다는말의 실제 내용이다.

(2) **divergence.** student 는 어느 T 에서도 $(1/3, 1/3, 1/3)$ 이므로 $\mathcal{F}(p^{\text{te}}||p^{\text{st}}) = \ln 3 - H(p^{\text{te}})$ 다. $\ln 3 = 1.09861$ 이고 엔트로피는 각각 0.44106, 1.02019 nat 이므로

$$\mathcal{F}|_{T=1} = 0.65755 \text{ nat}, \quad \mathcal{F}|_{T=4} = 0.07842 \text{ nat}.$$

같은 두 모델인데 loss 값이 **8.38 배** 차이 난다. T 를 바꾸면 loss 의 눈금 자체가 바뀐다는 뜻이고, 이것이 (4-2) 에 정규화상수가 필요한 이유다.

(3) **gradient 와 T^2 .** (4-3) 으로 student logit 에 대한 gradient 를 직접 계산한다.

T	$\partial\mathcal{F}/\partial z^{\text{st}}$
1	(-0.53348, +0.21602, +0.31746)
4	(-0.04329, +0.00653, +0.03675)

첫 좌표의 크기 비는 $0.53348/0.04329 = 12.32$ 다. 여기에 $T^2 = 16$ 을 곱하면 $0.04329 \times 16 = 0.69269$ 가 되어 $T = 1$ 의 0.53348 을 **30% 초과한다.** 즉 T^2 보정은 이온도 · 이 로짓 간격에서 정확하지 않다 — 정확한 것은 로짓차이가 T 에 비해 작은 고온 극한이다. 실무적 함의는 분명하다: T 를 크게 흔들면 w_{KD} 의 실효값이 따라 움직이므로 두 값을독립적으로 튜닝했다고 믿으면 안 된다.

(4) **바이트.** 이제 같은 계산을 실제 어휘 크기로 옮긴다. 예제값으로 $|\mathcal{V}| = 128,000$, 시퀀스 길이 $L = 2,048$, dtype bf16(2 bytes) 을 쓴다. teacher logit 전체를 들고 있으려면

$$2,048 \times 128,000 \times 2 \text{ B} = 524,288,000 \text{ B} = 500 \text{ MiB}$$

가 시퀀스 한 개당 필요하다. 8B 모델의 bf16 가중치가 16 GB 이므로, 시퀀스 하나의 logit 텐서가 이미 가중치의 약 3.3%다. distillation 집합이 200 시퀀스면 97.7 GiB — 캐시할 수 있는 크기가 아니다. top- k 로 자르면 ($k = 64$, 값 bf16 2 B + 인덱스 int32 4 B)

$$2,048 \times 64 \times 6 \text{ B} = 786,432 \text{ B} = 768 \text{ KiB},$$

즉 **666.7 배** 작아진다. 잘려 나간 $128,000 - 64$ 개 좌표가 정확히 (1) 에서 확대했던 꼬리라는 점이 이 절의 결론이다. **temperature 로꼬리를 키우는 일과 top- k 로 꼬리를 버리는 일은 같은 파이프라인 안에서서서로 상쇄한다.** 두 값을 따로 정하면 안 된다.

(5) **표본 출처가 이 계산을 무의미하게 만드는 경우.** $\lambda_{\text{on}} > 0$ 이면 위의 캐시 논리의 자체가 성립하지 않는다. student 가 매 step 새로 생성한 y 위에서 divergence 를 재므로 teacher logit 을 미리 계산해 둘 대상이 없고, teacher 는 매번 student 의 생성물을다시 forward 해야 한다. 저장 문제가 사라지는 대신 지배 커널이 decode 로바뀐다. 이 교환의 크기를 재려면 절대 비용이 필요한데 논문이 내놓는 것은비율 하나뿐이다 — [LM Need Sleep App. B.5] 는 같은 step 수에서 SFT 가자기 방법보다 $4 \times$ 효율적이라고 보고하고, 그 비교 대상은 SFT 하나뿐이다.

요약

- distillation 은 gradient 의 출처를 label 에서 다른 모델의 출력 분포로 바꾸는 절차이고, teacher 와 student 는 크기가 아니라 **파라미터의 이동 여부**로 정의된다.
- temperature T 는 teacher 분포의 꼬리를 학습 신호로 끌어올리는 다이얼이며, T^2 보정은 hard/soft 혼합 비율 w_{KD} 의 실효 의미를 T 와 분리하기 위한 것이다. 그보정이 정확한 것은 고온 극한뿐이고, 이 책의 예시 계산 ($T = 4$) 에서 실제 비는 12.32 대 16 이다.
- logit distillation 의 비용은 $|V|$ 폭 텐서의 저장이고 feature distillation 의 비용은 teacher activation 의 수명 연장이다. 전자는 top- k 로 줄일 수 있으나 (이 책의 예시 계산에서 666.7 배) 그대가가 정확히 dark knowledge 의 꼬리다.
- self-distillation 이 이득을 내는 이유는 지식의 이동이 아니라 세 가지다: 표본 출처의 정합 (on-policy), 목표의 부드러움, 그리고 옛 함수를 붙잡는 앵커 효과.
- on-policy 비율 λ_{on} 은 sleep job 의 지배 커널을 GEMM 에서 autoregressive decode 로 바꾼다. 그 결과 sleep 은 training 클러스터가 아니라 wake 트래픽과 같은 병목을 놓고 경쟁한다.
- upward distillation 에서 옮겨지는 것은 정보량이 아니라 좌표다. 용량확장 자체가 옛 함수에 대한 손상이고, Knowledge Seeding 은 그 손상을 되돌려 **비어 있고 정렬된 용량**을 남긴다. sender 블록의 리셋이 이것을 복사가 아니라 전송으로 만든다.
- [LM Need Sleep Table 1, p.11] 에서 Sleep 이 OPSD 를 이기는 마진 +2.6 / +1.6 / +1.0 중 +1.3 / +0.8 / +0.8 은 Knowledge Seeding 없이 구조적 확장만으로 얻어지고, Semantic Reward 를 빼면 AIME-25 는 69.0 에서 69.2 로 오른다. 시드도 신뢰구간도 없다.
- distillation 은 전부 Θ 층의 도구다. W 도 E 도 건드리지 않고 L_w 를 직접 올리지 않으며, 정면으로 겨냥하는 ρ 는 이장의 근거 논문 어디에도 측정되어 있지 않다.

자가 점검체크리스트

- teacher/student 를 크기가 아니라 파라미터의 이동 여부로 정의했고, 그 정의 덕분에 “작은 teacher, 큰 student” 가 형용모순이아님을 설명할 수 있다.
- (4-2) 에서 T^2 이 붙는 이유와, 그 보정이 정확해지는 조건을 함께 말할 수 있다.
- logit distillation 과 feature distillation 중 어느 쪽이 batch 크기를 더깎는지, 그 이유를 저장의 성질 (일회성 대 점유) 로 설명할 수 있다.
- λ_{on} 을 0 에서 1 로 올렸을 때 sleep job 의 지배 커널과 roofline 위치가 어떻게 바뀌는지 말할 수 있다.
- upward distillation 에서 새 expert 로 옮겨지는 것과, 반대로 그 절차로는 **생기지 않는 것**을 구분해 말할 수 있다.
- [LM Need Sleep Table 1] 의 어느 행 비교가 “구조적 확장만의 기여” 를 분리해 주는지 지목하고 그 값을 숫자로 답할 수 있다.
- (Rosetta) “모델 재배포 = $(U-\Theta)$ 1 회” 라는 대응을 이 장의 절차에 적용해, distillation 이 끝나는 순간 serving 에서 정확히 무엇이 달라지는지 말할 수 있다. 그리고 자신이 이미 배포 중인 speculative decoding 의 draft model 이 바로 이 장의 student 라는 것을 확인했다.

다음 장으로

이 장의 모든 절차는 하나의 전제 위에서 있다 — **teacher 의 출력이 따라갈 만하다**는 전제다. 그 전제가 성립할 때 divergence 를 줄이는 것은 곧 개선이다. 그러나 sleep-time compute 의 목표는 배포된 모델이 **자기보다 나아지는 것**이고, 자기 자신을 teacher 로 삼는 순간 divergence 를 0 으로 만드는 최적해는 아무것도 바꾸지 않는 것이다. 실제로 (4-6) 은 이미 그 벽을 넘고 있었다: divergence 항 옆에 붙은 imitation reward 항은 “teacher 를 닮아라” 가 아니라 “채점을 통과하라” 는 신호이며, 그것은 distillation 의 문법이 아니다.

따라서 다음 질문은 이것이다 — **teacher**가 없거나 **teacher**를 넘어서야 할 때, 학습 신호를 어디서 얻는가. ch05가 그 답을 다룬다: 정답분포 대신 스칼라 보상으로 Θ 를 움직이는 절차, 그 절차가 요구하는 채점기, 그리고 채점기가 있으면 반드시 따라오는 reward hacking이다. [SEAL]과 [LM Need Sleep]이 공유하는 ReST^{EM} 루프도 거기서 정의된다.

ch05. RL for LLM — 보상으로 가중치를 움직이기

이 장의 목표 — 독자가 이 장을 마치면 다음을 할 수 있어야 한다. (1) 임의의 LLM 학습 루프를 보고 무엇이 state, 무엇이 action, 무엇이 reward 인지 지목하고, 그 셋이 decode loop 의 어느 부분에 해당하는지 대응시킬 수 있다. (2) policy gradient 의 최소 골격 (rollout $\rightarrow$ reward $\rightarrow$ advantage $\rightarrow$ 확률 이동) 을 손으로 한 step 돌리고, 왜 reward 가 미분불가능해도 되는지 설명할 수 있다. (3) RLHF 파이프라인 · DPO · RLVR 를 “무엇이 채점하는가” 와 “어떤 하드웨어를 먹는가” 의 두 축으로 구분하고, 각각이 sleep 라운드에 었힐 때의 비용 모양을 말할 수 있다. (4) reward hacking 을 정의하고, RL 이득 보고에 어떤 통제군이 붙어있어야 하는지 판정할 수 있다.

왜 필요한가 — Part II 의 세 장이 이 어휘 없이는 읽히지 않는다. **ch20 의 SEAL** (Self-Adapting Language Models, arXiv:2506.10943) 은 [SEAL § 3.1] 전체가 RL 표기로 쓰여 있다 — 목적함수 [SEAL Eq. 1], 이진 보상 [SEAL Eq. 2], gradient 추정량 [SEAL Eq. 3 – 4], 그리고 알고리즘 선택의 근거 (“GRPO 와 PPO 는 불안정했다”) [SEAL § 3.1, p.4]. 이 식들을 읽으려면 policy·rollout·advantage 가 무엇인지 알아야 한다. **ch21 의 Language Models Need Sleep** (arXiv:2606.03979) 의 Dreaming 단계는 semantic reward 와 distillation 을 계수로 섞는다 — 그 기호들 (ρ_r , λ_{KD} , r_{sem}) 은 NM 에서 이미 만났지만 ($\rightarrow$ NM ch17), “reward 를 섞는다” 가 최적화 관점에서 무엇을 하는 일인지는 이 장이 공급한다. (reward 혼합 계수는 ch21 에서 ρ_r 로 쓴다. 맨 ρ 는 이 책에서 비용 4 종의 망각 · 열화율로 예약되어 있어 덮어쓸 수 없다.) **ch15 의 ReasoningBank** (arXiv:2509.25140) 은 agent 를 $\pi_L(\cdot | M, A)$ 라는 policy 표기로 쓰고 자기 루프가 “RL 의 학습동역학을 닮았다” 고 서술하는데 [ReasoningBank § 3.1, § 5], 정작 gradient 는 한 번도 계산하지 않는다. 그 차이를 독자가 스스로 판정하려면 RL 이 실제로 무엇을 하는지 먼저 알아야 한다.

NM 과의 관계 — 넘길 것: backward pass 와 optimizer state 의 객체적 이해는 전제한다 ($\rightarrow$ NM ch02). stream 위의 단일 패스 최적화, regret, comparator 는 $\rightarrow$ NM ch03. inner/outer loop 의 형식화는 $\rightarrow$ NM ch04. Sleep 의 GKD 계수 표기는 $\rightarrow$ NM ch17. 더할 것은 하나다: **NM 의 어느 장에도 reward 가 없다.** NM ch03 의 online protocol 에서 loss 는 stream 이 자기 label 을 들고 도착하는 self-supervised 회귀였다 ($\ell(W; k_t, v_t) = \|\mathcal{M}(k_t; W) - v_t\|_2^2$). RL 의 reward 는 생성이 끝난 뒤 **밖에서** 도착하는 스칼라 하나다. 이 차이가 이 장의 전부다. 그리고 어휘 충돌 하나를 미리 못 박는다 — NM ch03 의 “online” t 는 token 인덱스이고, 이 장의 episode step 도 token 이지만, **NM ch03 의 regret 보장을 이 장의 policy 성능주장으로 옮겨 읽으면 안 된다.** 둘은 다른 comparator 에 대한 다른 보장이다.

05.1 LLM 위에서 무엇이 state 이고 무엇이 action 이고 무엇이 reward 인가

RL 을 처음 만나는 systems 엔지니어가 막히는 지점은 알고리즘이 아니라 **대응**이다. “환경과 상호 작용하며 누적 보상을 최대화한다” 는 문장은 로봇이나 게임을 떠올리게 하지만, LLM 에는 로봇도

물리 환경도 없다. 대응은 아주 구체적이고, 한 번 못 박으면 다시 흔들리지 않는다.
 표 5-1 — LLM 위의 RL 대응표. 오른쪽 열이 독자가 이미 매일 다루는 대상이다.

RL 의 항	LLM 에서의 정체	decode 세계의 대응물
state s_t	지금까지의 token 열 = prompt q + 생성된 $z_{<t}$	KV cache 가 담고 있는 바로 그것
action a_t	다음 token 하나	한 번의 decode step 의 출력
action space $\mathcal{A}$	vocabulary 전체	logits 벡터의 차원
전이 s_{t+1}	s_t 에 방금 뽑은 token 을 붙인 것	KV cache append
episode	한 번의 완결된 생성	한 request 의 decode 전체
horizon	생성 길이	max_new_tokens
reward R	생성이 끝난 뒤 밖에서 도착하 는 스칼라 하나	대응물 없음 — 이것이 새것 이다

이 표에서 셋을 강조한다.

첫째, 전이가 결정론적이고 완전히 알려져 있다. s_{t+1} 은 s_t 뒤에 a_t 를 이어 붙인 것, 곧 문자열 concat 이다. 로봇 RL 이 씨름하는 환경 모델 학습이 여기에는 없다. 이것은 LLM RL 을 예외적으로 다루기 쉽게 만드는 성질이고, 동시에 “환경” 이라는 단어가 오해를 부르는 이유다 — 환경은 KV cache append 연산이다.

둘째, 보상은 거의 항상 terminal 이다. 중간 token 에는 보상이 없고, 생성이 끝난 뒤 채점기가 한 번 호출되어 스칼라 하나를 준다. 즉신호는 극도로 성기다. 500 token 을 뽑고 “0” 을 받으면 그 500 개 중 어느결정이 잘못이었는지는 알려주지 않는다. RL 알고리즘 설계의 절반은 이 credit assignment 문제를 다루는 장치다.

셋째, discount factor 를 쓰지 않는다. episode 가 유한하고 짧으며, LLM RL 의 관행은 undiscounted return 이다. 그래서 이 책은 discount 기호를 아예 도입하지 않는다 — $\gamma_{t,i}$ 는 NM 이 Omega rule 의 window gate 로 이미 예약한 기호이고, 덮어쓰면 두 책이 갈라진다.

[해설] systems 관점에서 이 대응의 실용적 귀결은 하나다. RL 의 state 가 KV cache 와 같은 것이라면, **RL 훈련의 절반은 batch decode job 이다**. rollout 을 만드는 단계는 독자가 매일 튜닝하는 그워크로드 — memory-bandwidth bound, KV cache 가 HBM 을 지배하고, batch size 와 prefix 공유가 처리량을 정하는 — 와 문자 그대로 같다. “RL 로 후처리한다” 는 말을 들었을 때 무엇이 GPU 를 먹는지 감이 서지 않았다면, 절반은 서빙이고 절반은 훈련이라고 생각하면 정확하다.

05.2 policy, rollout, advantage — policy gradient 의 최소골격

policy 는 state 에서 action 으로 조건부 분포다. LLM 에서는 별도의 policy network 가 없다 — **모델 자신이 policy 다**: $\pi_{\Theta}(z_t | q, z_{<t})$ 가 곧 softmax 출력이고, policy 를 갱신한다는 것은 Θ 를 움직인다는 것, 즉 식 (U- Θ) 를 실행한다는 것이다. 이 동일시가 LLM RL 의 출발점이자, RL 을 이 책의 3 층프레임에 놓을 수 있는 이유다.

rollout 은 현재 policy 로 prompt 하나에서 끝까지 sampling 한 완결된 생성 하나와, 그것에 매겨진 보상의 쌍이다. rollout 하나 = decode 한 번. 표기는 생성된 token 열을 z , 그 t 번째 token 을 z_t , 보상을 R 로 쓴다.

이 장에서만 쓰는 보조 기호를 먼저 선언한다 (이 책의 예약 기호와 충돌하지 않도록 전부 개명했다).

표 5-2 — 장-국소 기호 선언

이 장의 기호	의미	왜 이 기호인가
π_{Θ}	policy = 모델 자신	Θ 가 학습 대상임을 기호에 드러냄
z, z_t	rollout(생성 token 열) 과 그 t 번째 token	y_t (layer 출력) 와 충돌 회피
$\hat{A}_i$	rollout i 의 advantage	NM 의 보조 accumulator A_t 와 hat·첨자로 구분
R_{ψ}	reward model	첨자 ψ 로 $\mathcal{R}_k$ (학습집합)· R_t (NM 의 retention 항) 와 구분
β_{KL}	KL penalty 계수	β_t (inner momentum decay) 와 반드시 구분
M	한 prompt 당 rollout 수	—
c	context(질의 이전에 주어진 문서·시연)	Atlas 의 window 길이 c 는 이 장에 등장하지 않음

표 5-2 각주 — 보상 기호의 대응. 이 절에서 만나는 원 논문들 (PPO, GRPO, ReST^{EM}, SEAL 포함) 은 보상을 소문자 r 로 쓴다. 이 책은 r 을 LoRA rank 에 예약했으므로 ($\rightarrow$ ch03) 보상은 전부 대문자 R 로 옮겨쓴다. 원문 직접 인용 안의 r 은 그대로 둔다. NM 에서 승계한 r_{sem} ($\rightarrow$ NM ch17) 만 예외이며, 승계 기호는 재정의하지 않는다.

policy gradient 의 핵심 식은 하나다. 목적은 기대 보상을 최대화하는 것, 즉 outer loss 를 그 음수로 놓고,

$$\nabla_{\Theta} \mathcal{L}_{\text{RL}} = -\mathbb{E}_{z \sim \pi_{\Theta}(\cdot|q)} \left[\hat{A}(z) \nabla_{\Theta} \log \pi_{\Theta}(z | q) \right] \quad (5-1)$$

이 식이 말하는 것은 한 문장이다: **샘플이 좋았으면 그 샘플을 다시낼 확률을 올리고, 나빴으면 내린다.** 그리고 결정적으로, gradient 는 보상자체를 미분하지 않는다. 미분되는 것은 $\log \pi_{\Theta}$ 뿐이다. 따라서 **보상은 미분 불가능해도 된다** — 사람이어도, 프로그램이어도, 다른모델이어도, 심지어 “finetune 한 번 돌려서 평가한 점수” 여도 된다. 이것이 SFT 대신 RL 을 쓰는 유일하고 충분한 이유다: RL 은 **미분 불가능한채점기를 loss 로 만드는 장치**다.

식 (5-1) 의 gradient 는 sequence 하나에 대한 것이지만 실제 계산은 token 단위로 풀린다. $\log \pi_{\Theta}(z | q) = \sum_t \log \pi_{\Theta}(z_t | q, z_{<t})$ 이므로

$$\nabla_{\Theta} \log \pi_{\Theta}(z | q) = \sum_{t=1}^{|z|} \nabla_{\Theta} \log \pi_{\Theta}(z_t | q, z_{<t}) \quad (5-1')$$

이고, 여기에 스칼라 $\hat{A}(z)$ 가 곱해진다. 이 형태가 § 05.1 에서 말한 credit assignment 의 실체다 — **생성 전체에 매겨진 점수 하나가 그안의 모든 token 에 똑같은 계수로 곱해진다.** 어느 token 이 잘했는지에 대한 정보는 어디에도 없고, 여러 rollout 에 걸쳐 통계적으로만 분리된다. SEAL 의 추정량이 정확히 이 token 전개 형태로 쓰여 있다 [SEAL Eq. 4, § 3.1, p.4]. systems 관점의 귀결: 갱신 단계의 GEMM shape 은 SFT 와 완전히 같고 (이미 확정된 token 열 위의 forward+backward), 달라지는 것은 per-sequence 스칼라 하나가 loss 에 곱해진다는 것뿐이다. **RL 의 갱신단계는 가중치가 붙은 SFT 다.**

$\hat{A}$ 자리에 보상 R 을 그냥 넣으면 두 가지가 망가진다. 분산이크고, 더 나쁘게는 모든 rollout 이 $R > 0$ 이면 전부 확률을 올려 아무 것도 구분하지 않는다. **advantage** 는 그 rollout 의 보상에서 같은 prompt 에 대한 기준선 (baseline) 을 뺀 값이다: $\hat{A}_i = R_i - b(q)$. 기준선을 빼도 gradient 의 기댓값은 변하지 않고 분산만 준다 — advantage 가하는 일은 “이 rollout 이 평소보다 좋았는가” 로 질문을 바꾸는 것이다.

baseline 을 만드는 방식이 알고리즘 계보를 가른다. 하나는 별도의 value network 를 학습해 $b(q)$ 를 예측하는 것이고 (PPO 계열, Schulman et al. 2017, arXiv:1707.06347; advantage 추정은 Schulman et al. 2015, arXiv:1506.02438), 다른 하나는 같은 prompt 에서 M 개의 rollout 을

뽑아 그 표본 평균을 baseline 으로 쓰는 것이다 (group-relative 방식, Shao et al. 2024, arXiv: 2402.03300). 후자는 value network 를 통째로 없애 HBM 에서 모델 하나를 지운다.

여기에 두 개의 안전장치가 붙는다. 하나는 **KL penalty**: 참조 policy π_{ref} (보통 RL 시작 시점의 SFT 모델) 에서 멀어지는것을 β_{KL} 로 벌해, policy 가 보상만 좇다가 언어모델이기를 그만두는 것을 막는다. 다른 하나는 **clipping**: 한 batch 의 rollout 으로 여러 gradient step $s = 1, 2, \dots$ 를 밟을 때 policy 가 rollout 을 뽑은 분포에서 너무 멀어지지 않도록 확률비를 잘라낸다 (PPO 의 정의적 장치).

여기서 첨자 하나를 못 박는다. s 는 **optimizer 가 밟는 SGD step** 이고, k 는 **sleep 라운드**. 한 sleep 라운드 k 하나가 여러 step s 를 품으며, 식 (U- Θ) 의 Θ_{k+1} 은 그 step 들이모두 끝난 뒤의 결과물이다. rollout 을 다시 뽑아야 하는 주기는 k 의 시계이고, clipping 이 지키는 것은 s 의 시계다. 이 책은 두 첨자를 섞어쓰지 않는다.

[해설] RL 한 iteration 은 서로 arithmetic intensity 가정반대인 세 단계다. ① **roll-out 단계** — decode. memory-bandwidth bound, KV cache 가 지배, 배치가 클수록 좋다. ② **채점 단계** — 무엇이 채점하느냐에 따라 GPU forward 일 수도, CPU 프로그램일 수도, 외부 API 일 수도 있다. ③ **갱신 단계** — 이미 확정된 token 열 위의 forward+backward. prefill 모양의 큰 GEMM, compute bound. 실무 스택이 sampler(vLLM 류) 와 trainer 를 따로 두고 weight 를 왕복시키는 이유가 이것이고, ch20 이 다룰 SEAL 의 구현도 정확히 그 구조다 — 한 sleep 라운드에 vLLM 과 DeepSpeed ZeRO-3 가 동시에 상주한다 [SEAL § B.5, p.21].

05.3 RLHF 파이프라인 — 채점기를 학습해서 사람을 대신세운다

식 (5-1) 은 보상이 있다고 가정한다. 문제는 우리가 원하는 것 대부분에 채점기가 없다는 것이다. “도움이 되는 답”, “요약이 원문에 충실한가” 같은 목표에는 정답 문자열도 단위 테스트도 없다. RLHF 는 이 공백을 **채점기를 학습해서** 메운다.

reward model 은 사람이 매긴 선호 비교 데이터로 학습된, (prompt, 응답) 쌍을 스칼라 점수로 보내는 함수 $R_\psi(q, z)$ 이다. 사람에게 절대 점수를 매기게 하지 않고 **둘 중 어느 쪽이 나은가**만 묻는다는 것이 설계의 핵심이다 — 절대 점수는 평가자 간에 재현되지 않지만 쌍 비교는 상당히 재현된다. 선호를 확률로 바꾸는 표준 모형은 Bradley – Terry 이고,

$$P(z^+ \succ z^- | q) = \sigma(R_\psi(q, z^+) - R_\psi(q, z^-)) \quad (5-2)$$

R_ψ 는 이 로그가능도를 최대화하도록 학습된다 (Christiano et al. 2017, arXiv:1706.03741). 식 (5-2) 가 말하는 것: reward model 이 배우는 것은 점수의 절대 눈금이 아니라 **차이**뿐이다. 상수를 더해도 식이 변하지 않으므로 눈금 원점은 정의되지 않는다.

RLHF 파이프라인은 이 reward model 을 가운데 두고 학습을 세단계로 나누는 후처리 절차다: (1) 사람이 쓴 시연으로 supervised fine-tuning 을 해 policy 의 형식을 잡고, (2) 그 policy 가 뽑은 응답들에 대한 사람의 쌍 비교로 reward model R_ψ 를 학습하고, (3) R_ψ 를 보상으로 삼아 식 (5-1) 로 policy 를 갱신한다 (Stiennon et al. 2020, arXiv:2009.01325; Ouyang et al. 2022, arXiv:2203.02155).

표 5-3 — RLHF 세 단계에서 무엇이 움직이는가

단계	움직이는 것	데이터	프레임
SFT	Θ	사람이 쓴 (prompt, 응답)	(U- Θ), $\mathcal{R}$ 은 사람이 만들
reward model 학습	ψ (별도 모델)	사람의 쌍 비교	(U- Θ) 의 형태이나 대상이 policy 가 아님
RL	Θ	policy 가 스스로 뽑은 rollout	(U- Θ), $\mathcal{R}$ 은 policy 가 만들고 R_ψ 가 채점

1 단계가 먼저인 이유는 실용적이다 — rollout 이 최소한의 형식을 갖추어야 reward model 이 의미 있는 점수를 준다. 그리고 3 단계가 RL 이어야 하는 이유는 § 05.2 의 답 그대로다: R_ψ 의 출력을 Θ 에 대해 미분할 수 없기 때문이 아니라 (사실 R_ψ 는 미분가능하다), z 를 뽑는 **sampling** 이 미분 불가능하기 때문이다. 식 (5-1) 의 score-function 추정량이 정확히 그 벽을 넘는 장치다.

[해설] 3 단계의 HBM 회계를 세어 보면 RLHF 가 왜 무거운지가 바로 보인다. 동시에 상주해야 하는 모델이 넷이다 — 학습 중인 policy, KL 항을 계산할 frozen reference, frozen reward model, 그리고 학습 중인 value network. 넷을 모두 7B 로 잡으면 bf16 가중치만 $4 \times 14 \text{ GB} = 56 \text{ GB}$ 이고, 여기에 학습 대상 둘의 optimizer state 가 더 붙는다 ($\rightarrow$ NM ch02). 이 56 GB 는 어느 논문의 보고값도 아니라 자릿수를 잡기 위한 **이책의 예시 계산**이다 — 7B 라는 크기와 bf16 2 bytes/elem 를 이책이 골랐다. 같은 데이터로 SFT 만 할 때와 비교하면 몇 배다. 이후 알고리즘계보가 전부 “이 넷 중 몇 개를 지울 수 있나” 의 역사인 것은 우연이 아니다 — group-relative baseline 은 value network 를 지우고, 다음 절의 DPO 는 sampler 와 reward model 을 함께 지운다.

05.4 DPO — reward model 을 지우고 preference 를 직접 loss 로

DPO(Direct Preference Optimization) 는 reward model 을 명시적으로 학습하지 않고, 선호 쌍에 대한 닫힌 형태의 분류 loss 로 policy 를 직접 갱신하는 방법이다 (Rafailov et al. 2023, arXiv: 2305.18290).

출발점은 KL 로 정규화된 RL 목적의 최적해가 닫힌 형태를 갖는다는 관찰이다: $\pi^*(z | q) \propto \pi_{\text{ref}}(z | q) \exp(R(q, z)/\beta_{\text{KL}})$. 이 관계를 보상에 대해 뒤집으면 보상을 policy 로 표현할 수 있고, 그것을 식 (5-2) 에 대입하면 정규화 상수가 소거되어 R_ψ 가 통째로 사라진다.

$$\mathcal{L}_{\text{DPO}}(\Theta) = -\mathbb{E}_{(q, z^+, z^-)} \left[\log \sigma \left(\beta_{\text{KL}} \log \frac{\pi_\Theta(z^+ | q)}{\pi_{\text{ref}}(z^+ | q)} - \beta_{\text{KL}} \log \frac{\pi_\Theta(z^- | q)}{\pi_{\text{ref}}(z^- | q)} \right) \right] \quad (5-3)$$

식 (5-3) 에서 무엇이 없는지가 무엇이 있는지보다 중요하다. **sampling 이 없다**. rollout 이 없고, 스칼라 보상이 없고, advantage 가 없고, value network 가 없다. 남은 것은 고정된 선호 쌍 위의지도 학습이며, 필요한 forward 는 policy 와 reference 각각 두 번씩, 총 네번이다.

표 5-4 — 같은 선호 데이터를 쓰는 두 경로의 부품 대조

부품	RLHF (3 단계)	DPO
reward model R_ψ	학습하고 상주	없음
sampler(rollout 생성)	매 iteration	없음
value network	있음 (또는 group baseline)	없음
reference 모델	있음 (KL)	있음 (식 (5-3) 안에)
워크로드 모양	decode + 채점 + 훈련	훈련만

대가는 **off-policy** 라는 것이다. 선호 쌍이 현재 policy 의 분포에서 오지 않으므로, policy 가 데이터를 만든 분포에서 멀어질수록 신호가 낡는다. RLHF 의 rollout 은 정의상 항상 현재 policy 의 것이고, DPO 의 쌍은 한 번 고정된다.

[평가] 이 책은 RLHF 와 DPO 사이에 우열을 매기지 않는다. 두 방법의 선택을 정하는 변수는 목적 도메인과 선호 데이터의 갱신 주기이지 알고리즘의 우아함이 아니며, 그 두 변수를 명시하지 않은 비교는 판정으로 성립하지 않는다.

sleep 라운드의 관점에서 이 절의 요점은 하나다. **DPO 의 워크로드 모양은 SFT 와 같다**. 고정 배치, sampler 없음, 채점기 없음. 유휴 시간에 없기가 비교할 수 없이 싸고, 스케줄링이 훈련 job 하나로 끝난다. ch21 이다를 LM Need Sleep 의 Dreaming 단계가 보상 항과 distillation 항을 계수로 섞는 형태를 취하는 것도 같은 압력 아래에 있다 (그 계수 표기는 $\rightarrow$ NM ch17).

05.5 RLVR — 채점기를 프로그램으로 바꾼다

RLVR(Reinforcement Learning with Verifiable Rewards, 검증가능보상)은 보상을 사람이 나 학습된 모델이 아니라 **결정론적 검증프로그램**이 산출하는 RL이다. 정답 문자열 대조, 단위 테스트 실행, 파서·컴파일러 통과 여부, 출력 형식 정규식이 전형적인 검증기다. 이 이름은 Lambert et al. 2024(Tulu 3, arXiv:2411.15124)가 붙였고, 규칙 기반보상만으로 대규모 추론 학습을 돌린 사례로 DeepSeek-R1(arXiv:2501.12948)이 널리 인용된다.

식은 새로울 것이 없다 — 식 (5-1) 그대로이고 R 의 출처만 바뀐다. 바뀌는 것은 전부 회계와 실패 모드다. 검증기의 종류를 먼저 갈라 두면 그 회계가 보인다.

표 5-5 — 검증기의 네 종류와 그 성질

검증기	예	결정론	어디서 도는가	이 corpus의 사례
문자열 대조	수학 정답, 추출형 QA	완전	CPU, 무시 가능	—
실행	단위 테스트, 컴파일	완전 (샌드박스 고정 시)	격리 프로세스	—
형식 검사	JSON 파싱, 정규식, 스키마	완전	CPU	SEAL의 few-shot self-edit 이 JSON 설정이다 [SEAL Fig. 3, p.5]
모델 채점	LLM-as-a-Judge, 루브릭	아님	GPU 또는 API	Reasoning-Bank의 j_k [Reasoning-Bank § 3.2]; SEAL의 proxy-reward ablation [SEAL Table 9]

앞의 셋만이 엄밀한 의미의 RLVR다. 네 번째 행은 “검증기”라는 이름을 쓰되 실제로는 학습된 채점기이므로 § 05.6의 첫 번째 실패 모드를 그대로 물려받는다 — § 05.7이 다루는 두 자동 채점기가 모두 네 번째 행에 걸린다는 사실은 거기서 다시 만난다.

검증기가 프로그램일 때 얻는 것은 셋이다.

1. **사람 라벨이 필요 없다.** reward model 학습 단계가 통째로 사라진다.
2. **reward hacking의 표면이 좁다.** 채점기가 학습된 함수가 아니므로 policy가 채점기의 일 변화 오차를 타고 오를 수 없다 (§ 05.6). 채점기 자체의 허점은 남는다.
3. **보상을 무인으로, 임의 횟수, 거의 공짜로 만들 수 있다.**

이 책의 관심은 3번에 있다.

[해설] sleep 라운드는 정의상 사람이 없는 시간이다. $(U-\Theta)$ 를 돌리려면 학습 집합 $\mathcal{R}_k$ 가 쓸모 있었는지 판정할 신호가 필요한데, 그 신호를 사람이 준다면 라운드의 처리량은 사람의 처리량에 묶인다 — 유틸 시간을 쓴다는 sleep-time compute의 전체 자체가 무너진다. RLVR는 그 신호를 프로그램으로 바꾼다. 그래서 “sleep 라운드에서 Θ 를 옮긴다”는 설계는 검증 가능한 도메인(수학, 코드, 구조화 출력)에서 먼저 나타난다. **이 연결이 이 장이 Part II에 남기는 예고다.** 실제로 어떤 형태의 자동 채점기가 쓰이는지는 § 05.7이 보이고, 그것이 충분한지는 ch20·ch21이, 세 경로 전체에 대한 답이 되는지는 ch27이 판정한다. 이 장은 판정하지 않는다.

한계도 같은 자리에서 나온다. **검증 가능한 것은 corpus의 일부다**. 임의의 문서를 읽고 그것을 가중치에 넣었는지 판정하려면 그 문서에 대한 문제와 답이 있어야 한다. SEAL은 이 벽에 정면으로 부딪히고, 논문 자신이 그것을 명시한다 — 모든 context에 라벨된 downstream task가 짝지어져 있어야 하며 “이 결합이 보상 계산을 단순하게 만들지만 SEAL의 RL 학습이 라벨 없는 코퍼스로 확장되는 것을 막는다” [SEAL § 5 Context-dependent evaluation, p.9]. 논문이 제안하는 탈출구는 모델이 문서를 문맥에 두고 있는 동안 **평가 문제까지 스스로 생성하게** 하는 것이다 [SEAL § 5, p.9].

[평가] 이 탈출구는 문제를 옮길 뿐 없애지 못한다. 평가문제를 모델이 만들면 보상의 신뢰도는 검증기가 아니라 문제 생성기의 품질에 걸리고, 그 생성기를 채점할 독립 신호는 다시 없다. 즉 라벨 의존이 사라지는 것이 아니라 라벨 의존이 한 단계 안쪽으로 들어간다. 이 장은 이관찰만 남기고, 그것이 SEAL의 실증 범위에 실제로 어떤 상한을 거는지는 ch20이 판정한다.

systems 접점: 검증기는 GPU 밖에 산다. 샌드박스, 테스트 러너, 파서는 CPU·IO 서비스이고, 결정론이므로 캐시할 수 있고 batch로 돌릴 수 있으며 실패해도 재시도가 안전하다. RL 루프의 tail latency가 GPU에서 그 서비스로 옮겨간다. reward model 방식 (rollout마다 채점 모델의 forward가 한 번 더 붙는다)과 비교하면 비용이 완전히 다른 자리에 선다 — 한쪽은 HBM과 GEMM, 다른 쪽은 프로세스 격리와 타임아웃 정책이다.

05.6 reward hacking — 보상이 목적을 대신할때

reward hacking은 policy가 의도한 목적을 달성하지 않은 채 보상신호의 결합을 최대화하는 현상이다. Goodhart의 법칙의 기계학습판이며 (Amodei et al. 2016, arXiv:1606.06565), 버그가 아니라 구조적 귀결이다 — reward는 목적의 대리 (proxy) 이고, RL은 그 대리를 최대화하도록 설계된 최적화기다. 대리과 목적이 갈라지는 지점을 policy가 찾아내는 것은 실패가 아니라 **정확히 시킨 일**이다.

두 종류로 나누면 대응책이 달라진다.

첫째, **학습된 채점기의 오차를 타는 것**. reward model은 고정된 분포에서 학습되었는데 policy는 학습하면서 분포를 옮긴다. policy가 reward model의 훈련 분포 밖으로 나가면 R_ψ 의 점수는 계속 오르는데 실제 품질은 꺾인다 — reward model overoptimization으로 불리며 그 함수적 형태를 측정한 연구가 있다 (Gao·Schulman·Hilton 2022, arXiv:2210.10760). 대응은 세 가지다: KL 항 β_{KL} 로 참조 policy 근처에 묶기, reward model을 새 분포에서 재학습하기, ensemble로 불확실성을 벌하기.

둘째, **결정론적 채점기의 허점을 타는 것**. 단위 테스트만 통과시키는 코드, 형식만 맞춘 빈 답, 채점 스크립트가 참조하는 파일을 건드리는 행위. RLVR도 면역이 아니다. 다만 이 종류는 **재현 가능**하다 — 허점은 프로그램에 있으므로 찾아서 고칠 수 있고, 고치면 사라진다. 학습된 채점기의 오차는 그렇게 국소적이지 않다.

가장 흔한 형태는 따로 이름을 붙일 만하다: **길이 인플레이션**. 길수록 점수가 높아지는 채점기 (사람 평가자를 포함해)에서 policy는 길어진다. 그리고 이 corpus 안에 문서화된 사례가 있다.

SEAL은 RL iteration이 진행될수록 self-edit이 길어지는 것을 그림으로 보고하고 [SEAL Fig. 5, p.8], 논문 자신이 “RL이 예시 응답의 길이를 극적으로 늘린 것으로 나타난다. 그래서 더 긴 생성을 요구하는 prompt를 실험한다”고 쓴다 [SEAL § B.11, p.23]. 그 실험의 결과가 표로 남아있다. RL을 한 라운드도 돌리지 않고 prompt만 “implications-long”으로 바꾸면 49.3, “rewrite”로 바꾸면 49.4이며, 두 값 모두 2라운드 RL을 돌린 “implications”의 47.0보다 높다 [SEAL Table 10, § B.11, p.24].

[평가] 이 표가 보여주는 것은 reward hacking의 교과서적 진단 절차다 — 보상과 상관된 표면적 변량 (여기서는 길이)을 의심하고, RL 없이 그 변량만 prompt로 직접 조작해 같은 이득이 나오는지 본다. SEAL은 그 검사를 실제로 수행했고 결과를 실었다. 이 장이 못 박는 규범은 그 절차 쪽이다: **RL 이득을 보고할 때는 보상과 상관된 표면변량**

을 **prompt** 만으로 재현해 보는 통제군이 있어야 하고, 없으면 그 이득은 기제에 귀속될 수 없다. 논문 본문이 그 통제군 대신 47.0 을 헤드라인으로 계속 쓰는 것은 별개 문제이며, 그 판정은 ch20 이 한다. 덧붙여, 47.0 이라는 값 자체도 세 열 중 단일 passage 열의 값이고 나머지 두 continued-pretraining 열에서는 GPT-4.1 합성 데이터가 SEAL 을 앞선다 [SEAL Table 2, p.8] — 이 범위 조건 없이 47.0 을 인용하지 않는다.

systems 접점: 길이 인플레이션은 품질 지표만 흐리는 것이 아니라 회계를 망가뜨린다. rollout 이 길어지면 decode 시간이 선형으로 늘고 KV cache 가 선형으로 늘며, 채점기가 LLM 이면 채점 비용까지 함께늘어난다. 즉 reward hacking 은 sleep 예산 B_s 를 라운드마다 키운다. 상각식 (A) 의 분자가 학습이 진행되는 동안 자란다는 뜻이고, 학습 초기에 계산한 손익분기 N_q^* 는 나중 라운드에서 성립하지 않는다.

05.7 이 corpus 가 실제로 쓰는 RL 루프 — 하나는 진짜, 하나는 어휘만

Part II 에서 만날 두 논문이 RL 을 정반대 방식으로 사용한다. 나란히 놓으면 장의 어휘가 무엇을 판별하는 도구인지가 드러난다.

05.7.1 SEAL — 보상이 “훈련 실행 한 번” 인 RL

먼저 소유권을 밝힌다. 이 절이 소유하는 것은 SEAL 의 **RL 기제** — 세항의 대응, 추정량의 형태, 보상이 무엇으로 만들어지는가 — 뿐이다. self-edit 이라는 산출물의 형식과 그 실증, 즉 [SEAL Table 2]·[SEAL Table 9]·[SEAL Table 10] 의 수치가 결국 무엇을 입증하는가는 ch20 이 소유한다. 이 절과 § 05.6 이 그표들의 값을 인용하는 것은 기제를 설명하고 통제군의 존재를 확인하기 위해서이며, 그 값들에 대한 판정은 여기서 내리지 않는다.

SEAL 의 outer loop 는 진짜 RL 이다. 그러나 PPO 도 group-relative 방식도 아니다. 논문은 “GRPO 와 PPO 같은 다양한 on-policy 방법을 실험했으나 학습이 불안정했다” 고 쓰고 [SEAL § 3.1, p.4], 대신 **ReST^{EM}** (Singh et al. 2023, arXiv:2312.06585) 을 채택한다 — 논문 자신의 표현으로 “filtered behavior cloning 에 기반한 더 단순한 접근, 달리 말해 rejection sampling + SFT” 다 [SEAL § 3.1, p.4].

세 항의 대응은 § 05.1 의 표준형과 다르다. **state** 는 (c, Θ_k) 인데 관측은 c 뿐이다 — 보상이 파라미터 자신에 의존하지만 “ θ 를 직접 문맥에 넣는 것은 불가능하다” [SEAL § 3.1, p.4]. **action** 은 token 하나가 아니라 self-edit $\mathcal{R}_k$ 전체, 즉 한 번의 완결된 생성이다. **reward** 는 그 $\mathcal{R}_k$ 로 $(U-\Theta)$ 를 돌려 얻은 모델을 downstream 과제 $\mathcal{T}$ 에서 채점한 이진 점수다 [SEAL Eq. 2, § 3.1, p.4].

세 번째 항이 이 corpus 의 핵심 장치다. 채점기가 프로그램도, 사람도, reward model 도 아니고 **학습 실행 그 자체**다 — finetune 한 번 + 평가 한 번이 보상 함수의 본체다. 그 실행 비용은 단위 테스트가 아니라 SFT 한 번이고, 실행의 마지막 채점은 프로그램이 아니라 GPT-4.1 이 greedy decoding 으로 내리는 yes/no 다 [SEAL § B.4, p.20]. 게다가 self-edit 의 점수는 3 개 seed 의 inner-loop 실행 위에서 매겨진다 [SEAL § B.5, p.21].

[평가] 이 보상은 § 05.5 의 표 5-5 의 어느 한 행에도 깨끗이 들어가지 않는다. 이 장의 분류가 여기서 한 칸 모자란다는 뜻이고, 정정은 이렇게 한다 — 빼대는 두 번째 행 (실행) 이 되 실행의 마지막 채점이 네 번째 행 (모델 채점) 이므로, SEAL 의 보상은 **결정론적 검증기가 아니라 모델 채점을 끝에 단 학습 실행**이다. 따라서 § 05.6 의 첫 번째 실패 모드 (학습된 채점기의 오차를 타는 것) 를 물려받으며, RLVR 의 “재현 가능한 허점” 이라는 이점은 여기에 적용되지 않는다.

gradient 추정량은 [SEAL Eq. 3–4] 이고, 이 책의 표기로 옮기면

$$\nabla_{\Theta_k} \mathcal{L}_{\text{RL}} \approx -\frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M R_{ij} \nabla_{\Theta_k} \log \pi_{\Theta_k}(\mathcal{R}_{ij} | c_i) \quad (5-4)$$

식 (5-4) 를 식 (5-1) 과 나란히 놓으면 차이가 정확히 하나다: advantage 자리에 baseline 을 빼지 않은 이진 보상 R_{ij} 가 그대로 들어간다. 그결과 $R = 0$ 인 항은 전부 사라지고, 남는 것은 “보상 1 을 받은 self-edit 들에 대한 SFT” 다. 이것이 filtered behavior cloning 이라는 이름의내용이며, 결정적 성질이 하나 따라온다 — **확률을 내리는 방향의 gradient 가 존재하지 않는다**. 실패한 rollout 은 벌받지 않고 그냥 무시된다.

정직성 주석 둘을 붙인다. 첫째, **식에 쓰인 보상과 실제로 쓴 보상이 다르다**. [SEAL Eq. 2] 는 성능을 개선한 모든 self-edit 에 $R = 1$ 을주는 임계값 규칙인데, 각주 2 는 knowledge incorporation 실험에서 “샘플된후보 중 가장 크게 개선한 단 하나의 self-edit” 에만 보상을 준다고 밝힌다 [SEAL 각주 2, p.4] — 임계값이 아니라 M 개 중 $\arg\max$ 다. 둘째, **보상이 Θ_k 에 의존하는데 그것을 무시한다**. 논문은 “보상항이 θ_t 에 의존하지만 미분 가능하지 않다. 우리는 보상을 θ_t 에 대해 고정된 것으로 취급함으로써 이를 처리한다” 고 명시한다 [SEAL § 3.1, p.4].

[평가] 두 번째 주석의 귀결을 이 책은 논문보다 강하게읽는다. 식 (5-4) 가 최적화하는 것은 보상에 stop-gradient 를 적용한 대리목적이지 [SEAL Eq. 1] 에 쓰인 목적 그 자체가 아니다. 두 목적이 같은최적점을 갖는다는 논증은 논문에 없다. 이 장이 SEAL 의 outer loop 를 “진짜 RL” 이라고 부르는 근거는 $\eta_{\Theta} \nabla_{\Theta}$ 가 실제로 실행된다는사실이지, 쓰인 목적함수가 그대로 최적화된다는 사실이 아니다.

비용은 채점기가 지배한다. self-edit 하나를 평가하는 데 30 – 45 초가 걸리고 [SEAL § 5 Computational overhead, p.9], 한 ReST^{EM} 라운드는 $50 \text{ passage} \times 5 \text{ completion} \times 3 \text{ seed} = 750$ 회의 inner-loop 실행으로 $2 \times \text{H100}$ 에서약 6 시간이다 [SEAL § B.5, p.21]. 그리고 논문 자신이 그 채점기를지우는 ablation 을 싣는다 — TTT 기반 보상을 GPT-4.1 루브릭 채점 (4 개 범주각 1 – 5 점 합) 으로 바꾸면 47.0 대 **45.6** 이고, 시간은 약 6 시간 대**약 5** 분이다 [SEAL Table 9, § B.10, p.23].

[해설] 이 한 줄이 systems 독자에게 이 장에서 가장 정보량이크다. 보상을 “훈련 실행” 에서 “추론 호출” 로 바꾸면 sleep 라운드의원크로드 모양이 통째로 바뀐다 — 훈련 GPU 예약이 API 호출 예산으로내려온다. 두 설계의 품질 차이가 그 비용 차이를 정당화하는지의 판정은 ch20 이 한다. 이 장이 못 박는 것은 대응 자체다: **보상의 종류가 곧 sleep 라운드가 먹는 하드웨어의 종류다**.

05.7.2 ReasoningBank — policy 라고 쓰고, gradient 는없다

ReasoningBank 은 agent 를 policy 표기로 쓴다: “agent policy $\pi_L(\cdot | M, A)$ 는 backbone LLM L 로 파라미터화되고, memory module M 과 action space A 에 조건화된다” [ReasoningBank § 3.1]. (원문 기호를 그대로 옮긴 인용이다. 이 책의 표기로는 원문 L 이 frozen Θ 이고 원문 M 이 E 다. 이 책에서 맨 L 은 sequence 길이로, 맨 M 은 한 prompt 당 rollout 수로 이미 예약되어있으므로 인용 밖에서는 원문 기호를 쓰지 않는다.) [ReasoningBank § 5] 는 이 루프가 “RL 의 학습 동역학을 닮았다” 고 서술한다.

그런데 gradient 가 한 번도 계산되지 않는다. backbone 은 frozen 체크포인트이고 (주력 세 모델은 Vertex AI API 뒤에 있다) [ReasoningBank § 4.1, App B.1], 이 논문에서 갱신되는 것은 JSON 파일 하나뿐이다 [ReasoningBank App A.2]. 부품별 대응을 세면 이렇다.

표 5-6 — ReasoningBank 에서 RL 부품의 실제 정체

RL 부품	ReasoningBank 의 대응물	실제로 있는가
policy π_{Θ}	frozen backbone + 검색된 기억을 붙인 system prompt	있으나 학습되지 않음
rollout	MaTTS 는 같은 질의에 m 개의 궤적을 굴린다 [ReasoningBank § 3.3]	있음

RL 부품	ReasoningBank 의 대응물	실제로 있는가
reward	LLM-as-a-Judge 의 이진 라벨 j_k , temperature 0.0 [ReasoningBank § 3.2, App A.2]	있음 (단 정답 라벨 없음)
advantage	—	없음
policy 갱신 $\eta_{\Theta} \nabla_{\Theta}$	이 자리에 (U-E) 가 들어가 있다	없음

[평가] RL 어휘 (“policy” , “self-evolving” , “test-time learning”) 를 쓰면서 어떤 층의 파라미터도 움직이지 않는 것은 이 corpus 에서 반복되는 서술 패턴이다. 판별 기준은 단순하다 $-\eta_{\Theta} \nabla_{\Theta}$ 가 어디에선가 실제로 실행되는가. ReasoningBank 에서는 실행되지 않는다. 이것이 결함이라는 뜻은 아니다. E-경로는 그렇게 작동하도록 설계된 것이고, 그 설계가 가진 이점 (체크포인트관리도, optimizer state 도, 망각 대응도 필요 없다) 은 실재한다. 결함은 두개의 다른 기제를 같은 단어로 부르는 데서 생기며, 그 부분이 ch11 판별식의일이다.

그럼에도 ReasoningBank 은 RL 의 결정적 부품 하나를 진짜로 가지고 있다: **자동 채점기**. 그리고 그 채점기의 정확도가 측정되어 있다 — ground truth 대비 **72.7%**다 [ReasoningBank § 5]. 이 숫자가 § 05.5 의 RLVR 논의와 만나는 자리다. 검증기가 결정론적 프로그램이 아니라 LLM 일 때보상 신호의 신뢰도가 얼마인지에 대한, 이 corpus 에서 유일하게 측정된값이기 때문이다. (그 견고성 실험이 실제 judge 의 편향 대신 대칭 잡음으로 대체되었다는 사실은 ch15 가 다룬다.)

systems 접점: ReasoningBank 의 오프라인 비용은 토큰으로 보고되어 있다. 과제당 judge 2186.3 tokens + memory extraction 1562.1 tokens = **3748.4 tokens** 이고, wake 의 action generation 은 49306.1 tokens 다 [ReasoningBank Table 5, § C.2]. 즉 “채점 + 요약” 이 과제당 총 토큰의 7% 남짓이다 ($3748.4/53054.5 = 7.07\%$. 이 나눗셈은 이 책의 산술이며, 논문은 비율을 이 형태로 신지 않는다). SEAL 의 self-edit 하나당 30 – 45 초짜리채점과 자릿수가 다르다 — 한 쪽은 추론 호출 두 번, 다른 쪽은 finetune 한번이다. **보상을 어떻게 만드느냐가 sleep 예산 B_s 를 자릿수단위로 가른다**는 것이 이 절의 결론이며, ch25 의 통일 비용 회계가 이자리에서 시작한다.

(state, update, cost) 정리

이 장의 개념들은 전부 Θ 층에서 벌어지는 일이거나, 그 층을 움직이기 위한 보조 장치다. RL 의 부품 중 어느 것도 W 나 E 에 놓이지 않는다 — § 05.7.2 에서 본 (U-E) 는 policy 갱신이 있어야 할 자리에 대신들여선 것이지 RL 의 부품이 아니다. 그것이 RL 을 세 층 프레임에 놓을 때의 첫 사실이다.

표 5-7 — 이 장의 개념을 세 층 프레임에 배치

개념	state (어느 층인가)	update (어느 규칙인가)	cost (어느 예산인가)
policy π_{Θ}	Θ — policy 는 모델 자신	(U- Θ). gradient 는 식 (5-1)	rollout M 개 decode + backward
rollout z	어느 층도 아님 — 소모품	저장되지 않고 버려짐	B_s 의 대부분. 길이에 선형
reward model R_{ψ}	별도 모델의 Θ	자체 (U- Θ), 선호 데이터 위 1 회	학습 1 회 + rollout 마다 forward 1 회
advantage $\hat{A}$	상태 아님 — 추정량의 계수	—	baseline 방식이 value network 유무를 정함

개념	state (어느 층인가)	update (어느 규칙인가)	cost (어느 예산인가)
RLHF 파이프라인	$\Theta (+ \psi)$	(U- Θ) 3 단계	넷의모델 상주. HBM 이 병목
DPO	Θ	(U- Θ) 1 회, sampling 없음	SFT 와 같은 모양. sampler· 채점기 부재
RLVR	Θ	(U- Θ)	rollout(GPU) + 검증 (GPU 밖)
reward hacking	상태가 아니라 실패 모드	—	B_s 를 라운드마다 키움 → (A) 의 분자 증가

Rosetta 로 옮기면 한 줄이다. 독자의 세계에서 **RL 후처리 한라운드**는 “**offline batch inference job** 이 **training job** 을 먹여 살리는파이프라인” 이고, 세 층 프레임에서는 (U- Θ) 한 번, 즉 배포 아티팩트가 바뀌는 사건이다 (→ ch01 § 01.6 표 1-4 의 “모델재배포 ↔ (U- Θ) 1 회” 행).

Worked micro-example — 한 위치의 확률이 어디로가는가

숫자 네 개와 확률 하나로 이 장 전체를 손으로 확인한다. 계산기 없이따라갈 수 있다. 이 예제의 설정값과 그로부터 나오는 표 5-8 의수치는 전부 이 책이 만든 예시 계산이며, 어느 논문의 보고값도 아니다. 4 단계에서만 SEAL 의 보고 수치를 끌어 쓰고, 그 자리에 출처를 병기한다.

설정. prompt q 하나에 rollout $M = 4$ 개를 뽑았다. 채점기는검증 프로그램이고 보상은 이진이다. 결과는 $R = (1, 0, 1, 0)$. 그리고 어떤 한 decode 위치에서 두 후보 token 만 살아 있다고 하자 $-p_A = 0.25, p_B = 0.75$. rollout 이 그 위치에서 A 를 뽑았다. sleep learning rate 는 $\eta_\Theta = 0.1$.

1 단계 — 두 알고리즘의 계수를 만든다.

group-relative 방식은 같은 prompt 의 표본 평균을 baseline 으로 쓴다. $\bar{R} = 0.5$ 이고 네 값의 표준편차 ($1/M$ 정의) 는 $\sqrt{\frac{1}{4} \cdot 4 \cdot 0.25} = 0.5$ 이므로

$$\hat{A}_i = \frac{R_i - \bar{R}}{0.5} = (+1, -1, +1, -1)$$

ReST^{EM} (SEAL 의 선택) 은 baseline 을 빼지 않고 식 (5-4) 처럼 R_{ij} 를 그대로 쓴다. 계수는 (1, 0, 1, 0) 이다. 음수가없다.

2 단계 — 한 위치의 logit 을 옮긴다.

두 후보만 있을 때 $\log p_A$ 의 logit 에 대한 gradient 는 $\partial \log p_A / \partial \text{logit}_A = 1 - p_A = 0.75$, $\partial \log p_A / \partial \text{logit}_B = -p_B = -0.75$ 이다. 계수 w 로 한 step 올라가면 두 logit 의 차가

$$\Delta(\text{logit}_A - \text{logit}_B) = \eta_\Theta w [(1 - p_A) + p_B] = 0.1 \cdot w \cdot 1.5 = 0.15 w$$

만큼 움직인다. 초기 log-odds 는 $\ln(0.25/0.75) = -1.0986$ 이므로 결과는표 5-8 이다.

표 5-8 — 같은 rollout, 같은 위치, 다른 알고리즘

그 rollout 이 받은 보상	group- relative 계수	ReST^{EM} 계수	새 log-odds (group)	새 p_A (group)	새 p_A (ReST^{EM})
$R = 1$	+1	1	-0.9486	0.279	0.279
$R = 0$	-1	0	-1.2486	0.223	0.250 (불변)

두 알고리즘은 성공한 rollout 에 대해 정확히 같은 일을 하고, 실패한 rollout 에 대해 완전히 다른 일을 한다. ReST^{EM} 은 실패를**별하지 않고 버린다**. 이것이 § 05.7.1 에서 말한 “확률을 내리는 gradient 가 존재하지 않는다” 의 수치적 내용이다.

3 단계 — 퇴화 지점을 확인한다.

같은 prompt 의 rollout 4 개가 전부 $R = 1$ 이면 어떻게 되는가. group-relative 에서는 $\bar{R} = 1$ 이고 편차가 0 이므로 $\hat{A}_i = 0$ — **gradient 가 정확히 0 이고 아무것도 배우지 않는다**. 실무에서 prompt 를 난이도로 걸러 “전부 맞거나 전부 틀리는” 문제를 빼는 이유가 이것이다. 반면 ReST^{EM} 에서는 계수가 (1, 1, 1, 1) 이 되어 네 rollout 모두 확률이 올라간다 — 즉 자기 샘플 위의 평범한 SFT 로 퇴화한다. 두 방식은 극단에서 서로 다른 방향으로 무너진다.

4 단계 — 이 한 prompt 의 채점 비용을 센다.

SEAL 의 채점기로 이 예제를 돌리면 self-edit 하나 평가에 30 – 45 초이므로 [SEAL § 5, p.9], $M = 4$ 에 대해 2 – 3 분이 gradient 를 한 번도 계산하기 전에 소비된다 (이 곱셈은 이 책의 예시 계산이다). 그중 절반 ($R = 0$ 인 둘) 은 식 (5-4) 에서 계수 0 을 받아 갱신에 전혀 기여하지 않는다 — **채점 비용의 50%가 버려진다**. 논문이 보고한 라운드비용과 대조해 보면 자릿수가 맞는다: 750 회 실행에 약 6 시간이면 $21600/750 = 28.8$ 초/회로, 명시된 30 – 45 초와 정합적이다 (이 나눗셈은 이 책의 산술이며, 논문은 초당 값을 따로 실지 않는다) [SEAL § B.5, p.21].

이 4 단계가 이 장의 요약이다. 보상이 확률을 얼마나 옮기는지는 η_Θ 와 계수가 정하고, 그 계수를 어떻게 만드느냐가 알고리즘 이름이며, 계수를 만들기 위해 채점기를 몇 번 부르느냐가 B_s 다.

요약

- LLM RL 에서 state 는 KV cache 가 담고 있는 token 열, action 은 다음 token, reward 는 생성이 끝난 뒤 밖에서 도착하는 스칼라 하나다. 전이는 결정론적 concat 이고 관행상 discount 는 쓰지 않는다.
- policy gradient 는 보상을 미분하지 않는다. 미분되는 것은 $\log \pi_\Theta$ 뿐이며, 그래서 RL 은 **미분 불가능한 채점기를 loss 로 바꾸는 장치**다.
- advantage 는 보상에서 baseline 을 뺀 값이고, baseline 을 만드는 방식 (value network 나 같은 prompt 의 표본평균이나) 이 RL 단계의 HBM 예산을 정한다.
- RLHF 파이프라인은 SFT → reward model → RL 의 3 단계이며, RL 단계에서 policy·reference·reward model·value network 넷이 동시에 상주한다. 이후 알고리즘계보는 그 넷을 줄이는 역사다.
- DPO 는 KL 정규화 최적해의 닫힌 형태를 이용해 reward model 과 sampler 를 함께 지운다. 워크로드 모양이 SFT 와 같아 sleep 라운드에 엮기가 구조적으로 단순하며, 대가는 off-policy 라는 것이다.
- RLVR 는 보상을 결정론적 프로그램에 맡겨 사람 없이 임의 횃수의 보상을 만든다. sleep 라운드가 사람의 처리량에 묶이지 않으려면 이 성질이 필요하다 — 이 연결이 실제로 성립하는지의 판정은 ch20·ch21·ch27 이 한다.
- reward hacking 은 버그가 아니라 대리 최적화의 구조적 귀결이며, 길이인플레이션이 가장 흔한 형태다. RL 이득 보고에는 보상과 상관된 표면변량을 prompt 만으로 재현해 보는 통제군이 있어야 한다.
- SEAL 의 self-edit 루프는 ReST^{EM} (rejection sampling + SFT) 이고 PPO·GRPO 가 아니며, 실패한 rollout 을 벌하는 gradient 가 없다. ReasoningBank 은 policy 표기와 rollout 과 자동 채점기를 모두 갖추고도 $\eta_\Theta \nabla_\Theta$ 를 한 번도 실행하지 않는다.

자가 점검체크리스트

- 임의의 LLM 학습 루프를 보고 state·action·reward 를 지목하고, 그 reward 가 terminal 인지 dense 인지 판정할 수 있다.
- 식 (5-1) 에서 보상이 미분되지 않는다는 점을 지적하고, 그것이 왜 SFT 대신 RL 을 쓰는 이유가 되는지 설명할 수 있다.
- $M = 4$, 보상 (1, 0, 1, 0), $p_A = 0.25$, $\eta_\Theta = 0.1$ 에서 group-relative 와 ReST^{EM} 가 p_A 를 각각 어디로 옮기는지 손으로 계산하고, 전부 정답일 때 두 방식이 어떻게 다르게 퇴화하는지 말할 수 있다.

- RLHF·DPO·RLVR 를 “무엇이 채점하는가” 와 “어느하드웨어를 먹는가” 의 두 축으로 구분하고, 각각을 sleep 라운드에 었었을때의 비용 모양을 말할 수 있다.
- 어떤 논문의 RL 이득 주장을 보고, reward hacking 을 배제하는 통제군이있는지 없는지 판정할 수 있다.
- “policy” · “self-evolving” 이라고 쓰인 논문에서 $\eta_{\theta} \nabla_{\theta}$ 가 실제로 실행되는지를 확인해 진짜 RL 과어휘 차용을 구분할 수 있다.
- (Rosetta) RL 후처리 한 라운드를 독자의 어휘로 “offline batch inference job 이 training job 을 먹여 살리는 파이프라인” 이자, 세 층 프레임에서 (U- θ) 1 회 = **배포 아티팩트 교체**로 옮겨 말할 수 있다.

다음 장으로

이 장은 보상이 이미 있다고 가정하고, 그것으로 θ 를 옮기는 법을다뤘다. 그런데 식 (U- θ) 에는 아직 채워지지 않은 자리가 하나 남아있다 — $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 의 gen 이다. SEAL 에서 policy 의 action 이 곧 다음 gradient step 이먹을 학습 데이터였다는 사실은 이 장에서 이미 보았지만, **모델 이자기 학습 데이터를 만들 때 무엇이 좋아지고 무엇이 무너지는가**는 이 장의도구로는 답할 수 없다. reward 는 만들어진 데이터를 채점할 뿐, 그 데이터가몇 라운드까지 자기 자신을 먹여도 되는지는 말해주지 않는다.

ch06 이 그 자리를 맡는다. augmentation 에서 자기생성 코퍼스까지 $\mathcal{R}$ 을 만드는 방법들을 정리하고, 자기 출력을 반복해서 먹었을때의 붕괴가 (U- θ) 를 반복하는 설계에 어떤 상한을 거는지 — 즉 ρ 의 한 원천을 — 다룬다. self-edit 이라는 산출물의 형식과 그 실증은 ch20 이 소유한다.

ch06. 데이터를 만드는 법 — augmentation, synthetic, 그리고 붕괴

이 장의 목표 — 이 장을 마치면 다음을 할 수 있어야 한다. 1. 임의의 sleep 라운드 학습 집합 $\mathcal{R}_k$ 를 augmentation · synthetic data · self-generated data 로분해하고, 생성기가 누구인지로 셋을 판별한다. 2. $\mathcal{R}_k$ 의 구성 스케줄을 replace / accumulate / replace-multiple 로 분류하고, 각 스케줄의라운드당 열화율 ρ_k 를 닫힌 형식으로 적는다. 3. “model collapse 가 일어난다” 는 진술을 네 현상 중 어느 것인지 지정하지 않고는 쓰지 않는다. 4. 자기 생성 $\mathcal{R}_k$ 로 (U- Θ) 를 반복할 때 무엇이 유계로 남고 무엇이 무너지는지 분리하고, 그 경계가 어느 전제 위에서 있는지 말한다.

왜 필요한가 — Part II 의 Θ -경로 전체가 이 장을 요구한다. SEAL(ch20) 은 모델이 자기 자신이 쓴 self-edit 을 학습 집합으로 삼아 (U- Θ) 를 돌리고 [SEAL § 3.1], Language Models Need Sleep(ch21) 의 Dreaming 은 자기 생성 샘플을 sleep 라운드의 $\mathcal{R}_k$ 로 쓴다. 두 논문 중 어느 쪽도 그루프를 K 번 반복하면 Θ 가 어디로 가는가에 대한 경계를 제시하지 않는다. corpus 에서 그 경계를 닫힌 형식으로 가진 논문은 한편뿐이고, 그 논문은 Θ -경로의 어느 논문도 인용하지 않으며 Θ -경로의 어느 논문도 그것을 인용하지 않는다. 이 장이 그 다리를 놓는다.

NM 과의 관계 — backward pass, optimizer state, inner/outer loop, test-time learning 은 전제한다 ($\rightarrow$ NM ch02). Neural Memory 에는 이 장에 대응하는 장이 없다. NM 은 W 층만 다뤘고, W 의 학습 데이터는 test-time 토큰 스트림 그 자체였다 — 만들 필요가 없었다. Θ 층으로 올라오는 순간 “무엇으로 학습하는가” 가처음으로 설계 변수가 된다. 이 장이 더하는 것이 정확히 그 변수다.

06.1 $\mathcal{R}_k$ 는 어디서 오는가

식 (U- Θ) 에는 이 책이 아직 열어보지 않은 상자가 하나 있다. $\text{gen}(\mathcal{H}_k; B_s)$ 다.

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k), \quad \mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$$

이 식이 말하는 것은 sleep 라운드가 두 부분으로 나뉜다는 사실이다. 오른쪽절반 (gen) 은 학습 집합을 만드는 절차이고, 왼쪽절반은 그것으로 Θ 를 움직이는 절차다. ch03(PEFT) 과 ch05(RL) 는 왼쪽 절반을 다뤘다. 이 장은 오른쪽 절반만 다룬다.

gen 의 산출물은 세 가지 출처를 갖는다. 이 장은 그 셋을 여기서 정의하고, 이후 모든 장은 참조만 한다.

data augmentation 은 이미 가진 데이터 항목에 라벨을 보존하는 변환을 적용해 항목 수를 늘리는 절차다. 원 항목이 반드시 존재하고, 변환이 라벨을 바꾸지 않는다는 것이 정의의 두 조건이다.

synthetic data 는 사람이 쓰지 않고 모델이 생성한 학습데이터다. 원 항목이 없어도 된다.

self-generated data 는 synthetic data 중에서 **생성기가 지금 학습받는 모델 자신**인 것이다. 즉 $\text{gen}(\cdot; B_s) = \text{LM}(\cdot; \Theta_{k-1})$ 인 경우다.

세 정의는 포함 관계가 아니라 축이 다르다. augmentation 은 “원 항목이 있는가” 를 묻고, synthetic 은 “사람이 썼는가” 를 묻고, self-generated 는 “생성기가 누구인가” 를 묻는다. 실무의 파이프라인은 대개 셋 중 둘 이상에 동시에 걸친다.

표 6-1. $\mathcal{R}_k$ 의 출처와 그 결과

출처	생성기	원 항목 필요	루프가 닫히는가	B_s 가 붙는 곳
사람이 쓴 데이터	사람	—	아니오	없음 (수집 비용은 계산 예산이 아니다)
data augmentation (고전형)	결정론적 변환	예	아니오	CPU dataloader
synthetic data (타 모델)	다른 모델 Θ^{ext}	아니오	아니오	타 모델의 decode
self-generated data	Θ_{k-1} 자신	아니오	예	자기 모델의 decode

마지막 행만 루프가 닫힌다. Θ_{k-1} 이 만든 데이터로 Θ_k 를 만들고, Θ_k 가 만든 데이터로 Θ_{k+1} 을 만든다. 이 되먹임이 이 장 나머지의 주제이고, Part III 의 반증 축 하나가 여기서 나온다.

이 장의 주 논문은 Gerstgrasser 외, Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data (arXiv:2404.01413) 이다. 이하 [MCI] 로 인용한다.

[해설] systems 독자에게 이 절의 실질은 워크로드 하나가 새로 생긴다는 것이다. gen 이 self-generated 이면 sleep 라운드의 앞절반은 **대규모 batch decode job** 이다 — 가중치가 HBM 에 상주하고, 프롬프트가 스트리밍으로 들어오고, 출력이 스토리지로 나가는, 독자가 매일보는 그 작업이다. 실제로 [MCI] 는 데이터 생성을 vLLM 으로 구현했다고 밝힌다 [MCI App. C]. 그것이 그 논문이 독자의 세계와 닿는 유일한 지점이다.

06.2 LLM 에서 augmentation 은 decode 다

augmentation 의 정의는 vision 에서 왔고, 그 정의의 두 조건 중 하나가 text 에서 깨진다. 깨지는 쪽은 “라벨을 보존하는 변환” 이다.

vision 에서 augmentation 이 쓸모 있는 이유는 라벨을 보존하는 변환군이 **명시적으로 알려져 있기** 때문이다. 고양이 사진을 좌우 반전해도 고양이이고, 밝기를 10% 올려도 고양이이다. 이 불변군은 도메인 지식에서 공짜로 온다. 비용도 싸다 — CPU dataloader 에서 돌고, GPU 는 그동안 backward 를 돈다.

text 에는 그런 불변군이 없다. 동의어 치환은 의미를 바꿀 수 있고, 역번역은 사실을 흘릴 수 있고, 문장 순서 교체는 논증을 파괴할 수 있다. 그래서 LLM 실무의 augmentation 은 결정론적 변환을 포기하고 **모델에게 다시쓰게 하는 것**으로 대체되었다. 그 순간 표 6-1 의 두 번째 행과 세 번째·네 번째 행의 경계가 사라진다. “원 항목이 있는 augmentation” 과 “원 항목을 조건으로 받은 synthetic data” 는 구현상 같은 것이 된다.

SEAL 의 self-edit 이 정확히 그 경계 위에 있다. SQuAD passage 한 건을 주 모델에게 그 passage 의 **함의 (implications)** 목록을 생성하게 한 뒤, 원 passage 가 아니라 그 함의 문장들로 SFT 를 돈다 [SEAL § 3.2]. 원 항목이 있으므로 augmentation 이고, 생성기가 모델이므로 synthetic data 이며, 생성기가 학습 대상 자신이므로 self-generated data 다. 세 이름이 모두 맞다.

[평가] 세 이름이 모두 맞을 때 이 책은 **self-generated** 로 분류한다. 이유는 분류의 목적이 이름 붙이기 때문이 아니라 위험 판정이기 때문이다. § 06.6 이 보일 붕괴 경계는 “생성기가누구인가”에만 걸리고 “원 항목이 있는가”에는 걸리지 않는다. 원 항목이 있다는 사실은 안전을 조금도 보장하지 않는다 — 원 항목은 조건일 뿐, 출력의 분포를 정하는 것은 Θ_{k-1} 이다.

systems 접점은 비용의 이동이다. 고전형 augmentation 의 비용은 CPU dataloader 에 있었고 GPU 시간에 잡히지 않았다. LLM augmentation 의 비용은 **decode** 에 있고, 그래서 B_s 의 대부분을 차지한다. 문서 1 건당 후보 M 개를 뽑는 파이프라인이면 생성 비용은 그대로 M 배다. 이 곱이 § 06.4 에서 다시 나온다.

06.3 자기 생성이 남의 생성을이기는가

답은 “설정에 따라 다르다” 이고, 그 설정 의존성이 이 장이 쓰는가장 중요한 실측 사실이다.

SEAL 은 이 질문을 정면으로 켤다. no-passage-in-context SQuAD v1.1 에서 Qwen2.5-7B 를 놓고, 같은 passage 에 대해 (a) passage 원문으로만 학습, (b) 모델 자신이 생성한 함의로 학습 (RL 없음), (c) GPT-4.1 이 생성한 synthetic data 로 학습, (d) 생성 정책을 RL 로 학습한 뒤의 자기 생성 (SEAL) 을 비교한다. 세 열은 각각 passage 1 건 (LoRA), 200 건 continued pretraining(full-FT), 2067 건 continued pretraining(full-FT) 이다 [SEAL Table 2].

표 6-2. 생성기에 따른 no-context SQuAD 정확도 (%) ([SEAL Table 2] 그대로)

방법	$n=1$ (LoRA)	CPT $n=200$	CPT $n=2067$
Base model	32.7	32.7	29.0
Train on Passage	33.5	36.0	31.2
Passage + 자기 생성 (RL 없음)	39.7	50.6	43.4
Passage + GPT-4.1 synthetic	46.3	59.4	49.2
SEAL(RL 로 학습된 자기 생성)	47.0	58.2	46.4

이 표는 세 줄로 읽힌다. 첫째, 원문을 그대로 학습시키는 것은 거의아무 일도 하지 않는다 — 32.7 에서 33.5 로 0.8 점이다. 데이터를 만드는 절차자체가 일의 대부분을 한다. 둘째, 무필터 자기 생성은 남의 생성에진다 — 39.7 대 46.3. 셋째, 생성 정책을 RL 로 학습하면 47.0 으로 GPT-4.1 을 0.7 점 앞선다.

셋째 줄에는 조건이 붙는다. 그 역전은 세 열 중 한 열에서만성립한다. CPT 두 열에서는 GPT-4.1 이 각각 59.4 대 58.2, 49.2 대 46.4 로이긴다. 논문 자신이 이 사실을 쓴다 [SEAL § 4.2]. 초록과 서론은 GPT-4.1 비교를 범위 조건 없이 앞세운다 [SEAL Abstract, § 1]. 이 caveat 의 의무 장은 ch20 이지만, 수치를 여기서 쓰는 이상 여기서도병기한다.

같은 논문의 부록에는 더 불편한 행이 둘 있다. 첫째, self-edit 프롬프트를 7 가지로 바꿔 본 실험에서 RL 을 전혀 돌리지 않은 implications-long(49.3) 과 rewrite(49.4) 가 RL 로 학습된본문 헤드라인 47.0 을 이미 넘는다 [SEAL Table 10]. 프롬프트한 줄이 RL 루프 전체보다 크다. 둘째, 학습된 생성 정책 대신구조화된 휴리스틱으로 합성 데이터를 만드는 Entigraph 가 가장 큰규모에서 SEAL 을 앞선다 — CPT $n=2067$ 에서 48.6 대 46.4 다 (문서당 생성 10 건 대 5 건이라 계산량이 맞춰져 있지 않고, 논문이 그 사실을 밝힌다) [SEAL Table 8, § B.9]. 규모가 커질수록 휴리스틱이 따라잡는 방향의추세다.

[평가] 표 6-2 에서 이 장이 가져갈 결론은 “자기 생성이났다”가 아니라 “**형식이 생성기보다 먼저다**”이다. 33.5 → 39.7 의 6.2 점은 데이터의 형식을 원문에서 함의 목록으로 바뀌서 얻었고, 39.7 → 46.3 의 6.6 점은 생성기 품질로 얻었으며, 46.3 → 47.0 의 0.7 점은 생성 정책 학습으로 얻었다. 세 항의 크기 순서가 이 장의실천 지침이다. B_s 를 어디에 쓸지 정할 때 마지막 항부터 손대는 것은순서가 틀렸다.

06.4 데이터 품질 필터링

필터는 gen 의 절반이다. 생성만으로 $\mathcal{R}_k$ 가 정해진다고 보는 것은 실제 파이프라인을 잘못 그린 것이다.

데이터 품질 필터링은 생성된 후보 집합에서 $\mathcal{R}_k$ 에 실제로 넣을 부분집합을 고르는 절차다. 형식적으로 gen 을 두단계로 쪼갬다: 후보 생성 $\tilde{\mathcal{R}}_k \sim \text{LM}(\cdot|c; \Theta_{k-1})$, 채택 $\mathcal{R}_k = \{z \in \tilde{\mathcal{R}}_k : \phi(z) = 1\}$. 판정 함수 ϕ 가 필터다. 후보 하나를 가리키는 표본 기호는 z 다 — s 는 SGD step 첨자로 예약되어 있다.

ϕ 는 세 종류로 나뉜다.

1. 규칙 · 휴리스틱 필터. 중복 제거, 길이 범위, 형식검사, n-gram 오염 검사. 비용이 거의 0 이고 판정이 결정론적이다.
2. 검증기 기반 필터. 과제 점수, 실행 결과, 단위 테스트, 보상모델. 판정이 정확하지만 비용이 후보마다 붙는다. 보상을 최적화하는 알고리즘 자체는 ch05 가 소유한다 — 이 장은 그 보상을 채택판정으로 쓰는 것만 다룬다.
3. 사람 필터. 가장 정확하고 가장 비싸며, 유희 시간에 상각되지 않는다.

SEAL 의 ϕ 는 2 번의 극단이다. 후보 self-edit 으로 실제로 Θ 를 갱신하고, 갱신된 모델로 downstream 문항을 풀어, 점수가 오르면 채택한다 [SEAL Eq. 2]. 헤드라인 실험에서는 이보다 더 좁아서, 후보 M 개 중 가장 크게 개선한 하나만 채택한다 [SEAL footnote 2]. 즉 ϕ 가 후보당 학습 1 회 + 평가 1 회를 요구한다.

그 ϕ 의 값이 얼마인지도 같은 논문이 켜다. 학습-후-평가 보상을 GPT-4.1 rubric 채점으로 바꾸면 정확도는 47.0 에서 45.6 으로 1.4 점떨어지고, RL 학습 시간은 약 6 시간에서 약 5 분으로 줄어든다 [SEAL Table 9]. 1.4 점에 약 72 배다.

[평가] 이 ablation 은 필터의 비용-품질 곡선이 매우 평평하다는 뜻이다. 가장 비싼 ϕ 가 가장 싼 ϕ 보다 1.4 점 낫다면, B_s 가 제약인 시스템에서 비싼 ϕ 를 고를 근거는 논문안에 없다. 논문 자신도 proxy 가 “true” 보상을 넘어설 수도 있다고 쓴다 [SEAL § B.10].

필터가 못 하는 것도 같은 표에 있다. 프롬프트를 주지 않고 모델이 스스로 형식을 정하게 한 No-Prompt 행은 13.8 에서 출발해 RL 2 라운드뒤에도 18.9 다 [SEAL Table 10]. 소스가 나쁘면 필터가 구제하지 못한다. 필터는 분포를 자르는 연산이지 옮기는 연산이 아니다.

마지막으로, 다음 두 절이 다룰 붕괴 경계는 필터가 없는 경우에만 증명되어 있다. [MCI] 의 gen 은 온도 0.3 또는 1.0 의 무조건부 샘플링이고, 검증 · 보상 · 중복제거 · 사람 검토가 전혀 없다 [MCI § 2.1]. 필터링은 논문이 future work 로 명시한다 [MCI § 4]. 이 사실은 양쪽으로 작용한다 — 경계가 최악의 gen 을 덮는다는 뜻인 동시에, 필터가 있는 파이프라인 (즉 배포된 모든 self-improvement 파이프라인) 에 그 경계를 그대로 적용할 수 없다는 뜻이다.

systems 접점은 B_s 의 곱셈이다. 후보 M 개를 뽑아 1 개를 채택하면 생성 예산은 M 배이고, ϕ 가 검증기 기반이면 여기에 후보당 판정비용이 더해진다. SEAL 식 ϕ 에서는 그 판정 비용이 SFT 한 번 + 평가한 번이므로, $B_s \approx M \times (\text{decode} + \text{SFT} + \text{eval})$ 이된다. 상각식 (A) 의 분자에 들어가는 것은 이 값이지 decode 항 하나가 아니다.

06.5 model collapse — 이름 하나에 현상넷

model collapse 는 하나의 현상이 아니다. 이 절의 결론은 정의하나가 아니라 “어느 뜻인지 지정하라” 는 규율이다.

model collapse는 모델이 자기 게보가 생성한 데이터로 반복 학습될때 성능이 라운드가 진행될수록 악화되는 현상이다. [MCI]의 조작적정의는 이보다 좁고 날카롭다 — “model collapse란 model-data 루프의 반복이늘어남에 따라 오차가 뚜렷하게 악화되는 것을 가리키고, model collapse를피한다는 것은 그 반복에 대해 오차가 **유계로 남는 것을 가리킨다**” [MCI § 2]. 이론 절에서는 더 좁혀서 “test error가 (어떤 속도로든) 무한대로 발산하는 상황”과 동일시한다 [MCI 각주 3].

정의가 “유계인가 발산인가”라는 점이 중요하다. 이 정의 아래에서는 오차가늘어나도 수렴하면 붕괴가 아니고, 아무리 천천히 늘어도 발산하면 붕괴다. § 06.6의 결과 전체가 이 이분법 위에 있다. [MCI] 자신이 이 이름의 남용을 경고하며 네 현상을 분리한다 [MCI § 4 Discussion].
표 6-3. model collapse라는 이름이 가리키는 네 현상 ([MCI § 4] 분류)

#	현상	무엇이 관측되는가
(0)	test error의 무계 폭증	held-out loss가 라운드마다 오르고수렴하지 않는다
(1)	modal collapse	생성 분포가 한두 개 mode로 수축한다
(2)	uniformity로의 붕괴	생성 분포가 특징 없이 평평해진다
(3)	artifact 증폭	앞선 합성 데이터가 남긴 결함이 라운드마다커진다

[해설] 이 네 항목은 서로를 함의하지 않는다. loss가 유계로남으면서 mode가 사라질 수 있고, 그 반대도 가능하다. § 06.7에서 보겠지만 [MCI]의 VAE 실험이 정확히 그 경우다 — (0)은 완화되는데 (1)은 그대로일어난다. 따라서 이 책은 “붕괴한다/안 한다”를 번호 없이쓰지 않는다. Part II와 Part III에서 Θ -경로 논문의 주장을 검증할때에도 같은 규율을 적용한다.

systems 접점은 모니터링이다. 프로덕션에서 관측 가능한 것은 대개 (0) 뿐이다 — held-out perplexity는 대시보드에 있지만 생성 분포의 mode 수는 없다. 그래서 (1)과 (3)은 **알람이 울리지 않는 붕괴**다. § 06.9에서 이 함정으로 돌아온다.

06.6 accumulate vs replace — 스케줄이 수렴을정한다

루프의 안정성은 학습 규칙이 아니라 $\mathcal{R}_k$ 를 구성하는스케줄이 정한다. 이것이 [MCI]의 결과이고, 이 장이 Part III에 넘기는핵심 명제다.

replace는 라운드마다 이전 학습 집합을 버리고 새로 생성한 것으로바꾸는 스케줄이다: $\mathcal{R}_k = \text{gen}(\mathcal{H}_{k-1}; B_s)$, $|\mathcal{R}_k| = T$. **accumulate**는 이전 학습 집합을 버리지않고 새 생성분을 덧붙이는 스케줄이다: $\mathcal{R}_k = \mathcal{R}_{k-1} \cup \text{gen}(\mathcal{H}_{k-1}; B_s)$, $|\mathcal{R}_k| = kT$. 라운드 1의 실데이터 $\mathcal{R}_1$ 은 영원히남는다. (T 는 라운드당 추가되는 표본 수다. 이 장에서 계속 그 뜻으로쓴다.)

세 번째 스케줄이 하나 더 있다. **replace-multiple**은 실데이터를버리되 합성 코퍼스를 kT 까지 키우는 스케줄로, 데이터 양을 accumulate와 맞춘 통제군이다 [MCI App. E].

[평가] 여기서 이 책의 프레임을 한 번 정정한다. **ch01 § 01.3이 세운 (U- Θ)의 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 는 글자 그대로 replace다**. 표준형을 그대로 반복하는 시스템은 [MCI]가 발산한다고증명한 바로 그 레짐에 있다. 표준형은 스케줄을 지정하지 않았고, 지정하지않은 채로 읽으면 replace로 읽힌다. 따라서 스케줄은 (U- Θ)의세부가 아니라 η_Θ 와 나란한 1급 파라미터이며, 이 책은 이후 Θ -경로 장에서 그렇게 다룬다.

[MCI]의 이론은 잘 지정된 ridgeless 선형 회귀에서 전개된다. 입력 차원 d_{in} , 참 파라미터 Θ^* , 라벨 잡음 분산 σ^2 . 라운드마다 이전 모델로 타깃을 합성할 때 새 잡음 $\mathcal{E}_i \sim \mathcal{N}(0, \sigma^2 I_T)$ 가 주입된다. accumulate 에서 파라미터의 정확한 재귀는 다음과 같다 [MCI Thm. 1, Eq. 2].

$$\Theta_k = \Theta^* + (X^\top X)^{-1} X^\top \left(\sum_{i=1}^k \frac{\mathcal{E}_i}{i} \right) \quad (6-1)$$

이 식이 결과 전체를 이미 말한다. **라운드 i 에 주입된 잡음은 계수 $1/i$ 로 파라미터에 도달한다** — 그것이 코퍼스에 들어간 시점에 i 개 블록 중 하나였기 때문이다. 오차는 제곱이므로 기여는 $1/i^2$ 가 되고, $\sum 1/i^2$ 가 수렴하므로 총합이 유계로 남는다 [MCI § 3.2]. 결론은 다음 세 줄이다.

$$\mathcal{L}_{\text{test}}^{\text{acc}}(\Theta_k) = \frac{\sigma^2 d_{\text{in}}}{T - d_{\text{in}} - 1} \sum_{i=1}^k \frac{1}{i^2} \leq \frac{\sigma^2 d_{\text{in}}}{T - d_{\text{in}} - 1} \cdot \frac{\pi^2}{6} \quad (6-2)$$

$$\mathcal{L}_{\text{test}}^{\text{rep}}(\Theta_k) = \frac{\sigma^2 d_{\text{in}}}{T - d_{\text{in}} - 1} \times k \quad \mathcal{L}_{\text{test}}^{\text{rep-mult}}(\Theta_k) = \frac{\sigma^2 d_{\text{in}}}{T - d_{\text{in}} - 1} \sum_{i=1}^k \frac{1}{i} \approx \frac{\sigma^2 d_{\text{in}}}{T - d_{\text{in}} - 1} \log k \quad (6-3)$$

(6-2)는 MCI Eq. 3, (6-3)의 왼쪽은 MCI Eq. 4, 오른쪽은 [MCI Eq. 11]이다. 성립 조건은 $T \geq d_{\text{in}} + 2$, 등방 공분산 $\Sigma = I_{d_{\text{in}}}$, ridge 계수 0, X 열 full rank 다 [MCI Thm. 2].

$\pi^2/6 \approx 1.6449$ 다. **accumulate 루프를 영원히 돌린 총열화가 한 라운드 별점의 1.645 배를 넘지 않는다.** 이 책의 프레임이 각 논문에 요구하는 ρ 칸을, corpus 에서 유일하게 측정치가 아니라 공식으로 채우는 것이 이 결과다. 세 닫힌 형식을 1 차 차분하면 (이책의 산술이며 추가 가정은 없다) 라운드당 열화율이 나온다.

$$\rho_k = \frac{\sigma^2 d_{\text{in}}}{T - d_{\text{in}} - 1} \cdot \begin{cases} 1 & \text{replace} \\ 1/k & \text{replace-multiple} \\ 1/k^2 & \text{accumulate} \end{cases} \quad (6-4)$$

설계 규칙은 여기서 바로 읽힌다. 반복되는 (U- Θ) 루프는 $\sum_k \rho_k$ 가 수렴할 때에만 지속 가능하다. [MCI]의 정의상 $\log k$ 발산도 붕괴로 센다 [MCI 각주 3] — replace-multiple 은 데이터를 accumulate 만큼 늘리고도 붕괴한다. **데이터를 더 만드는것은 기제가 아니다. 실데이터를 계속 보유하는 것이 기제다.**

실측은 이론과 같은 방향이다. TinyStories(470M tokens) 에서 라운드마다 새모델을 초기화 해 1 epoch 씩 학습하고 [MCI § 2.1], GPT-2 9M 의 held-out cross-entropy 를 재면 라운드 1 에서 1.82 이고, 라운드 4 에서 accumulate 1.74(− 0.07), replace 2.39(+0.58), 라운드 10 replace 2.91(+1.09) 이다. 데이터 양을 맞춘 replace-multiple 도 라운드 4 에서 2.18(+0.36) 로악화된다 [MCI Table 2]. 가장 큰 모델 (본문은 125M, Table 2 와 Fig. 2 범례는 126M 으로 표기한다 — 논문 내부가 1M 어긋나 있어 양쪽을 그대로 옮긴다)에서는 1.71 $\rightarrow$ accumulate 1.59(− 0.12), replace 2.23(+0.53) 이다 [MCI Table 2].

accumulate 열의 부호에는 단서가 붙는다. **표의 어느 행도 계산량이 맞춰져 있지 않다.** “1 epoch” 은 accumulate 에서 kT 개 표본에 대한 1 epoch 이므로 gradient step 수가 replace 보다 많고, 논문 자신이 그림캡션에서 그 사실을 밝힌다 [MCI Fig. 3 캡션 주석]. 따라서 1.82 $\rightarrow$ 1.74 처럼 accumulate 가 **좋아지는** 방향의 delta 일부는 코퍼스가 커지고 최적화가 늘어난 결과이며, 정규화에 쓸 계산량 수치는 논문 어디에도 없다. 이 장이 accumulate 에서 가져가는것은 개선이 아니라 **유계성**이다.

디코딩 온도가 이 그림을 크게 흔든다. 같은 GPT-2 9M 을 온도 0.3 으로 생성하면 replace 는 라운드 4 에서 5.82(+4.00), 라운드 10 에서 9.85(+8.04) 로 무너지는 반면 accumulate 는 1.75(− 0.06) 로 사실상 불변이다 [MCI Table 2]. 낮은 온도가 더 나쁘다. 그리고 온도 0(greedy) — 프로덕션 기본값 — 은 검증되지 않았고, 논문이 future work 로 명시한다 [MCI § 4].

systems 접점은 비용의 비대칭이다. accumulate 에서 생성은 라운드마다 T 로 일정하지만 학습은 $|\mathcal{R}_k| = kT$ 로 커진다. K 라운드 총 학습량은 $\Theta(K^2 T)$ 이고 replace 는 $\Theta(KT)$ 다. 논문 자

신의 5 라운드 스왑을 이 책의 산술로 환산하면 accumulate 는 $470M \times (1+2+3+4+5) = 7.05B$ training tokens, replace 는 $470M \times 5 = 2.35B$ 이며 생성량은 양쪽 모두 $470M \times 4 = 1.88B$ 로 같다. 논문은 이 세합계 중 어느 것도 쓰지 않는다 — 비대칭을 말로만 두 번 언급한다 [MCI Fig. 3 캡션 주석; MCI App. E].

06.7 헤드라인이 성립하지 않는자리

[MCI]의 헤드라인은 세 모달리티 중 하나에서 성립하지 않는다. 이 책은 자기 논지에 유리한 논문에도 같은 자를 댈다.

CelebA 위의 VAE 실험에서 논문은 이렇게 쓴다: “언어 모델링과 달리, 데이터를 accumulate 할 때에도 test error 는 반복 횟수에 따라증가한다 (데이터를 replace 할 때보다 훨씬 느리기는 하지만)” [MCI § 2.3]. 논문 자신의 정의 — 붕괴를 피한다는 것은 오차가 유계로남는 것 [MCI § 2] — 을 그대로 적용하면, 이 실험은 붕괴 회피를 보이지않았다. 그럼에도 해당 그림의 캡션은 “Data Accumulation Avoids Model Collapse in Variational Autoencoders for Image Generation” 이고 [MCI Fig. 5], 초록은 “다른 종류의 실데이터 위의 심층 생성 모델에서도 유사한결과를 얻었다” 고 쓴다 [MCI Abstract]. 캡션과 본문 측정이 어긋난다.

같은 실험에 두 번째 사실이 붙는다. accumulate 아래에서도 생성 다양성은떨어진다 — “성별처럼 데이터셋의 주요 변동 축은 여전히 표현하지만, 안경이나 액세서리 같은 데이터 다양체의 부차적 축의 세부는 더 이상생성하지 못하는 것으로 보인다” [MCI § 2.3]. 부록은 반대 방향의 사실을하나 더 준다: 같은 모델의 재구성은 안경과 모자를 유지한다 [MCI App. D]. 즉 encoder-decoder 능력은 살아남고 샘플링분포의 꼬리만 죽는다.

[평가] 표 6-3의 언어로 옮기면 이것이 정확히 (0) 은 완화되고 (1) 은 일어나는 경우다. 그리고 sleep 라운드를 반복하는 시스템에게는이쪽이 더 나쁜 이야기다 — gen 은 재구성이나 샘플링을 쓰기 때문이다. 라운드 k 의 샘플 분포에서 사라진꼬리는 라운드 $k+1$ 의 학습 집합에 없고, 그 손실은 자기 증폭한다. held-out loss 는 그동안 완만하게 오를 뿐이다 [MCI § 2.3].

논문이 스스로 인정하는 외부 반례도 있다. “Martínez 외 (2023a) 는 다소상반된 증거를 찾았다 — 훨씬 작은 데이터셋 위의 다른 아키텍처에서는데이터를 accumulate 해도 성능이 빠르게 열화한다” 이고, 왜 이런 불일치가생기는지는 future work 로 남긴다 [MCI § 2.3]. 초록의 일반화 주장 (“여러모델 크기 · 아키텍처 · 하이퍼파라미터에 걸쳐 성립” [MCI Abstract]) 에는 문헌 안에 이름이 붙은 반례가 있고 이 논문은그것을 재현하지도 반박하지도 않는다.

증거의 성격도 한 항목에서 다르다. 부록의 근거 하나는 새 실험이 아니라남의 그림을 다시 읽은 것이다. Alemohammad 외 (2023) 의 Figure 7 을그대로 옮겨 싣고, 원저자들이 “실데이터를 고정해 두는 것은 불가피한열화를 늦출 뿐” 이라고 내린 결론에 대해 “자세히 보면 오차가 낮은 값에서평탄해진다” 고 반박한다 [MCI App. A, Fig. 8]. 이 책은 이것을 증거가아니라 해석 분쟁으로 인용한다 — 저자들이 돌리지 않은 실험의곡선에 대한 판독이기 때문이다.

세 번째 모달리티도 이론과 형태가 다르다. 분자 conformation diffusion 에서 replace 는 “합성 데이터로 학습한 첫 라운드에서 크게 나빠진 뒤 이후라운드에서 실질적으로 더 악화되지 않는다” [MCI § 2.2]. 식 (6-3) 의왼쪽은 k 에 선형인 증가를 예측한다. 한 번 떨어지고평평해지는 곡선은 그 예측이 아니며, 논문은 이 불일치를 논평하지 않는다.

마지막으로 언어 모델 실험의 “실데이터” 는 실데이터가 아니다. TinyStories 를 논문은 “GPT-3.5/4 가 생성한 470M 토큰의 유치원수준 짧은 이야기 데이터셋” 이라고 소개한다 [MCI § 2.1]. 보호받는 분포자체가 모델 출력이고, 엔트로피가 낮으며 구조가 단거리다. 웹 규모의무거운 꼬리 분포에서 $1/i^2$ 논증이 살아남는지는 검증되지 않았다.

06.8 경계의 전제 — 이 결과를 SEAL과 Dreaming에 옮길 수 있는가

옮길 수 없다. 그리고 그 사실이 이 장이 Part III에 넘기는 반증축이다.

식 (6-2)가 성립하는 설정은 다음을 모두 가정한다. (i) 라운드마다 Θ_k 를 처음부터 다시 학습한다 — 이론에서는 닫힌 해 $\Theta_k = \hat{X}_k^\dagger \hat{Y}_k$ 이고, 실험에서는 “새로초기화된 모델을 사전학습한다” [MCI § 2.1, § 3.1]. 그래서 η_Θ 는 이론에 아예 등장하지 않고, optimizer state도 checkpoint 연속성도 catastrophic forgetting도 모델링되지 않는다. (ii) 공변량이 동결되어 있다 — accumulate된 설계 행렬은 같은 X 를 k 번 쌓은 것이고 타깃만 재생성된다 [MCI § 3.1]. 즉 정리가 경계짓는 것은 누적된 라벨 잡음이지 입력 분포의 이동이 아니다. (iii) 모델 클래스가 참 함수를 포함하고 (well-specified), ridge 계수가 0이며, 공분산이 등방이다 [MCI Thm. 2]. test error가 $\mathbb{E}\|\Theta - \Theta^*\|_\Sigma^2$ 로 정의되므로 bias 항이 분석 전체에 없다 [MCI Eq. 1]. (iv) gen 이무필터다 (§ 06.4).

[평가] 네 전제는 Θ -경로의 모든 시스템에서 깨진다. SEAL은 살아 있는 checkpoint를 LoRA로 증분 편집하고 [SEAL § 3.2], 보상으로 후보를 고르며 [SEAL Eq. 2], 입력 (생성된 함의 문장)을 매라운드 새로 만든다. Language Models Need Sleep의 Dreaming도 마찬가지다. (6-2)의 기제 — 라운드 i 의 데이터가 코퍼스의 $1/i$ 이므로 잡음이 $1/i$ 로 들어온다 — 는 이전 데이터가 아니라 이전 파라미터가 운반자일 때 대응물이 없다. 따라서 이 정리는 그시스템들을 보증하지도, 반증하지도 않는다. 이 책이 하지 말아야 할 일은 $\pi^2/6$ 을 안전 증서로 인용하는 것이다.

반대 방향으로도 한 칸을 깎아야 한다. 논문은 자기 설정을 “합성 데이터가 통제 없이 인터넷에 쏟아지는 미래를 상정한, 어떤 의미에서 최대로비관적인” 것이라고 부른다 [MCI § 1]. 그러나 accumulate에서 원실테이터는 규정상 모든 라운드의 코퍼스에 절대량 그대로 남고, 이론에서는 실공변량 X 가 매라운드 문자 그대로 재사용된다 [MCI § 3.1]. 보호 성분을 제거할 수 없게 만들어 둔 설정은 최악의 경우가 아니라 유리한 구성이다. 인터넷에는 2023년 이전 사람 코퍼스가 이후 모든 학습집합에 줄지 않은 크기로 계속 포함된다고 보장하는 주체가 없다. 이 논문을 “인터넷 규모의 안심 근거”로 옮겨 쓰는 것은 이 구성을 지우는 일이다.

측정 자체의 한계도 기록해 둔다. 시드·반복·오차막대·신뢰구간이 전혀 없어 Table 2의 모든 곡선과 항목이 단일 실행이다. downstream 벤치마크도 없어 모든 정량 주장이 held-out loss이고, 정성 증거는 손으로 고른 생성문 여섯 개뿐이다 [MCI Table 1]. FLOPs·GPU-hour·wall-clock·bytes·latency 중 어느 것도 보고되지 않으며, 논문 전체의 유일한 비용 진술은 “계산적으로 비싼 시도”라는 형용사다 [MCI § 2.1]. 그리고 실증 상한은 126M 파라미터 $\times$ 470M 토큰 $\times$ 5 accumulate 라운드인데 [MCI § 2.1], 논문이 서론에서 동기로 든 규모는 Llama 1/2/3의 1.4T / 2T / 15T 토큰이다 [MCI § 1]. 200 라운드의 긴 지평선은 10 차원 선형 장난감에만 존재한다 [MCI App. F].

이 corpus에서 가장 깨끗한 침묵도 여기 있다. [MCI]는 지속학습 문헌을 하나도 인용하지 않는다 — EWC도, GEM도, Deep Generative Replay도, LoRA도, 모델 편집도 없다. 반대 방향도 같다: Θ -경로의 어느 논문도 이 결과를 인용하지 않는다. Language Models Need Sleep은 반복 self-distillation이 “training collapse”를 일으킬 수 있다는 위험을 관련 연구 논의에서 직접 다루면서도, 그 위험을 2026년 on-policy self-distillation 문헌 (He 외, Wang 외)에 귀속시키고 2024년 model collapse 문헌에는 닿지 않는다 (원문 위치는 ch21 이소유한다). 두 공동체가 같은 위험을 발견해 서로 다른 이름을 붙이고 서로를 0번 인용한다. 이 다리는 논문이 아니라 이 책이 놓는다. ch11의 판별식과 ch22의 반증 축, ch30의 반증 조건이 그것을 이어받는다.

06.9 그래서 decode에서 무엇이 바뀌는가

decode 자체는 아무것도 바뀌지 않는다. 그리고 그것이 이 장의 비용이 다른 데 숨어 있다는 뜻이다.

sleep 라운드의 산출물은 아키텍처도 파라미터 수도 그대로인 Θ 다 [MCI § 2.1]. 추가 상태도, KV cache 변화도, 검색 hop 도, 토큰당오버헤드도 없다. L_w 는 정의상 갱신 전후가 같다. 이 칸은 “논문에없음” 이 아니라 **구조적으로 0** 이다. E -경로와 비교했을 때 Θ -경로의 구조적 강점이 정확히 이것이고, 그래서 비용을 다른 데서찾아야 한다.

찾으면 세 곳에 있다.

첫째, sleep 트래픽은 성격이 반대인 두 작업이다. 생성은 Θ_{k-1} 에서 코퍼스를 뽑아내는 batch decode —memory-bandwidth-bound, 가중치 HBM 상주, 출력은 스토리지로 스트리밍 —이고, 학습은 k 에 선형으로 커지는 코퍼스를 순차 읽는 compute-bound 패스다. 어느 쪽도 지연에 민감하지 않고 어느 쪽도 캐시 계층에서 흥미로운일을 하지 않는다. 압력은 스토리지 용량과 순차 읽기 대역폭에 걸린다.

둘째, C_{cap} 이 둘로 쪼개진다. accumulate 아래에서지속되는 상태는 가중치 delta 가 아니라 **보존된 코퍼스**다. 가중치상태는 모델 크기에 고정 ($\Theta(1)$) 이고, 데이터 상태는 라운드 수에선형 ($\Theta(k)$) 이며 버려지지 않는다. 이 책의 산술로 크기를 보면 470M tokens 를 uint16 id 로 저장할 때 라운드당 약 0.94 GB, 5 라운드누적 약 4.7 GB 다 (논문은 어떤 바이트 수치도 보고하지 않는다). 절대값은작지만 **모양**이 요점이다. ch01 § 01.5 가 세운 C_{cap} 칸은 첫 번째만 가정한다. Θ -경로 장들이 두 칸을 모두 요구하는이유가 이 논문이다 ($\rightarrow$ ch09, ch26).

[평가] 셋째, **batching 은 살아남는다 — 단, 이레짐에서만.** Rosetta 의 “batch 로 weight 공유 $\rightarrow$ Θ -경로에서 깨짐” 행은 **per-user** 가중치 delta 에 대한 진술이다. [MCI] 의 루프는전체 인구에 대해 라운드당 하나의 전역 Θ 를 만들므로 shared-weight batching 이 깨지지 않는다. SEAL 식 per-context self-edit 만깨뜨린다. 따라서 이 책은 **인구 수준 Θ -경로와사용자 수준 Θ -경로**를 구분해서 부른다. 이 구분 없이 “ Θ -경로는 batching 을 깬다” 고 쓰면 절반이 틀린다.

마지막으로 모니터링 함정 하나. § 06.7 의 VAE 결과는 **집계 loss 가유계인 채로 샘플링 분포의 꼬리가 사라질 수 있음**을 보인다. held-out perplexity 를 보는 대시보드는 이때 울리지 않는다. sleep 루프의 출력이다음 라운드의 입력이 되는 시스템에서 꼬리 손실은 자기 증폭하므로, 관측지표에 **분포 다양성 측정치**가 없으면 (1) 번 붕괴는 조용히진행된다.

(state, update, cost) 정리

이 장의 개념은 전부 (U- Θ) 의 오른쪽 절반, 즉 $\text{gen}(\mathcal{H}_k; B_s)$ 안에 있다. W 도 E 도 등장하지않는다. 표 6-4. 이 장의 개념을 세 층 프레임에 놓기

개념	state	update	cost
data augmentation	$\mathcal{R}_k$ 의 항목 수를 늘림	(U- Θ) 이전 단계. $\Theta \cdot W \cdot E$ 불변	B_s : 고전형은 CPU, LLM 형은 decode
synthetic data (타 모델)	$\mathcal{R}_k$ 전체를 대체 가능	(U- Θ) 이전 단계. 루프 안 닫힘	B_s 가 타 모델 decode 로외부화
self-generated data	$\mathcal{R}_k = \text{LM}(\cdot; \Theta_{k-1})$	(U- Θ) 가자기 출력을 되먹임	B_s : 자기 decode. ρ 가 여기서 발생
데이터 품질 필터링	$\mathcal{R}_k \subseteq \tilde{\mathcal{R}}_k$	gen 을 생성+채택 2 단계로 분해	$B_s \approx M \times (\text{decode} + \phi \text{ 판정})$
accumulate vs replace	$ \mathcal{R}_k = kT$ 대 T	(U- Θ) 의 1 급 스케줄 파라미터	$\rho_k \propto 1/k^2$ 대 1; 총 학습 $\Theta(K^2T)$ 대 $\Theta(KT)$; C_{cap} 에데이터 꼬리
model collapse	Θ_k 의 열화	(U- Θ) 반복의 극한 거동	ρ 칸 자체. $\sum_k \rho_k$ 수렴여부가 판정

L_w 는 이 장의 어느 개념에 대해서도 구조적으로 0 이다. B_s 와 ρ 가 이 장이 채우는 두 칸이고, C_{cap} 은 accumulate 에서만 움직인다.

Worked micro-example — 1 차원 가우시안으로 세 스케줄을 손으로 따라가기

식 (6-1)~(6-4) 를 암산으로 재현할 수 있는 가장 작은 설정을 만든다. 이 절의 모든 숫자는 이 책의 산술이다.

설정. 진짜 분포는 $p^* = \mathcal{N}(0, 1)$ 이다. 모델은 파라미터가 평균 하나뿐이다: $\Theta_k = \hat{\mu}_k$, 예측 분포는 $\mathcal{N}(\hat{\mu}_k, 1)$. 학습은 학습 집합의 표본평균을 취하는 것이고 (이것이 이 모델의 $\arg \min$ 이다), 생성은 $\mathcal{N}(\hat{\mu}_{k-1}, 1)$ 에서 뽑는 것이다. 라운드당 표본 수는 $T = 100$. 라운드 1 은 실데이터 100 개로 시작한다.

test error 는 [MCI Eq. 1] 과 같은 형태로 잡는다. 모델 클래스가 참분포를 포함하므로 bias 가 없고 순수 분산이다.

$$\mathcal{L}_{\text{test}}(\Theta_k) = \mathbb{E}[(\hat{\mu}_k - 0)^2] = \text{Var}[\hat{\mu}_k]$$

표본 100 개의 평균이 갖는 분산은 $\sigma^2/T = 1/100 = 0.01$ 이다. 이 0.01 이 한 라운드 별점이고, 이후 모든 숫자가 그배수다.

단계 1 — replace. 라운드 k 는 $\hat{\mu}_{k-1}$ 에서 뽑은 100 개만으로 학습한다. 그 100 개의 표본평균은 $\hat{\mu}_{k-1} + \bar{\varepsilon}_k$ 이고 $\text{Var}[\bar{\varepsilon}_k] = 0.01$ 이다. 따라서

$$\hat{\mu}_k = \hat{\mu}_{k-1} + \bar{\varepsilon}_k = \sum_{i=1}^k \bar{\varepsilon}_i, \quad \text{Var}[\hat{\mu}_k] = 0.01 k.$$

random walk 다. 매 라운드 별점 0.01 이 그대로 쌓인다. 이것이 식 (6-3) 의 왼쪽이다.

단계 2 — accumulate. 라운드 k 의 코퍼스는 블록 k 개 (100 k 개 표본) 이고, $\hat{\mu}_k$ 는 블록 평균 m_j 들의 평균이다. $m_1 = \bar{\varepsilon}_1$ 이고 $j \geq 2$ 에 대해 $m_j = \hat{\mu}_{j-1} + \bar{\varepsilon}_j$ 다. 손으로 두 줄만 전개해보면 계수가 보인다.

$$\hat{\mu}_2 = \frac{1}{2}(\bar{\varepsilon}_1 + (\hat{\mu}_1 + \bar{\varepsilon}_2)) = \bar{\varepsilon}_1 + \frac{1}{2}\bar{\varepsilon}_2, \quad \hat{\mu}_3 = \frac{1}{3}(m_1 + m_2 + m_3) = \bar{\varepsilon}_1 + \frac{1}{2}\bar{\varepsilon}_2 + \frac{1}{3}\bar{\varepsilon}_3.$$

일반형은 $\hat{\mu}_k = \sum_{i=1}^k \bar{\varepsilon}_i/i$ 이고, 이것이 식 (6-1) 이다. 잡음이 제곱으로 들어가므로

$$\text{Var}[\hat{\mu}_k] = 0.01 \sum_{i=1}^k \frac{1}{i^2} \leq 0.01 \times \frac{\pi^2}{6} = 0.016449.$$

단계 3 — replace-multiple. 실데이터를 버리되 라운드 k 에 100 k 개를 $\hat{\mu}_{k-1}$ 에서 뽑는다. 블록 평균의 분산이 $0.01/k$ 이므로 $\text{Var}[\hat{\mu}_k] = 0.01 \sum_{i \leq k} 1/i$ —조화급수이고 $\log k$ 로 발산한다.

표 6-5. 세 스케줄의 $\mathcal{L}_{\text{test}}(\Theta_k)$ ($\sigma^2=1, T=100$, 이 책의 산술)

k	replace	replace-multiple	accumulate	accumulate 의 실데이터 비율
1	0.0100	0.0100	0.0100	1
2	0.0200	0.0150	0.0125	1/2
3	0.0300	0.01833	0.01361	1/3
4	0.0400	0.02083	0.01424	1/4
5	0.0500	0.02283	0.01464	1/5

k	replace	replace- multiple	accumulate	accumulate의 실데이터 비율
10	0.1000	0.02929	0.01550	1/10
100	1.0000	0.05187	0.01635	1/100
∞	∞	∞	0.016449	0

$k = 100$ 행이 이 표의 요점이다. replace의 오차 1.00은 **참 분포의 분산과 같다** — 100 라운드를 돌고 나면 모델의 평균추정이 실표본 하나보다 나을 것이 없다. 같은 라운드에서 accumulate는 0.01635이고 영원히 0.016449를 넘지 않는다. 약 61 배 차이가 학습 규칙이 아니라 스케줄 하나에서 나온다.

단계 4 — 혼합 비율을 고정하면. 이제 질문을 바꾼다. 코퍼스크기를 $T = 100$ 으로 **고정한 채** 매 라운드 실데이터 비율을 λ 로 유지하면 (이 장에서만 쓰는 기호다) 분포는 어디로 가는가. 라운드 k 의 학습 집합은 새 실표본 λT 개와 $\hat{\mu}_{k-1}$ 에서 뽑은 합성 표본 $(1 - \lambda)T$ 개다. 표본평균을 그대로 쓰면

$$\hat{\mu}_k = (1 - \lambda)\hat{\mu}_{k-1} + u_k, \quad \text{Var}[u_k] = \frac{\lambda^2}{\lambda T} + \frac{(1 - \lambda)^2}{(1 - \lambda)T} = \frac{1}{T} = 0.01.$$

계수 $(1 - \lambda)$ 의 AR(1)이므로 정상 분산이 존재한다.

$$\text{Var}_\infty = \frac{0.01}{1 - (1 - \lambda)^2} = \frac{0.01}{\lambda(2 - \lambda)} \quad (6-5)$$

$\lambda = 1$ 이면 0.0100(실데이터만), $\lambda = 0.5$ 이면 0.01333, $\lambda = 0.3$ 이면 0.01961, $\lambda = 0.1$ 이면 0.05263, $\lambda = 0.01$ 이면 0.50251이고, $\lambda \rightarrow 0$ 에서 발산한다. **λ 가 0보다 크기만 하면 분포는 유계인 자리에 멈추고, 정확히 0일 때만 random walk가 된다.** 실데이터의 역할은 참평균으로 끌어당기는 복원력이며, 그 세기가 λ 다.

두 결과를 겹쳐 놓으면 이 장에서 가장 반직관적인 사실이 나온다. accumulate에서 실데이터 비율은 $1/k \rightarrow 0$ 으로 **사라진다**. 그런데도 오차는 0.016449에 멈춘다. 식 (6-5)를 뒤집어 풀면 그 값은 실데이터 비율을 영구히 $\lambda \approx 0.37$ 로 고정한 것과 같은 자리다. **중요한 것은 실데이터의 비율이 0으로 가느냐가 아니라 얼마나 빨리 가느냐다.** $1/k$ 로 줄어드는 것은 충분히 느리다.

[평가] 두 처방의 가격표는 전혀 다르고, 이것이 systems 독자에게 실제 선택지다. accumulate는 유계성을 **계산**으로 산다 — 코퍼스가 kT 로 커지므로 총 학습량이 $\Theta(K^2T)$ 이고, 보존코퍼스가 C_{cap} 의 데이터 꼬리로 남는다. 고정 λ 는 유계성을 **새 실데이터 공급**으로 산다 — 라운드마다 λT 개의 신선한 사람 데이터가 계속 들어와야 하고, 대신 코퍼스크기와 학습량은 $\Theta(KT)$ 로 평평하다. 후자는 [MCI]가 첫 번째 open question으로 남긴 레짐이고 [MCI § 4], corpus의 어느 논문도 답하지 않는다. 배포된 시스템은 대부분 후자에 있다.

요약

- (U- Θ)의 절반은 $\text{gen}(\mathcal{H}_k; B_s)$ 이고, 이장은 그 절반만 다뤘다. data augmentation·synthetic data·self-generated data는 포함 관계가 아니라 서로 다른 질문 (원 항목이 있는가 / 사람이 썼는가 / 생성기가 누구인가)에 답하는 세 축이다.
- LLM에서 augmentation은 결정론적 변환이 아니라 decode다. 그래서 비용이 CPU dataloader에서 GPU로 옮겨 왔고, augmentation과 synthetic data의 경계가 구현상 사라졌다. 셋 중 둘 이상에 걸치면 이 책은 self-generated로 분류한다.
- 데이터를 만드는 절차가 일의 대부분을 한다. no-context SQuAD에서 원 passage 학습은 $32.7 \rightarrow 33.5$ 에 그치고, 합의 형식으로 바꾸면 39.7, 생성기를 GPT-4.1로 바꾸면 46.3, 생성 정책을 RL로 학습하면 47.0이다 [SEAL Table 2]. 세 증분의 크기 순서가 B_s 배분의 순

서다. 그리고 자기 생성이 남의 생성을 이긴다는 주장은 그 세 열 중 한 열에서만 성립한다 — CPT $n=200$ 과 $n=2067$ 에서는 GPT-4.1 이 각각 59.4 대 58.2, 49.2 대 46.4 로 이긴다 [SEAL Table 2, § 4.2].

- 필터 ϕ 는 gen 의 나머지 절반이고 B_s 의 곱셈인자다. 가장 비싼 ϕ (실제 학습-후-평가) 와 가장 싼 ϕ (rubric 채점) 의 차이는 1.4 점이고 계산은 약 72 배다 [SEAL Table 9]. 필터는 분포를 자를 뿐 옮기지 못한다 — 소스가 나쁘면 구제되지 않는다 [SEAL Table 10].
- model collapse 는 네 현상의 이름이며, 번호를 지정하지 않은 붕괴 진술은 이 책에서 결함이다 [MCI § 4].
- 반복 (U- Θ) 의 안정성은 학습 규칙이 아니라 $\mathcal{R}_k$ 의 스케줄이 정한다. ρ_k 는 replace 에서 상수, replace-multiple 에서 $1/k$, accumulate 에서 $1/k^2$ 이고, 지속 가능 조건은 $\sum_k \rho_k < \infty$ 다. accumulate 의 총 열화는 한 라운드 별점의 $\pi^2/6 \approx 1.6449$ 배를 넘지 않는다 [MCI Eq. 3, Eq. 4, Eq. 11].
- 그 경계는 VAE/CelebA 에서 성립하지 않는다 — accumulate 에서도 test error 가 증가한다고 논문이 본문에 쓰고 [MCI § 2.3], 같은 그림의 캡션은 반대로 단정한다 [MCI Fig. 5]. 집계 loss 가 유계인 채로 샘플링 분포의 꼬리가 사라지는 것이 이 장이 넘기는 가장 위험한 실패 양식이다.
- 경계의 네 전제 (처음부터 재학습, 공변량 동결, well-specified·ridgeless, 무필터 생성) 는 Θ -경로의 모든 시스템에서 깨진다. $\pi^2/6$ 은 SEAL 과 Dreaming 의 안전 증서가 아니다.

자가 점검체크리스트

- ☐ 내가 보고 있는 파이프라인의 $\mathcal{R}_k$ 를 표 6-1 의 네 행 중 하나로 분류할 수 있는가. 특히 생성기가 학습 대상 모델 자신인지 아닌지를 말할 수 있는가.
- ☐ 그 파이프라인의 스케줄이 replace 인지 accumulate 인지 replace-multiple 인지 말할 수 있는가. 말할 수 없다면 그것은 설계가 정해지지 않았다는 뜻이다.
- ☐ ρ_k 를 적고 $\sum_k \rho_k$ 의 수렴 여부를 판정할 수 있는가. 판정할 수 없다면 무엇이 빠져서인가.
- ☐ “붕괴한다” 고 쓸 때 표 6-3 의 네 번호 중 어느 것인지 매번 지정했는가.
- ☐ 필터 ϕ 가 B_s 를 몇 배로 만드는지 계산했는가 (M 개 생성 + 후보당 판정 비용).
- ☐ 식 (6-2) 를 인용하기 전에 네 전제 (처음부터 재학습 · 공변량동결 · well-specified/ridgeless · 무필터) 가 내 시스템에서 성립하는지 확인했는가.
- ☐ **Rosetta**: “warm-up / 사전 컴파일 $\leftrightarrow$ sleep 라운드” 대응에서 sleep 라운드만 갖는 성질을 말할 수 있는가 — warm-up 은 품질을 바꾸지 않지만 sleep 라운드는 바꾸며, **나쁜 쪽으로도 바꾼다**. 그리고 “checkpoint 크기 $\leftrightarrow C_{\text{cap}}$ ” 대응이 accumulate 에서 왜 깨지는지 (상태가 가중치가 아니라 보존 코퍼스로 옮겨간다) 설명할 수 있는가.

다음 장으로

이 장은 $\mathcal{R}_k$ 를 무엇으로 채울 것인가에 답했고, 그답의 유효 범위가 전제 (i) — 라운드마다 Θ_k 를 처음부터 다시 학습한다 — 에 걸려 있음을 보였다. 실제 시스템은 그렇게 하지 않는다. 살아있는 checkpoint 를 라운드마다 증분 갱신한다. 그 순간 이 장의 계산에 없던 실패 양식이 등장한다: 새 데이터를 배우면서 **이전에 알던 것을 잃는 것**이다.

ch07 이 그 실패를 다룬다. 그리고 그 대응책 중 하나가 이 장의 accumulate 와 사촌이라는 사실 — 옛 데이터를 코퍼스에 계속 섞어 넣는 것 — 이 두 장을 잇는다. ch07 은 그 사촌 관계를 정확히 어디까지 밀 수 있는지, 그리고 옛데이터를 실물로 보관하지 않고도 같은 효과를 낼 수 있는지를 묻는다.

ch07. 망각과 replay — ρ 의 정체

이 장의 목표 — 이 장을 마친 독자는 다음을 할 수 있다. 1. catastrophic forgetting 을 식 (U- Θ) 의 어느 항의 부재로 정의하고, 그 대응책 네 가족 (rehearsal · pseudo-rehearsal · 정규화 · 제약) 이 각각 갱신식의 어디를 건드리는지 분류할 수 있다. 2. EWC 의 Fisher 정보, GEM 의 gradient 사영, deep generative replay 의 scholar 재귀를 기제 수준에서 설명하고, 세 방법의 비용 구조가 왜 서로 다른 자원 (대역폭 · 라운드 수 · 생성 FLOP) 에 걸리는지 말할 수 있다. 3. 망각률 ρ 를 “무엇을 · 언제 · 무엇에 대해 정규화해 재는가” 세 항목으로 조작적으로 정의하고, 실제 성능 행렬에서 손으로 계산할 수 있다. 4. 어떤 논문이 “망각을 막았다” 고 주장할 때 그 주장이 ρ 의 어느 정의를 쓰는 지, 그리고 그 값이 실제로 보고되었는지 판정할 수 있다.

왜 필요한가 — 비용 4 종 (B_s , L_w , C_{cap} , ρ) 중 ρ 만이 반복에 대한 값이다. 나머지 셋은 sleep 라운드 한 번의 값이지만 ρ 는 라운드가 쌓일 때 무엇이 무너지는지를 켜다. 그래서 Θ -경로를 다루는 Part II 의 장들이 전부 이 장을 요구한다: ch18(Nested Learning) 은 지속학습을 헤드라인으로 걸면서 Θ -경로 선행 연구로 EWC 하나만 인용하고, ch20(SEAL) 과 ch21(Language Models Need Sleep) 은 sleep 라운드마다 배포가중치를 바꾸며, ch22(반증 축) 의 질문 “가중치에 사실을 계속 넣을 수 있는가” 는 그 자체로 ρ 의 상한 질문이다. E -경로도 면제되지 않는다 — ch15의 ReasoningBank 은 경험이 쌓일수록 성능이 단조 악화한다. Part III 에서는 ch25(통일 비용 회계) 가 상각식 (A) 를 K 라운드로 확장할 때, ch30(반증 조건) 이 “몇 라운드까지 버티는가” 를 판정할 때 이 장의 정의를 그대로 쓴다. 실험 X4(반복 consolidation 열화) 의 측정 규칙도 여기서 정한다.

NM 과의 관계 — Neural Memory ch11 이 catastrophic forgetting 의 기제 (공유 weight 위 gradient 간섭), 완화책 3 가족의 (state, update, cost) 개괄, 2-parameter forgetting/EWC 손계산, CLS 이론, knowledge distillation 을 이미 다뤘다. 그 내용은 반복하지 않는다 — 간섭의 기하와 2 차원 손계산은 → NM ch11 § 11.1 · § 11.7, CLS 의 fast/slow 분업은 → NM ch11 § 11.3(이 책에서는 ch08 이 소유), distillation 의 문법은 → NM ch11 § 11.5 와 이 책 ch04. 이 장이 더하는 것은 정확히 둘이다. 첫째, EWC · GEM · DGR 세 원형의 기제 상세와 각 논문이 실제로 보고한 (그리고 보고하지 않은) 증거. 둘째, ρ 를 이 책의 비용 4 종 중 하나로 조작적으로 정의하는 것 — NM 에는 이 정의가 없다. 용량 상한은 ch09, CLS 와 생물학적 sleep 은 ch08 이 소유하므로 여기서 참조만 한다.

07.1 catastrophic forgetting — (U- Θ) 에 없는 항

세 경로 중 Θ -경로만이 망각을 겪는다. E -경로는 레코드를 덮어쓸 수 있고 W -경로는 상태를 감쇠시키지만, 배포 아티팩트 자체가 라운드마다 다른 함수로 바뀌는 것은 Θ -경로뿐이다. 그래서 이 장은 Θ -경로의 근본 제약을 다루는 장이고, 다른 두 경로에는 유비로만 적용된다.

catastrophic forgetting 은 과제 A 를 이미 학습한 network 를 과제 B 의 데이터만으로 계속 훈련할 때, A 의 성능이 점진적이 아니라 급격하게 — 종종 거의 전부 — 무너지는 현상이다. EWC 는 이것을 뒤집을 대상으로 소개한다: “catastrophic forgetting 이 connectionist 모델의 불가피한 성질이라고 널리 여겨져 왔다” [EWC Abstract, lines 19-20]. 현상의 기제 — 지식이 수많은

weight 에 분산 저장되고 gradient descent 는 현재 objective 만 본다 — 는 $\rightarrow$ NM ch11 § 11.1 이 이미 세웠다.

이 책의 프레임에서 그 기제는 한 줄로 다시 읽힌다. 표준형 (U- Θ) 는

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta_k), \quad \mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$$

인데, $\text{gen}(\cdot)$ 이 라운드 k 의 데이터만 돌려주면 $\mathcal{L}$ 어디에도 과거 과제의 항이 없다. 망각은 부작용이 아니라 그 경우의 갱신식이 정의하는 바로 그 동작이다. 따라서 **망각 대응의 설계** = $\text{gen}(\cdot; B_s)$ 또는 $\mathcal{L}$ 의 설계이며, 이장이 다루는 네 가족은 그 둘 중 어디를 건드리느냐로 정확히 갈린다.

여기서 EWC 가 하나의 갈래를 달는다. 명백한 답 — 과거 데이터를 보관했다가 섞어 넣기 — 을 EWC 는 **정확도가 아니라 용량 논거**로 거부한다: “과제가 순차적으로 제시되면 multitask learning 은 데이터가 episodic memory system 에 기록되어 훈련 중 network 에 replay 되는 경우에만 쓸 수 있다. 이 접근 (흔히 system-level consolidation 이라 불리는) 은 많은 수의 과제를 학습하는데 비현실적인데, 우리 설정에서는 저장하고 replay 해야 하는 기억의 양이 과제 수에 비례해야 하기 때문이다” [EWC § 1, lines 50-54]. 이 한 문장이 Θ -경로의 분기점이다. EWC 는 저장소를 갖지 않기로 결정했고, deep generative replay 는 저장소를 유지하되 그것을 파라미터로 바꾸기로 결정했다.

[해설] 독자의 세계에는 망각이 없다. serving 중 weight 는 움직이지 않고 KV cache 는 append-only 여서, 어제 넣은 entry 가 오늘의 write 때문에 변질되지 않는다. Rosetta 사전의 “모델 재배포 $\leftrightarrow$ (U- Θ) 1 회” 행이 이 불변식을 깨는 지점이다. sleep 라운드가 하루 한 번 돈다면 1 년에 365 개의 배포 아티팩트가 생기고, 그때 ρ 는 낮선 학술 지표가 아니라 **릴리스마다 회귀 스위트에서 잡히는 성능 회귀의 크기**가 된다. § 07.6 에서 정의할 성능 행렬 Π 는 실제로 “매 릴리스마다 회귀 스위트 전체를 돌린 결과표” 와 같은 물건이다.

07.2 rehearsal 와 pseudo-rehearsal — 데이터를 되돌려주는 두 방법

$\text{gen}(\cdot; B_s)$ 을 건드리는 가족이 둘이다. 차이는 되돌려주는 입력이 진짜냐 아니냐다.

rehearsal 은 과거 라운드의 실제 예제 일부를 보관했다가 현재 라운드의 학습 집합에 섞는 것이다. 즉 $\mathcal{R}_k = \mathcal{D}_k \cup \mathcal{B}$ 이고, $\mathcal{B} \subset \mathcal{H}_{<k}$ 는 실제 데이터의 부분집합이다. 갱신식 자체는 바뀌지 않고 데이터 분포만 교정된다.

pseudo-rehearsal 은 실제 예제 대신 **만들어 낸 입력**과 그 입력에 대한 **이전 모델 자신의 응답**을 쌍으로 삼아 $\mathcal{R}_k$ 를 채우는 것이다. 원형은 Robins 1995 이며, DGR 은 그 개념을 명시적으로 계승한다 [DGR § 2.2, lines 122-124]. 핵심은 정답 label 이 아니라 이전 모델의 출력이 target 이라는 점이다 — pseudo-rehearsal 은 언제나 self-distillation 의 한 형태다 ($\rightarrow$ ch04).

두 논문이 rehearsal 을 거부하는 방식이 서로 다르다는 점이 중요하다. EWC 는 § 07.1 에서 인용한 대로 **과제 수에 비례하는 저장량**을 이유로 든다. DGR 은 같은 거부를 두 이유로 반복한다: “큰 memory 를 요구하고, 더 나쁘게는 과거 데이터 접근이 제한된 실제 응용에서는 종종 실행 불가능하다” [DGR Abstract, lines 17-19]. DGR 은 여기에 프라이버시 논거를 덧붙인다 — 과거 실제 데이터를 다시 보지 않으므로 “프라이버시 문제가 걸린 다양한 실무상황에 쓸 수 있다” [DGR § 1, lines 78-80].

[평가] 두 논문 모두 자기 거부의 근거를 **측정하지 않았다**. EWC 의 논거는 “과제 수에 비례” 라는 점근적 진술뿐이고 바이트도 예제 수도 없다. DGR 은 자기가 데이터셋을 대체하겠다고 내세운 generator + solver 의 크기를 그 데이터셋 크기와 **한 번도 비교하지 않는다** — 논문 어디에도 파라미터 수, 아키텍처, 바이트 수가 없다 [DGR § 4.2 line 387 에서 supplement 로 유예]. 즉 이 분야의 두 원형 논문은 “저장소는 너무 크다” 를 전제로 출발하면서 그 크기를 양쪽 다 재지 않았다. 상태 용량을 바이트로 보고한 논문이 corpus 에 하나도 없다는 사실 ($\rightarrow$ ch26) 은 여기서 시작된다.

systems 점점 하나. rehearsal buffer 는 모델 파일이 아니라 **data-plane artifact** 다 — 저장 용량, 표본화 대역폭, 보존 정책, 감사 대상, 삭제 요청 대응을 갖는 데이터 자산이며 배포 아티팩트와 다른수명주기로 관리된다. pseudo-rehearsal 은 그 자산을 **GPU 시간으로교환**한다: 저장과 I/O 는 0 이 되고 대신 sleep 라운드마다 생성 job 이 돈다. 이것이 이 장에서 처음 나오는 비용 교환이고, B_s 와 C_{cap} 사이의 교환으로 기록된다.

07.3 EWC — 경험을 곡률로압축한다

EWC(elastic weight consolidation; Kirkpatrick et al., arXiv:1612.00796 [EWC line 4]) 는 과거라운드의 데이터를 하나도 남기지 않는 대신, 직전 해를 중심으로 한 좌표별이차 우물을 목적함수에 더해 과거를 기억하는 방법이다. 우물의 강성은 그좌표가 과거 과제에서 갖는 중요도이고, 그 중요도의 추정치가 Fisher 정보의대각이다. 즉 EWC 의 수는 $\mathcal{R}_k$ 를 포기하고 $\mathcal{L}$ 을건드리는 것이다. 유도는 세 단계다.

먼저 Bayes 규칙을 데이터의 두 부분에 대해 쪼갬다. 두 부분이 독립이라는가정 아래 [EWC § 2 line 116],

$$\log p(\Theta | \mathcal{D}) = \log p(\mathcal{R}_k | \Theta) + \log p(\Theta | \mathcal{H}_{k-1}) - \log p(\mathcal{R}_k) \quad (7-1)$$

이다 [EWC Eq. 2]. 이 식이 말하는 것은 하나다: **과거 경험 $\mathcal{H}_{k-1}$ 은 데이터로 남을 필요가 없고 Θ 위의 분포하나로 대체될 수 있다.** “과제 A 에 관한 모든 정보는 따라서 사후분포 $p(\theta | \mathcal{D}_A)$ 에 흡수되어 있어야 한다” [EWC § 2, lines 135-136]. 이것이 정규화 갈래 전체의 형식적 면허다.

두 번째 단계는 그 사후분포를 다룰 수 있는 형태로 근사하는 것이다. EWC 는 MacKay 의 Laplace 근사를 따라 평균이 직전 해이고 정밀도가 Fisher 정보행렬의 대각인 Gaussian 을 쓴다 [EWC § 2, lines 137-146]:

$$p(\Theta | \mathcal{H}_{k-1}) \approx \mathcal{N}(\Theta_{k-1}^*, \text{diag}(F^{(k-1)})^{-1}). \quad (7-2)$$

여기서 Θ_{k-1}^* 은 라운드 $k-1$ 이 끝난 순간의 가중치스냅샷 (**anchor**) 이고, $F^{(k-1)}$ 은 그 지점에서 계산한 Fisher 대각이다. **Fisher 정보**는 좌표별로 그 좌표가 과거 과제의 예측에얼마나 하중을 싣는지를 재는 국소 2 차 통계이며, EWC 는 그것이 필요한이유로 세 성질을 든다: 최솟값 근처에서 loss 의 2 차 도함수와 같고, 1 차도함수만으로 계산되므로 큰 모델에서도 쉽게 구할 수 있으며, 반드시 양의준정부호다 [EWC § 2, lines 142-146].

세 번째 단계가 목적함수다.

$$\mathcal{L}_{\text{EWC}}(\Theta) = \mathcal{L}(\mathcal{R}_k; \Theta) + \sum_i \frac{\lambda}{2} F_i^{(k-1)} (\Theta_i - \Theta_{k-1,i}^*)^2 \quad (7-3)$$

[EWC Eq. 3]. 두 번째 항에는 **데이터가 없다**. 기억하는 비용전체가 이 항이다. λ 는 “옛 과제가 새 과제에 비해 얼마나중요한지를 정한다” [EWC § 2, line 156]. EWC 자신의 비유로는 이전 해에파라미터를 묶는 스프링이고, 스프링 상수가 좌표마다 다르다는 것이요점이다 — “그 강성은 모든 파라미터에 대해 같아서는 안 되고, 과제 A 의성능에 가장 중요한 파라미터에 대해 더 커야 한다” [EWC § 2, lines 103-107].

식 (7-3) 을 미분하면 실제 갱신이 나온다. 라운드 k 안의 SGD step 을 s 로 색인한다 — **step s 와 sleep 라운드 k 는 서로 다른시계다**. 한 라운드는 수천 개의 step 으로 이루어지고, ρ 가 재는 것은 step 이 아니라 라운드다.

$$\Theta^{(s+1)} = \Theta^{(s)} - \eta_{\Theta} \left[\nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta^{(s)}) + \lambda F^{(k-1)} \odot (\Theta^{(s)} - \Theta_{k-1}^*) \right] \quad (7-4)$$

이다. EWC 논문은 갱신식을 쓰지 않으므로 (7-4) 는 이 책이 (7-3) 에서 유도한형태다. systems 어휘로 읽으면 정체가 분명해진다: **기억하는 비용은 step 마다 전체 파라미터 벡터에 대한 elementwise 곱셈-덧셈 한 번이다**. FLOP-bound 연산이 아니라 대역폭-bound AXPY 이고, step 당 2| Θ | 개원소를 더 읽는다. EWC 자신의 복잡도 주장도 이와 일치한다 — “파라미터수와 훈련 예제 수 둘다에 선형인 실행 시간” [EWC § 3, lines 347-348].

Θ -경로 방법으로서 EWC의 결정적 성질은 **서빙 비용이 정확히 0**이라는 것이다. 식 (7-3)은 훈련 목적함수만 바꾸고, 평가는 훈련을 끈 채로 수행된다 [EWC § 2.2, lines 237-239]. 읽기식 (R)은 $\hat{y} = f(q; \Theta)$ 로 붕괴한다 — W 도 $\text{ret}(E, q)$ 도 없다. per-user delta가 없으므로 shared-weight batching도 온전하다. EWC는 지속학습의 값을 지연이나 배칭이 아니라 **정확도 여유**로 치른다.

라운드가 쌓일 때의 상태량은 논문이 스스로 미결로 남겨 놓았다. “세 번째 과제 C로 옮겨갈 때 EWC는 파라미터를 A와 B 둘 다의 학습된 파라미터 근처에 두려 한다. 이것은 두 개의 별도 penalty로 강제하거나, 두 이차 penalty의 합이 그 자체로 이차 penalty임을 이용해 하나로 강제할 수 있다” [EWC § 2, lines 158-160]. 전자는 $O(k|\Theta|)$, 후자는 $O(|\Theta|)$ 이고 **논문은 실험에서 어느 쪽을 썼는지 말하지 않는다**. 훈련 측상태량에 대해 논문이 주는 유일한 수치는 “EWC에서는 시냅스마다 세 값을 저장해야 한다: 가중치 자체, 그 분산, 그 평균” [EWC § 3, lines 369-370] — 즉 $3|\Theta|$ 이며, 바이트도 dtype도 파라미터 수도 없다.

07.3.1 근간 논문에 결과표가 없다는 사실

이 절의 사실 하나가 이 장 전체에서 가장 중요하다. **EWC 논문에는 결과표가 없고, EWC의 정확도 수치가 본문 어디에도 인쇄되어 있지 않다**. 논문의 표는 정확히 두 개이고 둘 다 하이퍼파라미터 표다 [EWC Table 1 line 515; Table 2 line 764]. 전 파일에서 유일한 정량적 성능 진술은 **baseline**을 서술한다: 평범한 gradient descent로는 “총 human-normalized 점수가 1 미만에 머문다” [EWC § 2.2, lines 281-283]. EWC 쪽 값은 없다.

주장 쪽은 그렇지 않다. permuted MNIST의 헤드라인은 “EWC는 많은 수의 과제를 순차적으로 학습하게 하며 오차율은 완만하게만 증가한다” [EWC § 2.1, lines 206-207] 인데, **과제가 몇 개인지가 본문 어디에도 없다**. 그 수는 Figure 2B의 라벨 없는 x축에만 있다. 숫자로 명시된 최대 과제 수는 Atari의 10개다 [EWC § 2.2, lines 234-235]. 독립 학습 대비 격차도 인정되지만 크기가 없다 — “10개의 별도 DQN을 훈련시켰을 때 얻었을 점수에는 도달하지 못한다” [EWC § 2.2, lines 305-307].

논문은 자기 근사의 절반을 스스로 반증하고 그것을 적는다. Fisher의 영공간에 weight 잡음을 주입하면 근사가 옳을 경우 성능이 전혀 변하지 않아야 하는데, 실제로는 역 Fisher 방향 잡음과 같은 정도로 성능이 떨어진다: “우리가 특정 파라미터가 중요하지 않다고 과신하고 있음을 시사한다. 따라서 현 구현의 주된 한계는 파라미터 불확실성을 과소평가하는 것일 가능성이 높다” [EWC § 2.2, lines 314-319; Fig. 3C].

Atari 결과에는 세 겹의 교란이 더 있다. 첫째, 참조 agent 대비 여섯 가지가 동시에 바뀌었고 (파라미터가 더 많은 network, 더 작은 transition table, 층마다의 과제별 bias·gain, 전체 action set, 과제 인식 모델, EWC penalty) **EWC penalty만 분리하는 ablation이 없다** [EWC § 2.2, lines 252-255]. 확대된 network 하나만으로도 파라미터가 약 4배다 [EWC Appendix § 4.2, lines 551-553]. 둘째, Fisher 추정식이 논문에 한번도 쓰이지 않는다 — empirical Fisher 인지 model Fisher 인지, 어느 분포에 대해 평균하는지가 없고 이 선택은 알고리즘을 실질적으로 바꾼다 [EWC § 2 lines 137-146]. 셋째, Atari 시스템은 “추론된 과제마다 별도의 단기 memory buffer”를 유지한다 [EWC § 2.2, lines 268-272] — 서론에서 “많은 수의 과제에 비현실적”이라며 거부한 저장-후-replay 접근을 부분적으로 되살린 것이고, 그 buffer는 추론된 과제 수에 비례해 자란다.

MNIST 쪽에도 세 가지가 더 있다. 첫째, 세 그림의 arm이 서로 다른 설정에서 돌았다 — 학습률·dropout·early stopping·깊이·폭·epoch가 그림마다 다르고, 새 과제와 옛 과제의 균형을 정하는 유일한 하이퍼파라미터인 λ 의 MNIST 값은 표에 아예 없다 [EWC Table 1, lines 484-515]. baseline도 uniform L2와 dropout+early stopping 둘뿐이어서 rehearsal 대조군도 joint-training 상한도 없고 [EWC Fig. 2 caption, lines 184-188], 분산·시드 수는 서술되지 않는다 [EWC Appendix § 4.1, lines 525-537]. 둘째, 초록의 기제 서술이 구현과 어긋난다 — “옛 과제에 중요한 weight의 학습을 선택적으로 늦춘다” [EWC Abstract, lines 22-23]는 좌표별 학습률 조절을 말하지만, 식 (7-3)은 η_θ 를 건드리지 않고 복원력을 더할 뿐이다. 셋째, 벤치마크가 지속학습의 온전한 끝에 있고 논문이 그 이유를 자기 입으로 적는다: permutation은 입력 영역만 바꾸고 출력 영역 (클래스 라벨)은 공유된다 [EWC § 2.1, lines 220-222]. 검증 규모도 여기서 못 박아 둔다 — 가장 큰 실증은 약 4배로 키운 DQN 하나와 최대 2000 폭의 MLP이고 숫자로 명시된 최장 과제열은 10이며 [EWC Table 1, lines 484-515; Appendix § 4.2, lines 548-553, 582-586], sequence

모델도 language 모델도 없다.

[평가] 이 장은 EWC 를 Θ -경로 정규화 갈래의 **원형**으로 세우되, 그 실증 지위를 정직하게 적는다. 널리 인용되는이 논문에서 독자가 가져갈 수 있는 것은 기제와 식 (7-1) 의 면허이지 수치가아니다. 이후 문헌이 “EWC 는 X%를 달성한다” 고 적을 때 그 X는 전부 **재구현에서 나온 값**이다 — 실제로 § 07.4 에서 인용할 EWC 수치는 GEM 저자들의 재구현이다. 근간 논문의 증거가 전부 곡선일 때 그 위에 쌓인 10 년치 비교는 서로 다른 재구현을 비교하고 있는 것이며, 이것이 ρ 를조작적으로 정의해야 하는 첫 번째 실무적 이 유다.

07.4 GEM — 과거를 제약으로삼는다

세 번째 가족은 $\mathcal{R}_k$ 도 $\mathcal{L}$ 도 건드리지 않는다. **갱신 방향**을 건드린다.

GEM(Gradient Episodic Memory; Lopez-Paz & Ranzato, arXiv:1706.08840, NIPS 2017 [GEM line 54]) 은 과거 과제의 실제 예제 일부를 과제별 buffer 에 보관하되**그것으로 학습하지 않고**, 현재 gradient 가 그 buffer 위의 loss 를증가시키지 않도록 갱신 방향을 사영하는 방법이다. 예산 M 개의 저장위치를 과제 수로 나눠 과제마다 M/K 개를 배정하고, 단순히 그 과제의 마지막 M/K 개 예제로 채운다 GEM § 3, 원문은 이 몫을 $m = M/T$ 로 쓰고 과제 수를 T 로 쓴다. 설정도 이 corpus 의 sleep 라운드보다 훨씬 가혹하다 — 학습자는 예제를 한 번에하나씩 (또는 작은 minibatch 로) 받고, 같은 예제를 두 번 보지 않는다 [GEM § 2, lines 105-108].

원 문제는 제약 최적화다. 라운드 k 의 데이터로 학습하되, 과거 buffer 위의 loss 가 직전 라운드 값을 넘지 않게 한다:

$$\min_{\Theta} \mathcal{L}(\mathcal{D}_k; \Theta) \quad \text{s.t.} \quad \mathcal{L}(\mathcal{B}_j; \Theta) \leq \mathcal{L}(\mathcal{B}_j; \Theta_{k-1}) \quad \forall j < k \quad (7-5)$$

[GEM Eq. 6]. 부등식이라는 점이 요점이다. 등식 (distillation 으로 과거예측을 고정) 이면 과거 성능이 **좋아지는** 것도 막히지만, 부등식은증가만 막고 감소는 허용한다 — 논문은 이것을 EWC·iCaRL 대비자기 이점으로 명시한다: “우리 모델은 따라서 양의 backward transfer 를허용한다” [GEM § 3, lines 200-201].

그대로는 못 푼다. 과거 예측기 Θ_{k-1} 을 들고 있어야 하고 제약검사가 비싸다. GEM 은 국소 선형성을 가정해 제약을 **gradient 사이의각도**로 바꾼다:

$$\langle g, g_j \rangle \geq 0 \quad \forall j < k, \quad g = \nabla_{\Theta} \mathcal{L}(\mathcal{D}_k; \Theta), \quad g_j = \nabla_{\Theta} \mathcal{L}(\mathcal{B}_j; \Theta) \quad (7-6)$$

[GEM Eq. 7]. 논문의 진술은 이 조건이 만족되면 제안된 갱신 g 가 “과거 과제의 loss 를 증가시킬 가능성이 낮다” 는 것이다 [GEM § 3] — 국소선형성과 buffer 대표성 가정 위의 진단이지 보증이 아니다. 위반이 생기면제약을 만족하는 가장 가까운 방향으로 사영한다:

$$\tilde{g} = \arg \min_z \frac{1}{2} \|g - z\|_2^2 \quad \text{s.t.} \quad \langle z, g_j \rangle \geq 0 \quad (7-7)$$

[GEM Eq. 8]. 이것은 변수가 $|\Theta|$ 개인 이차계획 (QP) 이고 $|\Theta|$ 는 수백만이 될 수 있다. GEM 의 실행 가능성은 쌍대 문제가 $k - 1$ 개 변수의 QP 라는 데 있다 — 지금까지 관측한 과제수만큼이다 [GEM Eq. 11]. 논문의 CIFAR-100 실험에서 $|\Theta| = 1,109,240$ (원문 기호 p) 이고 과제는 20 개다 [GEM § 4.4, line 598].

[해설] 세 원형을 갱신식 기준으로 나란히 놓으면 이렇게 된다. EWC 는 과거를 **곡률**로 압축해 $\mathcal{L}$ 에 항을 더한다. GEM 은과거를 **제약**으로 유지해 갱신 방향을 사영한다. DGR (§ 07.5) 은과거를 **생성기**로 압축해 $\mathcal{R}_k$ 를 다시 만든다. 세 방법모두 최종 배포 아티팩트는 평범한 가중치 벡터 하나이고 서버 비용은동일하다 — 차이는 전부 sleep 라운드 안에서 어느 자원을 쓰느냐다.

비용 구조가 EWC와 근본적으로 다르다. EWC의 기억 비용은 step 당 2| Θ | 원소 읽기로 **라운드 수와 무관**하다. GEM은 step마다 과거 과제 buffer 전부에 대해 gradient를 계산해야 하므로 step 비용이 라운드 수에 **선형**이다. 논문 자신이 이것을 병목으로 지목한다: “GEM의 병목은 매 학습 반복마다 이전 과제 gradient를 계산해야 한다는 것이다” [GEM § 4.4, line 599]. MNIST 실험의 CPU 훈련 시간이 이를 보여준다 — permutations에서 baseline 11 초, EWC 179 초, GEM 77 초; rotations에서 11 초 / 169 초 / 135 초 [GEM Table 1]. baseline 대비 GEM은 7.0 배 · 12.3 배다 (비율 계산은 이 책의 것).

[평가] sleep-time compute의 관점에서 이 비용 구조는 치명적이다. Θ -경로가 매일 또는 sleep 라운드로 운영된다면 k 는 무한정 자라므로, GEM류 제약 방법의 라운드당 sleep 예산 B_s 도 함께 자란다. EWC의 상수 비용이 이 축에서는 정확히 옳은 설계이고, 실제로 GEM이 저장하는 것도 EWC가 거부한 바로 그 실제예제다 (MNIST·CIFAR 실험 모두 예산 5120 개 [GEM App. A]). GEM은 EWC의 용량 논거를 논박한 것이 아니라 **다른 자원으로 옮긴 것**이다: 저장은 다시 늘고 step 비용이 라운드 수에 실린다.

GEM이 이 장에 반드시 있어야 하는 진짜 이유는 알고리즘이 아니라 **계기**다. 논문은 지속학습을 재는 세 지표를 정의하면서 성능 행렬 Π 를 도입한다 — $\Pi_{i,j}$ 는 과제 i 의 마지막 표본을 관측한 직후 과제 j 에서 얻은 테스트 정확도이고, $\bar{b}$ 는 무작위 초기화시점의 정확도 벡터다 [GEM § 2, lines 125-128]. 표기 대응: **원문은 이 행렬을 R 로 쓴다**. 이 책에서 R 은 강화학습의 보상이고 $\mathcal{R}_k$ 는 sleep 라운드의 학습 집합이므로, 성능 행렬은 Π 로 옮겨 쓴다. 여기서 **backward transfer**가 나온다:

$$\text{BWT} = \frac{1}{K-1} \sum_{i=1}^{K-1} (\Pi_{K,i} - \Pi_{i,i}) \quad (7-8)$$

[GEM Eq. 3, 원문은 과제 수를 T 로 쓴다]. 논문의 표현으로 “크게음인 backward transfer는 (catastrophic) forgetting이라고도 불린다” [GEM § 2, lines 116-118]. § 07.6의 ρ 는 이 계기를 라운드 축으로 일반화한 것이다.

수치도 이 장에 필요하다. 20개 과제 permuted MNIST에서 최종 ACC / BWT는 baseline 0.6018 / -0.1980, EWC 0.6185 / -0.1653, GEM 0.8260 / +0.0247이다 [GEM App. B.1]. rotations에서는 0.5307 / -0.0896, 0.5461 / -0.2047, 0.8607 / +0.0048 [GEM App. B.2]. CIFAR-100에서는 0.4631 / -0.1226, EWC 0.4984 / -0.0799, iCaRL 0.5462 / -0.0830, GEM 0.6783 / +0.0042 [GEM App. B.3]. 과제당 epoch를 늘리면 망각이 심해진다는 것도 표로 나온다 — rotations에서 baseline의 ACC/BWT가 1·2·5 epoch에 대해 0.53/-0.08 → 0.49/-0.25 → 0.43/-0.40인 반면 GEM은 0.86/+0.05 → 0.88/+0.02 → 0.89/-0.02이고, 5 epoch에서 GEM은 iid 학습 oracle의 0.89와 같다 [GEM Table 3].

이 수치들에 두 개의 단서를 반드시 단다. 첫째, **여기 인쇄된 EWC 값은 GEM 저자들의 재구현**이다 (학습률과 정규화 강도를 격자 탐색으로 고른 [GEM App. A]) — EWC 논문에는 비교할 값이 없다 (§ 07.3.1). 둘째, **인쇄된 BWT 값은 식 (7-8)이 인쇄한 분모 $K-1$ 이 아니라 K 로 나눈 값**이다. 부록의 성능 행렬 15개에서 직접 재계산하면 ACC는 인쇄값과 소수 넷째 자리까지 일치하지만 BWT·FWT는 분모 K 일 때만 일치한다 — permutations의 baseline 블록에서 $\sum_{i < K} (\Pi_{K,i} - \Pi_{i,i}) = -3.9610$ 이고, 19로 나누면 -0.2085, 20으로 나누면 인쇄값 -0.1980이다 [GEM App. B.1.1, 재계산은 이 책의 것]. 모든 인쇄 magnitude가 균일하게 19/20 배이므로 순위 · 부호 · 질적 주장은 하나도 바뀌지 않지만, 아래 worked micro-example처럼 인쇄값에서 라운드당 값을 역산하려면 이 분모를 반드시 맞춰야 한다.

[평가] MNIST rotations의 forward transfer는 방법의 성질이 아니라 벤치마크의 성질이다. GEM 0.6647, EWC 0.5524인데 **아무 대응도 하지 않는 baseline이 0.4254**다 [GEM App. B.2] — forward-transfer 기제를 하나도 갖지 않은 arm이 그 값을 낸다. 회전된 숫자는 서로 닮았고, 그래서 이 지표는 과제 유사도를 재고 있다. 같은 지표가 permutations에서는 전 arm이 0.0054 - 0.0093에 몰리고 CIFAR-100에서는 GEM이 -0.0078로 음수다 [GEM App. B.1, B.3]. 논문은 rotations의 큰 값을 지표 해석의 단서 없이 보고한다.

07.5 generative replay — 데이터를 생성기로바꾼다

DGR(Deep Generative Replay; Shin et al., arXiv:1705.08690 [DGR line 3], NIPS 2017 [DGR line 57]) 은 EWC 가 § 07.1 에서 거부한 저장소 문제에 정반대로 답한다: replay 를 유지하되 저장되는 대상을 데이터셋에서 생성기로바꾼다.

generative replay 는 과거 입력 분포를 학습한 생성 모델에서 입력을 표본화하고, 그 입력에 대한 **직전 라운드 모델 자신의 응답**을 target 으로 붙여 $\mathcal{R}_k$ 를 만드는 것이다. DGR 은 이 쌍을 **scholar** $\Sigma_k = \langle \Theta_k^G, \Theta_k \rangle$ 라고 부른다 — 생성기와 solver 의 튜플이며, 배우면서 동시에 가르치기 때문에 teacher 도 student 도 아닌 이름을 붙였다 [DGR § 3 Def. 2, lines 162-166].

프레임 언어로 옮기면 DGR 이 채운 자리가 정확히 보인다. 표준형 (U- Θ) 의 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 에서 $\mathcal{H}_k$ 는 어떤 형태로든 존재한다고 전제되는데, DGR 은 $\mathcal{H}_k$ 를 **삭제**하고 파라미터 대리물 Σ_{k-1} 로 치환한다:

$$\mathcal{R}_k = \left\{ (\tilde{x}, f(\tilde{x}; \Theta_{k-1})) : \tilde{x} = G(z; \Theta_{k-1}^G), z \sim p_z \right\} \quad (7-9)$$

[DGR § 3.1 lines 182-187; Fig. 1(c)]. 그리고 목적함수는 실제 데이터와 replay 를 상수 비율 ω 로 섞는다 (표기 대응: **원 논문은 이 비율을 r 로 쓴다**. 이 책에서 r 은 LoRA rank 이므로 혼합 비율은 ω 로 옮긴다. $\mathcal{L}_{\text{ex}}$ 는 예제 하나에 대한 loss 다):

$$\mathcal{L}_{\text{train}}(\Theta_k) = \omega \mathbb{E}_{(x,y) \sim \mathcal{D}_k} [\mathcal{L}_{\text{ex}}(f(x; \Theta_k), y)] + (1 - \omega) \mathbb{E}_{\tilde{x}} [\mathcal{L}_{\text{ex}}(f(\tilde{x}; \Theta_k), f(\tilde{x}; \Theta_{k-1}))] \quad (7-10)$$

[DGR Eq. 1]. 둘째 항은 self-distillation 이다 — 교사는 직전 solver 이고, 문제지는 직전 생성기가 지어낸 입력이다.

재귀가 이중이라는 점이 이 장에서 가장 중요한 사실이다. solver 만자기 전임자에게서 배우는 것이 아니라 생성기도 그렇다: 생성기는 현재 입력과 **직전 생성기의 표본**을 섞은 분포에 적합되어 누적 입력 분포를 재현한다 [DGR § 4.3, lines 475-478]. 라운드가 쌓이면 두 재귀가 곱해지고, 외부 앵커는 매 라운드의 새 실제 데이터 ω 몫뿐이다. 이것이 ch06(합성 데이터와 붕괴)의 model collapse 질문과 같은 구조이며, 그 연결이 ρ 가 이 라인에서 왜 지속가능성 지표인지를 말해준다.

DGR 이 이 문제에 대해 내놓는 증거는 딱 한 표다. 실제 데이터로 배운 첫 solver 부터 이전 scholar 에게서만 배운 다섯 번째 solver 까지, 전체 MNIST 테스트 정확도가 98.81% → 98.64% → 98.58% → 98.53% → 98.56%다 [DGR Table 1, § 3.2]. 세대당 변화는 -0.17, -0.06, -0.05, +0.03 퍼센트포인트이고 네 세대 누적 -0.25pp 이며 마지막 단계는 비단조다 (차분 계산은 이 책의 것). 논문의 결론은 “scholar 모델은 정보손실 없이 지식을 전달한다” 이다 [DGR § 3.2, lines 290-291].

[평가] 이 표는 **지속학습 결과가 아니다**. 과제가 하나뿐이고, 다섯 solver 전부를 같은 전체 MNIST 테스트 집합에서 쟀다 [DGR Table 1 caption, lines 292-293]. 즉 측정된 것은 “한 과제에 대해 자기복제 사슬이 무슨 실인가” 이지 “과제들이 이어질 때 무엇을 잇는가” 가 아니다. 그럼에도 이 표는 corpus 에서 **반복 self-training 의 열화를 숫자로 인쇄한 몇 안 되는 자리**이며, 그래서 이 책은 이것을 ρ 의 최초 실측치로 취급하되 범위 단서를 붙여 인용한다. 실험 X4 가 같은 측정을 LLM 규모의 sleep 라운드에서 재현하려는 이유가 이것이다.

논문에 불리한 사실이 여기서 여럿 나온다. 첫째, [DGR § 4.1 – § 4.3] 의 네 주요 실험은 **전부 곡선이고 숫자가 하나도 없다** — 표는 Table 1 하나뿐이고 그것이 방금 본 예비 실험이다. 둘째, generative replay 는 자기상한 대조군 (ER: 실제 과거 입력 + 직전 solver 의 예측 target) 을 **이긴 적이 없고 비길 뿐이다**: “결과는 직전 solver 의 예측 응답과 짝지은 실제 과거 입력을 replay 하는 것보다 나쁘지 않았다” [DGR § 4.2, lines 416-417]. 그 상한 자체도 무르다 — ER 조차 정답 label 이 아니라 예측 target 을 쓰므로 진짜 joint training 은 어느 실험에도 없다 [DGR § 4, lines 337-339]. 셋째, 방법의 대가를 저자들이 직접 적는다: “두 경우 모두 replay 데이터 없이 훈련한 모델이 새 과제에서 약간 더 나은 성능을 냈다” [DGR § 4.2, lines 417-419]. 넷째, **EWC 가 baseline 으로 한번도 돌지 않았다** — [DGR § 2.1] 과 [DGR § 5] 에서 길게 논의하는 주된 경쟁

자인데 실험 arm 은 GR/ER/Noise/None 넷뿐이다 [DGR § 4, lines 336-342]. 다섯째, 논문 자신의 실패 사례가 크기 없이 보고된다: “§ 4.3 과 동일한 설정에서 SVHN 데이터셋으로 훈련할 때 일부 성능손실을 관측했다. 상세 분석은 supplementary materials 에 있다” [DGR § 5, lines 505-507]. 여섯째, 이론적 보증이 조건부인데 그 조건이 측정되지 않는다 — “생성기가 입력 분포를 복원하는 한 누적 실제 데이터에 대한 joint training 과 동등하다” [DGR § 2.2, lines 133-134] 이고, 분포 복원 정도를 재는 지표 (FID·coverage 등) 는 논문 어디에도 없다. 생성기 품질은 Figure 4 와 7 의 표본 격자를 눈으로 보는 것으로만 평가된다. 일곱째, GR 의 마진이 가장 큰 [DGR § 4.3] 의 class-incremental 설정은 저자들이 이 논문을 위해 제안한 벤치마크다 — “우리는 network 를 서로 겹치지 않는 데이터로 순차 학습시키는 새 실험을 제안한다” [DGR § 4.3, lines 444-446]. 여덟째, Noise arm 은 무작위 가우시안 입력을 되돌려주는 약한대조군이고 [DGR § 4, lines 339-341], 실험 전체가 아키텍처하나·생성기 하나 (WGAN-GP [DGR § 4, lines 325-326])· ω 값 하나·시드 반복없음의 단일 구성 위에 있다. 아키텍처도 파라미터 수도 ω 값도 본문에 없고 supplement 로 유예되었다 [DGR § 4.2, line 387].

systems 관점에서 DGR 의 sleep 작업은 셋으로 쪼개진다. (i) 생성기에서의 조상 표본화 — 표본 간 데이터 의존이 없는 대규모 dense 생성 job, (ii) 동결된 직전 solver 의 forward pass 로 target 만들기, (iii) 혼합 집합위에서 solver 와 생성기를 각각 다시 훈련. 셋 다 지연 SLA 가 없는 throughput-bound 배치 작업이고, 서버 fleet 의 비수기 용량이 자연서식지다. 저장 측면에서는 과거 데이터 plane 이 통째로 사라진다 — 저장도 I/O 도 0 이 되고 대신 GPU 시간이 든다. **compute-for-storage 교환**이며 논문은 양쪽 어느 것도 값을 매기지 않았다.

07.6 ρ 의 조작적정의

지금까지 세 원형이 전부 “망각을 줄인다” 고 주장했고, 그 주장을 검증할 수 있었던 것은 GEM 하나였다. GEM 만이 계기를 정의했기 때문이다. 이 절은 그 계기를 이 책의 비용 4 종에 맞게 다시 세운다.

ρ 의 정의는 세 질문에 답해야 한다.

(1) **무엇을 재는가.** 고정된 과제 집합 위의 1 차 지표다. 라운드 k 가 끝난 직후 과제 i 에서 잔 값을 $\Pi_{k,i}$ 로 쓰고, 이 행렬을 **성능 행렬**이라 부른다 [GEM § 2, lines 125-128 의 행렬을 라운드축으로 재해석]. 세 가지를 명시해야 값이 정의된다 — (a) 과제 집합: 이미 학습한 과제만이며 아직 안 본 과제는 ρ 가 아니라 forward transfer 의 대상이다, (b) 지표: 그 과제의 1 차 지표하나 (정확도·EM·pass@1) 로 고정하고 라운드마다 바꾸지 않는다, (c) **기준점**: 무작위 초기화 값 $\bar{b}_i$ 가 아니라 그 과제를 막 마쳤을 때의 값 $\Pi_{i,i}$ 다. 기준점을 $\bar{b}_i$ 로 잡으면 망각이 아니라 순 학습량을 재게 된다.

(2) **언제 재는가.** 라운드 k 의 갱신이 끝난 직후, 다음 라운드가 시작하기 전이다. 시간 첨자는 k (sleep 라운드) 이지 t (토큰) 가 아니다 — W -경로의 상태 감쇠와 혼동하지 않기 위해 이 구분을 지킨다. 라운드 경계가 외생적으로 주어지지 않는 설정 (경계를 추론해야 하는 경우 [EWC Appendix § 4.2] 또는 경계가 아예 없는 연속 자기복제 [DGR § 5, lines 513-514]) 에서는 **측정 시점 자체가 설계 결정**이 되며, 그 사실을 함께 보고해야 한다.

(3) **무엇에 대해 정규화하는가.** 두 축이 있고 둘을 섞으면 안 된다. 하나는 **라운드 수** (누적값을 라운드 하나의 값으로 환산), 다른 하나는 **각 과제 자신의 기준값** (퍼센트포인트를 상대 손실로 환산). 이 책의 기본형은 둘 다 적용한 증분·상대형이다:

$$\rho_k = \frac{1}{k-1} \sum_{i < k} \frac{\Pi_{k-1,i} - \Pi_{k,i}}{\Pi_{i,i}} \quad (7-11)$$

망각률 ρ_k 는 sleep 라운드 k 하나가, 그 시점까지 이미 학습한 과제들에, 각 과제 자신의 anchor 값 대비 입힌 상대 성능 손실의 평균이다. 값이 양이면 열화, 음이면 양의 backward transfer 다. 누적형이 필요하면 $\rho_K^{\text{cum}} = \frac{1}{K-1} \sum_{i < K} (\Pi_{i,i} - \Pi_{K,i}) / \Pi_{i,i}$ 를 쓰되 반드시 이름을 달리 적는다. 분모의 $k-1$ 은 이 책이 고정한 값이므로, 인쇄된 BWT 에서 ρ 를 역산할 때는 그 논문이 실제로 쓴 분모를 먼저 확인한다 (§ 07.4 의 둘째 단서).

식 (7-8) 의 BWT 와의 관계는 정확히 이렇다. -BWT 는 (7-11) 의 절대·누적 판본이며, 마지막 라운드에서 한 번만 잔값이다. 증분형과 누적형은 서로 자동 변환되지 않는다 — 변환에는 각 과제

가 몇 번의 후속 라운드에 노출되었는지가 필요하다. 이 장의 worked micro-example (c) 가 그 보정을 빠뜨리면 같은 실행에서 왜 2 배 가까이 어긋나는지 숫자로 보인다.

ρ 가 필요한 자리는 상각식이다. 식 (A) $C_{\text{avg}} = C_{\text{wake}} + C_{\text{sleep}}/N_q$ 는 sleep 라운드 **한 번**의 회계다. 라운드가 K 번 반복되면 분자는 K 배가 되지만 분모의 품질도 ρ 만큼씩 깎인다. ρ 를 모르면 “ K 를 늘리면 이득이 누적된다” 와 “ K 를 늘리면 모델이 무너진다” 중 어느 쪽인지 판정할 수 없고, 그것이 이 라인이 유망한지에 대한 답의 절반이다 ($\rightarrow$ ch25, ch30).

[평가] ρ 는 비용 4 종 중 **가장 자주 비어 있는 칸**이다. 이 장의 두 배정 논문이 그 전형이다 — EWC 는 ρ 를 숫자로 전혀 보고하지 않고 (“많은 수의 과제” , “완만한 증가” 가 전부 [EWC § 2.1, lines 206-207]), DGR 이 인쇄한 유일한 수열은 지속학습 설정이 아니다. Part II 에서 각 장의 비용 4 종 표를 채워 보면 ρ 칸이 숫자로 채워지는 논문은 예외에 가깝다는 것이 드러난다. 게다가 채워진 경우조차 정의가 서로 다르다 — E -경로의 ch15 는 라운드가 아니라 검색 경험량에 대한 단조 악화를 보고하고, Θ -경로의 ch21 은 구성 요소 제거 실험의 표에서만 열화가 드러난다. **비교 가능한 ρ 가 없다는 것 자체가 이 책의 발견 중 하나이며, 그것을 발견으로 부를 수 있는 이유는 정의를 먼저 고정했기 때문이다.**

(state, update, cost) 정리

표 7-1 — 이 장의 개념을 세 층 프레임에 배치한다. 네 방법 모두 갱신 층은 Θ 하나이고 W 와 E 는 비어 있다. 읽기식 (R) 은 전부 $\hat{y} = f(q; \Theta)$ 로 붕괴한다.

개념	라운드 사이에 남는 상태	(U- Θ) 의 무엇을 바꾸는가	라운드당 sleep 비용 B_s 의 성격	서빙 비용 L_w
rehearsal	실제 예제 buffer $\mathcal{B}$ (모델 밖 byte)	$\mathcal{R}_k = \mathcal{D}_k \cup \mathcal{B}$	섞어 넣은 예제수 만큼의 추가 FLOP	변화 없음
pseudo-rehearsal / generative replay	$\Sigma_{k-1} = \langle \Theta_{k-1}^G, \Theta_{k-1} \rangle$, 크기 $O(\Theta + \Theta^G)$	$\mathcal{R}_k$ 를 식 (7-9) 로 생성	생성 job(decode-heavy) + 생성기 재훈련	변화 없음
EWC	anchor Θ^* 와 Fisher 대각 F , 합쳐 $3 \Theta $ [EWC § 3, lines 369-370]	$\mathcal{L}$ 에 데이터 없는 이차항 추가, 식 (7-3)	Fisher 추정 1 회 (Atari: minibatch 100 개 [EWC Table 2]) + step 당 $2 \Theta $ 원소 읽기	변화 없음
GEM	과제별 실제 예제 buffer $\mathcal{B}_j$, 예산 M (실험값 5120 [GEM App. A])	갱신 방향 을 사영, 식 (7-7)	step 마다 과거 gradient $k-1$ 개 + $(k-1)$ 변수 QP — 라운드 수에 선형	변화 없음
ρ	성능 행렬 Π (측정 산출물, 모델 상태 아님)	— (측정대상)	라운드마다 회귀 스위트 1 회 실행	—

표에서 읽어야 할 것은 두 가지다. 첫째, **네 방법의 L_w 칸이 전부 같다**. Θ -경로 방법의 선택은 서빙 지연으로는 구별되지 않으며, 이것이 E - W 경로와의 결정적 차이다 — E -경로는 검색이 prefill 토큰을 더하고 W -경로는 상태 갱신이 토큰당 일을 더한다 ($\rightarrow$ ch10, ch17). 둘째, B_s 칸의 **성격이 전부 다르다** — 데이터 볼륨 (rehearsal), 생성 FLOP (DGR), 메모리 대역폭 (EWC), 라운드 수 (GEM). 통일 회계 없이 “어느 쪽이 싸가” 를 물으면 답이 없다는 사실이 이 표 하나에 이미 들어 있다 ($\rightarrow$ ch25).

Worked micro-example

GEM 논문의 부록은 성능 행렬 Π 를 전부 인쇄한다. 20 개 과제 permuted MNIST 의 base-line(“single”, 아무 대응도 하지 않는 순차 학습) 행렬에서 왼쪽 위 3×3 을 그대로 가져온다 [GEM App. B.1.1]. **행렬의 값만 논문 수치이고, 아래 (a) – (f) 에서 그 값으로 뽑아내는 모든 수는 이 책의 예시 계산이다** — 논문이 보고한 값이 아니다.

	과제 1	과제 2	과제 3
라운드 1 후	0.7838	0.1264	0.0709
라운드 2 후	0.7374	0.7939	0.0981
라운드 3 후	0.6564	0.7196	0.7783

굵은 값이 anchor $\Pi_{i,i}$ 다. 아직 학습하지 않은 과제의 값이 0.07 – 0.13 에 머무는 것은 무작위 수준이다 — 각과제가 10 개 클래스이므로 ≈ 0.1 이다 [GEM § 4.1, lines 331-333].

(a) 라운드 2 의 ρ . 이미 학습한 과제는 1 번 하나다. 식 (7-11) 로

$$\rho_2 = \frac{0.7838 - 0.7374}{0.7838} = \frac{0.0464}{0.7838} = 0.0592.$$

라운드 하나가 과제 1 의 성능을 상대 5.92% 깎았다.

(b) 라운드 3 의 ρ . 이제 과제가 둘이다.

$$\rho_3 = \frac{1}{2} \left[\frac{0.7374 - 0.6564}{0.7838} + \frac{0.7939 - 0.7196}{0.7939} \right] = \frac{1}{2} [0.1033 + 0.0936] = 0.0985.$$

ρ 는 상수가 아니다. 5.92%에서 9.85%로 올랐다. 이 한 줄이 “ ρ 는 스칼라가 아니라곡선이고, 스칼라로 보고하려면 어느 라운드의 값인지를 함께 적어야 한다” 는 규칙의 근거다.

(c) 총계에서 라운드값을 복원하기 — 여기서 2 배가 어긋난다. 같은실행의 20 라운드 총계는 BWT = -0.1980 이다 [GEM App. B.1.1]. 이것을 라운드당 값으로 바꾸는 가장 흔한 계산은 과제 수로 나누는 것이다: $0.1980/19 = 0.0104$, 즉 1.04pp. 틀렸다. BWT 는 과제별 총낙폭의 평균인데, 과제 i 는 이후 $K - i$ 번의 라운드에 노출되므로 노출횟수가 과제마다 다르다. 라운드마다 절대 δ 씩 깎인다고 두면 과제 i 의 총 낙폭은 $\delta(K - i)$ 이고, 낙폭의 총합은 분모와무관하게

$$\sum_{i=1}^{K-1} \delta(K - i) = \delta \cdot \frac{K(K - 1)}{2} \implies \delta = \frac{2}{K(K - 1)} \sum_{i=1}^{K-1} (\Pi_{i,i} - \Pi_{K,i})$$

이다. 이 실행의 총합은 3.9610 이므로 $\delta = 2(3.9610)/380 = 0.0208$, 즉 2.08pp 다. 총합 대신 인쇄된 스칼라에서 출발하려면 § 07.4 의 둘째 단서를 먼저 적용해야 한다 — 인쇄값은 K 로나눈 것이므로 $\delta = 2|BWT_{\text{인쇄}}|/(K - 1) = 2(0.1980)/19$ 이고, 식 (7-8) 의 분모로 재계산한 0.2085 에서 출발하면 $\delta = 2(0.2085)/K = 2(0.2085)/20$ 이다. 둘 다 2.08pp 로같다. **분모를 틀리면 5%가 어긋나고, 노출 보정을 빠뜨리면 2 배가어긋난다.** 순진한 계산 1.04pp 와 2.08pp 의 비가정확히 2 인 것이 그 사실이다.

(d) 세 숫자를 나란히 놓는다. 같은 실행에 대해 — 순진한 총계나누기 1.04pp, 노출 보정 2.08pp, 그리고 (a) 의분자 즉 라운드 2 가 실제로 입힌 절대 낙폭 4.64pp. 교차 확인으로 과제 1 만 끝까지 따라가면 $0.7838 \rightarrow 0.4690$ 이니 19 라운드에 걸쳐 라운드당 1.66pp 다. 즉 열화는 앞쪽 라운드에 몰려 있고, 어느 정의를 쓰느냐에 따라 같은 실행의 “라운드당 망각” 이 4 배 넘게벌어진다. **정의를 밝히지 않은 ρ 는 숫자가 아니다.**

(e) 상대형으로 옮기고 외삽해 본다. 이 행렬의 대각 평균은 0.7999 이므로 (20 개 값 평균, 이 책의 계산) 절대 2.08pp 는 상대 $0.0208/0.7999 = 0.0261$, 약 2.6%/라운드다. 이 비율이 상수라면 $0.974^K = 0.5$ 에서반감기는 약 26 라운드다. 그런데 (b) 가 이미 상수 가정을 반증했다 — 반감기 26 은 측정이 아니라 요약이 만들어 낸 인공물이다. 이것이 이 책이 ρ 를 라운드별 곡선으로 정의하고 스칼라 요약을 파생값으로만 쓰는이유다.

(f) 다른 레짐과 비교해 본다. DGR의 자기복제 사슬은 98.81% → 98.56%, 네 세대에 걸쳐 -0.25pp였다 [DGR Table 1]. 세대당 상대 $0.25/4/98.81 = 0.00063$, 약 0.063%/라운드로 (e)의 2.6%보다 40배 넘게 작다 (두 비율 모두 이 책의 계산). 그러나 이 비교는 **방법 비교로 성립하지 않는다**: GEM 쪽은 20개의 서로 다른 과제를 순차 학습한 뒤 과거과제에서 켜 값이고, DGR 쪽은 과제가 하나뿐인 전달 사슬을 같은 테스트집합에서 켜 값이다 [DGR Table 1 caption]. 두 수를 나란히 인쇄할 수 있다는 것 자체가 함정이며, 그래서 § 07.6의 정의는 값뿐 아니라 **과제 집합 · 측정 시점 · 기준점**을 함께 보고하도록 요구한다.

요약

- Θ -경로만이 배포 아티팩트를 라운드마다 바꾸고, 그래서 catastrophic forgetting은 Θ -경로의 고유 제약이다. 망각은 식 (U- Θ)에서 $\mathcal{R}_k$ 가 과거를 담지 않을 때 갱신식이 정의하는 동작 그 자체다.
- 대응책은 갱신식의 어디를 건드리느냐로 넷으로 갈린다: rehearsal과 generative replay는 $\mathcal{R}_k$ 를, EWC는 $\mathcal{L}$ 을, GEM은 갱신 방향을 바꾼다. 네 방법 모두 서버 비용 L_w 는 변하지 않고 shared-weight batching도 온전하다.
- EWC는 과거 경험을 anchor Θ^* 와 Fisher 대각 F 두 벡터로 압축한다 (식 (7-3)). 비용은 step당 $2|\Theta|$ 원소 읽기의 대역폭세금이고 라운드 수와 무관하다. 다만 라운드가 쌓일 때 상태량이 $O(|\Theta|)$ 인지 $O(k|\Theta|)$ 인지를 논문 스스로 결정하지 않았다 [EWC § 2, lines 158-160].
- **EWC 논문에는 결과표가 없고 EWC의 정확도 수치가 본문 어디에도 없다.** 표 두 개는 모두 하이퍼파라미터 표이고, 유일한 정량 성능 진술은 baseline이 “1 미만”이라는 것이다 [EWC § 2.2, lines 281-283]. 이후 문헌의 모든 EWC 수치는 재구현에서 나온 값이다. 논문은 자기 근사의 절반도 스스로 반증한다 — Fisher 영공간 방향의 잡음이 역 Fisher 방향잡음만큼 성능을 떨어뜨리며, 논문은 이를 “파라미터 불확실성 과소평가”로 적는다 [EWC § 2.2, lines 314-319].
- GEM은 과거를 부등식 제약으로 유지해 양의 backward transfer를 허용하지만, step 비용이 라운드 수에 선형이고 실제 예제를 저장한다 — EWC의 용량 논거를 논박한 것이 아니라 다른 자원으로 옮긴 것이다. GEM이 이 장에 필요한 진짜 이유는 성능 행렬 Π 와 BWT라는 계기를 정의했기 때문이다. 그 계기조차 인쇄된 BWT가 식 (7-8)이 아니라 분모 K 로 계산되어 있다.
- DGR은 저장소를 생성기로 바꿔 상태량을 $O(|\Theta| + |\Theta^G|)$ 로 고정하지만 재귀가 이중이고 (solver도 생성기도 자기 전임자에게서 배운다), 그 표류를 측정할 자리는 과제가 하나뿐인 예비 실험 하나뿐이다. generative replay는 자기 상한 대조군을 이긴 적이 없고 비길 뿐이며, EWC는 baseline으로 한 번도 돌지 않았다.
- ρ 는 라운드 k 하나가 이미 학습한 과제들에 입힌 상대 손실의 평균으로 정의한다 (식 (7-11)). 기준점은 무작위 초기화가 아니라 그 과제를 막 마쳤을 때의 값이고, 시간 첨자는 토큰이 아니라 sleep 라운드다. — BWT는 이 값의 절대 · 누적 판본이며 둘은 자동 변환되지 않는다.
- 비용 4종 중 ρ 가 가장 자주 비어 있다. 채워진 경우조차 정의가 서로 달라 비교되지 않으며, 그 사실이 발견으로 성립하는 것은 정의를 먼저 고정했기 때문이다.

자가 점검체크리스트

- catastrophic forgetting을 식 (U- Θ)의 어느 항의 부재로 설명하고, rehearsal · generative replay · EWC · GEM이 각각 그 식의 어디를 고치는지 지목할 수 있다.
- 식 (7-1)이 왜 “과거 데이터를 버려도 된다”는 면허인지, 그리고 식 (7-2)의 Laplace 근사가 그 면허의 대가로 무엇을 버리는지 설명할 수 있다.
- EWC 논문이 실제로 보고한 수치가 무엇이고 무엇이 아닌지 말할 수 있다. 다른 문헌에서 “EWC = X%”를 보면 그 X의 출처를 의심할 수 있다.
- worked micro-example의 (a) – (d)를 다른 행렬 (예: 같은 부록의 GEM arm)로 다시 계산하고, 노출 보정이 순진한 총계 나누기의 약 2배가 되는 이유를 $\sum_{i < K} (K - i) = K(K - 1)/2$

- 에서 유도할 수 있다. 인쇄된 BWT를 쓸 때 분모가 K 인지 $K - 1$ 인지 먼저 확인할 수 있다.
- 어떤 논문이 “망각을 줄였다”고 주장할 때 (1) 어느 과제의 어느 지표를, (2) 어느 라운드에서, (3) 무엇에 대해 정규화해 했는지 세 질문으로 검증하고, 셋 중 하나라도 없으면 그 칸을 “논문에 없음”으로 적을 수 있다.
 - GEM의 step 비용이 라운드 수에 선형이라는 사실이 매일 도는 sleep 라운드 운영에서 무엇을 뜻하는지, EWC의 상수 비용과 대비해 말할 수 있다.
 - Rosetta 항목: “모델 재배포 $\leftrightarrow$ (U- Θ) 1 회” 행을 이 장의 어휘로 채울 수 있다 — 왜 sleep 라운드마다 새 배포 아티팩트가 생기는지, ρ 가 왜 릴리스마다 회귀 스위트에서 잡히는 성능 회귀와 같은 물건인지, 그리고 왜 네 방법 모두 L_w 칸은 비어 있는지.

다음 장으로

이 장은 ρ 를 썰 자를 만들었지만, 왜 오프라인에 그 일을 해야 하는지는 답하지 않았다. EWC와 DGR은 둘 다 sleep 국면을 갖지 않는다 — EWC의 consolidation은 과제 경계에서, 훈련 루프 안에서 일어나고, ‘sleep’이라는 단어는 논문 전체에서 인용된 참고문헌 제목 안에 딱 한 번 나온다 [EWC lines 480-481]. 그럼에도 두 논문 모두 신경과학을 자기논거의 척추로 삼는다. EWC는 시냅스 공고화를, DGR은 “hippocampus는 replay buffer보다 생성 모델에 더 잘 대응된다”는 한 문장을 [DGR §1, lines 63-65].

넘기는 것이 하나 더 있다. 이 장은 EWC · GEM · DGR을 개념의 원형으로 다루므로 세 논문의 실증적 결함 중 기재이해에 필요한 것만 실었다. 세 논문의 나머지 불리 사실 — EWC 쪽 7항, DGR 쪽 2항 — 은 논문 평가에 속하므로 **ch22(반증 축)가 받아 간다**. 이장에서 그 항목들이 보이지 않는 것은 누락이 아니라 이송이다.

다음 장은 그 은유의 출처를 정면으로 다룬다. 왜 학습 시스템이 둘이어야 하는가 (CLS 이론), 그리고 오프라인 국면이 실제로 무엇을 하는지에 대해 생물학 실험이 **보인 것과 보이지 않은 것**은 무엇인가. ch08은 이장이 만든 자를 들고 그 문헌으로 들어가서, 은유가 기제로 번역되는 지점과 번역되지 않는 지점을 갈라낸다.

ch08. CLS 이론과 생물학적 sleep — 은유의출처

이 장의 목표 — 이 장을 마치면 (1) Complementary Learning Systems 가 무엇을 주장하고 무엇을 주장하지 않는지 한 문단으로 말할 수 있고, (2) SWS와 REM 을 서로 다른 outer loss 를 가진 두 개의 오프라인라운드로 번역할 수 있으며, (3) 생물학적 replay 와 deep learning 의 replay 를 갈라 놓을 수 있고, (4) 이 책의 corpus 가 생물학에서 실제로이송한 것과 이름만 빌린 것을 구분해 말할 수 있다.

왜 필요한가 — Part II 의 두 장이 이 장을 직접 요구한다. ch21 Language Models Need Sleep(2606.03979) 은 한 절전체를 SWS/REM 분업과 hippocampus - neocortex 대화의 서술에쓰고 그 위에 자기 기제를 세운다 [LM Need Sleep § 2]. ch14 Sleep-time Compute(2504.13171) 은 제목과 본문에서 sleep 이라는 단어를 170 회 쓰면서 (vendored 원문 전문 계수, § 08.5) 생물학문헌을 한 편도 인용하지 않는다. 같은 이름 아래 두 논문이 서로 다른 것을하고 있다는 판정은 생물학 쪽 원본을 알아야만 내릴 수 있다. ch07 의 replay-interleaving 도 계보상 여기서 출발한다.

NM 과의 관계 — Neural Memory ch17 이 Google Sleep 논문의 lifecycle 을 다루면서 sleep 이라는 단어를 이미 썼지만 생물학 배경은 한문단으로 넘겼다. 그 배경을 이 장이 소유한다. inner/outer loop, backward pass, optimizer state 는 전제한다 (→ NM ch02). 이 장이더하는 것은 오프라인 위상이 무엇을 근거로 필요하다고주장되었는가의 원출처, 그리고 그 근거가 LLM 으로 이송될 때 어디서끊기어는가의 경계선이다.

08.1 왜 두 시스템인가 — Complementary Learning Systems

Complementary Learning Systems(CLS) 는 하나의 학습 시스템이 빠른개별 경험 기억과 느린 일반화를 동시에 감당할 수 없으므로 뇌가 두 개의상보적 시스템을 가진다고 설명하는 이론이다. McClelland, McNoughton, O' Reilly 가 1995 년 Psychological Review 에 낸 논문이 그 원전이다 (이책의 corpus 에서는 vendored 원문이 아니라 후속 논문들의 인용으로만접근한다 — 아래 모든 CLS 서술은 인용하는 쪽 논문에 귀속시킨다).

hippocampus/neocortex 분업 은 CLS 가 제시하는 그 두시스템의 역할 분배다. Deep Generative Replay 는 이렇게 요약한다: “The hippocampal system rapidly encodes recent experiences, and the memory trace that lasts for a short period is reactivated during sleep or conscious and unconscious recall. The memory is consolidated in the neocortex through the activation synchronized with multiple replays of the encoded experience” [DGR § 1, L50-59]. ch21 의 LM Need Sleep 은 같은 분업을 더 날카롭게 쓴다: “The hippocampus serves as a high-fidelity temporary storage system, capable of rapidly encoding specific daily experiences. In contrast, the neocortex is a vast, long-term repository better suited for the gradual learning of generalized rules and semantic knowledge from these experiences” [LM Need Sleep § 2].

이 분업의 요점은 속도가 아니라 **간섭**이다. 빠르게 쓰는 시스템은새 항목이 옛 항목을 덮어쓰는 것을 감수해야 하고, 느리게 쓰는 시스템은여러 경험에 걸친 통계를 뽑아내는 대신 한 번의 경험을

즉시 담지 못한다. 두 요구를 한 파라미터 집합에 얹으면 둘 다 망가진다. CLS 는 그 충돌을 아키텍처로 푼 것이다.

[해설] 이 책의 세 층으로 읽으면 hippocampus 는 E (external store) 자리에, neo-cortex 는 Θ (slow weights) 자리에 놓인다. 빠른 쓰기 · 낮은 간섭 · 제한된 용량은 E 의 성질이고, 느린 쓰기 · 높은 일반화 · 배포 아티팩트의 변경은 Θ 의 성질이다. **다만 이 대응은 이 책의 재구성이지 CLS의 주장이 아니다.** CLS 는 1995 년의 심리학 · 연결주의 이론이고 external store 도 slow weights 도 그 어휘에 없다. 이 장은 대응을 세우되 대응을 증거로 쓰지 않는다.

CLS 가 남긴 실용적 처방은 **interleaving** 이다. 여러 과제를 하나의 학습 집합에 섞어 제시하면 순차 학습이 일으키는 덮어쓰기를 피할 수 있다. PLOS 논문은 이 처방의 출처를 명시적으로 CLS 로 지목한다: “interleaved training was originally construed to be an approximation to what the brain may be doing during sleep to consolidate memories; spontaneously re-activating memories from multiple interfering tasks in an interleaved manner” [PLOS Sleep Introduction, L106-108].

여기서 이 책 전체를 지배하는 잔여물이 하나 남는다. 같은 논문이 곧바로 이어 쓴다: “explicit use of interleaved training, in contrast to memory consolidation during biological sleep, imposes the stringent constraint that the original training data be perpetually stored for later use and combined with new data to retrain the network” [PLOS Sleep Introduction, L109-113]. **interleaving 은 작동하지만 과거데이터의 영구 저장을 요구한다.** CLS 는 뇌가 그 저장 없이 같은 일을 한다고 말했고, interleaving 은 그 주장을 저장이 있는 형태로만 재현했다. 이 간극이 Θ -경로 전체의 문제의식이다.

[해설] systems 독자에게 이 잔여물은 익숙한 모양이다. 과거데이터의 영구 저장은 곧 per-user 또는 per-task 상태량이고, 식 (A) 의 분자에 들어가는 것이 아니라 C_{cap} 에 들어간다. interleaving 을 그대로 LLM 에 옮기면 사용자마다 원본 대화를 통째로 보관해야 한다는 뜻이 된다 — 이것은 성능 문제가 아니라 저장 · 프라이버시 · 수명 관리 문제다. 이후 모든 replay 계열이 하려는 일은 이 C_{cap} 을 줄이거나 없애는 것이다 (→ ch07).

08.2 sleep 의 두 상태 — SWS 와 REM

SWS(slow-wave sleep) 는 NREM sleep 의 가장 깊은 단계로, 동기화된 고진폭 · 저주파 신경 활동으로 특징지어지는 상태다. **REM(rapid eye movement) sleep** 은 각성 상태에 가까운 고주파 · 저진폭 뇌파로 특징지어지고 꿈과 가장 강하게 연관되는 상태다. 두 정의 모두 [LM Need Sleep §2] 의 서술을 따른다.

기능 분업에 대한 주장은 다음과 같이 귀속시킨다. [LM Need Sleep §2] 는 SWS 에 두 기능을 배정한다 — synaptic homeostasis(“globally downscale synaptic strengths to counteract the net increase in connectivity from waking experiences”) 와 memory consolidation(“the transformation of fragile, recent experiences into stable, long-term knowledge”). 그리고 consolidation 이 raw data 의 replay 가 아니라고 명시한다: “this transfer does not simply replay raw data; instead, it re-architects the knowledge acquired during waking hours, extracting abstractions and integrating them into a cohesive semantic network” [LM Need Sleep §2]. REM 에는 다른 기능을 배정한다 — “the selective strengthening of newly formed synapses and the integration of new information with pre-existing emotional and semantic networks” [LM Need Sleep §2].

[해설] 이 문단이 이 장에서 systems 독자에게 가장 값싼 이득이다. **SWS 와 REM 은 “sleep 라운드 하나” 가 아니라 목적함수가 다른 두 라운드다.** 프레임 어휘로 옮기면, 하나의 k 안에서 $\mathcal{L}$ 이 두 번 바뀐다 — 식 (U- Θ) 의 $\mathcal{L}(\mathcal{R}_k; \Theta_k)$ 에서 $\mathcal{L}$ 이 위상의 함수가 된

다. 그래서 생물학을 실제로 인용하는 논문들은 예외 없이 **두 개 이상의 sleep 목적함수**를 갖는다: ch21은 Knowledge Seeding과 Dreaming 두 단계를 (두 단계의 정의는 → ch21), § 08.4의 PAD는 NREM과 REM에 서로 다른 outer loss를 둔다. 반대로 sleep을 이름으로만 쓰는 논문은 오프라인 위상이 하나다. 오프라인 위상의 개수가 은유를 실제로 이송했는지 아닌지의 값싼 판별기다.

한 가지를 이 장에서 못 박는다. SWS/REM 분업 서술의 근거는 신경과학 문헌이고, 이 책의 corpus는 그 문헌을 직접 갖고 있지 않다. corpus가 가진 것은 두 편의 **계산 모델**이다 — PLOS 논문 (§ 08.3)과 PAD (§ 08.4). 둘 다 동물 실험이 아니라 시뮬레이션이며, 생물학적 권위는 인용으로만 확보된다. PLOS 논문은 자기가 모델링한 것이 REM 뿐임을 밝힌다: “In vivo, activity of the neocortical neurons during REM sleep is low-synchronized and similar to baseline awake activity. Therefore, to simulate REM sleep-like activity in the model, the rewarded STDP rule was replaced by unsupervised STDP” [PLOS Sleep Results, L292-296]. **NREM은 이 논문에서 모델링되지 않는다.** 그런데 이 논문이 인용하는 declarative memory consolidation 문헌은 NREM에 그 기능을 배정한다. 즉 corpus 안에서 SWS 쪽 증거는 계산 모델로 뒷받침되지 않은 채 인용으로만 흘러간다.

08.3 생물학적 replay — 데이터 없이 다시 도는것

생물학적 replay는 각성 중 활동했던 신경 양상들이 오프라인 상태에서 저장된 감각 입력 없이 자발적으로 재활성화되어 시냅스를 변화시키는 현상이다. 이 정의에서 결정적인 부분은 “저장된 입력 없이”다. deep learning의 replay는 같은 이름을 쓰지만 다른 것을 가리킨다. PLOS 논문이 두 용법을 직접 갈라 놓는다 — deep learning 쪽 replay를 “storing a subset of previous veridical inputs and mixing them with more recent inputs to update the networks”로 규정하고, 자기 기여를 “the data intensive process of storing old data and using them for retraining can be avoided”로 규정한다 [PLOS Sleep Discussion, L671-677].

[평가] 이 구분은 용어 위생의 문제가 아니라 이 책의 판정에 직접 들어간다. **“replay를 한다”는 서술은 층도 데이터 출처도 특정하지 않으므로 그 자체로는 정보가 없다.** 어느 층에 쓰는지 (Θ 인가 E 인가), $\mathcal{R}_k$ 를 어디서 얻는지 (저장된 원본인가, 생성 모델인가, 모델 자신의 자발 활동인가)를 밝히지 않은 replay 주장은 이 책에서 미완성 서술로 취급한다.

08.3.1 PLOS 실험이 실제로 보인것

corpus에서 오프라인 위상이 각성 위상이 만들 수 없는 가중치 상태를 만든다는 것을 가장 직접적으로 보인 것은 Golden·Delanois·Sanda·Bazhenov의 PLOS Computational Biology 논문 (2022, doi 10.1371/journal.pcbi.1010628)이다. 이 논문은 수식에 번호를 붙이지 않으므로, 아래 인용은 절 이름과 vendored 원문 줄번호를 병기한다.

설정은 작다. 842개의 spiking 뉴런, 7×7 입력층 · 28×28 은닉층 · 3×3 출력층, 연속적으로 학습되는 시냅스는 은닉 → 출력의 흥분성 6272개뿐이다 [PLOS Sleep Methods: Network structure, L743-755; L995]. 과제는 두 개의 상보적 foraging 과제이고 성능 지표는 pattern discriminability로 chance가 0.5다 [PLOS Sleep Results, L166-167]. 모든 수치는 최소 10회 시행의 평균 $\pm$ 표준편차다 [L168-169].

네 조건의 결과가 이 장의 뼈대다.

표 8-1. PLOS Sleep의 핵심 네 조건 [Results, L228-229, L275-276, L282, L304-305].

조건	Task 1	Task 2	과거 데이터 저장
Task 1 단독 학습 후	0.70 ± 0.02	0.53 ± 0.02	—
Sequential (T1 → T2)	0.52 ± 0.02	0.69 ± 0.03	불필요
Interleaved _{T1,T2} (고전 interleaving)	0.68 ± 0.03	0.65 ± 0.04	필요
Interleaved _{S,T2} (sleep 교대)	0.70 ± 0.03	0.68 ± 0.05	불필요

Sequential 학습은 옛 과제를 chance 로 떨어뜨린다 ($0.70 \rightarrow 0.52$). sleep 을 새 과제 학습 과 100 movement cycle 단위로 교대시키면 옛 과제가 전혀 떨어지지 않는다 ($0.70 \rightarrow 0.70$). 그리고 결정적으로, “Importantly, no training on Task 1 was performed at any time during Interleaved_{S,T2}” [PLOS Sleep Results, L303-304]. 옛 과제데이터는 한 번도 다시 제시되지 않았다.

기제에 대한 논문 자신의 설명은 다음과 같다: “Having a sleep phase after a short period of Task 2 training enables spontaneous forward replay between hidden and output layers (H->O) that preferentially benefits the strongest synapses. Thus, if Task 1 synapses are still strong enough to maintain replay, they are replayed and weights are increased” [PLOS Sleep Results, L307-311]. 각성 중에는 입력층이 현재과제의 통계를 은닉층에 강제하므로 형성되는 상관은 새 과제의 것이다. 입력을 끊고 무구조 noise 로 대체해야만 **가중치 행렬 자신**이 어떤상관을 되살릴지 고를 수 있다. 그 선택은 가중치만의 성질이고, 그래서 각성절차로는 재현되지 않는다.

이 지점에서 계보상의 공백을 하나 기록한다. 저장 없이 $\mathcal{R}_k$ 를만든다는 목표에는 두 개의 독립된 해법이 있다 — 생성 모델을 따로 학습시켜과거 분포를 그리게 하는 것 (Deep Generative Replay, → ch07) 과, 모델자신을 noise 로 돌려 나오는 활동을 그대로 쓰는 것 (이 논문) 이다. PLOS Sleep 은 **DGR 을 인용하지 않는다**. 참고문헌 77 편은 대부분신경과학 · 심리학 · spiking network 모델링과 고전연결주의 망각 문헌이고, 기계학습 쪽 인용은 EWC [Results, L546-547], deep learning replay 리뷰와 지속학습 문헌 [Introduction, L102-105; Discussion, L673-674], meta-learning 한 건 [Discussion, L647-648], 그리고 같은 그룹이 이 기제를 feedforward ANN 에 옮긴 자기 인용 [Discussion, L677-679] 에 한정된다.

[평가] 같은 목표의 두 해법이 서로를 보지 않고 나란히자랐다. 이 침묵은 연대가 강제한 것이 아니다 — DGR 은 2017 년이고 이 논문은 2022 년이다. 인용이 가능했는데 이루어지지 않은 자리이므로 이 책은 이것을계보 사실이 아니라 발견으로 취급하며, Part II 의 경로 간 침묵 지도에서되풀이되는 형태의 첫 사례로 둔다 (→ ch11).

sleep 을 정적 보호와 구분하는 통제도 있다. Task 1 학습 후 상위 x% 시냅스를 열려 두는 baseline 은 1%에서 $0.54/0.68$, 5%에서 $0.65/0.61$, 10%에서 $0.70/0.53$ 을 낸다 [PLOS Sleep S6 Fig, L1181-1189]. 세 점 모두 두 축을 맞바꾸는 곡선 위에 있고, sleep 의 $0.70/0.68$ 은 세 점 전부를 두축에서 동시에 지배한다. 논문은 이 baseline 을 EWC 와 같은 계열로명시적으로 연결한다 [PLOS Sleep Results, L517-547] — 다만 EWC 자체는실행하지 않는다 (→ ch07).

08.3.2 이 실험에 불리한 사실 — 반드시 함께 읽어야 하는것

강한 baseline 이 방법과 거의 같다. 표 8-1 에서 Interleaved_{T1,T2} 는 $0.68/0.65$ 이고 sleep 은 $0.70/0.68$ 이다. 차이는 0.02 와 0.03 이며 표준편차는 $0.03 - 0.05$ 다. 논문은 본문에서 sleep 이 고전 interleaving 을 “exceeding” 한다고 쓰지만 [PLOS Sleep Results, L305-306], Discussion 에서 스스로 정정한다: “Although classical interleaved training of the old and new tasks showed similar performance results in our model as interleaving new task training with sleep, **we believe the latter to be superior on the following theoretical grounds**” [PLOS Sleep Discussion, L654-655, 강조는 이책]. 이어지는 근거는 전부 이론적이다 — 고전

interleaving 은 각 주기마다 단일 과제 비용함수를 쓰므로 이중 과제 최적점 주위에서 진동하는 반면 sleep 은 두 과제가 섞인 replay 를 지원할 수 있다는 것 [L655-670]. 그런데 replay 내용을 실제로 분석한 실험은 이 논문에 없다. 혼합 replay 주장은 인용으로 수입된다.

[평가] 이 논문이 실증한 우위는 정확도가 아니라 **저장의 제거**다. 과거 데이터가 이미 있다면 이 논문은 sleep 을 택할정확도상의 이유를 하나도 제공하지 않는다. Θ -경로를 옹호할 때 이 논문을 근거로 들 수 있는 것은 “가중치만으로 $\mathcal{R}_k$ 를 만들 수 있다” 까지이고, “그렇게 만든 $\mathcal{R}_k$ 가 저장된 데이터보다 낫다” 까지는 아니다. 이 두 문장을 붙여 쓰는 서술은 이 책에서 결함이다.

나머지 제약도 함께 기록한다. 이 논문에는 **어떤 통계 검정도 없다**— 모든 수치가 10 회 이상 시행의 평균 $\pm$ 표준편차이고, “significantly” 라는 단어를 최소 세 번 쓰면서 검정 이름도 p-값도 제시하지 않는다 [L168-169; L418, L444, L506]. 과제는 **두 개뿐이며** 같은 네 방향 어휘에서 뽑은 대칭 상보 과제다 — 논문 자신이 유사성을 특징으로 강조한다 [L233-237]. 따라서 반복 sleep 라운드에 걸친 열화를 ρ 는 **측정되지 않았다**: 세 번째 과제도, 두 번째 Interleaved 위상도 실행되지 않았다. 보호되는 성능 밴드는 chance 0.5 에서 천장 0.70 까지 **0.20** 폭이고 조건별 표준편차가 0.02 – 0.05 로 밴드의 10 – 25%다. 과제·환경·평가 지표가 모두 저자 제작이며 표준지속학습 벤치마크는 하나도 쓰이지 않았다 [L166-167, L1037-1095]. 그리고 sleep 은 **복원하지 않고 보존한다** — “a single episode of new task training using reinforcement learning could quickly erase old memories to the point that they cannot be recovered by subsequent sleep” [PLOS Sleep Discussion, L639-640]. 각성 손상이 아직 부분적인 창안에서만 오프라인 위상이 각성 위상이 못 하는 일을 한다.

마지막으로 비용이다. 이 논문은 **FLOPs·wall-clock·메모리사용량·에너지를 하나도 보고하지 않는다**. B_s 는 계산량이 아니라 시뮬레이션 시간으로만 존재한다 — 한 sleep 구간은 100 movement cycle 이고 이는 직전 각성 구간과 정확히 같아 duty ratio 가 1:1 이다 [PLOS Sleep Results, L301-303; Methods: Simulated sleep, L985-986]. sleep 길이·비율·교대 횟수에 대한 sweep 은 없다. ch01 §01.5 가 정한 회계 규칙대로, 비용 4 종이 비어 있으므로 이 논문을 근거로 “효율적” 이라고 쓸 수 없다.

[해설] 그럼에도 systems 독자가 이 논문에서 가져갈 구조적 사실이 하나 있다. sleep 위상을 구동하는 데 필요한 각성 유래 상태는 은닉뉴런 784 개의 목표 발화를 스칼라인데, Uniform-Noise Sleep 통제에서 이것을 **집단 평균 하나로 축약해도 결과가 유지된다** ($0.67 \pm 0.05 / 0.69 \pm 0.03$) [PLOS Sleep Results, L355-362; S4E/F, L1144-1150]. 즉이 모델에서 각성→sleep 경계를 넘는 상태는 $O(1)$ 스칼라 하나로 줄어든다. 이것이 이 논문에서 나오는 **유일한** 용량 주장이고, gradient 로 학습되는 모델로의 이송은 이 논문이 보이지 않았다.

08.4 dreaming 과 표현학습 — 재생이 아니라 생성

dreaming 과 표현학습은 오프라인 위상이 과거 경험을 재생하는 대신 **경험한 적 없는 입력을 생성**하고 그 생성물로 학습함으로써 표현의질을 높인다는 가설이다. §08.3 의 replay 가 “있었던 것을 다시 돌린다” 였다면이쪽은 “없었던 것을 만들어 돌린다” 다. Deperrois·Petrovici·Senn·Jordan 의 eLife 논문 Learning cortical representations through perturbed and adversarial dreaming(2022, doi 10.7554/eLife.76384, 이하 PAD) 이 이 가설의 계산 모델이다.

출발점은 관찰 하나다. Wake – Sleep·Helmholtz machine 계열은 오프라인 위상을 생성 모델 개선에 쓰지 않고 각성 중 입력 재구성에만 학습시키는데, “most dreams during REM sleep exhibit realistic imagery beyond past sensory experience” [PAD Introduction, L84-86]. 꿈은 겪은 것의 재생이 아니다. PAD 는 그 사실을 잡음이 아니라 **계산의원재료**로 삼는다.

08.4.1 한 사이클에 세 개의 outer loss

PAD의 한 wake-sleep 사이클 k 는 세 위상으로 구성되고 위상마다 목적함수가 다르다. 논문 자신의 표현으로 “the three brain states only differ in their objective function and the presence or absence of external input” [PAD Results, L275-276].

- **Wake:** 실제 입력 x_k 를 encoder에 통과시켜 latent을 얻고, 그것을 두 슬롯짜리 hippocampal buffer에 쓴다. 동시에 재구성 손실 + latent prior 정규화 + discriminator의 “real” 향으로 파라미터를 갱신한다 [PAD Eq. 1-5].
- **NREM:** buffer의 latent을 generator로 되그려가림 (occlusion)을 씌운 뒤, 그 훼손된 이미지를 다시 encode한 결과가 원래 latent과 같아지도록 encoder만 갱신한다 [PAD Eq. 6]. 저장된 latent이 정답이고 generator가 입력을 만든다.
- **REM:** 현재와 직전 사이클의 latent 두 개와 가우시안 noise를 볼록 결합해 $z' = \frac{1}{4}z + \frac{1}{4}z_{\text{old}} + \frac{1}{2}\epsilon$ 을 만들고, 그것을 generator로 그린 뒤 discriminator에게 “internal”로 판정하도록 학습시키면서 **generator에는 부호를 뒤집어 상승방향으로** 갱신한다 [PAD Eq. 7-8, Figure 2c]. 논문의 핵심 생물학적 예측이 이 부호 전환이다: “identical local errors lead to opposing weight changes between Wake and REM sleep” [PAD Figure 7 caption].

[해설] 프레임으로 옮기면 PAD는 (U-E)를 한 번, (U- Θ)를 세 번 도는 시스템이다. 그런데 (U-E)의 $S(c_k; B_s)$ 는 encoder forward 한 번이고 w_r 은 2 슬롯 shift register이며, $\text{ret}(E, q)$ 가 존재하지 않는다 — 읽기는 질의가 아니라 라운드 첨자로만 이루어진다. 즉 PAD의 E 는 검색 절반이 잘려 나간 store로, 오직 $\mathcal{R}_k$ 를 만들기 위해서만 존재한다. 그리고 평가시점에는 $\hat{y} = \Theta^{(\text{ro})} \text{enc}(x)$ 한 번의 forward가 전부다 — W 층이 없고 E 도 읽지 않는다. **오프라인 위상이 벌어들인 것이 전부 Θ 에 흡수되어 있으므로 wake 비용이 정확히 0 만큼 늘어난다.** 이것이 상각 논증이 가장 좋아하는 모양이지만, 논문은 어느 쪽 비용도 재지 않는다.

[평가] PAD가 자기 기여로 내세우는 것은 부호 전환이라는 생물학적 예측이다 — 각성과 REM에서 동일한 국소 오차가 반대 방향의 가중치변화를 낳는다는 것. 계산 쪽에서 보면 이것은 한 파라미터 블록에 대한 gradient의 부호를 뒤집는 것, 즉 표준 GAN 학습 절차 그 자체다. 새 연산도, 새 커널도, 추가 상태도 없다. 이 사실을 이 책은 약점이 아니라 **이송 가능성의 지표**로 읽는다: 생물학 쪽에서 “각성과 sleep은 다른 종류의 가소성을 쓴다”로 서술되는 것이 계산 쪽에서는 “오프라인 라운드는 목적함수와 부호만 다르다”로 환원된다. 반대로 말하면, 이 라인이 생물학에서 가져올 수 있는 것은 대개 **스케줄과 목적함수의 배치**이지 새로운 갱신 primitive가 아니다. ch21의 두 단계 sleep이 기존 distillation과 RL 조각의 재배치로 구성되는 것도 같은 이유다 (→ ch04, ch21).

이 논문의 기호는 이 책의 예약 기호와 정면 충돌한다. 원문을 대조해 읽을 독자를 위해 세 행만 못 박는다.

표 8-2. PAD 표기 대응 — 예약 기호를 덮어쓰는 두 충돌과 시간 단위 충돌.

원 논문 기호	원문에서의 뜻	이 책 표기
E, E_z, E_d	encoder 신경망과 그 두 출력 [PAD Methods: Network architecture]	$\Theta^{\text{enc}}, \Theta^{\text{disc}}$ — 예약된 $E(\text{external store})$ 는 논문이 이름을 주지 않은 hippocampal buffer 에 배정
W	선형 readout 가중치 $W \in \mathbb{R}^{10 \times 256}$ [PAD Eq. 10]	$\Theta^{(\text{ro})}$ — 모델이 열려진 뒤 학습되는 평가용 probe 이며, 예약 기호 $W(\text{fast weights})$ 와 무관하다

원 논문 기호	원문에서의 뜻	이 책 표기
epoch	데이터셋 1 회 통과 [PAD Methods]	한 wake – sleep 사이클 k 는 미니배치 하나 다 — 이 모델에서는 sleep 라운드 k 와 optimizer step s 가 1:1 로 겹친다. 논문 그림의 x 축은 epoch 이므로 k 의 수는 그보다 훨씬 많다

08.4.2 무엇이 측정되었고 무엇이손봐졌는가

평가는 표현의 **선형 분리도**다 — 모델을 열린 뒤 latent 위에 선형분류기를 학습시켜 얻은 테스트 정확도로 정의한다 [PAD Methods: Linear separability]. epoch 50 시점, 초기값 4 종에 대한 평균 $\pm$ SEM 은 다음과 같다 [PAD Appendix 1—table 1].

표 8-3. PAD 와 병리 조건의 최종 선형 분리도 [PAD Appendix 1—table 1].

데이터셋	PAD	w/o memory mix	w/o REM	w/o NREM	Wake only
CIFAR-10	58.25 $\pm$ 0.70	53.87 $\pm$ 0.85	46.00 $\pm$ 0.43	58.00 $\pm$ 0.34	42.25 $\pm$ 0.54
SVHN	78.92 $\pm$ 0.40	60.87 $\pm$ 5.07	42.30 $\pm$ 1.51	73.25 $\pm$ 0.22	41.93 $\pm$ 0.65

REM 을 제거하면 두 데이터셋 모두 크게 무너진다 (58.25 $\rightarrow$ 46.00, 78.92 $\rightarrow$ 42.30). REM 을 두되 여러 기억의 혼합도 noise 도없이 단일 기억만 재생하면 역시 떨어진다 (53.87, 60.87) [PAD Results, L547-549]. 논문의 결론은 여기서나온다 — 표현을 만드는 것은 오프라인이라는 사실 자체가 아니라 **오프라인에서 무엇을 생성하는가**다.

[평가] 표를 그대로 읽으면 논문의 두 번째 주장은 훨씬약하다. **CIFAR-10 에서 w/o NREM 은 58.00 $\pm$ 0.34 로 PAD 의 58.25 $\pm$ 0.70 과 오차 안에서 같다** [PAD Appendix 1—table 1]. NREM 의 기여는 깨끗한이미지의 선형 분리도가 아니라 가림이 있을 때의 견고성으로만 나타난다 [PAD Results, L571-577]. 논문은 본문에서 이 구분을 지키지만, 표 없이요약만 읽으면 “네 구성요소가 모두 기여한다” 로 읽히기 쉽다.

더 무거운 사실이 Methods 에 있다. **절제 대조군은 손봐진 상태로비교된다**. REM 을 제거한 모델은 “we observed a decrease of linear separability after some (>25) epochs” 라는 이유로 encoder 출력에 $\epsilon \sim \mathcal{N}(0, 0.5I)$ 를 더한 변형 재구성손실로 학습되고 [PAD Eq. 12], 나아가 “we reduced the effect of NREM by scaling down its loss with a factor of 0.5” 라는 조정까지 받는다 [PAD Methods: Modifications specific to pathological models]. 논문은이 조정을 공정 비교를 위한 것이라고 밝히며 숨기지 않는다.

[평가] 조정을 밝힌 것은 정직하다. 그러나 결과의 지위는달라진다. 표 8-3 의 w/o REM 열은 “REM 을 뺀 PAD” 가 아니라 “**REM 을 빼고 저자가 손본 다른 모델**” 이다. 이 책이 절제 수치를인용할 때는 그 사실을 함께 옮긴다. 그리고 PLOS 논문과 마찬가지로 PAD 에도**검정 이름이나 p-값이 없다** — 본문은 “significantly” 를 반복해서쓰지만 vendored 원문 어디에도 검정이 없다. 두 편의 생물학 논문 모두, 이라인의 근거로 인용되는 바로 그 논문들이, 유의성 주장을 검정 없이 한다.

저자 자신의 통제가 논문 제목의 성분을 지지하지 않는다. PAD 의 REM 은 혼합된 episodic memory 와 자발 활동을 함께 주입해 구동되는데, 저자는 그 혼합이 필요 없다고 보고한다: “we do not observe significant differences between using a combination of episodic memories

with spontaneous activity or only using spontaneous activity” [PAD Discussion, L1034-1036; Appendix 1—figure 4]. 표 8-3의 w/o memory mix는 혼합만 없앤 조건이 아니라 **noise** 까지 없애고 단일 기억만 되돌린 조건이다 [PAD Results, L547-549]. 두 결과를 함께 읽으면 REM에서 결정적인 성분은 기억의 혼합이 아니라 자발 활동의 주입이다. 논문 제목이 가리키는 성분 (mixed episodes로부터의 dreaming)은 통제 대비 효과를 보이지 않았다.

단계 순서도 무관하다. “our model does not show significant differences in performance when the order of sleep phases is switched” [PAD Discussion, L1046-1047; Appendix 1—figure 5]. 저자는 생물학쪽에서 NREM → REM 순서가 중요하다고 가설된다는 점 (sequential hypothesis)을 인정하면서, 모델의 순서 독립성을 단계별 시냅스 변화가 작기 때문으로 돌린다 [PAD Discussion, L1050-1055].

표현 품질의 지표는 하나뿐이다. 측정된 것은 linear readout 정확도 하나이고, 저자는 이것을 “an obvious simplification with regard to cortical processing”이라고 인정한다 [PAD Discussion, L1039-1041]. downstream 과제 성능은 측정되지 않았다. 그 하나뿐인 지표의 절대 값도 낮다—CIFAR-10에서 약 59%다 [PAD Results, L534; Figure 4c].

[평가] 세 사실을 합치면 이 논문을 인용할 수 있는 범위가 좁아진다. PAD는 **기제의 존재 증명**이지 성능 주장이 아니다. 절대성능을 근거로 이 라인의 유망함을 말하는 데 이 논문을 쓰는 것은 오용이다. 그리고 sleep 단계를 개수와 순서까지 모사하려는 LLM 설계에 대해 이 논문은 순서의 필요성을 지지하지 않는다. § 08.2에서 오프라인 위상의 개수를 판별기로 쓴 것은 은유가 실제로 이송되었는지를 재는 지표이지, 개수와 순서가 성능을 낳는다는 주장이 아니다.

비용은 다시 비어 있다. PAD는 FLOPs·GPU 시간·wall-clock·메모리 사용량을 어느 것도 보고하지 않는다. C_{cap} 은 구조적으로 2 슬롯으로 고정되고 보존 기간은 정확히 한 사이클이다 — $z_{old} \leftarrow z$ 가 매 사이클 덮어쓴다 [PAD Algorithm 1]. 누적되는 것은 아무것도 없다. ρ 에 가장 가까운 관측은 방향성뿐이다: NREM이 없는 조건에서 “linear separability tends to decrease after many training epochs” [PAD Appendix 1, ‘Linear classification performance’] — 즉 오프라인 위상 하나를 빼면 반복라운드가 성능을 **깎기 시작한다**.

08.5 은유의 이송한계

은유의 이송 한계란, 생물학적 sleep 서술에서 계산 시스템으로 실제로 옮겨지는 구조적 주장과, 이름만 옮겨지고 근거는 남겨진 부분 사이의 경계를 말한다. 이 절이 이 장의 결론이다.

먼저 무엇이 이송되는지 정리한다. § 08.3과 § 08.4가 지지하는 것은 다음 네 문장이며, 각각 **적어도 하나의 계산 모델에서 실증되었다**.

표 8-4. 생물학 두 편에서 이송 가능한 구조적 주장.

이송되는 구조적 주장	근거
식 $(U-\Theta)$ 의 $\mathcal{R}_k$ 는 데이터셋일 필요가 없다. 이 설정에서는 모델을 noise로 돌려 만든 스트림으로 충분했고, $\mathcal{H}_k$ 는 Θ_k 안에만 있어도 된다	PLOS Sleep Results, L300-311
각성→오프라인 경계를 넘는 보조 상태는 $O(1)$ 스칼라 하나로 줄어든다	PLOS Sleep S4E/F, L1144-1150
오프라인 위상의 가치는 “오프라인”이라는 스케줄이 아니라 거기서무엇을 생성하는가 에서 나온다. 단 PAD에서 결정적인 성분은 기억의 혼합이 아니라 자발 활동의 주입이다	PAD Appendix 1—table 1 (w/o memory mix 53.87 vs PAD 58.25) + Appendix 1—figure 4(혼합 유무는 차이 없음)

이송되는 구조적 주장	근거
오프라인 위상이 하나가 아니라 목적함수가 다른 둘일 때 서로 다른성질 (정확도 / 섭동 견고성) 이 갈라져 나온다	PAD Results, L534-577; Appendix 1—table 1

셋째 행은 이 표에서 유일하게 자기 근거와 긴장 관계에 있으므로 그자리에서 긴장을 밝힌다. 이 행이 이송하는 주장은 PAD의 절제결과 (w/o memory mix 53.87 vs PAD 58.25)에서 나오는데, 같은 저자가 통제에서 “we do not observe significant differences between using a combination of episodic memories with spontaneous activity or only using spontaneous activity” 라고 보고한다 [PAD Discussion, L1034-1036; Appendix 1—figure 4].

[평가] 두 결과를 함께 놓으면 이송되는 것은 “오프라인에서 무엇을 생성하는가가 중요하다”의 약한 형태뿐이다 — 생성물이 자발 활동으로 넓게 퍼져야 한다는 것까지다. 논문 제목이 가리키는 강한 형태 (혼합된 에피소드로부터의 dreaming)는 저자 자신의 통제에서 효과를 보이지 않았으므로 이송되지 않는다. 표 8-4의 나머지 세행과 달리 이 행은 그 절단선을 달고서만 인용할 수 있다.

이송되지 않는 것은 더 길다. PLOS 모델에는 **loss 도 gradient 도 optimizer 도 backward pass 도 없다** — 갱신은 국소 eligibility trace를 낀 Hebbian 곱과 열 합을 고정하는 정규화 사영이고, 학습률 η_Θ 는 독립된 존재조차 아니다 [PLOS Sleep Methods: Synaptic plasticity, L861-963]. 그 기제는 transformer가 갖지 않은 장치에 의존한다: spike timing이라는 상관 신호, 뉴런별 목표 발화율 항상성, 열 합을 보존하는 heterosynaptic 정규화, 균형 잡힌 전방 억제. 논문이 sleep의 우위를 설명할때 기대는 것이 바로 이 정규화 제약이다 [L509-517]. 제약을 빼면 그설명에는 대체물이 없다. 규모도 이송되지 않는다 — 연속적으로 가소적인시냅스 6272개, 과제 2개, 밴드 0.20이다. PAD는 gradient를 쓰지만 언어도 토큰도 attention도 없고, 저장은 2슬롯이며, 두 논문 모두 **replay·dream의 내용이 의미상 옳은지를 한번도 검사하지 않는다**. LLM에서 이 검사 누락에 대응하는 실패 — 유창하지만거짓인 생성물로 Θ 를 갱신하는 것 — 이 Θ -경로의 중심 위험이며 (→ ch06), 생물학 쪽 두 편은 그것에 대해 아무 말도 하지 않는다.

이송의 반대편, 즉 이름만 옮겨 간 쪽의 표본은 E-경로의 창시 논문이다. Sleep-time Compute(2504.13171)은 sleep이라는단어를 170회 쓰지만 (vendored 원문 전문, 대소문자 무시 계수) 그 단어를도입하는 문장은 순전히 운영적이다: “inference is done between interactions with the model while it would otherwise be idle in sleep-time” [Letta STC § 1]. **여기서 sleep은 consolidation 이아니라 유희를 뜻한다**. 반대쪽 끝에는 Θ -경로의 Language Models Need Sleep(2606.03979)이 있다 — 한절 전체를 SWS/REM 분업과 hippocampus-neocortex 대화에 쓰고자기 기제를 그 위에 얹는다 [LM Need Sleep § 2]. 두 논문의 참고문헌을전수 대조해 어느 침묵이 연대가 강제한 것이고 어느 것이 선택된 것인지등급을 매기는 일은 이 장의 몫이 아니다 — 그 판정은 생물학 두 편을정면으로 재독하는 ch23이 소유한다 (→ ch23 § 23.8).

[해설] Rosetta로 옮기면 차이가 즉시 드러난다. Sleep-time Compute의 오프라인 위상은 **warm-up 또는 사전컴파일**과 같은 자리에 있다 — 유희 시간에 미리 해 두고, 배포 아티팩트는그대로다. 반면 CLS·PLOS·PAD가 말하는 consolidation은 **모델 재배포 1회**와 같은 자리에 있다 — 끝나고나면 서빙되는 가중치가 다른 것이 된다. 이 둘은 상각 구조도, 배칭가능성도, 롤백 절차도 같지 않다. 앞의 것은 캐시 무효화 문제이고 뒤의것은 배포 문제다. 같은 단어 하나가 이 구분을 덮고 있다.

(state, update, cost) 정리

이 장의 개념을 세 층 프레임에 배치한다. 비용 칸은 **논문이 실제로보고한 것만** 적는다.
표 8-5. 이 장의 개념과 세 층 프레임의 대응.

개념	이 책의 층	갱신식	시간척도	비용에 대해 알려진 것
hippocampus (CLS)	E 에 대응 (이 책의 재구성)	—	경험 단위	원전미확보. corpus 인용으로만 접근
neocortex (CLS)	Θ 에 대응 (이 책의 재구성)	(U- Θ)	다수 replay에 걸쳐	동일
interleaving 처방	Θ	(U- Θ), $\mathcal{R}_k =$ 저장된 원본	학습 라운드	$C_{\text{cap}} =$ 과거 데이터 전체 (정성적 서술)
SWS	Θ	(U- Θ), 목적 = homeostasis + consolidation	sleep 라운드	논문에 없음
REM	Θ	(U- Θ), 목적 = 선택적 강화 + 통합	sleep 라운드	논문에 없음
생물학적 replay	Θ	(U- Θ), $\mathcal{R}_k = \text{gen}(\Theta_k; B_s) -$ 저장 없음	sleep 라운드 k	B_s : 100 movement cycle, wake와 1:1. FLOPs·시간·바이트 전부 논문에 없음
PLOS sleep 위상	Θ 만 ($W = \emptyset$, $E = \emptyset$)	(U- Θ), gradient 아님	$k =$ 교대 1회	L_w 변화없음 (구조적). ρ 미측정 (과제 2개)
PAD Wake write	E (2 슬롯)	(U- E), $S(c; B_s) =$ forward 1회	사이클 k	$C_{\text{cap}} = 2$ latent 슬롯, 보존 1 사이클
PAD NREM	Θ^{enc}	(U- Θ), $\mathcal{R}_k =$ 저장 latent + 가림	사이클 k	논문에 없음
PAD REM	Θ^{disc} 하강 / Θ^{gen} 상승	(U- Θ), 부호 반전	사이클 k	논문에 없음

두 가지가 이 표에서 바로 읽힌다. 첫째, 이 장의 개념 중 W 층에 놓이는 것이 **하나도 없다**. 생물학 쪽 두 모델 모두 fast weights를 갖지 않는다 — 질의 시점에 움직이는 상태가 없고, 오프라인에서 번 것은 전부 Θ 에 흡수된다. W -경로는 생물학에서 오지 않았다 ($\rightarrow$ ch17). 둘째, 비용 칸의 대부분이 “논문에 없음”이다. **이 라인의 생물학적 뿌리는비용 회계를 하나도 물려주지 않았다**. 식 (A)의 어느 항도 이 장의논문들에서 조달할 수 없다.

Worked micro-example — 밴드 정규화 점수와 상각불능

PLOS Sleep의 네 조건을 손으로 다시 계산해, “sleep이 interleaving을 이겼다”는 문장이 얼마나 큰 주장인지 재어 본다. 필요한 수치는 표 8-1과 S6 뿐이고 계산은 나눗셈 네 번이다. 입력만 논문 수치이고 **이 절에서 나오는 유도값은 전부 이 책의 예시 계산이다** — 논문은 정규화 점수도, 밴드 환산도, 바이트 환산도 보고하지 않는다.

1 단계 — 밴드 정규화. 이 과제의 chance는 0.50이고 단일 과제 천장은 Task 1이 0.70, Task 2가 0.69다 [PLOS Sleep Results, L166-167, L228-229, L276]. 따라서 보호 대상 밴드는 각각 0.20과 0.19다. 정규화 점수를 $g(x) = (x - 0.50) / (\text{천장} - 0.50)$ 으로 두면 $g = 1$ 이 “단일 과제 학습과 같음”, $g = 0$ 이 “chance”다.

표 8-6. 밴드 정규화 점수 — 표 8-1 과 S6 의 논문 수치를 입력으로 이 책이 계산한 값이며, 논문이 보고한 값이 아니다.

조건	Task 1 g	Task 2 g	합
Sequential	$0.02/0.20 = 0.10$	$0.19/0.19 = 1.00$	1.10
Interleaved $_{T1,T2}$	$0.18/0.20 = 0.90$	$0.15/0.19 = 0.79$	1.69
Interleaved $_{S,T2}$	$0.20/0.20 = 1.00$	$0.18/0.19 = 0.95$	1.95
Freeze 5%	$0.15/0.20 = 0.75$	$0.11/0.19 = 0.58$	1.33

Sequential 은 옛 과제 밴드의 90%를 잃는다. sleep 은 0%를 잃는다. freezing 은 두 축을 맞춰 바꿔 1.33 에 머문다.

2 단계 — 표준편차를 같은 축으로 옮긴다. Interleaved $_{S,T2}$ 의 표준편차는 Task 1 이 0.03, Task 2 가 0.05 다. 같은 나눗셈을 적용하면 $0.03/0.20 = 0.15$ 밴드, $0.05/0.19 = 0.26$ 밴드다. 그런데 sleep 과 고전 interleaving 의 합 차이는 $1.95 - 1.69 = 0.26$ 밴드다. **두 방법의 격차가 한 조건의 1σ 와 같은 자릿수다.** 논문이 본문에서 “exceeding” 이라고 쓰고 Discussion 에서 우위를 이론적 근거로 돌린 이유가 이 한 줄에 들어 있다 [L305-306 vs L654-655].

3 단계 — 그러면 무엇이 교환되었는가. 두 방법의 차이는 정확도가 아니라 저장이다. Interleaved $_{T1,T2}$ 는 Task 1 환경에 대한 접근을 계속 유지해야 한다 [L282-286]. Interleaved $_{S,T2}$ 가 각성에서 sleep 으로 넘기는 것은 은닉 뉴런 784 개의 목표 발화율이 전부이고, Uniform-Noise 통제에서는 집단 평균 1 개로 줄어든다 [L982-985; L1144-1150]. **논문은 dtype 도 바이트도 밝히지 않으므로** 여기서부터는 이 책의 산수다: fp32(4 B/elem) 로 가정하면 $784 \times 4 \text{ B} = 3,136 \text{ B} \approx 3.1 \text{ KiB}$ 이고, 통제군은 4 B 다. 환경 전체와 3.1 KiB 사이의 비교는 자릿수 비교조차 되지 않는다.

4 단계 — 상각식에 넣어 본다. 식 (A) 는 $C_{\text{avg}} = C_{\text{wake}} + C_{\text{sleep}}/N_q$ 다. duty ratio 가 1:1 이므로 한 주기에서 $C_{\text{sleep}} = C_{\text{wake}}$ 다 [L301-303]. 그런데 이 논문에는 질의 스트림도, 문맥도, 사용자도 없어 N_q 가 정의되지 않는다. 정의하지 않은 채 $N_q = 1$ 로 놓으면 $C_{\text{avg}} = 2C_{\text{wake}}$ 다. 손익분기 N_q^* 도 구할 수 없다 — 구하려면 sleep 없이 같은 품질을 내는 wake 예산이 있어야 하는데 이 논문에 그 대안은 없다 (Sequential 은 품질이 다른 결과다).

[평가] 4 단계의 결론을 그대로 쓴다. 이 논문에서 sleep 은 총 계산을 두 배로 쓰고, wake 지연은 그대로 두고, 저장 바이트를 환경 전체에서 킬로바이트 이하로 바꾼다. 교환은 $\text{sleep} \leftrightarrow \text{wake}$ 가 아니라 $\text{계산} \leftrightarrow \text{저장}$ 이다. Part III 에서 세 경로를 비교할 때 이 축 배치 를 잊지 않는다 — 생물학적 조상은 상각 증거를 하나도 제공하지 않았고, 제공한 것은 상태 용량 논증이다.

요약

- CLS 는 속도 논증이 아니라 **간접 논증**이다. 빠른 쓰기와 느린 일반화를 한 파라미터 집합에 있으면 둘 다 망가지므로 뇌가 두 시스템을 갖는다는 것이며, 그 처방인 interleaving 은 과거 데이터의 영구 저장을 요구한다.
- SWS 와 REM 은 하나의 sleep 이 아니라 **목적함수가 다른 두 오프라인 라운드**다. 오프라인 위상의 개수는 어떤 논문이 은유를 실제로 이송했는지 판별하는 값싼 지표다.
- 생물학적 replay 는 저장된 입력 없이 일어나는 자발적 재활성화이고, deep learning 의 replay 는 저장된 원본을 쉼의 것이다. 두 용법은 이름만 같다.
- PLOS Sleep 은 과거 과제 데이터를 한 번도 다시 쓰지 않고 옛 과제 성능을 0.70 에서 0.70 으로 유지했다. 이것이 corpus 에서 오프라인 위상의 필요성을 지지하는 가장 직접적인 증거다.
- 같은 논문에서 데이터를 저장하는 고전 interleaving 은 0.68/0.65 로 sleep 의 0.70/0.68 과 표준편차 안에서 같고, 논문은 우위를 **실증이 아니라 이론적 근거**로 돌린다. 이 논문이 실증한 것은 정확도 우위가 아니라 저장의 제거다.

- PAD 는 오프라인의 가치가 스케줄이 아니라 **생성물**에 있음을보인다. REM 을 빼면 CIFAR-10 선형 분리도가 58.25 에서 46.00 으로 무너진다. 다만 저자 자신의 통제에서 기억의**혼합 자체는 필요하지 않고** (자발 활동만으로도 차이 없음), sleep 단계의 **순서도 무관하다**.
- 두 생물학 논문 모두 검정 이름도 p-값도 없이 “유의하게” 를 쓰고, 계산비용을 어느 형태로도 보고하지 않는다. PAD 의 절제 대조군은 저자가 손본상태로 비교되고, 표현 품질 지표는 linear readout 하나뿐이며 CIFAR-10 절대 성능은 약 59%다 — 성능 근거로 인용하면 오용이다.
- E-경로의 창시 논문은 sleep 을 170 회 쓰면서 그 단어를 consolidation 이아니라 **유휴**의 뜻으로 도입한다. 이름의 공유는 이론의 공유가아니며, 두 방향 침묵의 등급 판정은 ch23 이 소유한다.

자가 점검체크리스트

- CLS 의 두 시스템이 왜 필요한지를 “속도” 가 아니라 “간섭” 으로 설명할 수있다.
- 어떤 논문이 “replay 한다” 고 쓸 때, **어느 층에 쓰는지와 $\mathcal{R}_k$ 를 어디서 얻는지** 두 질문을 먼저 던진다.
- 표 8-1 의 네 조건에서 PLOS Sleep 이 실증한 것과 실증하지 않은 것을 각각한 문장으로 말할 수 있다.
- 표 8-3 에서 w/o NREM 열이 CIFAR-10 에서 PAD 와 오차 안에서같다는 사실을 지적할 수 있고, 그럼에도 NREM 의 기여가 어디서 나타나는지말할 수 있다.
- 어떤 논문이 sleep 이라는 단어를 쓸 때 그것이 유휴를 뜻하는지 consolidation 을 뜻하는지를 **참고문헌으로** 판별할 수 있다.
- 이 장의 개념 중 W 층에 놓이는 것이 하나도 없는 이유를 설명할 수있다.
- (Rosetta) Sleep-time Compute 의 오프라인 위상은 **warm-up / 사전 컴파일** 자리에, CLS·PLOS·PAD 의 consolidation 은 **모델재배포 1 회** 자리에 놓인다는 것을 설명할 수 있다. 앞의 것은 캐시 무효화문제이고 뒤의 것은 배포 문제다.

다음 장으로

이 장은 오프라인 위상이 저장 없이 $\mathcal{R}_k$ 를 만들 수 있음을보였다. 그러나 두 질문이 열린 채 남는다. 첫째, 그렇게 만든 $\mathcal{R}_k$ 를 몇 번이나 반복해서 넣을 수 있는가 — PLOS Sleep 은과제 두 개에서 멈췄고 ρ 를 재지 않았으며, PAD 는 오프라인 위상하나를 빼면 반복 라운드가 성능을 깎기 시작한다는 방향성만 보고한다. 둘째, 더 앞선 질문으로, Θ 에 애초에 **얼마나** 넣을 수있는가.

ch09 가 두 번째 질문을 받는다. 파라미터당 몇 bit 가 저장되는지, 그 상한이 Θ -경로의 C_{cap} 을 어디서 끊는지, 그리고 그 상한이 이 장이말한 “저장을 계산으로 바꾼다” 는 교환을 어디까지 허용하는지를 다룬다. 첫번째 질문은 반증 축인 ch22 가, 그리고 두 생물학 논문의 정면 재독은 ch23 이 받는다.

ch09. 기억 용량 — 얼마나 쓸 수 있는가

이 장의 목표 — 이 장을 마치면 다음 넷을 할 수 있어야 한다. (1) memorization 과 generalization 을 조작적으로 분리하고, 어떤 층에 무엇을 쓰든 그 층이 담을 수 있는 정보량의 상한을 bits 단위로 계산한다. (2) “3.64 bits per parameter” 가 어떤 실험 조건에서 나온수이고 왜 상한이 아니라 하한인지 진술하고, 그 수를 LoRA delta 에 적용하는 것이 외삽임을 지적한다. (3) 주어진 d, L_{layer}, r 에서 LoRA delta 의 정보 규모와 저장 바이트를 손으로 계산한다. (4) 용량과 계산 (FLOPs) 이 분리 가능한 축임을 memory layer 의 실측으로 설명하고, 저장 dtype 이 정보 밀도를 정한다는 것을 보인다.

왜 필요한가 — Part II 의 Θ -경로 장들 (ch19 LoRA·모델 편집, ch20 SEAL, ch21 Language Models Need Sleep) 은 전부 “sleep 라운드마다 가중치에 새 지식을 쓴다” 를 전제한다. 그런데 그 장들 중 어느 것도 “그 가중치가 몇 bits 를 담을 수 있는가” 를 묻지 않는다. Part III 의 ch26(상태 용량과 정보밀도)·ch27(경로별판정)·ch29(system·infra 기회) 가 쓰는 용량 논증전체가 이 장에서 세우는 자를 쓴다. ch22(반증 축) 는 이 장이 세우는 $H(\Theta)$ 천장을 Θ -경로의 falsifier 로 세운다.

NM 과의 관계 — Neural Memory 는 W 층만 다뤘고, 용량을 두 번 정의했다: NM ch05 의 고전 Hopfield capacity 와 NM ch14 의 Atlas associative-memory capacity (fast-weight 행렬이 담을 수 있는 key-value 쌍의 수). 둘 다 W 의 용량이고 단위는 **쌍의 개수**다. 이 장이 더하는 것은 Θ 층의 용량이며 단위가 다르다 — 개수가 아니라 **bits** 다. 세 용량은 같은 단어를 쓰지만 같은 양이 아니므로 이장에서는 항상 C_{cap} 으로만 쓴다. backward pass·optimizer state·outer/inner loop 는 전제한다 ($\rightarrow$ NM ch02).

09.1 무엇을 재는지 먼저 정한다 — memorization 과 generalization 의 분리

용량을 재려면 “모델이 데이터에 대해 아는 것” 중 그 데이터가 아니면 알 수 없었을 부분만 세야 한다. 모델이 “프랑스의 수도는 파리” 를 맞히는 것은 훈련 집합의 그 문장을 담고 있어서일 수도 있고, 세상에 대한 일반적 압축이 그 답을 함의해서일 수도 있다. 후자를 세면 용량이 무한대로 나온다. 분리가 먼저다.

How much do language models memorize(2505.24832) 는 이 분리를 참조 모델로 조건화해서 수행한다. 논문이 스스로 건요구는 세 가지다: 일반화와 분리될 것, 확률변수가 아니라 실현된 표본 하나와 실현된 모델 하나에 대해 정의될 것, 학습 알고리즘과 무관할 것 [lm-memorization-capacity §2, txt L285–L295].

정의. unintended memorization 은 참조 모델 Θ_{ref} 로 먼저 조건화한 뒤 남은 것 중 대상 모델 Θ 가 여전히 아는 양이다.

$$\text{mem}_U(\mathcal{X}, \Theta, \Theta_{\text{ref}}) = I([\mathcal{X} \mid \Theta_{\text{ref}}]; \Theta) = H(\mathcal{X} \mid \Theta_{\text{ref}}) - H(\mathcal{X} \mid (\Theta_{\text{ref}}, \Theta)) \quad (9-1)$$

이 식이 말하는 것은 하나다 — 조건화의 순서가 곧 분리 장치다. 참조 모델이 이미 아는 것은 빼고 시작하므로, 남는 양은 “이 데이터셋을 봤기 때문에 생긴” 정보다 [lm-memorization-capacity § 2.1, txt L321]. 그 반대편, 즉 일반화는 **잔차**로 정의된다: $\text{mem}_I = \text{mem} - \text{mem}_U$ [lm-memorization-capacity § 2.1, txt L323]. 일반화를 직접 재지 않고 뺄셈으로 얻는다는 것은 논문의 정의상 선택이지 결과가 아니다.

정의. memorization 과 generalization 의 분리는 이 두식의 쌍이다 — 측정되는 것은 mem_U 이고, generalization 은 $\text{mem} - \text{mem}_U$ 라는 잔차로만 존재한다. 이 책에서 “용량” 이라고 할 때 세는 것은 언제나 앞의 것이다.

표본 하나 · 모델 하나에 대해 정의하려면 Shannon 엔트로피로는 안 된다. 실현된 문자열 하나에는 엔트로피가 없기 때문이다. 논문은 Kolmogorov 복잡도로 갈아탄다 — 문자열 하나에도 기술 길이는 있다 [lm-memorization-capacity Def. 2–3, txt L363–L378]. 그리고 실제계산에서는 기술 길이를 산술부호 길이, 즉 음의 로그가능도로 근사한다 [lm-memorization-capacity § 2.3, txt L406–L413]. 여기서 실무적 결론이 하나 나온다: **용량 측정은 forward pass 두 번과 뺄셈 한 번이다**. 학습이 끝난 뒤에 하는 일이므로 갱신 기제가 필요 없다.

[해설] inference 엔지니어의 어휘로 옮기면 (9-1) 은 “같은 문자열을 두 코덱으로 압축했을 때의 코드 길이 차이” 다. 대상 모델이라는 코덱이 참조 코덱보다 이 문자열을 얼마나 더 짧게 쓰는가 — 그 차이가 대상모델이 이 문자열에 대해 별도로 들고 있는 bits 다.

정의에는 대가가 따른다. mem_U 는 모델 하나의 성질이 아니라 **모델 쌍의 성질**이다. 참조 모델이 바뀌면 값이 움직인다. 그리고 논문은 참조 모델 규약을 최소 세 가지로 쓴다 — (a) “같은 아키텍처의 더 큰 모델을 훨씬 넓은 분포에서 오래 학습시킨 것” [lm-memorization-capacity § 2.3, txt L411–L413], (b) “같은 파라미터 수를 최대 데이터로 학습시킨 것” [§ 4, txt L688–L690], (c) “평균 집합에서 최저 손실을 내는 oracle 모델” [§ 4, txt L690–L692] — 그리고 어느 선택에 대해서도 민감도분석을 보고하지 않는다. 텍스트 쪽 헤드라인 그림은 (c) 를 쓴다 [Fig. 2 캡션].

[평가] 그러므로 텍스트에 대한 모든 암기량 수치는 참조모델을 바꾸면 보고되지 않은 만큼 움직인다. 합성 데이터에서는 이 문제가 없다 — 참조가 알려진 균등분포이므로 해석적이다 [lm-memorization-capacity § 2.3, txt L550–L553]. **논문의 수치가 견고한 구간과 그렇지 않은 구간이 정확히 이 선으로 갈린다**. 다음절이 견고한 쪽이다.

systems 관점에서 이 절이 남기는 것은 검증 규칙이다. 논문은 membership inference 점수가 0.97 인데 32-token prefix 추출률이 0 인 사례를 보고한다 [lm-memorization-capacity § 5.1, txt L867–L869; Fig. 10]. 그리고 반대 방향에 대해서도 못을 박는다 — “언어모델은 거의 아무문자열이나 출력하도록 유도될 수 있으므로, 모델이 무언가를 출력한다는 사실 자체가 암기의 증거는 아니다” [lm-memorization-capacity § 1, txt L169–L171].

[평가] 그러므로 sleep 라운드가 지식을 실제로 심었는지를 **프롬프트를 넣고 출력이 맞는지 보는 방식으로 검증하는 것은 양방향으로 틀린다**. 모델은 담고 있으면서 뱉지 않을 수 있고, 담지 않은 채로 뱉을 수 있다. 옳은 게이트는 (9-1) 이 쓰는 가능성이다. Part II 의 Θ -경로 논문 중 이 게이트를 쓰는 논문은 없다.

09.2 bits per parameter — 3.64 는 어떻게 측정된 값인가

(9-1) 은 정의일 뿐 측정이 아니다. 측정으로 바꾸려면 $H(\mathcal{X})$ 를 정확히 아는 데이터가 필요하다. 논문의 수법은 **일반화가 원리적으로 불가능한 데이터**를 쓰는 것이다: 어휘 $|\mathcal{V}| = 2048$ 에서 토큰을 균등 · 독립으로 뽑아 길이 $L = 64$ 의 문자열을 만든다. 그러면 데이터셋 엔트로피가 닫힌 형태로 나온다.

$$H(\mathcal{D}) = N_D \cdot L \cdot \log_2 |\mathcal{V}| \quad (9-2)$$

토큰이 서로 독립이므로 예측할 것이 없고, 따라서 $\text{mem}_U \approx \text{mem}$ 이 되어 참조 모델 문제도 사라진다 [lm-memorization-capacity § 3.2, txt L628 – L637]. 이 조건에서 데이터셋 크기 N_D 만 키우며 학습하면, 암기량이 어느 지점에서 평평해진다. 그평평해진 높이가 그 모델의 용량이다 [lm-memorization-capacity § 3.2 Results, txt L650 – L654; Fig. 1].

정의. bits per parameter(이하 bpp)는 측정된 용량을 파라미터 수로 나눈 기울기다. 즉 $C_{\text{cap}}(\Theta) \approx \text{bpp} \cdot |\Theta|$ 이며, GPT 계열 half precision 에서 $\text{bpp} = 3.64$ 로 보고된다 [lm-memorization-capacity Fig. 6 캡션].

α 는 예약 기호 (retention gate) 이므로 논문의 α 를 그대로 쓰지 않고 여러 글자 이름 bpp 를 쓴다. 그리고 이 수를 인용할때 반드시 함께 옮겨야 할 측정 조건은 다음과 같다 [lm-memorization-capacity § 3.2, txt L640 – L646].

조건	값
데이터	균등·독립 무작위 토큰 문자열 (실제 텍스트 아님)
아키텍처	GPT-2, from scratch 학습
규모	Table 1 기준 $8.04 \times 10^4 \sim 6.86 \times 10^6$ 파라미터
학습	10^6 steps, batch 2048, Adam, A100 1 장, bfloat16, 시드 5 개
시퀀스	$L = 64$ tokens, $ \mathcal{V} = 2048$

이 조건이 곧 이 수의 사용 범위다. 무작위 문자열을 처음부터 학습시킨 소형 GPT 에서 켜 값이며, 부분 파라미터 학습·adapter·저랭크 delta 는 한 번도 측정되지 않았다 [lm-memorization-capacity § 3.2; 논문 전체에 adapter 실험 없음].

09.2.1 이 수는 여섯 가지로 인쇄된다

논문은 자기 헤드라인 상수를 한 값으로 쓰지 않는다. 초록은 3.6 [txt L12 – L13], [lm-memorization-capacity § 1] 은 “3.5 와 4 사이” [txt L194 – L195], [lm-memorization-capacity § 3.2] Results 는 “3.5 와 3.6 사이” [txt L659], Fig. 6 캡션은 3.64, [lm-memorization-capacity § A.1] 은 3.642 [txt L1310], Table 1 평균은 bf16 3.51 / fp32 3.83 이다. Table 1 의 16 개 설정별 값은 bf16 에서 2.86 – 3.94, fp32 에서 3.02 – 4.23 으로 흩어지며, 표에는 “Mean (± 0.1)” 이라고 적혀 있다.

바이트로 읽을 때 걸리는 단위 오류도 하나 있다. Fig. 5 는 6.86×10^6 파라미터 모델의 용량을 “23.9 MB” 로 인쇄하지만 [txt L238, L264], Table 1 이 같은 모델에 주는 용량은 $2.51 \times 10^7 \text{ bits} = 3.14 \text{ MB}$ 다. $2.51 \times 10^7 / 2^{20} = 23.9$ 이므로 bit 수를 2^{20} 으로 나눈 값에 MB 를 붙인 것이다 (본서 산술). 이 그림에서 저장 바이트를 읽으면 8 배 과다 산정한다.

[평가] 즉 존재하는 것은 하나의 상수가 아니라 $\pm 0.5 \text{ bpp}$ 폭의 밴드이고, 중심값은 논문의 어느 문장을 인용하느냐에 따라 달라진다. 이 장은 계산에 3.64 (Fig. 6 캡션값) 를 쓰되, 결론은 항상 자릿수와 순서로만 진술한다. 3.64 를 소수둘째 자리까지 믿는 서술은 이 논문이 뒷받침하지 않는다.

09.2.2 이것은 상한이 아니라 하한이다

논문은 자기 수치의 방향을 스스로 밝힌다 — “우리 모델은 gradient descent 로 학습되므로 전역 최적을 찾는다 는 보장이 없다. 따라서 우리는 언제나 모델 용량의 하한만을 측정한다” (원문: “we are only ever measuring a lower bound on model capacity”) [lm-memorization-capacity § 3.2, txt L664]. 게다가 가장 큰 두 실험은 수렴하지 않았다고 명시한다 [lm-memorization-capacity § 3.2, txt L670 – L672]. 이 문장이 3.64 bpp 에 대한 이 책의 정본 서술이고, 다른 장이 이 상수를 인용할 때 함께 옮겨야 할 방향이다.

[평가] 하한은 Θ -경로를 반증할 수 없다. “적어도이만큼은 들어간다” 는 진술은 Θ -경로에 **유리한** 방향이다. 그러므로 이 장은 3.64 bpp 를 “가중치가 담을 수 있는최대치”로 쓰지 않는다. 쓸 수 있는 것은 두 문장뿐이다 — (i) 암기량은 $H(\Theta)$ 를 넘을 수 없다 (§ 09.3), (ii) 실측된 도달점은 파라미터당 약 3.6 bits 였다. 이 책의 실험 X2 는 초판에서 3.64 를 상한으로놓고 Θ 와 E 의 바이트 밀도를 비교했고, 방향이 보장되지않으므로 그 경로 간 비교를 철회했다 [X2 F3_withdrawn] — § 09.7 이철회 후 남는 것만 신는다.

세 가지 제약이 더 붙는다. 첫째, bpp 가 모델 형태에안정적이지 않다. Table 1 bf16 에서 ($L_{\text{layer}}=1, d=256$) 은 2.88, ($L_{\text{layer}}=2, d=256$) 은 3.66 이고, ($L_{\text{layer}}=8, d=32$) 은 2.86, ($L_{\text{layer}}=1, d=32$) 은 3.93 이다 — 깊이와폭이 서로 다른 지점에서 반대로 움직인다. 둘째, 이 상수는 6.86×10^6 파라미터 위에서 다시 측정된 적이 없는데, 논문의 membership-inference 법칙은 이를 1,556,075,200 파라미터 모델에그대로 대입한다 — 약 227 배 외삽이다 [lm-memorization-capacity Table 1 vs Table 2]. 셋째, **텍스트에 대한 bpp 값은 논문어디에도 없다.** [lm-memorization-capacity § A.2] 는 텍스트 쪽 암기량합이 “아마도 모델 용량 상한에서” 일정해진다고 쓰는데, 그 “아마도” 가 논문자신의 표현이다 [txt L1405 – L1408].

systems 점점 하나. bf16 파라미터는 16 bits 를 차지하지만 3.51 bits 의 데이터셋 정보를 담고, fp32 는 32 bits 를 차지하고 3.83 을 담는다 — 저장 효율 21.9%와 12.0%다. 정밀도를 두배로 올려도 용량은 9.1%만 오른다 [lm-memorization-capacity § 3.2, txt L677 – L680; 백분율은 본서 산술]. 논문은 이것을 “정밀도를 bfloat16 에서 float32 로 올릴 때 추가된 모델 bits 의 대부분은 원시 저장에 쓰이지않는다” 고 정리한다 [lm-memorization-capacity § 3.2, txt L678 – L680].

[평가] 이 결과는 bpp 를 저장 밀도로 읽는 논문자신의 틀과 어긋난다. bpp 는 **그릇의 성질이 아니라최적화의 성질이다.** 그릇의 성질이라면 bits 를 두 배로 주었을 때 용량도 두배가 되어야 한다. 남는 문장은 하나다 — **가중치는 특정 내용을담기에 매우 비효율적인 매체이고, 그 비효율의 원인은 매체가 아니라 그매체를 채우는 절차다.**

09.3 C_{cap} 의 조작적 정의 —용량은 아키텍처만의 성질이아니다

앞 절의 측정 절차를 정의로 굳히면 다음이 된다.

정의. C_{cap} 의 조작적 정의: 학습 절차 $\mathcal{A}$ 의 용량은 데이터 분포를 바꿔가며 포화까지 학습시켰을 때언어지는 암기량의 최댓값이다.

$$C_{\text{cap}}(\mathcal{A}) = \max_{\mathcal{X}} \text{mem}(\mathcal{X}, \mathcal{A}(\mathcal{X})) \quad (9-3)$$

인자가 $\mathcal{A}$ 라는 점이 핵심이다 — 용량은 **(아키텍처, 학습 알고리즘) 쌍의 성질**이지 아키텍처만의 성질이 아니다 [lm-memorization-capacity Def. 5, § 3.1, txt L571 – L620]. 논문의 모든실험은 Adam 하나만 쓰고, 두 번째 optimizer 는 시도되지 않는다 [txt L643].

[평가] § 09.2.2 의 정밀도 결과와 합치면 결론이 분명해진다: bpp 는 검증되지 않은 optimizer 가정을 안고 있다. 정의 (9-3) 이 인자로 $\mathcal{A}$ 를 받는 이상, 단일 optimizer 에서詹 값을상수로 옮기는 것은 논문의 정의가 허락하지 않는다.

용량에는 정보이론적 천장도 붙는다. 개별 표본의 암기량 합은 데이터셋전체의 암기량을 넘지 못하고, 그것은 다시 가중치 자체의 엔트로피를 넘지못한다.

$$\sum_{i=1}^{N_D} \text{mem}_U(z_i, \Theta, \Theta_{\text{ref}}) \leq \text{mem}_U(\mathcal{D}, \Theta, \Theta_{\text{ref}}) \leq H(\Theta) \quad (9-4)$$

[lm-memorization-capacity Prop. 1, txt L326 – L330; 증명 § A.6]. 오른쪽부등식이 이 책이 상속하는 문장이다 — **어떤 $(U-\Theta)$ 통합절차도 $H(\Theta)$ 를 넘겨 쓸 수 없다.** 단, 증명은 표본의 i.i.d. 가정을 쓴다 [§ A.6 각주 4, txt L1592 – L1594]. 자기생성 $\mathcal{R}_k$ 처럼 표본끼리 상관성이 큰 학습 집합에서는 이 부등식을 그대로 옮길 수 없다.

실무형은 더 단순하다. 암기량은 용량과 데이터 엔트로피 중 작은 쪽이다.

$$\text{mem}(\mathcal{D}, \mathcal{A}(\mathcal{D})) \approx \min(C_{\text{cap}}(\mathcal{A}), H(\mathcal{D})) \quad (9-5)$$

[lm-memorization-capacity § A.1, txt L1309]. 즉 **찰 때까지 채우고, 차면 멈춘다.** L 을 바꿔가며 검증했을 때 평균 오차 1.7%, $|\mathcal{V}|$ 를 바꿨을 때 1.8%다 [txt L1399; Tables 3, 4].

이 예측자에서 두 개의 쓸모 있는 귀결이 나온다.

첫째, 레짐 경계가 사전에 계산된다. $H(\mathcal{D})$ 가 $\text{bpp} \cdot |\Theta|$ 를 넘는 지점에서 double descent 가 시작한다 — 논문은 “double descent 는 데이터 용량이 모델 용량을 넘는 바로 그때 시작한다” 고 쓴다 [lm-memorization-capacity § 4, txt L832 – L833; Fig. 3]. 논문의 모델에서는 대략 10^5 개의 64-token 표본이다 [txt L824 – L826].

[해설] consolidation scheduler 를 만든다면 이 식이 그 판정규칙이다. sleep 라운드를 돌리기 전에, 쓰려는 자료의 엔트로피와 대상파라미터 블록의 크기만으로 “이 블록은 이미 찾는가” 를 계산할 수 있다. 논문은 그런 scheduler 를 제안하지 않는다 — 이것은 본서의 재구성이다.

둘째, 포화 후 쓰기는 제로섬이다. 데이터셋이 N 배로 늘면 모델은 암기량을 표본들에 나눠 배분하고, 합은 일정하게 측정된다 [lm-memorization-capacity § A.2, txt L1405 – L1408]. **즉 반복되는 Θ -경로 sleep 루프는 언젠가 반드시 잊는다.** 이것은 설계 실패가 아니라 (9-4) 의 귀결이다.

[평가] 그런데 이 논문은 망각의 **필연성**만 주고 **속도**는 주지 않는다. 라운드 개념이 없으므로 ρ 가 정의조차 되지 않는다. 망각률은 다른 데서 가져와야 한다 ($\rightarrow$ ch07). 이 장이 소유한것은 천장이지 감쇠 곡선이 아니다.

마지막으로 어떤 내용이 비싼가. 희귀어를 포함한 표본이 더 많이 암기되며, 가장 많이 암기된 상위 20 개 시퀀스 중 17 개가 일본어 · 중국어 · 히브리어 등 비영어 토큰열이다 [lm-memorization-capacity § A.3, txt L1549 – L1554; Fig. 16].

[평가] per-user sleep 스킴이 통합하려는 자료가 정확히 이런것이다 — 그 사용자만의 특이한 사실. 좋게 말하면 “잘 달라붙는 내용” 이고 나쁘게 말하면 “그릇을 가장 빨리 채우는 내용” 인데, 둘은 같은 사실이다.

09.4 LoRA 의 실측 용량 — 저랭크 delta 는 얼마나 담는가

앞 두 절의 수는 **전체 파라미터를 처음부터 학습시킨** 모델에서 나왔다. 그런데 Θ -경로의 sleep 라운드가 실제로 쓰는 대상은 전체 Θ 가 아니라 저랭크 delta $\Delta\Theta^{(j)} = A^{(j)}B^{(j)}$ 다 (기제는 ch03 이 소유한다. 여기서는 용량만 본다). 두 상황은 같지 않다.

[평가] 3.64 bpp 를 $2rd$ 개짜리 LoRA delta 에 곱해서 “이 adapter 는 $3.64 \times 2rd$ bits 를 담는다” 고 쓰는 것은 **외삽**이다. How much do LMs memorize 는 adapter 를 재지 않았고, 저랭크 갱신을 재지 않았으며, 부분 파라미터 학습을 재지 않았다 — Table 1 의 모든 모델은 전 파라미터가 자유로운 from-scratch 학습이다 [lm-memorization-capacity § 3.2, txt L640 – L646]. 게다가 § 09.3 에서 본대로 C_{cap} 은 정의상 학습 알고리즘 $\mathcal{A}$ 의 함수이고 (9-3), 저랭크 제약은 다른 $\mathcal{A}$ 다. 그리고 3.64 는 하한이므로 (§ 09.2.2) 이 곱은 상한조차 아니다. 본서는 이 곱을 **파라미터 수가 시사하는 규모 추정**으로만 쓰고, 그렇게 쓸 때마다 외삽임을 표기한다.

그러면 실측은 어디에 있는가. Understanding LoRA as Knowledge Memory(2603.01097) 가 그것을 직접 쟀다. 다만 정보이론적 bits 가 아니라 **행동적 토큰수**로 쟀다.

정의. LoRA 의 실측 용량은 성능이 임계 π_{thr} 이상으로 유지되는 최대 지식 적재량 T_{max} (토큰)이며, 파라미터 효율은 그것을 학습 가능 파라미터 수로나는 값이다.

$$\text{Efficiency} = \frac{T_{max}(\pi_{thr}, r)}{N_{params}(r)} \quad (9-6)$$

[lora-as-knowledge-memory Eq. 1, § 4 Q3]. 이 논문 전체에서 번호가붙은 유일한 식이다. 이 식은 § 09.2 의 bits-per-parameter 와 **형태가 같고 단위가 다르다** — 분자가 bits 가 아니라 회수 가능한 토큰이다.

논문이 보고하는 형태는 셋이다. 첫째, 용량은 rank 와 함께 커진다. CounterFact 8K/10K/12K/14K 적재에서 r 을 2 에서 1024 까지 올리면 efficacy 가 단조 증가하되, “rank 증가분당 한계 이득은 감소한다” 고 논문이 명시한다 [lora-as-knowledge-memory § 4 Q1; Fig. 1]. 둘째, 고정 rank 에서적재량을 1K → 20K 토큰으로 늘리면 “어느 임계까지는 유지되거나 오르다가, 그지점을 넘으면 성능이 급격히 떨어진다” [§ 4 Q2; Fig. 2, Fig. 14]. 셋째, **파라미터 효율은 rank 에 대해 단조가 아니고 작은 rank 에서최대다** — “가장 효율적인 설정은 일관되게 $r = 4$ 같은 작은 rank 에서발견되었다” [App. H].

$\mathcal{R}$ 을 만드는 방식이 용량을 좌우한다는 것도 이 논문이 처음실측한다. PaperQA LLM-judge 기준 [lora-as-knowledge-memory Table 1], 원문 그대로 학습하면 Llama-3.1-8B 에서 3.187 인데 QA 40 쌍으로바꾸면 5.893, 네 형식을 모두 섞으면 6.822 다. 같은 문서인데 학습 집합의형식만 바뀌서 두 배가 된다. 이것이 (U- Θ) 의 $gen(\cdot; B_s)$ 에 실증적 내용이 붙은 최초의 지점이다 (자세한것은 ch06).

09.4.1 이 실측이 뒷받침하지 않는것

정직하게 옮겨야 할 것이 여럿이다. T_{max} 의 수치가 **논문어디에도 표로 없다**. $N_{params}(r)$ 도 어느 rank 에 대해서도명시되지 않는다. 즉 (9-6) 의 값을 재현할 수 있는 점이 논문에 한 개도없고, 값은 오직 그림의 비율 축에만 존재한다 [lora-as-knowledge-memory Eq. 1; Figs. 3, 15, 16]. 시드 · 표준편차 · 오차막대도 논문 전체에 없다.

세 벤치 중 둘이 저자 제작이다 (PhoneBook, PaperQA) [App. B, App. C]. 그리고 최고 형식으로 뽑힌 QA 에 대해 저자 스스로 오염을 인정한다 — “QA 형식의 우위는 평가 과제 역시 QA 라는 구조적 정렬에 부분적으로 기인할 수있다” [App. I].

가장 중요한 부정 결과는 **조합이 안 된다**는 것이다. 라우팅 오류를완전히 제거하고 정답 모듈만 병합해도, LLM-judge 점수는 $N_m = 1$ 에서최고이고 $N_m = 5$ 까지 단조 감소한다 [lora-as-knowledge-memory § 6 Q11; Fig. 7 오른쪽; App. P]. 실제 라우터를 쓰면 임베딩 라우팅 (Llama 4.94, Qwen 5.52) 이 단일 LoRA 기준선 (5.24, 5.65) 보다 **낮다** [§ 6 Q9]. closed-book NarrativeQA 에서는 multi-LoRA 가 네 모델 전부에서 단일 LoRA 에 진다 [Table 2].

용량을 비용과 함께 읽어야 할 자리에서 항 하나가 빠져 있다. 논문의 유일한비용 비교 (한 문서에 대한 30 개 질문의 벽시계 시간) 는 구성요소로 모델로딩 ·adapter 로딩 ·병합 ·활성화 ·토큰화 ·추론을열거하는데 **학습 항이 없다** [lora-as-knowledge-memory App. U]. 즉 그 비교는 상각식 (A) 에서 $N_q = 30$, $C_{sleep} = 0$ 으로놓은 것이다. 그리고 그 비교 안에서 RAG 가 논문이 제안한 multi-LoRA preloaded 보다 빠르다 — FlashAttention 기준 43.5 초 vs 65.9 초 [Fig. 22]. 논문은 ICL 과만 비교한다.

[평가] 용량을 rank 로 늘리는 축과 모듈을 쪼개서 늘리는 축은다르다. 이 논문은 전자에 대해서는 “유한하고 rank 로 조절된다” 는 쓸 만한형태를 주지만, 후자에 대해서는 **음의 결과**를 준다. per-user-per-document adapter 를 여러 개 쌓아 용량을 키우는 설계는 이 논문의 증거로는 지지되지 않는다. 실증 상한도 낮다 — 논지를떠받치는 결과는 전부 8B 이하에서 나왔고 (모델 크기 스왑 하나만 14B 까지간다 [App. K]), 지식 64K 토큰 이하, adapter 15 개 이하, 5-way 병합이하, 추론 배치 1, 워크스테이션 GPU 한장이다 [App. D; App. Q].

systems 점점 셋. 첫째, **rank** 는 **용량 knob** 이자 **바이트 knob** 이며 두최적점이 어긋난다 — 절대 용량은 r 과 함께 오르지만 파라미터당 효율은 작은 r 에서 최대다. 바이트로 사이징해야 하는 per-user 저장소는 작은쪽을 택하게 된다. 둘째, delta 트래픽이 **FFN에 실린다** — “모든설정에서 FFN-only LoRA 가 attn-only 를 일관되게 능가한다” [App. E.1]. 셋째, 질의마다 병합 (TIES) 이 일어나므로 요청마다 가중치가 달라진다. 두요청이 서로 다른 파티션으로 라우팅되면 같은 GEMM 을 공유할 수 없다 [본서 추론; 논문은 batching 을 논하지 않고 추론 배치 크기를 밝히지않는다]. Rosetta 로 옮기면 “batch 로 weight 공유 $\rightarrow$ Θ -경로에서깨짐” 행이 여기서 실제로 발생한다.

09.5 memory layer — 용량과 FLOPs 는분리된다

Θ 의 용량을 늘리는 방법이 “모델을 크게 만든다” 하나뿐이라면 용량논의는 곧 FLOPs 논의로 환원 된다. Memory Layers at Scale(2412.09764) 이 그 환원을꾼는다.

기제는 FFN 층 하나를 학습 가능한 key-value lookup 으로 교체하는 것이다. 토큰 표현 q_t 에 대해

$$I_t = \text{TopK}_{k_{\text{top}}}(K q_t), \quad a_t = \text{softmax}(K I_t q_t), \quad y_t^{\text{mem}} = a_t V_{I_t} \quad (9-7)$$

이며 $\{K, V\}$ 는 activation 이 아니라 **학습된 파라미터**다 — 논문스스로 경계를 긋는다: “memory layer 의 key 와 value 는 activation 이 아니라 학습 가능한 파라미터다” [memory-layers § 3]. 즉 이것은 KV cache 가아니고 E 도 아니다. Θ 안이다. 본서는 이 부분을 Θ_m (capacity 파라미터), 나머지를 Θ_d (dense compute 파라미터) 로 부른다.

정의. memory layer 의 용량-FLOPs 분리란, 토큰마다 k_{top} 개의 value 행만 읽히므로 Θ_m 을자릿수 단위로 키워도 토큰당 부동소수점 연산이 거의 늘지 않는 성질이다. 논문의 회계 전제가 이것이다 — “같은 base model 설정을 가진 모델들은 FLOPs 차이가 무시할 만하다” [memory-layers § 5.1].

실측은 다음과 같다 [memory-layers Table 1; 1.3b base, 1T tokens, Memory+].

설정	총 파라미터	NaturalQuestions	TriviaQA
dense 1.3b	1.3b	7.76	32.64
Memory+ 1m values	3.377b	13.68	42.89
Memory+ 4m values	9.823b	14.43	51.18
Memory+ 16m values	35.618b	20.14	58.67
Memory+ 64m values	138.748b	20.78	62.14

dense 1.3b 에서 138.748b 까지 총 파라미터가 **106.7 배**로 늘고 ([본서 산술]) FLOPs 는 사실상 그대로인데 NQ 는 7.76 에서 20.78 로, TQA 는 32.64 에서 62.14 로 오른다. 최대 설정의 memory 파라미터는 8192^2 개 value $\times$ 2048 차원 = $2^{37} = 128 \times 2^{30}$ 개로, [memory-layers § 5.2] 의 “128 billion memory parameters” 는 근사가아니라 gibi 단위의 정확값이다 [본서 산술].

[해설] 이 표가 이 장에 있는 이유는 용량 추과 계산 추이직교할 수 있다는 실증을 corpus 에서 유일하게 제공하기 때문이다. § 09.2 의 3.64 bpp 는 “파라미터 하나가 몇 bits 를 담나” 를말하고, 이 표는 “그 파라미터를 늘리는 값이 반드시 FLOPs 는 아니다” 를말한다. 둘을 합치면 Θ 용량을 늘리는 데 드는 비용이 계산이아니라 **바이트와 대역폭**이라는 결론이 나온다.

09.5.1 이 논문은 sleep-time compute의 사례가 아니다

명시해야 할 사실이 있다. 이 논문에는 배포 후 갱신이 없다. Θ 가 움직이는 시점은 pre-training 한 번뿐이고, W 도 E 도 존재하지 않는다. 판별식이 겨냥하는 것은 배포된 모델의 유휴시간에 어느 한 층의 상태를 바꾸고 그 변화를 이후 질의에 쓰는 절차인데, 이 논문에는 pre-training 밖의 갱신이 없다. 그래서 이 논문은 sleep-time compute이 아니라 **그것이 쓰일 그릇의 용량 실측치**로만 인용된다. 논문이 지속학습을 언급하는 유일한 자리는 실험이 아니라 희망 서술이다 — “우리는 memory layer가 가능하게 한 희소 갱신이 모델이 학습하고 정보를 저장하는 방식에 깊은 함의를 가질 수 있다고 믿는다” [memory-layers § 6].

정직성 항목이 더 있다. 16m → 64m 구간에서 파라미터를 103b 더 넣고도 NQ는 +0.64점만 오른다 (TQA 증분도 8.29 → 7.49 → 3.47로 감쇠한다) [본서 산술]. [memory-layers § 5.2]는 이 구간을 “예측 가능하게 계속 오른다”고 쓰고 Memory+ 64m을 “Llama2 7B에 근접한다”고 쓰는데, 보고된 다섯 열의 값은 NQ 20.78 vs 25.10, TQA 62.14 vs 64.00, PIQA 77.31 vs 78.40, OBQA 30.00 vs 33.20, HotpotQA 20.47 vs 25.00이다 [memory-layers Table 1]. 그리고 **dtype이 논문 어디에도 없고 k_{top} 의 수치도 없다**. 따라서 128 Gi 파라미터가 몇 바이트인지, 토큰당 몇 바이트가 움직이는지를 논문만으로는 계산할 수 없다. 추론 지연·처리량 측정은 0건이며, 저자 스스로 프로덕션 효율이 미해결이라고 쓴다 [memory-layers § 6].

[평가] 두 서술 모두 논문 자신의 표가 반증한다 — 증분은 감쇠하고, 다섯 열 전부에서 뒤진다. 이 장은 memory layer를 용량-FLOPs 분리의 실증으로만 인용하고, [memory-layers § 5.2]의 스케일 서술은 인용하지 않는다.

조건부로 환산하면 규모가 드러난다. 2^{37} 개 파라미터를 bf16에 두면 274.9 GB, fp8이면 137.4 GB다 (본서 조건부 산술 — 논문에 dtype이 없으므로 이 값은 논문 수치가 아니다). H100 80GB 한 장에 들어가지 않으며, 이것이 [memory-layers § 3.1.2]의 memory-parallel 샤딩이 선택이 아니라 **필수인** 이유다. 그리고 샤딩되면 토큰 하나를 읽는 데 두 번의 collective가 임계경로에 들어온다 — 인덱스를 process group 전체에 공유하고, 각 워커가 자기 샤드에서 lookup·집계한 뒤 자기담당 인덱스의 부분 임베딩을 모은다 [memory-layers § 3.1.2, Fig. 2; 학습 맥락의 서술을 추론 맥락으로 옮긴 것은 본서 추론]. memory 층 3개가 공유풀을 쓰므로 이 왕복이 층마다 반복된다.

systems 점점. 이 층은 FLOP-bound가 아니라 **bandwidth-bound**다 — 논문의 forward 커널은 3TB/s에 도달하며 이는 H100 사양 3.35TB/s의 89.6%다 [memory-layers § 3.1.4; 백분율은 본서 산술]. 그리고 dense GEMM과 달리 배치에 gather를 상각하지 못한다: 토큰마다 다른 행을 건드리므로 접근 행 수가 대략 $\min(B \cdot k_{\text{top}}, n_{\text{mem}})$ 까지 배치와 함께 늘어난다 [본서 추론; 논문은 이 축을 측정하지 않는다]. 마지막으로 Θ_m 은 전 사용자 공유이므로 shared-weight batching이 유지된다 — 대가는 대칭적이다. **per-session C_{cap} 이 0이다**. 사용자별 지식을 넣을 자리가 이 설계에는 없다.

09.6 rate-distortion 관점 — 상한이 정해지면 남는 질문은 “무엇을 버릴까”다

여기까지의 세 절은 그릇의 크기를 잴다. 그릇이 부족할 때 무슨 일이 생기는지는 다른 문제다. What to Keep, What to Forget(2607.08032)이 그 질문을 하나의 제약 최적화로 세운다. 이 논문의 budget B 는 **계산 예산이 아니라 상태 예산**이므로 본서 표기에서는 C_{cap} 으로 옮긴다 ($B_s \cdot B_t$ 와 혼동하면 안 된다).

정의. rate-distortion 관점은 기억 관리를 “예산 C_{cap} 안에서 하류 과제 손실을 최소화하는 압축기 선택” 문제로 놓는 것이다.

$$\min_{\Theta} \mathbb{E}_{(\mathcal{H}, q, y^*)} \left[\mathcal{L}(\text{use}(\text{cmp}_{\Theta}(\mathcal{H}), q), y^*) \right] \quad \text{s.t.} \quad \text{rate}(Z) \leq C_{\text{cap}}, \quad Z = \text{cmp}_{\Theta}(\mathcal{H}) \quad (9-8)$$

[rate-distortion-compaction Eq. 1]. 여기서 $\mathcal{H}$ 는 모델이 읽은 이력, Z 는 그것을 줄인 표현이다. 그리고 이 절이 이 장에 있는진짜 이유는 다음 부등식이다. 과제가 요구하는 정보량을 $I^*(q) = I(y^*; \mathcal{H} | q)$ 라 할 때, $C_{\text{cap}} < I^*(q)$ 이면

$$P_e = \Pr[\hat{y} \neq y^*] \geq \frac{H(y^* | q) - C_{\text{cap}} - 1}{\log |\mathcal{Y}|} \quad (9-9)$$

[rate-distortion-compaction Eq. 2]. Fano 부등식을 data-processing 단계에 적용해 얻은 것이며, 논문은 이 하한의 적용 범위를 명시한다 — “이하한은 KV cache 든, gist 벡터든, recurrent state 든, 에이전트의 노트저장소든, **어떤 압축 기억 Z 에 대해서도 성립한다**” [§ 2, ¶ 4].

[해설] 이 문장이 세 경로를 같은 자로 재는 근거다. Θ 에 쓰든 W 에 접든 E 에 적든, 상태가 과제의 정보요구량보다 작으면 **같은 식의 오류율 바닥**을 맞는다. 아키텍처로빠져 나갈 구멍이 없다 [§ 2, ¶ 5 (i)]. 그리고 압축 가능 비율은 $1/I^*(q)$ 에 비례하므로, 같은 evictor 가 요약에서는 거의 무손실이고정확 검색에서는 무너지는 이유도 여기서 나온다 [§ 2, ¶ 5 (ii)].

sleep-time compute 에 직접 걸리는 항이 하나 더 있다. 질의를 모르는압축기는 예산을 질의 분포 전체에 퍼야 하므로 유효 예산이 대략 $C_{\text{cap}} - H(q)$ 로 줄고, **그 차 $H(q)$ 가 질의를모르는 값의 정확한 가격**이다 [rate-distortion-compaction § 2, ¶ 5 (iii)]. 논문은 이것을 Letta 의 Sleep-time Compute 에 직접적용해, 그 방식이 “결코 질의되지 않을 수도 있는 bits 에 예산을 투기적으로쓴다”고 쓴다 [§ 7].

[평가] ch01 의 조작적 정의상 sleep-time compute 은**질의가 도착하기 전에** 계산한다. 그러므로 세 경로 전부가구조적으로 $H(q)$ 를 낸다. 예외는 저장한 것을 나중에 다시 질의할 수있는 경우뿐인데, 이것이 논문의 첫 번째 성질 (P-rev, 되돌릴 수있는가) 이다. 논문은 성질 셋을 전면에 세운다 — (P-rev) 버린 내용을 나중에다시 끌어올 수 있는가, (P-q) 압축 시점에 질의를 보는가, (P-fid) 손실프로파일이 무엇인가 [§ 2, ¶ 6]. **본서의 세 경로 판정은 이 셋을그대로 쓴다.**

개념적 정리 하나가 더 이 장에 유용하다. 논문은 온갖 중요도 휴리스틱을 **distortion 의 대리 추정량**으로 통합한다 — 압축 시점에 정답 y^* 를 모르므로 아무도 distortion 을 직접 평가할 수 없고, 그래서각 방법은 “이걸 버리면 얼마나 아플까” 의 값싼 대리값을 고른다는 것이다 [rate-distortion-compaction § 2, ¶ 5]. 누적 attention mass(H2O·SnapKV), 토큰 자기정보 (LLMLingua), 명시적 상호정보추정 (QUITO-X), 유발 출력 오차의 상한 (Ada-KV), LLM 자신의 중요도판단 (Mem0·reflection) 이 전부 같은 양의 추정량이다.

[해설] NM 의 retention gate α_t 는 이 가족의**학습된** 구성원이다. 손으로 고른 휴리스틱 대신 gradient 로 “남길비율” 을 배우게 한 것이 W -경로가 한 일이었다 (→ NM ch13). 이 논문은 KV 층에서 같은 구조에 도달하면서 α_t 와 연결하지 않는다 — 그 경로의문헌을 인용하지 않기 때문이다. 경로 간 침묵의 또 한 사례다.

되돌릴 수 있는가가 실제로 얼마나 차이를 내는지도 이 논문이 쟁다. 에이전트가 긴 문서를 청크로 읽으며 12 개 사실을 모으고 주기적으로 작업기억을 압축하는 설정에서, 되돌릴 수 있는 연산 (청크를 보관하고 질의 시검색) 은 모든 압축 빈도에서 recall 0.95 근처를 유지하고, 되돌릴 수 없는 연산 (LLM 요약으로 덮어쓰기) 은 0.33 – 0.56 에 머문다 [rate-distortion-compaction § 14.2; Fig. 6]. 논문의 표현으로 “같은평균 예산의 두 연산이 기억이 재사용되는 순간 recall 에서 약 0.5 만큼갈라진다.”

[평가] 다만 이 논문은 스스로 “이것은 survey 이며 형식화와벤치마크 프로토콜을 제한할 뿐 어느 쪽도 대규모로 실증하지 않는다” 고 쓴다 [Limitations]. 실험은 Qwen2.5-1.5B-Instruct 한 모델을 RTX 4060 8GB 한 장에서 돌린 것이고, 예측 네 개 중 둘은 검증하지 않았다 [§ 14; § 2]. 그리고 예산 측인 BPT(bytes-per-token-of-history) 는

끝까지 전체 cache 대비 백분율로만 보고되며 절대 바이트가 한 번도 나오지 않는다 [§ 14, ¶ 2].

더 무거운 제한 두 가지를 논문 자신이 밝힌다. 첫째, (9-9)의 유일한 자유변수 $I^*(q)$ 가 한 번도 측정되지 않는다 — “실제 과제에 대해 $I^*(Q)$ 를 추정하는 것 자체가 열린 문제다” [Limitations; § 12]. 그러므로 이 하한은 어느 설계 결정도 수치로 판정하지 못하고, 쓸수 있는 것은 부등호의 방향뿐이다. 둘째, 하한의 유도는 **질의를 모르는 압축기**를 가정한 Markov 단계를 쓰며, 질의를 보는 압축기에 대한 대응 하한은 제시되지 않는다 [§ 2, ¶ 4; Limitations]. $H(q)$ 별점은 그 비교의 한쪽만 유도된 상태에서 나온 값이다. 형식은 쓸 수 있고 수치는 쓸 수 없다.

systems 접점: BPT는 독자가 이미 쓰는 자다. 토큰을 비율 f_{keep} 만큼 남기면 f_{keep} , b_q -bit 양자화는 $b_q/16$, 프롬프트를 n 에서 m 으로 줄이면 m/n — KV cache 회계와 프롬프트 압축과 에이전트 기억이 하나의 단위로 모인다 [rate-distortion-compaction § 14, ¶ 2]. 분모의 16이 fp16을 무압축 기준으로 고정한다는 점은 다음 절로 이어진다.

09.7 정보 밀도는 dtype이 정한다 —[X2]

앞 절들의 수치는 전부 “파라미터 몇 개”, “토큰 몇 개” 였다. 서빙 설계는 바이트를 요구한다. 그런데 **corpus 전체에서 상태 용량을 바이트로 보고한 논문이 0 편이다** [X2 F1]. 그래서 본서는 구성 파라미터에서 바이트를 복원하는 결정론적 회계 X2를 돌렸다.

핵심 결과 하나. 저장 바이트 1개에 담기는 정보량은 표현의 한계가 아니라 **저장 dtype**이 정한다. 8B 앵커 ($d = 4096$, $L_{\text{layer}} = 32$)에 $r = 16$ LoRA를 $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}$ 에 붙인 delta 기준 [X2 Q2]:

저장 형식	저장 크기	bits / 저장 바이트
Θ delta, bf16	16.78 MB	1.82
Θ delta, int8	8.39 MB	3.64
Θ delta, int4	4.19 MB	7.28
E 텍스트 (액면가)	56.0 KB	8.00
E 텍스트 (3:1 압축 가정)	—	2.67

이 표에서 단정할 수 있는 것은 Θ 세 행뿐이다. 3.64가 하한이므로 (§ 09.2.2) Θ 행의 밀도도 하한이고, 하한과 E 의 액면가를 나란히 놓아 순서를 매길 수는 없다 — X2 초판이 낸 경로 간 밀도 비 (bf16 1.47 / int8 0.73 / int4 0.37)는 방향이 보장되지 않으므로 **철회하고 판정 불가로 남긴다** [X2 F3_withdrawn]. 남는 것은 Θ 열 안에서만 정해지는 문장이다: Θ -경로가 바이트 비효율로 보이는 것은 파라미터당 최소 3.64 bits를 16 bits 그릇에 담기 때문이지 표현이 정보를 못담아서가 아니다.

[평가] 그러므로 **delta 양자화는 Θ -경로 배포의 선택 사항이 아니라 전제다**. 이 결론은 3.64가 하한이든 상한이든 성립한다 — 밀도의 분모인 저장 dtype만으로 정해지기 때문이다. 그리고 corpus의 어느 논문도 delta를 양자화한 상태에서 지식 회수율을 측정하지 않았다. 이것이 이 책이 지목하는 미탐색 영역 하나다.

규모로 옮기면 사용자 10^6 명 기준 웹 스토리지가 LoRA $r=16$ bf16에서 16.777 TB, int4로 내리면 4.194 TB, $r=64$ bf16이면 67.109 TB다. 모델을 통째로 복제하는 직접 편집 방식이면 16000 TB로 세 자릿수가 뛴다 [X2 Q3].

[해설] 기회의 성격이 여기서 드러난다. 이것은 HBM 대역폭 문제가 아니라 **요청 지연 안에 delta를 끌어올 수 있는 웹 계층과 그 계층의 쓰기 내구성** 문제다. NM이 W -경로에서 짙은 HBM read-modify-write 기회와는 다른 자리다. 두 책의 결론을 하나로 잇지 않는다 (→ ch29).

등급 규칙을 지킨다. X2는 측정이 아니라 회계이고, 3.64 bpp를 LoRA delta에 적용한 것은 § 09.4에서 못 박은 외삽이며, E -경로 바이트는 토큰 수에서 환산한 가정치다. 따라서 본문 단정문으로 쓰는 것은 **자릿수 차이와 Θ 안의 dtype 순서**이고, 경로 간 밀도순서는 쓰지 않으며, 절대 바이트는 방향성으로만 읽는다 [X2 honest_limits, F3_withdrawn].

(state, update, cost) 정리

이 장의 개념을 세 층 프레임에 배치하면 다음과 같다. 비용 칸의 “논문에없음”은 추정으로 채우지 않는다.

개념	어느 층의 용량인가	갱신 규칙과의 관계	단위	비용 4 층에서의 위치
unintended memorization (9-1)	Θ	갱신 아님. 사후 측정 절차	bits	없음 (측정 도구)
bits per parameter, $\text{bpp} = 3.64$	Θ	($U - \Theta$)를 포화까지 돌린 고정점	bits/param	C_{cap} 의 분자
그릇 상한 $\text{mem}_U \leq H(\Theta)$ (9-4)	Θ	모든 ($U - \Theta$)에 걸리는 제약	bits	C_{cap} 천장
LoRA 실측 용량 $T_{\text{max}}/N_{\text{params}}$ (9-6)	Θ (저랭크 부분 공간)	($U - \Theta$) + 부분공간 제약	tokens/param	C_{cap} ; 바이트는 논문에없음
memory layer, Θ_m	Θ (공유, per-session 0)	pre-training 1회. sleep 라운드 없음	params (dtype 논문에 없음)	C_{cap} ; $B_s \cdot L_w \cdot \rho$ 전부 논문에 없음
Fano 하한 (9-9)	$\Theta \cdot W \cdot E \cdot \text{KV}$ 무관	갱신 규칙과 독립. 읽기 (R)의 품질 바닥	오류율	C_{cap} 과품질을 잇는 유일한 부등식
$H(q)$ 별점	세 경로 공통	질의 전 갱신이라는 정의 자체에서 발생	bits	C_{cap} 의 유효분 감소
bits / 저장 바이트 [X2]	Θ (dtype 별 하한; E 와의 비교는 철회)	갱신 아님. 저장 형식 선택	bits/byte	C_{cap} 을 바이트로 옮기는 환산율

Worked micro-example — 8B 모델의 $r=16$ LoRA delta는 몇 bits인가

암산으로 따라갈 수 있는 계산 하나를 끝까지 한다. 이 절의 수는 전부 이 책의 예시 계산이다 — 앵커 구성은 이 책이 고른 것이고, 어느 논문도 이 구성을 보고하지 않았다. 출처가 붙은 값만 논문 수치다. 대상은 dense 8B 급 앵커다: $d = 4096$, $L_{\text{layer}} = 32$, LoRA 적용 행렬집합이 $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}$ 이므로 층당 적용행렬 수 $m = |\mathcal{S}| = 2$, rank $r = 16$.

1 단계 — 파라미터 수. 적용 행렬 하나당 $A \in \mathbb{R}^{d \times r}$ 와 $B \in \mathbb{R}^{r \times d}$ 이므로 $2rd$ 개다.

$$N_{\text{params}} = 2rd \cdot m \cdot L_{\text{layer}} = 2 \cdot 16 \cdot 4096 \cdot 2 \cdot 32 = 8,388,608 = 2^{23}$$

행렬 하나당 $2 \cdot 16 \cdot 4096 = 131,072$, 층당 $\times 2 = 262,144$, 32 층이니 $\times 32$. 8.39M 파라미터다.
2 단계 — 정보 규모 추정 (외삽). § 09.2의 $\text{bpp} = 3.64$ 를 공급한다.

$$\text{bpp} \cdot N_{\text{params}} = 3.64 \times 8,388,608 \approx 3.05 \times 10^7 \text{ bits} \approx 3.82 \text{ MB}$$

여기서 반드시 붙일 표기. 이 곱은 외삽이다 — 3.64 는무작위 문자열을 전 파라미터 자유도로 from-scratch 학습시킨 소형 GPT 에서나온 값이고 (§ 09.2), 저랭크 부분공간에서 재추정된 적이 없다 (§ 09.4). 게다가 3.64 자체가 하한이므로 (§ 09.2.2) 이 곱은 상한이 아니다. 그러므로 이 값은 “이 delta 가 담는 양” 도 “넘을 수 없는 양” 도아니고, **파라미터 수가 시사하는 정보 규모일 뿐**이다. 이 delta 에실제로 걸리는 천장은 (9-4) 의 $H(\Theta)$ 하나뿐이다.

3 단계 — 저장 바이트와 밀도. bf16 이면 $8,388,608 \times 2 = 16,777,216$ 바이트 = 16.78 MB. 밀도는 $3.05 \times 10^7 / 1.68 \times 10^7 = 1.82$ bits per stored byte 다. int4 로 내리면 저장은 4.19 MB, 밀도는 7.28 이된다 [X2 Q2]. **정보량은 그대로인데 바이트가 4 배 줄었다** — § 09.7 의 결론이 이 두 줄에 들어 있다.

4 단계 — 이게 얼마만큼의 내용인가. E-경로 세션 저장이 14K 토큰이면 UTF-8 텍스트로 약 56 KB 다 [X2 Q1]. delta 의 정보 규모 3.82 MB 는 그 **68 배**다 ($3,816,817 / 56,000 = 68.2$, 본서 산술). 즉 규모만놓고 보면 rank 16 짜리 adapter 하나가 시사하는 정보량이 그런 세션 68 개분에 해당한다. 이것은 밀도 비교가 아니라 자릿수 감각이다 (§ 09.7).

5 단계 — 실측과 대조한다. § 09.4 의 LoRA 실측 스왑은 단일 모듈에대해 1K – 20K 토큰까지만 훑는다 [lora-as-knowledge-memory § 4 Q2; Fig. 14]. 20K 토큰을 액면가로 환산하면 $20,000 \times 4 \times 8 = 6.4 \times 10^5$ bits 이고, 이는 2 단계의 3.05×10^7 bits 의 **약 2%**다 (본서 산술). 단 20K 는추정된 T_{max} 가 아니라 논문이 훑은 범위의 끝이며, 논문은 T_{max} 를 어떤 rank 에 대해서도 표로 주지 않는다. 따라서 이 2%가말하는 것은 값이 아니라 방향 하나다 — **실측 적재량은 파라미터 수가 시사하는 정보 규모에 두 자릿수 못 미친다.**

6 단계 — 서빙 회계로 옮긴다. 같은 8B 모델의 공유 가중치는 bf16 으로 약 16 GB 다. 사용자 당 delta 16.78 MB 는 그 **0.105%**다 (본서 산술). 사용자 10^6 명이면 16.777 TB, int4 면 4.194 TB 다 [X2 Q3]. 참고로 논문 쪽 앵커에서도 같은 비율이 나온다 — 1,556,075,200 파라미터모델의 정보 용량은 3.6 bits/param 기준 약 700 MB 인데 bf16 가중치는 약 3.11 GB 다 (본서 산술, Table 2 의 파라미터수와 초록의 3.6 을 사용).

[해설] 6 단계가 이 장에서 systems 독자가 가져갈 한 줄이다. **delta 는 작다.** 문제는 크기가 아니라 개수와 위치다 — 사용자 수에선형으로 늘고, 요청 지연 안에 웹 계층에서 끌어와 병합해야 하며, 병합된순간 그 요청은 다른 사용자와 GEMM 을 공유하지 못한다.

요약

- 용량을 재려면 memorization 과 generalization 을 먼저 분리해야 하고, 그분리는 참조 모델로 조건화한 뒤 남는 양을 세는 것으로 수행된다 [lm-memorization-capacity § 2.1].
- “3.64 bits per parameter” 는**균등·독립 무작위 토큰 문자열을 GPT-2 아키텍처로 from-scratch 학습시킨**, 6.86×10^6 **파라미터 이하의 모델에서**추정된 값이다. 텍스트에 대한 값도, adapter 에 대한 값도 논문에 없다. 게다가 이 상수는 논문 안에서 여섯 가지로 인쇄된다 ($3.6 / “3.5 - 4” / “3.5 - 3.6” / 3.64 / 3.642 / \text{Table 1}$ 평균 $3.51 \cdot 3.83$) — 존재하는 것은상수가 아니라 밴드다.
- 저자들은 이것이 **하한**이라고 명시한다 (“we are only ever measuring a lower bound on model capacity” [§ 3.2, txt L664]). 따라서 3.64 bpp 는 Θ -경로를 반증하지 못하며, 이장이 상속하는 것은 $\text{mem}_U \leq H(\Theta)$ 라는 그릇 상한과 $\text{mem} \approx \min(C_{\text{cap}}, H(\mathcal{D}))$ 라는 포화규칙이다.

- 용량이 포화한 뒤의 쓰기는 제로섬이다. 반복되는 Θ -경로 sleep 루프는 반드시 잇는다 — 다만 이 논문은 필연성만 주고 망각률 ρ 는주지 않는다.
- LoRA delta 의 용량은 rank 와 함께 커지지만 파라미터당 효율은 작은 rank($r \approx 4$) 에서 최대이며, 모듈을 여러 개 병합해 용량을 키우는축은 $N_m = 1$ 에서 최고이고 그 뒤로 단조 악화한다 [lora-as-knowledge-memory § 6 Q11].
- 용량과 FLOPs 는 분리 가능한 축이다 — 총 파라미터를 106.7 배로 늘리고 FLOPs 를 사실상 고정한 채 NQ 7.76 $\rightarrow$ 20.78 을 얻은 실측이 있다 [memory-layers Table 1]. 단 이 논문은 배포 후어떤 층도 갱신하지 않으므로 sleep-time compute 의 사례가 아니다.
- 상태가 과제 정보 요구량보다 작으면 $\Theta \cdot W \cdot E \cdot KV$ 어디에 두든 같은 형태의 오류율 바닥을 맞고, 질의 전에 압축하는 모든방식은 $H(q)$ 만큼의 유효 예산을 잃는다 [rate-distortion-compaction Eq. 2, § 2].
- 저장 바이트당 정보 밀도는 표현의 한계가 아니라 dtype 이 정한다: bf16 1.82 / int8 3.64 / int4 7.28 bits per byte 다 [X2 Q2]. 3.64 가 하한이므로 이 값들도 하한이며, E -경로와의 밀도비교는 이 회계로 판정할 수 없어 철회했다 [X2 F3_withdrawn].

자가 점검체크리스트

- 3.64 bits per parameter 를 인용할 때 데이터 (무작위 균등문자열)· 아키텍처 (GPT-2 from scratch)· 규모 ($\leq 6.86 \times 10^6$ 파라미터) 를 함께말할 수 있다.
- 이 값이 상한이 아니라 하한인 이유를 논문의 문장으로 댈 수 있고, 그래서무엇을 주장하면 안 되는지 말할 수 있다.
- 주어진 $(d, L_{\text{layer}}, r, |S|)$ 에서 $N_{\text{params}} = 2rd |S| L_{\text{layer}}$ 와 그정보 규모, bf16/int4 저장 바이트를 종이 없이 계산할 수있다.
- 이 장의 세 “용량” 을 구분해 부를 수 있다 — NM ch05 의 Hopfield capacity, NM ch14 의 Atlas associative-memory capacity(쌍의 개수), 이 장의 C_{cap} (bits).
- C_{cap} 이 아키텍처만이 아니라 (아키텍처, 학습 알고리즘) 쌍의 성질인 이유를 (9-3) 으로 설명하고, 그것이 3.64 bpp 의신뢰 범위에 어떤 제약을 거는지 말할 수 있다.
- memory layer 가 용량-FLOPs 분리의 실증이면서 동시에 sleep-time compute 이 아닌 이유를 판별식으로 판정할 수 있다.
- **Rosetta**: “checkpoint 크기 $\leftrightarrow C_{\text{cap}}$ ” 행을 수로 채울 수 있다 — 8B bf16 checkpoint 약 16 GB, 그 안의 $r=16$ LoRA delta 16.78 MB(0.105%), 그 delta 의 정보 규모 3.82 MB. 세 수가 왜 전부 다른 양인지 설명할 수 있다.

다음 장으로

이 장이 세운 것은 Θ 의 천장이다. $\text{mem}_U \leq H(\Theta)$ 는 가중치에만 걸리며, 모델 **밖에** 든 바이트에는 걸리지 않는다. 그렇다면 자연스러운 다음 질문은 이것이다 —상태를 E 로 옮기면 이 천장을 피할 수 있는가. 형식적 답은 절반만그렇다: (9-4) 의 그릇 상한은 사라지지만 (9-9) 의 Fano 하한은그대로 남고, E 는 대신 질의마다 검색 지연과 바이트를 낸다. 다음 장은그 E 의 비용 구조 — 무엇이 E 의 용량을 제한하고, 읽기 한 번에무엇이 얼마나 움직이는가 — 를 세우고, 그 위에서 Part II 의 E -경로계보 (ch12 – ch15) 를 받을 준비를 한다.

ch10. 외부 기억과 검색비용

이 장의 목표 — (1) E 층의 세 구현형태 (text·vector·graph) 를 구분하고 각각의 $\text{ret}(E, q)$ 가 무엇을 계산하는지 말한다. (2) RAG 를 식 (R) 과 (U-E) 로 환원해, 이 논문이 읽기만 완성하고 쓰기를 비워두었다는 사실을 수식으로 확인한다. (3) $\text{ret}(E, q)$ 의 비용을 검색·prefill·쓰기 세 항으로 분해하고, 그중상각식 (A) 의 분모에 들어가는 항이 하나뿐임을 계산한다. (4) chunk 크기 C 와 회수 폭 K_{ret} 이 서빙 비용의 어디를 각각 움직이는지 자기 config 숫자로 판정한다.

왜 필요한가 — Part II 의 E -경로 네 장 (ch12 Memorizing Transformers, ch13 RAG → MemGPT, ch14 Letta Sleep-time Compute, ch15 Mem0·Zep·ReasoningBank·SCM) 이전부 ch01 이 정의하고 이 장이 형태와 비용을 채우는 E 위에서 돈다. 특히 ch14 의 상각 논증은 식 (A) 를 쓰는데, (A) 의 분자에 들어갈 항과 분모를 통과하지 못하는 항을 가르는 것이 이 장의 § 10.5 다. ch25(통일 비용 회계) 는 그 분해를 회계 단위로 그대로 쓴다.

NM 과의 관계 — Neural Memory 에는 이 장에 대응하는 장이 없다. NM 은 W 층만 다뤘고 E 는 v2 에서 신설한 층이다. 따라서 이 장은 NM 에서 아무것도 넘겨받지 않고, 대신 NM 이 세운 회계 도구를 빌려 쓴다: prefill/decode 비대칭과 state 트래픽 회계는 → NM ch10 § 10.2·§ 10.4, KV cache 와 고정 크기 state 의 crossover 산수는 → NM ch10 § 10.3, chunk 크기를 rooeline 독립변수로 놓는 관점은 → NM ch09·ch10. 이 장이 더하는 것은 세 가지다 — 모델밖 저장소의 형태 분류, 검색 연산의 비용 구조, 그리고 상각으로 사라지지 않는 항의 식별.

10.1 E 는 무엇으로 만들어지는가 — text·vector·graph

E 는 하나의 자료구조가 아니라 세 구현 형태의 총칭이며, 형태를 가르는 것은 저장 방식이 아니라 $\text{ret}(E, q)$ 가 무엇을 계산하는가다.

정의. E 자체의 정의는 ch01 이 소유한다 — 모델 밖 텍스트·벡터·그래프로 저장되는 기억이다 (→ ch01). 이 장이 소유하는 것은 그 구현 형태이고, 형태는 세가지다. **text store** 는 레코드를 원문 문자열로 저장하고, 질의와레코드의 어휘 겹침 (BM25 등) 또는 전체 스캔으로 읽는다. **vector store** 는 레코드를 고정 길이 실수 벡터로 바꿔 저장하고, 질의 벡터와의 유사도 상위 몇 개를 읽는다. **graph store** 는 레코드를 노드로, 레코드 사이의 관계를 엣지로 저장하고, 순회로 읽는다. 세 형태는 배타적이지 않다 — 실제 시스템은 같은 레코드를 둘 이상의 형태로 동시에 색인한다.

이 장이 다루는 RAG 는 vector store 와 text store 를 겹쳐 쓴다. 저장된 것은 원문 텍스트이고 (“A key feature of our memory is that it is comprised of raw text rather distributed representations” [RAG § 5]), 검색은 그 텍스트의 embedding 위에서 일어난다. graph store 형태의 실제 시스템은 ch15 가 소유하므로 여기서는 형태만 정의하고 넘어간다.

[해설] 세 형태의 차이는 독자에게 익숙한 언어로는 접근 패턴의 차이다. text store 는 inverted index 조회 — 정수 posting list 의 순차 읽기와 교집합. vector store 는 고차원 벡터 배열의 무작위 접근. graph store 는 포인터 추적. 셋 중 어느 것도 GEMM

이 아니고, 셋 다 arithmetic intensity가 낮다. 독자가 커널 최적화에서 익힌 직관 — tiling으로 재사용률을 올린다 — 은 이 세 형태 어디에도 그대로 이식되지 않는다. 그리고 셋 다 HBM이 아니라 host DRAM 또는 NVMe에 산다. E 는 정의상 가속기 밖의 층이며, 이 위치가 § 10.5의 비용 분해를 강제한다.

C_{cap} 의 단위에는 처음부터 주의가 필요하다. RAG의 E 는 전역이고 읽기 전용이며 모든 사용자가 같은 것을 본다. 따라서 이 논문의 C_{cap} 은 **corpus 당** 값이다 [RAG App. C]. 반면 이후 E -경로 논문들이 보고하는 상태량은 **사용자당 · 세션당**이다 (→ ch13·ch15). 단위가 다른 두 수를 나란히 놓으면 비교가 성립하지 않는다. 이 장은 corpus 당 값만 다룬다.

10.2 RAG — 읽기 경로에 붙은 E

RAG는 E 를 일급 객체로 만든 논문이고, 동시에 E 에 **누가쓰는가**를 통째로 비워 둔 논문이다.

정의. RAG(retrieval-augmented generation)는 사전학습된 seq2seq generator에 사전학습된 dense index를 붙이고, 검색된 레코드를 잠재변수로 두어 그 위에서 주변화한 우도를 최대화하도록 generator와 질의 encoder를 함께 fine-tuning하는 방법이다 [RAG Abstract, § 1]. arXiv:2005.11401.

통일 표기로 환원하면 RAG는 식 (R)에서 W 자리가 비어 있는 형태다.

$$\hat{y} = f(q; \Theta, \emptyset, \text{ret}(E, q)) \quad (10-1)$$

이 식이 말하는 것은 하나다 — RAG에서 test time에 움직이는 것은 파라미터가 아니라 **뽑혀 나온 레코드의 정체**뿐이다. 논문에는 fast weight가 없다. corpus 전체에서 $W = \emptyset$ 이 가장 깨끗하게 성립하는 사례이므로, W -경로 장들은 RAG를 영점으로 쓸 수 있다.

주변화를 어디서 하느냐로 두 변형이 갈린다. RAG-Sequence는 답 전체에 대해 레코드 하나가 책임지고 (주변화가 토큰 곱 **밖**), RAG-Token은 토큰마다 다른 레코드가 책임질 수 있다 (주변화가 토큰 곱 **안**) [RAG § 2.1]. 이 논문의 표시 수식에는 번호가 없으므로 이 책은 절위치로만 인용한다. E 쪽 갱신을 (U-E)에 넣어 보면 퇴화가 눈에 보인다.

$$E_{k+1} = \text{wr}(E_k, \hat{c}_k), \quad \hat{c}_k = S(c_k; B_s) = c_k, \quad B_s = 0 \quad (10-2)$$

식 (10-2)가 말하는 것은 RAG의 $S(\cdot; B_s)$ 가 추론을 전혀 하지 않는다는 것이다. 저장되는 물건은 원문 chunk 그 자체이고, 요약도 파생사실도 없다 [RAG § 3, § 5]. 그리고 wr을 실행하는 주체가 모델이 아니다 — 쓰기는 corpus 파이프라인이나 사람이 하고, 그것도 레코드 삽입이 아니라 **index 전체 교체**로만 한다 [RAG § 4.5]. 논문 자신의 표현으로 이 기억은 “human-readable”이고 “human-writable”이다 [RAG § 5]. 빠진 세 번째 성질이 machine-writable이고, E -경로의 이후 연구 전부가 그 빈칸을 채우는 작업이다 (→ ch13·ch14).

index 전체 교체가 실제로 작동한다는 증거는 하나 있다. 2016년 12월과 2018년 12월 사이에 재임자가 바뀐 세계 지도자 82명을 “Who is {position}?” 템플릿으로 물었을 때, index와 세계가 맞으면 70%(2016 index/2016 지도자)와 68%(2018/2018)를 맞히고, 어긋나면 12%(2018 index/2016 지도자)와 4%(2016 index/2018 지도자)로 떨어진다 [RAG § 4.5]. 이 값들이 이장에서 유일하게 ρ 에 가장 가까운 측정치다 — 다만 이것은 반복 sleep 라운드의 열화율이 아니라 저장소 신선도의 함수다. 82 항목 템플릿 프로브라는 규모도 함께 기록해 둔다.

[해설] 독자의 어휘로 번역하면, RAG는 세계 지식이 바뀔 때 **배포 아티팩트를 건드리지 않는다**. checkpoint를 다시 쓰지 않고, 서빙 가중치를 교체하지 않으며, 사용자별 delta가 생기지 않는다. 그래서 shared-weight batching이 그대로 살아 있다 — 배치 안의 모든 요청이 같은 Θ 와 같은 index를 쓰고, 요청마다 다른 것은 프롬프트에 실려들어오는 토큰뿐이다. Rosetta 표의 “모델 재배포 ↔ (U- Θ) 1 회” 행이 여기서 왜 결정적인지가 드러난다: E -경로는 그 행을 아예 밟지 않는다.

E 를 붙여서 무엇을 사는가에 대한 이 논문의 답은 **용량**이다. RAG-Sequence 는 Natural Questions 에서 44.5 EM 을 기록하는데, 파라미터 수가 가장 가까운 T5-large(770M) 는 같은 벤치에서 28.9 EM 이고 [RAG App. G], 훨씬 큰 T5-11B 의 closed-book 점수도 34.5 에 그친다 [RAG Table 1]. 즉 21M 개 텍스트 레코드가 [RAG § 3], 수십억 파라미터를 더 쌓아서 얻었을지식을 대신 들고 있다.

이 비교에는 논문이 직접 밝힌 제한이 셋 붙는다. 첫째, T5 수치는 재실행이 아니라 발표된 값을 옮긴 것이다 [RAG App. G]. 둘째, RAG 쪽 생성기는 BART-large 가 상한이고 ([RAG § 2.3] 은 400M, [RAG App. G] 는 406M 으로 두 값을 인쇄한다), 수십억 파라미터 생성기 위에서도 이 우위가 남는지는 논문이 답하지 않는다. 셋째, 검색기는 Natural Questions 와 TriviaQA 에 검색감독을 받은 DPR 로 초기화되어 있고, 그 둘이 헤드라인 SotA 를 주장하는 바로 그 벤치다 — 논문 자신이 이 사실을 적는다 [RAG § 4.1]. Θ 의 용량 상한을 다룬 ch09 의 논지가 여기서 반대편에서 확인된다 — E -경로의 창립 논거는 “ Θ 에 다 넣지 말고 밖에 두라”이며, 그 대가로 지불하는 것이 § 10.5 의 세 항이다.

이 논문은 generator 가중치를 “parametric memory”, index 를 “non-parametric memory” 라 불렀다 [RAG § 2.2, § 2.3]. 이 이분법이 문헌에 들어온 자리가 여기다.

[평가] 이 책은 그 이분법을 쓰지 않고 Θ 와 E 로 바꿔 쓴다. 이유는 정확도다 — 이후 경로 들은 W 라는 세 번째 항을 갖는데, 그 항은 “parametric” 에도 “non-parametric” 에도 들어가지 않는다. fast weight 는 파라미터의 모양을 하고 있으므로 “non-parametric” 이 아니고, 배포 아티팩트에 속하지 않으므로 논문이 뜻한 “parametric” 도 아니다. 두 칸짜리 어휘로는 세 층을 셀 수 없다. ch01 이 세운 3 층 표기가 이 장 이후 전장의 기준이다.

10.3 chunking — 저장 단위를 정하는 결정

검색은 레코드 단위로 일어나므로, corpus 를 어떻게 자르느냐가 검색기 성능보다 먼저 결정되는 변수다.

정의. chunking 은 corpus 를 검색 단위 (레코드) 로 자르는 결정이다. 단위 하나의 크기를 C 로 쓴다. C 는 예약 기호이며, NM 에서는 시퀀스를 나누는 단위였고 이 장에서는 corpus 를 나누는 단위다 — 둘 다 “한 번에 묶어 다루는 토큰 수” 라는 같은 역할을 한다. 용량은 어떤 경우에도 C_{cap} 으로 쓰고 맨 C 로 쓰지 않는다. § 10.5 가 쓰는 $C_{\text{search}} \cdot C_{\text{prefill}} \cdot C_{\text{write}}$ 도 같은 규약을 따르는 비용 항이며, 첨자 없는 C 만이 chunk 크기다.

RAG 의 chunking 은 단순하다. 2018 년 12 월 Wikipedia dump 의 각 문서를 겹치지 않는 100-word chunk 로 잘라 총 $n = 21\text{M}$ 개 레코드를 만든다 [RAG § 3]. 즉 $C = 100$ words 이고, 이 선택은 선행 open-domain QA 연구의 관행을 따른 것이다 [RAG § 3].

C 가 정하는 것은 세 가지이고, 셋의 방향이 서로 반대다.

1. **레코드 수** $n \propto 1/C$. C 를 절반으로 줄이면 색인대상이 두 배가 되고, 벡터 저장 용량 C_{cap} 도 두 배가 된다.
2. **회수당 프롬프트 질량** $K_{\text{ret}} \cdot C$. 이것이 § 10.5 의 두 번째 비용 항이며, C 에 정비례한다.
3. **레코드의 자족성**. C 가 작으면 문장이 경계에서 잘리고 대명사의 선행사가 다른 chunk 에 남는다. 검색이 정답 chunk 를 뽑아도 그 chunk 만으로 답이 나오지 않는 실패가 여기서 나온다. 세 번째 항목은 논문이 보고한 실패가 아니라 chunking 정의로부터의 연역이다 — RAG 는 겹치지 않는 절단을 쓰므로 [RAG § 3] 경계에서 잘린 문장을 복구할 경로가 정의상 없다. 이 실패를 계측한 논문은 corpus 에 없다.

세 방향이 반대이므로 C 에는 최적값이 있고, 그 최적값은 corpus 와 질의 분포에 의존한다.

[평가] RAG 는 $C = 100$ 을 고정하고 그 knob 을 스윙하지 않는다. 논문이 스윙한 것은 회수 폭 K_{ret} 뿐이다 (Figure 3, $K_{\text{ret}} \in [10, 50]$) [RAG § 4.5]. 그런데 § 10.5 에서 보듯

서빙비용의 두 번째 항은 K_{ret} 과 C 의 곱에 비례한다. 즉 논문은 곱의 한 인자만 움직여 보고 다른 인자는 관행값에 두었다. E -경로 전체가 이 관행값을 물려받았고, 이후 어느 논문도 C 를 비용측으로 놓지 않는다. 원문 전체에서 chunk 크기가 인쇄되는 자리는 [RAG § 3] 의 한 문장뿐이며, 100 words 이외의 값이 등장하는 실험은 없다.

systems 점점 하나로 이 절을 달는다. C 는 독자가 아는 tile 크기와 다르다. FlashAttention 의 tile 은 계산되는 함수를 바꾸지 않지만, chunking 은 **검색 대상 집합 자체를 바꾼다**. C 를 바꾸면 같은질의에 다른 레코드가 뿔히고, 프롬프트 내용이 달라지고, 답이 달라진다. E 층에서 C 는 성능 knob 이 아니라 semantic knob 이다.

10.4 embedding 과 유사도검색

vector store 의 $\text{ret}(E, q)$ 는 두 개의 encoder 와 하나의 $\text{top-}K_{\text{ret}}$ 연산으로 완전히 결정된다.

정의. embedding 은 텍스트를 고정 길이 실수 벡터로 보내는 함수다. 저장 쪽 함수를 ϕ_d , 질의 쪽 함수를 ϕ_q 로 쓰고, 출력 폭을 d_e 로 쓴다 (d 는 모델 폭으로 예약되어 있으므로 구분한다). **유사도 검색** 은 질의 벡터에 가장 가까운 저장 벡터 K_{ret} 개를 찾는 연산이며, 가까움을 내적으로 재면 maximum inner product search(MIPS) 다. K_{ret} 은 회수 폭을 가리키는 이장의 기호다 — 예약 기호 k 는 sleep 라운드 침자이므로 회수 개수에 절대쓰지 않는다.

RAG 의 검색기는 이 형태 그대로다 [RAG § 2.2].

$$\text{ret}(E, q) = \underset{e \in E}{\text{top-}K_{\text{ret}}} \exp(\phi_d(e)^\top \phi_q(q)) \quad (10-3)$$

이 식이 말하는 것은 검색이 학습 가능한 대상이 되었다는 것이다. 어느레코드가 정답인지에 대한 감독 없이, 과제 loss 의 gradient 가 주변화통과해 ϕ_q 까지 흘러간다 [RAG § 2.4]. 그러나 두 encoder 의지위는 대칭이 아니다. ϕ_q 는 Θ 의 일부로 학습되고, ϕ_d 는 DPR 에서 가져와 **동결된다** — 논문의 문장은 “we ... keep the document encoder (and index) fixed, only fine-tuning the query encoder and the BART generator” 이다 [RAG § 2.4].

동결의 이유가 곧 E 층의 첫 번째 구조적 제약이다. ϕ_d 를 한 번바꾸면 저장된 n 개 벡터를 전부 다시 계산해야 한다. 논문도그렇게 쓴다 — document encoder 를 훈련 중에 갱신하는 것은 index 를주기적으로 다시 만들어야 해서 비싸다는 것이다 [RAG § 2.4]. 비용은주장되고 측정되지 않는다. 반면 ϕ_q 는 공짜로 바꿀 수 있다. 읽기함수의 두 반쪽이 서로 다른 가격표를 달고 있다는 이 비대칭은 E 층 설계전체를 지배한다.

검색 자체의 비용 구조는 exhaustive 형태에서 가장 선명하다. 전수 MIPS 는 n 개 벡터를 전부 읽어 내적을 계산하므로 $2nd_e$ FLOPs 를 쓰고**저장된 벡터 배열 전체**를 트래픽으로 소비한다. 원소당 b bytes 라 하면 arithmetic intensity 는

$$I = \frac{2nd_e}{nd_e b} = \frac{2}{b} \text{ [FLOP/byte]} \quad (10-4)$$

이다. 식 (10-4) 가 말하는 것은 검색의 intensity 가 **레코드 수에도 벡터 폭에도 의존하지 않는다**는 것이다. 오직 저장 정밀도만이 정한다: fp32 면 0.5, 8 비트면 2 FLOP/byte. bf16 GEMV 의 1 FLOP/byte 와 같은 자릿수이며, 독자가 아는 decode GEMV 가 이미 bandwidth-bound 라는 사실 ($\rightarrow$ NM ch10 § 10.6) 이 그대로 이 연산에도 적용된다. 전수 검색은 계산이 아니라 **대역폭 문제**다.

식 (10-4) 에는 반직관적인 따름정리가 하나 붙는다. index 를 압축하면트래픽은 줄지만 arithmetic intensity 는 **올라간다** — b 가작아지기 때문이다. RAG 가 보고한 CPU 메모리 100 GB $\rightarrow$ 36 GB 압축은 [RAG App. C] index 가 상주하는 바이트를 2.8 배 (이 책의 나눗셈) 줄인다는 뜻이고, 동시에 그 위의 검색을 대역폭 한계에서 조금 더 떼어 놓는다는 뜻이다. 벡터 양자화가 E 층에서 표준이 된 이유는 저장 공간이 아니라 이 두 효과의 결합이다.

ANN 이 이 그림을 바꾸는 방식은 트래픽의 양이지 성질이 아니다. RAG 는 FAISS 에 Hierarchical Navigable Small World 근사를 얹어 방문 벡터 수를 n 보다 훨씬 작게 줄이고 [RAG § 3], 논문은 MIPS 가 “sub-linear time” 에 근사적으로 풀린다고 쓴다 [RAG § 2.2]. 그러나 방문 순서는 질의에 따라 달라지는 그래프 순회이므로 접근이 무작위다. 읽는 양은 줄고 접근 지역성은 나빠진다.

systems 접점은 위치다. RAG 는 index 벡터를 CPU 에 둔다 — “doing Maximum Inner Product Search with FAISS is sufficiently fast on CPU, so we store document index vectors on CPU” [RAG App. C]. 무압축 약 100 GB, FAISS 압축 후 36 GB 이고, 8비트로 저장할 수 있다고 부록이 덧붙인다 [RAG App. C, App. G]. 즉 E 는 HBM 밖 host DRAM 에 사는 read-mostly 구조이고, GPU 가 400M 파라미터 생성기를 [RAG § 2.3] 붙들고 있는 동안 CPU 가 그래프를 걷는다. 독자의 회계로 옮기면 TTFT 안에 **프롬프트 길이가 아니라 index 크기에 비례하는 host-side 항**이 하나 들어온 것이다. 논문은이 항을 한 번도 측정하지 않는다.

보고된 이 footprint 와 식 (10-4) 의 $nd_e b$ 는 같은 수가 아니며, 이장은 둘을 구분해 쓴다. App. G 는 index 가 21M 개 벡터, 총 15.3B 개 값으로 이루어진다고 인쇄한다 [RAG App. G] — 즉 $nd_e = 1.53 \times 10^{10}$ 이다. 이 곱을 fp32 로 저장하면 벡터 배열 자체는 $nd_e b \approx 61$ GB 이고 (이 책의 곱셈), 8비트로 저장하면 15 GB 다. 논문이 보고하는 값은 무압축 약 100 GB, 압축 후 36 GB 로 양쪽 다 그보다 크다 [RAG App. C]. 차이의 이유는 회계 대상이 다르다는 것이다 — 보고값은 “all of Wikipedia” 에 대한 **index 전체의 CPU 상주메모리**이고, 여기에는 벡터 배열 외에 [RAG § 3] 이 얹은 HNSW 그래프와 저장된 원문이 함께 들어간다 [RAG § 3, App. C]. 논문은 이 100 GB 를 항목 별로 분해하지 않는다. 따라서 이 장은 논문 수치를 인용할 때는 100 GB/36 GB 를 그대로 쓰고, 식 (10-4) 를 돌릴 때는 벡터 배열만세는 이 책의 예시 구성을 쓴다. 폭 d_e 자체는 인쇄하지 않는다 — 곱 nd_e 를 논문이 직접 인쇄하고 아래 계산이 곱만으로 전부 성립하므로, 폭을 확정할 이유가 없다.

10.5 $\text{ret}(E, q)$ 의 비용 — 세 항으로의 분해

검색은 공짜가 아니고, 비용이 한 곳에 있지도 않다. 질의 하나가 E 층에 지불하는 값은 세 항의 합이다.

$$C_E(q) = \underbrace{C_{\text{search}}}_{\text{① 검색 자체}} + \underbrace{C_{\text{prefill}}}_{\text{② prefill 증가}} + \underbrace{\frac{C_{\text{write}}}{N_q}}_{\text{③ 쓰기}} \quad (10-5)$$

식 (10-5) 가 말하는 것은 세 항이 서로 다른 자원을 서로 다른 주기로 먹는다는 것이다. 셋을 하나씩 본다.

① **검색 자체.** § 10.4 가 유도한 항이다. host DRAM 위의 벡터 연산이고, bandwidth-bound 이며, 크기는 index 크기와 ANN 파라미터가 정한다. 주기는 **질의마다**다. 이 항은 GPU 를 쓰지 않지만 TTFT 의 앞쪽에 직렬로 들어간다 — 질의를 embedding 하고, host 에서 그래프를 걷고, K_{ret} 개 레코드를 다시 가속기로 보낸 뒤에야 prefill 이 시작된다. 이 왕복이 프롬프트 길이가 아니라 index 크기에 따라 늘어난다는 것이 E 층이 서빙 경로에 넣는 세 항의 성질이다.

② **prefill 증가.** 이 장이 소유하는 정의다.

정의. prefill 증가는 $\text{ret}(E, q)$ 의 결과를 프롬프트에 실어 보냄으로써 생기는 추가 prefill 비용이다. 크기는 회수된 토큰 질량 $K_{\text{ret}} \cdot C$ 가 정하고, 발생 주기는 **질의마다**이며, 어떤 상각으로도 줄지 않는다.

RAG 에서 이 항의 형태는 노골적이다. 입력과 회수된 레코드를 결합하는 방법이 “we simply concatenate them” 이고 [RAG § 2.3], 회수 폭은 open-domain QA 테스트에서 RAG-Token 15 개, RAG-Sequence 50 개, MS-MARCO 와 Jeopardy 에서 양쪽 10 개다 [RAG App. A]. 즉 질의 하나가 최대 50 개의 (질의 + 100-word 레코드) 컨텍스트를 generator 에 통과시킨다.

decode 쪽 배수도 여기서 갈린다. RAG-Token 은 주변화된 전이 확률을 표준 beam decoder 에 그대로 꽂으므로 decode 루프는 하나지만 매 스텝이 K_{ret} -원 혼합이고, 따라서 K_{ret} 개의 KV cache 를 동시에 들고 있어야 한다 [RAG § 2.5]. RAG-Sequence 의 Thorough Decoding 은 레코드마

다 beam search 를 한 번씩 돌린 뒤, 그 레코드의 beam 에없는 가설마다 forward pass 를 추가로 돌린다 — 논문 자신이 “For longer output sequences, $|Y|$ can become large, requiring many forward passes” 라고 쓴다 [RAG § 2.5]. 생성 과제에서는 이 비용 때문에 Fast Decoding 을 채택했고, 그 이유는 Thorough Decoding 이 성능을 올리지못했기 때문이다 [RAG App. A].

[해설] 오늘날 실무의 형태는 조금 다르다. K_{ret} 개 레코드를 각각 독립 컨텍스트로 돌리는 대신 하나의프롬프트로 이어 붙인다. 그러면 컨텍스트 길이가 $K_{\text{ret}} \cdot C$ 만큼 늘고, prefill FLOPs 는 그 길이에비례하며 (dense 모델에서 파라미터 P 당 $\approx 2PT$), attention 항은 길이의 제곱으로 붙고, KV cache 는 길이에 비례해 커진다. 두 형태는비용의 배분이 다를 뿐 **질의마다 새로 지불한다**는 성질에서 같다.

[해설] prompt cache 관점에서 한 번 더 본다. 회수 결과는질의마다 달라지므로 프롬프트의 그 구간은 캐시 hit 이 나지 않는다. 앞이고정 system prompt 를 두면 거기까지만 hit 이고, 회수 블록이 시작되는지점부터 끝까지는 전부 miss 다. 즉 E -경로는 prefix cache 를 늘려 주는것이 아니라 **prefix cache 가 덮는 구간의 경계를 앞으로 밀어낸다**. Rosetta 표의 “prefix cache $\leftrightarrow \hat{c}$ ” 행을 조심해서 읽어야 하는이유가 이것이다 — 둘다 재사용이 목적이지만, cache 는 같은 계산의재사용이고 회수 결과는 매번 다른 내용이다.

③ **쓰기 비용**. RAG 에서 wr 은 corpus 파이프라인이 하는일이다: n 개 chunk 를 동결된 ϕ_d 로 한 번씩 embedding 하고 FAISS index 를 만든다 [RAG § 3]. 문맥당 B_s 는 0 이다 — 저장 전에 어떤추론도 하지 않기 때문이다 (식 (10-2)). 그리고 이 비용은 corpus 스냅샷당한 번 지불되고 배포 수명 전체에 걸쳐 나뉜다. 논문은 이 항의 시간도 FLOPs 도 보고하지 않는다.

세 항을 상각식에 넣으면 이 장의 결론이 나온다. 식 (A) 의 형태로다시 쓰면

$$C_{\text{avg}} = \underbrace{C_{\text{search}} + C_{\text{prefill}}}_{C_{\text{wake}}} + \underbrace{\frac{C_{\text{write}}}{N_q}}_{C_{\text{sleep}}/N_q}, \quad \lim_{N_q \rightarrow \infty} C_{\text{avg}} = C_{\text{search}} + C_{\text{prefill}} > 0 \quad (10-6)$$

이 극한이 말하는 것은 하나다 — **상각은 ③만 지운다**. ①과 ②는 N_q 를 아무리 키워도 남는다. 이것은 회계 방식의 선택이 아니라 주기의정의다. ③은 문맥당 한 번 지불되므로 그 문맥을 공유하는 질의 수로 나뉘고, ②는 질의당 한 번 지불되므로 나눌 것이 없다.

RAG 는 이 구조의 극단에 있다. ③이 corpus 당 한 번이라 N_q 가 사실상배포 수명 전체의 질의 수이고, 그래서 손익분기 N_q^* 를 묻는 것자체가 공허하다. 상각 논증이 필요해지는 것은 ③이 **문맥당** 항으로올려져간 뒤다 — 그것이 E -경로의 이후 논문들이 한 일이다 ($\rightarrow$ ch13:ch14). 그때 ②는 그대로 남는다. 이 장은 여기까지만확정하고 판정은 Part III 로 넘긴다 ($\rightarrow$ ch25).

[해설] 세 항이 batching 에 거는 압력도 정리해 둔다. 좋은소식부터 — E 는 전역이고 Θ 는 fine-tuning 후 고정이므로, 배치안의 모든 요청이 같은 가중치와 같은 index 를 쓴다. shared-weight batching 은 온전하다. 나쁜 소식은 요청 하나의 유효 시퀀스 수가 1 이 아니라 K_{ret} 이라는 것이다. continuous-batching 스케줄러가 보는요청의 토큰 비용을 사용자의 프롬프트가 아니라 **검색하이퍼파라미터**가 정하고, 운영자가 K_{ret} 다이얼을 돌리면재학습 없이 SLA 와 정확도를 맞바꿀 수 있다 [RAG § 4.5]. 이 다이얼의품질 수익은 단조가 아니다 — RAG-Sequence 는 K_{ret} 을 50 까지올릴수록 좋아지지만 RAG-Token 은 10 에서 정점을 찍는다 [RAG Figure 3].

[평가] RAG 는 세 항 중 어느 것도 측정하지 않는다. 지연 · 처리량 · FLOPs · index 빌드시간이 논문 어디에도 없고, 부록이 보고하는 비용은 메모리 footprint 뿐이다 [RAG App. C]. 회수 폭을 바꾸면 “performance and runtime” 이 영향을받는다고 명시해 놓고도 [RAG § 4.5], Figure 3 의 세 패널은 전부 K_{ret} 을 품질 측에만 대응시킨다 [RAG Figure 3]. 서버비용을 정하는 유일한 knob 을 세 번 스위치하면서 비용 축을 한 번도 그리지않은 것이다. E -경로의 이후 비용 주장은 전부 이 침묵 위에 세워진다.

10.6 학습된 검색기가 항상 이기지않는다

비용을 지불할 가치가 있는지는 검색기가 실제로 이기는지에 달려 있고, 이논문의 표에는 지지 않는 반례가 들어 있다.

FEVER 에서 BM25 가 학습된 검색기를 이긴다. dev 세트 ablation 에서 BM25 로 갈아 끼운 RAG-Token-BM25 가 FEVER-3-way 75.1, FEVER-2-way 91.6 을 기록하고, 학습된 검색기를 쓰는 RAG-Token 은 74.5 와 90.6 에 그친다 [RAG Table 6]. 논문도 인정한다 — “For FEVER, BM25 performs best, perhaps since FEVER claims are heavily entity-centric and thus well-suited for word overlap-based retrieval” [RAG § 4.5]. 파라미터가 하나도 없는 어휘 겹침 검색기가, 이 논문의 중심 기여인 학습된 검색기를 한 과제군에서 이긴다.

“모든 과제에서 개선된다” 는 문장도 자기 표에서 정확히 성립하지 않는다. 논문은 동결 검색기 ablation 을 두고 “learned retrieval improves results for all tasks” 라고 쓰지만 [RAG § 4.5], 같은 표의 MS-MARCO Bleu-1 에서 RAG-Token-Frozen 과 RAG-Token 은 둘 다 49.4 로 동점이다 [RAG Table 6]. 개선이 아니라 무승부다.

한 과제군에서는 기제가 통째로 무너진다. story generation 에서는 검색기가 붕괴해 입력과 무관하게 같은 레코드를 뽑고, 그러면 generator 가 레코드를 무시하는 법을 배워서 “the RAG model would perform equivalently to BART” 가 된다 [RAG App. H]. 논문은 이것을 부록에 수치 없이, 데이터셋 상세 없이, 수정 시도 없이 적는다. 이 실패 양식에는 이름이 붙어있다 — **retrieval collapse** 이며, E -경로가 처음 보고한 읽기 쪽 실패다.

반대 방향의 사실도 함께 기록해 둔다. 회수 품질은 절대적이지 않다 — FEVER gold evidence 기준으로 1 위 레코드가 정답 문서인 비율이 71%, 상위 10 개안에 있는 비율이 90%다 [RAG § 4.4]. 그런데 NQ 에서는 정답이 회수된 어떤 레코드에도 없을 때조차 11.8%를 맞힌다 [RAG § 4.1]. 즉 Θ 와 E 는 서로를 부분적으로 대체한다.

[평가] 세 사실을 § 10.5 의 비용 측에 얹으면 다음이 따라나온다. 항 ①과 ②는 검색기의 품질과 무관하게 **항상 지불된다**. FEVER 에서는 그 비용을 내고 산 학습된 검색기가 무료 baseline 에 지고, story generation 에서는 지불한 비용 전부가 버려진다. “검색을 붙이면 좋아진다” 는 명제는 과제 의존적이고, 비용은 과제 의존적이지 않다. 이비대칭이 E -경로 평가의 기본 구도이며, ch15 에서 품질 1 위가 반복적으로 “기억 시스템 없음” 으로 나오는 현상의 조상이 여기에 있다.

(state, update, cost) 정리

이 장의 개념을 세 층 프레임에 배치하면 RAG 가 어느 칸을 채우고 어느 칸을 비웠는지가 한눈에 보인다.

표 10-1 — E 층 개념의 (state, update, cost) 배치. 수치는 전부 [RAG] 보고값이고 합산·차감만 이 책의 산술이며, 빈칸은 “논문에 없음” 이다.

대상	층	갱신식	시간척도	비용
chunk embedding index (21M 레코드, $C = 100$ words)	E	(U-E). 단 $\hat{c}_k = c_k$ 항등, $B_s = 0$	corpus 스냅샷마다 (실증은 2016 → 2018 2 회)	$C_{\text{cap}} = 36 \text{ GB}$ (압축) / 약 100 GB(무압축), corpus 당 [RAG App. C]

대상	층	갱신식	시간척도	비용
generator + 질의 encoder ϕ_q	Θ	(U- Θ) 의 모양만 . $\mathcal{R}$ 이 사람이 라벨 한 고정 corpus 라 sleep 라운드 가 아니다	배포 전 1 회	논문 표기 “trainable 626M” [RAG App. G]. 단 그 합에는 같은 문장 이 학습하지 않는 다고 밝힌 ϕ_d 110M 이들어 있 어, 실제로 움직 이는 것은 generator 406M + ϕ_q 110M 이다
저장 encoder ϕ_d	Θ (동결)	없음	—	110M [RAG App. G]. 바꾸면 n 개 벡터 전량 재 계산 [RAG § 2.4]
fast weights ret(E, q)	W 읽기 (R)	없음 ($W = \emptyset$) —	— 질의마다	0 ① host DRAM 대역폭 + ② prefill $K_{\text{ret}} \cdot C$ 토큰. 둘 다 논문 미측정
index 교체	E	wr 을 사람 · 파이 프라인이실행	세계가 바뀔 때	신선도 실패 시 68% $\rightarrow$ 4% [RAG § 4.5]

표에서 읽어야 할 것은 두 가지다. 첫째, $B_s = 0$ 칸이 RAG 를 sleep-time compute 판별식에
서 탈락시킨다 — 질의 전에 예산을 쓰는 오프라인 추론이없다 ($\rightarrow$ ch11). 둘째, 비용 열에서 실제로
채워진 칸은 C_{cap} 하나뿐이고 그것도 단위가 corpus 당이다. $B_s \cdot L_w \cdot \rho$ 는 이 논문에 없다.

Worked micro-example — 질의 하나가 지불하는 세항

식 (10-5) 의 세 항을 실제 숫자로 채워, ②가 ③을 몇 자릿수로 지배하는지암산으로 확인한다. **아래
계산은 전부 이 책의 예시 계산이며 논문의보고값이 아니다**. 논문에서 오는 값과 이 예제가 고르는
값을 먼저 갈라둔다.

논문에서 오는 값: $n = 21\text{M}$ 레코드, $C = 100$ words, $K_{\text{ret}} = 50$ (RAG-Sequence, open-
domain QA 테스트), index 값 개수 $nd_e = 1.53 \times 10^{10}$, 저장 encoder $\phi_d = 110\text{M}$, 보고된 CPU
메모리 무압축 약 100 GB / 압축 후 36 GB [RAG § 3, App. A, App. C, App. G]. **이 예제가 고르는
값** (논문에 없다): 벡터를 fp32 로 저장 ($b = 4$ bytes/elem), 1 word ≈ 1.3 tokens, generator 는
8B dense 모델이라 prefill FLOPs $\approx 2PT$, 가속기 실효 처리량 125 TFLOP/s(peak 312 TFLOP/
s $\times$ MFU 40%), host DRAM 대역폭 100 GB/s, 질의 자체는 15 words.

전제를 하나로 고정한다. 트래픽 항은 **벡터 배열만** 세고, 그 크기는위 두 값에서 직접 나온다:
 $nd_e b = 1.53 \times 10^{10} \times 4 = 61$ GB. 논문이 보고하는 약 100 GB 는 § 10.4 에서 밝힌 대로 HNSW 그
래프와원문까지 포함한 index 전체의 상주 메모리이므로 이 예제의 트래픽 항과회계 대상이 다르다
[RAG § 3, App. C]. 아래 3 단계는 61 GB 쪽으로계산하고, 100 GB 를 쓰면 같은 값이 1.6 배 커진
다 — 결론이 되는 자릿수는바뀌지 않는다.

1 단계 — 프롬프트 질량. 회수 질량 = $K_{\text{ret}} \cdot C = 50 \times 100 = 5,000 \text{ words} \approx 6,500 \text{ tokens}$. 질의 자체는 $15 \times 1.3 \approx 20 \text{ tokens}$. 프롬프트가 $20 \rightarrow 6,520 \text{ tokens}$, 즉 **326 배** 된다.

2 단계 — 항 ②, prefill. $2PT = 2 \times 8 \times 10^9 \times 6,520 \approx 1.04 \times 10^{14} \text{ FLOP} = \mathbf{104 \text{ TFLOP}}$. 검색 없이 질의만 넣으면 $2 \times 8 \times 10^9 \times 20 = 3.2 \times 10^{11} = 0.32 \text{ TFLOP}$. 시간으로는 $104/125 \approx 0.84 \text{ s}$ 대 2.6 ms 다. TTFT가 밀리초에서 초로 넘어간다.

3 단계 — 항 ①, 검색. 전수 MIPS의 계산량은 $2nd_e = 2 \times 1.53 \times 10^{10} = 3.06 \times 10^{10} = 30.6 \text{ GFLOP}$ 뿐이다. 같은 가속기면 0.24 ms — 계산량으로는 무시할수준이다. 그런데 같은 연산이 벡터 배열 61 GB를 훑으므로 100 GB/s DRAM에서 **0.61 s**가 걸린다. 계산시간의 2,500 배다. 식 (10-4)가 말한 것이 숫자로 확인된다: 검색은 FLOPs가 아니라 대역폭 문제이고, ANN은 바로 이 0.6 초를 깎으려고 존재한다. 논문은 그 결과를 “sufficiently fast on CPU” 라고만 쓰고 값을 주지 않으므로 [RAG App. C], 여기서 멈춘다.

4 단계 — 항 ③, 쓰기. index를 만들려면 저장된 전체 텍스트를 ϕ_d 에 한 번 통과시켜야 한다. 레코드당 $C = 100 \text{ words} \approx 130 \text{ tokens}$ 이므로 총량은 $21 \times 10^6 \times 130 = 2.73 \times 10^9 \text{ tokens}$ 이고, $2 \times 1.1 \times 10^8 \times 2.73 \times 10^9 \approx 6.0 \times 10^{17} \text{ FLOP} = \mathbf{600 \text{ PFLOP}}$. 같은 가속기 한 장이면 $6.0 \times 10^{17} / 1.25 \times 10^{14} = 4,800 \text{ s} \approx 1.3 \text{ 시간}$ 이다 (FAISS/HNSW 그래프 빌드는 여기 포함되지 않았고 논문도 보고하지 않는다).

5 단계 — 나눗셈 하나. ③이 질의당 prefill 아래로 내려가는 지점은

$$N_q^* = \frac{C_{\text{write}}}{C_{\text{prefill}}} = \frac{6.0 \times 10^{17}}{1.04 \times 10^{14}} \approx 5.8 \times 10^3 \quad (10-7)$$

질의 약 5,800 개다. 배포 수명 동안 $N_q = 10^9$ 개를 처리하면 ③은 질의당 $6.0 \times 10^{17} / 10^9 = 0.6 \text{ GFLOP}$ 로 줄어 ②의 1.7×10^5 분의 1이 된다.

읽어야 할 결론. ③은 다섯 자릿수 아래로 사라지고 ②는 104 TFLOP에 그대로 남는다. N_q 를 더 키워도 이 그림은 바뀌지 않는다 — 식 (10-6)의 극한이 그것이다. 반대로 ②를 줄이는 방법은 K_{ret} 이나 C 를 줄이는 것뿐이고, 둘 다 품질 knob이다 (§ 10.3, § 10.5).

[평가] 위 절대치 중 논문에서 오는 것은 $n \cdot C \cdot K_{\text{ret}} \cdot nd_e \cdot \text{footprint} \cdot \phi_d$ 크기뿐이고 나머지는 이 예제가 고른 값이다. 따라서 이 계산이 단정하는 것은 **비율과 순서**다 — ②가 ③을 다섯 자릿수 차이로 지배하고, ②만 N_q 에 무감하다. 절대치는 각자의 스택 값으로 알아 끼워야 하는 하한 · 방향성이다. 이 책은 자체 계산에서 비율 · crossover · 순서만 단정문으로 쓰고 절대치는 그렇게 쓰지 않는다.

요약

- E 의 구현 형태는 text·vector·graph 세가지이고, 형태를 가르는 것은 저장 방식이 아니라 $\text{ret}(E, q)$ 가 무엇을 계산하는가다. 셋 다 GEMM이 아니고 셋 다 arithmetic intensity가 낮다.
- RAG는 식 (R)에서 $W = \emptyset$ 인 형태이며, 읽기 함수 $\text{ret}(E, q)$ 를 학습 가능한 일급 객체로 만든 논문이다. 반면 (U-E)는 퇴화해 있다 — $\hat{c}$ 가 항등사상이고 문맥당 $B_s = 0$ 이며 wr을 실행하는 주체가 모델이 아니다.
- chunk 크기 C 는 레코드 수 n ·프롬프트 질량 $K_{\text{ret}} \cdot C$ ·레코드의 자족성을 서로 반대방향으로 움직이는 semantic knob이다. RAG는 $C = 100 \text{ words}$ 를 관행값으로 고정하고 스윙하지 않는다.
- 읽기 함수의 두 반쪽은 가격이 다르다. 질의 encoder ϕ_q 는 공짜로 바꿀 수 있지만 저장 encoder ϕ_d 를 바꾸면 n 개 벡터를 전량 재계산해야 하고, 그래서 RAG는 ϕ_d 를 동결한다 [RAG § 2.4].
- $\text{ret}(E, q)$ 의 비용은 세 항이다: ① 검색 자체 (host DRAM 대역폭), ② prefill 증가 ($K_{\text{ret}} \cdot C$ 토큰), ③ 쓰기. 상각식 (A)의 분모를 통과하는 것은 ③뿐이고, ①과 ②는 $N_q \rightarrow \infty$ 에서도 남는다.
- prefill 증가는 질의마다 발생하며 상각되지 않는다. 회수 결과가 질의마다 다르므로 prompt cache로도 회수되지 않는다.

- 학습된 검색기가 항상 이기지는 않는다. FEVER 에서 BM25 가이기고 (75.1/91.6 대 74.5/90.6), 논문도 “For FEVER, BM25 performs best” 라고 인정하며 [RAG Table 6, § 4.5], story generation 에서는 retrieval collapse 로 기체 자체가 무너진다 [RAG App. H].
- RAG 는 지연 · 처리량 · FLOPs · index 빌드 시간을 하나도 보고하지 않는다. 비용 축이 비어 있는 채로 E -경로가 시작되었고, 이후 논문들의 비용 주장은 그 침묵을 물려받았다.

자가 점검체크리스트

- E 의 세 구현 형태를 각각 한 문장으로 정의하고, 각 형태의 ret 이 계산하는 것을 말할 수 있다.
- RAG 를 식 (R) 과 (U-E) 로 각각 환원해 쓸 수 있고, 어느 자리가 왜 비어있는지 ($W = \emptyset$, $\hat{c}$ 항 등, $B_s = 0$) 설명할 수 있다.
- C (chunk 크기) 와 C_{cap} (용량) 을 혼동하지 않고, K_{ret} 을 sleep 라운드 첨자 k 와 구분해 쓸 수 있다.
- 식 (10-5) 의 세 항을 자기 서빙 config 숫자로 채우고, (10-6) 에서 N_q 로 나뉘는 항이 하나뿐인 이유를 주기의 정의로 설명할 수 있다.
- ϕ_q 와 ϕ_d 의 갱신 비용이 왜 비대칭인지 말할 수 있고, 그것이 E 층 설계에 거는 제약을 하나 들 수 있다.
- FEVER 에서 BM25 가 이긴다는 사실을 수치와 함께 인용할 수 있고, 그 사실이비용 논증에 무엇을 더하는지 말할 수 있다.
- Rosetta: “vector DB 조회 $\leftrightarrow \text{ret}(E, q)$ ” 행이 왜 완전한대응이고, “prefix cache $\leftrightarrow \hat{c}$ ” 행이 왜 부분적 대응인지 (회수결과는 매 질의 달라진다) 각각 한 문장으로 설명할 수 있다.

다음 장으로

이 장은 E 층의 구현 형태를 셋으로 가르고 그 위의 읽기 비용을 세항으로 갈랐다. 그러나 정작 RAG 는 이 책의 주제가 아니다 — 문맥당 $B_s = 0$ 이고 저장 전에 어떤 추론도 하지 않으므로, “질의 전에 예산을 써서 어느 층의 상태를 바꾼다” 는 조건을 만족하지 않는다. 이 장이 남기는 질문은 두 개다. 무엇이 E 에 쓸 내용을 만드는가 ($S(c; B_s)$ 를항등에서 떼어내면 무엇이 되는가), 그리고 그 판정을 무슨 기준으로 하는가. Part II 의 첫 장인 ch11 이 두 번째 질문을 먼저 처리한다 — 판별식을 명시하고 corpus 29 편 전체를 통과 · 불통과로 가르 뒤, 통과한것들을 $E \cdot W \cdot \Theta$ 세 경로로나눈다. 첫 번째 질문은 그 분류 위에서 ch12·ch13 이 이어받는다.

이 장이 다루지 않은 것도 지목해 둔다. RAG 의 평가설계에 붙는 나머지불리한 사실들 — baseline 선택, 데이터셋 구성, 비교 조건의 대칭성 — 은 이장이 아니라 ch13(RAG 에서 MemGPT 로) 이 소유한다. 이 장은 E 층의 형태와 비용 분해를 세우는 개념 장이므로, 평가설계쟁점은 같은 논문을 다시 다루는 ch13 에 모아 한 번에 처리한다.

Part II — 계보: 판별식을 대본다

Part II 가 답하는 질문은 둘이다. “sleep-time compute” 이라는 이름아래 실제로 무엇이 있는가, 그리고 그 각각은 어느 층을 언제 무슨비용에 쓰는가. 두 질문 모두 사실의 질문이지 가치의 질문이 아니다. 이부는 아무것도 판정하지 않는다 — 유망한가, 어느 경로가 나은가, 비용이감당되는가는 전부 Part III 의 몫이다. 분류는 판정의 재료이지 판정이아니다.

ch11 이 문을 연다. Part I 이 만든 판별식을 적용 가능한 다섯 단계절차로 정밀화하고 corpus 전체에 기계적으로 적용해통과·불충족·배경을 가른다. 저자의 자기 규정도성능도 입력되지 않으므로 판정에 재량이 개입하지 않는다. 이 장에서 이책의 첫 실질 발견이 나온다 — 이름이 기계보다 넓다. 그리고통과한 것들을 경로별로 세로로 세우면 세 줄이 나오는데, 세 줄이 서로를거의 인용하지 않는다는 사실이 함께 나온다. 어느 침묵이 연대에 강제된것이고 어느 침묵이 선택된 것인지를 가르는 것도 이 장이다.

나머지 열두 장은 세 세로줄과 두 개의 결산이다. *E-경로*는 ch12(학습 중 캐시 적재라는 초기형) → ch13(항등사상 $\hat{c}$ 에서벗어나는 과정, 그리고 판별식 (1) 을 만족하지 않는 경계 사례) → ch14(이 라인의 이름을 만든 논문. 상속받을 open question 이 없는 시작) → ch15(프로덕션 네 시스템의 wr 형태 비교) 로 간다. *W-경로*는 ch16(경첩 장 — 산출물은 Θ 모양인데갱신식은 (U-W) 이고 시 계는 wake 다) → ch17(경로의 유일한 정면 사례) → ch18(갱신 주파수 연속체가 표기의 다리인지 기제의 다리인지) 이다. *Θ -경로*는 ch19(가중치에 쓰는 두 primitive 와 각각의상한) → ch20(자기 생성 학습 데이터) → ch21(wake W → sleep Θ 의 순서) 이다. 그리고 **ch22** 가 Θ -경로의 falsifier 후보를 세우고, **ch23** 이 ch08 에서 빌린 은유의 담보를확인한다 — 생물학 논문들이 통제군 대비 실제로 보인 것과 보이지 않은 것.

열세 장 모두 같은 8 절 격자를 따른다: 전작이 명시적으로 남긴 open question(bridge-in) → 논문 자신의 문제의식 → 통일 표기로 환원한 core mechanism 과 표기 대응표 → 어느 층을 언제 쓰는가 → 비용 4 중 → 실험과 스케일 → systems/serving 함의 → 한계와 bridge-out. 넷째·다섯째 절이 이 격자의 무게중심이다. 비용칸이 비어 있으면 “논문에 없음” 이라고 쓰고, 비어 있는 절차를 “효율적” 이라고 서술하지 않는다. 그리고 각 장의 마지막 절은 그 논문이다른 경로를 인용했는지를 반드시 밝힌다.

열세 장을 겹치면 남는 것은 논문 열세 편이 아니라 비교 불가능성의지도다. 같은 이름을 쓰는 절차들이 서로 다른 층을 건드리고, 서로를 읽지않으며, 비용을 같은 단위로 보고하지 않는다. 그 지도 위에서 회계를 세우고판정을 내리는 것이 Part III 이며, 인계는 ch24 가 받는다.

ch11. 분기 장 — 판별식과 전수분류

11.1 Bridge-in: 자를 만들었으니 대본다

Part I 은 자를 만들었다. ch01 이 상태를 $\Theta \cdot W \cdot E$ 세 층으로 갈랐고, 쓰기를 $(U-W) \cdot (U-\Theta) \cdot (U-E)$ 세 식으로 갈랐고, 비용을 $B_s \cdot L_w \cdot C_{cap} \cdot \rho$ 네 값으로 갈랐다. 그리고 판정 도구인 판별식을 진술한 자리에서 곧바로유예를 걸었다 — “전수 적용의 결과와 그 판정 근거는 ch11 이 낸다” ($\rightarrow$ ch01 § 01.4). ch10 도 같은 지점에서 끝났다. RAG 를 다 뜯어 놓고 마지막에 “정작 RAG 는 이 책의 주제가 아니다” 라고 쓴 뒤, 그 판정을 무슨 기준으로 하는지를 장에 넘겼다 ($\rightarrow$ ch10).

이 장이 그 두 유예를 한꺼번에 갚는다. 할 일은 셋이다. 판별식을 **적용 가능한 절차**로 정밀화하고, corpus 전체에 기계적으로 적용해통과 · 불통과를 가르고, 그 결과가 드러내는 사실을 진술한다. 그 사실이 이 책의 첫 번째 실질 발견이다 — **이름이 기제보다 넓다**.

이 장이 소유하는 것은 판별식의 엄밀한 진술과 적용 절차, 전수 분류표, 이름과 기제의 불일치, 경로 간 침묵 지도, 그리고 강제된 침묵과 선택된 침묵의 구분이다. 소유하지 않는 것도 못 박아 둔다. 개별 논문의 기제상세는 각 논문장이 갖는다 — 이 장은 어느 층을 언제 바꾸는가까지만 보고, 왜 그렇게 바꾸는지는 보지 않는다. 그리고 이 장은 **아무것도 판정하지 않는다**. 유망한가, 어느 경로가 나은가, 비용이 감당되는가는 전부 Part III(ch24 – ch30) 의 몫이다. 분류는 판정의 재료이지 판정이 아니다.

11.2 판별식 — 네 조건과 각 조건이 막는것

11.2.1 진술

판별식. 절차 P 가 sleep-time compute 인가? (1) **시계** — 상태를 바꾸는 연산이 질의 q 가 도착하기 **전에** 실행되는가. (2) **예산** — 그 연산이 $B_s > 0$ 의 추가 계산을 소비하는가. (3) **대상** — 그 연산이 $\Theta \cdot W \cdot E$ 중 **어느 한 층의 상태를 실제로 바꾸는가**. (4) **소비자** — 바뀐 상태가 그 연산을 유발한 질의가 아닌 **이후 질의**에 쓰이는가. 넷을 모두 만족해야한다. 하나라도 빠지면 인접 문헌이지 이 범주가 아니다.

판별식이 **묻지 않는 것** 두 가지를 먼저 밝힌다. 첫째, 논문의 자기규정을 묻지 않는다. 제목에 sleep 이나 consolidation 이 들어 있는지, 저자가 자기 절차를 무엇이라 부르는지는 판정에 들어 오지 않는다. 둘째, 성능을 묻지 않는다. 이득이 있는지 없는지는 이 범주에 속하는지와 무관하며, 이득의 크기는 Part III 가 다룬다. 판별식이 보는 것은 오직 **갱신식과 시계의 쌍**이다.

11.2.2 적용 절차

판정을 재현 가능하게 만들려면 절차가 필요하다. 이 책은 다섯 단계를 순서대로 밟는다.

적용 절차. 1 단계 — **갱신 연산자를 찾는다**. 논문의 수식중 상태를 좌변에 놓고 이전 상태를 우변에 놓는 것을 전부 뽑는다. 하나도 없으면 조건 (3) 에서 탈락이다. 2 단계 — **좌변을 세 층에 배정한다**. 배포 아티팩트의 일부이면 Θ , 요청 · 세션에 붙는가중치 형태의 상태이면 W , 모델 밖 레코드이면 E 다. 3 단계 — **그 연산자의 시계를 읽는다**. 침자가 토큰 · chunk 이면 t , sleep 라운드이면 k , 세션이면 τ 다. 시계가 t 이고 그 t 가 답을 만든

는 중의 토큰이면 조건 (1) 에서 탈락이다. **4 단계 — 예산을확인한다.** 그 연산자가 추가 추론을 소비하는가. 저장·복사·색인만 한다면 $B_s = 0$ 이고 조건 (2) 에서 탈락이다. **5 단계 — 소비자를 확인한다.** 바뀐 상태를 읽는질의가 그 상태를 만든 질의와 다른가. 같으면 조건 (4) 에서 탈락이다.

절차가 논문마다 같은 순서로 돌기 때문에 판정 결과에 재량이 개입하지 않는다. 1-2 단계는 논문의 수식만 보고, 3-5 단계는 논문의 실험 설정만 본다. 저자의 서술은 어느 단계에도 입력되지 않는다.

11.2.3 각 조건을 빼면 무엇이들어오는가

네 조건이 전부 필요하다는 것은 각각을 제거해 보면 드러난다. 조건마다 실제 corpus 에서 다른 논문이 잘못 들어온다.

(1) 을 빼면 — 모든 쓰기가 sleep 이 된다. MemGPT(arXiv:2310.08560) 는 쓰기를 사용자 턴 안에서 수행한다. 함수 호출을 디코딩해 external store 를 고치는 연산이 답을 만드는 것과 같은 예산에서 나가고, 실행 오류는 그 자리에서 프로세서로 되먹여진다 [MemGPT § 2.3]. SCM(arXiv:2604.20943) 의 요약 성분도 마찬가지로 wake 쪽에 있다 — $\hat{c}$ 를 만드는 인코더 호출이 매 턴 일어난다 [SCM § 3.2]. 조건 (1) 이 없으면 이 둘의 쓰기와 sleep 라운드의 쓰기가 구분되지 않고, ch01 이 세운 시금식 — sleep 은 정의상 L_w 에 0 을 기여한다 (→ ch01 표 1-4 마지막 행) — 이 무너진다. 그러면 “sleep-time compute” 은 그냥 “쓰기” 의 다른 이름이 된다.

(2) 를 빼면 — 저장이 추론으로 승격된다. RAG(arXiv:2005.11401) 는 문서를 잘라 인코딩해 색인에 넣을 뿐 저장 전에 어떤 추론도 하지 않는다. $\hat{c}$ 가 항등 사상이고 문맥당 $B_s = 0$ 이다 (→ ch10). Memorizing Transformers(arXiv:2203.08913) 도 같은 자리에 있다 — 저장되는 것이 그 층의 forward pass 가 이미 만든 (k, v) 텐서 그대로이고, 논문 스스로 “kNN 조회는 긴 거리의 토큰을 평균하거나 요약하지 않고 먼 문맥에서도 정확한 값을 회수한다” 고 쓴다 [Memorizing Transformers § 1]. 조건 (2) 가 없으면 색인 구축이 전부 sleep-time compute 이 되고, 상각식 (A) 의 분자 C_{sleep} 이 정의를 잃는다.

(3) 을 빼면 — 읽기를 바꾼 논문이 쓰기를 바꾼 논문으로 둔갑한다. Memory Caching(arXiv:2602.24281) 이 정확히 이경우다. 갱신 규칙 $f(\cdot)$ 은 baseline 과 글자 그대로 같고 [Memory Caching Eq. 4, 6, 8, 14], 세그먼트 경계에서 일어나는 일은 갱신이라 아니라 과거 iterate 의 보존이다. 바뀐 것은 읽기 — 단일 W_t 대신 캐시집합 위의 게이트된 혼합을 읽는다 [Memory Caching Eq. 7, 9, 15, 17]. 조건 (3) 이 없으면 식 (R) 을 고친 논문과 (U-*) 를 고친 논문이 한 범주에 섞이고, 세 경로 구분 자체가 성립하지 않는다.

(4) 를 빼면 — 같은 질의 안에서 소비되고 끝나는 계산이 들어온다. chain-of-thought 와 scratchpad 가 대표다. 둘 다 상태를 만들고 그 상태를 읽지만, 만든 질의가 끝나면 사라진다. Nested Learning(arXiv:2512.24695) 이 문헌 쪽 사례다 — 논문 스스로 “획득된 in-context 지식은 현재 문맥이 제거되면 존속하지 않는다” 고 쓰고 [NL § 6], CMS 의 블록은 일정 step 뒤 재초기화된다 [NL § 7.1]. 조건 (4) 가 없으면 test-time compute 전체가 sleep-time compute 이 되어 두 개념의 대립이 사라진다. 이 라인을 만든 논문이 test-time 예산의 대안으로 자신을 제시했다는 점에서, 이 조건은 특히 양보할 수 없다.

11.2.4 판별식이 판정하지 못하는것

도구의 한계를 먼저 밝혀 둔다. 회색지대가 둘 있고, 둘 다 의도된 것이다.

첫째, **시계를 스스로 정하지 않는 논문**은 절차로 판정할 수 없다. LoRA·ROME·MEMIT 가 여기 해당한다. 셋 다 “무엇을 어디에 어떻게 쓰는가” 는 완전히 규정하지만 “언제 쓰는가” 는 규정하지 않는다. 3 단계에서 읽을 첨자가 없으므로 판정이 성립하지 않는다. 이 책은 이들을 불충족으로 적되 탈락 사유를 **시계 부재**로 따로 표기하고, 쓰기 primitive 라는 별도의 지위를 준다. 이들을 어느 시계에 엮을지는 이들을 쓰는 논문의 선택이며, 그 선택이 곧 Θ -경로 논문들의 설계다 (→ ch19).

둘째, **조건 (2) 는 부등식이라 크기를 묻지 않는다.** SCM 의 sleep 라운드는 LLM 호출 없이 그 래프 연산만 돌고 [SCM Algorithm 1], LM Need Sleep 의 sleep 라운드는 distillation 과 RL 을 돈다. 두 예산의 자릿수가 전혀 다른데 판별식은 둘에 같은 판정을 준다. 이 무차별은 결함이 아니라

역할 분담이다 — 판별식은 **소속**을 정하고, 크기는 통일 비용 회계 (ch25)가 정한다. 판별식 하나로 “어느 쪽이 비싼가”에 답하려면 도구를 잘못 쓰는 것이다.

이 두 회색지대를 명시하는 것이 판별식의 반증 조건이기도 하다. 네 조건을 모두 만족하는데 이 책이 배제한 절차가 발견되면 그것은 이 책의 오류이고, 표 11-2의 해당 행을 고쳐야 한다.

[해설] 네 조건은 서로 독립이 아니라 **직교**한다. (1)은 시간축, (2)는 비용축, (3)은 상태축, (4)는 재사용축을 각각 한 번씩 자른다. 그래서 어느 하나를 빼면 그 축이 통째로 열리고, 열린 축을 따라다른 종류의 논문이 들어온다. 위 네 문단에서 잘못 들어온 논문이 매번 다르다는 것이 네 조건이 중복되지 않는다는 증거다.

11.3 전수 분류

적용 절차를 corpus 전체에 돌리면 세 군이 나온다. **A 군**은 네조건을 모두 만족한다. **B 군**은 이 이름 아래에서 자주 함께거론되지만 어느 조건에선가 탈락한다. **C 군**은 애초에 이 범주를 자처하지 않는 배경·제약 문헌으로, 판별식의 판정 대상이 아니라 다른 장들이 근거로 쓰는 재료다.

집계 규약을 먼저 못 박는다. corpus는 **29 편**이고, 갈림은 **통과 9 편 + 경계 사례 1 편 + 불충족 19 편**이다. 경계 사례는 MemGPT 한 편이며 조건 (1)을 만족하지 않으므로 통과군에 세지 않는다 (§ 11.5). 불충족 19 편은 B 군 9 편과 C 군 10 편의 합이다. 이 책의 모든 집계는 이 규약을 쓴다.

표 11-1 — A 군: 판별식 통과 9 편 + 경계 사례 1 편 (MemGPT)

논문	arXiv	갱신 층	시계	이 책의 자리
Letta Sleep-time Compute MemGPT	2504.13171 2310.08560	<i>E</i> <i>E</i>	질의 전 1 회 대화 중	ch14. 이 라인의 이름을 만든 논문 ch13. 경계사례 — (1) 불충족 (§ 11.5)
Mem0	2504.19413	<i>E</i>	메시지쌍마다	ch15. wr 이 ADD·UPDATE·DELETE·NOOP 를 판단
Zep	2501.13956	<i>E</i>	에피소드마다 + 주기적 community refresh	ch15. 이중 시간 그래프, wr 비 파괴
Reasoning-Bank SCM	2509.25140 2604.20943	<i>E</i> <i>E</i>	과제 종료 후 대화 후	ch15. wr 이 순수 합집합 ch15. 제목은 consolidation, 가중치 불변 (§ 11.4.1)
Multi-Timescale Do LMs Need Sleep?	2605.05097 2605.26099	<i>E</i> <i>W</i>	이벤트마다 KV cache 비우기 전 <i>N</i> 회 재귀	ch15. 유일 실험에서 LLM 미사용 ch17. <i>W</i> -경로의 유일한 정면 사례
SEAL	2506.10943	Θ	두 시간척도	ch20. 자기생성 self-edit
LM Need Sleep	2606.03979	$\Theta (\leftarrow W)$	sleep 라운드 <i>k</i>	ch21. wake 의 $W \rightarrow$ sleep 의 Θ 순서

표 11-2 — B 군: 판별식 불충족 (9 편). 탈락 조건을 지목한다.

논문	arXiv	탈락 조건	실제로 일어나는 일	이 책의 자리
Nested Learning	2512.24695	(1), (4)	모든 갱신 시계가 토큰이고, 문맥이 끝나면 소멸한다 [NL § 6, § 7.1]	ch18
Memory Caching	2602.24281	(2), (3)	$B_s = 0$. 갱신식이 아니라 읽기식을 바꾼다 [Memory Caching Eq. 4, 7]	ch18
Memory Layers	2412.09764	(1), (3)	pre-training 1 회. 배포 후 어느 층도 바뀌지 않는다	ch09·ch19
Generative Adapter	2411.05877	(1)	갱신은 (U-W) 이 고시계는 wake 다. 산출물만 Θ 모양이다	ch16
LoRA	2106.09685	(1) 시계 부재	쓰기 primitive 이지시계를 가진 절차가 아니다	ch03·ch19
ROME	2202.05262	(1) 시계 부재	직접 편집. 저자가 실용적 방법으로 의도하지 않았다고 명시한다	ch19
MEMIT	2210.07229	(1) 시계 부재	같은 편집을 대량으로 확장한 판	ch19
RAG	2005.11401	(2)	$\hat{c}$ 가 항등사상. 저장 전에 추론이 없다	ch10·ch13
Memorizing Transformers	2203.08913	(1), (2)	쓰기가 forward pass 중의 (k, v) 캐시 적재다	ch12

표 11-3 — C 군: 판별식의 대상이 아닌 배경·제약 문헌 (10 편)

논문	왜 대상이 아닌가	이 책이 쓰는 방식
EWC	지속학습 일반. “질의 전” 이라는 좌표계 자체가 없다	ch07·ch22 — 망각 대응의 원형
Deep Generative Replay	“ ”	ch07·ch22 — replay 의 원형
GEM	“ ”	ch07·ch22 — 성능행렬 Π 의 출처
model collapse	기제가 아니라 $\mathcal{R}$ 이 자기생성 일 때 걸리는 제약	ch06·ch22 — 반증 축
How much do LMs memorize	용량 상한 이론	ch09·ch22

논문	왜 대상이 아닌가	이 책이 쓰는 방식
LoRA as Knowledge Memory	저랭크 쓰기의 용량	ch09·ch19
Rate – Distortion Memory Compaction	무엇을 버릴 것인가의 이론	ch22
Can a LM Learn Facts Continually	Θ -경로의 반증 후보	ch22
Sleep prevents catastrophic forgetting (PLOS)	생물 실증	ch08·ch23
PAD (eLife)	생물 실증	ch08·ch23

11.3.1 통과군의모양

표 11-1 을 두 방향으로 읽으면 이 라인의 현재 상태가 그대로 드러난다.

층으로 읽으면 극단적으로 치우쳐 있다. 통과 9 편 중 E 가 6 편, Θ 가 2 편, W 가 1 편이고, 경계 사례 MemGPT 를 더하면 E 쪽이 7 편이 된다. W -경로는 정면 사례가 하나뿐이고, 그 하나마저 자기 경로의주류 계보와 접점이 없다 (§ 11.7). 이 분포는 Part III 의 비교 가능성에 직접영향을 준다 — 경로별 판정을 하려면 각 경로에 비교할 논문이 있어야하는데, W -경로는 표본이 1 이다.

시계로 읽으면 더 흥미롭다. 통과군 안에서 “질의 전”의 주기가 논문마다 다르다. Letta STC 는 문맥 하나에 대해 **한 번**, Mem0 는 **메시지쌍마다**, Zep 은 **에피소드마다** 쓰고 여기에 주기적 community 갱신을 얹으며, ReasoningBank 는 **과제가 끝날 때마다**, SCM 은 스케줄이 아니라 **버퍼 엔트로피 · 충돌밀도 · 경과 시간의 임계**가 걸릴 때 [SCM § 3.6], LM Need Sleep 는 명시적인 **sleep 라운드 k** 마다 돈다.

[해설] 이 이질성이 상각식 (A) 에 그대로 얹힌다. (A) 의 분모 N_q 는 “한 문맥을 공유하는 질의 수” 인데, 무엇이 한 주기인지가 논문마다 다르면 N_q 가 세는 대상도 달라진다. Letta STC 에서 N_q 는한 문맥에 대한 질의 수지만, Mem0 에서 같은 기호는 메시지쌍 하나당 질의수가 되고, ReasoningBank 에서는 과제 하나당 질의 수가 된다. 세 값은 자릿수가 다르다. 통과군에 속한다는 사실만으로 두 논문의 상각 논증을나란히 비교하면 안 되며, 그 비교를 가능하게 만드는 것이 ch25 의 통일회계다.

11.3.2 판별식은 비용을 묻지않는다

조건 (2) 는 $B_s > 0$ 만 요구하고 그 값을 요구하지 않는다. 그래서 통과군안에도 비용 침묵이 그대로 남고, 불충족군도 다르지 않다. 이 장이 § 11.4 에서 개별로 지목하는 네 편의 비용 4 종을 논문이 실제로 적은 만큼만 옮기면 표 11-5 다. 논문이 보고하지 않은 칸은 이 책이 추정치로 메우지않는다.

표 11-5 — § 11.4 의 네 편이 보고한 비용 4 종

논문	B_s	L_w	C_{cap}	ρ
SCM	논문에 없음. 사이클당 비용을 FLOPs·tokens·초 어느 단위로도 적지 않는다. 유일한 계산 수치는 벤치마크 전체의 총 벽시계 시간 약 12 분이다 [SCM § 3.8]	검색 부분만 — 개념 10 개에서 0.1 ms 미만, 360 개에서 0.3 ms 미만 [SCM § 4.3]. 매 턴 일어나는 추출 호출의 지연은 논문에 없음	개념개수로만 — 10 턴 대화 후 24 개 [SCM Table 2]. 바이트는 논문에 없음	논문에 없음. 반복 sleep 실험 자체가 없다 [SCM § 4.3]
Memory Caching	논문에 없음	논문에 없음	논문에 없음	논문에 없음
Nested Learning	논문에 없음	논문에 없음	논문에 없음	논문에 없음
Memory Layers	논문에 없음	논문에 없음	파라미터 개수로만 — 최대 6400 만 key, 즉 1,280 억 memory 파라미터 [Memory Layers § 5.2]. dtype 미기재로 바이트 환산 불가	논문에 없음

열여섯 칸 중 값이 들어간 칸이 셋이고, 그 셋의 단위가 서로 다르다 — ms, 개념 개수, 파라미터 개수. 셋 중 어느 것도 다른 논문의 같은 칸과 나란히놓이지 않는다. 상태 용량을 바이트로 보고한 논문이 corpus 에 0 편이라는사실은 ch09 가 이미 확인했고, 그 공백을 메우는 회계는 ch26 이 만든다.

[평가] 이름이 기제보다 넓다. A 군의 통과 9 편은 sleep-time compute 이고, B 군 9 편은 아니다. 그런데 B 군 9 편 중 다수가 이이름 아래에서 함께 인용되고, 일부는 제목·초록 수준에서 이범주를 시사한다. 반대 방향의 어긋남도 있다 — A 군의 SCM 은 제목이 Θ -경로를 가리키지만 가중치를 한 번도 건드리지 않는다 (§ 11.4.1). 즉 어긋남은 통과·불통과의 경계에서만 생기는 것이 아니라 **경로 배정에서도** 생긴다. 이것이 이 책이 판별식을 먼저 세운이유이며, 이 판정은 논문들의 자기 규정과 어긋나는 이 책의 판단이다. 근거는 표 11-1·11-2 의 전수 적용이고, 근거의 세부는 § 11.4 가다섯 사례로 보인다.

11.4 이름과 실제가 갈리는 다섯사례

다섯 편을 개별로 다룬다. 결론만 나열하지 않고 § 11.2.2 의 다섯 단계를실제로 밟아 보인다. 이 다섯 편은 어긋남의 유형이 서로 다르기 때문에 골랐다 — 제목이 어긋난 경우, 예산이 없는 경우, 축이 표기에만 있는 경우, 시계가 멈춘 경우, 산출물과 시계가 갈린 경우.

11.4.1 SCM — 제목은 consolidation, 가중치는 불변

SCM(Sleep-Consolidated Memory, arXiv:2604.20943) 은 이름의 세 낱말 중 둘이 Θ -경로를 가리킨다. 판별식을 밟으면 다른 답이 나온다.

1 단계에서 갱신 연산자는 다섯 개다 — hebb(기존 edge 강화), 전역 downscale, 장기 저장소로의 transfer, REM 단계의 dream, 그리고 prune [SCM Algorithm 1]. 2 단계에서 이 연산자들의 좌변은 전부 NetworkX 그래프와 텍스트 버퍼이고 둘 다 모델 밖에 있다 [SCM § 3.5]. 층은 E

다. 3 단계에서 시계는 대화 후이며, 트리거가 스케줄이 아니라 상태의존이다 — 버퍼의 엔트로피, 그래프의 충돌 밀도, 경과 시간 중 하나가 임계를 넘으면 사이클이 돈다 [SCM § 3.6]. 4 단계에서 예산은 0 보다 크되극단적으로 작다. Algorithm 1 의 다섯 연산 어디에도 LLM 호출이 없기 때문이다 [SCM Algorithm 1] — 이 논문의 sleep 라운드는 그래프연산만으로 이루어진다. 5 단계에서 바뀐 그래프는 이후 턴의 검색으로 소비된다.

넷을 다 만족하므로 SCM 은 통과다. 그러나 경로는 E 다. 논문 자신이 못박는다 — “학습이나 fine-tuning 이 필요 없다. 모든 구성요소는 기존 pre-trained 모델 아니면 알고리즘적 논리를 쓴다” [SCM § 3.8]. 가중치 delta 도, gradient 도, fast weight 상태도 논문 어디에도 없다.

[평가] 이 어긋남의 원인은 제목이 아니라 **연산자의 이름을 층과 혼동한 것이다**. consolidation·downscaling·pruning 은 신경과학에서 시냅스에 걸리는 연산이고, 시냅스의 인공 대응물은 가중치다. SCM 은 같은 연산자를 그래프의 edge 에 건다. 연산자가 같아도 좌변이 다르면 다른 층이고, 비용·용량·롤백 단위가 전부 달라진다 (→ ch01 § 01.2). 부수적으로 이 사례는 § 11.6 의 침묵과도 맞물린다 — SCM 의 edge downscale 은 W -경로가 이미 형식화한 retention gate 와 대수적으로 같은 형태인데, 논문은 그 문헌을 한 편도 인용하지 않는다.

11.4.2 Memory Caching — $B_s = 0$, 읽기식을 바꾼 논문

Memory Caching(arXiv:2602.24281) 은 recurrent 모델의 고정 용량이 recall 을 무너뜨린다는 진단에서 출발해, 과거 memory 상태의 체크포인트를 버리지 않고 들고 있다. sleep-time compute 과 문제의식이 겹치므로 함께 거론되기 쉽다.

절차를 밟으면 두 자리에서 걸린다. 첫째, 갱신 규칙이 baseline 에서 한글자도 바뀌지 않는다 [Memory Caching Eq. 4, 6, 8, 14]. 바뀐 것은 읽기다 — 단일 W_t 를 읽던 자리에서 캐시 집합 위의 게이트된 혼합을 읽는다 [Memory Caching Eq. 7, 9, 15, 17]. 이 논문이 기여한 연산은 상태를 바꾸는 연산이 아니라 상태를 **보존하고 고르는** 연산이므로 조건 (3) 에서 탈락한다. 둘째, 그 보존은 추가 추론을 소비하지 않는다. 세그먼트 경계에서 일어나는 일은 이미 계산된 iterate 를 남겨 두는 것뿐이므로 $B_s = 0$ 이고 조건 (2) 에서 탈락한다. 시계까지 따지면 (1) 과 (4) 도 성립하지 않는다 — 모든 캐싱이 온라인 forward pass 안에서 일어나고, 캐시는 하나의 시퀀스 안에서만 살아 질의 간에 재사용되지 않는다. 논문에 오프라인 성분이 등장하는 곳은 사후 학습 변형을 논하는 대목 한군데 뿐이고, 그 변형의 예산과 수치는 논문에 없다 [Memory Caching § 4.3].

이 논문이 Part II 에 남는 이유는 다른 데 있다. 고정 크기 W 하나로는 recall 이 무너진다는 것을 정량화하고, 그러면 W 를 몇 개나 들고 있어야 하는지를 차수로 답한다 — 로그 분할이면 캐시 개수가 $\log_2 L$ 이하다 [Memory Caching § 4.2]. W 층의 용량 천장에 관한 이 답이 ch18 의 재료가.

11.4.3 Nested Learning — 연속체는 표기의 다리다

Nested Learning 은 갱신 주파수라는 하나의 축 위에 pre-training 부터 attention 까지를 늘어 놓고, Θ 를 그 축의 아무것도 움직이지 않는 끝점으로 놓는다. 축이 하나이므로 W 와 Θ 사이에 다리가 놓인 것처럼 읽힌다.

3 단계에서 그 읽기가 무너진다. 논문의 모든 시계가 살아 있는 시퀀스의 토큰이다. 실험이 스윙하는 최저 주파수가 512 토큰과 2K 토큰이므로 [NL § 9.1], 가장 느린 층조차 긴 문서 하나 분량이다 — 세션도 아니고 sleep 라운드도 아니다. 질의 도착 전에 실행되는 pass 가 없으므로 조건 (1) 에서 탈락한다. 5 단계도 마찬가지다. 논문 스스로 “획득된 in-context 지식은 현재 문맥이 제거되면 존속하지 않는다” 고 쓰고 [NL § 6], CMS 의 블록은 일정 step 뒤 초기값으로 되돌아간다 [NL § 7.1]. 조건 (4) 에서 탈락한다.

논문 자신이 오프라인 단계를 범위 밖에 둔다는 사실이 결정적이다. § 1 은 신경과학의 두 단계 — 빠른 온라인 consolidation 과 오프라인 consolidation — 를 소개한 뒤 “본 연구에서는 첫 단계, 즉 온라인 과정으로서의 memory consolidation 에 집중한다” 고 선언한다 [NL § 1]. 논문에 등장하는 유일한 진짜 (U- Θ) 는 평가 전에 한 번 도는 15B 토큰 continual pre-training 인데, 그것이 ICL baseline 에도 동일하게 적용되므로 기제가 아니라 통제군이다 [NL § 9.1].

[평가] 연속체는 표기의 다리이지 기제의 다리가 아니다. 하나의 축 f 가 두 끝점을 이름 위에서 잇는 것과, 어떤 갱신이 실제로 문맥 경계를 넘어 체크포인트에 되쓰이는 것은 다른 사건이다. 이 논문에서는 후자가 한 번도 일어나지 않는다. 축이 있다는 사실만으로 W -경로의 결과를 Θ -경로로 옮겨 읽으면 안 된다.

11.4.4 Memory Layers — 배포 후정지

Memory Layers(arXiv:2412.09764) 는 Θ 의 일부를 대역폭 지배 구간으로 떼어 내 10^2 배 키운다. 최대 설정이 6400 만 key, 즉 1,280 억 개의 memory 파라미터다 [Memory Layers § 5.2]. 파라미터를 그만큼 늘려도 FLOPs 는 사실상 불변이다 [Memory Layers § 5.1].

그럼에도 판별식은 통과하지 못한다. Θ 가 움직이는 유일한 시점이 pre-training 이고, 배포 후에는 $\Theta \cdot W \cdot E$ 어느 것도 바뀌지 않는다. 3 단계에서 볼 시계가 없으므로 (1) 에서, 배포 후 상태 변화가 없으므로 (3) 에서 탈락한다. 논문이 continual learning 과 망각 감소를 거론하는 대목은 결론의 희망 서술 한 문단이며, 실험이 아니다 [Memory Layers § 6].

이 논문의 자리는 기제가 아니라 용량의 실측 공급원이다 (→ ch09). 서빙 쪽으로 옮기면 한 가지 예외적 성질이 따라온다 — Θ -경로는 일반적으로 사용자별 delta 를 만들어 shared-weight batching 을 깨지만 (→ ch01 표 1-4), 여기서 늘어난 파라미터는 전 사용자 공유이므로 batching 이 그대로 성립한다. 배포 후 정지가 대가이자 동시에 이점이다.

11.4.5 Generative Adapter — 산출물의 모양과 시계가 갈린다

Generative Adapter(arXiv:2411.05877) 는 문맥을 받아 한 번의 forward 로 adapter 를 만들어 낸다. 그 산출물의 모양이 LoRA 다 — 논문이 그 형태를 명시적으로 유도한다 [Generative Adapter Eq. 8–9]. 산출물만 보면 전형적인 Θ -경로 아티팩트다.

갱신식을 보면 다른 그림이 나온다. 상태 갱신은 $S_t = S_{t-1} + A_2 H_t^\top H_t B_1$ 이고 읽기용 가중치는 $W_t = A_1 \text{norm}(S_t) B_2$ 다 [Generative Adapter Eq. 3, 6]. (U-W) 의 항과 하나씩 대조하면 α_t 가 없고 (W_t 가 W_{t-1} 을 이어받지 않는다), β_t 가 항등적으로 1 이며, η_t 가 없고, gradient 자리에 chunk 자신의 Gram 행렬의 학습된 쌍선형 사영이 들어간다. 즉 gradient-free 변종이지만 형태는 (U-W) 다. 그리고 결정적으로, base 가중치는 미분되지도 수정되지도 않는다 — contextualization 이후의 체크포인트가 이전과 바이트 단위로 같다. 배포 아티팩트가 바뀌지 않으므로 Θ -경로가 아니다.

시계는 wake 다. 문맥이 주어지면 그 자리에서 접고, 그 결과로 그 문맥에 대해 closed-book 으로 답한다 [Generative Adapter § 4.1]. 질의 도착전에 실행되는 pass 가 아니므로 조건 (1) 에서 탈락한다.

[해설] 다섯 사례가 함께 말하는 것은 하나다. 분류는 산출물의 모양이 아니라 (갱신식, 시계) 의 쌍으로 한다. 산출물 모양으로 분류하면 Generative Adapter 가 Θ -경로로, SCM 이 Θ -경로로, Memory Caching 이 sleep-time compute 으로 들어온다. 셋다 틀린다. 반대로 (갱신식, 시계) 로 분류하면 세 판정이 동시에 바로잡히고, 그 판정이 곧 비용 · 용량 · 룰백 단위의 판정이 된다 — 그것이 이 분류가 회계에 쓸모 있는 이유다.

11.5 경계 사례의 처리규칙

MemGPT 는 조건 (2)·(3)·(4) 를 만족하고 (1) 만 만족하지 않는다. 쓰기가 사용자 턴 안에서, 답을 만드는 것과 같은 예상으로 일어나기 때문이다 [MemGPT § 2.3]. 그런데 그 쓰기의 산출물은 이후 질의에 쓰이고, E -경로의 통과 논문들이 자기 계보를 여기서 시작한다. 규칙이 필요하다.

경계 사례 규칙. 판별식 네 조건 중 셋을 만족하고, 통과군의 논문이 그 논문을 자기 계보의 조상으로 지목하는 경우에 한해, 그 논문을 해당 경로의 출발점으로 서술한다. 단 두 가지 의무가 붙는다. (i) 탈락한 조건이 무엇인지를 그 장의 본문에 명시한다. (ii) 통과군의 일원으로 세지 않는다.

규칙이 있어야 하는 이유는 계보적 조상과 범주 소속이 다른 문제이기 때문이다. 조상을 빼면 bridge 사슬이 끊겨 뒤 논문들의 문제의식이 허공에서 시작하고, 조상을 통과로 세면 조건 (1) 이 무력해져 판별식 전체가 무의미해진다. 규칙은 두 손실을 모두 피한다. MemGPT의 지목 근거는 Letta STC가 자기 절차를 운영체제의 prefetching에 빚대며 그 계보를 자기 조상에 연결하는 대목이다 [Letta STC § 2 ‘Pre-computation’] — 다만 그것은 유비이지 문제의 상속이 아니며, 그 사실은 § 11.7이 따로 다룬다.

반대 방향의 예도 못 박아 둔다. RAG는 조건 (2)에서 탈락하고, 통과군 논문이 조상으로 지목하지도 않는다 — Letta STC의 참고문헌 22개에 RAG는 없다 [Letta STC 참고문헌]. 따라서 RAG는 경계 사례가 아니라 *E* 층의 형태와 읽기 비용을 정의한 배경이다 (→ ch10). 경계 사례 자격은 탈락 조건의 개수만으로 주어지지 않는다.

11.6 경로 간 침묵지도

분류가 끝나면 다음 질문은 세 경로가 서로를 읽는가이다. 이 책은 corpus의 참고문헌을 전수 확인했다. 확인 방법은 각 논문의 참고문헌 목록과 전문에서 다른 경로의 대표 논문·핵심 용어를 검색하는 것이고, 결과는 표 11-4다. 표의 셀은 인용의 **유무와 깊이**를 적은 것이지 인용의 정당성을 평가한 것이 아니다.

표 11-4 — 경로 간 인용 지도 (참고문헌 전수 확인 결과)

논문	<i>E</i> -경로 인용	<i>W</i> -경로 인용	Θ -경로 인용	생물·CLS 인용
Letta STC	자기 계보 1 건 (MemGPT)	없음	없음	없음 (제목에만 빌려씀)
MemGPT	있음	없음	없음	없음
Mem0	조밀	1 건, 문제의 증거로만	없음	수사적 인용만
Zep	조밀	없음	없음	없음
Reasoning-Bank	agent memory	없음	없음	없음 (“sleep” 0 회)
SCM	조밀	1 건, 기각	2 건, 기각	조밀 — 실제 계보
Do LMs Need Sleep?	한 문장 (Letta STC 2 회)	자기 주류 계보	한 문장	조밀
LM Need Sleep	2 건, 두 문장으로 기각	아키텍처 라인 조밀, sleep 논문 0	자기 계보	조밀
Nested Learning	없음	조밀	EWC 1 건	절단 인용 (온라인 단계만)
Memory Caching	없음	조밀	없음	없음 (“sleep” 0 회)
Generative Adapter	1 건, 데이터셋 프로토콜용	조밀 (2022 까지)	조밀 (PEFT 갈래)	없음
Memory Layers	없음	없음	없음	없음 (44 개 중 0 건)
LoRA	없음	없음	없음	없음 (51 개 중 0 건)
ROME	없음	없음	같은 층의 다른 갈래도 없음	없음
model collapse	없음	없음	없음	없음
Rate – Distortion	전면	확인됨	확인됨	—

표를 행 방향으로 읽으면 “이 논문이 누구를 읽었는가”가 나오고, 열 방향으로 읽으면 “누가 읽

하지 못했는가”가 나온다. 열 방향이 더 많은 것을 말한다.

W -경로 열과 Θ -경로 열은 거의 비어 있다. 채워진 칸도 대부분한 건짜리 기각이거나 한 문장짜리 언급이다 — SCM은 W -경로 논문 한편을 “개념이 아니라 토큰 수준의 캐시 관리”라며 [SCM § 2.1], Θ -경로 두 편을 “파라미터 수준이지 기억 구조 수준이 아니다”, “이미지 분류가 대상이다”라며 [SCM § 2.2] 각각 물린다. 세 기각 모두 **측정된 비교가 아니라 추상화 수준에 대한 판단**이며, 어느 것도 baseline으로 실행되지 않았다.

생물·CLS 열은 더 선명한 모양을 만든다. 이 열이 조밀하게 채워진 논문은 SCM·Do LMs Need Sleep?·LM Need Sleep 셋뿐이고, 정작 이 라인에 sleep이라는 이름을 준 논문의 칸이 비어 있다. Letta STC는 § 1에서 그 단어를 순전히 조작적으로 도입한다 — 모델이 상호작용 사이에 유희 상태로 있는 시간을 가리키는 말로 쓰며, consolidation 이론을 인용하지 않는다 [Letta STC § 1]. **이름은 생물에서 왔고 논증은 오지 않았다**. 그러므로 뒤에 온 논문들이 같은 단어를 쓴다는 사실은 공유된 이론적 입장의 증거가 아니다. 이름의 충돌일 뿐이다.

11.6.1 연대가 강제한 침묵과 선택된 침묵

표 11-4를 그대로 읽으면 부당한 비난이 된다. 2021년 논문이 2025년 논문을 인용하지 않는 것은 침묵이 아니라 연대이기 때문이다. 구분이 필요하다.

정의. 논문 A 가 논문 B 를 인용하지 않는 사실은 두 종류다. B 의 공개가 A 보다 늦으면 연대가 강제한 침묵이며, 이는 계보 사실일 뿐 저자의 선택이 아니다. B 가 A 보다 앞서고 같은 문제를 다루는데도 인용이 없으면 선택된 침묵이며, 이것이 서술 대상이다. 공개 간격이 수 주 이내인 근접 동시성은 강제 쪽으로 분류한다.

강제된 침묵부터 정리한다. ROME(2022-02)·MEMIT(2022-10)·EWC(2016-12)·Deep Generative Replay(2017-05)와 생물 실증 논문들은 세 경로의 어느 논문보다 앞선다. 이들이 sleep-time compute 문헌을 인용하지 않는 것은 판정 대상이 아니다. Generative Adapter(2024-11)가 Letta STC(2025-04)를 인용하지 못하는 것도, SCM(2026-04)이 LM Need Sleep(2026-06)을 인용하지 못하는 것도 같은 이유다. 이 방향의 침묵에 대해 물을 수 있는 것은 하나뿐이다 — **역방향은 어떠한가**. 뒤에 온 논문이 앞선 논문을 인용했는가 실제 질문이며, 그것이 선택된 침묵의 정의다.

선택된 침묵은 네 건이 두드러진다. 첫째, Memory Layers(2024-12)는 MemGPT(2023-10)·LoRA(2021-06)·ROME(2022-02)·EWC(2016-12)를 하나도 인용하지 않는다. 넷 다 선행하고, 논문 자신이 결론에서 continual learning과 망각을 거론한다 [Memory Layers § 6]. 둘째, Nested Learning(2025-12)은 Letta STC(2025-04)를 인용하지 않는다 — 8개월 선행이고, 그 논문이 이 이름을 만들었다. 셋째, Letta STC(2025-04) 자신이 W · Θ ·생물 세 방향 모두에 침묵한다. 인용했어야 할 대상이 전부 선행한다 — CLS는 1995년, LoRA는 2021년, ROME는 2022년, Memorizing Transformers는 2022년이다. 넷째, ReasoningBank(2025-09)는 Letta STC를 인용하지 않을 뿐 아니라 “sleep”이라는 단어가 논문 전체에 한 번도 나오지 않는다.

경계에 놓인 사례도 있다. Mem0(arXiv:2504.19413)는 Letta STC(arXiv:2504.13171)를 인용하지 않는데, 두 논문의 v1 공개 간격이 11일이다 (2025-04-17 → 2025-04-28). 이 책은 이것을 근접 동시성으로 보고 강제 쪽에 놓는다. 같은 사실을 선택된 침묵으로서 술하면 근거 없는 비난이 된다.

접촉은 했으나 교류가 없는 중간 상태도 기록해 둔다. Multi-Timescale은 W -경로와 Θ -경로를 각각 한 편씩 인용하지만, 두 인용이 같은한 문장 안에 있고 그 문장 이후로 다시 등장하지 않는다 [Multi-Timescale § 3.2]. Do LMs Need Sleep?도 Letta STC를 두 번 인용하되 [Do LMs Need Sleep? § 2] 실험으로 비교하지 않는다. 인용이 있다는 사실만으로 사슬이 이어졌다고 쓰면 안 된다 — 이 책은 인용의 유무와 깊이를 분리해 적는다.

11.7 같은 경로 안에서도 창시 논문이 인용되지 않는다

경로 사이의 단절보다 더 놀라운 것은 경로 안의 단절이다.

[평가] *E*-경로에서 이 라인의 이름을 만든 논문은 Letta STC 다. 그런데 같은 경로의 후속 논문 둘이 그것을 인용하지 않는다. Mem0 는 11 일 차이의 근접 동시성으로 설명되지만, ReasoningBank 는 5 개월 뒤에나왔고 “sleep” 이라는 단어를 한 번도 쓰지 않는다. 두 논문 모두 **다음 질의가 오기 전에 과거 문맥의 요약**을 외부 저장소에 쓴다 — Letta STC 와 같은 연산을 한다. 같은 연산을 하는 논문이 그 연산에 이름을붙인 논문을 보지 않는다는 것은 인용 누락이 아니라 **공동체가 다르다**는 뜻이다. Mem0 의 대화 상대는 경쟁 제품이고, ReasoningBank 의 대화상대는 agent memory 갈래다. sleep-time compute 은 그 두 공동체 어느쪽에도 검색어로 존재하지 않는다.

사슬은 반대 방향으로도 끊겨 있다. Letta STC 자신이 상속받은 open question 이 없다. 참고문헌 22 개이고, 계보 진술이라 할 것은 같은 저자의 MemGPT 에 대한 운영체제 prefetching 유비 하나뿐이며 [Letta STC § 2], 그 논문이 남긴 문제를 인용해 받는 문장은 없다. 즉 이 경로는 **상속없이 시작해서 계승 없이 이어진다**. ch14 의 bridge-in 을 이 책이 “상속 없는시작” 으로 쓰는 이유가 이것이며, 없는 계보를 지어내지 않는 것이 정직성요구다.

같은 종류의 단절이 *W*-경로 안에도 있다. Do LMs Need Sleep?는다른 두 경로를 각각 한 문장씩 언급하면서, 정작 자기 경로의 주류 계보 —Titans·Miras·Atlas·TNT·Nested Learning — 를 한 편도 인용하지 않는다. 이 논문은 자기 갱신 규칙이 gradient step 이 **아니라는** 것을 명시적으로 방어하는데 [Do LMs Need Sleep? § 2], 그 주장이 정확히 Titans 라인이반대편에 서 있는 명제다. 두 반쪽이 서로를 모른 채 반대 입장을 취하고있다.

반례도 정확히 기록한다. LM Need Sleep 는 Nested Learning 이오프라인 통합을 범위 밖에 둔 자리를 명시적으로 이어받는다. corpus 전체에서 bridge-in 이 교과서적으로 성립하는 거의 유일한 사례다. 그리고 세경로를 모두 읽는 문헌이 한 편 있다 — Rate - Distortion Memory Compaction 은 Letta STC 를 네 번 인용하고 *W*-경로와 Θ -경로를 함께 다룬다. 이 라인을 통합적으로 읽은 문헌이 **기제 논문이 아니라 이론·서베이 쪽에서 나왔다**는 사실 자체가 corpus 의 상태를 말해 준다.

11.8 이 분류가 Part II 를배치한다

분류 결과가 곧 Part II 의 순서다. *E*-경로는 ch12(Memorizing Transformers) 에서 표현 수준의 조상으로 시작해 ch13(RAG 에서 MemGPT 로) 에서경계 사례를 통과한 뒤 ch14(Letta STC) 에서 상속 없는 시작을 맞고 ch15(Mem0·Zep·ReasoningBank·SCM) 로갈라진다. *W*-경로는 ch16(Generative Adapter) 의 경첩에서 시작해 ch17 이유일한 정면 사례를 다루고 ch18 이 표기의 다리와 용량 천장을 함께처리한다. Θ -경로는 ch19(LoRA 와 모델 편집) 에서 쓰기 primitive 를 세우고 ch20(SEAL)·ch21(LM Need Sleep) 로 이어지며, ch22 가 반증 축을 세운다.

B 군과 C 군의 논문이 사라지지 않는다는 점을 강조해 둔다. 판별식을 통과하지못한다는 것은 이 범주가 아니라는 뜻이지 쓸모없다는 뜻이 아니다. Memory Layers 는 Θ 층 용량의 실측을 공급하고, Memory Caching 은 *W* 층용량 천장의 차수를 공급하며, model collapse 와 용량 이론 4 편은 Part III 의반증 조건을 공급한다. 분류의 목적은 배제가 아니라 **각 문헌을자기가 실제로 증거가 되는 자리에 놓는 것이다**.

마지막으로 이 장의 결과가 이 책의 원 질문과 어떻게 이어지는지 못 박는다. ch01 의 질문 지도에서 Q2(“무엇이 실제로 이 범주인가”) 는 나머지 전부의전제였다 (→ ch01 표 1-5). 이 장은 그 질문의 **증거**를 냈다 —판별식, 전수 분류표, 다섯 사례, 침묵 지도. 증거와 판정은 다르고, 판정은 ch24 가 낸다. 그러나 증거만으로도 한 가지는 확정된다. “sleep-time compute 은 유망한가” 라는 질문을 문헌 전체에 대해 던지면 답할 수 없다는것이다. 이 이름 아래에 실제로는 서로를 모르는 세 경로가 있고, 그중 하나는 표본이 1 이며, 이름을 공유한다는 사실은 이론적 입장의 공유를뜻하지 않기 때문이다. 질문은 경로를 지정해야 형태를 갖춘다. Part II 의나머지 열두 장은 그 세 경로를 하나씩 읽어 Part III 가 경로별로 답할재료를 만든다.

요약

- 판별식은 네 조건이다 — (1) 질의 전, (2) $B_s > 0$, (3) 어느 한 층의상태를 실제로 변경, (4) 이후 질의가 소비. 네 조건은시간·비용·상태·재사용 축을 각각한 번씩 자르며, 어느 하나를 빼면 매번 다른 종류의 논문이 잘못들어온다.
- 판별식은 논문의 자기 규정도 성능도 묻지 않는다. 판정에 들어가는 입력은갱신식과 시계뿐이며, 다섯 단계 절차로 기계적으로 적용한다.
- 전수 적용 결과 29 편이 통과 9 편 + 경계 사례 1 편 (MemGPT) + 불충족 19 편 (B 군 9 편, C 군 10 편) 으로 갈린다. 통과한 9 편은 E 6 편, W 1 편, Θ 2 편으로 극단적으로 치우쳐 있고, 경계 사례를 더하면 E 쪽이 7 편이 된다.
- 이름이 기제보다 넓다. 어긋남은 통과·불통과 경계에서만아니라 경로 배정에서도 생긴다 — SCM 은 제목이 Θ 를 가리키지만가중치를 한 번도 건드리지 않고, 논문 스스로 학습이나 fine-tuning 이필요 없다고 쓴다 [SCM § 3.8].
- 분류는 산출물의 모양이 아니라 (갱신식, 시계) 의 쌍으로 한다. 모양으로분류하면 Generative Adapter 가 Θ -경로로, Memory Caching 이 sleep-time compute 으로 잘못 들어온다.
- 판별식은 비용을 묻지 않으므로 통과군 안에도 비용 침묵이 남는다. Memory Caching·Nested Learning·Memory Layers 는 $B_s \cdot L_w \cdot C_{cap} \cdot \rho$ 네 칸이 전부 논문에 없고, SCM 은 B_s 와 ρ 가 없으며 C_{cap} 을 개념 개수로만 보고한다.
- 경로 간 침묵은 실재하며, 일부는 연대가 아니라 선택이다. 선택된 침묵의대표는 Memory Layers·Nested Learning·Letta STC·ReasoningBank 네 건이고, Mem0 의 11 일 차 미인용은 근접동시성으로 강제 쪽에 놓는다.
- 같은 경로 안에서도 창시 논문이 인용되지 않는다. Letta STC 는 상속 없이시작했고 ReasoningBank 는 그것을 계승하지 않았다. 세 경로를 모두 읽은문헌은 기제 논문이 아니라 이론 쪽한 편뿐이다.

자가 점검체크리스트

- ☐ 판별식 네 조건을 순서대로 진술하고, 각 조건을 제거했을 때 corpus 에서잘못 들어오는 논문을 하나씩 댈 수 있다.
- ☐ 임의의 논문에 다섯 단계 적용 절차를 밟아 통과·불통과를판정하고, 불통과이면 걸린 조건 번호를 지목할 수 있다.
- ☐ SCM·Memory Caching·Nested Learning·Memory Layers·Generative Adapter 다섯 편이 각각 어떤 유형의 어긋남인지 구분해 말할 수 있다.
- ☐ 경계 사례 규칙을 진술하고, MemGPT 는 왜 경계 사례이며 RAG 는 왜 아닌지설명할 수 있다.
- ☐ 어떤 미인용 사실을 보고 연대가 강제한 침묵인지 선택된 침묵인지 판정할수 있고, 근접 동시성을 어느 쪽에 놓는지 말할 수 있다.
- ☐ 통과군 9 편이 세 경로에 어떻게 분포하는지 말하고, 그 분포가 Part III 의비교 가능성에 무엇을 의미하는지 한 문장으로 옮길 수 있다.
- ☐ Rosetta: 어떤 절차가 L_w 를 늘린다면 왜 조건 (1) 에서 탈락하는지를자기 서빙 스택의 SLA 항목으로 설명할 수 있다.

다음 장으로

이 장은 대상을 확정했다. 남은 것은 확정된 대상을 하나씩 읽는 일이고, 순서는 경로별 연대순이다. 첫 경로는 E 다 — 통과군 9 편 중 6 편이 여기있고 경계 사례 한 편도 여기 붙으며, 이 라인의 이름도 여기서 나왔다.

ch12 는 그 경로의 가장 이른 조상에서 시작한다. Memorizing Transformers 는조건 (1) 과 (2) 에서 탈락하므로 이 범주가 아니지만, **미분 불가능한외부 저장소를 attention 으로 읽는 것이 미분 가능한 문맥을 키우는 것보다 낫다**는 명제를 처음 세운 논문이다. E 가 무엇이고 그것을 읽는 비용이얼마인지는 ch10 이 이미 세웠으니, ch12 가 답할 질문은 그다음이다 —저장되는 것이 요약이

아니라 원본 텐서일 때 무엇이 얻어지고 무엇이 잃어지는가. 이 장의 표현으로 옮기면, $S(c; B_s)$ 가 항등에 가장 가까운 자리에서 E -경로가 어디까지 갈 수 있는가이다. 거기서 한 걸음 나아가 $S(·; B_s)$ 에 실제로 추론을 넣는 것이 ch13 과 ch14 의 주제다.

ch12. Memorizing Transformers — 외부 기억의 초기형

12.1 Bridge-in: 전작이 남긴 문제

ch10 은 E 층을 세웠다. 구현 형태를 text·vector·graph 셋으로 갈랐고, 읽기 비용을 세 항 — 검색 자체 ·prefill 증가 ·쓰기 — 으로 분해했으며 (→ ch10 식 (10-5)), 상각이 지우는 항이 쓰기 하나뿐임을 확정했다. ch11 은 그 문헌 전체에 판별식을 댔고 Memorizing Transformers(arXiv: 2203.08913) 를 조건 (1) 과 (2) 에서 탈락시켰다 (→ ch11 § 11.3). 그러므로 이 장은 sleep-time compute 의사례를 읽는 장이 아니라 B_s 축의 원점을 읽는 장이다.

원점을 먼저 읽는 이유는 회계 때문이다. ch10 의 RAG 는 저장 전 추론 없이도 쓰기 비용을 corpus 당 한 번 지불했다. 이 논문은 한 걸음 더 내려간다 — 쓰기의 산출물이 wake pass 가 이미 만든 텐서 그 자체여서 한계 예산이 0 이다. E -경로가 B_s 를 한 푼도 쓰지 않고 얻을 수 있는 것의 상한이여기서 정해지고, 이후 논문들이 지불하는 B_s 는 이 상한 위에서 얼마를 더 샀는가로 평가되어야 한다.

이 논문은 어떤 선행 논문의 open question 도 인용해 받지 않는다. Related Work([Memorizing Transformers § 2]) 는 3 인칭 현재시제의 위치선정 서베이이고, 인용부호도 “X 가 남긴 문제” 도 없다. 모든 비교는 저자들이 스스로 세운 대조다 — “In contrast, we use a very large cache without compression” [Memorizing Transformers § 2]. 이 책은 없는 계보를 지어내지 않는다 (→ ch11 § 11.7).

그래서 이 장의 bridge-in 은 상속 진술이 아니라 **암묵적으로 답해진 세 질문**의 복원이다. 첫째, Transformer-XL 과 Compressive Transformer 가 단계 사이로 나르는 비미분 캐시는 어디까지 키울 수 있고, 나르는 상태를 반드시 압축해야 하는가. 답은 262,144 쌍까지이고 압축은 불필요하다 — dense attention 을 근사 top- k_{nn} 회수로 바꾸면 된다 [Memorizing Transformers § 2, Table 5]. 둘째, 긴 문맥 모델이 주어진 문맥을 실제로 쓰는가 (Sun et al. 2021 의 부정적 결과). 논문은 그 질문을 인용하지 않고 [Memorizing Transformers § 4.6] 을 그 답으로 지은 뒤 같은 결과를 방증으로 인용한다 [Memorizing Transformers § 4.6]. 셋째, kNN-LM(Khandelwal et al. 2020) 이 출력 softmax 에 둔 비미분 저장소를 stack **안쪽**에 둘 수 있는가. 답은 그렇다이고, 자리는 중상위 attention layer 한 곳이며 학습된 per-head 게이트가 딸린다 [Memorizing Transformers § 2, § 3.1].

ch11 이 이 장에 넘긴 질문은 하나였다 — 저장되는 것이 요약이 아니라 원본텐서일 때 무엇이 얻어지고 무엇이 잃어지는가. 미리 적으면 이렇다. 얻는 것은 **정확한 회수와 0 에 가까운 쓰기 비용**, 잃는 것은 **어떤 체크포인트에나 붙일 수 있는 이식성**이다.

12.2 문제의식

논문의 첫 두 문장이 이 라인 전체의 대립 구도를 세운다.

“Language models typically need to be trained or finetuned in order to acquire new knowledge, which involves updating their weights. We instead envision language models that can simply read and memorize new data at

inference time, thus acquiring new knowledge immediately.” [Memorizing Transformers Abstract]

이 책의 어휘로 옮기면 Θ -경로와 E -경로의 대립이다. 그런데 2022 년 3 월의 이 논문에는 sleep 도 consolidation 도 replay 도 없다. 대신저자들이 쓰는 대비는 **저장 속도**다.

“Facts and information which are stored in the form of weight matrices must be slowly trained over hundreds of thousands of training steps. By using attention, however, a model can simply memorize facts ... In this case, attention acts as a form of information retrieval.” [Memorizing Transformers § 1]

수사적 핵심은 attention 과 information retrieval 의 **동일**이다. 그것이 성립하면 문맥을 키우는 대신 저장소를 키우는 선택지가 열린다. 저장소를 키우려면 미분 가능성을 포기해야 한다는 것이 두 번째 주장이다.

“gradients are not backpropagated into the external memory, which is critical to the scalability of our technique. ... if the external memory is not differentiable, then we can instead reuse keys and values that were previously computed on prior training steps, which drastically reduces the amount of computation for large memories.” [Memorizing Transformers § 1]

세 번째 주장은 압축의 불필요다. Compressive Transformer 계열이 나르는상태를 요약해 줄이는 것과 정반대로, 이 논문은 “kNN lookup does not do averaging or summarization of tokens at long distances, but retrieves exact values” 라고 쓰고 설계를 “a very large cache without compression” 으로 요약한다 [Memorizing Transformers § 1, § 2].

[해설] 세 주장을 이 책의 기호로 옮기면 ch11 이 이 장에 넘긴자리가 그대로 나온다. $S(c; B_s)$ 를 표현 공간의 **항등사상**으로두고, 대신 $\text{ret}(E, q)$ 를 모델 내부의 attention 연산 자체로만든다. RAG 도 $S(\cdot; B_s)$ 를 항등으로 두지만 ret 을 별도 encoder 와 프롬프트 접합에 맡겼다 ($\rightarrow$ ch10 § 10.2). 두 논문은 같은 자리에서반대로 갈라진다 — RAG 는 읽기를 모델 **밖**으로 밀어내 학습 가능한일급 객체로 만들었고, 이 논문은 읽기를 모델 **안**으로 끌어들여 attention 의 한 종류로 만들었다.

한 가지를 미리 못 박아 둔다. 논문은 첫 문장에서 Θ -경로를대립항으로 지목하면서 인용을 하나도 달지 않고, 145 항의 참고문헌에 PEFT 도 LoRA 도 모델 편집도 catastrophic forgetting 도 없다. E -경로는자기가 읽지 않은 문헌을 **상대로 자기를 정의하며** 시작했다 (§ 12.8).

12.3 Core mechanism (통일표기)

이 절이 이 장의 두 정의를 소유한다 — 구조가 먼저이고 접합 방식이다음이다.

정의. 비미분 외부 기억을 attention 이 읽는 구조는, 저장소 E 에 어떤 gradient 도 흐르지 않고 ($\nabla_E \equiv 0$), E 의 내용이 모델 자신이 이미 계산한 내부 표현이며, 읽기가 프롬프트접합이 아니라 stack 내부의 특정 layer 에서 residual stream 으로 합류하는구조를 말한다. 이 셋 중 하나라도 어긋나면 다른 구조다 — gradient 가흐르면 Θ 의 일부이고, 내용이 텍스트면 프롬프트 접합이며, 읽기가입력단에서 일어나면 문맥 확장이다.

배치는 이렇다. 12 층 decoder-only transformer($d=1024$, $8 \text{ heads} \times d_{\text{head}}=128$, $d_{\text{ff}}=4096$) 에서 layer 9 하나만 kNN-augmented layer 이고 나머지 11 개 층은 E 를 보지 못한다 [Memorizing Transformers § 3.1, § 4.2]. 쓰기는 chunk 경계마다일어난다.

$$E_t = \text{wr}(E_{t-1}, \hat{c}_t), \quad \hat{c}_t = S(c_t; B_s) = \{(k_i, v_i)\}_{i \in c_t}, \quad \text{wr} = \text{append+FIFO evict to } n_E \quad (12-1)$$

식 (12-1) 이 말하는 것은 (U-E) 에서 $S(\cdot; B_s)$ 가 항등이라는 것이다. c_t 는 forward pass 가 방금 소비한 $C = 512$ (또는 2048) 토큰의 chunk 이고, (k_i, v_i) 는 그 layer 의 attention 이 이미 물질화한 텐서그대로다 — “after each training step, the (key, value) pairs in the local context are appended to the end of the external memory. If the document is very long, old (key, value) pairs will be dropped from the memory to make room for new ones” [Memorizing Transformers § 3.1]. 여기서 시간 첨자가 t (chunk) 이고 k (sleep 라운드) 가 아니라라는 점이이 장 전체의 회계를 결정한다.

두 번째 정의가 읽기 쪽이다.

정의. kNN 검색을 attention 에 접합하는 방식은, attention 의 질의 벡터 q_t 를 그대로 검색 질의로 사용해 E 에서 근사 top- k_{nn} 쌍을 뽑고, 뽑힌 쌍에 대해 다시 softmax attention 을 돌린뒤, 그 출력과 local attention 출력을 학습된 per-head 스칼라 게이트로 섞어 residual stream 에 되돌리는 접합을 말한다. 별도의 encoder 도, 유사도 임계값도, 프롬프트 토큰도 개입하지 않는다.

$$\text{ret}(E, q_t) = \text{Top-}k_{nn}\left(\{\langle q_t, k_i \rangle : (k_i, v_i) \in E\}\right), \quad k_{nn} = 32 \quad (12-2)$$

$$V_m = \text{softmax}(q_t K_{nn}^\top) V_{nn}, \quad (K_{nn}, V_{nn}) = \text{ret}(E, q_t) \in \mathbb{R}^{k_{nn} \times d_{\text{head}}} \quad (12-3)$$

$$g = \sigma(b_g), \quad b_g \in \Theta \text{ (head 당 스칼라 하나)}, \quad V_a = V_m \odot g + V_c \odot (1 - g) \quad (12-4)$$

식 (12-2) – (12-4) 는 원문 [Memorizing Transformers § 3.1] 의 산문 서술과 Eq. (1) – (2) 에 대응한다. (12-2) 의 요지는 검색의 키 공간이 모델 자신의 attention 키 공간이라는 것이다 — “The same queries are used for both the local context, and for the external memory. The keys and values also belong to the same distribution” [Memorizing Transformers § 3.1]. 즉 ret 은 attention 이 어차피 계산하는 내적의 argmax 일 뿐이다. (12-3) 의 K_{nn}, V_{nn} 은 회수된 k_{nn} 개 쌍을 쌍은 행렬이고 여기에는 position bias 를 붙이지 않으며, (12-4) 의 V_c 는 local chunk 와 Transformer-XL 캐시에 대한 통상의 dense attention 출력이며 V_a 가 residual stream 으로 올라간다.

읽기 게이트 g 는 **내용에 의존하지 않는다** — “the value of the gate g does not depend on the content of the token at each position, although that would be a trivial extension to implement” [Memorizing Transformers § 3.1]. test-time 의 라우팅 결정이 아니라 outer loop 에서 한번 학습되는 head 별 구조적 선호다. 그리고 학습이 끝나면 “most heads learned to attend almost exclusively to external memory” [Memorizing Transformers § 3.1].

전체를 표준형 (R) 로 환원하면 이렇다.

$$\hat{y} = f(q; \Theta, \emptyset, \text{ret}(E, q)), \quad \text{ret는 정확히 한 layer 에서만 진입} \quad (12-5)$$

$W = \emptyset$ 이다. inner loss 도 test-time gradient 도 고정 크기압축 상태도 없다. Transformer-XL 캐시는 무손실 KV 이월이지 W 가 아니다 — α_t 도 S_t 도 η_t 도 없다.

논문 자신이 (12-5) 의 결과에 놀란다. “This is surprising, because the external memory is not differentiable, and is added only to one layer of the Transformer ... We conclude that the lower layers of a Transformer don’t necessarily need long-range context, and having a differentiable memory is not as important as one might suspect.” [Memorizing Transformers § 4.3]

훈련 중에만 나타나는 향이 하나 있다. E 에 담긴 (k_i, v_i) 는 더 오래된 Θ 가 만든 것이라 질의와 분포가 어긋나고, 논문은 이를 staleness 라 부르며 key/query 정규화로 대응한다 — “Normalization

does not eliminate staleness, but it at least ensures that older keys and newer keys do not differ in magnitude” [Memorizing Transformers § 3.2]. Θ 가 동결된 순수 추론에서는 이 항이 사라진다. 훈련 시점과 배포 시점의 거동이 같리는 유일한 자리다.

12.3.1 표기 대응표

표 12-1 — Memorizing Transformers 원 표기 → 이 책의 표기

원 표기	이 책의 표기	주의
M — memory size, head 당 (k, v) 쌍 수	n_E (개수), C_{cap} (바이트)	맨 M 금지 — $\mathcal{M}(\cdot; W)$ (읽기 함수)로 읽힌다
k — 회수 이웃 수, $k = 32$ [§ 4.2]	k_{nn}	이중 충돌: k 는 sleep 라운드 첨자, k_t 는 key 벡터
g — per-head 게이트, $g = \sigma(b_g)$ [Eq. (1)]	g (“읽기게이트 g ”)	α_t (쓰기 시점 retention)와 혼동 금지. b_g 는 Θ 에 속한다
“context” / “context size” = 512 또는 2048	C (chunk)	용량은 언제나 C_{cap} . Table 4의 Context 열이 C , Memory 열이 n_E
“external memory”	E (표현 수준 E)	MemGPT·Mem0의 E 와 같은 단어, 다른 물질 — “활성값을 담는 E ” 대 “텍스트를 담는 E ”
V_m, V_c, V_a — memory·local·결합 출력	그대로 유지	저장된 값 벡터는 v_i . V_i 로 쓰지 않는다
Δ_i — $\text{CE}_{8192}(x_i) - \text{CE}_{32K}(x_i)$ [§ 4.6]	Δ_i	부호 관행: 양수 = 큰 n_E 가 개선
“training step” — E 를 쓰는 단위	chunk 경계, 첨자 t	학습사건이 아니다. 추론에서도 동일하게 일어나며, 원문이 그렇게 쓴 이유는 모든 측정이 훈련 측정이기 때문이다
(장·국소) ch10의 K_{ret}	k_{nn}	같은 역할, 다른 단위 — ch10은 텍스트 레코드, 여기서는 (k, v) 쌍
(장·국소) 회수된 키·값 행렬	K_{nn}, V_{nn}	K_{ret} 을 재사용하지 않는다 — ch10에서 그 기호는 회수개수(스칼라)다

12.4 어느 층을 언제쓰는가

test-time에 학습되는 것은 아무것도 없다. 움직이는 것은 E 뿐이고 그것도 gradient가 아니라 memcpy로 움직인다. 게이트를 포함한 모든 학습 객체가 배포 전 pre-training에서 고정된다 — corpus 전체에서 가장 깨끗한 분리다.

표 12-2 — 이 값은 누가 학습하는가

객체	학습 주체	언제	test-time 이동	출처
transformer weights (embedding, 12 층 attention+FFN, 출력 투영)	outer loop	500K steps, Adafactor, lr 1.0, 32 TPU cores	아니오	§ 4.2
T5 relative position bias	outer loop	Θ 와 함께	아니오. local dense attention에만 적용, E 에서 회수한 K_{nn} 에는 미적용	§ 3.1
게이트 바이어스 b_g (head 당 스칼라 1 개)	outer loop	Θ 와 함께. 대부분의 head에서 1 근처로 수렴	아니오. 내용 비의존이고, 내용의 존화는 “trivial extension”이라 부르며 실험하지 않는다	§ 3.1 Eq. (1)
E_{knn} 의 내용 — (k, v) 쌍	없음. gradient가 흐른 적이 없다	chunk 경계마다 축자 append, 용량 초과 시 FIFO 축출, 문서 경계마다 소거	예 — 유일하게 움직이는 상태	§ 1, § 3.1
E_{xl} 의 내용 — Transformer-XL 캐시	없음	매 step 직전 chunk의 (k, v) 로 교체, 모든 layer에서	예	§ 3
근사 kNN 색인	없음. recall 약 90%의 고정 근사	해당 없음	E 위의 자료구조	§ 3.3
kNN layer의 위치 (12 층 중 9 층) n_E 자체	없음 — 하이퍼파라미터	훈련 전 고정	아니오	§ 4.2, App. A.1
	없음. 그러나 훈련 후에 바꿀 수 있는 유일한값이다	추론에서 올리거나 20K step 추가 finetune	배포 시점 knob — 이논문의 실질 기여	§ 1, § 4.5, Table 5

실행되는 갱신 규칙은 (U-E) 하나뿐이다. Θ 는 배포 전 outer loop에서만 움직이고 W 는 존재하지 않으므로 순서 문제가 생기지 않는다. 시계는 chunk 경계 t 이며 sleep 라운드 k 가 아니다 — 쓰기는 유희시간이 아니라 wake pass 안에서 동기적으로 일어나고, 별도의 오프라인 job도 쓰기 배치도 없다. 이것이 ch11의 조건 (1)·(2)가 걸린자리다.

wr은 append와 FIFO 축출뿐이다. merge도 rewrite도 중복제거도 중요도 점수도 없다 — MemGPT의 working-context 치환 ($\rightarrow$ ch13)이나 Mem0의 ADD/UPDATE/DELETE ($\rightarrow$ ch15)에 해당하는것이 없다. 그래서 E 는 창 안에서 단조 증가하고 창 밖에서 완전망각이며, 어떤 실험도 “축출되었다”와 “top- k_{nn} 경쟁에서 졌다”를 구분하지 않는다. 범위는 문서당·batch 원소당이고 문서경계에서 소거되므로 [Memorizing Transformers § 3.1] 상각은 문서 내부에서만 성립한다. 식 (A)의 N_q 는 여기서 같은 문서에 남은 토큰 수다.

[해설] ch01의 Rosetta 표는 KV cache를 W 의 유비로놓았다 ($\rightarrow$ ch01). 이 논문은 그 유비가 문자 그대로 성립하는 유일한자리이되 층이 다르다 — 저장되는 것이 실제 (k, v) 이고 창 안에서 append-only 무손실이지만, 놓이는 층은 W 가 아니라 E 다. 뒤집어말

하면 이 E 는 뒤의 E -경로 논문들과 달리 읽기 함수를 바꾸지 않는다. 읽는 연산은 여전히 softmax attention 이고, 바뀐 것은 그대상뿐이다.

12.5 비용 4 종

이 논문은 서빙 수치를 하나도 보고하지 않는다. 유일한 벽시계 측정은 가장작은 모델의 **훈련** step 시간이며 한 문장으로 한 번 나온다.

표 12-3 — 비용 4 종

기호	값	근거
B_s	논문에 없음. 구조적으로 0 이다 — 오프라인 pass 가없으므로 분리 가능한 sleep 예산이 성립하지 않는다. 쓰기의 한계 비용은 memcpy 와 색인 삽입 뿐이고 논문은 측정하지 않는다	§ 3.1, § 1
L_w	논문에 없음. 추론 지연도 TTFT 도 ITL 도 decode 측정도없고 “latency” 라는 단어가 나오지 않는다. 131K·262K 가 “maintaining a reasonable step time” 이라는 진술은 정성적이며 “reasonable” 에 숫자가 없다	§ 1, § 4.2
C_{cap}	개수로만 보고. 바이트는 논문에 없음. $n_E \in \{1536, 8192, 65K, 131K, 262K\}$, 범위는 head 당 (8 heads)· 한 layer· 문서당· batch 원소당, $d_{head}=128$. 논문은 이 값들을 곱한 적이 없고 dtype 도 적지 않는다	§ 3.1, § 4.2, Table 4·5
ρ	수치로는 논문에 없음. 열화 기제는 staleness 하나이고그것은 훈련 시점 아티팩트다. 정량화되지 않았다	§ 3.2, § 4.5

논문에 있는 유일한 계산량 수치는 다음이다. “the step time increased from 0.2s to 0.25s when we added a memory of size 8K, and to 0.6s when we added a memory of size 65K (measured on TPUv3)” [Memorizing Transformers § 4.2]. $n_E=8192$ 에서 +25%, $n_E=65536$ 에서 3 배다 [본서 산술]. 이 값은 kNN 검색· 추가 attention· 쓰기를 **한덩어리로** 묶고 있고, 추론 step 이 아니라 훈련 step 이며, 약 200M 파라미터에서만 보고되었다. 셋 중 하나만 어겨도 서빙 감각으로 옮길 수없다 [본서 판단].

ρ 칸은 hedge 가 다섯 겹이다. “In some of our experiments” , “sometimes resulted in worse performance” , “could be due to staleness” [Memorizing Transformers § 3.2], 다시 “In some cases, training was unstable … possibly due to distributional shift early in the training” [Memorizing Transformers § 4.5]. 불안정했다는 실행은 논문 어디에도그려지거나 표로 실리지 않았다. Θ 가 배포 후 움직이지 않으므로 (U- Θ) 의미의 라운드 간 망각은 애초에 적용되지 않고, FIFO 축출로인한 손실은 한 번도 분리되지 않았다.

[평가] 상각식은 여기서 붕괴한다. 식 (A)의 C_{sleep} 이 0 이므로 $C_{\text{avg}} = C_{\text{wake}}$ 이고 손익 분기 N_q^* 를 계산할 대상이 없다 — E 쓰기는 $N_q = 1$ 에서 이미 자기 값을 한다. corpus에서 상각식이 정당하게 붕괴하는 유일한 논문이다.

[평가] 그 대신 이 논문은 corpus의 어떤 E -경로 논문도 갖지 못한 것을 갖고 있다 — N_q 가 뿔혀 나올 분포다. 문서길이 히스토그램이 다섯 corpus 전부에 실려 있고 arXiv 논문은 최대 거의 1.6M 토큰, Github 저장소는 최대 900 만 토큰을 조금 넘는다 [Memorizing Transformers App. A]. 이 논문에는 상각할 것이 없는데 분포는 있고, 상각을 주장하는 뒤의 논문들에는 분포가 없다.

읽기 쪽 비용은 측정이 아니라 인용으로 주장된다. 확장 가능한 구현으로 ScaNN 과 Faiss 가 지목되고 근사 kNN 이 “can scale into the billions” 라고 적히지만 [Memorizing Transformers § 1, § 3.3], FLOP 수도색인 빌드 시간도 gather 대역폭도 없다. 중심 효율 논증이 측정되지 않은 항위에서 있다 [본서 판단].

12.6 실험과 스케일

이 논문의 유일한 정량 지표는 token 단위 평균 perplexity 다. downstream 과제가 하나도 없다 — QA 도 code completion pass@k 도 요약도 긴문맥 벤치마크도 없다. 초록의 능력 주장 (“capable of making use of newly defined functions and theorems during test time”)을 떠받치는 것은 Isabelle 에서 손으로 확인한 10 개 예시이고 그중 8 개를 저자들이 성공으로 판정했다 [Memorizing Transformers § 4.6].

표 12-4 — 500K step 훈련 후 평균 token perplexity [Memorizing Transformers Table 4]

C	n_E	XL 캐시	arXiv	PG19	C4(4K+)	GitHub	Isabelle
512	없음	없음	3.29	13.71	17.20	3.05	3.09
2048	없음	없음	2.69	12.37	14.81	2.22	2.39
512	없음	512	2.67	12.34	15.38	2.26	2.46
2048	없음	2048	2.42	11.88	14.03	2.10	2.16
512	1536	없음	2.61	12.50	14.97	2.20	2.33
512	8192	없음	2.49	12.29	14.42	2.09	2.19
512	8192	512	2.37	11.93	14.04	2.03	2.08
512	65K	512	2.31	11.62	14.04	1.87	2.06
2048	8192	2048	2.33	11.84	13.80	1.98	2.06
2048	65K	2048	2.26	11.37	13.64	1.80	1.99

논문이 표 12-4 에서 뽑아 쓰는 주장은 셋이다. C4(4K+)에서 $n_E=8192$ 을 붙이면 vanilla 가 $17.20 \rightarrow 14.42$, Transformer-XL 이 $15.38 \rightarrow 14.04$ 가 된다. 다섯 열전부에서 최소값이 마지막 행이다 [본서 관찰]. 그리고 “using even a small external memory of size 1536 provides a gain in perplexity which is almost as good as using a local context of size 2048 but no memory” [Memorizing Transformers § 4.3]. 셋째 주장은 다섯 열 중 둘에서 뒤집힌다 — $n_E=1536$ 행과 $C=2048$ 행을 대조하면 arXiv(2.61 대 2.69)·GitHub(2.20 대 2.22)·Isabelle(2.33 대 2.39)에서는 성립하지만 PG19(12.50 대 12.37)와 C4(4K+)(14.97 대 14.81)에서는 진다 [본서 관찰]. 논문은 이 열별 분해 없이 주장만 적는다.

용량 사다리는 Table 5 가 있다. 500K step 사전학습 뒤 20K step 만 더 돌려 n_E 를 올리면 arXiv perplexity 가 $C=2048$ ·사전학습 65K 기준 65K에서 2.26, 131K에서 2.23, 262K에서 2.21 까지 내려가고, 논문은 상한을 문서 길이로 귀속시킨다 [Memorizing Transformers § 4.5, Table 5]. 같은 표에 [Memorizing Transformers § 3.2]의 불안정성 서사와 반대 방향의 행이 있다: $C=512$ 에서 65K로 직접 훈련하면 2.31, 8192로 사전학습뒤 65K로 finetune 하면 2.32 다 [본서 관찰]. 두 프로토콜을같은 용량에서 비교한 유일한 자리에서 직접 훈련이 낫다 [본서 관찰].

스케일 주장 — $n_E=8192$ 을 붙인 작은 모델이 파라미터 5 배인 vanilla 와 맞먹는다 — 은 Figure 1 하나에만 근거한다 [Memorizing Transformers § 4.4, § 5]. 1B 와 8B 의 perplexity 값이 논문 어디에도 없고, 데이터셋 arXiv Math 하나, $n_E=8192$ 하나, $C=2048$, XL 캐시 없음, 오차막대 없음이다. 소급 적용 주장도 Figure 6 하나에 근거한다: E 없이 사전학습된 1B 모델을 20K step(사전학습의 4%) finetune 하면 격차의 85%가, 100K step 이면 전부 닫힌다 [Memorizing Transformers § 4.5].

부록의 ablation 은 전부 한 점에서 돌았다 — $C=512$, XL 캐시 512, $n_E=8192$, 약 200M 파라미터, 데이터셋 1 개 [본서 관찰]. layer 위치는 3 층 2.40, 6 층 2.36, 9 층 2.37, 12 층 2.43 이고 [Memorizing Transformers Table 14], 논문 전체가 쓰는 기본값 9 층은 자기 스왑의 최고값이 아니다 [본서 관찰]. 이웃 수는 32 에서 2.38, 128·256 에서 2.37 이다 [Table 15]. 시드 표준편차는 Transformer-XL 2.67 ± 0.01 , Memorizing Transformer 2.37 ± 0.005 이며 [Table 16], 이것이 유일한 분산 추정이고 Figure 1·Figure 6 에도 오차 막대가 없다 [본서 관찰]. 즉 layer 스왑은 전체 폭 0.07 을 점당 1 회 실행으로 가르고, Table 15 의 2.38 은 Table 4 가 2.37 로 보고한 바로 그 구성이다 — 시드 폭 기준 약 2σ 이며 논문은 언급하지 않는다 [본서 산술]. 두 정성 연구도 **다른 아키텍처**에서 돌았다: Δ_i 분석은 게이트 없는 구버전을, Isabelle 사례 연구는 부록이 이득 없다고 보고한 2-layer 32K 구성을 썼다 [Memorizing Transformers § 4.6, App. A.1].

[평가] 다섯 corpus 중 셋을 저자들이 만들었고, 그중 둘은 측정 대상인 장거리 의존성을 **제조하는** 방식으로 만들어졌다. Github 은 저장소 하나를 무작위 디렉터리 순회로 이어 붙이고, Isabelle 은 theory 파일을 “ordered according to their import dependencies, so that later files use sub-theorems that are proved in earlier files” 로 이어붙인다 — 정의가 참조보다 반드시 앞선다 [Memorizing Transformers § 4.1]. 표 12-4 에서 가장 큰 이득이 나오는 열이 정확히 그 둘이다 (GitHub 3.05 → 1.80, Isabelle 3.09 → 1.99). Table 4 의 캡션은 모든 모델이 500K step 훈련되었다고 적지만 Isabelle 은 과적합 때문에 100K step 에서 멈췄다 [Memorizing Transformers § 4.2]. 회수 baseline 은 한 번도 돌지 않았다 — kNN-LM, Compressive Transformer, Big Bird 가 [Memorizing Transformers § 2] 에서 논의되고 아무것도 재현되지 않으며, 유일한 대조군은 저자 자신의 vanilla 와 Transformer-XL 이다. 근사 회수는 recall 약 90%에서 돌았고, 더나쁜 근사에도 견고하다는 주장에 수치도 구현체 이름도 없다 [Memorizing Transformers § 3.3, § 4.2]. 이 장은 위 어느 것도 개별 결함으로 세지 않는다. 세는 것은 **결론의 폭**이다 — 실증된 것은 “저자들이 만든 장거리 의존 corpus 에서, 저자들의 baseline 대비, 200M 규모에서 perplexity 가 내려간다”이며 초록이 주장하는 능력은 그보다 넓다.

Δ_i 분석이 이득의 실제 모양을 보여 준다. 22K 토큰짜리 arXiv 논문에서 $n_E=8192$ 과 $n_E=32K$ 를 비교하면 앞의 8192 토큰 구간에서 차이가 정확히 0 이고, 그 뒤로도 “the benefit of external memory is somewhat sparse ... mainly driven by a small percentage of tokens” 이며 일부 토큰은 오히려 **나빠진다** [Memorizing Transformers § 4.6]. 논문은 원인을 top- k_{nn} 탈락으로 추정하지만 추출과 혼잡을 가르는 실험은 없다. 이득이 물리는 자리는 희귀 토큰이다 — 고유명사, 참조, 인용, 함수 이름.

12.7 Systems/serving 합의

서빙 감각으로 직접 옮겨지는 문장은 하나다. “Unlike standard dense attention, the retrieved memories contain a different set of (key, value) pairs for each query.” [Memorizing Transformers § 3.1] K 와 V 가 질의들 사이에서 공유되지 않는다는 뜻이고, 그러면 그 layer 는 GEMM 이기를 그만두고 **gather** 가 된다 (나머지 11 개 층은 그대로 GEMM 이다). 프롭트 접합형 E -경로와의 결정적 차이 — ch10 의 항② (prefill 증가) 가 여기서는 0 이고, 대신 토큰당 gather 트래픽 항이 새로 생긴다.

[해설] 바이트로 환산해 본다. **아래 계산은 이 책의 산술이며 논문의 보고값이 아니다.** 논문이 dtype 을 적지 않으므로 bf16(2 bytes/elem) 을 가정하고 나머지는 논문 자신의

shape 다 ($8 \text{ heads} \times d_{\text{head}}=128$, key-value 양쪽 저장, kNN layer 1 개). $n_E=65,536$ 이면 $65536 \times 8 \times 128 \times 2 \times 2 = 268,435,456 \text{ bytes} = 256 \text{ MiB}$, 같은 방식으로 $1536 \rightarrow 6 \text{ MiB}$, $8192 \rightarrow 32 \text{ MiB}$, $262,144 \rightarrow 1 \text{ GiB}$ 이고 전부 **문서 하나당**이다. 계산: 같은 토큰 수의 12 층 KV cache 전체가 3 GiB 이므로 한 층에만 사는 저장소가 그 12 분의 1 인 것이 맞다. 절대치는 dtype 가정에 걸려 있으니 하한 · 방향성으로 읽고, 단정하는 것은 자릿수와 비율이다.

집계하면 문제가 보인다. $C=512$ 에서 batch 가 문서 256 개이고 [Memorizing Transformers § 4.2] batch 원소마다 별도의 E 가 필요하므로, 곱하면 $n_E=8192$ 에서 8 GiB, $n_E=65,536$ 에서 64 GiB 다 [본서 산술]. 논문은 두 값을 나란히 적어 놓고 한 번도 곱하지 않는다.

[평가] 용량이 batch 와 곱해진다는 것이 이 설계의 서빙 질문전부다. 논문은 그것을 아키텍처 사실로만 진술하고 회계하지 않는다 — batch 원소마다 별도의 E 가 필요하다는 문장과 batch 크기를 적은 문장 사이에 곱셈이 없다 [Memorizing Transformers § 3.1, § 4.2].

[평가] gather 트래픽에는 정확한 손익분기가 있다. 회수 후 gather 는 kNN layer 하나에서 토큰당 $k_{nn} \times d_{\text{head}} \times 2 \times 2 \times 8 = 131,072 \text{ bytes} = 128 \text{ KiB}$ 를 옮기고 이 값은 n_E 와 무관한 반면, 저장소 전체를 dense scan 하면 $n_E \times 8 \times 128 \times 2 \times 2 \text{ bytes}$ 를 옮긴다. 둘이 같아지는 지점은 $n_E = C \cdot k_{nn} = 512 \times 32 = 16,384$ 쌍이다. 그보다 작으면 **회수가 전부 옮기보다 더 많은 바이트를 읽고**, 크면 회수가 이기며 이득이 선형으로 커진다. 논문의 1536 과 8192 는 둘 다 손익분기 아래다. 이항등식은 shape 에서 정확히 따라 나오지만, § 12.5 의 step-time 삼중값은 검색 · attention · 쓰기를 묶은 훈련 측정이므로 그 증거가 되지 못한다.

[해설] 독자의 세계로 옮기면 둘이 남는다. 첫째, **shared-weight batching 이 온전하다**. 사용자별로 Θ 에 쓰이는 것이 없으므로 요청들은 가중치 사본 하나를 공유하고, 요청별 상태는 KV cache 처럼 거동한다 — 문서 시작에 할당, 상한까지 성장, 문서 끝에 해제. Rosetta 표에서 이 논문에 대응하는 항목은 “모델 재배포” 가 아니라 “KV cache” 다. 논문 스스로 그 대비를 윤리 절에 적으며 사용자 지식의 $O(1)$ 삭제를 아키텍처 속성으로 내세운다 [Memorizing Transformers Ethics]. 둘째, 접근 패턴이 직관과 반대다. 문서당 256 MiB 는 SRAM 이 아니라 HBM 에 내려앉고, 논문이 가리키는 10 억 규모에서는 host DRAM 이나 디스크의 ANN 색인 뒤로 간다 [Memorizing Transformers § 3.3]. 대부분이 차가운 큰영역에 대한 무작위 gather 이고, decode 에서 익숙한 streaming KV cache 의 정반대다.

decode 제약도 하나 명시해 둔다. 쓰기가 chunk 경계에서만 일어나므로 생성 중인 모델은 **현재 chunk 안에서 자기가 만든 토큰을 회수할 수 없다** — 최대 512(또는 2048) 토큰이 chunk 가 닫힐 때까지 E 에 보이지 않는다 [본서 추론]. 논문은 생성을 돌리지 않아 표면화되지 않지만, 이기제를 decode 경로에 놓는 어떤 설계에도 걸린다 [본서 추론].

12.8 한계와 bridge-out

논문이 남긴 문제는 다섯이다. 거대한 E 를 무엇으로 채울 것인가 — “How to make the best use of this capability is a topic for future work” [Memorizing Transformers § 5]. 읽기 게이트를 내용 의존으로 만들 것인가 (명명만 하고 실행하지 않음). staleness 가 실제로 불안정성의 원인인가. n_E 의 수익 체감점을 문서 길이가 정하는가 top- k_{nn} 혼잡이 정하는가 — [Memorizing Transformers § 4.5] 는 전자로 [Memorizing Transformers § 4.6] 은 후자로 귀속시키고 화해시키지 않는다. 왜 하위 층은 장거리 문맥이 필요 없는가 — 관찰 하나에서 결론으로 곧장 간다.

여섯 번째가 이 책에 가장 중요하다. 윤리 절이 삭제 가능성을 아키텍처 요구사항으로 제기한다 — “The same is not true of differentiable model parameters” [Memorizing Transformers

Ethics]. 2022 년에 미리 적힌 Θ -경로에 대한 반론이며 ch19 – ch21 이 통과해야 할관문이다. 반대로 한 번도 묻지 않은 것도 있다: E 와 Θ 를 함께 쓸 수 있는가. 논문은 둘을 대안으로만 세운다.

경로 간 인용. E -경로의 전 (前)-에이전트 회수 계보는 읽는다 — kNN-LM, Yogatama et al. 2021, REALM, MARGE, RAG, Faiss, ScaNN [Memorizing Transformers § 2]. W -경로는 전무하다: fast weight 도 Hebbian 도 meta-learning 도 test-time training 도 없다. 이 corpus 의 W -경로 논문은 전부 2022 년 3 월 이후이므로 그쪽은 연대가 강제한 침묵이지만, 2022 년 이전 fast-weight 문헌의 부재는 선택이다 (감사의 글은 Imanol Schlag 의 교정을 사례하면서 참고문헌에는 그 계보를 넣지 않는다 — 이 병기는 이 책의 주석이다). Θ -경로도 전무하고 이쪽은 명백히 선택이다: LoRA 는 2021 년 6 월부터, ROME 은 2022 년 2 월부터 있었다. 생물학·CLS 도 전무하다 — 이중 시간척도 기억 시스템을 지으면서 CLS 어휘를 한 번도 쓰지 않는다.

[평가] 침묵은 양방향이다. 뒤의 E -경로도 이 논문을 인용하지 않는다 — MemGPT·Mem0·Zep·ReasoningBank·Letta STC 어느 쪽도 참조하지 않으며 corpus 에서 이 논문을 인용하는 파일은 둘뿐이다. 그중 하나가 압축 주장을 정면으로 기각한다: 이런 방법들은 “sparsify retrieval but not storage. The full KV still lives somewhere, so the true rate(Z) is unchanged and the compression is of compute, not memory” [Rate – Distortion Memory Compaction, arXiv:2607.08032 — 이 논문 자신의 진술이 아니다. → ch22]. 이 책의 판단은 그 기각이 옳다는 것이다. § 12.7 의 문서당 256 MiB 가 표 12-3 의 빈 C_{cap} 칸을 채운 값이며, **표현 수준 E 는 계산을 압축하고 저장을 압축하지 않는다.**

ch13 이 받아 가는 것은 둘이다. B_s 축의 원점 — 쓰기에 아무것도 쓰지않고 얻어지는 것의 상한 — 과, 프롬프트 접합이 아닌 읽기의 손익 회계다. ch13 은 같은 E 를 텍스트로 되돌린다. 그때 항 ② (prefill 증가) 가 되살아나고, 대신 어떤 체크포인트에나 붙는 이식성이 생기며, $S(c; B_s)$ 가 항등에서 떨어져 나가기 시작한다.

요약

- Memorizing Transformers 는 sleep-time compute 이 아니다. 쓰기가 wake pass 안 chunk 경계에서 동기적으로 일어나고 한계 예산이 0 이므로 조건 (1)·(2) 에 걸린다 (→ ch11). 이 장은 그것을 B_s 축의 원점으로 읽는다.
- 갱신 규칙은 (U-E) 하나뿐이고 $S(\cdot; B_s)$ 는 표현 공간의 항등사상이다. $\hat{c}_t$ 는 forward pass 가 이미 만든 (k_i, v_i) 그대로이며, W 는 없고 Θ 는 배포 후 움직이지 않는다.
- 읽기는 프롬프트 접합이 아니라 12 층 중 layer 9 에서의 top- k_{nn} ($k_{nn}=32$, recall 약 90%) 회수 후 attention 이고, local 출력과 내용 비의존 per-head 게이트로 섞인다.
- 비용 4 중 중 값이 든 칸이 없다. $B_s \cdot L_w \cdot \rho$ 는 논문에 없고 (B_s 는 구조적으로 0, ρ 는 이름만), C_{cap} 은 개수로만 보고되며 바이트가 없다. 유일한 벽시계 값은 200M 모델의 훈련 step 시간 0.2/0.25/0.6 초다.
- 실증 범위는 좁다. 지표는 perplexity 하나, 다섯 corpus 중 셋이 저자제작이고 그중 둘은 장거리 의존성을 제조하는 방식으로 이어 붙였으며, 회수 baseline 은 없고 헤드라인 스케일 주 장에는 표도 오차 막대도 없다.
- 서빙 순이득은 shared-weight batching 유지와 $O(1)$ 삭제이고, 순비용은 한 층의 GEMM → gather 전환과 batch 와 곱해지는 문서당 상태 (bf16 가정 시 $n_E=65,536$ 에서 문서당 256 MiB).
- 상속도 계승도 없다. 뒤의 E -경로는 이 논문을 인용하지 않고, 인용하는 한 편은 압축 주장을 기각한다 — 이 기제는 저장이 아니라 계산을 압축한다.

자가 점검체크리스트

- 이 논문이 조건 (1)·(2) 에 걸리는 이유를 갱신식과 시계로 설명하고, 그럼에도 E -경로의 조 상으로 읽는 이유를 말할 수 있다.

- 식 (12-1) – (12-5) 를 $(R) \cdot (U-E)$ 와의 차이로 진술하고, $S(:, B_s)$ 가 항등일 때 상각식 (A) 가 왜 붕괴하는지 설명할 수 있다.
- 표 12-3 의 네 칸이 각각 왜 비어 있는지 근거와 함께 말하고, 유일한벽시계 수치를 서빙 지연으로 옮기면 안 되는 이유를 셋 달 수 있다.
- $n_E = C \cdot k_{nn}$ 손익분기를 유도하고, 그 아래에서 회수가 전부훔기보다 더 많은 바이트를 읽는다는 결론을 자기 shape 로 재계산할 수 있다.
- Rosetta: 이 E 가 왜 “모델 재배포” 가 아니라 “KV cache” 행에대응하는지, 그 대응이 ch01 의 “KV cache $\leftrightarrow W$ ” 행과 어디서 갈라지는지 설명할 수 있다.

ch13. RAG에서 MemGPT로 — 검색에 서운영체제로

13.1 Bridge-in: 전작이 남긴문제

MemGPT(arXiv:2310.08560)가 명시적으로 상속한문제는 긴 문맥을 넓혀도 모델이 가운데를 쓰지 못한다는 것이다. 논문 [MemGPT § 1]이 그 상속을 인용으로 적는다.

“even if we could overcome the computational challenges of context scaling, recent research shows that long-context models struggle to utilize additional context effectively (Liu et al., 2023a). As consequence, given the considerable resources needed to train state-of-the-art LLMs and diminishing returns of context scaling, there is a critical need for alternative techniques to support long context.” [MemGPT § 1]

같은 진술이 실험 절에서 한 번 더 반복된다. “Recent research (Liu et al., 2023a) also raises doubts about the utility of simply scaling contexts, since they find uneven attention distributions in large context models …To enable reasoning across documents, more flexible memory architectures like MemGPT are needed.” [MemGPT § 3.2]

상속은 수사에 그치지 않는다. 네 개 벤치마크 중 둘이 그 선행 논문에서 그대로 왔다 — retriever-reader document QA [MemGPT § 3.2.1]와 합성 key-value retrieval [MemGPT § 3.2.2]. 문제를 물려받고 그 문제를 정의한 측정 도구까지 물려받았다.

다만 정확히 적어 둔다. MemGPT는 어떤 선행 연구에 대해서도 “open question”, “future work of”, “left open by”라는 표현을 쓰지 않는다. 위상속은 두 구절과 벤치마크 재사용으로부터 이 책이 복원한 것이다.

두 번째 상속은 이 책 안에서 온다. ch10의 결론은 RAG의 쓰기 쪽 공백이었다— 그 기억은 논문 자신의 표현으로 “human-readable”이고 “human-writable”이며 [RAG § 5], 빠진 세 번째 성질이 machine-writable이다 (→ ch10 § 10.2). Liu et al.의 문제가 “창을 키우지마라”라면 ch10의 공백은 “그럼 무엇이 쓰는가”이고, MemGPT는 두 질문에같은 답을 낸다: **모델 자신이 쓴다.**

13.2 문제의식

MemGPT의 문제 진술은 한 문장이다 — “In this paper, we study how to provide the illusion of an infinite context while continuing to use fixed-context models” [MemGPT § 1]. 고정 창을 그대로 두고 무한 문맥의 착시를 만들겠다는 것이다.

그 착시를 만드는 방법이 이 장의 첫 번째 소유 개념이다.

정의. 계층화된 외부 기억은 E 를 하나의 평평한 레코드 집합으로 두지 않고, **프롬프트 안에 상주하는 층과 프롬프트 밖에 사는 층**으로 나눈 뒤 두 층 사이의 이동을 명시적 연산으로 관리하는 관점이다. 이동을 촉발하는 것은 질의의 내용이 아니라 **상주 층의 용량 초과**이며, 따라서 E 설계의 1차 변수가 검색 품질이 아니라 “어느 층에 무엇을 남길 것인가”로 바뀐다.

RAG의 E 와 대비하면 세 축이 전부 뒤집힌다. RAG의 E 는 corpus 당전역·읽기 전용·평평이라 승격도 강등도 연산으로 존재하지 않고, 검색을 촉발하는 것은 언제나 질의다. MemGPT의 E 는 사용자당·읽기쓰기·최소 2계층이며, 쓰기를 촉발하는 것은 질의가 아니라 프롬프트 길이다.

최적화 대상도 함께 갈아엎는다. RAG가 올린 것은 $\text{ret}(E, q)$ 의 정확도였지만 ($\rightarrow$ ch10 § 10.4), MemGPT는 ret 을 손대지 않는다 — `text-embedding-ada-002`를 그대로 쓰고 [MemGPT § 3.2.1] 검색 품질에 어떤 기여도 하지 않는다. 대신 검색 결과를 언제 어떻게 받아 어디에 남길지를 모델의 결정으로 옮긴다.

[해설] 익숙한 대비로 옮기면, RAG는 캐시 **적중률**을 올리는 논문이고 MemGPT는 캐시 **교체 정책**을 만드는 논문이다. 그리고 그 교체 정책을 하드웨어도 런타임도 아닌 **모델 자신**이 실행한다. 이 계보의 실질적 시작은 저장소가 커진 것이 아니라 저장소를 운영 하는 주체가 처음으로 모델이 된 것이다.

논문이 이 관점에 붙인 이름이 이 장의 두 번째 소유 개념이고, 그 이름은 곧바로 한계를 데리고 온다.

정의. virtual context management는 고정 크기 프롬프트를 OS의 main memory로, 모델 밖 저장소를 disk로 놓고 둘 사이의 데이터 이동을 시스템이 관리해 무한 문맥의 착시를 만드는 기법이며, 논문 자신이 붙인 이름이다 [MemGPT Abstract]. **이 이름이 빌려 온 OS의 기제와 논문이 실제로 구현한 것이 갈라지는 자리가 셋이다** (판정은 아래 [평가]). 첫째, MemGPT의 page fault는 비자발적 트랩이 아니라 프롬프트 안에 텍스트로 배달되는 **경고**이고, 모델이 그것을 무시하면 아무 일도 일어나지 않는다 [MemGPT § 2.2]. 둘째, 주소 변환도 보호도 상주 보장도 없다 — 이동은 전부 자발적이다. 셋째, 논문이 인용하는 유일한 OS·아키텍처 문헌은 Patterson, Gibson & Katz 1988이며 [MemGPT § 1], 그것은 가상 메모리 논문이 아니라 RAID 논문이다.

[평가] 은유는 검증되지 않고 주장된다. 그럼에도 이 장이 은유를 지우지 않는 이유는 세 자리가 정확히 이후 E -경로가 채우려 한 자리이기 때문이다. “자발적 page fault”는 **정책이 학습되지 않았다**의 다른 표현이고, “상주 보장 없음”은 **쓰기의 성공이 보장되지 않는다**의 다른 표현이다. ch14·ch15가 손대는 지점이 이 둘이며, 은유의 결합 목록이 그대로 후속 연구 목록이 된다.

13.3 Core mechanism (통일표기)

먼저 못 박아야 할 사실이 있다. **이 논문에는 번호 붙은 수식이 하나도 없고, 수학 표기 자체가 없다.** 순수 systems 논문이며 갱신 규칙 전부가 영어 산문으로 [MemGPT § 2.1 – § 2.4]에 쓰여 있다. 따라서 아래식은 전부 이 책이 그 산문을 통일 표기로 옮긴 것이고, 원문 수식 번호 대신 **절 위치**를 병기한다. 이 부재 자체가 보고 대상이다 — E -경로의 조상이 자기 갱신 규칙을 수식으로 진술하지 않는다.

상태 분해부터 한다. Θ 는 동결된 상용 API 세 개다: gpt-4-1106-preview(문맥 128,000), gpt-4-0613(8,192), gpt-3.5-turbo-1106(16,385) [MemGPT § 3]. W 는 아예 존재하지 않는다 — Neural Memory가 통째로 다른 층이 여기서 빈칸이다 ($\rightarrow$ NM ch12). E 와 그 프롬프트 내 투영은 다섯 블록으로 나뉘며, 결정적인 분할은 **프롬프트 안이나 밖이냐**다 [MemGPT § 2, § 2.1].

프롬프트 안의 셋은 I_{sys} (제어 흐름 설명과 함수스키마를 담은 읽기 전용 정적 블록), E^{wc} (working context — “a fixed-size read/write block of unstructured text, writeable only via MemGPT function calls”), Q (FIFO queue — 롤링 메시지 이력이며 0번 자리에 추출 메시지들의 재귀 요약 σ 가 들어간다)다 [MemGPT § 2.1, § 2.3]. 프롬프트 밖의 둘은 E^{rec} (recall storage — 들어온 모든 메시지와 나간 모든 출력을 담은 메시지 DB)와 E^{arc} (archival storage — “a read/write database storing arbitrary length text objects”)다 [MemGPT § 2.2].

앞의 셋은 이 책의 표기에서 E 가 아니라 E 의 **프롬프트 내 투영**, 즉 식 (R)에서 f 의 인자로 들어가는 텍스트다. 실제 E 는 E^{rec} 와 E^{arc} 뿐이다. 이 구분을 놓치면 MemGPT의 “main memory”를 W 로 오독하게 된다.

프롬프트 조립은 세 블록의 연접이다 [MemGPT § 2.1].

$$P_t = [I_{\text{sys}}; E_t^{\text{wc}}; Q_t], \quad Q_t = [\sigma_t; m_i, \dots, m_t] \quad (13-1)$$

읽기는 식 (R) 에 $W = \emptyset$ 을 넣고 `ret` 을 반복으로 바꾼 형태다 [MemGPT § 2.3, § 2.4].

$$\hat{y} = f(q; \Theta, \emptyset, \text{ret}(E, q)), \quad \text{ret}(E, q) = \bigcup_{j=0}^{J-1} \text{search}(E^{\text{rec}} \cup E^{\text{arc}}, \tilde{q}_j) \quad (13-2)$$

식 (13-2) 에서 두 가지가 RAG 와 다르다. 검색 인자 $\tilde{q}_j$ 가 고정 encoder 로 q 에서 유도되지 않고 **모델이 디코딩하며**, J 가 하이퍼파라미터가 아니다 — 모델이 `request_heartbeat=true` 를 내는 동안 루프가 돌고 모델이 멈추면 끝난다. 검색 결과는 페이지 단위로 잘려 들어오므로 $\tilde{q}_j$ 가 같은 질의의 다음 페이지일 수도 있다 [MemGPT Fig. 6].

쓰기는 **성격이 정반대인 두 갈래**가 공존한다. 모델이 지시하는 갈래는 손실이 있고 확률적이다 [MemGPT § 2.3].

$$(E_t^{\text{wc}}, E_t^{\text{arc}}) = \text{wr}(E_{t-1}^{\text{wc}}, E_{t-1}^{\text{arc}}; a_t), \quad a_t \sim p_{\Theta}(\cdot | P_t) \quad (13-3)$$

큐 관리자가 실행하는 갈래는 결정론적이고 무손실이다 [MemGPT § 2.2].

$$E_t^{\text{rec}} = E_{t-1}^{\text{rec}} \cup \{m_t, \hat{y}_t\} \quad (13-4)$$

식 (13-3) 과 (13-4) 의 대비가 이 논문의 구조적 핵심이다. 전자에서 쓰기행위 a_t 는 **샘플링된 토큰열**이고 `wr` 은 그것을 해석하는 인터프리터다 — `append`, `replace`, `archival_storage.insert`. 파싱에 실패하면 런타임 오류가 텍스트로 모델에게 되돌아간다 [MemGPT § 2.3]. 즉 표준형 (U-E) 에 없는 **실패 모드**가 있다. 후자는 모델이 개입하지 않는 순수 로그이고 예산도 요약도 없다. 이후 E -경로 논문들은 이 둘 중 어느 쪽을 지배적으로 만들 것인가를 다시 설계한 것이다 ($\rightarrow$ ch15).

쓰기를 촉발하는 것은 질의가 아니라 프롬프트 길이다 [MemGPT § 2.2].

$$|P_t| > N_{\text{warn}} \Rightarrow Q_t \leftarrow \text{append}(Q_t, \text{"memory pressure"}), \quad N_{\text{warn}} = 0.70 N_{\text{ctx}} \quad (13-5)$$

$$|P_t| > N_{\text{flush}} \Rightarrow Q_t \leftarrow [\sigma_t; Q_t^{\text{keep}}], \quad \sigma_t = S(\sigma_{t-1}, Q_t^{\text{evict}}; \Theta), \quad N_{\text{flush}} = 1.00 N_{\text{ctx}} \quad (13-6)$$

축출량은 문맥창의 약 50%다 [MemGPT § 2.2]. 위 세 백분율 (70%, 100%, 50%) 은 원문이 전부 “e.g.” 를 달아 적은 예시값이지 실험에서 쓴 설정값이 아니다 — 그대로 인용하되 측정된 구성으로 읽으면 안 된다 [본서 판단].

식 (13-6) 의 $S(\cdot)$ 는 표준형 (U-E) 의 $S(c; B_s)$ 와 **방향이반대**다. (U-E) 에서 $S(c; B_s)$ 는 문맥을 압축해 저장소 **안으로** 넣지만, MemGPT 의 $S(\cdot)$ 는 축출된 프롬프트 내용을 압축해 프롬프트 **밖으로** 밀어내고 그동안 E^{rec} 는 식 (13-4) 에 의해 아무것도 잃지 않는다. 손실은 저장소가 아니라 프롬프트에서만 일어난다. 그리고 이 압축의 예산은 계산 예산이 아니라 **토큰 개수 목표**다 — MemGPT 에 압축 예산이 존재하는 유일한 자리이고, 그것이 B_s 가 아니라는 사실이 § 13.5 의 결론을 미리 정한다.

마지막으로 제어 흐름이다 [MemGPT § 2.4].

$$\text{사이클 } j: \quad o_j \sim p_{\Theta}(\cdot | P^{(j)}), \quad P^{(j+1)} = [P^{(j)}; \text{exec}(o_j)] \quad (\text{heartbeat=true}), \quad \text{아니면 정리} \quad (13-7)$$

식 (13-7) 이 말하는 것은 **사용자 톰 하나가 forward pass 하나가 아니라는 것**이다. 연쇄된 함수 호출 하나하나가 직전보다 긴 프롬프트위에서 도는 완전한 추론 사이클이며, 논문은 이 사이클 수를 한 번도 세지 않는다.

13.3.1 표기 대응표

표 13-1 — MemGPT 원 논문 기호 → 이 책 기호. 이 논문에는 번호 붙은 수식이없으므로 원문 위치는 절·그림 번호로 병기한다.

원 논문 표기	원문 위치	이 책 표기	처리
“memory” / “main context” / “external context”	§ 2, § 2.1	E (프롬프트 밖만) / f 의 인자 (프롬프트안)	논문은 프롬프트 블록과 DB를 둘 다 memory라 부른다. 이 책에서 E 는 프롬프트 밖 부분만이다
working context	§ 2.1	E^{wc}	프롬프트 내 투영. 이후 Letta 계열의 “core memory” 개명은 이 논문에 없다 (→ ch14)
FIFO queue	§ 2.1	Q_t	원문의 Q 는 attention의 query 행렬로 오독되므로 필기체로 옮긴다
recursive summary	§ 2.1, § 2.2	σ_t	(U-E)의 $S(\cdot; B_s)$ 에 대응하나, S_t (momentum)와 절대 혼동하지 않는다. 이 장은 $W = \emptyset$ 이므로 S_t 가 등장하지 않는다
“context window” (토큰 단위)	Table 1, § 3	N_{ctx}	예약기호 C (chunk)· C_{cap} (용량)과 절대 혼용하지 않는다
warning / flush token count	§ 2.2	$N_{\text{warn}} / N_{\text{flush}}$	예약 기호 τ (세션 첨자)와 충돌하므로 N 계열로 옮긴다. 셋 다 토큰 수다
K (회수 문서 수)	§ 3.2.1	K_{ret}	ch10 § 10.4가 도입한 회수 폭 기호를 그대로 쓴다. 예약 기호 k (sleep 라운드)· K (key 행렬)와 구분
“function” (MemGPT의 호출 가능 객체)	§ 2.3	search, append, replace, insert	예약기호 f (식 (R)의 읽기 사상)로 축약하지 않는다
“page” / “paging” / “page fault”	§ 1, § 2.2	(번역하지 않음)	기호 충돌은 없으나 의미 함정이 있다 — MemGPT의 page fault는 자발적이고 모델이 발행한다.
70% / 100% / 50%	§ 2.2	그대로	§ 13.2 정의 참조 원문이 “e.g.”로 적은 예시값. 측정된 설정으로 제시 금지

13.4 어느 층을 언제쓰는가

이 절의 답은 한 줄이다. Θ 는 한 번도 움직이지 않고, W 는 존재하지 않으며, E 만 매 턴 움직인다. 그런데 더 중요한 사실이 하나 더 있다.

MemGPT에서는 아무것도 학습되지 않는다. gradient 가 하나도 계산되지 않고 loss 가 없으며 이 방법에 속하는 파라미터가 없다 — 논문에 training 절 자체가 없다 [MemGPT § 3]. 학습 가능해 보이는 모든 객체는 셋 중 하나다: 남이 사전학습해 동결한 것, 사람이 손으로 쓴 영어, wake time 에 디코딩으로 만들어진 텍스트. 독자의 1 번 질문 “이 값은 누가 언제학습하는가” 에 대한 답은 전 항목에서 **아무도, 한 번도다**.

표 13-2 — MemGPT 의 모든 상태 객체와 그 갱신 주체. “학습” 은 gradient 갱신을 뜻한다.

객체	층	누가 정하는가	언제	test time 에 움직이는가	출처
기본 LLM 가 중치	Θ	OpenAI 의 사전학습 · 후 처리	논문 이전. 전 구간 동결	아니오	§ 3
fast weights	W	없음 — 객체 자체가 없다	—	해당 없음	전논문에 부재
I_{sys} (기억 계층 설명 + 함수 스키마)	프롬프트	저자가손으로 작성	저작 시점. “read-only (static)”	아니오	§ 2.1, § 2.3, App. § 6.1
E^{wc} (working context)	E 의 프롬프트 내 투영	LLM 이 append/replace 를 발화	wake time , 사용자 턴의 임계 경로 위	예 — 이 논문의 중심 상태	§ 2.1, § 2.3, Fig. 1, Fig. 4
Q 와 σ	프롬프트	큐 관리자 (결정론적 정책) 가 요약 텍스트 생성에만 Θ 호출	wake time, flush 시점	예	§ 2.2
E^{rec} (recall storage)	E	큐 관리자가 무조건. 모델 결정 없음	wake time, 매 메시지	예 (append-only, 압축 없음)	§ 2.2
E^{arc} (archival storage)	E	쓰기는 LLM 의 함수호출, 벌크 적재는 운영자	wake time / 실험 전 사전 적재	예	§ 2.2, § 3.2.1
embedding 모델 + HNSW index	E 의 읽기 경로	OpenAI(모델), 빌드 (index)	동결. embedding 사전 계산	아니오	§ 3.2.1
기억 관리 정책 (언제 쓰고 언제 검색하고 언제 멈추는가)	—	아무도 — 학습되지 않고 프롬프트로 지시된다	해당 없음	매 사이클 디코딩으로 처음부터 재유도	§ 2.3, § 5

표의 마지막 행이 이 장의 무게중심이다 — “We implement self-directed editing and retrieval by providing explicit instructions within the system instructions” [MemGPT § 2.3]. 정책은 지시되지 학습되지 않는다.

한 턴 안의 순서는 이렇다. 메시지가 Q 에 붙는다 → 식 (13-4)의 로그 쓰기 → 필요하면 식 (13-5)의 압력 경고 → 식 (13-2)의 읽기와 식 (13-3)의 쓰기가 식 (13-7)의 연쇄 안에서 **자유롭게 섞이며** 반복 → 필요하면 식 (13-6)의 flush와 요약 → 답변. 읽기 단계와 쓰기 단계의 분리가 없고, 이 무단 계성이 비용을 세지 못하게 만드는 직접 원인이다.

이 장의 의무 caveat 하나를 여기서 명시한다. **MemGPT는 이 책의 판별식 조건 (1)을 만족하지 않는다.** 판별식은 질의 q 가 도착하기 전에 예산을 써서 상태를 바꿀 것을 요구하는데 (→ ch11), MemGPT의 쓰기는 전부 대화 중에, 사용자 턴의 임계 경로 위에서, 답을 만드는 것과 같은 디코딩 예산으로 일어난다 (식 (13-3), (13-6)). 그럼에도 E -경로의 출발점으로 두는 근거는 둘이다 — 그 쓰기의 산출물이 **이후 질의**에 쓰이고 (조건 (4)는 만족), ch14가 이 계보를 자기조상으로 지목한다. MemGPT는 sleep-time compute의 구조적 조상이 되 sleep-time compute이 아니다.

[해설] 유희 예산의 자리는 논문 안에 이미 뚫려 있다. [MemGPT § 2.4]가 이벤트 종류를 열거하며 “timed events that are run on a regular schedule (allowing MemGPT to run ‘unprompted’ without user intervention)”을 적는다 [MemGPT § 2.4]. 정기 스케줄로 도는 사용자 개입 없는 실행 — 이것이 B_s 가 들어갈 구멍이다. 그런데 어떤 실험도 이혹을 쓰지 않고 어떤 계산도 여기에 귀속되지 않는다. ch14가 하는 일을 한 문장으로 줄이면 **이 혹을 실제로 실행하고 그 비용에 이름을 붙인 것**이다.

13.5 비용 4종

표 13-3 — MemGPT의 비용 4종. 논문이 침묵한 칸은 “논문에 없음”으로 적었고, 추정치는 넣지 않았다.

기호	값	논문이 대신 말하는 것
B_s	논문에 없음	sleep 단계가 없으므로 예산도 없다. 혹은 있다 — “timed events that are run on a regular schedule” [MemGPT § 2.4]. 어떤 실험도 이 혹을 쓰지 않고 계산이 귀속되지 않는다
L_w	논문에 없음	TTFT 도, ms/token 도, wall-clock 도, 토큰 수도, 달러 비용도 논문 전체에 없다. 지연에 관한 유일한 문장은 시스템이 아니라 벡터 index에 대한 것이다 — “which uses an HNSW index to enable approximate, sub-second query times” [MemGPT § 3.2.1]. 그것은 archival 검색 1회 의 상한이고, MemGPT는 한 턴에 여러 번 호출한다 (식 (13-7))

기호	값	논문이 대신 말하는 것
C_{cap}	논문에 없음 (바이트 기준)	주어진 것은 상태 용량이 아니라 문맥창 N_{ctx} 뿐이다: 8,192 / 16,385 / 128,000 토큰 [MemGPT § 3]. E^{wc} 는 “fixed-size” 라고만 하고 그 크기를 끝내 밝히지 않는다 [MemGPT § 2.1]. $E^{\text{arc}} \cdot E^{\text{rec}}$ 에는 용량 정책이 없고, recall storage 는 명시적으로 무한하다 (“stored indefinitely”) [MemGPT § 2.2]. corpus 규모 수치는 공개 아티팩트 하나 뿐 — Wikipedia 문서 2 천만 건의 embedding [MemGPT § 3]
ρ	논문에 없음	반복 consolidation 실험도, 라운드별 열화 측정도, staleness 연구도 없다. Fig. 4 가 working_context.replace 로 “Boyfriend named James” 를 “Ex-boyfriend named James” 로 고치는 장면을 보여 주지만 [MemGPT Fig. 4] 지표가 붙지 않은 예시 대화다. DMR 과제는 이전 5 개 세션 + 새 세션 1 개를 쓰고 [MemGPT § 3.1], 세션 수를 변화시켜 열화를 드러내지 않는다
N_q	논문에 없음	한 문맥을 몇 개 질의가 공유하는지에 대한 분포가 없다

[평가] N_q 가 없다는 사실의 귀결은 명확하다. 식 (A) 를 이 논문으로 채울 수 없다 — 분자에 넣을 C_{sleep} 이 없고 (B_s 부재) 분모에 넣을 N_q 도 없다. 실서빙 N_q 분포를 보고한 논문이 corpus 에 없다는 사실의 한 사례다 (→ ch14, ch25).

[평가] C_{cap} 칸에는 특히 주의가 필요하다. ch10 이 RAG 에서 채운 C_{cap} 은 36 GB / 약 100 GB 였고 단위가 **corpus** 당이었다 (→ ch10 § 10.1). MemGPT 의 상태는 **사용자당**인데 그 값이 바이트로 보고되지 않는다. E -경로는 단위가 corpus 당에서 사용자당으로 바뀌는 바로 그 논문에서 숫자를 잃는다. 두 논문의 C_{cap} 을 나란히 놓아 비교할 수 없다는 뜻이며, 이결손은 corpus 전체로 확대된다 (→ ch26).

[평가] 비용 4 종이 전부 비어 있으므로 이 책은 MemGPT 에 대해 “효율적” 이라는 서술을 쓰지 않는다. 그런데 이 침묵은 중립적이지 않고 방향이 있다 — 식 (13-7) 이 사용자 턴 하나를 여러 번의 추론 사이클로 만들고 각 사이클은 직전보다 긴 프롬프트 위에서 돈다. 따라서 MemGPT 는 답하나당 반드시 baseline 보다 비싸다. 얼마나 비싼지를 묻는 질문자체가 논문에 등장하지 않는다.

13.6 실험과 스케일

이 절은 두 논문의 실험을 차례로 감사한다.

13.6.1 RAG의 평가설계 —전수

ch10은 RAG를 E 층의 형태와 비용으로만 읽었고 학습된 검색기가 FEVER에서 BM25에 지는 사실까지만 실었다 (→ ch10 § 10.6). 남은 평가설계 쪽사실 여덟 항을 여기서 전부 적는다.

(1) TriviaQA의 SotA 주장은 **split 의존적이고, 유리한 split이 쉬운쪽이다**. 헤드라인 문장부터 단서가 붙어 있다 — “On all four open-domain QA tasks, RAG sets a new state of the art (only on the T5-comparable split for TQA)” [RAG § 4.1]. 부록은 그 이유를 official Wiki test set의 문제가 Wikipedia에서 답하기 더 쉽기 때문이라고 적는다 [RAG App. D]. 헤드라인 68.0은 그 쉬운 쪽 split의 값이다. 관행적인 open-domain split에서는 RAG-Sequence가 56.8이고 같은 표의 DPR이 57.9다 [RAG Table 1] — 이 split에서는 진다.

(2) MS-MARCO 비교가 **like-for-like가 아니며, 논문도 그렇게 쓴다**. 저자들은 과제가 제공하는 gold passage 열 개를 버리고 open-domain 설정으로 푼 뒤 [RAG § 3.2], 캡션에 “*Uses gold context/evidence” 라고 표시된 SotA와 비교한다 [RAG Table 2]. 논문 자신이 어떤 질문은 gold passage 없이는 참조 답과 일치하게 답할 수 없고 어떤 질문은 Wikipedia만으로 답할 수 없다고 덧붙인다 [RAG § 3.2]. 조건이 다른 두 시스템의 점수가 한 표에 나란히 인쇄된다.

(3) Jeopardy는 저자가 제안한 과제이고, 평가도 **baseline**도 저자가 만들었다. “As this is a new task, we train a BART model for comparison” [RAG § 3.3]. 인간 평가는 BART와 RAG-Token 생성물 452쌍의 쌍대 비교이며 선택지는 A 우세/B 우세/둘 다 좋음/둘 다 나쁨 네 개다 [RAG § 3.3, § 4.3]. 임베드된 gold 문장 검사를 통과하지 못한 평가자 두 명이 제거되었고 [RAG App. B], 평가자 수도 평가자 간 일치도도 신뢰구간도 보고되지 않는다.

[평가] (4) 그 인간 평가 표의 한 열이 100%가 되지 않는다. Table 4의 Specificity 열은 BART better 16.8%, RAG better 37.4%, Both good 11.8%, Both poor 6.9%, No majority 20.1%로 합이 93.0%다 [RAG Table 4]. 같은 표의 Factuality 열은 7.1 + 42.7 + 11.7 + 17.7 + 20.8 = 100.0%로 맞는다. 논문은 이 차이를 언급하지 않는다.

(5) 같은 표에 대해 산문과 표가 어긋난다. [RAG § 4.3]은 “both RAG and BART were factual in a further 17% of cases” 라고 쓰지만, Table 4의 Factuality “Both good”은 11.7%이고 17.7%는 “Both poor”다 [RAG Table 4]. 어느 쪽을 옳긴 것인지 원문에서 결정되지 않는다. 이 책은 표값을 인용하고 불일치를 그대로 기록한다.

(6) **null-document** 기제는 세 변형을 만들고 전부 버렸는데, 숫자가 하나도 없다. 학습되는 null embedding, 정적 학습 bias 항, logit을 예측하는 작은 신경망을 시도하고 “We did not find that these improved performance, so in the interests of simplicity, we omit them” 으로 끝낸다 [RAG App. F]. 수치가 없으므로 답할 수 없는 질의에서 손해가 얼마인지 알 수 없다 [본서 추론].

(7) RAG-Sequence와 RAG-Token의 순서가 과제마다 뒤집히고, 고르는 규칙이 없다 [본서 관찰]. RAG-Token이 이기는 곳은 Jeopardy B-1 17.3 대 14.7, QB-1 22.2 대 21.4, WebQuestions 45.5 대 45.2 이고, RAG-Sequence가 이기는 곳은 MS-MARCO R-L 40.8 대 40.1 과 B-1 44.2 대 41.5, Natural Questions 44.5 대 44.1, CuratedTrec 52.2 대 50.0 이다 [RAG Table 1, Table 2]. 논문의 설명은 Jeopardy에 대한 사후 이야기 — RAG-Token이 여러 문서의 내용을 결합할 수 있어서일 수 있다는 것 — 뿐이다 [RAG § 4.3].

(8) 모든 증거가 단일 corpus에서 나온다. 2018년 12월 Wikipedia dump 하나이고 2016년 12월 dump는 index 교체 프로브에만 쓰인다 [RAG § 3, § 4.5]. 웹 corpus도 도메인 corpus도 다중 corpus 혼합도 사용자별 파티션도 시험되지 않는다 — Broader Impact 절이 “by endowing it with a medical index” 같은 용도를 직접 제안하는데도 그렇다 [RAG Broader Impact].

[평가] 여덟 항을 묶으면 성격이 드러난다. 넷은 비교조건의 문제이고 (1·2·3·8), 둘은 표의 내적 정합성 문제이며 (4·5), 둘은 보고되지 않은 음의 결과다 (6·7). 어느 것도 RAG

의기제를 반증하지 않는다. 반증되는 것은 그 기제의 이득 크기를 이논문에서 읽어낼 수 있다는 믿음이다. 이 결손은 비용 추의 공백과 독립이 아니다 (→ ch10 § 10.5) — 비용을 세지 않으면 이득을 어떤 척도에도 걸 수없으므로 비교 조건을 조이려는 압력이 생기지 않는다.

13.6.2 MemGPT의 실험

평가 대상은 폐쇄 API 세 개뿐이고 [MemGPT § 3], 파라미터 수는 어디에도 없다 — 전부 비공개 endpoint라 셀 수가 없다. open-weights 모델은 하나도 돌리지 않는데 정작 문제를 동기화하는 Table 1은 Llama, Llama 2, Mistral 7B, Yi-34B-200k로 채워져 있다. 그 Table 1의 메시지 수 열도 측정값이 아니다 — “preprompt of 1k tokens, and an average message size of ~50 tokens (~250 characters)”를 가정해유도한 값이며 [MemGPT Table 1], 논문의 동기 주장 (“이 만큼밖에 안들어간다”)을 지탱하는 것이 바로 그 열이다.

Deep Memory Retrieval (DMR). Multi-Session Chat 위에 저자들이 세션 6을 만들고 QA 쌍을 “generated using a separate LLM”으로 생성한 저자제작 벤치다 [MemGPT § 3.1, § 3.1.1].

표 13-4 — DMR 정확도와 ROUGE-L recall [MemGPT Table 2]. 정확도는 GPT-4 판정기 채점이다.

시스템	정확도	ROUGE-L recall
GPT-3.5 Turbo	38.7%	0.394
GPT-3.5 Turbo + MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
GPT-4 + MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
GPT-4 Turbo + MemGPT	93.4%	0.827

[평가] 이 표에는 세 가지가 함께 붙어야 한다. 첫째, **비교가 정보 동등하지 않다**. 논문 자신의 문장이 “The baselines are able to see a lossy summarization of the past five conversations ...while MemGPT instead has access to the full conversation history”다 [MemGPT § 3.1.1]. 92.5%대 32.1%의 격차는 방법의 효과와 입력접근권의 효과를 합쳐서 낸 값이고, 접근권을 맞춘 baseline은 돌지않았다. 둘째, **baseline이 뒤집혀 있다**. GPT-4 baseline(32.1%, 0.296)이 GPT-3.5 Turbo baseline(38.7%, 0.394)보다 낮다. 논문은 이 역전을 언급하지 않으며, 역전은 GPT-4의 상승폭(+60.4점)을 GPT-3.5의 상승폭(+28.2점)보다 커 보이게만든다. 셋째, **판정기가 관대하게 지시받았다** — “you should be generous with your grading - as long as it touches on the same topic as the gold answer, it should be counted as CORRECT” [MemGPT App. § 6.1.2]. 판정기는 피험 시스템과 같은 모델 계열이며, DMR 평가 집합의 크기는 논문 어디에도 없다.

Conversation Opener. 저자가 정의한 지표로 저자가 만든 과제다 [MemGPT § 3.1.2].

표 13-5 — Conversation Opener 유사도 [MemGPT Table 3]. SIM-1·SIM-3은 gold persona 라벨과의 유사도, SIM-H는 사람이 쓴 opener와의 유사도다.

시스템	SIM-1	SIM-3	SIM-H
Human	0.800	0.800	1.000
MemGPT (GPT-3.5 Turbo)	0.830	0.812	0.817
MemGPT (GPT-4)	0.868	0.843	0.773
MemGPT (GPT-4 Turbo)	0.857	0.828	0.767

[평가] 이 표에는 MemGPT 를 쓰지 않은 LLM 행이 아예 없다. 비교 대상이 사람이 쓴 opener 하나뿐이므로 이 실험은 MemGPT 에귀속되는 상승을 하나도 확립하지 못한다. “사람을 넘어선다” 는 주장을 지탱하는 것도 persona 라벨 대비 유사도인 SIM-1·SIM-3 뿐이다— SIM-H 에서는 사람이 정의상 1.000 이라 모든 변형이 지고, **가장 강한 backend 가 가장 낮다** ($0.767 < 0.773 < 0.817$). 저자들 자신이 MemGPT 의 opener 가 “more verbose and cover more aspects of the persona information” 라고 적는데 [MemGPT § 3.1.2], 그것은 persona 라벨 유사도를 부풀리는 바로 그 성질이 오히려 opener 가 더 낮다는 증거가 아니다.

Document QA 와 nested KV. 둘 다 표가 없고 결과가 그림으로만 존재한다 [MemGPT Fig. 5, Fig. 7]. Document QA 는 Liu et al. 2023a 의 과제를 그대로 쓰고 Natural Questions-Open 에서 질문 50 개를 표본으로 뽑았으며 [MemGPT § 3.2.1], nested KV 는 UUID 키-값 쌍 140 개 (약 8k 토큰) 에 대해 중첩 수준마다 순서 배치 30 개를 뽑았다 [MemGPT § 3.2.2]. 이 표본 크기에서 분산도 오차막대도 보고되지 않은 것은 중대한 결손이다 [본서 판단].

Document QA 에서 “MemGPT” s performance is unaffected by increased context length” 이고 [MemGPT Fig. 5 캡션] GPT-4 는 회수 문서가 늘수록 계속 좋아진다 [MemGPT § 3.2.1] — 고정 문맥 baseline 은 우상향하고 MemGPT 는 평평하다. 큰 K_{ret} 에서 **baseline 이 MemGPT 를 추월하는지, 교차점이 있다면 어디인지를 논문은 말하지 않는다.** nested KV 에서는 GPT-3.5 가 중첩 1 에서 0%로 떨어지고 GPT-4 와 GPT-4 Turbo 는 중첩 3 에서 0%에 도달하며 MemGPT+GPT-4 는 중첩 수에 영향받지 않는다 [MemGPT § 3.2.2].

[평가] 불리한 사실 셋이 이 두 그림에 붙는다. 첫째, **더 강한 기반 모델이 MemGPT 를 더 나쁘게 만든다** — “While GPT-4 Turbo performs better as a baseline, MemGPT with GPT-4 Turbo performs worse than MemGPT with GPT-4” [MemGPT Fig. 7 캡션]. 논문은 설명하지 않는다. 둘째, **약한 모델에서는 기제가 작동하지 않는다** — “MemGPT has significantly degraded performance using GPT-3.5, due to its limited function calling capabilities” [MemGPT § 3.2.1]. 셋째, **중심이론적 장점이 실현되지 않는다.** 무한 페이지징으로 검색기 한계를 넘는다는 것이 논지인데 관측은 그 반대다 — “we observe that MemGPT will often stop paging through retriever results before exhausting the retriever database” [MemGPT § 3.2.1]. nested KV 에서 MemGPT+GPT-4 Turbo 와 MemGPT+GPT-3.5 가 중첩 2 에서 떨어지는 것도 “failing to perform enough lookups” 로 귀속된다 [MemGPT § 3.2.2]. 적용된 유일한 대책은 프롬프트에 대문자로 지른 고함이다 — “DO NOT STOP SEARCHING UNTIL YOU VERIFY THAT THE VALUE IS NOT A KEY” [MemGPT App. § 6.1.6].

ablation 은 하나도 없다. working context·recall·archival·압력인터럽트·페이지네이션 중 어느 것도 제거하고 다시 재지 않았고, 구성요소에 대한 유일한 주장은 수치 없는 산문이다 — “we can see the storing information in working context is key to generating engaging openers” [MemGPT § 3.1.2].

실증 상한. 가장 긴 대화는 MSC 의 5 개 세션에 저자가 만든 세션 6 을 더한 것이다. 원문이 인쇄하는 값은 세션당 “roughly a dozen messages” 뿐이고 [MemGPT § 3.1], 메시지 60 – 70 개라는 수는 그 표현에 5 를 곱한 **이 책의 산수**이지 논문이 보고한 값이 아니다. 논문의 동기는 “weeks, months, or even years” 에 걸친 대화이고 문서 쪽 동기는 “million token mark” 를 넘는 SEC 10-K 인데 [MemGPT § 3.1, § 3.2], 실제로 평가된 문서 작업량은 상위 200 개 Wikipedia passage 다 [MemGPT Fig. 5]. 저장소 상한은 archival storage 에 벌크 적재된 Wikipedia 문서 2 천만 건의 embedding 이다 [MemGPT § 3].

13.7 Systems/serving 합의

논문이 직접 진술한 사실부터 정리한다. **모든 기억 트래픽이 디코딩된다.** LLM 은 프롬프트를 하나의 문자열로 받아 출력 문자열을 내고, MemGPT 가 그것을 파싱하며, 런타임 오류를 포함

한 결과가 다시 텍스트로 모델에게 들어간다 [MemGPT § 2.3]. DMA 도 별도 제어 평면도 없다. **archival 계층은 가속기 메모리가 아니라 데이터베이스다** — PostgreSQL + pgvector, HNSW index, “approximate, sub-second query times”, text-embedding-ada-002 로 사전 계산한 embedding [MemGPT § 3.2.1]. **검색은 토큰 인식이다** — 페이지네이션으로 회수 결과가 문맥창을 넘치지 않게 한다 [MemGPT § 2.3]. **압력 임계값은 프롬프트 길이 임계값이고 대역 내 텍스트로 배달된다** — 경고와 flush 사이에 N_{ctx} 의 약 30%를 여유로 두고 flush 에서 창의 약 50%를 버린다 [MemGPT § 2.2].

[해설] 독자의 질문 — 그래서 decode 에서 뭘 바꾸는가. **커널 안에서는 아무것도 바꾸지 않는다.** 새 상태도 새 GEMM 도 attention 변경도 없다. 바뀌는 것은 모델 위다. 논리적 토큰 하나가 예측 불가능한 횟수의 forward pass 가 되고 (식 (13-7)), 그 pass 들이 도는 프롬프트의 앞부분을 모델 자신이 다시 쓴다. E -경로는 긴 문맥을 **가속기 상태 바이트로** 사지 않고 **decode 사이클과 prefix 캐시 지역성**으로 산다.

사이클 수의 하한은 논문 그림의 trace 에서 읽힌다. Fig. 8 의 nested KV 해결과정은 archival 검색 네 번에 답변 한 번, 즉 질문 하나에 최소 다섯 사이클이고 각 사이클의 프롬프트가 직전보다 길다 [MemGPT Fig. 8]. Fig. 6 은 Wikipedia 질문 하나에 페이지네이션된 검색 두 번을 보인다 [MemGPT Fig. 6]. **논문은 사이클 수를 세지 않으므로 이 값들은 그림의 trace 에서 읽은 것이지 보고된 수치가 아니다.**

[평가] 세 가지를 이 책의 판단으로 덧붙인다. 첫째, **shared-weight batching** 은 온전하다. 어떤 파라미터도 사용자마다 움직이지 않으므로 (Θ 는 동결 API, $W = \emptyset$) 서버 fleet 은 가중치 사본 하나로 에이전트 사이클을 평범한 채팅 세션처럼 배치한다. 사용자별 상태는 Postgres 행과 프롬프트 토큰에 살고 둘 다 기존 스택의 일급 객체다. 이것이 E -경로가 $W \cdot \Theta$ -경로에 대해 갖는 **구조적** 이점이며, Rosetta 표의 “batch 로 weight 공유” 행이 왜 경로별로 갈리는지가 여기서 처음 드러난다.

둘째, **재귀 요약**을 **FIFO 0** 번에 두는 것은 **prefix KV cache 재사용에 최악의 배치다**. 식 (13-1) 의 레이아웃에서 flush 마다 다시 쓰이는 σ 가 앞쪽에 있으므로 뒤따르는 전부의 KV 가 무효화되고, E^{wc} 에 대한 append·replace 도 메시지 꼬리 전체에 같은 일을 한다. ch10 § 10.5 는 RAG 가 prefix cache 의 경계를 앞으로 민다고 했는데, MemGPT 는 한 걸음 더 간다 — **경계 앞쪽 구간 자체를 모델이 다시 쓴다**. 논문은 KV cache 를 한 번도 언급하지 않는다.

셋째, **턴당 이동 바이트는 저장소가 아니라 re-prefill 이 지배한다**. Fig. 6 에서 회수된 한 페이지는 텍스트 결과 열 개, 즉 킬로바이트 규모다 [MemGPT Fig. 6]. 그것을 삽입한 결과 — 다음 사이클에서 최대 N_{ctx} 토큰짜리 프롬프트에 attention 을 다시 도는 일 — 은 자릿수가 다른 가속기 트래픽이다. E -경로는 메모리 계층을 거의 건드리지 않으면서 HBM 쪽 작업을 배로 늘린다. 논문은 두 양 중 어느 것도 보고하지 않는다.

[평가] 운영 결손 둘을 마지막으로 적는다. **실패 양식이 서버에서는 tail latency 문제로 번역된다**. 보고된 실패는 “페이징을 너무 일찍 멈춘다” 이고 [MemGPT § 3.2.1], 처방인 “더 오래 찾게 만든다” 는 정확도 실패를 **무한 사이클 수 실패**로 바꾼다. 사이클상한도 타임아웃도 기술되지 않는다. **그리고 용량 계획이 구조적으로 정의되지 않는다**. recall storage 는 모든 것을 무기한 보관하므로 사용자당 저장 비용에 상한이 없고 [MemGPT § 2.2], **정작 쓸 수 있는 사용자당 상태는 크기가 공개되지 않은 고정 E^{wc} 가 제한한다** [MemGPT § 2.1].

13.8 한계와 bridge-out

논문 자신이 남긴 문제는 여덟 개다. (a) external context 를 무슨 저장기술로 받칠 것인가 — “integrating different memory tier technologies like databases or caches” [MemGPT § 5]. (b) 기억 관리 **정책**을 어떻게 개선할 것인가 — 같은 문장이 “further improving control

flow and memory management policies” 를 적지만 [MemGPT § 5] 정책을 학습시킨다는 선택지는 검토되지 않는다. (c) 에이전트가 충분히 오래 찾게 만드는 법 [MemGPT § 3.2.1, § 3.2.2]. (d) 왜 더 강한 기반모델이 더 약한 MemGPT 를 주는가 [MemGPT Fig. 7]. (e) 기억 계층의 비용 — 제기조차 되지 않는다. (f) E 가 무한히 커지면 어떻게 되는가 — 압축·추출·용량 정책이 없다 [MemGPT § 2.2]. (g) 사실 정정 기제가 작동하는가 — `working_context.replace` 는 예시될 뿐 측정되지 않는다 [MemGPT Fig. 4]. (h) open-weights 모델로 옮겨 가는가 — GPT-3.5 의 실패를 “limited function calling capabilities” 로 귀속시켜놓고 검증하지 않는다 [MemGPT § 3.2.1].

[평가] 아무것도 학습되지 않는다는 사실을 이 장은 완곡화하지 않는다. MemGPT 의 기여 전부가 동결된 폐쇄 API 세 개 위의 prompt-engineered function calling 이며 gradient 도 loss 도 이 방법에 속하는 파라미터도 없다 [MemGPT § 3]. 따라서 이 논문의 모든 능력 주장은 MemGPT 아키텍처에 대한 주장인 만큼 gpt-4-0613 의 지시 따르기 능력에 대한 주장이기도 하다.

그럼에도 이 장이 E -경로의 출발점인 이유는 넷이다. 첫째, **wr 에 처음으로 행위자가 생겼다** — ch10 이 남긴 machine-writable 공백을 이 논문이 메운다. 둘째, **쓰기의 출발 조건이 질의에서 용량으로 옮겨 갔다**. 식 (13-5) 의 인터럽트는 이후 모든 E -경로 시스템이 어떤 형태로든 갖는 부품이다. 셋째, **$S(\cdot)$ 가 처음 등장한다**. 방향은 (U-E) 와 반대지만 (§ 13.3) 문맥을 압축해 다른 층으로 옮기는 연산이 파이프라인에 들어온 것은 여기가 처음이다. 넷째, **B_s 가 들어갈 구멍이 뚫려 있다** — [MemGPT § 2.4] 의 timed events 혹이다. 학습이 없다는 사실과 계보의 출발점이라는 사실은 모순되지 않는다. 이 논문이 공급한 것은 학습된 부품이 아니라 인터페이스이고, 이후 연구는 그 인터페이스 뒤에 무엇을 넣을 것인가를 다룬다.

경로 간 인용. MemGPT 는 W -경로와 Θ -경로 어느 쪽과도 접촉하지 않는다. 참고문헌 약 40 개가 전부 장문맥 transformer 아키텍처, retrieval-augmented generation, LLM 에이전트, 평가 인프라다. fast weights, test-time training, 모델 편집, PEFT, 지속 학습, catastrophic forgetting, complementary learning systems, 생물학적 sleep·consolidation 인용이 0 건이다. 유일한 근접 사례는 [MemGPT § 4] 가 장문맥 문단 안에서 인용하는 “neural memory (Lee et al., 2019)” 인데, 그 참조는 Set Transformer 이고 fast-weight·TTT 의미의 neural memory 가 아니라 효율 기제다.

연대가 강제한 침묵과 선택된 침묵은 갈라야 한다. MemGPT 는 2023 년 10 월 (vendored 판본은 2024 년 2 월 v2) 이므로 이후의 sleep-time 계열 논문을 인용할 수 없었고 그 부재는 흠이 아니다. 그러나 **그 시점에 이미 존재했고 인용할 수 있었던 W · Θ -경로 문헌** — fast weight programmer 계열, test-time training, LoRA, ROME/MEMIT, EWC, CLS — 은 하나도 등장하지 않는다. RAG 쪽도 같은 방향으로 어긋난다. RAG 의 참고문헌에는 elastic weight consolidation 을 제목에 단 논문이 들어 있지만 [RAG § 3.4, Table 2], 그것은 FEVER-2-way SotA 92.2 라는 표의 한 칸으로만 쓰이고 catastrophic forgetting 내용은 아무 데도 쓰이지 않는다.

[평가] 세 경로의 침묵은 이 책이 사후에 부과한 분류가 아니라 첫 경로의 첫 논문에서 이미 관측되는 사실이다. E -경로는 같은 정보를 W 나 Θ 에 넣는 선택지를 **반박한 뒤** 출발한 것이 아니라 그 선택지를 **한 번도 제기하지 않은 채** 출발했다. 문제 정의가 “there is a critical need for alternative techniques to support long context” 라는 형태이기 때문이다 [MemGPT § 1] — context 가 유일한 축으로 놓이면 파라미터 기억은 설계 공간에 들어오지 않는다. 이 관측이 Part III 판정의 재료가 된다 (→ ch27).

ch14 가 받아 가는 것. 다음 장은 같은 저자들이 그다음에 무엇을 했는가를 다루고, 넘어가는 물건은 셋이다. 첫째, [MemGPT § 2.4] 의 timed events 혹이 실제로 실행되고 그 계산에 B_s 라는 이름이 붙는다. 둘째, 식 (13-3) 의 쓰기가 사용자 턴 밖으로 나가면서 판별식 조건 (1) 이 해소된다. 셋째, $\hat{e}$ 의 내용이 바뀐다 — MemGPT 의 $\hat{e}$ 는 대화에서 골라낸 사실 문장이지만 다음 장의 $\hat{e}$ 는 질의를 보기 전에 수행한 추론의 산출물이다. 물려받지 못하는 것도 있다: 비용 4 종이 전부 비어 있으므로 상각 논증을 세울 기준선이 이 장에는 없다.

요약

- MemGPT 가 명시적으로 상속한 문제는 Lost in the Middle 이다 — “there is a critical need for alternative techniques to support long context” [MemGPT § 1]. 상속은 수사에 그치지 않고 벤치마크 두 개를 함께물려받는다.
- 이 장의 관점은 **계층화된 외부 기억**이다. 이동을 촉발하는 것이질의가 아니라 상주 층의 용량 초과가 되면 E 설계의 1 차 변수가 검색품질에서 잔류 정책으로 바뀐다. 논문이 붙인 OS 은 유는 세 곳에서 샌다 —page fault 가 자발적 경고이고, 주소변환·보호·상주 보장이 없으며, 인용된 유일한 OS 문헌이 RAID 논문이다 [MemGPT § 1, § 2.2].
- **아무것도 학습되지 않는다**. 동결된 폐쇄 API 세 개 위의 prompt-engineered function calling 이며 gradient 도 loss 도 이 방법의파라미터도 없다 [MemGPT § 3].
- 쓰기가 대화 중 임계 경로 위에서 일어나므로 판별식 조건 (1) 을만족하지 않는다. 구조적 조상이되 sleep-time compute 은 아니다 — $w_r \cdot S(\cdot)$ 인터럽트를공급하고 B_s 를 공급하지 않는다.
- 비용 4 종이 전부 비어 있고 N_q 도 없어 식 (A) 를 채울 수없다. 그럼에도 식 (13-7) 이 턴 하나를 여러 사이클로 만들므로 답 하나당비용은 반드시 baseline 보다 크다.
- 실험은 ablation 0 건이고, DMR 비교가 정보 동등하지 않으며, GPT-4 baseline 이 GPT-3.5 baseline 보다 낮고 (32.1% 대 38.7%), 판정기는관대하게 지시받은 동계열 모델이며, Table 3 에는 MemGPT 를 쓰지 않은 LLM 행이 없다 [MemGPT Table 2, Table 3, § 3.1.1].
- RAG 의 평가설계 결함 여덟 항 — TriviaQA split 의존성 (56.8 대 DPR 57.9), MS-MARCO 비대칭 비교, 저자 제안 Jeopardy 와 452 쌍 인간 평가, Table 4 Specificity 합 93.0%, 산문-표 불일치, null-document 3 변형 무수치폐기, RAG-Seq/Tok 순서 비밀관, 단일 corpus 증거 — 은기제가 아니라 **이득의 크기**를 읽어낼 수 있다는 믿음을 반증한다.
- 서빙 관점에서 커널 안은 아무것도 바뀌지 않는다. 바뀌는 것은 턴당 forward pass 수와 prefix cache 지역성이다. 재귀 요약이 프롬프트 앞쪽에있어 flush 마다 뒤따르는 KV 전부 가 무효화되고, 논문은 KV cache 를 한번도 언급하지 않는다.

자가 점검체크리스트

- MemGPT 의 상속을 원문 인용으로 복원할 수 있고, 그것이 “open question” 이라는 표현 없이 두 구절과 벤치마크 재사용으로 복원된 것임을밝힐 수 있다.
- 식 (13-3) 과 (13-4) 가 왜 성격이 정반대인 두 개의 w_r 인지, 식 (13-6) 의 $S(\cdot)$ 가 왜 (U-E) 의 $S(c; B_s)$ 와 방향이 반대인지말할 수 있다.
- 표 13-2 의 전 행에 대해 “누가 언제 학습하는가” 를 답할 수 있고, 답이전부 같다는 사실이 왜 계보의 출발점 자격을 무효화하지 않는지 네 가지이유로 말할 수 있다.
- MemGPT 가 판별식 네 조건 중 무엇을 만족하지 못하는지 지목하고, 그럼에도 E -경로에 두는 근거 둘을 델 수 있다.
- 비용 4 종이 전부 “논문에 없음” 인 상태에서 단정할 수 있는 비용 명제 (턴당사이클 수 증가) 와 단정할 수 없는 명제 (그 배수) 를 가를 수 있다.
- RAG 평가설계 여덟 항 중 넷을 수치와 함께 인용하고, 비교 조건 문제와 표내적 정합성 문제를 구분할 수 있다.
- Rosetta: “prefix cache $\leftrightarrow \hat{c}$ ” 가 왜 무효화 문제로 나타나고 “KV cache $\leftrightarrow W$ ” 가 왜 아예 걸리지 않는지 ($W = \emptyset$) 각각 한문장으로 설명할 수 있다.

ch14. Sleep-time Compute — 이 라인의 이름

14.1 Bridge-in: 전작이 남긴문제

이 장에는 상속받을 open question 이 없다.

Part II 의 다른 장들은 전작이 명시적으로 남긴 미해결 문제를 인용으로복원하며 시작한다. ch14 는 그렇게 시작할 수 없다. Letta 의 Sleep-time Compute(2504.13171) 는 어떤 선행 연구의 open question 도 인용하지 않고, 요약하지도 않는다. 참고문헌은 22 개이며 [References, pp.13 – 15], Related Work[Letta STC § 2, p.3] 는 미해결 질문이 아니라 **기법 계열**로 조직되어 있다 — test-time scaling, speculative decoding, 그리고 고전적 pre-computation.

계보에 가장 가까운 문장은 하나뿐이고, 그것은 문제의 계보가 아니라**유비의 계보**다.

‘sleep-time compute builds on the idea of pre-fetching in traditional operating systems, in the context of LLMs à la Packer et al. (2023), storing frequently used computational results to avoid higher latency at test-time.’

[Letta STC § 2 ‘Pre-computation’ , p.3]

Packer et al. 2023 은 MemGPT 다 (→ ch13). 같은 저자들과 같은 회사 (Letta) 다. 그런데 이 문장이 MemGPT 에서 가져오는 것은 “운영체제의 pre-fetching 처럼” 이라는 비유이지 MemGPT 가 남긴 문제가 아니다. MemGPT 의 open question 은 인용되지도, 바꿔 쓰이지도 않는다. 이 장이 § 14.8 에서 다시확인하겠지만, E-경로 내부 인용은 이 한 건이 전부다 — RAG 도, vector store 도, graph memory 도, 검색 문헌 어느 것도 참고문헌에 없다.

상속이 없는 자리를 논문은 **공백 제조**로 채운다. 논문은 test-time scaling 문헌 안에 있는 가정 하나를 이름 붙여 꺼낸다.

‘These drawbacks are in part due to the fact that the current approach to applying test-time compute assumes that problems are stateless, i.e. queries (user queries at test-time) and the contexts (background information) required for answering them are provided to the model together at test-time.’

[Letta STC § 1, p.1]

이 문장이 지목하는 논문들 (Snell et al. 2024, OpenAI 2024, DeepSeek-AI 2024, Brown et al. 2024) 은 그 가정을 자기 open question 으로 진술한 적이 없다. stateless 가정은 저자들이 그 문헌을 읽고 **부여한 성격규정**이다. 그러므로 이 장의 bridge-in 은 “전작이 남긴 문제를 받는다” 가 아니라 “전작에게 문제를 만들어 준다” 이다.

[평가] 상속이 비어 있다는 것은 이 장의 결함이 아니라관측이다. 그리고 그 관측은 이 책 전체에서 반복될 형태의 첫 사례다. 한라인의 이름을 만든 논문이 그 라인의 조상을 갖지 않는다. 더 정확히는, 조상을 **다른 분야에서** 골랐다 — 논문이 자기 지적 부모로 세운 것은 캐시 (Smith 1982), data cube (Gray 1997), speculative decoding (Leviathan et al. 2023, Stern et al. 2018, Cai et al. 2024) 이다. 기억 연구도 아니고 학습 연구도 아니며, 제목이 빌려 쓴 생물학은 더더욱 아니다. 이 사실이 § 14.8 의 침묵 지도와 ch11 의 판별식 논의로 그대로 이어진다.

ch13 이 이 장에 넘겨주는 것은 따라서 문제가 아니라 **사실 두가지**다. 첫째, 외부 기억을 운영체제 유비로 다루는 관행이 이미 있었다는것. 둘째, 그 관행의 저자들이 이 논문의 저자들과 같다는 것. 이 장은 그위에서, 상속 없이 시작한다.

14.2 문제의식

논문이 스스로 정의하는 문제는 비용의 크기가 아니라 **비용이청구되는 시점**이다.

논문은 test-time scaling 이 정확도를 사면서 지연과 달러를 낸다는 것을출발점으로 놓되, 그 지불이 불가피한 물리량이 아니라 모델링 가정의산물이라고 주장한다 [Letta STC § 1, p.1]. 가정은 § 14.1 에 인용한 stateless 가정이다. 그 가정 아래에서는 “여러 관련 질의가 문맥에 대해비슷한 추론을 요구하면 모델은 매번 중복 계산을 다시 해야 하고, 그만큼지연과 비용이 추가된다” [Letta STC § 1, p.1].

반대 관측은 응용의 형태에서 온다.

‘In reality, many LLM applications are inherently stateful, and work in conjunction with persisted, re-used context.’ [Letta STC § 1, p.1]

논문이 드는 사례는 세 가지다 — document QA, 공통 저장소 위에서 작동하는 coding agent, 대화 이력을 들고 있는 대화형 assistant. 셋 모두에서 “다음사용자 입력이 오기 전에 이미 확보된 문맥 (문서, 코드베이스, 대화 이력) 이존재한다” [Letta STC § 1, p.1].

여기서 조작적 질문이 나온다. 프롬프트가 문맥과 질의의 쌍으로 분해되고문맥이 질의보다 먼저 도착한다면, **문맥에만 쓴 계산은 test-time 에무엇을 사주는가** [Letta STC § 3, p.3].

이 질문에 답하려면 먼저 두 국면을 분리해야 한다. 이 장이 소유하는 첫번째 정의다.

정의 (sleep/wake 분리의 조작적 형태). 하나의 서빙경로를 두 국면으로 쪼갬다. **sleep** 국면은 질의 q 가 아직도착하지 않은 구간이며, 입력으로 문맥 E_0 만을 가지고 예산 B_s 를쓴다. **wake** 국면은 q 가 도착한 뒤의 구간이며, 예산 B_t 를쓴다. 두 국면은 세 가지가 다르다 — (i) 입력 가용성 (q 의 유무), (ii) 지연 제약 (wake 만 사용자 대기 시간에 묶인다), (iii) 단가 (wake 토큰이 sleep 토큰보다 비싸다고 가정된다). 이 세 차이 중 하나라도 없으면 분리는아무것도 사주지 않는다.

sleep-time compute 자체의 일반 정의 — 어느 층을 갱신하는지 지정하지 않는정의 — 는 ch01 이 소유한다. 이 장이 도입하는 것은 그 정의를 **한논문이 실제로 구현 가능한 형태로 좁힌 결과**다. Letta 의 논문은 이 좁힘을가장 단순한 방식으로 한다: sleep 국면의 산출물을 자연어 문자열 하나로두고, wake 국면에서 그 문자열을 원래 문맥 자리에 끼워 넣는다.

[해설] 독자의 어휘로 옮기면 이렇다. prefix cache 는 “이토큰들의 KV 를 다시 만들지 않겠다” 는 장치다. 같은 계산의 재사용이고, 무손실이며, 방향이 없다 — 어떤 질의가 오든 같은 KV 가 유효하다. sleep 국면의 산출물은 다르다. “이 문맥에서 도출될 결론을 미리 도출해 두겠다” 는장치이고, 새로운 계산의 **선행 수행**이며, 손실적이고 방향이 있다 —어떤 질의를 예상했느냐에 따라 유용성이 달라진다. 이 차이가 § 14.6 의고예산 역전과 § 14.7 의 prefill 증가를 동시에 설명한다. 미리 뽑아 둔결론은 맞을 때는 계산을 대신해 주고, 빗나갈 때는 프롬프트를 길게 만들뿐이다.

문제의식의 마지막 조각은 평가에 있다. 논문은 자기가 정의한 문제를 기준벤치마크로 시험하지 않는다. stateful reasoning 을 재는 기준 벤치마크가논문 안에 하나도 인용되지 않는다. 대신 저자들은 기존 과제를 잘라만들었고, 그 절단이 § 14.6 의 모든 수치의 전제가 된다.

14.3 Core mechanism (통일표기)

14.3.1 기제의 다섯줄

먼저 짚어야 할 형식적 사실이 있다. **논문에는 번호가 붙은 수식이하나도 없다.** 기제 전체가 Letta STC §3의 산문 속 화살표세 개로 진술된다. 따라서 이 절의 모든 출처 표기는 절·부록위치이지 원문 식 번호가 아니다.

논문의 출발 선언은 프롬프트의 분해다. 이 책의 표기로 옮기면 원 문맥이 external store 의 초기 상태가 된다.

$$\text{prompt} = (E_0, q), \quad E_0 := c \quad (14-1)$$

여기서 c 는 논문의 기호이고, 이 책은 c 를 다른 뜻으로 예약해두었으므로 (§14.3.3) 본문에서는 쓰지 않는다. baseline 은 식 (R) 의 특수화다— 원 문맥 전체가 프롬프트에 들어가고 검색 연산은 항등이며 wake 예산이크다.

$$\hat{y} = f_{B_t}(q; \Theta, W, \text{ret}(E_0, q)), \quad \text{ret}(E_0, q) = E_0 \quad (14-2)$$

sleep 국면의 쓰기는 식 (U-E) 의 특수화다. 그리고 이 특수화가 세 겹으로 이루어져 있다는 것이 이 장의 핵심 관측이다.

$$E^{(j)} = \text{wr}(E^{(j-1)}, \hat{c}^{(j)}) = \hat{c}^{(j)}, \quad \hat{c}^{(j)} = S(E^{(j-1)}; B_s/J), \quad j = 1, \dots, J, \quad J \leq 10 \quad (14-3)$$

wake 국면의 읽기는 다시 식 (R) 이되, 저장소가 이제 재표현을 들고 있고 wake 예산이 작다.

$$\hat{y} = f_{B_t^{\text{lo}}}(q; \Theta, W, \text{ret}(E_1, q)), \quad \text{ret}(E_1, q) = E_1 = \hat{c}^{(J)}, \quad B_t^{\text{lo}} \ll B_t \quad (14-4)$$

식 (14-3) 과 (14-4) 의 쌍이 논문의 주장 전체다. 같은 읽기 사상을, 훨씬작은 wake 예산으로, 저장소가 중간 결과를 미리 들고 있기 때문에 쓸 수 있다는 것.

B_s 를 키우는 손잡이는 두 개다. 비추론 모델에서는 병렬 sleep 궤적을 늘린다.

$$\hat{c}_i = S(E_0; B_s/\kappa_{\parallel}), \quad i = 1, \dots, \kappa_{\parallel}, \quad E_1 = \text{concat}(\hat{c}_1, \dots, \hat{c}_{\kappa_{\parallel}}), \quad \kappa_{\parallel} \in \{1, 2, 5, 10\} \quad (14-5)$$

추론 모델 (o1, o3-mini) 에서는 API 의 ‘reasoning effort’ $\in \{\text{low}, \text{medium}, \text{high}\}$ 를 돌린다 [Letta STC §5.2, p.8; Fig. 8 범례, p.10]. 즉이 경우 B_s 는 토큰 수가 아니라 **벤더 쪽 서수**다.

마지막으로, 세 층 중 둘은 움직이지 않는다. 논문이 식으로 쓰지는 않지만설정이 강제한다 — 모든 대상 모델이 폐쇄 API 이고 논문 어디에도 training, fine-tuning, adapter, optimizer 가 없다 [Letta STC §4.2, p.5].

$$\Theta_{k+1} = \Theta_k \quad \forall k, \quad W \text{ 는 이 아키텍처에 존재하지 않는다} \quad (14-6)$$

14.3.2 정규형과 어긋나는 지점다섯

식 (U-E) 의 정규형은 wr 과 $S(c_k; B_s)$ 의 내부를 지정하지않는다. 이 논문은 그 빈칸을 다음과 같이 채우고, 채우는 방식마다 결과가달라진다.

첫째, wr 은 파괴적 덮어쓰기다. 정규형의 wr 은 append 든 merge 든 index-insert 든 허용한다. 여기서는참조 구현이 블록 값을 그대로 세팅한다 —agent_state.memory.update_block_value(label=target_block_label, value=new_memory)[Letta STC App. F Listing 1, p.22]. 부록 K 의 서술도 같다: 함수가 “현재 문맥을새 문자열로 교체한다” [Letta STC App. K, p.27]. 결과로 저장소에이력이 없고 rollback 이 없다. sleep 국면 안에서 잘못된 재작성은 복구불가능하다.

둘째, $S(\cdot; B_s)$ 는 단발이 아니라 반복이다. 깊이 $J \leq 10$ 이며, 각 단계는 현재 블록과 지정된 원본 블록 하나를 읽고, 모델이 `finish_rethinking` 을 호출하면 끝난다 [Letta STC App. K, p.27; Fig. 17, p.18]. 정규형은 B_s 를 한덩어리 예산으로 쓰지만 여기서는 **순차 사슬**로 소비된다. 이것이 § 14.7 의 `sleep wall-clock` 논의를 만든다.

셋째, 서로 다른 두 개의 `wr` 이 이름 없이 공존한다. 궤적 안에서는 덮어쓰기 (식 14-3), 궤적 사이에서는 이어붙이기 (식 14-5) 다. 논문은 두 연산자를 쓰고 있다는 사실 자체를 언급하지 않는다. 이어붙이기쪽에서는 상태 크기가 $\kappa_{||}$ 에 선형으로 자라고, § 14.6 의비단조성 ($\kappa_{||} = 5$ 가 $\kappa_{||} = 10$ 을 이긴다) 이 그 성장의 첫 번째 가시적 대가다.

넷째, `ret` 이 항등이다. 정규형의 $\text{ret}(E, q)$ 는 선택을 함의한다. 여기서는 저장소 전체가 원 문맥 자리에 통째로 끼워진다 [Letta STC § 3, p.4]. 인덱스도, 임베딩도, 유사도 점수도, `top-m` 절단도 없다. 튜닝할 검색기가 없고 검색 실패모드도 없다 — 그리고 그 말은 **E-경로가 통상 치르는 검색 비용이 이논문에서 하나도 행사되지 않았**다는 뜻이기도 하다 (→ ch10, ch15).

다섯째, E 는 문맥 범위이고 휘발성이다. 문맥 하나당 저장소하나이며, 지속되지 않고, 문맥 사이에서 병합되지 않고, 재통합되지 않는다. 논문 자신이 이것을 작업의 경계로 명시한다 [Letta STC § 7, p.13]. 그리고 `sleep` 라운드 첨자 k 는 모든 실험에서 1 을 넘지 않는다.

기계에 없는 것도 같은 무게로 기록해야 한다. 축출 · 만료 · 압축 · 용량 상한이없다 ($J \leq 10$ 이라는 재작성 횟수 제한이 유일한 구조적 경계다). 세션을넘는 상태가 없다. 그리고 $\hat{c}$ 의 **검증이 없다** — 블록에 적힌추론이 옳은지 확인하는 절차가 어디에도 없다. 유일한 시늉은 레벨 4 verbosity 프롬프트가 모델에게 “블록의 숫자를 쓸 거면 먼저 다시 계산해서확인하라” 고 지시하는 대목인데 [Letta STC Fig. 16, p.17], 그것은 검증비용을 `wake` 국면으로 되돌린다.

14.3.3 표기 대응표

표 14-1 — Letta Sleep-time Compute 의 원 논문 기호와 이 책의표기. 원문에 번호 붙은 수식이 없으므로 ‘원문 위치’ 는절 · 부록 · 그림 위치다.

원 논문 기호	논문에서의 의미	이 책의 예약 충돌	이 책의 표기
c [§ 3, p.3]	기존 문맥 (코드베이스, 문서, 수학 문제의 앞부분)	소문자 c 는 Neural Memory 에서 Omega rule window 길이로 예약되어있고 (→ NM), 대문자 chunk 크기 C 와도 구분해야 한다	E_0 — 원문맥은 external store 의 초기 상태다
c' [§ 3, p.4; Fig. 1 라벨 ‘Learned Context’]	<code>sleep</code> 국면이 만든재표현 문맥	없음. 다만 ‘Learned’ 가 오도한다 — 이 논문에 gradient 가없다	$\hat{c}$, 그리고 그 결과 상태로서 E_1
$S(\cdot)$ [§ 3, p.4]	<code>sleep</code> 절차, $S(c) \rightarrow c'$	S_t = inner momentum(surprise) 버퍼 (ch01 예약)	$S(\cdot; B_s)$. 판별 규칙: momentum 은 항상 시간 첨자 S_t , <code>sleep</code> 연산자는항상 예산 인자 ($\cdot; B_s$). 첨자 없는 맨 S 는 쓰지 않는다
$T_B(q, c) \rightarrow a$, $T_b(q, c') \rightarrow a$ [§ 3, pp.3–4] B, b ($b \ll B$) [§ 3, p.4]	예산 B 또는 b 로 적용한 test-time 방법 큰/작은 test-time 예산	T 는 sequence 길이 의미로 이미 L 에 흡수됨 맨 b 사용 금지 (mini-batch 크기 의미와 충돌)	읽기 사상 $f(\cdot)$ 에 예산 첨자: $f_{B_t}, f_{B_t^{\text{lo}}}$ B_t, B_t^{lo}

원 논문 기호	논문에서의 의미	이 책의 예약 충돌	이 책의 표기
$t = 10$ [§ 5.3, p.8]	test-time 생성 토큰 이 sleep-time 생성 토큰보다 비싼 배수	$t =$ 토큰 인덱스 (ch01 예약)	κ_{cost} (장-국소)
k [§ 5.2, p.8 / § 5.1, p.7]	두 가지로 쓰인다 — (i) 병렬 sleep 생성 개수, (ii) pass@ k 의 k	$k =$ sleep 라운드첨자 (예약), $k_t =$ key 벡터	(i) $\kappa_{\parallel}$ (장-국소), (ii) pass@ κ
$N, q_1 \dots q_N$ [§ 3, p.3]	한 문맥을 공유하는 질의 수	없음	N_q
$p = (c, q)$ [§ 3, p.3]	전체 사용자 프롬프트	$p = \ell_p$ 노름 차수	기호를 새로 만들지 않고 쌍 (q, E) 를 그대로 쓴다
q [§ 3, p.3]	사용자 질의, 자연어 문자열	$q_t =$ query 벡터 (항상 첨자)	q 유지. E-경로에서 q 는 투영된 벡터가 아니라 토큰 문자열이다
a [§ 3, p.3]	추론 흔적 뒤에 나오는 최종 답	없음	$\hat{y}$

장-국소 기호 두 개를 추가로 선언한다. $J = \text{sleep}$ 국면의 재작성깊이 (상한 10). κ_{cost} 와 $\kappa_{\parallel}$ 은 이장에서만 쓰며, Neural Memory 가 Newton – Schulz 반복 횟수로 예약한 맨 $\kappa(\rightarrow \text{NM})$ 와는 첨자로 구분된다.

14.3.4 이 장이 소유하는 두정의

정의 (learned context $\hat{c}$). learned context 는 질의 q 가 도착하기 전에 문맥 E_0 만을 입력으로 예산 B_s 를 써서수행한 추론의 산출물이며, wake 국면에서 E_0 을 대체하여 프롬프트에 놓이는 자연어 문자열이다. 이름의 ‘learned’ 는 gradient 를 뜻하지 않는다 — 이 논문에는 backward pass 가 없다 (식 14-6). 학습되는 것은파라미터가 아니라 **표현이 놓이는 공간**이고, 그 공간은 자연어다.

이 정정은 이 책의 재구성이 아니라 논문 본문의 진술과 일치한다. 논문은 [Letta STC § 7] 에서 스스로 이렇게 쓴다.

‘Unlike traditional representation learning (Bengio et al., 2014), which typically operates in model parameter or activation space, we instead form representations in the space of natural language.’ [Letta STC § 7, p.13]

이 한 문장이 이 책의 3 경로 분할에 대한 가장 좋은 1 차 근거다. parameter space 가 Θ -경로, activation space 가 W -경로, natural-language space 가 E -경로다 — 그리고 그 삼분을 E-경로 저자가 직접 썼다 ($\rightarrow$ ch11). ‘Learned Context’ 라는 오도하는 라벨은 Figure 1(p.2) 의 그림 안에만 있고, 논문 본문은 정확하다.

정의 (상각 논증의 원형). 상각 논증은 다음 형태의 논증이다 —sleep 국면의 비용은 문맥당 한 번 발생하고 wake 국면의 비용은 질의당발생하므로, 한 문맥을 공유하는 질의 수 N_q 가 커질수록 질의당 평균비용이 식 (A) 를 따라 떨어진다. 이 논증이 숫자로 성립하려면 세 값이필요하다: 분자 B_s , wake 비용, 그리고 N_q .

논문이 쓰는 형태는 식 (A) 에 wake/sleep 단가비를 넣은것이다.

$$\text{Cost}_{\text{avg}} = \kappa_{\text{cost}} \cdot B_t^{\text{lo}} + \frac{B_s}{N_q}, \quad \kappa_{\text{cost}} = 10 \quad (14-7)$$

원문은 이렇게 쓴다 (논문의 배수 기호는 표 14-1 에 따라 κ_{cost} 로 옮긴다).

‘we consider a simple linear model where tokens generated at test-time are a factor t the cost of the tokens at sleep-time. In our analysis, we set $t = 10$ ’
[Letta STC § 5.3, p.8]

이 식이 이 장의 나머지 절반을 지배한다. § 14.5 는 위 세 값 중 무엇이 논문에 있고 무엇이 없는지를 따진다.

14.4 어느 층을 언제 쓰는가

답부터 쓴다. 이 논문은 (U-E) 만 실행한다. 그리고 정확히 한 번 실행한다.

표 14-2 — 이 논문에서 움직이는 값과 움직이지 않는 값. “이 값은 누가 정하는가” 에 전부 답한다.

대상	누가 정하는가	언제	이 논문에서 움직이는가	출처
Θ — GPT-4o-mini, GPT-4o, o1, o3-mini, Claude Sonnet 3.7 Extended Thinking, DeepSeek-R1 의 backbone	모델 벤더가, 이 논문이 존재하기 전에	outer-loop 사전 · 사후학습, 전부 논문 밖	아니오. 동결	§ 4.2, p.5
W — fast weights	아무도	해당 없음	아니오. 이 층이 사용된 아키텍처에 존재하지 않는다	§ 7, p.13(쓰지 않는다고 명시)
E — rethink memory 블록, 단일 가변 텍스트 블록	동결된 모델 자신이, 프롬프팅으로	sleep 국면. 최대 10 회 순차 rethink_memory 호출, finish_rethinking 으로 종료	예. 유일한 공동부	App. K, p.27; App. F Listings 1–2, p.22; Fig. 17, p.18
sleep 시스템 프롬프트 (Letta-Offline-Memory 페르소나, rethink/finish 프로토콜)	저자가 손으로	고정	아니오	Fig. 17, p.18; AIME 전용 부기 Fig. 18, p.18
wake verbosity 프롬프트 5 종 (레벨 0–4)	저자가 손으로	실험별 선택, 고정	아니오	§ 5.1, p.6; App. A Figs. 12–16, pp.16–17
$\kappa_{ } \in \{1, 2, 5, 10\}$	아무도 학습하지 않음. 손 스위치	실험 구성	스왑될 뿐 학습되지 않음	§ 5.2, p.8; Fig. 7, p.10
sleep ‘reasoning effort’ $\in \{\text{low, medium, high}\}$	손 스위치. effort → 토큰 매핑은 벤더 내부이며 비공개	실험 구성	스왑될 뿐 학습되지 않음	§ 5.2, p.8; Fig. 8, p.10

대상	누가 정하는가	언제	이 논문에서 움직이는가	출처
B_t — wake 예산	손 스위. 모델별로 세 가지 다른 방식 (verbosity 프롬프트 / API reasoning-effort / budget-forcing)	실험 구성	스윙될 뿐 학습되지 않음	§ 5.1, p.6
$J \leq 10$ — 재작성 호출 상한	아무도. 하드코딩된 상수	고정	아니오	App. K, p.27
$\kappa_{\text{cost}} = 10$ — wake:sleep 단가비	아무도. 외부 벤더 문서에서 단언, 측정 아님	고정된 모델링 가정	아니오	§ 5.3, p.8 + 각주 4
Multi-Query 문항 생성기 (o3-mini + Fig. 19 프롬프트)	아무도. 동결된 벤더 모델	오프라인 데이터 셋 구성 — 방법의 일부가 아님	아니오	§ 4.1, p.4; App. C Fig. 19, p.19
SWE-Features PR 군집기 (claude-sonnet-3-7-20250219) — 무엇이 '관련문맥' 인지를 정한다	아무도. 동결된 벤더 모델	오프라인 벤치마크 구성	아니오	App. D, p.20
질의 예측가능성 스코어러 (Llama2-70B base, $\log P(q E_0)$)	아무도. 동결된 제 3 자 base 모델	사후 분석 도구	아니오	§ 5.4, p.9

표 14-2 에서 읽어야 할 것은 outer-loop 열이 **비어 있다**는 사실이다. 이 논문에서 학습되는 것은 없다. 학습 가능해 보이는 모든 대상은 (a) 동결된 벤더 모델이거나, (b) 손으로 쓴 프롬프트이거나, (c) 손으로 스위칭한 하이퍼파라미터다. 움직이는 것은 텍스트 블록 하나뿐이고, 그것이 sleep 국면에 움직인다.

[해설] 그래서 이 장이 Part II 에서 E-경로의 정면 사례로 먼저오는 것이 옳다. Part I 의 T1 – T5 가 가르치는 모든 장치 — fine-tuning 과 PEFT(→ ch03), distillation(→ ch04), RL(→ ch05), 합성 데이터 (→ ch06), replay 와 망각 대응 (→ ch07) — 이 여기서는 **전부 부재한다**. E-경로는 backward pass 한 번 없이 끝까지 시연된다. 뒤의 W -경로 $\cdot \Theta$ -경로 장들은 같은 발상을 gradient 값으로 사게 되고, 그 가격 차이가 Part III 의 판정 재료다. W 층이 실제로 어떻게 움직이는지는 이 책이 다시 설명하지 않는다 — 그 층의 갱신식은 이 책의 (U-W) 와 글자 그대로 같으며, 전개는 Neural Memory 모노 그래프가 갖는다 (→ NM 식 (M)).

시간척도는 단일하다. 갱신 첨자 k 가 1 을 넘지 않는다. sleep 은 문맥당한 번 일어나고, 그 뒤 wake 읽기가 N_q 번 일어나며, 그 사이에 문맥이 갱신되어 다시 sleep 이 도는 일은 논문의 어떤 실험에도 없다. 순서로 쓰면 $E_0 \rightarrow E_1(\text{sleep}) \rightarrow$ 읽기 (wake) 뿐이며, 층 간 전이도 cascade 도교차도 없다.

이 단일성이 § 14.5의 마지막 칸을 미리 결정한다. 반복 열화 ρ 는 이 논문에서 미측정이 아니라 구성상 범위 밖이다. 라운드가 하나뿐이면 라운드당 손실은 정의되지 않는다. 논문 자신이 이 경계를 명시한다.

‘we make the simplifying assumption that interactions fall into two phases: sleep-time and test-time. However, real-world LLM use cases can be more complex, with multiple rounds of interaction and context modifications between rounds (e.g. multiple edits to a code-base).’ [Letta STC § 7, p.13]

독자의 1번 질문에 대한 답도 이 절에서 이미 나온다. **decode에서 바뀌는 것은 없다.** 커널도, 가중치 배치도, batching 형태도 그대로다. 프롬프트의 내용만 다르다. 그것이 E-경로가 오늘 당장 배포되는 이유이자, E-경로가 Θ -경로처럼 용량을 올릴 수 없는 이유다. § 14.7이 이 답을 서빙 스택의 각 층으로 전개한다.

14.5 비용 4종

표 14-3 — Letta Sleep-time Compute의 비용 4종. 네 칸이 전부 “논문에 없음”이다. 추정치를 넣지 않는다.

값	논문의 보고	대신 보고된 것	출처
B_s — sleep 계산 예산	논문에 없음	서수 손잡이만: $J \leq 10$ 재작성 호출, $\kappa_{ } \in \{1, 2, 5, 10\}$, sleep reasoning effort $\in \{\text{low}, \text{medium}, \text{high}\}$	App. K, p.27; § 5.2, p.8; Figs. 7–8, p.10
L_w — wake 지연 기여	논문에 없음	질문당 생성된 test-time 토큰 수 (지연의 대리값). Stateful GSM-Symbolic 약 0 – 600, Stateful AIME 약 1000 – 22500, SWE-Features 3000 – 10000	Fig. 3, p.6; Fig. 4, p.7; Fig. 11, p.12
C_{cap} — 상태 용량	논문에 없음	구조적 경계만: 재작성 $J \leq 10$ 회, Letta core-memory 블록 배치 (persona· human 읽기 전용, rethink 읽기/쓰기)	App. K, p.27; Fig. 17, p.18
ρ — 망각·열화율	논문에 없음	sleep 라운드가 모든 실험에서 정확히 1 회. 반복 통합이 구성상 범 위 밖	§ 7, p.13

값	논문의 보고	대신 보고된 것	출처
$(N_q - \text{참고})$	스윙값만	$N_q \in \{1, 2, 5, 10\}$, 합성 데이터셋위에서 구성. 실제 워크로드의 N_q 분포는 없음	§ 5.3, pp.8 – 9; Fig. 9, p.11

14.5.1 B_s 가 없다는것

표 14-3의 첫 줄이 이 장에서 가장 무거운 사실이다. **상각 논증의분자가 숫자로 존재하지 않는다.**

논문은 sleep-time 토큰 수도, FLOP 수도, 달러 액수도 보고하지 않는다 — 본문에도, 어떤 축에도, 어떤 표에도 없다. 모든 그림의 x 축은 'Avg. Test Time Tokens / Question'이며, sleep 비용은 자기 축을 한번도 갖지 못한다. 유일하게 sleep 비용이 등장하는 곳은 Figure 9(p.11)의 x 축 'Total Inference Cost / Query'인데, 이 축은 식 (14-7)로 만들어진 **파생 합성량**이므로 B_s 가 그 안에 $\kappa_{\text{cost}} B_t$ 와 엉켜 있고 분리해 읽을 수 없다.

보고된 것은 서수뿐이다. 재작성 호출 상한 10, 병렬 궤적 개수 네 가지, 그리고 벤더가 정의를 공개하지 않은 reasoning effort 세 단계. 상한 10은 실제로 몇 번 호출되었는지를 말해 주지 않고, 논문은 실현된 평균 호출 수를보고하지 않는다.

14.5.2 그래서 N_q^* 를 계산할 수 없다

식 (14-7)에서 손익분기를 유도해 보면 무엇이 빠졌는지가 정확히 드러난다. 비교는 **같은 정확도에서** 이루어져야 한다 — 그것이 논문의 pareto 서술이 하는 일이다. sleep을 쓰는 팔이 wake 예산 B_t^{lo} 로 baseline의 B_t 와 같은 정확도에 도달한다고 두면, sleep을 쓰는 쪽이 총비용에서 이기는 조건은

$$\kappa_{\text{cost}} B_t^{\text{lo}} + \frac{B_s}{N_q} < \kappa_{\text{cost}} B_t \iff N_q > N_q^* = \frac{B_s}{\kappa_{\text{cost}} (B_t - B_t^{\text{lo}})} \quad (14-8)$$

이다. 우변에 필요한 값은 셋이다. κ_{cost} 는 논문이준다 — 다만 측정이 아니라 벤더 문서 인용이다 (§ 14.5.4). B_s 는 없다 (§ 14.5.1). B_t 와 B_t^{lo} 는 생성 토큰 축의눈금으로만 존재하며, 논문에 결과 표가 없으므로 주석 없는 선 그래프에서읽어야 한다.

따라서 N_q^* 는 **이 논문의 1차 자료만으로 계산되지 않는다**. 논문이 주는 것은 $N_q \in \{1, 2, 5, 10\}$ 네 점에서의 곡선과, 손익분기가그 구간 어딘가에 있다는 정성적 진술이다. 그 진술은 Figure 9의 캡션에있다 — “문맥당 질문 수가 적으면 총비용 관점에서 sleep-time compute이 덜유리하다” [Letta STC Fig. 9 캡션, p.11]. 즉 논문 스스로 손익분기의존재를 인정하되 그 위치를 숫자로 주지 않는다.

여기에 더 근본적인 공백이 겹친다. **실제 서버에서 N_q 의 분포를보고한 논문이 이 책이 다루는 corpus 전체에 없다**. 상각 논증은 N_q 를안다고 가정하는데, 그 값은 워크로드의 성질이 방법의 성질이 아니다. 이논문은 N_q 를 합성으로 구성해 네 값에서 재고, 그 네 값이 실제트래픽에서 어떤 빈도로 나타나는지는 다루지 않는다 (→ ch25).

14.5.3 비용모델이 생성 토큰만센다

식 (14-7)의 두 항 모두 **생성된 토큰만** 센다. § 14.3.4에 인용한원문이 그렇게 말한다 — ‘tokens generated at test-time’ [Letta STC § 5.3, p.8]. prefill 항, 즉 입력 토큰 항이 어느 쪽에도 없다.

그런데 이 방법의 기제 자체가 **입력을 길게 만드는 것이다**. $\hat{c}$ 는 E_0 보다 길다 (Figure 1, p.2의 두 문맥 상자를 나란히 보면확인된다). 그리고 병렬 스케일링에서는 $\kappa_{\parallel}$ 개의재표현이 이어붙여 프롬프트에 실리며, 그 값은 최대 10이고, 그 프롬프트는**매 질의마다** 다시 처리된다 [Letta STC § 5.2, p.8]. 이 입력비용은 $5\times$ 에도, 13% 에도, 18% 에도, $2.5\times$ 에도 나타나지 않는다.

14.5.4 $\kappa_{\text{cost}} = 10$ 은단언이다

상수 κ_{cost} 는 곱셈으로 결론에 들어간다. 식 (14-8) 에서 N_q^* 는 κ_{cost} 에 반비례한다 — κ_{cost} 를 1 로 두면 손익분기 질의 수가 10 배로 된다. 그만큼 무거운 상수인데, 논문이 제시하는 근거는 측정이 아니라 인용이다.

‘Since at test-time, there are strict latency constraints, and latency optimized inference can be roughly $10\times$ more expensive, we model the total cost of inference between both sleep-time and test-time, by up-weighting the cost of test-time tokens.’ [Letta STC § 5.3, p.8]

이 문장의 각주 4 는 Databricks provisioned-throughput 문서 페이지를 가리킨다. 논문 안에서 이 비율을 재는 실험은 없고, κ_{cost} 를 흔들었을 때 $2.5\times$ 가 어떻게 변하는지에 대한 민감도 분석도 없다.

14.5.5 네 칸의 공백이 뜻하는것

[평가] 비용 4 종이 전부 absent 인 것은 이 논문 한 편의 부주의가 아니다. 이 라인에 이름을 준 논문이 비용을 보고하지 않았고, 같은경로의 후속 연구들이 그 관행을 그대로 물려받았다 (→ ch15, ch26). 그 결과 “sleep-time compute 이 싸다” 는 명제는 **현재 문헌 상태에서 검증도 반증도 되지 않는다** — 비교할 수 있는 회계 단위가 없기 때문이다. 이 책은 비용이 침묵한 자리를 성능 주장으로 메우지 않는다 (→ ch01). 그래서 표 14-3 의 네 칸은 “효율적” 이라는 서술 대신 “논문에 없음” 으로 남는다.

이 공백을 이 책은 실험 X1 로 다룬다. X1 은 논문들이 각자의 측으로 흩어놓은 값을 하나의 회계 단위로 옮기고, prefill 항을 포함했을 때 상각논증이 어느 레짐에서 살아남고 어느 레짐에서 뒤집히는지를 결정론적으로 계산한다. **판정은 이 장에서 하지 않는다**. 통일 비용 회계의 판정은 ch25 가 소유한다. 이 장이 하는 일은 판정에 필요한 세 값 중 두 값이 1 차자료에 없다는 사실을 확정하는 데까지이다.

14.6 실험과 스케일

14.6.1 읽기 전에 알아야 할 형식

논문에 결과 표가 없다. 표는 정확히 하나 있고 (Table 1, p.19), 그것은 데이터셋 통계다. 모든 정확도·F1·비용수치는 주석 없는 선 그래프로 제시된다. 오차 막대도, 신뢰구간도, 분산통계도 어디에도 없다. 그러므로 아래에 쓰는 수치는 **논문이 산문으로 진술한 값과 그림 축의 범위로 한정된다**. 곡선 위의 점을 눈으로 읽어 옮기지 않는다.

실행 횟수는 arm 마다 다르다. o1·o3-mini·R1 은 3 회 평균, Claude 3.7 Sonnet 은 10 회 평균이며, 그 이유를 논문이 밝힌다 — “초기 실험에서 잡음이 더 많이 관측되어서” [Letta STC § 5.1, p.6]. 즉표본 수가 결과를 본 뒤에 arm 별로 달라졌고, 분산은 보고되지 않아 독자가 그 조정의 타당성을 판단할 재료가 없다.

14.6.2 벤치마크는 저자가 만들었다

정의 (Stateful 벤치마크 구성). Stateful 벤치마크 구성이란, 기존의 단일 프롬프트 과제를 문장 경계에서 **문맥 절반과 질의 절반으로 잘라** 만든 변형 벤치마크를 말한다. 자르는 원 과제의 난이도를 보존하되, “문맥이 질의보다 먼저 도착한다” 는 조건을 인공적으로 성립시킨다. 이 구성은 sleep/wake 분리를 측정 가능하게 만드는 동시에, 그 분리가 실제 워크로드에서 성립하는지를 **평가밖으로 밀어낸다**.

표 14-4 — Letta Sleep-time Compute 의 네 벤치마크. 전부 저자제작이다.

벤치마크	크기	구성 방식	출처
Stateful GSM-Symbolic P1	5000 examples	GSM-Symbolic P1(GSM8K 에 절 1 개 를 더한 변형) 을 의문 문 앞에서 절단	§ 4.1, p.4; Fig. 2, p.5
Stateful GSM-Symbolic P2	2500 examples	GSM-Symbolic P2(절 2 개 추가), 같은 절단	§ 4.1, p.4
Stateful AIME	60 questions	AIME 2024 + AIME 2025 합본. 구두점으 로분리한 statement 중 마지막 하나를 질의 로, 나머지를 문맥으로	§ 4.1, p.4; App. J, pp.27 – 28
Multi-Query GSM-Symbolic P1	12043 questions / 1095 contexts / 1095 original / 10948 generated	템플릿당 1 개 인스턴 스 표집 후, o3-mini 가 “같은 난이도, 같은 추론 단계수” 의 Q/A 쌍을 추가 생성	Table 1, p.19; § 4.1, p.4; App. C Fig. 19, p.19
Multi-Query GSM-Symbolic P2	5497 questions / 500 contexts / 500 original / 4997 generated	같은 방식	Table 1, p.19
SWE-Features	33 examples (Aider-AI/aider 18 + comfyanonymous/ ComfyUI 15)	.py/.js 파일 3 개 이 상을 수정한 PR 수집. gpt-4o-mini 가 제목 · 본문을심사하고, claude-sonnet-3-7- 20250219 가 PR 을 군 집화해 형제 PR 을 문 맥으로삼는다	§ 6, p.12; App. D, pp.19 – 20

표 14-4 에 이 장의 첫 번째 의무 caveat 이 이미 들어 있다. $5\times\cdot 13\%\cdot 18\%\cdot 2.5\times$ 는 **전부이 표의 벤치마크에서 나온 값이다**. 논문 안에 기존의 stateful reasoning 벤치마크는 하나도 없다. 그 숫자들 뒤에 독립적인 평가 기준이 없다는 뜻이다.

Multi-Query 는 기반 데이터셋의 약 5 분의 1 만 쓴다 — P1 의 5000 예제 중 1095 개 문맥, P2 의 2500 개 중 500 개 문맥이며, 이유는 “템플릿당 한인스턴스를 표집한다” 이다 [Letta STC Table 1, p.19; § 4.1, p.4].

14.6.3 논문이 보고하는수치

$5\times$ (test-time compute 감소). 초록의 표현은 “같은 정확도에도달하는 데 필요한 test-time compute 을 Stateful GSM-Symbolic 과 Stateful AIME 에서 약 $5\times$ 줄인다” 이다 [Letta STC Abstract, p.1]. 본문은 이배율에 레짐 제한을 붙인다 — “낮은 test-time 예산에서, sleep-time compute 의 성능이 baseline 보다 유의하게 좋으며, baseline 이 5 배 많은 test-time 토큰으로 내는 성능과 비슷한 성능에 도달한다” [Letta STC § 5.1, p.6]. 즉 $5\times$ 는 **낮은 예산 구간의 수평 이동**이다.

13% (Stateful GSM-Symbolic). “pareto 곡선을 바깥으로 밀어비슷한 test-time 예산에서 성능을 최대 13%까지 개선한다” [Letta STC § 5.2, p.8; Fig. 7, p.10]. 이 값은 $\kappa_{||}$ 스왑에서 나온다. 저자는 “더 어려운 과제와 더 강한 모델에서 이득이 가장 크다 (예: P2 + gpt-4o)” 고 덧붙인다.

18% (Stateful AIME). “sleep-time 에 계산을 키우면 대체로 pareto 곡선이 바깥으로 이동하며 성능이 최대 18%까지 개선된다” [Letta STC § 5.2, p.8]. 다만 내부 참조가 어긋난다 — [Letta STC § 5.2] 는 이결과를 ‘Figure 26’ 으로 지목하는데, 그 그림의 캡션은 ‘Scaling sleep-time compute on Stateful AIME2025’ (App. M, p.31) 로 연도별부분집합이다. 60 문항 합본의 스케일링 그림은 Figure 8(p.10) 이다.

2.5× (질의당 평균 비용 감소). “문맥당 질문이 10 개일 때 단일질의 baseline 대비 질의당 평균 비용을 최대 2.5× 줄일 수 있다” [Letta STC § 5.3, pp.8–9; Fig. 9, p.11]. 조건은 Multi-Query GSM-Symbolic, $N_q = 10$, $\kappa_{\text{cost}} = 10$, 그리고 생성 토큰만 세는 식 (14-7) 이다.

pass@ κ 대비 우위. “모든 과제와 모든 모델에서, 같은 test-time 토큰 예산일 때 sleep-time compute 이 pass@ k 병렬 스케일링을 일관되게 능가한다” [Letta STC § 5.1, p.7; Figs. 5–6, pp.8–9]. 저자는 자기 baseline 의 강점을 명시한다 — pass@ k 는 “test-time 에 정답검증기에 oracle 로 접근한다는 비현실적 가정” 을 하며, 그 가정을 하지 않는방법이 그것을 이긴다면 유의미한 개선이라는 것이다.

문맥만 주는 통제. 부록 I 는 질의를 지운 채 문맥만 준 baseline(‘ablate question’) 을 세워, 데이터셋의 질문이 문맥에서 자명하게추측되지 않음을 보인다 [Letta STC App. I, p.26; Figs. 21–22, pp.26–27].

[해설] 위 두 항목은 이 논문이 잘한 일이다. oracle 우위를가진 baseline 을 세우고 그 우위를 본문에 밝힌 것, 그리고 “질문이 문맥에서그냥 보이는 것 아니냐” 는 가장 명백한 반론에 대한 통제를 부록에 실은 것. 이 책은 두 설계를 신뢰할 만한 것으로 본다. 아래의 caveat 목록은 이 두항목을 지우지 않는다.

SWE-Features. “낮은 test-time 예산에서 sleep-time compute 이성능을 개선하며 test-time 토큰을 최대 약 1.5× 줄인다. 다만 test-time 예산이 높으면 test-time compute 만 쓰는 쪽이 더 나을 수 있다” [Letta STC § 6, p.12]. F1 측은 약 0.30–0.55, test-time 토큰 측은 3000–10000 이다 [Fig. 11, p.12].

질의 예측가능성. “질문이 문맥으로부터 더 예측 가능해질수록 sleep-time compute 과 표준 test-time compute 의 정확도 격차가벌어진다” [Letta STC § 5.4, pp.9–10]. 프로토콜은 Llama2-70B base 아래의 $\log P(q | E_0)$ 를 5 분위로 나누고, GPT-4o-mini 에 verbosity 0(가장 낮은 wake 예산) 을 적용한 것이다. 정확도 차 측은 P1 이 0.0–0.5, P2 가 0.00–0.40 이다 [Fig. 10, p.11].

14.6.4 논문 자신이 보고하는 불리한결과

이득이 높은 test-time 예산에서 역전한다. 두 과제 모두에서. Stateful GSM-Symbolic 에서는 이렇다.

‘However, at the test-tome [원문 오타] compute budgets, the test-time compute only baseline slightly outperforms sleep-time compute. We hypothesize that this may be because the standard test-time compute only has the content relevant to the specific question, so there is less distracting information in the prompt.’ [Letta STC § 5.1, p.6]

SWE-Features에서도 같은 역전이 나오고, 저자는 정밀도 쪽에서 그것을관측한다 — “높은 test-time 예산 설정에서는 표준 test-time compute 이정밀도가 더 높고 재현율은 비슷하다” [Letta STC § 6, p.12]. 두 경우모두 가설이 제시되고 시험되지 않는다. 그리고 초록은 이 레짐제한을 달지 않는다.

sleep 스케일링이 비단조다. “성능을 개선하는 병렬 에이전트수에는 한계가 있는 듯하며, 병렬 생성 5 개가 대체로 10 개보다 낮다” [Letta STC § 5.2, p.8]. 13%라는 헤드라인이 바로 이 스왑에서 나온다. 최적점이구간 내부에 있고, 그것을 찾는 규칙도 임계도 제시되지 않는다.

가장 강한 추론 모델이 거의 이득을 보지 못한다. “모든 모델에서유의한 test-time 정확도 pareto 이동을 관측했으나, o1 은예외로 이득이 제한적이다” [Letta STC § 5.1, p.6; Fig. 4, p.7]. 설명은 제시되지 않는다.

질의 수가 적으면 순손실이다. “문맥당 질문 수가 적으면 총비용관점에서 sleep-time compute 이 덜 유리하다” [Letta STC Fig. 9 캡션, p.11].

14.6.5 이 책이 확인한 불리한사실

Stateful AIME 의 구성에 수동 개입이 있고, 문맥이 비어 있는 인스턴스가 포함된다.

“There are a couple of edge cases where the question is posed in e.g. the second to last statement rather than the last statement. In these cases, we manually rearrange the statements to ensure the query being used corresponds to the question. In a few cases, there is only one statement in the problem. In these cases, the context is empty.” [Letta STC App. J, pp.27 – 28]

LaTeX 그림도 수동으로 제거되었고, 제거 후에도 문제가 풀리는지를 수동으로 확인했다고 같은 부록이 밝힌다. 여기서 무거운 것은 빈 문맥이다. 문맥이 비어 있으면 sleep 국면이 읽을 것이 없으므로 sleep-time compute 은 구성상 아무것도 할 수 없다. 그런 인스턴스가 몇 개인지 논문은 밝히지 않는다. 분모는 60 이다 — 한 문항이 정확도 1.67 점이다.

Multi-Query 데이터셋의 90.9%가 모델 생성 문항이다. P1 은 12043 개 중 10948 개, P2 는 5497 개 중 4997 개이며 두 비율 모두 90.9%다 [Letta STC Table 1, p.19]. 생성된 답의 정확성을 검증했다는 보고는 논문 어디에도 없다. 그리고 논문 자신의 예시 (Figure 20, p.20) 를 보면 생성 문항 중 여럿이 문맥에서 숫자 하나를 옮겨 적으면 답이 되는 단일조회형이다 (‘How many stacking rings are on the tower?’). 상각 논증의 분모 N_q 가 바로 그 문항들로 채워져 있다.

모델 선택 기준이 측정 대상과 상관된다. “각 데이터셋에서 우리는 test-time compute 을 적게 썼을 때 성능이 나쁘지만 test-time compute 을 키우면 개선되는 모델을 평가한다” [Letta STC § 4.2, p.5]. 이것은 test-time-compute 대 정확도 기울기가 가파른 모델을 고른다는 뜻이고, 가파른 기울기는 정확히 $5\times$ 수평 이동을 크게 보이게 만드는 성질이다.

baseline arm 의 프롬프트가 공개되지 않았다. 부록 A 에 실린 다섯개 verbosity 프롬프트 (레벨 0 – 4) 는 전부 sleep 산출물의 존재를 전제한다 — 모델에게 “rethink memory 블록을 확인하라”, “rethink memory 블록에 이미있는 것은 다시 계산하지 말라” 고 지시한다 [Letta STC App. A, Figs. 12 – 16, pp.16 – 17]. 대응하는 baseline arm 의 프롬프트는 실려 있지 않다. 논문 텍스트만으로는 두 arm 이 $\hat{c}$ 의 유무에서만 다른지 확인할 수 없다.

SWE-Features 는 테스트를 돌리지 않고 채점한다. “PR 을 GitHub 에서 긁어 온 것이라 평가에 쓸 만한 직관적인 테스트가 없다. 대신 예측된 수정파일 집합과 정답 수정 파일 목록을 비교해 F1 을 보고한다” [Letta STC § 6, p.12; App. D, p.21]. 에이전트는 Docker 안에서 테스트를 작성하라는 지시를 받지만 채점은 그것을 무시한다. 즉 지표는 동작하는 코드가 아니라 파일 집합을 맞추는 것에 보상을 준다. 그리고 이것이 이방법의 이득이 가장 작고 (약 $1.5\times$) 고예산에서 역전하는 바로 그 지표다.

같은 모델 계열이 과제를 정의하고 또 푼다. claude-sonnet-3-7-20250219 가 PR 을 군집화해 어떤 선행 PR 이 대상 PR 의 ‘관련 문맥’ 인지를 정하고 [Letta STC App. D, p.20], Figure 11 에 서 평가되는 에이전트도 Claude 3.7 Sonnet 이다 [Fig. 11 범례, p.12]. 무엇이 유용하게 관련된 문맥인지를 그것을 이용해야 할 모델이 정한다. 이에 대한 통제는 보고되지 않는다.

같은 부록이 프롬프트 교란을 밝히고 ablation 하지 않는다. “현재단계와 전체 단계를 명시적으로 알려 주면 에이전트 성능이 특히 저예산설정에서 유의하게 개선된다는 것을 발견했다” [Letta STC App. D, p.21]. 저예산 구간은 정확히 이 방법이 이긴다고 주장되는 구간이다.

지연이 한 번도 측정되지 않았다. wall-clock, TTFT, ms/token 수치가 논문에 0 개다. 동기 진술 — “답을 얻기 위해 잠재적으로 수 분을 기다리고 질의당 수십 달러까지 든다” [Letta STC § 1, p.1] — 은 OpenAI 가격 페이지를 가리키는 각주로 지지되고, “지연 최적화 추론이 대략 $10\times$ 비싸다” 는 진술은 Databricks 문서 페이지를 가리키는 각주로 지지된다. 초록의 문제 제기 (‘high latency and inference cost’) 는 전적으로 생성 토큰 대리값 위에서 있다.

내부 참조 오류가 여럿이다. [Letta STC § 5.2] 의 18% 지목이 부분집합 그림을 가리키는 것 (위), [Letta STC § 4.1] 과 App. C 가 ‘Table C’ 를 지시하는데 캡션은 ‘Table 1’ 인 것, [Letta STC § 6] 이 sleep arm 에이전트가 “test-time 국면에 더 많은 파일을 탐색했다” 고 쓰는데 문맥상 sleep 국면인 것, 그리고 역전을 진술하는 [Letta STC § 5.1] 문장에 ‘test-tome’ 오타가 있는 것.

14.6.6 스케일 상한

모델 규모 상한은 없다. 모든 대상 모델이 파라미터 수 비공개 폐쇄 API 다 [Letta STC § 4.2, p.5]. 논문 전체에서 파라미터 수가 진술된 모델은 Llama2-70B 하나이고, 그것은 예측가능성 분석의 도구가 대상이 아니다 [§ 5.4, p.9]. 방법이 모델 무관한 프롬프팅이므로, 규모에 대한 주장은 이 논문으로 확인되지도 반증되지도 않는다.

학습 규모는 0이다. 어떤 종류의 학습도 일어나지 않는다.

평가 규모의 최대치는 Multi-Query GSM-Symbolic P1 의 12043 문항/ 1095 문맥이며, 그중 10948 개 (90.9%) 가 생성 문항이다.

문맥 길이 상한이 보고되지 않는다. 문맥 길이도, 프롬프트 길이도, $|\hat{c}|$ 도 논문 어디에도 없다. GSM-Symbolic 문맥은 문장 몇 개이고 (Fig. 2, p.5), SWE-Features 문맥은 PR 제목 · 본문 · 패치와 에이전트의 저장소 탐색이다 (App. D, p.20). **긴 문맥 레짐이 한 번도 행사되지 않는다** — 동기 예시가 코드베이스와 문서 QA 인 논문에서 이것은 큰 공백이다.

현실성 상한은 SWE-Features 다. 저장소 2 개에서 온 PR 33 건이고, 테스트 없이 파일 집합 F1 으로 채점된다. 이것이 유일한 비수학 · 비합성 평가이며, 동시에 이득이 가장 작고 고예산에서 뒤집히는 설정이다. 논문이 동기론 시나리오와 대조하면 — 큰 저장소 위의 coding assistant 는 PR 33 건으로, 대화 이력 위의 assistant 는 대화 세션 0 건으로 평가되었다.

[평가] 위 목록의 길이가 이 논문의 기여를 지우지 않는다. 시간 비대칭을 이용한다는 발상 자체는 옳다 — 문맥이 질의보다 먼저 도착하는 워크로드가 실재하고, 그 시간 동안 노는 계산 자원이 실재하며, 그것을 쓰면 wake 국면의 생성 토큰이 준다는 것은 관측된 사실이다. 생성 토큰 축의 $5\times$ 는 관측이다. 이 책이 반박하는 것은 그 관측이 아니라, **그 축이 비용 축과 같다는 암묵 등식이다.** 생성 토큰이 줄었다는 관측과 총비용이 줄었다는 결론 사이에는 prefill 항, 측정되지 않은 B_s , 단언된 κ_{cost} , 그리고 미지의 N_q 분포가 놓여 있다. 그 네 가지를 통과해야 등식이 성립한다.

14.7 Systems/serving 합의

독자의 1 번 질문 — 그래서 decode 에서 뭐가 바뀌는가 — 에 대한 답은 이 논문에서 유난히 깨끗하다. **decode 커널은 아무것도 바뀌지 않는다.**

기제는 **프롬프트 토큰 치환**이고 **그 이상이 아니다.** 논문의 표현은 “test-time 에 c 대신 새 문맥 c' 을 제공할 수 있다” 이다 [Letta STC § 3, p.4]. 서빙 스택의 변경은 논문 어디에도 기술되지 않는다. wake 국면의 요청은 종류상 평범한 요청과 같고, 다른 것은 프롬프트의 내용뿐이다.

따라서 상태는 KV cache 와 애플리케이션 텍스트 저장소에 내려앉고, 가중치에는 절대 내려앉지 않는다. 결과로 **shared-weight batching** 이 보존된다. ch01 의 Rosetta 표가 말하는 대로, per-user fast-weight 나 per-user 가중치 delta 가 생기면 가중치를 공유하는 배치가 깨지고 grouped-GEMM 계열의 우회가 필요해진다 (→ ch01, ch19; per-request W 상태의 decode 비용 전개는 → NM 식 (M) 이후 장들). E-경로는 그대가를 치르지 않는다. 이 논문은 batching 도, KV cache 도, prefix caching 도한 번 언급하지 않지만, 기제가 프롬프트 치환이라는 사실만으로도 결론이 따라 나온다.

$\hat{c}$ 는 **한 문맥의 N_q 개 질의에 걸친 공유 prefix** 다. prefix caching 이 있는 스택에서 상각은 경제적 비유가 아니라 문자 그대로의 KV 재사용이 된다.

[해설] 여기서 논문의 비용 모델이 양쪽으로 어긋난다. 식 (14-7) 은 생성 토큰만 세므로 공유 prefix 의 KV 재사용을 **절약으로 잡지 못하고**, 동시에 길어진 prefix 의 prefill 을 **비용으로도 잡지 못한다.** 즉 논문의 회계는 자기 방법의 비용을 과소평가하고 자기 방법의 절약도 과소평가한다. 두 오차의 부호는 반대이지만 크기를 비교할 재료가 논문에 없으므로, 상쇄 여부는 이 논문으로 판정되지 않는다.

wake prefill 이 오르고, 논문은 얼마나 오르는지 한 번도 경계 짓지 않는다. 병렬 스케일링에서는 $\kappa_{\parallel}$ 개의 이어붙인재표현이 매 질의의 프롬프트에 실린다 (최대 10 개)[Letta STC § 5.2, p.8]. $\hat{c}$ 가 보고되지 않으므로 prefill 증가분은 논문에서 계산되지 않는다. 이것이 이 논문의 가장 큰 systems 공백이다 — decode 토큰 절약을 주장하면서 경계 없는 prefill 토큰 비용을 측정하지 않는다.

sleep 작업은 배치 가능하고, 지연에 둔감하며, throughput 으로 최적화되는 decode 잡이다. 서버 합대의 언어로 옮기면, 이 논문의 제품은 latency-critical decode 를 background decode 로 바꾸는 것이고, 논문은 그 전환에 10:1 의 가격을 붙였다 [Letta STC § 5.3, p.8]. interactive tier 의 일을 batch tier 로 옮기는 낮익은 모양이다. 다만 그 10:1 은 이 합대에서 측정된 값이 아니라 벤더 문서에서 인용된 값이다 (§ 14.5.4).

sleep 의 wall-clock 은 임계 경로 밖이어도 공짜가 아니다. sleep 쓰기는 문맥당 최대 10 회의 순차 모델 턴 사슬이다 — 각 호출이 직전 블록을 읽고 덮어쓴다 [Letta STC App. K, p.27]. 여기에 병렬 궤적 $\kappa_{\parallel}$ 까지 곱하면, 최악의 구성은 첫 질의가 도착하기 전에 문맥 하나당 100 회의 모델 턴이다. 합대가 단위 시간에 몇 개의 문맥을 예열할 수 있는지가 이 값으로 정해진다. 논문은 실제로 사용된 턴 수도, 처리율 수치도 보고하지 않는다.

트래픽이 계층의 어디에 내려앉는가. 애플리케이션 텍스트 저장소 (Letta memory 블록 값)[Letta STC App. F Listing 1, p.22] → 토큰나이저 → prefill → KV cache. HBM 상주 가중치도 아니고, optimizer 상태도 아니며, per-user 가중치 샤드도 아니다. 이것이 Θ -경로와의 가장 날카로운 대비다 — E-경로의 쓰기 트래픽은 데이터베이스로 들어가는 텍스트이고, Θ -경로의 쓰기 트래픽은 디바이스 메모리로 들어가는 가중치 delta 다. memory-device 기회를 만드는 쪽은 후자다 (→ ch26, ch29). 이 장에서 memory-centric 논증을 더 밀지 않는다. 이 논문에 그 논증을 지탱할 바이트 수치가 하나도 없기 때문이다.

입장 제어 신호가 하나 제안되어 있고, 제안된 형태로는 작동하지 않는다. 논문은 $\log P(q | E_0)$ 를 sleep-time compute 이 이득을 낼 문맥의 신호로 제시하고, 그것으로 결과를 5 분위로 나눈다 [Letta STC § 5.4, p.9; Fig. 10, p.11]. 계산 자체는 싸다 — 작은 동결 base 모델의 forward 한 번이면 된다.

[평가] 그러나 이 신호는 sleep 결정을 게이팅할 수 없다. 신호가 q 를 입력으로 요구하는데, q 는 sleep 시점에 정의상 존재하지 않는다. 논문은 할당 정책을 future work 로 명명하면서 이 순환을 관측하지 않는다. 서버에서 이 신호를 쓰려면 q 가 아니라 E_0 만으로 계산되는 대리 지표로 바뀌야 하고, 그 변환은 논문에 없다.

14.8 한계와 bridge-out

14.8.1 논문 자신이 남긴 문제

논문의 Letta STC § 7 은 네 가지를 남긴다.

1. 어느 문맥이 예측 가능한 질의를 갖는가, 그리고 계산을 어떻게 배분하는가. “흥미로운 향후 방향은 어떤 문맥이 예측 가능한 질문을 가질지 식별하고, 서로 다른 문맥과 질의에 걸쳐 sleep-time 과 test-time 사이에 추론 계산을 최적으로 배분하는 것이다.” 이 논문에는 언제 잠들지를 결정하는 컨트롤러가 없다.
2. 두 국면 가정을 넘어서기. “실제 LLM 사용 사례는 더 복잡할 수 있고, 여러 라운드의 상호작용과 라운드 사이의 문맥 수정이 있다 (예: 코드베이스에 대한 여러 번의 편집). 또한 sleep-time 의 길이 자체도 상호작용마다 크게 다를 수 있다 (사용자 타이핑 사이의 짧은 간격부터 며칠의 비활동까지).” 이것이 정확히 ρ 의 질문이며, 저자들이 이미 해결이라고 명시한다.
3. 자연어 공간의 표현 학습으로 밀 수 있는가. § 14.3.4 에 인용한 문장이 여기서 나오고, 이 책의 3 경로 분할의 1 차 근거가 된다.
4. $\hat{c}$ 를 합성 학습 데이터로 쓸 수 있는가. “향후 연구는 sleep-time compute 의 출력 자체를 합성 데이터의 한 형태로 쓰는 것을 탐색할 수 있다.” 이것은 E-경로에서 Θ -경로로의 명시적 인계다 — $\hat{c}$ 가 (U- Θ) 의 학습 집합 $\mathcal{R}_k$ 가 되는 구성이다. 저자들이 제안하고, 시험하지 않는다.

논문이 질문으로 세우지 않았으나 이 책이 미해결로 기록하는 것도 있다. 고예산 역전의 기제 (가설만 있고 시험 없음), $\kappa_{\parallel}$ 비단조성의 임계, o1 예외의 원인, 실서빙의 N_q 분포, $\kappa_{\text{cost}} = 10$ 의 타당성, 그리고 $\hat{c}$ 의 검증·만료·축출·버전 관리 — 이 마지막 묶음은 논문 어디에서도 다루어지지 않는다. 쓰기 검증 단계도, 용량 정책도, staleness 처리도 없다.

14.8.2 경로 간 인용여부

참고문헌은 22 개다 [References, pp.13 – 15]. 전수 확인 결과는 다음과 같다.

- **W-경로 인용: 0 건.** fast weight, test-time training, 선형 attention, state-space model, recurrent memory, weight-space test-time 상태 — 어느 것도 없다. 'state' 라는 단어는 'stateful application' 의 뜻으로만 나온다. KV cache 는 한 번도 언급되지 않는다.
- **Θ-경로 직접 인용: 0 건.** LoRA 도, PEFT 도, 모델편집 (ROME/MEMIT) 도, 지속학습도, catastrophic forgetting 도, knowledge distillation 도 없다. 가장 가까운 이웃은 합성데이터 두 편 (Yang et al. 2024, Gunasekar et al. 2023) 과 Bansal et al. 2024 인데, 셋 다 [Letta STC § 7] 의 future work 문단에서 sleep 산출물의 소비자로서만 등장하지 경쟁 기제로 등장하지 않는다. 다른 두 공간을 이름으로 부르는 유일한 문장 (§ 14.3.4 인용) 이 그 공간의 어떤 연구도 인용하지 않고 2014 년 리뷰 하나만 단다.
- **생물학·CLS 인용: 0 건.** McClelland 도, hippocampal replay 도, sleep consolidation 신경과학도, dreaming 문헌도 없다. 이논문의 중심 은유인 'sleep' 에 생물학 인용이 하나도 붙어 있지 않다. Letta STC § 1 은 이 용어를 순전히 조작적으로 도입한다 — “모델과의 상호작용 사이, 그렇지 않았으면 놓고 있었을 sleep-time 에추론을 수행한다.” 단어는 유희를 위해 선택된 것이지 consolidation 을 위해 선택된 것이 아니다.
- **E-경로 내부 인용: 정확히 1 건.** Packer et al. 2023(MemGPT), 같은 저자·같은 회사, [Letta STC § 2] 의 OS/pre-fetching 유비에서. RAG 도, vector database 도, graph memory 도, 검색 문헌 어느 것도 참고문헌에 없다 — ret 을향등으로 두는 설계 선택은 검색 문헌이 반대할 만한 선택인데도 그렇다.
- **논문이 실제로 교전하는 것:** test-time scaling(Snell 2024, OpenAI 2024, DeepSeek-AI 2024, Brown 2024, Muennighoff 2025), speculative decoding(Leviathan 2023, Stern 2018, Cai 2024, DeepSeek-V3), 고전적 systems 선계산 (Smith 1982 캐시 메모리, Gray 1997 data cube).

이 네 건의 침묵을 두 종류로 갈라야 한다. 어떤 침묵은 **연대가강제한다** — 인용될 수 없는 것을 인용하지 않은 것은 계보 사실이지 선택이 아니다. 어떤 침묵은 **선택된다** — 인용할 수 있었는데 하지 않은것이다. 이 논문의 발표 시점은 2025 년 4 월이고, 위 네 묶음은 **전부후자다**. W-경로의 주류 계보 (→ ch16, ch17), Θ-경로의 쓰기 primitive 인 LoRA(2106.09685) 와 모델편집 (2202.05262, 2210.07229)(→ ch19), 망각대응의 원형인 EWC(1612.00796)(→ ch07), CLS 이론과 생물학적 sleep 실험 (→ ch08, ch23), 그리고 E-경로 내부의 RAG(2005.11401)(→ ch10) 와 Memorizing Transformers(2203.08913)(→ ch12) — 어느 것도 이 논문보다뒤에 나오지 않았다. 인용 가능한 상태로 전부 존재했다.

[평가] 이 라인의 이름을 만든 논문은 자기를 캐시와 speculative decoding 의 후손으로 놓았지 기억 연구나 학습 연구의 후손으로놓지 않았다. 그리고 뒤에 'sleep-time compute' 이라는 이름을 쓴 W-경로·Θ-경로의 논문들은, 자기를 인용한 적없고 생물학을 인용한 적도 없는 논문에서 그 이름을 물려받았다. 이름의충돌은 공유된 이론적 약속이 아니다. 이 관측이 ch11 의 판별식과 ch27 의판정에서 다시 쓰인다.

14.8.3 다음 장이 받아가는것

ch15 는 이 장이 세운 (U-E) 의 특수화를 프로덕션 시스템 쪽으로 가져간다. 이장에서 항등이었던 ret 이 거기서는 진짜 검색기가 되고, 이장에서 파괴적 덮어쓰기 하나였던 wr 이 거기서는 연산자집합으로 갈라지며, 이 장에서 문맥과 함께 죽던 E 가 거기서는 세션을넘어 지속된다. 그리고 이 장이 라운드 1 로 고정해 둔 k 가 거기서는늘어나기 시작한다.

[평가] 그런데 ch15 가 다루는 논문들은 **이 논문을 인용하지 않는다**. 같은 경로의 후속 연구가 그 경로에 이름을 준 논문을보지 않는다. § 14.8.2 가 보인 침묵이 바깥으로만 향한 것이 아니라안쪽에서도 되돌아온다는 뜻이다. 그 단절 자체가 ch15 의 재료이며, 연대가강제한 침묵이 아니라 선택된 침묵이라는 점에서 ch11·ch27 이다시 쓴다.

이 장이 채우지 못하고 넘기는 것은 표 14-3 의 네 칸이다. 미리 말해 두면 **ch15 도 그 칸을 채우지 못한다**. 채우려는 시도가 이 책의 실험 X1(비용) 과 X2(용량) 이고, 그 위에서의 판정은 ch25 와 ch26 이 소유한다.

요약

- Letta 의 Sleep-time Compute(2504.13171) 는상속받은 open question 없이 시작한다. 참고문헌 22 개이며, 계보 진술은같은 저자의 MemGPT 에 대한 운영체제 pre-fetching 유비 하나뿐이다 [§ 2, p.3]. 논문은 문제를 물려받는 대신 test-time scaling 문헌에 stateless 가정을 부여해 공백을 제조한다 [§ 1, p.1].
- 기제는 (U-E) 하나이며, 네 겹으로 특수화되어 있다: wr 은파괴적 덮어쓰기, $S(\cdot; B_s)$ 는 깊이 $J \leq 10$ 의 반복 자기재작성, ret 은 항등, E 는 문맥 범위이고 휘발성이다. Θ 는 동결이고 W 는 존재하지 않는다. 논문에는 번호 붙은수식이 하나도 없다.
- learned context $\hat{c}$ 의 'learned' 는 gradient 를 뜻하지 않는다. 논문 본문이 스스로 정확하게 쓴다 — 표현이 형성되는 공간이 parameter 도 activation 도 아닌 자연어라는 것이고, 그 문장이 이 책 3 경로 분할의 1 차근거다 [§ 7, p.13].
- 비용 4 종이 **전부 논문에 없다**. $B_s \cdot L_w \cdot C_{cap} \cdot \rho$ 어느 것도 숫자로 보고되지 않는다. 상각 논증의 분자 B_s 가 없으므로손익분기 N_q^* 를 식 (14-8) 로 계산할 수 없고, 논문은 $N_q \in \{1, 2, 5, 10\}$ 네 점의 곡선과 “질의 수가 적으면 불리하다” 는정성 진술만 준다 [Fig. 9 캡션, p.11].
- 비용모델 (14-7) 은 생성 토큰만 센다. prefill 항이 없는데 이 방법의 기제자체가 입력을 길게 만드는 것이며, $\kappa_{||} = 10$ 에서는 열개의 재표현이 매 질의마다 prefill 된다. $\kappa_{cost} = 10$ 은벤더 문서 인용이고 민감도 분석이 없다.
- $5 \times 13\% \cdot 18\% \cdot 2.5 \times$ 는 전부 저자제작 벤치마크 (Stateful GSM-Symbolic, Stateful AIME, Multi-Query GSM-Symbolic) 에서 나온 값이다. Stateful AIME 60 문항에는 문맥이 빈인 스텐스가 개수 미상으로 포함되고 수동 재배열이 있었으며 [App. J, pp.27–28], Multi-Query 의 90.9%가 모델 생성 문항이고 생성 답의검증은 보고되지 않았다 [Table 1, p.19].
- 이득은 높은 test-time 예산에서 역전한다. Stateful GSM-Symbolic 과 SWE-Features 두 과제 모두에서이며, 기제는 제시되지 않았다 [§ 5.1, p.6; § 6, p.12]. sleep 스케일링도 비단 조여서 $\kappa_{||} = 5$ 가 10 을 이긴다 [§ 5.2, p.8].
- 서빙 관점에서 decode 커널은 아무것도 바뀌지 않는다. 상태가 가중치에내려앉지 않으므로 shared-weight batching 이 보존되고, 그 대신 Θ -경로가 만드는 종류의 memory-device 기회도 생기지 않는다. 대가는 경계 없는 wake prefill 증가이며 논문은 그것을 측정하지 않는다.

자가 점검체크리스트

- 이 논문이 갱신하는 층이 (U-E) 하나뿐이고, 그 (U-E) 가 정규형과 어긋나는지점 다섯 개를 말할 수 있는가 (wr 파괴적 덮어쓰기/ $S(\cdot; B_s)$ 반복 / 두 번째 wr 인 concat / ret 항등/ E 휘발).
- 식 (14-8) 을 쓰고, 우변 세 값 중 어느 것이 논문에 있고 어느 것이 없는지지목할 수 있는가. 없는 값 때문에 N_q^* 가 계산되지 않는 이유를 한문장으로 말할 수 있는가.
- “생성 토큰이 $5 \times$ 줄었다” 와 “비용이 $5 \times$ 줄었다” 사이에 놓인 네 가지 —prefill 항, 미보고 B_s , 단언된 κ_{cost} , 미지의 N_q 분포 — 를 열거할 수 있는가.
- $5 \times 13\% \cdot 18\% \cdot 2.5 \times$ 가 각각 어느벤치마크, 어느 레짐의 값인지 말할 수 있는가. 그리고 이득이 역전하는레짐을 두 과제 모두에서 지목할 수 있는가.

- 'learned context' 에서 무엇이 학습되지 **않는지**, 그리고 논문본문이 그 사실을 어디에서 스스로 밝히는지 말할 수 있는가.
- 이 논문이 W -경로 $\cdot \Theta$ -경로 $\cdot$ 생물학을 각각 몇건 인용하는지 말할 수 있는가. 그 답이 왜 ch11의 재료가 되는지 설명할 수 있는가.
- **Rosetta**: prefix cache 와 $\hat{c}$ 의 차이를 재사용의 종류로 설명할 수 있는가 (같은 계산의 재사용 대 새 추론의 선행 수행). 그리고 이 논문에서 shared-weight batching 이 깨지지 않는 이유를, per-user 가중치 delta 가 생기는 경우와 대조해 말할 수 있는가.

ch15. Mem0 · Zep · ReasoningBank · SCM — 프로덕션과 그대가

15.1 Bridge-in: 전작이 남긴문제

이 장의 다섯 시스템 중 어느 것도 ch14 를 인용하지 않는다.

사실부터 고정한다. Zep(arXiv 2501.13956) 은 2025 년 1 월 20 일자이고 ch14 의 논문 (arXiv 2504.13171) 보다 석 달 앞선다. Mem0(arXiv 2504.19413) 은 2025 년 4 월 28 일자로 열흘 뒤에 나왔다. ReasoningBank(arXiv 2509.25140) 은 이 책이 읽은판본이 v2(2026 년 3 월 16 일) 이고, SCM(arXiv 2604.20943) 은 2026 년 4 월, Memini(arXiv 2605.05097) 는 v3 가 2026 년 6 월이다. 앞의 둘은 ch14 를 인용하지 못할 연대적 이유가 있다. 뒤의 셋은 그렇지 않다. ReasoningBank 전문에 “sleep” 이라는 문자열은 한 번도 나오지 않고, Memini 9 쪽에도 나오지 않으며, SCM 은 제목에 sleep 을 걸고도 ch14 를참고문헌에 넣지 않았다.

그렇다면 이들은 무엇을 상속했는가. 다섯 논문의 어느 것도 선행 논문이 **명시적으로 남긴 open question** 을 인용하지 않는다. Zep 은 MemGPT 가남긴 질문이 아니라 MemGPT 의 점수를 다룬다 — “In the DMR benchmark, which the MemGPT team established as their primary evaluation metric, Zep demonstrates superior performance (94.8% vs 93.4%)” [Zep Abstract]. Mem0 은 MemGPT 의 계층화 질문에 답하되 그 사실을 밝히지 않고, 학습된 정책을 쓰자는 선택지는 명시적으로 거절한다 — “Rather than using a separate classifier, we leverage the LLM’ s reasoning capabilities to directly select the appropriate operation” [Mem0 § 2.1]. ReasoningBank 은 AWM 과 Synapse 를 자기 목소리로 진단하고, SCM 과 Memini 는 각각 [SCM § 2] 와 [Memini § 2] 에서 선행 연구 세 갈래를 저자 자신의판정으로 반박한다.

[평가] 이것은 인용 누락이 아니라 사술의 종류가 다른 것이다. ch14 까지의 E -경로는 질문을 주고받았고, 이 장의 다섯은 **리더보드를 주고받는다**. 대화 상대가 경쟁 질문이 아니라 경쟁 제품이면, 상속할 것은 벤치마크와 점수뿐이고 비용 회계는상속되지 않는다. 다섯 논문이 서로의 B_s 를 인용하지 못하는 이유가여기 있다 — 인용할 수 있는 B_s 를 아무도 인쇄한 적이 없기 때문이다.

ch14 가 남긴 문제는 두 가지였다. 하나는 저자 제작 벤치마크에 걸린 이득을어떻게 읽을 것인가이고, 다른 하나는 생성 토큰만 세는 비용 모델이 무엇을감추는가였다. 이 장의 다섯은 첫 번째 문제를 절반쯤개선하고 (Mem0·Zep·ReasoningBank 는 남이 만든벤치마크를 쓴다) 두 번째 문제를 그대로 물려받는다. 정확히는, 더 나빠진형태로 물려받는다. ch14 는 적어도 sleep 예산을 세려고 시도했다. 이 장에서 B_s 를 숫자로 인쇄한 논문은 다섯 중 하나뿐이다.

15.2 문제의식

다섯 논문의 문제 진술은 놀랄 만큼 같은 모양이다. 고정된 문맥창, 계속자라는 스트림, 그리고 창을 키워도 해결되지 않는다는 주장이다.

- “these systems are fundamentally limited by their reliance on fixed context windows, which severely restrict their ability to maintain coherence over extended

interactions” [Mem0 § 1]. 창을 키우는 것은 자연일 뿐이라는 근거로 두 가지를 든다 — 관계는 어떤 창보다 오래 지속되고, 실제 대화는 주제적으로 연속하지 않는다 [Mem0 § 1].

- “Current approaches using RAG have focused on broad domain knowledge and largely static corpora—that is, document contents added to a corpus seldom change” [Zep § 1].
- “By approaching each new task in isolation, they are doomed to (i) repeat similar errors observed in the past, (ii) discard valuable insights gained from related problems” [ReasoningBank § 1].
- LLM 은 “remain fundamentally amnesic. Each conversation begins anew” [SCM § 1].
- “A system whose memory merely grows is accumulating. A system whose memory consolidates, forgets, and reshapes its organization in response to experience is learning” [Memini § 1].

같은 모양이지만 답은 다섯 방향으로 갈라진다. Mem0 은 무엇을 얼마나 적게 다시 읽을 것인가를 묻고, Zep 은 어떤 사실이 언제 참이었는지를 어떻게 기록할 것인가를 묻고, ReasoningBank 은 경험을 어떤 형태로 저장해야 재사용되는가를 묻고, SCM 은 무엇을 지울 것인가를 묻고, Memini 는 저장소 자체가 동역학계이면 어떻게 되는가를 묻는다.

여기서 한 가지가 빠져 있다. 다섯 중 어느 것도 이 문제를 **계산을 언제 쓸 것인가**의 문제로 세우지 않는다. Mem0 전문에서 “sleep”, “offline”, “test-time”, “inference-time” 은 0 회 등장하고, Zep 전문에서 “sleep”, “consolidat”, “offline”, “amortiz” 는 0 회이며, Memini 전문에서 “sleep” 과 “offline compute” 는 0 회다. ch01 의 조작적 정의는 예산 B_s 를 언제 쓰느냐로 이 범주를 가르는데, 이 논문들의 자기 진술은 **표현과 비용의 문제**이지 **시점의 문제**가 아니다. 그러면서도 다섯 모두 판별식은 통과한다 — 질의가 도착하기 전에 E 의 상태를 실제로 바꾸고 그 변화가 이후 질의에 쓰이기 때문이다 (→ ch11).

이 어긋남이 이 장의 진짜 주제다. 다섯 논문은 각자 다른 것을 고치겠다고 선언하지만, 자기 표를 그대로 읽으면 **품질 1 위가 반복적으로 “기억시스템 없음”** 이다. Mem0 의 Table 2 에서 최고 J 점수는 full-context $72.90 \pm 0.19\%$ 이고 Mem0 의 $66.88 \pm 0.15\%$ 보다 6.02 점 높다. Zep 의 Table 1 에서 gpt-4-turbo full-conversation 94.4%는 이미 비교 대상인 MemGPT 93.4%를 넘어서는 선다. SCM 의 Table 2 에서 아무것도 지우지 않는 No-Forget Graph 는 SCM 과 똑같이 22/22 를 맞힌다. ReasoningBank 은 검색 경험을 하나에서 둘로 늘리면 49.7 에서 46.0 으로 떨어진다. Memini 는 검색을 한 번도 돌리지 않았다.

이 관찰을 다섯 번 반복하는 것이 이 장의 논증이고, § 15.8 의 [평가] 가 그 결론이다.

15.3 Core mechanism (통일표기)

다섯 시스템은 읽기에서 완전히 같은 식을 쓴다. 표준형 (R) 에서 W 자리가 비어 있다.

$$\hat{y} = f(q; \Theta, \emptyset, \text{ret}(E, q)) \quad (15-1)$$

Θ 는 다섯 논문 전체에서 단 한 번도 움직이지 않는다. Mem0 은 모든 호출 지점에 GPT-4o-mini 를 쓰고 [Mem0 § 2.1], Zep 은 그래프 구축에 gpt-4o-mini-2024-07-18 을 쓰며 [Zep § 4.1], ReasoningBank 은 Vertex AI 로 Gemini-2.5-Flash/Pro·Claude-3.7-Sonnet 을 부르고 [ReasoningBank § 4.1], SCM 은 “It requires no training or fine-tuning; all components use existing pretrained models or algorithmic logic” [SCM § 3.8] 이라고 명시하며, Memini 는 “The backbone can be replaced or upgraded without disturbing what the memory has learned” [Memini § 3.1] 라고 쓴다. 이 장에는 gradient 가 하나도 없다. 손실 함수도 ℓ 도 $\mathcal{L}$ 도 등장하지 않는다.

쓰기도 모두 (U-E) 이고, 차이는 전부 $S(\cdot; B_s)$ 와 $\text{wr}(\cdot, \cdot)$ 의 모양에 있다. 다섯의 이탈을 표준형과의 차이로 적으면 이렇다.

Mem0 — 재귀적 인자. 요약이 저장소에서 검색되어 추출 프롬프트로 들어가므로 $S(c_k, E_k; B_s)$ 의 인자에 E 자신이 들어간다 [Mem0 § 2.1].

$$E_{k+1} = \text{wr}(E_k, S(c_k, E_k; B_s)) \quad (15-2)$$

표준형에는 이 되먹임 경로가 없다. 결과적으로 k 라운드의 쓰기가 $k+1$ 라운드의 추출 품질을 바꾼다. 트리거 단위는 메시지쌍 (m_{k-1}, m_k) 이고 최근 창은 $m_{\text{rec}} = 10$, 갱신 판단에참조하는 이웃은 $s_{\text{nn}} = 10$ 이다 [Mem0 § 2.1].

Zep — 타입 있는 그래프 delta. $\hat{c}_k$ 가 문자열이 아니라다섯 종류의 델타 묶음이다 [Zep § 2.2].

$$\hat{c}_k = (\Delta \mathcal{N}_e, \Delta \mathcal{E}_e, \Delta \mathcal{N}_s, \Delta \mathcal{E}_s, \Delta \mathcal{N}_c) \quad (15-3)$$

이 델타를 만드는 $S(\cdot; B_s)$ 는 아홉 단계다 — 개체 추출, reflexion 재확인, 1024 차원 임베딩, 개체 해소, 사실 추출, 간선 중복 제거, 시간추출, 모순 검사 · 무효화, community 라벨 전파 [Zep § 2.2.1 – § 2.3]. 이 중 여섯에서 일곱 단계가 frozen LLM 호출이다. $S(c_k; B_s)$ 가 보는 문맥은 현재 메시지와 직전 네 개, 즉 $n_{\text{ctx}} = 4$ 로 고정된다 [Zep § 2.2.1].

ReasoningBank — 라벨로 분기하는 요약. $S(\mathcal{H}_k^\Delta; B_s, j_k)$ 의 인자가 문맥이 아니라 에이전트 자신의 궤적이고, 자기 판정 라벨 j_k 가 프롬프트를 고른다 [ReasoningBank § 3.2].

$$\hat{c}_k = S(\mathcal{H}_k^\Delta; B_s, j_k), \quad j_k \in \{\text{success, failure}\}, \quad |\hat{c}_k| \leq 3 \quad (15-4)$$

성공 분기는 검증된 전략을, 실패 분기는 예방 교훈을 뽑는다. 두 프롬프트모두 “Do not mention specific websites, queries, or string contents” 라는제약을 건다 [ReasoningBank App A.1 Fig. 9].

[평가] 이 논문이 기여로 내세우는 추상화는 아키텍처가 아니라프롬프트 한 줄에서 나온다. (15-4) 에서 $S(\mathcal{H}_k^\Delta; B_s, j_k)$ 의 모양을 정하는 것은 라벨 j_k 가 고르는 프롬프트뿐이고, 학습되는 성분도 강제되는 스키마도 없다.

SCM — 요약이 wake 로 옮겨간 경우. 추출은 매 턴 wake 에서 일어나고 [SCM § 3.2], sleep 예산은 이미 쓴 것을 재조직하고 지우는데만 쓰인다.

$$E_{k+1} = \text{prune}_{\vartheta_f}(\text{dream}(\alpha^E \cdot \text{hebb}(E_k, E_k^{\text{wm}}))), \quad \Theta_{k+1} = \Theta_k \quad (15-5)$$

간선 강도의 갱신만 떼어 쓰면 아래 형태가 되는데, 이것은 (U-W) 의 retention gate 와 대수적으로 같은 모양이다 — 다만 상태가 W 가 아니라 E 의 스칼라이고, 쓰기 향이 gradient 가 아니라 중요도의 곱이며, α^E 가 데이터 의존 gate 가 아니라 상수 0.8 이다 [SCM Eq. 6, Eq. 7].

$$s_{ij}^{(k+1)} = \alpha^E (s_{ij}^{(k)} + \eta_{\text{heb}} I(c_i) I(c_j)), \quad \alpha^E = 0.8, \quad \eta_{\text{heb}} = 0.1 \quad (15-6)$$

[해설] 반복 강화되는 간선의 고정점은 $s^* = 0.08 I(c_i) I(c_j) / 0.2 = 0.4 I(c_i) I(c_j)$ 이고, 강화되지않는 간선은 k 사이클 뒤 0.8^k 로 줄어 20 사이클이면 원래의 약 1.15%가 남는다. 이 장에서 유일하게 대수적으로 닫히는 용량상한인데, 논문은 이 계산을 하지 않는다.

Memini — 저장소 전체가 선형 필터 बैंक. 간선마다 읽히는 빠른성분 E_e^f 와 읽히지 않는 느린 성분 E_e^s 가결합한다 [Memini Eq. 1, Eq. 2].

$$\dot{z}_e = A z_e + \begin{bmatrix} I_e(t) \\ 0 \end{bmatrix}, \quad z_e = \begin{bmatrix} E_e^f \\ E_e^s \end{bmatrix}, \quad A = \begin{bmatrix} -(\frac{1}{\tau_f} + \mu) & \mu \\ \mu & -(\frac{1}{\tau_s} + \mu) \end{bmatrix} \quad (15-7)$$

행렬형은 논문에 없다. 논문이 쓴 것은 두 개의 스칼라 미분방정식과 “a forward Euler step $\Delta t = 1$ corresponding to one document per step” 이라는 이산화 규칙뿐이다 [Memini App. A.2]. A 가 모든 간선에대해 같고 I_e 가 임펄스열이므로, Memini 의 쓰기 층은 $|\mathcal{E}|$ 개의 동일한 2 극 선형 필터 बैंक다. 여기서도 학습되는것은 없다.

15.3.1 쓰기 연산자의 종류

이 절이 도입하는 첫 번째 정의다. E -경로의 쓰기 연산자는 **저장소의 크기를 어떻게 움직일 수 있는** 가로 분류된다.

정의. 쓰기 연산자 $wr(E, \hat{c})$ 가 **확장전용 (append-only)** 이라 함은 모든 입력에 대해 $|E_{k+1}| \geq |E_k|$ 가 성립하는 경우를 말하고, **수축 가능 (contraction-capable)** 이라 함은 어떤 입력에 대해 $|E_{k+1}| < |E_k|$ 가 되는 경우를 말한다. 수축가능한 연산자는 ρ 를 실재하는 위험량으로 만든다.

이 축 위에서 다섯 시스템의 연산자는 네 가지 형태로 나뉜다.

정의. 네 연산 문법 (four-operation grammar) 은 후보 사실 ω_i 마다 frozen LLM 이 {ADD, UPDATE, DELETE, NOOP} 중 하나를 함수 호출로 고르는 쓰기 규칙이다 [Mem0 § 2.1, Alg. 1]. ADD 는 $E \leftarrow E \cup \{(\text{id}, \omega)\}$, UPDATE 는 기존 항목의 교체, DELETE 는 되돌릴 수 없는 제거, NOOP 는 항등이다.

정의. 합집합 쓰기 (union write) 는 $wr(E_k, \hat{c}_k) = E_k \cup \hat{c}_k$, 즉 병합 · 중복 제거 · 감쇠 · 축출이 전혀없는 순수 합집합이다 [ReasoningBank § 3.2]. 논문 자신의 표현으로는 “a minimal consolidation strategy: newly generated items are directly added without additional pruning” [ReasoningBank App A.2] 이다.

정의. 그래프 delta 쓰기는 $\hat{c}$ 가 문자열이 아니라 노드 · 타입 간선 · 메타데이터의 델타 묶음이고, 쓰기가 스키마로 강제되는 형태다 [Zep § 2.2]. Zep 은 이 델타를 LLM 이 만든 질의가 아니라 미리 정의된 Cypher 로 적용한다 — “We chose this approach over LLM-generated database queries to ensure consistent schema formats and reduce the potential for hallucinations” [Zep § 2.2.1].

네 번째는 **동역학 쓰기**다. SCM 과 Memini 의 wr 은 인자에 없는 항목까지 건드린다. SCM 은 사이클마다 모든 간선에 α^E 를 곱하고 임계 미달 노드를 지우며, Memini 는 문서가 하나 도착할 때마다 감쇠와 결합 항을 **모든 간선에** 적용한다 [Memini Eq. 1 – 2; App. A.2]. 표준형 (U-E) 는 $O(|\hat{c}_k|)$ 개 레코드를 건드리지만 이 둘은 $O(|\mathcal{E}_k|)$ 개를 건드린다.

표 15-1. 다섯 시스템의 쓰기 연산자

시스템	wr 의 형태	수축 가능성	논문의 근거 위치
Mem0 (base)	네 연산 문법, frozen LLM 이 선택	가능 — 하드 DELETE	[Mem0 § 2.1, Alg. 1]
Mem0 ^g	그래프 delta + 무효화 표시	불가 — 삭제 대신 INVALID 표시	[Mem0 § 2.2]
Zep	그래프 delta, 스키마 강제 Cypher	불가 — 전 계층 append-only	[Zep § 2, § 2.2.3]
ReasoningBank	합집합, 항목 상한 3 개 (MaTTS 5 개)	불가	[ReasoningBank § 3.2]
SCM	동역학: hebb $\rightarrow$ 수축 $\rightarrow$ dream $\rightarrow$ prune	가능 — 임계 미달 노드 삭제	[SCM Alg. 1]
Memini	동역학: 전 간선 forward Euler 1 스텝	불가 — 가중치만 감쇠, 간선 제거 규칙 없음	[Memini Eq. 1 – 2]

Mem0 한 논문 안에서 두 정책이 화해 없이 공존한다는 사실은 그대로 적어야 한다. base 는 모순된 기억을 하드 삭제하고 [Mem0 Alg. 1], 그래프 변형은 “marking them as invalid rather than physically removing them to enable temporal reasoning” [Mem0 § 2.2] 라고 쓴다. 논문은 돌의불일치를 지적하지도, 둘 중 어느 쪽이 옳은지 측정하지도 않는다. 게다가 UPDATE 분기의 가드인 $\text{InformationContent}(f) > \text{InformationContent}(m_i)$ 에서 InformationContent 는 논문 어디에도 정의되어 있지 않다 [Mem0 Alg. 1].

15.3.2 시간을 표현하는 방식

두 번째 정의다. 이 장의 시스템들은 시간을 세 가지 방식 중 하나로 표현하며, 셋은 서로 바꿔 쓸 수 없다.

정의. 명시적 타임스탬프 방식은 저장 레코드에 절대시각을 필드로 붙이는 방식이다. 그 극단이 **이중 시간 (bi-temporal) 표현**으로, 사실이 세계에서 참이었던 구간을 다루는 타임라인 T 와 시스템이 그것을 알게 되거나 철회한 순서를 다루는 타임라인 T' 를 독립적으로 유지한다 [Zep § 2.1]. Zep 은 의미 간선마다 네 개의 시각을 단다 — $t_{\text{valid}}^{\text{wall}}, t_{\text{invalid}}^{\text{wall}} \in T$ 와 $t_{\text{created}}', t_{\text{expired}}' \in T'$ [Zep § 2.2.3].

이 표현이 낳는 쓰기 규칙이 **구간 폐쇄 (interval closure)** 다. 모순되는 새 간선이 오면 옛 간선을 지우는 대신 그 유효 구간을 닫는다.

$$\text{contradict}(e_{\text{new}}, e_{\text{old}}) \wedge \text{overlap}_T(e_{\text{new}}, e_{\text{old}}) \Rightarrow t_{\text{invalid}}^{\text{wall}}(e_{\text{old}}) \leftarrow t_{\text{valid}}^{\text{wall}}(e_{\text{new}}) \quad (15-8)$$

정의. 감쇠 상수 방식은 시각을 저장하지 않고, 상태가 시간에 따라 줄어드는 속도로 시간을 표현하는 방식이다. SCM 은 $\delta(c) = \exp(-\lambda_{\text{rec}}\Delta t)$, $\lambda_{\text{rec}} = 0.01$ 을 쓰고 [SCM Eq. 9], Memini 는 두시상수 $\tau_f = 2$, $\tau_s = 10$ 을 **문서단위로** 쓴다 [Memini App. A.2]. 이 방식에서 “언제” 는 절대 시각이 아니라 “그 뒤로 몇 번의 이벤트가 지났는가” 다.

정의. 순서 첨자 방식은 시간을 라운드 번호로만 표현하고 지속 시간을 아예 모형화하지 않는 방식이다. ReasoningBank 은 과제 스트림의 순서 $k = 1..N$ 만 갖는다. 항목에 시각도 없고 감쇠도 없다 [ReasoningBank § 3.1 – § 3.2].

세 방식은 서로 다른 실패 모드를 갖는다. 명시적 타임스탬프는 정확하지만 **모델이 그것을 읽어야** 쓸모가 있다. Zep 의 이중 시간은 결국 프롬프트 안에 문자열로 들어간다 — 템플릿 머릿글이 “These are the most relevant facts and their valid date ranges. ... format: FACT (Date range: from - to)” [Zep § 3] 이다. 시간 임베딩도, 검색 단계의 유효성 필터링도, 시간 전용 attention 도 없다. 그래서 § 15.6 에서 보게 될 결과 — 약한 모델에서 knowledge-update 가 **되행한다**는 것 — 은 표현의 결함이 아니라 표현이 추론자에게 전가된 결과다. 감쇠 상수 방식은 절대시각을 버리는 대신 재언급되지 않은 사실을 조용히 잃고, 순서 첨자 방식은 시간을 아예 다루지 못한다.

15.3.3 표기 대응표

다섯 논문의 국소 기호를 이 책의 예약 기호로 옮긴다. 본문에서는 오른쪽열만 쓴다.

표 15-2. 논문 기호 → 이 책의 표기

논문	원 기호 (원문에서의 뜻)	이 책의 표기
Mem0	S (대화 요약 텍스트, § 2.1)	$\hat{c}^{\text{sum}} - E$ 의 한 원소
Mem0	E (그래프의 간선 집합, § 2.2)	$\mathcal{E}$
Mem0	L (노드 타입 라벨, § 2.2)	L_{lab}
Mem0	M (기억 저장소, Alg. 1)	E
Mem0	m (메시지 / 최근 창 크기 / 저장항목, § 2.1)	$m_k / m_{\text{rec}} = 10 / E$ 색인
Mem0	t (메시지 색인 / 노드 생성 시각 / 병합 임계, § 2.1 – 2.2)	$k / t_v / t_{\text{sim}}$
Mem0	k (검색 청크 수, § 3.3)	k_{RAG}
Mem0	s (갱신 판단에 쓰는 유사 기억 수, § 2.1)	$s_{\text{nn}} = 10$
Mem0	ϕ (LLM 추출 함수, § 2.1)	$S(\cdot; B_s)$

논문	원 기호 (원문에서의 뜻)	이 책의 표기
Mem0	$\Omega = \{\omega_1.. \omega_n\}$ (후보 사실 집합)	$\hat{c}$ (집합값)
Mem0	F (Alg. 1의 검색된 사실 집합)	$\hat{c}$
Mem0	P (추출 프롬프트 튜플, § 2.1)	P_k
Mem0	J (LLM-as-a-Judge 점수, § 3.2)	J (유지)
Zep	E (그래프 간선 집합, § 2)	$\mathcal{E}$
Zep	ρ (재순위 함수, § 3)	rrank
Zep	α (질의 문자열, § 3)	q
Zep	β (출력 문맥 문자열, § 3)	$\text{ret}(E, q)$
Zep	S (문자열 집합, § 3)	Σ^*
Zep	f (그래프 검색 API — 생성 아님, § 3)	$\text{ret}(E, q)$
Zep	φ / ϕ (부착 함수 / 검색단계, § 2·§ 3)	φ / srch
Zep	χ (문맥 구성기, § 3)	cons
Zep	$t_{\text{ref}}, t_{\text{valid}}, t_{\text{invalid}}, t'_{\text{created}}, t'_{\text{expired}}$	t^{wall}, t'
Zep	T, T' (두 타임라인, § 2.1)	T, T' (유지)
Zep	n (직전 메시지 수 / BFS 홑 / 리 스트길이)	$n_{\text{ctx}} = 4 / h / \cdot $
Zep	G, G_e, G_s, G_c (그래프와 세 계층)	$E, \mathcal{G}_e, \mathcal{G}_s, \mathcal{G}_c$
Zep	N, N_e, N_s, N_c (노드 집합)	$\mathcal{N}_e, \mathcal{N}_s, \mathcal{N}_c$
Zep	e_i, n_i (개별 간선 · 노드)	유지
ReasoningBank	M (기억 모듈, § 3.1)	E
ReasoningBank	L (backbone LLM, § 3.1)	Θ (frozen)
ReasoningBank	$\pi_L(\cdot M, A)$ (에이전트 정책)	$f(q; \Theta, \cdot, \text{ret}(E, q))$
ReasoningBank	$\mathcal{H}$ (단일 궤적, Fig. 2)	$\mathcal{H}_k^\Delta$ (라운드 증분)
ReasoningBank	k (MaTTS 스케일 인자, § 3.3)	m
ReasoningBank	k (검색 top- k , 기본 1, App A.2)	$n_{\text{ret}} = 1$
ReasoningBank	t (궤적 내부 스텝)	t (틱 = 환경 상호작용 1 회)
ReasoningBank	N (과제 스트림 길이, § 3.1)	$N - N_q$ 와다름
ReasoningBank	q_i, Q (과제 질의 · 질의열)	q_k
ReasoningBank	$\mathcal{A}/A$ (행동 공간), $T(\cdot)$ (전이)	유지
ReasoningBank	SR, AS (성공률 · 평균 스텝, App B.1)	유지 — AS 는 L_w 가 아니다
SCM	W (7 칸 working memory 버퍼, § 3.4)	E_{wm}
SCM	$S(c)$ (보존 점수, Eq. 9)	$r_{\text{ret}}(c)$ — 맨 r 은 rank 예약이므로 첨자를 단다
SCM	s_{ij} (간선 강도, Eq. 6–7)	s_{ij} 유지 — 장-국소 행. 첨자를 둘 달아 step 첨자 s 와 구분한다
SCM	$\eta = 0.1$ (Hebbian 강화율, Eq. 6)	η_{heb} — inner 의 $\eta_t \cdot \text{sleep}$ 의 η_Θ 와 다르다
SCM	$\alpha = 0.8$ (시냅스 축소, Eq. 7)	α^E
SCM	$\beta_1 = 0.8, \beta_2 = 0.2$ (보존 점수 혼합, Eq. 9)	$w_{\text{imp}}, w_{\text{rec}}$
SCM	$\lambda = 0.01$ (최근성 감쇠, Eq. 9)	λ_{rec}

논문	원 기호 (원문에서의 뜻)	이 책의 표기
SCM	$\rho(G)$ (모순 간선 밀도, § 3.6)	ρ_{cf}
SCM	$\theta_e, \theta_c, \theta_f$ (트리거 · 삭제 임계, § 3.6)	$\vartheta_e, \vartheta_c, \vartheta_f$
SCM	e_c (384 차원 문장 임베딩, Eq. 1)	$x_c \in \mathbb{R}^{384}$
SCM	e (버퍼의 episode 객체, Alg. 1)	산문 ‘에피소드’
SCM	M (기존 개념 집합, Eq. 1)	E_{ltm}
SCM	E ($ \mathcal{E}_{\text{contradicts}} / E $ 의 분모, § 3.6)	$\mathcal{E}$
SCM	σ_I (중요도 표준편차, Eq. 10) / σ (활성함수, Eq. 2)	$\text{sd}(I) / \sigma(\cdot)$
SCM	c (개념, Eq. 1–5)	c (장-국소 재정의; 이 장에 Ω 규칙 없음)
SCM	G (장기 기억 그래프)	$E_{ltm}, G \rightarrow \mathcal{V} $
SCM	$H(W)$ (버퍼 중요도 분포의 엔트로피)	$H(E_{wm})$
SCM	$\tau = 1$ 시간 (경과 시간 트리거)	τ_{sleep}
Memini	w_{fast} (읽히는 간선 변수)	E_e^f
Memini	w_{slow} (읽히지 않는 간선 변수)	E_e^s
Memini	C (두 변수의 결합 강도, Eq. 1–2)	μ
Memini	E (그래프 간선 집합) / V (노드 집합)	$\mathcal{E} / \mathcal{V}$
Memini	$u_i^{(t)}$ (확산 활성값, Eq. 3)	$a_i^{(t)}$
Memini	δ (반복당 감쇠, $(1 - \delta)$ 로 등장)	$\alpha_{sa} := 1 - \delta$
Memini	S (전역 확산 계수, Eq. 3)	s_{sp}
Memini	b (이진 입력 크기, = 1)	b_{in}
Memini	n (Benna–Fusi 사슬 길이 / 공기 이벤트 수)	$n_{\text{BF}} = 2 / N_{\text{ev}}$
Memini	T (확산 반복 수) / top- k	$T_{\text{sa}} / k_{\text{ret}}$
Memini	$\tau_{\text{fast}}, \tau_{\text{slow}}$	τ_f, τ_s (유지)

이 표에서 가장 위험한 두 행을 따로 짚는다. 첫째, Zep의 f 는 (R)의 f 가 **아니다**. Zep의 $f(\alpha) = \chi(\rho(\phi(\alpha))) = \beta$ 는 질의 문자열을 문맥문자열로 바꾸는 검색 API이고, 답을 만드는 모델은 그 바깥에 있다 [Zep § 3]. 둘을 겹쳐 읽으면 Zep이 답을 생성한다고 오해하게 된다. 둘째, Memini의 fast/slow는 이 책의 W/Θ 가 **아니다**. Memini의 두 변수는 외부그래프 간선에 붙은 스칼라 쌍이고, 모델 안에는 아무것도 없다. 같은 낱말이다른 층을 가리킨다.

15.4 어느 층을 언제쓰는가

다섯 시스템 전부 (U-E)만 실행한다. (U- Θ)는 항등사상이고 (U-W)는 W 가 존재하지 않으므로 정의되지 않는다. 그래서 층의 순서문제는 발생하지 않는다. 대신 발생하는 것은 **시계 (時計)의 문제**다—쓰기가 어느 시점에 걸려 있는가.

이 장에서 k 는 쓰기 라운드 첨자다. 다섯 시스템에서 한 라운드가무엇인지가 다르고, 그중 실제로 유희 시간에 도는 것은 일부뿐이다.

표 15-3. 어느 층을, 어느 시계로, 어떤 순서로

시스템	실행 규칙	라운드 k 의 단위	요청 경로 위인가	진짜 오프라인 성분
Mem0	(U-E)	메시지쌍 (m_{k-1}, m_k)	위 — ingestion 경로에서 동기 실행	비동기 요약 갱신 1 종 [Mem0 § 2.1]
Zep	(U-E)	에피소드 1 건 = 메시지 1 건	위 — 메시지 도착 에결합	주기적 community 전 면 재계산 [Zep § 2.3]
Reasoning- Bank	(U-E)	과제 1 건	아래 — 과제 종료 후, 다음 질의 전	판정 + 추출 2 회 호출
SCM	(U-E)	트리거 발화 1 회	추출은 위, 재조 직 · 삭제는아래	NREM·REM· prune 전 과정
Memini	(U-E)	문서 1 건	아래 — 문서 수집 시점	전 간선 Euler 1 스텝

Mem0 과 Zep 은 ch01 의 조작적 정의 중 “질의 q 가 도착하기 전” 은만족하지만 “유휴 시간 에” 는 만족하지 않는다. Zep 의 쓰기는 메시지 도착률에결합되어 있고, Mem0 의 추출 · 갱신은 “incremental processing paradigm, enabling it to operate seamlessly within ongoing conversations” [Mem0 § 2.1] 이라고 논문 자신이 밝힌다. 두 시스템에서유일하게 배치 작업의 모양을 갖는 것은 Mem0 의 비동기 요약 모듈과 Zep 의 community 전면 재계산인데, **둘 다 주기가 공개되어 있지 않고 둘 다어떤 실험에서도 돌지 않았다**. Zep 은 그 필요성을 스스로 인정한다 — “the resulting communities gradually diverge from those that would be generated by a complete label propagation run. Therefore, periodic community refreshes remain necessary” [Zep § 2.3].

SCM 은 유일하게 트리거를 상태의 함수로 만든다. 엔트로피 $H(E_{wm}) > \vartheta_e = 0.9$, 모순 밀도 $\rho_{cf} > \vartheta_c = 0.3$, 경과 시간 $\Delta t > \tau_{sleep} = 1$ 시간, 또는 수동 호출 중 하나면발화한다 [SCM § 3.6]. 서빙 시스템에 맞는 모양이다. 다만 로그의 밀이명시되어 있지 않고, 7 칸 버퍼에서 항목이 균등 중 요도일 때 $H = \log k$ 이므로 nat 이면 $k \geq 3$, bit 이면 $k \geq 2$ 에서 이미 임계를넘는다. 논문은 어떤 실험에서 sleep 이 몇 번 돌았는지 보고하지 않는다.

“이 값은 누가 학습하는가?” 에 대한 답은 다섯 논문 전체에서 하나다 —**아무도 학습하지 않는다**. 정확히 세 부류만 존재한다. (a) 남이사전학습해 동결한 체크포인트, (b) 사람이 손으로 고른 상수, (c) 그 동결모델이 프롬프트에 따라 뱉은 텍스트. Mem0 의 $m_{rec} = 10$ 과 $s_{nn} = 10$ 은 한 번 정해지고 한 번도 스왑되지 않았으며, 노드병합 임계 t_{sim} 과 삼중항 관련도 임계는 “configurable” 이라고만 적히고 값이 공개되지 않았다 [Mem0 § 2.1–2.2]. SCM 의 중요도 가중치 (0.30, 0.20, 0.35, 0.15) 는 “determined through ablation on the benchmark suite” [SCM § 3.3] 이고, $w_{rec} = 0.2$ 는 “grid search over β_2 in [0.1, 0.5], selecting the value that maximized noise reduction” [SCM § 3.6] 이다. Memini 의 결정적 파라미터인 시상수 비 $\tau_s/\tau_f = 5$ 는 “chosen so that the three expected regimes are clearly distinguishable within the 13-step window” [Memini App. A.2] 다.

[해설] training 을 처음 배우는 독자에게 이 절은 Part II 에서유일하게 “누가 학습하는 가” 의 답이 “사람이 숫자를 골랐다” 인 자리다. 이것자체는 결함이 아니라 E -경로의 정의적 성질이다 ($\rightarrow$ ch11). 결함이 되는지점은 § 15.6 이다 — 그 숫자를 고른 벤치마크와 그 숫자로 낸 결과를 보고한벤치마크가 같을 때다.

ReasoningBank 의 MaTTS 는 이 표에서 유일하게 예산의 성격이 다르다. 병렬스케일링은 같은 질의에 대해 m 개의 궤적을 굴린 뒤 그 대조를 한 번의프롬프트로 요약해 E 에 넣는다 [ReasoningBank § 3.3]. 통일 표기로쓰면 $\mathcal{H}_k^\Delta = \text{gen}(q_k, \text{ret}(E_k, q_k); B_s^{\text{roll}})$ 다음에 $\hat{c}_k = S(\mathcal{H}_k^\Delta; B_s^{\text{sum}})$ 이다. 즉 예산의 일부가 **요약할 경험을제조하는 데** 쓰인다. $\text{gen}(\cdot; B_s)$ 는 (U- Θ) 의부품인데, ReasoningBank 은 그것을 만들어놓고 gradient 대신 프롬프트로보낸다. 이 한 걸음의 차이가 E -경로와 Θ -경로를 가른다.

15.5 비용 4 종

표 15-4. 다섯 시스템의 $B_s \cdot L_w \cdot C_{\text{cap}} \cdot \rho$

시스템	B_s	L_w	C_{cap}	ρ
Mem0	논문에 없음 — 유일한 구축 비용 진술은 “graph construction completes in under a minute even in worst-case scenarios” [§ 4.5]	search p50 0.148 s / p95 0.200 s; total p50 0.708 s / p95 1.440 s. Mem0 ^g 0.476/0.657, 1.091/2.590. full-context total 9.870/17.117 [Table 2]	토큰 대응치만: 저장소 ~7k(Mem0) / ~14k(Mem0 ^g) / >600k(Zep) / 원문 ~26k [§ 4.5]. 바이트·dtype·임베딩 차원 모두 논문에 없음	논문에 없음
Zep	논문에 없음 — 토큰·FLOP·시간·호출 수 전부 없음	LongMemEval _s end-to-end: gpt-4o-mini 31.3 s → 3.20 s (IQR 8.76 → 1.31), gpt-4o 28.9 s → 2.58 s (IQR 6.01 → 0.684) [Table 2]	논문에 없음 — 노드·간선 수, 저장 바이트 전무. 1024 차원 임베딩만 언급되고 dtype 없음 [§ 2.2.1]	논문에 없음
Reasoning-Bank	과제당 3748.4 토큰 = judge 2186.3 + extraction 1562.1. 총 53054.5 vs No Memory 50847.4 [Table 5, § C.2]. MaTTS 구성의 B_s 는 논문에 없음	논문에 없음 — ms·TTFT·시간 전무. 대응치는 평균 스텝 수 AS 뿐	논문에 없음 — 항목 수·바이트·임베딩 차원 전무. 상한 규칙만 존재 (과제당 3 개)	논문에 없음
SCM	논문에 없음 — 벤치 전체 wall-clock 약 12 분 (8 개 시험 × 5 회) 이 유일한 수치 [§ 3.8]	검색만: 개념 10 개에서 0.1 ms 미만, 360 개에서 0.3 ms 미만 (Apple M1 MacBook Air, 질의 1000 회) [§ 4.3]. 지배적 비용인 wake 추출 LLM 호출은 추정 안 함	개념 수만: SCM 24 / No-Forget 72 / Vector DB 55 / FIFO 7 [Table 2]; 자연 시험 360 개; 기본 target_size 100 [§ 3.6]. 바이트 논문에 없음	논문에 없음 — 90.9%는 의도된 삭제율이 지 열화율이 아니다

시스템	B_s	L_w	C_{cap}	ρ
Memini	논문에 없음 — 실제로 돌린 실험에서는 LLM 추출을 문자열매칭으로 대체해 토큰 비용이 0 이었다 [App. A.2]	논문에 없음 — 읽기 경로를 한 번도 실행하지 않았다 [App. A.5]	형태만: 방향 간 선당 스칼라 2 개; 실험은 개체 20 개 · 고유 쌍 68 개 [§ 3.1, App. A.2]. 바이트 · 정밀도 논문에 없음	감쇠 시상수 $\tau_f = 2$, $\tau_s = 10$ (문서단위). 성능 손실률은 논문에 없음

네 칸 중 ρ 는 다섯 논문 전부에서 비어 있고, C_{cap} 은 바이트 기준으로 다섯 전부에서 비어 있다. B_s 는 넷에서 비어 있고 ReasoningBank 하나만 토큰으로 채워져 있다. L_w 는 둘에서 비어 있고 하나 (SCM)는 지배적 성분을 빼고 채워져 있다.

ρ 의 부재가 특히 심각한 두 자리가 있다. Mem0은 이 코퍼스에서 상태를 되돌릴 수 없이 파괴할 수 있는 첫 E-경로 시스템인데 [Mem0 Alg. 1], ADD/UPDATE/DELETE/NOOP 분류의 정확도, 오삭제율, 회차에 따른 열화 곡선을 하나도 측정하지 않는다. 게다가 추출이 저장소에서 가져온 요약에 조건화되므로 (15-2)의 되먹임 고리가 드리프트를 누적시킬 바로 그 구조인데, 논문은 각 대화를 한 번만 수집하고 재수집 실험을 하지 않는다. SCM은 임계 미달 노드를 삭제하면서, 20 사이클 결과를 유일하게 제시하는 Figure 3을 논문 스스로 “simulated memory growth over twenty sleep cycles” [SCM § 4.3]라고 부른다. 측정된 결과는 전부 sleep 1회 이내다.

상각식 (A)로 손익분기 N_q^* 를 계산할 수 있는 논문은 다섯 중 하나도 없다. $C_{sleep} = B_s$ 가 넷에서 비어 있어 분자가 없고, 남은 하나인 ReasoningBank도 N_q — 한 문맥을 공유하는 질의 수 — 를 보고하지 않는다. ReasoningBank에서 각 과제는 자기 자신의 문맥이므로 $N_q = 1$ 이고, 재사용은 과제들 사이에서 top-1 코사인 검색으로 일어나는데, 저장된 항목이 몇 번 검색되었는지에 대한 분포가 없다. 계산할 수 있는 것은 라운드 1회의 오버헤드 비뿐이다: $3748.4/49306.1 = 7.60\%$ (이 책의 산술, [ReasoningBank Table 5]의 값에서). 이것은 N_q 에 대한 상각이 아니라 한 라운드의 회계다.

Zep의 경우는 더 날카롭다. DMR 프로토콜은 “500 multi-session conversations, each containing 5 chat sessions with up to 12 messages per session. Each conversation includes a question/answer pair” [Zep § 4.2]다. 최대 60개 메시지를 아홉 단계 파이프라인으로 전부 수집한 뒤 질의 하나로 상각한다. (A)에서 $N_q = 1$ 이면 $C_{avg} = C_{wake} + B_s$ 이고, B_s 는 인쇄되지 않았다.

[평가] 비용 4종 표의 빈칸은 우연이 아니다. 다섯 논문 모두 읽기 쪽 비용은 정밀하게 세고 쓰기 쪽 비용은 세지 않는다. Zep은 이 비대칭을 스스로 선언한다 — “we’ve focused heavily on the accuracy, latency, and scalability of its memory retrieval mechanisms” [Zep § 1, 강조는 이 책]. 그러면서 결론에서는 “while reducing token costs” [Zep § 5]라고 쓰는데, 논문에 있는 토큰 수치는 전부 읽기 쪽 프롬프트 크기다. 이 장의 집필 규칙상, 비용 칸이 비어 있는 시스템에 대해 “효율적”이라고 쓸 수 없다. 쓸 수 있는 것은 “읽기 쪽이 싸다” 뿐이다.

15.6 실험과 스케일

증거 기반부터 정직하게 적는다. Mem0은 LOCOMO 대화 10건이 전부이고, 각 대화는 평균 약 600 토큰 · 약 26000 토큰 · 약 200 문항이다 [Mem0 § 3.1]. 모든 호출 지점이 GPT-4o-mini 한 모델이고, 판정자 모델은 “a separate, more capable LLM” [Mem0 § 3.2] 이라고만 적힌 채 끝까지 이름이 나오지 않는다. Zep은 DMR 대화 500건 (각 최대 60 메시지, 문항 1개)과 LongMemEval_s(평균 약 115,000 토큰)를 쓴다 [Zep § 4.2 – 4.3]. ReasoningBank은 WebArena 684건 (Map 도메인 제외; Shopping 187 / Admin 182 / Gitlab 180 / Reddit 106 / Multi 29), Mind2Web 1341건, SWE-Bench-Verified 500건을 쓴다 [ReasoningBank App B.1 – B.2]. 셋 다 남이 만든 벤치마크를 쓴다 — ch14와 비교할 때 실질적 개선이며, 의무 caveat가 둘을 뭉뚱그리지 않도록 명시

해 둔다. 나머지 둘은 자체 제작이다. SCM의 여덟 시험과 세 baseline은 모두 저자가 만든 단순 재구현이고 [SCM § 4.2], Memini는 저자가 고른 COVID-19 위키백과 13 편에 저자가 미리 정한 개체 20 개를 엮었다 [Memini App. A.1 – A.2].

첫 번째 증거 — Mem0. Table 2의 J 순위는 full-context $72.90 \pm 0.19\% > \text{Mem0}^g 68.44 \pm 0.17\% > \text{Mem0} 66.88 \pm 0.15\% > \text{Zep} 65.99 \pm 0.16\%$ 다. 기억 구조가 전혀 없는 구성이 1 위이고, 격차는 base 대비 6.02 점, 그래프변형 대비 4.46 점으로 보고된 ± 1 표준편차의 바깥에 한참 있다. 논문도 본문에서 인정한다 — “a full-context method ... still achieves the highest J score (approximately 73%)” [Mem0 § 4.3]. 그런데 초록은 순서를 뒤집어 “Beyond accuracy gains, we also markedly reduce computational overhead” 라고 쓴다. 더 나아가, 이 논문의 헤드라인 확장인 그래프 기억은 절반의 범주에서 **더 나쁘다**: single-hop J 65.71 vs 67.13, F1 38.09 vs 38.72; multi-hop J 47.19 vs 51.15, F1 24.32 vs 28.64 [Mem0 Table 1]. 저자들은 이를 그대로 적으면서 원인을 분리하는 ablation 을 제시하지 않는다 — “the addition of graph memory in Mem0^g does not provide performance gains here, indicating potential inefficiencies or redundancies in structured graph representations” [Mem0 § 4.2]. 그리고 그 확장은 검색 지연 $p50\ 0.148\text{ s} \rightarrow 0.476\text{ s}$, 검색 토큰 $1764 \rightarrow 3616$ 을 치른다 — 이 책의 산술로 각각 3.22 배와 2.05 배다. 초록이 “consistently outperform all existing memory systems across four question categories” 라고 쓰는 동안, open-domain에서는 Zep이 F1 49.56 과 J 76.60 으로 Mem0^g의 49.27·75.71 을 이기고 있고, 결론 [Mem0 § 5]는 네 범주 중 open-domain 만 조용히 빼고 세 범주의 상대개선만 나열한다.

두 번째 증거 — Zep. 이 논문에는 **ablation 이 하나도 없다**. 스스로 novelty 로 내세운 이중 시간 표현 — “This bi-temporal approach represents a novel advancement in LLM-based knowledge graph construction” [Zep § 2.1] — 을 뺀 변형을 한 번도 돌리지 않았고, reflexion 패스도, 개체 해소 단계도, community 계층도 분리되지 않았다. 그 상태에서 DMR 결과를 보면, gpt-4-turbo 에서 full-conversation baseline 94.4%가 이미 비교 대상 MemGPT 93.4%를 넘고, Zep은 94.8%로 그 위에 0.4%p 를 얻는다. gpt-4o-mini에서는 full-conversation 98.0%, Zep 98.2%로 0.2%p 다. 대화 500 건에서 0.2%p 는 대화 한 건이다. 신뢰구간도, 시드도, 반복 실행도, 유의검정도 논문 어디에도 없다. 게다가 비교 상대인 MemGPT 93.4%는 인용값이다 — Table 1이 “† Results reported in [3]” 로 표시하고, 본문은 “We were unable to reproduce MemGPT’s results using gpt-4o-mini” [Zep § 4.2] 라고 적으며, LongMemEval_s에서는 MemGPT를 아예 돌리지 못했다 [Zep § 4.3.1]. 직접 비교는 존재하지 않는다. 분모도 일관되지 않다 — Table 3의 상승 델타는 baseline 을, 하락 델타는 Zep 자신을 분모로 쓰고, [Zep § 4.3.2]가 gpt-4o에 대해 내세우는 18.5%는 Table 2의 값으로 $(71.2 - 60.2)/60.2 = 18.27\%$ 다 (이 책의 산술). 초록은 18.5%를 그대로 옮긴다. 저자들은 “showing marginal improvements over both MemGPT and the respective full-conversation baselines” [Zep § 4.2] 라고 적고, 같은 절에서 이 벤치마크가 부적절하다고 스스로 판정한다 — “The high performance achieved by simple full-context approaches using modern LLMs further highlights the benchmark’s inadequacy for evaluating memory systems” [Zep § 4.2]. 더 결정적인 것은 LongMemEval_s의 범주별 결과다. gpt-4o-mini에서 knowledge-update는 $76.9\% \rightarrow 74.4\%$ 로 **떨어진다** [Zep Table 3]. knowledge-update는 사실이 바뀔 때 저장소가 버티는가를 재는 범주, 즉 무효화 기제가 존재하는 바로 그 이유다. 같은 표에서 single-session-assistant는 두 모델 모두에서 퇴행한다 ($81.8 \rightarrow 75.0$, $94.6 \rightarrow 80.4$). 논문은 후자만 언급하고 [Zep § 4.3.2], 전자는 언급하지 않는다. 그리고 gpt-4o에서는 knowledge-update가 $78.2\% \rightarrow 83.3\%$ 로 오른다 — **이득의 부호가 생성 모델의 성능에 따라 바뀐다**. 원인은 § 15.3.2에서 이미 봤다. 유효 구간이 프롬프트 안의 문자열로 전달되므로, 그것을 쓰는 일은 검색의 보장이 아니라 추론자의 과제다. 논문자신의 표현으로 “additional development may be needed to improve less capable models’ understanding of Zep’s temporal data” [Zep § 4.3.2] 이다.

세 번째 증거 — ReasoningBank. 헤드라인은 실재한다. WebArena 전체 SR이 Gemini-2.5-flash에서 $40.5 \rightarrow 48.8$, Gemini-2.5-pro에서 $46.7 \rightarrow 53.9$, Claude-3.7-Sonnet에서 $41.7 \rightarrow 46.3$ 이고 [ReasoningBank Table 1], 평균 스텝은 $9.7 \rightarrow 8.3$, $8.8 \rightarrow 7.4$, $8.0 \rightarrow 7.3$ 으로 줄며, SWE-Bench-Verified에서도 $34.2 \rightarrow 38.8(\text{flash})$, $54.0 \rightarrow 57.4(\text{pro})$ 다 [Table 2]. 문제는 그 다음이다. [ReasoningBank § C.1]은 검색해 넣는 경험의 개수를 0에서 4까지 늘리며 SR을

젠다: 39.0, 49.7, 46.0, 45.5, 44.4 [Figure 13]. 하나를 넘어가면 **단조 악화**다. 그리고 출하기본 값은 $n_{\text{ret}} = 1$, 즉 0 이 아닌 최소값이다 [App A.2]. 이 곡선은 “경험이라는 새로운 스케일 축” 이라는 논문의 틀과 정면으로부딪히는데, 논문의 해석은 “excessive experiences may introduce conflicts or noise” 에서 멈춘다. 일반화 주장의 근거도 쏘다. [ReasoningBank § 4.2] 가 내세우는 “최강 baseline 대비 평균 +4.6” 은 Multi 부분집합에서나오고, 그 부분집합은 **29 인스턴스**다 — 한 과제가 3.4 점이다. Claude-3.7-Sonnet 의 Multi 열은 0.0 / 0.0 / 0.0 / 3.4 / 10.3, 즉 0 건 · 0 건 · 0 건 · 1 건 · 3 건이다. Mind2Web 의 과제 수준 SR 은 바닥에 붙어 있다 (cross-domain flash 1.0 $\rightarrow$ 1.6, pro 1.4 $\rightarrow$ 1.7). 이 모든 수치에 시드도, 반복 실행도, 신뢰구간도 없다. 에이전트는 temperature 0.7, 추출자는 1.0 으로 디코딩하므로 [ReasoningBank App A.2, App B.1] 인쇄된 값은 각각 한 번의 확률적추출이고, 29 건 · 106 건짜리 부분집합의 델타는 과제 한두 건안에 들어간다. 산문이 자기 표와 어긋나는 자리도 있다 — [ReasoningBank § 4.4] 는 “scaling actually reduces performance for weaker memories, where Synapse slightly increases from 40.6 to 41.2, and AWM from 44.4 to 45.5” 라고 쓰는데 인용된 두 수치는 모두 상승이다. 마지막으로, 모든 쓰기를여닫는 자기 판정기의 실제 정확도는 **72.7%**다 [ReasoningBank § 5]. 그 견고성을 재는 실험은 실제 판정기를 쓰지 않고 정답 라벨을확률적으로 뒤집어 만든 가상 판정기를 쓴다 — “A 90% accurate verifier is simulated by using the correct (ground-truth) label 90% of the time and the incorrect (flipped) label 10% of the time” [§ 5]. 이 잡음은독립이고 대칭이다. 실제 판정기의 오류는 그렇지 않다. 게다가 100% 지점은정답 라벨을 그대로 쓰는 오라클이며, 이는 “no ground truth is available during test-time” [§ 3.1] 이라는 이 방법의 전제를 그 자리에서소비한다.

네 번째 증거 — SCM. 제목이 consolidation 을 말하지만 가중치는 한번도 바뀌지 않는다 — “It requires no training or fine-tuning” [SCM § 3.8]. ch11 의 판별식으로 보면 이 논문은 (U-E) 를 실행하므로 sleep-time compute 이지만, 제목이 약속하는 Θ 갱신은 존재하지 않는다. 그리고자기 표를 그대로 읽으면 여기서도 같은 일이 벌어진다. Table 2 에서아무것도 지우지 않는 No-Forget Graph 는 22/22(100%) 로 SCM 과**동률**이고, SCM 이 이기는 것은 저장소 크기 24 대 72 뿐이다. 즉품질에서 삭제 기제의 기여는 0 이고, 기여는 전부 용량 쪽에 있다. 여덟시험은 모두 정확히 1.00 이고 다섯 번 실행에서 분산이 0 이다 [SCM Table 3, § 4.1] — 포화된 벤치마크는 대안을 순위 매길 수도, 퇴행을 잡아낼 수도 없다. 논문도 난이도를 인정한다 (“This represents a lower bound on memory system capability” [§ 4.3]). 헤드라인 숫자 90.9%는 두 개의 서로 다른 개수에 동시에 붙어 있다 — Table 3 은 “50/55 noise concepts removed (90.9%)” 이고 [SCM § 4.3] 과 Figure 5 는 “removing forty-five of the fifty noise concepts” 인데 45/50 은 90.0%다. 이 숫자를 만든 계수는벤치마크 위에서 격자 탐색으로 정해졌고, 그 이전 값 $w_{\text{rec}} = 0.4$ 에서는 노이즈 제거율이 0%였다 [§ 3.6, § 4.3]. 용량을 묶는다는 핵심 기제 Eq. 10 은 인쇄된 형태 그대로는 본문의 주장과반대 방향으로 움직인다 — $\vartheta_f = \bar{I} - \text{sd}(I) \cdot |G| / \text{target_size}$ 에서 $\text{sd}(I) \geq 0$ 이면 $|G|$ 가 커질수록 임계가 **낮아진다**. Eq. 9 도 같은 상태다. $\delta(c) = \exp(-\lambda_{\text{rec}} \Delta t)$ 이면 방금 접근한개념은 $1 - \delta = 0$ 인데, [SCM § 3.6] 은 고쳐낸 버그를 “ $1 - \delta(c) \approx 1$ 인 새 개념을 부양했다” 로 설명한다. 같은 절의 $\lambda_{\text{rec}} = 0.01$ 교정 (“한 시간 전 개념이 최근성 점수의 약 96%를 유지”) 도 Δt 의 단위가 없어 재현되지 않는다 — 시간 단위면 99.0%, 분 단위면 54.9%다 (이 책의 산술). 코드는 공개되지 않았다.

다섯 번째 증거 — Memini. 유일한 실험에서 LLM 을 한 번도 돌리지않았다. 설계상 쓰기 경로는 LLM 개체 추출인데 [Memini Fig. 1(a)], 실제로 쓴 것은 “case-insensitive word-boundary string matching against the 20 predefined terms and a small set of hand-curated aliases” [App. A.2] 다. 그래프도 자라지 않았다 — 노드 20 개가 실행 전에 손으로고정되었다. 읽기 경로는 아예 실행되지 않았고, 확산 활성의 네하이퍼파라미터 ($\alpha_{\text{sa}}, s_{\text{sp}}, T_{\text{sa}}, k_{\text{ret}}$) 중 값이 주어진 것은 하나도 없다. 저자들이 그렇게 적는다 — “we report no retrieval metrics or comparison against published systems” [App. A.5]. 남은 정량 결과 Table 3 에서 헤드라인 “약 30 배” 는 $N = 5$ 쌍에서 0.104 대 0.003 이고, 세 구성의 지표가 서로 비교가능하지 않다 (Uniform 열은 사실상 그룹별 평균 이벤트 수라서 모든 행에서가장 크다). 검색이 활성값으로 순위를 매긴다는 사실을 함께 놓으면 더곤란한 결과가 드러난다: 이 기제가 존재하는 이유인 “반복되었으나 최근에언급되지 않은” 그룹은 0.104 로 끝나고, “드물게 언급되었으나최근인” 그룹은 0.463 이다. 논문은 이 순위역전을 논하지 않는다.

표 15-5. 다섯 시스템의 실증 상한

시스템	데이터	모델	저장소 최대	반복 라운드
Mem0	LOCOMO 대화 10 건, 단일 데이 터셋	GPT-4o-mini 단일	~14k 토큰 (Mem0 ^g)	1 회 수집, 재수집 없음
Zep	DMR 500 건 + LongMemEval _s (~11k 토큰)	gpt-4o-mini / gpt-4o / gpt-4-turbo	미보고	1 회 수집, 장기 실행 없음
Reasoning- Bank	WebArena 684 + Mind2Web 1341 + SWE-Bench-V 500	Gemini-2.5- flash/pro, Claude-3.7, Gemma-3-12B	미보고 (과제당 항목 ≤ 3)	과제 스트림 최대 500
SCM	저자 제작 8 시험, 10 턴 대화 22 사실	Llama 3.2(2B, Q4_K_M) + MiniLM-384	개념 360 개 (지 연 시험만)	측정은 1 사이클, 20 사이클은 시물 레이션
Memini	위키백과 13 편, 개체 20 개, 이벤 트 124 건	없음 — 문자열 매칭	고유 쌍 68 개	Euler 13 스텝

15.7 Systems/serving 합의

decode 에서 바뀌는 것은 없다. 다섯 시스템 전부 Θ 가 동결되어있으므로 요청별 가중치 상태가 생기지 않고, **shared-weight batching** 이 그대로 성립한다. Rosetta 사전의 “batch 로 weight 공유 $\rightarrow \Theta$ -경로에서 깨짐” 행은 여기서 적용되지 않는다 ($\rightarrow$ ch01). 백만 명의 사용자가 한 벌의 가중치를 공유하고, 차이는 데이터베이스가 어떤 행을 돌려주는지에만 있다. 이것이 E -경로의 구조적 서빙 이점이고, 다섯 논문중 아무도 명시하지 않는다.

읽기 쪽에서 실제로 바뀌는 것은 **prefill 길이**다. Mem0 은 질의당문맥을 26031 토큰에서 1764 토큰으로 줄이고 (이 책의 산술로 14.76 배), 총 지연 p95 는 17.117 s 에서 1.440 s 로 줄어든다 (11.89 배) — 두 비가 가까운것은 이 길이대에서 지연이 prefill 지배적임을 시사한다. Zep 은 115k 에서 1.6k 로 줄인다 (이 책의 산술로 71.9 배). 어느 논문도 prefill 과 decode 를분해하지 않고, 하드웨어 · 배치 크기 · 동시성을 밝히지 않으며, Zep 은 보스턴의 소비자용 노트북에서 AWS us-west-2 의 호스팅서비스에 접속해 측정했다고 공개한다 [Zep § 4.3]. 네트워크 지연이 Zep 쪽에만 붙었으므로 방향은 Zep 에 불리하지만, 두 팔이 프롬프트 길이에서 71.9 배 차이 나는 이상 이 비교는 기억 구조의 비교가 아니라 프롬프트길이의 비교다.

[평가] 이 절에서 반드시 짚어야 할 공백이 하나 있다. Mem0 전문에 “cache” 라는 단어는 한 번도 나오지 않는다. 그런데 비교 대상인 full-context baseline 은 같은 대화에 대한 약 200 개 질의마다 동일한 26k 프리픽스를 다시 prefill 한다 [Mem0 § 3.1, § 4.3]. prefix caching 이 걸린서빙 스택에서는 그 prefill 이 한 번만 청구된다. 91%/92%라는 지연 절감의 얼마가 아키텍처의 몫이고 얼마가 baseline 구성의 몫인지는 이 논문에서 판정할 수 없다. 반대로 Mem0·Zep 의 검색 결과는 질의마다 달라 prefix cache 로 재사용되지 않고, ReasoningBank 은 검색 항목을 시스템 지시문 맨 앞에붙이므로 과제 간 공유 프리픽스를 위치 0 에서 깨뜨린다 [ReasoningBank App A.2]. 셋 다 논문이 다루지 않는다.

쓰기 쪽은 메시지 속도로 도는 두 번째 추론 워크로드다. Zep 은 메시지 하나마다 여섯에서 일곱 번의 frozen LLM 호출과 임베딩 · 색인 검색을 돌리고 [Zep § 2.2], Mem0 은 메시지쌍마다 추출 프롬프트 한 번과 갱신 tool call 한 번을 돌린다 [Mem0 § 2.1]. ReasoningBank 은 과제당 두 번,

합계 3748.4 토큰이다. 호의적인성질 하나는 어느 논문도 말하지 않는다 — Zep의 $S(c_k; B_s)$ 는 메시지다섯 개만 보므로 쓰기 경로는 **짧고 균일하며 배치하기 좋은프롬프트의 흐름**이고, 이는 그것이 대체하는 115k 단일 프롬프트보다 좋은 prefill 워크로드다. 불리한 성질도 아무도 말하지 않는다 — 쓰기 비용은대화 트래픽에 비례하고 읽기 절감은 질의 트래픽에 비례하므로, 순이익은전적으로 N_q 에 달려 있다.

상태가 앉는 자리도 다르다. Zep의 검색 backend는 Neo4j의 Lucene 구현이고 [Zep § 3.1] 배포는 네트워크 너머의 호스팅 서비스다. E -경로의상태는 호스트 DRAM과 SSD에, 벡터 색인과 역색인 뒤에 있고, HBM에는 평범한모델 가중치만 있다. 그래서 이 경로의 지연 하한은 커널이 아니라 네트워크왕복이다 — Zep의 상대 산포가 baseline보다 나쁜 것이 그 흔적이다 (IQR/지연 = $1.31/3.20 = 0.409$ 대 $8.76/31.3 = 0.280$, 이 책의 산술).

SCM과 Memini는 sleep 작업의 성격을 한 번 더 갈라놓는다. SCM의 sleep 절차에는 **LLM 호출이 하나도 없다** — Hebbian 곱, 전 간선 스칼라곱, 5 스텝 무작위 보행 3회, 임계 검사 한 번이 전부다 [SCM Alg. 1]. 이것은 GPU 학습 작업이 아니라 CPU 배경 압축 작업이고, LSM-tree의 compaction에 훨씬 가깝다. Memini의 쓰기는 문서마다 저장소 전체를 훑는 $O(|\mathcal{E}|)$ 작업이다.

[해설] Memini의 (15-7)은 LTI이므로 각 간선을 마지막으로건드린 시점에서 $z_e(t) = e^{A(t-t_{\text{last}})} z_e(t_{\text{last}})$ 로 지연계산하면 전수 훑기가 $O(|\hat{c}_k|)$ 로 줄어든다. 그리고 “Each pair is integrated independently using the same parameters” [Memini App. A.2]이므로 간선 단위 샤딩은 안전하다 — 문서 단위 샤딩은 궤적 의존성때문에 안전하지 않은데도 그렇다. 논문은 비용도 이 완화책도 언급하지않는다.

마지막으로 운영 위험 하나를 남긴다. 되돌릴 수 없는 삭제가 두 시스템에있고 (Mem0의 DELETE, SCM의 prune), 둘 다되돌리기 ·tombstone· 감사 로그가 없으며, 둘 다삭제 판단의 정확도를 측정하지 않는다. SCM의 경우 계수 하나가 잘못설정되면 삭제 기제 전체가 아무 신호 없이 무력화된 사례가 논문 안에기록되어 있다 [SCM § 4.3, Figure 5].

C_{cap} 을 바이트로 환산할 수 있는 논문이 하나도 없으므로, 이 다섯 위에 memory-device 논증을 세울 수는 없다. 격차를 기록하고 여기서멈춘다 (→ ch26, ch29).

15.8 한계와 bridge-out

다섯 시스템은 서로를 거의 인용하지 않으면서 같은 뼈대를 세 번씩 다시만들었다. 그 뼈대를 여기서 못 박는다.

정의. E -경로 프로덕션 삼단 구조는 (i) 동결 LLM 이도착한 문맥에서 후보 레코드를 뽑는 **추출**, (ii) 같은 동결 LLM 이기존 저장소와 대조해 쓰기 연산자를 고르는 **통합**, (iii) 임베딩 · 역색인 · 그래프 탐색으로 후보를 뽑아프롬프트 문자열로 직렬화하는 **검색**의 세 단계로 이루어진 구조를말한다. 학습되는 파라미터는 없고, 시스템의 실질적 ‘가중치’는 프롬프트와사람이 고른 상수다.

정의. E -경로 프로덕션 **공통 결함**은 이 구조를 채택한논문들에서 반복 관찰되는 다섯 가지 보고 결함이다. (D1) 쓰기 예산 B_s 미보고. (D2) 상태 용량을 바이트로 보고하지 않음. (D3) ablation 부재, 또는 자기 표가 자기 캡션을 반증. (D4) 기억 없는 baseline이 품질에서이기는 사실을 본문에서 인정하고 초록 · 결론에서 누락. (D5) 추출자 · 판정자 · 응답자가 같은 동결 모델인 닫힌루프에 외부 검증 없음.

표 15-6. 다섯 시스템의 공통 결함 분포

시스템	D1	D2	D3	D4	D5
Mem0	○	○	○ (제안 시스템 ablation 0 건; 스위치는 RAG baseline 에만 14 구성)	○ (§ 4.3 인 정, 초록 · 결론 누락)	○ (판정자 모델미공개)
Zep	○	○	○ (ablation 0 건)	○ (§ 4.2 인 정, 초록은 SOTA 대비만)	○ (Long-MemEval 판정자 GPT-4o, 피평가 모델과 동일 계열)
Reasoning-Bank	— (토큰으로 보고)	○	— (Figure 7 이 기여를 분리)	부분 (검색 개수 곡선을 본문에 실음)	○ (판정자 · 추출자 · 에이전트가 같은 backbone)
SCM	○	○	— (Table 4 존재; 단 두 성분이 무효과)	○ (No-Forget 이 품질 동물)	○ (벤치 · baseline · 튜닝 모두저자)
Memini	○	○	부분 (단일 시 상수 대조군 1 종)	○ (Uniform 열이 모든행에서 최대)	○ (개체 · 문서 · 순서 모두 저자선정)

논문들이 스스로 남긴 open question 은 서로 겹친다. [Mem0 § 5] 는 그래프지연 최적화, 계층적 기억, 그리고 “developing more sophisticated memory consolidation mechanisms inspired by human cognitive processes” 를 든다 —CLS 도, 재생도, 수면 신경과학도 인용하지 않고, “sleep” 이라는 단어를 한번도 쓰지 않은 채 이 책의 주제 쪽으로 손을 뻗는 한 문장이다. [Zep § 5] 는 추출기를 fine-tune 하면 E -경로 파이프라인이 짜지겠는가를 묻고 (“Similar models fine-tuned for Graphiti prompts may enhance knowledge extraction”), 이어서 이 책의 비용표와 같은 질문을 제기한 다음 그 절반만 답한다 — “Current literature on LLM memory and RAG systems insufficiently addresses production system scalability in terms of cost and latency. We have included latency benchmarks for our **retrieval** mechanisms to begin addressing this gap” [Zep § 5, 강조는 이 책]. ReasoningBank App D 는 합성 가능한 기억, 타입별 기억, 그리고 “long-term ... consolidated knowledge with **decay/refresh policies**” 를 미래 과제로 든다 — 논문자신이 ρ 에 붙인 이름이 미래 과제 목록에 있다. SCM 은 하이퍼파라미터 근거와 10,000 개 개념 천장을, Memini 는 검색을 포함한 전면검증을 남긴다.

[평가] 이 다섯은 **비용 · 지연 결과를 성능결과로 제시하고 있다**. 논증은 § 15.6 이 다섯 번 반복한 그대로다. Mem0 의 자기 표에서 품질 1 위는 full-context 이고, Zep 의 DMR 에서 자명한 full-conversation baseline 이 이미 비교 대상을 이기며 남은 마진은 0.4%p 이고, SCM 의 Table 2 에서 아무것도 지우지 않는 구성이 품질 동물이며, ReasoningBank 은 검색 경험을 하나만 넘겨도 단조 악화하고, Memini 는 검색을 한 번도 돌리지 않았다. 다섯 논문의 초록은 모두 성능의 언어로 쓰여 있고, 다섯 논문의 증거는 모두 비용의 언어로 되어 있다.

이것이 무가치하다는 뜻이 아니다. **비용 절감은 실재한다**. 질의당문맥이 26031 토큰에서 1764 토큰으로, 115k 에서 1.6k 로 줄어드는 것은 측정된 사실이고, 그것은 서버에서 곧바로 돈이다. ReasoningBank 의 과제당 3748.4 토큰 오버헤드로 WebArena 전체 SR 이 40.5 에서 48.8 로 오르는 것도 측정된 사실이다. 문제는 **주장의 종류가 바뀌어 있다**는 것이다. “우리 시스템이 더 정확하다” 와 “우리 시스템은 훨씬 싼 값에 거의 그만 큼 정확하다” 는 다른 주장이고, 후자를 방어하려면 싼 값 쪽 — 즉 B_s — 을 세어야 한다. 다섯 중 넷은 그것을 세지 않았다. 그러므로 이 장의 결론은 “이 시스템들이 나쁘다” 가

아니라 “이 시스템들의 주장은 현재형태로는 검증 대상이 아니다”이며, 검증 가능한 형태로 바꾸는 일이 Part III의 과제다 (→ ch24, ch25, ch27).

경로 간 인용. 침묵은 거의 완전하고, 방향이 셋이다. Θ -경로로: Mem0의 참고문헌에는 LoRA도, adapter·PEFT도, 모델 편집도, 지속학습도, 증류도, 용량연구도 없다. ReasoningBank은 gradient를 한 번도 취하지 않으면서 $\mathcal{R}_k$ 에 해당하는 자기 생성·자기 라벨링된 성공/실패 분할 코퍼스를 실제로 만들고, Θ -경로가 존재한다는 사실을 언급하지 않는다. SCM은 EWC를 “operates at the parameter level rather than the memory architecture level” [SCM § 2.2]로, Memini는 파라미터 지속학습을 서버 수준에서 각각 한 번씩 한다. W -경로로: Mem0의 유일한 순환 상태 인용 (Recurrent Memory Transformer)은 문제의 증거로만 쓰이고 해결책 후보로는 쓰이지 않는다. Zep 전문에서 ‘test-time’, ‘fast weight’, ‘gradient’, ‘KV’, ‘cache’는 0회다. Memini는 Titans와 Nested Learning을 한 문장에서 “validated independently”로 부르고 다시 언급하지 않는다. 생물학으로: Mem0은 어휘만 빌리고 문헌은 빌리지 않는다 (McClelland도, CLS도, 수면 신경과학도 없다). 반면 SCM과 Memini에서는 생물학 인용이 실제로 하중을 진다 — Rasch & Born, Tononi & Cirelli, Benna & Fusi, Kaplanis, McClelland, Tulving 이기제의 출처다.

[평가] 이 침묵의 가장 날카로운 형태는 인용 누락이 아니라 **재발명**이다. SCM의 Eq. 7은 외부 그래프의 간선 강도에 상수 배율을 곱해 오래된 것을 지우는데, 식 (15-6)이 보여 주듯 이것은 (U-W)의 retention gate와 대수적으로 같은 형태다. Memini의 두 시상수도 같은 계보를 외부 저장소 위에서 다시 세운다. 두 논문 모두 그 형식이 이미 W 층에서 정리되어 있다는 사실을 모르는 채로 도달했다.

연대와 선택의 구분은 **대상별로** 해야 한다. ch14를 인용하지 않은 것은 Zep과 Mem0에서만 연대가 강제된 침묵이다 — Zep은 석 달 앞서 나왔고 Mem0은 열흘 뒤다. 그러나 다섯 논문이 $W \cdot \Theta$ 경로에 대해 지키는 침묵은 어느 논문에서도 연대의 문제가 아니다. LoRA는 2021년, ROME·MEMIT은 2022년, TTT라인 전체가 Zep보다 앞선다. 그리고 선택된 침묵에서 대가는 인용 예절이 아니라 중복 노동이다.

다음 장이 받아가는 것. 이 장에서 $\text{ret}(E, q)$ 의 산출물은 언제나 **문자열**이었다. Zep의 이중 시간은 “FACT (Date range: from - to)”라는 텍스트로 모델에 전달되고, ReasoningBank의 전략은 시스템 지시문에 붙는 몇 문장이며, Mem0의 기억은 1764개의 prefill 토큰이다. 그래서 E -경로의 읽기 비용은 **검색된 바이트가 곧 prefill 토큰이 되는** 형태이고, 항목의 개수보다 간결성이 더 중요하다는 § 15.6의 관찰 (검색 경험 1개 → 4개에서 49.7 → 44.4)이 그 대가다.

ch16은 여기서 층을 옮긴다. Generative Adapter는 같은 문맥을 문자열이 아니라 **한 번의 forward로 만든 파라미터 모양의 산출물**로 바꾼다. 옮겨 가는 질문은 셋이다. 첫째, § 15.3.1의 쓰기 연산자 분류 — 확장전용·수축 가능·그래프 delta·동역학 — 중 어느 것이 W 층에서 살아남는가. 둘째, § 15.3.2의 세 가지 시간 표현 중 무엇이 파라미터 모양의 상태 위에서 표현 가능한가. 셋째, 그리고 가장 중요하게, 이 장이 발견한 패턴 — 비용 결과가 성능 결과의 자리에 놓이는 패턴 — 이 W -경로에서도 반복되는가. ch16은 그 검사의 첫 시험대이고, 초록의 헤드라인 수치가 본문 표 어디에도 없는 논문이라는 점에서 이미 예고편을 갖고 있다.

요약

- 이 장의 다섯 시스템은 전부 (U-E)만 실행한다. Θ 는 한 번도 움직이지 않고 W 는 존재하지 않으며, gradient도 손실 함수도 논문안에 없다.
- 쓰기 연산자는 네 형태로 갈린다: 네 연산 문법 (Mem0), 그래프 delta (Zep·Mem0^g), 합집합 (ReasoningBank), 동역학 (SCM·Memini). 이 중 수축 가능한 것은 Mem0의 DELETE와 SCM의 prune 둘뿐이고, 둘 다 삭제 정확도를 측정하지 않는다.
- 시간은 세 방식으로만 표현된다: 명시적 타임스탬프 (Zep의 이중 시간), 감쇠 상수 (SCM·Memini), 순서 첨자 (ReasoningBank). Zep의 타임스탬프는 프롬프트 문자열로 전달되므로 그것을 쓰는 일은 검색의 보장이 아니라 추론자의 과제다.

- 비용 4 종에서 ρ 는 다섯 전부에서, 바이트 기준 C_{cap} 도 다섯 전부에서 비어 있다. B_s 는 ReasoningBank 의 과제당 3748.4 토큰이 유일한 수치이고, 그것조차 MaTTS 구성은 덮지 않는다. 상각식 (A) 의 N_q^* 는 다섯 어디에서도 계산되지 않는다.
- 자기 표를 읽으면 품질 1 위가 반복적으로 기억 시스템 없는 구성이다: full-context 72.90% 대 Mem0 66.88%, full-conversation 94.4% 대 MemGPT 93.4%, No-Forget Graph 22/22 대 SCM 22/22.
- E -경로 프로덕션 시스템은 shared-weight batching 을 그대로 보존한다. 이것이 Θ -경로 대비 결정적 서빙 이점이며, 다섯 논문 중 아무도 명시하지 않는다.
- 다섯 논문은 비용 · 지연 결과를 성능 결과로 제시하고 있다. 비용 절감은 실재하지만 주장의 종류가 바뀌어 있고, 그 주장을 방어하려면 세지 않은 쪽인 B_s 를 세어야 한다.
- 침묵의 대가는 인용 예절이 아니라 중복 노동이다. SCM 은 (U-W) 의 retention gate 를 외부 그래프 위에서 재발명했다.

자가 점검체크리스트

- 다섯 시스템 각각에 대해 “라운드 k 가 무엇인가” 와 “그 라운드가 요청경로 위인가 아래인가” 를 표 15-3 을 보지 않고 답할 수 있다.
- Mem0 의 (15-2) 가 표준형 (U-E) 와 다른 점 두 가지를 말할 수 있고, 그 차이가 왜 ρ 를 실재하는 위험량으로 만드는지 설명할 수 있다.
- Zep 이 knowledge-update 에서 퇴행한 사실 (76.9% $\rightarrow$ 74.4%, gpt-4o-mini) 과 같은 범주가 gpt-4o 에서는 오른 사실 (78.2% $\rightarrow$ 83.3%) 을 § 15.3.2 의 시간표현 방식으로 설명할 수 있다.
- “이 시스템은 full-context 보다 효율적이다” 라는 문장이 이 장의 다섯 논문중 어느 것에 대해서도 쓸 수 없는 이유를, 비용 4 종 표의 빈칸을 지목해 말할 수 있다.
- ReasoningBank 의 판정기 정확도 72.7%와 대칭 잡음 시뮬레이션의 관계를 설명하고, 왜 후자가 전자의 견고성 근거가 되지 못하는지 말할 수 있다.
- SCM 이 판별식은 통과하면서도 제목이 약속한 Θ 갱신은 하지 않는다는 진술을, ch11 의 판별식 네 조건에 비추어 검증할 수 있다.
- (Rosetta) prefix cache 와 $\hat{c}$ 의 차이를 이 장의 사례로 말할 수 있다 — Mem0·Zep 의 검색 결과는 질의마다 달라 prefix cache 로 재사용되지 않고, ReasoningBank 은 검색 항목을 시스템 지시문 맨앞에 붙여 공유 프리픽스를 위치 0 에서 깨뜨린다. 즉 E -경로의 $\hat{c}$ 는 “같은 계산의 재사용” 이 아니라 “새 추론의 선행 수행” 이고, 그 대가로 캐시 재사용을 잃는다.

ch16. Generative Adapter —경첩

16.1 Bridge-in: 전작이 남긴문제

ch12 – ch15 는 E -경로를 끝까지 따라갔다. 그 경로의 구조적 약속은 하나였다 — 문맥의 산출물을 모델 밖에 레코드로 두고, 질의 시점에 $\text{ret}(E, q)$ 로 다시 끌어온다. 그 약속의 대가는 ch10 이 항 ② 로분해한 prefill 증가이고, ch15 가 마지막 체크리스트 항목에서 확인한 캐시재사용의 상실이다. 레코드는 질의마다 다르게 뿔히므로 공유 프리픽스를 위치 0 에서 깨뜨린다.

이 장은 반대편 선택지를 연다. 문맥을 밖에 두는 대신 **파라미터의모양으로 접어 넣는다**. 접어 넣은 뒤에는 원 문맥 토큰을 입력에서 지우고 closed-book 으로 답한다. 그러면 항 ②가 사라지고 공유 프리픽스 문제도 사라진다. 결정적인 것은 그 접기가 **학습이 아니라 forward** 라는 점이다 — backward pass 도, optimizer 상태도, 학습률 스케줄도 없다.

이 논문은 어떤 선행 논문의 open question 도 인용해 받지 않는다. 오히려 반대를 명시한다 — “As far as we know, we are the first to explore this direction” [Generative Adapter § 1], 그리고 기여 목록의 첫항에서 “To our knowledge, we are the first to explore retaining the relevant temporary knowledge through generated parameter-efficient model updates for state-of-the-art pretrained LMs” [Generative Adapter § 1]. Related Work ([Generative Adapter § 6]) 는 세 개의 대조 문단으로 짜여 있고 (“Different from previous work…” , “Unlike those methods…” , “Instead, GenerativeAdapter is…”), 각 문단은 받은 질문이 아니라 차이를 주장한다.

[평가] 그 최초성 주장은 논문 자신의 [Generative Adapter § 6] 과 나란히 놓으면 좁혀진다. 같은 절이 Tack et al. (2024) 를 “meta-learned amortization network 를 써서 개별 문맥 문서에 대한 base LM 의 parameter-efficient fine-tuning modulation 을 직접 예측한다” 고 요약한다 [Generative Adapter § 6, Adapting LMs via Meta-Learning]. 방향이 같다. 논문이 대는 차이 둘 — nested training loop 없이 end-to-end 로 학습된다는 것, generator 가 self-programming 이라는 것 — 은 실재하지만 학습 절차의 차이이지 방향의 차이가 아니다. 그리고 그 가장가까운 선행 연구는 baseline 으로 돌려지지 않는다.

ch14 가 E -경로에서 상속 없는 시작이었다면 이쪽은 종류가 다르다. ch14 는 같은 공동체 안에서 계보 진술을 생략했지만, 이 논문은 **다른공동체에서 입장한다** — fast-weight 와 PEFT 엔지니어링 쪽이며, 지속학습으로도 에이전트 기억으로도 실이 닿지 않는다.

없는 계보를 지어내지 않는 대신, 이 논문이 **암묵적으로 답한** 질문하나를 복원해 둔다. Schmidhuber(1992·1993) 의 fast-weight programmer 질문 — slow network 가 fast network 의 가중치를 프로그램할 수 있는가 — 이 여기서는 “동결된 7B pretrained LM 과의 접촉에서 살아남는가” 라는 형태로 던져진다. 논문 자신이 고전적 구성과의 차이를 이름붙인다.

“Instead of using a slow network to program a separate fast model, our method can be viewed as a self-programming model, i.e., context encoded by the base LM is used to update the base LM itself.” [Generative Adapter § 6, Fast Weights]

그리고 이 논문이 **질문이 아니라 기정사실로** 수입한 것이 하나 더있다. “acquiring knowledge through continual pretraining has shown to be data-inefficient (Yang et al., 2024; Allen-

Zhu & Li, 2024)” [Generative Adapter § 1]. 이 문장이 논문 전체의 동기를 떠받치는데, 질문이 붙어 있지 않다.

ch11 이 이 장에 넘긴 일은 하나다 — 산출물의 모양과 시계가 갈리는 사례를 끝까지 따라가는 것 (→ ch11 § 11.4.5). 이 장은 그 판정을 결론으로 옮겨 적지 않고 판별식으로 다시 밟는다 (§ 16.4).

16.2 문제의식

논문이 세우는 대립은 두 항이다.

“However, there is an accuracy compute tradeoff—finetuning incurs significant training cost and prompting increases inference overhead.” [Generative Adapter Abstract]

[Generative Adapter § 1] 이 그 두 항을 각각 풀어 쓴다. prompting 쪽은 저장 · 추론 부담이다 — “to maintain additional memory across sessions, some extra prompts must be added to the input, which incur an inference-time or storage overhead (Chevalier et al., 2023)” . finetuning 쪽은 학습 부담이다 — “it requires a training phase that is more computationally expensive than a single forward pass” [Generative Adapter § 1].

목표는 [Generative Adapter § 2.1] 이 한 문장으로 못 박는다. “To contextualize a base model, Θ base, to a given context C , our goal is to obtain an updated model, ΘC , that can respond to user instructions using the information provided in the context C .” 그리고 그 갱신이 스트리밍임을 곧바로 덧붙인다.

“We specifically focus on test-time contextualization, where context arrives incrementally as a stream of data... In this online adaptation scenario, the model must be efficiently adapted to each new context chunk as it becomes available.” [Generative Adapter § 2.1]

적용 대상 시나리오는 셋이다 — 문서에서의 지식 획득, 시연에서의 학습, 사용자 개인화 [Generative Adapter Abstract, § 1, § 4].

[해설] 이 책의 어휘로 옮기면 논문이 그린 2×2 가 사실은 1×2 다. prompting 은 문맥을 토큰으로 두고, finetuning 은 문맥을 가중치로 옮기되 gradient 로 옮긴다. 논문이 원하는 것은 가중치이되 gradient 가 없는 것이다. 그것은 Θ -경로의 어휘로 진술된 W -경로의 제안이다. 이 문장이 이 장 전체의 요약이며, § 16.4 가 그것을 판별식으로 증명한다.

한 가지를 먼저 못 박는다. 이 논문에는 “sleep” 도 “offline” 도 “idle” 도 “latency” 도 한 번 나오지 않는다 (전문 검색으로 확인). 이 논문은 ch01 의 조작적 정의에서만 이 범주에 접근하고, 자기 규정으로는 한 번도 sleep-time compute 을 자처하지 않으며, 실제로 판별식 조건 (1) 에서 탈락한다 (→ ch11 § 11.3). 그럼에도 이 장이 Part II 의 한 가운데 놓이는 이유는 이 논문이 경첩이기 때문이다 — Θ -경로가 gradient 로 쓰는 바로 그 아티팩트를, W -경로의 시계와 쓰기 규칙으로만 들어 낸다.

16.3 Core mechanism (통일표기)

이 절이 이 장의 두 정의를 소유한다.

정의. 한 번의 forward 로 문맥을 파라미터화하는 기제는, 문맥을 소비하는 계산이 base 모델의 forward pass 하나와 파라미터 없는 후처리만으로 끝나고, 그 산출물이 가중치와 같은 모양의 텐서이며, 읽기시점에 원 문맥 토큰이 입력에 존재하지 않는 기제를 말한다. 셋 중 하나라도 어긋나면 다른 기제다 — backward pass 가 있으면 학습이고, 산출물이 토큰이면 압축이며, 읽기에 원 문맥이 남아 있으면 prompting 이다.

배치부터 적는다. base LM 은 Mistral-7B-Instruct v0.2 또는 Llama2-7B-Chat 이고 전 과정에서 동결된다. adapter 가 실리는 자리는 **본 실험 전체에서 블록당 하나** — multi-head attention 의 출력 투영행렬이다 [Generative Adapter § 3]. 문맥은 1,024 토큰의 chunk 로 잘려들어온다 ($C = 1,024$) [Generative Adapter § 3 Training].

읽기를 표준형 (R) 로 환원하면 이렇다.

$$\hat{y} = f(q; \Theta, W_t, \emptyset), \quad \Theta = \Theta_{\text{base}} \cup \Theta_G \quad (16-1)$$

$E = \emptyset$ 이다. **Part II 에서 외부 저장소가 아예 없는 첫장이다.** 읽기는 Θ 와 W 만 통과하고 $\text{ret}(\cdot, \cdot)$ 이 등장하지 않는다. 층별로는 유효 행렬이 덧셈으로 만들어진단다 — 원문 $W^{(l)} = W_{\text{base}}^{(l)} + W_{\Delta}^{(l)}$ [Generative Adapter § 2.2] 가 이 책의 기호로 $\Theta^{(l)} + W_t^{(l)}$ 이며, $l = 1, \dots, L_{\text{layer}}$ 중 adapter 가 실린 자리에서만 두 번째항이 0 이 아니다.

쓰기는 두 식이다. 상태 갱신이 먼저다.

$$S_t = \beta_t S_{t-1} + A_2 H_t^\top H_t B_1, \quad \beta_t \equiv 1, \quad S_0 = 0 \quad (16-2)$$

$H_t \in \mathbb{R}^{M_{\text{ctx}} \times d}$ 는 동결된 base LM 이 t 번째 chunk 에 대해 이미 만든 hidden state 행렬이고, A_2, B_1 은 학습된 사영 행렬이며, $S_t \in \mathbb{R}^{d_r \times d_r}$ 은 $d_r = 1,024$ 의 정사각 상태다 [Generative Adapter Eq. 3; § 3 Hyperparameters]. 읽기용 가중치는 그 상태에서 매 chunk 다시계산된다.

$$W_t = A_1 \text{norm}(S_t) B_2 \quad (16-3)$$

식 (16-3) 이 표준형 (U-W) 와 갈라지는 첫 지점이다. W_t 는 W_{t-1} 을 이어받지 않는다 — retention 이 통째로 S_t 쪽으로 옮겨가 있다 [Generative Adapter Eq. 4, Eq. 6]. 그리고 (16-2) 의 재귀는 배치형태와 정확히 같다.

$$S_t = A_2 \left(\sum_{i=1}^t H_i^\top H_i \right) B_1 = A_2 \left(\sum_{m=1}^{M_{\text{ctx}}} h_m \otimes h_m \right) B_1 \quad (16-4)$$

식 (16-4) 는 원문 Eq. 1–2 의 형태다. 오른쪽 끝이 말하는 것은 쓰기가 **토큰들의 합**이라는 것이다. 이 성질이 chunk 병렬화를 가능하게하고 — “GenerativeAdapter is able to generate the adaptors for prefixes of chunks simultaneously by processing the context chunks in parallel” [Generative Adapter App. B] — 동시에 recency 도 순서도 축출도 금지한다.

정규화는 안정화 장치다.

$$\text{norm}(M) = UV^\top, \quad M = U\Sigma V^\top \quad (16-5)$$

양의 특이값을 전부 1 로 되돌린다 [Generative Adapter Eq. 7]. 이것이 없으면 학습이 수렴하지 않는다는 것이 도입 이유이고 [Generative Adapter § 2.4], 부수 효과로 rank r 까지 절단하면 곱이 두 개의 얇은 행렬이된다 — **LoRA 형태다** ($\rightarrow$ ch03 § 03.2 의 정의). 논문이 그 유도를 직접적는다 [Generative Adapter Eq. 8, Eq. 9]. $r = 128$ 이다 [Generative Adapter § 3 Hyperparameters].

학습되는 것은 사영 행렬뿐이고, 그 학습은 배포 전 outer loop 한 번이다.

$$\Theta_G \leftarrow \Theta_G - \eta_\Theta \nabla_{\Theta_G} \mathcal{L}, \quad \mathcal{L} = - \left[\log P(x_{1:m} | \Theta_{\text{base}} + W(x_{1:m})) + \log P(x_{m+1:n} | \Theta_{\text{base}} + W(x_{1:m})) \right] \quad (16-6)$$

앞 항이 reconstruction, 뒤 항이 completion 이다 [Generative Adapter Eq. 5, § 2.3]. 원문은 두 log-likelihood 를 **최대화**하므로 이 책의 최소화 관행에 맞춰 부호를 뒤집었다. gradient 는 동결된 base LM 을 **통과해** Θ_G 에 도달한다 [Generative Adapter § 2.3]. 여기서 $\mathcal{R}$ 에 해당하는 것은 replay 도 합성 데이터도 아니다 — SlimPajama 원시 웹 토큰 1B 와 뒤이은 instruction-tuning 혼합이다 [Generative Adapter § 3 Training].

표준형 (U-W) 와 항별로 대조하면 넷이 비어 있고 하나가 바뀐다.

- α_t : 없다. W_t 는 W_{t-1} 의 게이트된 이월이 아니라 S_t 의 무상태 함수다. $\alpha_t = 0$ 에 retention 을 통째로 S_t 로 옮긴 것과 같다.
- β_t : 항등적으로 1 이다. 감쇠도 게이팅도 축출도 없다. 아무것도 잊히지 않는다.
- η_t : 없다. 쓰기에 step size 가 없고, 가장 가까운 것은 손으로 정한 고정 adapter 배율뿐이다 — Mistral 1/16, Llama2 1/8 [Generative Adapter App. B]. 스케줄이 아니라 상수다.
- $\nabla_W \ell(W_{t-1}; k_t, v_t)$: **chunk 자신의 Gram 행렬의 학습된 쌍선형 사영으로 대체된다.** 이 논문에는 inner loss ℓ 이 없다. $H_t^\top H_t = \sum_m h_m \otimes h_m$ 이므로 연상쌍 (k_t, v_t) 가 (h_m, h_m) 으로 붕괴한다 — key 와 value 가 분리되어 있지 않고, 모델이 각 hidden state 를 자기 자신에 대해 쓴다. Hebbian 쓰기다.
- 비선형이 놓인 자리가 다르다. (U-W) 계열은 통상 쓰기에 비선형을 두고 읽기를 선형으로 두는데, 여기서는 읽기 쪽 (16-3) 에 SVD 가 매 chunk 들어간다. 그리고 상태 $S_t(d_r \times d_r)$ 가 산출물 $W_t(d_{\text{out}} \times d_{\text{in}})$ 보다 작다.

두 번째 정의가 이 장의 방법론적 소유물이다.

정의. 산출물의 모양과 갱신의 시계를 분리해 보는 관점은, 어떤 절차를 분류할 때 (i) 그 절차가 남기는 상태의 물리적 모양 (텍스트 레코드인가, fast weight 텐서인가, 가중치 delta 인가) 과 (ii) 그 상태가 움직이는 시간 척도 (토큰 · chunk 인가, 세션인가, 학습라운드인가) 를 독립된 두 축으로 두고, **경로 판정을 시계와 갱신식 쪽에만 맡기는 관점**이다. 모양은 저장 · 전송 · 롤백의 단위를 정하고, 시계는 갱신식과 비용 구조를 정한다. 두 축은 함께 움직이지 않으며, 이 논문이 그 사실의 가장 깨끗한 사례다.

16.3.1 표기 대응표

표 16-1 — Generative Adapter 원 표기 → 이 책의 표기

원 표기	이 책의 표기	주의
$\Delta, \Delta_t, W_\Delta, W_{\Delta t}$ — 문맥 의존 가산 adapter	W_t	이 장에서 가장 중요한 치환이다. 논문의 단어 “adapter” 는 LoRA 에서 온 Θ -경로 어휘인데, 여기서 그것이 가리키는 대상은 W -경로 객체다
$S_t - d_r \times d_r$ 누적 부분합, “the memory of history context chunks” [§ 2.2]	S_t	슬롯은 정확히 (U-W) 의 S_t 자리다. 내용이 다르다 — 이 책의 S_t 는 gradient momentum, 논문의것은 Hebbian 누적기다
$\Theta_{\text{base}}, \Theta_C, \Theta_{\Sigma(t)}$ — 동결 base 가중치, 문맥화된 모델	$\Theta = \Theta_{\text{base}} \cup \Theta_G$; $\Theta_{\Sigma(t)} \rightarrow \Theta + W_t$	outer loop 에서 움직이는 것은 Θ_G 뿐이고 Θ_{base} 는 이 논문에서 한 번도 움직이지 않는다. $\Theta_{\Sigma(t)}$ 라는 표기가 Θ 가 갱신된다는 인상을 주지 않게 한다
C, C_t — 문맥, t 시점 도착 chunk [§ 2.1] $\Sigma, \Sigma(t)$ — 스트리밍 문맥 ($C_1, \dots, C_t$) [§ 2.1]	내용은 c_t , 크기는 $C = 1,024$ 토큰 $c_{1:t}$	용량은 언제나 C_{cap} . 논문의 C_t 를 본문에 그대로 쓰지 않는다 논문 내부 충돌: 같은 Σ 를 Eq. 7 의 특이값 행렬로 재사용한다. 특이값 쪽 Σ 는 식 (16-5) 안에 서만 쓴다

원 표기	이 책의 표기	주의
$G, G^{(l)}$ — layer 별 adapter generator	G 유지	파라미터 $\{A_1, A_2, B_1, B_2\}^{(l)}$ 는 Θ 의 부분집합이다. (U- Θ) 의 $\text{gen}(\cdot; B_s)$ 와 혼동 금지 — 그 쪽은 학습집합을 만드는 절차다
$H, H^{(l)}, H_t$ — base LM hidden state 행렬, $M \times d_h$ [§ 2.2] M — 문맥 토큰 수 [§ 2.2] 이자 $\text{norm}(M)$ 의 일반 행렬 [§ 2.4] L — Transformer 블록 수 [§ 2.2]; $L_{\text{reconstruction}}, L_{\text{completion}}$ [§ 2.3]	H_t (로만체, hidden states) 토큰 수는 산문에서 M_{ctx} , 일반 행렬은 식 (16-5) 안에서만 블록수는 L_{layer}	예약된 $\mathcal{H}_k$ (누적 경험) 는 이 논문에 인스턴스가 없다 — sleep 라운드가 없다 맨 M 금지 — $\mathcal{M}(\cdot; W)$ (읽기 함수) 로 읽힌다 L 은 sequence 길이로 예약되어 있다. 원문의 두 L 은 최대화 대상 log-likelihood 이므로 식 (16-6) 으로 옮길 때 부호를 뒤집었다 충돌 없음. 논문자신이 Eq. 8 – 9 에서 LoRA rank 와 동일시한다
r — SVD 절단 rank, 128 [§ 3]	$r = 128$	d_r 은 r 과 다른 값이다 (1,024 대 128)
$d_r, d_h, d_{\text{in}}, d_{\text{out}}$ — generator 중간차원 (1,024), hidden 차원, layer 입출력 차원 A_1, A_2, B_1, B_2 — 학습 가능한 generator 행렬 [Eq. 1]	$d_h = d$; d_r 은 장-국소 기호로 유지 그대로 유지	α_t, β_t 는 게이트 전용 기호이므로 이 넷과 무관하다. 이 $A \cdot B$ 는 outer loop 에서 학습되며 test-time 에 적합되는 LoRA 의 $A \cdot B$ 가 아니다 논문 내부 중복이다
x — 선형층 입력 벡터 [§ 2.4] 이자 토큰 $x_1 \dots x_n$ [§ 2.3]	인용할 때마다 구분	

16.4 어느 층을 언제쓰는가

먼저 “이 값은 누가 학습하는가” 에 전부 답한다.

표 16-2 — 이 값은 누가 학습하는가

객체	학습 주체	언제	test-time 이동	출처
Θ_{base} — 동결 base LM (Mistral-7B-Instruct v0.2 또는 Llama2-7B-Chat, 7B)	이 논문에서는 없음	없음. 다른 곳의 pre-training 에서 물려받음	아니오	§ 1, § 3

객체	학습 주체	언제	test-time 이동	출처
$\Theta_G = \{A_1^{(l)}, A_2^{(l)}, B_1^{(l)}, B_2^{(l)}\}$ — adapter generator, 약 500M 파라미터	outer loop gradient descent(Adam), 동결된 base LM 을 통과해 역전파	배포 전 한 번, 2 단계: (i) SlimPajama 1B 토큰 자기지도 사전학습, (ii) QA·ICL·instruction 혼합 2 epoch	아니오	§ 2.3, § 3, App. B
$S_t^{(l)} \in \mathbb{R}^{1024 \times 1024}$ — 누적 상태	학습되지 않음. forward-only 재귀로 움직인다	도착하는 1,024 토큰 chunk 마다 한 번. 문맥 스트림 시작마다 $S_0 = 0$	예	Eq. 3, § 2.2, § 3
W_t — 생성된 adapter, 32M 파라미터	학습되지 않음. S_t 에서 식 (16-3) 으로 결정론적 재계산 (gradient 도 optimizer 상태 없음)	S_t 갱신 직후 chunk 마다. 매 chunk 덮어씀	예	Eq. 6, § 3, Table 1
norm($\cdot$) — SVD 정규화	파라미터 없음	문맥화 시점, 매 chunk, adapter 가 실린 매 layer. <code>torch.svd_lowrank()</code> 반복 1 회로 구현	해당 없음	Eq. 7, App. B
adapter 배율 (Mistral 1/16, Llama2 1/8)	손으로 정한 하이퍼파라미터	outer loop 학습 전 고정, base 모델마다	아니오	App. B
$d_r = 1,024$, $r = 128$	손으로 정한 하이퍼파라미터	outer loop 학습 전 고정	아니오	§ 3
adapter 를 어느 모듈이 지는가 (본 실험은 attention 출력 투영만; feedforward down-projection 은 ablation 예만)	설계 결정, 학습되지 않음	outer loop 학습 전 고정	아니오	§ 3, Table 2

시간척도는 셋이고, 셋 중 **sleep 라운드는 없다**. (i) outer loop 학습 — 배포 전 한 번, gradient 기반, Θ_G 대상. (ii) chunk 단위문맥화 — forward-only, S_t 와 W_t 대상. (iii) 토큰 단위 decode — 아무것도 움직이지 않는다. sleep 라운드 첨자 k 와 세션 첨자 τ 는이 논문에서 **한 번도 인스턴스를 갖지 않는다**. wake 시점에움직이는 것은 S_t 와 W_t 뿐이고 나머지는 전부 정적이다.

이제 경로를 판정한다. 결론을 옮겨 적지 않고 ch01 § 01.4 의 세 판별 조건을하나씩 댄다.

Θ-경로 조건 — 배포 아티팩트가 바뀌는가. 바뀌지 않는다. 논문이 정면으로 진술한다 — “the pretrained base LM remains frozen while we train the LM-specific adapter generator” [Generative Adapter § 1], 그리고 [Generative Adapter § 2] 서두에서 “Unlike continual pretraining and supervised fine-tuning which update the pretrained LM via gradient descent, our method achieves adaptation using forward passes only” . Θ_{base} 는 미분되

지도 수정되지도 않으므로 문맥화 이후의 체크포인트는 이전과 바이트 단위로 같다. **탈락.**

E-경로 조건 — (U-E) 만 실행하고 Θ, W 가 불변인가. 텍스트도 벡터도 그래프도 기록되지 않고, 읽기 시점에 $\text{ret}(E, q)$ 가 없다. 논문의 요점 자체가 문맥을 다시 제시하지 않는 것이다 — 문맥화된 모델은 “in a closed-book fashion” 으로 답한다 [Generative Adapter § 4.1]. E 자리가 비어 있으므로 (U-E) 의 인스턴스가 없다. **탈락.**

W-경로 조건 — 문맥을 fast weight 로 접어 넣고 원 문맥을 버리는가. 그렇다. 식 (16-2) – (16-3) 이 (U-W) 의 자리마다 대응물을 갖는다 (gradient-free 변종이지만 형태는 (U-W) 다), 상태는 고정 크기이고, 원 문맥 토큰은 읽기 입력에서 사라진다. **통과.**

W-경로 논문이다. 여기까지가 경로 판정이고, 소속 판정은 별개다. ch11 § 11.2 의 네 조건을 대면 이렇다 — (1) 질의 전인가: **아니다.** 접기는 문맥이 도착한 그 자리에서 wake pass 안에 일어나고, 모델은 방금 접은 그 문맥에 대해 답한다. (2) $B_s > 0$ 인가: 구조적으로는 그렇다 (forward pass 하나 + SVD). (3) 어느 한 층의 상태가 실제로 바뀌는가: 그렇다, W 가 움직인다. (4) 이후 질의에 쓰이는가: 같은 스트림 안에서는 그렇다. 조건 (1) 에서 탈락하므로 **sleep-time compute 이 아니다.** 이 논문은 W-경로의 **wake** 쪽 끝점이다.

[평가] 이 판정이 왜 중요한가. 산출물만 보면 이 논문은 Θ -경로로 분류된다 — 만들어내는 물건이 LoRA 이고, 논문이 그 형태를 직접 유도하며 [Generative Adapter Eq. 8 – 9], 크기도 ch03 § 03.6 의 delta 아티팩트 공식으로 그대로 계산된다. 그런데 시계는 chunk 이고, 쓰기 규칙에는 loss 도 학습률도 backward pass 도 없다. **산출물의 모양으로 경로를 나누면 틀린다** — 이것이 이 책의 방법론적 주장이고, 이 논문이 그 주장의 가장 깨끗한 반례 제조기다. 모양으로 나누면 이 논문이 Θ -경로로 들어오고, 같은 오류가 SCM 을 Θ -경로로, Memory Caching 을 sleep-time compute 으로 들여보낸다 (→ ch11 § 11.4). 시계와 갱신식으로 나누면 세 판정이 동시에 바로잡히고, 바로잡힌 판정이 곧 비용 · 용량 · 롤백 단위의 판정이 된다. 이 장에서 그 실익이 구체적으로 나타나는 자리는 § 16.7 이다 — 롤백 단위가 “이전 체크포인트로 되돌리기” 가 아니라 “ $S_0 = 0$ 으로 초기화” 이고, 그 차이가 서버 설계를 통째로 바꾼다.

갱신 규칙의 순서 문제는 생기지 않는다. 이 논문은 (U-W) 하나만, 한 단계로 실행한다. 앞서는 (U-E) 도 없고 뒤따르는 (U- Θ) 도 없다.

16.5 비용 4 종

표 16-3 — 비용 4 종

기호	값	근거
B_s	수치로는 논문에 없음. 구조로만 진술된다 — 문맥에 대한 동결 base LM 의 forward pass 1 회, adapter 가 실린 layer 마다 ·1,024 토큰 chunk 마다 rank-128 randomized SVD 1 회 (torch.svd_lowrank, 반복 1 회), 그리고 식 (16-2) – (16-3) 의 사영 GEMM 넷. Figure 3 의 왼쪽 패널이 문맥화 FLOPs 를 teraFLOPs 단위 로그축으로 512 – 32K 구간에 그리지만 본문이 그 패널의 어떤 수치도 인용하지 않는다	§ 3, § 4.1, App. B, Figure 3

기호	값	근거
L_w	논문에 없음. “latency” , “ms” , “TFTT” , “time to first token” 이 한 번도 나오지 않는다 (전문 검색으로 확인). 어떤 종류의 벽시계 decode 측정도 없다	전문 검색
C_{cap}	개수로만 보고. 바이트는 논문에 없음. 문맥당 32M floats — “the generated adapter of 32 million parameters” [§ 3], Table 1 의 “Extra Storage (M floats)” 열이 32. dtype 이 어디에도 없고, 저장 대상이 S_t 인지 Eq. 9 의 저랭크 인수쌍인지도 적혀 있지 않다	§ 3, Table 1
ρ	논문에 없음. 라운드당 열화량이 정의되지도 측정되지도 않았다. “forget” , “forgetting” , “catastrophic” 이 한 번도 나오지 않는다 (전문 검색으로 확인). 같은 모델이 문맥화를 반복한 뒤 앞선 자료로 재평가되는 실험이 없다	전문 검색, Eq. 3

B_s 에 붙은 유일한 정량 수식어는 비교급 하나다 — “GenerativeAdapter requires preprocessing time (forward passes only) that is orders of magnitude smaller than CPT (which involves multiple forward and backward passes), as demonstrated in Figure 3” [Generative Adapter § 4.1]. “orders of magnitude” 가 전부다. 일회성 상각 성분은 따로 있다: generator 의 outer loop 학습이 SlimPajama 1B 토큰에 대해 “approximately 20 hours using 8 NVIDIA H100 GPUs” 이고 [Generative Adapter App. B], 뒤이은 instruction tuning 2 epoch 의 벽시계는 보고되지 않았다.

L_w 칸에 가장 가까운 대용물은 MSC 에서의 질의당 추론계산량이다 (TFLOPS): Generative Adapter 0.505, closed-book 0.505, full-conversation prompting 2.059, UltraGist 는 64/128/256/512/1K/2K 압축 토큰에서 각각 0.514/0.552/0.627/0.772/1.067/1.658 [Generative Adapter Table 1]. 문맥화 이후의 질의당 비용이 소수 셋째자리까지 base 모델과 같게 보고된 것이 이 논문의 핵심 수치다. Figure 3 의 가운데 패널이 StreamingQA 에서 같은 평탄함을 문맥 길이에 대해보인다.

ρ 자리에 있는 것은 다른 양이다. 하나는 한 스트림 안에서 문맥 길이에 따른 품질 저하다 — Mistral/StreamingQA 에서 F1 이 512 부터 32K 까지 51.5 $\rightarrow$ 49.3 $\rightarrow$ 44.7 $\rightarrow$ 40.9 $\rightarrow$ 36.7 $\rightarrow$ 32.7 $\rightarrow$ 32.0 으로 떨어진다 [Generative Adapter Table 5]. 이것은 상태가 차면서 생기는 압축 손실이지 라운드당 망각률이 아니다. $\beta_t = 1$ 이 스트림 전체를 하나의 누적으로 만들기 때문에 라운드를 분리할 수조차 없다. 다른 하나는 쓰기 후에도 일반 언어모델링 능력이 남는지를 보는 completion perplexity 7.40 이다 [Generative Adapter Table 2]. 그런데 문맥화되지 않은 base 모델의 기준 perplexity 가 논문에 없어 이 값이 함의하는 열화량을 정량화할 수 없다.

상각식 (A) 는 닫히지 않는다. 분자 쪽은 알려져 있다 — MSC 에서 질의당 2.059 $-$ 0.505 = 1.554 TFLOPS 가 절약된다 [Generative Adapter Table 1][본서 산술]. 분모 쪽 C_{sleep} 은 Figure 3 의 눈금 없는 로그축 곡선으로만 존재하고 본문에 수치로 인용된 적이 없다. 따라서 손익분기 N_q^* 를 계산할 수 없다. 그럼에도 상각 논증은 두번 등장한다 — “In practical scenarios with many queries from the same user on edge computing devices, the benefits of our method

are even more evident” [Generative Adapter § 1], “In real world scenarios with many queries from the same user, the benefits of our method are even more pronounced” [Generative Adapter § 4.3]. N_q 값도 없다.

[평가] 이 논문은 ch14의 상시 caveat — 상각식이 N_q 를안다고 가정하는데 실서빙 N_q 분포를 보고한 논문이 없다 — 에 한 겹을더한다. 여기서는 N_q 분포가 없을 뿐 아니라 **sleep 쪽 비용자체가 수치로 없다**. 비율의 두 항이 모두 비어 있는 상태에서 비율이 주장된다. ch01이 정한 규칙에 따라 이 장은 이 논문에 대해 “효율적” 이라고쓰지 않는다. 쓸 수 있는 문장은 하나뿐이다 — **질의당 추론 FLOPs가 base 모델과 같다고 보고** 되었다.

16.6 실험과스케일

스케일 상한부터 적는다. base 모델은 7B 둘뿐이고 그 아래도 위도 없다 [Generative Adapter § 3]. generator는 약 500M 파라미터, $d_r = 1,024$, $r = 128$, chunk 1,024 토큰, adapter가 실린 자리는블록당 attention 출력 투영 하나다. 사전학습 코퍼스는 SlimPajama에서무작위 추출한 1B 토큰 (8,192 토큰 세그먼트로 분할) 이고, instruction tuning은 COQA·DROP·NarrativeQA·PubMedQA·Quail·MS MARCO·MetaICL·BookSum·PwC 혼합이다 [Generative Adapter § 3, App. A, Table 3]. 문맥은 32K 까지평가된다.

표 16-4 — 문서 QA, F1, Mistral-7B-Instruct v0.2 [Generative Adapter Table 5]

데이 터셋	방법	512	1K	2K	4K	8K	16K	32K
SQuAD	zero-shot prompting	10.8						
SQuAD	super- vised fine- tuning	20.7						
SQuAD	contin- uous pre- train- ing	30.0						
SQuAD	in- context prompt- ing	45.4	44.9	43.6	42.6	42.5	38.6	35.1
SQuAD	Gener- ative Adapter	48.8	43.0	39.9	35.9	33.8	30.3	28.0
Stream- ingQA	zero- shot prompt- ing	13.6						

데이터셋	방법	512	1K	2K	4K	8K	16K	32K
StreamingQA	super-vised fine-tuning	19.5						
StreamingQA	continuous pre-training	22.2						
StreamingQA	in-context prompting	47.2	48.7	48.1	48.7	48.0	46.0	39.3
StreamingQA	Generative Adapter	51.5	49.3	44.7	40.9	36.7	32.7	32.0

closed-book 행 (zero-shot·SFT·CPT) 은 문맥길이와 무관한 단일 값이라 첫 열에만 적었다. 원문 표의 (모델, 데이터셋) 블록 배정은 초록이 인용한 SFT 19.5(Mistral/StreamingQA) 와 [Generative Adapter § 4.1] 의 Llama2 4K 절단 규칙으로 고정했다.

초록의 헤드라인 수치가 본문 표와 화해하지 않는다. 초록은 “achieving a 63.5% improvement in F1 score over the model with supervised fine-tuning (from 19.5 to 31.5) for contexts as long as 32K tokens” 라고 쓴다 [Generative Adapter Abstract]. 기준값 19.5 는 표 16-4 의 Mistral/StreamingQA SFT 와 일치한다. 그런데 **31.5 는 논문 어디에도 없다** — 표 16-4 의 32K 값은 32.0 이다. 게다가 19.5 → 31.5 는 63.5%가 아니라 61.5% 증가이고, 19.5 → 32.0 이라면 64.1%다. 목표값도 백분율도 맞지 않는다. 이 책은 두 값을 나란히 적고, 어느 쪽도 초록의 진술을 지지하지 않는다고 기록한다.

[평가] 가장 센 baseline 과의 대조는 다르게 읽힌다. Mistral 에서 Generative Adapter 가 in-context prompting 을 이기는 칸은 열넷중 셋뿐이다 — SQuAD 512(48.8 대 45.4), StreamingQA 512(51.5 대 47.2), StreamingQA 1K(49.3 대 48.7). 32K 에서는 SQuAD 28.0 대 35.1, StreamingQA 32.0 대 39.3 으로 진다. 초록의 표현 (“effective in injecting knowledge into the LM’ s parameters”) 이 재는 상대는 prompting 이 아니라 가장 약한 baseline 인 SFT 다. Llama2 에서는 8K 이상에서 이기지만 (SQuAD 28.2/24.9/23.6 대 25.2/9.6/6.4; StreamingQA 28.7/26.0/25.7 대 27.8/17.5/11.6), 그 구간의 baseline 은 **설계상 잘려 있다** — “for Llama2-7B-Chat, if the context length exceeds the maximum limit of 4K tokens, we truncate the prompt to include only the last 4K tokens” [Generative Adapter § 4.1]. 둘 다 문서 전체를 보는 512 에서는 in-context prompting 64.8 대 Generative Adapter 36.2 로 28.6 점 차이다. 장문맥 우위 증거의 절반이 능력 비교가 아니라 절단 아티팩트다.

CPT 와의 대조에는 교차가 있다. SQuAD/Mistral 에서 512 – 16K 는 전부 CPT 의 30.0 을 넘지만 32K 에서 28.0 으로 진다. SQuAD/Llama2 도 32K 에서 23.6 대 23.9 로 진다. StreamingQA 두블록에서는 모든 길이에서 이긴다 [본서 관찰].

[평가] 나머지 두 시나리오의 증거의 성격이 다르다. MetaICL 은 26 개 테스트 과제, $K \in \{1, 2, 4, 8, 16\}$ 시연, 설정마다 5 회 반복표집이다 [Generative Adapter § 4.2]. 그런데 헤드라인 “our method achieves an average accuracy of 44.9 across 26 tasks” [Generative Adapter Abstract] 는 **초록에만 있다** — [Generative Adapter

§ 4.2] 에도, 어떤 표에도 없고, 어느 base 모델인지도 어느 K 인지도 붙어 있지 않다. [Generative Adapter § 4.2] 가 내놓는 것은 Figure 4 의 범주별 곡선과 Figures 5-6 의 과제별 곡선뿐이고 본문에 수치 집계가 없다. 세 헤드라인 중 하나를 논문 자신의 본문으로 검증할 수 없다.

표 16-5 — 개인화, Multi-Session Conversation, Mistral-7B-Instruct v0.2 [Generative Adapter Table 1]

방법	F1	추론 계산 (TFLOPS)	추가 저장 (M floats)
Closed-book	8.1	0.505	0
Full-conversation Prompting	66.0	2.059	128+
UltraGist (512 토큰)	40.8	0.772	32
UltraGist (1K 토큰)	44.4	1.067	64
Generative Adapter	40.2	0.505	32

UltraGist 의 나머지 네 설정 (64/128/256/2K 토큰) 은 F1 26.5/32.2/38.3/42.4, 추론 0.514/0.552/0.627/1.658, 저장 4/8/16/128 이다.

[평가] “4× 절감” 주장은 산술이 맞는다 — $2.059/0.505 = 4.08$ 배 계산, $128/32 = 4$ 배 저장 [Generative Adapter Abstract, § 4.3]. 그 대가도 같은 표에 있다: full-conversation prompting 66.0 대 Generative Adapter 40.2 로, 전체 문맥 품질의 61%를 유지한다. 25.8 점 F1 결손이 4× 절감의 가격이며, 초록의 형용사 (“highly competitive”) 는 비율에 대한 것이지 정확도에 대한 것이 아니다. 그리고 같은 절에 표와 어긋나는 문장이 하나 있다 — “Comparing to UltraGist at the same level of storage cost (compressed into 512 tokens), GenerativeAdapter further reduces inference cost without performance drop” [Generative Adapter § 4.3]. 표 16-5 는 UltraGist(512) 40.8, Generative Adapter 40.2 다. 0.6 점 하락이 하락 없음으로 서술되어 있다. 계산 절감 ($0.772 \rightarrow 0.505$) 은 실재하지만 문장은 옆의 표가 지지하지 않는다.

표 16-6 — ablation, Mistral-7B-Instruct v0.2, 검증 perplexity [Generative Adapter Table 2]

설정	reconstruction ppl	completion ppl
기본값 (두 과제 + SVD norm + attention 출력 투영)	1.75	7.40
사전학습 과제: reconstruction 만	1.75	34.34
사전학습 과제: completion 만	6.38	6.71
정규화: Frobenius	7.72	7.32
모듈: feedforward (down projection; 기본값의 3 배 갱신 파라미터)	1.68	7.26

[평가] 표 16-6 에 두 가지 역전이 있다. 첫째, “relying solely on one task does not yield good perplexity on the validation set for both metrics” [Generative Adapter § 5.1] 는 reconstruction-only 행 (1.75 / 34.34) 에만 참이다. completion-only 행은 completion perplexity 에서 기본값보다 낫다 (6.71 대 7.40). 두 과제 혼합은 Pareto 개선이 아니라 교환이고 논문이 그렇게 쓰지 않는다. 둘째, feedforward

배치가 두 지표 모두에서 기본값보다 좋은데 (1.68 / 7.26 대 1.75 / 7.40) **본 실험전체가 기본값 구성을 쓴다** — “For efficiency, our main experiments train adapter generators to only update the output projection layers” [Generative Adapter § 3]. 모든 헤드라인 수치가 저자 자신의 ablation 이열등하다고 판정한 구성에서 나왔고, 더 나은 구성은 어떤 downstream 벤치마크에서도 돌려지지 않았다.

[평가] 그 ablation 전체가 두 개의 검증 perplexity 위에 서있다는 점도 함께 적어야 한다. perplexity 와 downstream 품질의 연결은 보고되지 않은 예비 실험에서 주장된다 — “As we observe in our preliminary study, the quality of the resulting adapter generator is highly correlated with these metrics” [Generative Adapter § 5.1]. 어떤 ablation 행도 SQuAD·StreamingQA·MetaICL·MSC 에서 평가되지 않았다. 설계 결정 셋 (정규화·과제혼합·모듈 배치) 의 근거가 전부 대응 지표다.

[평가] 실증 상한을 정직하게 정리하면 이렇다. 모델은 7B 둘, 문맥은 32K 까지, 그러나 **우위의 상한은 훨씬 낮다** — Mistral 에서 in-context prompting 대비 우위는 1K 토큰까지다. MSC 대화는 평균 2.5K 토큰이고, 단일 연속 스트림을 넘는 실험이 없다 — 다중 세션도, 다일 (多日) 도, 반복 라운드도 없다. MSC (Table 1) 와 ablation (Table 2) 은 Mistral 하나로만 돌았다. 분산은 어디에도 없다: MetaICL 은 5 회 반복 표집을 하면서 표준편차를 본문에도 그림에도 적지 않고, QA 결과에는 오차 막대가 없다. 그리고 가장 아픈 공백 하나 — **문맥마다 gradient 로 적합한 LoRA 가 baseline 으로 없다**. 그것이 비용을 맞춘 Θ -경로 비교군이자이 방법이 배출하는 바로 그 물건인데, 가장 치 꼭 baseline 은 full-parameter SFT 와 CPT 둘뿐이다.

16.7 Systems/serving 합의

독자의 1 번 질문에 논문이 직접 답한다. **문맥화 이후 decode 비용은 base 모델의 decode 비용이다** — 0.505 대 0.505 TFLOPS, 소수 셋째 자리까지 같다 [Generative Adapter Table 1]. 문맥 길이에 대해서도 평탄하다 [Generative Adapter Figure 3 중앙 패널]. 이등식이 성립하는 조건은 하나뿐인데 논문이 그 조건을 명시하지 않는다 — delta 가 **merge 된 형태로** 서빙되어야 한다 (→ ch03 § 03.5)[본서추론].

[해설] 아래는 이 책의 산술이며 논문의 보고값이 아니다. 논문이 블록 수를 적지 않으므로 7B 관행대로 $L_{\text{layer}} = 32$, $d = 4,096$ 을 가정한다. **분리된 delta 형태로** 서빙하면 adapter 가 실린 행렬마다 토큰당 2rd MAC 이 더붙는다 — $r = 128$ 에서 행렬당 약 1.05M MAC, 블록당 하나씩 32 개면 토큰당 약 33.5M MAC (약 67 MFLOPs) 이고, 7B forward 의 토큰당 약 14 GFLOPs 에 대해 약 0.5%다. 어느 형태든 덧셈 항은 작다. 문제는 크기가 아니라 형태 선택이 서빙 구조를 정한다는 것이다.

상태량도 같은 산술로 확인된다. ch03 § 03.6 의 delta 아티팩트 공식 $|\Delta\Theta| = 2N_{\text{mod}}dr$ 에 $N_{\text{mod}} = L_{\text{layer}} \times |S| = 32 \times 1$ 을 넣으면 $2 \times 32 \times 4096 \times 128 = 33,554,432$ 이고, 상태 S_t 쪽도 $32 \times 1024 \times 1024 = 33,554,432$ 로 같은 수가 나온다. 논문의 “32 million floats” 와 자릿수·값이 모두 맞는다 [Generative Adapter § 3, Table 1]. 바이트로 옮기면 bf16 에서 약 64 MB, fp32 에서 약 128 MB 인데 **dtype 이 논문에 없으므로 이 두 값은 가정 위의 값이다**. 단정할 수 있는 것은 자릿수와 비율이다.

이 상태량이 KV cache 와 갈리는 지점은 성장 방식이다. 표 16-5 의 full-conversation prompting 은 2.5K 토큰 대화에서 128+ M floats 인데 Generative Adapter 는 문맥 길이와 무관하게 32 M floats 로 고정이다. 교차점은 이르다 — 위 두 점을 선형으로 이으면 약 600 토큰 대화 아래에서는 KV cache 쪽이 더 작다 (이 추정 은 이 책의 산술이다). 그 아래에서 이 기제는 **저장을 늘린다**.

문맥화 자체는 학습 단계가 아니라 prefill 이다. hidden state 를 얻으려면 동결 base LM 의 full forward 가 문맥 전체에 대해 필요하므로, FLOPs 는 그 문맥을 한 번 prefill 하는 것과 같은 자릿수다 — prompting-with-KV-cache 가 어차피 지불하는 그 값이다. **절감은 첫 pass 에 있지 않**

고 그 뒤의 모든 pass 에 있다. prefill 위에 얹히는 추가 작업은 chunk 마다 · layer 마다 1024×1024 행렬의 rank-128 randomized SVD 인데, $O(d_r^2 r)$ 로 layer 당 chunk 당 약 1.3×10^8 MAC, 32 layer 면 chunk 당 약 4×10^9 이다. 1,024 토큰 chunk 를 7B 로 prefill 하는 약 1.4×10^{13} FLOPs 에비하면 작다 (이 대조도 이 책의 산술이다). 작지만 **GEMM 이 아니고 배치가 잘 되지 않는 커널**이 깨끗한 prefill 파이프라인 한가운데에 들어온다.

[평가] 배치가 이 설계의 다루어지지 않은 결과다. merge 된 형태면 동시 서빙되는 사용자마다 7B 가중치 사본이 따로 필요해지고 shared-weight batching 이 죽는다 — ch01 Rosetta 표의 “batch 로 weight 공유 → Θ -경로에서 깨짐” 행이 그대로 발동한다. 분리된 delta 형태면 요청별 LoRA 이므로 multi-LoRA 서빙 방식으로 배치되지만, 손으로 적합한 $r=16$ LoRA 의 수 M floats 이 아니라 요청당 32 M floats 다. 논문은 동시다중 사용자 서빙을 한 문장도 다루지 않는다 — “batch” 는 학습 하이퍼파라미터에 만 나온다.

여기서 § 16.4 가 예고한 실익이 드러난다. **롤백 단위가 다르다.** Θ -경로였다면 잘못 쓴 내용을 되돌리는 방법은 이전 체크포인트로의 롤백뿐이다. 이 기제에서는 $S_0 = 0$ 으로 초기화하는 것이 전부이고, 비용은 문맥을 다시 접는 forward 한 번이다. 배포 아티팩트가 불변이므로 사용자별 delta 가 오염되어도 base 모델은 감사 대상이 아니다. 이것이 산출물 모양이 아니라 시계로 분류했을 때 실제로 달라지는 운영 결론이다.

[평가] 논문이 두 번 기대는 edge 논증은 edge 측정 없이 서있다 — “most computations occur on edge devices without power GPUs” [Generative Adapter § 4.3]. 그런 장치의 구속 조건은 보통 메모리 대역폭과 용량이고, 자라는 KV cache 를 고정 64–128 MB adapter 로 바꾸는 것은 실제로 좋은 교환이다. 그러나 논문이 재는 것은 FLOPs 뿐이고, **대역폭 구속 장치에서 $4 \times$ FLOPs 절감은 $4 \times$ 지연 절감이 아니다.** 표 16-3 의 L_w 칸이 비어 있는 한 이 논증은 검증되지 않은 채로 남는다.

16.8 한계와 bridge-out

논문이 명시한 문제는 셋이고 하나로 모인다. “For future work, it would be interesting to further explore scaling up the adapter generator, such as by integrating adapters into additional layers, and to investigate more selective update rules (Schlag et al., 2021)” [Generative Adapter § 7]. 뒤쪽 절반이 **논문이 자기 결손을 이름 부르는 자리다** — Schlag et al. (2021) 은 fast-weight programmer 의 delta-rule 방식 쓰기 · 지우기로 인용되며, 그것이 정확히 식 (16-2) 에 없는 β_t 와 축출이다. 나머지 둘은 같은 공백의 재진술이다 — [Generative Adapter § 3] 의 “For efficiency... defer the full exploration of other modules for future work” 와 [Generative Adapter § 5.1] 의 “Due to computational constraints, a more thorough exploration was not feasible” .

논문이 남기지 않았지만 남는 문제는 여섯이다. **용량 포화** — $\beta_t = 1$ 과 고정 $d_r \times d_r$ 상태에서 품질은 스트림 길이에 대해 단조 감소할 수밖에 없고 표 16-4 가 그것을 보이는데 ($51.5 \rightarrow 32.0$), 어디서 포화하는지도 d_r 이나 r 을 키우면 어떻게 되는지도 특성화되지 않았다. 둘은 1,024 와 128 로 한 번 정해지고 끝까지 변하지 않는다. **덮어쓰기와 낚음** — 이미 누적된 정보를 제거하거나 정정할 방법이 기제 안에 없고, 모순되거나 갱신된 사실을 제시하는 실험이 없다. **합성** — 같은 S_t 에 들어간 두 문맥은 식 (16-4) 에 의해 무차별 합산되고, 따로 생성된 두 adapter 를 결합하는 실험은 없다. **순서** — 식 (16-4) 가 토큰들의 합이므로 쓰기는 순열 불변이고, 문맥을 섞어도 같은 adapter 가 나온다. 평가된 세 시나리오 중 둘 (MSC 의 다중 세션 대화, MetaICL 의 입출력 시연) 이 순서를 지니는데 순서 민감도를 재는 실험이 없다. **이 순열 불변성은 식 (16-4) 로부터의 이 책의 추론이며 논문이 진술하지 않는다.** **generator 의 이식성** — generator 는 base LM 마다 하나씩 학습되는데 (“the LM-specific adapter generator” [Generative Adapter § 1]) base 모델이 갱신되면 살아남는지에 대한 언급이 없다. 배포하는 쪽의 실제 질문이다. **대용 지표의 타당성** — 설계 결정전부가 [Generative Adapter § 5.1] 의 보고되지 않은 상관 주장 위에 서있다.

경로 간 인용. W -경로는 이름으로 광범위하게 인용된다 — [Generative Adapter § 6] 이 “Fast Weights” 문단으로 열리고 Hinton & Plaut (1987), Ba et al. (2016), Schmidhuber (1992·1993), Schlag et al. (2021) ([Generative Adapter § 6·§ 7] 에서 두번), Clark et al. (2022), 그리고 선형 attention 계열이 들어온다. 다만계보가 **2022 년에서 끝진다** — 2024–2025 년의 test-time training 라인은 없고, 이쪽은 연대가 강제한 침묵이다 (2024 년 11 월 8 일). Θ -경로는 PEFT와 continual pretraining 으로 인용된다 (LoRA, Houlsby et al. 2019, prefix tuning, AdaLoRA, DoRA, Yang et al. 2024, Allen-Zhu & Li 2024, Hu et al. 2023, Tack et al. 2024). 그러나 **consolidation** 으로는 **한 번도 인용되지 않는다** — EWC 도 replay 도 generative replay 도 CLS 도 생물학적 sleep 도 참고문헌에 없다. Θ -경로의 망각·통합 절반이 통째로 빠져 있다. **이쪽은 연대가 강제한 침묵이 아니다** — EWC(2016-12)·Deep Generative Replay(2017-05)·CLS(1995) 는 전부 이 논문보다 앞서고, 논문자신이 [Generative Adapter § 1] 에서 지속학습의 데이터 비효율을 동기화한다. 선택된 침묵이다 ($\rightarrow$ ch11 § 11.6.1). E -경로는 **정확히 한번**, 데이터셋 프로토콜로만 등장한다 — MSC 설정을 Packer et al. (2024) 를 따라 잡는 문장 하나이며 [Generative Adapter § 4.3], MemGPT 는 방법으로서술되지도 baseline 으로 돌려지지도 않는다. RAG 는 [Generative Adapter § 1] 에서 한 번 언급되고 버려진다. sleep-time compute 문헌은 전무하고, 이 논문은 Letta STC 보다 다섯 달 앞서므로 그 침묵은 **한방향으로만** 연대가 강제한 것이다. Part III 가 물어야 할 것은 역방향이다 — E -경로가 이 논문을 되받아 인용했는가.

[평가] 인접하되 구별해야 할 계열이 하나 더 있다. Chevalier et al. (2023)·ICAE·UltraGist·Compressed Context Memory 의 압축 라인은 이 논문에 조밀하게 인용되고 UltraGist 가 유일한 강한 효율 baseline 이다 (표 16-5). 이 책의 프레임에서 이들은 E -경로가 **아니다** — 산출물이 읽기 시점에 소비되는 토큰임베딩이므로 저장소에 쓰는 것이 아니라 프롬프트를 줄인다. 같은 표 안에서 경쟁하지만 층이 다르다. 그리고 가장 아쉬운 공백은 § 16.6 이 지목한 그것이다 — 같은 모양·같은 rank 의 delta 를 gradient 로 적합한 baseline 이 없으므로, **forward 쓰기가 gradient 쓰기 대비 품질을 얼마나 포기하는지 이 논문은 말할 수 없다.**

ch17 이 받아 가는 것은 둘이다. 첫째, **접기를 wake 경로 밖으로 옮기는 이동**. 같은 물건, 다른 시계다 — 그리고 그 이동이 정확히 W -경로 논문을 sleep-time compute 으로 바꾸는 조작이다. 이 장이 판별식조건 (1) 에서 탈락한 자리가 그대로 ch17 의 출발점이 된다. 둘째, $\beta_t = 1$ 과 **축출 부재라는 결론**, 그리고 논문이 [Generative Adapter § 7] 에서 이름으로 가리킨 그 수선택. 용량 천장은 ch18 이, forward 쓰기가 gradient 쓰기에 내주는 것은 ch19 가 받는다.

요약

- Generative Adapter 는 문맥을 한 번의 forward 로 파라미터화한다. 산출물은 LoRA 모양이고 논문이 그 형태를 직접 유도하지만 [Eq. 8–9], 갱신식은 (U-W) 의 gradient-free 변종이고 시계는 chunk 다.
- 경로 판정은 셋 다 뺏아 나온다 — Θ -경로 탈락 (배포 체크포인트가바이트 단위로 불변), E -경로 탈락 (ret 도 레코드도없음), W -경로 통과. 소속 판정은 별개이며 조건 (1) 에서 탈락한다: 접기가 wake 안에서 일어난다.
- **산출물의 모양으로 경로를 나누면 틀린다.** 이 장의 판정이 이책의 방법론적 주장을 실증한다 — 모양은저장·전송·롤백 단위를 정하고, 시계와갱신식이 경로를 정한다.
- (U-W) 의 항 중 넷이 비어 있다: α_t 없음, $\beta_t \equiv 1$, η_t 없음, inner loss ℓ 없음. gradient 자리에 chunk Gram 행렬의 학습된 쌍선형 사영이 들어가고 (k_t, v_t) 가 (h_m, h_m) 으로 붕괴한다.
- 비용 4 중 세 칸이 비어 있다. $L_w \cdot \rho$ 는 논문에없고 B_s 는 그림에만 있으며, C_{cap} 은 32M floats 로보고되되 dtype 이 없어 바이트로 환산되지 않는다. 상각식 (A) 는 닫히지않는다 — 절감 1.554 TFLOPS/질 의는 알지만 C_{sleep} 이 수치로 없다.
- 초록의 헤드라인 수치가 본문 표와 화해하지 않는다. “from 19.5 to 31.5, 63.5% improvement” 에서 31.5 는 논문 어디에도 없고 (표 5 는 32.0), $19.5 \rightarrow 31.5$ 는 61.5%다.

- 품질 우위는 좁다. Mistral 에서 in-context prompting 을 이기는 칸이 열넷중 셋이고, Llama2 의 장문맥 우위는 baseline 이 마지막 4K 로 잘린결과이며, MSC 에서 전체 문맥 품질의 61%를 유지한다 (40.2 대 66.0).
- 서빙 순이익은 문맥 길이와 무관한 고정 상태량과 base 와 동일한 질의당추론 FLOPs 이고, 순비용은 요청당 32M floats·prefill 중간의비-GEMM SVD·merge 형태에서의 shared-weight batching 상실이다.

자가 점검체크리스트

- $\Theta \cdot E \cdot W$ 세 경로 조건을 이논문에 각각 대고, 어느 조건이 어느 문장으로탈락·통과하는지 원문 위치와 함께 말할 수 있다.
- 식 (16-2) – (16-3) 을 표준형 (U-W) 와 항별로 대조해 비어 있는 넷을 짚고, $\beta_t \equiv 1$ 이 ρ 를 왜 측정 대상이 아니라 정의 대상밖으로 밀어내는지 설명할 수 있다.
- 표 16-3 의 세 빈칸이 각각 왜 비어 있는지 말하고, 이 논문에 대해 “효율적” 이라고 쓸 수 없는 이유를 상각식 (A) 의 두 항으로 설명할 수 있다.
- 초록의 31.5·63.5%와 표 16-4 의 32.0·19.5 의관계를 재계산하고, 이 책이 어느 값을 본문에 쓰는지 말할 수 있다.
- 32M floats 가 ch03 § 03.6 의 $|\Delta\Theta| = 2N_{\text{mod}}dr$ 과 S_t 양쪽에서 같은 수로 나오는 것을 재계산하고, 그럼에도 바이트를단정할 수 없는 이유를 말할 수 있다.
- 식 (16-4) 의 순열 불변성을 유도하고, 그것이 MSC 와 MetaICL 에서 왜위험한지, 그리고 그 진술이 논문의 주장이 아니라 이 책의 추론인 이유를말할 수 있다.
- (Rosetta) 이 논문이 왜 “모델 재배포” 행의 물건을 만들면서 “KV cache” 행의 시계로 움직이는지 설명하고, 그 어긋남이 “batch 로 weight 공유” 행에서 어떤 결과로 나타나는지 두 서빙 형태로 나누어 말할 수 있다.

ch17. Do Language Models Need Sleep? — 오프라인재귀

17.1 Bridge-in: 전작이 남긴문제

ch16 은 문맥을 파라미터 모양으로 접는 데 성공했다. Generative Adapter 는 chunk 하나를 forward pass 한 번으로 흡수해 LoRA 모양의 산출물을 만들고, base 체크포인트는 바이트 단위로 그대로 둔다. 그런데 시계가 wake 였다 — 문맥이 주어지면 그 자리에서 접고, 그 접은 결과로 같은 문맥에 답한다. 판별식 조건 (1) 에서 탈락한 이유가 그것이다 (→ ch11 § 11.4.5). **접는기제는 있었고 접는 시각이 없었다.**

이 장의 논문 (Do Language Models Need Sleep? Offline Recurrence for Improved Online Inference, arXiv:2605.26099, 이하 [Do LMs Need Sleep?]) 은 접는 시각을 만든다. 접기를 답 생성경로에서 떼어 내 **KV cache** 를 비우는 순간에 붙이고, 그 자리에서같은 chunk 를 여러 번 다시 통과시킨다. 그러면 W -경로에서 (U - W) 가 wake 의 토큰 시계 위가 아니라 별도의 시계 위에서 추가로 실행되는 구성이처음으로 성립한다. ch11 이 이 논문을 W -경로의 유일한 정면 사례로분류한 근거다 (→ ch11 표 11-2).

제목부터 정리하고 시작한다. 이 논문의 제목은 ch21 이 다루는 Language Models Need Sleep(2606.03979) 과 한 글자차이로 겹치고, 저자들이 그 충돌을 인지해 자기 제목을 고쳤다고 각주에적는다 [Do LMs Need Sleep? 각주 2]. 이 책은 이 논문을 [Do LMs Need Sleep?], ch21 의 논문을 [LM Need Sleep] 으로 부른다. 두 논문은경로도 기제도 다르다 — 이쪽은 W , 저쪽은 Θ 다.

그런데 이 논문에는 **상속받은 open question 이 없다**. “ X 가 남긴문제를 우리가 받는다” 에 해당하는 문장이 논문 어디에도 없고, 인용부호로복원할 선행 질문도 없다. ch14 가 그랬듯 이 장도 없는 계보를 지어내지않는다 (→ ch11 § 11.7). 대신 이 논문이 실제로 물려받은 것은 질문이 아니라**진단**이고, 논문의 첫 동작은 그 진단을 뒤집는 것이다.

물려받은 진단은 이렇다. attention-SSM 하이브리드가 긴 문맥에서 지는이유는 고정 크기 상태의 **용량**이 모자라서다 — Jelassi et al. 의복사 과제 결과와 Arora et al. 의 recall-throughput 트레이드오프가 그자리를 잡았다. 이 논문은 같은 관찰을 받아 원인 귀속을 바꾼다.

“This suggests that the bottleneck is not merely memory capacity as suggested by prior work [32, 2], but the amount of computation available for transforming evicted context into a useful internal state.” [Do LMs Need Sleep? § 1]

그리고 § 2 에서 같은 말을 명시적 반박의 형태로 다시 쓴다 — “Contrary to these works, we show that such models can fail as the required reasoning depth to solve a task increases, even when the amount of information to store is held fixed.” [Do LMs Need Sleep? § 2]

세 갈래의 암묵적 답이 더 붙는다. 첫째, Allen-Zhu 의 k -hop 지식 회수과제에서 SSM 이 transformer 에 크게 지는데 저장할 정보량은 충분하다는 관찰 — 이 논문은 거기에 기제 (consolidation 계산 부족) 와 처방 (오프라인 재귀) 을공급하고 그 과제와 학습 프로토콜을 그대로 가져다 쓴다 [Do LMs Need Sleep? § 6.2]. 둘째, chunk 마다 gradient step 을 **한 번** 도는 test-time training 계

열에 대해 “한 번이면 충분한가” 를 아니오로 답한다 [Do LMs Need Sleep? § 2]. 셋째, depth-recurrence 계열이 예측 시점의 재귀가 도움이 된다는 것을 확립했는데, 이 논문은 그 계열이 묻지 않은 질문에 답한다 — 그 재귀를 예측 경로에서 **통째로 건너낼 수 있는가**. “Unlike previous looped models, our model does not need to loop at prediction time.” [Do LMs Need Sleep? § 1]

[해설] 그래서 이 장의 bridge-in 은 상속 진술이 아니라 **상속된 오진의 교정**이다. 독자의 어휘로 옮기면 이렇다. KV cache 가 버려질 때 남는 것이 고정 크기 상태라는 사실은 모두가 안다. 지금까지 그상태의 문제는 **얼마나 담기는가**로 이야기되었다. 이 논문은 문제를 **담기 전에 얼마나 계산했는가**로 옮긴다. 담는 양이 아니라 담기까지의 계산량이 축이 되면, 그 축은 예산 B_s 이고 그 예산은 wake 밖에 둘 수 있다.

17.2 문제의식

논문이 자기 문제를 세우는 문장은 하나다.

“However, scalable memory is not the same as scalable reasoning. A fast weight memory may support long-range recall [48], but it is unclear whether it can support deep computation over tokens that are no longer present in the KV cache.” [Do LMs Need Sleep? § 1]

이 문장을 실험으로 옮기려면 두 가지를 동시에 고정해야 한다. 저장량을 고정한 채 요구되는 추론 깊이만 올려야 하고, 답을 만드는 쪽의 예산이 늘어나지 못하게 막아야 한다. 논문은 두 개의 제약으로 그것을 만든다.

첫째, **hard-eviction 제약**이다. C 토큰마다 KV cache 를 완전히 비운다 — “we impose a strict context window size as well as a hard-eviction constraint: we clear the context window every 24 tokens” [Do LMs Need Sleep? § 4]. 경계를 넘어 살아남는 것은 SSM 블록의 fast weight 뿐이다. 이 제약의 역할은 성능 향상이 아니라 **채널의 유일화**다. 경계 이후의 정확도는 전적으로 W 에 무엇이 접했는지의 함수가 된다.

둘째, **prediction-phase latency 제약**이다.

“We impose a prediction-phase latency constraint: during the prediction phase, each answer token is predicted with a single standard forward pass. Extra loops or chain-of-thought tokens are disallowed because they increase prediction latency.” [Do LMs Need Sleep? § 4]

이 두 제약 아래에서 wake 는 **토큰당 표준 forward pass 정확히 한번**으로 못 박히고, chain-of-thought 가 데이터에서 제거되어 있어 우회로가 없다. 비교군과 처리군의 wake 경로가 커널 수준에서 동일하다.

[평가] 이 책이 보기에 이 두 제약이 논문의 기제보다 오래남을 기여다. 지금까지 이 책이 다른 논문 중 “wake 비용을 고정한다” 를 조작적으로 정의한 것이 없다. Letta STC 는 wake 토큰 수를 줄였다고 보고했지만 prefill 항을 세지 않았고 (→ ch14 § 14.5.3), E -경로 전반은 회수된 레코드가 프롬프트를 늘리는 항을 회계에 넣지 않는다 (→ ch10). 논문자신은 이 두 제약을 실험 통제로만 소개하고 기여로 내세우지 않는다.

논문의 핵심 주장은 이 무대 위에서 한 문장으로 진술된다 — “Our key insight is that recurrence can be used not only for prediction but also for memory consolidation.” [Do LMs Need Sleep? § 1] 재귀를 예측에 쓰면 test-time compute 이고, consolidation 에 쓰면 sleep-time compute 이다. 같은 연산자의 시계만 바꾼 것이다.

여기에 논문이 스스로 붙이는 방법론적 단서가 하나 있다. 개선의 축이 학습토큰 예산 고정 하의 정확도라는 것이다 — 논문은 과제 실패를 “고정된 학습토큰 예산 안에서 풀리지 않음” 으로 정의하고 그 위에서 곡선을 그린다 [Do LMs Need Sleep? § 4]. 이 정의가 § 17.6 의 해석 전부를 지배한다.

17.3 Core mechanism (통일표기)

17.3.1 표기 대응표

이 논문은 예약 기호와 **정면으로** 충돌한다. 충돌이 세 건이고 그중두 건은 뜻이 정반대로 뒤집히는 종류라, 수식을 하나라도 보이기 전에 대응표를 먼저 놓는다.

표 17-1 — Do LMs Need Sleep? 원 표기 → 이 책의 표기

원 표기	이 책의 표기	심각도	주의
S_t — fast weight 상 태 그 자체 [Eq. 3]	W_t	정면 충돌	예약된 S_t 는 inner momentum 버퍼다. 이 논문에는 momentum 이 아예 없으므로 예약된 뜻의 S_t 는 이 장에 한 번도 등장하지 않는다
$\beta_t \in (0, 1)$ — 쓰기 크 기를 정하는 입력 게이트 [Eq. 3]	η_t	정면 충돌	예약된 β_t 는 momentum decay 다. 이논문의 β_t 는 토 큰당 쓰기 보폭이므로 η_t 의 역할이다
$\alpha_t \in (0, 1)$ — forget 게이트, $\alpha_t S_{t-1}$ [Eq. 3]	α_t (변환 불요)	없음	방향이 이미 통일 규약 과 같다 (남기는 비율). Titans 와 달리 부호 뒤집기가 필요 없다 (→ NM ch12)
L — context window 크기 = hard-eviction 주기 (24, 75, 2000, 512)	C (chunk)	정면 충돌	예약된 맨 L 은 sequence 길이다. 이 아키텍처에서 eviction 창이 곧 chunk 다
T — 전체 sequence 길이 (100, 360, 12,000)	L	교환	위 행의 귀결로 원문의 T 와 L 이 이름을 맞바꾼다 . 이 장에서 가장 위험한 대응이다
D — 블록 수 [Eq. 4 – 5]	L_{layer}	충돌	D 는 Atlas 의 lifted 차원에 예약되어 있다 (→ NM ch14)
t — Rule-110 rollout 단계 수 (4, 8, ..., 32) [§ 4, § 6.1]	n_{roll} (장-국소)	정면 충돌	원문이 같은 문서안에 서 t 를 토큰 첨자와 rollout 깊이 양쪽으로 쓴다. 예약된 t 는 토큰 첨자다
k — Depo hop 수 (1, 2, 4, 8, 16) [§ 3.2, § 6.2]	n_{hop} (장-국소)	정면 충돌	예약된 k 는 sleep 라 운드첨자, k_t 는 key 벡 터다
n — chunk·segment 첨자	j	정면 충돌	n 은 저장 항목 수 전역 예약이다

원 표기	이 책의 표기	심각도	주의
(원문에 없음) — sleep pass 첨자	p (장-국소 신설)	신설	chunk 첨자가 j 로 확정되면서 pass 첨자가 따로 필요해졌다. k (sleep 라운드)· k_t (key 벡터)· s (SGD step) 와 겹치지 않는 글자를 골랐다
h — Algorithm 1 의 은닉 활성화값	$H^{(j,p)} \in \mathbb{R}^{C \times d}$ (장-국소)	충돌	예약된 h_t 는 outer 2 차 moment 다. 검사 검사 pass 첨자 p 를 명시하게 된다
m, m_c — loss mask [Alg. 1]	μ, μ_j (장-국소)	충돌	예약된 m_t 는 outer 1 차 moment 다
c — 토큰 chunk [Alg. 1]	c_j (첨자 필수, 장-국소)	약한충돌	예약된 맨 c 는 Atlas Omega-rule 의 window 길이다
o_t — layer 출력 [Eq. 2–3]	y_t	없음	통상의 통일개명
$\mathcal{L}$ — MaskedCE [Alg. 1]	$\mathcal{L}$ (그대로)	없음	원문이 outer loss 에 $\mathcal{L}$ 을 써서 통일 규약과 우연히 일치한다. TNT 와는 반대다 ($\rightarrow$ NM ch15)
N — sleep pass 수	N — 이 장이 신설하는 예약 기호	신설	Titans 의 N_p (persistent memory 토큰) 와도, N_q (문맥을 공유하는 질의 수) 와도 다르다
q_t, k_t, v_t ; W_Q, W_K, W_V ; K_t, V_t	동일	없음	원문이 이미 열벡터로 선언한다 [§ 3.1]. 전치 변환이 필요 없는 드문 경우다
$\mathcal{M}(\cdot; W)$ — 읽기 함수	해당 없음	부재	이 논문에는 memory 모듈 추상이 없다. 읽기가 $y_t = W_t q_t$ 로 직접 쓰인다 [Eq. 3]. 없는 $\mathcal{M}$ 을 소급해 써주지 않는다

$T \leftrightarrow L$ 교환이 특히 위험하다. 아래 본문에서 $C = 2000$ 이고 L 이 2,000–3,300 이라고 쓰면 원문에서는 $L = 2000, T \in [2000, 3300]$ 이다. 원문과 대조할 독자는 이 표를 손에 들고 읽어야 한다.

17.3.2 한 pass 안의 쓰기

논문이 설명용으로 내거는 갱신 규칙은 Mamba-2 형태의 게이트된 Hebbian 쓰기다.

$$W_t = \alpha_t W_{t-1} + \eta_t v_t k_t^\top, \quad y_t = W_t q_t \quad (17-1)$$

식 (17-1) 은 [Do LMs Need Sleep? Eq. 3] 이며 개명 세건 ($S_t \rightarrow W_t, \beta_t \rightarrow \eta_t, o_t \rightarrow y_t$) 만 적용한 것이다. 표준형 (U-W) 와 두 군데가 다르다. 첫째, momentum 항이 **없다** — $\beta_t S_{t-1}$ 에 해당하

는 것이 아예 존재하지 않으므로 통일 표기로 옮기면 $S_t \equiv \eta_t v_t k_t^\top$, $\beta_t \equiv 0$ 이다. 둘째, gradient 자리에 raw outer product 가들어간다. $-\eta_t \nabla_W \ell(W_{t-1}; k_t, v_t)$ 가 아니라 $+\eta_t v_t k_t^\top$ 이다. 게이트 α_t, η_t 는 둘 다 x_t 에서 계산되는 데이터 의존 게이트다 [Do LMs Need Sleep? § 3.1].

논문은 이 차이를 실수가 아니라 입장으로 방어한다.

“our method uses a learned recurrent forward pass as the memory-update rule, allowing more flexible forms of consolidation that need not correspond to a one-step gradient descent on a fixed scalar objective.” [Do LMs Need Sleep? § 2]

그런데 논문이 보여 주는 식은 논문이 돌린 식이 아니다. 바로 다음문장이 그렇게 말한다 — “In our experiments we use Gated Delta Networks (GDNs), which add a delta-rule correction to this update; however, the specific update rule does not matter for our discussion.” [Do LMs Need Sleep? § 3.1] 실제로 돌아간 규칙은 delta 보정이 붙은 형태이고 논문은 그것을 한 번도 전개하지 않는다. [Do LMs Need Sleep? § 6.3]에서는 세번째 규칙이 등장한다 — Jet layer 는 “dynamic convolution instead of the fixed convolution in GDN” 으로만 서술된다 [Do LMs Need Sleep? § 6.2]. 세 규칙 중 어느 것이 일을 하는지 가르는 ablation 은 없다.

17.3.3 이 장이 소유하는 두정의

정의. KV cache 를 비우기 전 **offline recurrence** 로 **fast weight** 에 접는 기제란, eviction 경계에서 KV cache 를 버리기 직전, 같은 chunk 의 토큰에 대해 블록 스택을 N 번 더 통과시켜 각 SSM 블록의 fast weight $W^{(\ell)}$ 만 갱신하고, 그 과정에서 정제된 은닉 특징 H 와 재구성된 KV cache 는 경계에서 폐기하여 오직 W 만 경계를 넘게 하는 절차를 말한다. 세 요소가 모두 필요하다 — 경계직전이라는 시각, 같은 토큰의 반복 흡수, 그리고 W 이외 전부의 폐기. 하나라도 빠지면 다른 기제다. 특징을 함께 넘기면 depth-recurrent 예측모델이고, 경계 이후에 돌리면 문맥 재적재이며, 한 번만 돌리면 통상의 SSM 이다.

index 를 명시해 쓰면 이렇다. chunk 첨자를 j , pass 첨자를 p 로둔다.

$$\begin{aligned} W^{(0,N)} &= 0, \\ H^{(j,0)} &= \text{Embed}(c_j), \quad W^{(j,0)} = W^{(j-1,N)}, \\ (H^{(j,p)}, W^{(j,p)}) &= \text{Blocks}(H^{(j,p-1)}, W^{(j,p-1)}; \Theta), \quad p = 1, \dots, N, \\ &\text{그 뒤 } H^{(j,N)} \text{과 KV cache 를 폐기. } W^{(j,N)} \text{만 경계를 넘는다.} \end{aligned} \tag{17-2}$$

식 (17-2) 는 [Do LMs Need Sleep? Alg. 1, § 5] 의 의사코드를 첨자를 드러내 옮긴 것이다. 원문은 for $n = 1, \dots, N$ 안에서 $h, S \leftarrow \text{Blocks}(h, S)$ 로만 쓰고 첨자를 적지 않는다. 그 의사코드 가지고 있고 산문이 말하지 않는 사실이 셋이다.

1. W 는 **pass** 사이에 초기화되지 않는다. pass p 는 pass $p-1$ 이 끝난 상태에서 시작하므로 같은 C 개 토큰이 누적적으로 N 번 접힌다.
2. H 는 **chunk** 안에서 **pass** 를 건너 운반된다. 그래서 pass 마다 (k_t, v_t) 가 달라진다. 그리고 경계에서 파괴된다.
3. 따라서 chunk 를 넘는 유일한 채널이 W 다 — “Unlike prior depth-recurrent models where gradient flows through recursively refined feature vectors, the gradient flows through the refined fast weights because we discard the refined features after sleep.” [Do LMs Need Sleep? § 5]

토큰 수준으로 내리면 N 이 무엇을 하는지가 드러난다. $\xi(t, C)$ 를 chunk 시작 offset 이라 하면,

$$\begin{aligned} k_t^{(p)} &= W_K [H^{(j,p-1)}]_{t,:}, & v_t^{(p)} &= W_V [H^{(j,p-1)}]_{t,:}, \\ W_t^{(j,p)} &= \alpha_t^{(p)} W_{t-1}^{(j,p)} + \eta_t^{(p)} v_t^{(p)} (k_t^{(p)})^\top, & W_{\xi(t,C)}^{(j,p)} &= W^{(j,p-1)}. \end{aligned} \quad (17-3)$$

식 (17-3) 이 이 장의 중심 문장을 담는다. pass p 의 게이트 $\alpha_t^{(p)}, \eta_t^{(p)}$ 는 $W^{(j,p-1)}$ 이 만들어 낸 특징의 함수다. 즉 상태 $\rightarrow$ 특징 $\rightarrow$ 게이트 $\rightarrow$ 상태의 닫힌 고리를 **입력 토큰을 열려 둔 채** N 번 도는 것이다.

정의. sleep 깊이 N 을 늘린다는 것은, 같은 chunk 를 W 에 N 번 누적해 쓰되 매 pass 의 (k_t, v_t) 를 직전 pass 의 W 가 만든 특징에서 다시 뽑는다는 뜻이다. 그러므로 이것은 “(U-W) step 을 더 밟는 것” 이 아니라 **(U-W) 와 특징 정제 고정점 반복의 합성**이다. 늘어나는 것은 저장량이 아니라 evicted 문맥 위에서 수행 가능한 **계산의 깊이**이며, 고정된 스칼라 목적함수에 대한 N 번의 하강이 아니다.

이 정의가 왜 필요한지는 대조로 분명해진다. chunkwise-parallel 학습에서 한 chunk 안의 gradient 는 전부 stale snapshot $W_{\xi(t,C)}$ 에서 평가된다 ($\rightarrow$ NM ch09). 여기서는 pass 내부의 재귀가 통상의 online SSM 재귀그대로이고, snapshot 의미론이 **pass 수준으로 올라간다** — pass p 는 chunk 내용이 이미 $p-1$ 번 쓰인 상태에서 출발한다.

마지막으로 두 가지를 못 박는다. W_{init} 은 meta-learn 되지 않는다. 매 sequence 에서 0 으로 초기화된다 [Do LMs Need Sleep? Alg. 1]. Titans-TNT 계열이 지고 있던 학습된 초기 상태가 여기에는 없다 ($\rightarrow$ NM ch12, NM ch15). 그리고 sleep 중에 Θ 는 움직이지 않는다 — sleep 은 forward pass 다. “learned local rule” 의 learned 는 게이트 생산자와 투영 행렬이 **outer loop** 에서 학습되었다는 뜻이지, test-time 에 규칙이 학습된다는 뜻이 아니다.

17.4 어느 층을 언제 쓰는가

이 논문이 **test-time** 에 움직이는 층은 W 하나다. Θ 는 배포 전 outer loop 에서만 움직이고, E 는 존재하지 않는다 — 텍스트로도 벡터로도 그래프로도 아무것도 쓰지 않는다. 그러므로 층 사이의 순서 문제가 성립하지 않는다. corpus 에서 갱신 대상이 단일한 몇 안 되는 논문이다.

표 17-2 — 이 값은 누가 학습하는가

객체	사는 곳	outer loop 학습	sleep 에서 이동	wake 에서 이동	출처
Θ — W_Q, W_K, W_V , MLP, nor- malization, 출력 투영, embedding	slow weights	예. MaskedCE 에 대해 Muon(+AdamW, lr 5e-5) 으로 전체 그래프 관통	아니오 — sleep 은 forward pass 다	아니오	Alg. 1; § 6
α_t, η_t 의 게이 트 생산자 (“learned local rule”)	slow weights ($\subset \Theta$)	예. 규칙이 “학습된다” 는 말의 유일 한 의미	아니오. 생산 자는 고정이고 출력 $\alpha_t^{(p)}, \eta_t^{(p)}$ 만 입력 특징 을 따라 pass 마다 달라진다	아니오	§ 3.1

객체	사는 곳	outer loop 학습	sleep 에서 이동	wake 에서 이동	출처
$W^{(\ell)}$ — SSM/ GDN/Jet 블 록당 fast weight 행렬	fast weights	저장량으로는 아니오. 학습 중 gradient 가 통과할 뿐이 며 초기값은 meta-learn 이 아니라 0 이다	예 — sleep 이 움직이는 유일 한 객체. chunk 에 대 한 N 회 누적 sweep	예 — 답 토큰 을 처리하는 통상의 단일 pass 재귀도 W 에 쓴다	Alg. 1; § 5
$H^{(j,p)}$ — sleep 내부의 정제된 은닉 특징	활성값 (일 시적)	아니오 (파라 미터가 아 니다)	예 — N 개 pass 를 건너 운반	경계에서 파괴	Alg. 1; § 5
KV cache K_t, V_t	비모수 상태	아니오	N 개 pass 마 다 재구축된 뒤 비워짐	hard eviction 에 서는 예측 chunk 시작 시 비어 있음. sliding window 에서 는 최근 $C - 1$ 토큰 유지	§ 5; § 6.4
N — sleep 깊이	하이퍼파라 미터	아니오 — run 마다 고 정. N 값마다 별개로 학습된 모델 이다	아니오 — 적 응적이지 않고 halting 기제 가 없다	아니오	§ 6.1; § 6.3
C — eviction 창	하이퍼파라 미터	아니오 — 과 제마다 고정 (24 / 75 / 2000 / 512)	아니오	아니오	§ 4; § 6.2; § 6.3; § 6.4
Ouro 1.4B 에 삽입한 Jet layer 6 개	slow weights(신 규 추가, 파라 미터 10% 미 만 증가)	예. sliding- window 설 정에서는 2 단 계 — Jet layer 만 1 epoch warm-up 후 전체 모델 2 epoch	아니오	아니오	§ 6.3; § 6.4
E — external store	—	해당 없음	해당 없음	해당 없음	이 논문에 external store 가 없다

표에서 읽어야 할 행은 $W^{(\ell)}$ 행이다. 같은 상태가 두 시계에서 갱신된다. sleep 에서는 eviction 경계마다 N 번 누적으로, wake 에서는 답 토큰을 처리하며 토큰마다 한 번씩이다. 두 시계가 같은 갱신규칙 (17-1) 을 쓰고 같은 Θ 가 만든 게이트를 쓴다. 이 논문의 sleep 이 별도의 학습 절차가 아니라 같은 연산자의 반복 적용인 이유가 여기 있다.

언제 자는지는 누가 정하는가. 학습 시점에는 **loss mask** 가 정한다. Algorithm 1 은 chunk 의 mask μ_j 가 전부 0 이면 N 번 loop 를 돌고, 그렇지 않으면 한 pass 만 돌고 손실을 쟀다 [Do LMs

Need Sleep? Alg. 1]. 즉 sleep/wake 분할이 런타임 정책이 아니라 **라벨 배치**에서 나온다. 서빙에서 그 분할을 무엇이 정할지는 논문이다루지 않는다.

17.4.1 판별식을 정면으로 통과한다는것

정의. W -경로가 판별식을 정면으로 통과한다는 것은, $(U-W)$ 를 wake 의 토큰 시계 위에서가 아니라 **eviction 경계라는 별도의 시계 위에서 추가로** 실행하고 ($\Theta \cdot E$ 는 건드리지 않는다), 그 추가 실행이 $B_s > 0$ 을 소비하며, 그 결과인 W 가 그 실행을 유발하지 않은 질의에 소비되는 구성을 말한다. E -경로는 모델박 레코드를, Θ -경로는 배포 아티팩트를 바꾸는 데 반해, 여기서는 요청에 붙는 가중치 형태의 상태만 바뀐다.

네 조건을 이 논문에 하나씩 댄다 ($\rightarrow$ ch11 § 11.2.1). 조건 (2) 와 (3) 은 이론의 여지가 없다 — N 개의 추가 forward pass 가 $B_s > 0$ 이고, 바뀌는 것은 W 다. 조건 (1) 과 (4) 는 **실험 설정마다 답이 다르다.**

Depo 설정에서는 둘 다 만족한다. 한 cycle 이 최대 75 개노드 · 최대 300 토큰으로 서술된 뒤 그 뒤에 질의-답 쌍 10 개가 최대 60 토큰에 걸쳐 붙고 전체 $L = 360$ 이며, 창은 $C = 75$ 이라 cycle 하나가 네 개 cache 창에 걸쳐 조각난다 [Do LMs Need Sleep? § 6.2]. consolidation 이 질의보다 앞서고, 만들어진 W 를 **열 개의 질의가 공유한다**. 그리고 저자들이 그 설정의 성격을 명시한다 — “the model must form a query-agnostic representation because both k and the start node are randomly sampled for each example” [Do LMs Need Sleep? § 6.2].

헤드라인인 GSM-Infinite 설정에서는 다르다. 논문이 질문을 문맥앞에 놓고 chain-of-thought 를 데이터에서 제거한다.

“We place the question before the context and exclude Chain-of-Thought traces from the data... This order gives the model the query before it reads the long problem context, allowing it to selectively consolidate information relevant to the question while ignoring filler tokens.” [Do LMs Need Sleep? § 6.3]

이 배치에서 consolidation 은 질의 조건부다. 조건 (1) 이 문자 그대로는 성립하지 않는다 — 상태를 바꾸는 연산이 질의 도착 뒤에 실행된다. 조건 (4) 도 약해진다. 요청당 질의가 하나이므로 바뀐 상태를 읽는 질의가 그 상태를 만든 질의와 같다. 남는 것은 “답 토큰이 나오기 전에” 라는 약화된 시계뿐이다.

[평가] ch11 이 이 논문을 통과로 분류한 것은 옳고, 그 통과는 **Depo 설정이 지고 있다**. 헤드라인 실험은 판별식의 조건 (1) 과 (4) 를 동시에 약화한 구성이며, 그 대가는 정확히 상각의 소멸이다 — 질의조건부 consolidation 에는 나눌 N_q 가 없다. 이 책은 두 사실을 함께 적는다. 이 논문은 W -경로가 sleep-time compute 이 될 수 있음을 보인 유일한 정면 사례이면서, 자기 최대 성과를 그 성질을 포기한 설정에서 얻었다.

17.5 비용 4 종

네 칸이 전부 비어 있다. 하나도 수치로 보고되지 않는다. 대신 이 논문에는 다른 논문에 없는 것이 있다 — 세 칸이 왜 비었는지가 구조적으로 설명된다.

표 17-3 — 비용 4 종

기호	값	논문이 말하는 것	출처
B_s	논문에 없음 (수치). 구조로만 진술된다	eviction 경계마다 loop 범위에 대한 N 회 추가 forward pass. loop 범위는 from-scratch 모델과 Ouro 는 전 블록, Jet-Nemotron 은 28 개 중 가운데 14 개. 실행된 N 은 $\{1, 2, 3, 4\} \cdot \{1, 2, 4\} \cdot \{1, 2, 4, 6\}$. “training cost grows roughly linearly with the number of recurrent steps N ”	§ 6.3; § 6.5
L_w	논문에 없음 (측정). 아키텍처 보장으로만 존재	답토큰마다 표준 forward pass 정확히 한 번, 추가 loop·CoT 금지. “this shifts extra computation to the sleep while preserving the latency of wake-time prediction”	Abstract; § 4
C_{cap}	바이트는 논문에 없음. 고정 크기라는 사실과 shape 재료만 있다	“Unlike the KV cache K_t and V_t , the fast-weight S_t does not grow in size with t .” $d = 256$ (4 층하이브리드), $d = 512$ (10 층 Depo), 28 블록 (Jet, 14 개 loop), Jet layer 6 개삽입 (Ouro)	§ 3.1; § 6; § 6.3
ρ	논문에 없음	이름이 붙은 유일한 retention 기제는토큰당 forget 게이트 $\alpha_t \in (0, 1)$ 이고, 이는 sequence 안의감쇠이지 측정된 열화율이 아니다	§ 3.1

빈칸의 성격이 칸마다 다르므로 하나씩 적는다.

B_s 는 FLOP 도 경계당 토큰 수도 없다. 유일한 계산량 수치는 실험 전체의훈련 비용이다 — “The automaton experiments require less than one A6000 GPU-day. The Depo and GSM-Infinite experiments require roughly 1–2 H100 GPU-days per run.” [Do LMs Need Sleep? § 6] 이 값은 세션당 sleep 예산이 아니라 run 당 학습 예산이다. **추론 시점의 sleep 비용은 어떤형태로도 보고되지 않는다.**

C_{cap} 은 유도조차 불가능하다. 블록당 상태차원 ($d_k \times d_v$), head 수, dtype 이 전부 없다. 세션

당 바이트를 쓰려면 GDN 과 Jet 의 기본값을 논문 밖에서 들여와야 하므로, 이 책은 그 칸을 채우지 않고 공백으로 기록한다.

ρ 는 관측될 레짐에 논문이 들어가지 않는다. 실제로 지나가는 consolidation 경계 수가 automaton 에서 4 개 ($L = 100, C = 24$), Depo 에서 4 개, GSM-Infinite hard eviction 에서 1 – 2 개 ($L \in [2000, 3300], C = 2000$), sliding window 에서 대략 4 – 6 개다 [Do LMs Need Sleep? § 4, § 6.2, § 6.3, § 6.4]. 게다가 W 가 sequence 마다 0 으로 초기화되어 요청을 넘어 존속하지 않는다. 반복 consolidation 의 열화를 썰 대상이 애초에만들어지지 않는다.

L_w 가 가장 중요한 칸이다.

[평가] 헤드라인 지연 주장은 측정된 적이 없다. “wake-time 예측의 지연을 보존하면서 추가 계산을 sleep 으로 옮긴다” 는 초록의 문장은 프로토콜 구성상 참이다 — 답 토큰마다 표준 forward pass 한번으로 못 박아 두었으니 wake 경로가 baseline 과 같을 수밖에 없다. 그러나 그것은 정의에 의한 참이지 측정이 아니다. ms/token 도, TTFT 도, wake 시점 tokens/s 도 논문에 없다. 그리고 같은 침묵이 반대편에도 있다 — sleep 자체의 서빙 시점벽시계 비용도 측정되지 않는다. 논문의 유일한 throughput 그림은 명시적으로 혼란 측정이다 (“Training throughput comparison on 1 NVIDIA H200 GPU” [Fig. 6 캡션]). systems 독자를 향해 지연 성질로 팔면서 systems 측정을 하나도 제시하지 않는다. 이 책은 표 17-3 의 L_w 칸에 “논문에 없음” 이라고 쓰고, “효율적” 이라는 서술을 본문에 쓰지 않는다 (→ ch01 의 회계 규약).

상각식 (A) 는 설정마다 다르게 거동한다. Depo 에서는 $N_q = 10$ 이 구조적으로 정해져 있다 — 한 번 접은 W 를 질의 열 개가 쓴다. corpus 에서 N_q 가 설계로 확정된 드문 자리이지만, 정작 이 논문은 상각을 논증하지 않고 C_{sleep} 을 재지 않아 손익분기 N_q^* 를 계산할 수 없다. GSM-Infinite 헤드라인에서는 $N_q = 1$ 이므로 (A) 가 $C_{\text{avg}} = C_{\text{wake}} + C_{\text{sleep}}$ 으로 붕괴한다. **상각이 아니라 재배치다.** wake 에서 뻥 계산이 같은 요청 안의 다른 국면으로 옮겨 갈 뿐이고, 요청 전체의 계산량은 N 에 대략 비례해 늘어난다.

17.6 실험과 스케일

17.6.1 프로토콜 — 읽기 전에 알아야 할 것들

첫째, 세 설정이 모두 합성이고 전부 저자가 만들었거나 재구성했다. Rule-110 무대는 온전히 저자들의 구성이고, Depo 는 Allen-Zhu 의 과제이되 저자들이 다시 파라미터를 잡았으며, GSM-Infinite 는 절차적 생성기라 저자들이 학습·평가 집합을 스스로 뽑았다 [Do LMs Need Sleep? § 4, § 6.2, § 6.3]. 확립된 긴 문맥 벤치마크는 각각의 대상으로만 등장한다 — “Unlike retrieval-focused long-context tasks such as RULER [31]...” [Do LMs Need Sleep? § 6.3]. 자연 텍스트 위의 결과가 없다.

둘째, **learning rate 프로토콜이 양방향으로 비대칭인데 한 방향만 진술된다.** “We tune the Muon learning rate on the $N = 1$ model, giving the no-loop baseline an advantage, and use the selected value, $2e-3$, for all looped models.” [Do LMs Need Sleep? § 6] baseline 에 이점을 준다는 서술은 정당하지만, 같은 문장이 **처리군은 한 번도 튜닝되지 않았**다는 뜻이기도 하다. AdamW lr 은 탐색 없이 $5e-5$ 로 고정했고, [Do LMs Need Sleep? § 6.3] 의 Muon lr $1e-3$ 은 선행 연구를 따랐을 뿐 튜닝하지 않았 [Do LMs Need Sleep? § 6]. loop 모델은 유효 깊이가 다르므로 $N = 1$ 에서 고른 lr 을 이식하는 것은 방향을 알 수 없는 교란이다.

셋째, **시드가 하나다.** “For fair comparison, we fix random seeds ensuring that all runs use exactly the same data ordering.” [Do LMs Need Sleep? § 6] Fig. 2 – 6 어디에도 신뢰구간도 음영대도 반복 실행도 없다. batch 크기는 512(automaton)·128(Depo)·256(GSM-Infinite) 이다.

그리고 이 절 전체를 지배하는 사실이 하나 더 있다. **계산량을 맞춘 baseline 이 없다.** loop 는 비용을 N 에 대략 비례해 늘리는데 [Do LMs Need Sleep? § 6.5], Fig. 2b·3·4·5 의 모든 비교는 학습 토큰 또는 **step** 을 맞춘 것이지 FLOP 을 맞춘 것이 아니다. 같은 x 좌표에서 $N = 4$ 실행은

$N = 1$ 실행의 대략 4 배 계산을 이미 썼다. “재귀가 통상의 용량이나 통상의 계산으로는 살 수 없는 것을 산다” 는 중심주장이 더 넓거나 더 깊거나 더 오래 학습한 비-loop 모델과 겨뤄진 적이 없다.

17.6.2 Rule-110 — 계산이 병목이라는 논증

무대는 이렇다. 길이 24 의 독립 이진 문자열 넷, 문자 단위 토큰라이저, 상태토큰 96 개 + 라벨 토큰 4 개로 $L = 100$, hard eviction 주기 $C = 24$. 라벨 i 는 상태 i 를 Rule-110 으로 n_{roll} 번 전이시킨 뒤의 첫 비트다 [Do LMs Need Sleep? § 4]. 모델은 4 층 GDN-attention 하이브리드, $d = 256$, 배치 attention $\rightarrow$ GDN $\rightarrow$ attention $\rightarrow$ GDN 이다.

Fig. 2a 가 논문의 진단을 보인다. n_{roll} 을 4 에서 16 까지 올리면 **sequence 길이를 고정한 채** 비-loop 하이브리드의 정확도가 무너진다. 저장할 정보량이 그대로인데 요구되는 계산 깊이만 올라간 것이므로, 실패의 원인이 용량이 아니라는 논증이 성립한다. 곡선의 수치 끝점은 인쇄되어 있지 않다.

Fig. 2b 가 처방이다. $n_{\text{roll}} = 32$ 에서 $N = 1$ 은 “remains close to random guessing, reaching only about 10% exact accuracy after nearly 5B training tokens”, $N = 2$ 는 “approximately 20% accuracy”, $N = 3$ 과 $N = 4$ 는 “above 30%” 다 [Do LMs Need Sleep? § 6.1]. 저자들의 귀속은 정확하다 — “Because the context length, eviction rule, and prediction-phase computation are fixed across these runs, the improvement comes from additional consolidation-time computation during sleep.” [Do LMs Need Sleep? § 6.1]

두 가지를 함께 적어야 한다. 첫째, **과제는 여전히 풀리지 않았다**. 거의 5B 학습 토큰 뒤의 최고 치가 30%대이고 baseline 은 무작위 추측근처다. sleep 이 돕는다는 논증은 성립하고 sleep 이 과제를 다룰 만하게만든다는 논증은 성립하지 않는다. 둘째, 초록의 “regular transformer as well as SSM-attention hybrid models fail” 은 결과가 아니라 **동어반복**이다. 논문 자신이 이유를 적는다 — “Under this hard eviction constraint, a standard transformer cannot do better than random guessing as the KV cache has been destroyed before prediction is made.” [Do LMs Need Sleep? § 4] 경계를 넘어 살아남는 상태가 정의상 없는 것이다. 캐시를 유지하도록 허용한 full-attention baseline 은 **어느과제, 어느 스케일에서도 실행되지 않았다**.

17.6.3 Depo — 수치가 없는 절

Depo 는 Allen-Zhu 의 n_{hop} -hop 지식 회수 과제다. cycle 하나가 최대 75 노드 · 최대 300 토큰이고 양쪽 시점에서 300 으로 left-padding 되며, 질의-답 쌍 10 개가 최대 60 토큰, $L = 360$, $C = 75$ 다. hop 수는 학습 중 [1, 16] 에서 균등 추출하고 시험은 $n_{\text{hop}} \in \{1, 2, 4, 8, 16\}$ 에서 켜다 [Do LMs Need Sleep? § 6.2].

이 절은 정량치를 하나도 보고하지 않는다. 정확도도, 최종손실값도, 표도 없이 0 - 100k step 의 시험 손실 곡선만 있다. 확보되는 결과는 계단 모양의 정성 진술이다 — “increasing the number of offline loops improves learning speed for queries that require 4 or more hops. The 1-loop model makes little progress on 4-hop and harder queries, and the 2-loop model similarly stalls on 8-hop and harder queries. Within our training budget, only the 4-loop model begins to improve on the hardest 16-hop task.” [Do LMs Need Sleep? § 6.2] 마지막 문장이 말하는 것은 16-hop 이 풀렸다가 아니라 **움직이기 시작했다**이다.

그리고 이 절에는 논문 내부 모순이 있다. [Do LMs Need Sleep? § 6] 의 실험 세부는 [Do LMs Need Sleep? § 6.2] 용으로 “10-layer 모델을 $d = 512$ 로 처음부터 학습했다” 고 적는데, Fig. 3 의 캡션은 같은 결과를 “4-layer GDN-attention hybrid” 의 것이라고 적는다 [Do LMs Need Sleep? § 6, Fig. 3 캡션]. 어느 모델이 Fig. 3 을 냈는지 독자가 판정할 수 없고, 확인할 부록이 본문에 없다.

17.6.4 GSM-Infinite — 실모델 · 실수치

여기가 유일하게 사전학습 모델 · 실제수치 · 포화/이득 패턴이 함께 나오는 자리다. 문제당 2,000 - 3,300 토큰, 연산 수는 [1, 8] 에서 균등 추출, 평가는 저자들이 뽑은 held-out 1,600 예제다 [Do LMs Need Sleep? § 6.3].

표 17-4 — GSM-Infinite 정확도 (원문 정밀도 그대로)

설정	모델 · loop 범위	연산 수	$N = 1$	최대 N	논문의 상대 개선 라벨
hard eviction $C = 2000$	Jet- Nemotron 2B · 28 블록 중 14	6	0.742	0.812 ($N=6$)	9%
"	"	8	0.351	0.388 ($N=6$)	11%
"	Ouro 1.4B · 전 블록	6	0.419	0.615 ($N=4$)	47%
"	"	8	0.210	0.272 ($N=4$)	30%
sliding window $C = 512$	Ouro 1.4B · 전 블록	2	0.596	0.905 ($N=4$)	52%
"	"	4	0.839	0.926 ($N=4$)	10%
"	"	6	0.251	0.320 ($N=4$)	27%
"	"	8	0.116	0.137 ($N=4$)	18%

라벨은 전부 **상대** 개선이다. 절대 점수 차는 각각 7.0·3.7·19.6·6.2·30.9·8.7·6.9·2.1 백분점이다 [본서 산술]. 출처: [Do LMs Need Sleep? § 6.3, § 6.4, Fig. 4, Fig. 5].

모델 두 개의 성격이 다르다. Jet-Nemotron 2B는 Qwen 2.5 1.5B에서 일부 attention layer를 Jet layer로 바꿔 만든 하이브리드이고, Ouro 1.4B는 원래 attention 전용 looped 모델이라 fast weight 기억을 주기 위해 MLP 없는 Jet layer 6개를 삽입했다 (파라미터 10% 미만 증가) [Do LMs Need Sleep? § 6.3]. Ouro의 이득이 큰 것에 대해 저자들은 “The gap is wider for Ouro, which may reflect its depth-recurrent pretraining” 이라고만 적는다.

표에 넣지 않은 네 줄이 더 중요하다.

쉬운 문제에서 N 은 아무것도 사지 않는다. Jet의 2연산 네 값은 0.985 / 0.985 / 0.980 / 0.979, 4연산은 0.995 / 0.994 / 0.993 / 0.992로 $N \in \{1, 2, 4, 6\}$ 전체의 폭이 각각 0.6과 0.3 백분점이다. Ouro의 2연산은 0.868 / 0.863 / 0.857 (폭 1.1), 4연산은 0.932 / 0.923 / 0.903 (폭 2.9)이다. **어느 끝이 어느 N 인지 논문은 배정하지 않는다.** 배정과 무관하게 확정되는 것은 폭 자체이고, 6·8연산의 7.0·19.6 백분점과 나란히 놓으면 이득이 어려운 꼬리에만 있다는 사실이 남는다. 저자들의 서술은 포화만 인정한다 — “For easier two- and four-operation problems, accuracy often approaches saturation regardless of the number of loops, especially for Jet, which has more fast weight memory capacity than Ouro.” [Do LMs Need Sleep? § 6.3]

N 에 대한 비단조가 있다. Ouro 8연산 패널에 인쇄된 세 값은 0.272 / 0.210 / 0.209이고, 산문이 no-loop를 0.210, 4-loop를 0.272로 지목하므로 [Do LMs Need Sleep? § 6.3] 남는 0.209가 2-loop다. 즉 가장 어려운 연산 수에서 $N = 2$ 가 $N = 1$ 보다 근소하게 **낮다**. 이 배정은 논문이 명시하지 않고, 반전자체를 논의하지도 않는다. 초록의 주장은 “sleep 깊이를 늘리면 성능이 오른다”이다.

가장 큰 헤드라인은 결손의 회복이다. sliding-window 2연산의 52%는 0.596에서 0.905로 가는 값인데, **같은 모델**이 hard eviction $C = 2000$ 에서 같은 2연산을 loop 없이 0.857–0.868로 푼다 (표 17-4 위쪽과 § 17.6.4 앞문단). 논문 스스로 “this baseline performs poorly even on two-operation problems” 라고 적는다 [Do LMs Need Sleep? § 6.4]. 새 능력이 아니라 설정이 만든 구멍을 메운 값이다.

배포에 가장 가까운 설정이 레시피 없이는 학습되지 않는다. sliding window에서는 sleep 뒤 최근 $C - 1$ 토큰을 남기고 그보다 오래된것만 축출하므로 $N = 1$ 이면 통상의 SWA-SSM 하이브리드가 된다. 그런데 학습에 2단계가 필요하다 — Jet layer만 1 epoch warm-up 한 뒤 전체 모델을 2 epoch 학습하며, “We find that for $N > 1$, using hard eviction for the warm-up stage is crucial for the model to learn to refine the fast weights.” [Do LMs Need Sleep? § 6.4] 왜 그런지는 설명되지 않고 레시피로만 보고된다.

17.6.5 throughput 과 스케일상한

논문의 유일한 systems 측정은 Fig. 6 이고 **훈련 측정이다**. Ouro 1.4B, H200 한 장, $L = 12,000$, FlashAttention 2, 창 크기 1K·2K·4K. 결과는 두 문장이다 — 창이 충분히크면 문맥 창을 가로지르는 직렬성이 완전 병렬 baseline 대비 throughput 을 의미 있게 바꾸지 않는다, 그리고 “Throughput is roughly inversely proportional to N ” [Do LMs Need Sleep? Fig. 6 캡션]. Fig. 6b 는 OOM 을 피하려고 문맥 chunk 측에 대한 activation checkpointing 을 켜야했다. 곡선 값은 인쇄되어 있지 않아 인용할 수 있는 숫자가 없다.

실증 상한을 정직하게 적으면 이렇다. 가장 큰 모델이 2B 와 1.4B, 가장 긴과제 sequence 가 3,300 토큰 (12,000 토큰은 아무것도 학습하지 않고 어떤과제도 평가하지 않는 throughput 프로브에서만 등장한다), 최대 sleep 깊이가 6 이고 그것도 블록 절반만 도는 Jet 에서다. run 당 계산량은 1-2 H100 GPU-day 다. 그리고 Ouro 의 N 은 성능이 아니라 **학습 메모리때문**에 4 에서 잘렸다 — “To keep memory cost during training manageable while using a reasonable batch size, we use $N = \{1, 2, 4\}$ for Ouro.” [Do LMs Need Sleep? § 6.3] 파라미터에서도 문맥에서도 배포 규모보다서너 자릿수 아래이며, 이미 1.4B·3.3k 토큰에서 메모리 제약에걸려 있다.

17.7 Systems/serving 합의

독자의 1 번 질문에 대한 이 논문의 답은 **decode**에서는 **아무것도바뀌지 않는다**이다. 그리고 그 답이 참인 방식이 곧 이 논문의 systems 공백이기도 하다.

wake 경로가 **baseline** 과 커널 단위로 같다. 답 토큰마다 표준 forward pass 한 번, 추가 loop 없음, chain-of-thought 토큰 없음이다 [Do LMs Need Sleep? § 1, § 4]. 따라서 토큰당 decode FLOPs 도, decode 가끝어오는 가중치 트래픽도, 산술 강도도 $N = 1$ 실행과 같다 [본서추론]. 달라지는 것은 W 의 **내용**뿐이다. E -경로가 회수레코드로 prefill 을 늘리고 ($\rightarrow$ ch14 § 14.7) Θ -경로가 배포아티팩트를 바꾸는 것과 대비된다 ($\rightarrow$ ch19).

대신 **eviction** 경계마다 **chunked prefill** 의 버스트가 붙는다. 그버스트는 안에서 성질이 아주 다른 두 부분으로 갈라진다.

첫째, SSM 블록의 fast weight 다. 고정 크기이고 t 에 따라 자라지않으므로 [Do LMs Need Sleep? § 3.1], sleep 동안 이 상태는 C 번이아니라 $N \times C$ 번 read-modify-write 된다. 작고 상주하는 상태에 대한**대역폭 가벼운 · 계산 무거운** 버스트이며, 독자가 아는 KV cache 바운드 decode 레짐의 정반대 모양이다.

둘째, 같은 loop 안의 attention 블록이다. pass 마다 특징이 달라지므로 KV cache 를 매 pass 다시 쌓아야 하고, 그러면 chunk 에 대한 $O(C^2)$ attention 일이 N 번 반복된다 [본서 추론]. $C = 2000$ 에서 이것은작은 재계산이 아니다.

[해설] 논문이 준 값으로 이 책이 산술만 해 두면 이렇다. Jet-Nemotron 설정은 28 개 블록 중 가운데 14 개만 loop 하고 $N = 6$ 까지 돈다 [Do LMs Need Sleep? § 6.3]. 그러면 한 chunk 의 consolidation 비용은 그 chunk 를 평범하게 prefill 하는 비용의 $1 + (N - 1) \cdot \frac{14}{28} = 3.5$ 배다. 요청당 경계 수는 hard eviction GSM-Infinite 에서 1-2 개, sliding window 에서 4-6 개다 (§ 17.5). 이 곱셈을논문은 하지 않는다. 그리고 이 값은 이 책의 산술이므로 표 17-3 의 B_s 칸은 여전히 “논문에 없음” 으로 남는다.

서빙에서 배치가 깨지는 자리가 두 겹이다. 첫째는 낮익다 — 요청마다 다른 W 를 들면 shared-weight batching 이 성립하지 않는다 ($\rightarrow$ ch01 의 Rosetta 표). 이 논문이 그 대가를 새로 만드지는 않는다. 둘째가새롭다. **eviction** 경계는 **요청별 토큰 수에서 발생하므로 배치안에서 정렬되지 않는다**. 한 요청이 자고 나머지가 decode 하는 배치에는커널 모양이 둘 살아 있다 — N -pass chunked prefill 과 batch-1 에 가까운 decode step 이다. 스케줄러는 자는 요청의 이웃을 세우든지, consolidation 을선점 가능한 별도 잡으로 빼든지 해야 한다. 논문은 경계 skew 를 다루지 않으며, batching 에 관한 유일한 언급은 GPU 활용률을 위해 설정마다 batch 크기를 조정했다는 것이다 [Do LMs Need Sleep? Fig. 6 캡션].

상태가 **요청보다 오래 살지 않는다**. W 는 sequence 마다 0 으로초기화되고 [Do LMs Need Sleep? Alg. 1] 요청이 끝나면 사라진다. 그래서 사용자당 · 세션당 지속 저장소가 **생기지않는**

다. checkpoint/restore 도, 세션 캐시 축출 정책도, 요청간 상각도 없다. 이것이 “요청 내부 consolidation” 이라는 말의 서빙 쪽의미다.

[평가] 이 한 줄이 W -경로와 E -경로의 구조적 차이를 가장 짧게 드러낸다. E -경로의 sleep 산출물은 요청보다 오래 살고, 그래서 식 (A)의 N_q 가 1보다 커질 수 있다. 이 논문의 sleep 산출물은 요청과 함께 죽는다. 같은 이름 아래 있지만 회계의 종류가 다르다 — 하나는 상각이고 하나는 요청 내부의 재배치다 (→ ch24, ch25).

독자가 원하는 비교 팔이 데이터에서 제거되어 있다. 서빙 관점에서 이 방법의 자연스러운 경쟁자는 wake에서 추론 깊이를 사는 표준 수단, 즉 chain-of-thought 토큰이다. “경계에서 N 번의 sleep pass”와 “decode에서 κ 개의 CoT 토큰” 중 어느 쪽이 같은 품질을 싸게 사는가가 배치·SLA 결정을 가르는 질문인데, 논문은 CoT를 데이터에서 빼는 것으로 그 팔을 없앴다 [Do LMs Need Sleep? § 6.3]. 두 번째 팔은 어느 과제에서도 실행되지 않는다.

memory-centric 논증은 여기서 멈춘다. 작고 상주하는 상태에 대한 $N \times C$ 회 read-modify-write는 그 논증의 근거가 실제로 있는 드문 패턴이다. 그러나 논문에 바이트가 없으므로 (§ 17.5) 이 장은 패턴만 기록하고 정량화는 이 책의 실험이 소유하는 자리로 넘긴다 (→ ch26, ch29).

17.8 한계와 bridge-out

17.8.1 논문 자신이 남긴 문제

논문의 § 7이 세 가지를 스스로 적는다.

1. **학습 비용과 불안정.** “this gain is not free: during training, we need to perform N deeper forward and backward passes, which can make training slow and unstable.” [Do LMs Need Sleep? § 7] 처방으로 implicit gradient, truncated BPTT, 안정화 기법을 이름으로 열거하지만 **하나도 시도하지 않는다**. 불안정이 얼마나 자주 발생하는지에 대한 증거도 없고, 모든 곡선이 단일 시드라 불안정은 그림에 나타나지 않는다 (§ 17.6.1).
2. **sequence 축 병렬성의 상실.** 학습이 문맥 창을 가로질러 재귀적이므로 sequence 축을 완전히 병렬화할 수 없다. 논문은 창이 충분히 크면 wall-clock을 해치지 않는다고 하지만, 근거는 H200 한장 $\cdot L = 12,000 \cdot C \in \{1K, 2K, 4K\}$ 뿐이다 [Do LMs Need Sleep? § 6.5]. 작은 C 나 다중 GPU에서의 거동은 열려있다.
3. **이득이 순차적이지 않은 과제에서도 남는가.** “Sleep makes training sequential across context and depth dimension, but this sequentiality is also why our method shows gains on the tasks we consider, whose solutions are themselves sequential.” [Do LMs Need Sleep? § 7] 저자들이 스스로 적는 범위 제한이다 — **세 과제는 이 방법이 통하는 성질을 가졌기 때문에 골라졌다**.

논문이 질문으로 세우지 않았으나 이 책이 미해결로 기록하는 것이 넷 더 있다.

N 을 어떻게 고르는가. N 은 run마다 고정된 하이퍼파라미터이고 값마다 별개로 학습된 모델이다 (표 17-2). 적응 깊이 연구를 관련 연구에서 인용하면서도 halting 기준도, 예제별 배분도 없다 [Do LMs Need Sleep? § 2]. § 17.6.4가 보인 대로 이득은 어려운 꼬리에만 있는데 선택적으로 지출할 기제가 없으므로, 서빙에서는 모든 요청이 같은 N 배의 consolidation을 낸다.

어디서 포화하는가. 최대 시험값이 $N = 6$ 이고 그것도 블록 절반만도는 Jet에서다. Ouro의 N 은 학습 메모리 때문에 4에서 잘렸다 [Do LMs Need Sleep? § 6.3]. 포화점은 측정되지 않았다.

어느 국소 규칙이 일을 하는가. 논문이 명시적으로 범위 밖으로 선언한다 — “the specific update rule does not matter for our discussion” [Do LMs Need Sleep? § 3.1]. 초록이 “a learned local rule”을 기여성분으로 내거는데, GDN·Mamba-2·Jet·delta 보정을 가르는 ablation은 없다 (§ 17.3.2).

인용한 대안과의 경쟁·합성이 전부 미시험이다. context distillation, Cartridges, in-context autoencoding, 실제 gradient step을 밟는 test-time training, chunk당 LoRA adapter, 그리고 Θ -경로의 sleep — 여섯 묶음이 § 2에서 서술되고 **어느 것도 baseline으로 돌지 않으며 어느**

것과도 결합되지 않는다. 논문의 전체 비교집합은 같은 아키텍처의 $N = 1$ 판본이다. 그래서 § 2 가 말로 반박한 “chunk 당 gradient 한 번” 계열보다 이 방법이 낫다는 증거가 논문에 없다.

17.8.2 경로 간 인용여부

참고문헌 65 개를 전수 확인하면 이 논문의 인용 지도는 corpus 의 다른논문들과 모양이 다르다.

E -경로: 교전한다. Lin et al. = Letta 의 Sleep-time Compute(2504.13171, → ch14) 를 두 번 인용한다 — 오프라인 계산으로 예상 질문을 미리 푸는방법으로 한 번, “offline planning phase 를 sleep 이라 부른다” 로 한 번 [Do LMs Need Sleep? § 2]. Cartridges 는 경로 구분을 가장 또렷하게 쓰는자리에서 인용된다 — “These methods shorten what remains in the attention context, whereas our method transfers evicted context into weight-based memory.” [Do LMs Need Sleep? § 2] 그러나 MemGPT(2310.08560)·Mem0(2504.19413)·Zep(2501.13956)·ReasoningBank(2509.25140) 은 **참고문헌에 없다** (→ ch13, ch15). 프로덕션 외부기억 문헌이 통째로 빠져 있다.

Θ -경로: 교전한다. Behrouz et al. 의 Language models need sleep(→ ch21) 이 인용되어 있는데, 그항목에는 arXiv ID 도 venue 도 연도도 없이 제목만 있다 — OpenReview 투고본에서 인용했다는 뜻이다 [Do LMs Need Sleep? § 2, References]. Generative Adapter(→ ch16) 도 context distillation 목록 안에 들어 있다. 반면 SEAL(2506.10943)·Nested Learning(2512.24695)·SCM(2604.20943)·LoRA(2106.09685)·ROME(2202.05262)·MEMIT(2210.07229)·Memory Layers(2412.09764) 는 전부 없다 (→ ch18, ch19, ch20).

자기 경로: 침묵한다. fast weight programming 계보 (Schlag 등, Hebb) 와 현대 gated/delta 선형 attention 계열 (Dao·Gu, Samba, Griffin, Hymba) 은 두텁게 인용한다. 그런데 Titans(2501.00663)·Miras(2504.13173)·Atlas(2505.23735)·TNT(2511.07343)·Nested Learning(2512.24695) 은 65 개 항목 중 **하나도 없다** (→ NM ch12, NM ch14, NM ch15).

침묵은 두 종류로 갈라야 한다. 연대가 강제한 침묵은 계보 사실이고, 선택된침묵이 발견이다. 이 논문의 v3 스탬프는 2026 년 6 월 5 일이고 위에 열거한부재 항목은 **전부 그보다 앞선다**. 인용 가능한 상태로 전부존재했다. 연대가 실제로 개입한 자리는 반대 방향에 하나 있다 — 제목이겹치는 Behrouz et al. 은 당시 arXiv 에 오르기 전 OpenReview 투고본이었고, 그럼에도 인용되어 있다.

[평가] 그러므로 이 논문에 대해 “경로가 서로를 인용하지않는다” 고 쓰면 틀린다. 이 논문은 E -경로에 이름을 준 논문도, Θ -경로의 제목 쌍둥이도 인용하고 산문으로 자기와 구별한다. 정확한 발견은 더 좁고 날카롭다. 첫째, **교전의 깊이가 균일하게 한문장이고 실험적 접촉이 0 이다** — 모든 그림의 비교 집합이 자기 자신의 $N = 1$ 판본이다. 둘째, 더 큰 단절이 경로 사이가 아니라 **경로안에** 있다. 이 논문의 “learned local rule” 은 gradient step 이아님을 입장으로 내세우는데, 그것은 Titans 계열이 정확히반대편을 잡은 자리다. W -경로의 두 절반이 서로 말하지 않는다. ch11 의침묵 지도에서 이 항은 경로 사이가 아니라 경로 안에 기록된다.

17.8.3 다음 장이 받아가는것

이 장은 W -경로에 “질의 전에 실행되는 pass” 라는 자리를 만들었다. 그자리가 생기자마자 두 질문이 따라온다.

첫째, W 에 접는 것과 Θ 에 쓰는 것이 정말 다른 종류의행위인가. 이 장의 sleep 은 갱신 규칙을 바꾸지 않고 같은 규칙의 적용횟수만 늘렸다. 갱신 주파수를 축으로 놓으면 그 조작은 축 위의 이동으로읽힌다. ch18 이 그 축을 주장하는 논문과 그 축 위에서 상태를 키우는 논문을함께 받아 검증한다 — 연속체가 표기의 다리인지 기제의 다리인지가 거기서판정된다.

둘째, C_{cap} 의 빈칸. 이 장은 바이트를 쓸 수 없다고기록했고, 그 칸을 채우는 것은 이 책의 실험이며 판정은 ch26 이 소유한다. 계산량을 맞춘 비교가 없다는 § 17.6.1 의 공백은 B_s 를 축으로 세우는 ch28 로 가고, forward 쓰기가 gradient 쓰기에 무엇을 내주는가라는 § 17.3.2 의 미결은 ch16 에서 이어져 ch19 로 간다.

그리고 이 장이 남기는 가장 무거운 한 줄을 ch24·ch25 가받는다. 이 논문은 W -경로가 **sleep-time compute** 이 될 수 있음을보인 유일한 정면 사례이면서, 그 **sleep**의 산출물이 요청보다 오래 살지 않는사례이기도 하다. 판별식을 통과하는 것과 상각되는 것은 같은 일이 아니다.

요약

- Do LMs Need Sleep?(2605.26099) 은 상속받은 open question 없이 시작한다. 이 장의 bridge-in 은 상속 진술이 아니라**상속된 오진의 교정**이다 — attention-SSM 하이브리드의 실패원인을 용량에서 계산으로 옮기고, 선행 연구를 명시적 반박의 형태로인용한다 [§ 1, § 2].
- 기제는 (U-W) 하나이며 두 군데가 표준형과 다르다. gradient 자리에게이트된 outer product 가 들어가고 momentum 이 없으며, (U-W) 에 없는 pass 첨자 N 이 추가된다. 같은 chunk 를 W 에 N 번 누적해 쓰되 매 pass 의 (k_t, v_t) 를 직전 pass 의 W 가 만든 특징에서 다시 뽑는다 — (U-W) 와 특징 정제 고정점 반복의 합성이다. 표기 충돌은 corpus 에서 가장심하다: 원문의 S_t 가 이 책의 W_t, β_t 가 η_t 이고, 원문의 L 과 T 가 이름을 **맞바꾼다** (표 17-1).
- **논문이 보여 주는 식은 논문이 돌린 식이 아니다**. Eq. 3 은 Mamba-2 형태이고 실험은 GDN 이며 [Do LMs Need Sleep? § 6.3] 에는 Jet layer 가 또 다르게 등장한다. 국소 규칙 ablation 은 없고, 논문 자신이 “the specific update rule does not matter” 라고 범위 밖으로 선언한다 [Do LMs Need Sleep? § 3.1].
- 층은 W 하나다. Θ 는 outer loop 에서만 움직이고 E 는 존재하지 않으며, 학습 시점의 sleep/wake 분할은 런타임정책이 아니라 **loss mask** 가 정한다 [Alg. 1]. 판별식 통과는 **Depo 설정이 진다** — 거기서만 consolidation 이 질의보다 앞서고 $N_q = 10$ 이 구조로 정해진다. 헤드라인인 GSM-Infinite 는 질문을 문맥앞에 놓아 consolidation 이 질의 조건부가 되고, 그 대가로 $N_q = 1$ 에서식 (A) 가 $C_{\text{avg}} = C_{\text{wake}} + C_{\text{sleep}}$ 으로붕괴한다.
- 비용 4 종이 **전부 논문에 없다**. B_s 는 구조로만, L_w 는아키텍처 보장으로만 있고, C_{cap} 은 상태 차원 · head 수 · dtype 이 없어 유도조차불가능하며, ρ 는 관측될 레짐 (경계 1–6 개, sequence 마다 0 초기화) 에 논문이 들어가지 않는다. 헤드라인 지연 주장은 프로토콜 구성상참이지 측정된 값이 아니다.
- 실험은 세 설정 모두 합성이고 저자 제작 · 재구성이며, 시드하나에 오차막대가 없고, learning rate 는 $N = 1$ 에서만 튜닝되었고, **계산량을 맞춘 baseline 이 없다**. 캐시를 유지하도록 허용한 full-attention baseline 은 어느 과제 · 어느 스케일에서도실행되지 않았다.
- 수치가 나오는 곳은 GSM-Infinite 하나다. 이득은 6·8 연산꼬리에만 있고 (Ouro 6 연산 0.419 $\rightarrow$ 0.615), 쉬운 문제에서는 폭이 0.3–2.9 백분점이며, Ouro 8 연산에는 $N = 2$ 가 $N = 1$ 보다 낮은 반전이 있다. 가장 큰 상대 이득 52%는 같은 모델이다른 설정에서 loop 없이 이미 도달하는 수준의 회복이다.
- 서빙에서 decode 는 바뀌지 않고, 경계마다 chunked prefill 버스트가붙으며, 그 산출물은 요청과 함께 죽는다. 사용자당 지속 저장소가 생기지않으므로 요청 간 상각도 없다.

자가 점검체크리스트

- 이 장의 bridge-in 이 왜 상속 진술이 아니라 오진의 교정인지 말하고, 노트가 “none-stated”인 논문에 계보를 지어내지 않는 것이 왜 이 책의규율인지 설명할 수 있는가.
- 식 (17-3) 을 표준형 (U-W) 와 항별로 대조해 다른 곳 둘 (momentum 부재, gradient 자리의 outer product) 과 **추가된 것 하나** (pass 첨자 p) 를 지목하고, 그것이 왜 “(U-W) step 을 더 밟는 것” 과 다른지 말할수 있는가.
- 표 17-1 에서 L 과 T 가 맞바뀌는 행을 짚고, 본문의 “ $C = 2000, L \in [2000, 3300]$ ” 이 원문에서는 무엇으로 쓰여 있는지 말할 수 있는가.
- 판별식 네 조건을 Depo 설정과 GSM-Infinite 설정에 각각 대고, 어느조건이 어느 설정에서 왜 약해지는지, 그 약화가 식 (A) 에서 무엇을없애는지 말할 수 있는가.
- 표 17-3 의 네 칸이 각각 **어떤 종류로** 비어 있는지 구분할 수있는가 (수치 부재 / 측정 부재 / 유도 불가/ 레짐 미진입). 그리고 이 논문에 대해 “효율적” 이라고 쓸수 없는 이유를 한 문장으로

- 말할 수 있는가.
- 계산량을 맞춘 baseline 의 부재가 Fig. 2b·3·4·5 의 모든 비교에서 무엇을뜻하는지 말하고, 같은 x 좌표에서 $N = 4$ 실행이 이미 쓴 계산량을 어림할수 있는가.
 - 이 논문이 E -경로와 Θ -경로를 각각 몇 문장 인용하는지, 그리고 자기 경로의 어느 계보에 침묵하는지 말할 수 있는가. 그 침묵이연대가 강제한 것이 아니라 선택된 것임을 어떻게 판정하는가.
 - (Rosetta) 이 논문의 sleep 을 “warm-up / 사전 컴파일” 행으로 옮겨 설명하고, 그럼에도 “prefix cache” 행과 달리 산출물이요청을 넘지 못하는 이유를 W 의 초기화 규칙으로 말할 수 있는가.

ch18. Nested Learning과 Memory Caching — 연속체는다리인가

18.1 Bridge-in: 전작이 남긴문제

ch17 은 오프라인 재귀를 W 에 접었다. KV cache 를 비우기 전에 같은 문맥위로 (U-W) 를 N 회 더 돌리는 것이 곧 sleep 이라는 구성이었고, 그 결과 W -경로에는 “질의 전에 실행되는 pass” 라는 자리가 처음으로 생겼다. 그자리가 생기자마자 따라오는 질문이 하나 있다. W 에 접는 것과 Θ 에 쓰는 것은 정말 다른 종류의 행위인가, 아니면 같은 축 위와 다른 눈금인가.

Nested Learning(Nested Learning: The Illusion of Deep Learning Architecture, arXiv:2512.24695, 이하 [NL]) 은 후자를 주장한다. 갱신 주파수라는 축 하나를 세우고 그 위에 attention 부터 pre-training 까지를 순서지은 뒤, 학습과 추론의 구분 자체가 “지식 전달 과정을 최고 주파수 level 에서 최저 주파수 level(즉 pre-training) 로부터 끊어낸 결과” 라고 쓴다 [NL § 6]. 같은 절에 “Neural Learning Module 에는 training time 도 test time 도 없다” 는 박스가 붙는다 [NL § 6]. 이 주장이 성립하면 이 책의 3 층 프레임은 축 하나로 접힌다.

앞 문단의 질문은 이 책이 읽기 순서로 세운 것이지 [NL] 이 물려받은 것이 아니다. 사실을 그대로 적는다 — [NL] 은 어느 선행 논문의 open question 도 인용하지 않는다. 논문이 밝히는 출발점은 자기 관찰돌뿐이고 ($\rightarrow$ § 18.2), Miras 에서 associative memory 정의를 통째로 들여오고 [NL § 2] Titans 에서 deep memory 블록 · meta-learn 된 초기상태 · chunkwise 학습 절차를 받지만, 어느 쪽도 “이 논문이 답할 질문을 남긴 전작” 으로 지목되지 않는다. 연대도 반대 방향이다 — ch17 이 읽은 논문 (arXiv:2605.26099) 은 NL 보다 다섯 달 뒤에 나왔다. 따라서 이 장의 bridge-in 은 인용의 복원이 아니라 구조적 복원이다: [NL] 은 W -경로가 이미 세워 둔 설계공간 (Miras·Titans·Atlas) 위에 주파수 축을 하나더 얹는다.

이 장은 그 주장을 검증한다. 검증의 결론을 먼저 적는다. 연속체는 표기의 다리이지 기제의 다리가 아니다. [NL] 이 구현한 모든 갱신은 (U-W) 모양이고, Θ 는 그 축 위에서 test-time 에 아무것도 움직이지 않는 $f = 0$ 끝점으로만 등장한다. 축이 두 끝점을 이름으로 잇는 것과 상태가 두 끝점 사이를 실제로 이동하는 것은 다른사건이며, [NL] 에는 후자가 없다. 이 결론은 논문의 진술이 아니라 이책의 판단이다 — 근거는 § 18.4 의 표 18-3 이고, 판정문은 § 18.4 와 § 18.8.2 의 [평가] 블록에 놓인다.

같은 장에 Memory Caching(Memory Caching: RNNs with Growing Memory, arXiv:2602.24281, 이하 [MC]) 을 함께 놓는다. 두 논문은 같은 저자 그룹에서 나왔고, [MC] 는 [NL] 과 Miras 의 “memory 상태를 forward pass 중 최적화되는 동적 객체로 보는” 관점을 명시적으로 승계한다고 [MC § 2] 에서 밝힌다. 즉 [MC] 쪽에는 상속 진술이 있고, 받는 상대가 [NL]/Miras 라인이다 — sleep-time compute 세 경로 어느 논문의 open question 도 받지 않는다. 그러나 [MC] 가 실제로 한 일은 갱신식이 아니라 읽기식을 바꾼 것이고, $B_s = 0$ 이다. 세그먼트 경계에서 일어나는 사건은 갱신이 아니라 체크포인트 저장이다. 그래서 이 장은 [MC] 를 sleep-time compute 사례로 읽지 않고 W 층 용량 천장 논증으로 읽는다 — 고정 크기 W 하나로 recall 이 무너지면 W 를 몇 개나 들고있어야 하는가에 대한 정량적 답이다.

ch11 이 이 장에 넘긴 것은 두 편의 탈락 사유였다: [NL] 은 조건 (1) 과 (4) 에서, [MC] 는 조건 (2) 와 (3) 에서 걸린다 ($\rightarrow$ ch11 § 11.4.2, § 11.4.3). 이 장은 그 판정을 반복하지 않고 증거로 보인다 — 어느 객체가 어느 시계로 움직이는지의 표로.

18.2 문제의식

두 논문이 스스로 정의한 문제는 다르다. 순서대로 옮긴다.

[NL]의 출발점은 Transformer가 두 개의 극단 주파수만 쓴다는 관찰이다.

“Transformer architectures are based on two extreme frequencies of update: i.e., infinity and 0 for attention and MLP blocks, respectively” [NL § 1]

이 문장을 이 책의 기호로 옮기면 $f = \infty$ 는 attention의 KV 읽기이고 $f = 0$ 은 Θ 다. [NL]의 모든 기여는 그 사이에 놓인다. 두 번째출발점은 Transformer에서 사영 W_k, W_v, W_q 가 최저 주파수 level에서 최적화되므로 문맥 안에서 얼어 있고, 그래서 토큰이 문맥화되는 방식에상한이 걸린다는 관찰이다 [NL § 8, § 8.1]. 해법은 그 사영들 자체를 갱신되는 memory로 승격시키는 것이다.

여기에 신경과학 유비가 붙는다. [NL § 1]은 gamma(30–150 Hz), beta(13–30 Hz), delta/theta(0.5–8 Hz) 대역을 나열하고, 빠른온라인 consolidation과 느린 오프라인 consolidation의 2단계 그림을 세운뒤 곧바로 범위를 좁힌다.

“in this work, we focus on the first stage: memory consolidation as an online process” [NL § 1]

논문이 오프라인 단계를 이름 붙여 부르고 명시적으로 사양한다. 이문장이 이 장의 판정에서 가장 무거운 증거이며, 동시에 corpus 전체에서가장 쓸모 있는 bridge-out이다 (→ § 18.8).

[MC]의 출발점은 recall이다. recurrent 모델이 recall 집약 과제에서 뒤쳐지는 원인을 “자라는 시퀀스를 압축해야 하는 고정 용량이 과거 정보를 잊게 만들며, 이것이 결정적 병목”이라고 진단하고 Zoology 계열의 recall-throughput tradeoff를 인용한다 [MC § 1]. 해법은 놀랄 만큼 단순하다 — 과거 memory 상태를 버리지 않는다. 시퀀스를 N 개 세그먼트로 자르고 각 세그먼트 끝의 iterate를 보존한 뒤, 읽기를 그 캐시 집합 위의 게이트된 혼합으로 바꾼다. 명시적 상속이 하나 더 있다 — log-linear attention이 위치 편향과 검색 과정의 문맥 의존성 결여를 겪는다고 지목하고, 그 결함을 고친 판을 Log-Linear++라는 이름으로 자기 baseline에 세운다 [MC § 4.3].

[해설] 두 문제의식은 같은 자리를 반대편에서 누른다. [NL]은 “ W 의 갱신 주기가 하나뿐이라서 문제다”라고 하고, [MC]는 “ W 의 개수가 하나뿐이라서 문제다”라고 한다. 전자는 시간축을 쪼개고 후자는 상태축을 쪼갬다. 그리고 어느 쪽도 문맥경계를 넘지 않는다 — 쪼갬 것이 시간이든 개수든, 시퀀스가 끝나면 전부 사라진다. 이 공통점이 두 논문을 한 장에 묶는 이유이고, 동시에 둘 다 sleep-time compute이 아닌 이유다.

이 장이 소유하는 구분 하나를 여기서 못 박는다.

정의. 어떤 논문이 갱신식을 바꾼다는 것은 $(U-W) \cdot (U-\Theta) \cdot (U-E)$ 중 한 규칙의 우변을 바꾸는 것이다. 읽기식을 바꾼다는 것은 식 (R) 의 세 인자 $\Theta, W, \text{ret}(E, q)$ 중 하나를 읽는 방식을 바꾸는 것이며, 상태를 만드는 규칙은 그대로 둔다. 두 변경은 비용 회계에서 서로 다른 항으로 들어간다 — 갱신식 변경은 B_s 또는 쓰기 시점의 지연에 들어가 식 (A) 로 상각될 수 있고, 읽기식 변경은 질의마다 다시 지불되므로 상각되지 않는다.

18.3 Core mechanism (통일표기)

18.3.1 Nested Learning — 두 갱신규칙

[NL]의 축은 Definition 2가 정의한다. 한 데이터 포인트에 대한 갱신 한걸음을 단위 시간으로 삼아, 임의의 성분 A 에 대해 단위 시간당 갱신했수를 f_A 로 둔다 [NL Def. 2, § 3.2]. 성분들은 f 로 순서지어 level이 되고, 높은 level일수록 낮은 주파수다. 이 장에서 f 는 장-국소 기호이며 단위는 언제나 토큰이다 — level ℓ 의 chunk 크기를 $C^{(\ell)}$ 이라 하면 $f_\ell = 1/C^{(\ell)}$ 이다.

구현된 갱신은 둘이다. 첫째가 Continuum Memory System(CMS)이다.

$$W_i^{(\ell)} = W_{i-1}^{(\ell)} - \eta^{(\ell)} \sum_{t=i-C^{(\ell)}}^i \nabla_W \mathcal{L}(W_t^{(\ell)}; x_t) \quad \text{if } i \equiv 0 \pmod{C^{(\ell)}}, \quad \text{else } W_i^{(\ell)} = W_{i-1}^{(\ell)} \quad (18-1)$$

식 (18-1) 은 [NL Eq. 71] 의 통일 표기 환원이다. 표준형 (U-W) 와의 차이는 셋이다. retention gate 가 없고 ($\alpha_t \equiv 1$), chunk 안의 합 자체가 momentum 누적기 역할을 하며 경계에서 초기화되고, **objective** 가 inner loss ℓ 이 아니라 outer loss $\mathcal{L}$ 이다. 세 번째가 결정적이다. 논문은 이렇게 쓴다.

“the objective of $\mathcal{L}(\cdot)$ is the objective of choice for the task at hand, e.g., for language modeling it is next token prediction objective” [NL § 7.1]

즉 (18-1) 은 (U- Θ) 의 목적함수를 (U-W) 의 시계와 소멸성위에서 계산한다. 이 한 줄이 논문 전체의 모호성이다. 게다가 CMS 가 움직이는 대상은 보조 상태가 아니라 모델 자신의 MLP 블록이다 — vanilla Transformer 블록이 $k = 1$, 주파수 0 인 퇴화 사례로 나오고 [NL § 7.1], retrofit 경로에서는 문자 그대로 pre-trained Llama-3 의 MLP 텐서에서 시작한다 [NL § 7.3]. 겉모습만 보면 Θ -경로다.

둘째가 self-modifying Titans 다.

$$W_t^\square = W_{t-1}^\square (\alpha_t I - \eta_t k_t k_t^\top) + S_t^\square, \quad S_t^\square = -\eta_t \nabla_W \ell(W_{t-1}^\square; k_t, \hat{v}_{\square,t}) \quad (18-2)$$

$\square \in \{k, v, q, \eta, \alpha, \text{memory}\}$ 이고 각각이 2 층 residual MLP 다 [NL Eq. 88 – 89]. (U-W) 와의 차이 셋: retention gate 가 스칼라가 아니라 데이터 의존 행렬 $\alpha_t I - \eta_t k_t k_t^\top$ 이고, η_t 와 α_t 가 스케줄이 아니라 자기 자신도 W 층 **memory** 의 출력이며, 회귀목표 $\hat{v}_{\square,t} = \mathcal{M}_\square(v_t; W_{t-1}^\square)$ 가 갱신 대상 memory 자신에게서 생성되므로 외생 목표가 없다.

두 규칙 모두 병렬화를 위해 stale snapshot 으로 실행된다. chunk $[t/C]$ 의 $k_t, v_t, \hat{v}_{\square,t}, \eta_t, \alpha_t$ 와 모든 inner gradient 가 **직전 chunk 끝 상태**에서 생성된다 [NL Eq. 90, § 8.2]. Hope 는 (18-2) 의 작은 상태 · 표현력 높은 규칙과 (18-1) 의 큰 용량 · 단순한 규칙을 결합한 블록이고, Hope-Attention 은 CMS 의 기여만 분리하려고 softmax attention 에 CMS 를 붙인 변형이다 [NL § 7.1, § 8.3].

18.3.2 Memory Caching — 읽기식만바뀐다

[MC] 는 시퀀스를 N 개 세그먼트 $S^{(1)}, \dots, S^{(N)}$ 으로 자른다. 각 세그먼트 안에서 갱신은 baseline 에서 **한 글자도 바뀌지 않는다**.

$$W_t^{(s)} = \text{upd}(W_{t-1}^{(s)}; k_t, v_t), \quad 1 \leq t \leq L^{(s)} \quad (18-3)$$

위첨자 (s) 는 현재 세그먼트 인덱스이며, 본서 표기 규약이 SGD step 에 예약한 아래첨자 s 와 다른 대상이다. upd 는 baseline 아키텍처의 갱신 규칙 그대로다 ([MC Eq. 4] 는 이 자리에 f 를 쓰지만, 이 책에서 f 는 식 (R) 의 읽기 함수이고 이 장에서는 [NL] 의 갱신주파수이기도 하므로 기호를 옮긴다) — linear attention 이면 $W_{t-1} + v_t k_t^\top$, Titans 이면 (U-W) 의 $\alpha_t, \beta_t, \eta_t$ 세 항이 그대로 [MC Eq. 4, 34 – 35]. 세그먼트 끝에서 일어나는 일은 갱신이 아니라 보존이다: $W^{(i)} := W_{L^{(i)}}^{(i)}$.

바뀌는 것은 식 (R) 이다. 일반형은 캐시 집합 위의 집계다 [MC Eq. 5]. 논문이 제시하는 네 구체형 중 둘을 적는다.

$$\hat{y}_t = \gamma_t^{(s)} \mathcal{M}(q_t; W_t^{(s)}) + \sum_{i=1}^{s-1} \gamma_t^{(i)} \mathcal{M}(q_t; W^{(i)}), \quad \gamma_t^{(i)} = \text{softmax}_i \langle u_t, \text{MeanPool}(S^{(i)}) \rangle \quad (18-4)$$

$$\tilde{\gamma}_t^{(i)} = \langle u_t, \text{MeanPool}(S^{(i)}) \rangle, \quad \mathcal{I}_t = \arg \text{Top-}k(\{\tilde{\gamma}_t^{(i)}\}_{i=1}^{s-1}), \quad \hat{y}_t = \gamma_t^{(s)} \mathcal{M}(q_t; W_t^{(s)}) + \sum_{i \in \mathcal{I}_t} \gamma_t^{(i)} \mathcal{M}(q_t; W^{(i)}) \quad (18-5)$$

(18-4) 가 Gated Retrieval Memory(GRM) [MC Eq. 9 – 10], (18-5) 가 Sparse State Caching(SSC) [MC Eq. 16 – 17] 이다. 나머지 둘은 게이트를 1 로 둔 Residual [MC Eq. 7] 과 파라미터 공간에서 섞어 토큰마다 자기 회수용 $W_t^* = \sum_i \gamma_t^{(i)} W^{(i)}$ 를 짓는 Memory Soup [MC Eq. 14 – 15] 이다. $u_t = x_t W_u$ 는 q_t 와 별개의 사영이다.

두 극단이 이 설계의 좌표를 준다. $N = 1$ 이면 보통의 RNN 이고, $N = L$ 이면 모든 과거 토큰의 상태가 캐시되어 attention 과 거의 일치한다 [MC § 3.1]. 세그먼트 크기 1 에 value 없는 벡터 memory 를 두면 게이트된 global softmax attention 블록이 문자 그대로 복원된다 [MC Eq. 18 – 20]. 즉 이 논문은 RNN 과 Transformer 사이의 보간을 읽기 쪽에서 구성한것이다.

이 장이 소유하는 두 번째 정의가 여기서 나온다.

정의. W 층 용량 성장이란, 한 시퀀스가 진행되는 동안 그 시퀀스가 보유하는 fast-weight 상태의 총량이 늘어나는 것을 말한다. 자라는 것은 W 하나의 크기가 아니라 **보존된 (U-W) iterate 의 개수** N 이고, 유효 용량은 N 에 선형이며 $1 \leq N \leq L$ 이다 [MC § 3.1]. W 층 용량 천장은 그 N 의 상계를 정하는것이 아키텍처가 아니라 **세그먼트 분할 스케줄**이라는 사실을 말한다— 균일 분할이면 $N = L/C$ 로 문맥 길이에 선형이어서 상계가 없고, 로그분할이면 $N \leq \log_2 L$ 로 상계가 생기지만 먼 과거 토큰의 해상도가 현저히 떨어져 recall 집약 과제의 능력이 제한된다 [MC § 4.2].

로그 분할의 구성은 이진 표현을 그대로 쓴다. $L = 37 = (100101)_2$ 이면 세그먼트 길이가 32, 4, 1 로 잡히고, 그래서 총 비용이 $O(pL \log L)$ 로 내려간다 [MC § 4.2]. 균일 분할이면 $O(pL^2/C)$ 다. **상계를 얻는 대가가 해상도이며, 이 교환이 W 층 용량 논의의 핵심이다.**

한 가지 함정이 있다. W 가 행렬이고 게이트가 없으면 residual 변형은 사전 합산되어 고정 크기로 붕괴한다: $\hat{y}_t = (W_t^{(s)} + \sum_{i < s} W^{(i)}) q_t$ [MC Eq. 13]. 성장이 실패하려면 게이트가 문맥 의존이어야 하고, 그 결과 게이트는 토큰마다 다시 계산되어야 하며 사전 계산도 다음 토큰 재사용도 불가능하다 [MC § 3.1]. **용량 성장의 대가가 정확히 이 지점에서 발생한다.**

18.3.3 표기 대응표

표 18-1 — [NL] 원 표기 → 이 책의 표기

원 표기	이 책의 표기	주의
$\Theta_i^{(k)} / \theta^{(f_\ell)}$ — 임의 level 의 파라미터	pre-training level 위의 모든 level 은 W . Θ 는 level 1 에만	하드 충돌. 논문의 θ 를 그대로 Θ 로 옮기면 논문이 조용히 Θ -경로로 재분류된다. 본서 예약 기호 규칙상 금지
$\mathcal{L}(\cdot)$ — Eq. 71 에서 CMS 시계로 평가되는 task objective	$\mathcal{L}$ (outer/task loss)	기호도 뜻도 같다. 단 outer loss 를 inner 시계로 평가한다는 사실을 본문에 적어야 한다
$\tilde{L}$ — inner memory objective	ℓ	소문자 ℓ 이 inner loss 예약 기호다
$\mathcal{M}, \mathcal{M}_\square$ — 가변 객체	상태는 W , 읽기는 $\mathcal{M}(\cdot; W)$	논문은 $\mathcal{M}$ 으로 객체 자체를 가리킨다. Eq. 88 인용 시 분리 용량 C_{cap} 자리로 흘러들어가지 않게 한다
$C^{(\ell)}$ — level 별 chunk 크기 (토큰)	C (chunk)	비용 기호 아님. 단위는 항상 토큰, $f_\ell = 1/C^{(\ell)}$
f_A — 갱신 주파수	f (장-국소)	

원 표기	이 책의 표기	주의
$\hat{v}_{\square,t}$ — 자기 생성 회귀 목표	$\hat{v}_{\square,t}$ 유지. $\hat{c}$ 가 아니다	$\hat{c}$ 는 E -경로의 learned context 예약 기호다. 혼동하면 없는 E -경로 독법이 생긴다
$\eta^{(\ell)}, \eta_t$	전부 η_t 계열 (inner)	η_Θ 는 § 9.1 의 15B 토큰 continual pre-training에만 해당
S_t — 누적 momentum 기여 “level”	S_t “level” 그대로	참자를 유지해 $S(c; B_s)$ 와분리 이 책의 “층” 으로 번역하지 않는다. 세 층은 $\Theta/W/E$ 이고, [NL] 의 level 은 전부 W 안에 있다

표 18-2 — [MC] 원 표기 → 이 책의 표기

원 표기	이 책의 표기	주의
$M, M_t, M_t^{(s)}$ — 모듈 겹 상태	상태는 W , 캐시된체크포인트는 $W^{(i)}$	읽기 함수 $\mathcal{M}(q; W)$ 와 상태를 분리
$\theta_{M^{(i)}} = \{W_1^{(i)}, \dots, W_c^{(i)}\}$ — memory MLP 층가중치	$W^{(i)}$ 하나로 접는다	논문의 아래참자가 이 책의 층 기호와충돌하므로 본문에서 원 표기를 쓰지 않는다
L = 시퀀스 길이, L_M = memory MLP 깊이	L = 시퀀스 길이, 깊이는 L_M , 블록 수는 L_{layer}	충돌 없음. NM 승계 규칙상 L 은 sequence 길이다
$\mathcal{L}(M(k_t); v_t)$ — attentional bias	ℓ	논문의대문자를 그대로 옮기면 층이 뒤집힌다
$S^{(i)}$ — 세그먼트	$S^{(i)}$ 유지	같은 논문 안에서 위참자 $S^{(i)}$ =세그먼트, 아래참자 S_t =momentum 이 공존한다 [MC Eq. 35]
N — 세그먼트 수	N (장-국소: 캐시 개수)	예약 기호 N_q (문맥 공유 질의 수) 와 혼동 금지. § 18.1 이 ch17 에서 인용한 오프라인재귀 횟수 N 과도 다른 대상이다 우연히 일치. 용량은 반드시 C_{cap}
$C = L/N$ — 세그먼트 크기	C (chunk)	
$\gamma_t^{(i)}$ — 캐시 i 의 기여 게이트	$\gamma_t^{(i)}$ 유지. α_t 로 매핑 금지	α_t 는 시간축 retention 이고 γ 는 읽기축 mixture weight 다. 시간축 감쇠가 아니다
$\alpha_t, \beta_t, \eta_t$ (Titans 대입)	동일	(U-W) 와 글자 그대로대응
$u_t = x_t W_u$ — connector	u_t	q_t 와 별개 사영. $u_t = q_t$ 대안은 § 3.1 에 언급되나 그 행의 수치는 비어 있다 (→ § 18.6)

원 표기	이 책의 표기	주의
$r_t^{(i)}$ — 세그먼트 관련도 점수, R_t — 선택된 캐시 인덱스 집합 [MC Eq. 16 – 17]	$\tilde{\gamma}_t^{(i)} / \mathcal{I}_t$	예약 충돌: 본서 표기 규약상 r 은 rank, R 은보상이다. 논문 기호를 옮긴다. $\tilde{\gamma}$ 는 softmax 전 점수이고 $\gamma_t^{(i)}$ 가 정규화된 mixture weight 라는 관계가 기호에 그대로남는다. top- k 의 k 는 이 장에서 선택 개수를 뜻하는 장-국소기호이며, 예약된 sleep 라운드 첨자 k ·키 벡터 k_t 와 다르다
f — baseline 갱신 규칙 [MC Eq. 4, 17]	$\text{upd}(\cdot)$	이 책의 f 는 식 (R)의 읽기 함수이고 이 장에서는 [NL]의 갱신주파수다. 한 글자를 세 뜻으로 쓰지 않는다
위첨자 (s) — 현재 세그먼트	위첨자 (s) 유지 (장-국소)	예약된아래첨자 s (SGD step)와 다른 대상임을 § 18.3.2에서 선언한다

18.4 어느 층을 언제쓰는가

이 절이 이 장의 판정을 증거로 만든다. 방법은 하나다 — 두 논문에 등장하는 모든 움직임은 객체를 나열하고, 각각에 대해 (a) 누가 움직이는가, (b) 어느 시계인가, (c) 문맥이 끝난 뒤에도 남는가, (d) 배포 아티팩트가 바뀌는가를 채운다. (c)와 (d)가 판별식의 조건 (4)와 Θ -경로판별식에 각각 대응한다 (→ ch11 § 11.2).

표 18-3 — [NL]: 이 값은 누가, 어느 시계로 움직이는가

객체	움직이는 주체	시계	문맥 종료 후 존속	배포 아티팩트 변화	출처
임베딩, norm, window 4 short con- volution, 정 적 사영 W_q	outer loop (AdamW)	학습 라운드	— (test-time 불변)	아니오	[NL § 8.1, § 8.3]
여섯 self- modifying memory의 초기 상태 $W_0^\square$	outer loop (meta- learn)	학습 라운드	매 문맥 초기 값으로 복귀	아니오	[NL § 8.1]
여섯 memory의 running weights	inner, 식 (18-2)	chunk C (직 전 chunk 스 냅샷 기준)	소멸	아니오	[NL Eq. 88, 90]
η_t, α_t	학습되지 않는 다 — W^η, W^α 가 토큰마다 생성	토큰	소멸	아니오	[NL Eq. 86 – 88]

객체	움직이는 주체	시계	문맥 종료 후 존속	배포 아티팩트 변화	출처
회귀 목표 $\hat{v}_{\square,t}$	학습되지 않는 다 — 갱신 대 상 memory 자신이 생성	토큰 (실행은 chunk 스 냅샷)	소멸	아니오	[NL Eq. 87 – 88]
CMS level 파 라미터의 running state	inner, 식 (18-1)	$C^{(\ell)}$ 토큰. 실 측된 최저 주 파수는 2K 토큰	Nested 변형 은 $[C^{(s)}/C^{(s+1)}]$ step 뒤 초기 값으로 재초 기화	아니오	[NL Eq. 71 – 72, § 9.1]
CMS level 초 기값	outer loop (meta-learn; Nested 에서 는 level s 가 $s+1$ 의 초기 값을 학습)	학습 라운드	—	아니오	[NL Eq. 72 – 73]
retrofit 의 15B 토큰 continual pre-training M3 optimizer state	outer loop, 진짜 (U- Θ) optimizer 상태	평가 전 1 회, 저자측 학습 step	영구 서빙에 존재하 지 않음	예 아니오	[NL § 9.1] [NL § 7.2, Alg. 1]

표 18-3의 마지막에서 두 번째 행이 이 표의 전부다. **배포아티팩트가 바뀌는 행이 정확히 하나이고, 그 하나는 기제가 아니라 통제군이다.** 15B 토큰 continual pre-training은 Hope에도 ICL baseline에도 동일하게 적용된다 — “MLP 블록에 어떤 변경도 없이 같은 15B 토큰 continual pre-training 과정을 거친” baseline과 비교한다고 논문이 직접 쓴다 [NL § 9.1]. 실험 설계상 arm 사이에 고정된 값은 기여로 셀 수 없다.

나머지 모든 행의 “문맥 종료 후 존속” 칸이 소멸이다. 논문 자신이 자기범주를 그렇게 정의한다 — parametric in-context learning에 대해 “획득된 in-context 지식은 현재 문맥이 제거되면 존속하지 않는다”고 박스로 진술한다 [NL § 6]. 그리고 시계 칸에 세션도 sleep 라운드도 없다. 전부 토큰이고, 가장 느린 실측 눈금이 2K 토큰이다 [NL § 9.1].

정의. 갱신 주파수 연속체란, 서로 다른 갱신 주기를 갖는 성분들을 단위 시간당 갱신 횟수 f 라는 하나의 축 위에 순서지어 놓고 $f = \infty$ (attention) 와 $f = 0$ (pre-trained MLP) 을 그 축의 두 끝점으로 삼는 서술을 말한다 [NL Def. 2, § 1].

정의. 연속체 주장이 기제의 다리 이려면 두 조건이 필요하다. (i) 축 위의 어떤 갱신이 문맥 경계를 넘어 존속하고, (ii) 그 산출물이 배포 아티팩트에 되돌아 쓰인다. 둘 중 하나라도 없으면 그 연속체는 **표기의 다리** 다 — 축 하나가 두 끝점을 이름으로 이을 뿐, 어떤 상태도 끝점 사이를 실제로 이동하지 않는다.

[평가] 표 18-3에 (i) 을 만족하는 행이 없고 (ii) 를 만족하는 행은 통제군 하나뿐이다. 따라서 [NL]의 연속체는 표기의 다리다. 이 판정은 논문의 기여를 깎지 않는다 — [NL]은 W 층을 주파수로 구조화하는 어휘 (level, update frequency, persistent 대 adaptive 지식저장) 를 만들었고, Θ -경로는 뒤에 그 어휘를 빌려 쓴다 ($\rightarrow$ ch21). 판정이 말하는 것은 어휘가 건너간 자리에 기제는 건너가지 않았다는 것이다. 물리적으로 그 자리를 건너려면 **문맥이 끝난 뒤에도 살아남는 쓰기가** 있어야 하고, 그것을 하려면 오프라인 pass가 있어야 하며, 그 pass를 [NL § 1]에서 명시적으로 범위 밖에 두었다.

한 가지 유혹을 미리 차단한다. [NL § 7.3] 의 ad-hoc level stacking 은 $\eta^{(\ell)} \rightarrow 0$ 으로 두면 “갱신된 memory 블록이 초기 상태에 가깝게 유지되어, 적응 없이 pre-trained 블록을 그대로 쓰는 결과”가 된다고 쓴다. 스칼라 하나가 동결된 Θ 와 적응하는 W 사이를 연속적으로 보간하는 것이며, 진짜 다리라면 이렇게 생겼을 것이다. 그러나 보간의 한쪽 끝은 “ Θ 처럼 행동하는 W ” 이지 Θ 가 아니다. 체크포인트에 되돌아 쓰이는 경로가 식 어디에도 없다.

표 18-4 — [MC]: 이 값은 누가, 어느 시계로 움직이는가

객체	움직이는 주체	시계	문맥 종료 후 존속	출처
Θ — W_k, W_v, W_q , connector W_u , backbone	outer loop (AdamW, lr 4e-4, batch 0.5M tokens)	학습 라운드	불변	[MC App. B]
$W_t^{(s)}$ — 온라인 fast weight (2 층 MLP, expansion 4, GELU)	inner, 식 (18-3) = 표준 (U-W)	토큰	소멸	[MC Eq. 4]
$\{W^{(i)}\}_{i < s}$ — 캐시된 체크포인트	학습되지 않는다. 과거 inner iterate 를 보존한 것	세그먼트 경계에서 동결, 이후 읽기 전용	소멸	[MC § 3]
$\gamma_t^{(i)}, \tilde{\gamma}_t^{(i)}$ — 게이트 · router 점수	W_u 를 outer 가 학습, 값은 입력 의존으로 매 토큰 재계산	토큰	소멸	[MC § 3.1, § 3.3]
MeanPool($S^{(i)}$) — 세그먼트 요약	학습 없음 (평균 또는 합)	세그먼트 확정 시 1 회, 사전 계산 가능	소멸	[MC § 3.3]
N , 세그먼트 분할 스케줄	학습되지 않는 하이퍼파라미터	설계 시 고정	—	[MC § 5 Setup]

표 18-4 에는 “배포 아티팩트 변화” 열이 없다. Θ 가 한 번도 갱신되지 않기 때문이다. 그리고 세 번째 행이 이 논문의 정체를 드러낸다 — 캐시는 학습되지도, 갱신되지도 않는다. 이미 계산된 iterate 를 버리지 않았을 뿐이다. 세그먼트 경계에서 소비되는 추가 추론은 0 이다.

[해설] Rosetta 표의 어느 항목에 붙는지가 두 논문의 차이를 한 줄로 준다 ($\rightarrow$ ch01). [NL] 의 상태는 “batch 로 weight 공유” 가 깨지는 자리에 붙는다 — 가변 객체가 MLP 블록 자체이므로 요청마다 사본이 필요하다. [MC] 의 상태는 “paged KV cache / session cache” 에 붙는다 — 시퀀스 시작에 할당되고 시퀀스 끝에 해제되는, 페이지단위만 훨씬 큰 cache class 다. 어느 쪽도 “모델 재배포” 에 붙지 않는다. 그것이 두 논문이 Θ -경로가 아니라는 말의 서빙 쪽 번역이다.

18.5 비용 4 종

표 18-5 — 비용 4 종. 두 논문 모두 네 칸이 전부 비어 있다

기호	[NL]	[MC]
B_s	논문에 없음 — 정의되지 않는다. 질의 전에 실행되는 pass 가 없다. 가장 가까운 보고치는 wake 시점의 상각 갱신 비용 $O(\frac{1}{f} \times \frac{L_{\text{layer}}}{5} \times d_{\text{in}}^2)$ 파라미터 이며 FLOPs 도 토큰도 아니다 [NL § 7.1]	논문에 없음 — $B_s = 0$. 오프라인 성분은 § 4.3의 post-training 변형 하나뿐이고 그변형의 수치가 논문에 없다 [MC § 4.3]
L_w	논문에 없음 — Hope 에 대한 지연 · TTFT · ITL · 처리량 · wall-clock 측정이 전무하다. 논문의 유일한 효율 측정은 M3 optimizer 의 학습시간이다 [NL § 9.7]	논문에 없음 — 보고된 것은 차수 $O(p \times N)/\text{token}$ 과 training throughput 곡선 뿐이다. decode 지연은 측정되지 않았다 [MC § 3.1, § 5.7]
C_{cap}	논문에 없음 — 바이트도, 파라미터 수도, 평가된 Hope 모델의 $d_{\text{in}} \cdot L_{\text{layer}} \cdot \text{level}$ 개수도 적혀 있지 않다. 그래서 논문 자신의 $O(\cdot)$ 식조차 수치로 평가할 수 없다 [NL § 7.1, § 9.1]	논문에 없음 — 차수만 있다. 캐시개수 $1 \leq N \leq L$, 로그 분할이면 $N \leq \log_2 L$ [MC § 3.1, § 4.2]
ρ	논문에 없음 — 라운드당 열화가 보고되지 않는다. 유일하게 정량화된 망각 지평은 모델이 아니라 optimizer의 것이다 ($\beta = 0.9$ 에서 최근 6개 gradient가 누적 기여의 50% 이상, 43개가 99% 이상) [NL § 4.3]	논문에 없음 — 라운드 반복이 없으므로 개념 자체가 정의되지 않는다. 논문이 말하는 forgetting은 시퀀스 내 memory overflow 다 [MC § 1, § 3]

식 (A)를 두 논문 어디에도 적용할 수 없다. [NL]은 $C_{\text{sleep}} = 0$ 이고, 모든 문맥이 meta-learn된 초기 상태에서 다시시작하므로 질의 사이에 공유되는 아티팩트가 없어 N_q 가 정의되지 않는다. [MC]도 마찬가지로 $C_{\text{sleep}} = 0$ 이고, 캐시는 하나의시퀀스 안에서만 살아 질의 간에 재사용되지 않는다. 두 논문 모두손익분기 N_q^* 가 존재하지 않는다.

[평가] 여기서 § 18.2가 도입한 구분이 회계로 값을 한다. [MC]가 바꾼 것은 읽기식이므로, 그 비용은 상각 대상이 아니라 **토큰마다 반복되는 고정 부담**이다. 식 (A)의 분자에 들어갈 수 있는항은 세그먼트 요약 $\text{MeanPool}(S^{(i)})$ 하나인데 그것은 평균한 번이라 사실상 0이고, 실제로 무거운 항 — 캐시 N 개에 대한 per-token forward — 은 분모 N_q 가 커져도 줄지 않는다. 상각 논증이 이 논문에달지 않는 이유는 비용이 커서가 아니라 **비용이 놓인 자리가 다르기때문**이다. 읽기식을 바꾼 논문을 sleep-time compute으로 분류하면 이구조적 사실이 통째로 감춰진다.

본서의 정직성 규칙을 그대로 적용한다. 네 칸이 전부 비어 있으므로 이 장은두 논문에 대해 “효율적이다”라는 서술을 쓰지 않는다. [NL § 9.1]이 최저주파수 2K를 “significantly more efficient forward pass”라는 이유로권장하고, [MC § 5.7]이 SSC의 오버헤드가 최소라고 서술하지만, 두진술모두 단위가 붙은 값을 동반하지 않는다.

18.6 실험과스케일

18.6.1 [NL] 이 보고하는것

from-scratch 학습은 FineWeb-Edu + 장문맥 혼합, vocab 32K, AdamW 로 760M/30B tokens 와 1.3B/100B tokens 두 규모다. 760M 에서 Hope 의 Wiki perplexity 18.68, LAMBADA perplexity 20.07, 평균 정확도 52.28(Titans 51.68, RWKV-7 50.55). 1.3B 에서 Wiki 14.39, LAMBADA 10.08, 평균 58.04(Titans 56.82, Samba 54.46, Transformer++ 53.38) [NL Table 2, § 9.3]. RULER 계열은약 50B tokens 학습에서 Hope 가 attention-free 중 최고이고, S-NIAH-3 16K 에서 Hope 24.8 대 Titans 21.2 대 RWKV-7 5.8, MK-NIAH 16K 에서 Hope 14.8 대 Titans 8.2 다. Hope-Attention 은 S-NIAH-1 에서 4K/8K/16K 전부 100/100/100 으로 Transformer 의 88.6/76.4/79.8 을 넘는다 [NL Table 1, § 9.2]. 형식 언어 6 종 (parity, $(aa)^*$, $(abab)^*$, $a^n b^n$, $a^n b^n c^n$, Shuffle-2) 에서는 전부 100.0 으로 LSTM·SRWM 과 동률이다 [NL Table 5, § 9.5].

불리한 결과도 같은 표들 안에 있다. 짧은 in-context recall 에서는 Transformer 가 결정적으로 앞선다 — FDA 67.3 대 Hope 41.9 이고, Hope 는 attention-free 중 최고일 뿐이다 [NL Table 3, § 9.4]. 10M 토큰까지버틴다는 BABILong 결과는 **fine-tuned 모델의 결과**이며, 그림 이 fine-tuned 군과 zero-shot 대형 모델 군을 분리해 그리고 Hope 는 전자에있다 [NL Figure 9, § 9.2]. 그리고 논문이 스스로 가장 가까운 비교대상으로 지목한 Cartridges 는 비교에서 빠지는데, 그 이유가 수치 없는 비용시인이다 — “Hope 는 memory 사용량이 더 높고 계산 비용에 근본적인 차이가있다” [NL § 9.1]. **자원 열세를 인정하는 유일한 자리가 곧그것을 측정하지 않는 유일한 자리다.**

18.6.2 캡션이 자기 표에 반증되는 자리 — [NL] Table 6

표 18-6 — [NL Table 6, § 9.6] Hope ablation. perplexity 는 낮을수록 좋고 reasoning 정확도는 높을수록 좋다

구성	ppl	reasoning acc
Hope	12.24	58.1
w/o DGD	13.41	56.5
w/o momentum	13.58	56.9
w/o weight decay	13.71	57.2
w/o CMS	13.04	57.3
w/o inner-projection k	13.77	56.9
w/o inner-projection v	13.90	55.1
w/o inner-projection q	12.19	57.4

Table 6 의 캡션은 “Hope 의 모든 구성요소가 성능에 긍정적으로 기여한다” 고적는다. 마지막 행이 그것을 반증한다 — inner-projection q 를제거하면 perplexity 가 12.24 에서 12.19 로 내려간다 [NL Table 6, § 9.6]. level 을 하나 더한것이 언어모델링을 악화시킨 것이며, 이 사실은 § 18.8 이 인용할 “level 설계는 경험적이다” 라는 미해결 항목과 정확히 같은 자리를 가리킨다.

지속학습 헤드라인에는 다른 종류의 문제가 있다. class-incremental 설정 (CLINC 150 intent, Banking 77, DBpedia 70 개 2 차 클래스) 에서 Hope 가 ICL·EWC·InCA 를 셋 다 이긴다 [NL Figure 6, § 9.1]. 그런데 **restructuring 을 분리하는 통제군이 없다**. Hope 와 ICL baseline 이 똑같이 15B 토큰 continual pre-training 을 받지만, MLP 블록을 여러 level 로 재구성하는 처치는 Hope 쪽에만 들어간다. 재구성이함께 들어오는 추가 적응 파라미터와 추가 갱신 계산을등파라미터·등계산 arm 으로 분리한 실험이 없는 상태에서, 논문은 격차를 “Hope 의 다중 level in-context learning” 덕으로 귀속시킨다 [NL § 9.1].

주파수 축 자체의 실측은 한 자리뿐이다. LongHealth(환자 문서 20 편, 5.1K – 6.8K words, 문항 200 개), QASPER, MK-NIAH 에서 level 수와 최저주파수를 교차한 결과이며, level 이 많을수록 좋고 “최저 주파수가 높을수록성능이 낮다” 고 보고한다 [NL Figure 7, § 9.1]. 그런데 권장

동작점은 더나쁜 쪽이다 — 최저 주파수 2K가 512에 “가까운 성능”을 내면서 forward pass가 훨씬 효율적이라는 이유로 최적으로 제시된다 [NL § 9.1]. **주파수 연속체를 품질 축으로 팔고 비용으로 튜닝하며, 양쪽 다수치가 없다.**

효율 측정은 논문에 정확히 하나 있고 자기 산출물에 불리하다. M3 optimizer는 140M과 1.3B에서 Muon보다 느리고 AdaMuon과 비슷하다 [NL Figure 12, § 9.7]. Hope 자체의 wall-clock·지연·처리량·메모리측정은 없다. 한 가지 더 적는다 — “Adam은 최적 associative memory다”라는결과는 상정된 element-wise 목적함수와 gradient 분산을 목표로 택했을 때의존재 구성이며 유일성 결과가 아니다 [NL § 4.2, App. B].

스케일 상한은 이렇다. from-scratch는 1.3B/100B tokens가 최대이고 7B 이상 Hope는 없다. retrofit 경로는 Llama3-8B와 Llama-3B 위에 15B 토큰 continual pre-training이며, optimizer 연구는 ViT-86M/ImageNet-21K다 [NL § 9.1, § 9.3, § 9.7]. **그리고 실측된 가장 느린 CMS 눈금이 2K 토큰이다.** 연속체 자체의실증 상한이 2K 토큰이라는 뜻이고, 세션은 물론이고 sleep 라운드와는자릿수가 세 자리 이상 떨어져 있다.

[평가] 주파수 논증의 동기가 된 생물학은 뇌파 대역이다 —gamma 30 – 150 Hz에서 delta/theta 0.5 – 8 Hz까지 약 300 배범위이며 전부 지각 시간 규모 안에 있다 [NL § 1]. 논문은 이 유비를구현 범위가 1 토큰에서 2K 토큰인 계산 스펙트럼으로 옮긴 뒤, 같은 유비로축이 pre-training까지 닿는다고 말한다. 유비는 끝점의 증거가 아니다. 논문은 이 양보를 하지 않으므로 이것은 이 책의 판단이다.

18.6.3 [MC]가 보고하는것

설정은 760M(24 block, dim 1536, 16 head, peak lr 1.25e-3, 30B tokens)과 1.3B(18 block, dim 2048, 8 head, 7e-4, 100B tokens)이고, FineWeb + Long-Data-Collections, vocab 32K, AdamW, batch 0.5M tokens다. 학습 문맥은 {2K, 4K, 8K, 16K, 32K}, 세그먼트는 {16, 32, 64, 128, 256, 512}에서 고르며 Table 1의 기본은 문맥 4K·세그먼트 256이다. NIAH·in-context retrieval·LongBench는 16K 문맥으로 학습했다 [MC § 5, App. B].

1.3B에서 Titans(LMM) 평균 56.82가 +Log-Linear++ 57.19, +SSC 57.58, +Memory Soup 57.91, +GRM 58.33으로 오르고, DLA 53.72 → +GRM 55.96, SWLA 52.55 → +GRM 54.60이다. 같은 표의 baseline은 Transformer++ 53.19, Samba* 54.46, Miras(Memora) 55.76이다. 760M에서는 Titans 51.56 → +GRM 52.55, DLA 50.48 → +GRM 51.41이고 Transformer++ 49.64다 [MC Table 1]. NIAH 16K에서 S-NIAH-3(uuid)은 Titans 21.2 → +SSC 27.0 → +Soup 28.6 → +GRM 32.2, DLA 4.0 → +GRM 18.2이며 Transformer는 40.8이다 [MC Table 2]. in-context retrieval 평균은 Titans(LMM) 31.75 → +GRM 40.50, DLA 30.51 → +GRM 38.03이고 Transformer 41.00이다 [MC Table 3].

즉 격차는 좁혀졌고 닫히지 않았다. 가장 어려운 recall 설정에서 Transformer 40.8대 최고 MC 32.2이고, retrieval 평균에서도 41.00대 40.50이다. 초록의 “close the gap with Transformers”는 방향으로는 맞고 정도로는 표에 앞선다.

18.6.4 캡션이 자기 표에 반증되는 자리 — [MC] Table 5와그이웃들

표 18-7 — [MC Table 5, p.13] ablation. 세 열은 ppl / commonsense / retrieval

구성	Titans(GRM)	Titans(SSC)
전체	13.3 / 58.3 / 40.5	13.4 / 57.6 / 36.3
– context-dependent	13.4 / 57.4 / 33.0	13.4 / 57.1 / 32.6
– gating	13.5 / 56.9 / 32.4	13.5 / 56.8 / 31.9
– linear memory	13.7 / 56.3 / 34.5	13.8 / 56.8 / 33.4
– shared u and q	00.0 / 00.0 / 00.0	00.0 / 00.0 / 00.0

마지막 행이 양쪽 모두 000/000/000 으로 비어있다. [MC § 5.6] 본문은 앞의 세 설계 선택만 논하고 이 행을 언급하지 않는다. 그런데 Table 5 의 캡션은 “MC 의 모든 설계 선택이 그 효과에 긍정적으로 기여한다” 고 단정한다 [MC Table 5, § 5.6]. **빈 행위에 세운 단정이다.** 같은 표에서 SSC 의 context-dependent γ 제거는 perplexity 를 전혀 움직이지 않는다 (13.4 $\rightarrow$ 13.4) — 그 설계 요소의 이득은 retrieval(36.3 $\rightarrow$ 32.6) 에 국한되며 언어모델링 측에서는 “평균적으로 유의한 개선” 이라는 서술이 지지되지 않는다. 그리고 Table 5 의 Titans(GRM) ppl 13.3 은 Table 1 의 어떤 Titans+GRM perplexity 와도 맞지 않는다 (760M Wiki 19.14 / LMB 20.21, 1.3B 15.37 / 11.29). ablation 의 모델 크기 · 문맥 · 세그먼트가 명시되지 않아 Table 1 과 대조할 수 없다.

표 18-8 — [MC] 본문 진술과 자기 표의 대조

본문 진술	표가 말하는 것
“Titans + MC and DLA + MC achieves +0.8% performance gain over the Titans” [§ 5.1]	대응하는 쌍이 없다. 760M Titans 51.56 $\rightarrow$ +GRM 52.55(+0.99), 1.3B 56.82 $\rightarrow$ 58.33(+1.51), 1.3B DLA+GRM 55.96 은 Titans 56.82 보다 0.86 낮다 [Table 1]
“GRM and then SSC achieves better results among our provided methods” [§ 5.1(3)]	Avg 에서 Memory Soup 이 모든 비퇴화 블록에서 SSC 보다 위다 — 760M DLA 51.33 vs 50.85, 1.3B DLA 55.08 vs 54.64, 760M Titans 52.33 vs 52.29, 1.3B Titans 57.91 vs 57.58 [Table 1]
SSC 가 “performs on par or better compared to other variants” [§ 5.7]	Table 3 Avg 에서 SSC 는 GRM 보다 DLA 33.09 vs 38.03, Titans 36.27 vs 40.50 으로 뒤지고 S-NIAH-3@16K 도 27.0 vs 32.2 다. SSC 는 일관되게 열등하며 그 대가로 효율을 산다
“All MC-enhanced variants provide performance gains compared to their base RNNs” [§ 5.4]	Table 4 에 Avg 열이 없다. 14 개 과제 단순평균은 Titans 19.53 $\rightarrow$ +GRM 19.81 로 +0.28 이다 [본서 산술]. TRC 37.1 $\rightarrow$ 14.8, MNs 11.8 $\rightarrow$ 3.1, GvR 10.5 $\rightarrow$ 8.4 로 세 과제가 역전되고, 이득 전부가 TQA 26.2 $\rightarrow$ 49.7 하나에서 나온다

세 가지를 더 적는다. 첫째, 효율 근거가 training throughput 곡선 하나뿐이다 [MC Figure 4, § 5.7] — decode 지연, TTFT/ITL, 상태 바이트 중 아무것도 측정되지 않았고 비용 주장은 $O(pNL)$ 이라는 차수 서술에 머문다. 상수 p 의 실측이 없는데 p 가 이 설계의 전부다. 둘째, [MC § 3.4] 가 두 갈래 (체크포인트 연쇄 대 독립 압축기) 를 제시하고 [MC § 5.6] 을 보라고 넘기지만 [MC § 5.6] 과 Table 5 어디에도 그 측의 ablation 이 없다. **논문이 스스로 지정한 검증이 누락되었다.** 셋째, [MC § 4.3] 의 post-training MC 는 길이 외삽을 “크게” 개선한다고 주장하지만 이를 뒷받침하는 표도 그림도 없다 [MC § 4.3] — v2 기준으로 이 논문에서 유일하게 오프라인에 해당하는 변형이 정확히 미측정 상태다.

스케일과 통계의 한계도 정직하게 적는다. 1.3B/100B tokens, 학습 문맥 최대 32K(평가는 16K) 이고 instruction tuning 이나 RLHF 단계가 없다. MQAR(5 seed 평균) 을 제외한 Table 1 – 5 전부가 단일 시드이며 오차막대가 없어, Avg 차이의 상당수 — 예컨대 760M Titans+Soup 52.33 대 +SSC 52.29 의 0.04 — 는 시드 잡음과 구분되지 않는다. 760M Transformer++ 평균 49.64 는 RetNet(48.19) 을 제외한 모든 recurrent baseline 보다 낮고 S-NIAH-1@4K 에서도 88.6 으로 DLA 96.4 에 지므로, 이 규모에서 Transformer 대비 비교의 기준선이 약하다 [MC Table 1, Table 2]. 마지막으로 [MC § 4.1] 은 attention – RNN 하이브리드가 세그먼트 크기 1 의 MC 와 동치라고 논증하지만 MC 를 실제로 하이브리드에 얹은 실험이 없다 — Samba*와 Titans(MAL) 은 baseline 으로만 등장한다.

18.7 Systems/serving 합의

독자의 질문은 언제나 하나다 — 그래서 decode 에서 뭐가 바뀌는가. 두 논문의답이 다르다.

[NL] 에서 바뀌는 것은 **가변 객체의 정체**다. Hope 블록의 서빙상태는 여섯 개의 2 층 residual MLP memory 와 흘러가는 CMS tier 들이며, 전부가중치 모양이다 [NL Eq. 71, 89]. 여기서 Rosetta 표의 “batch 로 weight 공유” 행이 깨진다 — 동시 요청이 이 텐서들을 공유할 수 없으므로 in-flight 시퀀스마다 사본이 필요하다. W -경로 논문인데도 그렇게 되는 이유는가변 객체가 작은 행렬 상태가 아니라 MLP 블록이기 때문이다.

[해설] 그 대신 얻는 것이 있다. 상태가 문맥 길이에 대해 $O(1)$ 이므로 읽기 트래픽이 문맥에 평평하다 — KV cache 의 $O(L)$ 과대비된다. decode 지연은 문맥에 평평해지고, 대가는 블록마다 토큰마다추가되는 여섯 번의 작은 MLP forward 다. 그 교환의 손익 분기가 짧은 recall 에서의 열세로 나타난다 (FDA 67.3 대 41.9). 그리고 용량은 연속체를따라 **평평하다** — CMS tier 하나의 상태는 $C^{(\ell)}$ 과무관하게 MLP 블록 하나 전체이며, 주파수가 사는 것은 바이트가 아니라 쓰기대역폭이다. 논문 자신의 $O(\frac{1}{f} \frac{L_{layer}}{5} d_{in}^2)$ 과 라미터 식이 정확히 이 진술의 쓰기 쪽이고, 바이트로도 $d_{in} \cdot L_{layer}$ 의 실제 값으로도주어 지지 않는다 [NL § 7.1].

[MC] 에서 바뀌는 것은 **상태 성장의 모양**이다. 균일 세그먼트 C 면 캐시 개수가 $N = L/C$ 이므로 상태 바이트가 문맥 길이에 선형으로자란다 [MC § 3.1, § 4.2]. W -경로의 간판인 “고정 크기 상태” 라는장점이 여기서 사라진다. 이 논문의 진짜 주장은 고정 상태로 recall 을하려면 대가를 치러야 한다는 것이고, 그 대가가 정확히 KV cache 와 같은**선형 성장**이며, 다른 것은 성장의 상수뿐이다.

[해설] 상수를 잡아 본다. **아래 산술은 이 책의계산이며 논문의 보고값이 아니다.** 논문이 바이트를 적지 않으므로 bf16(2 bytes/elem) 을 가정하고 나머지는 논문 자신의 제원을쓴다 (1.3B: 18 block, dim 2048, 8 head; memory 는 2 층 MLP·expansion 4). memory 모듈이 head 별로 놓인다고 가정하면 $d_{head} = 2048/8 = 256$ 이고 head 당 $2 \times 256 \times 1024 = 524,288$ 파라미터, block 당 8 head = 4.19M, 18 block = 75.5M 파라미터/체크포인트, bf16 으로 **약 151 MB/체크포인트**다. 문맥 16K·세그먼트 256 이면 $N = 64$ 이므로 시퀀스당 약 9.7 GB 다. 같은 제원의 KV cache 는 토큰·layer 당 $2 \times 2048 \times 2 = 8$ KB, 18 layer 면 147.5 KB/token 이고 16K 문맥에서 약 2.4 GB 다. 즉 이 가정 아래 MC 는 KV cache 의 **약 4 배** 바이트를 쓰고, 손익분기 세그먼트 길이는 $C \approx 1024$ 이다. memory 가 head 별이 아니라 model-dim 전체에 놓이면체크포인트가 604M 파라미터로 8 배가 되어 배수는 더 커진다. **단정하는 것은 방향과 자릿수다 — 두 배치 가정 어느 쪽에서도 MC 의시퀀스당 상태는 같은 모델의 KV cache 보다 무겁다.** 논문은 이 비교를 하지않는다.

읽기 쪽 결과가 더 날카롭다. 회수는 토큰마다 모든 캐시에 대한 forward 이므로 $O(p \times N)$ 이다 [MC § 3.1]. 위 가정에서 GRM 과 Soup 은 decode 토큰마다 9.7 GB 를 읽고, FLOPs 로는 $2 \times 75.5M \times 64 \approx 9.7$ GFLOP/token 으로 1.3B dense forward(2.6 GFLOP/token) 의 약 3.7 배다 [본서 산술]. **decode 가완전히 대역폭 지배가 된다.**

SSC 가 서빙 가능한 유일한 변형인 이유가 여기서 나온다. top- k 만로드하므로 읽기가 $O(p \times k)$ 로 고정되고, 논문이 명시적으로 시스템논거를 단 유일한 지점이다 — 이런 계산은 “캐시된 memory 의 상태를 accelerator 에 상주시킬 필요가 없으며 토큰마다 ‘선택된’ memory 만 로드하면된다” [MC § 3.3]. MeanPool 이 사전 계산 가능하므로 router 는 캐시 상태를 읽지 않고 돌고, HBM 밖 (호스트 DRAM·NVMe) 에 캐시를 두고 선택된 것만 끌어오는 계층 구조가성립한다. **다만 k 의 값이 논문에 없다.** 그리고 § 18.6 이 보인대로 SSC 는 품질에서 일관되게 열등하다 — 서빙 가능한 변형과 성능이 좋은변형이 갈린다.

배치 쪽도 적는다. 캐시는 시퀀스별로 다르지만 Θ 는 공유되므로 shared-weight batching 이 깨지지 않는다. 대신 per-sequence 상태가 KV cache 보다 훨씬 크므로 동시 시퀀스 수가 상태 바이트로 제한되고, paged KV 의 페이지 단위가 토큰이 아니라 수십~수백 MB 짜리체크포인트가

되어 단편화와 스케줄링 난도가 오른다 [본서 추론]. SSC의 top- k 는 이 문제를 읽기 측에서만 완화하고 저장 측은 그대로 둔다.

18.8 한계와 bridge-out

18.8.1 논문 자신이 남긴문제

[NL]이 남긴 것 중 이 책에 가장 중요한 항목은 [NL § 1]의 범위진술이다. 오프라인 consolidation을 이름 붙여 소개하고 “본 연구에서는 첫단계, 즉 온라인 과정으로서의 memory consolidation에 집중한다”고 선언한다. **corpus 전체에서 가장 쓸모 있는 bridge-out이며, 이것이 뒤 장에서 상속으로 실현된다** (→ ch21). 그 밖에 catastrophic forgetting이 일반적으로 해결되지 않았고 “압축의 자연스러운 귀결”이며 level 사이 용량배분이 열려 있다는 진술 [NL § 10], level 설계가 경험적이라는 사실 (512 대 2K는 원리가 아니라 효율 판단이고, Table 6은 level 하나를 더한 것이 perplexity를 악화시킴을 보인다), Cartridges 비교의 유보, M3의 대형 확장시 계산 오버헤드 우려 [NL § 7.2], chunkwise stale snapshot이 진짜 self-reference를 얼마나 근사하는지에 대한 충실도 분석의 부재 [NL § 8.2]가 남는다. 자기 MLP 블록을 gradient로 갱신하는 모델의 서빙 경제 —테넌트별 분기, 표류한 상태의 체크포인트, 롤백—는 제기조차 되지 않는다.

[MC]가 명시적으로 남긴 future work는 하나다: 더 표현력 있는 pooling·routing 기제 [MC § 6]. 현재 설계는 MeanPooling + 내적 + top- k 라는 최소형이다. 나머지 미결은 § 18.6이 적은 세 공백 —체크포인트 연쇄 대 독립 압축기의 비교 부재, 세그먼트 분할 스케줄의 최적형태 (균일이 로그보다 나았다는 관찰만 있고 이유가 없다), post-training MC의 미측정 —과, q_t 를 이용해 모델이 자기 입력 시퀀스를 짓는다는 [MC § 4.1]의 관찰을 아키텍처로 만들지 않은 것이다.

18.8.2 이 책의 판정

[평가] [NL]의 연속체는 표기의 다리다 (§ 18.4). 그러나 이판정에는 정확한 부호가 붙는다 — **다리가 아니라는 사실이 Part II의 구조를 정당화한다**. 만약 갱신 주파수 하나로 Θ 와 W 가 이어졌다면 이 책의 세 경로 구분은 눈금의 구분에 불과했을 것이고, E -경로만 따로 놓으면 그만이었을 것이다. 그런데 표 18-3이 보인 대로, 축 위에서 아무리 주파수를 낮춰도 얻는 것은 “느리게 움직이는 W ”이지 Θ 가 아니다. 문맥 경계를 넘는 존속과 배포 아티팩트의 되쓰기는 주파수를 낮춰서 도달할 수 있는 상태가 아니라 **다른 종류의 연산**이다. 세 경로를 따로 세우는 이유가 여기 있다.

[평가] [MC]는 sleep-time compute이 아니다. 그러나 이장에서 가장 값비싼 수치를 공급한다 — W 층에서 recall을 사려면 상태가 문맥에 선형으로 자라야 하고, 상계를 걸려면 해상도를 내놓아야 하며, 그 성장의 상수는 같은 모델의 KV cache보다 크다 (§ 18.7). W -경로가 “고정크기 상태”라는 이름으로 파는 이점은 recall 요구가 높아지는 순간 사라진다. Part III의 용량 회계는 이 사실 위에서 세워진다.

18.8.3 경로 간 인용여부

두 논문 모두 다른 두 경로에 침묵한다. 종류가 다르다.

먼저 연대를 갈라 둔다. 침묵에는 **연대가 강제한 것과 선택된 것**이 있고, 둘을 섞으면 계보 사실이 발견으로 둔갑한다. [NL]은 2025년 12월, [MC]는 2026년 2월에 나왔다. 이 장이 아래에서 비어 있다고 지목하는 문헌 — MemGPT(2023-10), RAG(2020-05), LoRA(2021-06), ROME(2022-02), MEMIT(2022-10), Zep(2025-01), Letta STC·Mem0(2025-04), SEAL(2025-06), ReasoningBank(2025-09) —은 **전부 두 논문보다 앞선다**. 따라서 이들의 부재는 연대가 아니라 선택이다. 반대로 ch17이 읽은 Do LMs Need Sleep?(2026-05), ch21이 읽는 LM Need Sleep(2026-06), ch15의 SCM(2026-04)의 부재는 연대가 강제한 것이며 이 장은 그것을 두 논문의 흐름으로 세지 않는다. 아래 서술은 전부 앞쪽 — 선택된 침묵 — 에 대한 것이다.

[NL]의 E -경로 인용은 사실상 전무하다 — MemGPT도, 8개월 앞서 이라인의 이름을 만든 Letta STC도, Mem0도 Zep도 ReasoningBank도 없고 RAG의 방법 인용도 없다. “RAG”는

BABILong 에서 Llama-8B 의 인용 없는 baseline 변형으로만 등장한다 [NL § 9.2]. Θ -경로는 얇고 오래되었다 —EWC 한 건이 지속학습 baseline 으로 쓰일 뿐 LoRA 도 ROME 도 MEMIT 도 SEAL 도 모델 편집 문헌 전체가 없다. W -경로는 조밀하고 (Titans, Miras, Atlas, TTT, DeltaNet, RWKV-7 등) 생물학은 대량으로 인용되 온라인 단계에만 쓰인다. 모든 시간척도에서 갱신되는 모든 것을 통일하겠다는 논문이외부 기억 시스템을 한 편도, 가중치 경로를 2017 년 정규화 기법 하나로만 읽는다. 통일된 것은 W -경로와 optimizer 이며, 어휘만 셋을 덮을 만큼 넓다.

[MC] 의 침묵은 더 완전하다. 전문에서 “sleep” 이 0 회 등장하고, E -경로 인용 0 건, Θ -경로 인용 0 건이다. 인용되는 것은 전부 W -경로와 효율 아키텍처 계열이다 (Titans, Atlas, Nested Learning/Miras, log-linear attention, MoBA, model soups, MoE, fast weight programmers, Hopfield, linear attention, Zoology/Based). Θ -경로 문헌과 공유되는 조상은 model souping 과 MoE 둘뿐인데, [MC] 는 이들을 학습이 아니라 읽기측 혼합으로만 쓴다.

역방향 관찰을 하나 덧붙인다. $E \cdot \Theta$ 경로 논문들은 용량을 아키텍처 문제로 다루지 않고, W -경로만 용량을 복잡도 차수로 정량화한다. 비용을 차수로라도 쓰는 경로가 정확히, sleep-time compute 담론과 어휘 수준에서도 점점이 없는 경로다.

18.8.4 다음 장이 받아가는것

ch19 는 LoRA 와 모델 편집을 읽는다. 이 장이 넘기는 것은 정확히 이 장이 없다고 판정한 것 — Θ 에 실제로 쓰는 연산이다. 주파수를 낮춰서는 Θ 에 닿지 못하므로, Θ -경로는 낮은 주파수의 W 가 아니라 별도의 쓰기 primitive 에서 출발해야 한다. ch19 가 그 primitive 를 세우고 그 상한을 쟁다. 그리고 이 장이 [MC] 에서 얻은 용량천장 — 상태를 늘리면 recall 을 사지만 성장이 선형이라는 사실 — 은 ch19 에서 rank r 로 다시 나타난다. 두 경로가 같은 질문을 다른 자원으로 묻는다.

요약

- [NL] 이 구현한 모든 갱신은 (U-W) 모양이고, 시계가 전부 살아 있는 시퀀스의 토큰이며, 실측된 가장 느린 눈금이 2K 토큰이다 [NL § 9.1].
- 표 18-3 에서 배포 아티팩트를 바꾸는 행은 15B 토큰 continual pre-training 하나뿐이고, 그것은 ICL baseline 에도 동일하게 적용되므로 기제가 아니라 통제군이다 [NL § 9.1].
- 따라서 갱신 주파수 연속체는 표기의 다리이지 기제의 다리가 아니다. 주파수를 낮춰 얻는 것은 느리게 움직이는 W 이며, 문맥경계를 넘는 존속과 체크포인트로의 되쓰기는 주파수로 도달할 수 없는 다른 연산이다.
- [NL] Table 6 의 캡션은 모든 구성요소가 기여한다고 적지만, inner-projection q 를 제거하면 perplexity 가 12.24 에서 12.19 로 내려간다 [NL Table 6]. 지속학습 헤드라인에는 restructuring 을 분리하는 통제군이 없다 [NL § 9.1].
- [MC] 는 갱신식을 한 글자도 바꾸지 않는다. 바꾼 것은 읽기식이고 $B_s = 0$ 이며, 세그먼트 경계의 사건은 갱신이 아니라 체크포인트저장이다 [MC Eq. 4, 7].
- [MC] Table 5 의 한 행이 000/000/000 으로 비어 있는데 캡션은 모든 설계 선택이 긍정적으로 기여한다고 단정한다 [MC Table 5].
- W 층 용량의 상계를 정하는 것은 아키텍처가 아니라 세그먼트 분할스케줄이다 — 균일 분할이면 $N = L/C$ 로 상계가 없고, 로그 분할이면 $N \leq \log_2 L$ 이 되 면 과거의 해상도를 잃는다 [MC § 4.2].
- 두 논문 모두 비용 4 종의 네 칸이 전부 비어 있고, 식 (A) 의 손익분기 N_q^* 가 존재하지 않는다. 읽기식 변경은 질의마다 다시 지불되므로 원리적으로 상각되지 않는다.

자가 점검체크리스트

- 연속체 주장이 기제의 다리인지 표기의 다리인지 판정하는 두 조건을 말할 수 있고, [NL] 에 대해 그 판정을 표로 재구성할 수 있다.

- “outer loss 를 inner 시계로 평가한다” 가 왜 [NL] 의 모호성 전부인지, 식 (18-1) 을 $(U-W) \cdot (U-\Theta)$ 와 대조해 설명할 수 있다.
- 갱신식을 바꾼 논문과 읽기식을 바꾼 논문을 구분하고, 그 구분이 비용회계의 어느 항으로 나타나는지 말할 수 있다.
- W 층 용량 천장을 세그먼트 분할 스케줄의 함수로 진술하고, 균일분할과 로그 분할의 교환을 계산할 수 있다.
- 두 논문의 캡션이 자기 표에 반증되는 자리를 각각 수치로 지목할 수 있다.
- [MC] 의 체크포인트 하나를 바이트로 환산하고, 같은 모델의 KV cache 와비교해 결론이 배치 가정에 어떻게 의존하는지 말할 수 있다.
- [NL] 의 서빙 상태와 [MC] 의 서빙 상태를 각각 Rosetta 사전의 어느항목 (“batch 로 weight 공유” , “paged KV cache / session cache” , “모델 재배포”) 에 대응시키고, 어느 것도 마지막 항목이 아닌이유를 설명할 수 있다.

ch19. LoRA와 모델 편집 — 가중치에 쓰는 두방식

19.1 Bridge-in: 전작이 남긴문제

ch18은 갱신 주파수 연속체가 Θ 에 닿지 않음을 보였다. 축 위에서 주파수를 아무리 낮춰도 얻는 것은 느리게 움직이는 W 이고, 문맥 경계를 넘는 존속과 배포 아티팩트의 되쓰기는 눈금을 옮겨 도달할 수 있는 상태가 아니라 다른 종류의 연산이다 ($\rightarrow$ ch18 § 18.4). 그렇다면 질문은 하나만 남는다. Θ 에는 무엇으로 쓰는가.

이 장이 그 도구를 다룬다. 도구는 두 계열이다. 하나는 갱신을 저랭크곱으로 제약해 분리 가능한 **delta 아티팩트**를 만드는 계열이고 (LoRA, LoRA: Low-Rank Adaptation of Large Language Models, arXiv:2106.09685, 이하 [LoRA]), 다른 하나는 배포 가중치의 지정된 행렬을 **제자리에**서 **고쳐 쓰는** 계열이다 (ROME, Locating and Editing Factual Associations in GPT, arXiv:2202.05262, 이하 [ROME]; MEMIT, Mass-Editing Memory in a Transformer, arXiv:2210.07229, 이하 [MEMIT]). 두 계열의 대비가 이 장의 축이다.

bridge-in의 상태를 정직하게 적는다. **세 논문 중 어느 것도 sleep-time compute 계보의 open question을 물려받지 않았다**. 상속 진술이 있는 것은 [LoRA] 하나이고, 그 상대는 이 라인 이 아니라 intrinsic dimension 문헌이다 — “과매개화된 학습 모델이 실제로는 낮은 intrinsic dimension 위에 놓인다는 것을 보인 Li et al. (2018a), Aghajanyan et al. (2020)에서 영감을 얻는다. 우리는 **모델 적응 중 가중치의 변화량 역시** 낮은 intrinsic rank를 갖는다고 가설한다” [LoRA § 1, 동일취치 § 4.1]. [ROME]은 어느 선행 논문의 미해결 질문도 인용문으로 복원하지 않으며, 가장 가까운 문장은 저자 자신의 공백 진단이다 — “그것들이 지식을 저장하는 방식은 아직 충분히 탐구되지 않았다” [ROME § 1]. [MEMIT]도 상속 진술이 없다. 이 논문이 물려받은 것은 문장이 아니라 **규모의 수열**이다: 편집 1개 (단일 편집 패러다임) $\rightarrow$ 10개 (De Cao et al.의 개선 목적함수) $\rightarrow$ 75개 (Mitchell et al.) $\rightarrow$ 그리고 이 논문의 10,000개 [MEMIT § 1, § 2].

[평가] 이 비대칭 자체가 관측이다. 도구는 위로 흘렀고 질문은 아래로 흐르지 않았다. [ROME]이 Θ -경로의 primitive가 된 것은 후행 논문들이 이 도구를 가져다 쓴 결과이며, 이 논문 자신은 sleep·consolidation·replay·forgetting·external memory 중 어떤 어휘도 한 번도 쓰지 않는다. 그러므로 이 장의 bridge-in은 인용의 복원이 아니라 **재구성**이다. 재구성이라는 사실은 § 19.2에서 논문 자신의 문장으로 못 박는다.

19.2 문제의식

세 논문이 스스로 정의한 문제는 서로 다르고, 셋 중 어느 것도 기억 문제가 아니다.

[LoRA]의 문제는 배포 경제학이다. “fine-tuning의 주된 단점은 새 모델이 원 모델과 같은 수의 파라미터를 갖는다는 것이다” [LoRA § 1]. 175B 규모에서 이것은 불편이 아니라 배포 불가능성이 된다 — “사전학습 모델이 크면 (GPT-3의 경우 $|\Phi_0| \approx 1750$ 억) 독립적으로 fine-tune된 인스턴스를 여러 벌 저장하고 배포하는 것은, 가능하다 해도, 어렵다” [LoRA § 2]. 여기에 두 번째 제약이 붙는다. 기존 parameter-efficient 대안은 추론 지연을 늘리거나 쓸 수 있는 시퀀스 길이를 잡아먹고, 품질에서 fine-tuning을 따라가지 못한다 [LoRA § 1].

[ROME]의 문제는 해석가능성이다. “대규모 언어모델은 사실을 어디에저장하는가. 이 논문에서 우리는 GPT의 factual association이 직접 편집할수 있는 국소화된 계산에 대응한다는 증거를 보고한다” [ROME § 1]. 절의제목 자체가 “사실 연관 저장을 이해하기 위한 가중치 개입”이며, 편집은가설 검정의 도구로 도입된다. 그리고 논문은 자기 범위를 명시적으로제한한다.

“ROME의 목적은 지식 저장 기제를 이해하기 위한 도구로 쓰이는 것이다. 이것은 한 번에 사실 하나만 편집하며, **대규모 모델 학습을 위한실용적 방법으로 의도되지 않았다.**” [ROME § 3.7]

[평가] 이 문장을 먼저 실어야 한다. 이 책이 [ROME]을 Θ -경로의 쓰기 primitive로 세우는 것은 **이 책의재구성이며 저자의 주장 범위를 넘는다.** 이후 이 장이 [ROME]에서 끌어내는 모든 함의 — 상태량, 서빙, 상각 — 는 논문이 하지 않은 논의이고, 그 사실은 각 자리에서 다시 표기한다. 재구성이 부당하다는 뜻은 아니다. 후행 문헌이 실제로 이 도구를 그렇게 썼다. 다만 논문에 없는 주장을 논문에귀속시키지 않는다는 것이다.

[MEMIT]의 문제는 용량이다. 첫 문장이 곧 문제다 — “가중치를 직접편집해서 딥네트워크에 몇 개의 memory를 넣을 수 있는가” [MEMIT § 1]. 재학습이 비용상 금지적이므로 지식을 직접 갱신하는 방법을 찾되 [MEMIT § 1], 성공 기준은 “효과 있음” 하나가 아니라 generalization(재구문프롬프트에서도 새 사실을 회상)·specificity(편집하지 않은유사 주어가 오염되지 않음)·fluency(반복 붕괴 없음)를 함께만족하는 것이라고 [MEMIT § 3]이 정의한다. 그리고 선행 기법의 순차반복이라는 우회로가 실패한다는 것을 자기 근거로 세운다 — “현재 최고 성능지식편집 기법을 순진하게 순차 적용하면 규모가 나지 않는다” [MEMIT § 1].

이 장이 소유하는 첫 구분을 여기서 못 박는다.

정의. Θ 에 쓰는 방식은 산출물의 형태로 갈린다. **delta 방식**은 갱신을 base 가중치와 분리 저장 가능한객체 $\Delta\Theta$ 로 만들고, 배포 시점에 그것을 더할지 말지를 선택할수 있게 남긴다. **직접 편집 방식**은 지정된 가중치 행렬을제자리에서 고쳐 갱신을 base에 **흡수시키며**, 분리 저장 가능한객체를 남기지 않는다. 두 방식은 wake-time 연산 그래프에 대해서는 같은결과를 내고 ($L_w = 0$), **배포 아티팩트의 단위와 되돌리기 연산에대해서는 정반대다.**

[해설] 이 구분은 rank의 크기와 무관하다. [MEMIT]의갱신도 구조상 rank가 편집 개수 이하인 저랭크 갱신이지만 [본서 관찰], 그것을 $\Theta_{\text{out}}^{(l)}$ 에 흡수시키는 순간 분리 가능한아티팩트가 사라진다. 반대로 [LoRA]는 merge를 실행하면 같은 자리에도달하지만, (A, B) 를 그대로 보관하면 언제든 되돌릴 수 있다 ($\rightarrow$ ch03 § 03.5의 task switch). **갈림길은 저랭크냐 아니냐가 아니라아티팩트를 남기느냐다.**

19.3 Core mechanism (통일표기)

세 논문 모두 (U- Θ) 하나만 실행한다. (U-W)도 (U-E)도 없고, 식 (R)은 $\hat{y} = f(q; \Theta)$ 로 붕괴한다 — 편집 후에도 forward pass는원래 아키텍처 그대로이고 바뀐 것은 Θ 의 일부 값뿐이다.

19.3.1 delta 방식 — 저랭크 부분공간으로의사영

기제는 ch03이 소유한다. 여기서는 대비에 필요한 두 줄만 다시 놓는다. 갱신대상은 적용 행렬 $m \in \mathcal{S}$ 마다 붙는 저랭크 곱이고,

$$\Theta^{(m)} = \Theta_0^{(m)} + \frac{\alpha_{\text{LoRA}}}{r} B^{(m)} A^{(m)}, \quad B^{(m)} \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A^{(m)} \in \mathbb{R}^{r \times d_{\text{in}}} \quad (19-1)$$

학습은 (A, B) 에 대해서만 돈다 [LoRA § 4.1, Eq. 3]. $B^{(m)} = 0$ 초기화 때문에 쓰기는 반드시 항등원에서 출발한다 [LoRA § 4.1].

(U- Θ)와의 차이는 셋이다. (i) gradient를 Θ 전체가 아니라 (A, B) 에 대해 취하므로 $\Theta_k - \Theta_0$ 가 항상 $\text{rank} \leq r$ 인 다양체 위에 머문다. (ii) $\mathcal{R}_k$ 는 사람이 만든 고정 labeled dataset이며 $\text{gen}(\cdot; B_s)$ 가 항등이다 — B_s 라는 **knob 자체**가 없다. (iii) 첨자 k 가 sleep 라운드가 아니라 한 번의 adaptation run 안의 평범한 optimizer step s 다. 라운드가 반복되지 않으므로 누적 열화 문제가 제기되지 않는다 [LoRA App. D: GPT-3은 2 epoch, GPT-2는 5 epoch].

19.3.2 직접 편집 — 학습률이 없는 쓰기

[ROME]은 (U- Θ)의 극단적 특수형이다. gradient step이 아니라 **등식 제약 최소화**의 닫힌 해로 단 하나의 행렬에 rank-one delta를 더한다. 쓰기 문제의 정식화가 먼저다.

$$\min_{\hat{\Theta}_{\text{proj}}^{(l^*)}} \|\hat{\Theta}_{\text{proj}}^{(l^*)} K - V\| \quad \text{s.t.} \quad \hat{\Theta}_{\text{proj}}^{(l^*)} k_* = v_* \quad (19-2)$$

기존 key-value 대응은 최소화 문제로 유지하면서 새 쌍만 등식 제약으로 강제한다 [ROME Eq. 2, Eq. 7]. “층에 저장된 다른 기억과의 간섭을 최소화하면서”라는 표현이 이 제약형에서 나온다 [ROME Fig. 4 캡션]. 해는 닫혀 있다.

$$\Delta \Theta_k = \frac{(v_* - \Theta_{\text{proj}}^{(l^*)} k_*) (C_K^{-1} k_*)^\top}{(C_K^{-1} k_*)^\top k_*}, \quad \text{rank}(\Delta \Theta_k) = 1 \quad (19-3)$$

[ROME Eq. 2, 유도는 App. A Eq. 5–17]. $C_K = K K^\top$ 는 기존 키 분포의 uncentered second moment이며, Wikipedia 2020-05-01 스냅샷에서 뽑은 hidden state 100,000개로 한번 추정해 캐시하고 이후 모든 편집이 같은 것을 재사용한다 [ROME App. E.5].

k_* 는 학습되지 않는다 — 무작위 prefix를 붙인 텍스트들에 대해 subject 마지막 토큰의 활성을 평균한 값이다 [ROME Eq. 3]. v_* 만 최적화되는데, **최적화 변수가 가중치가 아니라 활성 벡터** 하나다.

$$v_* = \arg \min_z \ell_{\text{ROME}}(z), \quad \ell_{\text{ROME}}(z) = -\frac{1}{N} \sum_j \log P[o_* | x_j + p] + \lambda_{\text{KL}} D_{\text{KL}}(\cdot \| \cdot) \quad (19-4)$$

첫 항이 새 객체의 확률을 올리고, 둘째 항이 “{subject}는 …” 류프롬프트의 예측 분포를 원 모델에 묶어 subject의 정체가 표류하는 것을 막는다 [ROME Eq. 4]. 논문이 명시한다 — “이 최적화는 모델 가중치를 직접 바꾸지 않는다. 벡터 표현 v_* 를 찾을 뿐이다” [ROME § 3.1].

구조적 결론 하나를 여기서 못 박는다. (19-3)에는 η_Θ 가 존재하지 않는다. 반복도 없다. Θ -경로의 쓰기가 반드시 SGD여야 하는 것은 아니라는 존재 증명이며, 이것이 (U- Θ) 표준형과의 가장 큰 차이다. 부속 최적화 (19-4)의 learning rate 0.5는 활성 벡터 z 에 대한 것이므로 η_Θ 로 번역해서는 안 된다 [ROME App. E.5].

[MEMIT]은 이 구조에서 두 가지를 바꾼다. 첫째, 등식 제약을 목적 함수로 흡수한다.

$$\Theta_1^{\text{out}} = \arg \min_{\hat{\Theta}} \left(\underbrace{\sum_{i \leq n} \|\hat{\Theta} k_i - m_i\|^2}_{\text{옛 연관 보존}} + \underbrace{\sum_{i > n} \|\hat{\Theta} k_i - m_i\|^2}_{\text{새 연관 삽입}} \right) \quad (19-5)$$

“Meng et al. (2022)과 달리 우리는 새 연관 하나만 더하는 제약으로는 문제를 풀 수 없으므로 확장된 목적 함수를 정의한다” [MEMIT § 4.2, Eq. 9]. 해는 다시 닫혀 있고,

$$\Delta^{(l)} = \text{Res} K_1^\top (\Gamma_0 + K_1 K_1^\top)^{-1}, \quad \text{Res} = M_1 - \Theta_0^{\text{out}} K_1, \quad \Gamma_0 = \lambda \mathbb{E}_k [k k^\top] \quad (19-6)$$

[MEMIT Eq. 14, Eq. 15]. 접근할 수 없는 사전 학습 (key, value) 쌍이 유도 과정에서 소거되고 K_0 의 **2차 통계만 남는다**는 것이 이식의 관건이다. 즉 replay 데이터 없이 replay 효과를 통계로 근사한다 — Θ -경로에서 $\mathcal{R}_k$ 를 명시적 데이터 없이 대체한 드문 사례다. λ 는 새것 대 옛것의 가중

이며 전형값이 1.5×10^4 라고 논문이 쓴다 [MEMIT § 4.2]. 구조적으로 λ 는 (U-W) 의 α_t 가 하던 **retention gate** 역할을 한다 $-\lambda \rightarrow \infty$ 면 $\Delta^{(l)} \rightarrow 0$ 이고, 이것이 Θ -경로에서 유일하게 명시적인 retention 손잡이다.

둘째, 쓰기를 한 층이 아니라 임계 MLP 층 집합 Λ 에 분산한다. 근거는 residual stream 전개다 — 각 MLP 가 최종 은닉 상태에 **덧셈**으로 기여하므로 원하는 변화량을 여러 층에 나눠 실을 수 있다 [MEMIT Eq. 6, § 4.1]. 분배 규칙은 균등이다.

$$r_i^{(l)} = \frac{z_i - h_i^{l*}}{l^* - l + 1} \quad (19-7)$$

목표 잔차를 남은 층 수로 나눈다 [MEMIT Eq. 20]. 동기는 “파라미터변화량을 작게 유지하면 robust 해진다” 는 선행 관찰이며, [MEMIT § 4.3] 이풀겠다고 세운 최적화 문제를 실제로 풀지 않고 이 heuristic 으로 근사한다.

그리고 **순서가 본질이다**. 한 층을 고치면 아래층 활성이 전부바뀌므로, Λ 를 오름차순으로 돌며 층마다 활성을 다시 수집한다 [MEMIT § 4.3, Algorithm 1]. 이것이 (U- Θ) 1 회를 $|\Lambda|$ 회의 직렬 sub-pass 로 만들고, 배치화가 불가능한 유일한 축이된다.

z_i 는 활성 공간의 목표이며 이 논문에서 gradient descent 가 도는 유일한 곳이다 [MEMIT Eq. 16]. 즉 두 단계다 — **활성 공간에서 목표를 gradient 로 정하고, 가중치 공간으로는 대수로 사영한다**. 표준형에 없는 분리이며, 세 논문 중 직접 편집 두 편이 공유하는 구조다.

19.3.3 쓸 자리를 정하는계측기

두 편집 논문 모두 “어디에 쓸 것인가” 를 학습하지 않는다. causal mediation analysis 로 재고 사람이 읽어 고정한다. [ROME] 은 clean run / corrupted run / corrupted-with-restoration run 세 번을 돌려 각 은닉 상태의 indirect effect 를 재고 사실 1000 개에대해 평균한다 [ROME § 2.1]. GPT-2 XL 에서 average total effect 는 18.6%, 개별 은닉 상태의 최대 average indirect effect 는 15 번째층 ·subject 마지막 토큰에서 8.7%, MLP 기여 최대는 6.6%, 같은 토큰에서 attention 은 1.6%다 [ROME § 2.2]. 편집 층은 이 지도를보고 18 로 고정된다 [ROME App. E.5]. [MEMIT] 은 같은 계측을 GPT-J 에 다시 수행해 층 하나가 아니라 **범위**를 고른다 — “Meng et al. (2022) 이 이 검정으로 단일 편집 층을 식별한 것과 달리, 우리는 임계 MLP 층의 전 범위를 고른다” [MEMIT § 4.1]. 결과는 GPT-J $\Lambda = \{3, 4, 5, 6, 7, 8\}$, GPT-NeoX $\Lambda = \{6, 7, 8, 9, 10\}$ 이다 [MEMIT App. B.4]. **모델이 바뀌면 사람이 다시 정해야한다** [본서 추론].

19.3.4 표기 대응표

표 19-1. 원 논문 기호 $\rightarrow$ 이 책 기호. [LoRA] 의 대응표는 ch03 이소유하므로 여기서는 편집 두 편만 옮기고, 두 논문에서 같은 자리를차지하는 행은 논문 열로 합쳤다 (의미 변경 ·삭제 없음). 마지막 두 행은 이 장에서 새로 생긴 장-국소 행이다.

원 논문 기호	논문	원 의미	이 책 기호	심각도
$W, W_{\text{proj}}^{(l)}, W_{\text{fc}}^{(l)}$	ROME § 3.1	편집 대상 MLP 행렬	$\Theta_{\text{proj}}^{(l*)}, \Theta_{\text{fc}}^{(l*)}$	치명 — 놓치면 W -경로로오분류된다
$W_y, W_{\text{out}}^l, W_{\text{in}}^l, W_0, W_1, \hat{W}$	MEMIT Eq. 1 · 4·7·9	가중치 행렬 일반	$\Theta_y, \Theta_{\text{out}}^{(l)}, \Theta_{\text{in}}^{(l)}, \Theta_0^{\text{out}}, \Theta_1^{\text{out}}, \hat{\Theta}$	치명 — 전부 slow weights 의 부분집합
$C / C_0, C^l$	ROME Eq. 2 / MEMIT Eq. 15	옛 key 의 uncentered covariance	$C_K / \Gamma_0, \Gamma^{(l)}$	치명 — $C(\text{chunk}) \cdot C_{\text{cap}}(\text{용량})$ 과 충돌

원 논문 기호	논문	원 의미	이 책 기호	심각도
E	ROME App. D	이웃 subject 집합	기호 없이 산문 으로	치명 — 3 층 프레 임 기호를 덮는다
E	MEMIT Eq. 5 / Eq. 1	편집 요청 집합 / 마지막 토큰 인 덱스	$\mathcal{E}_{\text{edit}} / T_{\text{end}}$	치명 + 논문 내부 이중사용
S	ROME § 3.3	ES·PS·NS 의 조 화평균	“종합점수” 로 풀 어 씀	치명 — 맨 S 금지
S	MEMIT § 4 / § 5.2.2 / Algorithm 1	마지막 subject token / Editing Score / 편집 층 집합	$t_S / \text{Score} / \Lambda$	치명 + 논문 내부 삼중사용. Algorithm 1 헤 더와 본문 (§ 4.1 의 R) 이 서로 다 른 기호를 쓴다
R	MEMIT § 4.1 / § 4.2	임계 MLP 층 집 합 / 잔차 행렬	Λ / Res	높음 + 논문내부 이중사용. $\mathcal{R}_k(\text{sleep 학습집합})$ 와도 충돌
D	MEMIT Eq. 1	transformer 층 수	L_{layer}	중 — 이 책에서 L 은 sequence 길 이다
$L \triangleq \max(R)$	MEMIT Eq. 16	목표 층	l^*	치명 — 개명 없이 는 식 (19-7) 이 읽히지 않는다
H, D	ROME App. A	은닉 폭, MLP 확 장 폭	d , MLP 확장 폭	중요 — 논문이 두 값의 수치를 명시 하지 않아 C_{cap} 계산이 막힌다
r (in (s, r, o))	ROME § 2 / MEMIT § 3	relation	ϱ	높음 — r 은 rank 예약
r_i^l	MEMIT Eq. 20	층 l 에 실을 잔차 벡터	$r_i^{(l)}$ (첨자 필수)	중 + 논문 내부 삼 중사용
k_*, v_*, K, V	ROME § 3.1	linear associative memory 의 키 · 값	별표 유지 k_*, v_*	높음 — $k_t, v_t(\text{U-W})$ 와 시 간척도가 다르다
$k_i, k_i^l / m_i, m_i^l$	MEMIT Eq. 19· 20	subject key / 기 역된 값	$k_i^{(l)} / m_i^{(l)}$	중 — k (sleep 라 운드첨자) 와 구 분해 항상 첨자를 단다
$L(z)$	ROME Eq. 4	v_* 최적화 목적 함수	$\ell_{\text{ROME}}(z)$	중 — $\mathcal{L}$ 도 ℓ 도아 니다
λ	ROME App. E.5 / MEMIT Eq. 15	KL 항 계수 1×10^2 / covariance 가중	$\lambda_{\text{KL}} / \lambda$	낮음. 후자는 α_t 의 Θ -경로 대응물
η	ROME / MEMIT	부속 최적화의 학 습률 (0.5)	η_{Θ} 로 번역 금지	치명 — Θ 갱신에 는 학습률이 존재 하지 않는다

원 논문 기호	논문	원 의미	이 책 기호	심각도
u / n	MEMIT Eq. 9 / Eq. 7·§ 5.2.2	새 연관 개수 / 기존 연관수 · 편집 수	n_{edit} / 문맥별 명시	중 + 논문 내부 이중사용
P / σ	MEMIT Eq. 16 / Eq. 4·App. A	prefix 개수 / 비선형성 · 임베딩 표준편차	$N_{\text{pref}} / \sigma \cdot \sigma_{\text{emb}}$	중 + 논문 내부 충돌
Λ	ROME App. A	rank-one 갱신의 좌벡터	본문에서 쓰지 않고 (19-3)에 전 개했다	장-국소 — MEMIT의 Λ (임계 층집합)와 충돌한다
G, G'	ROME § 2·§ 3.2	편집 전 · 후 모델	$f(\cdot; \Theta_k), f(\cdot; \Theta_{k+1})$	장-국소 — 편집 이배포 아티팩트를 바꾼다는 사실을 이 대응이 그대로 드러낸다

19.4 어느 층을 언제쓰는가

세 논문 전부 (U- Θ) 단독이다. 표 19-2가 “이 값은 누가학습하는가”에 전부 답한다.

표 19-2. 세 논문의 객체별 (누가 · 언제 · 무슨규칙). W 와 E 행이 셋 다 비어 있다는 것이 이 표의 첫 결론이다.

객체	논문	누가 정하는가	언제	규칙
Θ_0 (backbone 전체)	셋 다	이 논문들 밖의 pre-training	범위 밖. 이후 동결	—
$A^{(m)}, B^{(m)}$	LoRA	outer-loop 학습 (AdamW)	adaptation 구간. test-time 불변	(U- Θ)를 rank- r 부분공간에 사영
merge된 $\Theta_0^{(m)} + \frac{\alpha_{\text{LoRA}}}{r} B^{(m)} A^{(m)}$	LoRA	학습되지 않음. 배포 시 1회 계산	deploy 시점	산술
$r, \alpha_{\text{LoRA}}, \mathcal{S}, C_K / \Gamma^{(l)}$	LoRA ROME / MEMIT	사람. heuristic 학습이 아니라 표본 추정 . Wikipedia 100,000 / Wikitext 100,000(GPT-J) · 50,000(GPT-NeoX)	adaptation 이전 모든 편집 이전, 모델당 1회	— 2차 통계
l^* / Λ	ROME / MEMIT	학습되지 않음. causal tracing 지도를 사람이 읽어 고정	모델당 1회	—

객체	논문	누가 정하는가	언제	규칙
$\lambda, \lambda_{\text{KL}}$	MEMIT / ROME	사람이 고르는 hyperparameter	편집 이전	retention 강도
$k_* / k_i^{(l)}$	ROME / MEMIT	학습아님. forward pass 로 읽어 prefix 평균	편집 시점, 사실 마다	(19-3)·(19-6) 의 입력
v_* / z_i	ROME / MEMIT	gradient descent — 단, 변수는 가중치가 아니라 활성 벡터	편집시점, 사실마 다. MEMIT 은 편집 간 독립이라 병렬 가능	$\ell_{\text{ROME}}(z) /$ [MEMIT Eq. 16]
$\Theta_{\text{proj}}^{(l*)} /$ $\Theta_{\text{out}}^{(l)}, l \in \Lambda$	ROME / MEMIT	아무도 학습하지 않는다. 닫힌 해 를 계산해 더한다	sleep 라운드 1 회. MEMIT 은 층 오름차순 순차	(19-3) / (19-6)
$\mathcal{R}_k$ (무엇을 쓸 것 인가)	셋 다	생성되지 않는다. 사람이 만든 dataset·데이터 셋 레코드에서 온다	편집·학습 입력	$\text{gen}(\cdot; B_s)$ 가 항등
W (fast weights)	셋 다	존재하지 않는다	—	—
E (external store)	셋 다	존재하지 않는다	—	—

시계와 순서를 산문으로 정리한다.

[LoRA]의 순서는 단일하다. 고정 dataset 을 받아 (A, B) 에 대해 optimizer step s 를 돌고, 배포 직전 merge 한다. 층 간 순서 문제가 없다 — 적용 행렬들이 서로 독립이다.

[ROME]의 순서는 넷이다. (a) 모델당 1 회 C_K 추정, (b) 모델당 1 회 causal tracing 으로 l^* 고정, (c) 편집마다 k_* 읽기, (d) 편집마다 v_* 최적화 후 (19-3) 으로 쓰기. (a) 와 (b) 가 고정비이고 (c)·(d) 가 한계비다.

[MEMIT]의 순서는 안쪽에 직렬 구간을 갖는다. $\Gamma^{(l)}$ 사전 추정 $\rightarrow$ 모든 i 에 대해 z_i 산출 (편집 간 완전독립, 논문 표현으로 “embarrassingly parallel” [MEMIT § 5.2.2]) $\rightarrow l = \min \Lambda$ 부터 $\max \Lambda$ 까지 오름차순으로 $K^{(l)} \cdot \text{Res}^{(l)}$ 을 재수집한 뒤 $\Delta^{(l)}$ 적용. 마지막 단계의 순서를 바꾸면 활성 재수집 전제가 깨진다.

두 가지를 명시한다. 첫째, 세 논문 모두 **sleep 라운드가 $k = 1$ 에서 멈춘다.** [ROME] 은 정의상 한 번에 사실 하나만 편집하고 [ROME § 3.7], [MEMIT] 은 배치 편집을 한 번 실행하고 끝나며 반복적용 실험이 없다. [LoRA] 는 delta 하나를 학습하고 두 delta 를누적하거나 합성하는 실험을 하지 않는다. 그러므로 ρ 를 **절실험이 이 장의 어느 논문에도 없다** ($\rightarrow$ § 19.5).

둘째, $\text{gen}(\cdot; B_s)$ 가 **셋 다 항등이다.** [LoRA] 의 $\mathcal{R}$ 은 GLUE·WikiSQL·E2E 같은 기성 dataset 이고, [ROME]·[MEMIT] 의 $\mathcal{R}$ 은 zsRE·CounterFact 레코드다. 무엇을 쓸지는 언제나 외부에서 주어진다.

[평가] 이것이 이 장의 위치를 정한다. **이 장은 Θ 에 쓰는 도구를 완성하지만 무엇을 쓸지는 아무것도 제공하지않는다.** 세 논문 중 어느 것도 예산 B_s 를 써서 학습 집합을 만드는 단계를 갖지 않으므로, ch11 의 판별식 조건 (2) 를 만족시킬 방법이 이 장안에는 없다 ($\rightarrow$ ch11 § 11.3). 그 빈칸을 채우는 것이 다음 장의 일이다.

19.5 비용 4 종

표 19-3. 세 논문의 비용 4 종. 논문이 침묵한 칸은 “논문에 없음”이며 추정치를 넣지 않았다.

	[LoRA]	[ROME]	[MEMIT]
B_s	논문에 없음 (FLOPs·토큰·wall-clock 전 부). 보고된 것은 학습 throughput 32.5 → 43.1 tokens/s per V100 과 VRAM 1.2TB → 350GB 뿐 [LoRA § 4.2, 각주 5]	GPT-2 XL 편집 1 회 약 2 초 (NVIDIA A6000) [ROME App. E.5]. FLOPs·토큰 없음, GPT-J 편집 시간 없음, C_K 추정 비용 없음	GPT-J 6B, 10,000 편 집 총 7.44 hr [MEMIT § 5.2.2]. 분 해: z_i 산출 23,546.65 초 + 나머지 갱신 3,226.35 초 [App. B.4]. GPT-NeoX 실행 시간 없음
L_w	merge 된 형태에서 0 — “구성상 fine-tune 된 모델 대비추가 지연을 전혀 도입하지 않는다” [LoRA § 4.1]	논문에 없음. 편집 후 추론 지연을 한 번도 재 지 않는다. 유일한 시간 수치 (2 초, 100ms) 는 편집 비용이다	논문에 없음. latency·throughput·TTFT 가 전문에 없다
C_{cap}	$ \Delta\Theta = 2N_{\text{mod}} dr$. GPT-3 175B· $r=4$ · $\mathcal{S} = \{\Theta^{(q)}, \Theta^{(v)}\}$ 에서 350GB → 35MB [LoRA § 4.2, § 5.1] (정본은 → ch03 § 03.6)	논문에 없음. 바이트 수치가 0 건이다. 갱신이 rank 1 이므로 상태량은 두백터 (좌백터는 은닉 폭 차원, 우백터는 MLP 확장 폭 차원) 인데 논문이 두 차원의 수치를 어디에서도 밝히지 않는다 [ROME App. A]	바이트는 논문에 없음. 논문이 보고하는 용량 단위는 편집개수이고 그 값이 10,000 이다 [MEMIT Table 1, Table 2]
ρ	논문에 없음. 순차 적응·delta 누적·반복 write 실험이 0 건	논문에 없음. 모든 표가 편집 1 회 후의 값이다	논문에 없음. 반복 라운드 실험이 없다. 유일한 반복 열화증거는 경쟁 기법의 것 — 순차 ROME 은 zsRE Specificity 0.9 (± 0.1), Score 2.6 으로 붕괴한다 [MEMIT Table 1]

식 (A) 를 이 장에 적용하면 세 논문 모두에서 손익분기 N_q^* 가 정의되지 않는다. 이유가 E-경로와 정반대다. 직접 편집은 C_{wake} 를 바꾸지 않으므로 — 갱신이 $\Theta_{\text{out}}^{(l)} += \Delta^{(l)}$ 라는 같은 shape 의 제자리 덧셈이다 [MEMIT Algorithm 1] — 상각은 “손익분기” 가 아니라 “한 번 지불하고 무한히 상각” 의 형태가 된다. 즉 $N_q \rightarrow \infty$ 극한이며, 분모에 들어갈 것은 질의 수가 아니다.

[해설] Θ -경로에서 상각의 실제 분모는 한번의 sleep 라운드에 묶여 넣은 편집 수다. [MEMIT] 의 진짜 기여를 이언어로 옮기면 그 분모를 1 에서 10,000 으로 키운 것이다. 편집당 시간은 26,773.00 초 ÷ 10,000 = 2.6773 초이고, z_i 단계가 총 시간의 87.95% 를 차지한다. 이 두 값은 이 책의 나눗셈이다 — 논문은 자기 방법에 대해서는 총시간과 두 단계의 초만 보고하고 편집당 값도 비율도 계산하지 않는다. 대조군에는 같은 나눗셈을 한다 — “ROME 은 GPT-J 10,000 편집에 44,248.26 초 ≈ 12.29 시간이 걸리며, 이는 편집당 약 4 초에 해당한다” [MEMIT App. B.3]. 그 4 초가 이 장에서 유일하게 논문이 인쇄한 편집당 비용이고, MEMIT 의 2.6773 초는 그보다 낮다.

[ROME]에서 실제로 상각되는 항은 하나뿐이다. C_K (hidden state 100,000 개 표본 추정)는 편집 횟수에 상각되고 편집당 한계비는 약 2 초다. 반면 hypernetwork 계열은 반대 구조다 — “KE와 MEND 같은 hypernetwork 는 추론 시점에는 훨씬 빠르지만 (100ms 수준) 수 시간에서 수 일의 추가 학습 오버헤드를 요구한다” [ROME App. E.5]. 즉 두 계열의 비교는 고정비 대 한계비의 문제이고, 손익분기 편집 수를 논문이 계산하지 않으며 “수 시간에서 수 일”이라는 범위가 너무 넓어 원문 수치만으로는 계산할 수도 없다.

비용 회계에서 빠진 항이 어느 논문에나 있다. [ROME]은 C_K 추정비용과 causal tracing 비용을 값으로 주지 않는다. [MEMIT]은 covariance 추정 (Wikitext 100,000 표본)과 causal tracing(진술 501 개 $\times$ 노이즈 표본 10 개)이 7.44 hr 에 포함되는지 명시하지 않고, “가장 계산이 비싼 단계”로 지목한 대형 정사각 행렬의 역행렬에 개별 시간을 붙이지 않는다 [MEMIT App. B.4]. **더 큰 빈칸은 쓰기 검증이다.** [ROME]은 사람 15 명이 반사실 시나리오 50 개를 평가했고 편집 후 모델이 근거 없는 새 사실을 지어낸다고 인정하지만 [ROME § 3.6, § 3.7], 그 검증비용은 어떤 비용 수치에도 들어 있지 않다.

[평가] “2 초”는 쓰기 연산의 비용이지 **올바르게 썼는지 확인하는 비용**이 아니다. Θ -경로에서 쓰기가 싸다는 인상은 검증을 회계 밖에 둔 결과이며, 이 장의 세 논문 중 어느 것도 그 항을 세지 않는다. Part III(ch25~ch27)가 이 항을 회계 안으로 들여온다.

같은 누락이 후속 문헌에서 반복된다는 것을 기록해 둔다. Understanding LoRA as Knowledge Memory(arXiv:2603.01097, 이하 [LoRA-KM])의 유일한 비용 비교는 문서 하나에 대한 질문 30 개의 벽시계 시간인데, 구성요소 목록이 모델 로딩 · adapter 로딩 · RAG 설정 · 질의임베딩 · 검색 · merge · 활성화 · 토큰화 · 추론으로 되어 있고 **학습 항이 없다** [LoRA-KM App. U]. 즉 그 비교는 식 (A)에서 $C_{\text{sleep}} = 0$, $N_q = 30$ 으로 놓은 것이다. 그리고 그 비교 안에서 RAG가 논문이 제안한 multi-LoRA preloaded 보다 빠르다 — FlashAttention 적용 시 43.5 초 대 65.9 초, 미적용 시 86.1 초 대 109.3 초인데 논문은 ICL 과만 비교한다 [LoRA-KM Fig. 22, § 7 Q15].

19.6 실험과 스케일

19.6.1 [ROME] — 사실 하나를 넣는 데 드는 것

실증 상한부터 적는다. **편집 실증의 상한은 GPT-J 6B 다** [ROME Table 4]. causal tracing 은 GPT-NeoX 20B 까지 올라가지만 그 모델의 편집 결과는 없다 [ROME App. B.3]. 편집 규모의 상한은 회당 사실 1 개이고, 지식 종류의 상한은 factual association 이며 “논리 · 공간 · 수치 지식 같은 다른 종류의 학습된 믿음은 조사하지 않았다”고 논문이 쓴다 [ROME § 3.7].

헤드라인은 저자 제작 벤치마크 위의 값이다. CounterFact 는 21,919 레코드로 저자가 만들었고 종합점수의 가중 (ES·PS·NS 의 조화평균)도 저자가 정했다 [ROME § 3.3, App. D]. 그 위에서 ROME 은 GPT-2 XL 종합점수 89.2, GPT-J 91.5 로 모든 기준선을 앞선다 [ROME Table 4]. **표준 벤치마크에서는 결론이 다르다.** zsRE paraphrase 에서 ROME 88.1 (± 0.5)은 MEND-zsRE 99.3 (± 0.1)과 KE-zsRE 90.0 (± 0.3)에 진다 [ROME Table 1]. 저자는 이들이 zsRE 분포에 맞춰 학습되었기 때문이라고 설명한다 [ROME § 3.2]. 스케일 하단에서도 역전이 있다 — GPT-2 Medium 의 zsRE efficacy 에서 ROME 96.6 (± 0.2)이 FT+L 97.2 (± 0.2)보다 낮다 [ROME Table 6].

사실 **하나**를 넣는 대가가 이미 측정되어 있다. GPT-2 XL 에서 편집 후 specificity 는 75.4 (0.7)로 편집 전 78.1 (0.6)보다 낮고, fluency 는 621.9 (0.5)로 편집 전 626.6 (0.3)보다 낮다 [ROME Table 4]. 그리고 사람 평가가 자동 지표를 뒤집는다 — 평가자는 ROME 을 FT+L 보다 삽입된 사실에 일관적이라고 볼 확률이 1.8 배 높지만, **더 유창하다고 볼 확률은 1.3 배 낮고**, 저자는 이 손실이 “우리의 다른 지표에 포착되지 않는다”고 인정한다 [ROME § 3.6].

쓰기 주소 규칙이 보편적이지 않다는 것도 논문 안에 있다. Windows Media Player 는 마지막 토큰이 아니라 ‘Windows’가 결정적이고, Mitsubishi Electric 은 ‘Mitsubishi’가, Madame de Montesson 은 ‘Madame’이 결정적이며, subject 안에 결정적 토큰이 아예 없는 사례와 마지막 subject 토큰의 효과가 음수인 사례가 있다 [ROME Fig. 11, Fig. 14]. **논문은 그 비율을 보고하지 않는다.** 자동화된 Θ 쓰기 파이프라인에서 이것은 주소 실패율이 된다 [본서 판단].

19.6.2 [MEMIT] — 편집 용량의 실측상한

이 장이 소유하는 두 번째 정의를 여기서 못 박는다.

정의. 편집 용량의 실측 상한은 한 번의 ($U-\Theta$) 라운드에 써넣었을 때 efficacy·generalization·specificity 세 지표가동시에 무너지지 않는 최대 편집 수다. 이 값은 아키텍처의 성질이 아니라**쓰기 알고리즘의 성질**이며, 붕괴가 관측되지 않은 sweep의 끝점은상한이 아니라 **관측된 하한**이다.

[MEMIT]의 sweep은 log-scale로 $n \in \{1, 2, 3, 6, 10, 18, 32, 56, 100, 178, 316, 562, 1000, 1778, 3162, 5623, 10000\}$ 이며 [MEMIT § 5.2.2 각주 1], 10,000에서 끝난다. 그 지점에서 MEMIT는 붕괴하지 않는다. 반면 기준선의 붕괴점은 명시된다 — “ROME은 $n = 10$ 까지는 잘 작동하지만 $n = 32$ 부터 열화한다”, “MEND는 $n = 1$ 에서 잘 작동하다가 $n = 6$ 에서 급락하고 $n = 1,000$ 전에 efficacy를 전부 잃는다” [MEMIT § 5.2.2]. 따라서 **10,000은 MEMIT의 한계가 아니라 이 corpus가 인용할 수 있는 관측된 하한**이다 [본서 추론].

10,000 편집 지점의 수치를 그대로 옮긴다 [MEMIT Table 2, CounterFact]. GPT-J 편집 없음: 종합점수 22.4 / ES 15.2 (0.7) / PS 17.7 (0.6) / NS 83.5 (0.5) / GE 622.4 (0.3) / RS 29.4 (0.2). MEMIT: 85.8 / 98.9 (0.2) / 88.6 (0.5) / **73.7 (0.5)** / 619.9 (0.3) / 40.1 (0.2). GPT-NeoX 20B에서는 82.0 / 97.2 (0.8) / 82.2 (1.6) / **70.8 (1.4)** / 606.4 (1.0) / 36.9 (0.6)이며 편집 없음 대비 NS가 81.6에서 70.8로 내려간다. 즉 **편집하지 않은 이웃 주어의 정답률이 10pt 가까이 떨어진다** [본서 산술]. 이것이 Θ -경로 대량 쓰기의 실측부작용이다.

그리고 다른 벤치마크에서는 그 값이 훨씬 나쁘다.

zsRE에서 MEMIT의 종합점수는 **50.7**이다 [MEMIT Table 1]. 분해하면 Efficacy 96.7 (± 0.3), Paraphrase 89.7 (± 0.5), **Specificity 26.6 (± 0.5)**이고, 조화평균이 이 낮은 항에 지배된다. 그런데 그 26.6은 **편집하지 않은 GPT-J의 27.0 (± 0.5)보다 낮다**. 논문은 이 격차를 편집 없는 모델의 zsRE specificity 자체가 낮다는 사실과 연결해 설명하지 않고 “bleedover가 최소”라고만 쓴다 [MEMIT § 5.2.1].

기준선과의 비교에도 유보가 붙는다. naive fine-tuning(FT-W)이 efficacy에서 MEMIT를 앞서고 (99.4 대 98.9), 시간도 1,716.21 초로 26,773.00 초보다 약 15.6 배빠르다 (배수는 이 책의 나눗셈이다) [MEMIT Table 2, App. B.1, App. B.4]. MEMIT의 우위는 efficacy가 아니라 specificity(73.7 대 46.9)와 fluency(619.9 대 293.9)에 있다. 작은 n 에서는 ROME이 generalization으로 MEMIT를 이기며, 저자는 이를 ROME의 hard equality 제약덕분이라고 설명한다 [MEMIT § 5.2.2] — **soft least-squares 완화는 공짜가 아니다**. 선행 최대 규모 주장의 출처인 SERAC과는 공개코드가 없어 비교하지 않았고 [MEMIT § 5.1], λ ablation은 역전 관계다 — “ λ 가 커지면 specificity와 fluency가 단조증가하지만 동시에 efficacy와 generalization이 떨어진다” [MEMIT App. F.3]. 모든 축에서 좋은 λ 는 없으며, 보고된 운용점은 조화평균이라는 저자의 집계 선택이 최대가 되는 지점이다.

마지막으로 평가 범위의 상한이다. 일반 능력 평가가 전혀 없다 — perplexity도 downstream benchmark도 0 건이고, 모델 손상의 유일한 대리지표가 bi/tri-gram 엔트로피다 [MEMIT App. C.2]. GPT-NeoX 20B는 데이터 점이 하나뿐이며 scaling curve·카테고리 실험·ablation·실행 시간이 전부 GPT-J에서만 수행됐다. 그리고 저자가 지식 범위를 명시한다 — “우리가 다루는 지식 표현은 방향성 (s, r, o) 관계로 범위가 제한된다. 공간·시간 추론, 수학 지식, 언어 지식, 절차 지식, 심지어 **대칭 관계도** 다루지 못한다” [MEMIT § 6]. 즉 “10,000개 memory”는 10,000개 삼중항이지 일반적 의미의 기억 10,000개가 아니다 [본서 추론].

19.6.3 delta 방식의 실측 — 후속 문헌이 보고한 것

[LoRA] 자신의 표에도 유보가 있다. GLUE에서 QQP는 두 RoBERTa 크기모두에서 full fine-tuning보다 낮고 (base 90.8 대 91.9, large 91.6 대 92.2), **Adapter가 LoRA를 이기는 행이 존재한다** — RoBERTa large의 QQP에서 Adapter_H(6.0M) 92.1이 LoRA_† (0.8M) 91.6보다 높고, SST-2에서 Adapter_P(0.8M) 96.6이 LoRA_† 96.2보다 높다 [LoRA Table 2]. (†는 Houlsby

et al. (2019)에 맞춘 제한 setup 이다 — 전 과제 공통 batch size, sequence 길이 128 고정, 그리고 MRPC·RTE·STS-B에서 MNLI checkpoint warm-start 제거 [LoRA § 5.1, App. D.1].)

[LoRA-KM]은 저랭크 delta를 지식 저장소로 쓸 때의 경계를 측정한다. 결과가 이 장의 논지에 불리한 쪽으로 선명하다.

첫째, RAG에 LoRA를 붙이면 RAG가 무너진다. QuALITY에서 Llama-3.1-8B는 RAG 63.79 → Single LoRA+RAG 42.42로 21.4점 떨어지고, Qwen3-8B는 64.53 → 44.10, Qwen3-1.7B는 43.43 → 33.15다 [LoRA-KM Table 2]. 세 큰 모델에서 방향이 같다. 남은 Llama-3.2-1B만 26.12 → 29.86으로반대인데, 그 29.86은 아래 둘째 항의 복사 정황에 걸린 값이다 [LoRA-KM Table 2]. 그리고 논문 본문은 이 패턴을 한 번도 언급하지 않는다. 오히려 구현 합의 상자에는 “LoRA는 외부 문맥과 결합할때 더 강한 성능을 내며 standalone LoRA·RAG·ICL을 능가한다”고 적혀 있다 [LoRA-KM § 7 Q13].

둘째, 그 42.42는 복사 오류의 정황을 띤다. 같은값이 Llama-3.1-8B QuALITY의 Single LoRA closed-book, Multi Top3 closed-book, Single LoRA+RAG open-book 세 칸과 [Table 6]의 QuALITY Embedding Top-3까지 네 번 등장한다. 나아가 QuALITY 열에서는 네모델 전부 “Single LoRA + RAG” 행이 같은 모델의 “Single LoRA” closed-book 행과 정확히 같다 (29.86 / 42.42 / 33.15 / 44.10) [LoRA-KM Table 2]. 문맥을 붙였는데 네 모델의소수점 둘째 자리까지 하나도 움직이지 않는 것은 측정이 아니라 복사의서명이다. 이 장은 이 사실을 매끄럽게 넘기지 않고 그대로 적는다.

셋째, 선행 연구의 기준선이 절반의 모델에서 저자의 single-LoRA를이긴다. closed-book NarrativeQA에서 KMSDCD(Caccia et al. 2025)는 Llama-3.1-8B 29.07대 27.05, Qwen3-8B 28.68대 25.78로 앞서고, 네 모델 전부에서 모든 multi-LoRA 구성을 이긴다. 논문은 이 행에 논평하지 않는다 [LoRA-KM Table 2].

넷째, 100K 토큰 이상 구간의 증거가 통계적으로 비어 있다. LongBench v2는 문서 30개로 평가되고 보고된 정확도가 전부 3.33의 배수다 (30.00, 20.00, 33.33, 16.67, 23.33, 36.67, 26.67). 무작위 추측 바닥이약 25%인 객관식 벤치에서 ICL이 두 모델 모두 20.00, RAG가 20.00/16.67로 바닥아래다. ∞ Bench는 문서 20개를 쓴다 [LoRA-KM App. Q, Table 7]. “LoRA 기반 방법이 100K+ 토큰 레짐에서 일률적으로 붕괴하지는않는다”는 결론은 이 표본 크기로는 지지되지 않는다.

19.6.4 Θ층 용량의 실측 공급원 —Memory Layers

Memory Layers at Scale(arXiv:2412.09764, 이하 [MemLayers])은 이 장의 도구가 아니다. 배포 후 어떤 층도 갱신되지 않으므로 sleep-time compute의 사례가 아니며 (→ ch11 § 11.4.4), 이 책이 인용하는 지위는 하나다 — Θ층의 용량이 FLOPs와 분리가능하다는 실측 공급원이다. 용량 논의의 정본은 ch09 가소유한다 (→ ch09 § 09.5). 이 장이 흡수해야 할 것은 그 실측에 붙는 유보다.

채택된 변형이 자기 ablation 표에서 최고가 아니다. [MemLayers Table 3]에서 +random values가 (2.06, 11.32, 7.20)로 세 지표 전부 최고인데 채택된+swilu는 (2.07, 11.79, 7.64)이고, +gated(2.08, 11.60, 7.54)조차 뒤의 두 지표에서+swilu를 앞선다. 저자는 학습 속도 비용과 “더 큰 모델크기에서는 이득이 일관되지 않았다”를 이유로 배제하지만 [MemLayers § 5.4], 그 대형 모델 수치를 논문에 실지 않는다. 배제 근거가보고되지 않은 실험에 있다.

vanilla Memory는 PEER에 진다. 1.3b에서 Memory의 NQ 9.83이 PEER 12.33에, TQA 39.47이 42.46에 뒤진다 [MemLayers Table 1]. [MemLayers § 5.1]의 “PEER 아키텍처는 Memory와 유사한 성능을 보인다”는 NQ에서 25%가 넘는 상대 격차 (9.83대 12.33, 상대화는 이 책의 나눗셈이다)를 완곡화한 서술이다. 즉 이 논문의 우위는 memory layer 자체가 아니라 Memory+의 추가 장치에서 온다.

MOE 기준선은 학습과 평가의 설정이 다르다. “MOE 모델은 expert choice로 학습하고 top-1 routing으로 평가한다” [MemLayers § 4.1]. 이불일치가 MOE 성능을 얼마나 떨어뜨리는지 논문이 측정하지 않으므로, MOE가 “큰 차이로” 뒤진다는 결론이 이 선택에 얼마나 의존하는지 알 수 없다.

8B에서 Memory+는 Llama3.1 8B에 9개 중 7개가 뒤진다. 앞서는 것은 HellaSwag(60.29대 60.05)와 PIQA(79.82대 79.16) 둘뿐이고, HotpotQA(26.06대 27.85)·HumanEval(31.

71 대 37.81)·MBPP(42.20 대 48.20)·MMLU(63.04 대 66.00)·NQ(27.06 대 29.45)·OBQA(34.40 대 34.60)·TQA(68.15 대 70.36) 에서 뒤진다 [MemLayers Table 2]. 비교 대상이 15 배의 토큰을 받았다는 점을 논문이 밝히지만, [MemLayers § 5.3] 의 “Llama3.1 8B 의 성능에 근접한다” 는 표현은 방향을 흐린다.

19.7 Systems/serving 합의

이 장이 소유하는 세 번째 정의를 여기서 못 박는다.

정의. Θ -경로 쓰기 도구의 서빙 합의는 세항으로 갈린다. (a) wake-time 연산 그래프가 바뀌는가, (b) 배포 아티팩트의 단위가 무엇인가, (c) shared-weight batching 이 살아남는가. 세항은 독립이 아니며, (a) 를 0 으로 만드는 방식이 (b) 와 (c) 를 정한다.

(a) 부터 답한다. 세 도구 모두 wake-time 연산 그래프를 바꾸지 않는다. [LoRA] 는 merge 후 GEMM shape 가 base 와 글자 그대로 같으므로 이것을 초록 수준의 셀링 포인트로 삼는다 — “adapter 와 달리 추가 추론 지연이 없다” [LoRA Abstract].

[해설] 직접 편집도 같은 자리에 도달하지만 논문이 그 사실을 주장조차 하지 않는다 [본서 추론]. 갱신이 $\Theta_{\text{out}}^{(l)} += \Delta^{(l)}$ 라는 같은 shape 의 제자리 덧셈이므로 [MEMIT Algorithm 1] 편집 후 모델의 파라미터 수 · 연산 그래프 · 토큰당 메모리 트래픽이 편집 전과 동일하다. 그러나 두 논문 어느 쪽도 추론 지연을 측정하지 않았고 [ROME] 의 유일한 시간 수치는 편집 비용이므로, 표 19-3 의 L_w 칸은 “논문에 없음” 으로 남는다. 구조가 뻗은 것과 논문이 켜진 것은 다르다.

(b) 와 (c) 에서 두 방식이 갈린다. Rosetta 사전으로 옮기면 세 도구 전부 “모델 재배포 = (U- Θ) 1 회” 행에 놓이지만, 재배포되는 단위가 다르다. delta 방식의 단위는 base 350GB 와 별도의 35MB 파일이다 — base 는 HBM 에 상주하고 갈아끼우는 것은 35MB 뿐이며, 되돌리기는 뺄셈 한 번이다 (→ ch03 § 03.5). 직접 편집의 단위는 체크포인트 전체다. 편집이 원 가중치에 흡수되므로 별도 저장 아티팩트가 없고, 그래서 “checkpoint 크기 = C_{cap} ” 이라는 Rosetta 행이 이 방식에서는 무의미해진다 — 편집을 100 개 하든 10,000 개 하든 체크포인트 크기가 같다 [본서 추론].

[평가] 이 “무료” 가 함정이다. 상태량이 0 으로 보이는 대가로 세 가지가 사라진다. 첫째, 되돌릴 수 없다 — [ROME] 과 [MEMIT] 어디에도 편집 취소 연산이 없고, 롤백은 이전 체크포인트로 교체하는 것뿐이다. 둘째, 편집 집합이 다른 두 배포본은 완전히 별개의 체크포인트가 된다. 셋째, 사용자별 개인화가 구조적으로 불가능하다. 직접 편집은 공유 가중치를 제자리에서 고치므로 애초에 분리가 가능한 아티팩트가 생기지 않고, 사용자마다 다른 사실을 넣으려면 사용자마다 모델 전체 사본이 필요하다. 이 책의 실험 [X2] 가 네 서빙구조를 비교해 이 갈림길을 정량화했으며, 그 판정은 ch26 과 ch29 가 내린다. 이 장은 갈림길이 어디에 있는지만 지목한다.

(c) 를 마저 적는다. 편집이 전역적이면 — 10,000 개 사실이 모든 사용자에게 같은 사실이면 — 가중치 1 벌을 계속 공유할 수 있으므로 shared-weight batching 이 살아남는다. 사용자별 편집을 시작하는 순간 per-user Θ 가 생기고 대전제가 깨진다. 논문은 이 구분을 제기조차 하지 않는다 [본서 판단]. 상각 논증도 같은 조건에 걸린다 — § 19.5 의 “한 번 지불하고 무한히 상각” 은 편집이 전역일 때만 성립한다.

운영 쪽 항 셋을 더 적는다. 어느 것도 논문에 없고 전부 이 책의 추론이다.

KV-prefix cache 무효화. [MEMIT] 은 GPT-J 의 층 3–8 MLP 를 고치므로 그 층 이후의 캐시된 KV 가 전부 stale 해진다. 논문은 KV cache 를 한 번도 언급하지 않는다 (검색어 ‘KV’ 0 건). prefix cache 를 대량으로 운용하는 서빙에서 Θ 갱신은 캐시 전면 무효화 이벤트이며, 이것이 Θ -경로 배포의 숨은 운영 비용이다.

sleep 라운드의 자원 격리. [MEMIT]은 GPT-NeoX 편집에 “float16 으로 모델을 돌릴 48GB GPU 하나와, 편집 방법을 실행할 조금 작은 GPU 하나”가 필요하다고 쓴다 [MEMIT § 9]. 그리고 covariance 통계는 fp32 로 수집 · 저장하는데 모델은 float16 으로 돈다 [MEMIT App. B.3, B.4]. **sleep 라운드는 서빙과 같은 커널 · 같은 dtype 으로 돌지 않으며, 편집기의 peak memory 발자국이 모델과 같은 급이다.** 유틸리티 서빙 GPU 에 sleep 을 엮는 그림은 이 수치와 맞지 않는다.

갱신 주기는 일 단위다. 10,000 사실에 7.44 시간 (GPT-J 6B, A6000) 이고, 논문의 뉴스 신선도 데모도 11 월 8 일 선거 결과를 11 월 14 일 이전에 확보하는 며칠 단위 서술이다 [MEMIT § 5.2.2, App. E]. 하루 1 회 배치가 현실적 상한이며 실시간 갱신은 이 방법의 범위 밖이다.

마지막으로 E -경로와의 비용 위치를 대조한다. 편집 이후의 질의는 검색대역폭을 0 지불한다 — 지식이 이미 가중치 안에 있다. E -경로는 질의마다 $\text{ret}(E, q)$ 의 대역폭과 지연을 지불한다. 반대로 [MEMIT]은 지식 하나를 바꾸는 데 편집당 2.6773 초의 오프라인 비용을 지불하고, E -경로는 레코드 하나를 쓰는 데 거의 0 을 지불한다. **쓰기가 비싸고 읽기가 공짜인 층 대 쓰기가 싸고 읽기가 유료인 층** — 이 대비가 Part III 의 3 경로 비용 분해의 뼈대다.

19.8 한계와 bridge-out

19.8.1 논문 자신이 남긴 문제

[ROME]이 남긴 것 중 이 책에 가장 무거운 항목은 쓰기 검증이다 — “모델에 저장된 사실 연관이 성공적으로 바뀐 뒤에도, 모델은 증거에 근거가 없고 거짓일 가능성이 높은 그럴듯한 새 사실을 추측한다. 이것은 언어모델을 사실의 출처로 쓰는 유용성을 제한할 수 있다” [ROME § 3.7]. 두 번째는 C_{cap} 회계에 직접 들어온다 — “ROME 이 편집하는 연관은 방향성을 가진다. 예컨대 ‘시애틀의 상징적 랜드마크는 스페이스 니들이다’ 는 ‘스페이스 니들은 시애틀의 상징적 랜드마크다’ 와 따로 저장되므로 둘 다 바꾸려면 두 번 편집해야 한다” [ROME § 3.7]. **per-fact 상태량이 사실 수가 아니라 연관 방향 수로 스케일한다 [본서 추론].** 나머지는 다중 동시 편집 (후속 논문으로 넘김), 사실 이외의 학습된 믿음, 학습된 속성을 표현하는 벡터 공간의 구조다. 그리고 “중간층 MLP 어디에나 같은 사실을 동등하게 저장할 수 있다고 추측한다” 는 문장이 실험이 아니라 conjecture 로 남는다 [ROME § 2.3] — Θ -경로 쓰기 자유도의 전제가 미검증 상태로 후속 논의에 상속된다.

[MEMIT]이 물음표로 남긴 문장은 하나다 — “해석가능성 기반 방법이 전통적인 불투명 fine-tuning 의 흔한 대안이 될 수 있을까” [MEMIT § 6]. 그 밖에 대칭 관계를 포함한 지식 범위, 어떤 관계는 왜 robust 한 specificity 로 편집하기 어려운가, 그리고 z_i 산출이 “embarrassingly parallel 이므로 배치화할 수 있다” 는 주장만 있고 실행되지 않은 것이 남는다 [MEMIT § 5.2.2].

[LoRA]가 남긴 것 중 이 장에 직접 걸리는 항목은 셋이다. 어느 행렬에 걸지를 heuristic 으로 정한다는 자인 (“더 원리적인 방법이 있는가” [LoRA § 8]), “모델 크기와 적응에 최적인 rank 사이의 관계는 여전히 열린 질문이다” [LoRA App. H.2], 그리고 merge 와 배치가 양립하지 않는다는 한계다 [LoRA § 4.2].

19.8.2 이 책의 판정

[평가] 이 장은 Θ -경로의 primitive 를 세우지만, 그 primitive 를 제공한 두 논문 중 하나는 스스로 실용 방법이 아니라고 선언했고 ([ROME § 3.7], → § 19.2) 다른 하나는 기억 문제가 아니라 배포경제학을 풀었다 ([LoRA § 1]). **재구성은 이 책의 것이고, 재구성이 성립하는 근거는 후행 문헌이 실제로 이 도구를 그렇게 썼다는 사실 하나다.** 그러므로 이 장에서 확립된 것은 “ Θ 에 쓸 수 있다” 이지 “ Θ 에 써야 한다” 가 아니다.

[평가] 두 방식의 갈림길은 rank 도 정확도도 아니라 **아티팩트**다. 같은 $L_w = 0$ 을 두 방식이 정반대 대가로 산다 — delta 방식은 아티팩트를 남겨되돌리기 · 개인화 · 다중 배포를 얻는 대신 요청마다 다른 가중치를 감수하고, 직접 편집은 아티팩트를 없애 shared-weight batching 을 지키는 대신 되돌리기와 개인화를 통째로 포기한다. **이 갈림길은**

알고리즘의 세부가 아니라 서빙 구조의 분기점이며, 정량 판정은 ch26·ch29 가 [X2] 위에서 내린다.

[평가] 그리고 이 장의 도구는 절반만 완성되어 있다. 세 논문전부에서 $\text{gen}(\cdot; B_s)$ 가 항등이므로 ($\rightarrow$ § 19.4) **무엇을 쓸지를 만드는 단계가 없다.** 쓰기 검증도 마찬가지다 — [ROME] 이 그 필요를 명시했지만 어떤 비용 수치에도 그 향이 없다. Θ -경로가 sleep-time compute 이 되려면 두 빈칸이 채워져야 하고, 그중 첫 번째를 다음 장이 채운다.

19.8.3 경로 간 인용여부

세 논문 모두 다른 두 경로에 완전히 침묵한다. 먼저 연대를 갈라 둔다. E -경로 (MemGPT 2023-10, Letta STC 2025-04) 와 W -경로의 sleep 논문들은 **전부 이 장의 세 논문보다 뒤에 나왔으므로**, 그 부재는 연대가 강제한 것이고 흠이 아니다. 진짜 관측은 **동시대의 침묵**이다.

[LoRA] 는 참고문헌 51 항목 안에 retrieval·외부 기억 문헌 0 건, fast weights·test-time training·linear attention 0 건, 지속학습·catastrophic forgetting·replay·EWC·CLS 0 건이다. [ROME] 은 전문 검색에서 sleep 0 건, consolidat 0 건, continual 0 건, forget/catastroph 0 건, replay 0 건, fast weight 0 건, external memory 0 건, adapter 0 건이다. [MEMIT] 도 sleep·consolidat·replay·catastrophic·continual·forgetting 이전부 0 건이며 (‘forgetfulness’ 가 FT-W 기준선 설명에 1 회), LoRA 0 건·adapter 0 건·low-rank 0 건이다.

[평가] 두 방향의 침묵이 이 장의 발견이다. 첫째, **같은 Θ 층 안에서 두 공동체가 서로를 모른다.** LoRA 는 ROME 보다 8 개월 앞서 나왔고 둘 다 같은 종류의 저랭크 가중치 쓰기인데, [ROME] 은 [LoRA] 를 인용하지 않고 자기 갱신을 low-rank adaptation 문헌이 아니라 associative memory 문헌 (Kohonen 1972, Anderson 1972, Bau et al. 2020) 에 연결한다. [MEMIT] 도 PEFT 계보를 한 번도 언급하지 않으며, fine-tuning 은 오직 naive 기준선으로만 등장한다. 두 논문은 **서로를 모른 채 같은저랭크 쓰기 공간을 반대 방향에서 채운다** — [LoRA] 는 rank 를 설계변수로 고정하고, [MEMIT] 의 rank 는 편집 개수의 부산물로 결정된다 [본서 관찰]. 둘째, Θ -경로의 쓰기 기법이 **지속학습 공동체의 어휘를 하나도 쓰지 않고 만들어졌다.** [MEMIT] 의 Γ_0 보존 항은 기능적으로 EWC 의 Fisher 정칙화와 같은 자리에 있지만 (옛 지식을 2 차 통계로 보호한다) EWC 를 인용하지 않는다.

연결점이 하나 있다. [MEMIT] 은 Kohonen(1972) 과 Anderson(1972) 을 [MEMIT § 2] 와 Eq. 7 에서 명시적으로 인용하며 “선형 층을 associative memory 로 보는 고전적 관점을 채택한다”고 선언한다. 이 두 문헌은 Neural Memory 모노그래프가 정리한 fast-weight·linear-attention 계보의 공통 조상이기도 하다 ($\rightarrow$ NM ch03). W -경로와 Θ -경로는 **서로를 인용하지 않지만 같은 1972 년 조상을 공유한다** — 이 장에서 쓸 수 있는 유일한 실제 연결점이다.

마지막으로 [MEMIT] 이 실제로 인용하는 “외부 기억”의 정체를 밝혀 둔다. [MEMIT § 2] 첫 문단이 통째로 상징 knowledge base 계보 (Cyc, WordNet, DBpedia, Freebase, YAGO, ConceptNet, NELL, Wikidata) 이며, retrieval·RAG·vector database 언급은 0 건이다. **같은 external memory 라는 말을 쓰지만 대상이 세계 지식과 사용자문맥으로 갈린다** — 이 corpus 의 E -경로와는 문제 자체가 다르다. Part III 가 이 구분을 유지한다.

19.8.4 다음 장이 받아가는것

ch20 은 SEAL 을 읽는다. 이 장이 넘기는 것은 정확히 이 장이 비었다고 판정한칸이다 — **도구는 있는데 무엇을 쓸 것인가.** [MEMIT] 이 “무엇을 쓸지 이미 안다”는 전제 위에서 “어떻게 많이 쓸지”만 푼 자리에서, 다음장의 논문은 $\mathcal{R}_k$ 를 모델이 스스로 만들게 한다. 그 순간 $\text{gen}(\cdot; B_s)$ 가 항등이 아니게 되고 B_s 가 처음으로 조절가능한 knob 이 되며, ch11 의 판별식 조건 (2) 가 비로소 만족된다. 이 장이 전편집 용량의 실측 상한과 쓰기 부작용은 그때 그대로 제약으로 따라간다.

요약

- Θ 에 쓰는 방식은 산출물의 형태로 갈린다. **delta 방식은 분리 가능한 아티팩트를 남기고, 직접 편집은 base에 흡수시켜 아무것도 남기지 않는다.** 갈림길은 저랭크냐 아니냐가 아니다.
- [ROME]의 갱신 (19-3)에는 η_Θ 가 존재하지 않는다. 등식제약 최소제곱의 닫힌 해이며, Θ -경로의 쓰기가 반드시 SGD여야 하는 것은 아니라는 존재증명이다.
- [ROME]은 스스로 “지식 저장 기제를 이해하기 위한 도구이며 대규모모델 학습을 위한 실용적 방법으로 의도되지 않았다”고 쓴다 [ROME § 3.7]. 이 논문을 Θ -경로 primitive로 세우는 것은 **이책의 재구성이며 저자의 주장 범위를 넘는다.**
- 편집 용량의 실증치는 GPT-J 6B에 10,000개 (s, ϱ , o) 연관이고, 그 지점에서 specificity가 83.5에서 73.7로 떨어진다 [MEMIT Table 2]. **10,000은 붕괴점이 아니라 관측된 하한이다.**
- 같은 방법의 zsRE 종합점수는 50.7이며 Specificity 26.6에 지배되는데, **그 값은 편집하지 않은 GPT-J의 27.0보다 낮다** [MEMIT Table 1].
- 10,000 편집의 sleep 비용은 7.44시간 (GPT-J 6B, A6000)이고 그중 87.95%가 z_i 산출이다 [MEMIT § 5.2.2, App. B.4]. **편집당 2.6773초는 이 책의 나뉠셈이고 논문에 없다.**
- 세 논문 모두 $\text{gen}(\cdot; B_s)$ 가 항등이고 sleep 라운드가 $k = 1$ 에서 멈춘다. 그러므로 B_s 의 knob도 ρ 를 켤 실험도 이장 안에 없다.
- [MemLayers]는 배포 후 갱신이 없으므로 sleep-time compute의 사례가 아니라 Θ 층 용량의 실측 공급원이며 ($\rightarrow$ ch11 § 11.4.4), 채택된 +swilu가 자기 ablation에서 최고가 아니고 8B에서 Llama3.1 8B에 9개 중 7개가 뒤진다 [MemLayers Table 2, Table 3].

자가 점검체크리스트

- delta 방식과 직접 편집을 산출물의 형태로 정의하고, 두 방식이 같은 $L_w = 0$ 을 서로 다른 대가로 산다는 것을 (b)·(c)항으로 설명할 수 있다.
- (19-3)과 (19-6)을 (U- Θ) 표준형과 대조해 **학습률이 없는 쓰기의 구조를 진술**하고, λ 가 α_t 의 어느 역할을 대신하는지 말할 수 있다.
- 편집 용량의 실측 상한을 정의하고, 왜 10,000이 상한이 아니라 하한인지, 그리고 그 값이 어느 지표를 얼마나 희생한 값인지 수치로 말할 수 있다.
- 표 19-3의 12칸 중 “논문에 없음”인 칸을 짚고, C_{cap} 이 [ROME]에서 계산 불가능한 정확한 이유 ($H \cdot D$ 미명시)를 달 수 있다.
- 이 장의 세 논문이 어느 경로를 왜 인용하지 않는지 설명하면서, **연대가 강제한 침묵과 동시대의 선택된 침묵**을 구분할 수 있다.
- Rosetta 사전의 “모델 재배포 = (U- Θ) 1회”, “checkpoint 크기 = C_{cap} ”, “batch로 weight 공유 $\rightarrow \Theta$ -경로에서 깨짐” 세 행을 delta 방식과 직접 편집에 각각 적용했을 때 어느 행이 어느 방식에서 무의미해지는지 말할 수 있다.

ch20. SEAL — 스스로 학습 데이터를 만든다

20.1 Bridge-in: 전작이 남긴문제

ch19는 Θ 에 쓰는 도구를 손에 쥐여 주었다. LoRA는 갱신을 저랭크부분공간에 가두어 delta 아티팩트를 만들고, ROME·MEMIT은 특정 사실을 지목해 직접 덮어쓴다. 세 도구 모두 **어떻게 쓰는**가에 답한다. 어느 것도 **무엇을 쓸 것인가**에는 답하지 않는다. LoRA는 학습 집합을 받아서 쓰고, MEMIT은 삼항 관계를 받아서 쓴다. 그 입력을 누가만드는지는 도구 밖에 남는다.

ch16이 같은 공백을 다른 쪽에서 건드렸다. Generative Adapter는 delta의 내용을 forward pass 하나로 결정하되, 그 결정 규칙은 배포 전에 한 번 학습되고 그 뒤로 고정된다. 문맥마다 무엇을 접을지는 학습된 사영 행렬이 정하고, 그 정책은 문맥을 보고 다시 조정되지 않는다.

SEAL(Self-Adapting Language Models, arXiv:2506.10943, 이하 [SEAL])은 그 빈자리에 모델 자신을 놓는다. 새 문맥이 도착하면 모델이 먼저 **그 문맥으로 자기를 어떻게 학습시킬지를 글로 쓴다**. 그 글이 학습 집합이 되고, 그것으로 $(U-\Theta)$ 를 돌려 얻은 모델의 downstream 점수가 다시 그 글을 쓴 정책의 보상이 된다.

정직하게 적어야 할 것이 하나 있다. **이 논문은 어떤 선행 논문의 open question도 인용해 받지 않는다**. 대신 두 갈래의 **실무 한계**를 명시적으로 이어받고, 두 갈래 모두 저자가 곁친다. 하나는 test-time training 프로토콜이다 — “손으로 조율한 heuristic에 의존해 augmentation과 optimization 설정을 고르는 대신, 우리는 SEAL이 그 결정을 학습하도록 훈련한다” [SEAL § 3.2, p.6]. 다른 하나는 합성 데이터 생성이다 — “SEAL은 정적이거나 heuristic한, 손으로 조율된 생성 전략에 의존하는 대신, gradient 기반 자기 갱신에 적용되었을 때 합성 데이터의 downstream 효용을 직접 최대화하는 생성 policy를 RL로 학습함으로써 이 라인 위에 선다” [SEAL § 2, p.2]. 두 인용의 공통 구조가 이 장 전체의 형태다 — **손으로 고정된 규칙 하나를 학습되는 정책으로 바꾼다**.

ch14가 E-경로에서 상속 없는 시작이었고 ch16이 다른 공동체에서의 입장이었다면, 이쪽은 세 번째 종류다. 계보는 조밀한데 **질문의 형태로 진술되지 않는다**. 그래서 이 책은 논문이 인용한 두 한계 진술을 그대로 옮기고, 그것이 ch19가 남긴 공백과 같은 공백임을 이 책의 판단으로 잇는다.

20.2 문제의식

논문이 세우는 대답은 한 줄이다.

“Large language models (LLMs) are powerful but static; they lack mechanisms to adapt their weights in response to new tasks, knowledge, or examples.” [SEAL Abstract, p.1]

[SEAL § 1]이 그 정적임의 내용을 줬다. 현재의 모델은 과제 데이터를 fine-tuning이나 in-context learning을 통해 “**있는 그대로 (as-is)**” 소비하고 학습하는데, “그런 데이터는 학습에 최적인 형식(또는 분량)이 아닐 수 있고, 현재의 접근법은 모델이 자기 학습 데이터를 어떻게 변환하고 그로부터 어떻게 가장 잘 배울지에 대한 맞춤 전략을 개발하도록 하지 못한다” [SEAL § 1, pp.1–2].

그래서 가설이 던져진다 — “흥미로운 가설 하나를 탐구한다: LLM 이 자기자신의 학습 데이터와 학습 절차를 변환하거나 생성함으로써 self-adapt 할 수있는가?” [SEAL § 1, p.1].

목표 능력은 둘로 갈린다 [SEAL § 3.2 서두, p.5]. (1) 새 정보를가중치에 통합해서 **문맥 없이** 회상되게 하는 것 — no-context SQuAD. (2) 소수 예제로부터 새 과제로 일반화하는 것 — ARC.

[해설] 이 두 목표는 이 책의 프레임에서 같은 층에 대한 두질문이다. (1) 은 E -경로가 $\text{ret}(E, q)$ 로 푸는 문제를 Θ 에 밀어 넣은 것이고, (2) 는 ch16 이 forward 로 만든 delta 를 gradient 로 만들되 그 gradient 의 설정까지 생성하게 한 것이다. 논문은 두목표를 “적용 사례 둘” 로 나열하지만, 통일 표기로 보면 하나의 식 ($U-\Theta$) 에서 $\text{gen}(\cdot; B_s)$ 가 무엇일 뻔느냐만 다르다— (1) 에서는 데이터, (2) 에서는 데이터와 갱신 규칙 둘 다.

신규성의 자리도 못 박아 둔다. 가중치 갱신 자체는 새롭지 않고 (LoRA SFT), RL 알고리즘도 새롭지 않다 (ReST^{EM}). 새로운 것은 **루프를닫은 것**이다 — 갱신된 모델의 점수가 그 갱신의 데이터를 만든 생성기의학습 신호가 된다. 논문의 표현으로는 “별도의 adaptation 모듈이나 보조 network 에 의존하는 선행 접근과 달리, SEAL 은 모델의 생성을 자기 adaptation 과정을 파라미터화하고 제어하는 데 직접 쓴다” [SEAL Abstract, p.1] 이다.

20.3 Core mechanism (통일표기)

먼저 이름 충돌 하나를 막는다. SEAL 이 말하는 “inner loop” 는 NM 이 말하는 inner loop 가 아니다. NM 과 Titans/TTT 계열에서 inner loop 는 토큰마다 W 를 미는 per-token 갱신이다. SEAL 에서 inner loop 는 **SFT 실행 한 번 전체**다 [SEAL Alg. 1 line 5, p.4]. 걸리는시간은 초 단위에서 분 단위다 — self-edit 평가 1 회가 30 – 45 초이고 [SEAL § 5, p.9], few-shot 의 과제당 TTT 는 30 초에서 수 분이다 [SEAL § A.4, p.19]. 이 논문에는 per-token inner loop 가 어디에도 없다. 이 책의독자가 오독할 확률이 가장 높은 지점이 여기다.

배치를 적어 둔다. 지식 통합의 base 모델은 Qwen2.5-7B [SEAL § B.1, p.20], few-shot 은 Llama-3.2-1B-Instruct 다 [SEAL § 4.1, p.6]. 아키텍처는 전 과정에서 수정되지 않는다.

sleep 시점의 첫 동작은 **decode 한 번**이다.

$$\mathcal{R}_k \sim \text{LM}(\cdot \mid c; \Theta_k) \quad (20-1)$$

c 는 문맥이다 — SQuAD passage 하나이거나 few-shot 시연 묶음 하나 [SEAL § 3.1, p.3]. (이 장에서 소문자 c 는 문맥을 뜻한다. NM 이 c 에 준 Atlas 의 Omega-rule 창 길이는 이 장에 등장하지 않는다.) 식 (20-1) 이 표준형 ($U-\Theta$) 의 $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$ 와 대응하는자리이며, 대응은 두 군데에서 어긋난다. gen 이 외생 절차가아니라 $\text{LM}(\cdot; \Theta_k)$ 자신이고, $\mathcal{H}_k$ 가누적된 경험이 아니라 **문맥 하나**다.

두 번째 동작이 실제 쓰인다.

$$\Theta' \leftarrow \text{SFT}(\Theta_k, \mathcal{R}_k), \quad \Theta' = \Theta_k + \Delta, \quad \Delta = \frac{\alpha_{\text{LoRA}}}{r} BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d} \quad (20-2)$$

이 식은 **논문에 없다**. SEAL 에는 inner 갱신에 대한 번호 붙은 식이한 개도 없고, [SEAL § 3.1] – [SEAL § 3.2] 가 산문으로만 진술한다 — “supervised finetuning 을 통해 파라미터를 갱신한다: $\theta' \leftarrow \text{SFT}(\theta, SE)$ ” [SEAL § 3.1, p.4], “각 sequence 에 대해 표준 causal language-modeling loss 를 계산하고파라미터를 갱신해 θ' 을 얻는다” [SEAL § 3.2, p.5]. LoRA 재파라미터화는 ch19 의 원문에서 상속되며 재진술되지 않는다. 식 (20-2) 의우변은 그 산문과 하이퍼파라미터 표를 합친 **이 책의 복원**이다. 갱신이 저랭크인 이유도 산문에 있다 — “갱신당 데이터 양이 적고 전체 갱신회수가 많으므로, 효율적이고 가벼운 튜닝을 위해 low-rank adapter 를 쓴다” [SEAL § 3.2, p.5].

세 번째 동작이 이 논문의 본체다. Θ' 을 downstream 과제 $\mathcal{T}$ 에서 채점하고, 그 점수를 $\mathcal{R}_k$ 를 쓴 정책의보상으로 되먹인다.

$$\mathcal{L}_{\text{RL}}(\Theta_k) := -\mathbb{E}_{(c, \mathcal{T}) \sim \mathcal{D}} \left[\mathbb{E}_{\mathcal{R} \sim \text{LM}(\cdot \mid c; \Theta_k)} [R(\mathcal{R}, \mathcal{T}, \Theta_k)] \right] \quad (20-3)$$

$$R(\mathcal{R}, \mathcal{T}, \Theta_k) = \begin{cases} 1 & \mathcal{R} \text{로 adaptation 한 결과가 } \mathcal{T} \text{에서 } \text{LM}_{\Theta_k} \text{의 성능을 개선하면} \\ 0 & \text{그 외} \end{cases} \quad (20-4)$$

식 (20-3) 은 [SEAL Eq. 1, § 3.1, p.4], 식 (20-4) 는 [SEAL Eq. 2, § 3.1, p.4] 다. 보상이 행동만이 아니라 **파라미터 자신에의존한다**는 것을 논문이 명시한다 — “표준 RL 설정과 달리, 우리 설정에서주어진 행동에 배정되는 보상은 그 행동이 취해진 시점의 모델 파라미터 θ 에 의존한다” [SEAL § 3.1, p.4]. 따라서 RL 의 state 는 (c, Θ) 인데 관측은 c 뿐이다 [본서 추론].

gradient 추정량과 그 성질 — 왜 filtered behavior cloning 이 되는지, 왜확률을 내리는 방향의 gradient 가 없는지, stop-gradient 근사가 무엇을바꾸는지 — 은 **ch05 § 05.7.1 이 소유한다**. 이 장은 그 결론만 받아쓴다: 식 (20-3) 의 gradient 는 $R = 1$ 을 받은 self-edit 들에 대한 SFT 로추약되고, 실제 outer step 은

$$\Theta_{k+1} = \Theta_k - \eta_{\Theta} \nabla_{\Theta_k} \mathcal{L}_{\text{RL}} \quad (20-5)$$

이며 구현은 LoRA SFT 다 — batch 10, 2 epoch, 학습률 3e-4, rank 64, alpha 128, “모든 MLP 과 attention projection layer 에 적용” [SEAL § B.2, p.20].

읽기는 표준형 (R) 에서 두 항이 비어 있다.

$$\hat{y} = f(q; \Theta', W = \emptyset, \text{ret}(E, q) = \emptyset) \quad (20-6)$$

문맥은 질의 시점에 사라진다 — “갱신된 모델은 원문에 접근하지 않은 채 그 passage 에 관한 질문으로 평가된다” [SEAL Fig. 2 캡션, p.5]. 답변프롬프트는 두 줄이다: “Let’ s answer a question directly and concisely. Question: {question} Answer:” [SEAL § B.3, p.20]. **corpus 전체에서 가장 순수한 Θ -경로 읽기다** [본서 판단].

20.3.1 표기 대응표

표 20-1 — SEAL 원 표기 $\rightarrow$ 이 책의 표기

원 논문 기호	이 책	근거
θ — LM_{θ} 의 파라미터 [§ 3.1]	Θ	slow weights. 시간 첨자 붙은 소문자 금지
θ' — inner 갱신 후 [§ 3.1]	Θ'	outer 의 Θ_{k+1} 과 다른 이동임을 눈에 보이게
θ_t — outer RL 반복 t [Eq. 1, Alg. 1]	Θ_k	t 는 토큰 첨자로 예약. SEAL 의 t 는 토큰이 아니다
C — 문맥 (passage 또는 시연) [§ 3.1]	c	맨 C 는 chunk 크기, 용량은 C_{cap}
τ — downstream 평가 [§ 3.1]	$\mathcal{T}$	τ 는세션 첨자로 예약
SE — self-edit	$\mathcal{R}, \mathcal{R}_k$	(U- Θ) 의 학습 집합 자리
L_{RL} [Eq. 1]	$\mathcal{L}_{\text{RL}}$	outer loss 는 calligraphic. 맨 L 은 sequence 길이
$y_s^{(i,j)}$ — self-edit 의 s 번째 토큰, 길이 T [Eq. 4]	$z_t^{(i,j)}, t = 1, \dots, L_{\mathcal{R}}$	y 는 layer 출력, s 는 SGD step
α — LoRA alpha [Tables 3 – 4]	α_{LoRA}	$\alpha_t \in [0, 1]$ 은 retention gate
r — 보상 [Eq. 2–4] 이자 LoRA rank [Table 3]; 같은 rank 를 Table 4 는 R 로 인쇄	보상 $\rightarrow R$, rank $\rightarrow r$. 두 뜻이 맞바뀐다	ch01 의 예약 규칙

원 논문 기호	이 책	근거
“training steps” — “375 step 초과 설정은 폐기” [§ A.2]	s ; ARC 필터는 $s \leq 375$	k (라운드) 와 절대 혼동 금지
layer 수 — 기호 없음. “모든 MLP 과 attention projection layer” [§ B.2]	필요하면 L_{layer} , 대상 모듈 집합은 S	논문의 개수를 주지 않는다
learning rate — inner 와 outer 양쪽	inner η_{in} , outer η_{Θ}	few-shot 에서 η_{in} 은 모델이 생성한다
$D = (C, \tau)$ 쌍의 데이터셋 [Alg. 1]	D	맨 D 는 예약됨
N, M — minibatch 문맥 수, 문맥당 self-edit 표본 수 [Eq. 3]	그대로	충돌 없음

경고 한 줄을 크게 붙인다. 이 corpus 에서 원문과 이 책의 $r \cdot R$ 이 정확히 반대 뜻인 논문은 SEAL 하나다 [본서관찰]. 본문에서 [SEAL Eq. 2] 와 [SEAL Table 3 – 4] 를 나란히 인용할 때마다 이 맞바꿈이 작동한다 — [SEAL § 3] 에서 r 은 보상이고, 부록에서는 r 과 R 이 둘 다 rank 다.

20.3.2 이 장이 소유하는 두정의

정의. self-edit 은 모델 자신이 생성한 자연어산출물로서, 자기 가중치 갱신의 내용을 지정한다. 최소 형식은 갱신에 쓰일 학습 집합 그 자체이고, 최대 형식은 학습 집합에 더해 optimizer 설정 (학습률 · epoch 수 · loss mask) 과 데이터 변환 도구의 호출까지를 포함한다. 통일 표기로 self-edit 은 $\mathcal{R}_k$ 이고 식 (20-1) 로 만들어진다 — 즉 $(U-\Theta)$ 의 $\text{gen}(\cdot; B_s)$ 가 **decode pass** 한 번이다.

정의만으로는 부족하다. 이 정의가 무엇을 가리키는지는 실제로 생성된 두형식이 정한다.

형식 A — 데이터. 지식 통합 실험의 self-edit 은 passage 의 “함의 (implications)” 목록이다. 프롬프트가 전문 그대로다: “Let’s read the following passage and produce a list of implications derived directly or indirectly from the content. / Passage: {passage} / Implications:” [SEAL § B.3, p.20]. 생성된 sequence 는 single-passage 레짐에서 **줄바꿈으로 쪼개져 여러 개의 학습 문서**가 되고, multi-passage 레짐에서는 통째로 하나의 학습 문서가 된다 [SEAL § B.3, p.20]. [SEAL Fig. 2] 가 그 예를 보인다 — Apollo 계획 passage 에서 “1. The Apollo program faced opposition from Kennedy’s science advisor, Jerome Wiesner, who had...” 라는 문장이 나오고, 그문장으로 LoRA SFT 를 돌린 모델이 원문 없이 “Jerome Wiesner” 를 답한다 [SEAL Fig. 2, p.5].

형식 B — 갱신 규칙. few-shot 실험의 self-edit 은 데이터가 아니라 **설정**이다. [SEAL Fig. 3] 이 생성된 내용을 그대로 인쇄한다: basic_augmentations: true, size_augmentations: false, chain_augmentations: false, repeat_augmentations: false, strategy: loss on all tokens, learning rate: 1e-05, epochs: 3 [SEAL Fig. 3, p.5]. 선택 가능한 도구는 회전 · 뒤집기 · 반사 · 전치 · 해상도변경과 그 연쇄 · 반복, 그리고 학습률 · epoch 수 · loss 를 전체 토큰에 걸지 출력 토큰에만 걸지다 [SEAL § 3.2, p.6].

[해설] 형식 B 가 이 장에서 가장 저평가된 사실이다. 표준형 $(U-\Theta)$ 에서 $\text{gen}(\cdot; B_s)$ 은 데이터생성기로만 정의되어 있고, $\eta_{\Theta} \cdot \text{epoch} \cdot \text{loss mask}$ 는 식 밖의 하이퍼파라미터다. 형식 B 는 그 하이퍼파라미터를 $\mathcal{R}_k$ 안으로 끌어들인다. 프레임에 대한 SEAL 의 가장 날카로운 확장이 이것이며, 이 책은 이후 gen 이 **갱신 규칙까지 뱉을 수 있는 채널**을 가진 것으로 다룬다.

두 번째 정의가 루프 자체다.

정의. 자기 생성 데이터로 Θ 를 움직이는 루프는, $(U-\Theta)$ 의 생성기 $\text{gen}(\cdot; B_s)$ 와 갱신 대상 Θ 가 같은 **파라미터 벡터**인 루프다. 라운드 k 에서 (i) Θ_k 가 $\mathcal{R}_k$ 를 쓰고, (ii) $\mathcal{R}_k$ 가 Θ_k 를 Θ' 으로 밀고, (iii) Θ' 의 점수가 (i) 을수행한 정책을 Θ_{k+1} 로 민다. 갱신 대상과 갱신 저자가 분리되지않는다는 것이 이 루프의 정의적 성질이고, 논문의 표현으로는 “SEAL 은모델의 생성을 자기 adaptation 과정을 파라미터화하고 제어하는 데 직접 쓴다” [SEAL Abstract, p.1] 다.

이 루프가 표준형 $(U-\Theta)$ 와 갈리는 지점은 여섯이다.

1. **gen 이 Θ 다.** 표준형은 gen 을외생 절차로 둔다.
2. **outer 목적이 $\mathcal{R}_k$ 위의 loss 가 아니다.** 표준형은 $\nabla_{\Theta} \mathcal{L}(\mathcal{R}_k; \Theta)$ 를 취하는데, SEAL 의 outer gradient 는 $\log p(\mathcal{R}_k | c)$ 에 대한 것이다 —gradient 는 $\mathcal{R}$ 을 썼을 확률을 통해 흐르고, $\mathcal{R}$ 을 소비한 SFT 를 통해서는 흐르지 않는다.
3. **$\mathcal{H}_k$ 가 문맥 하나다.** 누적 경험치 아니다 — “우리방법이 단일 예제 TTT 에피소드로 학습되기는 하지만...” [SEAL § 2 Test-Time Training, p.3].
4. **갱신이 부분공간에 갇힌다 — 한 레짐만 빼고.** 상세는 § 20.4.
5. **gen 이 데이터만이 아니라 갱신 규칙을 뱉는다** (형식 B).
6. **보상이 문맥마다 라벨된 downstream 과제를 요구한다.** 표준형에없는 oracle 이며, 논문 자신이 스케일의 걸림돌로 지목한다 [SEAL § 5, p.9].

20.4 어느 층을 언제쓰는가

“이 값은 누가 학습하는가” 에 전부 답한다.

표 20-2 — 이 값은 누가 학습하는가

객체	학습 주체	언제 움직이는가	질의 후 남는가	출처
Θ — base backbone (Qwen2.5-7B / Llama-3.2-1B-Instruct)	여기서 사전학습하지 않음. 동결도 아님 —레짐에 따라 다름 (아래 세 경우)	(a) few-shot ARC: 안 움직임 (b) 지식통합 $n=1$: 안 움직임 (c) 지식 통합 CPT $n=200, 2067$: 움직인다 (full finetuning)	(a),(b) base 는 불변, adapter 가 남음 (c) 움직인 backbone 이 곧 산출물	§ 4.1 p.6; § 4.2 p.7; Table 2 p.8; § A.2 p.18; § B.2 p.20
self-edit 생성 policy (= 생성기로 본 Θ_k)	outer RL 학습 (배포 전, 오프라인)	ReST ^{EM} 라운드마다 1 회. 지식 통합 2 라운드, few-shot 1 라운드	예 — 출하되는 모델이 이것	§ 3.1 pp.3–4; § A.2 p.18; § B.2 p.20
$\mathcal{R}_k$ — self-edit 자체	파라미터가 아님. sleep 시점 decode 로 생성됨	새 문맥 c 마다 1 회, 질의 도착 전	single-passage 에서는 갱신 후 폐기 . CPT 에서는 전체를 코퍼스로 보존	§ B.2 p.20; § 4.2 p.7

객체	학습 주제	언제 움직이는가	질의 후 남는가	출처
Δ — inner SFT 가 만든 LoRA adapter (실제 새 지식)	sleep 시점 갱신 (문맥마다, gradient 기반)	문맥마다 1 회. decode 중에는 아님	배포 시 예 . RL 학습 중에는 아니 오 —라운드당 750 회 inner 실 행이 전부 평가 후 버려짐	Abstract p.1; § 3.2 p.5; § B.5 p.21
inner 하이퍼파 라미터 (η_{in} , epoch 수, loss mask)	few-shot 에서만 모델이 생성하고 그 생성기가 RL 로 학습됨. 지식 통합에서는 저 자가 grid search	few-shot: 과제 마다 sleep 시점. 지식 통합: 오프 라인 고정	정책은 남고 값은 과제별	§ 3.2 p.6; Fig. 3 p.5; § A.2 p.18; Tables 3 – 4
augmentation 도구 선택	few-shot 에서만 모델이 생성. 도 구 자체는 고정 함수	과제마다 sleep 시점	정책이 남음	§ 3.2 p.6
LoRA rank· alpha· 대상 모듈 S W — fast weights	저자가 고정. 학 습되지도 생성되 지도 않음 이 논문 에 존재하 지 않음	없음	해당 없음	§ A.2 p.18; § B.2 p.20; Tables 3 – 4 전편부재.
E — external store	이 논문 에 존재하 지 않음	—	—	per-token 상태 갱신이 정의되지 않음 전편부재. wake 프롬프트가 비어 있음 § B.3 p.20
wake 시점의 이동	아무것도 움직이 지 않음	—	—	§ 3.2 p.5, Fig. 2

순서는 라운드 k 마다 다섯 단계다. (i) $\mathcal{R}_k \sim \text{LM}(\cdot|c; \Theta_k)$, (ii) inner (U- Θ) 로 $\Theta_k \rightarrow \Theta'$, (iii) $\mathcal{T}$ 에서 Θ' 을 채점해 R 을 얻음, (iv) Θ' 을 버림, (v) outer (U- Θ) 로 $\Theta_k \rightarrow \Theta_{k+1}$. 배포 시에는 (i) – (ii) 만 들고 Θ' 을 남긴다.

시간척도가 셋이고 침자가 셋이다. s 는 하나의 self-edit 을 소비하는 SFT 안의 optimizer step 이고 (few-shot 의 폐기 필터가 $s \leq 375$ 다 [SEAL § A.2, p.18]), 그 위에 모델이 고르는 epoch 수 가있으며, 그 위에 sleep 라운드 k 가 있다. SEAL 이 쓰는 “step” 이라는 단어를 k 로 옮기면 틀린 다. 이 셋을 겹쳐 세는 논문이 corpus 에서 SEAL 하나다 [본서 관찰].

판별식 네 조건을 낸다 ($\rightarrow$ ch11 § 11.3). (1) 질의 전인가: **그렇다** — 문맥이 도착하면 self-edit 생성과 SFT 가 먼저 끝나고 질문은 그 뒤에 온다. (2) $B_s > 0$ 인가: **그렇다** — decode 한 번 + SFT 한 번. (3) 어느 한 층의 상태가 바뀌는가: **그렇다** — Θ 가 움직이고 배포 아티팩트가 바뀐다. (4) 이후 질의에 쓰이는가: **그렇다** — “이 self-edit 들은 지속적인 가중치 갱신을 낳아 지속적인 adaptation 을 가능하게한다” [SEAL Abstract, p.1]. 네 조건 전부 통과이고, 실행되는 식은 (U- Θ) **하나뿐**이므로 순서 문제도 생기지 않는다.

여기서 한 가지를 정정해 둔다. “backbone 은 절대 안 움직인다” 는 서술은 이 논문에 대해 참이 아니다. 논문이 두 번 명시한다 — “표는 두레짐에서의 평균 no-context SQuAD 정확도를 보고한다: single-passage 갱신 (LoRA) 과 소규모 continued pretraining(full finetuning)” [SEAL § 4.2, p.7], 그리고 Table 2 의 열 머리말이 “Single Passage (n = 1; LoRA)” , “Continued Pretraining (n = 200; full-FT)” , “(n = 2067; full-FT)” 다 [SEAL Table 2, p.8]. 네 칸 (도메인 $\times$ 루프) 중 정확히 한 칸에서만 full finetuning 이 일어난다 — 지식 통합 CPT 레짐의 inner 갱신 이다. 나머지 셋은 전부 LoRA 이고, 특히 outer 루프는 두 도메인 모두에서 LoRA 다. 다시 말해

SEAL 이 RL 로 배우는 것은 어느 경우에도 base tensor 를 움직이지 않는다. 움직이는 것은 CPT 의 inner 갱신뿐이다.

[평가] 이 구분은 회계 문제가 아니라 서빙 문제다. 세 칸에서 지속 상태는 adapter 이고 한 칸에서는 **7B 체크포인트 전체**다. 그 한 칸에서 C_{cap} 이 자릿수 단위로 뛰고, ch01 Rosetta 의 “shared-weight batching $\rightarrow \Theta$ -경로에서 깨짐” 행의 발동 조건도 정반대로 바뀐다 (§ 20.7). 그리고 § 20.6 이 보일 Table 2 의 열 사이 비교가 **스케일 계열이 아닌** 이유가 바로 이것이다 — 47.0 $\rightarrow$ 58.2 는 n 의 변화와 갱신 파라미터화의 변화를 함께 담고 있다.

20.5 비용 4 종

표 20-3 — 비용 4 종

기호	값	근거
B_s	일부만 보고. 벽시계로만이고, RL 학습 루프에 대해서만이며, 배포 시점 sleep 단계에 대해서는 없다. self-edit 평가 1 회 “대략 30 – 45 초” ; 지식 통합 1 라운드 = 50 passage $\times$ 5 completion $\times$ 3 seed = 750 회 inner 실행, 2 $\times$ H100 에서 “약 6 시간” ; 헤드라인은 2 라운드; few-shot 은 과제당 TTT “30 초에서 수 분” , ReST ^{EM} 전체 “약 2 – 3 시간” (A100/H100/H200 한 장); proxy-reward 변형은 RL 학습 전체가 약 5 분. FLOPs·self-edit 토큰수는 논문에 없음. 배포 시점 1 문맥 sleep 비용 (decode 1 회 + LoRA SFT 1 회) 을 보상 평가와 분리해 낸 값도 논문에 없음. CPT 실행 비용도 논문에 없음	§ 5 p.9; § A.4 p.19; § B.5 p.21; Table 9 § B.10 p.23
L_w	논문에 없음. 지연도 TTFT 도 tokens/s 도질의당 비용도 어떤 형태로도 보고되지 않는다. wake 에 대한 진술은 구조적인 것 하나뿐이다 — 원문 없이 평가하고 프롬프트가 두 줄이다. 그것이 함의하는 prefill 절감에 수치가 붙어 있지 않다	§ B.3 p.20; Fig. 2 캡션 p.5

기호	값	근거
C_{cap}	rank 로만 보고. 바이트도 파라미터 수도없음. few-shot TTT adapter 는 rank 128·alpha 16 을 q_proj, v_proj, gate_proj, down_proj, up_proj 에; few-shot RL SFT adapter 는 rank 16·alpha 16; 지식 통합 RL SFT adapter 는 rank 64·alpha 128 을 “모든 MLP 과 attention projection layer” 에; inner adapter 탐색 공간 은 rank·alpha 모두 [32,64]. adapter 당 바이트, 학습 파라미터 수, 사용자·세션당 상태 예산, 질의 후 adapter 의처분 (유지·병합·축출) 은 전부 논문에 없음. CPT 레짐의 상태는 adapter 가 아니라 7B 모델 전체인데 그 비용도 회계되지않는다	§ A.2 p.18; § B.2 p.20; Tables 3 – 4 pp.20 – 21; § 4.2 p.7
ρ	정성적으로만. 열화를 보이되 rate 를 인쇄하지 않는다. “Figure 6 에서 보듯 편집 횟수가 늘수록 이전 과제의 성능이 점진적으로하락하며, 이는 SEAL 이 여전히 catastrophic forgetting 에 취약함을 시사한다. 그래도 완전한 붕괴 없이 여러 번의 갱신을 수행할 수 있다” [§ 5, pp.8 – 9]. [SEAL Fig. 6] 은 8 회까지의 히트맵이고, Table 5 는 그히트맵의 항목별 표준오차 (비영 항목에서 0.0211 – 0.0406) 를 싣는다. 정확도 값자체는 어디에도 표로 없다. 따라서 편집 1 회당 손실 %p, 실패 임계까지의편집 횟수, replay·EWC 통제군은 전부 논문에 없음 — “현재학습 설정에서 retention 을 명시적으로 최적화하지 않는다” [§ 5, p.8]	§ 5 pp.8 – 9; Fig. 6 p.8; Table 5 § B.6 p.22

상각식 (A) 는 닫히지 않는다. C_{wake} 쪽은 구조적으로 0 에 가깝다 — 질의 시점에 추가 상태 읽기도 검색 hop 도 없다. 그런데 C_{sleep} 이 FLOPs 로도 토큰으로도 없고, 배포 시점 sleep 단계가 격리되어 측정된 적도 없다. 분모 N_q 도 마찬가지다. 논문에서 가장가까운 수는 평가 설계에서 나온다 — 200 passage 에 974 개 질문, “passage 당대략 5 개” [SEAL § B.4, p.20]. $N_q \approx 5$ 로 읽는 것은 이 책의 대응 해석이며, 논문은 이 수를 상각 인자로 제시하지도 상각 분석을수행하지도 않는다.

[평가] 이 논문은 계산을 질의 이전으로 옮기자고 제안하면서, 옮기는 계산도 사는 절감도 측정하지 않는다. 보고된 두수 (self-edit 평가 30–45 초, 라운드당 6 시간) 는 전부 학습 루프 비용이다. ch01 § 01.5 가 정한 실천 규칙에 따라 이 장은 SEAL 에 대해 “효율적” 이라고 쓰지 않는다. 쓸 수 있는 문장은 둘뿐이다 — wake 시점 프롬프트에서 passage 가 사라진다, 그리고 RL 학습 라운드 하나가 $2 \times H100$ 6 시간이다. ch01 § 01.5 의 비용 4 종 표를 이 논문에서 채울 수 없다는 사실 자체가 ch25 가 받아야 할 자료다.

20.6 실험과 스케일

스케일 상한부터 적는다. 최대 모델은 Qwen2.5-7B 이고, 스케일 점은 3B 와 7B 두 개뿐이다 [SEAL Table 6, § B.7]. 이 논문의 가장 새로운 기제 — 모델이 자기 optimizer 설정을 쓰는 형식 B — 는 **Llama-3.2-1B-Instruct** 에서만 돌아다. RL 학습이 닿은 데이터는 지식 통합에서 2 라운드 $\times$ 50 문맥 = 100 문맥, few-shot 에서 “11 개 학습문제에 대한 단 한 번의 outer loop” 다 [SEAL § A.2, p.18; § B.2, p.20]. 본문과 [SEAL Fig. 1] 이 outer loop 를 반복되는 루프로 제시하는 데 반해 few-shot 도메인에서 실제로 돈 라운드는 하나이며, 두 도메인의 학습량이 이렇게 비대칭이라는 사실은 본문에 정리되어 있지 않다. 하드웨어 상한은 $2 \times H100$ 또는 $2 \times H200$ 이다. 토큰 예산은 어디에도 보고되지 않는다.

지식 통합. no-passage-in-context SQuAD v1.1 이고, 채점은 gpt-4.1-2025-04-14 가 greedy decoding 으로 하는 자동 채점이다 [SEAL § B.4, pp.20–21]. RL 학습 집합과 평가 집합이 겹치지 않음을 논문이 명시한다 [SEAL § B.1, p.20].

표 20-4 — no-context SQuAD 정확도 [SEAL Table 2, p.8]

방법	Single Passage ($n=1$; LoRA)	CPT ($n=200$; full-FT)	CPT ($n=2067$; full-FT)
Base model	32.7	32.7	29.0
Train on Passage	33.5	36.0	31.2
Train on Passage + Synthetic (RL 없는 base 모델 self-edit)	39.7	50.6	43.4
Train on Passage + GPT-4.1 Synthetic	46.3	59.4	49.2
SEAL	47.0	58.2	46.4

초록의 헤드라인은 세 열 중 한 열에서만 성립한다. 초록과 [SEAL § 1] 이 모두 “SEAL 의 자기 생성 데이터가 GPT-4.1 이 생성한 합성 데이터를 능가한다” 고 쓰는데 [SEAL Abstract, § 1, p.1], 그것이 참인 칸은 $n=1$ 의 47.0 대 46.3 — **마진 0.7 점** — 하나뿐이다. 두 continued-pretraining 열에서는 GPT-4.1 이 이긴다: 59.4 대 58.2, 49.2 대 46.4. 논문 본문은 이 사실을 진술한다 — “single-passage 의 경우와 유사한 방법 순위를 관찰하되, GPT-4.1 의 합성 데이터가 SEAL 을 근소하게 앞선다” [SEAL § 4.2, p.7]. 진술은 [SEAL § 4.2] 에 있고 범위 조건은 초록에 없다.

열 사이를 스케일 계열로 읽어서는 안 된다. § 20.4 가 정정한 대로 $n=1$ 은 LoRA 이고 $n=200$ · $n=2067$ 은 full finetuning 이므로, 47.0 $\rightarrow$ 58.2 에는 문서 수의 변화와 갱신 파라미터화의 변화가 함께 들어 있다. $n=200 \rightarrow 2067$ 의 하락도 방법의 열화로만 읽을 수 없다 — 평가 집합이 함께 바뀌어 base 모델도 32.7 $\rightarrow$ 29.0 으로 떨어진다. 읽을 수 있는 것은 **base 대비 마진**이다 (이 뻔셈은 이책의 산술이다): SEAL 은 25.5 $\rightarrow$ 17.4, GPT-4.1 은 26.7 $\rightarrow$ 20.2, 그리고 heuristic 기준선인 Entigraph(10 생성) 는 23.3 $\rightarrow$ 19.6 이다. **세마진이 같은 방향으로 줄되 SEAL 이 가장 많이 준다.**

RL 라운드의 수익 형태도 여기서 읽힌다. ‘Implications’ 프롬프트의 꺾적은 39.7 $\rightarrow$ 43.7 $\rightarrow$ 47.0 이다 [SEAL Table 10, § B.11, p.24]. 논문의 읽기는 “두 번의 반복이면 SEAL 이 GPT-4.1 데이터를 넘어서기에 충분하고, 이후 반복은 수익이 체감한다” [SEAL § 4.2, p.7] 이다. 이것이 B_s -품질 곡선의 형태에 대한 이 논문의 유일한 증거이며, 두 라운드에서 포화한다.

few-shot ARC. ICL 0%, RL 없는 TTT + Self-Edit 20%, SEAL 72.5%, Oracle TTT 100% 다 [SEAL Table 1, p.7]. 이 네 수에는 즉시붙어야 할 단서가 있다. 학습 11 개 · 평가 8 개의 ARC 과제가 “base Llama-3.2-1B-Instruct 의 최적 TTT 설정 아래에서 풀릴 수 있도록 필터링” 되었다 [SEAL § 4.1, p.6; § A.1, p.18]. 따라서 **Oracle TTT 의 100%는 실증 결과가 아니라 필터의 정의적 귀결이고, ICL 의 0%도 ICL 이 못 하고 TTT 가 하는 과제만 고른 결과다.** 헤드라인 전체가 손으로 고른 19 개 과제 위에 서 있다. 그리고 그 위에서도 학습된 정책은 손으로 정한 설정에 **27.5 점 뒤진다** — 논문이 인정한다 [SEAL § 4.1, p.6].

형식이 RL 보다 세다. 논문은 7 개 프롬프트에 대해 ReST^{EM} 2 라운드를 돌린 표를 부록에 신는다.

표 20-5 — self-edit 프롬프트 민감도, no-context SQuAD [SEAL Table 10, § B.11, p.24]

프롬프트	RL 없음	라운드 1	라운드 2	GPT-4.1 합성
Implications (본문의 주 설정)	39.7	43.7	47.0	46.3
Implications-long	49.3	52.4	54.4	54.1
Implications-very-long	45.0	51.5	52.1	40.9
Rewrite	49.4	55.3	55.6	54.4
Self-QA	37.3	42.8	48.7	39.2
No-Prompt	13.8	12.7	18.9	28.6

세 가지가 한 표에서 나온다. 첫째, **RL 을 한 번도 돌리지 않은 두기준선 (49.3, 49.4) 이 논문의 RL 헤드라인 (47.0) 을 넘는다.** 프롬프트 한 단어를 바꾸는 것이 RL 루프 전체보다 값이 크다. 둘째, 그럼에도 **RL 의 기여는 실재한다** — 논문의 진술대로 “모든 경우에서 ReST^{EM} 은 성능을 대략 6 에서 11 퍼센트포인트 향상시켰다” [SEAL § B.11, p.23]. 좋은 프롬프트를 출발점으로 삼으면 55.6·54.4 까지 간다. 문제는 기제가 아니라 **본문이보고한 기준선의 선택이다.** 셋째, **“self-directed adaptation” 이 이표에서 반증된다.** 모델이 자기 self-edit 형식을 스스로 정하게 두면 (No-Prompt) RL 없이 13.8, 2 라운드 뒤에도 18.9 이고, 같은 설정의 GPT-4.1(28.6) 에도 진다. 자기 편집은 **저자가 고른 형식 안에서만 작동한다.**

비싼 부품을 지워도 거의 잃지 않는다. TTT 기반 보상 — finetune-and-evaluate 루프, 즉 이 논문이 선행 합성 데이터 연구 대비내세우는 바로 그 기여 — 를 GPT-4.1 루브릭 채점 (4 범주 각 1-5 점 합) 으로 바꾸면 47.0 대 **45.6** 이고, 시간은 약 6 시간 대 **약 5 분**이다 [SEAL Table 9, § B.10, p.23]. 약 72 배의 학습 비용 절감에 1.4 점이다. 논문 자신이 “정제하면 post-finetuning 성능을 직접 최적화하는 ‘참’ 보상을 능가할 수도 있다” 고 쓴다 [SEAL § B.10, p.23].

비교 대상들. ch16 의 Generative Adapter 를 같은 벤치에 세우면 single-passage 에서 66.8 대 47.0 으로 SEAL 이 **19.8 점 진다** — 헤드라인 수치가 나온 바로 그 설정이다. CPT 에서는 58.2 대 28.0 으로 SEAL 이 이긴다 [SEAL Table 7, § B.8, p.22]. 논문이 교란을 밝힌다: Generative Adapter 는 Mistral-7B 기반이고 SEAL 은 Qwen2.5-7B 이며, SEAL 을 Mistral 에서 돌리지 않았다. heuristic 기준선 Entigraph 는 $n=200$ 에서 56.0, $n=2067$ 에서 **48.6** 으로 SEAL 의 46.4 를 넘는다 (10 생성 기준; SEAL 은 5 생성이므로 계산량이 맞춰져 있지 않고 논문도 그렇게 밝힌다) [SEAL Table 8, § B.9, pp.22-23]. 논문의 읽기는 “RL 로 학습된 self-edit 과 구조화된 heuristic 방법 모두 합성 데이터 생성의 강한 접근” 이다.

모델 크기 논증은 출발 정확도가 다른 두 점 위에 있다. 3B 에서 25.1 → 31.9 → 37.0, 7B 에서 32.7 → 39.7 → 47.0 이고 개선 비율이 1.75 배 대 2.04 배다 [SEAL Table 6, § B.7, p.22]. 논문이 곧바로 물러선다 — “실제로 더 키워 보지 않고는 결론을 내리기 어렵다” .

보고된 수치 자체의 균열 셋. (i) base 정확도가 표마다 다르다 — Table 2·6 은 32.7, Table 7·9 는 32.0 이고 논문은 화해시키지 않는다. (ii) 지식 통합의 모든 정확도가 gpt-4.1-2025-04-14 채점이며 **같은 벤더 모델이 동시에 가장 강한 기준선이다.** 사람과의 일치도도 채점기 분산도 없고, 그 겹침을 논문 어디서도 논의하지 않는다. (iii) 식에 쓰인 보상과 실제로 쓴 보상이 다르다 — 각주 2 가 지식통합에서는 임계값이 아니라 후보 중 argmax 에 보상을 준다고 밝히고, few-shot 에는 문서화되지 않은 필터가 더 붙는다 ($s > 375$ 인 설정 폐기) [SEAL 각주 2 p.4; § A.2 p.18]. 어느 쪽도 Eq. 2 와 Alg. 1 에 없다. RL 알고리즘 선택의 근거도 곡선 없는 부정 결과다 — “GRPO 와 PPO 같은 다양한 on-policy 방법을 실험했으나 학습이 불안정했다” [SEAL § 3.1, p.4].

망각. 순차 self-edit 8 회에 걸쳐 이전 passage 의 성능이 단조하락한다 [SEAL § 5, pp.8 – 9; Fig. 6, p.8]. 이 결과의 정확도 값은 표로 인쇄되지 않고 **표준오차만** 인쇄된다 [SEAL Table 5, § B.6, p.22]. 통제군도 없다. 따라서 이 장은 ρ 를 수치로 옮길 수 없다.

20.7 Systems/serving 합의

독자의 1 번 질문에 한 문장으로 답한다. **decode**에서는 **아무것도 바뀌지 않는다** — 그리고 그것이 이 설계의 전부이자 위험 전부다. 적응된 모델은 평범한 transformer 이고, 붙은 adapter 하나 (또는 CPT 에서는 그냥가중치) 를 제외하면 추가 상태 읽기도, 검색 hop 도, 토큰당 재귀도 없다. 이전 술은 wake 시점 기제가 아예 정의되지 않았다는 사실로부터의 이 책의 추론이다 — [SEAL § 3] 은 데이터와 optimizer 만 바꾼다.

wake 프롬프트는 **비어 있는 쪽으로 줄어든다**. passage 도 검색레코드도 없이 두 줄이다 [SEAL § B.3, p.20]. E -경로와의 대비가 여기서 가장 깨끗해진다 — 검색 시스템은 질의마다 prefill 로 값을 치르고, SEAL 은 질의마다 0 을 치르고 한 번에 전부를 치른다. 식 (A) 에서 $C_{\text{wake}} \approx 0$ 이면 평균 비용이 C_{sleep}/N_q 로 축약되므로, **이 설계는 전부가 상각 베타이다 [본서 추론]**. 그런데 질의당 절감의 크기는 원래 문맥에 있었을 passage 길이인데, 논문이 passage 토큰 수를 적지 않으므로 **이 장은 절감 수치를 인쇄하지 않는다**.

상태량도 인쇄할 수 없다. C_{cap} 을 ch03 § 03.6 의 식 (3-6) 으로 계산하려면 $(r, L_{\text{layer}}, \mathcal{S}, d)$ 넷이 필요한데, 지식통합 실험의 $\mathcal{S}$ 가 “모든 MLP 과 attention projection layer” 라는 산문으로만 있어 재구성되지 않는다. 단정할 수 있는 것은 **레짐 사이의 대비**뿐이다 — 세 칸의 지속 상태는 adapter 이고 CPT 한칸에서는 7B 체크포인트 전체이며, ch03 의 기준점 (GPT-3 175B 에서 $r=4$ delta 가 체크포인트를 약 10,000 배 줄인다) 에 비추면 그 대비는 자릿수 몇개짜리다.

배치가 두 갈래로 갈린다. single-passage 레짐은 “문서마다 adapter 를 따로 학습하고 평가한다” [SEAL § B.8, p.22]. 문서마다 가중치상태가 생기므로 ch01 Rosetta 의 “shared-weight batching $\rightarrow \Theta$ -경로에서 깨짐” 행이 그대로 발동하고, 요청별 adapter 라우팅 (grouped GEMM, multi-adapter 서빙) 이 전제가 된다. ch06 § 06.9 가 이대비에 이미 이름을 붙여 두었다 — single-passage 는 **사용자 · 문서 수준 Θ -경로**이고 CPT 는 **인구 수준 Θ -경로**이며, Rosetta 의 그 행이 발동하는 것은 전자뿐이다. CPT 레짐은 반대다 — 모두가 한 벌의 가중치를 쓰므로 배치가 살아나지만 문서 · 사용자 단위 격리가 사라지고, 그 레짐이 바로 $n=2067$ 에서 heuristic 에 지는 자리다 (46.4 대 48.6). Θ -경로 서빙 문제의 두 뿔이며, 논문은 둘을 모두 보고하면서 그 교환을 이름 부르지 않는다. 서빙 · 배치 · adapter 라우팅에 대한 논의가 논문에만 줄도 없다.

sleep 단계의 워크로드 모양도 여기서 결정된다. 논문의 실행 구성은 생성 · 평가에 vLLM, SFT 에 DeepSpeed ZeRO-3 다 [SEAL § B.5, p.21]. 즉 **SEAL 을 돌리는 fleet 은 문맥마다 학습 job 을 하나씩 돌린다** — 서빙 스택과 학습 스택이 같은 자리에 상주해야 한다 [본서 추론]. 그리고 한 라운드는 750 개의 순차적 finetune-and-evaluate 단위이므로 통상적 의미의 배치도 되지 않는다 [본서 추론].

[해설] 그래서 § 20.6 의 proxy-reward 결과가 systems 독자에게이 논문에서 가장 정보량이 크다. 보상을 “학습 실행” 에서 “채점 호출” 로 바꾸면 sleep 라운드가 먹는 하드웨어의 종류가 바뀐다 — 학습 GPU 예약이 API 호출 예산으로 내려온다. 47.0 대 45.6, 약 6 시간 대 약 5 분이다 [SEAL Table 9, § B.10, p.23]. 그리고 sleep 시점에 전선을 건너가는 것은 가중치가 아니라 **텍스트**다. 가중치 delta 는 그 텍스트를 소비하는 SFT 가 로컬에서 만든다 — 논문이 이 성질을 이점으로 주장한다 (“생성된 데이터는 CPT 에 재사용되거나 임의의 base 모델에 적용될 수 있다” [SEAL § B.8, p.22]).

배포의 실제 걸림돌은 계산이 아니다. **RL 루프가 문맥마다 라벨된 downstream 과제를 요구한다**. 서빙 fleet 에는 그런 라벨이 없다. 논문이 스스로 이름 붙인다 — 이 결합이 “SEAL 의 RL 학습이 라벨 없는 코퍼스로 확장되는 것을 막는다” [SEAL § 5, p.9].

20.8 한계와 bridge-out

논문이 명시한 문제는 셋이다 [SEAL § 5, pp.8–9]. **망각** — 순차 self-edit 이 누적되면 이전 과제가 무너지는데 완화책이 구현되어 있지 않다. 제안된 방향은 회귀에 벌점을 주는 reward shaping, null-space 제약 편집, 표현 중첩, inner loop 를 SFT 대신 RL 로 두는 것이고 **전부미실행**이다. **계산 부담** — self-edit 평가 하나가 모델 전체를 finetune 하고 평가하므로 30–45 초다. **문맥 의존 평가** — 위에서 본라벨 oracle 문제이며, 논문의 제안은 문맥이 아직 창에 있을 때 모델이 자기평가 문항까지 생성하게 하는 것이다.

남긴 문제 넷을 더 적는다. 3B/7B 두 점으로는 이득이 모델크기와 함께 커지는지 판정할 수 없다 [SEAL § B.7]. teacher–student 분리 — 학생을 별도 교사의 편집으로 갱신하고 교사를 RL 로 학습시키는 구성 — 는 전문이 제시되되 **한 번도 실행되지 않았다** [SEAL § 3.1, p.4]. self-edit 형식이 지배적인데 7 개 프롬프트로도 해결되지 않았다 [SEAL § B.11]. 그리고 [SEAL § 6] 이 제안하는 전망 — 사전학습 코퍼스생성기, chain-of-thought 중간의 가중치 갱신, 상호작용마다 self-edit 을 합성하는 에이전트 — **셋 모두 실험이 없다**.

[평가] 이 장이 논문보다 강하게 읽는 것 둘. 첫째, 식 (20-3) 이 실제로 최적화되지 않는다. 보상은 Θ_k 에 의존하지만 미분불가이므로 논문은 그것을 Θ_k 에 대해 고정된 것으로 취급한다 [SEAL § 3.1, p.4]. 증명되는 것은 stop-gradient 를 적용한 대리 목적 이지식 (20-3) 자체가 아니고, 두 목적이 같은 최적점을 갖는다는 논증은 없다 ($\rightarrow$ ch05 § 05.7.1). 둘째, **상태의 수명이 논의되지 않는다**. 질의가 끝난 뒤 adapter 를 사용자별로 유지하는지, 병합하는지, 축출하는지, 버전을 매기는지 — Θ -경로 서빙의 첫 번째 질문인데 논문에 진술이 없다. 가장 가까운 문장이 초록의 “지속적인 가중치 갱신” 이다.

ch06 이 세운 제약이 이 루프에 걸리는 방식. ch06 § 06.8 이 이미 못박은 대로, accumulate 의 유계성 결과 (6-2) 를 떠받치는 네 전제가 SEAL 에서 전부 깨진다 — 라운드마다 재초기화하지 않고 살아 있는 checkpoint 를 증분편집하며, 공변량이 동결되어 있지 않고, 보상으로 후보를 거른다. 따라서 $\pi^2/6$ 은 이 루프의 안전 증서가 아니다. SEAL 쪽에서 확인할 수 있는 것은 셋이다. (i) 관측된 열화는 **라운드 축이 아니라 문맥 축**에 있다 — [SEAL Fig. 6] 은 k 가 아니라 순차 self-edit 8 회에 대한 것이다. (ii) 라운드 축은 $k = 2$ 에서 멈추므로 자기생성 반복의 지속가능성이 관측된 적이 없다. (iii) 스케줄로 보면 single-passage 는 Θ' 을 매번 버리므로 ch06 의 accumulate 도 replace 도 아니고, **CPT 레짐만이 생성물을 코퍼스로 누적한다**. 판정은 ch22 가 한다.

경로 간 인용. W -경로는 조밀하게 인용되되 **한갈래만**이다 — TTT 계열 (Sun 2020·2024, Gandelman, Akyürek 2025) 과 자기참조 network 계열 (Schmidhuber 1987·1992, Irie 2022) 이며, ch16 의 Generative Adapter 는 인용될 뿐 아니라 baseline 으로 돌아간다. 반면 **fast-weight sequence layer 계열은 전무하다** — Titans(2501.00663) 도 Atlas(2505.23735) 도 DeltaNet 도 Mamba 도 선형 attention 도 없고, “fast weights” · “linear attention” · “state space” 라는 어구 자체가 등장하지 않는다. 이 목록에서 **Nested Learning 은 빼야한다** — 2512.24695 는 이 장이 읽는 판본 (v2, 2025-09-18) 보다 뒤에 나왔으므로 연대가 강제한 침묵이고, 인용 부재를 이 논문의 선택으로 셀 수 없다 ($\rightarrow$ ch18). 나머지 다섯은 전부 v2 보다 앞서 나왔고, 따라서 선택된 침묵이다. E -경로는 **완전한 침묵**이다: MemGPT·RAG·vector store·Letta·Mem0·Zep 어느 것도 85 개 참고문헌과 본문에 없다. 생물학과 CLS 도 **0 회**이고, catastrophic forgetting 은 1989 년 · 2014 년의 고전적 틀로만 인용된다. 용량 이론도 걸리지 않는다.

[평가] **corpus 전체에서 한 경로가 다른 경로들의존재를 모르는 가장 강한 사례가 이 논문이다**. 결정적인 것은 침묵의 위치다. [SEAL § 6] 은 “확장된 상호작용에 걸쳐 작동하며 변화하는 목표에 동적으로 적응하는 에이전트 시스템” 을 목표로 제시하고, “상호작용 뒤에 에이전트가 가중치 갱신을 촉발하는 self-edit 을 합성” 하자고 쓴다 [SEAL § 6, p.9]. 그것은 E -경로의 문제 진술 그 자체이며, E -경로는 그 문제를 가중치가 아니라 레코드에 써서 푼다. SEAL 은 그대안을 인용하지도 비교하지도 않고, 왜 레코드 대신 가중치여야 하는지 논증하지 않는다. 연대도 변명이 되지 않는다 — Letta STC 는 동시대 (2025-04) 이고 MemGPT 는 1 년 반 넘게 앞선다. 같은 자리에서 생물학 쪽 침묵도 기록해 둔다: 이 논문은 sleep-time 의 문제 (오프라인 통합, 망각, 지속 적응) 에 전부 도달하

면서 “sleep”이라는 단어를 한 번도 쓰지않는다. CLS(1995)도 EWC(2016)도 Deep Generative Replay(2017)도 이논문보다 8년 이상 앞서므로 이 침묵 역시 연대가 아니라 선택이다. 이장에서 연대가 강제한 침묵으로 분류되는 것은 Nested Learning 한 건뿐이다.

ch21이 받아 가는 것은 둘이다. 첫째, **완화책 없는 망각**. [SEAL Fig. 6]이 보인 하락은 SEAL이 문제로 등록만 하고 남긴 것이고, Language Models Need Sleep의 consolidation·dreaming 수명주기가 정확히 그 자리를메우겠다고 주장한다 — 자기 생성에 replay와 증류를 더하면 무엇이 달라지는지가 다음 장의 질문이다. 둘째, $\mathcal{H}_k$ **의확장**. SEAL의 $\mathcal{H}_k$ 는 문맥 하나이고 라운드는 둘이다. 경험이너적되고 라운드가 반복될 때 (U- Θ)가 어떤 모양이 되는지는 이논문이 답하지 않는다. 반증 축 — $n=2067$ 에서 heuristic에 지는 첫 신호, 그리고 가중치에 얼마나 계속 쓸 수 있는가 —는 ch22가 받는다.

요약

- SEAL은 (U- Θ)를 두 시계로 두 번 실행한다. inner는 문맥마다 self-edit으로 SFT하고, outer는 그 결과 점수를 보상으로 self-edit 생성 정책을 민다. (U-W)도 (U-E)도 인스턴스를 갖지않는다.
- **self-edit**은 모델이 자기 가중치 갱신의 내용을 지정해 쓴 자연어산출물이다. 실증된 형식은 둘 — 합의 목록 (데이터)과 augmentation·학습률·epoch·loss mask 명세 (갱신 규칙). 후자가 표준형 $\text{gen}(\cdot; B_s)$ 에 대한이 논문의 가장 날카로운 확장이다.
- 판별식 네 조건을 전부 통과하며, 읽기는 $W = \emptyset \cdot \text{ret}(E, q) = \emptyset$ 인 **corpus**에서 가장 순수한 Θ -경로 읽기다. wake 프롬프트에 passage가 없다.
- 초록의 “GPT-4.1 능가”는 세 열 중 한 열에서만 성립한다 — $n=1$ 의 47.0대 46.3 뿐이고, 두 CPT 열에서는 GPT-4.1이 이긴다 (59.4대 58.2, 49.2대 46.4). [SEAL § 4.2]는 이를 진술하고 초록은 범위조건을 달지 않는다.
- 표의 열은 스케일 계열이 아니다. $n=1$ 은 LoRA, $n=200 \cdot 2067$ 은 full finetuning이고, 후자에서만 base tensor가 움직인다. outer 루프는 두 도메인 모두 LoRA이므로 **RL이 배우는 것은 어느 경우에도 backbone을 움직이지 않는다**.
- 자기 표가 논지를 세 번 깎는다 — RL 없는 프롬프트 둘 (49.3, 49.4)이 RL 헤드라인 (47.0)을 넘고, 정의적 기여인 TTT 보상을 지워도 45.6에 약 72 배싸며, 모델이 형식을 스스로 정하면 18.9로 무너진다. few-shot 벤치는 TTT 가해성으로 필터된 19개 과제여서 Oracle의 100%가 구성상 참이다.
- 비용 4종 중 셋이 사실상 비어 있다. 계산을 질의 이전으로 옮기자는 논문이 옮기는 계산도 사는 절감도 재지 않았고, 상각식 (A)는 닫히지 않는다.
- 서빙 합의는 두 뿔이다 — per-document adapter는 shared-weight batching을 깨고, CPT는 배치를 되살리되 격리를 잃고 그 레짐에서 heuristic에 진다. 실제 걸림돌은 계산이 아니라 문맥마다 라벨을 요구하는보상 oracle이다.

자가 점검체크리스트

- 식 (20-1)–(20-2)를 표준형 (U- Θ)와 항별로 대조해 gen이 Θ 자신이라는 점과 outer gradient가 $\log p(\mathcal{R}_k|c)$ 를 통해 흐른다는 점을 말할 수 있고, 식 (20-2)가 논문에 없는 이 책의 복원임을 밝힐 수 있다.
- SEAL의 “inner loop”가 NM·Titans 계열의 inner loop와 무엇이 다른지 한 문장으로 말하고, $s \cdot \text{epoch} \cdot k$ 세 첨자를 각각 어디에붙이는지 구별할 수 있다.
- 표 20-4의 세 열에서 “GPT-4.1 능가”가 어느 칸에만 성립하는지 수치로말하고, 열 사이 비교가 스케일 계열이 아닌 이유를 갱신 파라미터화로 설명할 수 있다.
- 표 20-5로부터 RL의 기여와 프롬프트의 기여를 분리해서 진술하고, No-Prompt 행이 “self-directed adaptation”에 대해 무엇을 반증하는지 말할 수 있다.

- 표 20-3의 빈칸 넷이 각각 왜 비어 있는지 말하고, 이 논문에 대해 “효율적” 이라고 쓸 수 없는 이유를 식 (A)의 두 항으로 설명할 수 있다.
- ch06의 accumulate 유계성 결과가 이 루프에 적용되지 않는 이유를 네전제로 대고, SEAL에서 관측된 열화가 라운드 축이 아니라 문맥 축에 있다는 사실을 [SEAL Fig. 6]으로 설명할 수 있다.
- (Rosetta) “모델 재배포” 행과 “shared-weight batching” 행이 이 논문의 두 레짐에서 각각 어떻게 작동하는지 말하고, 그 차이가 C_{cap} 과 롤백 단위에 무엇을 하는지 설명할 수 있다.

ch21. Language Models Need Sleep — 증류와 dreaming

21.1 Bridge-in: 전작이 남긴문제

이 장의 bridge-in 은 이 책에서 유일하게 교과서적으로 성립한다. 독자는 앞의 다섯 장에서 그 반대를 봤다. ch14의 Letta Sleep-time Compute 는 상속할 open question 이 아예 없어 상속 없는 시작으로 써야 했고, ch15의 Mem0·ReasoningBank 은 같은 경로에서 이 라인의 이름을 만든 논문을 인용하지 않았으며, ch17의 Do LMs Need Sleep? 은 다른 두 경로를 한 문장씩 언급하면서 정작 자기 경로의 주류 계보에 침묵했고, ch18의 Nested Learning 은 상속 진술 자체가 없었다. 이 장은 다르다. [LM Need Sleep] (Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories, arXiv:2606.03979) 은 전작이 남긴 공백을 **이름과 함께 지목하고 그 자리에서 시작한다.**

전작이 적은 문장은 ch18 이 이미 인용했다 — [NL] 은 오프라인 consolidation 을 개념으로 소개한 뒤 “본 연구에서는 첫 단계, 즉 온라인과정으로서의 memory consolidation 에 집중한다” 고 범위를 스스로 달았다. 이 논문은 그 문장을 받아 이렇게 연다.

“Behrouz et al. (2025), recently, presented the Nested Learning (NL) paradigm and aimed at the first form of memory consolidation (i.e., online consolidation). ... In this paper, we focus on the second type of consolidation: i.e., offline consolidation via sleep.” [LM Need Sleep § 1, p.3]

양쪽이 다 닫혀 있다. 전작이 “여기까지” 라고 적었고 후속이 “그다음” 이라고 적었다. 상속은 여기서 끝나지 않는다. 이 논문은 전작이 **자기 실패조건까지** 적어 둔 것을 그대로 물려받는다.

“While actively updating this memory system can enhance the resistance to CF, the CF can happen when the update period of all models matched at some point (Behrouz et al. 2025). Therefore, it is crucial that before each update of a memory block, a mechanism consolidates the abstracted knowledge of that block to more stable parameters.” [LM Need Sleep § 2.2, p.6]

이 문장이 § 21.3 의 스케줄을 통째로 결정한다. sleep 은 임의의 시각이 아니라 **어떤 블록이 자기 가중치를 덮어쓰기 직전에** 발화한다. 이유가 상속된 진단이기 때문이다.

논문은 온라인 전용 consolidation 의 결함을 셋으로 정리한다. (i) 같은 추상화 수준에서 지식을 옮기므로 “it still uses the same amount of model’s capacity” , (ii) “Is selective and retrieval-dependent” — 활발히 회수되는 기억만 강화된다, (iii) “Depends on the context: the update caused by the online consolidation is based only on the context of the model, and so misses the higher-level understanding of new knowledge with existing one” [LM Need Sleep § 1, p.3]. 세 항목 각각이 뒤에서 기제 하나씩에 대응한다 — (i) 에 parameter expansion, (ii) 에 teacher 분포로부터의 재표집, (iii) 에 Dreaming 이 대응한다.

이차 상속도 명시적이다. Dreaming 절은 SEAL(Zweiger et al. 2025, arXiv:2506.10943, → ch20) 이 남긴 장애물 셋을 추가적으로 옮겨 적는다: “(1) Due to the cost of supervised fine-tuning (SFT) in SEAL’s inner-loop, it is limited to small number of self-edits (dreams

in our terminology). (2) Potential catastrophic forgetting as the cause of iterative self-improvement in sleep periods. (3) The sampling process only samples from the existing knowledge space of the model” [LM Need Sleep § 3.4, p.9]. 두 번째 항목은 SEAL 이 스스로보고한 한계이며 그렇게 귀속되어 있다.

[평가] 상속 진술이 성립하려면 두 가지가 다 있어야 한다 — 전작이 공백을 자기 언어로 적었을 것, 그리고 후속이 그 공백을 지목했을 것. 이 corpus 에서 둘이 함께 성립하는 사례는 이 쌍이 거의 유일하다. 그런데 이 예외가 왜 성립하는지를 적지 않으면 서술이 정직하지 않다. **[NL] 과 [LM Need Sleep] 의 제 1 저자가 같은 사람이다.** 사슬이 이어진 것이 아니라 한 연구 프로그램 안에서 이어졌다. 이 책이 § 21.8 에서 보일 침묵의 지도 — 이 논문이 W-경로의 sleep 문헌을 한 편도인용하지 않고 E-경로를 두 문장으로 닫는다는 사실 — 와 나란히 놓으면, 계보가 작동하는 단위는 경로가 아니라 저자군이다. 상속의 예외성 자체가 단절의 증거다.

21.2 문제의식

논문이 스스로 세운 문제는 배포 이후의 정지다. LLM 은 “are largely static after their initial deployment ... their knowledge and skills become progressively stale, operating with a fixed “knowledge cutoff” date” [LM Need Sleep § 1, p.1]. 기존 처방 둘을 이름과 함께 기각한다. 확장된 데이터로 재-pre-training 하는 것은 “computationally expensive and impractical for frequent updates”이고, 지속적 파라미터 갱신 또는 low-rank adaptation 은 “with iterative updates often results in Catastrophic Forgetting” 이다 [LM Need Sleep § 1, p.1].

그 위에서 논문이 이탤릭으로 세운 질문이 이 장 전체의 축이다.

“How the model can effectively transfer the fragile short-term memories into more stable long-term knowledge?” [LM Need Sleep § 1, p.2]

[해설] 이 책의 어휘로 옮기면 문장 하나가 남는다. 이번 세션의 지식을 담은 W 가 있고 pre-training 을 담은 Θ 가 있을 때, $W \rightarrow \Theta$ 연산자는 무엇이고 언제 도는가. ch01 이 세운 세 층 프레임에서 이 질문은 자명하게 읽히지만, 원 논문에는 그 층 구분이 없다 — 논문은 처음부터 끝까지 모든 것을 “parameters” 라고 부른다. 이 장이 하는 일의 절반은 그 한 단어를 두 층으로 갈라 읽는 것이다.

논문의 재구획은 더 넓다. train/test 분할 자체를 폐기한다. [LM Need Sleep § 3.1] 의 박스 진술이 그것이다 — “For a continual learner, there is no boarder and clear distinction between training and test time” (원문 오기 그대로) [LM Need Sleep § 3.1, p.3]. 그 자리에 wake/sleep lifecycle 을 놓고, 선행 문헌 전체가 그 구분에 묶여 있다고 주장한다: “To the best of our knowledge, the existing literature (including sleep-inspired studies) remains firmly anchored in the conventional distinction of training and testing phases” [LM Need Sleep § 2.3, p.6].

마지막으로 catastrophic forgetting 의 원인 자체를 바꿔 놓는다. CF 는 “an inherent cause of model’ s limited capacity (e.g., number of parameters), where parameters need to be overridden to incorporate the new knowledge” 이며 [LM Need Sleep § 3.2, p.7], 부록이 이를 정식으로 선언한다: “This reframes catastrophic forgetting as a problem of insufficient capacity rather than of sampling distribution, and addresses it by gradual parameter growth between consolidation steps rather than by re-conditioning a fixed teacher” [LM Need Sleep App. A.4 (i), p.24].

이 재정의가 기제를 강제한다. CF 가 표집 분포의 문제라면 replay 가 답이고 (→ ch07), 용량의 문제라면 답은 용량을 늘리는 것이다. 논문은 후자를 택했고, 그래서 이 장의 sleep 은 **가중치를 고쳐 쓰는 절차가 아니라 가중치를 늘린 뒤 새로 생긴 자리에만 쓰는 절차**가 된다.

[평가] CF를 용량 문제로 재정의하면서 용량을 측정한 문헌을한 편도 인용하지 않는다. How much do LMs memorize(arXiv:2505.24832, 2025-05) 도 Understanding LoRA as Knowledge Memory(arXiv:2603.01097, 2026-03) 도 이 논문의 v2 시점 (2026-07-10) 보다 앞서 나왔고 둘 다참고문헌에 없다 (→ ch09). 용량 주장을 용량 문헌과 접촉 없이 세운 것이며, 얼마의 rank가 얼마의 망각을 막는지에 대한 형식적 진술도 논문 안에 없다. 이 공백의 판정은 ch22가 받는다.

21.3 Core mechanism (통일표기)

이 논문의 기제는 NM ch17이 단계별로 이미 전개했다. 이 절은 그것을반복하지 않고, 세 층 프레임이 강제하는 세 가지만 한다 — wake가 어느 층을움직이는가, sleep의 쓰기가 표준형 (U- Θ)와 어디서 갈라지는가, 그리고 그 둘의 순서.

먼저 읽기를 고정한다. 식 (R)은 이 논문에서 다음으로 붕괴한다.

$$\hat{y} = f(q; \Theta, W, \varnothing) \quad (21-1)$$

E 가 비어 있고, 논문이 그렇게 적는다 [LM Need Sleep App. A.3, p.23]. 이 사실 자체가 § 21.8의 재료다.

21.3.1 wake — 주파수 사다리와 (U-W)

Θ 는 이 논문에서 한 덩어리가 아니다. 갱신 주파수로 분할된 블록사슬 $\theta^{(1)}, \dots, \theta^{(K_{lv})}$ 이며 $f_1 \geq \dots \geq f_{K_{lv}}$ 이다 [LM Need Sleep § 2.1, Def. 1]. sequence layer(attention의 KV cache 또는 Titans/TTT 류의 고정 크기 memory)가 $f \rightarrow \infty$ 끝점에있고, 동결된 pre-trained MLP가 $f = 0$ 끝점에 있다 [LM Need Sleep § 2.2, p.5-6]. 정의와 그 계보는 ch18이 소유하므로 여기서 다시 세우지않는다.

유한 주파수 블록의 wake 갱신은 NM 식 (M5)의 형태다.

$$\theta_{i+1}^{(\ell)} = \theta_i^{(\ell)} - e_{i,\ell}, \quad e_{i,\ell} = \begin{cases} \sum_{t=i-C^{(\ell)}+1}^i \eta_t^{(\ell)} \varepsilon(\theta_t^{(\ell)}; x_t) & i \equiv 0 \pmod{C^{(\ell)}} \\ 0 & \text{otherwise} \end{cases} \quad (21-2)$$

원문 [LM Need Sleep Eq. 2]의 합은 하한이 $t = i - C^{(\ell)}$ (포함) 이어서 크기 $C^{(\ell)}$ 인 chunk에 대해 $C^{(\ell)} + 1$ 개 항을 더한다. 이 책은 NM 식 (M5)의 하한 $t = i - C^{(\ell)} + 1$ 을 쓴다. 원문의 off-by-one이며, 조용히 고치지않고 여기 적어 둔다.

$\varepsilon(\cdot)$ 은 임의의 optimizer의 오차 항이고 chunk 크기는 $C^{(\ell)} := \max_{\ell'} C^{(\ell')}/f_{\ell}$ 로 정의된다 [LM Need Sleep § 2.2]. 원문은 이 오차 항을 $f(\cdot)$ 로 쓰는데 같은 식의위첨자에서 f_{ℓ} 을 주파수로 쓰고 있어 기호가 겹친다 — 대응표에기록한다.

핵심은 판정이다. 식 (21-2)의 갱신은 (U-W)다. 시계가 살아 있는시퀀스의 토큰이고, 산출물이 문맥 경계를 넘어 존속하지 않으며, 배포아티팩트를 바꾸지 않는다. ch18이 [NL]에 대해 내린 판정 — 주파수를낮춰 얻는 것은 느리게 움직이는 W 이지 Θ 가 아니다 — 이 그대로적용된다. 논문이 이 블록들을 “parameters”라 부른다는 사실은 판정을바꾸지 않는다.

21.3.2 sleep — 확장, seeding, 리셋

sleep은 스케줄로 발화한다. 모든 $b \in \mathbb{N}$ 에 대해 step 인덱스 $\{C^{(1)}b, C^{(2)}b, \dots, C^{(K_{lv})}b\}$ 에서 돈다 [LM Need Sleep § 3.2, p.7]. 학습되지 않는다. 주파수가중첩되므로 빠른 블록은 느린 이웃이 한 번 갱신되기 전에 여러 번통합되는데, 논문 자신의 예시가 $f = 1K$ 가 $f = 10K$ 에먹이는 경우로 느린 갱신 1 회당 통합 10 회다 [LM Need Sleep § 3.2, p.7].

한 번의 sleep 은 세 조각으로 이루어진다. 순서가 그림으로도 명시되어 있다— Figure 2 는 모델이 **먼저** 자기 파라미터 수를 늘리고 **그다음** 높은 주파수 memory 에서 낮은 주파수 memory 로 추상화를 seeding 하는 두 단계로 통합을 그린다 [LM Need Sleep Fig. 2, p.7]. 첫째, 받는 쪽 블록 ℓ^* 가 커진다.

$$\theta^{(\ell^*)} \leftarrow \theta^{(\ell^*)} \cup \{A^{(\ell^*), n_{\ell^*+1}}, B^{(\ell^*), n_{\ell^*+1}}\}, \quad A \in \mathbb{R}^{d \times d_{\text{low}}}, B \in \mathbb{R}^{d_{\text{low}} \times d}, d_{\text{low}} \ll d \quad (21-3)$$

n_ℓ 은 블록 ℓ 의 expert 수이고 매 sleep 마다 1 증가한다. 각블록은 “without loss of generality” sparse MoE 로 가정되며 router $\mathcal{R}^{(\ell)}$ 이 expert 집합 위에서 dispatch 한다 [LM Need Sleep § 3.2, p.7]. $\dim \Theta$ 가 매 sleep 마다 엄격히 커진다— 표준형 (U- Θ) 로부터의 가장 큰 이탈이 여기서 [본서 판단].

둘째, 새로 생긴 자리에 쓴다. 이것이 이 장이 소유하는 첫 번째 정의다.

Knowledge Seeding(upward distillation) 은 확장 전의 자기 자신을 teacher 로, 확장 후의 자기 자신을 student 로 삼아, 새로 활성화된 저랭크 expert 에만 gradient 를 흘려 teacher 의 출력 분포와 표집 거동을 재현하도록 학습시키는 절차다. teacher 가 student 보다 작다는 점에서 고전적 distillation 의 방향이 뒤집혀 있다.

논문은 이를 세 단계 (a)/(b)/(c) 로 적는다 [LM Need Sleep § 3.3, p.7-8]. (a) 보내는 쪽 블록 $\ell^* - 1$ 의 갱신 e_{i, ℓ^*-1} 을 계산만 하고 적용하지 않는다. (b) teacher 는 갱신 전 Θ , student 는 예정 갱신과 새 expert 를 엮은 Θ_{exp} 로 두고 목적함수를 돈다. (c) 갱신을 실제로 적용하고, 새 expert 를 활성화하고, 보내는 쪽 블록에 그동안 쌓인 저랭크 expert 들을 리셋한다 — 논문의 표현으로 “the low-rank experts previously added to $\text{MLP}^{(f_{\ell^*-1})}$ are reset (synaptic pruning)” [LM Need Sleep § 3.3(c), p.8].

목적함수는 두 항의 결합이다. distillation 항은 GKD 의 on-policy 혼합이다.

$$\mathcal{L}(\Theta, \Theta_{\text{exp}}) = (1 - \lambda_{\text{on}}) \mathbb{E}_{(x,y) \sim \mathcal{D}} [\mathcal{F}(\text{LM}_\Theta \parallel \text{LM}_{\Theta_{\text{exp}}})(y|x)] + \lambda_{\text{on}} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \text{LM}_{\Theta_{\text{exp}}}(\cdot|x)} [\mathcal{F}(\text{LM}_\Theta \parallel \text{LM}_{\Theta_{\text{exp}}})(y|x)] \quad (21-4)$$

$\mathcal{D}$ 는 “by sampling from the teacher model” 로 만들어진다. 같은 문단이 설계 결정 둘을 명시한다 — student 의 표집 분포를 관통해 backprop 하지 않으며, “we freeze all the parameters in the student model and only updates the expanded parameters” [LM Need Sleep § 3.3, p.8]. 뒤의 것이 이 쓰기를 rank-bounded 로 만든다 (GKD 자체는 → ch04)[본서추론].

보상 항은 **Learning to Imitate**(LTI) 다. 약어 LTI 는 NM ch07 의 linear time-invariant 와 무관하며 이 장 안에서만 이 뜻으로 쓴다. teacher 가 생성한 $d^{(i)}$ 의 무작위 prefix 를 student 에게 완성시키고 두성분을 섞는다.

$$R(\hat{d}^{(i)}; d^{(i)}; \text{LM}_{\Theta_{\text{exp}}}) = \rho_r R_{\text{sem}}(\cdot) + (1 - \rho_r) R_{\text{abs}}(\cdot), \quad R_{\text{abs}} = \begin{cases} 1 - \frac{z(\hat{d}^{(i)}, d^{(i)})}{\max\{|\hat{d}^{(i)}|, |d^{(i)}|\}} & z \leq z_0 \\ 0 & \text{otherwise} \end{cases} \quad (21-5)$$

$z(\cdot, \cdot)$ 은 Levenshtein 거리, z_0 는 유사도 문턱이다 [LM Need Sleep Eq. 3, Eq. 4]. R_{sem} 은 이진값이며 “we use a reward model that is frozen” 으로부터 서술된다 — 그 **reward model** 은 논문 어디에서도 이름도 크기도 밝혀지지 않는다 [LM Need Sleep § 3.3, p.8]. 두 항을 합친 최종 목적함수는 다음이다.

$$\mathcal{L}_{\text{KS}}(\Theta, \Theta_{\text{exp}}) = \mathbb{E}_{x \sim \mathcal{D}} \left[(1 - \lambda_{\text{KD}}) \mathbb{E}_{y \sim \text{LM}_{\Theta_{\text{exp}}}} [R(y)] - \lambda_{\text{KD}} \mathbb{E}_{y \sim \text{LM}_{\Theta_{\text{exp}}}} \mathcal{F}(\text{LM}_\Theta \parallel \text{LM}_{\Theta_{\text{exp}}})(y|x) \right] \quad (21-6)$$

식 (21-4) 와 (21-6) 이 이 논문의 두 중심 목적함수인데 둘 다 원문에서 번호가 없다 — 번호는 보상의 세부인 Eq. 3-5 에만 붙어 있다. 그리고 원문은 식 (21-6) 의 divergence 를 $\mathcal{D}$ 로 쓰면서 같은

줄에서 $\mathcal{D}$ 를 표집 데이터셋으로도 쓰고, 한 문단 앞에서는 같은 divergence 를 $\mathcal{F}$ 로 썼다. 이 책은 divergence 를 $\mathcal{F}$ 로 통일한다. teacher 를 정의하는 문장 자체에도 층 첨자가어긋나 있다 — [LM Need Sleep § 3.3(b)] 는 받는 쪽 블록을 두고 “MLP^(f_{ℓ*})(·) still uses its original parameters $\theta^{(f_{\ell^*-1})}$ ” 라고 적어, 블록 ℓ^* 의파라미터를 $\ell^* - 1$ 의 것으로 지시한다 [LM Need Sleep § 3.3(b), p.8]. 이 절이 세운 보내는 쪽 $\ell^* - 1$ / 받는쪽 ℓ^* 구획으로만 읽어야 문장이 성립하며, 이 책은 그렇게 읽었다. 더 무거운 결함이 하나 남는다: 식 (21-6) 은 보상에서 divergence 를 뺀 형태이므로 최대화 대상인데, 같은 페이지의 산문은 저랭크 파라미터가 “**optimized via the Knowledge Seeding objective to minimize the distillation loss**” 라고 적는다 [LM Need Sleep § 3.3(b), p.8]. 논문은최적화 방향을 끝내 명시하지 않는다.

21.3.3 Dreaming

sleep 의 두 번째 단계는 이 장이 소유하는 두 번째 정의다.

Dreaming 은 sleep 라운드의 두 번째 단계로, 모델이 과제 문맥 c_{task} 를 조건으로 자기 샘플 DREAM⁽ⁱ⁾ 을 생성하되 MoE router 가 의도적으로 무작위 expert 를 하나 더 고르게 하고, 생성된 후보를 gradient norm 으로 거른 뒤 살아남은 것을 각각 격리된 모델 사본에 LoRA SFT 로 적용하는 절차다.

$$\{\text{DREAM}^{(i)}\}_{i=1}^{n_{\text{dr}}} \sim \text{LM}_{\Theta}(\cdot | c_{\text{task}}), \quad g_{\text{DR}}^{(i)} = \nabla_{\Theta} \mathcal{L}_{\text{SFT}}(\text{DREAM}^{(i)}, \Theta), \quad \theta^{(i)} \leftarrow \text{SFT}(\theta^{(i)}, \text{DREAM}^{(i)}) \quad (21-7)$$

선택은 $\|g_{\text{DR}}^{(i)}\|$ 상위 n_{top} 개에다 양성 유지를 위한 무작위 n_{rand} 개를 더한 것이다. 무작위 expert 선택의 목적은 논문 표현으로 “incorporates random irrelevant knowledge to the dreaming” 이다 [LM Need Sleep § 3.4, p.10]. 생성정책은 SEAL 의 이진 개선 보상을 그대로 물려받아 ReST^{EM} 으로 학습된다.

$$R(\text{DREAM}^{(i)}, \tau(\cdot), \text{LM}_{\theta^{(i)}}) = \begin{cases} 1 & \text{LM}_{\theta^{(i)}} \text{가 } \tau \text{ 아래에서 LM}_{\theta^{(i)}} \text{을 개선하면} \\ 0 & \text{그 외} \end{cases} \quad (21-8)$$

$\tau(\cdot)$ 는 “a measure to asses the performance in the downstream evaluation” 이다 [LM Need Sleep Eq. 5, § 3.4, p.9].

[평가] 식 (21-8) 이 이 논문의 lifecycle 주장과 정면으로부딪힌다. [LM Need Sleep § 3.1] 의 박스 정의는 자고 있는 모델이 “receives minimal (or none) input data” 라고 적는데, 식 (21-8) 은 sleep 중에 과제 쌍 (c_{task}, τ) 와 **평가 가능한 downstream 측정**이 주어져 있기를 요구한다. ARC 프로토콜에서는 dream 이야에 “from the few-shot demos” 로부터 생성된다 [LM Need Sleep App. B.3, p.25]. 즉 이 sleep 은 task-aware 이고, 새벽 3 시에 유희 상태로 도는 배포모델에는 τ 가 없다. 논문은 그때 무엇이 이진 개선 보상을 대체하는지말하지 않는다.

21.3.4 표준형 (U- Θ) 와의 차이

한 번의 sleep 라운드 k 를 표준형으로 환원하면 다음이 된다.

$$\Theta_{k+1} = \text{Reset}^{(\ell^*-1)} \left(\text{Grow}(\Theta_k) - \eta_{\Theta} \mathcal{P}_{\{A,B\}} \nabla_{\Theta} \mathcal{L}_{\text{KS}}(\mathcal{R}_k; \Theta_k) \right), \quad \mathcal{R}_k = \mathcal{D} \sim \text{LM}_{\Theta_k} \quad (21-9)$$

Grow(·) 은 식 (21-3) 의 차원 증가 매장, $\mathcal{P}_{\{A,B\}}$ 는 새로 활성화된 저랭크 expert 로의 사영, Reset^(ℓ*-1)(·) 은 보내는 쪽 블록의 누적 expert 소거다. 표준형 (U- Θ) 와의 차이는 여섯 개이고 전부 회계에 걸린다.

1. 차원이 상수가 아니다. Θ_{k+1} 은 Θ_k 보다 엄격히 큰 공간에 산다. 표준형은 고정 크기 Θ 에서 gradient 를 뺀다.
2. gradient 가 직전까지 존재하지 않던 부분공간으로 사영된다. 나머지가 전부 동결되므로 sleep 뒤에도 옛 Θ 는 비트 단위로 동일하다. EWC 류 정규화보다 훨씬 강한 비간섭 보장이며, 대가는 옛가중치를 아예 건드리지 않는 것이다 ($\rightarrow$ ch07 의 정규화 계열과 대조).
3. 학습 집합이 자기 생성이다. $\mathcal{R}_k$ 가 LM_{Θ_k} 에서 표집되므로 표준형의 $\mathcal{H}_k$ 는 저장된 경험이 아니라 모델이 자기 자신에게서 다시 뽑는 분포다. 실데이터 replay buffer 가 없다 (반복 시 무엇이 무너지는지는 $\rightarrow$ ch06).
4. 목적함수가 task loss 가 아니다. 표적이 과거 자기 출력분포이며, 토큰 수준 (GKD) 과 표집 수준 (LTI) 양쪽에서 그렇다.
5. 파괴 연산자가 있다. 표준형에 삭제는 없다. 매 sleep 이 리셋으로 끝나며, 이것이 C_{cap} 을 유계로 만드는 유일한 기제이자 ρ 를 썰 수 있었던 유일한 자리다.
6. 고정 시계로 발화하고, 한 주기에 여러 번 돈다. 연속한 블록쌍마다 통합이 일어나므로 주기당 $K_{\text{lv}} - 1$ 회다 [LM Need Sleep App. A.4 (ii), p.24]. 그리고 Dreaming 이 같은 주기 안에서 서다른 $\mathcal{R}_k$ · 다른 loss · 다른 학습 대상으로 ($\text{U-}\Theta$) 를 한 번 더 인스턴스화한다.

[평가] 이 여섯 중 1·5 번이 회계에서 가장 비싸다. 차원이 변하고 상태가 지워지는 절차는 “checkpoint 크기 = C_{cap} ” 이라는 Rosetta 대응 ($\rightarrow$ ch01) 을 라운드마다 다시 계산하게 만든다.

21.3.5 표기 대응표

원 논문 기호 $\rightarrow$ 이 책 기호. 방향은 항상 이쪽이며, 본문 수식에는 왼쪽 열이 등장하지 않는다.

표 21-1. [LM Need Sleep] 의 표기 대응

원 논문 기호	이 책 기호	주의
γ — 보상 혼합 계수 [Eq. 3]	ρ_r	NM 은 window gate 충돌 때문에 ρ 로 개명했는데, 이 책은 ρ 를 망각 · 열화율로 예약했다. 두 개명이 겹치므로 ρ_r 로 확정한다
α — distill 대 reward 계수	λ_{KD}	retention gate α_t 와 충돌
λ — GKD on-policy 비율	λ_{on}	outer weight decay λ 와 충돌
$r(\cdot)$, r_{sem} , r_{abs} — 보상	$R(\cdot)$, R_{sem} , R_{abs}	r 은 LoRA rank 전용. 같은 논문이 rank 를 $r = 64$ 로 쓰므로 충돌이 원문 내부에 있다 [Table 5]
C — dreaming 과제 문맥 [§ 3.4]	c_{task}	삼중충돌: chunk 크기 C , 용량 C_{cap} , 그리고 같은 논문이 두절 앞에서 쓰는 $C^{(\ell)}$
$\mathcal{D}$ — 데이터셋 겹 divergence	$\mathcal{D}(\text{데이터셋})$, $\mathcal{F}(\text{divergence})$	한 줄 안에서 같은 기호를 두 뜻으로 쓴 원문결함
$f(\cdot)$ — optimizer 오차 항 [Eq. 2]	$\varepsilon(\cdot)$	같은 식의 위첨자 f_ℓ (주파수) 과 충돌
$\theta^{(f_\ell)}$, $C^{(\ell)}$, $e_{i,\ell}$	$\theta^{(\ell)}$, $C^{(\ell)}$, $e_{i,\ell}$	NM 식 (M5) 표기
L / T — 시퀀스 길이 혼용	L	L_w (wake 지연) · $\mathcal{L}$ (outer loss) 과 구분
s_ℓ — 블록 ℓ 의 expert 수	n_ℓ	s 는 SGD step 첨자 전용
k — CMS level 수; Top- k 선택 크기	K_{lv} ; n_{top}	k 는 sleep 라운드 전용

원 논문 기호	이 책 기호	주의
m — 생성 dream 수; b — 무작위 유지 수	$n_{\text{dr}}; n_{\text{rand}}$	m_t 는 outer 1 차 moment
$\mathbf{A}, \mathbf{B}, d_{\text{low}}$ $\mathcal{R}^{(f_\ell)}$ — MoE router	$A^{(\ell),j}, B^{(\ell),j}, d_{\text{low}}$ $\mathcal{R}^{(\ell)}$	ch03 의 (A, B, r) 위에 얹힌다 $\mathcal{R}_k$ (sleep 라운드 학습 집합)와 첨자로 구분
(원 논문 기호 아님) 환원용 확장 · 리셋 · 사영연산자	$\text{Grow}(\cdot), \text{Reset}^{(\ell^*-1)}(\cdot), \mathcal{P}_{\{A,B\}}$	이환원을 Π 로 쓰면 지속학습 성능 행렬 Π (ch07 소유, ch22 가재사용)와 충돌한다. Π 와 맨 P 를 둘 다 비워 둔다

21.4 어느 층을 언제쓰는가

이 절이 STC 프레임에서 새로 보이는 것의 전부다. NM ch17 은 이 논문을아키텍처 라인의 완성형으로 읽었고, 그 독법에서 식 (21-2) 는 상속된 CMS 기계였다. 세 층으로 갈라 읽으면 그림이 달라진다 — 식 (21-2) 가 W 층 운동이고, § 21.3.2 전체가 $W \rightarrow \Theta$ 이송 연산자이며, 리셋이 그것을 복사가 아닌 이송으로 만든다. 이 논문은 corpus 에서 그연산자를 명시적으로 가진 유일한 사례다.

이 장이 소유하는 세 번째 정의가 그것이다.

wake $W \rightarrow \text{sleep } \Theta$ 순서란, 한 주기 안에서 (U-W) 가먼저 돌아 세션의 내용을 W 에 적재하고, 그다음 sleep 이 그 W 의내용을 읽어 새로 확장한 Θ 의 자리에 쓰고, 마지막에 W 쪽의누적분을 지우는 순서를 말한다. 지우기가 있으므로 이것은 복사가 아니라이송이다.

순서가 기제다. 세 단계 (a)/(b)/(c) 의 배치가 그증거다 — 보내는 쪽의 갱신을 계산만 하고 미룬 채 teacher/student 를 세우고, 쓰기가 끝난 뒤에야 적용과 리셋을함께 한다. 갱신을 먼저 적용했다면 teacher 는 이미 사라진 상태이고 distillation 의 표적이 없다. 즉 이 논문의 teacher 는 다른 모델도 더 큰모델도 아니라 아직 덮어쓰이지 않은 몇 밀리초 전의 자기자신이다 [본서 추론].

표 21-2. 이 값은 누가 학습하는가

대상	학습 주체	시계	wake 에 움직이나	sleep 에 움직이나	층
backbone weights (Llama-3B/8B, Llama-3.2-1B, Qwen3-1.7B/8B)	이 논문 이전의 누군가	outer-loop 1 회	아니오	아니오 (동결)	Θ
CMS 사다리 자체 ($f_\ell, C^{(\ell)}$)	손으로 설정	하이퍼파라미터	아니오	아니오	—
유한 주파수 블록의 base weights $\theta^{(\ell)}$	모델 자신, 서빙중인 토큰으로부터	$C^{(\ell)}$ 토큰마다	예	계산은 wake, 적용은 sleep 경계 (c)	W

대상	학습 주제	시계	wake 에 움직 이나	sleep 에 움직 이나	총
sequence layer 상태 (KV cache 또는 고정 크기 memory) 새 저랭크 expert $\{A, B\}$	모델, 토큰 마다	매 토큰	예	논의 없음 (블 랙박스)	W
보내는 블록의 기존 expert 들 MoE router $\mathcal{R}^{(\ell)}$	확장 전 자기 자신으로부터 터의 distillation 아무도 — 파 괴된다	sleep 1 회	아니오	예 — 이 논문 의 쓰기 연산	Θ
dream 별 LoRA adapter $\theta^{(i)}$ dream 생성 정책 frozen semantic reward model $\lambda_{on}, \lambda_{KD}, \rho_r, z_0, n_{dr}, n_{top}, n_{rand}$	격리된 모델 사본 ReST ^{EM} , 식 (21-8) 아무도 — 외 부 · 미식별	sleep (c)	아니오	리셋 (목표값 미명시)	$W \rightarrow \emptyset$
$\lambda_{on}, \lambda_{KD}, \rho_r, z_0, n_{dr}, n_{top}, n_{rand}$	미명세	미명세	미명세	미명세 (dream 표집 시 교란만 명시)	—
external store E	존재하지 않음	—	—	예	Θ 모양
					Θ
					외부
					—

표에서 세 줄이 특히 무겁다. 첫째, router 줄이 전부 “미명세” 다. 새로활성화된 expert 로 dispatch 를 배우는 과정도, 리셋된 expert 로의 dispatch 를 멈추는 과정도 논문에 없다 — 기제의 나머지가 전부 이 줄에 의존하는데도그렇다. 둘째, 하이퍼파라미터 줄의 값이 하나도 없다. Table 5 가 실는 것은 LR·batch·steps·LoRA rank·LoRA alpha 뿐이다 [LM Need Sleep Table 5, p.26]. 셋째, dream 별 LoRA 줄만이 **wake 경로에 gradient 를 들여놓는다** — ARC 에서 “for each unseen task, the model generates 5 dreams and applies them independently before predicting the held-out output” [LM Need Sleep App. B.3, p.25].

[해설] 마지막 줄이 이 논문의 유일한 요청별 상태다. 나머지전부는 전역이다. 논문은 사용자별·세션별 가중치를만들지 않고 모델 하나를 유지한다. 이 사실은 § 21.7 에서 다시 쓴다 — Θ -경로가 곧 per-user delta 라는 도식은 이 논문에 근거가 없다.

21.5 비용 4 종

표 21-3. [LM Need Sleep] 의 비용 4 종

값	논문이 보고하는 것	판정
B_s	FLOPs·토큰 수·절대 wall-clock 전부없음. 비율만 있다 — “With the same number of steps, SFT is 4x more efficient than our method” [App. B.5, p.26]. 동일 성능 기준으로는 SFT 가 AIME-24 $4.3\times$, AIME-25 $3.6\times$, HMMT-25 $4.8\times$ 의 wall-clock 을 쓴다 [App. B.5, p.26]. 절차량은 부분적으로 있다 — sleep 100 step, effective batch 32, LR 5e-6 [Table 5]; ARC 는 과제당 dream 60 개 생성 · 45 개 기각 [App. B.3]; SQuAD 지속 pre-training 은 passage 당 dream 5 개 $\times$ 200 passage [§ 4.1, p.12]	절대값 논문에 없음. teacher 표본 수, on-policy rollout 수, sleep 시 시퀀스 길이, 실험 당 sleep 주기 수, GPU 수 · GPU-hour 가 전부 없다
L_w	지연 · TTFT·ms/token· throughput 숫자가 논문 전체에 없다. 가장 가까운 진술은 파라미터 수 주장이다 — “5 MLP blocks with dimension 64 as the additional parameters and keep the active parameter count unchanged” [App. B, p.25]	논문에 없음. active parameter 불변은 계수 (計數) 주장이지 측정이 아니다. 식 (21-2) 가 요구하는 wake 시 optimizer 오차 누적에도, ARC 의 wake 내부 5 회 LoRA 적합에도 비용이 붙어 있지 않다
C_{cap}	파라미터로는 부분적으로 있다 — 통합 1 회당 저랭크 expert 하나, 즉 $2d d_{low}$ 개 [§ 3.2, p.7]. 실험 전체 추가분은 “5 MLP blocks with dimension 64” [App. B, p.25]. dreaming SFT 는 rank $r = 64$, alpha 128 [Table 5]	바이트로는 논문에 없음. d 와 d_{low} 의 값이 없고, expert pool 은 사전 할당 후 마스킹되므로 [§ 3.3 구현 주석, p.9] 상한이 $t = 0$ 에 정해지는데 그 pool 크기가 없다. 세션별 · 사용자별 값이 아니라 전역 값이다

값	논문이 보고하는 것	판정
ρ	라운드당 열화 수치가 없다. 정성 서술과 그림만 있다 — 지속 학습에서 ICL 은 급락하고 Hope 는 이득을 더 지킨다, “Performance improves monotonically with additional consolidation stages” [§ 4.1, p.11-12, Figure 5]. Cartridges 와 SFT 는 “outside of the plot” [§ 4.1, p.12]. “increasing the lowest frequency reduces performance” [§ 4.1, p.11, Figure 4]	논문에 없음. 그리고 측정 불가능하다 — 이논문에는 sleep 주기 수를 독립변수로 놓은 실험이 하나도 없다. Hope-1/2/3 은 통합 단계 수를 바꾼것이지 라운드 수가 아니다 [§ 4.1, p.11]

식 (A) 의 손익분기 N_q^* 는 이 논문에 대해 세울 수 없다. 이유가구조적이다. 쓰기가 문맥별이 아니라 스케줄별이고, 수해자가이후의 모든 질의이므로 상각의 분모 N_q 가 “앞으로의 전 트래픽” 이 된다. 분모가 무한하면 상각된 sleep 비용은 0 으로 간다.

[평가] 이것은 논문의 승리가 아니라 프레임에 대한발견이다. 식 (A) 는 한 문맥을 N_q 개 질의가 공유한다는 E -경로의모양을 전제한다. Θ -경로는 그 전제를 받지 않으므로 (A) 가퇴화하고, 그 순간 구속력 있는 질문이 두 개로 옮겨간다 — 쓰기가살아남는가 (ρ) 와 용량이 버티는가 (C_{cap}). 그리고 이논문은 그 둘을 다 보고하지 않는다. 세 층 프레임에서 Θ -경로의증거 공백은 정확히 이 모양이며, Part III 의 비용 회계가 이 자리에서시작 한다.

[평가] 그러므로 이 장은 이 논문을 “효율적” 이라고 쓰지않는다. 비용 4 종의 네 칸 중 절대값이 있는 칸이 0 개다. 논문이 가진 유일한효율 근거는 SFT 대비 상대비이고, § 21.6 이 보이듯 그 SFT 는 자기 표에서 base model 보다 낮은 baseline 이다. 비용 침묵을 성능 주장으로 메우지 않는것이 이 책의 규칙이다.

21.6 실험과스케일

실증 상한부터 적는다. 가장 큰 모델은 8B 다 — 지속학습에 Llama3-8B, 수학추론에 Qwen3-8B [LM Need Sleep § 4.1, p.11; Table 2, p.12]. 가장 작은것은 Llama-3.2-1B [App. B.3]. from-scratch pre-training 은 없고 8B 위의 모델도 없다. 가장 큰 데이터 스케일은 BABILong 의 10M 토큰 문맥이고, 지식 주입은 200 passage / 974 문항이다 [§ 4.1, p.12]. 실증된 sleep 예산은 effective batch 32 에서 100 optimization step 이며, 실증된 wake/sleep 주기 수는 어느 실험에서도진술되지 않았다 [본서 관찰].

데이터 쪽 규모도 적어 둔다. 지속학습 텍스트 분류는 CLINC150(150 in-scope intent, 10 도 메인, 총 23.7K 질의 중 in-scope 22.5K·OOS 1.2K), Banking77(77 intent, 13,083 예제, 불균형), DBpedia(level-2 70 클래스, 훈련 10K· 테스트 1K 부표집) 이고, 장문맥 쪽은 LongHealth(각 5.1K – 6.8K 단어의 사례 문서 20 편, 문항 200 개) 와 QASPER(약 1.6K 편의 논문에 대한 약 5K 문항, 전문을 문맥으로) 다 [LM Need Sleep App. B.1, p.25]. MK-NIAH 는 길이 격차가 진술되지 않았다.

수학 추론 표가 숫자를 가진 헤드라인이다. Qwen3-8B, AIME-24 / AIME-25 / HMMT-25, avg@16 [LM Need Sleep Table 2, p.12]: Base(Instruct) 73.8 / 68.1 / 42.4, SFT 75.5 / 66.4 / 43.7, GRPO 76.4 / 68.1 / 44.9, OPSD 76.6 / 67.4 / 45.1, Sleep 79.2 / 69.0 / 46.1. Qwen3-1.7B 에서는 Base 49.8 / 34.5 / 25.7, SFT 47.3 / 36.1 / 22.9, GRPO 51.0 / 38.6 / 26.1, OPSD 51.6 /

40.0 / 28.1, Sleep 53.2 / 40.2 / 29.3 이다. 최강 baseline OPSD 대비 마진은 8B 에서 +2.6 / +1.6 / +1.0, 1.7B 에서 +1.6 / +0.2 / +1.2 이다 [본서 산술]. 같은 표에서 SFT 는 1.7B 의 AIME-24(47.3 대 49.8) 와 HMMT-25(22.9 대 25.7), 8B 의 AIME-25(66.4 대 68.1) 에서 **base model 보다 낮다**.

지식 주입은 SQuAD 무문맥 정확도 평균이다 (단일 passage $n = 1$ / 지속 pre-training $n = 200$) [LM Need Sleep Table 3, p.12]: Base 31.9 / 31.9, dreaming 없는 fine-tuned 모델 33.4 / 32.0, SEAL 46.7 / 43.2, Sleep(Transformer) 48.1 / 44.3, Sleep(Transformer + four-level) 48.9 / 46.2. dreaming 제거는 35.7 / 36.2 로 떨어뜨려 - 13.2 / - 10.0 이며, **Dreaming 이 이 결과를 지고 있다** [본서 산술]. 반면 gradient 기반 선택 제거는 47.1 / 45.2, 무작위 expert 제거는 48.0 / 44.7 로 각각 - 1.8 / - 1.0, - 0.9 / - 1.5 에 그친다 [본서 산술]. ARC 는 성공률로 ICL 0, TTT 10, SEAL 72.5, Sleep 80 이다 [LM Need Sleep Table 4, p.12].

21.6.1 캡션이 자기 표에 반증되는자리

[LM Need Sleep § 4.2] 는 “All the components contribute positively to the performance of our method” 라고 적는다 [LM Need Sleep § 4.2, p.13]. 그 문장이 인용하는 표가 그것을 반증한다.

표 21-4. 구성요소 ablation, Qwen3-8B, AIME-24 / AIME-25 / HMMT-25, avg@16 [LM Need Sleep Table 1, p.11]

구성	AIME-24	AIME-25	HMMT-25
Sleep (전체)	79.2	69.0	46.1
- Imitation Learning	76.8	67.9	45.0
- Semantic Reward	78.9	69.2	44.5
- w/o Expansion	78.2	67.9	44.9
OPSD	76.6	67.4	45.1
OPSD + Expansion	77.9	68.2	45.9

Semantic Reward 를 빼면 AIME-25 가 69.0 에서 69.2 로 오른다. 구성요소를 제거했는데 성능이 올라간 칸이 표 안에 있고, 그 표를 인용하는문장이 모든 구성요소가 긍정적으로 기여한다고 적는다. 하필 그 구성요소가 § 21.3.2 에서 확인한 대로 **이름도 크기도 밝혀지지 않은 외부 frozen reward model** 에 의존하는 항이다. 논문의 효율 비교 (App. B.5) 에도 그모델의 비용은 계상되어 있지 않다.

두 번째로, 헤드라인 구조적 기여의 크기가 작다. 확장을 뺀 Sleep 이 78.2 / 67.9 / 44.9 이므로 확장의 기여는 1.0 / 1.1 / 1.2 다. 반대 방향이 더 날카롭다 — 확장만 baseline OPSD 에 없으면 76.6 / 67.4 / 45.1 → 77.9 / 68.2 / 45.9 로 +1.3 / +0.8 / +0.8 오른다. Sleep 이 OPSD 를 이기는 총마진이 +2.6 / +1.6 / +1.0 이므로 **그마진의 절반가량 — HMMT-25 에서는 1.0 중 0.8 — 이 Knowledge Seeding 없이구조적 확장만으로 얻어진다**. ch04 가 이 계산을 먼저 적었고 판정을 이 장에넘겼다. 판정은 이렇다.

[평가] 이 논문이 새로 도입한 것은 Knowledge Seeding 과 Dreaming 둘인데, 수학 추론에서 마진의 절반은 그 둘이 아니라 파라미터를더 준 것에서 나오고, 지식 주입에서 마진의 대부분은 Dreaming 에서 나온다. **Knowledge Seeding — 이 논문의 이름이 걸린 기여 — 이 수치로 자기몹을 보인 표가 없다**. 게다가 표 21-4 전체에 시드도 신뢰구간도 오차막대도없고, 여기서 논의되는 차이의 다수가 0.2 - 1.5 점이다. avg@16 이라는 표기만있고 분산은 어디에도 없다 [Table 1/2 캡션].

21.6.2 숫자가 없는 실험, 그리고 이름의충돌

일곱 개 실험 중 넷이 숫자 없이 그림으로만 보고된다 — 지속학습텍스트 분류 (Figure 3), 통합 단계 수 sweep(Figure 4), 신규 언어 지속번역 (Figure 5), BABILong(Figure 6) [LM Need Sleep § 4.1, pp.11-12]. 논문에도 부록에도 이들의 값을 싣는 표가 없다. 이 논문의 가장 강한 스케일주

장인 “Hope achieve almost perfect score in scaling to 10M of tokens” [§ 4.1, p.11] 에도 붙은 숫자가 없다.

그림뿐인 넷 가운데 Figure 4 는 이 논문이 자기 설계 손잡이를 쥘어 본유일한 자리라 특히 아깝다. 진술된 추세는 둘인데 두 번째가 반직관적이다 — 통합 단계 수를 늘리면 in-context learning 과 장문맥 이해가 좋아지고, “increasing the lowest frequency reduces performance, suggesting that making the most persistent memory more adaptive weakens retention” 이다 [LM Need Sleep § 4.1, p.11]. 가장 존속하는 memory 를 더 적응적으로 만들수록 retention 이 나빠진다는 이 관찰은 Θ -경로설계에 직접 걸리는 진술인데 수치가 없다. 같은 그림의 캡션은 세 패널 중 QASPER 만 축이 뒤집혀 있다고 경고한다 (“Lower values indicate better performance for QASPER”) — 재작도할 때 반드시 옮겨야 할 단서다.

그 10M 결과에는 저자 자신의 유보가 붙어 있다: “Finally, we observe that all small models, including Hope, suffer substantial performance drops without fine-tuning” [LM Need Sleep App. B.2, p.26]. 소형 baseline 은 전부 공식 BABILong 훈련 프로토콜로 fine-tune 되었고 비교 대상인 GPT-4·GPT-4o-mini 는 zero-shot 이다 — 그림 캡션이 그 사실을 스스로 적는다 (“Red points correspond to fine-tuned models, whereas blue points correspond to zero-shot evaluations of large-scale models”) [Fig. 6 캡션, p.11]. 동등 비교가 아니다.

이름의 충돌도 있다. [LM Need Sleep § 4.1] 은 plain Hope 를 baseline 으로 도입해 놓고 (“We also include Hope ... as a multi-level in-context updating baseline without explicit distillation process”) 바로 뒤에서 “Results in Figure 3 show that Hope performs best across datasets” 라고 쓴다 [LM Need Sleep § 4.1, p.11]. 승리를 선언하는 문장이 baseline 의 이름을 부르며, 결과가 그림뿐이라 본문만으로는 어느 쪽인지 판별되지 않는다.

내부 참조도 깨져 있다. [LM Need Sleep § 4.2] 는 “The results are reported in Figure 1” 이라고 적지만 Figure 1 은 관행적 ML 과 지속학습을 대비한 도식이고 실제 결과는 Table 1 에 있다. 같은 문장이 자기 개선 ablation 을 “Table 3” 으로 넘기는데 그것은 지식 주입 표다. 그리고 Table 1 과 Table 2 의 캡션이 바이트 단위로 동일하다 [LM Need Sleep § 4.2, p.13; Tables 1 – 2].

21.6.3 비교 조건과 기반구조

효율 주장은 SFT 하나에 대해서만 측정되었다. step 당 SFT 가 4 배 효율적이고, 동일 성능 기준으로 SFT 가 $4.3\times / 3.6\times / 4.8\times$ 의 wall-clock 을 쓴다 [LM Need Sleep App. B.5, p.26]. GRPO 와 OPSD 에 대한 등-계산 비교는 없다. 그런데 Table 5 는 GRPO 에 500 step 을, SFT 와 Sleep 에 각각 100 step 을 주고 LR 과 batch 는 동일하게 둔다 [LM Need Sleep Table 5, p.26] — **예산이 맞춰져 있지 않고**, 어느 arm 이 수렴했는지 보이는 학습 곡선도 없다.

ARC 헤드라인은 40 개의 이진 결과 위에 서 있다. 지표가 “the fraction of dreams that yield a correct answer” 이고 held-out 8 과제 $\times$ dream 5 개이므로, 80%는 32/40 이고 SEAL 의 72.5%는 29/40 이다 — **격차가 결과 3 개**다 [LM Need Sleep App. B.3, p.25]. 훈련 풀은 11 과제이며 과제 집합은 “to avoid tasks that remain unsolvable under standard configurations” 로 걸러졌다. 그리고 SEAL 비교를 싣는 Table 3 의 **backbone** 이 논문 어디에도 명시되지 않는다 — “We follow the experimental setup of Zweiger et al. (2025), including the choice of models and parameters, for the sake of fair comparison” 이 모델에 대한 유일한 진술이다 [LM Need Sleep § 4.1, p.12].

가장 무거운 것은 마지막이다. **이 논문의 두 기여가 자기 아키텍처위에서 함께 돈 적이 없다.** [LM Need Sleep § 3] 은 모든 것을 CMS/MoE backbone 위에서 정의하고 [LM Need Sleep § 4.1] 의 통합 실험은 Hope 를 쓰는데, Dreaming 결과는 “Sleep (Transformer)” 와 “Sleep (Transformer + four-level)” 이며 [Table 3, p.12] ARC 는 Llama-3.2-1B 다 [App. B.3, p.25]. 통합과 dreaming 을 Hope/CMS 위에서 함께 돌린 실험이 논문에 없다. 두 절반이 서로 다른 기반에서 검증되었고 그 합성은 미검증이다.

여기서 NM ch17 이 확정한 사실이 STC 프레임에서 다시 판정된다. 이 논문의 sleep 기계 — $\lambda_{on}, \lambda_{KD}, \rho_r, z_0$, reward model, ReST^{EM} 루프 — 는 어느 것도 pre-training 에서 end-to-end 로 meta-learn 되지 않으며, 실험의 대부분이 pre-trained Llama/Qwen 위에 sleep 기계를 얹은

graft 다 (→ NM ch17). NM 은 이것을 라인의 이름새로 읽었다 — 지금까지 update rule 자체를 outer loop 가 미분해 학습하던 계보가 여기서 알고리즘적 wrapper 로 바뀌었다는 것이다.

[평가] STC 프레임에서는 다른 값을 갖는다. **graft 라는 사실은 이 논문의 $W \rightarrow \Theta$ 이송 연산자가 학습된 연산자가 아니라 손으로 배치한 절차라는 뜻이다.** 이송의 시점 (스케줄), 이송의 폭 (d_{low}), 이송의 표적 (teacher 분포), 이송 뒤의 삭제 (리셋) 가 전부 하이퍼 파라미터이고 그 값들이 논문에 없다. 세 층사이에서 무엇을 언제 옮길지가 이 책의 중심 질문인데, corpus 에서 그 연산자를 가진 유일한 논문이 그것을 학습하지 않는다. 이것은 결함이 아니라 **이 경로의 현재 위치다** — Θ -경로는 이송을 할 줄 알지만 아직 언제 얼마나 이송할지를 데이터로 정하지 못한다. 반대로 이 사실은 graft 가 갖는 한 가지 이점도 정직하게 만든다: 기성 checkpoint 위에 얹을 수 있으므로 8B 실증이 가능했다. 라인의 from-scratch 상한 (1.3B/100B tokens, → ch18) 이 깨진 것은 아니다.

마지막으로 우선권 주장 하나를 기록한다. App. A.2 는 동시대 OPSD 계열과의 차별점 셋 중 **첫 번째로 발표 순서**를 든다 — “In addition to the fact that this study is older than such methods, Sleep fundamentally delivers a different messages that...” [LM Need Sleep App. A.2, p.23], 근거는 2025 년 9 월부터 OpenReview 에 공개되어 있었다는 p.1 각주다. 통제된 비교가 아니라 순서가 첫 논거로 놓였다 [본서 판단].

21.7 Systems/serving 합의

독자의 질문은 하나다 — 그래서 decode 에서 뭐가 바뀌는가. 이 논문의 답은 세 겹이다.

첫째, inference 가 forward 전용이 아니게 되고, 그 변화가 sleep 이 아니라 wake 에 착지한다. 식 (21-2) 는 유한 주파수 블록마다 토큰당 optimizer 오차 항을 누적하고 $C^{(\ell)}$ step 마다 가중치에 적용할 것을 요구한다. Figure 7 이 이 책에서 가장 값진 그림인 이유가 여기 있다 — 논문에서 갱신 주기가 토큰 수로 나타나는 **유일한** 자리이고, 사다리가 $1K \rightarrow 5K \rightarrow 10K$ 로 적혀 있다 [LM Need Sleep Fig. 7, p.22].

[해설] 서빙 엔진 쪽으로 옮기면 이렇다. decode 경로 안에 backward 가 가능한 kernel 이 상주해야 하고, 활성 블록마다 파라미터 크기의 accumulator 가 하나씩 필요하며, $1K$ 토큰마다 가중치 쓰기가 발생한다. Rosetta 표의 “decode = $C = 1$ 의 per-token write” (→ ch01) 가 여기서 $C^{(\ell)} = 1000$ 으로 커진 형태다. **논문은 이 wake 비용에 어떤 숫자도 붙이지 않는다.**

둘째, 정적 텐서 모양이 의도적 서빙 결정이고 그 이유가 적혀있다. 구현 주석이 말한다 — “Implementing the growing sparse modules can be extremely challenging if it requires a direct change in the dimensionality of tensors ... Alternatively, we can initially have those parameters in the model, but masked them in the forward and backward pass, before their initial activation in a sleep stage” [LM Need Sleep § 3.3, p.9]. 컴파일된 그래프 · kernel autotuning · checkpoint 레이아웃이 sleep 을 넘어 안정적으로 유지되므로, 이 corpus 에서 가장 서빙 친화적인 설계 결정이다 [본서 판단]. 대가는 § 21.5 가 적은 대로 $t = 0$ 에 정해지는 크기 미상의 하드 천장이다.

셋째, 존속하는 상태가 KV cache 에서 가중치로 옮겨간다 — 크기가 아니라 종류가 바뀐다. BABILong 설정은 $10M$ 토큰 문맥에 닿는데 [§ 4.1, App. B.2] 그 영역에서 $O(L)$ KV cache 는 선택지가 아니다. 대신 통합 1 회당 존속 delta 는 $A \in \mathbb{R}^{d \times d_{\text{low}}}$, $B \in \mathbb{R}^{d_{\text{low}} \times d}$ 하나, 즉 LoRA 모양의 객체다. 서빙 문제가 pagged-KV 관리에서 **adapter 버전 관리와 축출**로 옮겨간다 [본서 추론].

[평가] 여기서 반드시 끊어 둘 것이 있다. 이 상태는 **전역**이다. 논문은 모델 하나를 유지하며 세션별 · 사용자별 상태가 없다. 따라서 “ Θ -경로는 사용자별 가중치 delta 를 만들고, 그 delta 트래픽이 memory-centric 기회를 얻다” 는 서사를 이 논문에 귀속시킬 수 없다. 이 책은 근거가 있는 자리에만 그 논증을 쓴다 (NM D4 승계). 이 논문이 실제로 공급하는 것은 per-user 상태가 아니라 **버전 관리 문제**다 — sleep 이 $\{C^{(\ell)}b\}$ 마다

발화하고 매 발화가 base 갱신 · expert 활성화 · 다른 블록 expert 리셋을 수행하므로, checkpoint 관리 · cache 무효화 · 회귀테스트 · 롤백이 전부 주기 단위 운영이 된다. 그리고 리셋의 목표값이 명시되지 않았으므로, 외부 스냅샷 없이는 순진한 구현이 sleep 이전 모델로 되돌아갈 수 없다.

sleep job 의 모양도 통상과 다르다. Knowledge Seeding 은 teacher 표집 데이터셋, on-policy student rollout, LTI prefix 완성을 필요로 하고, Dreaming 은 n_{dr} 회 생성 + 후보마다 backward 1 회 + 살아남은 dream 마다 격리된 LoRA SFT 를 필요로 한다. ARC 수치로는 과제당 60 생성 · 60 backward · 15 LoRA 적합이다 [LM Need Sleep App. B.3, p.25]. 지배적 비용 모양이 training FLOPs 가 아니라 **autoregressive decode** 라는 뜻이며, 그러면 sleep job 은 훈련 클러스터가 아니라 wake 트래픽과 같은 하드웨어 · 같은 대역폭 지배 영역을 놓고 경쟁한다. 논문이 절대 비용을 보고하지 않으므로 이 추론은 검증할 수 없다. 덧붙여 후보 dream 마다 도는 gradient 계산은 App. B.5 의 4 배 수치에 **포함되어 있지 않다** — 그 비교는 Sleep step 과 SFT step 을 견줄 뿐 선택 오버헤드를 세지 않는다.

마지막으로 배칭이다. ARC 프로토콜은 요청 하나 안에서 dream 5 개를 생성하고 각각을 독립 적용한 뒤 예측한다. LoRA 로 갈라진 모델 인스턴스 5 개와 그 안의 fine-tuning 5 회가 사용자 응답 경로에 들어간다는 뜻이고, Rosetta 표의 “batch 로 weight 공유” 행이 정확히 이 지점에서 깨진다 [본서추론]. **이 프로토콜에 붙은 지연 수치는 없다.**

21.8 한계와 bridge-out

21.8.1 논문 자신이 남긴문제

먼저 형식을 적는다. **이 논문에는 limitations 절이 없다.** [LM Need Sleep § 5] Conclusion 은 다섯 문장이고 유보도 실패 모드도 부정결과도 담고 있지 않다 [LM Need Sleep § 5, p.13]. § 21.6 이 적은 불리사실은 전부 표 · 부록 · 내부 불일치에서 복원한 것이다. 참고문헌에도 기계적 정리의 흔적이 남아 있다 — Pang et al., Singh et al., Shenfeld et al., Zhang et al.(OPSDL), Zhao et al. 의 항목이 중복등재되어 있고 [pp.17-21], 그 다수가 동시대 OPSD 흐름을 두 페이지로 훑는 App. A.4 에 걸린다.

그럼에도 논문이 간접적으로 남긴 문제는 뚜렷하고, 대부분 이 책의 비용 4 종에 직결된다.

- **반복 안정성.** [LM Need Sleep § 3.2] 는 고정 크기 느린 블록에 반복 통합하는 것을 “a critical bottleneck” 이라 진단하고 확장을 처방한뒤, sleep 주기 수를 독립변수로 놓은 실험을 한 번도 보고하지 않는다. lifecycle 을 주장하는 논문에서 라운드 수가 측정되지 않았다.
- **pool 고갈.** 구현 주석의 사전 할당 · 마스킹은 무한성장을 하드 천장으로 바꾼다 [§ 3.3, p.9]. 천장에 닿으면 무슨 일이 일어나는지, pool 이 얼마나 큰지 둘 다 없다.
- **언제 잘 것인가.** 스케줄 $\{C^{(\ell)}b\}$ 는 주파수사다리에서 상속된다. 트리거를 학습하는 것이 없고, 실제 서빙 트래픽이 균일한 토큰 스트림이 아니라는 사실은 다뤄지지 않는다.
- **리셋의 수치적 정의.** “reset (synaptic pruning)” 이 0 인지재초기화인지 저장된 초기값 복원인지 명시되지 않는다 [§ 3.3(c), p.8]. 망각 거동과 checkpoint 서사가 이 선택에 함께 걸린다.
- **무엇이 보존되는가.** consolidation 이 원자료 replay 가 아니라 “추상화” 를 추출한다고 주장하면서, 무엇이 남고 무엇이 버려지는지에 대한 측정도 rate – distortion 형태의 진술도 없다. 통합 1 회당 얼마의 rank 가 얼마의 망각을 막는지도 정식화되어 있지 않다.
- **reward model 의 편향과 drift.** 고정 심판이 매 sleep 마다 바뀌는 student 를 채점하는데 그 편향 · drift · reward hacking 이 검토되지 않는다. 그리고 그 항의 제거가 벤치마크 하나를 개선한다 (표 21-4).
- **재귀적 자기 수정의 안전.** 스케줄에 따라 자기 가중치를 고쳐쓰고 무작위 expert 로 “random irrelevant knowledge” 를 주입하는 모델의 안전 · 거버넌스 논의가 논문 어디에도 없다.
- **서빙 회계.** 종단 wake/sleep 주기의 wall-clock 분해가 없고, replay buffer 라는 고전적 대안과의 등-계산 비교도 없다 — 생물학 프레임이 불러들이는 바로 그 대안인데도 그렇다.

21.8.2 이 책의 판정

[평가] 이 논문은 corpus 에서 $W \rightarrow \Theta$ 이송을 명시적 연산자로 가진 유일한 사례이며, 그 점에서 세 경로 사이의 실제 다리다. ch18 은 갱신 주파수를 낮추는 것만으로는 Θ 에 닿지 못한다고 판정했다. 이 논문은 그 판정을 우회하지 않고 인정한 뒤, 닿는 방법을 별도의 오프라인 절차로 만들었다 — 확장하고, 위로 증류하고, 아래를 지운다. **이 책이 Θ -경로를 별도 축으로 세우는 근거가 여기서 실증된다.**

그러나 다리가 놓였다는 것과 다리가 하중을 견딘다는 것은 다르다. 하중 시험에 해당하는 값이 ρ 인데, 그 값은 없을 뿐 아니라 **주어진 증거로부터 계산될 수도 없다** (§ 21.5). Θ -경로 논문에서 이보다 결정적인 결론은 없다. 이 논문의 논지 전체가 반복되는 wake/sleep 주기 위에 서 있고, 반복이 측정되지 않았기 때문이다.

[평가] 두 번째 판정은 기여의 배분에 대한 것이다. 표 21-4 가 말하는 바를 그대로 읽으면, 수학 추론에서 마진의 절반은 파라미터를 더 준것에서 나오고, 지식 주입에서 마진의 대부분은 Dreaming 에서 나오며, Semantic Reward 는 한 벤치마크에서 음의 기여를 한다. **이 논문의 이름이 걸린 기여인 Knowledge Seeding 이 자기 몫을 수치로 보인 자리가 없다.** 그럼에도 [LM Need Sleep § 4.2] 는 모든 구성요소가 긍정적으로 기여한다고 적는다. 이 corpus 에서 캡션이 자기 표에 반증되는 사례는 이것이 처음이 아니며 ($\rightarrow$ ch15, ch18), 반복된다는 사실 자체가 Part III 의 재료다.

21.8.3 경로 간 인용 여부

연대를 먼저 갈라 둔다. 이 논문의 v2 는 2026 년 7 월 10 일 자다. 따라서 Rate-Distortion Memory Compaction(2607.08032) 과 Can a LM Learn Facts Continually in Its Weights?(2607.11020) 의 부재는 **연대가 강제한 것**이며 이 장은 그것을 흠으로 세지 않는다 ($\rightarrow$ ch22). 아래는 전부 선택된 침묵이다.

E-경로 — 이름으로 부르고 두 문장으로 닫는다. Letta 의 Sleep-time Compute(2504.13171) 과 MemGPT(2310.08560) 가 둘 다 인용되어 있다. 교전 방식이 특징적이다. [LM Need Sleep § 2.3] 은 Letta 의 절차를 “mainly is designed as a proposal for spending compute on the context rather than training the model itself” 라 요약하고, App. A.3 이 이렇게 닫는다 — “Despite similarity in the name, their method is fundamentally different from our work. While this method aim to find a good summary of the interactions of the model with users in the text space, when the model is idle, our proposal is on using distillation to transfer text data knowledge into a form of parametric weight update” [LM Need Sleep App. A.3, p.23]. MemGPT 은 “orchestration systems like MemGPT manage context via virtual memory paging rather than architectural modification” 한 문장이다. Mem0·Zep·ReasoningBank 은 전부 없다.

[평가] 기각이 **정의로 이루어지고 실험으로 이루어지지 않았다**. E-경로 시스템은 일곱 실험 어디에도 baseline 으로 등장하지 않는다 — SQuAD 지식 주입처럼 검색이나 요약 baseline 이 자명한 통제군인 과제에서도 그렇다. 이 논문은 자기 이름의 소유권을 두고 다투면서, 상대를 한 번도 돌려 보지 않는다. 그리고 두 경로를 **함께** 쓰는 구성 (외부 요약과 파라미터 통합의 합성) 은 질문으로도 제기되지 않는다 — 정의로 닫혔기 때문이다. ch12 가 2022 년에 이미 이 질문이 한 번도 제기된 적 없다고 기록했고, 2026 년에도 여전히 그렇다.

W-경로 — 아키텍처는 전수, sleep 은 전무. 인용은 두렵다. Titans(2501.00663) 는 BABI-Long baseline 으로 실제로 돌기까지 하고, TTT(2407.04620) 는 ARC baseline 이며, Nested Learning(2512.24695) 은 직계 부모다. linear attention, Mamba, xLSTM, Schmidhuber 의 Fast Weight Programs(1992), Cartridges, DuoAttention 이 모두 있다. 그런데 **Do Language Models Need Sleep?(2605.26099) 이 참고문헌에 없다.** 제목이 한 글자 차이로 겹치는 W-경로의 sleep 논문이고, 두 논문의 간격은 한 달이다. Memory Caching(2602.24281) 도 없다.

침묵은 **일방향**이다. ch17 이 확인한 대로 Do LMs Need Sleep? 쪽은 이 논문을 인용하고 있으며, 다만 그항목에 arXiv ID 도 venue 도 연도도 없이 제목만 있다 — OpenReview 투고본에서 인용했다는 뜻이다. 즉 W-경로 쪽은 Θ -경로의 이 논문을보았고, Θ -경로 쪽은 W-경로의 sleep 논문을 보지 않았다. W-경로 문헌을 가장 두껍게 인용하는 논문이 정확히 W-경로의 sleep 문헌만 비워 두었다. 아키텍처는 전수 조사하고 같은 경로의 같은 주제는 0 건이다.

Θ -경로 형제와 이론 축. SEAL 은 인용되고 Table 3·4 의 baseline 으로도 돈다. LoRA 와 EWC 도 있다. 반면 SCM(2604.20943) 은 없고 — 같은 경로의 다른 sleep 논문이다 — ROME(2202.05262)·MEMIT(2210.07229) 도 없어 **가중치에 직접 쓰는 다른 방식 전체가 교전되지 않는다** ($\rightarrow$ ch19). Generative Adapter(2411.05877) 와 Memory Layers(2412.09764) 도 없고, 생물학 프레임이 기대는 바로 그기제인 Deep Generative Replay(1705.08690) 도 없다. 용량 축은 § 21.2 가 적은 대로 비어 있다.

생물학 축 — 가장 두껍게 인용되고 가장 얇게 쓰인다. [LM Need Sleep § 1] 이 두 페이지 가까이를 인간 sleep 신경과학에 쓰고, CLS(McClelland 외 1995), Tononi – Cirelli 의 synaptic homeostasis, sharp-wave ripple 연구가 줄지어 인용된다. 그런데 **어느 생물학결과도 정량적 예측으로 바뀌지 않는다**. 가장 직접적으로 관련된 두계산신경과학 결과 — Gonzalez 외 2020 과 Tadros 외 2022, 둘 다 sleep 유사 replay 가 인공신경망의 CF 를 줄인다는 것 — 은 [LM Need Sleep § 2.3] 의목록 한 줄에 담긴 뒤 한 번도 비교 대상이 되지 않는다 ($\rightarrow$ ch08, ch23). 은유는 이 송되었고 측정은 이송되지 않았다.

21.8.4 다음 장이 받아가는것

이 장은 Θ -경로가 실제로 작동하는 형태를 보였다 — 확장하고, 위로증류하고, 아래를 지우면 배포 가중치가 바뀐다. ch22 가 받아가는 것은 그절차가 **계속** 둘 수 있는가라는 질문이다. 이 장이 남긴 두 결측이정확히 그 질문의 두 변수다. ρ 는 반복이 열화를 낳는지 묻고, C_{cap} 은 사전 할당된 pool 이 언제 바닥나는지 묻는다. 이논문은 둘 다 보고하지 않으며, 논지 전체가 그 둘에 걸려 있다. ch22 는가중치에 사실을 계속 넣을 수 있는가를 정면으로 묻는 반증 축을 세우고, ch09 가 세운 용량 천장과 ch06 이 세운 자기 생성 데이터의 경계를 그 위에엮는다. 이 장의 자기 생성 $\mathcal{R}_k$ 와 리셋 연산자는 그때 다시호출된다.

요약

- [LM Need Sleep] 의 bridge-in 은 이 corpus 에서 거의 유일하게 양쪽이달혀 있다 — [NL] 이 오프라인 consolidation 을 범위 밖으로 선언했고, 이 논문이 “In this paper, we focus on the second type of consolidation: i.e., offline consolidation via sleep” 으로 그 자리를받는다 [LM Need Sleep § 1, p.3]. 두 논문의 제 1 저자가 같다.
- 세 층 프레임에서 새로 보이는 것은 순서다. 식 (21-2) 의 wake 갱신은 (U-W) 이고, [LM Need Sleep § 3.3] 의 세 단계가 $W \rightarrow \Theta$ 이송연산자이며, 마지막의 리셋이 그것을 복사가 아니라 **이송**으로만든다. corpus 에서 이 연산자를 명시적으로 가진 유일한 논문이다.
- 표준형 (U- Θ) 와의 차이 여섯 중 둘이 회계에 치명적이다 — $\dim \Theta$ 가 라운드마다 커지고, 매 라운드가 파괴 연산자로 끝난다.
- 비용 4 종의 절대값이 네 칸 모두 없다. B_s 는 SFT 대비 상대비만있고 (step 당 4 배, 동일 성능 $4.3 \times / 3.6 \times / 4.8 \times$ [App. B.5, p.26]), L_w 는 어떤 시간 수치도 없으며, C_{cap} 은 바이트가 없고, ρ 는 **주어진증거로부터 계산될 수도 없다** — sleep 주기 수를 독립변수로 놓은 실험이없기 때문이다.
- 상각식 (A) 가 이 논문에서 퇴화한다. 쓰기가 문맥별이 아니라 스케줄별이라 N_q 가 “앞으로의 전 트래픽” 이 된다. (A) 는 E·W-경로의 도구이고 Θ -경로는 그것을 받지않는다.
- Semantic Reward 를 제거하면 AIME-25 가 69.0 에서 69.2 로 **오른다** [LM Need Sleep Table 1, p.11]. 그럼에도 [LM Need Sleep § 4.2] 는 “All the components contribute positively to the performance of our method” 라고 적는다 [p.13]. 그 구성요소는 이름도 크기도 밝혀지지않은 외부 frozen reward model 에 의존한다.

- OPSD 대비 마진의 절반가량이 Knowledge Seeding 없이 구조적 확장만으로 얻어진다 (OPSD + Expansion 이 76.6 / 67.4 / 45.1 → 77.9 / 68.2 / 45.9). 일곱 실험 중 넷은 숫자가 없고, 시드 · 신뢰구간 · 오차막대가 논문 전체에 없다.
- 기제는 pre-trained Llama/Qwen 위의 **graft** 이며 라인 최초로 end-to-end meta-learn 되지 않았다 (→ NM ch17). STC 프레임에서 이 사실의 뜻은 하나다 — 이송의 시점 · 폭 · 표적 · 삭제가 전부 손으로 정한 하이퍼파라미터이고, 그 값들이 논문에 없다.
- 경로 간 침묵이 특징적이다. *E*-경로는 이름으로 부르고 두 문장으로 기각되 baseline 으로 한 번도 돌리지 않는다. *W*-경로는 아키텍처 계보를 전수 인용하면서 *W*-경로의 sleep 문헌 (2605.26099, 2602.24281) 만 0 건이며, 그 침묵은 일방향이다.

자가 점검 체크리스트

- 이 장의 bridge-in 이 왜 예외인지 두 조건으로 진술하고, 그 예외가 성립하는 이유가 저자 동 일성이라는 사실까지 말할 수 있다.
- wake $W \rightarrow$ sleep Θ 순서를 세 단계 (a)/(b)/(c) 로 재구성하고, 왜 보내는 쪽 갱신의 적용을 미뤄야 하는지 설명할 수 있다.
- 식 (21-9) 가 표준형 ($U-\Theta$) 와 갈라지는 여섯 지점을 열거하고, 그중 어느 둘이 C_{cap} 과 ρ 에 직접 걸리는지 지목할 수 있다.
- 이 논문에 대해 상각식 (A) 를 세울 수 없는 이유를 N_q 의 정의로 설명하고, 그 결과 구속력 있는 질문이 어느 두 값으로 옮겨가는지 말할 수 있다.
- 표 21-4 에서 캡션 주장이 반증되는 칸을 수치로 지목하고, Knowledge Seeding 과 parameter expansion 의 기여를 분리해 계산할 수 있다.
- “graft 이지 meta-learn 이 아니다” 라는 사실을 NM 의 독법과 이 책의 독법에서 각각 어떤 뜻으로 읽는지 구분해 말할 수 있다.
- 이 논문의 존속 상태를 Rosetta 사전의 어느 항목 (“모델 재배포” , “checkpoint 크기 = C_{cap} ” , “paged KV cache / session cache”) 에 대응시킬지 정하고, 마지막 항목이 **아닌** 이유 — 상태가 세션별이 아니라 전역이라는 것 — 을 설명할 수 있다.

ch22. 반증 축 — 가중치에 사실을 계속 넣을 수 있는가

22.1 Bridge-in: 전작이 남긴문제

ch19는 Θ 에 쓰는 도구를 주었고 (LoRA와 직접 편집), ch20은 무엇을 쓸지를 모델이 스스로 정하게 했으며 (SEAL의 self-edit), ch21은 W 에서 Θ 로 옮기는 이송 연산자를 세웠다. 세 장이 답한 질문은 전부 같은 형태다 — **한 번의 쓰기를 어떻게 하는가**. 이 장은 방향을 뒤집는다. 같은 루프를 스무 번, 백 번 돌린 뒤에 무엇이 남는가.

이 질문을 실제로 돌린 논문이 corpus에 한 편 있다. [Continual Facts] (Can a Language Model Learn Facts Continually in Its Weights?, arXiv:2607.11020)는 발명된 사실 하나씩을 rank- r adapter로 학습해 Θ 에 merge하는 절차를 20회, 이어서 100회 반복하고, **같은 내용을 프롬프트에 넣은 대조군**을 나란히 세운 뒤 잔여물을 잔다. Θ -경로의 다른 논문은 전부 통합 라운드 1회 또는 짧은 수열만 보인다.

그런데 이 장의 bridge-in은 ch21처럼 닫혀 있지 않다. **이 논문은 sleep-time compute 문헌의 어떤 open question도 상속하지 않는다**. 이라인의 논문을 한 편도 인용하지 않고, “sleep-time compute”이라는 말도 쓰지 않는다 (§ 22.8.5). 대신 상속을 두 곳에서 명시적으로 적는다.

“This extends the finding of Shenfeld et al. (2025), that KL from the base policy predicts forgetting, to knowledge writing.” [Continual Facts § 6.1, p.14]

“These nulls accord with the warning of Hase et al. (2023) that localisation need not identify the best site to edit.” [Continual Facts § 7.3, p.18]

그리고 공백은 스스로 만든다 — “These observations lack a common account of why some writes produce usable knowledge, why some survive, and what a forgotten fact leaves behind” [Continual Facts § 1, p.1]. 여기서 “these observations”가 가리키는 다섯 편 (Gekhman 외 2024, Berglund 외 2024, Lampinen 외 2025, Allen-Zhu & Li 2024, Shenfeld 외 2025) 중 어느 것도 이 책의 세 경로에 속하지 않는다.

연대는 먼저 갈라 둔다. 이 논문의 v2는 2026년 7월 14일 자이고 ch21이 다른 논문의 v2는 7월 10일 자다. 따라서 ch21이 이 논문에 침묵한 것은 **연대가 강제한 것이다**. 반대 방향은 아니다 — SEAL(2506.10943), Nested Learning(2512.24695), Language Models Need Sleep(2606.03979), SCM(2604.20943)은 모두 선행하며 이 논문의 참고문헌에 없다 (§ 22.8.5).

[해설] 이 배치가 반증 축의 성격을 정한다. **가장 강한 반증 후보가 반증 대상을 읽지 않고 쓰였다**. 이것은 약점이면서 동시에 강점이다 — 약점은 이 논문의 판정이 Θ -경로 논문들이 실제로 제안한 절차를 대상으로 삼지 않았다는 것이고, 강점은 그 판정이 이 라인의 주장에 맞춰 설계되지 않았다는 것이다. 수렴이 독립적일 때 증거는 더 강해진다.

22.2 문제의식

논문은 지속 쓰기를 세 요구로 분해하고 각각에 판정을 내린다.

“Each write must create usable knowledge, general abilities must survive accumulating updates, and earlier facts must remain reachable. Data breadth makes the first tractable, while a frozen teacher or explicit penalty makes the second tractable. The third remains unresolved.” [Continual Facts § 9.3, p.20]

이 세 요구가 이 장이 소유하는 첫 정의를 낳는다.

정의. Θ -경로의 반증 조건은 다음 셋 중하나라도 성립하면 $(U-\Theta)$ 를 반복하는 설계가 목적을 달성하지못한다는 진술이다. (i) **생성 실패** — 한 번의 $(U-\Theta)$ 가회수는 되지만 사용은 되지 않는 객체를 만든다. (ii) **역량 실패** — 라운드가 누적될수록 Θ 가 원래 갖고 있던 일반 능력이 무너진다. (iii) **도달 실패** — 앞서 쓴 내용이 Θ 안에 남아 있는데도질의로 닿지 않는다. 세 조건은 독립이며, 하나를 고치는 처방이 다른 하나를고치지 않는다.

논문이 겨냥하는 것은 알고리즘이 아니라 쓰기가 만드는 **객체**다 — “the object that a write creates: what kind of knowledge it contains, whether later writes preserve it, and what remains after questions about it fail. This turns catastrophic forgetting into a property of the written object, measured against the same content placed in context” [Continual Facts § 1, p.1].

판정 문장은 두 개다. “When facts must be composed or survive later writes, the reliable channel is context rather than the weights” [Abstract, p.1], 그리고 “weights are therefore the wrong system of record: they store content without creating an address for it” [§ 9.3, p.20].

[평가] 두 번째 문장의 “weights” 는 이 책의 어휘로 Θ 만을 가리킨다. 논문에는 W 가 존재하지 않고 (§ 22.4) W -경로 문헌도 한 편 없다 (§ 22.8.5). 그러므로 이 판정을 fast weights 로 옮기는 것은 논문이 뒷받침하지 않는 확장이다. 본서는 이 문장을 인용할때마다 층을 명시한다.

22.3 Core mechanism (통일표기)

기제는 순수 $(U-\Theta)$ 루프이며, 표준형과 세 곳에서 갈라진다. 라운드 k 의 학습 집합은 한 문장짜리 발명 사실 s_k 에서 생성되고, 갱신은 rank- r 부분공간 $\mathcal{S}_r$ 안에서 수렴까지 최적화된 뒤 Θ 에 **merge**된다.

$$\Theta_{k+1} = \Theta_k + \Delta\Theta_k, \quad \Delta\Theta_k = B_k A_k = \arg \min_{\Delta\Theta \in \mathcal{S}_r} \mathcal{L}(\mathcal{R}_k; \Theta_k + \Delta\Theta) \quad (22-1)$$

merge가 루프를 지속적으로 만든다 — “Sequential conditions merge each fact’s adapter into the running model before the next fact trains” [Continual Facts App. A.3, p.25]. $\mathcal{S}_r$ 는 $r \in \{4, 16\}$, $\alpha_{\text{LoRA}} = 32$, dropout 0으로 attention q/k/v/o와 MLP gate/up/down 일곱 투영 행렬 전부에게 걸린다 [App. A.3, p.25]. full fine-tuning 조건에서는 $\mathcal{S}_r$ 가 전파라미터 공간이다 [§ 3.4, p.6].

이 논문의 실제 연구 대상은 optimizer가 아니라 $\text{gen}(\cdot; B_s)$ 다.

$$\mathcal{R}_k = \text{gen}(s_k; B_s), \quad |\mathcal{R}_k| = \begin{cases} 2 & \text{bare-statement} \\ 24 & \text{study} \end{cases} \quad (22-2)$$

bare-statement는 사실 문장 자체를 두 가지 사소한 틀에 넣은 것이고, study는 사실 하나당 24개 항목 — paraphrase, 질문-답 쌍, 함의를 전개한예, 위반된 기본값과의 대조 — 이다 [§ 3.1, p.4; App. A.1, p.25]. **corpus**의 다른 Θ -경로 논문에서 $\mathcal{R}_k$ 는방법이 만들어 내는 부산물이고, 여기서는 독립변수다.

대조군은 (U-E) 의 퇴화한 최선 사례다.

$$\hat{y} = f(q; \Theta_0, W = \emptyset, \text{ret}(E, q) = s_k), \quad \hat{c}_k = c_k, \quad B_s = 0 \quad (22-3)$$

즉 $S(\cdot; B_s)$ 가 항등사상이고 검색이 완벽하다. 이 대조군은 E -경로의 구현이 아니라 상한이다 [본서 추론]. Letta 식 요약도, 검색 오차도 모델링되지 않는다 [§ 2, p.3; Fig. 1, p.2].

계기는 두 개다. 첫째, 쓰기가 만든 객체의 종류를 재는 entailment gap.

$$\text{gap} = \text{acc}_{\text{lenient}} - \text{acc}_{\text{strict}} \quad (22-4)$$

strict 는 요구된 결론만 인정하고 lenient 는 그 결론을 함의하는 사실도 인정한다. 두 정책은 중첩이 아니라 독립 판정이라 작은 음수 gap 이 나올 수있다 [Eq. 2.1, § 2.2, p.4]. 둘째, 내용이 남아 있는지를 재는 저장 탐침.

$$R_{\log p}(k) = \frac{\log p_{\Theta_{j+k}}(s) - \log p_{\Theta_0}(s)}{\log p_{\Theta_j}(s) - \log p_{\Theta_0}(s)} \quad (22-5)$$

$R_{\log p} = 1$ 이면 완전 보존, 0 이면 쓰기 전 사전분포로의 소거다 [Eq. 5.1, § 5.1, p.11]. 세 번째로 역량 손상의 좌표는 고정 프롬프트 풀 P 위의 토큰당 KL 이다 $-\text{drift}(\Theta) = \mathbb{E}_{x \sim P}[\frac{1}{|x|} \sum_t \text{KL}(\pi_\Theta \| \pi_{\Theta_0})]$ [Eq. 6.1, § 6, p.13].

표준형 (U- Θ) 와의 차이는 다섯이다. (1) 한 라운드가 gradient step 1 회가 아니라 수렴까지의 완전한 최적화 (batch size 1 에서 24·96·192 step) 이고 그 뒤 merge 가 붙는다. (2) 갱신이 rank 로 제약되는데 이 제약이 원인은 아니다 (§ 22.6.1). (3) $\mathcal{R}_k$ 의 폭이 조작된다. (4) 근접항 $\lambda_{\text{KL}} \text{KL}(\pi_\Theta \| \pi_{\Theta_k})$ 의 anchor 가 Θ_0 가 아니라 현재의 Θ_k 여서 Θ_0 로부터 잔 KL 은 내려가지 않는다 [§ 6.3, p.15]. (5) 20 라운드마다 Θ 를 Θ_0 로 되돌리고 지금까지의 모든 사실을 batch distillation 으로 다시 쓰는 둘째 시간척도가 있다 [§ 4.3, p.10]. [NL]/[Sleep] 의 어휘로는 $f_1 = 1, f_2 = 1/20$ 의 2-level 시스템이되 위 층이 아래 층 위에 쌓이지않고 리셋한다. 본서는 이것을 **frozen-anchor consolidation** 이라부르고, ch11·ch21 이 소유한 consolidation(느린 층으로의 상향증류) 과 매번 구분해 적는다.

22.3.1 표기대응표

표 22-1 [Continual Facts] 의 원 표기와 본서 표기

원 논문 표기	본서 표기	이유
$\theta, \theta_0, \theta_j$	$\Theta, \Theta_0, \Theta_j$	전부 slow weights 다. 소문자 $\theta^{(\ell)}$ 은 CMS level 파라미터로 예약
$\Delta\theta$	$\Delta\Theta$	동일. W 의 이동과 구분
λ [§ 6.3, Fig. 14]	λ_{KL}	맨 λ 는 outer weight decay, λ_{KD} 는 ch21 이씀
$\alpha = 32$ [App. A.3]	α_{LoRA}	$\alpha_t \in [0, 1]$ 은 retention gate. 정면 충돌
ρ (Spearman 상관)	ρ_S	ρ 는 이 장의주제인 망각률이다. 바꾸지 않으면 비용 표가 읽히지 않는다
$R(k)$ [Eq. 5.1]	$R_{\log p}(k)$	$\mathcal{R}_k$ (라운드학습 집합)· R (보상)과의 근접 오독 방지
D_s [Eq. 3.1]	$\mathcal{R}_k$	문자 그대로 (U- Θ) 의 학습 집합이다
s (사실 문장)	s_k (항상 첨자)	S_t 는 momentum, s 는 SGD step 첨자로 예약

원 논문 표기	본서 표기	이유
k (수행한 쓰기 수)	k (유지)	sleep 라운드 첨자와 일치. 단 이장에서 k 를 key 로 쓰지 않는다
π_T, π_θ, π_0	$\pi_T, \pi_\Theta, \pi_{\Theta_0}$	대문자 Π 는 ch07 이 도입한 지속학습 성능 행렬로 예약되어 있다. 정책은 어떤 경우에도 대문자화하지 않는다
$L_{\text{SFT}}, L_{\text{offline}}$	$\mathcal{L}_{\text{SFT}}, \mathcal{L}_{\text{offline}}$	outer loss 는 calligraphic. 맨 L 은 sequence 길이
g_{use} [Eq. 7.1]	g_{use} (outer 전용선언 후)	이 논문에는 inner loop 가 없다
“consolidation” [§ 4.3]	“frozen-anchor consolidation”	기호가 아니라 용어 충돌. 리셋 후 재작성이지 상향 증류가 아니다

22.4 어느 층을 언제쓰는가

표 22-2 층별 갱신 배정 — 무엇이 언제 움직이는가

대상	누가 학습하는가	언제 움직이는가	규모 · 설정	출처
Θ_0 (Qwen3-4B, bf16, thinking 비활성)	이 논문 밖의 pre-training	LoRA 조건에서는 never(base frozen), full-FT 조건에서는 쓰기 중	4B. 복제 1 회만 8B	[§ 2, p.3; App. A.3, p.25]
adapter A_k, B_k	sleep 라운드의 쓰기 그 자체	사실 k 의 오프라인 쓰기 중. 이후 merge 되고 별도 객체로는 폐기	$r = 16$ (또는 4), $\alpha_{\text{LoRA}} = 32$, 일곱 투영 행렬	[App. A.3, p.25]
AdamW optimizer state	쓰기가 만들고 쓰기가 버린다	쓰기 중에만. wake 로 넘어가지 않는다	$\text{lr } 2 \times 10^{-4}$, 답토큰에만 loss, batch size 1	[App. A.3, p.25]
teacher π_T	아무도 학습하지 않는다	frozen-teacher 조건에서 never. own-merges 조건에서는 현재 merge 된 모델이라 움직인다	adapter 를 끄고 사실을 문맥에 넣은 forward pass	[§ 6.2, p.14]
$\mathcal{R}_k$ (24 항목 study set)	별도 모델이 한 번 생성	전 실행전에 고정. 생성기는 평가 문항을 보지 않는다	사실당 24 항목/bare 는 2	[App. A.3, p.26]
W (fast weights)	아무도 학습하지 않는다. $W = \emptyset$	never. 토큰당 · 청크당 움직이는 상태가 없다	해당 없음	논문 전체 (부재로 확인)
E (문맥 채널)	아무도 학습하지 않는다	wake 시각에, 매 질문마다 프롬프트 토큰으로	사실당 한 문장. 동시 투입 최대 2 개	[Fig. 1, p.2; § 5.2, p.12]

읽는 법은 한 줄이다. 이 논문에는 **wake-time 이동이 전혀 없다**. 서빙은 평범한 frozen forward pass 이고 흥미로운 일은 전부 요청과 요청사이에서 일어난다. 판별식 ($\rightarrow$ ch11) 의 네 조건은 그대로 충족된다 — 질의전에, $B_s > 0$ 을 써서, Θ 를 실제로 바꾸고, 그 변화가 이후질의에 쓰인다. 다만 층 배정이 비대칭이다. (U- Θ) 만처치균이고, (U-E) 는 통제균으로 병렬 실행되며, (U-W) 는 아예 등장하지않는다. 두 층이 순서로 결합하는 자리는 딱 하나, offline context distillation 이다.

$$\mathcal{L}_{\text{offline}}(\Theta) = \mathbb{E}_{x,y \sim \pi_T} \left[\sum_t \text{KL}(\pi_T \| \pi_\Theta) \right], \quad \pi_T(\cdot | x, y_{<t}) = \pi_{\Theta_0}(\cdot | s_k \oplus x, y_{<t}) \quad (22-6)$$

[Eq. 3.2, § 3.2, p.5]. 교사가 정확히 식 (R) 의 E-채널 읽기이고학생이 Θ 쓰기다. 즉 (U-E) 의 산출을 표적으로 삼아 (U- Θ) 를 도는 구성이며, 이것이 논문에서 가중치 기반 composition 의 최고 성적을 낸다 (70% 대 study SFT 60%, 프롬프트 상한 83%) [§ 3.1, p.5; App. B.1, p.27]. on-policy 변형은 $y \sim \pi_\Theta$ 로 표집하고 forward 또는 reverse KL 을 쓴다 [§ 3.2, p.5].

[평가] W 의 부재는 결함이 아니라 구조적침묵이며, 그 자체가 이 장의 판정 범위를 정한다. Θ -경로의 대표반증 논문이 W-경로를 인지하지 않은 채 쓰였으므로, “가중치는 잘못된기록 매체다” 라는 결론은 Θ 에 대한 결론이고 fast weights 로이월되지 않는다. ch17 이 다룬 W-경로의 오프라인 recurrence 는 이논문의 어떤 arm 과도 대응되지 않는다.

22.5 비용 4 종

표 22-3 [Continual Facts] 의 비용 4 종

기호	논문이 보고한 값	판정
B_s	optimizer step 수와 항목 수로만. 사실당 24·96·192 step(batch size 1, AdamW lr 2×10^{-4}) [§ 3.1, p.4; App. A.3, p.25]. batch 조건은 N 개 사실에 $24 \times N$ step, 즉 $N = 100$ 에 2,400 step, $N = 200$ 에 4,800 step [App. A.3, p.25]	FLOPs·tokens·wall-clock 논문에없음
L_w	논문에 없음. 지연·TTFT·처리량·서빙 비용 수치가한 개도 없다. wake 시각의 유일한 수치는 512-token 답변 상한과 그에 대한 truncation 비율 (96 step 에서 bare 6.1% 대 study 0.1%) [App. D.3, p.31] — 이것은 실패율이지 지연이 아니다	—
C_{cap}	논문에 없음. 바이트도 파라미터 수도 없다. 명시된 것은 $r = 16$ (또는 4), $\alpha_{\text{LoRA}} = 32$, 일곱 투영행렬, 그리고 매 adapter 가 merge 된다는 사실뿐 [App. A.3, p.25]	—

기호	논문이 보고한 값	판정
ρ	이 논문에서 가장 풍부한 값이지만 라운드당 비율이 아니라 곡선과 중점 수준으로만 보고된다. $k = 20 \cdot 192$ step 에서 bare 1% 대 study 46%, 짝지은 차 45.6pp(95% CI [38.8, 52.6], $n = 57$) [§ 4.1, p.8]. $k = 100$ 에서 study 는 25 – 28% 고원 [§ 4.3, p.9]. 역량 쪽 ρ : $k = 20$ 에서 SFT $r=4$ –11, SFT $r=16$ –16, CD $r=4$ –6, CD $r=16$ –28pp [§ 6.2· § 6.4, p.14 – 15]	라운드당 감쇠상수 · 적합 곡선 · 반감기 없음

B_s 의 step 수조차 계산 예산이 아니다. 논문이 직접 적는다 — “The steps are matched, but generation and token throughput differ, so this remains an operating-point comparison” [§ 3.2, p.6], “The two data conditions are matched on steps, not tokens or examples” [App. A.3, p.26]. study 쓰기는 step 당 12 배 많은 항목을 통과시킨다.

[평가] 그러므로 **상각식 (A) 를 이 논문에 대해 세울 수 없다**. B_s 가 FLOPs 로 없고 N_q 도 없다. 손익분기 N_q^* 를 만들어 내는 서술은 이 논문이 뒷받침하지 않는다. 그런데 결측의 방향이 한쪽이다 — Θ -경로는 merge 이후 사실당 decode 비용이 0 인데 (§ 22.7) 그 이점이 회계되지 않았고, E -경로 대조군은 매 질문마다 프롬프트 토큰을 내는데 그 비용도 회계되지 않았다. **두 채널을 비교한 논문이 두 채널의 값을 다 매기지 않았다**. ch25 가 이 결측을 통일회계로 받는다.

ρ 를 인용할 때는 어느 ρ 인지 반드시 밝혀야 한다. 저장에서 켜 ρ 와 접근에서 켜 ρ 가 정반대이기 때문이다 (§ 22.6.3). $k = 20$ 에서 strict 문항을 전부 틀린 사실도 자기 쓰기가 만든 log-probability 상승분의 중앙값 69%(bare) / 79%(study) 를 — drift 보정 후에도 57% / 67%를 — 그대로 갖고 있고, “No fact in either condition approaches the erased floor” [§ 5.1, p.11].

22.6 실험과 스케일

계기부터 밝힌다. **이 논문의 벤치마크는 전부 자체 제작이다**. 사실 247 개는 발명된 개체에 대한 한 문장 진술이고 대부분 익숙한 기본값을 위반한다. 학습 데이터는 모델이 생성했고, 문항은 다른 계열의 모델이 작성했으며 (정답을 노출하지 않고, 원 모델이 틀리는 것만 남긴다), 채점은 GPT-5.4-mini(temperature 0, reasoning off) 가 하되 통과는 다른 계열의 둘째 판정기를 통과해야 한다 [§ 2, p.3; § 2.2, p.4]. 문항은 recall·paraphrase·application·composition·counterfactual 다섯 유형이다 [Table 1, p.3]. 사전 인증 게이트 (floor <20%, ceiling >80%, truncation <5%, leak $\leq 3/30$, pass audit $\geq 24/30$) 를 floor 3%, ceiling 84%, leak 1/30, pass-audit 29/30 으로 통과했다 [§ 2.1, p.3]. 단, truncation 기준 (<5%) 은 두 조건에서 통과하지 못했고 (원 모델 floor 23%, bare-statement 24 step 11%) 논문은 게이팅 대신 공개를 택했다 [§ 2.1, p.3; App. A.4, p.26].

22.6.1 한 번의 쓰기는 무엇을 만드는가

96 step, 다섯 유형 평균, strict 정책에서 study 데이터가 10.1pp 이긴다 (95% CI [+7.0, +13.3], 243 개 사실 짝지음). 그런데 lenient 정책에서는 **bare-statement 가 13.6pp 이긴다** [§ 3.1, p.4]. 이부호 반전이 entailment gap 의 정의를 정당화한다 [본서 판단].

유형별로 보면 갈라지는 자리가 분명하다. bare-statement 는 24 step 안에 recall 97%, paraphrase 95%에 도달하고 96 step 에서 포화한다. study 는 96 step 에서 88%와 83%로 각각 11pp, 13pp 뒤진다. counterfactual 은 반대다 —bare 는 24 step 에서 192 step 까지 21 – 23%에 머물고, study 는 45 – 50%에도달해 96 step 에서 29pp 앞선다 [§ 3.1, p.4]. composition 은 study 60%, bare 40%, 프롬프트 상한 83%, context distillation 70%다 [§ 3.1, p.5; App. B.1, p.27]. 96 step 에서 넓은 데이터는 application 을 +21pp, composition 을 +18pp, counterfactual 을 +29pp 올리는데 “extra bare-statement optimisation improves none of them” [§ 3.1, p.5]. entailment gap 은 bare-statement 사용 문항에서 22 – 42pp, study 학습 후 1 – 5pp, 사실을 프롬프트에 넣으면 1 – 6pp 다 [§ 3.1, p.4].

이 gap 이 rank 제약의 산물이 아니라는 것을 논문이 직접 확인한다. 쓰기강도를 맞춘 full fine-tuning($lr\ 3 \times 10^{-5}$, 보류 사실 5 개로선택)에서 bare 28pp, study 4pp, 대비 +24.0pp(95% CI [+20.9, +27.2]) 로 재현된다 — “Narrow supervision, not the number of trainable parameters, creates the gap” [§ 3.4, p.7]. Qwen3-8B 에서도 bare gap 26pp, 헤드라인 대비 +22.9pp(4B 에서 +22.8) 로 재현된다 [§ 3.4, p.6].

여기서 논문이 자기 기제 설명을 스스로 반증한다. 통제된 생성요인분해 (같은 32 개 사실, rank-16, $lr\ 2 \times 10^{-4}$, 192 step, 시드 3 개, 9 개 조건) 에서 bare-statement 는 27.4pp gap(95% CI [20.2, 35.1]) 을 남기고, **파생 결론이 데이터 어디에도 없는** 다양한 recall·paraphrase 프롬프트 24 개만으로 gap 이 5.4pp([1.8, 8.8]) 로 줄어든다. 그리고 study set 이 그 주위에 설계된 바로 그 성분 —함의와 대조를 담은 use-bearing 예시 — 은 단순한 다양 반복보다 **8.7pp 낮은 점수를 받는다** (95% CI [- 15.7, - 1.9]) [§ 7.1, p.17].

“The supported causal variable is therefore prompt breadth, not the presence of an explicit reasoning step.” [Continual Facts § 7.1, p.17]

[평가] 이 한 줄이 § 22.8 의 처방을 바꾼다. 널리 퍼진 직관 —sleep 라운드에서 추론 흔적 (reasoning trace) 을 만들어 학습시키면 지식이 “사용 가능하게” 들어간다 — 이 이 논문의 자기 요인분해 안에서 부정되었다. 작동하는 것은 **프롬프트의 폭**이다. ch20 의 SEAL 이 self-edit 으로만드는 것도, ch21 의 Dreaming 이 만드는 것도 use-bearing 예시이며, 두 논문어느 쪽도 폭과 use-bearing 을 분리한 통제군을 갖고 있지 않다.

22.6.2 쓴 것은살아남는가

20 회 순차 쓰기 (사실당 192 step, 시드 3 개, 매번 merge) 후 study 는 46%, bare-statement 는 1%를 유지한다. 짝지은 차 45.6pp(95% CI [38.8, 52.6], $n = 57$ fact-seed cell) [§ 4.1, p.8]. 쓰기 실패가 아니다 —bare-statement 는 후속 쓰기가 오기 전에는 문항의 65%를 맞힌다. 예산 인공물도 아니다 — 사실당 24 step 으로 낮춰도 (역량 54%) bare 는 6%만유지한다 [§ 4.1, p.8]. $k = 20$ 의 방법별 순위는 bare 1, offline distillation 13, online forward 27, online reverse 31, study 46 이다 [Fig. 5b, p.8].

이 대비를 인용할 때 두 조건이 따라붙는다. 첫째, 헤드라인의 운영점을 논문자신이 “The heaviest bare-statement budget, 192 steps on a single sentence, is deliberately severe” 라고 적는다 [§ 3.1, p.4] — 질적주장은 24 step 조건에서도 서지만 1%라는 수는 온건한 설정을 서술하지않는다. 둘째, **순위표의 증류 계열은 이미 망가진 레짐에서 잔값이다** — 20 번째 쓰기에 이르면 출력의 35 – 46%가 반복 루프에 빠지고역량이 42 – 49%로 내려가며, 논문은 “the on-policy advantage is real in direction but modest in a degraded regime” 이라고 적는다 [§ 4.1, p.8]. 같은 자리에서 offline distillation 의 유지율이 본문 14%와 Fig. 5b 의 13 으로 어긋난다 [§ 4.1, p.8].

한 번의 쓰기가 만든 **종류**가 생존을 예측한다: 45 개 조건 $\times$ 문항유형쌍에서 entailment gap 과 생존율의 Spearman 상관인 $\rho_S = -0.526$ (95% CI [- 0.657, - 0.197]) 이고, 조건별 평균으로 접으면 -0.675 다 [§ 4.1, p.8].

100 회로 늘리면 study 유지율은 0 으로 가지 않고 **25 – 28%의 고원에**앉는다 [§ 4.3, p.9]. 20 회마다 도는 frozen-anchor consolidation 은 $k = 100$ 에서 유지율 25%로 무통합 28%와 잡음 안에서 같고, 첫 통합 직후 5 회 시점에서도 41% 대 38%로 “fails to restore reachability even

immediately” 다. 반면 일반 역량은 매 통합에서 회복하고 $k = 100$ 에서 무통합보다 12pp 높다 [§ 4.3, p.10].

22.6.3 잊힌 사실은 어디에있는가

$k = 20$ 에서 strict 문항을 전부 틀린 사실은 bare 54 개, study 3 개이고 [Fig. 9, p.11], 그 사실들의 $R_{\log p}$ 는 § 22.5 에 적은 대로 소거바닥에 근접하지 않는다. 틀린 답의 내용도 특징적이다 — bare-statement 모델은 실패의 70%에서 질의된 개체에 **가장 최근에 쓴 사실**의 내용을 붙이고, study 모델은 1%다 [§ 5.1, p.11]. 고전적 savings 효과는 역전된다: 잊힌 study 사실 하나를 재 학습시키는 데 step 예산 전부가 들었고, 한 번도 쓰지 않은 통제 사실은 8 step 이 들었다 [§ 5.1, p.12].

합성은 망각보다 먼저 무너진다. 두 사실을 함께 써야만 답할 수 있는 문항에서 둘 다 가중치에 있으면 32%, 둘 다 프롬프트에 있으면 91%다 (짜지은 차 - 58.1pp, 95% CI [- 76.8, - 35.4], 14 쌍) [§ 5.2, p.12]. 개별 사용 가능성 심사는 배치의 87.5%가 통과하므로 개별 실패로는 설명되지 않는다. 병목은 자기 검색이다 — 학습했고 문항에는 답할 수 있는 사실을 그냥 진술하라고 하면 34%만 올바른 내용을 낸다 (8%가 축자적). 논문의 문장은 “There is nothing to search” 다 [§ 5.2, p.12].

반대로 문맥에 문장을 넣어 주면 잊힌 사실이 study 조건에서 77 – 80%, bare 조건에서 14%로 돌아오고, 문맥 사용 능력 자체는 일반 역량보다 빨리 닳지 않는다 (20 회에 걸쳐 83% → 74%, 이중 차분 +9.2pp, 95% CI [+3.1, +15.3]) [§ 5.3, p.13].

22.6.4 쓰기의 대가는 무엇으로 측정되는가

12 개 조건에서 역량 손실은 Θ_0 로부터의 KL 을 따라간다 ($\rho_S = 0.83$) [§ 6.1, p.14]. 요인분해의 400 항목 역량집합에서는 종점 KL 과 $\rho_S = 0.946$ (95% CI [0.787, 0.991]), 누적 local KL 과 0.909 다 [§ 6.1, p.14].

두 처방이 작동한다. 첫째, **frozen teacher**: 순차 증류에서 교사를 Θ_0 로 고정하면 역량 +2pp, KL 0.48, 유지율 54%이고, 교사를 자기 누적 merge 로 두면 - 31pp, KL 1.70, 유지율 21% (상한에 걸린 출력 제외 시 34%) 다 — 짜지은 유지율 차 33.0pp (95% CI [24.6, 41.0]) [§ 6.2, p.14 – 15]. batch 판본은 200 개 사실을 - 3 ~ +1pp 역량으로 쓴다. 둘째, **근접향**: $\lambda_{KL} = 0$ 에서 - 66pp·1%이던 것이 0.5 에서 - 19pp·25%, 1.0 에서 - 5pp·36%가 된다 (쓰기 정확도는 53%, 49%로 떨어진다) [§ 6.3, p.15].

같은 12 개 조건이 논문 자신의 틀에 저항하는 해리를 둘 보인다. rank 4 의 context distillation 은 순차 조건 중 역량을 가장 덜 깎으면서 (- 6pp) 유지율은 같은 rank 의 SFT 보다 나쁘고 (27% 대 42%), rank 를 올리면 유지율이 오르고 역량이 내려간다 — 논문 표현으로 “the reverse of its usual reputation” 이며, 논문이 다른 곳에서 승인하며 인용하는 Biderman 외 2024 와 반대 방향이다 [§ 6.4, p.15; § 8, p.19].

[평가] 그런데 같은 sweep 에서 Θ_0 로부터 잔 KL 은 1.8 – 2.4 사이에 머문다 — “even as capability recovers by more than sixty points” [§ 6.3, p.15]. 논문 자신의 표현대로 KL 은 “a cost coordinate that holds across methods, not a sufficient causal mechanism” 이다 [§ 6.1, p.14]. **가장 깨끗한 정량 법칙이 조종간이 아니라 계기판이라는 뜻이고, 이것이 § 22.7 의 운영 함의를 정한다.**

간섭의 소재도 특정된다. 2×2 교차 설계 (저장 방식 $\times$ 이후 쓰기 방식, 24 개 저장 사실, 시드 3 개, 동일한 10 회 후속 쓰기) 에서 최종 유지율은 9.0 / 47.8 / 7.8 / 44.2%이고, 효과분해는 **이후 쓰기 효과 +37.6pp** (95% CI [28.9, 46.3]), 저장방식 효과 - 2.4pp ([- 9.2, 4.1]), 상호작용 +2.4pp ([- 11.5, 14.1]) 다 — “Interference is caused by the incoming write, not by how the earlier fact was stored” [§ 7.2, p.17 – 18].

통제 시도는 전부 실패한다. 사전 등록한 step-16 조기 예측자는 최종 이득과 순위상관 0.083, 선형화 예측자 (식 7.1) 는 다음 갱신과 $\rho_S = 0.795$ 이지만 전체 수열 후 망각과는 -0.258 이다. 측정된 충돌방향을 잘라내는 protected optimizer 는 +1.4pp (95% CI [- 2.1, 5.0], 목표 15pp) 로

norm 을 맞춘 무작위 사영 (+1.8) 과 구별되지 않는다. bridging 데이터 8 개 조합 중 네 조건 결합을 통과하는 것이 없고, activation 기반개입 (사영 ·patching·block swap) 의 최대 구제폭은 10pp 문턱에 대해 5.9pp 다 [§ 7.3, p.18].

22.6.5 이 결과가 서 있는바닥

둘째 평가의 주 대비는 실패했다. 목적 제작한 prior-conflict 계기 (240 개 사실, 4 개 레시피, 시드 3 개) 에서 strict 정책의 모든 신뢰구간이 0 을 가로지르고 사전 설정한 +10pp 지지 기준에 닿지 않는다. truncation 되지않은 행만으로 다시 계산하면 gap 이 +0.1pp(invert) 와 +0.2pp(neutral) 로 붕괴한다 — 잔여 신호가 truncation 인공물이었다 [App. D.3, p.31]. 다양성 효과는 application 문항만으로 사후 제한한 뒤에야 되살아난다 [App. D.4, p.33].

헤드라인 대비는 문항 양식에 의존한다. 기본값을 진술하고 대조를 묻는 cue 를 붙이면 bare-statement 의 strict 정확도가 23%에서 99%로오르고 entailment gap 이 31.8pp 에서 0.9pp 로 무너진다 [App. D.4, p.32]. [Continual Facts § 3] 을 지탱하는 22 – 42pp gap 은 cue 없는문항에서만 존재하며, 이 논문의 주 평가는 구성상 cue 가 없다.

문맥 채널은 가중치 채널이 도달한 규모에서 시험되지 않았다. 문맥에 동시에 들어간 사실은 최대 2 개인데 ([Continual Facts § 5.2] 의 쌍실험) 가중치 채널은 100 개까지 밀린다 [§ 5.2, p.12]. 게다가 그 대조군자체가 완벽하지 않다 — 프롬프트 상한은 composition 에서 83%이고 tier 별로 77 – 95%이며 [App. B.1, p.27; App. D.4, p.33], cue 유무로 76%와 95% 사이를 오가고 [App. D.4, p.32], bare-statement 쓰기 스트림아래에서는 프롬프트 기준선 자체가 45.5pp 떨어진다 [§ 7.2, p.18].

나머지 바닥은 이렇다. **100 회 확장 — 이 책이 가장 원하는 결과인 25 – 28% 고원 — 이 논문에서 가장 약한 증거다.** 범위가 12 개 셀에서 8 개로축소되었고 (“after partial records existed”), 종점 대비가 사후에선택되었으며, 조건별로 학습률이 조정되었고, 불확실성 구간이 없고, 문맥상한이 77.3%로 인증 기준 80%에 미달한 채 수용되었다 [§ 4.3, p.9; App. B.3, p.27]. 목적함수 한 계열은 아예 평가되지 못했다 — GRPO 조건은 “received almost no signal because the base model never produced the target counterfactuals” [§ 3.3, p.6]. 요인분해 16 셀 중 1 셀 (reverse-KL·bare·LoRA) 은 생성 상한 반복으로검열되어 생존 상관은 15 개 조건의 complete-case 분석이다 [§ 3.3, § 4.1]. 확증 결과는 전부 4B 이고 8B 는 gap 한 측정의 복제 1 회뿐이며 [App. C.4, p.29], 같은 조건의 유지율이 [Continual Facts § 6.2] 에서 11%, [Continual Facts § 6.4] 에서 12%로 인쇄된다 [p.14, p.15].

넷이 더 붙는다. **수치의 비교 가능 범위가 절마다 다르다** — [Continual Facts § 6.4] 의 12 개 조건은 “their own optimisation budget and question set, so retention levels here are not comparable to those of § 4” 이고 [§ 6.4, p.15], 요인분해는 네 쌍이 조건별로 조정된 학습률을 써서 “an association across operating points rather than a one-variable causal effect” 다 [§ 3.3, p.6]. **하중을 지는 주장 여럿이 작은 n 위에 있다** — 역 savings 는 잊힌 study 사실 3 개, protected optimizer 의 null 은 고유 저장 사실 5 개 (60 개 짝 단위로 재표집), 합성붕괴는 14 쌍 47 문항, 사전 tier 역전은 19 개 삼중쌍 (부호검정 $p = 0.064$) 이고, 요인분해의 864 단위는 32 개 사실의 반복측정이다 [§ 5.1, p.12; § 7.3, p.18; App. A.6, p.26; § 5.2, p.12; § 4.2, p.9; App. C.1, p.28]. **인쇄 오류가 더 있다** — App. D.1 이 프롬프트상한이라고 적은 “(neutral 96%, invert 80%, confirm 77%)” 은 실제로는 App. A.5 의 문항 생존율 (95.9 / 79.5 / 76.3) 이고같은 부록의 Fig. 20 은 상한을 97 / 87 / 89 로인쇄하며 [p.30; p.26], “13,728 records (240 facts $\times$ 4 recipes $\times$ 3 seeds)” 는 곱이 2,880 이다 [App. D.3, p.31]. 재현 쪽에서는 의존성 lockfile 을 고정해 두고도 “the exact base-model revision was not recorded in the run rows” 이다 [App. A.3, p.26]. 마지막으로 [Continual Facts § 7.2] 의 간접 대비에는 논문이 배제하지 못한 교란이 남는다 — bare-statement 스트림에서 반복 생성 실패가 4.2%·3.2%(study 스트림 0%) 이고, 논문은 “broad generation pathology remains a possible explanation” 이라고 적는다 [§ 7.2, p.18].

22.7 Systems/serving 합의

decode에서 바뀌는 것은 없다 [본서 추론]. fast weight 도, 토큰당 상태 갱신도, KV cache 를 넘어 자라는 상태도 없다. k 회 쓰기후의 서빙은 Qwen3-4B 모양 Θ 하나 위의 평범한 frozen forward pass 다 [App. A.3, p.25].

[해설] adapter 가 보존되지 않고 merge 되므로 저장된사실당 한계 **decode 비용이 정확히 0** 이다. 사실 100 개를 넣은 모델의 토큰당비용은 0 개를 넣은 모델과 같다. 이것이 이 논문에서 Θ -경로가 E -경로에 대해 갖는 유일한 명백한 우위이고, **논문은 이 사실을 한 번도 적지 않는다**. 반대편에서 문맥 채널은 질문마다 문장 토큰을 prefill 하고 KV 를 잡는다. 그 비용도 논문에 없다. 그래서 “context wins” 라는 결론은 서빙 산술이 뒷받침하는 것보다 강하게 읽힌다.

트래픽은 읽기가 아니라 쓰기 쪽에 있다. 사실 하나당 4B 모델 위의 forward+backward 192 step(batch size 1), adapter 용 AdamW moment, 그리고 merge — merge 는 전 층 일곱 투영 텐서에 걸친 full read-modify-write 다. 즉 100 회 쓰기는 전체 모델 RMW 100 회를 뜻하고, 논문은 이 중 어느 것도 측정하지 않는다. frozen-anchor consolidation 의 교환비는 그 위에서 더 나쁘다 — batch 조건이 N 개 사실에 $24 \times N$ step 이므로 $k = 100$ 의 통합 1 회가 2,400 step, 단일 쓰기의 열 배가 넘고 [App. A.3, p.25], m 회마다 통합하는 정책은 쓰기 k 까지 누적 $O(k^2/m)$ step 을 쓴다 [본서 산술]. **그 비용이 사는 것은 역량이지 도달성이 아니다** [§ 4.3, p.10].

batching 은 온전하다. 모든 사실이 전역 Θ 하나에 merge 되고사용자별·세션별 가중치 상태가 없기 때문이다. Rosetta 사전의 “batch 로 weight 공유 $\leftrightarrow$ Θ -경로에서 깨짐” 행은 **이 논문의설정에서는 발동하지 않는다**. 사용자별 사실로 옮기는 순간 per-user $\Delta\Theta$ 가 생기고 grouped-GEMM 문제가 상속되는데, 이 논문은사용자별 쓰기를 고려하지 않는다 — 이 책이 가장 궁금해하는 배포 형태에대해서는 아무 말도 하지 않는다.

운영 계기 하나는 즉시 쓸 수 있다. drift(식 6.1) 는 고정 프롬프트 풀에대해 고정된 참조 모델과 비교하는 forward pass 이므로 서빙 fleet 에서 싸게돌릴 수 있고, 12 개 조건에서 $\rho_S = 0.83$, 16 개 조건에서 0.946 으로역량 손상을 정렬한다. **단 계기판이지 조종간이 아니다** — [Continual Facts § 6.3] 이 KL 을 움직이지 않고 역량을 60pp 넘게회복시켰다. 카나리아로 쓰고 제어 신호로 쓰지 않는 것이 이 증거가허락하는 사용법이다 [본서 판단].

22.8 한계와 bridge-out

22.8.1 지속 주입의 실패양상

증거를 모으면 실패가 하나가 아니다. 이 장이 소유하는 둘째 정의다.

정의. 지속 주입의 실패 양상은 (U- Θ) 를 반복할때 관측되는 네 가지 서로 다른 붕괴다. (a) **회수-사용 분리** — 쓰기가 축자 회수는 만들되 사용 가능한 지식은 만들지 않는다 (entailment gap 22 – 42pp). (b) **역량 침식** — 누적 갱신이 Θ_0 의 일반능력을 깎으며, 그 크기는 Θ_0 로부터의 KL 로 정렬된다. (c) **주소 상실** — 내용은 Θ 에 남아 있으나 질의가 닿지 못하고, 실패한질의는 최근에 쓴 사실로 흘러간다. (d) **합성 불능** — 개별적으로 사용 가능한 두 사실이 함께 쓰이지 못한다. 네 양상은 서로 다른 처방을요구한다 — (a) 는 $\mathcal{R}_k$ 의 폭이, (b) 는 frozen teacher 또는근접항이 다르고, (c) 와 (d) 에는 이 논문이 시도한 어떤 처방도 듣지 않았다.

이 분해가 중요한 이유는 처방이 섞이지 않기 때문이다. (b) 를 고치는 두처방은 (c) 를 고치지 못한다 — frozen-anchor consolidation 이 역량을 12pp 회복시키면서 유지율을 잡음 범위 안에 두는 것이 그 증거다. 반대로 (a) 를고치는 데이터 폭은 (c) 를 크게 완화한다 (1% $\rightarrow$ 46%). 즉 **네 양상은하나의 “catastrophic forgetting” 이 아니다**.

22.8.2 이송된 불리 사실 — 결함의 유형학

Part I의 ch06·ch07·ch09가 개념을 실었고 논문평가에 속하는 불리 사실을 이 장으로 넘겼다. 개별 나열 대신 유형으로 묶는다. 각 행이 반증 측에 거는 제약이 다르다.

표 22-4 이송된 불리 사실의 유형학

유형	대표 사례와 출처
결과가 표 없이 곡선으로만 존재	EWC 논문에는 결과표가 없고 EWC의 정확도 수치가 본문 어디에도 없다 (두 표 모두 hyperparameter 표) [EWC Table 1, Table 2]. DGR의 § 4.1 – § 4.3 네 실험이 전부 곡선이며, 유일한 수치 표는 단일 과제 예비 실험이다 [DGR Table 1]
인쇄값이 자기 데이터와 어긋남	GEM Table 2가 App. B.3와 불일치 (GEM 0.654 vs 0.6783, iCaRL 0.508 vs 0.5462) [GEM F2]; Table 3의 1-epoch BWT가 App. B.2와 10배 차이 (+0.05 vs +0.0048) [F3]. EWC의 Atari replay buffer 크기가 본문 5×10^5 과 Table 2의 50,000으로 열 배 어긋난다 [EWC App. § 4.2 vs Table 2]. 기억 용량 논문은 같은 설정을 두 부록 표에서 다르게 인쇄하고 ($2.29 \times 10^6/2.97\%$ vs $2.39 \times 10^6/1.11\%$) [U13], 수렴문단에서 같은 4M 데이터셋을 $3.56 - 3.65 \times 10^6$ 과 2.95×10^6 두 값으로 적으며 같은 6.86M 모델의 Table 1 용량 (2.51×10^7 bits)과 약 7배 어긋난다 [U4], 모델 크기 범위를 세 가지로 적으며 [U9], Table 2의 ‘GPT2-Medium’ 행이 실제로는 GPT-2 small이고 [U8], § 4의 그림 참조 두 곳이 틀렸으며 [U17], 부록 데이터셋 크기가 캡션·패널제목·산문에서 서로 다르다 [U14]. BWT 분모 문제는 ch07 이미미 정정했다
교정되지 않은 원고 상태	EWC의 vendored v2는 최종본이 아니다 — Table 2의 캡션은 “MNIST 그림들의 hyperparameter”라고 적혀 있으나 내용은 Atari의 것이고, Table 1의 열 머리는 이 원고에 없는 “Figure 3”을 참조하며, 미해결 LaTeX 참조 Appendix app:atari가 남아 있고, 에이전트 차이 목록의 두 항목이 모두 ‘(e)’로 인쇄되어 있다. § 2.1의 SGD 실패 서술은 과제 A와 B가 뒤바뀐 채 인쇄되어 있어 축자인용하면 오류가 그대로 전파된다 [EWC § 2.1, § 2.2, Table 1, Table 2]

유형	대표 사례와 출처
통제군과 구별되지 않음	rate – distortion Experiment 1: 여섯 KV 압축법 중 어느 것도 random-eviction 대조군을 유의하게 이기지 못한다. 저자가 직접 적는다 — “The gaps are modest at this scale, and the ordering of methods is beside the point; what matters is the comparability the BPT axis delivers” [Rate – Distortion § 14.1]. GEM 이’ 오라클 상한’ 을 3 열 중 2 열에서 넘어선다 (iid 0.83/0.87 vs GEM 0.86/0.88) [F4]. DGR 은 자기상한 대조군 (ER) 을 이긴 적이 없고 비길 뿐이며 [DGR § 4.2], EWC arm 은아예 없다
기제가 자기 실험에 반증됨	EWC: Fisher 의 nullspace 방향 섭동이 역 Fisher 방향만큼 성능을 떨어뜨린다 — “we are over-confident about certain parameters being unimportant” [EWC § 2.2]. [Continual Facts]: use-bearing 예시가 단순 다양 반복보다 8.7pp 낮다 (§ 22.6.1)
저장을 기각한 뒤 저장을 되살림	EWC 의 Atari 시스템은 “separate short-term memory buffers for each inferred task” 를 유지한다 — 도입부가 replay 를 기각한 바로 그 자원이다. GEM 은 $M = 5120$ 으로 MNIST 20,000 의 25.6%, CIFAR 50,000 의 10.24%를 원문 그대로 저장한다 [F5]
조건이 붙었으나 검증되지 않음	EWC 의 task-recognition 보장은 과제 수가 $o(n/\log n)$ 으로 자랄 것을 요구하는데 실험이 이를 확인하지 않는다. Atari 의 Fisher 는 replay buffer 에서 뽑은 100 개 미니배치로 추정되어 stale·off-policy 분포 위에 서 있고, 논문은 이를 언급하지도 표본 수 민감도를 시험하지도 않는다 [EWC § 2.2, Table 2]. GEM 의 보장은국소 선형성과 대표성 있는 기억을 요구하는 선형화이며 결론 자체가 “이전과제의 손실을 높이지 않을 가능성이 크다 ” 인데, 투영된 갭신이실제로 과거 손실을 높인 횟수를 세는 실험이 없다 — Π 를 만든 코드로직접 썰 수 있는데도 그렇다 [GEM F11]. Atari 의 EWC penalty 는 게임당 2 천만 frame 전에는 꺼져 있고 그 게이트를 바꾼 실험이 없다. MNIST 의 λ 값이 본문에 없고, multi-task 형태 (과제별 penalty $O(k \Theta)$ vs 병합 $O(\Theta)$) 가 결정되지않아 논문이 자기 용량 거동을 확정하지 못한다. DGR 의 보장은 “생성기가입력 분포를 복원하는 한” 성립하는데 복원지표 (FID·coverage) 가 없다

유형	대표 사례와 출처
권고와 은유가 실증 없이 남음	DGR 이 권고하는 세 프레임워크 결합은 “there is no straightforward mixture of any two frameworks” 라며 약한결합 하나 (LwF-GR) 만 만들어졌고 그것은 테스트 시각 과제 문맥을 요구한다 [DGR § 5]. 설계의 하중을 지는 생물학적 근거 — “해마는 replay buffer 보다 생성 모델에 가깝다” — 는 인용된 신경과학 세 편에 만 기대며, 논문 자신은 생성적 해마와 buffer 를 구별할 어떤 시험도 하지 않는다 [DGR § 1]
예측이 세워졌으나 확립되지 않음	rate – distortion 예측 (4) 의 super-linearity 가 확립되지 않았다. 압축 이벤트 5 – 25 회에서 irreversible recall 이 0.33 – 0.56 구간이라고 보고할 뿐 적합곡선도 지수도 선형 귀무 대조도 없고, 그림 캡션의 표현은 “loses roughly half its facts at every frequency” 로 가속이 아니라 평평하고 나뭇을 뜻한다 [Rate – Distortion § 14.2]
단일 시드 · 단일 설정 · 좁은 baseline	GEM 의 15 개 20-과제 실행이 전부 단일 실행이고 오차막대가 없으며 [F12], 헤드라인을 만든 γ 의 0 ablation 이 없고 [F10], memory ablation 은 CIFAR-100 에서만이며 [F6], 비용표는 MNIST·CPU 만이고 [F7], CIFAR 는 과제별 최종 선형 분류기를 뒤 테스트 시각 과제 식별자를 요구하며 [F8], FWT 는 방법 품질이 아니라 데이터셋 유사도를 쟁다 [F9]. EWC baseline 은 재구현이고 정규화 강도가 데이터셋 간 세 자릿수 차이이며 [F13], 용어가 두 쪽 안에서 불일치한다 [F14]. EWC 의 MNIST 는 시드 · 분산 미보고에 arm 이 둘뿐이고 Fig. 3C 일 반화 주장이 단일 게임 위에 있다. DGR 은 아키텍처 · 생성기 · ω · 시드가 전부 단일이고 SVHN 실패가 크기 없이 supplement 로 유예된다
강한 결론에 자기 부록의 반례	기억 용량 논문의 membership-inference 법칙은 인쇄된 상수로 계산하면 $ D \rightarrow \infty$ 에서 $F_1 = 1.006$, 최대 1.17 > 1 이 나와 한 문단 뒤의 “0.5 로 수렴” 진술과반대다 [U6]. “예측이 참 F_1 의 1.5 점 이내” 라는 주장이 자기 Table 2 에서 최대 9.31 점 어긋난다 [U7]. 합성 데이터에서 성능이 0.54 아래로 내려가지 않는다는 부록 그림이 그 수렴 주장의반례다 [U20]. 초록의 “grokking” 은 본문의 double descent 이고 인용도틀렸다 [U16]

[평가] 이 표에서 반증 축이 실제로 가져가는 것은 두 줄이다. 첫째, **망각 대응의 두원형 (EWC·DGR·GEM) 은 재현 가능한 수치를 거의 남기지 않았고, 남긴 곳에서는 통제군이 방법과 같리지 않았다.** 그러므로 “이미 해결된 문제” 라는 전제로 (U- Θ) 를 반복하는 설계는 근거가 없다. 둘째, **압축 쪽 이론도 같은 상태다** — 무엇을 버릴지 정하는법이 무작위 폐기를 이긴다는 증거가 그 문헌의 유일한 자체 실험에 없다. Θ -경로가 기대는 두 축 (망각을 막는 법, 버릴 것을 고르는 법) 이 둘다 실증으로 서 있지 않다.

22.8.3 용량·망각·붕괴가 함께 거는 제약

세 축은 따로 인용되어 왔지만 함께 걸린다. 이 장의 셋째 정의다.

정의. 용량·망각·붕괴의 결합제약은 $(U-\Theta)$ 를 k 회 반복하는 설계가 동시에 만족해야 하는 세조건이다. (i) **용량** — 누적 저장량이 $H(\Theta)$ 를 넘을 수 없으며 ($\rightarrow$ ch09), 실측된 도달점은 파라미터당 3.64 bits 이되 ($\rightarrow$ ch09 § 09.2; [기억 용량 Fig. 6 캡션], 반정밀도) 이는 **하한**이라 Θ -경로를 반증하지 못한다. (ii) **망각** — 라운드마다의 상대 손실 ρ 가 0 으로 수렴하지 않으면 유효 저장량은 용량 천장보다 훨씬 앞에서 포화한다 ($\rightarrow$ ch07 의 Π 와 식 (7-11)). (iii) **붕괴** — $\mathcal{R}_k$ 가 자기 생성일 때 누적스케줄에서만 오차가 유계이고 ($\rightarrow$ ch06), 교체 스케줄에서는 발산한다.

세 축이 [Continual Facts] 에서 만나는 지점이 100 회 실험이다. 유지율이 0 으로 가지 않고 25 - 28% 고원에 앉는 것은 (i) 의 천장에 닿아서가 아니다 — 4B 모델에 사실 100 개는 어떤 용량 회계로도 근처에 가지 않는다. 원인은 (iii) 도 아니다 — $\mathcal{R}_k$ 는 별도 모델이 한 번 생성해 고정했고 자기 생성 루프가 없다. 남는 것은 (ii) 뿐이며, 그것도 저장의 (ii) 가 아니라 **접근의 (ii)** 다.

[평가] 그러므로 Θ -경로의 실제 병목은 지금까지 이라인이 인용해 온 용량 상한이 아니다. **가중치는 넣을 자리가 없어서지는 것이 아니라 넣은 것을 부를 이름이 없어서 진다.** 이 문장이 ch09 의 3.64 bpp 논의와 ch07 의 ρ 논의를 재배치한다 — 용량은 아직 구속력 있는 제약이 아니고, 구속하는 것은 주소다. 다만 자기 생성 $\mathcal{R}_k$ 를 쓰는 설계 ($\rightarrow$ ch20 의 SEAL, ch21 의 Dreaming)에서는 (iii) 이 살아 있고, 이 논문은 그 조합을 시험하지 않았다.

22.8.4 논문의 open questions 와 살아남기 위한조건

논문이 남긴 것 중 이 책이 받는 것은 넷이다. **도달성은 SFT 와 offline·online 증류 전반에서 미해결이다** — “The third remains unresolved” [§ 9.3, p.20]. **왜 모든 사실을 다시증류해도 다시 주소가 불지 않는지** 실험이 답하지 않는다 [§ 4.3, p.10]. **fine-tune 된 skill 이 discrete fact 와 같은 방식으로 무너지는지** 시험되지 않았다 [§ 9.2, p.20]. **지식을 쓸 수 있는 RL 목적함수는 GRPO 가 신호를 받지 못한 채 미래 과제로 남았다** [§ 3.3, p.6].

[평가] 이 증거가 허락하는 것은 판정이 아니라 **조건**이다. Θ -경로가 살아남으려면 다음 다섯을 동시에 만족해야 한다.

(C1) **쓰기의 표적은 폭이지 추론이 아니다.** $\mathcal{R}_k$ 는 넓어야 하고, 넓다는 것은 use-bearing 예시를 넣는 것이 아니라 같은 내용에 닿는 표면형을 늘리는 것이다 (– 8.7pp, § 22.6.1). SEAL·Dreaming 식 self-edit 이 이 조건을 만족하는지는 두 논문 어디에도 통제군이 없어 판정되지 않는다. (C2) **교사는 고정되어야 한다.** 자기 누적 merge 를 교사로 쓰는 순차 증류가 12 개 조건 중 최악이다 (– 31pp, KL 1.70, 유지율 21%). 반대로 Θ_0 고정 교사는 20 회 쓰기를 +2pp·KL 0.48·유지율 54%로 통과시킨다. **이송연산자를 자기 자신에 겨누는 설계는 이 증거에 정면으로 걸린다.** (C3) **근접함이 있어야 하되 KL 을 지표로 삼아서는 안 된다.** λ_{KL} 은 역량을 – 66pp 에서 – 5pp 로 되살리면서 Θ_0 KL 을 움직이지 않는다. KL 은 배포 카나리아이고 목적함수의 조종간이 아니다. (C4) **주소를 Θ 밖에 뒤야 한다.** 이 논문이 실증으로 지지하는 유일한 합성은 Θ 를 기록 매체가 아니라 **정본이 다른 곳에 있는 write-through cache** 로 쓰는 것이다 — “Weight updates remain useful as a cache when a canonical copy exists elsewhere, because frozen-teacher batch distillation preserves general capability while consolidating many facts” [§ 9.2, p.20]. 이 구성에서 E -경로와 Θ -경로는 경쟁자가 아니라 보완재다. (C5) **회계를 세워야 한다.** 이 논문은 두 채널을 비교하면서 FLOPs·지연·바이트를 하나도 재지 않았고, 문맥채널을 $|E| \leq 2$ 에서만 돌린 뒤 100 개 사실 규모로 결론을 옮겼다. $|E| = 100$ 에서 **문맥 채널이 내는 prefill 과 KV 비용은 merge 된 Θ 가 내지 않는 비용이다.** 이 결측이 메워지기 전에는 “context wins” 가 서빙 결론이 아니다.

최종 판정은 하지 않는다. 경로별 판정은 ch27 이, 반증 조건의 정식화는 ch30 이 소유한다. 이 장이 넘기는 것은 위 다섯 조건과, 그 조건이 어떤관측으로 깨지는지다.

22.8.5 경로 간 인용여부

비대칭이 뚜렷하다. E -경로는 사상으로 깊이 교전하되 이름은하나도 인용하지 않는다. Snell 외 2022(context distillation), Cartridges(2506.06266), gist token, Memory Layers(2412.09764), Ovadia 외 2024 가 인용되고, MemGPT·Letta STC·Mem0·Zep·ReasoningBank 은 전부 없다. 그러면서 E -경로의 논지에 독립적으로 도달한다 — “Retrieval-augmented systems preserve this handle outside the weights. The write stores content, while the context supplies its address at use time” [§ 9.1, p.20]. 수렴이 독립적이므로 증거로는 더 강하고, E -경로의 방법에 대해서는 아무 증거도 주지 않는다 (검색기가오라클이고 $B_s = 0$ 이므로).

W -경로는 완전한 침묵이다. 참고문헌에 Titans·Miras·Atlas·TNT·Nested Learning·Language Models Need Sleep·TTT-fast-weight programming·Mamba·DeltaNet·RWKV 가한 편도 없고, “fast weights” · “test-time training” 이라는말이 본문에 나오지 않는다.

Θ -경로는 옛 계보만 인용한다. LoRA·ROME·MEMIT·EWC·Biderman 외 2024·How much do LMs memorize(2505.24832) 는있고, SEAL·Nested Learning·LM Need Sleep·SCM·Generative Adapter·rate - distortion 은 전부 없다. [Continual Facts § 4.3] 에서 주기적 오프라인 통합을 직접 구현하면서 CLS 도, McClelland 도, 생물 sleep 문헌도 인용하지 않는다. 인용된 심리학은 Tulving & Pearlstone 1966 과 McGeoch 1932 이고, 논문은 “We use this vocabulary descriptively, without claiming a shared mechanism” 이라고 못박는다 [§ 8, p.19].

[평가] 이 침묵 지도가 ch11 의 발견을 반복한다. 그런데 방향이반대라 더 무겁다 — 앞의 장들에서는 기제 논문이 서로를 안 봤고, 여기서는반증 논문이 반증 대상을 안 봤다. 그러므로 이 논문의 결론은 Θ -경로 논문들이 제안한 절차에 대한 판정이 아니라, 그 절차들이공유하는 연산자에 대한 판정이다. 그 구분을 ch27 이 유지해야한다.

다음 장은 방향을 다시 돌린다. 이 논문이 오프라인 통합을 구현하면서생물학 문헌을 한 편도 보지 않았다는 사실은, 이 라인 전체가 은유는 빌려쓰고 원본은 읽지 않았다는 ch08 의 관찰과 같다. ch23 은 은유의 원본으로돌아가 생물학이 실제로 무엇을 보였는지를 확인한다 — sleep 이망각을 막는다는 실증이 어떤 baseline 위에서 성립하고, dreaming 의표현학습 역할이 어느 통제 대비 유의했는지.

요약

- [Continual Facts] 는 sleep-time compute 문헌의 어떤 open question 도상속하지 않는다. 명시된 상속은 Shenfeld 외 2025(KL 이 망각을 예측) 와 Hase 외 2023(국소화가 편집 지점을 지정하지 않는다) 둘뿐이고, 공백진술은 스스로 만들었다. 가장 강한 반증 후보가 반증 대상을 읽지않고 쓰였다.
- 기제는 순수 ($U-\Theta$) 루프이며 표준형과 다섯 곳에서 갈라진다 —라운드마다 수렴까지 최적화 후 merge, rank 제약, $\text{gen}(\cdot; B_s)$ 의 조작, 현재 Θ_k 를 anchor 로 하는근접항, 20 라운드마다 Θ_0 로 리셋하는 둘째 시간적도. 그리고 $W = \emptyset$ 이다 — 따라서 “weights are the wrong system of record” 는 Θ 에 대한 판정이며 fast weights 로 이월되지 않는다.
- 비용 4 종에서 L_w 와 C_{cap} 이 **논문에 없음**이고, B_s 는 optimizer step 수로만 있으며 논문 스스로 “matched on steps, not tokens or examples” 라고 적는다. 상각식 (A) 를 세울 수 없다.
- 데이터의 폭이 생성과 생존을 모두 정한다 (96 step strict 에서 +10.1pp; $k = 20$ 에서 46% 대 1%). 그러나 study set 이 설계된 근거였던 use-bearing 예시는 단순 다양 반복보다 **8.7pp 낮다** (95% CI [- 15.7, - 1.9]) — 인과 변수는 프롬프트의 폭이지 명시적 추론 단계가아니다.

- 망각은 소거가 아니라 접근 상실이다. 모든 strict 문항을 틀린 사실도 log-probability 상승분의 중앙값 69% / 79%를 유지하며, bare 조건 실패의 70%가 가장 최근에 쓴 사실의 내용으로 흘러간다. 두사실의 합성은 가중치 32% 대 프롬프트 91%로 무너진다.
- 간섭의 소재는 저장 방식이 아니라 **들어오는 쓰기**다 (이후 쓰기효과 +37.6pp 대 저장 방식 - 2.4pp). Θ_0 고정 교사와 근접하이역량을 되살리지만 도달성은 어느 처방으로도 회복되지 않았다.
- 이송된 불리 사실을 유형으로 묶으면 두 줄이 남는다 — 망각 대응의원형들이 재현 가능한 수치를 거의 남기지 않았고, 압축 이론의 유일한 자체 실험에서 어느 방법도 random-eviction을 유의하게 이기지 못했다. Θ -경로가 기대는 두 축이 둘 다 실증으로 서 있지 않다.
- 실증 상한은 확증 결과 전부 Qwen3-4B 이고 8B는 한 측정의 복제 1 회다. 문맥 채널은 $|E| \leq 2$ 에서만 돌았고 가중치 채널은 100 개까지 밀렸으므로, “context wins”는 실행된 적 없는 레짐으로의 외삽이다.

자가 점검체크리스트

- Θ -경로의 반증 조건셋 (생성 · 역량 · 도달) 을 진술하고, 각각에 대해이 논문이 내린 판정과 그 근거 수치를 딸 수 있다.
- 지속 주입의 실패 양상 네 가지를 구분하고, 어느 처방이 어느 양상만 고치는지를 frozen-anchor consolidation의 결과 (역량 +12pp, 유지율 25% 대 28%)로 설명할 수 있다.
- use-bearing 예시가 - 8.7pp 였다는 사실이 SEAL과 Dreaming의 설계 전제에 무엇을 요구하는지 말할 수 있다.
- 이 논문의 비용 4종 중 어느 칸이 왜 비어 있는지, 그리고 그 결측이 $\Theta \cdot E$ 어느 쪽에 유리하게 작동하는지 양방향으로 설명할 수 있다.
- 용량 · 망각 · 붕괴 결합 제약에서 100 회 실험의 고원 (25 - 28%) 을 만든 것이 어느 축인지 나머지 둘을 배제하는 논증으로 지목할 수 있다.
- 이송 표의 열한 가지 결합 유형 중 최소 넷을 대표 사례와 함께 재현하고, 그중 어느 둘이 Θ -경로의 전제를 직접 무너뜨리는지 말할 수 있다.
- merge 된 Θ 의 사실당 한계 decode 비용이 0 이라는 사실을 Rosetta 사전의 “모델 재배포 $\leftrightarrow$ (U- Θ) 1 회” · “batch 로 weight 공유 $\leftrightarrow$ Θ -경로에서 깨짐” 두 행으로 옮기고, 후자가 이 논문의 설정에서 **발동하지 않는** 이유를 설명할 수 있다.

ch23. 생물학이 실제로 보인것

23.1 Bridge-in: 전작이 남긴문제

이 장은 Part II에서 유일하게 뒤를 향해 열린다. ch12부터 ch22까지 열한 장이 세 경로를 훑는 동안 생물학은 계속 배경에 있었다. ch08이 CLS와 생물학적 sleep을 배경으로 세웠고, 그 위에서 ch17·ch21은 wake/sleep lifecycle을 자기 기제의이름으로 썼으며, ch14는 같은 단어를 전혀 다른 뜻으로 썼다. 이제 그 배경문헌을 논문장의 엄밀함으로 다시 읽는다. 물음은 하나다 — 이 두편은 정확히 무엇의 증거인가.

전작의 open question을 복원하는 이 절의 규약(→ ch11)을 두 논문에적용하면 결과가 갈린다.

PLOS 논문 (Golden·Delanois·Sanda·Bazhenov, Sleep prevents catastrophic forgetting in spiking neural networks by forming a joint synaptic weight representation, PLoS Comput Biol 18(11):e1010628, 2022)은 상속을 명시한다. 문장은 이것이다.

“While proposed approaches are capable of mitigating catastrophic forgetting in certain circumstances, a general solution which can achieve human level performance for continual learning is still an open question [9].” [PLOS Sleep Introduction, L103-105]

그리고 자기 작업 질문을 한 문장으로 적는다: “Can we implement a sleep like phase to our model to protect an old task and still accomplish new task learning without explicit re-training of the old task?” [PLOS Sleep Results, L290-292].

그런데 상속의 성격을 밝혀야 한다. 인용된 [9]는 Hayes·Krishnan·Bazhenov·Siegelmann·Sejnowski·Kanan의 replay 리뷰 (Neural Computation 2021)이고, 이 논문의교신저자가 그 리뷰의 공저자다 [PLOS Sleep References, L1257-1260]. 이차 상속으로 지목된 Gonzalez et al. 2020 (eLife)도 같은 그룹의논문이다 [ref 41, L1438-1441]. 공백을 선언한 문헌과 그 공백을 메우는문헌이 저자를 공유한다.

PAD 논문 (Deperrois·Petrovici·Senn·Jordan, Learning cortical representations through perturbed and adversarial dreaming, eLife 2022;0:e76384)에는 상속 진술이 없다. 선행 저자가 적은 open question을 한 문장도 인용하지않으며, 계보 진술은 전부 저자 자신이 진단한 선행 문헌의 한계다 — Wake-Sleep 계열은 “do not leverage offline states to improve their generative model but are explicitly trained to reproduce sensory inputs during wakefulness” [PAD Introduction, L82-86], trace transformation 이론은 “lack a mechanistic implementation compatible with cortical structures” [L87-89], 기존 dreaming 이론들은 표현 형성이라는 더기본적인 기능을 고려하지 않는다 [L104-105]. 그러므로 PAD에 대해서는 bridge-in을 공동체의 공백에서 연다: CLS 전통은 sleep이 일어난일을 다시 돈다고 말하고, dream 현상학은 REM의 꿈이 일어난 일이 아니라고말한다. PAD는 두 번째 사실을 잡음이 아니라 계산의 원료로 삼은 첫 기능모델이다.

[평가] ch21에서 상속이 교과서적으로 성립한 이유를 이 책은저자군의 동일성으로 설명했다. 생물학 쪽도 같다 — 두 편 중 하나는 자기그룹의 리뷰에서 문제를 받았고 다른 하나는 아무에게서도 받지 않았다. corpus에서 상속 진술이 성립하는 자리는 대체로 경로가 이어진자리가 아니라 연구 그룹이 이어진 자리다. 흠이 아니라 전파 방식에 대한 사실이며, § 23.8의 반대 방향 침묵과 짝을 이룬다.

23.2 문제의식

두 논문이 스스로 세운 문제는 다르다. 그리고 둘 다 이 책의 corpus 가나중에 쓰게 될 문제와도 다르다.

PLOS의 문제는 저장이다. 순차 학습이 옛 과제의 가중치를 덮어쓴다는 진단에서 출발하되, 고전적 처방인 interleaved training 이 “imposes the stringent constraint that the original training data be perpetually stored for later use and combined with new data to retrain the network” 라는 점을 문제의 본체로 잡는다 [PLOS Sleep Introduction, L109-111]. 그래서 논문이 한 문장으로 적은 과제는 정확도가 아니다: “Thus, the challenge is to understand how the biological brain enables memory reactivation during sleep **without access to past training data**” [L111-113, 강조는 이 책]. 이 문장이 이 논문의 모든 결과의 해석 범위를 정한다.

PAD의 문제는 표현이다. 라벨 없는 감각 경험에서 어떻게 선형 분리가 가능하고 섭동에 강건한 표현이 생기는가, 그리고 두 sleep 상태가 각각무엇을 기여하는가 [PAD Abstract, L11-14]. 가설은 dream 이 재생이 아니라 생성이라는 데 있다: “generating new but realistic sensory experiences, instead of merely reconstructing previous observations, requires the brain to understand the composition of its sensorium” [PAD Introduction, L113-115].

여기서 이 장이 소유하는 첫 정의를 못 박는다.

정의. 생물학 실증의 정확한 범위란, 한생물학·계산신경과학 논문이 실제로 뒷받침하는 진술들의 집합을 네 좌표로 담아 적은 것이다 — (**기제 클래스, 규모, 측정지표, 반복라운드 수**). 네 좌표 중 하나라도 밖으로 나간 진술은 그 논문이지지하지 않는다. 예를 들어 “sleep 이 망각을 막는다” 는 범위 밖 진술이고, “국소 spike-timing 가소성과 열합 보존 정규화를 갖춘 3층 spiking 망에서, 6272 개 가소성 시냅스와 두 개의 상보 과제에 대해, pattern discriminability 지표로, 한 번의 interleaved 위상 동안, 옛 과제 성능이 0.70 ± 0.02 에서 0.70 ± 0.03 으로 유지된다” 는 범위 안 진술이다.

이 정의는 사후 트집이 아니라 **인용 가능성의 계약**이다. § 23.6 이 두논문의 네 좌표를 채우고, § 23.8 이 그 좌표 밖으로 나간 인용을 목록으로 잘라낸다.

[해설] 계약이 필요한 이유는 corpus 의 인용 패턴에 있다. LLM 쪽 논문이 생물학을 인용할 때 옮겨지는 것은 거의 언제나 **결론문장**이지 좌표가 아니다. “sleep 은 catastrophic forgetting 을 막는다” 는 한줄은 네 좌표를 전부 지운 상태로 이송된다. 지워진 좌표를 복원하는 것이 이장의 실용적 산출물이다.

23.3 Core mechanism (통일표기)

두 논문의 기제는 ch08 § 08.3 – § 08.4 가 서술했다. 이 절은 그것을 반복하지 않고 **표준형과의 차이**만 적는다.

23.3.1 PLOS — (U-Θ) 를 두 시계로 두번

PLOS 논문은 **수식에 번호를 붙이지 않는다**. 따라서 원문 인용은 Methods 의 절 이름과 vendored 원문 줄번호로만 한다. 이 책은 이 논문에 원문 수식 번호를 발명해 붙이지 않는다.

읽기는 식 (R) 의 극단적 붕괴다.

$$\hat{y}_t = f(q_t; \Theta, \emptyset, \emptyset) = \arg \max_o \#sp_o \left(q_t; \Theta^{I \rightarrow H}, \Theta_k^{H \rightarrow O}, \Theta_k^{H \rightarrow O, -} \right) \quad (23-1)$$

q_t 는 movement cycle t 의 7×7 이진 시야이고 $\#sp_o$ 는 출력 뉴런 o 의 전반부 스파이크 수다 [PLOS Sleep Methods: Policy, L784-789]. $W = \emptyset$ 이고 $E = \emptyset$ 이다 — movement cycle 사이에 넘어가는 상태는 가중치와 항상성 스칼라뿐이다. 이논문에는 fast weight 도 external store 도 없다.

쓰기는 (U- Θ) 하나뿐인데, **wake 와 sleep 이 같은 가중치블록에 같은 국소 규칙을 쓴다.** 갱신 대상은 은닉 $\rightarrow$ 출력 흥분성 시냅스 $\Theta^{H \rightarrow O}$ 6272 개다 [L995, L1035]. 표준형과의 차이는 네가지다.

첫째, **gradient가 없다.** $\nabla_{\Theta} \mathcal{L}$ 자리에 국소 eligibility trace 를 낀 세 인자 Hebbian 곱이 들어간다 [Methods: Synaptic plasticity, L879-901]. loss 함수도, 도함수도, optimizer 도, backward pass 도 논문 전체에 없다. η_{Θ} 는 독립된 존재조차 아니며 보상 R , trace 최대치 $K = \pm 0.04$, 항상성 보폭 Δ_{tar} 에 흡수되어 있다.

둘째, **갱신이 사영된다.** 매 STDP 사건 후 heterosynaptic 정규화가 각 출력 뉴런의 입력 합을 고정값으로 되돌린다 [L946-963]. 제약 집합은

$$\sum_i \Theta_{ij}^{H \rightarrow O} = \Theta_{.j}^0 \quad (\text{모든 } j) \quad (23-2)$$

이고, 이 사영을 이 책은 $\text{nrm}(\cdot)$ 로 쓴다 (장-국소 기호). **이것이 논문에서 가장 하중을 받는 조각이다.** 열 합이 고정되므로 모든 강화는 같은 열 안의 다른 시냅스의 약화로 지불된다 — 갱신이제로 섬이다. sleep 이 단순 동결보다 나은 이유에 대한 논문 자신의 설명이여기에 기댄다: “the sleep phase results in a large cluster of weights being renormalized around an intermediate value of synaptic strength in the network” [L511-512].

셋째, sleep 라운드의 $\mathcal{R}_k$ 가 데이터셋이 아니다. 감각수용기를 끄고, 환경에서 입자를 모두 없애고, 출력층을 운동 제어에서분리하고, 은닉층을 Poisson 잡음으로 구동한 채 망 자신을 돌린다 [Methods: Simulated sleep, L966-985]. 즉 $\text{gen}(\mathcal{H}_k; B_s)$ 가 문자 그대로 “ B_s movement cycle 동안 잡음 위에서 망을 전진시켜라” 이고, $\mathcal{H}_k$ 는 Θ_k 안 말고는 어디에도 저장되지 않는다.

$$\Theta_{k+1}^{H \rightarrow O} = \text{nrm} \left[\text{hebb}(\Theta_k^{H \rightarrow O}; R \equiv 0.5, q \equiv \emptyset, H \sim \text{Poisson}(\bar{f})) \right] \quad (23-3)$$

넷째, **wake 도 (U- Θ) 다.** 배포 가중치가 각성 중에도 계속움직인다. 이 구조적 사실이 § 23.7 의 serving 논의의 지배한다.

논문 안의 어휘 불일치도 그대로 옮긴다. Results 는 sleep 의 규칙을 “the rewarded STDP rule was replaced by unsupervised STDP” 라고 쓰고 [L294-295], Methods 는 “During periods of sleep the network received a constant reward of $Srp = 0.5$ on each movement cycle” 이라고 쓴다 [L936-937]. 앞 문장만 읽은 독자는 sleep 중 모든 trace 에 양의스칼라가 곱해진다는 것을 예상하지 못한다.

23.3.2 PAD — (U-E) 한 번 뒤 (U- Θ) 세번

PAD 는 한 사이클 k 가 mini-batch 하나이고 ($b = 64$) [PAD Methods, L1283], 순서는 wake $\rightarrow$ NREM $\rightarrow$ REM 이며 위상마다 outer loss 가 다르다. wake 의 쓰기는 (U-E) 의 가장 값싼 사례다.

$$E_k = \{\hat{c}_k, \hat{c}_{k-1}\}, \quad \hat{c}_k = S(c_k; B_s) = \text{enc}(x_k; \Theta^{\text{enc}}) \quad (23-4)$$

$S(\cdot; B_s)$ 가 encoder forward 한 번이고 wr 은 2 슬롯 shift register 이며 $\text{ret}(E, q)$ 가 존재하지 않는다 — 읽기는 질의가 아니라 라운드 첨자로만 이루어진다 [PAD Algorithm 1, L1300, L1351, L1423].

세 번의 (U- Θ) 는 위상별로 $\mathcal{L}$ 과 $\mathcal{R}_k$ 가갈린다. NREM 은 저장된 latent 를 목표로, generator 출력에 가림을적용한 것을 **입력**으로 삼는다: $\mathcal{R}_k = \{(\text{occ}(\text{gen}_{\text{pix}}(\hat{c})), \hat{c})\}$ [PAD Eq. 6]. store 가 입력을 주고 동결 teacher 가 목표를 주는 일반적 replay 도식과 **입출력이 뒤집혀 있다** ($\rightarrow$ ch07). REM 은 두 슬롯과잡음을 섞은 latent 를 렌더한다.

$$z_k^{\text{mix}} = \frac{1}{4} \hat{c}_k + \frac{1}{4} \hat{c}_{k-1} + \frac{1}{2} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (23-5)$$

계수는 $\lambda = \lambda' = 0.5$ 에서 나온다 [PAD Methods, L1556-1563; Figure 2c, L213-224]. **혼합의 절반이 잠음이다.**

표준형이 표현하지 못하는 것은 REM 의 부호다.

$$\Theta_{k+1}^{\text{disc}} = \Theta_k^{\text{disc}} - \eta_{\Theta} \nabla \mathcal{L}_{\text{rem}}, \quad \Theta_{k+1}^{\text{gen}} = \Theta_k^{\text{gen}} + \eta_{\Theta} \nabla \mathcal{L}_{\text{rem}} \quad (23-6)$$

같은 loss 에 대해 한 파라미터 블록만 상승한다 [PAD Algorithm 1, L1409, L1421]. 논문은 이 부호 전환을 자기 생물학적 예측으로 세운다: “Sign switch indicates that identical local errors lead to opposing weight changes between Wake and REM sleep” [PAD Figure 7 caption, L814-815]. 표준형 (U- Θ) 는 부호 필드 없이는 이 형태를 담지못한다.

W 층은 없다. 평가 시점에 움직이는 것이 아무것도 없다 $-\hat{y} = W_{\text{ro}} \text{enc}(x; \Theta^{\text{enc}})$ 한번의 forward 가 전부이고, 분류기를 학습할 때 “the connectivity of the network (E and G) is fixed” [PAD Results, L531-532].

23.3.3 표기대응표

표 23-1. PLOS Sleep 의 원 논문 기호 $\rightarrow$ 이 책 표기. 이 논문은 수식 번호가없으므로 출처는 Methods 절 이름으로 준다.

원 논문 기호	이 책 표기	주의
$W_{ij}, W_i, W_j, W_{i0}, W_{j0}, W_{Iij}$ (시냅스 가중치)	$\Theta^{I \rightarrow H}, \Theta^{H \rightarrow O}, \Theta^{H \rightarrow O, -}$	W 는 fast weights 예약. 이 논문에 W 층도 E 층도 없다
S_{rp} (보상 · 처벌 배율)	R	본서 표기 규약상 첨자없는 맨 S 는 금지이고, r 은 LoRA rank 이므로 보상은 R 이다 — 장-국소 정정
k (trace 합의 첨자)	m	k 는 sleep 라운드 전용
$R = 0.12$ (시냅스 방출 변동 계수)	r_{rel}	본서 수식에 등장하지 않음
H (은닉층)	산문의 “layer H” / $\mathcal{H}_k$ (누적 경험)	두 대상이 개념상 거의 같아 층 돌이 위험하다. 한 수식에 함께 두지않는다
$M_{T1}, M_{T2}, M_{T1 \cap T2}$ (해 manifold)	Man_{T1} 등	필기체 $\mathcal{M}$ 은 memory 읽기 함수 전용
$\alpha = 3.65$ (뉴런 비선형성)	α_{neu}	α_t 는 retention gate
β_n, σ_n (외부 전류 변수)	β_{neu}	β_t 는 momentum decay
$c = 1$ movement cycle (trace 감쇠 상수)	t_{off}	C 는 chunk, C_{cap} 은 용량
n (0.5 ms 시간 첨자 및 가중치 공간 차원 6272)	시간 첨자 s / 차원 $d_{\Theta} = 6272$	논문 자체가 n 을 두뜻으로 쓴다
epoch(= 600 timestep $\approx$ 300 ms), aeon(= 100 epoch)	“movement cycle” , “aeon(= 100 movement cycle)”	ML 의 epoch 와 무관. 수식어 없는 “epoch” 사용 금지
I (입력층 / 느린 뉴런 변수 / 시 냅스 전류)	산문의 “layer I” / u_s	논문 내부 삼중 사용
$T_c, K, \gamma, \mu, \delta, \Delta_{\text{tar}}$	그대로 인용만	본서 수식에 등장하지 않음. $\delta, \Delta_{\text{tar}}$ 는 vendored 원문에 수 치가 없다
(노트 내부 기호) 정규화 사영	$\text{nrm}(\cdot)$	Π 는 지속학습 성능 행렬 전용 — 장-국소 정정

표 23-2. PAD 의 원 논문 기호 $\rightarrow$ 이 책 표기.

원 논문 기호	이 책 표기	주의
E, E_z, E_d (encoder·latent·discriminator 사상)	$\Theta^{\text{enc}}, \Theta^{\text{disc}}, \text{enc}(\cdot), \text{disc}(\cdot)$	corpus 최대위험 충돌. 논문의 E 는 slow-weight 망이고 이 책의 E 는 external store 다. 예약 E 는 논문의 hippocampal 버퍼에 준다 이망이 문자 그대로 (U- Θ) 의 $\text{gen}(\cdot)$ 이다 fast weight 가 아니다 . 동결 후 학습되는 평가 probe 장-국소 선언
G (generator·feedback 경로) $W \in \mathbb{R}^{10 \times 256}$ (선형 readout)	$\Theta^{\text{gen}}, \text{gen}_{\text{pix}}(\cdot)$ W_{ro}	만 L 은 sequence 길이. 이 논문에는 inner loss ℓ 이 없다 모두 Θ_k 의 부분 블록 η_t 는 없다. 세 위상이 같은 값을 쓴다 평가 probe 소유 시간 첨자를 붙이지 않는다 retention gate 가 아니다. 상태 감쇠가 아니라 기록 두 개의 혼합비 $C(\text{chunk})$ 로 쓰지 않는다 d 는 모델폭
$z, z_{\text{old}}, Z, Z_{\text{old}}, Z_{\text{mix}}, z'$ $L_{\text{img}}, L_{\text{KL}}, L_{\text{real}}, L_{\text{Wake}}, L_{\text{NREM}}, L_{\text{REM}}, L_{\text{GAN}}, L_{\text{C}}$	z 유지; 저장된 것은 $\hat{c}_k, \hat{c}_{k-1}$ $L_{\text{img}}, L_{\text{KL}}, L_{\text{real}}, L_{\text{Wake}}, L_{\text{NREM}}, L_{\text{REM}}, L_{\text{GAN}}, L_{\text{C}}$	
θ_E, θ_G ADAM lr = 0.0002	$\Theta^{\text{enc}}, \Theta^{\text{disc}}, \Theta^{\text{gen}}$ $\eta_{\Theta} = 0.0002$	
$\eta = 0.2$ (readout 학습률) $\beta_1 = 0.5, \beta_2 = 0.999$ $\lambda = \lambda' = 0.5$ (혼합 계수)	η_{ro} $\beta_1^{\text{Adam}}, \beta_2^{\text{Adam}}$ 그대로, 장-국소 상수	
$b = 64$ (mini-batch) d (discriminator 스칼라 출력) $\epsilon \sim \mathcal{N}(0, I)$; 절제용 $\epsilon \sim \mathcal{N}(0, 0.5I)$ ω, Ω (가림 마스크)	b $\text{disc}(x)$ ϵ , 장-국소 $\text{occ}(\cdot)$	두 ϵ 은 분산도 역할도 다르다 ω 는 replay 혼합비 예약 (ch07) — 장-국소 정정 n 은 저장 항목 수
$n_z = 256$ (latent 차원) epoch(= 데이터셋 1 회 통과)	n_z $k = \text{wake-sleep 사이클 (= mini-batch 1 개)}$	한 epoch 은 많은 k 다. 혼동하면 sleep 예산이 세 자릿수만큼 작아 보인다

23.4 어느 층을 언제쓰는가

표 23-3. PLOS Sleep에서 움직이는 모든 것 [Methods: Network structure L742-758; Synaptic plasticity L861-963; Simulated sleep L965-986].

대상	기호	크기	누가 움직이나	언제
입력→은닉 흥분성 시냅스	$\Theta^{I \rightarrow H}$	$784 \times 9 = 7056$	비지도 STDP	초기 비지도 기간에만. 강화학습 기간 내내 동결 [L205-207]

대상	기호	크기	누가 움직이나	언제
은닉→출력 흥분성 시냅스	$\Theta^{H \rightarrow O}$	$784 \times 8 = 6272$	wake 는보상 STDP, sleep 은 $R \equiv 0.5$ 의 같은 규칙, 양쪽 다 매 사건 후 nrm	wake 와 sleep 둘 다. 명목 속도도 같다
은닉→출력 억제성 시냅스	$\Theta^{H \rightarrow O, -}$	6272, 자유도 0	아무도. 원천 뉴런의 흥분성 출력 평균의 음수로 재 계산	매 STDP 사건 후, wake·sleep 동일
출력 뉴런별 목표 입력 합	$\Theta_{\cdot j}^0$	스칼라 8 개	발화율기반 항 상성 scaling	매 movement cycle
은닉 뉴런별 목표 출력 합	Θ_i^0	스칼라 784 개	아무도 — 시작값으로 고정	없음
trace 합의 이동 평균	Avg _{tr}	스칼라	지수 평활	trace 평가 시마다
sleep 구동용 목표 발화율	$\bar{f}_i$	784 개 (통제에서 1 개)	학습되지 않음 — wake 에서 측정	직전 wake 에서 계산, sleep 중 고정
모델 하이퍼파라미터	—	약 15 개	손으로 설정. outer loop 없음, sweep 보고 없음	없음
SVM(RBF)·PCA/kPCA	—	입력 6272 차원	저자가 사후 분석 도구 로 적합	종료 후. 망은 한번도 보지 않는다

표 23-4. PAD 에서 움직이는 모든 것 [Algorithm 1, L1287-1423; Methods L1247-1663].

대상	기호	누가 움직이나	언제
encoder(conv 4 층)	Θ^{enc}	outer-loop gradient descent, ADAM	세 위상 전부. wake 는 $\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{KL}} + \mathcal{L}_{\text{real}}$, NREM 은 $\mathcal{L}_{\text{nrem}}$, REM 은 $\mathcal{L}_{\text{rem}}$
discriminator head	Θ^{disc}	같은 optimizer. encoder 와 앞 3 층 공유	wake(목표 1) 와 REM(목표 0). NREM 에는 달려 있다 [L866-868]
generator(deconv 4 층)	Θ^{gen}	wake 는 $\mathcal{L}_{\text{img}}$ 에 하강, REM 은 $\mathcal{L}_{\text{rem}}$ 에 상승	wake 와 REM. NREM 에는갱신 줄이 없다
store 현재 슬롯	E 의 $\hat{c}_k$	아무도 — 학습이 아니라 복사	wake. NREM 에서 목표로, REM 에서 혼합 항으로 읽힘
store 이전 슬롯	E 의 $\hat{c}_{k-1}$	아무도 — REM 끝에서 밀려들어감	보존 정확히 한 사이클
REM 잠음	ε	아무도 — 매 REM 재 표집	REM 만. 혼합의절반

대상	기호	누가 움직이나	언제
가림 마스크	$\text{occ}(\cdot)$	아무도 — mini-batch 마다 확 률 0-1, 사각 크기 1- 8 표집 [L1627-1628]	NREM 만
혼합 계수	λ, λ'	아무도 — 고정, 변주 되지 않음	REM 만
선형 readout	W_{ro}	라벨을 쓰는 지도 SGD	모델동결 후. 모델의 일부가 아니다
discriminator 교시 신호	파라미터화되지 않음	아무도 — 구현에서는 상수 1 과 0. 논문은 가 정된 뇌 상태 신호 라고 밝힌다 [L855-856]	wake 와 REM

두 논문의 시계와 순서를 명시한다. PLOS 에서 sleep 라운드 첨자 k 는 **한 번의 interleaved 주기**다 — 새 과제 학습 100 movement cycle 뒤 sleep 100 movement cycle 이고, duty ratio 가 시뮬레이션 시간 기준 정확히 1:1 이며, 두 블록이 같은 파라미터를 쓴다 [Results L300-306; Methods: Simulated sleep L985-986]. PAD 에서 k 는 **mini-batch 하나**이고, 그 안에서 (U-E) 한 번과 (U- Θ) 세 번이 고정 순서로 돌되 위상마다 갱신 블록이 다르다. 논문의 x 축은 epoch 단위 이므로 k 와 epoch 를 섞으면 오프라인 예산이 세 자릿수만큼 작아 보인다 [본서 추론]. 두 편 모두 오프라인 위상이 하나가 아니라는 ch08 § 08.2 의 판별기를 만족한다 — PLOS 는 두 목적, PAD 는 세 목적이다.

[해설] 독자의 1 번 질문에 대한 이 절의 답은 짧다. 어느 쪽에서도 **decode 가 바뀌지 않는다**. PLOS 의 결정 한 번은 고정된 3 층 전방 경로를 600 timestep 도는 것이고, sleep 은 $\Theta^{H \rightarrow O}$ 의 **값만** 바꾼다 [Methods: Policy L769-789; Simulated sleep L978-980]. PAD 의 평가는 동결된 encoder 의 forward 한번이다. 즉 두 편 모두 상각 논증이 가장 좋아하는 모양인데, **양쪽 다 어느 쪽 비용도 재지 않는다**.

23.5 비용 4 종

표 23-5. 비용 4 종. “논문에 없음” 은 추정으로 매우지 않는다.

	PLOS Sleep	PAD
B_s	부분. 계산량이 아니라 시뮬레이션 시간으로만 존재한다. 한 sleep 구간 = 100 movement cycle = 60,000 timestep, timestep 당 0.5 ms 이므로 시뮬레이션 시간 30 초 [본서 산술] [L769, L807, L985-986]. duty ratio 1:1 [L301-303]. FLOPs·wall-clock·CPU 시간·에너지 전부 논문에 없음. 길이·비율·교대 횟수 sweep 없음	논문에 없음. FLOPs·GPU 시간·wall-clock 어느 것도 없다. 구조적으로 확인되는 것은 사이클당 (U- Θ) 세 번 중 두 번이 오프라인이라는 사실뿐

	PLOS Sleep	PAD
L_w	논문에 없음. 지연 · 처리량 · 결 정당 비용이 어디에도 없다	논문에 없음
C_{cap}	바이트로는 논문에 없음 (dtype 없음). 개수로만: 연속 가소성 시냅스 6272 [L995, L1035], 한 번 학습 후동결된 입력→은닉 7056 [L751, L205-207], 자유도 0 인 억제성 6272 [L956-963], 항상성 스칼 라 8 [L938-945], 고정 목표합 784 [L928-930], sleep 구동 발화율 784 → 통제에서 1 [L982-985, L1144-1150]. 과 거 과제 데이터 0 바이트 [L611, L674-677, L727]	바이트로는 논문에 없음. 구조 적으로 2 슬롯 고정, 보존 기간 정확히 한 사이클 — $Z_{old} \leftarrow$ Z 가 매사이클 덮어쓴다 [Algorithm 1, L1423]. 누적 되는 것이 없다
ρ	부분. 간접 과제 하나의 종점 만: sequential 은 $0.70 \pm 0.02 \rightarrow 0.52 \pm 0.02$, sleep 교 대는 $0.70 \pm 0.02 \rightarrow 0.70 \pm 0.03$ [L228-229, L275, L304-305]. 라운드당 감쇠율 없음. 세 번째 과제도 두 번째 interleaved 위상도 없음	방향성만. NREM 이 없는 조건 에서 “linear separability tends to decrease after many training epochs” [PAD Appendix 1, ‘Linear classification performance’]. 수치화된 라운드당 손실 없음

식 (A) 의 손익분기 N_q^* 는 두 논문 모두에서 계산할 수 없고, 이유가 서로 다르다. PLOS 에는 질의 스트림도 문맥도 사용자도 없다 — 에이전트는 movement cycle 마다 결정 하나를 영원히 낸다. 1:1 duty ratio 를식 (A) 에 넣으면 주기당 $C_{sleep} = C_{wake}$ 가 되지만나눌 N_q 가 없다. PAD 의 평가는 고정된 테스트 집합 위의 선형 probe 이므로 한 문맥을 공유하는 질의라는 개념 자체가 없다.

[평가] 이 라인의 상각 논증은 생물학에서 오지 않았다. 생물학적 조상 두 편은 상각 증거를 하나도 제공하지 않는다. 채워진 칸조차 계산량이 아니라 시뮬레이션 시간이거나 파라미터 개수다. 그러므로 ch01 § 01.5 의 회계 규칙대로, 이 두 편을 근거로 “sleep-time compute 이 효율적이다” 라고 쓸 수 없다. 이 문헌이 실제로 제공하는 것은 **상태 용량 논증** 하나다 — 과거 과제 데이터 0 바이트, 경계를 넘는보조 상태 $O(1)$ 스칼라 ($\rightarrow$ ch25, ch26).

23.6 실험과스케일

이 절이 § 23.2 의 정의가 요구하는 네 좌표를 채운다. 수치는 원문 정밀도그대로 옮기고 반올림하지 않는다.

23.6.1 PLOS — 조건전량

과제는 50×50 격자에서 입자 밀도 10%의 foraging 두 개다 [Methods L731, L739-740]. 입자는 네 방향 중 하나이고, Task 1 은 수평을 보상하고음대각을 처벌하며 Task 2 는 수직을 보상하고 양대각을 처벌한다 [Results L225-232; Methods L932-935]. 지표는 pattern discriminability 이고 chance 가 0.5 다 [L166-167]. 모든 수치는 최소 10 회 시행의 평균 $\pm$ 표준편차이며, vendored

원문 전문에 검정 이름도 p-값도 한 건없다 [본서 관찰] [L168-169]. 그런데 본문은 “significantly”를 최소 세 번 쓴다 [L418, L444, L506]. 이 책은 두 사실을 함께 적고, 게재본의 그림·보충자료까지 포함한 판정은 유보한다.

표 23-6. PLOS Sleep 의 전 조건. 굵은 행이 헤드라인 결과다.

조건	Task 1	Task 2	과거 데이터	출처
Task 1 단독 학습 후	0.70 ± 0.02	0.53 ± 0.02	—	L228-229
Sequential (T1 → T2)	0.52 ± 0.02	0.69 ± 0.03	불필요	L275-276
Interleaved _{T1,T2} (고전 interleaving)	0.68 ± 0.03	0.65 ± 0.04	필요	L282
Interleaved_{S,T2} (sleep 교대)	0.70 ± 0.03	0.68 ± 0.05	불필요	L304-305
Uniform-Noise Sleep(집단 평균 1 스칼라 구동)	0.67 ± 0.05	0.69 ± 0.03	불필요	L1148-1150
나이프 망 + Task 2 통계로 만든 sleep 잠음	0.60 ± 0.03	0.49 ± 0.05	불필요	L1136-1137
Task 1 학습 후 Interleaved _{S,T1}	0.71 ± 0.02	0.51 ± 0.02	불필요	L1141-1143
상위 1% 시냅스 동결	0.54 ± 0.02	0.68 ± 0.03	불필요	L1184-1187
상위 5% 시냅스 동결	0.65 ± 0.02	0.61 ± 0.01	불필요	L1184-1187
상위 10% 시냅스 동결	0.70 ± 0.03	0.53 ± 0.03	불필요	L1184-1187

마지막 세 행에는 단서가 붙는다. 동결 baseline 은 세 비율만, 그리고 “Task 1 학습 후 크기 상위” 라는 단일 선택 기준으로만 시험되었고, 논문이 이 계열의 기계학습 사례로 지목한 EWC [ref 7] 는 이름만 볼릴 만한 번도 실행되지 않는다 [L546-547]. 즉 이 비교는 중요도 가중정규화가 제대로 구현되었을 때의 결과가 아니다 [본서 추론].

여섯째·일곱째 행이 sleep 이 하지 않는 일을 정한다. sleep 잠음에 Task 2 의 발화 통계를 실어 넣어도 나이브 망은 Task 2 를배우지 못하고 (0.49 ± 0.05), Task 1 만 학습한망에 sleep 을 돌려도 Task 2 는 baseline 에 머문다 (0.51 ± 0.02). 오프라인 위상은 학습된 적 없는지식을 만들어 내지 않는다 [본서 추론].

강한 baseline 이 방법과 거의 같다. 고전 interleaving 은 0.68/0.65 이고 sleep 은 0.70/0.68 이다. 차이는 0.02 와 0.03 이며 표준편차는 0.03 – 0.05 다. 논문은 Results 에서 sleep 이 고전 interleaving 을 “exceeding” 한다고 쓰지만 [L305-306], Discussion 에서 스스로 정정한다.

“Although classical interleaved training of the old and new tasks showed similar performance results in our model as interleaving new task training with sleep, **we believe the latter to be superior on the following theoretical grounds.**” [PLOS Sleep Discussion, L654-655, 강조는 이 책]

이어지는 근거는 전부 이론이다 — sleep 은 두 과제가 섞인 replay 를 지원할수 있다는 것 [L655-670]. 그런데 replay 내용을 분석한 실험은 이논문에 없고, 혼합 replay 주장은 인용으로 수입된다 [refs 41, 52, 53].

가중치 상태 판독도 기록한다. 6272 차원 가중치 벡터를 Task-1 형/Task-2 형으로 분류하도록 학습시킨 RBF-SVM 의 부호있는 평균 거리는 Task 1 이 – 0.069, Task 2 가+0.069, sleep 교대가

– **0.0047** 이다 [Fig 4D, L394-396]. 데이터 replay 버전에서는 Interleaved $_{T1,T2}$ 가 0.016 이다 [S5, L1165-1167]. 원문 수치를 단일 과제 크기 0.069 로 나누면 (이 책의 산술) 전자는 6.8%, 후자는 23%인데 **논문은 두 수치를 직접 비교하지 않는다.**

manifold 결과에는 방법론적 단서가 붙는다. 교집합 점집합 $\text{Man}_{T1 \cap T2}$ 가 “defined solely by the synaptic weight states from the last fifth of both InterleavedT1,T2 and InterleavedS,T1 training” 으로 구성되므로 [Methods L1013-1019], **그 점집합으로수렴하는 것으로 그려지는 궤적 자신이 점집합의 재료다.** 저자도 표본부족을 밝힌다 [L572-573]. 정확도 결과가 틀렸다는 뜻이 아니라 기하헤드라인이 독립 측정이 아니라는 뜻이다.

원문 내부 불일치 세 건은 어느 쪽으로도 정리하지 않고 양쪽을 그대로적는다. (i) 은닉 뉴런 제거 실험의 임계는 본문이 “around 70% of the hidden layer was pruned” [L241], 보충 캡션이 “remains high until ~25% of neurons are left” [L1109-1110] 로 어긋난다. (ii) uniform-noise 실험의 이름이 본문에서는 Interleaved $_{US,T1}$ [L359], S4 캡션에서는 Interleaved $_{US,T2}$ 다 [L1146] — 보고된수치 (Task 1 0.67 ± 0.05 , Task 2 0.69 ± 0.03) 가 학습된 쪽이 Task 2 임을 가린다. (iii) 표준편차 한 개가 “ 0.70 ± 002 ” 로 오식되어 있다 [L305]. 표 23-6 은 L228 의 올바른 값 0.70 ± 0.02 로 적었다.

논문 자신이 못 박는 적용 범위 제한도 옮긴다. “In our model, and indeed in all neural network models, the system begins as a ‘blank slate’ without knowledge of any previous learning or competing demands” 이며, 그래서 저자는 주장을 “the interference phenomena which follow training on an initial task as opposed to initial learning” 으로 줄인다 [Discussion, L685-697]. 즉 이 논문의 결과는 사전 경험이 있는시스템에 대한 진술이 아니다.

규모 천장. 뉴런 842 개 (입력 49 · 은닉 784 · 기능적 출력 8) [L743-746], 연속 가소성 시냅스 6272 개, 과제 2 개이며 그 둘은 같은 네 방향 어휘에서 뽑은 대칭 상보과제다 (논문 자신이 유사성을 특징으로 강조한다 [L233-237]). 보호되는성능 밴드는 0.20 폭이고 조건별 표준편차가 그 밴드의 10 – 25%다. 과제 환경과 두 평가 지표가 모두 저자 제작이고 표준 지속학습벤치마크는 하나도 쓰이지 않았으며 [L166-167, L1037-1095], 계산자원은 보고되지 않았다.

23.6.2 PAD — 절제와통제

설정은 conv 4 층 encoder(채널 64·128·256·256), deconv 4 층 generator, $n_z = 256$, $b = 64$, ADAM $\eta_\Theta = 0.0002$ 이고데이터셋은 CIFAR-10 과 SVHN 이다 [Methods L1247-1283, L1506]. 지표는모델을 동결한 뒤 latent 위에 학습시킨 선형 분류기의 테스트 정확도다.

표 23-7. epoch 50 시점의 선형 분리도. 초기값 4 종에 대한 평균 $\pm$ SEM [PAD Appendix 1—table 1].

데이터셋	PAD	w/o memory mix	w/o REM	w/o NREM	Wake only
CIFAR-10	58.25 ± 0.70	53.87 ± 0.85	46.00 ± 0.43	58.00 ± 0.34	42.25 ± 0.54
SVHN	78.92 ± 0.40	60.87 ± 5.07	42.30 ± 1.51	73.25 ± 0.22	41.93 ± 0.65

표를 정확히 읽는다. w/o memory mix 는 혼합만 없앤조건이 **아니다** — REM 을 단일 episodic memory 로, 잡음 없이 구동한조건이다 [PAD Results, L548-549]. CIFAR-10 에서 w/o NREM 은 58.00 ± 0.34 로 PAD 의 58.25 ± 0.70 과 오차안에서 같다 (그 함의는 ch08 § 08.4.2 가 다뤘다)[본서 관찰]. 절제대조군이 손봐진 상태로 비교된다는 사실도 함께 옮긴다 —w/o REM 모델은 encoder 출력에 $\varepsilon \sim \mathcal{N}(0, 0.5I)$ 를 더한 변형 재구성 손실로학습된다 [PAD Eq. 12].

이 장의 나머지 세 의무 caveat 가 여기서 나온다.

첫째, 저자 자신의 통제가 논문 제목의 성분을 지지하지 않는다. “we do not observe significant differences between using a combination of episodic memories with spontaneous activity or only using spontaneous activity” [PAD Discussion, L1034-1036]. 근거 그림은 세 팔을 비교한다— 혼합 기억 + 잡음, 순수 잡음만, 혼합 기억만 [Appendix 1—figure 4 caption]. 표 23-7 의 w/o memory mix 가 무너지는것은 잡음까지 뺀기 때문이고, **잡음만 남기고 기억을 빼면**

차이가 없다. 논문 제목이 가리키는 성분 (mixed episodes 로부터 adversarial dreaming) 이 통제 대비 효과를 보이지 않았다.

둘째, sleep 단계의 순서가 무관하다. “our model does not show significant differences in performance when the order of sleep phases is switched” [PAD Discussion, L1046-1047; Appendix 1—figure 5]. 저자는 생물학의 sequential hypothesis (Giuditta et al. 1995) 에서 NREM → REM 순서가 중요하다고 가설된다는 점을 인정하면서, 모델의 순서 독립성을 단계 별 시냅스 변화가 작기 때문으로 추정한다 [L1050-1055].

셋째, 표현 품질의 지표가 하나뿐이고 절대 성능이 낮다. 측정된 것은 linear readout 정확도 하나이며 저자는 이를 “an obvious simplification with regard to cortical processing” 이라고 인정한다 [L1039-1041]. downstream 과제 성능은 측정되지 않았다. 그 하나뿐인 지표의 절대값이 CIFAR-10 에서 “around 59% test accuracy” 다 [PAD Results, L534; Figure 4c]. 같은 시기 self-supervised 표현 학습의 CIFAR-10 linear probe 와의 비교는 논문이 수행하지 않는다. 따라서 논문은 기제의 존재 증명이지 성능 주장이 아니며, 성능 근거로 인용하면 오용이다. SVHN 은 표본 분포 불균형 때문에 나이브 분류기의 최고 성능이 18.9%여서 baseline 자체가 자명하지 않다 [Figure 5 caption, L643].

[평가] 여기서 검정의 부재가 양날이 된다. vendored PAD 전문에서 p-값 · 검정 이름을 검색하면 한 건도 나오지 않는다 [본서 관찰]. 그러므로 “유의한 차이가 없다” 는 두 보고 — 기억 혼합 무관, 단계 순서 무관 — 역시 검정에 근거하지 않는다. 이 책은 두 보고를 저자의 관측으로 귀속시켜 인용하되 “혼합이 무용하다는 증명” 으로 격상시키지 않는다. 정확한 진술은 이것이다 — 저자 자신의 통제는 혼합 성분의 효과를 보이지 못했고, 그 통제에는 검정이 없다. 두 절반을 다 적어야 정직하다.

표 23-8. § 23.2 정의가 요구하는 네 좌표.

좌표	PLOS Sleep	PAD
기제 클래스	국소 보상 STDP + 항상성 scaling + 열 합 보존 정규화. loss·gradient·optimizer 없음	ADAM gradient descent, GAN 목적함수, 한 블록의 부호 반전
규모	뉴런 842 개, 가소성 시냅스 6272 개, 행동 8 개	conv 4 층 + deconv 4 층, $n_z = 256$, CIFAR-10 / SVHN
측정 지표	저자 정의 pattern discriminability (chance 0.5, 천장 0.70)	선형 probe 정확도 하나 (저자가 “명백한 단순화” 라고 인정)
반복 라운드 수	과제 2 개, interleaved 위상 1 회. 3 과제 실험 없음	사이클마다 반복되나 ρ 의 수치화 없음. NREM 없는 조건에서 만감소 경향 보고

23.7 Systems/serving 함의

독자의 1 번 질문에 대한 답은 § 23.4 에서 이미 나왔다 — 어느 쪽에서도 decode 가 바뀌지 않는다. 이 절은 그 위에서 이송 가능한 systems 진술 세 개만 남기고 나머지를 명시적으로 금지한다.

첫째, 배포 아티팩트가 없는 극단. PLOS 에서는 wake 도 sleep 도 같은 $\Theta^{H \rightarrow O}$ 를 쓴다. 동결된 서빙 가중치가 존재하지 않는다. Rosetta 표 (→ ch01) 로 옮기면 “shared weights 로 배치한다” 는 전제가 첫 movement cycle 부터 깨진다 — 에이전트마다 가중치가 즉시 갈라진다. Θ -경로 per-user delta 문제의 극단값이고, 그 문제가 LoRA 류 구현의 부산물이 아니라 경로에 내재한다는 것을 보여 준다 (→ ch19, ch29). LLM 어휘로 옮기면 이 논문의 wake-time inference 는 서빙 중인 가중치를 끊임없이 fine-tuning 하는 것에 해당한다.

둘째, 오프라인 워크로드의 모양은 스케줄이 정하지 않는다. PLOS의 sleep 라운드는 (i) 은닉 784 유닛에 독립 Poisson 스파이크열을 만들고, (ii) 784×8 조밀 행렬을 60,000 timestep 동안 전진시키고, (iii) 사건마다 같은행렬의 열 정규화를 다시 도는 것이다. 작고, 조밀하고, 시간축으로 순차적이며, 병목이 행렬 곱이 아니라 **사건당 정규화**다. LLM의 sleep 라운드 (크고, 배치되고, gradient가 지배하는)와 정반대의 프로파일이다 — “오프라인 계산”은 스케줄의 이름이지 워크로드의 모양이 아니다. 세 경로의 sleep 비용을 한 축에 놓으려면 이 사실을 먼저 인정해야 한다 (→ ch25).

셋째, 유일한 용량 진술. wake → sleep 경계를 넘는 보조 상태는 은닉뉴런 784개의 목표 발화율인데, Uniform-Noise Sleep 통제가 이것을 집단평균 하나로 줄여도 결과가 유지된다 ($0.67 \pm 0.05 / 0.69 \pm 0.03$) [S4E/F, L1144-1150]. 과거 과제 데이터는 0 바이트다.

[평가] 셋째 항목은 **가설로만** 이송된다. 그 성질이 gradient로 학습되는 모델에 살아남는다면 Θ -경로의 sleep 라운드는 사용자별 replay buffer 없이 가중치만으로 돌 수 있다는 뜻이 되고, 그것은 C_{cap} 회계를 바꾼다. 그러나 이 논문은 그 이송을 보이지 않았다 — 이 논문의 갱신 규칙에는 도함수가 없고, 저자들이 자기 그룹의 feedforward ANN 이식을 인용하지만 여기서 실행하지는 않는다 [Discussion, L677-679]. 이 책은 이것을 결과가 아니라 반증 가능한 가설로 세우고 Part III로 넘긴다 (→ ch26, ch29).

금지도 명시한다. 두 논문 어느 쪽도 바이트·대역폭·arithmetic intensity·하드웨어를 보고하지 않는다. 따라서 **이장에서 memory-hierarchy 논증을 만들 수 없다**. HBM 이든 DRAM 이든 이워크로드를 어디에 놓는 진술은 전부 발명이 된다. NM D4의 억지 연결 금지규칙을 여기서 그대로 적용한다 — 이 책이 memory-centric 논증을 세우는 자리는 생물학 장이 아니라 자체 측정이 있는 자리다 (→ ch26, ch29).

[해설] PAD 쪽에서 가져갈 것은 더 적지만 더 날카롭다. 이논문의 오프라인 위상 두 개는 wake와 같은 **optimizer, 같은 η_θ , 동등 가중의 loss**를 쓰고 [L1618-1621], 유일한 차이가 목적함수의 선택과 한 블록에 대한 부호다. 새 연산도 새 커널도 추가상태도 없다. 생물학 쪽에서 “각성과 sleep은 다른 종류의 가소성을 쓴다”로서술되는 것이 계산 쪽에서는 “오프라인 라운드는 목적함수와 부호만 다르다”로 환원된다. 이 라인이 생물학에서 실제로 가져올 수 있는 것은 대체로 **스케줄과 목적함수의 배치**이지 새로운 갱신 primitive가 아니다.

23.8 한계와 bridge-out

논문 자신이 남긴 문제. PLOS는 세 과제 이상으로 확장되는지모르고, 중단 없는 새 과제 학습이 얼마나 견딜 수 있는지 모르며 (실패레짐을 지목만 하고 경계를 재지 않는다 [L639-642, L649-653]), gradient로 학습되는 망으로의 이송을 자기 인용으로만 주장하고 [L677-679], 우위 논증의 핵심인 혼합 replay의 실재를 분석하지 않았다 [L668-670]. 모델링된 것은 REM 류 활동뿐이고 NREM은 아예 없으며 [L292-297], 생물학적 예측도 미검증이다 [L721-722]. PAD는 latent 혼합 전략의 최적성을 미해결로 두고 [L1030-1033], 연속 학습에서 최근기억의 선호적 replay가 유용하다는 것을 **가설로** 적으며 [L1036-1038], readout 학습이 encoder를 바꾸지 않는다는 자기 가정을 스스로 문제로 지목하고 [L1041-1044], 단계 순서가 영향을 주려면 시냅스 변화가 더 커야 한다는 예측을 검증하지 않았으며 [L1050-1055], 진동·spindle·slow wave를 포함한 회로 모델을 만들지 않았다 [L940-944].

여기서 이 장의 두 번째 정의를 못 박는다. ch08 §08.5가 **은유의이송 한계**라는 경계를 이름 붙였다면, 이 장은 그 경계에 대한 판정 절차를 소유한다.

정의. 은유의 이송 한계 판정이란, 생물학 문헌을 근거로든 임의의 주장 P 에 대해 §23.2의 네 좌표를 차례로 묻는 결정 절차다. (1) P 가 요구하는 기제가 원 논문의 기제 클래스 안에 있는가. (2) P 가 주장하는 규모가 원 논문의 규모 안에 있는가. (3) P 가 쓰는지

표가 된 논문이 측정한 지표인가. (4) P 가 함의하는 반복 라운드 수를 원 논문이 실행했는가. 네 물음에 모두 “예” 라야 P 는 그 문헌이 지지하는 주장이다. 하나라도 “아니오”면 P 는 유비이며, 유비임을 표시하지 않고 인용하는 것은 이 책에서 결함이다.

[평가] 이 문헌이 지지하는 것. 1. 식 $(U-\Theta)$ 의 $\mathcal{R}_k$ 는 데이터셋일 필요가 없다. 한 시스템에서 모델을 잡음위에 돌려 만든 스트림으로 충분했고, $\mathcal{H}_k$ 는 Θ_k 안 말고 어디에도 없었다 [PLOS L300-311]. 2. 각성→오프라인 경계를 넘는 보조 상태는 $O(1)$ 스칼라로 줄어들 수 있다 [PLOS S4E/F, L1144-1150]. 3. 오프라인 위상은 옛 파라미터의 정적 보호와 같지 않다. 시험된 동결 비율 세 점 중 1%·5%는 두 축에서 동시에 뒤지고, 10%는 Task 1에서 동률 (0.70 ± 0.03) 이면서 Task 2를 배우지 못한다 (0.53 ± 0.03 대 0.68 ± 0.05) [PLOS S6 L1184-1187 vs L304-305]. 단 이 비교는 세 점·단일 선택기준이고 EWC는 실행되지 않았다. 4. interleaving의 입도에 레짐경계가 있다. 짧은 에피소드를 sleep과 교대하면 성공하고 한 번의 긴 에피소드는 회복 불가능하다 [PLOS L307-311, L639-653]. 5. 오프라인 위상의 가치는 스케줄이 아니라 거기서 무엇을 생성하는가에서 나온다. 단 PAD에서 결정적인 성분은 기억의 혼합이 아니라 자발 활동의 주입이다 [PAD Appendix 1—table 1 + Appendix 1—figure 4].

[평가] 이 문헌이 지지하지 않는 것. 1. gradient로 학습되는 망에 대해 아무것도. PLOS에는 loss도 도함수도 optimizer도 backward pass도 없다. SGD/AdamW sleep 라운드가 이 동역학을 물려받는다 고 볼 근거가 없다. 2. transformer·언어·시퀀스 모델링에 대해 아무것도. attention도 recurrence도 토큰도 없다. 3. 배포 규모에 대해 아무것도. 연속 가소성 시냅스 6272개 대 $10^9 - 10^{12}$ 파라미터. 4. 생성물의 의미적 정확성에 대해 아무것도. 두 논문 모두 replay·dream의 내용을 검사하지 않는다. LLM에서 이에 대응하는 실패 — 유창하지만 거짓인 생성물로 Θ 를 갱신하는 것 — 이 Θ -경로의 중심 위험인데 (→ ch06, ch21) 두 편 다 침묵한다. 5. sleep이 데이터 replay보다 정확도에서 낫다는 것에 대해 아무것도. 논문 자신이 우위를 이론적 근거로 돌린다 [PLOS L654-655]. 6. 반복 라운드의 ρ 에 대해 아무것도. 과제 2개, interleaved 위상 1회다. 7. 서버 비용에 대해 아무것도. FLOPs·지연·바이트·배치·하드웨어가 전부 없다. 8. 혼합 에피소드부터의 dreaming과 sleep 단계 순서의 필요성에 대해 아무것도. 저자 자신의 통제가 둘 다 지지하지 않는다 [PAD L1034-1036, L1046-1047].

경로 간 인용. PLOS는 E-경로와 W-경로에 완전히 침묵하고, 그 침묵은 연대가 강제한 것이다 — 2022년 11월 18일 게재로 corpus의 모든 E·W 경로 논문보다 앞선다 [L36]. Θ -경로 쪽으로는 2020년이전 지속학습 부문만 접촉한다: EWC [ref 7], CLS [ref 11], Hayes et al. 2021 리뷰 [ref 9], meta-learning 한 건 [ref 51]. Deep Generative Replay는 인용되지 않는다 — 저장 없이 $\mathcal{R}_k$ 를 만드는 두 해법이 서로를 보지 않은 자리이며 연대가 강제하지 않은 공백이다 (→ ch08). PAD도 세 경로 전부에 침묵하고 이 역시 연대가 강제한 것이다. vendored 전문 검색 계수가 그 상태를 보여 준다 — transformer 0, language model 0, LoRA 0, test-time 0, fast weight 0, 그리고 replay 38[본서 관찰].

반대 방향이 이 장의 결론이다. E-경로의 창시 논문은 제목에 sleep을 빌려 쓰면서 생물학·CLS 문헌을 한 편도 인용하지 않는다. Sleep-time Compute(2504.13171)의 참고문헌 22편 [Letta STC References, pp.13-15] 안에 McClelland도, hippocampal replay도, sleep consolidation 신경과학도, dreaming 문헌도 없다. 그 단어를 도입하는 문장은 순전히 운영적이다: “inference is done between interactions with the model while it would otherwise be idle in sleep-time” [Letta STC § 1]. 여기서 sleep은 consolidation이 아니라 유희를 뜻한다. 은유만 가져오고 근거는 두고 온 것이다.

[평가] 두 방향의 침묵에 같은 등급을 매기지 않는다. 생물학쪽의 침묵은 연대가 강제했으므로 계보 사실이고, Sleep-time Compute의 침묵은 선택이므로 발견이다 — CLS는 1995년, 이 장의 두 편은 2022년, 저 논문은 2025년이다. 그런데 결함은 인용의 부재가 아니다. 저 논문이 하는 일에 CLS의 두 시스템 논증은 필요하지 않다. 결함은 이름의 재사용이다. 서로 다른 세 경로가 같은 단어를 쓰기 때문에 독자는 그것들이 한 가

설의 변형이라고 읽게 되고, 이 책이 판별식을따로 세워야 했던 이유가 정확히 그것이다 (→ ch11, ch24).

Part III 가 받아가는 것. ch24는 위 판정 절차를 판별식과 나란히놓고 “무엇이 실제로 sleep-time compute인가”를 닫는다. ch25는 이 장이비운 비용 칸을 통일 회계의 결측으로 기록하고, ch26·ch29는지지하는 것 2 번을 반증 가능한 가설로 세워 자체 측정과 대조한다. ch27은 “생물학이 지지한다”는 문장을 쓸 때마다 이 장의 두 목록을통과시키고, ch30은 지지하지 않는 것 4 번을 반증 조건의 재료로쓴다. Part II는 여기서 끝난다 — 이제 판정으로.

요약

- PLOS는 상속받은 open question을 명시하지만 그 공백을 선언한 리뷰의공저자가 이 논문의 교신저자다. PAD에는 상속 진술이 없다. 상속이성립하는 자리는 경로가 이어진 자리가 아니라 연구 그룹이 이어진 자리다.
- PLOS는 (U- Θ)를 wake와 sleep 두 시계로 같은 6272개 시냅스에쓴다. gradient가 없고, 갱신이 열 합 보존 집합으로 사영되며, $\mathcal{R}_k$ 는 잡음 위에서 망 자신을 돌려 만든다.
- PAD는 사이클마다 (U-E)한 번과 (U- Θ)세 번을 돌고 REM에서 한파라미터 블록의 gradient 부호를 뒤집는다. 표준형 (U- Θ)는 부호필드 없이 이 형태를 담지 못한다.
- 비용 4종 여덟 칸의 절반이 “논문에 없음”이다. 생물학적 조상 두 편은상각 증거를 제공하지 않으며, 제공하는 것은 상태 용량 논증 하나다.
- PLOS의 강한 baseline(과거 데이터를 저장하는 고전 interleaving)은 0.68/0.65로 sleep의 0.70/0.68과 표준편차 안에서같고, 논문 자신이 우위를 이론적 근거로 돌린다. 실증된 우위는 정확도가아니라 저장의 제거다.
- PAD의 저자 통제는 논문 제목이 가리키는 성분(혼합 에피소드)의 효과도, sleep 단계 순서의 필요성도 보이지 못했다. 표현 품질 지표는 선형 probe 하나뿐이고 CIFAR-10 절대값은 약 59%다.
- E-경로의 창시 논문은 제목에 sleep을 쓰면서 생물학·CLS를한 편도 인용하지 않는다. 이름의 공유는 이론적 약속의 공유가 아니다.

자가 점검체크리스트

- ☐ PLOS와 PAD 각각에 대해 § 23.2의 네 좌표를 채워 말할 수 있다.
- ☐ 임의의 “생물학이 지지한다”는 문장에 § 23.8의 네 물음을 적용해지지/유비를 판정할 수 있다.
- ☐ 두 논문의 비용 4종 중 어느 칸이 왜 비어 있고 그 공백이 어떤 서술을금지하는지 설명할 수 있다.
- ☐ PLOS의 sleep이 고전 interleaving 대비 실증한 것과 이론으로만 주장한것을 갈라 말할 수 있다.
- ☐ PAD에서 REM의 결정적 성분이 혼합이 아니라 자발 활동의 주입임을, 그근거의 검정 부재까지 함께 말할 수 있다.
- ☐ 연대가 강제한 침묵과 선택된 침묵을 실제 사례로 각각 하나씩 들 수 있다.
- ☐ 이 두 논문의 오프라인 위상을 inference 어휘로 옮길 수 있다 —warm-up/사전 컴파일 자리인지 모델 재배포 자리인지, 그리고왜 두 편이 후자인지.

Part III — 판정

Part III 가 답하는 질문은 이 책의 원 질문이다. **sleep-time compute** 은 **유망한가**. Part I 과 Part II 는 그 질문에 답하지 않으려고 두부를 쓴 것이 아니라, 답할 수 있는 형태로 바꾸려고 쓴 것이다. 대상이 확정되지 않으면 주어가 없고, 비용이 같은 자로 재지지 않으면 술어가 없다. 그 둘을 갖춘 뒤에야 “유망한가” 는 답을 가진 질문이 된다. 그리고 답은하나가 아니라 **경로마다 다르다**.

이 부에서는 새 논문 사실을 처음 소개하지 않는다. 앞에서 확립된 것과 이책이 직접 돌린 실험 위에서만 결론을 낸다. 배치는 전제의 순서를 따른다. **ch24** 가 주어를 확정한다 — 무엇이 실제로 이 범주이며, 이름과기제의 간격이 일시적 지체인가 구조인가. **ch25** 와 **ch26** 이술어를 만든다: 세 경로를 하나의 비용 축에 올리고 (상각식에서 빠진 항과 κ 민감도가 여기서 드러난다), 상태 용량을 바이트로회계한다 (corpus 에서 아무도 보고하지 않은 값이다). **ch27** 이 그 셋위에서 원 질문에 답한다 — 경로별 판정과 각 판정이 성립하는 조건.

나머지 셋은 판정의 전개다. **ch28** 은 B_s 를 독립변수로 놓은점들을 통일 축에 재배치하고 scaling law 가 성립할 수 있는지를 보고 실태의전수 감사로 판정한다. **ch29** 는 판정을 배포 형태로 옮긴다 — shared-weight batching 이 깨지는 지점, 롤백 단위의 변화, 그리고 이 책이 memory-centric 논증을 펴는 **유일한** 자리인 웹 계층 delta 스테이징. 그 자리는 Neural Memory 가 짙은 지점과 다른 자리이며, 두 책의결론을 하나로 잇지 않는다. **ch30** 은 이 책의 판정을 뒤집을 관측을구체적으로 적는다.

일곱 장을 관통하는 규율은 증거 등급 계약이다. 이 부가 덮는 자체 실험넷은 어느 것도 GPU 측정이 아니다 — 둘은 결정론적 회계, 하나는 corpus 전수 감사, 하나는 고정 시드의 소규모 시뮬레이션이다. 따라서 본문단정문으로 올라가는 것은 **비율 ·crossover 위치 ·순서 ·bound** 분류뿐이고 절대치는 하한 또는방향성으로만 쓴다. 각 장의 증거 표는 Part II 가 원문 대조로 확립한 사실과 이 책의 실험을 등급을 갈라 적는다. 그리고 각 장은 다섯째 절 없이 끝나지않는다 — **반증 불가능한 판정은 이 책에서 쓰지 않는다**.

ch24. 무엇이 실제로 sleep-time compute인가

24.1 판정할 질문

Part II는 논문을 하나씩 읽었고 판정을 미뤘다. 열세 장이 같은 자리에서 멈췄다 — 기제를 통일 표기로 환원하고, 비용 네 칸을 채우거나 “논문에없음”으로 남기고, 불리한 사실을 본문에 적은 뒤, 그것이 무엇을 뜻하는지는 Part III로 넘겼다. Part III가 그 미룬 것을 갚는다. 이 부에서는 새 논문사실을 처음 소개하지 않는다. 앞에서 확립된 것 위에서 결론을 내리고, 그결론이 틀렸음을 보일 관측을 함께 적는다. 이 장이 그 첫 문이다.

판정할 질문. “sleep-time compute”이라는 이름 아래 모인 문헌 중무엇이 실제로 이 범주이며, 이름과 기제 사이의 간격은 우연인가구조인가.

답이 될 수 있는 것을 먼저 못 박는다. 세 가지다. 첫째, 통과·불충족·배경의 3분할과 그 분할이 닫는근거. 둘째, 이름과 기제가 어긋나는 원인에 대한 판정 — 아직정리되지 않은 일시적 지체인가, 아니면 서로 다른 공동체가 각자의 어휘를유지한 채 같은 단어를 쓰는 구조적 상태인가. 셋째, 그 간격이 실무의 어떤판단을 망가뜨리는가.

답이 될 수 없는 것은 넷이다. 어느 경로가 우월한가는 이 장의질문이 아니다 — ch27이 답한다. 각 경로의 비용이 얼마인가도아니다 — 통일 회계는 ch25가, 상태 용량은 ch26이 세운다. 개별논문이 좋은 논문인가도 아니다. 판별식은 논문의 자기 규정도 성능도 묻지않으며 (→ ch11 § 11.2.1), 이 범주에 속하지 않는다는 판정은 쓸모없다는판정이 아니다. 마지막으로 판별식 자체가 옳은가도 이 장에서 다시논증하지 않는다. 네 조건의 필요성과 다섯 단계 적용 절차는 ch11이 세웠고, 이 장은 그 절차의 결과가 무엇을 뜻하는지를 판정한다.

경계를 이렇게 좁히는 이유가 있다. 이 책의 원 질문은 “sleep-time compute은 유망한가”였다. 그 질문에 답하려면 질문의 주어가 무엇을가리키는지 먼저 정해져야 하고, 그 다음에 이득과 비용이 같은 자로 재져야한다. ch11이 첫 번째를 위한 증거를 냈고, 이 장이 그 증거를판정으로 바꾼다. 두 번째는 ch25·ch26이 한다. ch27은 이 장과 그 두 장 위에 선다 — 이 장이 무너지면 ch27의 질문 자체가형태를 잃는다.

24.2 증거

이 판정이 닫는 증거는 두 종류다. Part II가 원문 대조로 확립한 사실과, 이책이 직접 돌린 감사·회계·시뮬레이션이다. 두종류의 등급이 다르므로 표에서 갈라 적는다. 이 책의 실험은 어느 것도 GPU 측정이 아니며, 무엇을 단정문으로 쓸 수 있고 무엇을 방향성으로만 쓸 수있는지는 각 실험이 스스로 규정한 계약을 따른다.

표 24-1 — 이 판정이 닫는 증거

#	증거	등급	출처
A1	판별식 전수 적용 결과 A 군 9 편 + 경계 사례 1 편 (MemGPT)·B 군 9 편·C 군 10 편으로 29 편이갈린다. 통과 9 편의 경로 분포는 $E 6 / \Theta 2 / W 1$ 이며, 경계 사례를 더하면 E 쪽만 7 이 된다	Part II 확립	ch11 표 11-1·11-2·11-3
A2	어긋남은 양방향이다. B 군 다수가 이 이름 아래 함께 인용되고, A 군의 SCM 은 제목이 Θ 를 가리키는데 가중치를 한 번도 건드리지 않는다	Part II 확립	ch11 § 11.4.1
A3	분류의 기준은 산출물의 모양이 아니라 (갱신식, 시계) 의 쌍이다. 모양으로 나누면 Generative Adapter 가 Θ -경로로 들어온다	Part II 확립	ch11 § 11.4.5, ch16 § 16.4
A4	읽기식을 바꾼 논문은 상각 대상이 아니다. Memory Caching 의 무거운항은 N_q 가 커져도 줄지 않는다	Part II 확립	ch18 § 18.5
A5	갱신 주파수 연속체는 표기의 다라이지 기제의 다리가 아니다	Part II 확립	ch18 § 18.4
A6	이 라인의 이름을 만든 논문에서 sleep 은 consolidation 이 아니라 유희를 뜻한다. 참고문헌 22 편에 생물학·CLS 문헌이 0 편이다	Part II 확립	ch23 § 23.8, ch11 § 11.6
A7	learned context 의 'learned' 는 gradient 를 뜻하지 않는다. 논문 본문이 스스로 그렇게 쓴다	Part II 확립	ch14 § 14.3
A8	선택된 침묵이 네 건이고, 같은 경로 안에서 도 창시 논문이 인용되지 않는다. 세 경로를 모두 읽은 문헌은 기제 논문이 아니라 이론 쪽 한 편이다	Part II 확립	ch11 § 11.6.1, § 11.7

#	증거	등급	출처
A9	<i>E</i> -경로에서 품질 1 위 가 반복적으로 기억 시 스템 없는 구성이다 — Table 2의 J 순위에서 full-context 72.90% 가 Mem0 66.88%를 이긴다 [Mem0 Table 2]	Part II 확립	ch15 § 15.7
A10	이 이름을 만든 논문의 이득은 높은 test-time 예산에서 두 과제 모두 역전하고, sleep 스케 일링이 비단조다 — 병 렬 5 벌이 대체로 10 벌 보다 낫다 [Letta STC § 5.1, § 5.2]	Part II 확립	ch14 § 14.6
A11	경계 사례 규칙은 계보 적 조상과 범주 소속을 분리한다. MemGPT 는 조건 (1) 만 불충족 이고 통과군이 조상으 로 지목하므로 <i>E</i> -경로 의 출발점으로서 술하 되 통과군으로 세지 않 으며, RAG는 탈락 조 건이 하나여도 조상 지 목이 없으므로 경계 사 례가 아니다	Part II 확립	ch11 § 11.5, ch13, ch10
A12	표현 수준 <i>E</i> 는 계산을 압축하고 저장을 압축 하지 않는다. 그리고 이 논문에는 상각할 것이 없는데 문서 길이 분포 는 있고, 상각을 주장 하는 뒤의 논문들에는 그 분포가 없다	Part II 확립	ch12 § 12.5, § 12.7

#	증거	등급	출처
B1	이 corpus 29 편 중 B_s 가 absent 인 논문이 19 편이고, 절대 FLOPs 또는 토큰으로 보고한 논문은 1 편이다. 나머지는 optimizer step 5 편 · 벽시계 2 편 · 눈금 없는 서수 knob 2 편이다. MFU 또는 달성 throughput 을 보고한 논문도 1 편이어서 step · 벽시계를 FLOPs 로 옮길 다리가 없다	본서 감사 (측정 아님, corpus 29 편 한정)	→ ch28
B2	그 19 편 중 9 편은 “보고하지 않은” 것이 아니라 질의 전오프라인 pass 자체가 없어 B_s 가 정의되지 않는 경우이며, 그 집합이 ch11 표 11-2 의 B 군과 사실상 일치한다	본서 감사 (측정 아님)	→ ch28
B3	B_s 보고율의 경로 간 순서는 $\Theta > E$ 다. 원인은 기제가 아니라 관행 — Θ -경로는 학습 논문의 보고관습을, E -경로는 시스템 · 제품 논문의 관습을 물려받았다. (표본이 작아 순서만 인용 가능하며 비율은 쓰지 않는다)	본서 감사 (순서만 단정)	→ ch28
B4	B_s 축 위에 두 점 이상을 놓은 논문은 4 편이고 그중 점이 절대예산 단위인 논문은 2 편이며, 그 부분집합이 공유하는 벤치마크의 교집합은 공집합이다	본서 감사 (측정 아님)	→ ch28
B5	corpus 전체에서 상태 용량 C_{cap} 을 바이트로 보고한 논문이 0 편이다. 세 경로를 같은 자로 재려면 구성 파라미터에서 복원하는 수밖에 없다	본서 회계 (측정 아님)	→ ch26

#	증거	등급	출처
B6	세 경로의 사용자당 · 세션당 상태량은 자릿수가 다르다 . 순서와 자릿수는 단정 가능하고 절대 바이트는 아니다	본서회계 (자릿수 · 순서만)	→ ch26
B7	상각의 여유는 레짐마다 다르고 그 순서 는 미보고 값의 선택에 둔감하다. 순서는 단정 가능하고 임계의 절대치는 아니다	본서회계 (순서만)	→ ch25
B8	반복 sleep 라운드에서 과제 지표는 한동안 움직이지 않다가 임계로 무너지는 반면 저장은 1 라운드부터 깎인다. 그래서 이송해야 할 양은라운드당 비율이 아니라 성능이 기준선을 유지하는 마지막 라운드 다 (→ ch29 가 이 양에 기호를 준다). 부호와 모양만 이송하며 라운드수 · 비율은 이송하지 않는다	본서 시물 (장난감 모형, 부호 · 모양만)	→ ch29
B9	열화를 라운드의 함수로 적은 논문은 corpus 에 두 편뿐이고 그 둘은형태조차 서로 다르다 (달린 형태대 유지 곡선). 생성 분포의 퍼짐을 라운드함수로 보고한 논문은 0 편이다	본서 감사 (측정 아님)	→ ch29, ch30

[해설] 표의 위 절반과 아래 절반은 성격이 다르다. A 군 항목은 논문 원문에서 읽히는 사실이고, 반증하려면 원문을 다시 읽으면 된다. B 군 항목은 이 책이 만든 것이고, 반증하려면 이 책의 절차를 다시 돌리거나 더 나은 절차를 제시해야 한다. B1 – B4 와 B9 는 문헌의 상태를 **세는** 감사이므로 성능도 비용도 재지 않았고, B5 – B7 은 구성 파라미터에서 바이트와 임계를 **복원하는** 회계이므로 wall-clock 도 청구액도 아니며, B8 은 언어 모델 실험이 아니라 장난감 모형이다. 세 등급 전부에서 이 책이 본문 단정문으로 올리는 것은 **비율 · 순서 · 자릿수 · crossover 의 위치**이고, 절대치는 하한 또는 방향성으로만 쓴다.

등급 규칙을 지킨다는 것은 쓸 수 있는 것을 고르는 일이 아니라 **쓰지 않기로 한 것을 밝히는** 일이다. 이 장이 손에 쥐고 있으면서 본문에 올리지 않는 값이 넷 있다. B, 보고율의 경로별 **비율** — 순서만 쓰고 비율은 쓰지 않는다. 표본이 한 자릿수인 경로가 둘이라 비율의 분모가 신뢰구간을 감당하지 못한다 (B3). 사용자 10^6 명 규모의 웹 스토리지 **절대 바이트** — 자릿수와 순서만 쓴다. 구성 파라미터에서 복원한 회계이지 측정이 아니다 (B6). 상각 임계의 **절대 토큰 수** — 레짐 간 순서만 쓴다. 그 값의 입력 중 여럿이 원 논문에 없어 이 책이 가정한 것이다 (B7). 반복 라운드에서 과제 지표가 무너지는 **라운드수와 라운드당 감쇠율** — 부호와 모양만 쓴다. 장난감 모형의 산물이며 언어 모델 실험이

아니다 (B8). 넷 다 인용하면 문장이 훨씬 강해지지만, 그렇게쓰면 이 책이 논문들에 요구한 기준을 스스로 어기는 것이 된다. **비용 칸이 비어 있는 논문에 대해 “효율적” 이라고 쓰지 않는다는 규칙은 이 책의 실험에도 똑같이 적용된다.**

A 군 열두 항을 논지의 순서로 다시 묶으면 셋이 된다. **분류의결과가 A1 – A3 이고, 분류 기준이 자의적이지 않다는 증거가 A3 – A5·A11 – A12 이며, 이름이 기제와 갈라진 흔적이 A6 – A10 이다.** 가운데 묶음이 이 장에서 특히 무겁다. 판별식이 원하는 답을 얻기 위해 사후에 재단된 것이라면, 경계 근처의 판정이 이 책의 논지에 유리한 쪽으로 쏠려야 한다. 실제 판정은 그렇지 않다 — MemGPT 는 이 책이 서술의 출발점으로 삼고 싶은 논문인데도 통과군으로 세지 않고 (A11), Memory Caching 은 *W* 층 용량 천장의 차수를 공급하는 값진 논문인데도 범주에서 뺏으며 (A4), Generative Adapter 는 산출물이 이 책의 Θ -경로 서사에 가장 잘 맞는데도 시계가 wake 라는 이유로 뺏다 (A3). 세 판정 모두 편의와 반대로 갔다. [본서 판단]

증거 중 하나는 다른 것들과 지위가 다르므로 따로 세운다.

[평가] B2 는 독립 확증이다. ch11 의 분류는 논문의 갱신식과 시계를 읽어서 나왔고, B_s 감사는 각 논문의 비용 칸 하나만 보고 “질의전 오프라인 pass 가 존재하는가” 를 기계적으로 판정했다. 두 절차는 입력이 다르고 서로를 참조하지 않는다. 그런데 결과 집합이 사실상 겹친다 — 9 편이 같은 자리에 떨어진다. 이것은 판별식이 논문의 서술에서 읽어 낸 구분이 **비용 보고의 구조에도 새겨져 있다**는 뜻이다. 어떤 논문이 이 범주가 아니라는 판정과, 그 논문에는 sleep 예산이라는 칸 자체가 없다는 관측은 같은 사실의 두 얼굴이다.

24.3 분석

24.3.1 간격은 우연이 아니다

이름과 기제의 어긋남이 단순한 정리 부족이라면, 어긋남의 분포가 무작위여야 한다. 오래된 논문일수록 용어가 흔들리고 최근 논문일수록 수렴하거나, 특정 저자 그룹에서만 튀거나, 어느 경로에서든 비슷한 빈도로 나타나야 한다. 실제 분포는 그렇지 않다. 어긋남은 **경로를 따라 정렬되어** 있고, 정렬의 축은 기술이 아니라 공동체다.

세 갈래를 이름부터 갈라 적는다. *E*-경로의 대화 상대는 **에이전트 기억** 공동체다 — 경쟁 제품, 대화 벤치마크, 검색 지연과 정확도가 어휘를 이룬다. *W*-경로의 대화 상대는 **test-time learning** 공동체다 — 고정 크기 상태, 재귀, recall – throughput 트레이드오프가 어휘다. Θ -경로의 대화 상대는 **지속 학습** 공동체다 — 망각, replay, optimizer step, 성능 행렬이 어휘다. 세 어휘는 겹치는 낱말이 거의 없다.

표 24-2 — 같은 이름을 쓰는 세 공동체 (경로별 관행. 근거는 표 24-1 의 A8·B1·B3·B4)

	<i>E</i> -경로	<i>W</i> -경로	Θ -경로
물려받은 서식	시스템 · 제품 논문	아키텍처 논문	학습 논문
비용을 적는 칸	지연 · 정확도 (쓰기 비용 칸 없음)	상태 크기 · throughput	step · batch · GPU-hour
성공의 정의 [본서 판단]	프롬프트 토큰 절감, 정확도 유지	고정 상태에서의 recall	망각 없는 신규 습득
대조군의 관례 [본서 판단]	경쟁 제품, full-context	같은 폭의 아키텍처 변형	사전 학습 체크포인트
B_s 보고	대체로 없음	서수 knob	step 또는 벽 시계
이 이름을 쓰는가	창시 논문만, 후속은 쓰지 않음	제목에 쓰되 정의는 자기 것	제목에 쓰되 정의는 자기 것

W-경로 열은 통과군 표본이 1 편이므로 관행이 아니라 그 한 편의 서술이다. 세 열을 나란히 읽을 때 이 비대칭을 함께 읽는다. 그리고 여섯행의 지위가 같지 않다 — 「물려받은 서식」 · 「비용을 적는 칸」 · 「 B_s 보고」 · 「이 이름을 쓰는가」 는 A8·B1·B3·B4 에 근거가 걸려 있고, 「성공의 정의」 ·

「대조군의 관례」 두 행은 각 논문의 평가절을 읽고 이 책이 요약한 것이어서 전수 계수가 뒤에 없다. 그래서 두행에는 마커를 달았고, 이어지는 논증은 두 행에 단독으로 기대지 않는다.

표의 마지막 행이 이 절의 논지를 압축한다. 세 경로는 같은 낱말을 쓰지만 그 낱말이 각 경로에서 지시하는 절차가 다르고, 정의를 서로에게서 가져오지 않는다. 그리고 네 번째 행이 그 상태를 지속시키는 기제다 — 대조군의관례가 다르면 상대 경로의 방법은 애초에 실행할 대상으로 인식되지 않는다. E -경로 논문에게 자연스러운 대조군은 full-context 와 경쟁 제품이지 LoRA delta 가 아니고, Θ -경로 논문에게 자연스러운 대조군은 사전학습 체크포인트이지 벡터 데이터베이스가 아니다.

이 갈라짐이 사회적 사실이라는 증거는 **보고 관행**에 있다. B_s 를 무엇으로든 보고한 비율의 경로 간 순서가 $\Theta > E$ 다 (B3). 그리고 그 순서의 원인은 기제가 아니다 — Θ -경로가 계산을 더 많이 쓰기 때문에 세는 것이 아니라, Θ -경로 논문이 학습 논문의 서식을 물려받아 step·batch·GPU-hour 칸을 애초에 갖고 있기때문이다. E -경로 논문은 시스템·제품 논문의 서식을 물려받았고 그 서식에는 **쓰기 비용을 적는 칸이 없다**. 같은 연산을하는 두 논문이 다른 칸을 채우는 것은 기술적 선택이 아니라 서식의상속이다. [본서 추론]

두 번째 증거는 침묵의 방향이다. 침묵이 연대 때문이라면 뒤에 온 논문이 앞선 논문을 인용해야 하는데, 실제로는 반대 방향의 공백이 반복된다 (A8). 이 이름을 만든 논문은 8개월 뒤 다른 경로 논문에게 인용되지 않고, 5개월뒤 **같은 경로**의 후속 논문에게도 인용되지 않으며, 그 후속논문에는 “sleep” 이라는 단어가 한 번도 나오지 않는다 ($\rightarrow$ ch11 § 11.7). 한건이면 누락이지만 반복되면 검색어의 부재다. 그리고 세 경로를 모두 읽은문헌이 기제 논문이 아니라 이론 쪽 한 편뿐이라는 사실이 그 부재를확증한다 — 통합적으로 읽으려면 세 어휘를 모두 아는 자리에 서야 하고, 그자리는 기제를 만드는 자리가 아니다.

세 번째 증거는 이름 자체의 이동 경로다. **이름은 생물학에서 왔고논증은 오지 않았다** (A6). 이 라인의 이름을 만든 논문에서 sleep 은 consolidation 이론의 용어가 아니라 “상호작용 사이에 모델이 노는 시간” 을가리키는 조작적 낱말이고, 참고문헌 22 편에 CLS 도 hippocampal replay 도 없다 ($\rightarrow$ ch23 § 23.8). 그 뒤 이 단어가 다른 두 경로로 퍼졌지만, 퍼진 것은단어이지 그 단어가 원래 지시하던 두 시스템 논증이 아니다. 그래서 같은이름을 쓰는 세 논문이 같은 가설의 변형이라고 읽을 근거가 없다.

[**평가**] 세 증거가 같은 방향을 가리킨다. 어긋남은 문헌이 아직정리되지 않아 생긴 것이 아니라, **정리될 통로가 없어서** 유지되는상태다. 통로가 없다는 것은 물리적 은유가 아니다 — 공유 벤치마크의교집합이 공집합이고 (B4), 상태 용량 C_{cap} 을 바이트로 적은논문이 0 편이며 (B5), 예산을 절대 단위로 적은 논문이 1 편이다 (B1). 세경로가 서로를 읽기 시작하더라도 나란히 놓을 축이 없다. 그러므로 인간적은 시간이 해결하는 종류가 아니라 **누군가 축을 세워야 닫히는종류**다. 이 책이 판별식과 회계를 먼저 세우는 이유가 그것이다.

24.3.2 간격이 실무에 하는일

간격을 방치하면 세 가지 판단이 틀린다. 셋 다 문헌의 결함이 아니라**문헌을 읽는 쪽의 결함**이며, 그래서 이 절이 이 책에서 실무에 가장직접 닿는 자리다.

잘못된 비교. 같은 이름 아래 놓인 두 수치가 같은 축의 값이아니다. 이 이름을 만든 논문의 $5\times$ 는 **생성 토큰 축**의 수평 이동이고 prefill 항을 세지 않는다 ($\rightarrow$ ch14 § 14.5). Θ -경로가 보고하는예산은 optimizer step 이고, 그 step 을 FLOPs 로 옮기려면 MFU 나 달성 throughput 이 필요한데 그것을 보고한 논문이 이 corpus 29 편 중 1 편이다 (B1). 읽기식을 바꾼 논문의 비용은 애초에 상각 대상이 아니어서분모 N_q 를 키워도 줄지 않는다 (A4). 세 값은 단위가 다른 정도가 아니라**회계의 자리 자체가 다르다**. 그런데 이름이 같으므로 나란히놓인다. 이 책의 상각 회계가 확인한 것은 레짐 간 여유의 순서가미비고 값의 선택에 둔감하다는 것이고 (B7), 순서가 있다는 사실은 곧 같은방법이 한 레짐에서는 여유가 크고 다른 레짐에서는 이득이 사라지는 경계에붙어 있다는 뜻이다. 그 회계는 어느 레짐에서도 원 논문의 보고를 반박하지않는다 — 반박하는 것은 레짐을 지정하지 않은 채 그 수치를 옮기는 일이다. 이름만으로는 어느 레짐인지 알 수 없다.

이 실패가 가장 선명하게 드러나는 자리가 상각식 (A) 의 분모다. (A) 를쓰려면 N_q — 한 문맥을 공유하는 질의 수 — 의 분포를 알아야 하는데, 상각을 주장하는 논문 중 그 분포를 보고한 것이 없다. 그런데 그 분포를실제로 인쇄한 논문은 corpus 에 있다. 판별식에서 탈락해 이 범주가 아닌논

문, 즉 상각할 것이 애초에 없는 논문이 다섯 corpus의 문서 길이히스토그램을 싣고 있다 (A12). **상각을 주장하는 쪽에는 분모가 없고, 분모를 가진 쪽에는 상각할 것이 없다.** 이 어긋남은 우연이 아니라 § 24.3.1 이 말한 서식 차이의 귀결이다 — 데이터 분포를 싣는 것은 모델링논문의 관례이고, 비용 절감을 주장하는 것은 시스템 논문의 관례이며, 두 관례가 한 논문 안에서 만나지 않는다.

잘못된 기대. 이름이 consolidation 을 약속하므로 독자는 “sleep 라운드를 늘릴수록 좋아진다” 를 기대한다. 문헌은 그 기대를 지지하지 않는다. 이 이름을 만든 논문에서 sleep 스케일링은 비단조이고 — 병렬 5 벌이대체로 10 벌보다 낮다 — 이득은 높은 test-time 예산에서 두 과제 모두 역전하며 그 기제는 제시되지 않는다 (A10). *E*-경로에서는 자기 표의 품질 1 위가 반복적으로 기억 시스템 없는 구성이다 (A9). 반복 라운드의 열화를라운드의 함수로 적은 논문은 corpus 에 두 편뿐이고 그 둘은 형태조차 서로 다르며 (B9), 이 책의 반복 consolidation 시물에서 나온 모양은 기대와 반대방향이다 — 과제 지표는 한동안 전혀 움직이지 않다가 임계로 무너지는데 그동안에도 저장은 1 라운드부터 깎이고 있다 (B8, 부호 · 모양만). [본서 판단] sleep 라운드를 늘릴수록 좋아진다는 기대보다 위험한 것은 **손실 곡선이 평평하니 안전하다**는 기대다. 평평한 1 차 지표아래에서 무너지는 좌표가 따로 있다는 것이 그 시물이 보인 모양이며, 이함의는 ch29 가 운영 지표로 옮긴다.

잘못된 투자 판단. 세 경로는 서빙 아티팩트가 다르고, 그 차이가 인프라 결정을 통째로 가른다. *E*-경로는 사용자별로 움직이는 파라미터가없으므로 shared-weight batching 이 온전하고 상태는 데이터베이스 행과프롬프트 토큰에 산다 (→ ch13 § 13.7). *Θ*-경로는 사용자별 delta 가생기는 순간 그 batching 이 깨지고, 기회의 성격이 HBM 대역폭이 아니라 **요청 지연 안에 delta 를 끌어올 수 있는 웹 계층과 그 계층의 쓰기내구성**으로 옮겨 간다 (B6). *W*-경로의 상태는 활성 세션당으로 붙고자릿수가 위 둘과 다르다. 같은 *Θ*-경로 안에서도 갈림길이 하나 더있다 — 직접 편집 방식은 delta 아티팩트를 만들지 않고 공유 가중치를고치므로 사용자별 개인화가 구조적으로 불가능하고, 하려면 모델 전체사본이 필요하다 (→ ch19, ch26). “sleep-time compute 에 투자한다” 는 문장은 이 서로 다른 결정 셋을 하나로 뭉갠다. 조달 대상이 벡터 데이터베이스인지, 웹 스토리지 계층인지, 세션당 상주 메모리인지가 그 문장 안에서 결정되지 않는다.

[해설] 세 실패의 공통 구조는 하나다. 이름은 **무엇을하는가**를 가리키는데, 비교 · 기대 · 투자는 전부 **무엇이 바뀌는가**에 달려 있다. 판별식이 하는 일이 정확히 그 전환이다 — 절차를 (갱신식, 시계) 의 쌍으로 환원하면 바뀌는 것이 *Θ* 인지 *W* 인지 *E* 인지가 먼저 정해지고, 그 다음에야 비용 · 용량 · 롤백 단위 · batching 성립여부가 따라 나온다 (→ ch11 § 11.4.5 의 [해설]).

실무자의 어휘로 옮기면 판별식은 다음 순서의 질문표가 된다. 어느 층이바뀌는가 — 답이 *E* 이면 상태는 데이터베이스에 살고 batching 이 온전하며롤백은 행 삭제다. 답이 *Θ* 이면 배포 아티팩트가 바뀌므로 delta 형태인지 직접 편집인지를 다음으로 묻고, delta 라면 사용자당 몇바이트인지와 그것을 어느 계층에서 끌어오는지를 묻는다. 답이 *W* 이면상태가 세션에 붙으므로 동시 세션 수가 곧 상주 메모리다. 시계는 언제인가 — 질의 전이면 L_w 에 0 을 기여하고, 턴 안이면 그 비용은 사용자가기다린다. 예산은 얼마인가 — 이 칸이 비어 있으면 상각 논증을 검증할 수없으므로 그 방법에 대해 “효율적” 이라고 쓰지 않는다. 네 질문 모두 논문의제목으로는 답할 수 없고 갱신식과 시계로만 답할 수 있다는 것이 이 장이판별식을 판정의 전제로 세우는 실무적 이유다.

24.3.3 판별식 없이는 “유망한가” 에 답할 수 없다

이 절이 ch27 의 토대다. 판별식을 쓰지 않고 “sleep-time compute 은유망한가” 에 답하려 하면 세 지점에서 막힌다.

첫째, **주어가 정해지지 않는다.** 질문의 대상을 이 이름을 쓰는 문헌전체로 잡으면 그 안에 A 군과 B 군이 섞이고, B 군에는 배포 후 어떤 층도바뀌지 않는 논문, 읽기식만 바꾼 논문, 시계를 아예 갖지 않은 쓰기 primitive 가 함께 들어온다 (→ ch11 표 11-2). 이 집합의 평균적 전망을 묻는것은 답이 있는 질문이 아니다. 대상을 A 군으로 좁히면 질문은 형태를갖추지만, 그 순간 답은 하나가 아니게 된다 — A 군 안에 세 경로가 있고 셋의비용 구조가 다르다.

둘째, **표본이 경로마다 다르다.** 통과 9 편의 분포가 *E* 6 / *Θ* 2 / *W* 1 이다 (A1). *W*-경로에대한 어떤 판정도 표본 1 의 판정이며, 그 하나마저 자기 경로의 주류 계보와접점이 없다 (→ ch11 § 11.7). 세 경로에 같은 확신도를 부여하는 진술은 이분포를 무시하는 것이다.

이 두 번째 장애에는 비대칭이 하나 더 붙는다. 표본이 가장 큰 E -경로가 증거의 질까지 가장 좋은 것은 아니다. 그 경로에서 이 이름을 만든 논문의 헤드라인 수치는 저자 제작 벤치마크 위에서 나오고 이득이 역전하는 레짐이 그 논문 안에 함께 실려 있으며 (A10), 프로덕션 시스템 쪽에서는 자기 표의 품질 1위가 반복적으로 기억 시스템 없는 구성이다 (A9). [본서 추론] 그러므로 “표본이 6편이니 E -경로에 대해서는 강하게 말할 수 있다”는 추론은 성립하지 않는다. 편수와 확신도는 별개의 축이며, ch27은 둘을 따로적어야 한다.

셋째, **축이 아예 없다**. 유망함을 판정하려면 최소한 예산을 키울 때 품질이 어떻게 움직이는지를 봐야 한다. 이 corpus에서 B_s 축 위에 두 점 이상을 놓은 논문은 4편이고, 그 점이 절대 예산 단위인 논문은 2편이며, 그 부분집합이 공유하는 벤치마크의 교집합은 공집합이다 (B4). x 축을 복원해도 y 축이 붙지 않는다. 그러므로 현재 문헌으로는 B_s -품질 곡선을 그릴 수 없고, 그리는 것이 ch28의 기여가 아니라 곡선을 그리기 위한 최소 보고 요구를 명세하는 것이 ch28의 기여가 된다.

[평가] 세 장애는 성격이 다르다. 첫째는 판별식으로 즉시 해소된다 — 주어를 A 군으로 좁히고 경로를 지정하면 질문은 형태를 갖춘다. 둘째는 해소되지 않는다. 표본이 1인 경로에 대해서는 어떤 절차로도 강한 판정을 만들 수 없고, 그 사실을 밝히는 것이 유일한 정직한 처리다. 셋째는 이 책이 부분적으로만 메운다 — 통일 회계 (ch25)와 용량 회계 (ch26)는 비교의자를 만들지만 없는 데이터를 만들어 내지는 못한다. 따라서 ch27의 판정은 **경로별이고, 경로마다 확신도가 다르며, 일부 칸은 “현재 문헌으로 판정 불가”로 남는다**. 그 형태가 회피가 아니라 증거가 허용하는 최대치다.

24.4 판정

[평가] 이 장의 판정.

1. **이 범주는 A 군 9편이다**. 판별식 네 조건을 모두 만족하는 절차만 sleep-time compute이며, B 군 9편은 인접 문헌이지 이 범주가 아니다. 경계사례 1편 (MemGPT)은 조건 (1)을 만족하지 않으므로 E -경로의 출발점으로서 쓸 수 없고 통과군에 세지 않는다 (A11). 이 판정은 조건부가 아니다. 논문의 자기 규정과 어긋나는 자리가 여럿 있고, 어긋나는 자리에서는 이 책의 판정을 따른다 — 제목이 Θ -경로를 가리키면서 가중치를 한 번도 건드리지 않는 논문은 E -경로이고, 산출물이 LoRA 모양인데 시계가 wake인 논문은 이 범주가 아니다.

2. **이름과 기제의 간격은 구조적이다**. 우연한 지체가 아니다. 근거는 셋이다 — 보고 관행이 경로를 따라 갈라지고 그 갈라짐의 원인 기제가 아니라 서식의 상속이라는 것 (B3), 침묵이 연대가 강제하지 않은 방향으로 반복된다는 것 (A8), 그리고 이름이 생물학에서 옮겨 올 때 논증의 흐름에 오지 않았다는 것 (A6)이다. **조건**: 이 판정은 이 책이 원문 대조로 읽은 29편에 대한 것이다. 문헌 전체에 대한 표본이 아니므로 “이 분야의 몇 %”라는 형태로 일반화하지 않는다.

3. 그러므로 **“sleep-time compute은 유망한가”는 문헌 전체에 대해서는 형태를 갖추지 못한 질문이다**. 이것은 수사가 아니라 절차적 판정이다. 주어가 A 군과 B 군을 섞고, A 군 안에서도 경로마다 비용 구조가 다르며, 경로를 지정해도 예산 축과 품질 축이 둘 다 깨져 있다 (B1, B4). 질문이 답할 수 있는 형태를 갖추려면 **경로를 지정하고, 통일 회계를 세우고, 판정 불가 칸을 판정 불가로 남기는 세 단계가 선행해야 한다**.

4. **판별식은 이 범주에서 배제된 문헌을 폐기하지 않는다**. B 군과 C 군은 자리를 옮겨 증거가 된다 — Θ 층 용량의 실측 공급원, W 층 용량 천장의 차수, 자기생성 데이터의 제약, 반증 축. 분류의 목적은 배제가 아니라 각 문헌을 자기가 실제로 증거가 되는 자리에 놓는 것이다 ($\rightarrow$ ch11 §11.8).

판정하지 않는 것. 어느 경로가 우월한가 (ch27), 각 경로의 비용이 얼마인가 (ch25·ch26), B_s 를 키우면 품질이 어떻게 움직이느냐 (ch28)는 이 장의 판정에 들어 있지 않다. 이 장이 낸 것은 그셋을 물을 수 있는 자격이다.

24.5 반증 조건

위 네 판정은 각각 다른 관측으로 무너진다. 어느 것이 무너지면 무엇을 고쳐야 하는지까지 적는다.

F1 — 분류가 틀렸음을 보이는 관측. 판별식 네 조건을 모두만족하는데 이 책이 B 군에 넣은 절차가 있으면 판정 1 이 틀렸고, ch11 표 11-2의 해당 행을 고쳐야 한다. 가장 가까운 후보가 이미 지목되어 있다 — Memory Caching 이 사후 학습 변형을 논하는 대목이다. 그 변형에 예산과수치가 붙고 그 절차가 실제로 (U-W) 또는 (U- Θ) 를 질의 전에돌리는 것으로 확인되면, 이 책의 (2)·(3) 탈락 판정은 그변형에 대해서는 성립하지 않는다 (→ ch18 § 18.5). 반대 방향도 같다 — A 군의 어느 절차가 실은 조건 (1) 이나 (4) 를 어긴다는 것이 원문에서 확인되면 표 11-1 을 고친다.

F2 — 조건 하나가 잉여임을 보이는 관측. 네 조건 중 하나를 제거해도 A 군과 B 군의 원소가 하나도 바뀌지 않으면 그 조건은 이 corpus 에 대해 잉여이고, 판별식은 세 조건으로 단순화되어야 한다. 현재는 어느조건을 빼도 매번 다른 논문이 잘못 들어오므로 넷이 모두 일한다 (→ ch11 § 11.2.3). 이 반증은 corpus 를 넓히면 언제든 다시 검사할 수 있다.

F3 — 간격이 구조적이지 않음을 보이는 관측. 판정 2 가 틀렸다면 다음 셋 중 하나 이상이 관측되어야 한다. (a) E -경로 논문이 W 또는 Θ 방법을 **baseline** 으로 실행한다 — 한 문장 언급이나 추상화 수준에 근거한 기각이 아니라 같은 표 안에서 돌린 수치여야 한다. (b) 세 경로가 공유하는 벤치마크의 교집합이 공집합에서 벗어난다. (c) B_s 를 절대 단위로 보고하는 논문이 소수에서 다수로 바뀐다. 셋 중 어느것이라도 나타나면 간격은 사회적 구조가 아니라 시간이 해결하는 지체였다는 뜻이고, ch27의 경로별 판정은 통합 판정으로 대체되어야 한다.

F4 — “축이 아니라 각주” 라는 판정의 반증. B_s 를 FLOPs 또는 토큰으로 보고하고 같은 벤치마크 위에 두 점 이상을 놓은 논문이 나오면 B1·B4가 힘을 잃는다. 그런 논문이 세 편만 모여도 B_s -품질 곡선의 첫 조각이 그려지고, 그러면 ch28의 기여는 보고 요구명세에서 곡선 자체로 바뀐다. 이 반증은 가장 쉽게 일어날 수 있는 것이며, 이 책은 그것이 일어나기를 요구하는 쪽에서 있다.

F5 — 경로 간 차이가 서버에서 사라짐을 보이는 관측. § 24.3.2의 투자 판단 논증은 세 경로의 아티팩트가 실제로 다른 인프라를 요구한다는 전제 위에서 있다. 사용자별 delta 를 웹 계층에서 끌어오는 지연이 E -경로의 검색 지연과 같은 자릿수임이 실측되면 그 전제가 약해진다. 이 책의 용량 회계는 구성 파라미터에서 복원한 회계이지 측정이 아니므로 (B5·B6), 이 반증은 회계로는 달하지 않고 실제 서버스택의 측정을 요구한다. 그 측정이 나오기 전까지 § 24.3.2의 투자 판단 논증은 자릿수 논증으로만 서 있다.

F6 — 용어가 수렴함을 보이는 관측. 이후 논문들이 ‘sleep-time compute’ 을 판별식과 같은 뜻으로 정의해 쓰기 시작하면 이름과 기제의 간격은 해소된다. 관측 방법은 기계적이다 — 새 논문의 정의문이 네 조건 중 몇 개를 명시하는지 센다. 시계와 층을 함께 밝히는 정의가 표준이 되면 이 장의 판정 2와 3은 역사적 진술로 물러난다.

여섯 조건은 검사 비용이 서로 다르다. F1·F2는 원문과 표 11-1·11-2만 있으면 책상에서 판정된다. F3·F4·F6은 새 논문을 기다려야 하므로 관측 창이 열려 있고, 세 조건 모두 세는 방법이 이 책에 적혀 있으므로 후속관측자가 같은 절차를 다시 돌릴 수 있다. F5만 이 책의 도구로 달하지 않는다 — 회계는 자릿수를 주고 측정은 주지 않기 때문이며, 그 공백을 이책이 메우지 않고 남긴다는 사실 자체가 § 24.3.2의 투자 판단 논증의 강도에 상한을 건다.

반증되지 않는 것. 개별 논문의 성능 수치가 재현 실패로 내려가거나 후속 연구로 올라가도 이 장의 판정은 바뀌지 않는다. 판별식은 성능을 묻지 않기 때문이다 (→ ch11 § 11.2.1). 마찬가지로 A 군 논문의 이득이 전부 사라져도 그 논문들은 여전히 이 범주에 속한다 — 그때 바뀌는 것은 ch27의 판정이 이 장의 분류가 아니다. 분류와 평가를 같은 관측으로 무너뜨릴 수 없다는 것이 이 두 장을 나눈 이유다.

요약

- 이 범주는 A 군 9 편이다. 경계 사례 1 편은 통과군에 세지 않고, B 군 9 편은 인접 문헌이며, 어긋남은 통과·불통과 경계에서만 아니라 경로로 배정에서도 생긴다.
- 이름과 기제의 간격은 구조적이다. 세 공동체 — 에이전트 기억·test-time learning·지속 학습 — 가 각자의 어휘와 서식을 유지한 채 같은 단어를 쓴다.

- 그 구성성의 가장 강한 증거는 보고 관행이다. B_s 보고율의 경로 간순서는 기제가 아니라 서식의 상속에서 나오며, E -경로의 서식에는쓰기 비용을 적는 칸이 없다.
- 판별식의 분류와 B_s 보고 감사는 서로 다른 입력에서 같은 9편을지목한다. 범주 밖이라는 판정과 sleep 예산 칸이 없다는 관측은 같은사실의 두 얼굴이다.
- 이름은 생물학에서 왔고 논증은 오지 않았다. 같은 이름을 쓰는 세 경로가같은 가설의 변형이라고 읽을 근거가 없다.
- 간격을 방지하면 세 판단이 틀린다 — 회계의 자리가 다른 수치를 나란히놓는 비교, sleep 라운드를 늘릴수록 좋아진다는 기대, 서로 다른 인프라결정을 하나로 뭉개는 투자 판단.
- 따라서 “유망한가” 는 문헌 전체에 대해서는 형태를 갖추지 못한 질문이다. 경로를 지정하고 통일 회계를 세워야 답할 수 있는 형태가 되며, W -경로에 대한 어떤 판정도 표본 1의 판정이다.
- 이 판정은 여섯 가지 관측으로 반증된다. 그중 가장 쉽게 일어날 수 있는것은 B_s 를 절대 단위로 보고하는 논문의 출현이고, 이 책은 그것이일어나기를 요구하는 쪽에 서 있다.

다음 장으로

이 장은 대상을 확정했고 질문의 형태를 고쳤다. 남은 것은 자를 만드는일이다. 판정 3이 요구한 세 단계 중 첫째 — 경로 지정 — 는 이 장이 끝났고, 둘째 — 통일 회계 — 가 다음 두 장의 몫이다.

ch25는 비용의 자를 만든다. 상각식 (A)가 실제로 성립하는 범위를 레짐별로계산하고, 이 이름을 만든 논문의 5×가 생성 토큰 축의 관측에서 비용 축의결론으로 건너가려면 무엇을 통과해야 하는지를 회계로 적는다. 그 회계가확정하는 것은 임계의 절대치가 아니라 레짐 간 순서다. ch26은 용량의 자를만든다 — corpus 전체가 바이트로 보고하지 않은 상태량을 구성 파라미터에서복원해 세 경로를 같은 축에 올리고, 그 자릿수 차이가 서빙 구조의 어느결정에 걸리는지를 못 박는다. 두 자가 서고 나야 ch27이 경로별로 답할 수있다.

ch25. 통일 비용회계

25.1 판정할 질문

세 경로를 같은 자로 재면 무엇이 보이는가. 그리고 왜 지금은 쥔 수 없는가.

이 장은 Part II 가 스물한 번 반복해서 만난 빈칸을 한자리에 모아 판정한다. 각 논문장의 § N.5 는 같은 표를 채우려 했다 — B_s (sleep 예산), L_w (wake 지연 기여), C_{cap} (상태 용량), ρ (망각·열화율). 그리고 스물한 번 대부분의 칸에 “논문에 없음” 이라고 적었다. 개별 장에서 그것은 그 논문 한편의 정직한 기록이었다. 스물한 편을 겹쳐 놓으면 그것은 더 이상 개별 논문의 성질이 아니다.

이 질문에 답이 될 수 있는 것. 경로 간·논문 간비교가 지금 가능한지 아닌지에 대한 판정, 상각 논증이 어떤 구조적 결함을 갖는지에 대한 판정, 그리고 그 결함이 어느 레짐에서 물고 어느 레짐에서 물지 않는지에 대한 순서 판정. 이 셋은 원문 수치와 이 책의 회계만으로 결정된다.

이 질문에 답이 될 수 없는 것. 첫째, 어떤 경로가 절대적으로싼가. 자릿수를 맞춰 비교할 값이 문헌에 없으므로 이 장은 순위를 매기지 않는다. 둘째, 특정 서빙 구성에서의 실제 청구액이나 벽시계 시간. 이 장의 회계는 닫힌 형태이며 측정이 아니다. 셋째, 어느 논문의 헤드라인 수치가 거짓인가. **이 장은 [Letta STC] 의 $5\times$ 를 반박하지 않는다.** $5\times$ 는 생성 토큰 축에서 관측된 사실이고, 그 축 위에서 그것을 뒤집을 근거를 이 책은 갖고 있지 않다. 이 장이 반박하는 것은 그 축이 곧 비용 축이라는 **암묵 등식**이며, 그 등식이 어디까지 유효한지가 이 장의 실제내용이다.

한 가지를 미리 못 박는다. 비용 칸이 비어 있다는 것은 그 방법이 비싸다는 뜻이 아니다. ch01 의 회계 규약대로, 비어 있으면 비어 있다고 쓴다. 침묵을 성능 주장으로도 비용 비난으로도 메우지 않는다. 그래서 이 장의 판정은 “sleep-time compute 은 비싸다” 가 아니라 “**현재 문헌 상태에서 싼지비싼지가 결정되지 않는다**” 의 형태를 띤다. 결정되지 않는다는 것 자체가 판정이며, 그것이 반증 가능한 명제라는 것을 § 25.5 에서 보인다.

25.2 증거

25.2.1 비용 4 종의 corpus 전수집계

Part II 의 각 § N.5 가 채운 표를 논문 단위로 펼치면 스물한 편이다 — ch12 의 Memorizing Transformers, ch13 의 MemGPT, ch14 의 Letta STC, ch15 의 다섯편 (Mem0·Zep·ReasoningBank·SCM·Memini), ch16 의 Generative Adapter, ch17 의 Do LMs Need Sleep?, ch18 의 두편 (Nested Learning·Memory Caching), ch19 의 세편 (LoRA·ROME·MEMIT), ch20 의 SEAL, ch21 의 LM Need Sleep, ch22 의 Continual Facts, ch23 의 두편 (PLOS Sleep·PAD), 그리고 ch11 표 11-5 가 따로 확인한 Memory Layers.

집계 기준을 먼저 고정한다. 한 칸이 “채워졌다” 고 인정하려면 그 칸이 요구하는 **단위가 붙은 수치가 원문에 인쇄되어 있어야 한다**: B_s 는 FLOPs 또는 토큰, L_w 는 ms·TTFT·ITL, C_{cap} 은 dtype 이명시된 바이트, ρ 는 라운드당 손실률. 개수·rank·step 수·비율·차수 표기는 그 칸의 단위가 아니므로 “대용치” 로 따로 센다. 이 기준은 ch01 § 01.5 가 정한 것이며 Part II 전 장이 같은 기준으로 표를 채웠다.

표 25-1 — 비용 4 종의 corpus 집계. 대상은 Part II § N.5 와 ch11 표 11-5 가 원문 대조로 채운 21 편. [Part II 확정]

칸	요구 단위	단위대로 인쇄된 논문	대용치만 있는 논문	완전 침묵
B_s	FLOPs 또는 토큰	1 편 — ReasoningBank, 과제당 3748.4 토큰 (judge 2186.3 + extraction 1562.1) [ReasoningBank Table 5]	6 편 — 벽시계 (ROME 편집당 약 2 초, MEMIT 10,000 편집 7.44 hr → ch19 § 19.5; SEAL 라운드당 $2 \times H100$ 6 시간 → ch20 § 20.5), step 수 (Continual Facts 24·96·192 → ch22 § 22.5), 시뮬레이션시간 (PLOS 30 초 → ch23 § 23.5), 비율 (LM Need Sleep, SFT 대비 $4 \times$ → ch21 § 21.5)	14 편
L_w	ms · TTFT · ITL	0 편 . ms/token 도 TTFT 도 인쇄된 곳이 없다	5 편 — end-to-end 초 단위 (Mem0 total p50 0.708 s, Zep 31.3 s → 3.20 s), 검색 부분만 (SCM 0.3 ms 미만) — 셋다 → ch15 § 15.5; 질의당계산량 (Generative Adapter 0.505 TFLOPS → ch16 § 16.5), 구조논증 (LoRA merge 후 추가 지연 0 → ch19 § 19.5)	16 편

칸	요구 단위	단위대로 인쇄된 논문	대용치만 있는 논문	완전 침묵
C_{cap}	dtype 명시바이 트/세션 · 사용자	0 편	13 편 — 항목 개 수 (MemTrans $n_E \rightarrow$ ch12 § 12.5; SCM 개 념 24, Memini 간선, Mem0 토 큰 대용 약 7k $\rightarrow$ ch15 § 15.5), float 개수 (Generative Adapter 32M, dtype 없음 $\rightarrow$ ch16 § 16.5), rank(SEAL 128/ 64/16 $\rightarrow$ ch20 § 20.5; Continual Facts $r=16 \rightarrow$ ch22 § 22.5), 파 라미터 수 (Memory Layers 1,280 억 $\rightarrow$ ch11 표 11-5; LM Need Sleep 은 통합 1 회당 $2d d_{\text{low}}$ 와 rank 64 인데 $d \cdot d_{\text{low}}$ 의 값이 없다 $\rightarrow$ ch21 § 21.5), 시 냅스 · 슬롯 개수 (PLOS 가소성 시 냅스 6272, PAD 2 슬롯 고정 $\rightarrow$ ch23 § 23.5), 편 집 개수 (MEMIT 10,000), 그리고 단위가 다른 바이 트 (LoRA 35MB — 과제당 체크포 인트) — 뒤의 둘 은 $\rightarrow$ ch19 § 19.5	8 편 — MemGPT, Letta STC, Zep, Reasoning- Bank, Do LMs Need Sleep?, Nested Learning, Memory Caching, ROME

칸	요구 단위	단위대로 인쇄된 논문	대용치만 있는 논문	완전 침묵
ρ	라운드당 손실률	0 편	4 편 — 종점 수준 (Continual Facts $k=20$ 에서 bare 1% 대 study 46% → ch22 § 22.5; PLOS 0.70 → 0. 52, 그리고 방향 성만 있는 PAD → ch23 § 23.5), 그림만 (SEAL Fig. 6, 8 회 히트 맵 → ch20 § 20.5)	17 편

네 줄의 요지가 각각 다르다.

B_s 줄. 상각 논증의 분자가 corpus 전체에서 단 한 번 인쇄되었고, 그 한 번은 상각을 주장하지 않는 논문에서 나왔다. ReasoningBank 의 3748.4 토큰은 과제 하나의 쓰기 비용이며, 그 논문에서 각과제는 자기 자신의 문맥이므로 $N_q = 1$ 이다 (→ ch15 § 15.5). **상각을 주장하는 논문 중 분자를 숫자로 인쇄한 논문은 0 편이다.** [본서 관찰]

L_w 줄. 이 라인이 systems 독자에게 파는 성질이 정확하지 않은데, 지연의 단위로 잴 값이 없다. Do LMs Need Sleep?은 “wake-time 예측의 지연을 보존하면서 추가 계산을 sleep 으로 옮긴다” 고 초록에 쓰지만 ms/token 도 TTFT 도 없고, 유일한 throughput 그림은 캡션이 스스로 훈련 측정이라고 밝힌다 (→ ch17 § 17.5). [LoRA § 4.1]의 “추가 지연 0”은 merge 구조의 귀결이지 측정이 아니다. 즉 L_w 칸은 세 경로 모두에서 구조적 논증으로 대체되어 있다. 초단위 end-to-end 수치를 가진 셋 (Mem0·Zep·SCM)은 전부 E -경로이고, 셋 다 읽기 경로만 잰다.

C_{cap} 줄. dtype 이 붙은 바이트가 세션당 · 사용자당 단위로 인쇄된 곳이 없다. corpus 에 바이트가 등장하는 자리는 둘뿐인데 둘 다 단위가 다르다 — [LoRA § 4.2]의 35MB 는 과제당 체크포인트 크기이고, RAG 의 수십 GB 는 corpus 당 인덱스 크기다 (→ ch10, ch13 § 13.5). ch13 이 짚었듯 E -경로는 상태의 단위가 corpus 당에서 사용자당으로 바뀌는 바로 그 논문 (MemGPT) 에서 숫자를 잃는다. 이결손의 귀결은 ch26 이 받는다.

ρ 줄. 라운드당 열화율이 0 편이라는 것은 반복 sleep 의 지속가능성에 대한 정량 근거가 corpus 에 없다는 뜻이다. 그리고 그 이유가 논문마다 구조적이다 — Letta STC 는 모든 실험에서 sleep 라운드가 정확히 1 회이고 (→ ch14 § 14.5), Do LMs Need Sleep?은 W 가 시퀀스마다 0 으로 초기화되어 라운드를 넘길 상태가 만들어지지 않으며 (→ ch17 § 17.5), LM Need Sleep 은 sleep 주기수를 독립변수로 놓은 실험이 하나도 없다 (→ ch21 § 21.5). **측정되지 않은 것이 아니라 측정될 실험이 설계되지 않았다.** [본서 관찰]

네 칸이 전부 비어 있는 논문은 다섯이다: MemGPT, Letta STC, Do LMs Need Sleep?, Nested Learning, Memory Caching. 여기에 LM Need Sleep 을 더하면 여섯인데, 그 논문은 네 칸 중 절댓값이 있는 칸이 0 개이고 남은 것은 SFT 대비 상대비뿐이다 (→ ch21 § 21.5).

이 여섯의 명단이 이 절의 결론을 만든다. **세 경로의 대표 논문이 셋다 이 명단에 있다** — E -경로에 이름을 준 Letta STC, W -경로의 유일한 정면 사례 Do LMs Need Sleep?, Θ -경로에서 sleep 라운드를 정면으로 도는 LM Need Sleep (→ ch11 § 11.3, ch24). 세 경로를 같은 자로 재려고 자를 들이대면, 자를 대야 할 세 지점이 모두 비어있다. [본서 관찰]

25.2.2 손익분기 N_q^* 가 계산되는 논문은 0 편이다

식 (A) 는 이 책이 전 장에 강제한 상각식이다.

$$C_{\text{avg}} = C_{\text{wake}} + \frac{C_{\text{sleep}}}{N_q} \quad (\text{A})$$

Part II의 각 § N.5는 이 식으로 손익분기 N_q^* 를 계산하라고, 못하면 왜 못 하는지 쓰라고 요구 받았다. 스물한 편에서 **계산에 성공한장이 하나도 없다**. 실패의 이유가 한 종류가 아니라는 것이 이 절의내용이다.

표 25-2 — 식 (A)가 달하지 않는 이유의 분류. [Part II 확정]

실패 유형	뜻	해당 논문	대표 근거
I. 분자 부재	$C_{\text{sleep}} = B_s$ 가 숫자로 없다	Letta STC, MemGPT, Mem0, Zep, SCM, Memini, Generative Adapter, SEAL, LM Need Sleep, Continual Facts	Generative Adapter의 문맥화 FLOPs는 Figure 3의 눈금 없는 로그축 곡선으로만 존재하고 본문이 인용한 적이 없다 (→ ch16 § 16.5)
II. 분모 부재	B_s 는 있으나 N_q 가 없다	ReasoningBank	저장 항목이 몇 번 검색되었는지의 분포가 없다 (→ ch15 § 15.5)
III. 분자가 구조적으로 0	오프라인 pass가 없어 상각할 것이 없다	Memorizing Transformers, Nested Learning, Memory Caching, Memory Layers	비용이 커서가 아니라 비용이 놓인 자리가 다르기때문이다 . [Memory Caching]이 바꾼 것은 읽기식이고 그 부담은 토큰마다 반복된다 (→ ch18 § 18.5)
IV. 식의 모양이 경로에 맞지 않음	분모가 질의 수가 아니다	LoRA, ROME, MEMIT, LM Need Sleep	Θ-경로에서 실제분모는 한 라운드에 묶어 넣은 편집 수 이거나 (→ ch19 § 19.5) 앞으로의 전 트래픽 이다 (→ ch21 § 21.5)
V. 질의 스트림 자체가 없음	문맥·사용자·질의라는 대상이 정의되지 않는다	PLOS Sleep, PAD	에이전트가 movement cycle마다 결정 하나를 낼 뿐이다 (→ ch23 § 23.5)
VI. 상각이 아니라 재배치	$N_q = 1$ 이라 (A)가 합으로 붕괴	Do LMs Need Sleep?(GSM-Infinite 헤드라인)	wake에서 뺀 계산이 같은 요청 안의 다른 국면으로 옮겨 갈 뿐이고 요청 전체의 계산량은 늘어난다 (→ ch17 § 17.5)

LM Need Sleep은 유형 I과 IV에 함께 놓인다 — 분자가 없고, 있어도 분모가 질의 수가 아니다. 두 유형을 동시에 만족하면 값이 인쇄되어도 손익분기가 성립하지 않는다.

이 분류에서 읽어야 할 것은 실패의 **분포**다. 유형 I·II는 보고 관행의 문제이고 논문이 값을 인쇄하면 즉시 해소된다. 유형 III·IV·V·VI는 다르다 — 값을 인쇄해도 해소되지 않는다. 식 (A) 자체가 “한 문맥을 N_q 개 질의가 공유한다”는 E-경로의 모양을 전제하기 때문이고, 다른 두 경로는 그 전제를 받지 않는다. ch21이 이 사실을 처음 문장으로 만들었다 — Θ-경로에서는 분모가 무한대로 가고 상각된 sleep 비용이 0으로 수렴하므로, 구속력 있는 질문이 ρ 와 C_{cap} 으로 옮겨간다.

[평가] 그러므로 **상각식 (A)는 세 경로의 공통 자가아니다**. 이 책은 (A)를 전 장 공통 형식으로 강제했고, 그 강제가 유용했던이유는 값을 채우기 위해서가 아니라 **어디서 식이 깨지는지가경로별로 다르다는 사실을 드러내기 위해서였다**. 통일 회계의 첫 결과는통일된 숫자가 아니라, 하나의 회계 틀이 세 경로에 대해 각각 다른 방식으로무너진다는 관측이다. 이 관측은 corpus 전체에 대한 것이고 어느 논문 한편의 결함이나 아니다.

25.2.3 이 책의 통일 비용 회계 (실험 X1)

corpus 집계가 보인 것은 “잘 수 없다” 였다. 그러나 한 자리에서는 잘 수있다. [Letta STC]는 비용 4 종을 하나도 보고하지 않지만**비용모델은 인쇄했다**. 식 (14-7)이 그것이고, 그 식은 검사 가능한대상이다. 이 책은 그 식을 열어 무엇이 들어 있고 무엇이 빠졌는지를결정론적으로 계산한다. 이 계산을 **이 책의 통일 비용 회계 (실험 X1)**라고 부른다.

등급을 먼저 밝힌다. 이 실험은 EXPLORATION-GRADE / ACCOUNTING 이다 — 닫힌 형태 회계이며 측정이 아니다. 벽시계도 청구액도아니다. 결정론적이고 난수를 쓰지 않는다. 이 책이 승계한 정직성 계약 (→ ch01)에 따라 **비용 · 임계의 존재 · 레짐 간순서 · 민감도의 방향은 본문 단정문으로 쓰고, 절대 임계치는쓰지 않는다**. 뒤에 나오는 L^* 의 수치는 전부 그 등급 아래에서읽어야 한다.

모델. 비용 단위는 sleep 시점에 생성된 토큰 하나를 1로 놓은정규화 단위다. 장-국소 기호 하나를 선언한다 — w_{in} 은 입력토큰 하나의 상대 비용이며, 비용 4 종을 늘리는 것이 아니라 같은 회계안에서 입력 토큰에 매기는 가중치다. κ_{cost} (wake 생성토큰의 배수)와 $\kappa_{||}$ (병렬 재표현 별수)은 ch14 § 14.3.3이선언한 기호를 그대로 쓴다. J 는 sleep 국면의 재작성 깊이이고 상한은 10이다 [Letta STC App. K, p.27]. $|c| \cdot |\hat{c}| \cdot |q|$ 는 각각 원문맥 · learned context · 질의의 토큰 수다.

매 질의마다 내는 비용과 한 번만 내는 비용을 분리해 적으면 이렇다.

$$C_{wake} = \kappa_{cost} B_t + w_{in}(|c| + \kappa_{||}|\hat{c}| + |q|), \quad C_{sleep} = B_s + w_{in} J |c| \quad (25-1)$$

이 두 값을 식 (A)에 넣은 것이 이 장의 회계다. 논문의 식 (14-7)은 (25-1)에서 $w_{in} = 0$ 을 놓은 특수한 경우다 — 논문의 비용모델은 ‘tokens generated at test-time’만 세기 때문이다 [Letta STC § 5.3, p.8].

세 회계 변종. w_{in} 을 세 값으로 놓는다. $w_{in} = 0$ 은 논문의 암묵 회계다. $w_{in} = 1$ 은 FLOPs 회계다 — dense 모델에서 prefill 토큰과 decode 토큰의 토큰당 FLOPs가 같은자릿수이기 때문이다 (→ ch01의 Rosetta 항목, NM ch03). $w_{in} = 0.25$ 는 과금 회계다 — 공급자가 입력 토큰을 출력보다싸게 매기는 관행을 반영한다. 뒤의 둘은 **이 실험의 가정이며 논문의값이 아니다**.

STC에 유리한 쪽으로 몰아 놓은 가정. 이 실험은 결과가 가정탓으로 나오는 것을 막기 위해, 판정 대상에게 유리한 쪽으로 가정을고정했다.

표 25-3 — 이 회계의 가정과 그 방향. [본서 추론]

가정	내용	누구에게 유리한가
A1	sleep 생성 토큰에는 κ_{cost} 배수를 적용하지않는다 (throughput-optimized 로 돌린다고 본다)	STC에 유리
A2	sleep의 문맥 읽기를 J 회로만 센다. 실제 rethink 루프는 누적프롬프트를 다시 읽으므로 과소계상이다	STC에 유리
A3	baseline은 $\hat{c}$ 없이 원 문맥만 읽는다	공정
A4	두 팔의 정확도가 같다고 놓는다 (논문의 “같은 정확도 도달” 프레임)	중립

가정	내용	누구에게 유리한가
A5	원 문맥을 버리지 않는다. 논문이 원시 문맥과 learned context 를 함께 쓴다고 명시한다	논문 서술에 충실

논문에 없어 스윙한 값. $|c|$ (원 문맥 길이), $|\hat{c}|$ (learned context 길이), B_s (sleep 생성 토큰 수), 그리고 N_q 의 실제 분포 — 넷 다 원문에 없다. 앞의 셋은 가정하거나 스윙하고, 넷째는 논문이 스윙한 $\{1, 2, 5, 10\}$ 을 그대로 쓴다. 그리고 B_t 의 눈금은 논문에 수치표가 없어 **그림 축 범위에서 읽었다** (caveat: AXIS-READ). Stateful GSM-Symbolic 약 600, Stateful AIME 약 22500, SWE-Features 약 10000 이 baseline 쪽 상단이다 [Letta STC Fig. 3 p.6; Fig. 4 p.7; Fig. 11 p.12]. 절감률은 본문에 인쇄된 값을 쓴다 — 앞의 둘은 약 $5\times$, SWE-Features 는 약 $1.5\times$ 다 [Letta STC Abstract; § 5.1 p.6; § 6 p.12]. 이 값들이 전부 저자 제작 벤치마크의 값이라는 사실은 ch14 § 14.6 이 확정했다 (caveat: SELF-BUILT-BENCH).

25.2.4 두 개의 파생량 — 임계 길이와 손익분기

식 (25-1) 에서 두 값이 나온다. 유도는 초등적이며 이 책의 산술이다. [본서 산술]

임계 길이 L^* . sleep 을 쓰는 팔이 baseline 보다 매질의에서 싸려면 $C_{\text{wake}}^{\text{STC}} < C_{\text{wake}}^{\text{base}}$ 여야한다. 두 팔의 원 문맥 항과 질의 항이 같으므로 소거되고, 남는 조건은 $w_{\text{in}}\kappa_{\parallel}|\hat{c}| < \kappa_{\text{cost}}(B_t^{\text{base}} - B_t^{\text{lo}})$ 다. 따라서

$$L^* = \frac{\kappa_{\text{cost}}(B_t^{\text{base}} - B_t^{\text{lo}})}{w_{\text{in}}\kappa_{\parallel}} \quad (25-2)$$

이고, $|\hat{c}| > L^*$ 이면 N_q 를 아무리 키워도 **baseline 을 이길 수 없다**. L 이 이 책에서 sequence 길이를 뜻하므로 L^* 도 길이이며 단위는 토큰이다. $w_{\text{in}} = 0$ 이면 $L^* = \infty$ 다 — 논문의 회계에는 이 경계가 존재하지 않는다.

손익분기 N_q^* . 같은 회계에서 손익분기를 풀면

$$N_q^* = \frac{B_s + w_{\text{in}}J|c|}{\kappa_{\text{cost}}(B_t^{\text{base}} - B_t^{\text{lo}}) - w_{\text{in}}\kappa_{\parallel}|\hat{c}|} = \frac{B_s + w_{\text{in}}J|c|}{w_{\text{in}}\kappa_{\parallel}(L^* - |\hat{c}|)} \quad (25-3)$$

가 된다. 오른쪽 형태가 이 장에서 가장 중요한 식이다. **손익분기는 $\hat{c}$ 가 임계에 가까워질수록 발산한다.** $|\hat{c}| \rightarrow L^*$ 에서 $N_q^* \rightarrow \infty$ 이고, 임계를 넘으면 분모가 음수가 되어 해가 없다. 즉 상각 논증의 붕괴는 절벽이 아니라 **점근**이며, 임계에 닿기 한참전부터 필요한 질의 수가 실무적으로 도달 불가능해진다.

결과 1 — 레짐별 임계 길이. 세 레짐에서 (25-2) 를 계산한 값이다. 문맥 길이 $|c|$ 는 논문에 없으므로 각 레짐의 성격에 맞춰가정했다 (GSM-Symbolic 400, AIME 300, SWE-Features 8000 토큰). 괄호 안은 $L^*/|c|$, 즉 원 문맥의 몇 배까지 learned context 를 늘려도되는가다.

표 25-4 — 임계 길이 L^* (토큰) 와 문맥 대비 배수. 실험 X1, EXPLORATION-GRADE / AC-COUNTING. **절대치는 판정근거가 아니며 순서와 방향만 단정한다.**

레짐	회계	$\kappa_{\parallel} = 1$	$\kappa_{\parallel} = 5$	$\kappa_{\parallel} = 10$
Stateful GSM-Symbolic (절감 $5\times$, $ c =400$)	논문 ($w_{\text{in}}=0$)	경계 없음	경계 없음	경계 없음
	과금 ($w_{\text{in}}=0.25$)	19200 ($48.0\times$)	3840 ($9.6\times$)	1920 ($4.8\times$)
	FLOPs($w_{\text{in}}=1$)	4800 ($12.0\times$)	960 ($2.4\times$)	480 ($1.2\times$)

레짐	회계	$\kappa_{\parallel} = 1$	$\kappa_{\parallel} = 5$	$\kappa_{\parallel} = 10$
Stateful AIME (절감 5×, $ c =300$)	논문	경계 없음	경계 없음	경계 없음
	과금	720000 (2400×)	144000 (480×)	72000 (240×)
SWE-Features (절감 1.5×, $ c =8000$)	FLOPs	180000 (600×)	36000 (120×)	18000 (60×)
	논문	경계 없음	경계 없음	경계 없음
	과금	133333.3 (16. 67×)	26666.7 (3. 33×)	13333.3 (1. 67×)
	FLOPs	33333.3 (4. 17×)	6666.7 (0.83×)	3333.3 (0.42×)

결과 2 — 손익분기. B_s 를 baseline 생성 토큰의 2.5 배로, $J = 3$, $\kappa_{\parallel} = 1$ 로 놓고 (25-3) 을 계산한 값이다. $|\hat{c}|$ 는 $|c|$ 의 0.25 배 · 1 배 · 4 배로 스위쳤다. 이 세값은 전부 원문에 없어 이 실험이 놓은 값이다 (caveat: PAPER-ABSENT-SWEPT).

표 25-5 — 손익분기 N_q^* (질의 수). 실험 X1, EXPLORATION-GRADE / ACCOUNTING. **각 칸의 절대치는 가정한 $B_s \cdot |\hat{c}|$ 에 직접 비례하므로 방향성으로만 읽는다.** 단정하는 것은 회계 변종 간 순서다. 1 미만은 그 회계에서 첫 질의부터 이미 싸다는 뜻이다

레짐	논문 회계	과금 회계 ($ \hat{c} / c = 0.25 / 1 / 4$)	FLOPs 회계 (0.25 / 1 / 4)
Stateful	0.31	0.38 / 0.38 / 0.41	0.57 / 0.61 / 0.84
GSM-Symbolic			
Stateful AIME	0.31	0.31 / 0.31 / 0.31	0.32 / 0.32 / 0.32
SWE-Features	0.75	0.94 / 0.99 / 1.22	1.56 / 1.93 / 36.75

결과 3 — κ_{cost} 민감도. (25-2) 에서 L^* 는 κ_{cost} 에 정비례한다. 이 비례 관계가 이결과에서 단정하는 전부이고, 아래 배수들은 가정한 $|c|$ 위에서 그비례를 읽은 방향성 값이다. [본서 산술] SWE-Features·FLOPs 회계에서 κ_{cost} 를 흔들면 임계가 그대로 따라 움직인다: $\kappa_{\parallel} = 1$ 에서 $\kappa_{\text{cost}} = 1$ 이면 L^* 가 문맥의 0.42 배, $\kappa_{\text{cost}} = 2$ 면 0.83 배, $\kappa_{\text{cost}} = 5$ 면 2.08 배, $\kappa_{\text{cost}} = 10$ 이면 4.17 배, $\kappa_{\text{cost}} = 20$ 이면 8.33 배다. $\kappa_{\parallel} = 10$ 으로 옮기면 다섯 값 전부가 1 보다 작아진다 — 이회계에서는 어떤 κ_{cost} 에서도 learned context 가 원문맥보다 짧아야만 매 질의 비용에서 이긴다는 뜻이다.

25.3 분석

25.3.1 빠진 항은 우연이 아니라구조다

표 25-4 의 첫 줄들이 이 장의 핵심 관측이다. 논문의 회계에서는 세 레짐모두 **경계가 없다**. $w_{\text{in}} = 0$ 이면 (25-2) 의 분모가 0 이되어 L^* 가 무한대로 가고, learned context 를 아무리 길게 만들어도 매 질의 비용이 늘지 않는다. 그러면 상각 논증은 언제나 성립한다 — N_q 를 키우면 이긴다.

문제는 그것이 회계의 결론이 아니라 **회계의 전제**라는 데 있다. 식 (25-1) 을 다시 보면 두 항의 성질이 다르다. B_s 는 문맥당 한 번 내고 N_q 로 나뉜다. $w_{\text{in}}\kappa_{\parallel}|\hat{c}|$ 는 **매 질의마다** 낸다 — $\hat{c}$ 가 프롬프트에 실려 다시 처리되기때문이고, $\kappa_{\parallel}$ 개의 재표현을 이어붙이면 그만큼 곱해진다 [Letta STC § 5.2, p.8]. 나뉘는 항과 나뉘지 않는 항이 섞여 있는데, 논문의 식은 나뉘는 항만 세고 나뉘지 않는 항을 0 으로 놓는다.

[평가] 이것이 상각 논증의 **구조적 결함**이다. 어느상수를 잘못 골랐다거나 어느 실험이 부족하다는 층위의 문제가 아니다. 한번 내는 비용만 N_q 로 나뉘고 매 질의 내는 비용은 나뉘지 않는데, 논문의 회계는 후자를 항으로 갖고 있지 않다. 그 결과 이 회계는 “ N_q 를 키워도 이길 수 없는 영역”의 경계를 **무한대로 보내 표현조차 못한다**. 표 25-4의 “경계 없음” 세 줄이 그 상태다. 이 책이 하는 일은 그항을 되살려 경계가 어디에 있는지 계산하는 것이고, 항을 되살리면 경계는언제나 유한하다.

여기에 이 방법 특유의 사정이 겹친다. $\hat{c}$ 는 원 문맥을 압축한 것이아니다 — Letta STC의 learned context는 문맥에 대한 **추론의산출물**이고 원 문맥보다 길다 (→ ch14 § 14.5.3). 게다가 원 문맥을 버리지않는다 (가정 A5). 즉 이 방법의 기제 자체가 프롬프트를 늘리는 방향으로작동하는데, 늘어난 프롬프트의 비용이 회계에 없다. 항이 빠진 방향과방법이 작동하는 방향이 같다.

[해설] 이 독자의 어휘로 옮기면 w_{in} 은 **prefill**을 회계에 넣을지 말지의 손잡이다. $w_{in} = 0$ 은 “prefill 이 공짜”라는 뜻이고, 그것이 참인 서빙 구성이 실제로 있다 — $\hat{c}$ 가 프롬프트 접두사로 고정되어 접두사 캐시에 완전히 적중하면 매질의의 추가 prefill 이 0에 수렴한다. ch01의 Rosetta 표가 $\hat{c}$ 를 prefix cache 옆에 놓은 이유가 여기 있다. 다만 둘은 같지 않다. 접두사캐시는 **같은 계산의 재사용**이라 적중 조건이 문자열 동일성이고, $\kappa_{||}$ 개의 재표현이 질의마다 다르게 뽑히거나 질의 토큰이앞에 오면 적중이 깨진다. 그리고 캐시 자체가 KV 바이트를 점유하므로, 프롬프트가 길어지는 만큼 배치 크기의 상한이 내려간다 — 시간 축에서 지운비용이 바이트 축에서 다시 나타나는 것이며, 그 축은 ch26이 쟀다. 요약하면 w_{in} 을 0으로 놓는 것은 회계상의 단순화가 아니라**특정 서빙 구성에 대한 가정**이고, 논문은 그 가정을 명시하지않는다.

25.3.2 레짐 간 순서 — 이 실험이 단정하는것

표 25-4에서 단정할 수 있는 것은 순서다. 절대 임계치는 아니다. 그 이유는 L^* 의 절대값이 B_t 눈금 (그림 축에서 읽음) 과 $|c|$ (가정) 에직접 비례하는데, 두 값 다 논문에 없기 때문이다.

순서 판정 1 — SWE-Features 가 모든 회계 변종, 모든 $\kappa_{||}$ 에서 최하위다. 이것은 표 25-4에서 경계가 유한한여섯 설정 (회계 2종 $\times \kappa_{||}$ 3 값) 전부에서 성립하고, 두가지 방향성만으로 결정된다. 절감률이 $5\times$ 에서 $1.5\times$ 로 떨어지면 (25-2)의분자 $\kappa_{cost}(B_t^{base} - B_t^{lo})$ 가 줄고, 문맥이 몇 문장에서 코드베이스로 커지면 $L^*/|c|$ 의 분모가 커진다. 두 효과가 같은 방향으로 작용하므로 미보고 값을 어떻게 골라도 순서가뒤집히지 않는다. 이 순서는 본문 단정문이다.

순서 판정 2 — 여유가 큰 두 레짐 (Stateful AIME, Stateful GSM-Symbolic) 과 SWE-Features 사이의 간격은 자릿수 규모다. FLOPs 회계 $\cdot \kappa_{||} = 1$ 에서 $L^*/|c|$ 가 각각 600 배 $\cdot 12$ 배 $\cdot 4.17$ 배인데, 이 세 배수자체는 가정한 $|c|$ 와 축에서 읽은 B_t 위의 방향성 값이고 단정하는것은 그 사이에 자릿수가 놓인다는 사실이다. AIME 이 GSM-Symbolic 보다앞서는 것은 baseline 생성 토큰이 훨씬 크기 때문인데, 그 값 (약 22500 대약 600)은 **그림 축에서 읽은 값**이라 이 두 레짐 사이의 상대 순서는앞의 판정만큼 견고하지 않다. 이 책은 둘 사이의 순서를 단정하지 않고, **둘이 함께 SWE-Features 보다 크게 앞선다**만 단정한다.

순서 판정 3 — $\kappa_{||}$ 이 커질수록 여유가 정확히비례해 줄어든다. (25-2)에서 $L^* \propto 1/\kappa_{||}$ 이므로 병렬 재표현 열 벌을 붙이면임계가 10분의 1이 된다. 그런데 논문이 13% 헤드라인을 얻은 자리가 바로이 $\kappa_{||}$ 스왑이다 [Letta STC § 5.2, p.8; Fig. 7, p.10]. 품질을 올리는 손잡이가 임계를 좁히는 손잡이와 같은손잡이다. [본서 추론] 논문은 이 축에서 5 벌이 10 벌보다 대체로 낮다고보고하면서 기제를 제시하지 않는데 (→ ch14 § 14.6), 회계 쪽에서 보면 10 벌이나빠지는 데에는 품질과 무관한 이유가 하나 더 있다.

AIME 레짐의 큰 여유는 액면 그대로 읽으면 안 된다. 그 레짐은 문맥이 몇 백토큰짜리 문제 진술이고 일부 인스턴스는 문맥이 비어 있으며 그개수가 밝혀지지 않았다 [Letta STC App. J, pp.27–28]. 문맥이 비면 sleep 국면이 선행 처리할 대상 자체가 없다. 즉 이 레짐에서 임계가 멀리있는 이유는 방법이 잘 작동해서가 아니라, (25-2)의 분자에 놓인 생성 토큰차가 워낙 커서 프롬프트 항이 상대적으로 묻히기 때문이다. 문맥이짧을수록, 그리고 답이 길수록 이 회계는 sleep-time compute 에게유리해진다. 그 조건은 방법의 성질이 아니라 벤치마크의 성질이다. [본서 추론]

25.3.3 손익분기가 실제로 말하는것

표 25-5에서 논문 회계의 값이 세 레짐 모두 1보다 작다는 사실을 먼저 본다. 가정한 B_s (baseline 생성 토큰의 2.5 배) 아래에서, 이 회계는 **손익분기를 첫 질의 이전에 놓는다** — $N_q = 1$ 에서도 sleep 을 쓰는쪽이 싸다. 그렇다면 상각 논증은 필요가 없다. 논증이 필요한 이유는 $\kappa_{\text{cost}} = 10$ 이 wake 토큰을 열 배로 비싸게 만들어 두었기때문이고, 그만큼 sleep 토큰이 상대적으로 공짜에 가까워지기 때문이다. [본서 관찰]

반대편 끝의 36.75 가 이 장에서 가장 많은 것을 말하는 수다. SWE-Features·FLOPs 회계 · $|\hat{c}| = 4|c|$ 에서 손익분기가 같은 표의 다른 칸들보다 두 자릿수 가까이 뛴다. 이유는 (25-3) 의 오른쪽 형태에 있다 — 그 설정에서는 $|\hat{c}|$ 가 L^* 바로 아래까지 올라와 분모의 $L^* - |\hat{c}|$ 가 L^* 의 4% 남짓만 남는다. [본서 산술] 이 실험이 단정하는 것은 서른일곱이라는 값이 아니라 그 값이 생기는 구조다 — **같은 B_s 가 어느 회계에서는 첫 질의에서 회수되고 어느 회계에서는 수십 질의를 기다린다.** 그리고 조금만 더 가면 회수되지 않는다.

이 지점에서 표 25-2 의 유형 I 이 다시 걸린다. N_q^* 를 실무에서 쓰려면 N_q 의 분포를 알아야 하는데, **실서빙에서 한 문맥을 몇개 질의가 공유하는지의 분포를 보고한 논문이 corpus 에 없다.** 논문은 {1, 2, 5, 10} 네 점을 합성으로 구성해 재고, 그 네 값이 실제 트래픽에서 어떤 빈도로 나타나는지는 다루지 않는다 (→ ch14 § 14.5.2). N_q 는 방법의 성질이 아니라 워크로드의 성질이다. corpus 에서 N_q 의 분포를 실제로 가진 논문은 상각을 주장하지 않는 Memorizing Transformers 하나뿐인데, 그 논문에는 상각할 sleep 비용이 없다 (→ ch12 § 12.5). 분포를 가진 논문에는 상각할 것이 없고, 상각을 주장하는 논문에는 분포가 없다.

25.3.4 κ_{cost} — 회계전체를 지탱하는 미측정상수

결과 3 이 보이는 것은 단순하다. $L^* \propto \kappa_{\text{cost}}$ 이고 N_q^* 도 κ_{cost} 에 반비례로 얹힌다. 이 상수가 회계 전체를 위아래로 움직인다.

그런데 이 값의 근거는 측정이 아니라 인용이다. 논문은 “latency optimized inference can be roughly 10× more expensive” 라고 쓰고 각주로 벤더 문서 페이지를 가리킨다 [Letta STC § 5.3, p.8]. 논문 안에 이 비율을 재는 실험이 없고, 이 값을 흔들었을 때 2.5× 헤드라인이 어떻게 변하는지에 대한 민감도 분석도 없다 (→ ch14 § 14.5.4).

[평가] κ_{cost} 는 이 회계에서 **가장 무거운 단일 상수** 이면서 **가장 근거가 얇은 값** 이다. 그리고 그 값이 틀리는 방향은 예측 가능하다. wake 국면도 배치로 묶어 throughput-optimized 로 돌리는 서빙 구성 — 즉 이 방법이 상정하는 “문맥이 먼저 도착하고 질의 가나중에 오는” 비대화형 워크로드에서 자연스러운 구성 —에서는 κ_{cost} 가 1 에 가까워진다. 그러면 임계가 10 분의 1 로 줄어든다. SWE-Features·FLOPs 회계 · $\kappa_{\parallel} = 1$ 에서 $\kappa_{\text{cost}} = 1$ 이면 L^* 가 원 문맥의 0.42 배이고, 이는 **원 문맥보다 짧은 learned context 조차 매 질의 비용을 늘린다** 는 뜻이다. 이 방향의 민감도는 논문 어디에도 검토되어 있지 않다.

25.3.5 5× 는 반박되지 않는다

이 회계가 무엇을 하지 **않는** 지를 분명히 해야 한다.

표 25-4 의 GSM-Symbolic 줄을 다시 본다. FLOPs 회계 · $\kappa_{\parallel} = 1$ 에서 임계는 가정한 문맥의 12 배이고, $\kappa_{\parallel} = 10$ 까지 밀어도 1.2 배로 1 을 넘는다. 두배수 모두 방향성 값이지만 부등호의 방향은 이 실험이 스윙한 전 범위에서 유지된다. 즉 짧은 문맥과 큰 절감률이 함께 있는 레짐에서는 prefill 항을 넣어도 여유가 남는다. **논문의 5× 는 이 실험이 다루는 어떤 회계에서도 뒤집히지 않는다.** 5× 는 생성 토큰 축에서 관측된 사실이고, 그 축 위의 관측을 이 계산이 반박할 수는 없다.

반박 대상은 다른 것이다. “**생성 토큰 축 = 비용 축**” 이라는 암묵등식이다. 생성 토큰이 5 분의 1 로 줄었다는 관측과 비용이 5 분의 1 로 줄었다는 결론 사이에는 네 가지가 놓여 있다 — 회계에 없는 prefill 항, 인쇄되지 않은 B_s , 측정 없이 단언된 κ_{cost} , 그리고 미지의 N_q 분포. ch14 § 14.6 이 이 넷을 목록으로 만들었고, 이 장은 그중 첫째와 셋째를 수치로 열었다.

그리고 등식이 깨지는 자리가 어디인지가 이 실험의 실질적 결과다. 등식이 가장 위태로운 레짐은 논문 스스로 이득이 작다고 보고한 레짐과 일치한다. 논문은 SWE-Features 에서 절감이 약 $1.5\times$ 이고 “test-time 예산이 높으면 test-time compute 만 쓰는 쪽이 더 나을 수 있다” 고 쓴다 [Letta STC § 6, p.12]. 회계는 같은 자리에서 더 강한 진술을 허용한다 — 이득이 작을 뿐 아니라 이득이 사라지는 경계가 가깝다. 형제 PR 묶음처럼 여러 변경을 함께 설명해야 하는 문맥에서 learned context 가 원문맥의 몇 배가 되는 것은 이 방법의 기제상 자연스럽고 [본서 추론], $\kappa_{||}$ 을 키우면 경계는 그만큼 더 가까워진다.

25.3.6 같은 자를 세 경로에 대면 자가부러진다

지금까지의 계산은 E -경로 한 편에 대한 것이다. 같은 자를 다른 두 경로에 대면 무슨 일이 생기는지가 표 25-2 의 유형 III – VI 였다.

W -경로에서는 상각이 재배치로 바뀐다. Do LMs Need Sleep?의 GSM-Infinite 헤드라인 설정에서 $N_q = 1$ 이므로 식 (A) 가 $C_{avg} = C_{wake} + C_{sleep}$ 으로 붕괴한다. wake 에서 뺀 계산이 같은 요청 안의 다른 국면으로 옮겨 갈 뿐이고, 요청전체의 계산량은 recurrence 횟수에 대략 비례해 늘어난다 ($\rightarrow$ ch17 § 17.5). 이 경로에서 팔리는 것은 총비용 절감이 아니라 비용의 시간 배치다. 그리고 그 배치의 이득을 재려면 L_w 가 필요한데 그 칸이 비어 있다.

Θ -경로에서는 분모가 질의 수가 아니다. [MEMIT] 에서 실제분모는 한 라운드에 묶어 넣은 편집 수이고, 그 논문의 기여를 이 언어로 옮기면 분모를 1 에서 10,000 으로 키운 것이다 ($\rightarrow$ ch19 § 19.5). LM Need Sleep 에서 분모가 “앞으로의 전 트래픽” 이 되어 상각된 sleep 비용이 0 으로 수렴한다 ($\rightarrow$ ch21 § 21.5). 후자에서 상각식은 이기는 것이 아니라 퇴화한다 — 분모가 무한하면 그 식은 아무것도 말하지 않는다.

[평가] 그러므로 세 경로를 같은 자로 재려는 시도의 첫 결과는 자가 부러진다는 것이다. 식 (A) 는 E -경로의 모양을 전제하고있고, 다른 두 경로에서는 분모가 다른 것을 세거나 아예 정의되지 않는다. 이것은 이 책이 (A) 를 잘못 골랐다는 뜻이 아니다 — (A) 를 전장에 강제했기때문에 어디서 부러지는지가 경로별로 정확히 드러났다. 그리고 부러지는 지점이 곧 구속력 있는 질문이 옮겨가는 지점이다. Θ -경로에서 (A) 가 퇴화하는 순간 질문은 ρ 와 C_{cap} 으로 옮겨가고 ($\rightarrow$ ch21 § 21.5), 표 25-1 이 보이듯 그 두 칸은 corpus 전체에서 각각 0 편과 0 편이다. 경로를 갈아타면 질문이 옮겨가는데, 옮겨간 자리의 증거가 더 비어 있다.

Θ -경로에는 어느 경로에도 없는 항이 하나 더 있다. 쓰기검증 비용이다. [ROME] 은 편집당 약 2 초를 인쇄하지만 그 2 초는 쓰기연산의 비용이지 올바르게 썼는지 확인하는 비용이 아니다. 같은 논문이 편집후 모델이 근거 없는 새 사실을 지어낸다고 인정하고 사람 15 명을 동원한평가를 수행했는데, 그 비용은 어떤 비용 수치에도 들어 있지 않다 ($\rightarrow$ ch19 § 19.5). E -경로에서 쓰기가 틀리면 그 항목을 지우면 되지만, Θ -경로에서 쓰기가 틀리면 배포 아티팩트가 바뀐 뒤다. 검증을 회계 밖에 두면 Θ -경로의 쓰기는 실제보다 싸게계상된다. [본서 추론]

25.4 판정

[평가] 판정 1 — 세 경로는 지금 같은 자로 잴 수 없다. 그리고 못 재는 이유가 두 층위로 나뉜다.

표 25-1 의 요구 단위 열에 값이 인쇄된 논문은 스물한 편 중 B_s 에서 1 편, $L_w \cdot C_{cap} \cdot \rho$ 에서 각각 0 편이다. 그리고 세 경로의 대표 논문 셋이 모두 네 칸을 하나도 채우지 않았다. 첫 번째 층위는 보고 관행이다 — 값이 존재하는데 인쇄되지 않았고, 저자가 인쇄하면 즉시 해소된다. 두 번째 층위는 회계 틀 자체다. 식 (A) 는 E -경로의 모양을 전제하므로 W · Θ -경로에서는 분모가 다른 것을 세거나 정의되지 않고 (표 25-2 의 유형 III – VI), 값을 인쇄해도 해소되지 않는다. 따라서 “sleep-time compute 이 싼가” 는 현재 문헌 상태

에서 검증도반증도 되지 않는다. 이 책은 그 자리를 성능 주장으로도 비용 비난으로도 메우지 않는다.

[평가] 판정 2 — 상각 논증에는 구조적 결함이 있다. 조건 없이단정한다.

한 번 내는 비용만 N_q 로 나뉘고 매 질의 내는 비용은 나뉘지 않는다. learned context 는 매 질의의 프롬프트에 실려 다시 처리되고, 병렬 재표현을붙이면 $\kappa_{||}$ 배가 된다. 이 항이 0 이 아니면 “ N_q 를키우면 언제나 이긴다” 가 성립하지 않는 영역이 **원리적으로** 존재한다. [Letta STC] 의 비용모델은 생성 토큰만 세므로 이 영역의 경계 L^* 를 무한대로 보내고, 그래서 그 경계를 **표현조차 하지못한다**. 이것은 실험적 반례가 아니라 식의 성질이며, 미보고 값의 선택과무관하다.

[평가] 판정 3 — 조건부. 짧은 문맥과 큰 절감률이 함께 있는레짐에서는 $5\times$ 가 뒤집히지 않는다.

조건은 둘이다: (i) 원 문맥이 짧아 프롬프트 항이 생성 항에 비해 작고, (ii) 절감률이 커서 (25-2) 의 분자가 크다. 두 조건이 성립하는 Stateful GSM-Symbolic·Stateful AIME 에서는 FLOPs 회계로 계산해도, $\kappa_{||}$ 을 논문 최대치인 10 으로 밀어도 임계 안쪽에 머문다. **이 실험은 [Letta STC] 의 $5\times$ 를 반박하지 않는다**. 반박 대상은 “생성 토큰 축 = 비용 축” 이라는 암묵 등식이며, 그 등식은 회계 변종을바꾸는 것만으로 유효 범위가 유한해진다.

[평가] 판정 4 — 레짐 간 순서. 단정문으로 쓴다.

SWE-Features 레짐의 여유가 세 레짐 중 최하위다. 표 25-4 에서경계가 유한한 여섯 설정 (회계 2 종 $\times \kappa_{||}$ 3 값) 전부에서성립하고, 두 방향 (절감률 하락, 문맥 길이 증가) 만으로 결정되므로 미보고값의 선택에 둔감하다. 여유가 큰 두 레짐과의 간격은 자릿수 규모다. **단, 절대 임계치는 판정에서 제외한다** — L^* 의 절대값은그림 축에서 읽은 B_t 와 가정한 $|c|$ 에 비례하며, 이 실험의등급 (EXPLORATION-GRADE / ACCOUNTING) 이 허용하는 것은 순서와방향뿐이다. Stateful AIME 과 Stateful GSM-Symbolic 사이의 상대 순서도단정하지 않는다.

[평가] 판정 5 — 회계가 가장 위태로운 자리는 논문이 이미약하다고 보고한 자리와 일치한다.

논문은 SWE-Features 에서 절감이 약 $1.5\times$ 이고 고예산에서 역전할 수 있다고쓴다 [Letta STC § 6, p.12]. 통일 회계는 같은 자리에서 더 강한 진술을허용한다 — 이득이 작을 뿐 아니라 **이득이 사라지는 경계가 가깝다**. 이 일치는 우연이 아니다. 절감률이 작다는 것은 (25-2) 의 분자가 작다는것과 같은 말이고, 실제 코드베이스를 문맥으로 쓴다는 것은 분모 쪽 $|c|$ 가 커진다는 것과 같은 말이다. **논문의 이득이 작아지는축과 회계가 무너지는 축이 같은 축이다**. 그리고 그 축의 방향은 실무워크로드 쪽이다 — 문맥이 짧고 절감률이 큰 레짐은 저자 제작 벤치마크의성질이 지 코드베이스·문서고·대화 이력의 성질이아니다.

25.5 반증 조건

위 다섯 판정은 아래 관측으로 틀렸음이 드러난다. 각 항목은 어떤 판정을무너뜨리는지 명시한다.

1. $\hat{c}$ 와 B_s 의 실제 토큰 수가 인쇄되고, SWE-Features 에서 $\kappa_{||}$ 개를 이어붙인 프롬프트가 L^* 아래에 **넉넉히 머무는 것이 확인되는 경우**. 판정 5 가무너진다. 확인 방법은 원 저자 또는 재현자가 각 레짐의 평균 $|c| \cdot |\hat{c}| \cdot \text{sleep}$ 생성 토큰 수를보고하는 것이며, 이 셋은 실행 로그에 이미 존재하므로 새 실험이 필요없다.

2. 그림 축 ' Avg. 'Test Time Tokens / Question' 이프롬프트 토큰을 포함한다는 것이 확인되는 경우. 판정 2·3 이 함께 무너진다 — 그렇다면 $5\times$ 는 생성 토큰 절감이나 아니라 총 토큰 절감이고, 이 장이 되살린 항이 이미 축 안에 들어 있는것이 된다. 이 판정은 ch14 §14.5.3 이 미해결로 남긴 항목과 같은항목이며, 공개 결과 아티팩트의 필드 정의를 보면 결정된다.
3. **prefill** 이 실효적으로 공짜인 서빙 구성이 측정으로 제시되는경우. 판정 2 의 실무적 무게가 사라진다. 구체적으로, $\hat{c}$ 가 질의사이에 완전히 prefix-cache 되어 재계산되지 않고 $w_{in}^{eff} \approx 0$ 임이 벽시계로 확인되면 임계는실무에서 존재하지 않는다. 다만 $\kappa_{||}$ 개의 재표현이질의마다 달라지거나 질의 토큰이 앞에 놓이면 접두사 캐시가 성립하지않으므로, 반증하려면 캐시 적중률 까지 함께 보고해야 한다.
4. κ_{cost} 가 측정으로 확정되고 그 값이 10 이상인것이 서빙 구성 전반에서 유지되는 경우. 판정 5 의 민감도 논증이약해진다. 반대로 배치 서빙에서 κ_{cost} 가 1 에 가깝다는측정이 나오면 판정 5 가 강화된다. 어느 쪽이든 현재는 벤더 문서 인용 한줄뿐이므로 이 항목은 양방향으로 열려 있다.
5. 실서빙 로그에서 N_q 의 분포가 보고되고 그 중앙값이 손익분기위쪽에 안정적으로 있는 경우. 판정 1 의 두 층위 중 첫 번째가 해소된다. 이 관측은 corpus 전체에 대해 새롭다 — N_q 분포를 가진 논문이하나도 없으므로, 한 편이라도 인쇄하면 그 자체로 새 사실이다.
6. 세 편 이상의 논문이 B_s 를 FLOPs 로, C_{cap} 을 dtype 과 함께 세션당 바이트로 인쇄하는 경우. 판정 1 의 첫 층위가 corpus 수준에서 무너진다. 다만 둘째 층위 (식 (A) 가 경로마다 다르게 퇴화한다) 는이 관측으로 해소되지 않는다 — 그것을 무너뜨리려면 $W \cdot \Theta$ -경로에서 상각의 분모로 쓸 수 있는 양을정의하고 그 값이 측정되어야 한다.
7. 어떤 방법이 learned context 를 만들면서 원 문맥을 실제로 버리는것 (가정 A5 의 파기) 이 품질 손실 없이 성립하는 경우. 판정 2 의 결함이완화된다 — 매 질의 프롬프트가 $|c|$ 에서 $|\hat{c}|$ 로 줄어들면 나뉘지 않는 항이 음수 기여를 하고 임계가 사라진다. 이것이 이 판정에서 벗어나는 가장 깨끗한 설계 경로이며, W -경로가구조적으로 시도하는 것이기도 하다 ($\rightarrow$ ch17).
8. 한 편이라도 라운드당 열화율 ρ 를 sleep 라운드 수를독립변수로 놓은 실험에서 인쇄하는 경우. 판정 1 이 “네 칸 중 세 칸이 0 편” 이라는 corpus 사실 위 에 서 있으므로, 표 25-1 의 ρ 줄이 0 편에서 벗어나면 판정 1 이 덮는 범위가 그만큼 줄어든다. 값이 0 에가까우면 반복 sleep 의 지속가능성 쪽 공백이 함께 메워진다. 이 항목은 ch29·ch30 이 받는다.

이 여덟 중 둘 (2 번과 5 번) 은 새 실험 없이 기존 로그의 공개만으로판정된다. 판정 1 이 문헌의 상태에 대한 진술인 만큼, 그 상태를 바꾸는 데필요한 비용이 낮다는 사실도 함께 기록해 둔다.

요약

- Part II 의 §N.5 가 원문 대조로 채운 스물한 편에서, 요구 단위대로 인쇄된칸은 B_s 1 편 (ReasoningBank 의 과제당 3748.4 토큰) 뿐이고 $L_w \cdot C_{cap} \cdot \rho$ 는 각각 0 편이다.
- 네 칸에 절대값이 하나도 없는 논문이 여섯이며, 그중 셋이 세 경로의 대표논문이다 — Letta STC(E), Do LMs Need Sleep?(W), LM Need Sleep(Θ).
- 상각 손익분기 N_q^* 를 계산할 수 있는 논문이 스물한 편 중 0 편이다. 실패 이유는 여섯 유형이고, 그중 넷은 값을 인쇄해도 해소되지않는다 — 식 (A) 가 E -경로의 모양을 전제하기 때문이다.
- 상각 논증의 구조적 결함은 하나다. 한 번 내는 비용만 N_q 로 나뉘고매 질의 내는 비용은 나뉘지 않는데, 논문의 회계는 후자를 항으로 갖고있지 않다. 그래서 경계 L^* 가 무한대로 가고 표현되지 않는다.
- 이 책의 통일 비용 회계 (실험 X1) 는 그 항을 되살려 임계 길이 L^* 와 손익분기 N_q^* 를 계산한다. 손익분기는 $|\hat{c}| \rightarrow L^*$ 에서 발산하므로, 붕괴는 절벽이 아니라 접근이다.

- 레짐 간 순서는 단정한다 — SWE-Features 의 여유가 최하위이고, 여유가 큰 두 레짐과의 간격은 자릿수 규모다. 절대 임계치는 단정하지 않는다.
- $5\times$ 는 반박되지 않는다. 짧은 문맥 · 큰 절감률 레짐에서는 prefill 항을 넣어도 뒤집히지 않는다. 반박 대상은 “생성 토큰 축 = 비용축” 이라는 암묵 등식이다.
- 회계를 위아래로 움직이는 단일 상수 κ_{cost} 는 측정이 아니라 벤더 문서 인용이며, 배치 서빙 쪽으로 갈수록 1 에 가까워져 임계를 좁힌다. 민감도 분석이 원문에 없다.

다음 장으로

이 장은 시간 축의 회계를 판정했다 — 계산을 언제 내는가, 그리고 그것이 몇개 질의에 나뉘는가. 판정 1 의 둘째 층위가 남긴 것은 자리를 옮긴 질문이다. Θ -경로에서 식 (A) 가 퇴화하는 순간 구속력 있는 질문이 ρ 와 C_{cap} 으로 옮겨가는데, 표 25-1 은 그 두 칸이 각각 0 편과 0 편이라고 말한다.

그래서 ch26 은 자리를 바꾼다. 시간이 아니라 **바이트**로 잰다. 세경로가 한 세션 · 한 사용자에게 실제로 몇 바이트를 남기는지, dtype 이 정보 밀도를 어떻게 정하는지, 그리고 그 자릿수가 경로 간에 얼마나 벌어지는지를 이 책의 상태 용량 회계로 계산한다. 시간 축에서 비교가 불가능했던 세경로가 바이트 축에서는 비교되는지 — 그것이 다음장의 판정할 질문이다.

ch26. 상태 용량과 정보밀도

26.1 판정할 질문

세 경로가 만드는 상태는 얼마이고, 어디에 살며, 그 바이트에 무엇이 담기는가.

이 질문에는 특이한 사정이 하나 있다. 문헌이 답을 갖고 있지 않다. Part II 열세 장을 통과하며 비용 4 종 표를 채운 결과, **ch11의 판별식을 통과한 논문 중 상태 용량 C_{cap} 을 사용자당 · 세션당 바이트로 보고한 논문이 0 편이다.** 범위를 이렇게 좁혀 적는 이유는 배경 장의 반례 하나 때문이다 — [RAG]는 바이트를 인쇄하지만 그 단위가 **corpus 당**이어서 이 장의 회계와 같은 축에 놓이지 않는다 (→ ch10 § 10.4). 이 예외는 § 26.2에서 다시 짚는다. ch11이 판별식 통과 논문들의 비용 칸을 전수 대조했을 때 열여섯 칸 중 값이 들어간 칸은 셋이었고 그 셋의 단위가 각각 ms · 개념개수 · 파라미터 개수였다 (→ ch11). ch14는 Letta Sleep-time Compute의 C_{cap} 을 “논문에 없음” 으로 남겼고, ch17은 Do Language Models Need Sleep?에 대해 **유도조차 불가능**하다고 기록했다 — 블록당 상태 차원도, head 수도, dtype도 논문에 없기 때문이다 (→ ch17 § 17.5). ch13은 MemGPT에서 E 의 단위가 corpus 당에서 사용자당으로 바뀌는 바로 그 지점에 숫자가 사라진다는 것을 확인했다.

그러므로 이 장은 판정 이전에 자를 만드는 일부부터 한다. 이 책의 상태 용량 회계 (실험 X2)는 논문이 인쇄하지 않은 바이트를 **구성 파라미터에서 복원**한다. 측정이 아니라 회계이며, 그 등급이 이 장의 모든 문장의 상한을 정한다.

답이 될 수 있는 것. 세 경로의 상태가 서로 몇 자릿수 떨어져 있는가. 각 상태의 수명과 상주 위치. 저장바이트당 정보 밀도가 무엇에 의해 정해지는가 — 그리고 그 밀도의 순서. 사용자 수가 늘 때 상태 총량이 자라는 **모양**. 마지막으로, Θ -경로가 두 갈래로 갈라지는 **서빙 갈림길**이 어디에 있는가.

답이 될 수 없는 것. 첫째, 절대 바이트를 실측치처럼 쓰는 것. X2가 내놓는 수는 이 책이 고른 앵커 구성에 대한 산술이지 어느 시스템의 측정이 아니다. 둘째, **경로 간 정보 밀도 비교**. 뒤에서 보듯 X2 초판이 이 비교를 했다가 철회했고, 이 장은 철회를 판정으로 확정한다. 셋째, 어느 경로가 “용량이 크다”는 순위. 세 경로의 상태는 단위가 같아도 세는 대상이 달라 순위가 성립하지 않는다. 넷째, 지연 · 처리량 같은 시간 축의 값 — 이 장은 바이트만 다루고, 시간 축은 ch25가 소유한다.

한 가지를 더 못 박는다. 이 장은 “얼마나 답을 수 있는가” (ch09가 세운 $\text{mem}_U \leq H(\Theta)$ 천장)와 “얼마를 저장해야 하는가” (이장의 바이트)를 **다른 양으로** 취급한다. ch09의 Worked micro-example이 8B 앵커에서 세 수를 나란히 놓았듯이 — 공유 가중치 약 16 GB, 그 안의 $r=16$ LoRA delta 16.78 MB, 그 파라미터수가 시사하는 정보 규모 3.82 MB (3.64 bits/param을 저장 크 delta에 곱한 **외삽**이다 → ch09 § 09.4) — 이 셋은 같은 대상의 세 측면이지 같은 양이 아니다. 혼동하면 이장의 모든 판정이 무너진다.

26.2 증거

증거는 넷에서 온다: ch09의 용량 이론, ch19의 편집 상한, ch18의 W 증성장, 그리고 이 책의 실험 X2. 각 항목의 등급을 먼저 고정한다. 등급 규칙은 이 책의 정직성 계약을 그대로 따른다 — **비율 · 자릿수 · 순서 · bound** 분류는 본문 단정문으로, 절대치는 하한 또는 방향성으로만 쓴다.

표 26-1 — 이 장이 쓰는 증거와 그 등급. “본문 지위” 열이 그 항목으로무엇을 단정할 수 있는지를 정한다.

#	증거	출처	등급	본문 지위
E1	판별식 통과 논문 중 C_{cap} 을 사용 자당 바이트로 보고 한 논문 0 편	ch11·ch13· ch14·ch15· ch17 전수 대조 [X2 F1]	전수 확인된 사실	단정. 단 corpus 당 바이트는 [RAG] 에 있다 (→ ch10 § 10.4)
E2	$\text{mem}_U \leq H(\Theta)$ (그릇 상한)	ch09 § 09.3 [lm- memorization- capacity Prop. 1]	논문의 정리	단정. 단 i.i.d. 가정
E3	$\text{bpp} = 3.64$ 는 하 한이다	ch09 § 09.2.2 [lm- memorization- capacity § 3.2]	논문 자신의 명시	단정 (방향만). 값 은 밴드
E4	LoRA delta 에 bpp 를 곱하는 것 은 외삽	ch09 § 09.4	이책의 판정	단정
E5	MEMIT 10,000 편집 = 관측된 하한, 7.44 hr	ch19 § 19.6.2 [MEMIT § 5.2.2]	논문 수치	단정
E6	직접 편집은 delta 아티팩트 를 만들지 않는다	ch19 § 19.7 [MEMIT Algorithm 1]	구조적 귀결	단정
E7	Memory Caching 의 상태 는 문맥 길이에 선 형이다	ch18 § 18.7 [MC § 3.1, § 4.2]	논문의 구성	단정
E8	같은 앵커에서 MC 상태는 KV cache 의 약 4 배	ch18 § 18.7	이 책의 산술, 배 치 가정 의존	방향 · 자릿수만
E9	경로별 상태 바이 트 (앵커 2 종)	[X2 Q1]	회계 (측정 아님). 결정론적	자릿수 차이만 단정
E10	저장 바이트당 정 보 밀도의 dtype 의존	[X2 Q2]	회계. 분모만으로 결정	배수 · 순서 단정, 값은 외삽된 하한
E11	사용자 수 스케 일링	[X2 Q3]	회계. 선형 외삽	모양 · 자릿수만
E12	네 가지 서빙 구조 의 batching 성 립 여부	[X2 Q4]	구조적 분류	단정
E13	X2 초판의 경로 간 밀도 비교 철회	[X2 F3_withdrawn]	이 책의 자기 정정	단정 (철회 사실)

세 항목에 대해서는 등급의 세부를 더 적어야 한다. 회계의 신뢰도가 항목마다 다르기 때문이다.

E9 의 세 행은 인식론적 지위가 서로 다르다. Θ -경로 LoRA delta 의 바이트는 구성 파라미터에
서 정확히 나온다 — $N_{\text{params}} = 2rd|S|L_{\text{layer}}$ 에 dtype 폭을 곱하면 끝이고, 여기에 근사가 없다 (→
ch09 Worked micro-example). 반면 E -경로의 바이트는 가정치다. 논문들은 토큰 수만보고하
므로 (Mem0 의 LOCOMO 대화당 7,000 – 14,000 토큰이 이 corpus 에서 얻을 수 있는 유일한 상

태랑 수치다 [X2 sourced_constants; Mem0 § 4]) 토큰을 바이트로 옮기는 환산율은 이 책이 골랐다. 그리고 dense 앵커의 W -경로 행은 반사실이다. 아래에서 다시 못 박는다.

E -경로 가정의 취약성은 한 겹 더 있다. ch10 이 확인한 대로 RAG의 C_{cap} 은 **corpus 당 36 GB / 약 100 GB**였고, ch13 에서 MemGPT 로 넘어가며 단위가 **사용자당**으로 바뀌는바로 그 지점에 숫자가 사라진다 ($\rightarrow$ ch10 § 10.4, ch13). 즉 E -경로에는바이트가 인쇄된 논문이 하나 있지만 그 논문의 단위가 이 장의 단위와 다르다. 표 26-2 의 E 행이 사용자당 값이므로 100 GB 와 나란히 놓을 수 없고, 그 두 수를 비교하는 서술은 이 장에 없다.

E10 의 분자와 분모가 비대칭이다. 분자인 3.64 bits/param 은 하한이고 (E3), 게다가 그것을 저랭크 delta 에 적용하는 것은 외삽이다 (E4). 분모인 저장 바이트는 dtype 이 정확히 정한다. 따라서 이 비율은 외삽된 양에 대해 “적어도 이만큼”이며, 위가 아니라 **아래가 막혀 있다**. 이 비대칭이 § 26.3.4 에서 결정적으로 작동한다.

E11 은 선형 외삽이다. 사용자 N_u 명에 대해 총량을 $N_u \times$ (사용자당 상태) 로 놓았다 (N_u 는 사용자 수. 전역 n 은 저장된 항목 수이므로 여기에 쓰지 않는다 — ch29 가 같은 기호를 쓴다). 실서빙에서 활성 사용자 비율 $\cdot$ delta 보유율 $\cdot$ 중복제거를 고려하면 이 곱은 상한 쪽으로 치우친다. 그러므로 이 항목에서 단정할 수 있는 것은 자릿수와 **선형이라는 모양**이지 TB 값이 아니다.

마지막으로 이 장이 상속하는 이론 제약을 두 줄로 되짚는다. 첫째, 어떤 $(U-\Theta)$ 절차도 $H(\Theta)$ 를 넘겨 쓸 수 없다 (E2). 둘째, 상태가 과제의 정보 요구량보다 작으면 $\Theta \cdot W \cdot E$ 어디에 두든 같은 형태의 오류율 바닥을 맞는다 — Fano 하한은 압축 기억의 종류를 가리지 않는다 ($\rightarrow$ ch09 § 09.6 [rate-distortion-compaction § 2]). **이 두 제약은 이장의 바이트 회계와 독립이다.** 바이트를 늘려도 첫째가 걸리고, 바이트를 줄이면 둘째가 걸린다. 이 장의 회계는 그 사이의 실무 구간을 쟀다.

26.3 분석

26.3.1 세 경로의 상태를 하나의 표에 놓는다

앵커는 둘이다. 하나는 Neural Memory 모노그래프가 쓴 neural-mem-1.3B($d = 2048$, $L_{\text{layer}} = 24$, fast weight 있음) 이고, 다른 하나는 Θ -경로 논문들이 실제로 실험한 규모대 안의 대표점인 dense-8B($d = 4096$, $L_{\text{layer}} = 32$) 다 — [SEAL] 은 Qwen2.5-7B 까지, [LM Need Sleep] 은 Qwen3-8B 까지, [MEMIT] 은 GPT-J 6B 와 GPT-NeoX 20B 를 쓴다 ($\rightarrow$ ch19·ch20·ch21). 두 앵커를 쓰는 이유는 W -경로의 연속성을 유지하면서 Θ -경로 논문들의 실제 스케일을 함께 보기 위해서다.

표 26-2 — 앵커 두 종에서 경로별 상태바이트 $\cdot$ 수명 $\cdot$ 상주 위치 [X2 Q1]. LoRA 설정은 $r = 16$, $|S| = 2$ (query-value 사영), bf16 이다. E -경로는 대화당 14,000 토큰 기준. **dense-8B 의 W 행은 반사실이다** — 그 앵커는 dense transformer 이므로 W 층이 존재하지 않는다.

경로	앵커	상태 바이트	수명	상주 위치	이 값의 지위
E (텍스트)	두 앵커 공통	56.0 KB	세션~사용자	콜드/벡터 스토어	가정 (토큰 $\rightarrow$ 바이트 환산)
E (임베딩, fp32, 512-토큰 청크)	두 앵커 공통	168 KB	세션~사용자	콜드/벡터 스토어	가정
Θ delta (LoRA)	1.3B	6.29 MB	사용자 (영속)	웜 $\rightarrow$ 요청 시 로드	구성에서 정확
Θ delta (LoRA)	dense-8B	16.78 MB	사용자 (영속)	웜 $\rightarrow$ 요청 시 로드	구성에서 정확
Θ 직접 편집	1.3B	delta 0 $\cdot$ 개인 화 시 2.6 GB	배포본 (영속)	체크포인트 전체	구조적 귀결
Θ 직접 편집	dense-8B	delta 0 $\cdot$ 개인 화 시 16.0 GB	배포본 (영속)	체크포인트 전체	구조적 귀결

경로	앵커	상태 바이트	수명	상주 위치	이 값의 지위
W fast weights	1.3B (실재)	3,220.8 MB	세션 (transient)	HBM 상주, 토큰마다 RMW	NM 앵커에서 이송
W fast weights	dense-8B (반사실)	17,177.6 MB	세션 (transient)	—	반사실. 자릿수 비교용

반사실 행에 대해서는 한 문단을 따로 쓴다. dense-8B 에는 W 층이 없다. 17,177.6 MB 는 “같은 폭 · 같은 깊이의 TTT 류 모델이라면” 이라는조건문의 값이지 Llama-3-8B 급 모델이 실제로 들고 있는 상태량이 아니다. 이행을 표에 남기는 이유는 하나 — **경로 사이의 자릿수 차이가 앵커를바꿔도 유지되는지를 보기 위해서**다. 그 목적 외의 용도로 이 수를 인용하면오용이다 [X2 Q1 counterfactual_caveat].

자릿수부터 읽는다. dense-8B 앵커에서 E 56.0 KB $\rightarrow \Theta$ delta 16.78 MB $\rightarrow W$ 17.18 GB 이며, 양 끝의 비는 306,743 배, 즉 약 **5.5 자릿수**다 [본서 산술]. 1.3B 앵커에서는 56.0 KB $\rightarrow$ 6.29 MB $\rightarrow$ 3.22 GB 로 약 4.8 자릿수다 [본서 산술]. 두 앵커에서 순서가 같고, **큰 앵커에서격차가 더 벌어진다**.

[해설] 이 표를 Rosetta 사전의 “checkpoint 크기 $\leftrightarrow C_{\text{cap}}$ ” 행으로 옮기면 세 경로가 세 계층으로흘어진다. E 는 checkpoint 회계에 아예 나타나지 않고 (모델 밖이다), Θ delta 는 checkpoint 옆에 붙는 수 MB 짜리 별도 파일이며, 직접편집은 checkpoint 그 자체다. W 는 checkpoint 가 아니라 **런타임할당**이다. 네 가지가 같은 단어로 불러왔다는 것이 이 corpus 의 상태다.

26.3.2 자릿수가 벌어지는 이유는 세 경로가 서로 다른 변수를타기때문이다

바이트 차이는 우연이 아니라 각 상태가 무엇에 비례하는지에서 나온다. 세식을 나란히 쓴다. b 는 저장 dtype 의 바이트 폭, n 은 저장된 항목수다.

$$C_{\text{cap}}^{(E)} \propto n, \quad C_{\text{cap}}^{(\Theta)} = 2rd|\mathcal{S}|L_{\text{layer}} \cdot b, \quad C_{\text{cap}}^{(W)} \propto d^2 L_{\text{layer}} \cdot b \quad (26-1)$$

세 식의 변수 집합이 겹치지 않는다는 것이 요점이다. E 는**모델과 무관**하다 — 폭을 두 배로 늘려도 저장한 텍스트는 그대로다. Θ delta 는 d 에 **선형**이고, W 는 d 에**제곱**이다. 앵커를 1.3B 에서 8B 로 옮길 때 d 가 2 배, L_{layer} 가 $32/24 = 1.33$ 배이므로 배수는 Θ 가 $2 \times 1.33 = 2.67$ (6.29 $\rightarrow$ 16.78), W 가 $4 \times 1.33 = 5.33$ (3,220.8 $\rightarrow$ 17,177.6) 이다 [본서 산술]. 이 일치를 예측의 확인으로 읽으면 안 된다 — 표 26-2 의 두 행이 애초에 이지수로 만들어졌으므로 일치는 구성에 의한 항등이고, 확인되는 것은 산술의무모순성이지 지수의 타당성이 아니다 [본서 관찰]. 단정할 수 있는 것은지수가 서로 다르다는 사실 하나이며, 거기서 따라오는 것이 이것이다 —**모델이 커질수록 세 경로의 상태는 서로 멀어진다**.

W -경로에는 두 번째 성장 축이 있다. ch18 이 Memory Caching 에서 확인한대로, 고정 크기 상태라는 W -경로의 간판은 recall 을 얻는 대가로포기된다 — 균일 세그먼트에서 캐시 개수가 $N = L/C$ 이므로 상태 바이트가**문맥 길이에 선형**으로 자란다 ($\rightarrow$ ch18 § 18.7 [MC § 3.1, § 4.2]). ch18 의 산술로 1.3B 급 제원 · 문맥 16K · 세그먼트 256 이면 시퀀스당 약 9.7 GB 이고, 같은 모델의 KV cache 약 2.4 GB 의 약 4 배다. X2 의 3.22 GB 와 ch18 의 9.7 GB 는 앵커도 배치 가정도다르므로 나란히 놓지 않는다. 나란히 놓을 수 있는 것은**방향**이다— 어느 가정에서도 W -경로의 세션당 상태는 GB 단위이고 HBM 에 살며, 다른 두 경로보다 최소 두 자릿수 크다.

직접 편집으로 개인화하는 경우는 (26-1) 의 어느 항에도 들어가지 않고별도의 지수를 갖는다 — 사용자당 상태가 모델 전체이므로 $|\Theta|$ 에, 즉 $d^2 L_{\text{layer}}$ 에 비례한다. W 와 **같은 지수인데 수명이 다르다**. W 의 $d^2 L_{\text{layer}}$ 는 세션과 함께 사라지지만 직접 편집 사본의 $d^2 L_{\text{layer}}$ 는 사용자와 함께 남는다. 상태 회계에서 가장 나쁜조합은 큰 지수와 긴 수명의 결합이고, 네 가지 구조 중 이 조합에 해당하는 것은 이 하나뿐이다 [본서 관찰].

수명과 상주 위치를 곱하면 각 경로가 실제로 무엇에 제약되는지가 갈린다. W 는 **transient × 핫이므로 총량이 아니라 동시성에 걸린다** — 상태가 세션과 함께 사라지므로 누적되지 않는 대신, 활성 세션 하나하나가 HBM 을 GB 단위로 점유하므로 동시 세션 수가 상태 바이트로 제한된다 (→ ch18 § 18.7). Θ **delta 는 영속 × 읽이므로 총량과 스테이징 지연 둘 다에 걸린다** — 사용자 수에 선형으로 누적되면서 요청마다 끌어와야한다. E 는 **영속 × 콜드이므로 총량에만 걸린다** — 누적되지만 값이 작고 읽기는 질의당 한 번이다. 세 제약의 종류가 다르므로 셋을 같은 예산 항목으로 합산할 수 없다. 이것이 § 26.1 에서 “용량” 의 순위가 성립하지 않는다고 못 박은 이유의 절반이다 (나머지 절반은 세는 대상이 다르다는 것이다).

Θ -경로에도 두 번째 축이 있고, 그것이 rank 다. 같은 8B 앵커에서 $r = 4$ 면 4.19 MB, $r = 16$ 이면 16.78 MB, $r = 64 \cdot |S| = 4$ 면 134.22 MB 다 [X2 Q1]. 32 배 범위다. 그런데 ch09 가 옮긴 실측은 반대 방향을 가리킨다 — 파라미터당 효율은 작은 rank ($r \approx 4$) 에서 최대이고, 모듈을 여러개 병합해 용량을 키우는 축은 $N_m = 1$ 에서 최고이며 그 뒤로 단조악화한다 (→ ch09 § 09.4 [lora-as-knowledge-memory § 6 Q11]). **바이트를 아끼는 선택과 파라미터 효율이 좋은 선택이 같은 쪽에 있다는 뜻이고, 이것은 이 corpus 에서 드문 정렬이다.**

26.3.3 그 바이트에 무엇이 담기는가 — 정보 밀도는 dtype 이정한다

여기서 질문의 세 번째 절이 시작된다. 바이트 수를 세는 것과 그 바이트가무엇을 담는지는 다른 문제다.

표 26-3 — dense-8B 앵커, $r = 16$, $|S| = 2$, delta 파라미터 8,388,608 개일 때 저장 형식별 정보 밀도 [X2 Q2]. 유보가 두 겹 붙는다. **세 값은 전부 하한이다** — 분자인 3.64 bits/param 이 원 논문이 명시한 하한이기 때문이다 (→ ch09 § 09.2.2). 그리고 **그 분자를 저랭크 delta 에 곱하는 것 자체가 외삽이다** (표 26-1 의 E4) — 3.64 는 전 파라미터 자유도의 from-scratch 학습에서 나온 값이고 저랭크 부분공간에서 재측정된 적이 없다 (→ ch09 § 09.4). 따라서 세 값은 **외삽된 양의 하한**이며, 이절의 단정은 값이 아니라 행 사이의 배수에만 걸린다.

저장 형식	저장 크기	bits / 저장 바이트 (하한)
bf16	16.78 MB	1.82
int8	8.39 MB	3.64
int4	4.19 MB	7.28

이 표에서 단정할 수 있는 것과 없는 것이 갈린다. **단정할 수 있는것은 세 행의 순서와 배수다.** 분자가 무엇이든 분모는 dtype 이 정하므로, 저장 폭을 절반으로 내리면 저장 바이트당 정보 밀도가 정확히 두 배가 된다. 이 진술은 3.64 가 하한이든 상한이든, 심지어 그 값이 완전히 다른 수로 바뀌어도 성립한다 [X2 Q2 what_survives]. 단정할 수 없는것은 세 값 자체다 — 1.82 는 “적어도 1.82” 이지 “1.82” 가 아니다.

그리고 이 표의 핵심은 세 행의 비교가 아니라 **분자와 분모가 서로독립으로 정해진다는 사실**이다. 파라미터 하나에 담기는 데이터셋 정보는 **최소** 3.64 bits 인데, bf16 저장은 담긴 양이 얼마든 그 파라미터에 16 bits 를 쓴다. 간극의 **크기**는 여기서 확정되지 않는다 — 분자가 하한이므로 이 논증이 보장하는 것은 간극이 $16/3.64 \approx 4.4$ 배를 **넘지 않는다**는 것뿐이고, 실제정보량이 하한보다 크면 간극은 그보다 작다 [본서 추론]. 확정되는 것은 **크기가 아니라 원인**이다 — **저장 폭을 정하는 것은 알고리즘이 아니라 dtype 선택이다.**

표에 E 행이 없는 것도 의도다. E -경로 텍스트의 8 bits/byte 는 UTF-8 인코딩의 **액면가**이지 그 텍스트가담은 과제 정보량이 아니다 — 같은 56 KB 가 조밀한 사실 목록일 수도 있고 잉여로 가득한 대화 로그일 수도 있으며, 어느 쪽인지를 재는 자가 이 회계에 없다. 3:1 압축을 가정하면 2.667 로 내려가는데 그 가정도 이책의 것이다 [X2 Q2]. Θ 쪽은 **부등호가 붙은 추정**이고 E 쪽은 **부등호가 없는 액면가**라는 이 비대칭이 다음 절의 철회를 낳았다.

ch09 가 옮긴 보강 증거가 이 진단을 뒷받침한다. 정밀도를 bf16 에서 fp32 로 두 배 올리면 저장 바이트는 두 배가 되는데 측정된 용량은 3.51 에서 3.83 으로 **9.1%만** 오른다 (→ ch09 § 09.2.2 [lm-memorization-capacity § 3.2]). 원 논문의 표현으로 “추가된 모델 bits 의 대부분은 원시 저장에 쓰이지 않는다” . 즉 그릇을 키워도 내용이 따라오지 않는다 — 여유는 실재한다.

[평가] 그러므로 delta 양자화는 Θ -경로 배포의 최적화 항목이 아니라 전제다. 사용자당 상태의 크기를 정하는 1 차변수가 rank 도 적용 행렬 집합도 아니라 저장 dtype 이고, 그 변수는 알고리즘 설계와 독립적으로 조절된다. 그런데 **corpus** 의 어느 논문도 **delta** 를 양자화한 상태에서 지식 회수율을 측정하지 않았다 [X2 F3]. LoRA 원문에는 delta 의 load 시간도 대역폭도 나오지 않으며 ($\rightarrow$ ch03 § 03.6), Θ -경로 sleep 논문들 (ch20·ch21) 도 산출물의 저장형식을 논의하지 않는다. Θ -경로를 서버에 올리는 데 가장먼저 필요한 실험이 corpus 에 0 건이다.

26.3.4 이 책 자신의 정정 — 무엇이 죽고 무엇이 살아남았는가

앞 절의 표에는 원래 한 열이 더 있었다. X2 초판은 3.64 bits/param 을 상한으로 놓고 Θ 와 E 의 저장 밀도를 나눠 경로 간 비를 인쇄했다 — bf16 에서 1.47, int8 에서 0.73, int4 에서 0.37 이며, 읽으면 “ E 가 Θ 보다 바이트 효율이 높다” 가 된다. 그 비교는 철회했다 [X2 F3_withdrawn].

철회의 근거는 원 논문의 한 문장이다. How much do language models memorize 는 자기 수치의 방향을 스스로 밝힌다 — “we are only ever measuring a lower bound on model capacity” [lm-memorization-capacity § 3.2]. gradient descent 가 전역최적을 찾는다 는 보장이 없으므로 측정된 것은 언제나 도달점이지 천장이 아니다. ch09 가 이 문장을 3.64 bpp 의 정보 서술로 고정했고 ($\rightarrow$ ch09 § 09.2.2), 그 정본이 X2 초판의 산술과 충돌했다.

방향이 왜 결정적인지를 한 줄로 적는다. 하한과 액면가를 나눈 비는 어느 방향으로도 결론을 주지 않는다. 분자 쪽 Θ 값은 “적어도 이만큼” 이고, 분모 쪽 E 값은 UTF-8 텍스트의 액면가 8 bits/byte (또는 3:1 압축 가정에서 2.667) 다. 부등호가 한쪽에만 붙은 두 수의 비는 부등호를 상속하지 못한다. 초판은 Θ 쪽 값을 확정치로 취급해서 비를 만들었고, 그 순간 Θ 층의 용량을 알 수 없는 만큼 과소평가했다.

무엇이 죽고 무엇이 살아남았는지를 명시적으로 가른다.

죽은 것 하나. 경로 간 정보 밀도의 순서. “ E 가 Θ 보다 바이트당 정보를 더 많이 담는다” 는 문장은 이 회계로 지지되지 않으며, 반대문장도 마찬가지다. 이 죽은 판정 불가다.

살아남은 것 넷. (i) dtype 층의 결론 (§ 26.3.3) — 분모만으로 정해지므로 분자의 지위와 무관하다. (ii) 표 26-2 의 자릿수 차이 — 저장바이트를 세는 회계이지 정보량을 세는 회계가 아니므로 3.64 가 아예 등장하지 않는다. (iii) 사용자 수 스케일링의 모양 (§ 26.3.5). (iv) 서버 구조 분류 (§ 26.3.6). 즉 X2 의 네 질문 중 하나의 절반만 무너졌고 나머지는 그대로다.

[평가] 이 정정을 신는 것은 이 책의 규칙이다. Part II 는 논문들에게 같은 기준을 요구했다 — ch18 은 Nested Learning 과 Memory Caching 의 “모든 구성요소가 기여한다” 는 캡션을 그 논문 자신의 표로 반증했고, ch21 은 Language Models Need Sleep 의 Semantic Reward ablation 역전을 그대로 실었으며, ch15 는 Mem0 의 full-context baseline 우위를 본문에 옮겼다. 자기 실험에 같은 자를 대지 않으면 그 요구가 성립하지 않는다. ch09 가 이 사실을 이미 기록했고 ($\rightarrow$ ch09 § 09.7), 이 장은 그것을 반복이 아니라 판정으로 처리한다 — 이 사건이 정한 것은 수치 하나가 아니라 이 회계로 판정 가능한 축과 불가능한 축의 경계다.

같은 방향의 잔재가 한 군데 더 남아 있어 여기서 정정한다. X2 의 서술필드는 “파라미터 하나에 최소 3.64 bits 가 담기는데 bf16 은 16 bits 를 쓴다 — 표현 자체에서 최소 4.4 배의 저장 여유가 나온다” 고 적는다. 배수의 방향이 반대다. delta 가 담은 정보량을 I 라 하면 무손실저장에 필요한 것은 I bits 이상이고, bf16 저장은 $16|\Delta\Theta|$ bits 이므로 이 논증이 허가하는 압축 배수는 $16|\Delta\Theta|/I \leq 16/3.64 \approx 4.4$ 다 — 즉 4.4 배는 상한이며, 실제 정보량이 하한보다 크면 허가되는 압축은 그보다 작아진다 [본서 추론]. 다만 § 26.3.3 의 판정은 이 배수에 의존하지 않는다. 그 판정은 저장 폭이 담긴 정보량과 독립으로 정해진다는 사실에서만 나오고, 격차의 크기를 쓰지 않는다.

26.3.5 사용자 수를 곱하면 어느 계층이 문제가 되는가

사용자당 수 MB 는 작다. 문제는 개수다.

표 26-4 — dense-8B 앵커에서 사용자 수별 웹 스토리지 총량 (TB) [X2 Q3]. LoRA 는 query-value 사영에 적용. 선형 외삽이며 **모양과 자릿수만** 읽는다.

사용자 수	$r=4$ bf16	$r=16$ bf16	$r=16$ int4	$r=64$ bf16	직접 편집 (전체 사본)	E 텍스트
10^3	0.004	0.017	0.004	0.067	16.0	5.6×10^{-5}
10^5	0.419	1.678	0.419	6.711	1,600	0.0056
10^6	4.194	16.777	4.194	67.109	16,000	0.056
10^8	419.43	1,677.72	419.43	6,710.89	1,600,000	5.6

세 가지를 읽는다. 첫째, **delta 방식의 총량은 데이터센터 규모에서 크지 않다**. 사용자 10^6 명에 $r=16$ bf16 이면 16.777 TB 이고 int4 로 내리면 4.194 TB 다. 둘째, **직접편집으로 개인화하면 세 자릿수가 된다** — 같은 사용자 수에서 16,000 TB, 즉 16 PB 다. 셋째, E -경로는 다른 세계에 있다 — 0.056 TB 로 delta 방식보다 두 자릿수 이상 작다.

[해설] 그러므로 Θ -경로가 여는 memory-device 기회의 성격이 정해진다. 문제는 **용량 총량이 아니라 배치와 지연**이다. 16.777 TB 는 어디에 두어도 담기지만, 그 안의 16.78 MB 하나를 요청 지연 안에 끌어와 base 가중치와 합쳐야 한다. 필요한 것은 HBM 대역 폭이 아니라 **요청 지연 안에 delta 를 스테이징할 수 있는 웹 계층과 그 계층의 쓰기 내구성**이다 [X2 F4].

쓰기 쪽 부하는 갱신 주기가 정한다. sleep 라운드가 하루 1 회이고 사용자 10^6 명이 각자 delta 를 갱신하면 라운드당 쓰기가 16.777 TB, 즉 약 16.8 TB/day 다 [본서 산술 — 주기는 이 책의 가정이고 논문에서 오지 않는다]. 이 값은 읽기 트래픽과 성격이 다르다. 읽기는 요청마다 수 MB 의 랜덤 접근이고, 쓰기는 라운드마다 전량 교체다. **읽기 지연과 쓰기 내구성이 서로 다른 축에서 걸린다**.

읽기 쪽은 축 자체가 다르다. Θ -경로의 읽기는 **요청당 delta 로드 한 번**이고, W -경로의 읽기는 **토큰당 전 상태 read-modify-write** 다 [X2 Q4 traffic_character]. 1.3B 앵커에서 512 토큰을 생성하는 요청 하나를 놓고 보면 W -경로는 $512 \times 6.44 = 3,297$ GB, 즉 약 3.3 TB 를 움직이고 Θ -경로는 6.29 MB 를 한 번 움직인다 — 약 5.2×10^5 배다 [본서 산술]. 이 비를 “ Θ 가 싸다” 로 읽으면 안 된다. 두 값은 다른 축의 값이고, Θ -경로가 지불하지 않는 것은 그 대신 sleep 라운드에서 미리 지불했기 때문이다. **읽어야 할 것은 배수가 아니라 축의 분리**다 — W 의 비용은 생성길이에 비례하고, Θ 의 비용은 요청 수에 비례하며, E 의 비용은 바이트가 아니라 prefill 토큰으로 나타난다. 세 축을 하나의 회계로 합치는 일은 이 장이 아니라 ch25 가 한다.

[평가] 이 자리는 Neural Memory 가 W -경로에서 짙은 HBM read-modify-write 기회와 **다른 자리**다. 두 결론을 하나로 잊지 않는다. 오히려 이 장의 회계가 보이는 것은 분리다 — 식 (26-1) 의 세 지수가 세 경로를 메모리 계층의 세 층에 각각 내려놓는다. W 는 핫 (HBM, 토큰당 RMW), Θ 는 웹 (요청당 수 MB 스테이징), E 는 콜드 (질의당 1 회 검색; ch10 이 확인한 대로 RAG 의 index 는 host DRAM 에 산다 → ch10 § 10.4). **경로 선택은 알고리즘 선택이면서 동시에 메모리 계층 선택이며, 후자는 전자가 정한다**.

26.3.6 직접 편집 대 delta — 서빙 갈림길을 여기서 확정한다

ch19 가 갈림길의 위치를 지목하고 정량 판정을 이 장으로 넘겼다 (→ ch19 § 19.8.2). 여기서 닫는다.

표 26-5 — 네 서빙 구조에서 shared-weight batching 의 성립 여부와 트래픽 성격 [X2 Q4].

구조	shared-weight batching	트래픽 성격
E -경로	유지된다 . 가중치는 전부 공유, 상태는 프롬프트로 들어온다	질의당 1 회 검색 + prefill 증가. 읽기 편향

구조	shared-weight batching	트래픽 성격
W-경로	부분적으로 깨진다. 세션마다 별도 상태가 HBM에 상주	토큰당 6.44 GB RMW(1.3B 앵커). 쓰기 편향, 공유 불가
Θ -경로 delta	delta 단위로 깨진다. 배치 안의 사용자가 다르다면 가중치가 다르다	요청당 delta 로드 (수 MB). 갱신은 sleep 라운드에만. 읽기 편향
Θ -경로 직접 편집	완전히 깨진다. 개인화하려면 모델전체 사본	사용자당 체크포인트

마지막 행이 갈림길이다. MEMIT·ROME 류의 갱신은 $\Theta_{\text{out}}^{(l)} += \Delta^{(l)}$ 라는 같은 shape 의 제자리 덧셈이므로 ($\rightarrow$ ch19 § 19.7 [MEMIT Algorithm 1]) 편집을 10 개 하든 10,000 개 하든 체크포인트 크기가 같다. 상태량이 0 이라는 것이 이 방식의 겉보기 장점이고, 실제로 편집이 **전역**이면 — 10,000 개 사실이 모든 사용자에게 같은 사실이면 — 그 장점은 진짜다. 가중치 1 벌을 계속 공유하므로 batching 이 살아남고, ch19 가 쓴 “한 번 지불하고 무한히 상각” 이 성립한다.

개인화를 요구하는 순간 그 0 이 전체 사본으로 바뀐다. 분리 가능한 아티팩트가 애초에 생기지 않기 때문에 사용자마다 다른 사실을 넣으려면 사용자마다 모델 하나가 필요하고, 그 값이 8B 앵커에서 16.0 GB 다 [X2 Q1]. 표 26-4 의 세 자릿수 점프가 이 한 문장에서 나온다. 그리고 함께 사라지는 것이 둘 더 있다 — 되돌리기 (편집 취소 연산이 어느 논문에도 없다) 와 다중 배포본의 분리 (편집 집합이 다른 두 배포본은 완전히 별개의 체크포인트다). 셋 다 ch19 가 확인한 사실이다.

쓰기 쪽에서도 같은 갈림길이 나타난다. ch19 가 옮긴 대로 MEMIT 은 GPT-J 6B·A6000 에서 10,000 편집에 7.44 시간을 쓰고, 편집당으로는 2.6773 초다 ($\rightarrow$ ch19 § 19.5 [MEMIT § 5.2.2, App. B.4]; 편집당 값은 이 책의 나뉠셈이다). 이 절대치를 다른 모델·다른 가속기로 옮기는 것은 외삽이므로 여기서는 쓰지 않는다. 쓸 수 있는 것은 **상각의 구조**다. 편집이 전역이면 그 7.44 시간은 전 사용자에게 상각되지만, 개인화하면 사용자마다 따로 지불되고 상각의 분모가 1 이 된다. ch19 가 쓴 “한 번 지불하고 무한히 상각” 이 개인화 아래에서 “사용자마다 한 번씩 지불하고 그 사용자에게만 상각” 으로 바뀐다. **분리 가능한 아티팩트가 없다는 것은 저장이 없다는 뜻이 아니라 상각이 없다는 뜻이다.** delta 방식은 반대다 — delta 하나가 그 사용자의 이후 모든 요청에 재사용되므로 식 (A) 의 분모가 그 사용자의 질의 수 N_q 로 살아난다.

[평가] 그러므로 두 방식은 같은 경로의 두 구현이 아니라 **다른 용도의 두 도구**다. 직접 편집은 전역 사실 갱신의 도구이고, delta 는 개인화의 도구다. 이 구분을 제기한 논문이 corpus 에 없다 [X2 F5] — MEMIT 은 편집을 개인화 맥락에서 논하지 않고, LoRA 는 배포 경제학을 풀었지 기억 문제를 풀지 않았다 ($\rightarrow$ ch19 § 19.8.2).

갈림길의 양쪽 대가를 대칭적으로 적어야 공정하다. delta 를 택하면 batching 이 delta 단위로 깨지고, 요청마다 사용자 delta 를 로드·머지해야 하며, 서로 다른 delta 를 쓰는 두 요청은 같은 GEMM 을 공유하지 못한다 ($\rightarrow$ ch09 § 09.4). 이것도 실제 비용이다. 그러나 두 대가의 **종류**가 다르다 — delta 쪽 붕괴는 라우팅 문제라서 같은 delta 를 쓰는 요청을 모으는 스케줄링으로 완화되지만, 전체 사본은 용량 문제라서 완화 수단이 구조적으로 없다 [본서 추론]. 완화 가능한 대가와 완화 불가능한 대가를 같은 저울에 올리지 않는다.

26.4 판정

[평가] 판정 1 — “**상태 용량**” 은 하나의 수가 아니다. 세 경로의 상태를 한 숫자로 비교하는 것은 **범주 오류**다. 표 26-2 의 세 항목은 단위만 같고 세는 대상이 다르다. E 는 저장한 항목 수에 비례하고 모델과 무관하며, Θ delta 는 rdL_{layer} 에 비례하고 저장한 항목 수와 무관하며, W 는 d^2L_{layer} 에 비례한다 (식 (26-1)). 수명도 다르다 — W 는 세션과 함께 사라지고, Θ 와 E 는 사용자와 함께 남는다. 그러므로 “어느 경로의 용량이 큰

가”는 답할 수 없는 질문이고, 답할 수 있는 질문은 “어느 경로가 어느 계층에 얼마를 얼마 동안 두는가”다. 이 판정에 조건은 없다.

[평가] 판정 2 — 세 경로는 메모리 계층의 세 층에 각각 내려앉으며, 그 배치는 알고리즘이 아니라 식 (26-1)의 지수가 강제한다. W 는 핫이다 — 세션당 GB 단위이고 토큰마다 read-modify-write 이므로 HBM 밖에 둘 수 없다. Θ delta는 웜이다 — 사용자당 수 MB이고 요청지연 안에 한 번 스테이징되면 충분하다. E 는 콜드다 — 사용자당 수십 KB이고 질의당 한 번 읽힌다. 배치를 정하는 것은 두 앵커의 일치가 아니라 식 (26-1)의 지수 자체다 — 두 앵커의 수치가 그 지수로 만들어졌으므로 일치는 독립 관측이 아니고 (§ 26.3.2), 단정의 근거는 $n \cdot rdL_{\text{layer}} \cdot d^2 L_{\text{layer}}$ 가 서로 다른 변수라는 구조적 사실이다. 따라서 경로 선택은 곧 메모리 계층 선택이다. 이 판정의 조건은 하나 — W -경로가 고정 크기 상태를 유지한다는 전제다. ch18이 확인한 대로 recall을 얻으려고 상태를 문맥에선형으로 키우면 W 는 핫 계층에서 더 깊이 박힌다 (→ ch18 § 18.7).

[평가] 판정 3 — 저장 바이트당 정보 밀도는 표현의 한계가 아니라 저장 dtype이 정한다. 그러므로 delta 양자화는 Θ -경로 배포의 최적화가 아니라 전제다. 파라미터 하나에 담기는 데이터셋 정보는 최소 3.64 bits 인데 bf16 저장은 16 bits를 쓴다. 정밀도를 두 배로 올려도 측정된 용량은 9.1%만 오른다 (→ ch09 § 09.2.2) — 그릇을 키워도 내용이 따라오지 않는다. 저장 폭을 절반으로 내리면 밀도가 정확히 두 배가 되고, 이 진술은 분자의 값이나 그 부등호 방향과 무관하게 성립한다. 이 판정이 기대는 것은 분자가 아니라 분모다 — 분자는 하한이자 저랭크 delta 로의 외삽이므로 (→ ch09 § 09.4) 밀도의 절대치를 지지하지 않으며, 이 판정도 절대치를 주장하지 않는다. 그리고 corpus의 어느 논문도 delta를 양자화한 상태에서 지식 회수율을 측정하지 않았다 [X2 F3]. 조건: 이 판정은 저장·전송 바이트에 대한 것이고, 양자화가 회수 품질에 미치는 영향은 측정되지 않았다 — 그 측정이 § 26.5의 첫 번째 반증 실험이다.

[평가] 판정 4 — 경로 간 정보 밀도 비교는 이 회계로 판정불가이며, 이 책은 그것을 판정불가로 남긴다. X2 초판이 낸 비 (bf16 1.47 / int8 0.73 / int4 0.37)는 하한을 확정치로 취급해 만든 값이므로 철회했다 [X2 F3_withdrawn]. 어정쩡하게 남기지 않는다 — “ E 가 바이트당 정보를 더 담는다”도 “ Θ 가 더 담는다”도 이 책은 주장하지 않는다. 판정하려면 delta에 알려진 엔트로피의 정보를 주입해 회수율을 재는 실험이 필요하고, 그것은 X2의 범위 밖이다. 이 문단이 이 장에서 가장 중요한 방법론적 진술이다 — 회계는 자릿수와 배치를 판정할 수 있지만 밀도를 판정할 수 없다.

[평가] 판정 5 — Θ -경로가 개인화 서빙에 도달하려면 delta 형태여야 한다. 직접 편집은 전역 사실 갱신의 도구이지 개인화의 도구가 아니다. 직접 편집은 분리 가능한 아티팩트를 만들지 않으므로 (→ ch19 § 19.7) 사용자별 상태를 담을 그릇 자체가 없고, 담으려면 사용자당 모델 전체 사본이 필요하다 — 8B 앵커에서 16.0 GB, 사용자 10^6 명이면 16 PB로 delta 방식 대비 세 자릿수다 [X2 Q1, Q3]. 되돌리기와 다중 배포본 분리도 함께 사라진다. 조건을 명시한다: 편집이 전역이면 — 모든 사용자에게 같은 사실이면 — 직접 편집은 유효하고 상각도 성립한다. 갈림길은 알고리즘 품질이 아니라 개인화 요구의 유무가 정하며, 논문들은 이 갈림길을 논의하지 않는다 [X2 F5].

마지막으로 corpus 쪽 판정을 하나 더 붙인다.

[평가] 판정 6 — C_{cap} 의 침묵은 개별 논문의 부주의가 아니라 이 분야의 구조적 상태다. 판별식 통과 논문 중 사용자당 상태 바이트를 인쇄한 논문이 0 편이고 (E1), 하나 있는 바이트 보고는 단위가 corpus 당이어서 다른 층에 있다 (→ ch10 § 10.4). 비용 4 중 C_{cap} 은 가장 보고하기 쉬운 칸이다. B_s 는 FLOPs 계측이 필요하고 L_w 는 서빙 스택이 필요하며 ρ 는 여러 라운드의 실험이 필요하지만, C_{cap} 은 구성과 dtype만 적으면 독자가 곱셈으로 복원할 수 있다. 그런데 0 편이다. ch17의 사례가 가장 선명하다 — 그 논문은 상태가 고정 크기라는 것을 셀링 포인트로 삼으면서 그 크기를 계산할 재료를 인쇄하지 않았다 (→ ch17 § 17.5). 비교를 어렵게 만드는 것은 측정의 난이도가 아니라 비교하려는 의도의 부재다.

26.5 반증 조건

위 여섯 판정은 각각 구체적인 관측으로 틀릴 수 있다. 반증에 필요한 관측을 판정 번호와 함께 적는다.

R-1 (판정 1·2를 반증한다). 세 경로 중 둘 이상의 사용자당 상태를 같은 단위로 인쇄한 보고에서 자릿수 순서가 $E < \Theta_{\text{delta}} < W$ 가 아닌 경우. 특히 E -경로가 임베딩·그래프·다중 인덱스를 누적해 사용자당 저장장이 수십 MB를 넘어 Θ_{delta} 자릿수에 들어오면 표 26-2의 E 행이 무너진다. X2의 임베딩 환산(14K 토큰 → 168 KB)은 텍스트의 3 배에 그치지만, Zep 류의 이중 시간 그래프가 에피소드마다 노드·엣지·임베딩을 함께 쌓으면 성장률이 달라질 수 있다 (→ ch15). **필요한 관측은 실험이 아니라 인쇄 한 줄이다** — 사용자당 저장 바이트를 보고하기만 하면 된다.

R-2 (판정 3을 반증한다). rank·학습집합·학습 절차를 고정하고 delta의 저장 dtype만 bf16 → int8 → int4로 내렸을 때, int8 이전에 지식 회수율이 무너지는 측정. 그런 결과가 나오면 양자화는 “전제”가 아니라 “품질을 지불하고 바이트를 사는 거래”가 되고, § 26.3.5의 웹 계층 규모 추정에서 int4 열을 지워야 한다. 측정 형태를 명시한다 — 적재 토큰 수를 스왑하고, dtype 3 수준에서 회수율 곡선을 그리며, 회수 판정은 프롬프트 출력이 아니라 가능도비로 한다 (ch09 § 09.1 이 출력 기반 게이트가 양방향으로 틀린다는 것을 확정했다).

R-3 (판정 4의 “판정 불가”를 해소하거나 반증한다). delta에 알려진 엔트로피의 정보를 직접 주입하고 회수 가능한 bits/param 을 재는 실험. 회수량이 3.64 근방으로 수렴하면 하한이 사실상 도달점이라는 뜻이므로 경로 간 밀도 비교가 복원되고, 그 값이 3.64 보다 크게 나오면 X2 초판의 결론이 방향까지 뒤집힌다 (Θ 가 E 보다 밀도가 높아진다). 어느 쪽이든 이 장의 “판정 불가”는 사라진다. 설계 요건 셋: 데이터는 균등·독립이어야 하고 (참조 모델 문제를 없애기 위해), 학습대상은 저랭크 부분공간이어야 하며, 여러 rank에서 반복해야 한다.

R-4 (판정 5를 반증한다). 직접 편집의 $\Delta^{(l)}$ 을 분리가 가능한 아티팩트로 추출·주입·롤백하는 방법이나 오고, base가 중치를 공유한 채 사용자별 편집을 batching 안에서 적용할 수 있음을 보인 측정. 관측 형태는 둘이다 — 사용자당 저장장이 전체 사본이 아니라 편집 수에 비례하는 아티팩트로 보고될 것, 그리고 서로 다른 편집 집합을 쓰는 요청을 섞은 배치의 처리량이 전역 편집 모델 대비 유의하게 떨어지지 않을 것. 둘이 함께 성립하면 표 26-5의 마지막 행이 셋째 행으로 합쳐지고 판정 5는 폐기된다.

R-5 (판정 2의 조건절을 반증한다). W -경로 상태가 문맥 길이에 대해 $O(1)$ 을 유지하면서 recall이 KV cache 수준에 도달하는 결과. ch18이 확인한 선형 성장이 회피되면 W 행의 세션당 바이트가 내려가 Θ 자릿수와 겹칠 수 있고, 그러면 판정 2의 계층 배치가 흔들린다. 필요한 관측은 표 하나다 — 같은 모델·같은 문맥 길이에서 상태 바이트와 recall을 동시에 인쇄한 표. corpus에 그런 표가 0 개라는 것이 현재 상태다.

R-6 (판정 2의 웹 계층 논증을 반증한다). 실서빙에서 활성 delta의 working set이 사용자 수에 sublinear로 보고되는 경우. 대부분의 사용자가 delta를 갖지 않거나, delta가 집단 단위로 공유되거나, 중복 제거가 실패를 내면 표 26-4의 총량은 상한 쪽으로 크게 치우친 값이 된다. 관측: delta 보유율·재사용률·중복 제거율을 인쇄한 서빙보고. 이 반증은 판정 2의 규모를 깎지만 배치는 깎지 않는다 — 사용자 수가 줄어도 delta는 여전히 웹에 산다.

R-7 (이 장의 산술 자체를 반증한다). 같은 구성의 LoRA 체크포인트가 $2rd|S|L_{\text{layer}} \cdot b$ 와 다른 바이트로 저장되는 경우. scaling factor·bias·optimizer 상태·메타데이터가 함께 저장되어 실패 아티팩트가 2–3 배가 되면 표 26-2의 Θ 행과 표 26-4 전체가 그 배수만큼 틀린다. **이것이 가장 싼 반증이다** — 실제 체크포인트 하나의 파일 크기를 재면 끝난다. 이 장은 그 확인을 하지 않았고, 하지 않았다는 사실을 여기 적는다.

반증 목록에 들어가지 않는 것도 명시한다. 절대 바이트가 이 장의 인용값과 다르다는 관측은 반증이 아니다. 앵커 구성이 이 책의 선택이므로 다른 구성에서 다른 값이 나오는 것은 당연하다. 반증이 되는 것은 자릿수 순서·성장 지수·계층 배치·구조 분류가 어긋나는 경우다.

요약

- 판별식 통과 논문 중 상태 용량을 사용자당 바이트로 보고한 논문이 **0 편**이다 (유일한 바이트 보고인 [RAG]는 단위가 corpus 당이다→ ch10 § 10.4). 이 장의 회계는 그 공백을 구성 파라미터에서 복원한 것이며, 측정이 아니라 회계다 [X2 F1].
- 세 경로의 상태는 dense-8B 앵커에서 E 56.0 KB → Θ delta 16.78 MB → W 17.18 GB로 약 5.5 자릿수 벌어지고, 그 격차는 모델이 커질수록 커진다. dense 모델의 W 행은 **반사**이다 — 그 앵커에는 W 층이 없다.
- 격차의 원인은 세 경로가 서로 다른 변수를 타기 때문이다 — $E \propto n$, $\Theta \propto r d L_{\text{layer}}$, $W \propto d^2 L_{\text{layer}}$. 그래서 경로 선택은 곧 메모리 계층선택이다: W 는 핫, Θ 는 웜, E 는 콜드.
- 저장 바이트당 정보 밀도는 dtype 이 정한다 (bf16 1.82 / int8 3.64 / int4 7.28 bits per byte — 전부 하한이자 저랭크 delta 로의 외삽→ ch09 § 09.4). 단정하는 것은 세 값이 아니라 세 행의 배수다: 저장 폭을 절반으로 내리면 밀도가 정확히 두 배가 된다. 저장 폭이 담긴 정보량과 독립으로 정해지므로 **delta 양자화는 최적화가 아니라 전제**이며, corpus 의 어느 논문도 이를 논의하지 않는다.
- 이 책은 자기 실험의 오류를 신는다. X2 초판은 3.64 를 상한으로 놓고 경로 간 밀도 비를 계산했다가 철회했다 — 원 논문이 그 값을 명시적으로 **하한**이라고 밝힌다. 경로 간 밀도 비교는 **판정 불가**로 남긴다.
- 사용자 10^6 명 기준 웜 스토리지는 선형 외삽으로 LoRA $r=16$ bf16 에서 16.777 TB, int4 로 4.194 TB 이고, 직접 편집으로 개인화하면 16,000 TB 다. 단정하는 것은 TB 값이 아니라 **선형이라는 모양과 두 방식 사이의 세 자릿수 격차**다. 기회는 HBM 대역폭이 아니라 **요청 지연 안의 delta 스테이징과 그 계층의 쓰기내구성**에 있다.
- 서버 갈림길이 확정된다: **개인화를 요구하면 Θ -경로는 delta 형태여야 한다**. 직접 편집은 편집이 전역일 때만 유효하며, 개인화하려면 사용자당 모델 전체 사본이 필요하다.
- C_{cap} 은 비용 4 종 중 보고하기 가장 쉬운 칸인데도 비어있다. 침묵의 원인은 측정 난이도가 아니라 비교 의도의 부재다.

다음 장으로

이 장은 “얼마이고 어디에 사는가” 를 닫았다. 닫히지 않은 것은 그 바이트가 **값어치**를 하는가이다 — 사용자당 16.78 MB 의 웜 스토리지와 요청당 스테이징 지연을 지불하고 얻는 것이 무엇인지, 그리고 그 교환이 경로마다 어떻게 다른지는 용량 회계만으로 답이 나오지 않는다. ch25 가 세운 통일 비용 회계와 이 장의 용량 회계가 나란히 놓여야 비로소 “유망한가” 를 경로별로 물을 수 있고, 그것이 ch27 이 하는 일이다. ch27 은 이장에서 두 가지를 받아간다 — 세 경로가 서로 다른 계층에 산다는 사실 (그래서 하나의 순위표가 성립하지 않는다는 것) 과, 개인화 요구가 Θ -경로의 구현을 강제한다는 조건부 판정이다. 웜 계층 자체의 설계 — 무엇을 어디에 두고 어떤 지연 안에 끌어올 것인가 — 는 ch29 가 받는다. 그리고 이 장이 판정 불가로 남긴 축 하나는 ch30 의 미해결 목록으로 간다.

ch27. 유망한가 — 경로별판정

27.1 판정할 질문

sleep-time compute 은 유망한가 — 무슨 문제를 풀고, 그 해법이 최선이며, 대안 대비 우월하고, 주류가 될 것인가.

이 다섯 질문이 이 책을 시작하게 한 질문이다. ch24가 그 질문의 주어를 고쳤고, ch25와 ch26이 두 개의 자를 세웠다. 이 장은 그 위에서 답한다.

먼저 **답이 될 수 있는 것과 될 수 없는 것**을 못 박는다.

답이 될 수 없는 것 셋.

첫째, **하나의 답은 답이 아니다**. ch24의 판정 1이 이 범주를 A군 9편으로 확정했고, 그 9편은 E 6편 · W 1편 · Θ 2편으로 갈린다 (→ ch11 § 11.3.1). 세 경로는 갱신하는 층이 다르고, 상태가 사는 메모리 계층이 다르며 (→ ch26 판정 2), 상각식 (A)가 각각 다른 방식으로 퇴화한다 (→ ch25 판정 1). “sleep-time compute이 유망한가”에 예/아니오로 답하는 문장은 세 개의 서로 다른 명제를 하나로 뭉친 문장이고, 이 책은 그런 문장을 쓰지 않는다.

둘째, **성능 순위표는 답이 아니다**. 세 경로는 서로를 baseline으로 돌리지 않는다 (→ ch24 § 24.3.1). 공유 벤치마크의 교집합이 공집합이고, 예산측과 품질 측이 둘 다 깨져 있다. 순위표를 만들려면 없는 표를 지어내야 한다. 그러므로 이 장은 “어느 경로가 1등인가”에 답하지 않고, **각 경로가 무엇을 사고 무엇을 지불하는가**에 답한다.

셋째, **예측은 단정이 아니다**. “주류가 될까”는 미래에 대한 진술이므로 증거로 결정되지 않는다. 이 장은 그것을 **조건부로만** 답한다 — 무엇이 참이면 어느 경로가 이기는지를 조건으로 적고, 각 조건이 현재 어디까지 확인되었는지를 함께 적는다. 조건 없는 예측은 이 책의 문장이 아니다.

답이 될 수 있는 것 셋. 경로별 판정, 조건을 명시한 조건부 판정, 그리고 “**현재 문헌으로 판정 불가**”라는 판정이다. 셋째가 회피처럼 보인다면 그것이 오해다. 대조군이 실행되지 않은 비교에 대해 우열을 선언하는 것이 회피이고, 대조군이 없다는 사실을 명시하는 것이 판정이다. 이 장에는 판정 불가로 남는 칸이 있고, 그 칸마다 **무엇이 관측되면 판정 가능해지는지**를 § 27.5에 적는다.

다섯 질문을 답하는 순서로 다시 놓는다. **Q1** 각 경로가 실제로 푸는 문제는 무엇인가. **Q2** 그 해법은 유망한가 — 즉 문제가 실재하고 해법이 문제를 실제로 줄이는가. **Q3** 그것이 최선인가 — 같은 경로안에 더 나은 방법이 있는가. **Q4** 대안 대비 우월한가 — 경로 밖의 방법과 비교되었는가. **Q5** 주류가 될까. Q1이 먼저인 이유는 나머지 넷이 전부 Q1의 답에 상대적이기 때문이다. 무슨 문제를 푸는지 정하지 않으면 “최선”도 “우월”도 정의되지 않는다.

27.2 증거

증거는 세 곳에서 온다: Part II 13장이 원문 대조로 확정된 사실, ch24–ch26의 판정, 그리고 이 책의 실험 넷이다. 등급 규칙은 이 책의 정지성계약을 그대로 따른다 — **비율 · 순서 · 자릿수 · crossover의 존재와 이동 방향 · bound** 분류는 본문 단정문으로, 절대치는 하한 또는 방향성으로만 쓴다.

이 장이 처음 쓰는 실험이 둘 있다. 이 책의 **예산 측 감사 (실험 X3)**는 29편의 deep-read 노트를 규칙 기반으로 전수 감사해 B_s 가 어떻게 보고되는지를 센다. 등급은 AUDIT-GRADE / CORPUS-CENSUS이며 성능도 비용도 재지 않는다 — 문헌의 상태를 세는 실험이다. 이 책의 반복

consolidation 열화 실험 (실험 X4) 는가우시안 위치 모형과 선형 연상기억 위에서 (U- Θ) 를 K 라운드돌린다. 등급은 EXPLORATION-GRADE / MECHANISM-TRANSFER 이고 **LLM 실험이 아니다**. 두 실험을 본격적으로 받는 장은 각각 ch28 과 ch29 이며, 이 장은 그중 경로 판정에 직접 걸리는 항목만 인용한다.

표 27-1 — 이 장이 쓰는 증거와 그 등급. “본문 지위” 열이 그 항목으로무엇을 단정할 수 있는지를 정한다.

#	증거	출처	등급	본문 지위
A1	판별식 통과 9 편의 경로 분포는 E 6 · W 1 · Θ 2. 경제 사례 MemGPT 는 통과군에 세지 않는다	ch11 § 11.3.1, ch24 판정 1	전수 확인	단정
A2	E -경로에서 품질 1 위가 반복적으로 기억 시스템 없는 구성이다	ch15 § 15.6 [Mem0 Table 2; Zep Table 1; SCM Table 2]	논문 자기 수치	단정
A3	그 지연 · 비용 이득의 baseline 에 prefix cache 가 없다	ch15 § 15.6 [Mem0 전문에 ‘cache’ 0 회; Zep § 4.3]	부재 확인	부재는단정. 이득의 크기는 판정 불가
A4	E -경로 시스템 평가 결함 아홉 유형	표 27-2	원문 대조	항목별 단정. 종합은 [평가]
A5	E -경로는 shared-weight batching 을 보존한다	ch14 § 14.7, ch15 § 15.7	구조적 귀결	단정
A6	Letta STC 의 5× 는 짧은 문맥 · 큰 절감률 레짐에서 뒤집히지 않는다	ch25 판정 3 [X1 F2]	회계 (측정 아님)	단정 (조건부)
B1	W -경로의 A 군 논문은 1 편이다	ch11 § 11.3.1	전수 확인	단정
B2	그 1 편의 비용 4 종이 전부 논문에 없다. L_w 는 측정이 아니라아키텍처 보장으로만 있다	ch17 § 17.5	부재 확인	단정
B3	계산량을 맞춘 baseline 이 없다 — 캐시를 유지한 full attention 은 어느과제 · 어느 스케일에서도 실행되지 않았다	ch17 § 17.6	부재확인	단정

#	증거	출처	등급	본문 지위
B4	판별식 통과는 Depo 설정에서 만 성립하고, 헤드라인 설정은 $N_q = 1$ 이라 식 (A)가 붕괴한다	ch17 § 17.4	이 책의 판정	단정
C1	Θ -경로는 사실을 실제로 넣는다. 그러나 도달성은 라운드와함께 무너지고 $k = 100$ 에서 25–28%로 고원을 이룬다	ch22 § 22.6 [Continual Facts § 4.3]	논문 수치	단정
C2	A 군에서 sleep 라운드 수를 독립 변수로 놓은 실험이 0 편이다	ch25 § 25.2.1	전수 확인	단정
C3	delta를 양자화한 상태에서 회수율을 측정한 논문이 0 편이다	ch26 판정 3 [X2 F3]	부재 확인	단정
C4	Θ -경로의 쓰기 검증은 문맥마다의 라벨 또는 외부 frozen reward model에 의존한다	ch20 § 20.7, ch21 § 21.3.2	논문의 구성	단정
C5	개인화를 요구하면 Θ -경로는 delta 형태여야 한다	ch26 판정 5 [X2 F5]	구조적 귀결	단정
D1	B_s 보고 실태 — 29 편 중 19 편이 absent 이고 절대 FLOPs/토큰으로 보고한 논문이 1 편이다. 단 그 19 편 중 9 편은질의 전 오프라인 패스 자체가 없어 B_s 가 정의되지 않는 경우이므로, “보고가 부실하다”와 “범주에 속하지 않는다”를 섞어 세면 안 된다. 보고율의경로 간 순서는 $\Theta > E$ 다	실험 X3 [X3 F1, F1b, F4b]	감사	편수와 순서는 단정. 비율은 쓰지 않는다 (표본 작음)

#	증거	출처	등급	본문 지위
D2	B_s 축을 복원해도 y 축이 붙지 않는다 — 축 위에 두 점 이상을 가진 논문들의 공유 벤치마크 교집합이 공집합이다	실험 X3 [X3 F4]	감사	단정
D3	상각 여유의 레짐 간 순서는 GSM-Symbolic $\gg$ AIME $>$ SWE-Features 다	실험 X1 [X1 F4], ch25 판정 4	회계	순서만 단정. 절대 임계치 금지
D4	세 경로 상태의 자릿수 차이와 메모리 계층 배치	실험 X2, ch26 판정 1·2	회계	자릿수·배치 단정. 절대 바이트는방향성
D5	반복 (U- Θ) 에서 이송 가능한 양은 ρ 가 아니라 K^* 이며, 원본 보존 replay 가 자기 출력 replay 보다 모든 혼합비에서 오래 버틴다	실험 X4 [X4 F5, F6]	최소모형 (LLM 실험 아님)	부호와 조건 간 순서만 단정. K^* 의 라운드 수·유지율절대치 금지
D6	과제 지표는 저장 이 이미 깎이는 동안 움직이지 않다가 절벽으로떨어진다	실험 X4 [X4 F4]	최소모형	모양만 단정
E1	세 경로를 합성한 실험이 corpus 에 0 건이다	ch15 § 15.8, ch17 § 17.8, ch21 § 21.8	부재 확인	단정

세 항목의 등급을 더 적어야 한다.

A3 은 부재의 확인이지 크기의 판정이 아니다. [Mem0] 의 91%·92%와 [Zep] 의 약 90%는 전부 프롬프트 길이가 크게다른 두 팔 사이의 비교이고, 긴 쪽 팔에 prefix cache 가 걸려 있지 않다 — [Mem0] 는 전문에 'cache' 가 한 번도 나오지 않으며, [Zep] 의 두 팔은 115k 대 1.6k 토큰으로 71.9 배 차이다 [Zep Table 2] [본서 산술]. 그러므로 이 장이 단정하는 것은 “**이득의 방향은 구조적으로 참이고크기는 미확정**” 이다. 방향이 구조적인 이유는 명확하다 — 프롬프트가 짧아지면 prefill 이 준다. 크기가 미확정인 이유도 명확하다 — 상대방에게표준 서빙 최적화를 켜 주지 않았다.

D5·D6 은 LLM 실험이 아니다. X4 는 가우시안 추정문제와 선형 사상 위의 최소모형이며, 그 안의 절대치 (ρ 의 백분점, 유지율, K^* 의 라운드 수, 필요 혼합비) 는 본문에 숫자로 옮기지않는다 [X4 D1]. 이 장이 옮기는 것은부호·모양·조건 간순서·crossover 의 존재와 이동 방향뿐이다.

D1 의 비율을 쓰지 않는 이유는 표본이다. 경로별 보고율에서 W 는 1 편, bio 는 3 편이므로 어떤 비율도 신뢰구간이 넓다 [X3 D8]. 인용 가능한 것은 Θ -경로가 E -경로보다 B_s 를 자주보고한다는 순서뿐이고, 그 순서의 원인은 기제가 아니라 서식의상속이다 — Θ -경로는 학습 논문의 보고관습 (step·batch·GPU-hour) 을 물려받았고 E -경로는 시스템 논문의 관습 (지연·정확도) 을 물려받아 쓰기 비용을 적는 칸 자체가 없다 ($\rightarrow$ ch24 § 24.3.1).

27.2.1 E-경로 평가 결함의 유형별 묶음

ch15는 다섯 시스템의 기제와 비용을 다루면서 시스템별 평가 결함 약 스무항을 분량 안에 담지 못했다. 그 항목들은 개별 시스템의 서술이 아니라 E-경로 판정의 증거이므로 이 장이 받는다. 개별 나열이 아니라 **결함 유형별로 묶고 대표 사례와 원문 위치를 단다**. 유형화가요점이다 — 같은 결함이 서로 다른 저자·서로 다른 시스템에서 반복되면 그것은 개별 논문의 부주의가 아니라 이 경로의 평가 관행이다.

표 27-2 — E-경로 시스템 평가 결함의 유형별 묶음. 다섯 시스템은 Mem0(2504.19413)·Zep(2501.13956)·ReasoningBank(2509.25140)·SCM(2604.20943)·Memini(2605.05097)이다. “계”는 그 유형이 나타나는 시스템 수다.

유형	무엇이 무너지는가	대표 사례와 원문 위치	계
T1. 자기 표가 자기 주장을 반증한다	헤드라인의 근거가 논문 안에서 부정된다	[Mem0] 초록의 “consistently outperform all existing memory systems”를 open-domain 행이 반증한다 — Zep 49. 56 F1 / 76.60 J가 Mem0 ^g 49.27 / 75. 71 과 Mem0 47.65 / 72.93 을 이긴다 [Mem0 Table 1]. 동반: [ReasoningBank § 4.4]는 “scaling actually reduces performance” 라 쓰면서 근거로 든 두 수치가 둘 다 증가다 (40.6 → 41.2, 44.4 → 45.5); [Memini App. A.4]는 0.426 대 0. 260 을 “look similar” 라 쓰는데 그 비는 1.64 배다 [본서 산술]	4

유형	무엇이 무너지는가	대표 사례와 원문 위치	계
T2. 인쇄된 산술이 맞지 않는다	효율 헤드라인의 값 자체가 재현되지 않는다	[ReasoningBank § 5]의 “2.1 fewer steps (a 26.9% relative reduction)” — $2.1/6.8 = 30.9\%$ 이고 [ReasoningBank Table 4]의 어느 칸 쌍도 26.9%를 내지 않는다 [본서 산술]. 동반: [SCM]의 90.9%가 45/50 과 50/55 두 계수에 동시에 붙는데 앞의 것은 90.0%이고 잡음 개념은 50 개뿐이다 [본서 산술] [SCM Table 3] 대 [SCM § 4.3] 대 [SCM Fig. 5]; [Mem0]는 같은 자연 절감을 91%와 92%로 두 번 인쇄한다 ([Mem0 Abstract] 대 [Mem0 § 4.3])	3
T3. 만들어 놓고 쓰지 않는다	아키텍처 계층이 어느 보고 수치에도 기여하지 않는다	[Zep § 2.3]의 community 계층은 label propagation 과 map-reduce 요약 까지 기술되지만 두 실험의 검색은 edge 와 entity node 만둘려주고 프롬프트 템플릿에 도<FACTS>·<ENTITIES>만 있다 [Zep § 3, § 4, § 4.2]. episodic-provenance 간선도 “not directly examined” 다 [Zep § 2.1]. 동반: [SCM Table 4]의 REM dreaming 과 Self-Model 은 제거해도 수치가 그대로다 (저자 인정); [Memini]는 읽기 경로의 네 하이퍼파라미터값을 하나도 주지 않고 검색 지표를 보고하지 않는다 [Memini § 3.4, App. A.5]	3

유형	무엇이 무너지는가	대표 사례와 원문 위치	계
T4. 헤드라인 능력에 실험이 0 건이다	초록이 약속한 것이 측정되지 않는다	[Zep] 초록의 structured business data 합성은 실험이 0 건이고 저자가 벤치마크 부재를 인정한다 [Zep § 2.1, § 5]. 동반: [Mem0]의 “대화 길이와 무관하게 일정한 성능”은 대화 길이를 한 번도 바꾸지 않은 상태에서 나온 주장이다 (LOCOMO 평균 약 26k 토큰 고정 [Mem0 § 3.1]); [Memini] 초록은 “without experimental evaluation”이라 쓰는데 부록에 정량 표가 있다 [Memini Abstract 대 App. A]	3
T5. 결정적 파라미터 · 표본이 시연 가능성으로 정해진다	결과가 데이터가 아니라 설정에서 나온다	[Memini App. A.2] “The timescale ratio $\tau_{\text{slow}}/\tau_{\text{fast}} = 5$ is chosen so that the three expected regimes are clearly distinguishable” — consolidation 이 보이는지를 정하는 단 하나의 파라미터다. 동반: [Memini App. A.4]는 68 쌍 중 17 쌍 — 넷 중 하나다 [본서 산술] — 을 저자가 정한 경계 밴드로 배제하고, 그 밴드가 빠름 · 느림 구분이 가장 어려운 전이 구간이다; [SCM § 3.3, § 3.6]은 가중치를 결과를 보고하는 그 벤치마크 위에서 고른다	2

유형	무엇이 무너지는가	대표 사례와 원문 위치	계
T6. 일반화가 단일 조합·소표본에 얽혀 있다	범위 주장이 한칸에서 나온다	[ReasoningBank]의 스케일링 결과 전부가 WebArena-Shopping × Gemini-2.5-flash 하 나이고 (Fig. 4·5·7·8·13·14), 교차 도메인 일반화의 근거인 Multi subset 은 29 인스턴스여서 1 과제가 3.4 점이다 [ReasoningBank Table 1; App. B.1]. 동반: [Mem0] 는 LOCOMO 1 종 · GPT-4o-mini 1 종 · 대화 10 건이고 [Mem0 § 3.1], [SCM] 의 8 개 테스트 는 전부 1.00 으로 헤드 룸이 0 이다 [SCM Table 3]	3
T7. 쓰기 비용을 세지 않은 채 효율을 주장 한다	비용 주장이 반증불가 능해진다	[Zep] 은 ingestion 비용을 토큰 · FLOPs · 시간 · 호출 수 어느 것으로도 보고하지 않으면서 “significantly reduces latency and LLM inference costs” 류 진술을 세 곳에서 한다 [Zep § 2.2.2, § 2.3, § 5]. 동반: [Mem0] 는 B_s 가 전무하고, [ReasoningBank § C.2] 는 “실전 배포 를 위해” 라면서 절대 저장 · 지연 수치를 하 나도 내지 않으며, [Memini] 는 실제로 돌린 실험에서 LLM 추출을 문자열 매칭으 로 대체해 토큰 비용이 0 이었다 [Memini App. A.2]	4

유형	무엇이 무너지는가	대표 사례와 원문 위치	계
T8. 파괴적 쓰기가 미 검증 판정기와 결합 한다	저장소가 조용히 손상 될 수있다	[ReasoningBank § 5]의 판정기는 정답 대비 72.7% 정확한데, 견고성 실험은 실제 판 정기 편향 대신 정답 라벨을 대칭 확률로 뒤 집은잡음으로 대체하 고 100% 설정은 정답 라벨을 직접 쓰므로 달 성 가능한구성이 아니 다. 동반: [Mem0 Alg. 1]의 DELETE 는 동결 LLM의 감사 되지않은 판단으로 항 목을 하드 삭제하는데 삭제정확도·오삭제 율·DELETE 비활성 ablation이 전부없 고, 같은 논문이 무응 답 (adversarial) 범주 를 벤치마크에서 제거 했다 [Mem0 § 3.1]	2
T9. 평가의 독립성과 지표의 비교 가능성이 함께 무너진다	비교 자체가성립하지 않는다	[Mem0 § 2.1]은 비 동기 처리를 자기 시스 템의 장점으로제시하 고 [Mem0 § 4.5]는 같은 성질을 Zep의 결 함으로 드는데, 그 비 판은시행 횟수·실패 율·버전 없이 일화로 제시되며논문은 승자 시스템의 벤더가 쓰고 경쟁 시스템도 그 벤더 가 실행했다. 동반: [Memini Table 3]의 Uniform 열은 감쇠 없는 원시 누적 이벤트 수이고나머지 두 열은 감쇠 ODE 상태여서, Uniform이 모든 행 에서 가장 큰 것은정의 의 귀결이다; [SCM § 4.2]의 세 baseline 은 저자의 자체 단순화 재구현이다	3

아홉 유형 중 여섯이 다섯 시스템 중 셋 이상에 나타난다. 그리고 **결함의 방향이 한쪽이다**. T1·T2는 헤드라인을 실제보다 크게 만들고, T3·T4는 기여를 실제보다 넓게 만들며, T5·T6은 범위를 실제보다 멀리 밀고, T7은 비용 쪽을 세지 않아이득을 실제보다 싸게 만든다. 반대 방향으로 기운 결함 — 방법을 실제보다 나쁘게 보이게 하는 오류 — 은 이 표에 한 행도 없다.

[평가] 이 표가 허용하는 결론과 허용하지 않는 결론을 갈라둔다. **허용하지 않는 것**: “이 시스템들은 작동하지 않는다.” 표 27-2의 어느 항목도 그것을 보이지 않는다. 다섯 시스템은 실제로 돌아가는 소프트웨어이고 프로덕션에 배포되어 있으며, 짧아진 프롬프트가 지연을 줄인다는 것은 구조적으로 참이다. **허용하는 것**: 보고된 **이득의 크기**를 그대로 받을 수 없고, 보고된 **비교**를 통제된 비교로 읽을 수 없다는 것이다. 그리고 결함이 한 방향으로만 기울어있으므로 크기의 오차 역시 한 방향이다 — 실제 이득은 보고치보다 작을 가능성이 크고 클 가능성은 이 표에 근거가 없다. [본서 판단] 이것이 § 27.3에서 *E*-경로 판정의 형태를 정한다.

27.3 분석

27.3.1 Q1 — 세 경로는 같은 문제를 풀지 않는다

“유망한가”의 답이 하나일 수 없는 첫 번째 이유는 셋이 서로 다른 문제를 풀기 때문이다. 이 사실은 판정이 아니라 각 경로의 갱신식에서 곧바로 읽힌다.

표 27-3 — 세 경로가 실제로 푸는 문제와 지불하는 것. “풀지 않는 문제” 열이 이 장의 나머지를 규정한다 — 어느 경로도 그 열의 항목에 대해서는 대안이 아니다.

경로	실제로 푸는 문제	지불하는 것	풀지 않는 문제
<i>E</i>	긴 문맥을 매 질의 프롬프트에 다시 싣는 비용	매 질의의 검색과 $\hat{c}$ 프롬프트 항, 파괴적 쓰기의 손상 위험, 그리고 세지 않은 B_s	모델이 모르는 것을 알게 만드는 문제. Θ 와 W 가 불변이므로 저장은 모델 밖에 남는다
W	문맥 길이에 선형인 KV cache를 고정 크기 상태로 접는 문제	경계마다의 prefill 버스트, 손실 압축, 되돌릴 수 없는 요약	요청을 넘는 지속성. 상태가 시퀀스마다 0으로 초기화되어 세션과 함께 죽는다 (→ ch17 § 17.5)
Θ	사실과 능력을 배포 아티팩트 자체에 넣어 질의당 한계 비용을 0으로 만드는 문제	shared-weight batching, 사용자당 웹 스토리지, 반복라운드의 열화, 롤백 단위의 소실	즉시성. 갱신이 라운드 단위이므로 방금 도착한 사실을 지금 쓸 수 없다

세 행의 “풀지 않는 문제”가 서로의 “푸는 문제”와 겹치지 않는다는 점이 중요하다. *E*는 지속성을 주지만 모델을 바꾸지 않고, Θ 는 모델을 바꾸지만 즉시성을 주지 못하며, W 는 문맥 비용을 접지만 요청을 넘지 못한다. 그러므로 “*E* 대신 Θ 를 쓰면 되는가”는 성립하는 질문이 아니다 — 두 경로는 같은 자리를 놓고 경쟁하지 않는다.

[해설] 이것이 Rosetta 사전의 “모델 재배포 $\leftrightarrow$ (U- Θ) 1회” 항목이 실제로 뜻하는 바다. Θ -경로를 쓴다는 것은 배포 아티팩트를 바꾼다는 것이고, 배포 아티팩트를 바꾸는 절차는 아무리 빨라도 요청 하나의 지연 안에 들어가지 않는다. 같은 이유로 *E*-경로를 쓴다는 것은 배포 아티팩트를 건드리지 않겠다는 선택이고, 그 선택의 대가가 “모델은 여전히 모른다”이다.

27.3.2 *E*-경로 — 주장의 종류가 바뀌어 있다

E-경로에는 판별식 통과 논문 9편 중 6편이 있다 (A1). 문헌의 무게 중심이고, 이 라인의 이름도 여기서 나왔다. 그런데 그 6편의 자기 표를 읽으면 **품질 1위가 반복적으로 기억 시스템이 없는 구성**이다 (A2).

- [Mem0] 에서 full-context 가 $J = 72.90 \pm 0.19\%$ 로 Mem0⁹ $68.44 \pm 0.17\%$ 와 Mem0 $66.88 \pm 0.15\%$ 를 이긴다. 격차는 각각 4.46 점과 6.02 점으로, 논문이 인쇄한표준편차 (0.15 – 0.19 점) 의 20 배를 넘는다 [Mem0 Table 2] [본서 산술].
- [Zep] 의 DMR 에서 자명한 full-conversation baseline 이 이미 비교대상인 MemGPT 를 이긴다 (94.4% 대 93.4%). Zep 이 남기는 마진은 0.4 점이고, gpt-4o-mini 에서는 0.2 점이다 – 500 개 대화에서 0.2 점은 대화한 건이며 [본서 산술], 신뢰구간도 시드도 유의검정도 논문에 없다 [Zep Table 1, § 4.2].
- [SCM] 의 벤치마크에서 forgetting 없는 그래프 baseline 이 22/22 로 제안 시스템과 동률이고 [SCM Table 2], 여덟테스트 전부가 1.00 이다 [SCM Table 3].
- [ReasoningBank] 에서는 검색 항목이 하나를 넘어가면 성공률이 단조로나빠지고 (0 개 39.0 에서 하나까지는 오르지만 그 뒤로 $49.7 \rightarrow 46.0 \rightarrow 45.5 \rightarrow 44.4$), 출하 기본값이 최소 비영값이다 [ReasoningBank § C.1 Fig. 13]. 기억 baseline 이 No Memory 보다 나쁜 칸도 여럿이다 (WebArena-Multi 에서 AWM 3.4 대 No Memory 10.3) [ReasoningBank Table 1].

이 다섯 사실은 이 책의 재구성이 아니라 **논문들 자신의 수치**다. 그리고 표 27-2 의 아홉 유형이 그 위에 얹힌다 – 이득의 크기를 키우는방향으로만 기운 결함들이.

그럼에도 **비용과 자연의 이득은 실재한다**. 그 이득의 방향은 측정이아니라 구조에서 나온다. 26k 토큰의 대화 이력을 매 질의에 다시 싣는 대신수백 토큰의 $\hat{c}$ 를 싣는다면 prefill 이 준다 [본서 추론]. 이것은반박 대상이 아니고, ch25 의 회계도 이것을 반박하지 않는다 – [Letta STC] 의 $5\times$ 는 짧은 문맥 · 큰 절감률 레짐에서 prefill 항을되살려도 뒤집히지 않는다 (A6, $\rightarrow$ ch25 판정 3). 반박되는 것은**이득의 크기**뿐이다. [Mem0] 의 91%와 [Zep] 의 약 90%는상대편에게 prefix cache 를 켜 주지 않은 비교이고 (A3), 그 상태에서 얻은배수를 서빙 스택의 배수로 옮길 근거가 없다.

[평가] 그러므로 E -경로에서 일어난 일은 **주장의종류가 바뀐 것이다**. 논문들의 초록은 품질 주장의 문법으로 쓰여 있다 – “outperform”, “state-of-the-art”, “consistently”. 그러나 자기 표가지지하는 것은 비용 주장이다. 이 진단은 다섯 시스템 중 한 곳에서 이미문장으로 인정되어 있다 – “a full-context method ... still achieves the highest J score (approximately 73%)” [Mem0 § 4.3]. 이 책이 더하는것은 그 인정을 경로 전체의 판정으로 올리는 것이다.

그리고 종류가 바뀌면 방어 부담도 바뀐다. 품질 주장은 정확도 표로방어되지만 **비용 주장은 총비용으로 방어된다**. 총비용에는 B_s 가 들어간다 – 추출 · 갱신 · 요약 갱신에 쓴 LLM 호출이다. 그런데 B_s 가 이 경로에서 가장 비어 있는 칸이다 (D1, T7). E -경로 논문 아홉 편 중 B_s 에 대해 무언가라도 보고한 것이둘이고, 유일한 절대 수치는 [ReasoningBank] 의 과제당 3748.4 토큰이며그것조차 스케일링 구성은 덮지 않는다 ($\rightarrow$ ch15 § 15.5). 즉 이 경로는자기 주장을 성능 표로 방어하려다 지고, 비용 표로 방어하려면 세지 않은칸을 세어야 한다.

이 판정과 함께 반드시 실어야 할 반대편 사실이 하나 있다. E -경로는 **shared-weight batching 을 그대로 보존한다** (A5). 상태가 가중치에내려앉지 않으므로 서로 다른 사용자의 요청이 같은 가중치 위에서 한 배치로묶인다. 이것은 Θ -경로 대비 결정적인 서빙 이점이고, 사용자당 웹스토리지도 delta 스테이징 지연도 생기지 않는다 ($\rightarrow$ ch26 판정 2). **다섯 논문 중 아무도 이 이점을 주장하지 않는다**. 자기가 실제로가진 가장 강한 논거를 쓰지 않고, 가지지 않은 품질 논거를 쓰고 있다.

27.3.3 W -경로 – 표본 1 의판정

W -경로는 판별식을 정면으로 통과하는 논문이 **한 편**이다 (B1). Memory Caching 은 $B_s = 0$ 이라 조건 (2) 에서, Nested Learning 은 모든갱신이 wake 시계라 조건 (1) 에서 탈락하며, Generative Adapter 는 산출물이 Θ 모양이지만 시계가 wake 라 이 범주가 아니다 ($\rightarrow$ ch11 § 11.4, ch24 판정 1). 남는 것은 Do Language Models Need Sleep? 하나다.

그 한 편 의 증거 상태를 그대로 옮긴다.

- 비용 4 종이 전부 논문에 없다 (B2). B_s 는 구조로만 있고, C_{cap} 은 상태 차원 · head 수 · dtype 이 없어 유도조차 되지 않으며, ρ 는 관측될레짐에 논문이 들어가지 않는다 (→ ch17 § 17.5).
- 헤드라인 지연 주장은 측정이 아니다. 고정 크기 상태이므로 decode 가 문맥 길이에 무관하다는 것은 아키텍처의 보장이지 쟁 값이 아니다. L_w 칸에 시간 단위 수치가 하나도 없다 (B2).
- 계산량을 맞춘 baseline 이 없다 (B3). 캐시를 유지한 full attention 은 어느 과제 · 어느 스케일에서도 실행되지 않았다. N 번 재귀하는 실행이 이미 쓴 계산을 상대방에게 주지 않은 비교다.
- 판별식 통과가 설정 의존이다 (B4). 질의보다 consolidation 이 앞서는 것은 Depo 설정뿐이고, 수치가 나오는 헤드라인설정 (GSM-Infinite) 은 질문을 문맥 앞에 놓아 consolidation 이 질의조건부가 된다. 그 설정에서 $N_q = 1$ 이므로 식 (A) 가 $C_{avg} = C_{wake} + C_{sleep}$ 으로 붕괴한다 — 상각이 없다.
- 이득은 6·8 연산 꼬리에만 있고, 쉬운 문제에서는 폭이 0.3–2.9 백분점이며, 한 모델의 8 연산에서는 $N = 2$ 가 $N = 1$ 보다 낮은반전이 있다. 시드는 하나이고 오차막대가 없다 (→ ch17 § 17.6).

[평가] 이 상태에서 “ W -경로는 유망한가” 예예/아니오로 답하는 것은 방법론적으로 불가능하다. 표본이 1 이고, 그 1 편의 비용 칸이 모두 비어 있으며, 대조군이 실행되지 않았다. 그러나 “판정 불가” 는 이 경로에 대해 아무 말도 하지 않는 것이 아니다. 정확히 두 가지를 말한다.

첫째, 회계상 가장 유망한 형태를 가진 경로가 증거가 가장 적은경로다. ch25 판정 2 는 E -경로 상각 논증의 구조적 결함을 확정했다 — 매 질의 내는 프롬프트 항이 N_q 로 나뉘지 않기 때문에 이길 수 없는영역이 원리적으로 존재한다. 그 결함에서 벗어나는 가장 깨끗한 설계는 원 문맥을 실제로 버리는 것이고 (→ ch25 § 25.5 항목 7), 그것이정확히 W -경로가 구조적으로 하는 일이다. 문맥을 fast weight 로 접고원본을 버리면 매 질의 프롬프트가 줄어들어 나뉘지 않는 항이 음의 기여를한다. 이 경로에는 다른 두 경로가 갖지 못한 회계상 이점이 있고, 그 이점이 한 번도 측정되지 않았다.

둘째, 판별식 통과가 설정에 달려 있다는 것 자체가 이 경로의미성숙을 보여준다. 같은 논문의 두 설정 중 하나에서는 sleep-time compute 이고 다른 하나에서는 아니다. 경로가 성숙하면 이런 일이 일어나지않는다.

[해설] Rosetta 사전에서 이 경로는 “warm-up / 사전 컴파일 ↔ sleep 라운드” 행에 놓인다. 두 항목 모두 유틸 시간을 쓰고산출물을 캐시에 남긴다. 다른 점은 하나다 — 사전 컴파일의 산출물은프로세스를 넘어 남지만, 이 경로의 W 는 시퀀스마다 0 으로 초기화되어요청과 함께 죽는다. 그래서 사용자당 지속 저장소가 생기지 않고, 요청 간상각도 생기지 않는다 (→ ch17 § 17.7).

27.3.4 Θ -경로 — 용량은 있고 반복이없다

Θ -경로의 상태는 다른 둘과 다르다. 용량 축에서는 증거가있다. 가중치는 사실을 실제로 담고, 담긴 것은 쉽게 지워지지 않는다 — $k = 20$ 에서 strict 문항을 전부 틀린 사실도 log-probability 상승분의중양값 69%(bare)·79%(study) 를 유지한다 [Continual Facts § 5.1]. 그리고 담긴 사실의 질의당 한계 decode 비용은 0 이다 [본서추론] — 검색도 프롬프트 증가도 없기 때문이다. 이것은 E -경로가원리적으로 줄 수 없는 성질이다 (표 27-3).

무너지는 곳은 용량이 아니라 반복이다.

첫째, A 군에서 sleep 라운드 수를 독립변수로 놓은 실험이 0 편이다 (C2). [Letta STC] 의 모든 실험은 sleep 라운드가 정확히 1 회이고, Do LMs Need Sleep?의 W 는 시퀀스마다 초기화되며, LM Need Sleep 은 sleep 주기 수를 축으로 놓은 실험을 하나도 갖고있지 않다 (→ ch25 § 25.2.1). 측정되지 않은 것이 아니라 측정될실험이 설계되지 않았다.

둘째, 반복을 실제로 돌려 본 유일한 논문이 이 경로를 지지하려는논문이 아니라 반증하려는 논문이다. [Continual Facts] 는 $k = 100$ 까지 루프를 돌렸고, 유지율이 0 으로 가지 않는 대신 25–28%에서고원을 이룬다는 것과 (C1), 20 라운드마다 Θ_0 를 anchor 로 consolidation 을 넣으면 역

량은 회복되고 유지율은 회복되지 않는다는 해리를 보고한다 [Continual Facts § 4.3]. 이 경로의 지속가능성에 대해 우리가 가진 유일한 다라운드 증거가 반증 측에서 나온다는 사실은 그 자체로 문헌 상태에 대한 진술이다.

셋째, 이 책의 최소모형이 그 방향을 독립적으로 재현한다. 실험 X4는 언어도 의미도 없는 가우시안 위치 모형과 선형 연상기억 위에서 ($U-\Theta$)를 K 라운드 돌린다. 세 결과가 경로 판정에 걸린다.

- **이송해야 할 양은 ρ 가 아니라 K^* 다 (D5)** — 성능이기준선을 유지하는 마지막 라운드다. “라운드당 몇 %”라는 형태의 보고는 운영 지표가 될 수 없다.
- **replay는 보존된 원본에 앵커되어야 한다 (D5)**. 같은 예산·같은 혼합비에서 replay 타깃을 모델 자기 출력으로만 만들면 보존된 원본으로 만들 때보다 모든 혼합비에서 먼저 무너진다. 이 부호는 [Continual Facts § 6.2]의 frozen-teacher 대 own-merges 대비와 같고, ch06의 accumulate 대 replace 대비와도 같다. 세곳에서 같은 부호가 나온다.
- **과제 지표는 저장이 이미 깎이는 동안 움직이지 않다가 절벽으로 떨어진다 (D6)**. 연속 신호는 첫 라운드부터 꾸준히 감소하는데 과제정확도는 한동안 전혀 반응하지 않다가 좁은 구간에서 무너진다. 운영함의가 직접적이다 — held-out 정확도만 보는 모니터는 저장이 이미 오래깎이는 동안 침묵하다가 절벽에 이르러서야 발화한다. 얼마나 깎인되었는지는 이 최소모형으로 정할 수 없다 (모양만 이송한다).

X4의 등급을 다시 밝힌다. 이것은 LLM 실험이 아니며, K^* 의 라운드수도 유지율도 필요 혼합비도 본문에 숫자로 옮기지 않는다 [X4 D1]. 옮기는 것은 부호와 조건 간 순서뿐이다. 그리고 그 정도만으로도 판정에는 충분하다 — 이 절이 필요로 하는 것은 “얼마나 버티는가”가 아니라 “버티는지 아무도 재지 않았다”이기 때문이다.

넷째, 서버에 도달하기 위한 두 전제가 corpus에서 미논의다.

- **delta 양자화**. 저장 바이트당 정보 밀도를 정하는 것은 표현의한계가 아니라 저장 dtype이며 ($\rightarrow$ ch26 판정 3), 그 변수는 rank·적용 행렬 집합과 독립으로 조절된다 — 즉 알고리즘을 바꾸지 않고 사용자당 바이트를 내리는 유일한 손잡이다. 그런데 **delta를 양자화한 상태에서 지식 회수율을 측정**한 논문이 0 편이다 (C3). Θ -경로를 서버에 올리는 데 가장 먼저 필요한 실험이 corpus에 없다.
- **쓰기 검증**. 무엇을 $\mathcal{R}_k$ 에 넣을지 판정하는 절차가 두 논문 모두 외부 신호에 의존한다 (C4). [SEAL]의 보상은 문맥마다 라벨을 요구하고 — 실제 걸림돌은 계산이 아니라 그 oracle이다 ($\rightarrow$ ch20 § 20.7) — [LM Need Sleep]의 Semantic Reward는 이름도 크기도 밝혀지지 않은 외부 frozen reward model에 의존하며, 그것을 제거하면 AIME-25가 69.0에서 69.2로 **오른다** [LM Need Sleep Table 1, p.11]. 그럼에도 논문은 “모든 구성요소가 긍정적으로 기여한다”고 쓴다 [LM Need Sleep § 4.2, p.13].

[평가] Θ -경로의 판정은 세 측으로 갈린다. **용량 측은 통과한다** — 가중치는 실제로 담고, 담긴 것의 질의당 한계 비용은 0이며, 이것은 다른 두 경로가 줄 수 없는 성질이다. **반복 측은 미검증이다** — 지지 문헌에 다라운드 실험이 0 편이고, 존재하는 유일한 다라운드 증거는 반증 측에서 나오며, 최소모형은 그 방향을 재현한다. **서빙 측은 두 개의 미논의 전제 위에서 있다** — delta 양자화와 쓰기 검증이다. 따라서 이 경로에 대한 판정은 조건부일 수밖에 없고, 조건은 회피가 아니라 실행 가능한 실험 목록이다 (§ 27.5).

Rosetta 사전의 “batch로 weight 공유 $\leftrightarrow$ Θ -경로에서 깨짐” 행이여기서 값을 치른다. 사용자별 delta가 생기는 순간 shared-weight batching이 성립하지 않고, 그것이 E -경로가 공짜로 갖는 성질이다 (A5). Θ -경로는 **질의당 한계 비용 0을 사기 위해 배치 효율과 웹스토리를 지불한다**. 그 거래가 남는 장사인지는 반복 측이 검증된 뒤에만 계산할 수 있다.

27.3.5 Q3·Q4 — 최선인가, 대안대비우월한가

“최선인가”와 “대안 대비 우월한가”는 대조군이 있어야만 답이 있는 질문이다. 그러므로 이 절이 하는 일은 우열을 선언하는 것이 아니라 어느 대조가 실행되었고 어느 대조가 실행되지 않았는지를 세는 것이다.

표 27-4 — 각 경로가 푸는 문제에 대한 가장 강한 대안과 corpus 의 대조상태. “대조 상태” 열이 이 절의 결과다.

경로	같은 문제에 대한 가장 강한 대안	corpus 의 대조 상태	이 책의 판정
<i>E</i>	문맥 전부를 넣기 (full-context) — 품질 축	실행되었고 방법이 졌다. [Mem0] 72.90 대 66.88, [Zep] 의 full-conversation 이 비교 대상을 이미 이 김, [SCM] 의 무망각 baseline 이 동률	품질 축에서는 대안이 우월하다. 단정
<i>E</i>	full-context + prefix cache — 비용 축	실행되지 않았다. [Mem0] 전문에 ‘cache’ 0 회	비용 축의 우열은 판정 불가
<i>E</i>	질의 전 추론 없는 순수 검색 (RAG)	부분 실행이 되 비대칭이다. 스왑은 제안 시스템이 아니라 RAG baseline 에만 걸려 있고 제안 시스템의 ablation 은 0 건이다 (→ ch15 § 15.6)	방향만. 순수 검색보다 낫다는 진술은 한 설정의 관측
<i>W</i>	캐시를 유지한 full attention — 계산량 정합	실행되지 않았다. 어느 과제 · 어느 스케일에 서도 없다 (B3)	판정 불가
Θ	같은 사실을 문맥에 넣기 (in-context)	실행되었으나 레짐이 어긋난다. 합성 과제에서 가중치 32% 대 프 롬프트 91%이지만, 문 맥 채널은 $ E \leq 2$ 에 서만 돌았고 가중치 채널은 100 개까지 밀렸다 (→ ch22 § 22.6)	방향은 대안 우세. 그러나 동일 레짐 대조가 아니므로 크기는 판정 불가
Θ	검색 (<i>E</i> -경로)	실행되지 않았다. [LM Need Sleep] 은 <i>E</i> -경로를 이름으로 부르고 두 문장으로 기각하되 baseline 으로 한 번도 돌리지 않는다 (→ ch21 § 21.8)	판정 불가
Θ	같은 경로 안: 직접 편집 대 delta	대조 없음. 그러나 구조로 갈린다	용도가 다르다. 개인화면 delta, 전역이면 편집 (→ ch26 판정 5)

[평가] 표 27-4 의 일곱 행이 하나의 패턴을 이룬다. 실행된 대조에서는 방법이 지거나 겨우 비기고, 방법이 이기는 축에서는 대조가 실행되지 않았다. *E*-경로는 품질 대조를 돌렸고 졌다. 비용 대조 — 상대방에 prefix cache 를 켜 비교 — 는 돌리지 않았다. *W*-경로는 계산량을 맞춘 대조를 아예 갖고 있지 않다. Θ -경로는 문맥 채널과의 대조를 어긋난 레짐에서 돌렸고, 검색과의 대조는 두 문장의 기각으로 대신했다.

그리고 빠진 대조군이 무작위가 아니다. 빠진 것은 언제나 **가장 싸고 가장 자명한 baseline** 이다 — 표준 서빙 최적화를 켜 full-context, 캐시를 유지한 full attention, 같은 예산

의 in-context. 구현 난이도가 아니라 **이기기 어려운 상대**라는 성질이 공통이다. [본서 판단]

그러므로 Q3·Q4 에 대한 이 책의 답은 이것이다. “**최선인가**” 와 “**대안 대비 우월한가**” 는 세 경로 어디에서도 **현재문헌으로 판정되지 않는다**. 이 답은 세 경로에 대한 비난이 아니라 문헌상태의 기술이며, 그 상태를 바꾸는 데 필요한 실험은 전부 값이싸다 (§ 27.5).

27.3.6 Q5 — 주류가 될까: 조건으로만 답한다

“주류가 될까” 는 미래에 대한 진술이므로 이 책은 그것을 단정하지 않는다. 대신 **무엇이 참이면 어느 경로가 이기는지**를 조건으로 적는다. 조건마다 현재 확인 상태와 그 조건이 판정되는 자리를 함께 적는다 — 조건이 어디서 판정되는지 적지 않으면 조건부 판정도 반증 불가능해진다.

표 27-5 — 조건부 예측. “판정되는 자리” 가 문헌 밖이면 그 조건은 이 책의 증거로 닫히지 않는다.

조건	참이면 이기는 경로	현재 확인 상태	판정되는 자리
C1. 서비스가 요구하는 것이 지연과 비용이 높고 품질은 full-context 근방이면 족하다	<i>E</i>	이 조건은 A2 와 양립한다 — 품질에서 지는 것이 문제가 되지 않는 요구이기 때문이다	제품 요구. 문헌이 아님
C2. 워크로드의 N_q 중 양값이 손익분기 위쪽에 안정적으로 있고 문맥이 짧다	<i>E</i>	N_q 분포를 보고한 논문이 corpus 에 0 편이고, 상각여유는 실무 워크로드 쪽으로 갈수록 나빠진다 (D3)	서빙 로그. 새 실험불필요
C3. 개인화가 요구되고 사용자당 delta 를 요청 지연 안에 웹 계층에서 끌어올수 있다	Θ	웹 계층의 규모는 회계로 있고 (D4) 스테이징 지연은 미측정	ch26·ch29, 그리고 실제 서빙 스택의 측정
C4. 보존된 원본에 앵커된 replay 하에서 반복 sleep 이 서비스가 요구하는 K 를 버틴다	Θ	A 군에 다라운드 실험 0 편 (C2). 유일한 다라운드 증거가 반증 축에서 나온다 (C1)	X4 가 지정한 실험. ch30
C5. 문맥 처리 비용이 워크로드를 지배하고, 고정 크기 상태가 recall 을 지킨다	<i>W</i>	ch18 이 recall 을 얻으려면 상태가 문맥에 선형으로 커진다는 것을 확인했다 (→ ch18 § 18.7). 유일한 A 군 논문은 대조군 없이 돌아왔다 (B3)	ch18·ch29
C6. 세 경로가 합성 가능하고 합이 부분보다 낮다	셋 다 — 그리고 “어느 경로가 이기는가” 라는 질문이 폐기된다	합성 실험 0 건 (E1)	없음

[평가] 표 27-5 에서 두 가지가 읽힌다.

첫째, *E*-경로의 주류화 여부는 이 문헌의 품질과 거의 무관하게 결정된다. C1 과 C2 는 제품 요구와 서빙 로그가 정하는 조건이고 논문이 정하는 조건이 아니다. 그리고 *E*-경로 시스템들은 이미 프로덕션에 배포되어 있다. 배포를 정당화한 것은 논문의 성능 표가 아니라 프롬프트 길이의 산술이다 — 26k 토큰을 수백 토큰으로 줄이면 싸진다는, 표 27-2

의아함 유형 어느 것으로도 흔들리지 않는 사실이다. 이 경로는 문헌이약한 채로 주류가 될 수 있고, 이미 그렇게 되고 있다. [본서 판단]

둘째, 구조가 가장 강하게 가리키는 미래가 증거가 가장 없는미래다. C6 — 세 경로를 합성한 시스템 — 은 세 경로가 서로 다른 문제를 푼다는 표 27-3의 구조로부터 곧바로 따라 나오는 귀결이고, 세 경로 중 둘을 이미 배포한 조직이 다음으로 붙일 구성이다 [본서 판단]. 그런데 그조합을 돌린 실험이 corpus에 0건이다. [Mem0]는 자기 $\hat{e}$ 를 $\mathcal{R}$ 로 삼는 가능성을 언급조차 하지 않고, [Memini]는 W -경로 계보를 외부 방증으로 한 문장 인용하되 결합을 제안하지 않으며, [LM Need Sleep]은 E -경로를 기각한다. **경로 간 침묵의대가가 여기서 청구된다** — 침묵은 인용 예절의 문제가 아니라, 가장 유망한설계공간이 비어 있게 만든 원인이다 (→ ch24 판정 2).

27.4 판정

[평가] 판정 1 — 다섯 질문에는 하나의 답이 없다. 경로마다답이 다르고, 경로마다 확신도가 다르다. 조건 없이 단정한다.

근거는 표 27-3이다. 세 경로가 푸는 문제의 집합이 서로 겹치지 않으므로하나의 순위표가 성립하지 않는다. E 는 지속성을 주되 모델을 바꾸지않고, Θ 는 모델을 바꾸되 즉시성을 주지 못하며, W 는 문맥비용을 접되 요청을 넘지 못한다. “sleep-time compute 은 유망한가” 예예/아니오로 답하는 문장은 세 명제를 뭉겐 문장이다.

[평가] 판정 2 — E -경로는 유망하다. 단 비용방법으로서다. 품질 방법로서는 현재 문헌이 지지하지 않으며, 그반증은 이 책이 아니라 논문들 자신의 표에서 나온다.

품질 측: full-context 72.90% 대 66.88%, full-conversation 이 이미 비교 대상을 이김, 무망각 baseline 이 동물, 검색항목을 늘릴수록 단조 악화. 네 시스템에서 같은 방향이다 (A2). 비용 측: 프롬프트를 26k에서 수백 토큰으로 줄이면 prefill 이 준다 — **이득의방향은 구조적으로 참이다**. 그러나 크기는 미확정이다. 보고된배수는 상대편에게 prefix cache를 켜 주지 않은 비교이고 (A3), 총비용의분자인 B_s 는 이 경로에서 가장 비어 있는 칸이다 (D1, T7).

조건: 이 판정이 “유망하다”로 남으려면 B_s 를 포함한 총비용이같은 서빙 스택에서 full-context 보다 낮아야 한다. 그 계산은 아직 어느논문에도 없다. 그리고 이 경로가 실제로 가진 가장 강한 논거 — shared-weight batching 보존 (A5) — 를 주장하는 논문이 다섯 중 0편이다.

[평가] 판정 3 — W -경로는 현재 문헌으로 판정 불가이며, 그판정 불가 자체가 결과다. 이것은 보류가 아니다.

A 군 논문이 1편이고 (B1), 그 1편의 비용 4종이 전부 비어 있으며 (B2), 계산량을 맞춘 대조군이 없고 (B3), 판별식 통과조차 두 설정 중 하나에서만성립한다 (B4). 표본이 1인 경로에 대해서는 어떤 절차로도 강한 판정을 만들수 없다 (→ ch24 § 24.3.3).

그럼에도 이 경로에 대해 단정할 것이 하나 있다. ch25 판정 2가확정한 상각 논증의 구조적 결합 — 매 질의 프롬프트 향이 N_q 로 나뉘지않는다 — 에서 벗어나는 가장 깨끗한 설계가 원 문맥을 실제로 버리는것이고, 그것이 이 경로가 구조적으로 하는 일이다. **회계상 가장유리한 형태를 가진 경로가 증거가 가장 적은 경로다**. 이 불일치는 기제의성질이 아니라 문헌의 성질이다.

[평가] 판정 4 — Θ -경로는 조건부로 유망하다. 용량측은 통과하고, 반복 측은 미검증이며, 서빙 측은 두 개의 미논의 전제 위에서 있다.

통과: 가중치는 사실을 실제로 담고, 담긴 것의 질의당 한계 decode 비용이 0이며, 이것은 다른 두 경로가 원리적으로 줄 수 없는 성질이다. 미검증: A 군에서 sleep 라운드

수를 독립변수로 놓은 실험이 0 편이고 (C2), 존재하는 유일한 다라운드 증거는 반증 축에서 나오며 $k = 100$ 에서 유지율이 25–28%로 고원을 이룬다 (C1). 이 책의 최소모형은 그 방향을 독립적으로 재현하고, 이송 가능한 양이 ρ 가 아니라 K^* 임을 보인다 (D5). **전제 두 개:** delta 양자화 상태의 회수율이 측정된 논문 0 편 (C3), 그리고 쓰기 검증이 문맥마다의 라벨 또는 이름 없는 외부 reward model 에 의존한다 (C4).

조건을 명시한다. 이 경로가 유망해지는 조건은 셋이다 — (i) 보존된 원본에 앵커된 replay 하에서 K^* 가 서비스 요구 K 를 넘는 것이 측정되고, (ii) delta 를 int8 이하로 양자화해도 회수율이 유지되는 것이 측정되며, (iii) 라벨 없이 쓰기를 판정하는 절차가 나오고 그 정확도가 측정되는 것이다. 셋 중 어느 것도 현재 corpus 에 없다. **세 조건이 충족되지 않은 상태에서 “ Θ -경로가 유망하다” 고 쓰는 것은 이 책이 논문들에게 금지한 서술 방식이다.**

[평가] 판정 5 — “최선인가” 와 “대안 대비 우월한가” 는 세 경로 어디에서도 판정되지 않는다. 판정 불가로 남긴다.

이유는 대조군의 부재이지 성능의 부족이 아니다. 실행된 대조에서는 방법이 지거나 겨우 비겼고, 방법이 이기는 축에서는 대조가 실행되지 않았다 (표 27-4). 그리고 빠진 대조군은 매번 가장 싸고 가장 자명한 baseline 이다. **어정쩡하게 남기지 않는다** — 이 책은 “ E -경로가 full-context 보다 총비용에서 우월하다” 도 “우월하지 않다” 도 주장하지 않으며, 그 비교를 성립시키는 표가 corpus 에 없다고 주장한다.

[평가] 판정 6 — “주류가 될까” 는 조건부로만 답한다. 그리고 조건의 성격이 경로마다 다르다.

E -경로의 주류화 조건 (C1·C2) 은 **제품 요구와 서빙 로그가 정한다**. 문헌이 정하지 않는다. 그러므로 이 경로는 문헌이 약한채로 주류가 될 수 있고, 프롬프트 길이의 산술이 그것을 지탱한다. Θ -경로의 조건 (C3·C4) 은 **실험이 정한다**. 그 실험들은 아직 실행되지 않았지만 실행 가능하고 값이 싸다. W -경로의 조건 (C5) 은 **아키텍처가 정한다**. 고정 크기 상태가 recall 을 지키는지가 관건이고, ch18 은 반대 방향의 증거를 확인했다.

그리고 셋 중 어느 것도 아닌 결말이 표 27-3 의 분해에서 직접 따라 나온다 — 세 경로가 서로 다른 문제를 풀므로 합성이 가능하고, 합성되면 “어느 경로가 이기는가” 라는 질문 자체가 폐기된다 (C6). **그 결말을 지지하는 실험이 corpus 에 0 건이다**. 이 책이 예측하지 않고 조건만 적는 이유가 이것이다: 구조가 가장 강하게 가리키는 미래에 증거가 가장 없다.

27.5 반증 조건

여섯 판정은 각각 다른 관측으로 무너진다. 경로별로 적고, 각 항목이 어느 판정을 무너뜨리는지와 누가 어떤 비용으로 관측할 수 있는지를 함께 적는다. 반증 비용을 적지 않으면 반증 목록은 장식이 된다.

E -경로 판정 (판정 2) 의 반증

FE1 — 총비용 대조. B_s 를 포함한 총비용을, 상대방에 prefix cache 를 켜 full-context 와 같은 서빙 스택에서 비교한 표가 나오는 경우. E -경로가 이기면 판정 2 의 “크기 미확정” 이 해소되고 판정이 강화된다. 지면 판정 2 의 “유망하다” 가 무너지고, 이 경로는 품질에서도 비용에서도 방어되지 않는 상태가 된다. **이 항목은 양방향으로 열려 있고, 새 실험을 요구하지 않는다** — 추출·갱신 호출 수는 실행 로그에 이미 있고 prefix cache 는 스위치다.

FE2 — 통제된 품질 대조. 같은 예산·같은 서빙 최적화에서 E -경로가 full-context 를 품질에서 이기는 표가 나오는 경우. 판정 2 의 “품질 방법이 아니다” 가 무너진다. 관측 형태를 명시한다 — full-context 팔이 있어야 하고, 신뢰구간과 시드가 인쇄되어야 하며, 대화 길이가 최소 세 수준으로 스윙되어야 한다. 현재 corpus 에는 세 요건을 함께 만족한 표가 0 개다.

FE3 — 파괴적 쓰기의 안전성. 삭제·덮어쓰기의 오삭제율이 측정되고 무시 가능성이 보이는 경우, 표 27-3의 E 행에서 “손상 위험”이 지워진다. 관측: DELETE를 비활성화한 ablation, 같은 대화를 두 번 이상 ingest 한 실험, 그리고 제거된 무응답 (adversarial) 범주의 복원.

FE4 — 표 27-2 자체의 반증. 아홉 유형 중 어느 항목이 원문 대조로 틀렸음이 확인되면 그 행을 고친다. 이 반증은 책상에서 판정된다 — 인용된 결과 표를 열어 보면 끝난다. 다만 판정 2가 기대는 것은 아홉 유형의 총합이 아니라 A2와 A3 둘이므로, 개별 행이 수정되어도 판정 2는 유지된다. 무너지는 것은 결함의 방향 일방성 진술이며, 방법을 실제보다 나쁘게 보이게 하는 결함이 비슷한 수로 발견되면 표 27-2 뒤의 [평가]를 철회해야 한다.

W-경로 판정 (판정 3)의 반증

FW1 — 표본이 1을 벗어나는 경우. 판별식 네 조건을 정면으로 통과하는 W-경로 논문이 두 편 이상 추가되면 “표본 1” 판정이 사라진다. 가장 가까운 후보는 이미 지목되어 있다 — Memory Caching이 논하는 사후학습 변형에 예산과 수치가 붙고 그 절차가 질의 전에 (U-W)를 도는 것으로 확인되는 경우다 (→ ch24 § 24.5 F1, ch18 § 18.5).

FW2 — 계산량 정합 대조와 L_w 의 실측. 캐시를 유지한 full attention과 같은 계산 예산에서 비교한 표가 나오고 wake 지연이 시간단위로 측정되는 경우, 판정 3의 “판정 불가”가 해소된다. 그 표에서 W-경로가 지면 판정 3의 둘째 문단 — 회계상 가장 유리한 형태 — 이실무적 무게를 잃는다. 이 관측은 새 실험을 요구하지만 스케일이 작다.

FW3 — 상태가 요청을 넘는 경우. W를 세션 사이에 저장·복원하는 구성이 나오고 그 상태로 이후 질의가 개선되면, 표 27-3의 W 행에서 “요청을 넘는 지속성”이 “풀지 않는 문제”에서 빠진다. 그러면 이 경로가 E·Θ와 같은 자리에서 경쟁하게 되고, 사용자당 C_{cap} 이 GB 자릿수인 핫 계층 상태가 지속 저장 대상이 되므로 (→ ch26 판정 2) 서빙 함의가 전면 재계산되어야 한다.

Θ-경로 판정 (판정 4)의 반증

FT1 — K^* 의 측정. 보존된 원본에 앵커된 replay 하에서 sleep 라운드 수를 독립변수로 놓고 K^* 를 측정한 실험. 형태를 명시한다 — x 축은 라운드 k , y 축은 세 지표 (도달성·쓰기가 만든 연속 신호의 잔존률·일반 역량), 팔은 replay 타깃 두 종 (보존된 원본 대 모델 자기 출력)이다. K^* 가 서비스 요구 K 를 넘으면 판정 4의 조건 (i)이 충족되고 이 경로의 판정이 조건부에서 긍정으로 바뀐다. 넘지 못하면 그 값 자체가 이 경로의 상한이 된다. corpus 에이 실험이 0 건이라는 사실이 판정 4의 조건절 전체를 지탱한다.

FT2 — delta 양자화 하의 회수율. rank·학습집합·학습 절차를 고정하고 저장 dtype만 bf16 → int8 → int4로 내렸을 때의 회수율 곡선. int8 이전에 무너지면 ch26 판정 3의 “전제”가 “품질을 지불하고 바이트를 사는 거래”로 바뀌고 워م 계층 규모 추정의 int4 열을 지워야 한다. 유지되면 판정 4의 조건 (ii)가 충족된다 (→ ch26 § 26.5 R-2).

FT3 — 라벨 없는 쓰기 검증. 문맥마다의 정답 라벨도 외부 reward model도 없이 self-edit의 품질을 판정하는 절차가 나오고 그 정확도가 측정되는 경우, 판정 4의 조건 (iii)이 충족되고, [SEAL]의 실질 병목 — 계산이 아니라 보상 oracle — 이 사라진다 (→ ch20 § 20.7).

FT4 — X4의 부호가 틀렸음을 보이는 관측. LLM 실험에서 모델 자기 출력에 앵커한 replay가 보존된 원본에 앵커한 replay보다 오래버티는 결과가 나오면 D5의 부호가 틀렸고, ch06·[Continual Facts § 6.2]와 함께 세 곳에서 일치하던 방향이 깨진다. 그러면 판정 4의 조건 (i)에 붙은 “보존된 원본에 앵커된”이라는 수식을 떼야 한다.

판정 5(최선·우월 판정 불가)의 반증

FQ1 — 빈칸이 채워지는 경우. 표 27-4에서 “실행되지 않았다” 고적인 세 칸 중 하나라도 채워지면 그 행의 판정 불가가 해소된다. 이 표는 corpus가 커질 때마다 같은 절차로 다시 채점된다.

FQ2 — 이 책이 부재를 잘못 판정한 경우. “실행되지 않았다” 고 적은 대조가 실제로는 원문 어딘가에 실행되어 있는 경우, 확인 방법이 기계적이다 — 해당 논문의 표를 열어 baseline 열을 세면 된다. 이 반증이 성립하면 표 27-4의 해당 행과 판정 5의 범위를 함께 고친다.

판정 1-6(프레임 자체)의 반증

FZ1 — 경로 분해의 반증. 표 27-3의 “풀지 않는 문제” 열이 어느 경로에서 채워지면 그 행의 분해가 무너진다. 두 후보가 구체적이다 — Θ-경로의 갱신이 요청 지연 안에 완료되면 “즉시성 결여”가 사라지고, E-경로의 산출물이 모델의 파라미터를 실제로 바꾸는 구성이나 오면 그것은 E-경로가 아니라 경로 합성이다.

FZ2 — 합성이 부분을 이기는 경우. 세 경로를 합성한 시스템이 각 경로 단독보다 같은 표에서 낮

다는 것이 보이면, **경로별로 판정한다는 이 장의 분해 자체가 잘못된 분해다**. 그러면 이 장은 역사적 진술로 물러나고 판정은 합성 설계의 축으로 다시 쓰여야 한다. **이 책은 그 결과가 나오기를 요구하는 쪽에 서 있다** — C6 이 현재 증거 0 건이라는 사실은 그 방향의 연구를 막는 근거가 아니라 촉구하는 근거다.

반증되지 않는 것. 개별 시스템의 성능 수치가 후속 판본에서 오르거나 재현 실패로 내려가도 이 장의 판정 대부분은 바뀌지 않는다. 판정 2·5 가 기대는 것은 수치의 크기가 아니라 **대조의 구성**이기 때문이다 — 새 수치가 아니라 새 표만이 이 장을 무너뜨린다. 마찬가지로 어느 경로의 방법이 다른 벤치마크에서 더 좋은 점수를 내도 판정 5 는 유지된다. 벤치마크가 바뀌어도 빠진 대조군은 여전히 빠져 있다.

요약

- 다섯 질문에 하나의 답은 없다. 세 경로가 푸는 문제의 집합이 겹치지 않으므로 순위표가 성립하지 않는다 — E 는 지속성을 주되 모델을 바꾸지 않고, Θ 는 모델을 바꾸되 즉시성을 주지 못하며, W 는 문맥 비용을 접되 요청을 넘지 못한다.
- E -경로에서 품질 1 위가 반복적으로 기억 시스템 없는 구성이다 (72.90% 대 66.88%, full-conversation 의 선행 우위, 무망각 baseline 동률, 검색 항목 증가에 따른 단조 악화). 이것은 이 책의 재구성이 아니라 논문들 자신의 표다.
- 그럼에도 비용·지연 이득은 실재한다. **방향은 구조적으로 참이고 크기는 미확정이다** — 보고된 배수는 상대방에 prefix cache 를 켜 주지 않은 비교이며, 총비용의 분자인 B_s 가 이 경로에서 가장 비어 있는 칸이다.
- 그러므로 E -경로에서 일어난 일은 **주장의 종류가 바뀐 것**이다. 초록은 품질 주장의 문법으로 쓰였고 표가 지지하는 것은 비용 주장이며, 비용 주장은 성능 표가 아니라 총비용으로 방어된다. 그리고 이 경로가 실제로 가진 가장 강한 논거 (shared-weight batching 보존) 를 주장하는 논문이 0 편이다.
- E -경로 평가 결함 아홉 유형 중 여섯이 다섯 시스템 중 셋 이상에 나타나며, 결함의 방향이 전부 한쪽 — 이득을 크게 만드는 쪽 — 으로서 기울어 있다. 반대 방향으로 기운 결함은 한 행도 없다.
- W -경로는 A 군 논문 1 편, 비용 4 종 전부 부재, 계산량 정합 대조 부재, 설정 의존적 판별식 통과로 **판정 불가**다. 그러나 상각의 구조적 결함에서 벗어나는 가장 깨끗한 설계가 이 경로의 기제이므로, **회계상 가장 유리한 형태를 가진 경로가 증거가 가장 적은 경로다**.
- Θ -경로는 조건부다. 용량 축은 통과하고 질의당 한계 비용이 0 이지만, 반복 축은 A 군 실험 0 편으로 미검증이고 존재하는 유일한 다라운드 증거가 반증 축에서 나오며, 서빙에 도달하려면 delta 양자화와 라벨 없는 쓰기 검증이라는 두 미논의 전제가 먼저 채워져야 한다.
- “최선인가” 와 “대안 대비 우월한가” 는 세 경로 어디에서도 판정되지 않는다. 실행된 대조에서는 방법이 지거나 겨우 비겼고 방법이 이기는 축에서는 대조가 실행되지 않았으며, 빠진 대조군은 매번 가장 싸고 가장 자명한 baseline 이다.
- “주류가 될까” 는 조건부로만 답한다. E -경로의 조건은 제품 요구와 서빙 로그가 정하므로 문헌이 약한 채로 주류가 될 수 있고, Θ -경로의 조건은 실행 가능한 세 실험이 정하며, 경로 분해 자체에서 곧바로 따라 나오는 결말인 경로 합성에는 실험이 0 건이다.

다음 장으로

이 장은 경로별로 답했고, 답의 형태가 세 번 다르게 나왔다 — E 는 종류를 바꾼 유망, W 는 표본 1 의 판정 불가, Θ 는 세 조건이 붙은 조건부. 세 답에 공통으로 걸린 것이 하나 있다. **어느 경로에서도 “예산을 얼마 썼는가” 가 세어지지 않았다는 것이다**.

E -경로에서는 그것이 총비용 방어를 막았고 (판정 2), W -경로에서는 계산량 정합 대조를 불가능하게 했으며 (판정 3), Θ -경로에서는 라운드를 늘렸을 때 무엇을 지불하는지 알 수 없게 만들었다 (판정 4). 판정 5 의 “판정 불가” 도 절반은 이 결측에서 나온다 — 비용을 세지 않으면 같은 예산에서의 비교가 성립하지 않기 때문이다.

그래서 ch28은 B_s 를 축으로 세운다. 이 책의 예산 축 감사 (실험 X3) 가이 장에서 두 항목만 인용한 감사이고, 다음 장이 그 전체를 받는다. 다만결과를 미리 밝혀 두는 편이 정직하다 — **현재 문헌으로 B_s -품질곡선은 그릴 수 없다.** 축 위에 두 점 이상을 놓은 논문이 소수이고, 그소수가 공유하는 벤치마크의 교집합이 공집합이어서 x 축을 복원해도 y 축이불지 않는다 (D2). 그러므로 ch28의 기여는 곡선이 아니라 **곡선을그리기 위한 최소 보고 요구**이며, 그 요구가 왜 필요한지의 계수적 근거다. 이 장이 판정 불가로 남긴 칸들이 그 요구가 채워질 때 함께 채워진다.

ch28. scaling law — B_s 를축으로

28.1 판정할 질문

B_s 를 축으로 하는 scaling law를 지금 그릴 수 있는가. 없다면무엇이 필요한가.

ch01의 질문 지도가 이 장에 Q6을 배정했다 — “scaling law가 있는가 (B_s 축)”. 그 배정에는 전제가 하나 깔려 있었다. B_s 를 독립변수로 놓은점들이 문헌에 흩어져 있고, 동일 회계를 세우면 그 점들을 하나의 축 위에재배치할 수 있으리라는 전제다. 이 장은 그 전제를 먼저 검사한다. 점을 옮기려면 점이 있어야 하고, 점이 있으려면 B_s 가 눈금이 붙은 양으로인쇄되어 있어야 하기 때문이다.

이 질문에 답이 될 수 있는 것. 첫째, 현재 문헌으로 B_s -품질곡선이 복원되는가 — 이것은 논증이 아니라 세기로 결정된다. 둘째, 복원되지 않는다면 무엇이 인쇄되어야 복원되는가 — 최소 보고요구의목록이며, 이 장의 실질 기여가 여기 있다. 셋째, B_s 가 고전 scaling law의 (N, D) 와 같은 종류의 축인가 — 이것은 회계의 구조에서연역되는 판정이다. 넷째, 곡선을 그릴 수 없는 상태에서 말할 수 있는 것 — 곡선의 부호와 경계 조건이며, ch25의 비용 회계와 ch26의 용량 회계가 그 재료를 이미 만들어 두었다.

이 질문에 답이 될 수 없는 것. 첫째, 지수와 포화점. 어느경로에서도 이 책은 $A \propto B_s^\alpha$ 꼴의 값을 주장하지 않는다. 둘째, “자연에 그런 곡선이 없다”는 주장. 이 장의 판정은문헌의 보고 상태에 대한 것이지 현상에 대한 것이 아니며, 둘을섞으면 이 책이 논문들에게 요구한 기준을 스스로 어긴다. 셋째, 절대 FLOPs. 넷째, 경로 간 기울기 비교 — 뒤에서 보듯 세 경로의 B_s 는 wake 비용에서로 다른 부호로 결합하므로 같은 평면에 접칠 수 없다.

한 가지를 미리 못 박는다. **없는 곡선을 그리지 않는다.** 이 장이 할수 있는 가장 나쁜 일은 눈금 없는 서수 손잡이 위의 점 몇 개를 이어 붙여기울기를 읽는 것이다. 그렇게 얻은 지수는 어느 논문의 값도 아니고 이 책의회계 값도 아니며, ch25가 [Letta STC]의 비용모델에 대해 지적인 것과정확히 같은 종류의 결합 — 축이 아닌 것을 축으로 쓰는 것 — 을 이 책이되풀이하는 일이 된다.

장-국소 기호 셋을 선언한다. A 는 과제 품질지표 (정확도 · F1 등) 다. N 은 고전 scaling law의 파라미터수, D 는 학습 토큰 수다. 셋 다 이 장에서만 쓰며 이 책의 예약 기호가아니다 ($\rightarrow$ ch01). 특히 N 은 한 문맥을 공유하는 질의 수 N_q 와 다른양이고, 두 기호가 같은 문단에 나오면 그때마다 구분해 적는다.

28.2 증거

증거는 셋에서 온다. Part II 열세 장이 § N.5에서 채운 비용 4종 표, ch25·ch26이 그 표를 겹쳐낸 판정, 그리고 이 장의 warrant 인이 책의 예산 축 감사 (실험 X3) 다.

X3의 등급을 먼저 밝힌다. 이 실험은 AUDIT-GRADE / CORPUS-CENSUS 다 — **측정이 아니고 시뮬레이션도 아니다.** 성능도비용도 재지 않았다. 세는 것은 문헌의 보고 상태이며, 대상은 이 책이 deep-read한 29편의 노트다. 결정론적이고 난수를 쓰지 않으며, 편집자 override는 0건이라 집계는 전부 코드의 산물이다. 유보가 셋 붙는다. (i) **1차 감사 대상이 원문이 아니라 노트다** — 노트가 놓친 값은 이감사도 놓친다. 완화책으로 원문 텍스트 probe를 병기했으나 probe는 “어딘가에 그 문자열이 있다”만 말한다. (ii) **분류는 산문에 대한키워드+부정 규칙이다** — 범주별 편수와 순서는 인용해도 되지만 개별 논문의라벨은 증거 스니펫을 대조한 뒤에만 인용한다. (iii) **이 corpus**

는 문헌 전체의 표본이 아니다 — “문헌의 몇 %” 로 일반화하지 않고 “이 corpus 의 29 편 중” 으로 쓴다.

표 28-1 — 이 장이 쓰는 증거와 그 등급. “본문 지위” 열이 그 항목으로 무엇을 단정할 수 있는지를 정한다.

#	증거	출처	등급	본문 지위
E1	29 편의 B_s 보고 형태 6 분류 전수 집계	[X3 Q1]	감사 (census). 키워드+부정 규칙	범주별 편수와 순서 단정. 개별 라벨은 증거 대조 후
E2	(f) 19 편 중 9 편은 오프라인 패스 자체가 없어 B_s 가정되지 않는다	[X3 Q1 structural]	감사. ch11 판별식과 독립 산출	단정. “보고 안함” 과 반드시 구분
E3	[Letta STC] 전문 probe — “숫자 + tokens” 꼴 2 회, ‘FLOP’ 0 회	[X3 founding_paper_direct_probe]	원문 직접 probe. 결정론적	단정
E4	(a)~(c) 후보 8 편 중 노트만으로 FLOPs 환산이 달하는 것 1 편	[X3 Q2]	감사. 문자열 존재 판정 → 상한	단정하되 상한임을 밝힌다
E5	MFU 또는 달성 throughput 을 보고한 노트가 29 편 중 1 편	[X3 Q1 structural]	감사	단정
E6	B_s 축에 3 점 이상을 놓은 4 편, 그 중 눈금이 절대 예산단위인 것 2 편	[X3 Q1 structural, Q3]	감사	단정
E7	그 4 편의 벤치마크 교집합이 공집합, 두 편 이상이 공유하는 벤치 0 건	[X3 Q3]	이름 매칭 → 상한	단정하되 상한임을 밝힌다
E8	경로별 보고율의 순서 $\Theta > E$	[X3 Q1]	감사. 표본 작음	순서만 단정. 비율 인용 금지
E9	비용 4 종에서 요구 단위대로 인쇄된 칸은 B_s 1 편, $L_w \cdot C_{cap} \cdot \rho$ 각각 0 편	ch25 § 25.2.1	Part II 원문 대조	단정
E10	상각식 (A) 의 분모 N_q 의 실서빙 분포를 보고한 논문 0 편	ch25 § 25.3.3	Part II 확정	단정

#	증거	출처	등급	본문 지위
E11	판별식 통과 논문 중 C_{cap} 을 사용 자당 바이트로 보고한 논문 0 편. 용량 천장	ch26 § 26.2, ch09 § 09.3	확정 + 논문의 정리	단정
E12	$\text{mem}_U \leq H(\Theta)$ X1 이 가정한 $ c \cdot \hat{c} / c $ 범위가 원문 근거와 어긋난다	[X3 Q5]	문자/4 근사, 인쇄된 예시 1-2 건	자릿수·비율·방향만. 근사로 얻은 토큰 수는 유효 숫자를 인용하지 않는다 (논문이 토큰으로 직접인쇄한 값과 비는 예외)

28.2.1 B_s 는 이 corpus 에서 축이 아니라 각주다

표 28-2 — 29 편의 B_s 보고 형태 전수 분류 [X3 Q1]. 우선순위 $a > b > c > d > e > f$ 로 단일 범주에 배정했고, 다중 라벨은 감사 산출물이 따로 병기한다.

범주	보고 형태	편수	해당 논문
(a)	절대 FLOPs 또는 토큰 수	1	ReasoningBank
(b)	optimizer step 수	5	Continual Facts, EWC, LM Need Sleep, LoRA-as-Knowledge-Memory, ROME
(c)	벽시계	2	MEMIT, SEAL
(d)	눈금 없는 서수 손잡이만	2	Letta STC, Do LMs Need Sleep?
(e)	상대 비율만	0	—
(f)	없음	19	그 밖의 전부 (→ 아래)

이 표에서 세 가지를 읽는다.

첫째, (a) 가 한 편이다. 그리고 그 한 편은 상각을 주장하지 않는 논문이다 — ReasoningBank 의 과제당 3748.4 토큰은 과제 하나의 쓰기비용이고 그 논문에서 각 과제는 자기 자신의 문맥이므로 $N_q = 1$ 이다 (→ ch15 § 15.5, ch25 § 25.2.1). ch25의 집계는 모수가 다르다 — Part II가 § N.5에서 원문 대조로 채운 21 편을 요구 단위 기준으로 썼다. **모수도분류 규칙도 다른 두 집계는 같은 한 편을 지목한다.** [본서 관찰]

둘째, (f) 19 편은 한 덩어리가 아니다. 그중 9 편은 “보고하지않았다”가 아니라 **질의 전 오프라인 패스 자체가 없어 B_s 가정의되지 않는 경우**다 [X3 Q1 structural]. GEM·기억용량논문·LoRA·MemGPT·Memorizing Transformers·Memory Caching·Memory Layers·Nested Learning·RAG가 그 아홉이다. 아홉전부가 ch11이 통과군에서 제외된 논문이다 — 여섯은 B군, MemGPT는 조건 (1)만 탈락한 경계 사례, GEM과 기억용량 논문은 애초에 판정 대상이 아닌 C군이다 (→ ch11 § 11.3, § 11.5). 두 판정은 **서로 다른 필드에서독립으로 나왔다** — ch11은 기제 서술을 읽었고 이 감사는 비용 칸만 읽었다. 방향이 같다는 것은 판별식 쪽의 독립 확인이다 (→ ch24). [본서 관찰]

다만 두 목록은 동치가 아니고, 어긋나는 자리가 정보를 준다. ch11 이 불통과로 판정한 ROME·MEMIT·Generative Adapter 는 이 감사에서 B_s 가 정의되는 쪽에 남는다. 즉 판별식 불통과가 곧 B_s 미정의는 아니다. 두 자가 같은 것을 재지 않기 때문이다 — 판별식은 갱신을 도는 시계가 있는지를 묻고, 이 감사는 비용 칸에 오프라인 패스가 잡히는지를 묻는다. 포함방향으로 적으면 B_s 미정의 9 편은 판별식 불통과군의 진부분집합이며, 이 포함이 진짜라는 것이 두 자가 독립임을 보이는 증거다. [본서 추론]

셋째, 여섯 칸이 문헌의 보고 형태를 다 담지 못한다. 감사가 역지로 밀어 넣지 않고 남긴 형태가 넷이다 — PLOS Sleep 의 시물레이션 시간 (한 sleep 구간 = 100 movement cycle = 60,000 timestep = 시물레이션 시간 30 초. 원문 수치로부터의 산술이며 ch23 § 23.5 가 확정했다), EWC 의 표본 수 (Fisher 추정용 100 minibatch $\times$ 32, optimizer step 이 아니다), Generative Adapter 의 그림 축에만 있고 인쇄된 숫자가 없는 상태, 그리고 PAD 의 필드 부재다 [X3 Q1 taxonomy_gaps]. 여섯 칸조차 사후에 만든 분류인데 그 분류에 들어가지 않는 형태가 넷 나온다는 것은, 이 문헌에 B_s 를 세는 공통관행이 없다는 뜻이다. [본서 추론]

28.2.2 눈금을 공통 단위로 옮길 다리가 없다

(a)~(c) 의 8 편을 FLOPs 로 환산하는 레시피는 초등적이다. 토큰수면 파라미터 수와 forward/backward 구분을 곱하고, step 수면 batch·시퀀스 길이·파라미터수·학습 가능 비율을 곱하며, 벽시계면 장치종류·장치 수·달성 throughput 또는 MFU 를 곱한다 [X3 Q2 recipes]. 문제는 곱할 값이 없다는 것이다.

여덟 편 중 노트만으로 환산이 달히는 것은 1 편이다 [X3 Q2]. 가장 자주 막는 입력은 시퀀스 길이 (4 편) 이고 그다음이 파라미터수·batch·MFU (각 2 편) 다. 유일한 (a) 범주인 ReasoningBank 조차 backbone 모델 크기가 없어 토큰을 FLOPs 로 옮기지 못한다. 그리고 이 “1 편” 은 상한이다 — 판정 근거가 노트 안에 그 입력이 문자열로 존재하는지였으므로, 실제로 환산 가능한 점 수는 이 보다 많을 수 없다 [X3 D6]. 감사가 스스로 붙인 경고가 이 점을 예시한다: 달한다고 판정된 그 한 편 의 시퀀스 길이 히트는 “2,048-token chunk” 인데 이것은 RAG baseline 의 체크 크기이지 LoRA 학습 시퀀스 길이가 아니다 [X3 Q2 honest_limit].

벽시계 쪽 다리는 더 짧다. MFU 또는 달성 throughput 을 보고한 노트가 29 편 중 1 편이고, 그 1 편은 B_s 가 정의되지도 않는 LoRA 다 [X3 Q1 structural]. 그 값은 GPT-3 175B 학습의 V100 당 throughput 32.5 대 43.1 tokens/s 이고 [LoRA § 4.2], 이 책은 ch19 § 19.5 에서 이미 비용 칸에 적었다. 즉 (c) 범주 두 편의 시간을 계산량으로 옮길 계수가 corpus 안에 없다.

28.2.3 축 위에 점이 여럿인 논문과 그 y 축

표 28-3 — B_s 축에 3 점 이상을 놓은 4 편 [X3 Q3]. “눈금” 은 그 점들이 절대 예산 단위인지 눈금 없는 비례 손잡이인지를 가른다.

논문	눈금	축 위의 점	벤치마크	baseline
Continual Facts	절대 예산 단위	사실당 24 / 96 / 192 step	이름 붙은 공개 벤치 없음	SFT
LoRA-as-Knowledge-Memory	절대 예산 단위	250 / 500 / 1000 / 2000 step	CounterFact, PhoneBook, PaperQA, NarrativeQA, QuALITY	full-context, ICL, RAG/ BM25

논문	눈금	측 위의 점	벤치마크	baseline
Letta STC	비례 손잡이 (눈금 없음)	$\kappa_{ } \in \{1, 2, 5, 10\}$	Stateful GSM-Symbolic, Stateful AIME, Multi-Query GSM-Symbolic, SWE-Features	test-time only
Do LMs Need Sleep?	비례 손잡이 (눈금 없음)	재귀 횟수 1/2/3/4 · 1/2/4 · 1/2/4/6	GSM-Infinite, Depo, automaton	no-loop(재귀 1 회)

네 편의 벤치마크 교집합은 공집합이고, 두 편 이상이 공유하는 벤치마크가 하나도 없다 [X3 Q3]. 이 값도 상한이다 — 이름 매칭이므로 같은 이름이어도 split·프롬프트·채점기가 다르면 실제 비교 가능성은 더 낮다 [X3 D7]. 그리고 Letta STC 행의 벤치마크는 **넷이 전부 저자 제작 변형**이며, 논문 안에 기존의 stateful reasoning 벤치마크는 하나도 없다는 사실을 ch14 § 14.6 이 확정했다. 이 행에 관한 한 y 축은 깨져 있는 정도가 아니라 이 논문 밖으로 나가지 않는다.

여기에 ch17 이 확인한 사실 하나를 더한다. Do LMs Need Sleep?의 재귀 횟수 스위치는 **계산량을 맞춘 baseline 이 없다** — Fig. 2b·3·4·5 의 모든 비교가 학습 토큰 또는 step 을 맞춘 것이지 FLOPs 를 맞춘 것이 아니며, 같은 x 좌표에서 재귀 4 회 실행은 1 회 실행의 대략 네 배 계산을 이미 썼다 (→ ch17 § 17.6). 측 위에 점이 넷 있어도 그 측이 예산 측이 아니면 기울기는 예산에 대한 기울기가 아니다.

28.2.4 이송된 사실넷

이 장이 앞 장들에서 그대로 받아가는 것을 명시한다. 첫째, 요구 단위대로 인쇄된 비용 칸은 B_s 의 1 편뿐이고 $L_w \cdot C_{\text{cap}} \cdot \rho$ 는 각각 0 편이다 (→ ch25 § 25.2.1). 둘째, 상각식 (A) 의 분모 N_q 의 실서빙 분포를 보고한 논문이 corpus 에 없다 (→ ch25 § 25.3.3). 셋째, 판별식 통과 논문 중 사용자당 상태 바이트를 인쇄한 논문이 0 편이고, 어떤 (U- Θ) 절차도 $\text{mem}_U \leq H(\Theta)$ 를 넘겨 쓸 수 없다 (→ ch26 § 26.2, ch09 § 09.3). 넷째, LM Need Sleep 은 sleep 주기 수를 독립변수로 놓은 실험이 하나도 없어 ρ 가 계산될 수도 없다 (→ ch21 § 21.5).

넷을 한 문장으로 묶으면 이렇다. **scaling law 가 요구하는 네 재료 — x 축의 눈금, y 축의 공통 지표, 비용의 분모, 그리고 포화를 귀속시킬 용량 — 중 어느 것도 이 corpus 에 갖춰져 있지 않다.** [본서 관찰]

28.3 분석

28.3.1 (N, D) 와 B_s 는 같은 종류의 측이 아니다

먼저 측의 성격을 가른다. 이것이 갈리지 않으면 뒤의 계수 논의가 “보고가부실하다” 는 잔소리로 끝난다.

고전 scaling law 의 두 측을 독자가 이미 아는 것이다 — 파라미터 수 N 과 학습 토큰 수 D , 그리고 그 둘로 예측되는 손실. **이 책은 그 법칙의 지수를 인용하지 않는다.** 이 장에 필요한 것은 값이 아니라 측의 성질이고, 성질은 값 없이 비교된다.

표 28-4 — 고전 scaling law 의 측과 B_s 측의 구조 비교. 오른쪽 열은 이 책의 회계 (식 (A), (25-1)) 에서 따라 나온다. [본서 추론]

성질	(N, D)	B_s
누가 지불하는가	배포본 하나	문맥 하나 · 세션하나 · 사용자 하나
분모	그 배포본의 전 트래픽 — 사실 상 무한	N_q — 유한하고 워크로드가 정한다
상각의 지위	자명하므로 축에 나타나지 않는다	결론을 뒤집을 수 있으므로 축의 일부다
반복 여부	한 번	라운드 k 마다 재소비된다
wake 비용과의 결합	N 은 토큰마다 재발생, D 는 0	경로마다 부호가 다르다 (§ 28.3.5)
논금의 불변성	실험의 불변량	워크로드에 의존해 변한다

세 행이 결정적이다.

분모 행. (N, D) 도 상각된다 — 한 번 학습해 전 트래픽에 쓴다. 그러나 그 분모는 사실상 무한하므로 상각항이 0 으로 수렴하고, 그래서 고전법칙은 상각을 **회계에서 지워도 된다**. 축이 두 개인 이 유가 여기있다. B_s 는 다르다. 식 (A) 의 분모 N_q 는 유한하고, ch25 가 확정했듯 그 분포를 보고한 논문이 corpus 에 없다. **같은 FLOPs 가 N_q 에 따라 다른 비용이 된다**. 그러므로 B_s 를 x 축에 놓은 그림은 논금이 워크로드에 따라 늘었다 줄었다 하는 자 위에 점을 찍은 것이고, 두 논문의 점을 나란히 놓으려면 두 워크로드의 N_q 가 같아야 한다. 상각가능성은 축의 편의가 아니라 **축의 정의를 바꾼다**.

반복 행. D 는 한 번 쓰고 끝난다. B_s 는 라운드마다 다시낸다. 그러므로 “이 방법의 B_s ” 라는 말은 라운드 수를 지정하지 않으면 정의되지 않는다. ch22 의 Continual Facts 가 이 사정을 정면으로 보여 준다 — 사실당 192 step 이 단일 쓰기의 예산인데, 20 라운드마다 지금까지의 모든 사실을 다시 쓰는 둘째 시간척도가 붙고, 배치 조건에서는 사실 수에 24 를 곱한 step 이 들어가므로 $k = 100$ 의 통합 1 회가 2,400 step 으로 단일 쓰기의 열 배가 넘는다 (→ ch22 § 22.3, § 22.5). 라운드 수를 빼놓고 점 하나를 찍으면 그 점은 축 위의 어디에도 있지 않다.

결합 행. D 를 늘려도 추론 비용은 그대로다. N 을 늘리면 토큰마다 비용이 는다. 두 축의 wake 결합은 각각 0 과 양이고, **부호가 정해져 있다**. B_s 는 그렇지 않다 — § 28.3.5 에서 보듯 경로마다 부호가 다르다. 부호가 다른 세 양을 같은 x 축에 놓을 수는 없다.

이 셋을 합치면 그럴 수 있는 대상의 최소 형태가 정해진다.

$$\mathcal{A} = f(B_s, C_{\text{cap}}, k), \quad f \text{는 경로마다 다르다} \quad (28-1)$$

$$C_{\text{avg}} = C_{\text{wake}}(\hat{c}) + \frac{1}{N_q} \sum_k B_s^{(k)} \quad (28-2)$$

(28-1) 의 k 는 sleep 라운드 첨자이며 여기서는 몇 번째 라운드까지갔는가를 가리킨다. (28-2) 는 식 (A) 를 라운드에 대해 편 것이고, C_{wake} 가 $\hat{c}$ 의 함수라는 것은 ch25 가 되살린 항이다 (식 (25-1)). [본서 산술] 두 식을 나란히 두면 이 장의 형식적 결론이 나온다 — B_s **단독의 곡선은 미완성이다**. 품질 쪽에는 C_{cap} 과 라운드 수가 함께 들어가야 하고, 비용 쪽에는 N_q 와 wake 델타가 함께 들어가야 한다. 이것은 보고가 부실해서가 아니라 축이 그런 종류이기 때문이다.

[해설] 이 독자의 어휘로 옮기면 (N, D) 는 **빌드타임 파라미터**이고 B_s 는 **런타임 정책 파라미터**다. 빌드 타임값은 아티팩트에 박혀 모든 요청에 같게 적용되므로 하나의 수로 축에 놓인다. 런타임 정책 값은 요청 흐름의 성질에 따라 실효 비용이 달라지므로, 그 값을 축에 놓으려면 흐름의 통계를 함께 고정해야 한다. warm-up 이나 사전컴파일이 유효 시간을 쓴다는 점에서 sleep 라운드와 닮았지만 (→ ch01 의 Rosetta 표), 결정적 차이가 여기 있다 — 사전 컴파일의 산출물은 재사용횟수와 무관하게 같은 바이트이고 품질을 바꾸지 않는다.

28.3.2 x 축이 여섯 단위로 흩어져 있고 옮길 다리가없다

표 28-2가 세는 것은 단위의 분산이다. 단위가 여섯이면 축은 여섯 개이지하나가 아니다. 그러나 흩어진 것 자체는 치명적이지 않다 — 옮길 수 있으면된다. 옮길 수 없다는 것이 § 28.2.2의 계수다 — 여덟 편 중 노트만으로환산이 닫히는 것은 한 편이고, 그마저 근거가 된 시퀀스 길이가 다른 실험의 것이다. 그리고 **step**과 **벽시계**는 **그 자체로 축이 아니다**. step은 batch와 시퀀스 길이가 정해져야 계산량이 되고, 벽시계는 장치와 달성효율이 정해져야 계산량이 된다. 이 corpus에서 MFU 또는 달성 throughput을보고한 노트는 한 편이고 그 편은 B_s 가 정의되지도 않는다. **다리를 놓을 재료가 문헌 안에 없다**.

이 진단을 강화하는 사실이 하나 더 있다. 감사가 “ B_s 는 absent인데다른 계산 수치는 인쇄한” 논문을 따로 세었다 [X3 Q1 structural]. Generative Adapter는 outer-loop 학습의 벽시계와 step을인쇄하고 (SlimPajama 1B 토큰에 H100 8 대로 약 20 시간 [Generative Adapter App. B] → ch16 § 16.5), model collapse는서수·step·토큰을 인쇄한다. **계산량을세는 능력이 없어서가 아니다**. 세는 습관은 있는데 그 습관이 sleep 국면으로 확장되지 않았다.

[평가] 그러므로 B_s 의 침묵은 “**재기 어려워서**”가 아니다. ch26이 C_{cap} 에 대해 내린 판정과 같은 형태의 판정이 여기서도성립한다 — C_{cap} 은 구성과 dtype만 적으면 독자가 곱셈으로복원할 수 있는데도 0 편이었고 (→ ch26 § 26.4), B_s 는 대부분의 저자가자기 학습 로그에서 이미 알고 있는 값인데도 절대 단위로는 1 편이다. 두 칸모두 **측정의 난이도가 아니라 비교하려는 의도의 부재**가 원인이다. 차이는 하나다. C_{cap} 은 인쇄만 하면 복원되지만, B_s 는인쇄해도 § 28.3.1의 세 구조적 성질 때문에 그것만으로는 축이 되지 않는다.

28.3.3 y 축도 함께 깨져있다

x 축을 복원했다고 치자. 그래도 그림은 나오지 않는다.

B_s 축에 3 점 이상을 놓은 논문 네 편의 벤치마크 교집합이 공집합이고, 두 편 이상이 공유하는 벤치마크가 하나도 없다 (표 28-3). baseline도 SFT·full-context·test-time only·no-loop로 넷이 다 다르다. **두 축이 동시에 깨져있고, 한쪽만 고쳐도 그림은 나오지 않는다**.

여기서 이 장에서 가장 날카로운 관측 하나가 나온다. 축 위에 3 점 이상을놓은 네 편 중 그 점들이 **절대 예산 단위인 것은 둘인데**, 그 둘은 Continual Facts와 LoRA-as-Knowledge-Memory다. **둘 다 ch11의통과군에 없다** — 정확히는 판별식의 판정 대상조차 아닌 C 군이다 (→ ch11 § 11.3). 앞의 것은 Θ -경로의 반증 축이고 뒤의 것은 용량 이론이며, 어느 쪽도 질의 전 오프라인 갱신을 제안하는 논문이 아니다 (→ ch22, ch09). 뒤집어 적으면 이렇다.

판별식을 통과한 논문 중, B_s 축에 절대 예산 단위로 3 점 이상을 놓은 논문은 0 편이다.
[본서 관찰]

이 문장은 앞의 계수들을 하나로 묶는다. 예산을 눈금 있는 축으로 다룬 곳은이 라인 밖이고, 이 라인 안에서 점을 여러 개 찍은 두 편은 눈금없는 비례 손잡이 위에 찍었다. 그리고 그 손잡이 중 하나는 계산량이 맞춰진적이 없다 (→ ch17 § 17.6). scaling law를 그리려면 예산을 축으로 다루는 논문이 필요한데, 이 corpus에서 예산을 축으로 다룬 논문들은 sleep-time compute을 하지 않는다.

28.3.4 이름을 만든 논문에 분자가 없다 — 그리고 그것은 관행의문제다

[Letta STC]는 이 라인에 이름을 준 논문이고, 상각 논증의 원형을제시한 논문이며 (→ ch14 § 14.5), 그 논증의 분자가 B_s 다. 그 논문의 vendored 전문을 직접 훑은 결과는 이렇다 — 숫자 뒤에 tokens가 붙은 풀어나오는 자리가 두 군데이고 둘 다 AIME 문제 속의 게임말이며 (“removes either 1 token or 4 tokens”), ‘FLOP’은 0 회나온다 [X3 founding_paper_direct_probe]. 논문이 보고하는 것은서수 손잡이뿐이다 — 재작성 호출 $J \leq 10$, 병렬 재표현 $\kappa_{\parallel} \in \{1, 2, 5, 10\}$, sleep reasoning effort $\in \{\text{low, medium, high}\}$ (→ ch14 § 14.5.1). **상각 논증의 분자에 숫자가없다**.

이것이 개별 논문의 부주의인지 관행인지는 보고율의 분포가 답한다. 경로별로 B_s 에 대해 무언가라도 보고한 비율은 Θ -경로가 E -경로보다 뚜렷하게 높다 [X3 Q1]. 표본이 작은 경로 (W 1 편, 생물 3 편)는 순서 진술에서 빠고, 표본이 의미 있는 두 경로만 비교한다. **이 순서는 인용 가능하고 비율 자체는 아니다** [X3 D8].

[평가] 순서의 원인은 기제가 아니라 상속받은 보고 관습이다. Θ -경로 논문들은 ML 학습 논문의 관습을 물려받았으므로 step·batch·learning rate·GPU 시간을 적는 칸이 이미 있다 — 그 칸을 채우면 (b) 나 (c) 가 된다. E -경로 논문들은 시스템·제품 논문의 관습을 물려받았고, 그 관습에는 지연과 정확도를 적는 칸은 있어도 쓰기 **계산량을 적는 칸이 아예 없다**. Mem0 가 추출·갱신·비동기 요약 어느국면에 대해서도 토큰·FLOPs·LLM 호출수·비용을 한 줄도 인쇄하지 않는 것이 그 관습의 전형이다 ($\rightarrow$ ch15 § 15.5). 그러므로 이 공백은 새 측정을 요구하지 않는다. **표를한 칸 늘리면 메워진다**. 이 진단이 § 28.3.7 의 최소 요구 목록이 실행가능하다고 보는 근거다.

28.3.5 그래도 말할 수 있는 것 — 부호와 포화의귀속

곡선을 못 그린다는 것이 아무것도 모른다는 뜻은 아니다. ch25 와 ch26 이세운 회계 위에서 두 가지가 결정된다.

첫째, B_s 가 wake 비용에 결합하는 부호가 경로마다 다르다.

E -경로에서 B_s 의 산출물은 $\hat{c}$ 이고, $\hat{c}$ 는 매 질의의프롬프트에 실려 다시 처리된다. 즉 sleep 예산을 키워 $\hat{c}$ 를 길게만들수록 **wake 비용이 늘어난다**. ch25 가 되살린 항이 정확히 이것이고, 그래서 임계 길이 L^* 가 유한하며 $|\hat{c}| \rightarrow L^*$ 에서 손익분기가 발산한다 (식 (25-2), (25-3)). Θ -경로에서는 다르다. 산출물이 가중치 delta 이므로 merge 뒤 wake 의 토큰당 계산은 그대로이고 ($\rightarrow$ ch19 § 19.5), 남는 것은 요청당 수 MB 의 delta 스테이징 지연이다 ($\rightarrow$ ch26 § 26.3.5). W -경로에서는 또다르다. Do LMs Need Sleep?의 헤드라인 설정에서 $N_q = 1$ 이므로식 (A) 가 함으로 붕괴하고, wake 에서 뺀 계산이 같은 요청의 다른 국면으로 옮겨 갈 뿐 요청 전체의 계산량은 재귀 횟수에 대략 비례해 는다 ($\rightarrow$ ch17 § 17.5, ch25 표 25-2 유형 VI).

[평가] 그러므로 하나의 곡선이 아니라 세 곡선이다. y 축을품질로 놓으면 세 경로의 접을 같은 평면에 겹칠 수 있다 — 그때 x 축은예산이지 비용이 아니다. y 축을 비용으로 놓는 순간 겹칠 수 없다. 같은 B_s 증가가 E 에서는 wake 를 늘리고, Θ 에서는 wake 를 그대로두며, W 에서는 총계산을 늘리기 때문이다. 이 책이 “경로를 지정하지않은 일반화 주장을 금지” 한 규칙 ($\rightarrow$ ch01) 이 scaling law 에서는 형식적귀결이 된다. **경로를 지정하지 않은 B_s 곡선은 부호가 정의되지않는다.**

둘째, 포화점은 B_s 단독으로 귀속되지 않는다.

곡선에서 가장 값어치 있는 특징은 지수가 아니라 포화점이다 — 예산을 더넣어도 오르지 않는 지점이 어디인가가 서빙 예산을 정하기 때문이다. 그런데 sleep 라운드의 산출물은 어디가에 **저장** 되어야 하고, 그 그릇의크기가 C_{cap} 이다. ch09 가 확정한 두 제약이 여기서 동시에작동한다 — 위로는 $mem_U \leq H(\Theta)$ 의 천장이 있고, 아래로는 상태가 과제의 정보 요구량보다 작으면 어디에 두든 같은 형태의오류를 바닥을 맞는다 ($\rightarrow$ ch09 § 09.3, § 09.6). 즉 B_s 를 키워도 그릇이차면 오르지 않는다.

그래서 포화가 관측되어도 그것이 예산의 포화인지 그릇의 포화인지구별할 수 없다. 같은 B_s 를 작은 rank 의 delta 에 쓰는 것과 큰 rank 의 delta 에 쓰는 것은 천장이 다르고 (같은 앵커에서 rank 와 적용 행렬 집합만바뀌도 delta 의 저장 바이트가 32 배 범위로 벌어진다 $\rightarrow$ ch26 § 26.3.2), 같은 예산을 고정 크기 W 에 접는 것과 자라는 E 에 적는 것도 천장이다르다. 그런데 **판별식 통과 논문 중 C_{cap} 을 인쇄한논문이 0 편이다** ($\rightarrow$ ch26 § 26.4). 포화를 귀속시킬 두 번째 좌표가 없다.

ch22 가 이 문제의 구체적 사례를 제공한다. 사실당 24 step 에서 bare-statement 는 recall 97%·paraphrase 95%에 도달하고 96 step 에서 포화하는데, 같은 bare-statement 가 counterfactual 문항에서는 24 step 에서 192 step 까지 21 – 23%에 머문다. 그리고 데이터의 폭을 넓히면 같은예산에서 application·composition·counterfactual 이 각각+21·+18·+29pp 오르는 반면 “extra bare-statement optimisation improves none of them” 이다 Continual Facts

§ 3.1, p.5. 같은 x 좌표에서 결과를 정한 것은 예산이 아니라 무엇을 쓰는가였다. 그 곡선의 포화를 예산의 포화로 읽으면 오독이다. [본서 추론]

셋째, 라운드를 반복하면 축이 하나 더 붙는다. 식 (28-2)의 합이라운드에 대한 합인 이상 ρ 가 곡선에 들어온다 — 라운드마다 예산을 내는데 라운드마다 열화가 쌓이면, 누적 예산에 대한 품질은 단조가 아닐 수 있다. corpus에서 ρ 를 라운드당 비율로 인쇄한 논문은 0 편이고 (→ ch25 § 25.2.1), 가장 풍부한 열화 데이터를 가진 ch22 조차 곡선과 종점 수준으로만 보고한다 (→ ch22 § 22.5). 이 축은 ch29·ch30이 받는다.

28.3.6 이 책 자신의 가정에 대한 감사

같은 자를 이 책의 실험에도 낸다. X3은 ch25가 쓴 X1의 입력 가정을 원문근거와 맞춰 보았고, 세 군데가 어긋난다는 것을 확인했다 [X3 Q5].

첫째, 짧은 레짐의 $|c|$ 가정이 크다. X1은 Stateful GSM-Symbolic에 400 토큰, Stateful AIME에 300 토큰을 놓았다 (→ ch25 § 25.2.4). [Letta STC]가 직접 인쇄한 예시 문맥들을 문자 수로 재어 토큰으로 근사하면 그보다 약 3–12 배 작다 — [Letta STC Fig. 1]의 원 문맥, [Letta STC App. B]의 AIME 문맥 두 건, [Letta STC Fig. 20]의 GSM 문맥이 그 근거다 [X3 Q5]. 방향이 중요하다. L^* 는 $|c|$ 에 의존하지 않고 비 $L^*/|c|$ 만 $1/|c|$ 로 변하므로, $|c|$ 를 크게 잡은 X1은 그 두 레짐의 여유를 과소보고했다. ch25의 판정 3(“짧은 문맥·큰 절감률 레짐에서 $5\times$ 느뻐지지 않는다”)은 약해지지 않고 강해진다.

둘째, 긴 레짐의 sweep 상단이 잘려 있었다. X1이 가장 큰 레짐으로 잡은 SWE-Features의 8,000 토큰보다 이 corpus가 실제로 다루는 대화형 문맥이 훨씬 크다 — LOCOMO 대화 하나가 26,000 토큰 [Mem0 § 3.1], LongMemEval 대화 하나가 115,000 토큰이다 [Zep § 4.3]. X1의 상단보다 3–14 배 크다. corpus 전문에서 굵은 문맥 길이 표기의 분포도 같은 방향이다 (중앙값 8,192 토큰, p90 262,000 토큰, 93 건/15 편) [X3 Q5]. 그 영역에서 $L^*/|c|$ 는 X1이 보고한 SWE 값보다 더 작아지므로 ch25의 판정 4(SWE-Features의 여유가 최하위)도 방향이 유지되고 강해진다.

셋째, $|\hat{c}|/|c|$ 의 관측 범위가 X1의 그리드 양 끝 밖에 있다. 그리고 이 범위는 종류를 구분해 읽어야 한다 — 질의당 주입량 대 원 문맥은 Zep이 0.014, [Letta STC Fig. 1] 예시가 약 3인 반면, 저장소 크기 대 원 문맥은 Mem0가 0.27, 그래프 확장이 0.54이고, 경쟁사가 측정한 Zep 저장소는 원 문맥보다 한 자릿수 이상 크다 [X3 Q5]. X1이 스윙한 {0.25, 1, 4}는 이 범위의 가운데 한 자릿수만 덮는다.

[평가] 앞의 두 어긋남은 방향이 X1의 결론과 같은 쪽이므로 ch25의 순서 판정은 유지되고 강화된다. 셋째 어긋남은 그렇지 않다 — 관측범위의 양 끝이 모두 그리드 밖이므로 방향이 한쪽으로 정해지지 않으며, 이 감사는 셋째에 대해 방향을 주장하지 않는다. 그리고 L^* 의 절대치는 세 경우 모두에서 여전히 인용 불가이며, 이 감사는 그 금지를 완화하지 않는다. 오히려 근거를 하나 더 붙인다 — SWE-Features의 $|c|$ 는 논문이 “여러 관련 PR”이라고만 쓰고 그 수를 밝히지 않으며 예시의 changed_files는 “omitted here for brevity”로 잘려있으므로, X1의 8,000 토큰 가정은 하한으로만 지지된다 [X3 D5]. 그리고 셋째 항목은 보고 요구를 하나 넣는다. 저장소 크기와 질의당 주입량이 같은 단위로 인쇄되어 섞이기 쉽고, 섞으면 비가 한 자릿수 넘게 들어진다.

이 절의 수치에는 유보가 셋 붙는다 — 토큰 수는 전부 문자/4 근사이므로 자릿수와 비율만 쓰고, 문자 수는 인쇄된 예시 1–2 건에서 나오므로 “그 벤치마크 문맥의 평균 길이”로 인용하면 오용이며, 경쟁사 측정치는 방향만 쓴다 [X3 D2·D3·D4].

28.3.7 최소 보고 요구 R1–R9

이 장의 실질 기여는 곡선이 아니라 목록이다. 아래 아홉 항목이 인쇄되면 B_s 축이 복원된다. 각 항목은 이 corpus의 계수에서 직접 도출되었으며, 왜 필요한지가 계수로 대답된다.

표 28-5 — B_s -품질 곡선을 그리기 위한 최소 보고 요구 [X3 Q4]. “현재” 열은 이 corpus 29편에 대한 계수다.

#	요구	현재	이것이 여는 것
R1	B_s 를 절대 FLOPs 로. 못 하면 (토큰 수 + 파라미터 수 + prefill/decode 분해)를 함께 인쇄해 환산이 닫히게	(a) 범주 1 편, 그마저 backbone 크기 미 보고	x 축의 눈금
R2	step·벽시계로 보고할 거면 환산 입력을 함께 — step 이면 (batch, 시퀀스 길이, 파라미터 수, 학습 가능 비율), 벽시계면 (장치종류·수, 달성 throughput 또는 MFU)	환산이 닫히는 논문 1 편 (상한). MFU 보고 1 편	R1의 대체 경로
R3	B_s 를 한 점이 아니라 최소 3 점 으로 sweep 하고 각 점의 품질을 인쇄	3 점 이상 4 편, 그중 절대 단위 2 편, 판별식 통과 논문 중에는 0 편	기울기와 포화점
R4	품질은 공개 벤치마크 에서 같은 지표·같은 baseline arm 으로 각 점마다. 저자 제작 변형은 부가로만	sweep 보유 4 편의 벤치 교집합 공집합	y 축
R5	B_s 가 무엇당예산 인지 (문맥/세션/과제/라운드) 명시하고, 그것을 나눠 값을 N_q 의 분포를 보고	실서빙 N_q 분포 0 편	식 (A)의 분모
R6	각 점에서 wake 쪽 델타 도 — 특히 매 질의 프롬프트에 더해지는 $ \hat{c} $ 와 원 문맥 $ c $	L_w 요구 단위 0 편	총비용의 부호
R7	저장소 크기 C_{cap} 과 질의당 주입량 $ \hat{c} $ 을 분리해서	두 양이 같은 단위로 섞여 인쇄됨	포화의 귀속
R8	B_s 를 라운드마다 내는지 한 번만 내는지, 그리고 라운드 수	라운드 수를 독립변수로 놓은 실험 0 편	총예산과 ρ 의 짝
R9	그림 축에만 그리지 말고 표에 숫자로	그림만 있고 인용가능한 숫자가 없는 논문 1 편	점을 찍을 수 있음

아홉이 같은 무게는 아니다. **최소 실행 집합은 R1(또는 R2)·R3·R4 셋이다.** 이 셋만 갖춰지면 한 경로안에서 품질-예산 곡선의 첫 판이 그려진다 — 절대 단위의 x 축, 3 점 이상, 공유 벤치의 y 축. 여기에 R5·R6이 붙으면 곡선의 y 축을 품질에서 **총비용**으로 옮길 수 있고, 그때 비로소 ch25의 상각논증과 같은 평면에서 만난다. R7·R8·R9는 위생항목이지만 R7이 빠지면 포화의 귀속이 불가능해지고 R8이 빠지면 총예산이 정의되지 않는다.

[평가] 이 목록의 실행 비용이 낮다는 사실을 기록해 둔다. 아홉 중 새 실험을 요구하는 것은 R3 하나뿐이고, 그마저 “이미 돌린 설정을 두 점 더 돌린다”에 해당한다. 나머지 여덟은 전부 이미 존재하는 실행 로그의 인쇄다 — 저자는 자기 batch·시퀀스 길이·step

수·장치 수를 알고 있고, 프롬프트에들어간 토큰 수도 알고 있으며, 저장소 크기도 알고 있다. ch25가 자기 반증목록에서 짚은 것과 같은 구조다 — 문헌의 상태를 바꾸는 데 필요한 비용이이 분야의 다른 어떤 진전보다 낮다. 그런데도 비어 있다는 것이 § 28.3.4의 진단 (관행의 부재) 을 다시 확인한다.

28.4 판정

[평가] 판정 1 — 현재 문헌으로 B_s -품질 곡선은 그릴 수 없다. 조건 없이 단정한다.

이 corpus 29 편에서 B_s 를 절대 FLOPs 또는 토큰으로 인쇄한 논문은 1 편이고 그마저 파라미터 수가 없어 환산이 달하지 않는다. 19 편은 침묵하며 그중 9 편은 애초에 B_s 가 정의되지 않는다. 축 위에 3 점 이상을 놓은 4 편의 벤치마크 교집합은 공집합이고, 두 편 이상이 공유하는 벤치마크가 하나도 없다. x 축과 y 축이 동시에 깨져 있으므로 한쪽만 고쳐도 그림은 나오지 않는다. 이 판정의 범위를 명시한다 — 이것은 문헌의 보고 상태에 대한 판정이지 “그런 곡선이 존재하지 않는다” 는 주장이 아니다. 곡선은 있을 수 있고, 다만 지금 그것을 그린 사람이 없다. 그러므로 이 장은 지수도 포화점도 함수형도 진술하지 않으며, 없는 곡선을 지어내지 않는 것이 이 판정의 실행이다.

[평가] 판정 2 — B_s 는 (N, D) 와 같은 종류의 축이 아니다. 상각 가능성이 축의 정의를 바꾼다. 조건 없이 단정한다.

(N, D) 는 배포본 하나에 지불되고 그 배포본의 전 트래픽에 나뉜다 — 분모가 사실상 무한하므로 상각이 자명하고, 그래서 고전 법칙은 상각을 회계에서 지운 채 두 축만으로 성립한다. B_s 는 문맥·세션·사용자 하나에 지불되고 유한한 N_q 로 나뉜다. 같은 FLOPs가 워크로드에 따라 다른 비용이 되므로 눈금이 실험의 불변량이 아니다. 여기에 두 성질이 더 붙는다 — B_s 는 라운드마다 재소비되므로 라운드 수 없이는 총량이 정의되지 않고, 그 산출물이 wake 비용을 바꾸므로 y 축이 품질만으로 달하지 않는다. 그러므로 그릴 수 있는 최소 형태는 식 (28-1)·(28-2)이며, B_s 단독의 곡선은 보고가 부실해서가 아니라 축이 그런 종류이기 때문에 미완성이다.

[평가] 판정 3 — 하나의 곡선이 아니라 세 곡선이다. 경로를 지정하지 않은 B_s 곡선은 부호가 정의되지 않는다.

E -경로에서 B_s 를 키우면 $\hat{c}$ 가 길어지고 매 질의의 프롬프트 비용이 늘어난다 (→ ch25 식 (25-2)). Θ -경로에서는 merge 뒤 토큰당 계산이 그대로이고 요청당 delta 스테이징만 남는다 (→ ch19 § 19.5, ch26 § 26.3.5). W -경로에서는 $N_q = 1$ 이라 상각이 성립하지 않고 총계산이 재귀 횟수에 비례해 는다 (→ ch17 § 17.5). y 축을 품질로 놓으면 세 경로를 같은 평면에 겹칠 수 있으나, y 축을 비용으로 놓는 순간 겹칠 수 없다. 조건을 명시한다 — 이 판정은 경로별 곡선을 부정하지 않는다. 부정하는 것은 세 경로를 합친 단일 곡선이다.

[평가] 판정 4 — 포화점은 C_{cap} 과 함께 보고되지 않으면 귀속되지 않는다.

곡선에서 실무적으로 값어치 있는 특징은 지수가 아니라 포화점인데, sleep 의 산출물은 그릇에 담기고 그 그릇의 크기가 C_{cap} 이다. 위로는 $\text{mem}_U \leq H(\Theta)$ 의 천장이, 아래로는 상태가 과제정보량보다 작을 때의 오류율 바닥이 걸린다 (→ ch09 § 09.3, § 09.6). 같은 B_s 라도 그릇이 다르면 천장이 다르므로, 포화가 관측되어도 예산의 포화인지 그릇의 포화인지 구별되지 않는다. 그런데 판별식 통과 논문 중 C_{cap} 을 인쇄한 논문이 0 편이다 (→ ch26 § 26.4). ch22의 사례가 이 위험을 예시한다 — 같은 step 예산에서 결과를 정한 것은 예산이 아니라 무엇을 쓰는가였다 (→ ch22 § 22.6). 따라서 B_s 축의 보고 요구에 C_{cap} 이 함께 들어가야 한다 (R7).

[평가] 판정 5 — 이 장이 내놓는 것은 곡선이 아니라 최소 보고요구 R1 – R9이며, 그 실행 비용은 낮다.

최소 실행 집합은 셋이다 — R1(또는 R2)로 x 축의 눈금을, R3으로 3 점 이상을, R4로 공유 벤치의 y 축을 확보하면 한 경로 안에서 첫 곡선이 그려진다. R5·R6이 더해지면 y 축을 품질에서 총비용으로 옮길 수 있고 그때 ch25의 상각 회계와 같은 평면에서 만난다. 아홉 중 새 실험을 요구하는 것은 R3 하나뿐이고 나머지 여덟은 이미 존재하는 실행 로그의 인쇄다. 문헌의 상태를 바꾸는 데 필요한 비용이 낮다는 사실 자체가 판정의 일부다.

[평가] 판정 6 — B_s 의 침묵은 기제의 성질이 아니라 상속받은 보고 관습의 산물이다. 순서로만 단정한다.

보고율은 Θ -경로가 E -경로보다 뚜렷하게 높다 [X3 Q1]. Θ -경로는 ML 학습 논문의 관습을 물려받아 step·batch·GPU 시간을 적는 칸을 이미 갖고있고, E -경로는 시스템·제품 논문의 관습을 물려받지연과 정확도는 적지만 쓰기 계산량을 적는 칸이 없다. 이 판정에는 유보가 둘 붙는다. 첫째, 순서만 인용한다 — 표본이 작은 경로가 있고 비율 자체는 본문 수치가 아니다 [X3 D8]. 둘째, 원인귀속 (“관습”)은 이 책의 추론이며 관습을 직접 측정한 것이 아니다. 그러나 귀결은 측정과 무관하게 성립한다 — 침묵의 원인이 측정 난이도가 아니라면 R1 – R9는 실행 가능하다.

28.5 반증 조건

위 여섯 판정은 아래 관측으로 틀렸음이 드러난다. 각 항목은 무엇을 무너뜨리는지 명시한다.

1. 세 편 이상의 논문이 같은 공개 벤치마크에서, 절대 예산 단위의 B_s 3 점 이상과 그 점들의 품질을 함께 인쇄하는 경우. 판정 1이 즉시 무너진다. 이것이 가장 직접적인 반증이며 R1·R3·R4를 동시에 만족한다. 세 편이 같은 경로일 필요는 없으나, 다른 경로를 섞으면 판정 3에 따라 y 축을 품질로그로 고정해야 한다.
2. 기존 논문의 부록 또는 공개 아티팩트에서 batch·시퀀스 길이·파라미터수·MFU가 복원되어, (a)~(c) 후보 8 편 중 3 편 이상의 FLOPs 환산이 실제로 닫히는 경우. 판정 1의 x 축 절반이 해소된다. 이 항목은 이 감사의 가장 약한 고리를 정면으로 겨눈다 — X3의 1 차 대상은 원문이 아니라 이 책의 노트이고, 노트가 놓친 값은 이 감사도 놓친다. 원문에 있는데 노트에 없었다면 계수가 틀린 것이고, 그 경우 이 책의 잘못이다.
3. 실서빙 로그에서 N_q 의 분포가 보고되고, 그 분산이 작아 B_s/N_q 가 사실상 B_s 의 상수배가 되는 경우. 판정 2의 실무적 무게가 사라진다 — 눈금의 워크로드 의존성이 상수배로 흡수되면 B_s 를 그대로 축으로 써도 된다. 다만 구조적 진술 자체는 남는다는 점을 명시한다. 분포가 넓으면 반대로 판정 2가 강화되므로 이 항목은 양방향으로 열려 있다.
4. E -경로의 어떤 설계가 원 문맥을 실제로 버려 매 질의프롬프트를 $|c|$ 에서 $|\hat{c}|$ 로 줄이면서 품질을 유지하는 것이 측정으로 제시되는 경우. 판정 3이 약해진다 — 그 설계에서 B_s 의 wake 결합 부호가 Θ -경로와 같아지므로 두 경로를 같은 비용 평면에 놓을 수 있다. 이것은 ch25가 판정 2의 탈출경로로 지목한 관측과 같은 관측이다 (→ ch25 § 25.5).
5. 같은 B_s ·같은 벤치마크에서 C_{cap} 만 스윙했는데 (rank·저장 dtype·저장 바이트) 품질이 바뀌지 않는 경우. 판정 4가 무너진다 — 포화가 그릇과 무관하다면 B_s 단독 귀속이 정당해진다. 반대로 C_{cap} 을 키울 때 같은 B_s 에서 품질이 계속 오르면 판정 4가 강화된다. 어느 쪽이든 이 관측은 corpus에 0 건이므로 실행되기만 하면 새 사실이다.

6. 어떤 경로가 라운드 수를 독립변수로 놓고, 누적 B_s 에 대해 품질이 3라운드 이상에서 단조 증가함을 보이는 경우. § 28.3.5의 셋째항목(반복이 축을 하나 더 만든다)이 그 경로에서 해소되고 R8의 무게가 줄어든다. 반대로 비단조가 관측되면 B_s 축은 ρ 없이 읽을 수 없다는 것이 확정된다.
7. 이 감사의 라벨 규칙 자체가 틀렸음이 드러나, 재분류 후 (a) 범주가 3편 이상이 되는 경우. E1의 계수가 바뀌고 판정 1의 근거가 그만큼 약해진다. 확인 방법을 명시한다 — 각 논문의 증거 스니펫과 모호성플래그를 원문과 대조하고, PYTHONHASHSEED=0으로 두 번 실행해 산출물의 sha256이 같은지 확인하면 된다. 편집자 override가 0건이므로 집계는 전부 코드의 산물이고, 규칙을 고치면 계수도 재현 가능하게 바뀐다.
8. 이 corpus 밖에 이미 R1·R3·R4를 만족하는 논문이 세 편 이상 존재하는 경우. 판정 1은 “이 책이 deep-read한 29편에 대해서만” 참이 된다. 이 감사는 문헌 전체의 표본이 아니며, 그 한계는 이 책이 진 것이다 [X3 D9]. 그런 논문을 찾는 부담은 반박하는 쪽이 아니라 이 책 쪽에 있다.

반증이 아닌 것도 명시한다. 개별 논문의 범주가 (b) 나 (c) 나 하는 재분류 한두 건은 반증이 아니다 — 이 장의 판정은 범주별 편수의 자릿수와 순서 위에서 있고, 한 편이 옮겨 다녀도 (a) 1편 · (f) 19편 · 교집합 공집합은 바뀌지 않는다. 마찬가지로 어느 논문이 B_s 를 인쇄했는데 그 값이 예상보다 크거나 작다는 관측도 반증이지 아니다 — 이 장은 B_s 의 크기에 대해 아무 주장도 하지 않는다.

요약

- 이 corpus 29편에서 B_s 를 절대 FLOPs 또는 토큰으로 인쇄한 논문은 1편이고, 5편은 optimizer step, 2편은 벽시계, 2편은 눈금 없는 서수 손잡이로만 보고하며, 19편은 침묵한다. 그 19편 중 9편은 오프라인 패스 자체가 없어 B_s 가 정의되지 않는다 — “보고안 함”과 섞여 세면 안 된다.
- 단위를 공통 축으로 옮길 다리가 없다. (a)~(c) 후보 8편 중 노트만으로 FLOPs 환산이 달하는 것은 1편(상한)이고, 벽시계를 계산량으로 옮길 MFU·달성 throughput을 보고한 노트는 29편 중 1편인데 그 편은 B_s 가 정의되지도 않는다.
- y축도 함께 깨져 있다. B_s 축에 3점 이상을 놓은 4편의 벤치마크교집합은 공집합이고 두 편 이상이 공유하는 벤치마크가 0건이다. 그리고 판별식을 통과한 논문 중 절대 예산 단위로 3점 이상을 놓은 논문은 0편이다 — 예산을 축으로 다룬 두 편은 이 라인 밖의 이론·반증 논문이다.
- 이 라인에 이름을 준 논문의 전문에서 숫자 뒤에 tokens가 붙은 꼴은 두 번 나오는데 둘 다 문제 속의 게임 말이고 ‘FLOP’은 0회 나온다. 상각 논증의 분자에 숫자가 없다.
- B_s 는 (N, D) 와 같은 종류의 축이 아니다. 유한한 N_q 로 나뉘므로 눈금이 워크로드에 의존하고, 라운드마다 재소비되며, 산출물이 wake 비용을 바꾼다. 그래서 그릴 수 있는 최소 형태가 식 (28-1)·(28-2)이고 B_s 단독 곡선은 정의상 미완성이다.
- 하나의 곡선이 아니라 세 곡선이다 — B_s 증가가 E에서는 wake를 늘리고 Θ 에서는 그대로 두며 W에서는 총계산을 늘린다. 포화점은 C_{cap} 과 함께 보고되지 않으면 예산의 포화인지 그릇의 포화인지 귀속되지 않는다.
- 이 장의 산출물은 곡선이 아니라 최소 보고 요구 R1–R9이며, 최소 실행 집합은 R1(또는 R2)·R3·R4 셋이다. 아홉중 새 실험을 요구하는 것은 R3 하나뿐이고 나머지 여덟은 이미 존재하는 로그의 인쇄다.
- 이 책은 자기 실험에도 같은 자를 댔다. X3은 X1의 입력 가정에서 세어긋남을 찾았다 — 가정한 문맥 길이가 인쇄된 예시보다 크고, sweep 상단이 잘려 있었으며, $|\hat{c}|/|c|$ 의 관측 범위가 그리드 양 끝 밖에 있다. 앞의 둘은 방향이 X1과 같아 ch25의 순서 판정을 강화하고, 셋째는 방향을 주장하지 않는다. L^* 의 절대치는 어느 경우에도 인용 불가다.

다음 장으로

이 장은 축을 판정했다 — B_s 는 지금 축이 아니고, 축이 되려면 $N_q \cdot C_{cap}$ 라운드 수와 함께와야 한다. 그 요구 목록이 남긴 것 중 둘이 다음 장의 재료다.

첫째는 라운드 축이다. 식 (28-2) 의 합이 라운드에 대한 합인 이상 반복 sleep 의 열화율 ρ 가 곡선 안으로 들어오는데, corpus 에서 라운드당비율을 인쇄한 논문이 0 편이다. 둘째는 Θ -경로가 요구하는 워계층이다 — ch26 이 사용자당 수 MB 의 delta 를 요청 지연 안에 스테이징해야한다는 것까지 확정했고, 그 계층을 실제로 어떻게 설계할지는 열려 있다.

ch29 는 이 둘을 받아 system·infra 쪽으로 자를 옮긴다. 상태바이트가 어느 계층에 얼마나 있는지 (ch26), 반복 consolidation 이 얼마나 열화하는지, 그리고 그 위에서 memory-device 기회가 실제로 성립하는 자리가 어디인지 — 그것이 다음 장의 판정할 질문이다. 이 장이 남긴 “그릴 수 없다” 는 판정 자체와, 그것을 뒤집을 조건들은 ch30 의 미해결 목록으로 간다.

ch29. system·infra와 memory-device 기회

29.1 판정할 질문

이 축이 진행되면 시스템과 메모리 소자에 무슨 기회가 생기는가.

ch01 이 세운 독자의 1 번 질문 — “그래서 그게 decode 에서 뭘 바꾸는데?” — 의 마지막 형태가 이것이다. 앞의 장들이 경로마다 그 답을 한 조각씩 냈고, 이장은 조각들을 메모리 계층 위에 놓는다. 그러므로 먼저 답이 될 수 있는 것과 될 수 없는 것을 못 박는다.

답이 될 수 있는 것. 상태의 크기 · 수명 · 상주 계층 · 트래픽 성격은 앞의 세 장에서 이미 확정되었다. 상태량은 ch26 이 세 경로에 대해 복원했고, 그 상태가 언제 읽히고 언제 쓰이는지는 ch25 의 회계와 ch22 의 반복 실험이 정한다. 이 넷이 정해지면 **어느 메모리 계층에 어떤 새 요구가 걸리는가**는 연역된다. 구체적으로 이 장은 셋을 답한다 — (i) 새 요구가 걸리는 계층의 위치와 그 계층이 만족해야 할 요구의 **형태**, (ii) shared-weight batching 이 경로마다 어디서 어떻게 깨지고 그중 무엇이 완화가 가능한가, (iii) 그 기회가 성립하기 위해 **반복**이 견뎌야 하는지 평.

답이 될 수 없는 것. 첫째, **절대 지연 · 대역폭 · wall-clock** 은 **답이 아니다**. corpus 스물한 편에서 L_w 를 인쇄한 논문이 0 편이고 (→ ch25 § 25.2.1), 이 책의 실험도 silicon 측정이 아니라 회계다. 그러므로 “delta 스테이징에 몇 ms 가 필요한가” 는 이 장의 증거가 답할 수 없다. 둘째, **소자 종류를 지목하는 것은 답이 아니다**. 요구의 형태 (입도 · 마감 · 쓰기 주기) 는 연역되지만, 어떤 매체가 그 요구를 만족시키는지 는 이 책이 가진 어떤 증거로도 판정되지 않는다. 요구 사양을 쓰는 것과 제품을 고르는 것은 다른 일이며, 이 장은 앞의 것만 한다. 셋째, **Θ -경로가 실제로 배포된다는 예측은 답이 아니다**. 경로별 유망성 판정은 ch27 이 소유하고 반증 조건의 정식화는 ch30 이 소유한다. 이 장이 만드는 것은 **조건부 기회 지도**다 — “ Θ -경로가 사용자 수준 개인화로 간다면, 그때 새로 걸리는 요구는 이것이다.”

넷째를 따로 적는다. **이 장은 Neural Memory 모노그래프의 결론을 이어받지 않는다**. 그 책은 W -경로에서 decode 경로 위의 whole-state read-modify-write 를 memory-centric 기회로 지목했다 (→ NM ch20). 이 장이 지목하는 자리는 그것과 **다른 자리**이며, 두 결론을 하나의 서사로 잇지 않는다. 왜 다른 자리인지는 § 29.3.2 가 표로 확정한다. 두 책이 같은 단어 (“memory-centric”) 를 쓴다는 사실은 같은 자원을 가리킨다는 뜻이 아니다.

29.2 증거

이 장이 쓰는 증거는 Part II 가 확립한 사실 다섯과 이 책의 실험 둘이다. 각 항목에 등급을 붙인다. 등급 규칙은 이 책의 정직성 계약을 따른다 — 비율 · crossover · 순서 · 구조 분류는 본문 단정문으로 쓰고, 절대치는 하한 또는 방향성으로만 쓴다.

표 29-1 — 이 장의 증거와 등급

#	증거	출처	등급
E1	Part II 가 원문 대조로 채운 스물한 편 중 L_w 를 인쇄한 논문 0 편, C_{cap} 을 인쇄한 논문 0 편	→ ch25 § 25.2.1	corpus 사실. 단정
E2	경로별 상태 바이트와 사용자 수 선형 스케일링	X2 Q1·Q3 (→ ch26 § 26.3.5)	회계 (측정 아님). 자릿수와모양만 단정
E3	네 서빙 구조에서 shared-weight batching 의 성립 여부와 트래픽 성격	X2 Q4 (→ ch26 § 26.3.6)	구조 분류. 단정
E4	저장 바이트당 정보 밀도는 dtype 이 정한다 — 양자화는 전제이지 최적화가 아니다	X2 F3 (→ ch26 § 26.4 판정 3)	분모로 정해지는 배수. 단정. 분자는 하한
E5	직접 편집은 분리 가능한 아티팩트를 만들지 않는다 — 개인화하면 사용자당 전체 사본	X2 F5 (→ ch26 § 26.4 판정 5)	구조 사실. 단정
E6	merge 된 Θ 의 사실당 한계 decode 비용은 0 이고, 쓰기 쪽은라운드마다 전 모델 read-modify-write 다	→ ch22 § 22.7	원문 근거. 단정
E7	반복의 실증 상한은 $k = 100 \cdot 4B \cdot \text{LoRA}$ $r=16$ 이고, 그 설정은 사용자별 쓰기를 고려하지 않는다	→ ch22 § 22.6.2·§ 22.7	원문 근거. 단정
E8	반복 라운드의 열화는 무엇을 재느냐로 모양이 갈리고, 이송해야 할양은 라운드당 비율이 아니라 crossover K^* 다	X4 F4·F5	LLM 실험 아님. 부호 · 모양 · 순서만
E9	replay 를 보존된 원본에 앵커하면 모델 자기출력에 앵커할 때보다 모든혼합비에서 오래버틴다	X4 F6	// 조건 간 순서만
E10	예산을 맞춘 채 replay 혼합비를 올리면 유지가 늘고 새 항목 습득의여유가 줄어든다	X4 D5	// 방향만. 정량 금지

#	증거	출처	등급
E11	손실 곡선이 평평해지는 것은 안정의 증거가 아니다 — 1 차 모멘트 가포화하는 동안 생성 분포의 퍼짐 (2 차 모멘트) 이 죽는다	X4 F3	// 부호 · 모양만

E8 – E11 에 붙은 제한을 한 번 더 명시한다. 이 책의 실험 X4 는 가우시안 위치추정과 선형 연상 기억 위에서 돌았고 LLM 실험이 아니다. 그 결과에서 이송가능한 것은 **함수형 · 부호 · 조건 간순서 · crossover 의 존재와 이동 방향**이며, 라운드수 · 유지율 퍼센트 · 라운드당 열화 pp 는 이송불가다. 이 장은 X4 에서 단 하나의 절대치도 인용하지 않는다.

두 실험의 분업도 명시한다. X2 는 “**얼마이고 어디에 사는가**” 를 답하고 X4 는 “**얼마나 오래 사는가**” 를 답한다. X2 는 구성 파라미터에서 바이트를 복원하는 회계이므로 자릿수 · 모양 · 구조분류를 주고 절대 지연이나 처리량은 주지 않는다. X4 는 반복 루프의 함수형을 주지만 LLM 이 아닌 최소 모형 위에서 돌았으므로 부호와 순서만 준다. 그러므로 이 장의 판정은 전부 **형태에 대한 판정**이고, 값을 다루는 판정은 하나도 없다. 값을 다루려면 § 29.4 판정 6 이 지목하는 인쇄값셋이 필요하다.

X4 에 대해 한 가지를 더 적어 둔다. 이 책이 X4 에서 이송하는 E9 의 부호 —replay 는 보존된 원본에 앵커되어야 한다 — 는 우연한 일치가 아니다. X4 의 최소 모형은 [MCI] 의 세 데이터 스케줄이 설계행렬이 아예 없는 문제에서도 같은 함수형으로 재유도된다는 것을 확인했고, 그 일치가 “누적이 교체보다 낫다” 는 결론을 그 논문이 증명한 범위보다 넓게 정당화한다 [X4 F1] (→ ch06 § 06.6). 같은 부호가 [Continual Facts] 의 frozen-teacher 대비에서 독립적으로 나온다. **세 결과가 같은 부호를 가리키므로 이장은 그 부호를 판정에 쓴다** — 둘은 X4 안의 서로 다른 부분 (해석적 유도과 사실 주입 시물) 이고 하나는 원 논문이므로, 서로 독립인 근거는 둘이다 (§ 29.4 판정 4). 반대로 X4 가 재현하지 못한 효과들은 이장에서 쓰지 않는다 — 재현 실패는 원 논문의 반박이 아니라 기제 배제이며, 시스템 요구를 도출할 재료가 아니다.

증거 쪽에서 함께 기록해야 할 공백도 있다. K^* 를 실제모델에서 **젠 논문이 corpus 에 없고, sleep 라운드를 100 회 넘게 돌린 논문도 0 편이다**. 반복 열화율 ρ 를 라운드 수를 독립변수로 놓고 인쇄한 논문역시 0 편이다 (→ ch25 § 25.5 의 8 번 반증 항목이 이 장으로 넘어온다). 그러므로 이 장의 반복 관련 서술은 전부 **조건부**이며, 그 조건이 언제 깨지는지는 § 29.5 가 적는다.

29.3 분석

29.3.1 기회의 좌표를 셋으로 못 박는다 — 총량이 아니라 입도 · 마감 · 주기다

ch26 이 남긴 수치부터 옮긴다. dense-8B 앵커에서 사용자당 Θ delta 는 LoRA $r=16 \cdot S = \{W_q, W_v\} \cdot \text{bf16}$ 에서 16.78 MB 이고, 사용자 10^6 명이면 웹 스토리지 총량이 16.777 TB, int4 로 내리면 4.194 TB, $r=64$ bf16 이면 67.109 TB 다. 직접 편집으로 개인화하면 같은 사용자수에서 16,000 TB 로 세 자릿수가 된다 [X2 Q3] (→ ch26 표 26-4). 단정하는 것은 TB 값이 아니라 **선형이라는 모양과 두 방식 사이의 세자릿수 격차**다.

이 수치들이 시스템 요구로 번역될 때 중요한 것은 총량이 아니다. **16.777 TB 는 오늘의 데이터 센터에서 문제가 아니다**. 문제는 그 안의 16.78 MB 하나를 어떤 입도로, 어떤 마감 안에, 얼마나 자주 다시 쓰면서 다루느냐다. 세 좌표로 나눈다.

좌표 1 — 읽기 입도. Θ 경로의 읽기는 요청당 한 번이고 크기는 사용자당 delta 하나다. 8B 앵커에서 그 값은 구성에 따라 4.19 MB($r=4, \{W_q, W_v\}$) 에서 134.22 MB($r=64, \{W_q, W_k, W_v, W_o\}$) 사이다 [X2 Q1]. 이것은 KV cache 의 스트리밍 접근도 아니고 벡터 인덱스의 작은 무작위 gather 도 아니다. 수 MB 짜리 객체 하나에 대한 무작위 접근이며, 접근 키는 사용자 ID 다.

좌표 2 — 마감. 그 읽기는 요청 지연 안에 끝나야 한다. 여기서 쓰는 기호 셋을 못 박는다 — N_u 는 동시 서비스 사용자 수 (→ ch26 § 26.2 가 같은 뜻으로 쓴다), T_k 는 sleep 라운드 간격, Φ_{wr} 은

읽 계층이 받는 쓰기율이다. 뒤의 둘은 이 장안에서만 쓰며, 특히 Φ_{wr} 은 비용 4 종에 추가되는 다섯째항이 아니라 C_{cap} 에서 파생된 트래픽률이다. 네 번째 국소기호 K^* (성능이 기준선을 유지하는 마지막 sleep 라운드)는 § 29.3.5 에서 정의한다. 라운드 첨자 k 는 예약 기호 그대로이고, 대문자 K 는라운드의 총수를 가리킨다. 마감 조건은 형태로 쓰면이렇다.

$$t_{load}(C_{cap}) + t_{merge}(C_{cap}) \leq L_w \quad (29-1)$$

이 식이 말하는 것은 단순하다. 사용자 delta 를 읽 계층에서 끌어와 base 가중치에 적용하는 시간이 wake 지연 예산 안에 들어가야 한다. 그런데 corpus 는 좌변의 두 항도, 우변의 예산도 인쇄한 적이 없다 (E1). LoRA 논문자신이 분리된 delta 형태의 latency 를 측정하지 않았고 (→ ch03 § 03.6), Generative Adapter 는 동시 다중 사용자 서빙을 한 문장도 다루지 않는다 (→ ch16 § 16.7). 그러므로 (29-1) 은 값이 아니라 구조로만 쓸 수 있다— 이 부등식이 존재한다는 것, 그리고 좌변이 C_{cap} 에비례한다는 것.

좌표 3 — 쓰기 주기. sleep 라운드마다 사용자 delta 가 새로만들어지므로 읽 계층은 주기적으로 다시 쓰인다. 라운드당 쓰기량은 형태로이렇다.

$$\Phi_{wr} = \frac{N_u \cdot C_{cap}}{T_k} \quad (29-2)$$

$N_u = 10^6$, $C_{cap} = 16.78$ MB, $T_k = 1$ 일이면 하루 약 16.8 TB 의 쓰기가 된다 [본서 산술 — T_k 는 이 책의 가정이고 어느논문에서도 오지 않는다]. 이 값 자체는 단정하지 않는다. 단정하는 것은 Φ_{wr} 이 N_u 에 선형이고 T_k 에 반비례한다는 형태이며, 그 형태가 뜻하는 바는 하나다 — 라운드를 자주 돌수록 쓰기 부하가 그만큼 곧바로 오른다. sleep 라운드의 주기는 품질다이얼이면서 동시에 소자 수명 다이얼이다.

[해설] 세 좌표를 겹치면 이 계층의 성격이 혼한 캐시와 다르다는 것이 드러난다. 읽기는 소량·무작위·마감 있음이고, 쓰기는 대량·주기적·전량 교체다. 읽기 쪽만 보면 낮은 지연의 랜덤 접근 매체가 답이지만, 쓰기 쪽만 보면 내구성과 순차쓰기 대역폭이 답이다. 두 요구가 같은 매체에 동시에 걸리는 것이 이 자리의 특징이며, 서로 다른 축이므로 하나를 최적화해도 다른 하나가 따라오지 않는다 [본서 추론].

두 개의 나눗셈이 이 성격을 확정한다. 둘 다 이 책의 산술이며 앵커 구성에 의존하므로 값이 아니라 자릿수와 형태로 읽는다.

첫째, 스테이징 바이트는 대역폭 문제가 아니다. 8B 모델을 bf16 으로 서빙하면 base 가중치가 16.0 GB 이고 [X2 Q1], decode step 마다 그 전량이 읽힌다. 같은 요청에 붙는 사용자 delta 는 $r=16 \cdot S = \{W_q, W_v\} \cdot \text{bf16}$ 에서 16.78 MB 이므로 base 가중치 읽기의 약 0.105%다 [본서 산술]. 요청 하나가 수백 토큰을 생성한다면 그 비는 다시 세 자릿수 작아진다. 즉 delta 는 대역폭예산 안에서 눈에 띄지 않는 양이다. 그것이 문제가 되는 이유는 크기가 아니라 그 바이트가 첫 토큰 이전에도 도착해야 한다는 것이고, 그래서 이 자리의 지표는 GB/s 가 아니라 접근 지연과 꼬리분위수다.

둘째, 쓰기 내구성은 라운드 주기가 정한다. (29-2) 에서 라운드마다 모든 사용자 delta 가 교체되므로 하루 쓰기량과 상주 용량의 비가 곧 매체의 하루당 전체 재기록 횟수가 된다. 그 비는 형태로 이렇다.

$$\text{DWPD} = \frac{1 \text{ day}}{T_k} \quad (29-3)$$

용량이 몇 TB 인지과 무관하다. 사용자 수도, delta 크기도, dtype 도이 비에서 약분되어 사라지고 남는 것은 라운드 주기 하나다. 하루 한 번자면 1, 한 시간에 한 번 자면 24 다 [본서 산술]. 그러므로 “얼마나 자주자는가” 라는 알고리즘 다이얼이 저장 매체의 수명 사양을 직접정한다. 이 장이 소자를 지목하지 않으면서도 요구를 쓸 수 있는 이유가 (29-3) 이다 — 매체가 무엇이든 이 비는 같은 형태로 걸린다.

여기서 곧바로 따라오는 것이 이 장의 첫 번째 구체적 답이다. 기회의 자리는 HBM 대역폭이 아니라 요청 지연 안에 delta 를스테이징하는 읽 계층과 그 계층의 쓰기 내구성이다 [X2 F4]. HBM 이 아닌 이유는 자릿수로 정해진다 — 사용자당 수 MB 를 10^6 명분상주시키려면 TB 단위가 필요하고,

그 값은 오늘의 가속기 HBM 용량 밖이다. 그리고 상주시킬 필요도 없다. 요청당 한 번 읽히는 객체는 요청이 올 때올라오면 된다.

29.3.2 이 자리는 Neural Memory 가 짙은 자리와 다르다

두 책의 결론을 잇고 싶은 유혹이 여기서 가장 강하다. 둘 다 “가중치 형태의 상태가 메모리 계층에 새 요구를 만든다” 고 말하기 때문이다. 그 유혹을 표로 끊는다.

표 29-2 — 두 모노그래프가 지목한 memory-centric 기회의 좌표 대조. 각행은 두 책이 확정한 사실 (→ NM ch20, → ch26 § 26.3.5) 에서 이 책이 세운대조표다. [본서 추론]

축	Neural Memory — W-경로	본서 — Θ -경로
병목 자원	HBM read-modify-write 대역폭	웜 계층의 무작위읽기 마감과 쓰기 내구성
시간 입도	토큰당	읽기는 요청당, 쓰기는 sleep 라운드당
상태 수명	세션 (transient)	사용자 (영속)
상태량의 자리	세션당 GB 단위, HBM 상주	사용자당 MB 단위, 웜 상주
새 primitive	decode critical path 위의 backward pass + whole-state RMW	요청 경로 위의 delta 스테이징과 적용
배치 파손의 형태	grouped-GEMM 으로 batch 는 형성되거나 amortization 이 없다	delta 단위 라우팅. 같은 delta 를 쓰는 요청만모인다
트래픽의 방향	쓰기 편향, 공유 불가	읽기 편향, 사용자별

행 하나하나가 다르다. 특히 **병목 자원이 다르고 시간 입도가 다르다**. W-경로의 문제는 토큰마다 전 상태를 읽고 쓰느라 대역폭이 소진되는 것이고, Θ -경로의 문제는 요청이 도착한 뒤 수 MB 를 마감안에 끌어오는 것과 그 계층이 주기적 전량 교체를 건디는 것이다. 전자는초당 바이트의 문제이고 후자는 접근 지연과 쓰기 수명의 문제다.

[평가] 그러므로 이 책은 Neural Memory 의 pair thesis 를 sleep-time 쓰기 트래픽으로 확장하지 않는다. 확장 시도는 이 회계안에서 실패한다 — 두 기회가 같은 자원을 놓고 이야기하지 않기 때문이다. 그리고 더 중요한 것은 **두 경로가 한 배포에 공존할 때 두 요구가 상쇄되지 않고 합쳐진다는 점**이다. fast weight 를 쓰는 모델에 사용자별 delta 를 없으면 HBM 은 여전히 토큰당 RMW 로 포화하고, 그 위에 요청당 웜계층 스테이징이 추가로 얹힌다. 두 기회를 하나로 합쳐 부르는 서술은 이합산을 감춘다. 이 판정은 조건 없이 성립한다 — 두 상태의 수명과 시간입도가 다르다는 사실만으로 따라 나오기 때문이다.

한 가지는 공유된다. 두 경로 모두에서 **shared-weight batching 의무상 상각이 사라진다는 점**이다. 그러나 사라지는 방식이 다르고, 그 차이가다음 소절의 주제다.

29.3.3 shared-weight batching 은 경로별로 다른 자리에서 깨진다

“ Θ -경로는 batching 을 깬다” 는 문장은 절반만 맞다. ch06 과 ch20 이이미 구분을 세워 두었다 — 인구 수준 Θ -경로와 사용자 수준 Θ -경로는 서빙에서 정반대로 거동한다 (→ ch06 § 06.9, ch20 § 20.7). 그 구분을 다섯 행으로 확정한다.

표 29-3 — 경로·구조별 shared-weight batching 의 파손 지점과완화 가능성 [X2 Q4 위에 이 책이 완화 열을 더한다]

구조	batching	깨지는 단위	스케줄러에 새로 걸리는 요구	완화
E -경로	유지	—	없음. 프롬프트 길이와 K_{ret} 만 늘어난다	해당 없음
W -경로	부분 파손	세션	세션마다 상태가 HBM 상주 → 동시세션 수가 상태 바이트로 제한된다. 페이지 단위가 토큰이 아니라 수십~수백 MB 객체가 된다 (→ ch18 § 18.7)	부분적. 상태페이징 · 오프로드로 완화되나 대역폭 하한은 남는다
Θ -경로 delta (사용자 수준)	delta 단위 파손	사용자	요청마다 delta 로드 · 적용. 배치 안의 사용자가 다르면가중치가 다르다	가능. 같은 delta 를 쓰는 요청을 모으는라우팅 · 스케줄링
Θ -경로 delta (인구 수준)	유지	—	전 사용자 공유가 중치 1 별. Rosetta 의 해당 행이 발동하지 않는다	해당 없음
Θ -경로 직접 편집 + 개인화	완전 파손	사용자당체크포인트	사용자당 모델 전체 사본	불가능. 분리 가능한아티팩트가 없다

이 표에서 읽어야 할 것은 성립 · 불성립의 이분법이 아니라 **파손의 종류**다. 셋째 행의 파손은 **라우팅 문제**다 — 같은 delta 를 쓰는 요청을 한 배치에 모으면 배치 안에서 가중치가 다시 공유되고, 실패하면 grouped-GEMM 으로 내려간다. 다섯째 행의 파손은 **용량문제**이며 완화 수단이 구조적으로 없다 (→ ch26 § 26.3.6). 완화 가능한대와와 완화 불가능한 대가를 같은 저울에 올리지 않는 것이 이 표의요점이다.

셋째 행에는 그러나 완화의 상한이 있다. 개인화는 정의상 delta 가 사용자마다 다르다는 뜻이므로, 같은 delta 를 쓰는 요청을 모으는전략의 효과는 사용자당 동시 요청 수에 묶인다. 사용자 하나가 동시에 요청하나만 보내는 대화형 워크로드에서는 배치 폭이 1 에 가까워지고, 그때 남는것은 grouped-GEMM 경로뿐이다 [본서 추론 — corpus 어느 논문도 multi-adapter 배치의 처리량을 측정하지 않았다]. Neural Memory 가 W -경로에서 확인한 성질이 여기서 되풀이된다 — batch 는형성되지만 가중치 바이트의 재사용은 늘지 않는다 (→ NM ch20 § 20.2.5).

넷째 행을 따로 강조한다. 인구 수준 Θ -경로는 이 장이말하는 기회를 만들지 않는다. 전역편집 · CPT · 공유 memory layer 는 가중치 1 별을유지하므로 웹 계층도, delta 라우팅도, 쓰기 내구성 문제도 생기지 않는다 (→ ch09 § 09.5, ch20 § 20.7). 반복하면 — **memory-device 기회를 만드는것은 Θ -경로 자체가 아니라 Θ -경로의 개인화**다. 이조건절을 빼면 판정 전체가 과장된다.

29.3.4 delta 를 요청 지연 안에 넣는 방식은 셋이고, 셋이 서로다른 계층을산다

(29-1) 을 만족시키는 설계는 corpus 안에 이미 셋 다 나와 있다. 다만 어느논문도 그것을 서빙 선택지로 나란히 놓지 않았다.

표 29-4 — delta 를 요청 경로에 올리는 세 방식과 각각이 지불하는 것. 각행의 근거는 셀에 적은 장 참조이고, 세 방식을 서빙 선택지로 나란히 놓은것과 마지막 열의 전수 확인은 이 책의 것이다. [본서 관찰]

방식	저장	요청당 지연	batching	대표	측정된 L_w
(a) 미리 머지한 사본	사용자당 모델 전체	로드만. 적용 비용 0	완전파손	직접 편집 개 인화 (→ ch19 § 19.7)	없음
(b) 분리된 delta 로드 후 적용	사용자당 수 MB	스테이징 + 적용	delta 단위 파손	multi-LoRA 서빙 (→ ch03 § 03.5)	없음
(c) 요청 경로에서 delta 생성	0 (또는 원 문맥)	생성 forward pass	delta 단위 파손	Generative Adapter (→ ch16 § 16.4)	없음

세 방식은 **저장·지연·배칭을 세 방향으로 교환한다**. (a)는 지연을 최소화하는 대신 용량을 폭발시키고, (b)는 용량을 최소화하는 대신 요청마다 스테이징 지연을 지불하며, (c)는 저장을 아예 없애는 대신 그 지연을 계산으로 바꾼다. (c)의 대가는 ch16이 이미 수치로 옮겼다 — 요청당 32M floats의 adapter가 prefill 도중 생성되고, 그 경로에 GEMM이 아닌 연산이 낀다 (→ ch16 § 16.7).

[평가] 마지막 열이 이 표의 결론이다. 세 방식 중 어느 것에 대해서도 L_w 를 인쇄한 논문이 없다. 그러므로 오늘 이 셋사이의 선택은 측정이 아니라 취향으로 이루어진다. 이것은 개별 논문의 부주의가 아니라 ch25가 판정 1에서 확정한 corpus 상태의 직접적 귀결이며 $-L_w$ 칸이 0 편이다 — 그 공백이 가장 아프게 드러나는 자리가 바로 여기다. 뱀 계층을 설계하려는 엔지니어가 필요로 하는 첫 번째 수는 delta 크기가 아니라 스테이징 지연 대 생성 지연의 비인데, 그 비를 계산할 재료가 corpus에 없다.

한 가지는 재료 없이도 말할 수 있다. (b)와 (c) 사이의 갈림은 C_{cap} 과 함께 움직인다. delta가 작을수록 (b)의 스테이징이 싸지고, 클수록 (c)의 생성이 상대적으로 유리해진다. 그리고 C_{cap} 은 § 29.3.6이 보일 대로 dtype이 정한다. 즉양자화는 용량 문제일 뿐 아니라 이 갈림길의 위치를 옮기는 다이얼이기도 하다 [본서 추론].

29.3.5 반복이 이 기회의 시간 지평을 정한다

여기까지의 논의에는 숨은 전제가 하나 있다. **delta가 한 번만 들어지고 끝이라면 뱀 계층의 쓰기 내 구성은 문제가 되지 않는다**. (29-2)의 Φ_{wr} 을 0이 아니게 만드는 것은 반복이다. 그러므로 기회의 실재 여부는 (U- Θ)를 k 번 반복할 수 있느냐에 달려 있고, 그 질문의 증거를 가진 것이 ch22와 이 책의 실험 X4다.

ch22가 확정한 실증 상한부터 옮긴다. 반복을 실제로 돌린 유일한 논문의 상한은 $k = 100 \cdot Qwen3-4B \cdot LoRA\ r=16$ 이고, 그 설정은 **사용자별 쓰기를 고려하지 않는다** — 모든 사실이 전역 Θ 하나에 merge 되므로 그 논문의 설정에서는 batching이 온전하다 (→ ch22 § 22.7). 이 책이 가장 궁금해하는 배포 형태에 대해 그 논문은 아무말도 하지 않는다. 그리고 sleep 라운드를 100회 넘게 돌린 논문이 corpus에 0 편이다.

X4가 더하는 것은 절대치가 아니라 **모양과 순서**다. 다섯 가지를 옮긴다.

(가) **열화의 모양이 무엇을 재느냐로 갈린다**. 같은 반복 루프에서 임계로 채점하는 과제 지표는 한동안 전혀 움직이지 않다가 좁은 폭에서 무너지고, 쓰기가 만든 연속 신호는 첫 라운드부터 매끄럽게 깎인다 [X4 F4]. 두 곡선은 같은 루프의 같은 상태를 재는데 모양이 다르다. 그러므로 “라운드당 몇 퍼센트”라는 단일 비율로 ρ 를 보고하는 관행 자체가 오도다.

(나) **이송해야 할 양은 ρ 가 아니라 crossover다**. X4는 그 양을 K^* 로 부른다 — 성능이 기준선을 유지하는 마지막 라운드다 [X4 F5]. 이 책의 등급 규칙상 crossover의 위치는 이송 가능한 양이 고라운드당 비율은 아니다.

(다) **replay는 보존된 원본에 앵커되어야 한다**. 같은 혼합비 ω · 같은 예산에서 replay 타깃을 보존된 원본으로 두면 모델의 현재 출력으로 두는 것보다 **모든 ω 에서 오래버틴다** [X4 F6]. 단정하는 것은 이 순서이며, K^* 비의 배수는 그 장난감 모형 안의 값이므로 이송하지 않는다. 이 부호는 독립적인 두 번째 근거를 갖는다 — ch22가 옮긴 대로 교사를 Θ_0 로 고정 순차 증류는 역량 +2pp· 유지

율 54%인 반면 자기 누적 merge 를교사로 쓰면 - 31pp·유지율 21%다 [Continual Facts § 6.2, p.14 - 15]. 두 근거는 서로 다른 방법으로 얻어졌고 부호가 같다.

(라) **유지를 사면 습득을 지불한다.** 예산을 맞춘 채 ω 를올리면 유지 쪽 crossover 가 뒤로 밀리고 새 항목 습득의 연속 여유가줄어든다 [X4 D5]. 이 거래는 **방향만** 쓸 수 있다 - 임계지표로는 보이지 않고 연속 지표에서만 보이며, 정량은 이송 불가다.

(마) **평평한 손실 곡선은 안정의 증거가 아니다.** 같은 반복 루프를추정량만 바꿔 돌리면 1 차 모멘트로 켄 위험은 포화하는데, 그 포화는 생성분포가 죽어 갱신 스텝이 작아진 결과다 - 즉 곡선이 평평해지는 것과 상태가살아 있는 것은 다른 일이다 [X4 F3]. 죽는 좌표는 2 차 모멘트, 곧 생성분포의 퍼짐이고 그것은 held-out 손실에 잡히지 않는다. 단정하는 것은**두 좌표가 같린다는 사실과 그 부호**이며, 반감기 라운드 수는이송하지 않는다. 그리고 이 좌표를 라운드의 함수로 인쇄한 논문이 corpus 에 0 편이다.

[해설] 다섯을 시스템 요구로 번역하면 세 가지가 나온다. 첫째, **웹 계층 옆에 콜드 원본 아카이브가 필수다.** (다) 가 성립하면 sleep 라운드는 delta 만 다시 쓰는 것이 아니라 보존된 원본을 다시 읽어야한다. 즉 Θ -경로는 E -경로를 대체하지 않고 **요구한다** - ch22 가 조건 (C4) 로 적은 “정본이 다른 곳에 있는 write-through cache” 구성과 같은 결론이며, 여기서는 알고리즘 요건이 아니라 **저장 계층요건**으로 나타난다. 둘째, **sleep 라운드의 트래픽은 delta 바이트로회계되지 않는다.** 원본 재읽기와 전 모델 read-modify-write 가 함께들어가고 ($\rightarrow$ ch22 § 22.7), 주기적 통합 정책은 라운드 수에 대해 초선형으로누적된다. (29-2) 의 Φ_{wr} 은 웹 계층이 받는 쓰기의하한이다 [본서 추론]. 셋째, **모니터링 지표가 바뀐다.** (가) 때문에 임계 과제 지표만 보는 감시는 경보가 늦고, (마) 때문에 held-out 손실 곡선을 보는 감시는 아예 발화하지 않으며, (나) 때문에 SLA 로걸어야 할 양은 라운드당 열화율이 아니라 “몇 라운드까지 버티는가” 다.

셋째 항목이 운영에 거는 요구가 가장 구체적이다. K^* 가 유한하다는것은 **재기준화 주기가 서비스 설계 변수가 된다**는 뜻이다. 사용자 delta 는 무한히 누적되는 자산이 아니라 수명을 가진 캐시 엔트리이고, 수명이 다하면 Θ_0 에서 다시 쌓아야 한다. 그 재구축은 (29-2) 의정기 쓰기와 별도로 발생하는 **버스트 쓰기**이며, 사용자 집단의재기준화 시점이 겹치면 웹 계층은 정기 부하가 아니라 그 버스트로사이징된다 [본서 추론].

[평가] 그리고 (가) - (마) 의 처방과 ch22 의 다섯 조건을 모두지켜도 K^* 는 유한하다. 서비스가 요구하는 라운드 수를 그것이넘는지는 corpus 의 어느 논문도 답하지 않았다 - 100 회를 넘긴 실험이 0 편이기 때문이다. 그러므로 이 절의 결론은 “ Θ -경로는 반복을견딘다” 가 아니라 “**반복의 지평이 이 기회의 크기를 정하는데 그지평이 측정된 적이 없다**” 이다. 이 공백의 정식화는 ch30 이 소유한다.

29.3.6 양자화는 이 기회의 전제이고, 그 전제가 아직 측정되지않았다

ch26 이 확정한 것을 시스템 요구로 옮긴다. 저장 바이트당 정보 밀도는표현의 한계가 아니라 저장 dtype 이 정하므로, 저장 폭을 절반으로 내리면밀도가 정확히 두 배가 된다 [X2 F3] ($\rightarrow$ ch26 § 26.4 판정 3). 이 배수는이 장의 세 좌표 전부에 동시에 걸린다.

표 29-5 - dtype 이 세 좌표에 동시에 거는 배수 (8B 앵커, LoRA $r=16$, $\mathcal{S} = \{W_q, W_v\}$) [X2 Q1·Q3 위의 본서 산술]. X2 는 회계이지 측정이 아니므로 **각 칸의 절대치는 자릿수로만 읽고, 단정하는 것은 행 사이의 배수다.**

dtype	사용자당 delta	10 ⁶ 명 웹 총량	요청당 스테이징 바이트	라운드당 쓰기량
bf16	16.78 MB	16.777 TB	16.78 MB	기준
int8	8.39 MB	8.389 TB	8.39 MB	1/2
int4	4.19 MB	4.194 TB	4.19 MB	1/4

한 번의 dtype 결정이 총량·마감 여유·쓰기내구성을 같은 배수로 동시에 움직인다. (29-1)의 좌변은 C_{cap} 에 비례하므로 마감 여유가 네 배 넓어지고, (29-2)의 Φ_{wr} 은 네 배 줄어 소자 수명이 그만큼 늘어난다. 이 장이다루는 어떤 소자 선택보다도 큰 지렛대이며, **알고리즘 변경 없이 얻어진다.**

[평가] 그런데 corpus의 어느 논문도 delta를 양자화한 상태에서 지식 회수율을 측정하지 않았다 [X2 F3]. 그러므로 이 장이그리는 웹 계층 설계의 첫 번째 요구사항은 소자가 아니라 **측정**이다—rank·학습 집합·절차를 고정하고 저장 dtype만 내리며 회수율 곡선을 그리는 실험 하나. 그 실험은 새 하드웨어를 요구하지 않고 이미 있는 코드로 돌아간다. **이 분야가 웹 계층 소자를 논하기전에 지불하지 않은 것은 자본이 아니라 실험한 편이다.**

29.3.7 sleep 라운드 자체가 하나의 워크로드다 — 그리고 아무도그것을 스케줄링으로 다루지 않았다

지금까지 이 장은 wake 쪽 요구만 다뤘다. 그런데 sleep-time compute의 조작적 정의상 예산 B_s 가 실제로 소비되는 곳이 있고 (→ ch01), 그 소비는 서빙 fleet과 같은 하드웨어 위에서 일어난다. **sleep 라운드는 인프라 관점에서 별개의 워크로드 클래스이며, 이 라인의 어느 논문도 그것을 스케줄링 문제로 다루지 않았다.**

경로마다 그 워크로드의 모양이 다르다. E -경로에서 sleep은 **오프라인 LLM 추론 배치** 잡이다 — 문맥을 읽고 learned context를 생성하는 일이므로 prefill이 지배적이고, 산출물은 텍스트다 (→ ch14 §14.4). W -경로에서 sleep은 **forward pass**다 — 상태를 다시 접는 N -pass chunked prefill이고 Θ 는 움직이지 않는다 (→ ch17 §17.3.2). Θ -경로에서 sleep은 **훈련** 잡이다 — forward+backward와 optimizer 상태, 그리고 merge가 필요하고, merge는 전층 투영 텐서에 걸친 full read-modify-write다 (→ ch22 §22.7).

표 29-6 — 경로별 sleep 라운드의 워크로드 성격과 인프라 요구. 첫째 열은 앞 문단이 인용한 장 참조에서 오고, 지배자원·격리·선점 가능성 세 열은 그 사실에서 이책이 연역한 것이다. 어느 논문도 이 셋을 인쇄하지 않았다. [본서 추론]

경로	sleep에서 도는 것	지배 자원	요구되는 격리	선점 가능성
E	오프라인 추론 (prefill 지배)	계산 + 프롬프트 대역	서빙 fleet과 동일 스택	높다. 중단·재시작이 싸다
W	상태 재접기 forward	HBM 대역폭	같은 배치 안에서 커널 모양이들로 갈린다	낮다. 요청 경계에 묶인다
Θ	훈련 (fwd+bwd+optimizer+merge)	계산 + optimizer 상태 + 전모델 RMW	서빙과 다른 자원 프로파일	중간. 라운드 단위로 체크포인트된다

세 번째 열이 이 표의 요점이다. **세 경로의 sleep은 서로 다른 자원을 요구하므로 같은 스케줄러 정책으로 다룰 수 없다.** E -경로의 sleep은 서빙과 같은 커널을 쓰므로 유휴 슬롯에 끼워 넣기 쉽고, 그래서 이 경로가 오늘 당장 배포되는 이유의 절반이 된다. Θ -경로의 sleep은 optimizer 상태와 backward 활성값을 요구하므로 서빙 인스턴스의 메모리 여유로는 덮이지 않고, 별도의 훈련 자원을 잡거나 서빙 인스턴스를 잠시 비워야 한다. W -경로가 가장 곤란하다 — sleep이 요청 안에서 일어나므로 유휴 시간으로 밀 수 없고, ch17이 확인한 대로 한 요청이 자고 나머지가 decode하는 배치에는 커널 모양이 둘 살아 있다 (→ ch17 §17.7).

[해설] 독자의 어휘로 옮기면 이렇다. E -경로의 sleep은 **batch inference** 잡이고, Θ -경로의 sleep은 **fine-tuning** 잡이며, W -경로의 sleep은 **선점 불가능한 요청 내부 단계**다. 앞의 둘은 기존 스케줄러의 일급 객체이고 세 번째는 아니다. 그러므로 인프라 관

점에서 가장 새로운 요구를 만드는 것은 W -경로이고, 가장 큰 자원을 요구하는 것은 Θ -경로이며, 가장쉽게 배치되는 것은 E -경로다 — 셋의 순서가 판정 1 이 다른 저장쪽 순서와 다르다.

Θ -경로에는 시간 축의 요구가 하나 더 붙는다. ch22 가 옮긴 대로주기적 통합 정책은 라운드 수에 대해 초선형으로 누적된다 — m 라운드마다 통합하면 라운드 k 까지 누적 step 이 $O(k^2/m)$ 이다 ($\rightarrow$ ch22 § 22.7). 그러므로 **sleep 예산은 라운드마다 일정하지 않고 통합시점에 봉우리를 만든다.** 그 봉우리가 사용자 집단에서 겹치면 § 29.3.5 가말한 버스트와 같은 자리에서 계산 쪽 버스트가 함께 발생한다 [본서추론].

[평가] 그런데 B_s 를 요구 단위대로 인쇄한 논문이 corpus 스물한 편 중 한 편뿐이다 — ReasoningBank 의 과제당 3748.4 토큰이다 ($\rightarrow$ ch25 § 25.2.1). 나머지는 step 수나 라운드 수로만 적었고, 그 값들은 논문 자신의표현대로 “steps are matched, not tokens” 조건 아래 있다 [Continual Facts App. A.3, p.26]. **그러므로 이 표의 셋째 열은 형태로만채울 수 있고 크기로는 채울 수 없다.** sleep plane 의 용량 계획을 세우려는 엔지니어가 필요로 하는 것은 라운드당 FLOPs 인데, 그 수를 가진 논문이사실상 없다.

29.3.8 세 경로는 한 배포에서 세 계층에 동시에 요구를건다

마지막으로 셋을 합친다. 실서비스는 한 경로만 쓰지 않는다 — E -경로의검색 기억, W -경로의 긴 문맥 처리, Θ -경로의 개인화가 같은배포에 얹힐 수 있고, 판정 2 가 확정한 대로 그때 요구는 **상쇄되지않고 합쳐진다.**

표 29-7 — 세 경로가 한 배포에서 각 계층에 거는 요구. 세 경로를 한 배포에겹쳐 놓은 구성은 어느 논문에도 없으므로, 이 표는 앞 절들의 확정 사실을계층 축으로 재배열한 것이다. [본서 추론]

계층	E -경로	W -경로	Θ -경로 (개인화)
가속기 HBM	KV cache 증가 (검색 문서가 프롬프트에 실린다)	세션당 GB 단위 상태 상주 + 토큰당 RMW	base 가중치 + 적용된 delta
노드 DRAM	검색 index ($\rightarrow$ ch10 § 10.4)	상태오프로드 · 페이징 대상	delta 로컬 캐시
웹 계층	—	—	사용자당 수 MB, 요청 마감 안 스테이징 + 라운드 주기교체
콜드 계층	원문 저장소	—	replay 앵커용 원본 아카이브 (판정 4)
계산 plane	오프라인 추론 잡	요청 내부 forward	훈련 잡 + 통합봉우리

이 표가 이 장의 답을 한 화면에 담는다. 세 경로는 서로 다른계층에 서로 다른 성격의 요구를 걸고, 그 요구들은 겹치지 않으므로합산된다. E -경로는 KV 와 host DRAM 에, W -경로는 HBM 대역폭에, Θ -경로는 웹과 콜드에 앉는다. 그리고 다섯 행 중 오늘의 서빙스택에 자리가 아예 준비되어 있지 않은 것은 웹 행 하나뿐이다 — KV cache 도, 벡터 index 도, 체크포인트 저장소도 기존 스택의 일급 객체지만, “요청 지연 안에 끌어와야 하는 사용자당 수 MB 가중치 객체” 는 그렇지 않다.

[평가] 그러므로 이 장이 지목하는 새 자리는 정확히 한 칸이다 — 표 29-7 의 웹 행. 나머지 칸들은 새 소자를 요구하지 않고 기존 계층의사이징 변경을 요구한다. **이 구분을 흐리는 것이 이 분야의 기회서술에서 가장 흔한 과장이다** [본서 판단]. “sleep-time compute 이메모리 계층을 바꾼다” 는 문장은 다섯 행 중 한 행에 대해서만 참이고, 그 한행조차 Θ -경로가 개인화로 갈 때만 생긴다.

29.4 판정

[평가] 판정 1 — 기회는 실재한다. 그리고 그 자리는 HBM 대역폭이 아니라 요청 지연 안에 delta 를 스테이징하는 읽 계층과 그 계층의 쓰기 내구성이다.

근거는 세 좌표가 서로 다른 축에 있다는 사실이다. 읽기는 사용자당 수 MB 짜리 객체 하나에 대한 무작위 접근이고 요청 지연 안에 마감이 걸린다 (식 (29-1)). 쓰기는 sleep 라운드마다의 전량 교체이고 N_u 에선형 $\cdot T_k$ 에 반비례한다 (식 (29-2)). 총량은 사용자 10^6 명에 LoRA $r=16$ bf16 기준 16.777 TB 이며 [X2 Q3], 이 값은 데이터센터 규모에서 제약이 아니다. 이 판정이 총량에서 쓰는 것은 자릿수뿐이다 — X2 는 구성 파라미터에서 복원한 회계이지 측정치 아니고, 앵커 구성을 바꾸면 값이 바뀐다. 자릿수는 바뀌지 않는다. **그러므로 제약은 용량이 아니라 배치와 마감과 수명이다.** 대역폭이 아닌 이유는 산술로 정해진다 — 같은 요청에서 base 가중치가 16.0 GB 읽히는 옆에서 delta 는 그 0.105%이므로 대역폭 예산에서 보이지 않는다 [본서 산술]. 수명이 문제인 이유도 산술로 정해진다 — 식 (29-3) 에 따라 매체의 하루당 전체 재기록 횟수가 용량 $\cdot$ 사용자수 $\cdot$ dtype 과 무관하게 라운드 주기 하나로 정해지고, 그 주기는 알고리즘이 고르는 값이다. **조건을 명시한다: 이 판정은 Θ -경로가 사용자 수준 개인화로 갈 때만 성립한다.** 인구 수준 Θ -경로 — 전역 편집, CPT, 공유 memory layer — 는 가중치 1 벌을 유지하므로 이 요구를 만들지 않는다 (표 29-3 넷째 행). 기회를 만드는 것은 Θ -경로가 아니라 Θ -경로의 개인화다.

[평가] 판정 2 — 이 자리는 Neural Memory 가 W -경로에서 지목한 HBM read-modify-write 기회와 다른 자리다. 두 결론을 잊지 않는다. 조건 없다.

표 29-2 의 일곱 행이 전부 다르다. 병목 자원이 대역폭 대 접근마감 $\cdot$ 쓰기 수명으로 다르고, 시간 입도가 토큰당 대요청당 $\cdot$ 라운드당으로 다르며, 상태 수명이 세션 대 사용자로 다르다. 특히 두 요구는 공존할 때 상쇄되지 않고 합쳐진다. fast weight 를 쓰는 모델에 사용자별 delta 를 없으면 HBM 은 여전히 토큰당 whole-state RMW 로 포화하고 그 위에 요청당 읽 스테이징이 추가로 얹힌다. 두 기회를 “memory-centric” 이라는 한 단어로 묶는 서술은 이 합산을 감춘다. 이 판정에 조건이 없는 이유는 근거가 측정이 아니라 **상태의 수명과 시간 입도라는 정의적 사실**이기 때문이다 — 어떤 소자가 나와도 세션 상태가 사용자 상태가 되지는 않는다.

[평가] 판정 3 — shared-weight batching 은 깨진다/안 깨진다는 이분법이 아니라 네 단계이며, 완화 가능한 파손과 불가능한 파손을 같은 저울에 올려서는 안 된다.

표 29-3 이 확정한다. E -경로는 유지, W -경로는 세션 단위 부분 파손, 사용자 수준 Θ delta 는 delta 단위 파손, 직접 편집 개인화는 완전파손이다 [X2 Q4]. 셋째 행의 파손은 라우팅 문제이므로 같은 delta 를 쓰는 요청을 모으는 스케줄링으로 완화되지만 **완화의 상한이 사용자당 동시 요청 수에 묶인다** — 대화형 워크로드에서는 배치 폭이 1 에 가까워지고 남는 것은 grouped-GEMM 경로뿐이다 [본서 추론]. 다섯째 행의 파손은 용량 문제이므로 완화 수단이 구조적으로 없다. 그리고 넷째 행이 있으므로 “ Θ -경로는 batching 을 깬다” 는 문장은 그대로는 틀린다. 이 판정에 조건은 없다 — 표 29-3 의 다섯 행 모두 구조 분류이고 측정치 아니다.

[평가] 판정 4 — 반복이 이 기회의 시간 지평을 정하고, 반복은 보존된 원본 아카이브를 요구한다. 그러므로 Θ -경로는 E -경로를 대체하지 않는다.

반복이 없으면 (29-2) 의 Φ_{wr} 이 0 이고 쓰기 내구성 문제 자체가 사라진다. 기회를 만드는 것은 반복이다. 그리고 반복이 성립하려면 replay 가 보존된 원본에 앵커되어야 한다 — 이 부호는 독립적인 두 근거를 갖는다. 하나는 이 책의 실험으로, 같은 혼합비 $\cdot$ 같은 예산에서 원본 앵커가 자기 출력 앵커보다 **모든 ω 에서** 오래 버틴다 [X4 F6]. 다른 하나는 원 논문의 순차 증류 대비로, Θ_0 고정교사가 역량 +2pp $\cdot$ 유지율 54%인 반면 자기 누적 merge 교사는 -31pp $\cdot$ 유지율 21%다 [Continual Facts § 6.2, p.14 – 15]. **저장 계층으로 번역하면 읽 delta 계층 옆에 콜드 원본 아카이브가 상시 필요하다는 뜻이고,**

그 아카이브가 곧 E 다. 조건을 명시한다: $X4$ 는 LLM 실험이 아니므로 이 판정이 $X4$ 에서 받는 것은 부호와순서뿐이며 배수·라운드 수는 받지 않는다. 부호가 원 논문과 일치하지 않았다면 이 판정은 서지 않는다.

[평가] 판정 5 — 운영 지표는 라운드당 열화율이 아니라 crossover K^* 여야 하고, 감시는 임계 과제 지표가 아니라 연속신호에 걸어야 한다.

같은 반복 루프에서 임계 지표와 연속 신호의 열화 모양이 다르다 [$X4$ F4]. 임계 지표는 유예 구간 동안 움직이지 않다가 좁은 폭에서 무너지므로, 그것만 보는 감시는 저장이 이미 상당히 깎인 뒤에야 발화한다. held-out 손실 곡선을 보는 감시는 한 겹 더 늦다 — 1차 모멘트의 위험이포화하는 동안에도 생성 분포의 퍼짐은 계속 죽으므로, 평평해진 곡선이 안정을 뜻하지 않는다 [$X4$ F3]. 그리고 라운드당 비율은 라운드에 대해 상수가 아니므로 SLA로 걸 수 있는 양이 아니다. 걸 수 있는 양은 “몇 라운드까지 기준선을 유지하는가”이며, 그것이 재기준화 주기를 정하고 재기준화 주기가 워름 계층의 버스트 쓰기 사이클을 정한다. 조건: 이송가능한 것은 두 곡선의 모양 차이뿐이고 유예 구간의 길이나 열화 pp는 아니다. 이 판정은 이 책이 감시 설계에 거는 요구이지 Θ -경로의 성능에 대한 진술이 아니다.

[평가] 판정 6 — 이 기회를 사기 전에 필요한 것은 소자가 아니라 인쇄값 셋이며, 셋 다 corpus에 0 편이다.

첫째, delta 스테이징의 L_w — 표 29-4의 세 방식 어느 것에 대해서도 측정된 값이 없으므로 (29-1)의 좌변을 계산할 수 없다. 둘째, 양자화된 delta의 회수율 — 저장 폭이 세 좌표에 동시에 거는 배수 (표 29-5)가 이 분야에서 가장 큰 지렛대인데, 그 지렛대를 당겼을 때 품질이 어디서 무너지는지 켜 논문이 없다 [$X2$ F3]. 셋째, K^* — 반복 지평이 기회 크기를 정하는데 100회를 넘긴 실험이 0 편이고, 라운드당 열화율을 라운드 수의 합수로 인쇄한 논문도 0 편이다. 셋 중 어느 것도 새 하드웨어를 요구하지 않는다. 이 판정은 조건 없이 성립하며, 이 장의 나머지 판정 전부의 정밀도를 제한한다 — 위 다섯은 형태에 대한 판정이고, 값을 다루는 판정은 이 셋이 인쇄되기 전에는 어느 쪽으로도 쓸 수 없다.

29.5 반증 조건

기회 판정은 반증 가능해야 한다. 위 여섯 판정이 틀렸다면 무엇이 관측되는지를 판정 번호와 함께 아홉 항목으로 적는다. 각 항목은 관측의 형태로 적는다 — 의견이 아니라 인쇄되거나 측정되면 판정이 결정되는 것으로.

R-1 (판정 1을 반증한다). delta가 충분히 작고 재사용이 충분히 국소적이어서 별도의 워름 계층 없이 노드 로컬 캐시로 충분한 경우. 관측은 둘이 함께 필요하다 — (i) 활성 delta의 working set이 서버 노드의 DRAM 또는 로컬 저장에 상주하고, (ii) 같은 사용자의 요청이 같은 노드로 라우팅되는 비율 (sticky 적중률)이 높아서 delta가 요청마다 다시 끌려오지 않는다. 둘이 성립하면 (29-1)의 좌변에 원격 스테이징 항이 없어지고, 이장이 말한 “계층”은 별도의 계층이 아니라 노드 로컬 파일 캐시가 된다. 이것이 이 장의 판정을 무너뜨리는 가장 현실적인 관측이다 — 그리고 판정을 부분적으로만 무너뜨린다. sticky 라우팅이 깨지는 순간 (노드 장애, 재배포, 콜드 스타트) 원격 스테이징이 되살아나므로, 반증하려면 정상상태의 적중률이 아니라 꼬리 분위수의 적중률을 인쇄해야 한다.

R-2 (판정 1·4를 함께 반증한다). Θ -경로가 반복을 견디지 못해 개인화 배포에 도달하지 못하는 경우. 서비스가 요구하는 라운드 수보다 K^* 가 작으면 사용자 delta는 자산이 되기 전에 폐기되고, 워름 계층에 남는 것은 수명이 며칠인 캐시 엔트리뿐이다. 그러면 (29-2)의 정기 쓰기 대신 재구축 버스트가 지배하고, 애초에 개인화 delta를 유지할 이유가 사라진다 — 매번 다시 만들 것이면 E -경로가 같은 일을 더 싸게 한다. 필요한 관측은 하나다: 원본 보존 replay와 함께 $K > 100$ 에서 K^* 를 측정한 실험. 현재 corpus에 0 편이며, 이 반증항목의 정식화는 ch30이 소유한다.

R-3 (판정 1의 조건절 쪽에서 반증한다). 개인화가 사용자 수준이 아니라 집단 수준으로 수행하는 경우. 사용자 10^6 명이 아니라 집단 delta 수백 개로 품질이 유지된다면 표 29-3의 넷째 행이 지배적이 되고, 총량이 다섯 자릿수 줄며, shared-weight batching이 집단 단위로 되살아난다. 관측 형태: delta 사이의 중복 제거율·클러스터링결과와, 집단 delta로 대체했을 때의 품질 손실. 이 반증은 판정 1의 규모와 조건절을 동시에 깬다 — 웹 계층은 여전히 존재하지만 그 크기와 접근 패턴이 다른 문제가 된다.

R-4 (판정 6의 둘째 항과 표 29-5를 반증한다). rank·학습 집합·절차를 고정하고 저장 dtype 만 내렸을 때 int8 이전에 지식 회수율이 무너지는 측정. 그러면 양자화는 전제가 아니라 품질을 지불하고 바이트를 사는 거래가 되고, 표 29-5의 int4 행과 그 행이 주는 네 배의 마감 여유·수명 이득이 사라진다. 측정 형태는 ch26 § 26.5의 R-2와 같다 — 적재량을 스왑하고, dtype 세 수준에서 회수율 곡선을 그리며, 회수 판정은 출력이 아니라 가능도비로 한다.

R-5 (판정 1의 쓰기 쪽 절반을 반증한다). sleep 라운드의 쓰기가 전량 교체가 아니라 증분인 것이 확인되는 경우. 라운드 사이에 delta의 일부만 바뀐다면 (29-2)의 Φ_{wr} 은 그 변경 비율만큼 줄고, 쓰기 내구성 요구가 자릿수로 낮아진다. 관측: 연속한 두 라운드의 delta 사이 변경 바이트 비율. 이 값은 어느 논문도 인쇄한 적이 없고 측정비용이 거의 0이다 — 체크포인트 두 개를 diff하면 끝난다. 값이 작게 나오면 이 장의 쓰기 쪽 논증이 약해지고 읽기 쪽 논증만 남는다.

R-6 (판정 1의 읽기 쪽 절반을 반증한다). 요청 경로에서 delta를 생성하는 방식이 스테이징보다 싸다는 측정. 표 29-4의 (c)가 (b)를 지연에서 이기면 웹 계층 읽기가 계산으로 대체되고, 기회의 성격이 메모리에서 compute로 옮겨간다. 관측 형태는 같은 모델·같은 delta 크기에서 생성 지연 대 스테이징 지연의 직접 비교이며, 배치조건을 함께 인쇄해야 한다 — 생성은 배치 안에서 다른 요청과 GEMM을 공유할 수 있고 스테이징은 그럴 수 없으므로, 배치 크기가 이 비교의 결과를 뒤집을 수 있다 [본서 추론].

R-7 (판정 2를 반증한다). W-경로의 상태 트래픽과 Θ -경로의 delta 스테이징이 실제로 같은 자원에서 경합한다는 측정. 구체적으로, delta 스테이징이 HBM 대역폭을 유의하게 소비해 같은 배치의 decode RMW를 늦추는 것이 관측되면 표 29-2의 첫 두 행이 무너지고 두 자리가 하나로 합쳐진다. 그렇게 되면 두 모노그래프의 결론을 잇는 것이 정당해진다. 필요한 관측: 두 경로를 함께 돌리는 서빙 구성에서 delta 스테이징을 켜고 끄는 때와 켜고 끄는 때의 토큰당 지연 차이. 이 책이 억지연결을 금지한 것은 이 측정이 없기 때문이지 결과를 예단해서가 아니다.

R-8 (판정 3을 반증한다). 완화 불가능하다고 분류한 파손이 완화되거나, 완화 가능하다고 분류한 파손의 상한이 틀리는 측정. 앞쪽은 직접 편집의 갱신을 분리 가능한 아티팩트로 추출·주입·롤백하고, 서로 다른 편집 집합을 섞은 배치의 처리량이 전역 편집 모델 대비 떨어지지 않음을 보이는 측정이다 — 그러면 표 29-3의 다섯째 행이 셋째 행으로 합쳐진다 (관측 형태는 ch26 § 26.5의 R-4와 같다). 뒤쪽은 delta 단위 라우팅이 대화형 워크로드에서도 1보다 유의하게 큰 배치 폭을 유지한다는 처리량 측정이다 — 그러면 “완화의 상한이 사용자당 동시 요청 수”라는 이 장의 추론이 틀린다.

R-9 (판정 5를 반증한다). 임계 과제 지표가 연속 신호보다 먼저 또는 같은 라운드에 무너지거나, held-out 손실이 생성 분포의 퍼짐과 같은 라운드에 꺾이는 관측. 둘 중 하나라도 실제 LLM 반복 루프에서 나오면 “감시를 연속 신호에 걸라”는 처방의 근거가 사라지고, 라운드당 열화율을 SLA로 쓰지 못할 이유도 함께 약해진다. 필요한 인쇄는 하나다 — 라운드를 축으로 임계 과제 지표·연속 신호·생성 분포의 퍼짐 셋을 같은 그림에 올린 곡선. corpus에 0편이다.

반증 목록에 들어가지 않는 것도 명시한다. 첫째, 이 장이 인용한 절대 바이트·TB 값이 다른 구성에서 다르게 나오는 것은 반증이 아니다. 앵커 구성이 이 책의 선택이므로 다른 $r \cdot S \cdot d \cdot L_{\text{layer}}$ 에서 다른 값이 나오는 것은 당연하다. 반증이 되는 것은 선형이라는 모양, 세자릿수 격차, 계층 배치, 파손 분류가 어긋나는 경우다. 둘째, X4의 라운드 수·유지율·열화 pp가 실제 LLM에서 다르게 나오는 것도 반증이 아니다. 그 실험은 절대치를 주장하지 않았다. 반증이 되는 것은 부호와 순서가 뒤집히는 경우다 — 자기 출력 앵커 replay가 원본 앵커보다 오래 버티거나, 혼합비를 올렸는데 유지가 줄거나, 연속 신호가 임계 지표보다 늦게 무너지는 관측이 그것이다.

요약

- 이 장의 답은 한 문장이다. Θ -경로가 사용자 수준개인화로 가면 새 요구가 걸리는 자리는 HBM 대역폭이 아니라 요청 지연안에 delta 를 스테이징하는 워 메모리와 그 메모리의 쓰기 내구성이다 [X2 F4]. 총량은 사용자 10^6 명에 LoRA $r=16$ bf16 기준 16.777 TB 이고 (회계값이며 자릿수로 읽는다), 그 자릿수는 제약이 아니다 — 제약은 입도 (요청당 수 MB 무작위 접근) 와 마감 (식 (29-1)) 과 주기 (식 (29-2)) 다.
- 두 나눗셈이 그 성격을 확정한다. delta 는 같은 요청의 base 가중치 읽기대비 약 0.105%이므로 대역폭 문제가 아니고 [본서산술], 매체의 하루당 전체 재기록 횟수는 식 (29-3) 대로 용량 · 사용자 수 · dtype 과 무관하게 라운드 주기 하나로 정해진다. sleep 을 얼마나 자주 도느냐는 품질 다이얼이면서 동시에 소자 수명 사양이다.
- 이 자리는 Neural Memory 가 W -경로에서 지목한 HBM read-modify-write 기회와 다른 자리다. 병목 자원 · 시간입도 · 상태 수명 · 새 primitive 가 전부 다르며 (표 29-2), 두 경로가 공존하면 요구가 상쇄되지 않고 합쳐진다. 두 책의 결론을 잊지 않는다.
- shared-weight batching 은 이분법이 아니라 네 단계다 — E 유지, W 세션 단위 부분 파손, 사용자 수준 Θ delta 는 delta 단위파손 (라우팅으로 완화 가능하되 상한은 사용자당 동시 요청 수), 직접 편집개인화는 완전 파손 (완화 불가) [X2 Q4]. 인구 수준 Θ -경로는 batching 을 깨지 않으므로 이 장의 기회도 만들지 않는다.
- delta 를 요청 경로에 올리는 방식은 셋이고 저장 · 지연 · 배치를 세 방향으로 교환한다 (표 29-4). 세 방식 중 어느 것에 대해서도 L_w 를 측정 한 논문이 없다 — corpus 에서 L_w 칸이 0 편이라는 사실이 가장 아프게 드러나는 자리다.
- 기회를 만드는 것은 반복이며, 반복은 보존된 원본에 앵커된 replay 를 요구한다. 그러므로 워 메모리 delta 계층 옆에 콜드 원본 아카이브가 상시 필요하고, Θ -경로는 E -경로를 대체하지 않는다. 이부호는 이 책의 실험과 원 논문의 frozen-teacher 대비가 독립적으로 지지한다 [X4 F6; Continual Facts § 6.2].
- 운영 지표는 라운드당 열화율이 아니라 crossover K^* 여야 하고, 감시는 임계 과제 지표가 아니라 연속 신호에 걸어야 한다 — 두 곡선의 열화 모양이 다르기 때문이다 [X4 F4]. held-out 손실 곡선은 한 겹 더 늦다. 평평해진 곡선은 안정의 증거가 아니다 — 그 포화는 생성 분포가 죽어 갱신 스텝이 작아진 결과이고, held-out 손실만 보는 감시는 거기에 발화하지 않는다 [X4 F3]. 이송한 것은 모양과 순서뿐이고 라운드수 · 퍼센트 · pp 는 이송하지 않았다.
- 양자화는 세 좌표 (총량 · 마감 여유 · 쓰기 수명) 에 같은 배수로 동시에 걸리는 이 분야 최대의 지렛대이며 알고리즘변경 없이 얻어진다. 그런데 delta 를 양자화한 상태의 회수율을 쟀 논문이 0 편이다 [X2 F3].
- 기회를 사기 전에 필요한 것은 소자가 아니라 인쇄값 셋이다 — delta 스테이징의 L_w , 양자화된 delta 의 회수율, $K > 100$ 에서의 K^* . 셋 다 corpus 에 0 편이고 셋 다 새 하드웨어를 요구하지 않는다.

다음 장으로

이 장은 조건부 기회 지도를 그렸다. 조건은 셋이었다 — Θ -경로가 사용자 수준 개인화로 갈 것, 반복이 서비스가 요구하는 라운드 수를 견딜 것, 양자화가 품질을 깨지 않을 것. 세 조건 중 어느 것도 이 책이 참으로 확정하지 않았고, 각 조건이 어떤 관측으로 깨지는지만 § 29.5 가 적었다.

남은 일은 그 조건들을 이 책 전체의 미해결 목록과 함께 한 자리에 모으는 것이다. ch30 이 그것을 받는다. 이 장이 넘기는 것은 셋이다. 첫째, K^* 의 공백 — 반복 지평이 이 기회의 크기를 정하는데 100 회를 넘긴 실험이 corpus 에 0 편이라는 사실은 이 장의 판정을 넘어 Θ -경로 전체의 반증 조건이다. 둘째, 인쇄값 셋의 부재 (판정 6) — ch25 가 corpus 상태로 진단한 비용 침묵이 이 장에서 설계불가능성으로 나타났으므로, 그 침묵을 무엇이 해소하는지를 ch30 이 목록으로 확정한다. 셋째, 잊지 않은 두 결론 (판정 2) — Neural Memory 의 자리와 이 책의 자리를 하나로 묶지 않기로 한 판단은 근거가 아니라 측정의 부재에 기대고 있고, R-7 이 그 측정을 지정한다. ch30 은 이 셋을 이 책 이 스스로 틀릴 수 있는 방식의 목록에 넣는다.

ch30. 한계와 반증조건

30.1 판정할 질문

Part III의 여섯 장이 판정을 냈다. 대상을 확정했고 (ch24), 비용과 용량의자를 세웠으며 (ch25·ch26), 경로별로 답했고 (ch27), 예산 축이왜 그려지지 않는지를 썼고 (ch28), 기회의 좌표를 못 박았다 (ch29). 이 장은그 판정들에 같은 자를 댄다 — 이 책 자신에게.

판정할 질문. 이 책의 판정들은 무엇에 기대고 있으며, 어떻게 틀릴수 있는가.

답이 될 수 있는 것은 셋이다. 첫째, 판정이 딛는 **세 다리** — 이책의 실험 넷의 등급, corpus의 범위, 판별식의 선택 — 각각이 무엇을지탱하고 무엇을 지탱하지 못하는가. 둘째, Part II와 Part III의 주요판정이 각각 **어떤 관측으로 무너지는가**의 목록. 셋째, 이 책이 손에쥐고도 쓰지 않기로 한 것과 그 이유.

답이 될 수 없는 것도 셋이다. **이 장은 공백을 메우지 않는다.** GPU 실측이 없다는 사실은 이 장에서도 그대로이며, 한계를 정확히 기술하는 것과한계를 없애는 것은 다른 일이다. **판정이 실제로 옳은지도 이 장이정하지 않는다.** 반증 조건을 적는 것과 반증을 수행하는 것은 다른 일이고, 후자는 이 책 밖에서 일어난다. **반증 항목에 발생 확률을 매기지도않는다.** 어느 반증이 먼저 일어날지를 말하는 것은 예측이고, 조건 없는예측은 이 책의 문장이 아니다 (→ ch27 § 27.1).

규율을 하나 먼저 밝힌다. 이 장은 앞 장들이 § NN.5에 이미 적은 반증 조건을다시 논증하지 않는다. 각 장의 반증 조건은 그 장의 판정에 붙어 있고, 이장이 하는 일은 그것들을 **한 표로 모으고** 표 전체가 기대는**공통 전제**를 드러내는 것이다. 공통 전제가 무너지면 개별 반증조건을 하나도 통과하지 않고도 판정이 함께 무너지므로, 그 전제를 적지않은 반증 목록은 불완전하다.

30.2 증거

이 장의 증거는 다른 장과 종류가 다르다. 논문이 아니라 **이 책자신**이 증거다. 세 다리다 — 실험 넷의 등급, corpus의 범위, 판별식의형태.

첫째 다리 — 실험 넷. 통일 비용 회계 (실험 X1)는 상각식 (A)에 매질의 프롬프트 항을 되살려 임계 길이와 손익분기를 단힌 형태로 계산한다 (→ ch25). 상태 용량 회계 (실험 X2)는 corpus가 바이트로 보고하지 않은상태량을 구성 파라미터에서 복원한다 (→ ch26). 예산 축 감사 (실험 X3)는 29편의 deep-read를 규칙 기반으로 전수 감사해 B_s 가 어떻게보고되는지를 센다 (→ ch28). 반복 consolidation 열화 실험 (실험 X4)은가우시안 위치 모형과 선형 연상기억 위에서 (U- Θ)를 K 라운드돌린다 (→ ch29).

표 30-1 — 이 책의 실험 넷: 등급과 그 등급이 허용하는 것

실험	등급	본문 단정문으로 쓴 것	쓰지 않은 것
X1 통일 비용 회계	EXPLORATION-GRADE / ACCOUNTING (측정 아님)	레짐 간 여유의 순서 , 임계가 유한하다는 구조적사실	$L^* \cdot N_q^*$ 의 절대치, 레짐 상위 둘사이의 순서

실험	등급	본문 단정문으로 쓴 것	쓰지 않은 것
X2 상태 용량 회계	EXPLORATION-GRADE / ACCOUNTING (측정 아님)	경로 간 자릿수 와 계층 배치, dtype 이 정하는 밀도 배수 , 편집 대 delta 의 구조 분류	절대바이트 ·TB, 경로 간 정보 밀도 비 (철회)
X3 예산 축 감사	AUDIT-GRADE / CORPUS-CENSUS (측정 아님, 시뮬레이션 아님)	범주별 편수 와 경로 간 보고율의 순서 , 교집합 이 공집합이라는 사실	경로별 보고 율 , 개별논문의 라벨, 문자/4 근사로 얻은 토큰 수의 유효숫자
X4 반복 열화	EXPLORATION-GRADE / MECHANISM-TRANSFER (LLM 실험 아님)	함수형과 부호 , 조건 간 순서 , crossover 의 존재와 이동 방향	ρ 의 절대 pp, 유지율%, K^* 의 라운드 수, 필요 ω 값

넷의 공통점이 이 장의 첫 사실이다. **어느 것도 GPU 실측이 아니다**. X1 과 X2 는 닫힌 형태 회계이고, X3 은 문헌을 세는 감사이며, X4 는 numpy 위 의최소모형이다. 이 host 에 NVIDIA GPU 가 없으므로 wall-clock 은 한 줄도 없고, 따라서 이 책에는 지연 · 처리량 · 전력의**측정값**이 **하나도 없다**. 이 사실을 표 바로 아래에 적는 이유는 § 30.3.1 이 그 한계의 범위를 정확히 재기 때문이다.

둘째 사실은 재현성이다. 넷 다 결정론적이다 —X1·X2·X3 은 난수를 쓰지 않고, X4 는 고정 시드와단일 BLAS 스레드로 돌며 PYTHONHASHSEED=0 에서 두 번 실행해산출물의 sha256 이 같음을 확인했다. **재현성은 정확성이 아니다**. 같은 입력에서 같은 답이 나온다는 것은 구현이 안정하다는 뜻이지 그 답이실제 장치에서 참이라는 뜻이 아니며, 이 구분을 흐리는 것이 이 종류의실험이 저지르기 가장 쉬운 과장이다. [본서 판단]

셋째 사실은 **등급 계약을 이 책이 발명하지 않았**다는 것이다. 앞선모노그래프가 세운 계약을 글자 그대로 승계했다 —비율 ·crossover 위치 ·순서 ·bound 분류는 본문 단정문으로, 절대치는 하한 또는 방향성으로만 쓴다 (→ ch24 § 24.2). 계약이 먼저 있었고 실험이 나중에 왔다. 결론에 맞춰 사후에 조정된기준은 기준이 아니다.

둘째 다리 — corpus 의 범위. 이 책이 원문 대조로 읽은 것은 **29 편**이고 시점은 **2026-08** 이다. 문헌 전체의 무작위 표본이아니라 판별식을 세우기 위해 이 책이 고른 집합이며, 그래서 이 책의 계수진술은 전부 “이 corpus 의 29 편 중” 이라는 한정을 달고 있다 [X3 D9]. “이 분야의 몇 %” 라는 형태의 문장은 이 책에 하나도 없다 (→ ch24 판정 2).

셋째 다리 — 판별식. 네 조건 — (1) 질의 전, (2) $B_s > 0$, (3) 어느 한 층의 상태를 실제로 변경, (4) 이후 질의가 소비 — 은 논문들이합의한 정의가 아니라 **이 책이 그은 선**이다 (→ ch11 § 11.2). 계수진술 전부가 이 선 위 에 서 있으므로 선을 다르게 그으면 숫자가 달라지고, § 30.3.3 이 얼마나 달라지는지를 계산한다.

30.3 분석

30.3.1 GPU 실측이 없다는 것이 실제로 무엇을제한하는가

한계를 정직하게 다루는 방법은 크게 말하는 것이 아니라 **정확히**재는 것이다. 실측의 부재가 막는 것과 막지 않는 것을 갈라 적는다.

막는 것은 **값 주장**이다. 세 종류다. 첫째, 절대 지연과 처리량 — 이책은 delta 스테이징의 L_w 가 몇 ms 인지 말하지 않으며, 말할 수 없다는사실 자체를 판정으로 올렸다 (→ ch29 판정 6). 둘째, 실제 서빙 스택에서의병목 **순위** —가중치 ·활성 ·KV·delta 가 공존할 때무엇이 먼저 포화하는지는 회계로 닫히지 않는다. 셋째, 품질과 비용의 실제교환 곡선 — 이 책의 회계는 정확도를 동일하다고 놓고 비용만 세며 [X1 A4], 정확도 차이는 회계 안에 없다.

막지 않는 것은 **형태 주장**이다. 넷이다. 하나, **부재의진술**. “corpus 에 C_{cap} 을 바이트로 인쇄한 논문이 0 편” 은논문을 세면 판정되고 GPU 를 요구하지 않는다. 둘, **구조 분류**. 직접편집이 분

리 가능한 아티팩트를 만들지 않는다는 것은 알고리즘의 정의에서따라 나오지 측정에서 따라 나오지 않는다 (→ ch26 판정 5). 셋, **식의성질**. 한 번 내는 비용만 N_q 로 나뉘고 매 질의 내는 비용은 나뉘지않는다는 것은 회계 변종의 문제이지 실험의 문제가 아니다 (→ ch25 판정 2). 넷, **자릿수와 순서**. 세 경로의 상태량이 서로 다른 변수를 타드로자릿수가 벌어진다는 것은 지수의 문제다 (→ ch26 판정 2).

그러므로 실측의 부재가 무효로 만드는 것은 이 책이 애초에 쓰지 않기로 한종류의 문장이다. 다만 대가가 하나 있고 그것은 실재한다 — **이 책은자기 판정의 실무적 무게를 스스로 확정할 수 없다**. ch29 가 지목한 웹 계층기회는 자릿수 논증으로 서 있고, 그 자릿수가 실제 서빙에서 어느 정도의압력이 되는지는 측정 없이 말할 수 없다. ch24 § 24.5 의 F5 가 이 책의 도구로달하지 않는 항목인 것도 같은 이유다.

30.3.2 X2 초판의 오류 — 하한을상한으로

이 책의 실험 하나가 초판에서 틀렸다. 사건은 단순하다. X2 초판은파라미터당 3.64 bits 라는 값을 **상한**으로 놓고 경로간 정보 밀도 비를 계산했다. 원 논문은 그 값을 명시적으로**하한**이라고 쓴다 — “we are only ever measuring a lower bound on model capacity” [lm-memorization-capacity § 3.2]. 하한을 상한으로쓰면 Θ 층의 용량이 체계적으로 과소평가되므로, 그 비교는 방향 조차보장되지 않는다.

처리는 셋으로 갈렸다. **철회**한 것은 경로 간 밀도 비 자체이며, ch26 판정 4 가 그것을 “판정 불가” 로 남긴다. **살아남은** 것은 저장 dtype 이 정하는 밀도 배수다 — 저장 폭을 절반으로 내리면 밀도가 정확히 두배가 된다는 진술은 분모만으로 정해지므로 분자가 하한이든 상한이든성립한다 (→ ch26 판정 3). **새로 생긴** 것은 반증 실험의 설계다 — delta 에 알려진 엔트로피의 정보를 주입해 회수 가능한 bits/param 을 재면 판정 불가가 어느 쪽으로든 해소된다 (→ ch26 § 26.5 R-3).

이 사례를 본문에 실는 이유는 셋이다.

첫째, **이 책이 논문들에 요구한 것을 자신에게 적용한 기록이기때문이다**. 이 책은 corpus 의 여러 논문이 자기 표가 반증하는 문장을 캡션에그대로 두었다고 적었다 (→ ch18 § 18.6, ch21 § 21.6). 그 지적이 정당하려면같은 일이 이 책에서 일어났을 때 같은 처리를 해야 한다. 결론에 유리한값을 살려 두고 조용히 문장만 약화시키는 것은 그 논문들이 한 일과 같은일이다.

둘째, **오류의 종류가 이 분야에서 가장 흔한 종류이기 때문이다**. 틀린 것은 계산이 아니라 bound 의 방향이었다. 이 책이 인용하는 값 중여럿이 하한이거나 상한이며 — 용량은 하한이고, 환산 가능성 계수는상한이며, 일부 K^* 는 검열된 하한이다 [X3 D6, X4 D3] — 방향을잃은 채로 쓰면 부등호가 뒤집힌 결론이 나온다. 등급 표기가 장식이 아닌이유가 여기 있다.

셋째, **정정 이력이 없으면 반증 가능성도 없기 때문이다**. 어떤주장이 언제 무엇 때문에 철회되었는지가 남지 않으면, 살아남은 주장이검사를 통과해서 남은 것인지 검사를 받지 않아서 남은 것인지 독자가 구별할수 없다. 그래서 이 책의 실험 결과에는 철회 항목이 지워지지 않고 남아있고, 그 항목이 남아 있다는 사실이 나머지 항목의 등급을 뒷받침한다. [본서 판단]

자기 정정은 한 번에 그치지 않았다. X3 은 X1 의 입력 가정을 감사해 세어긋남을 찾았다 — 가정한 문맥 길이가 논문이 인쇄한 예시보다 크고, sweep 의 상단이 잘려 있었으며, 관측된 $|\hat{c}|/|c|$ 의 범위가 그리드 양 끝 밖에 있다 [X3 F6, F7, F8]. 앞의 둘은 방향이 X1 과 같아 ch25 의 순서 판정을**강화**했고, 셋째는 방향을 주장하지 않았다 (→ ch28 § 28.3.6). 이책의 실험이 이 책의 다른 실험을 감사했고 그 결과가 유리한 쪽으로만나오지 않았다는 사실이, 등급 규칙이 실제로 작동했다는 증거다.

30.3.3 판별식은 이 책의 선택이다 — 다르게 그으면 무엇이달라지는가

판별식은 발견이 아니라 결정이다. 네 조건 중 하나를 움직이면 A 군의 원소가바뀌고, A 군이 바뀌면 편수에 기대는 판정이 함께 움직인다. 네 변형을실제로 계산한다.

변형 (a) — 조건 (1) 의 “질의 전” 을 완화한다. 쓰기가 대화 중에일어나도 산출물이 이후 질의에 쓰이면 통과시킨다. 그러면 MemGPT 가 경계사례에서 E -경로 통과군의 정식 일원이 되고 (→ ch11 § 11.5), Generative Adapter 가 W -경로 2 번째로 들어온다 — 그 논문의 갱신은 (U-W) 이고산출물만 Θ 모양이며, 탈락 사유가 시계 하나였기 때문이다 (→ ch16 § 16.4). 결과가 무겁다. **ch27 판정 3 의 “표본 1” 이 “표본 2” 가된다**. [본서 추론] W -경로를 판정 불가로 남긴 근거의 절

반이 조건 (1) 의 엄격성에 직접 의존한다는 뜻이며, 이 의존은 ch16 을 경첩 장으로배치한 순간부터 예고되어 있었다.

변형 (b) — 조건 (3) 을 “읽기식 변경” 까지 확장한다. 상태를 바꾸지않아도 이후 질의의 읽기 경로를 오프라인에 준비하면 통과시킨다. 그러면 Memory Caching 이 들어오고, W 층 용량 천장의 차수를 공급하던 논문이배경에서 대상으로 자리를 옮긴다 (→ ch18 § 18.5). ch24 § 24.5 의 F1 이 이미 이후보를 지목해 두었다. 이 변형의 대가는 상각의 붕괴다 — 읽기식을 바꾼논문의 무거운 항은 N_q 가 커져도 줄지 않으므로, 식 (A) 가 정의되지않는 원소가 A 군 안으로 들어온다.

변형 (c) — 조건 (4) 를 뺀다. 이후 질의가 소비하지 않아도오프라인 계산이 상태를 바꾸면 통과시킨다. 범주가 “유휴 시간에 무언가를계산하는 모든 것” 으로 넓어진다. 네 조건 중 어느 것을 빼도 매번다른 종류의 논문이 잘못 들어오며, 그것이 네 조건이 모두일한다는 근거였다 (→ ch11 § 11.2.3, ch24 § 24.5 F2).

변형 (d) — 산출물의 모양으로 분류한다. (갱신식, 시계) 의 쌍 대신산출물이 무엇을 닮았는지로 경로를 배정하면 Generative Adapter 가 Θ -경로로 가고, 제목이 Θ 를 가리키는 SCM 은 E -경로에서 Θ -경로로 옮겨 간다 (→ ch11 § 11.4.1, § 11.4.5). Θ -경로 표본이 2 편에서 4 편으로 늘지만 [본서 산술], 그 넷은갱신식이 서로 달라 비용 표를 같은 자로 채울 수 없다. 모양 분류가 편수를 늘리고 비교 가능성을 줄인다.

여기서 이 장의 핵심 판정 하나가 따라 나온다. 네 변형 어디에서도이 책의 척추 논지는 무너지지 않는다. “이름이 기제보다 넓고, 남는경로들은 서로를 인용하지 않으며, 비용을 보고하지 않아 비교할 수 없다” 는진술은 판별식과 독립적으로 확인된다 — 경로 간 침묵은 참고문헌전수 확인에서 읽히고 (→ ch11 § 11.6), 비용 침묵은 각 논문의 § N.5 에서읽히며 (→ ch25 § 25.2.1), 이름과 기제의 어긋남은 논문의 제목과 본문을대조하면 판정된다. 판별식은 그 사실들을 정리하는 도구이지만 들어 내는 장치가 아니다.

무너지는 것은 편수에 기대는 판정들이다. 셋을 지목한다 — ch27 판정 3(W -경로 표본 1), ch28 판정 1 의 계수 (19 편 침묵 중 9 편은정의되지 않음), ch24 판정 1 의 통과 9 대 경계 1 대 불충족 19 분할. 이셋은 변형 (a) 와 (b) 에서 실제로 값이 바뀐다. 그러므로 이 책을 인용하는사람은 계수를 인용할 때 판별식을 함께 인용해야 한다. [본서 판단]

반대 방향의 변형 하나가 이 절의 결론을 만든다. 조건 (2) 를 “ B_s 가 절대 단위로 보고되어야 한다” 로 강화하면 어떻게 되는가. 절대 FLOPs 또는 토큰으로 B_s 를 인쇄한 논문은 29 편 중 1 편이고, 그 한 편이 A 군 안에 있다 [X3 F1]. 즉 A 군 9 편이 한 편으로 줄고, 이 책은출 대상을 잃는다. [본서 산술] 판별식이 보고 품질을 조건에 넣지 않은것은 이 책의 선택이며, 그 선택이 corpus 를 존재하게 했다. 동시에 그선택이 ch25·ch28 의 판정을 필연으로 만들었다 — 보고 품질을묻지 않고 들어 보낸 대상에 회계를 세우려 하면, 회계가 세워지지 않는다는것이 결과가 된다.

30.3.4 corpus 는 2026-08 의 것이다 — 어떤 논문이 나오면 무엇이바뀌는가

이 라인은 움직이는 중이다. 이 책의 판정은 문헌의 상태에 대한진술이므로, 상태가 바뀌면 판정도 바뀐다. 어떤 새 결과가 어느 판정을움직이는지를 표로 고정한다. 표의 목적은 예측이 아니라 **관측 의지정**이다 — 무엇을 보면 이 책의 어느 문장을 고쳐야 하는지가 미리 정해져있어야 한다.

표 30-2 — 새 결과의 유형과 그것이 움직이는 판정

새 결과의 유형	무엇이 바뀌는가	새 실험이 필요한가
세 편 이상이 같은 공개 벤치에서 절대 단위 B_s 3 점 이상과 품질을 함께인쇄	ch28 판정 1 이 무너지고 ch28 의 기여가 보고 요구에서 곡선으로바뀐다	예 (소규모 — R3 하나. 이미 돌린 설정에 두 점 추가)
실서빙 로그의 N_q 분포가 인쇄됨	ch25 판정 1 의 첫 층위가 해소되고, 분산이 작으면 ch28 판정 2 의 실무적 무게가 준다	아니오
사용자당 상태 바이트를 dtype 과 함께 인쇄한 논문 1 편	ch26 판정 6 이무너진다. 자릿수가 어긋나면 판정 1·2 도 함께	아니오

새 결과의 유형	무엇이 바뀌는가	새 실험이 필요한가
원본 앵커 replay 하에서 $K > 100$ 의 K^* 측정	ch27 판정 4 의 조건 (i) 이 결정되고, ch29 판정 4·판정 5 의 시간 지평이 값을 얻는다	예
delta 저장 dtype 만 내린 회수 을 곡선	ch26 판정 3 이 “전제” 에서 “거래” 로 바뀔 수 있고 ch29 의 int4 열이 지워진다	예 (소규모)
prefix cache 를 켜 full-context 와의 총비용 대조 판별식을 정면 통과하는 W -경 로 논문 1 편 추가	ch27 판정 2 가 양방향으로 결정된다 ch27 판정 3 의 “판정불가” 가 근거를 잃는다	아니오 새 논문
세 경로를 합성한 시스템이 각 단독을 이기는 표	ch27 판정 1·6 의 경로 분해 자 체가 잘못된 분해가 된다	예
delta 스테이징이 같은 배치의 decode 를 늦춘다는 측정	ch29 판정 2 가 무너지고 두 모 노그래프의 결론을 잇는 것이 정당해진다	예

표에서 읽히는 사실이 이 절의 판정이다. **아홉 유형 중 셋이 새 실험을 요구하지 않는다** — 둘은 이미 실행된 로그의 인쇄이고 하나는 스위치를 켜 재실행이다. [본서 산술] 넷째 유형 (절대 단위 B , 3 점) 은 새 실험을 요구하되 그 실행이 “이미 돌린 설정에 두 점을 더한다” 에그친다 (→ ch28 § 28.3.7). 표 밖에도 같은 종류가 더 있다 — 체크포인트하나의 파일 크기를 재는 것, 연속한 두 라운드의 delta 를 diff 하는 것이 그렇다 (→ ch26 § 26.5 R-7, ch29 § 29.5 R-5). 그러므로 **이 책이 가장 빨리 낚는 방식은 반박이 아니라 인쇄다**. 누군가 이 책을 틀리게 만들려면 새 방법을 발명할 필요가 없고, 이미 가진 값을 적기만 하면 된다.

반대로 잘 낚지 않는 것도 있다. 세 종류다. **역사적 사실** — 어느 논문이 무엇을 인쇄했고 무엇을 인용하지 않았는가는 후속 연구로 바뀌지 않는다. **식의 성질** — 상각식의 구조적 결함은 새 측정으로 사라지지 않고 설계를 바꿔야 사라진다 (→ ch25 § 25.5 항목 7). **구조 분류** — 상태의 수명과 시간 입도는 정의적 사실이므로 어떤 소자가 나와도 세션상태가 사용자 상태가 되지 않는다 (→ ch29 판정 2).

30.3.5 이 책이 지지 않은 집

한계 목록은 하지 못한 것만이 아니라 **하지 않기로 한 것도** 포함해야 한다. 세 가지다.

잊지 않은 것. 앞선 모노그래프가 W -경로에서 지목한 HBM read-modify-write 기회와 이 책이 Θ -경로에서 지목한 워 계층기회를 하나로 묶지 않았다. 근거는 두 자리의 병목 자원 · 시간입도 · 상태 수명이 다르다는 구조적 사실이고, 그것을 하나로 묶을 수 있는지는 **측정이 정한다** (→ ch29 판정 2, § 29.5 R-7). 잊지 않은 것은 결과를 예단해서가 아니라 그 측정이 없기 때문이며, 두 책의 결론을 한 단어로 묶는 서술은 두 요구가 공존할 때 합쳐진다는 사실을 감춘다.

판정하지 않은 것. 판정 불가로 남은 칸이 셋 있다 — 경로 간 정보밀도 (ch26 판정 4), W -경로 전반 (ch27 판정 3), “최선인가 · 대안 대비 우월한가” (ch27 판정 5). 셋 다 **무엇이 관측되면 판정 가능해지는지**를 함께 적었고, 그것이 붙는 순간 판정 불가는 회피가 아니라 판정이 된다. 대조군이 실행되지 않은 비교에 우열을 선언하는 것이 회피이고, 대조군이 없다는 사실을 관측 형태와 함께 명시하는 것이 판정이다.

쓰지 않은 것. 계산해 두고 본문에 올리지 않은 값이 넷 있다 — B , 보고율의 경로별 비율, 워 스토리지의 절대 바이트, 상각 임계의 절대 토큰 수, 반복 라운드의 열화 pp 와 라운드 수 (→ ch24 § 24.2). 넷 다 인용하면 문장이 강해진다. 쓰지 않은 이유는 하나다 — 비용 칸이 비어 있는 논문에 “효율적” 이라고 쓰지 않는다는 규칙은 이 책의 실험에도 똑같이 적용된다.

30.4 판정

[평가] 판정 1 — 이 책의 판정은 강도가 다른 네 종류의 근거위에서 있다. 그 순서를 명시한다.

가장 강한 것은 **부재의 진술**이다 — 어느 논문도 인쇄하지 않았다는 사실은 세면 판정되고, 반증에 필요한 것은 인쇄 한 줄이다. 다음이 **구조 분류와 식의 성질**이다 — 알고리즘의 정의와 회계 변종에서 따라 나오므로 측정에 의존하지 않는다. 다음이 **회계의 자릿수와 순서**다 — 구성 파라미터에서 복원한 값이므로 절대치는 지지하지 못하고 지수와 순서만 지지한다. 가장 약한 것은 **최소모형의 부호와 모양**이다 — LLM 실험이 아니므로 이송 가능한 것이 부호 · 함수형 · 조건 간 순서뿐이다. 이 책의 어떤 문장도 자기 근거보다 강하게 쓰이지 않았다고 주장하며, 그 주장 자체가 § 30.5의 검사 대상이다.

[평가] 판정 2 — GPU 실측의 부재는 이 책의 값 주장을 전부 무효로 만들지만, 형태 주장은 무효로 만들지 않는다. 조건을 명시한다.

형태 주장이란 부재 · 구조 분류 · 식의 성질 · 자릿수 · 순서 · 부호 · crossover 의존재다. 값 주장이란 지연 · 처리량 · 전력 · 정확도 차이 · 절대 바이트다. 이 책은 후자를 쓰지 않았고, 쓰지 않았다는 사실을 등급 표기로 매 장에 남겼다. **조건**: 이 판정은 형태 주장이 실측에서 뒤집히지 않는다는 뜻이 **아니다**. 순서 · 자릿수 · 부호는 측정으로 반증될 수 있고 § 30.5의 표가 그 관측을 지정한다. 이 판정이 말하는 것은 실측의 부재가 형태 주장을 미리 무효화하지 않는다는 것뿐이다.

[평가] 판정 3 — X2 초판의 오류를 본문에 실는 것은 겸손이 아니라 등급 체계의 필요조건이다. 조건 없이 단정한다.

등급은 어떤 문장이 어떤 검사를 통과했는지를 표시하는 장치다. 검사에서 떨어진 문장이 어디로 갔는지가 기록되지 않으면, 남은 문장이 검사를 통과해서 남은 것인지 검사를 받지 않아서 남은 것인지 구별되지 않고, 그러면 등급 표기 전체가 장식이 된다. 그리고 이 책이 corpus의 여러 논문에 대해 지적인 결함 — 자기 표가 반증하는 문장을 캡션에 그대로 두는 것 — 은 정확히 정정 이력을 남기지 않는 데서 생긴다. **같은 자를 자신에게 대지 않는 책은 그 지적을 할 자격이 없다**.

[평가] 판정 4 — 판별식은 이 책의 선택이며, 그 선택이 척추논지를 만들지 않았다. 무너지는 것은 편수에 기대는 판정들이다.

네 변형 — 조건 (1) 완화, 조건 (3) 확장, 조건 (4) 제거, 산출물 모양 분류 — 어디에서도 “이름이 기제보다 넓고 경로들이 서로를 인용하지 않으며 비용이 비교되지 않는다”는 논지는 살아남는다. 그 세 사실은 참고문헌 전수 확인 · 비용 칸 전수 집계 · 제목과 본문의 대조에서 각각 독립적으로 읽히기 때문이다. **조건**: 편수에 기대는 세 판정은 살아남지 않는다 — W-경로 표본이 1이라는 판정은 조건 (1)의 엄격성에 직접 의존하고, A 군 9편이라는 계수와 침묵 19편의 내부 분할도 함께 움직인다. 이 책의 계수를 인용할 때는 판별식을 함께 인용해야 한다.

[평가] 판정 5 — 이 책은 2026-08의 문헌 상태에 대한 진술이고, 가장 빨리 낡는 방식은 반박이 아니라 인쇄다. 그리고 이 책은 그 인쇄가 일어나기를 요구하는 쪽에서 있다.

표 30-2의 아홉 유형 중 셋이 새 실험을 요구하지 않고, 넷째는 이미 돌린 설정에 두 점을 더하는 것으로 끝난다. N_q 분포 한 줄, 사용자당 바이트 한 줄, prefix cache 스위치를 켜 재실행, 그리고 표 밖의 체크포인트 두 개 diff — 이런 것들이 이 책의 판정 여럿을 즉시 결정한다. **문헌의 상태를 바꾸는 비용이 낮다는 사실 자체가 이 책의 진단이 옳다는 증거이자 이 책이 오래 유효하지 않기를 바라는 이유다**. 이 판정에 조건은 없다. 다만 방향은 있다 — 이 책의 판정 중 인쇄로 결정되는 것이 실험으로만 결정되는 것만큼 많다면, 이 분야의 병목은 아직 연구가 아니라 보고다.

30.5 반증 조건

이 장의 다섯 판정부터 반증한다. 그 다음 이 책 전체의 판정을 표로 모은다.

M1 (판정 1 을 반증한다). 이 책의 어떤 문장이 자기 근거보다강하게 쓰였음이 확인되는 경우. 검사 절차가 기계적이다 — 그 문장의 근거를표 30-1 의 실험 또는 Part II 의 원문 위치까지 따라가서, 실험이 “쓰지 않은것” 옆에 넣은 양이 본문에서 단정문으로 쓰였는지 보면 된다. 한 건이라도나 오면 그 문장을 고쳐야 하고, 여러 건이 나오면 판정 1 이 틀렸다.

M2 (판정 2 를 반증한다). 실측에서 이 책의 형태 주장이 뒤집히는경우. 구체적으로 세 경로 상 태량의 자릿수 순서가 실제 배포에서 다르게나오거나, Θ -경로 반복에서 자기 출력 앵커 replay 가 원본 앵커보다오래 버티거나, 임계 과제 지표가 연속 신호보다 먼저 무너지는 관측이다. 셋중 하나 라도 나오면 “형태는 안전하다” 는 이 장의 구분이 실제로는 안전하지않았다는 뜻이다.

M3 (판정 3 을 반증한다). 이 책에 정정되지 않은 다른 오류가 남아있는 경우. 정정 이력을 남기 는 것이 방법의 일부라는 주장은, 남기지 않은오류가 발견되는 순간 약해진다. 가장 가까운 후보를 이 책이 스스로지목한다 — LoRA 체크포인트의 실제 파일 크기를 이 책은 확인하지 않았고, 그 확 인은 파일 하나를 재면 끝난다 (→ ch26 § 26.5 R-7).

M4 (판정 4 를 반증한다). 판별식의 어떤 변형에서 척추 논지가실제로 무너지는 경우. § 30.3.3 은 네 변형만 계산했다. 다섯째 변형에서 경로간 침묵이 사라지거나 비용 비교가 성립하면 판정 4 가 틀렸고, 그 경우 이책의 논지는 판별식의 산물이었다는 뜻이 된다.

M5 (판정 5 를 반증한다). 표 30-2 의 “인쇄만으로 결정되는” 항목들이 실제로는 인쇄되지 못하 는 경우 — 예를 들어 N_q 분포가 영업비밀이거나 서버 로그에 그 형태로 남지 않는다면, 이 책이 “낮 은비용” 이라고 부른 것이 실제로는 낮지 않다. 그러면 판정 5 의 방향 (“병목은연구가 아니라 보고 다”) 이 틀리고, 침묵의 원인이 관행이 아니라 구조라는다른 설명이 필요해진다.

이제 이 책 전체의 판정을 모은다. 각 행은 판정 하나와 그것을 무너뜨리는관측 하나다. 여러 관 측이 가능한 판정도 **가장 결정적인 하나**만적었다 — 나머지는 해당 장의 § NN.5 에 있다.

표 30-3 — Part II·III 주요 판정과 그 반증 관측

판정	어디	반증 관측
이름이 기제보다 넓다 — A 군 9 편, 경계 사례 1 편, B 군 9 편	ch11, ch24 판정 1	네 조건을 모두 만족하는데 B 군에 넣은 절차가 원문 대조로 확인됨
표현 수준 E 는 계산을 압축하 고 저장을 압축하지 않는다	ch12 § 12.5	검색을 줄이면서 전체 KV 의 실제 저장도 함께 줄어드는 구 성이 측정됨
MemGPT 의 쓰기는 대화 중이 며 조건 (1) 을 만족하지 않는다	ch13, ch11 § 11.5	원문에서 질의 전 배치 쓰기 경 로가 확인됨
Letta STC 의 비용모델에 prefill 항이 없고 $5\times$ 는 생성 토큰 축의 값이다	ch14 § 14.5	그림 축이 프롬프트 토큰을 포 함한다는 것이 공개 아티팩트 의필드 정의로 확인됨
그 이득은 높은 test-time 예산 에서 역전하고 기제가 없다	ch14 § 14.6	시드 · 신뢰구간과 함께 재측정 해 역전이 잡음임이 보임
E -경로에서 품질 1 위가 반복 적으로 기억 시스템 없는 구성 이다	ch15 § 15.6, ch27 판정 2	같은 예산 · 같은 서버 최적화에 서 E -경로가 full-context 를 이기는표 (시드 · 신뢰구간 · 길 이 스왑 포함)
Generative Adapter 는 산출 물이 Θ 모양이나 시계가 wake 다	ch16 § 16.4	같은 절차를 질의 전에 돌려 산 출물을 이후 질의에 재사용하 는구성이 보고됨
W -경로의 판별식 통과는 한 설 정에서만 성립한다	ch17 § 17.4	헤드라인 설정에서 consolidation 이 질의에 선행 함이 원문 또는 코드로확인됨

판정	어디	반증 관측
계산량을 맞춘 baseline 이 없다 — 캐시를 유지한 full attention 미실행	ch17 § 17.6	그 baseline 이 같은 예산에서 실행된 표가 나옴
갱신 주파수 연속체는 표기의 다리이지 기제의 다리가 아니다	ch18 § 18.4	연속체의 저주파 끝점에서 Θ 가 실제로 오프라인에 움직이는 구현과수치가 제시됨
개인화하려면 Θ -경로는 delta 형태여야 한다	ch19 § 19.7, ch26 판정 5	직접 편집을 추출·주입·롤백하고 혼합배치 처리량이 유지됨을 보인 측정
Θ -경로의 쓰기 검증은 라벨 또는 외부 reward model 에 의존한다	ch20 § 20.7, ch21 § 21.3	라벨 없이 쓰기 품질을 판정하는 절차와 그정확도가 보고됨
망각은 소거가 아니라 접근 상 실이다	ch22 § 22.6	쓰기가 만든 연속신호가 실제로 바닥에 닿는 (소거되는) 조건이 관측됨
Θ -경로가 기대는 두 축이 실증으로 서 있지 않다	ch22 § 22.8	random-eviction 대조군을 유의하게 이기는 압축법이 보고됨
이름은 생물학에서 왔고 논증은 오지 않았다	ch23 § 23.8	세 경로 논문이 CLS·생물 sleep 을 근거로 설계를 정당화하고 그 예측을시험함
이름과 기제의 간격은 구조적이다	ch24 판정 2	E -경로 논문이 $W \cdot \Theta$ 방법을 같은 표 안에서 baseline 으로 실행함
세 경로는 지금 같은 자로 썰 수 없다	ch25 판정 1	세 편 이상이 B_s 를 FLOPs 로, C_{cap} 을 dtype 과 함께 인쇄
상각 논증에 구조적 결함이 있다	ch25 판정 2	$\hat{c}$ 가 완전히 prefix-cache 되어 매 질의 향이 사라짐이 캐시 적 중률과 함께 벽시계로 확인됨
상각 여유의 레짐 간 순서 — SWE-Features 최하위	ch25 판정 4	각 레짐의평균 $ c \cdot \hat{c} \cdot \text{sleep}$ 생성 토큰이 인쇄되고 SWE 가 임계 아래에 넉넉히 머물름
경로 선택은 곧 메모리 계층 선택이다	ch26 판정 2	같은 단위로 인쇄된보고에서 자릿수 순서가
delta 양자화는 최적화가 아니라 전제다	ch26 판정 3	$E < \Theta_{\text{delta}} < W$ 가 아님
경로 간 정보 밀도는 판정 불가다	ch26 판정 4	rank·학습 집합을 고정하고 dtype 만 내렸을 때 int8 이전 예회수율이 무너지짐
E -경로는 비용 방법으로 유망하고 품질 방법으로는 지지되지 않는다	ch27 판정 2	delta 에 알려진엔트로피를 주입해 회수 bits/param 을 잴 (어느 쪽이든 해소)
W -경로는 표본 1 로 판정 불가다	ch27 판정 3	prefix cache 를 켜
Θ -경로는 조건부로 유망하다	ch27 판정 4	full-context 와 같은 스택에서의 총비용대조 (양방향)
“최선인가·대안 대비 우월한가” 는 판정 불가다	ch27 판정 5	판별식을 정면 통과하는 W -경로 논문이 한 편 더 나옴
		원본 앵커 replay 하에서
		$K > 100$ 의 K^* 가 측정됨
		표 27-4 의 빈칸 하나가 채워짐

판정	어디	반증 관측
현재 문헌으로 B_s -품질 곡선은 그릴 수 없다	ch28 판정 1	세 편이상이 같은 공개 벤치에서 절대 단위 B_s 3 점과 품질을 함께 인쇄
B_s 는 (N, D) 와 같은 종류의 축이 아니다	ch28 판정 2	실서빙 N_q 분포의 분산이 작아 B_s/N_q 가 B_s 의 상수배가 됨
기회는 워 계층과 쓰기 내구성이다	ch29 판정 1	활성 delta 의 working set 이 노드 로컬에 상주하고 꼬리 분위수 sticky 적중률이 높음
이 자리는 W -경로의 HBM RMW 자리와 다르다	ch29 판정 2	delta 스테이징이 같은 배치의 decode 를 유의하게 늦추는 것이 측정됨
운영 지표는 ρ 가 아니라 K^* 다	ch29 판정 5	실제 LLM 루프에서 임계 지표가 연속 신호보다 먼저 또는 같은 라운드에 무너짐
형태 주장은 실측 부재로 무효화되지 않는다	ch30 판정 2	실측에서순서·자릿수·부호 중 하나가 뒤집힘

반증되지 않는 것. 세 종류를 명시한다. 첫째, **개별 논문의성능 수치가 오르내리는 것은 이 책의 판정 대부분을 무너뜨리지 않는다.** 판별식은 성능을 묻지 않고 ($\rightarrow$ ch11 § 11.2.1), ch27 의 판정은 수치의 크기가 아니라 대조의 구성에 기대며, 새 수치가 아니라 새 표만이 그것을무너뜨린다. 둘째, **이 책이 인용한 절대치가 다른 구성에서 다르게나오는 것도 반증이 아니다.** 앵커 구성이 이 책의 선택이므로 다른 $r \cdot d \cdot L_{\text{layer}}$ 에서 다른 값이나오는 것은 당연하고, 반증이 되는 것은 자릿수·성장지수·계층 배치·구조 분류가 어긋나는 경우다. 셋째, **최소모형의 라운드 수·유지율·열화 pp 가 실제 LLM 에서 다른 것도 반증이 아니다** — 그 실험은 절대치를 주장하지않았다. 반증이 되는 것은 부호와 순서가 뒤집히는 경우다.

마지막으로 이 표 자체의 실패 조건을 적는다. **어떤 행의 반증관측이 원리적으로 관측 불가능하다면 그 행의 판정은 철회되어야 한다.** 표 30-3 의 서른두 행은 그 검사를 통과하도록 썼다 — 각 행의 관측은 원문 대조, 값의 인쇄, 스위치를 켜 재실행, 소규모 실험, 실제 서빙 스택의 측정 중하나이며 어느 것도 원리적 불가능에 속하지 않는다. 다만 마지막 종류는 이책의 도구로 닫히지 않는다. 회계는 자릿수를 주고 측정은 주지 않기때문이며 ($\rightarrow$ ch24 § 24.5 F5), 그 공백이 해당 판정들의 강도에 상한을 건다.

요약

- 이 책의 판정은 세 다리 위에 선다 — 실험 넷의 등급, 29 편·2026-08 이라는 corpus 의 범위, 네 조건 판별식이라는선택. 셋 다 이 책이 정한 것이고 셋 다 다르게 정할 수 있었다.
- 실험 넷 중 GPU 실측은 하나도 없다. 둘은 닫힌 형태 회계, 하나는 문헌감사, 하나는 numpy 최소모형이며 wall-clock 이 한 줄도 없다. 넷 다결정론적으로 재현되지만, 재현성은 정확성이 아니다.
- 실측의 부재는 값 주장을 전부 막고 형태 주장은 막지 않는다. 부재·구조 분류·식의성질·자릿수·순서·부호는 GPU 를요구하지 않는다. 대가는 하나 — 이 책은 자기 판정의 실무적 무게를스스로 확정할 수 없다.
- X2 초판은 파라미터당 3.64 bits 를 상한으로 놓고 경로 간밀도를 비교했다. 원 논문은 그것을 하한이라고 명시하므로 비교를철회했고, 분모만으로 정해지는 dtype 배수는 살아남았다. 정정 이력이없으면 살아남은 주장의 등급도 알 수 없다.
- 판별식을 네 방향으로 변형해도 척추 논지는 살아남는다 — 경로 간침묵·비용 침묵·이름과 기제의 어긋남은판별식과 독립적으로 확인되기 때문이다. 무너지는 것은 편수에 기대는 세판정이며, 특히 W -경로 “표본 1” 은 조건 (1) 의 엄격성에 직접의존한다.

- 조건 (2) 에 보고 품질을 넣으면 A 군 9 편이 한 편이 된다. 판별식이 보고품질을 묻지 않은 것이 corpus 를 존재하게 했고, 동시에 회계가 세워지지않는다는 결과를 필연으로 만들었다.
- 이 책이 가장 빨리 낡는 방식은 반박이 아니라 인쇄다. 표 30-2 의 아홉유형 중 셋이 새 실험을 요구하지 않으며, N_q 분포 한 줄과 사용자당바이트 한 줄이 여러 판정을 즉시 결정한다.
- Part II·III 의 주요 판정 서른두 개에 각각 반증 관측이 붙는다. 다수는 원문 대조 · 값의 인쇄 · 스위치재실행으로 결정되고, 실제 서빙 스택의 측정을 요구하는 행만이 이 책의 도구로 닫히지 않는다 — 그만큼 해당 판정의 강도에 상한이 걸린다.

다음 장으로

다음 장은 없다. 이 책은 여기서 끝나고, 남는 것은 표 30-3 이다.

책의 첫 질문은 “sleep-time compute 은 유망한가” 였다. 서른 장을 지나 그 질문은 다섯 개의 다른 질문으로 갈라졌고, 각각의 답에는 경로 이름과조건이 붙었으며, 세 칸은 판정 불가로 남았다. 그 과정에서 이 책이 실제로만든 것은 답이 아니라 **답할 수 있는 형태**다 — 판별식 하나, 자 두개, 감사 하나, 최소모형 하나, 그리고 그 다섯이 무엇을 지탱하고 무엇을지탱하지 못하는지를 적은 등급표.

그러므로 이 책을 쓰는 가장 좋은 방법은 인용이 아니라 반증이다. 표 30-3 의 어느 행이든 관측 하나로 무너지게 만들어 두었고, 무너지면 그 행을 고치면된다. 여러 행은 이미 존재하는 로그를 인쇄하는 것만으로 결정되므로, 이책의 수명은 이 분야가 자기 비용을 적기 시작하는 날까지다. **그날 이빨리 오는 것이 이 책이 바라는 결말이다.**

참고문헌

수록 원칙

이 목록에 실리는 문헌은 두 종류이고, 두 종류를 구획으로 나눈다. § 90.1 – § 90.3 은 **corpus 29** 편이다. 원문을 확보해 전문을 대조한 문헌이며, 본문의 사실 서술과 수치 인용은 전부 여기서만 나온다. § 90.5 는 **본문이 이름으로만 지목하는 문헌**이다. 계보를 설명하거나 어떤 논문이 무엇을 인용하지 않았는지를 지적할 때 이름이 필요해 등장할뿐, 이 책은 이들을 근거로 쓰지 않는다. 두 구획을 섞지 않는 것이 이 책의 정직성 규칙이다 — 대조하지 않은 문헌에서 사실을 옮기지 않는다.

각 항목은 다음 순서로 적는다.

저자 · 제목 · **arXiv ID** 또는 **venue** · 연도 · 이 책의 담당 장

규약 넷을 못 박는다.

- **저자가 다섯 명 이상이면 제 1 저자 (또는 공동 제 1 저자) 만 적고 “외” 를 붙인다.** 전원 명단은 원문 표제부에 있다.
- **venue 는 원문에 인쇄된 것만 적는다.** 원문에 venue 표기가 없으면 “venue 미기재” 로 적는다. 다른 경로로 알려진 게재 사실이 있더라도 그 원문에 인쇄되어 있지 않으면 여기에 적지 않는다 — EWC 가 대표사례다 (§ 90.3).
- **이 책이 읽은 판 (version) 을 병기한다.** arXiv 문헌은 판마다 표 · 수치가 달라지므로, 본문의 인용은 전부 여기 적힌 판을 가리킨다.
- **담당 장은 그 문헌을 정면으로 다루는 장이다.** 괄호 안은 그 문헌을 근거로 쓰는 다른 장이다. 전수 분류의 근거와 판정은 부록 A 와 ch11 에 있다.

각 장이 본문에서 쓰는 인용 표지 ([SEAL §4.2] 형식) 는 그 장이 첫 등장 시 고정한다. 표지가 장마다 다를 수 있으므로, 이 목록은 표지가 아니라 ch11 의 전수 분류표가 쓰는 짧은 이름으로 배열한다.

90.1 corpus — A 군: 판별식 통과 (9 편) + 경계 사례 (1 편)

판별식 네 조건을 모두 만족하는 문헌 9 편과, 조건 (1) 만 만족하지 않아 통과군에 세지 않는 경계 사례 1 편 (MemGPT) 이다. 조건별 판정은 부록 A 표 A-1 에 있다.

Letta Sleep-time Compute — Kevin Lin, Charlie Snell 외 (Letta; UC Berkeley). Sleep-time Compute: Beyond Inference Scaling at Test-time. **arXiv:2504.13171**, 2025-04-17. venue 미기재. → **ch14** (ch01, ch11, ch24, ch25, ch27, ch28)

MemGPT — Charles Packer 외 (UC Berkeley). MemGPT: Towards LLMs as Operating Systems. **arXiv:2310.08560**, v1 2023-10-12 / 이 책이 읽은 판 v2 2024-02-12. venue 미기재. → **ch13** (ch11, ch24)

Mem0 — Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. **arXiv:2504.19413**, 2025-04-28. venue 없음. → **ch15** (ch11, ch26, ch27)

Zep — Preston Rasmussen 외. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. **arXiv:2501.13956**, 2025-01-20. venue 없음. → **ch15** (ch11, ch27)

ReasoningBank — Siru Ouyang 외 (UIUC; Google Cloud AI Research; Yale). ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. **arXiv:2509.25140**, v1 2025-09 / 이 책이 읽은 판 v2 2026-03-16. venue 없음. → **ch15** (ch05, ch11, ch27)

SCM — Saish Shinde (Clyrai IP Studio), 단독 저자. SCM: Sleep-Consolidated Memory with Algorithmic Forgetting for Large Language Models. **arXiv:2604.20943v1**, 2026-04-22. venue 없음 — 저자 스스로 “a research preview” 라고 적는다. → **ch15** (ch11 § 11.4.1, ch27)

Multi-Timescale (Memini) — Andreas Pattichis, Constantine Dovrolis. Continual Knowledge Updating in LLM Systems: Learning Through Multi-Timescale Memory Dynamics. **arXiv:2605.05097**, v3 2026-06-24. **ICML 2026 Workshop “Continual Adaptation at Scale: Towards Sustainable AI”** — 부록 포함 9 쪽의 workshop 논문이다. → **ch15** (ch11, ch27)

Do Language Models Need Sleep? — Sangyun Lee, Sean McLeish, Tom Goldstein, Giulia Fanti (Carnegie Mellon University; University of Maryland). Do Language Models Need Sleep? Offline Recurrence for Improved Online Inference. **arXiv:2605.26099**, v3 2026-06-05. 원문이 스스로 “Preprint.” 라고 표기하며 venue 는 없다. → **ch17** (ch11, ch24, ch26, ch29)

SEAL — Adam Zweiger, Jyothish Pari 외 (Improbable AI Lab, CSAIL MIT). Self-Adapting Language Models. **arXiv:2506.10943**, 이 책이 읽은 판 v2 2025-09-18. **NeurIPS 2025** (원문 1 쪽 각주). → **ch20** (ch05, ch06, ch11, ch19, ch27)

Language Models Need Sleep — Ali Behrouz, Farnoosh Hashemi, Adel Javanmard, Vahab Mirrokni (Google; Cornell). Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories. **arXiv:2606.03979**, v2 2026-07-10. venue 없음 — 원문 각주는 2025 년 9 월부터 OpenReview 에 공개되어 있었다고만 적는다. → **ch21** (ch04, ch11, ch18, ch24, ch27)

90.2 corpus — B 군: 판별식 불충족 (9 편)

이 이름 아래에서 자주 함께 거론되지만 네 조건 중 하나 이상에서 탈락하는 문헌이다. 탈락한다는 것은 이 범주가 아니라는 뜻이지 쓸모없다는 뜻이 아니며, 각 문헌이 실제로 증거가 되는 자리를 담당 장이 지목한다.

Nested Learning — Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, Vahab Mirrokni (Google; Columbia). Nested Learning: The Illusion of Deep Learning Architecture. **arXiv:2512.24695v1**, 2025-12-31. **NeurIPS 2025** — 노트는 “A version published at NeurIPS 2025” 로 기록한다. → **ch18** (ch11, ch21, ch24, ch28)

Memory Caching — Ali Behrouz 외 (Google). Memory Caching: RNNs with Growing Memory. **arXiv:2602.24281v1**, 2026-02-27. venue 미기재. → **ch18** (ch11, ch26, ch29)

Memory Layers — Vincent-Pierre Berges, Barlas Oğuz 외 (Meta FAIR). Memory Layers at Scale. **arXiv:2412.09764v2**, 2024-12-20(arXiv 스탬프) / 2024-12-23(본문 Date 줄). venue 미기재. → **ch09 · ch19** (ch11, ch26, ch29)

Generative Adapter — Tong Chen 외 (University of Washington; Microsoft Research). GenerativeAdapter: Contextualizing Language Models in Parameters with a Single Forward Pass. **arXiv:2411.05877**, 2024-11-08. venue 미기재 — 원고는 conference 양식이나 학회명을 적지 않는다. → **ch16** (ch11, ch24, ch26)

LoRA — Edward Hu, Yelong Shen 외 (Microsoft). LoRA: Low-Rank Adaptation of Large Language Models. **arXiv:2106.09685v2**, 2021-10-16. venue 미기재. → **ch03 · ch19** (ch09, ch11, ch20, ch21, ch26)

ROME — Kevin Meng, David Bau, Alex Andonian, Yonatan Belinkov (MIT CSAIL; Northeastern; Technion – IIT). Locating and Editing Factual Associations in GPT. **arXiv:2202.05262**, 이 책이 읽은 판 v5 2023-01-13. **NeurIPS 2022**. → **ch19** (ch11, ch22)

MEMIT — Kevin Meng, Arnab Sen Sharma 외 (MIT CSAIL; Northeastern; Technion – IIT). Mass-Editing Memory in a Transformer. **arXiv:2210.07229v2**, 2023-08-01. **ICLR 2023** (전 페이지 running header). → **ch19** (ch11, ch22)

RAG — Patrick Lewis 외 (Facebook AI Research; UCL; NYU). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. **arXiv:2005.11401**, 이 책이 읽은 판 v4 2021-04-12. venue 미기재 — vendored 원문 1,450 줄 어디에도 게재 표기가 없다. → **ch10 · ch13** (ch11, ch25, ch26)

Memorizing Transformers — Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, Christian Szegedy. Memorizing Transformers. **arXiv:2203.08913**, 2022-03-16. **ICLR 2022** (running header). → **ch12** (ch11, ch26)

90.3 corpus — C 군: 판별식의 대상이 아닌배경 · 제약 문헌 (10 편)

판별식이 판정할 대상이 아니라 다른 장들이 근거로 쓰는 재료다. 지속학습의원형 셋, 제약 · 이론 넷, 생물 실증 둘, 그리고 Θ -경로의 반증 후보 하나로 이루어진다.

EWC — James Kirkpatrick 외 (DeepMind). Overcoming catastrophic forgetting in neural networks. **arXiv:1612.00796**, 이 책이 읽은 판 v2 2017-01-25. **venue 미기재** — vendored 원문은 arXiv preprint 판이고 게재 표기 · DOI 가 없다. 이 문헌에 venue 를 붙여인용하는 것은 이 책에서 결함이다. → **ch07 · ch22** (ch08, ch18)

Deep Generative Replay (DGR) — Hanul Shin, Jung Kwon Lee, Jaehong Kim, Jiwon Kim (MIT; SK T-Brain). Continual Learning with Deep Generative Replay. **arXiv:1705.08690**, 이 책이 읽은 판 v3 2017-12-12. **NIPS 2017** (원문 1 쪽 각주에 인쇄됨). → **ch07 · ch22** (ch06)

GEM — David Lopez-Paz, Marc’ Aurelio Ranzato (Facebook AI Research). Gradient Episodic Memory for Continual Learning. **arXiv:1706.08840**, 이 책이 읽은 판 v6 2022-09-13. **NIPS 2017** (원문 1 쪽 각주에 인쇄됨). → **ch07 · ch22** (ch18, ch25, ch30)

model collapse — Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov 외 (Harvard; Stanford). Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. **arXiv:2404.01413v2**, 2024-04-29. venue 미기재. → **ch06 · ch22** (ch20, ch28, ch30)

How much do LMs memorize — John X. Morris 외 (FAIR at Meta; Cornell). How much do language models memorize? **arXiv:2505.24832**, 이 책이 읽은 판 v3 2025-06-18. venue 미기재. 원문의 날짜 표기가 두 개이고 하루어긋난다 — 본문 Date 줄은 2025-06-19, arXiv 스탬프는 2025-06-18 이다. 이책은 두 값을 인쇄된 대로 병기한다. → **ch09 · ch22** (ch26, ch29)

LoRA as Knowledge Memory — Seungju Back, Dongwoo Lee 외 (KAIST; Samsung SDS). Understanding LoRA as Knowledge Memory: An Empirical Analysis. **arXiv:2603.01097v5**, 2026-07-29. **ICML 2026, PMLR 306** (원문 1 쪽 각주). → **ch09 · ch19** (ch26)

Rate – Distortion Memory Compaction — Ashwin Gerard Colaco, Nada Lahjouji (University of California, Irvine). What to Keep, What to Forget: A Rate – Distortion View of Memory Compaction in LLMs and Agents. **arXiv:2607.08032v1**, 2026-07-09. venue 미기재 — ACM 양식으로 조판되어 있으나 학회명이 없다. → **ch22** (ch11 § 11.7, ch26, ch27)

Can a LM Learn Facts Continually — Charles O’ Neill (Baseten), 단독 저자. Can a Language Model Learn Facts Continually in Its Weights? **arXiv:2607.11020v2**, 2026-07-14. venue 없음 — 표제는 “Preprint, July 2026” 이다. → **ch22** (ch27, ch30)

PLOS Sleep — Ryan Golden, Jean Erik Delanois, Pavel Sanda 외 (UCSD; Institute of Computer Science of the Czech Academy of Sciences). Sleep prevents catastrophic forgetting in spiking neural networks by forming a joint synaptic weight representation. **PLoS Computational Biology** 18(11): e1010628, 2022-11-18. arXiv preprint 없음. → **ch08 · ch23** (ch07)

PAD — Nicolas Deperrois, Mihai A. Petrovici, Walter Senn 외 (University of Bern; Heidelberg University). Learning cortical representations through perturbed and adversarial dreaming. **eLife** 2022;e76384, DOI 10.7554/eLife.76384, 2022-04-06 게재. arXiv ID 없음 (2021-09-09 preprint). → **ch08 · ch23** (ch06)

90.4 이 책이 근거로 삼는 이 책 자신의 산출물

corpus 의 어느 논문도 상태 용량을 바이트로 보고하지 않고, 비용 보고의 단위가 논문마다 다르다. 그래서 이 책은 비교에 필요한 두 회계를 스스로 만들었다. 이들은 외부 문헌이 아니므로 서지 항목이 아니라 **책 안의 위치**로 인용한다.

표 90-1 — 이 책이 스스로 만든 회계 · 감사 산출물과 그것을 싣는 장

산출물	무엇을 만드는가	위치
통일 비용 회계	논문별로 단위가 다른 비용 보고를 하나의 축에 재배치하고, 상각 논증의 유효 범위를 정한다	ch25
상태 용량 회계	세 경로의 상태량을 바이트로 환산해 자릿수를 비교하고, dtype 이 정하는 정보 밀도를 고정한다	ch26
B_s 축 재배치	보고된 B_s -품질 점들을 통일 축에 놓고, 대다수 논문이 B_s 를 보고하지 않는다는 사실을 정량화한다	ch28
반복 consolidation 열화	자기생성 $\mathcal{R}$ 로 라운드를 반복 할때의 ρ 를 소형 재현으로 측정한다	ch29, ch30

이 네 산출물의 수치는 **등급 규칙**을 달고 인용된다 — 비율 · crossover · 순서 · bound 분류는 본문 단정문으로 쓰고, 절대치는 하한 또는 방향성으로만 쓴다. 등급 규칙 자체는 Neural Memory 모노그래프의 정직성 계약을 승계한 것이다.

Neural Memory 모노그래프 — 이 책의 전작. W 층 하나를 master equation (M) 위에서 해부한다. 이 책의 식 (U-W) 는 (M) 과 글자 그대로 같고, $\Theta \cdot W \cdot S_t \cdot \ell \cdot \mathcal{L} \cdot \eta_t \cdot \alpha_t \cdot \beta_t \cdot \mathcal{M}(\cdot; W) \cdot C(\text{chunk})$ 의 의미는 전부 거기서 승계한다. 본문 참조 형식은 “→ NM ch12 § 12.3” 이다.

90.5 본문이 이름으로만 지목하는 문헌

아래 문헌은 원문 대조를 거치지 않았다. 본문에 등장하는 이유는 하나뿐이다 — **어떤 논문이 누구를 읽었고 누구를 읽지 않았는지를 적으려면 읽히지 않은 쪽의 이름이 필요하기 때문이다.** 이 문헌들에서 사실이나 수치를 옮겨 오지 않는다.

W-경로의 주류 계보 — ch11 § 11.7 이 Do Language Models Need Sleep?의 침묵을 지적하며 나열하는 네편이다. 네 편 모두 이 책의 corpus가 아니다.

표 90-2 — 이름으로만 지목하는 *W-경로* 주류 계보 네 편

문헌	식별자	연도	본문에서의 자리
Titans: Learning to Memorize at Test Time	arXiv:2501.00663	2024-12-31	ch11 § 11.7, ch17, ch18
It's All Connected (Miras)	arXiv:2504.13173	2025-04-17	ch11 § 11.7, ch18
Atlas: Learning to Optimally Memorize the Context at Test Time	arXiv:2505.23735	2025-05-29	ch11 § 11.7, ch18
TNT: Improving Chunkwise Training for Test-Time Memorization	arXiv:2511.07343	2025-11-10	ch11 § 11.7

이 네 편은 Neural Memory 모노그래프가 정면으로 다룬 대상이다. 이 책이이들을 다시 읽지 않는 것은 분업이며, *W-경로*의 기제 상세가 필요하다면그쪽으로 넘긴다 (→ NM).

CLS 이론의 원전 — McClelland, McNaughton, O' Reilly. Why there are complementary learning systems in the hippocampus and neocortex. **Psychological Review** **102(3): 419 – 457**, 1995. ch08 이 CLS 를 도입할 때, 그리고 ch11 § 11.6 이 “이름은 생물에서 왔고 논증은 오지않았다” 를 적을 때 등장한다 — 이 라인에 sleep 이라는 이름을 준 논문의참고문헌에 이 원전이 없다는 사실이 그 서술의 근거다.

agent memory 갈래 — ReasoningBank 이 자기 목소리로 진단하는 선행연구로 ch15 에 이름이 나온다 (AWM, Synapse). 이 책은 이들을 읽지 않았고, ReasoningBank 의 진단을 그 논문에 귀속시켜 옮기기만 한다.

서지 항목이 아닌 이름들 — 본문에는 문헌이 아니라도구·모델·데이터셋의 이름도 등장한다 (vLLM, NetworkX, GPT-4.1, Llama 3.1, PEER, BM25, DPR 등). 이들은 참고문헌이아니라 그것을 쓴 논문의 실험 설정이므로, 출처는 언제나 그 논문의 위치로병기한다. 별도 항목을 두지 않는다.

90.6 담당 장역색인

문헌에서 장을 찾는 것이 § 90.1 – § 90.3 이고, 장에서 문헌을 찾는 것이 이표다. 굵은 글씨가 그 장이 정면으로 다루는 문헌이고, 나머지는 근거로 쓰는문헌이다.

표 90-3 — 장 → 그 장이 쓰는 문헌 역색인

장	문헌
ch01	Letta STC, LM Need Sleep, Do LMs Need Sleep? (세 층의 필요를 보이는 예로만)
ch02	— (Neural Memory ch02 위에서 시작)
ch03	LoRA

장	문헌
ch04	LM Need Sleep, SEAL (distillation 실무의 실제 사용례로만)
ch05	SEAL, ReasoningBank (RL 루프의 실제 사용례로만 — § 05.7)
ch06	model collapse , SEAL, DGR, PAD
ch07	EWC, DGR, GEM, PLOS Sleep
ch08	PLOS Sleep, PAD, EWC
ch09	How much do LMs memorize, Memory Layers, LoRA as Knowledge Memory
ch10	RAG
ch11	corpus 29 편 전부
ch12	Memorizing Transformers
ch13	MemGPT, RAG
ch14	Letta STC
ch15	Mem0, Zep, ReasoningBank, SCM, Multi-Timescale
ch16	Generative Adapter
ch17	Do LMs Need Sleep?
ch18	Nested Learning, Memory Caching, GEM
ch19	LoRA, ROME, MEMIT, Memory Layers, LoRA as Knowledge Memory
ch20	SEAL
ch21	LM Need Sleep, Nested Learning
ch22	Can a LM Learn Facts Continually, Rate – Distortion, EWC, DGR, GEM, How much do LMs memorize, ROME, MEMIT
ch23	PLOS Sleep, PAD
ch24	ch11 의 분류 결과 — 통과 9 편 + 경계 사례 1 편
ch25	Letta STC, GEM, RAG + 이 책의 통일 비용 회계
ch26	Memory Layers, LoRA, How much do LMs memorize, Rate – Distortion + 이 책의 상태 용량 회계
ch27	통과 9 편 전부 + Can a LM Learn Facts Continually
ch28	Letta STC, Nested Learning, model collapse
ch29	Do LMs Need Sleep?, Memory Caching, Memory Layers
ch30	model collapse, GEM, Can a LM Learn Facts Continually

용어집

읽는 법

이 책의 모든 용어와 기호에는 **정의 소유 장**이 있다. 소유 장이볼드와 정의문으로 도입하고, 다른 장은 참조만 한다. 재정의는 이 책에서결함이므로, 어떤 용어의 뜻이 장마다 다르게 읽힌다면 그것은 독자의 오독이아니라 이 책의 결함이다. 이 목록은 그 소유 관계를 한 자리에 모은 것이다.

각 항목은 다음을 적는다.

기호 또는 용어 · 뜻 · **정의 소유 장** · 출처 (승계인가 신설인가)

출처란은 두 값 중 하나다. **NM 승계**는 Neural Memory 모노그래프에서 글자 그대로 물려받은 것이고 재정의가 금지된다. **v2 신설**은 이 책이 새로 세운 것이다. 승계 기호를 이 책이 다시 도입하는자리는 ch01 이지만, 그 **의미의 소유권은 여전히 Neural Memory 에있다** — 이 책이 뜻을 바꿀 수 없다는 뜻이다.

기호를 찾을 때는 § 91.1 – § 91.4 를, 용어를 찾을 때는 § 91.6 – § 91.10 을, 어느 장이 무엇을 소유하는지찾을 때는 § 91.11 을 본다.

91.1 기호 — 총과상태

표 91-1 — 상태 3 총과 그에 딸린 기호

기호	뜻	정의 소유 장	출처
Θ	slow weights. 배포 아티팩트를 이루는 파라미터. 학습 라운드 k 에 움직인다. 지우는 연산이 정의되지 않으며, 바꾸려면 새아티팩트를 배포한다	ch01 § 01.2	NM 승계
W	fast weights. test-time 에 토큰 · chunk 단위로 움직이는 가중치 형태의 내부 상태. 상태를 들고 있는 주체가사라지면 함께 사라진다	ch01 § 01.2	NM 승계

기호	뜻	정의 소유 장	출처
E	external store. 모델 밖에 텍스트·벡터·그래프로 저장되는 기억. 요청과 독립적으로 존속하고 임의 시점에 지울 수 있다	ch01 § 01.2	v2 신설
S_t	W 갱신의 momentum(surprise) 항. 항상 시간 첨자 t 를 단다	ch01 § 01.3	NM 승계
$\mathcal{M}(\cdot; W)$	memory 읽기 함수	ch01	NM 승계
$\hat{c}$	learned context. 질의 전 오프라인 추론의 산출물. E 의 한 레코드이며 L_w 에 0 을 기여한다	ch01 § 01.3	v2 신설
$\mathcal{R}_k$	k 번째 sleep 라운드의 학습 집합. replay·distillation·합성 데이터의 통칭	ch01 § 01.3	v2 신설
$\mathcal{H}_k$	k 라운드까지 누적된 경험	ch01 § 01.3	v2 신설

세 층의 결정적 차이는 크기도 정확도도 아니라 누가 소유하고 언제 사라지는가이며, 그 차이가 비용 4 종의 값을 전부 결정한다 (→ ch01 § 01.2).

91.2 기호 — 읽기·쓰기와절차

표 91-2 — 전역 표준형 수식

번호	식	무엇을 말하는가	정의 소유 장
(R)	$\hat{y} = f(q; \Theta, W, \text{ret}(E, q))$	답을만들 때 무엇을 보는가. 읽기는 세 층을 모두 통과한다	ch01 § 01.3
(U-W)	$W_t = \alpha_t W_{t-1} + S_t,$ $S_t = \beta_t S_{t-1} - \eta_t \nabla_W \ell(W_{t-1}; k_t, v_t)$	토큰 하나가 도착할 때 마다 inner loss 의 gradient 방향으로 W 를 한걸음 옮긴다. Neural Memory 의 master equation (M) 과 글자 그대로 같다	ch01 § 01.3 (NM 승계)
(U- Θ)	$\Theta_{k+1} = \Theta_k - \eta_\Theta \nabla_\Theta \mathcal{L}(\mathcal{R}_k; \Theta_k),$ $\mathcal{R}_k = \text{gen}(\mathcal{H}_k; B_s)$	누적 경험으로 학습 집합을 만들고 그것으로 outer loss 를 줄이는 라운드를 돈다	ch01 § 01.3

번호	식	무엇을 말하는가	정의 소유 장
(U-E)	$E_{k+1} = \text{wr}(E_k, \hat{c}_k),$ $\hat{c}_k = S(c_k; B_s)$	질의 전에 문맥을 예산 B_s 로 오프라인 처리해 $\hat{c}$ 를 만들고 store 에 기록한다	ch01 § 01.3
(A)	$C_{\text{avg}} =$ $C_{\text{wake}} + C_{\text{sleep}}/N_q$	질의하나당 실제로 내는 값 = wake 비용 + 상각된 sleep 비용	ch01 § 01.5

파생형 수식은 장 번호를 붙인다 (예: 식 (7-3)). 전역 표준형은 어느장에서도 번호만으로 인용한다.

표 91-3 — 연산자와 절차

기호	뜻	정의 소유 장	출처
$\text{ret}(E, q)$	질의 q 에 대한 검색 연산. 독자의 어휘로는 vector DB 조회 한 번이며 비용도 그렇게 회계한다. L_w 에만 잡힌다	ch01 § 01.3 (비용 분하는 ch10)	v2 신설
$\text{wr}(E, \hat{c})$	E 에 대한 기록 연산. 덧어쓰기인가·합집합인가·ADD/UPDATE/DELETE 를 판단하는가가 E-경로 논문들을 가르는 축이다	ch01 § 01.3 (형태별 분류는 ch15)	v2 신설
$\text{gen}(\cdot; B_s)$	예산 B_s 로 학습 집합 $\mathcal{R}$ 을 만드는 절차. 구현이 Part I 의 절반 (ch04 – ch07) 이다	ch01 § 01.3	v2 신설
$S(c; B_s)$	예산 B_s 로 문맥 c 를 오프라인 처리하는 절차. 항상 인자를 괄호로 받는다	ch01 § 01.3	v2 신설
$\ell(\cdot; \cdot)$	inner loss(소문자). 괄호를 받는다	ch01 (NM)	NM 승계
$\mathcal{L}$	outer/task loss(대문자)	ch01 (NM)	NM 승계
f	모델의 읽기 함수. 식 (R) 의 좌변을 만드는 것	ch01 § 01.3	NM 승계

91.3 기호 — 비용 4 종과 상각

비용 기호는 이 넷 외에 신설하지 않는다. 비교 가능성을 유지하기 위해서다. 논문이 어떤 칸을 보고 하지 않으면 그 칸에는 “**논문에없음**” 이라고 쓴다 — 추정치를 본문 수치로 쓰지 않는다.

표 91-4 — 비용 4 종

기호	뜻	단위	독자의 어휘	정의 소유 장
B_s	sleep-time 계산 예산. 질의 도착 전에 소비하는 추가계산	FLOPs 또는 tokens	유휴 시간 배치 job 의 비용	ch01 § 01.5
B_t	wake-time(test-time) 계산 예산	FLOPs 또는 tokens	질의당 추론 비용	ch01 § 01.4
L_w	wake 지연 기여. 정의상 sleep 은 여기에 0 을 기여해야한다	ms, TTFT/ITL	SLA 항목	ch01 § 01.5
C_{cap}	상태 용량. corpus 에서 가장 침묵당한값으로, 이것을 바이트로 보고한 논문이 0 편이다	bytes/세션또는 bytes/사용자	저장 sizing, checkpoint 크기	ch01 § 01.5 (회계는 ch26)
ρ	망각 · 열화율. 라운드를 반복할 때의 성능손실	라운드당 성능 손실	반복 실행의 지속 가능성	ch01 § 01.5

표 91-5 — 상각에 딸린 기호

기호	뜻	정의 소유 장	비고
N_q	한 문맥을 공유하는 질의 수. 식 (A) 의 분모. 비용이 아니라 워크로드의 성질이라 비용 4 종의 어느 칸에도 들어가지 않으면서 모든상각 주장의 결론을 좌우한다	ch01 § 01.5	실서빙 분포를 보고한 논문이 corpus 에 없다
N_q^*	손익분기. C_{avg} 가 sleep 없는 baseline 과 같아지는 N_q 값	ch01 § 01.5	계산 예제는 ch01 의 worked
κ	생성 토큰 대 입력 토큰의 비용비. 비용 기호가 아니라 회계 가정이다. 이 값 하나로 N_q^* 가 2.7 배 움직인다	ch01 § 01.5	micro-example 논문별 출처와 민감도는 ch14·ch25
$C_{wake}, C_{sleep}, C_{avg}$	식 (A) 의 세 항 — wake 비용, sleep 1 회 비용, 상각 후 질의당 비용	ch01 § 01.5	맨 C 와 혼동 금지 (§ 91.5)

ρ 는 논문별 변형을 아래첨자로 허용한다 (ρ_r 등). 다만 그장의 표기 대응표에 무엇이 비율인지 기록해야 한다. 첨자 없는 맨 ρ 는 언제나 비용 4 종의 망각 · 열화율이다.

91.4 기호 —첨자·차원·그 밖의 예약기호

표 91-6 — 시간 첨자. 시간척도 혼동이 이 분야 최대의 서술 오류이므로첨자가 척도를 드러낸다.

기호	무엇의 첨자인가	정의 소유 장
t	토큰 또는 chunk	ch01 § 01.3 (NM 승계)
k	sleep 라운드 . optimizer step 에 쓰지 않는다	ch01 § 01.3
τ	세션	ch01 § 01.2
s	SGD step . “step s 와 sleep 라운드 k 는 다르다” 는구분 자 체가 Part I 의 교육 목표다	ch02 § 02.3
j	청크·세그먼트 인덱스	ch17

표 91-7 — 차원·크기·그 밖

기호	뜻	정의 소유 장	비고
d	모델 폭	ch01 (NM 승계)	
L	sequence 길이	ch01 (NM 승계)	L 을 층 수 뜻으로 쓰는 것은 결함이다
L_{layer}	층 수	ch01 (NM 승계)	
n	저장된 항목 수	ch10	청크 인덱스로 쓰지 않는다 — 그것은 j 다
C	chunk 크기 . 맨 C 는 이 뜻으로만 쓴다	ch01 (NM 승계)	용량은 반드시 C_{cap}
r	LoRA rank	ch03 § 03.3	보상은 R 로 쓴다
$\mathcal{S}$	LoRA 를 적용하는 행렬 집합 (예: $\mathcal{S} = \{W_q, W_v\}$)	ch03 § 03.4	ch09·ch19 가그대로 쓴다
R	강화학습의 보상	ch05 § 05.1	원 논문이 r 을 쓰면 대응표로 옮긴다
Π	지속학습 성능 행렬 . Π_{ij} = 과제 j 까지 학습한 뒤 과제 i 의 성능. ACC·BWT·FWT 가 이행렬에서 나온다	ch07	원 논문 (GEM) 은 R 을 쓰지만 이 책에서 R 은보상이다
ω	replay 혼합비 . 한 배치에서 과거 데이터가 차지하는비율	ch07	
η_t	inner learning rate	ch01 (NM 승계)	
η_Θ	outer/sleep learning rate	ch01 § 01.3	η_t 와 섞어 쓰는 것은 결함이다
α_t	retention gate — 남기는 비율 이다	ch01 (NM 승계)	
β_t	momentum decay	ch01 (NM 승계)	
z	표본·후보	ch06 § 06.4	s 가 SGD step 으로 예약되어 있으므로

기호	뜻	정의 소유 장	비고
q, k_t, v_t	질의 · 키 · 값 벡터	ch01 (NM 승계)	벡터는 소문자, 행렬 · 텐서는 대문자

수 · 단위 규약 셋을 함께 못 박는다. **바이트는 항상 dtype 을 병기한다** (“bf16 기준 2 bytes/ elem”). 로그는자연로그이고 정보량은 bits 로 쓰되 명시한다. 논문 수치는 원문그대로 옮기고 반 올림하지 않는다.

91.5 기호 충돌 — 반드시 구분해야 하는 세 쌍

이 세 쌍은 이 책에서 가장 자주 어긋나는 자리다. 어느 쪽으로도 흡수되지않는다.

표 91-8 — 반드시 구분해야 하는 기호 세 쌍

충돌	구분 규칙	어디서 정해졌나
C 대 C_{cap}	맨 C 는 chunk 크기 (NM 승계). 상태 용량 은 언제나 C_{cap}	ch01 § 01.5
S_t 대 $S(c; B_s)$	S_t 는 (U-W) 의 momentum 이며 항상시간 첨자를 단다. $S(c; B_s)$ 는 (U-E) 의 오프라인 처리 절차 이며항상 인자를 괄호로 받는다. 첨자 없는 맨 S 는 쓰지 않는다	ch01 § 01.3
η_t 대 η_Θ	η_t 는 inner(토큰 척도), η_Θ 는 sleep 라운드 척도. 둘을 섞어 쓰는 것은 결함	ch01 § 01.3

ℓ 의 두 용법은 허용된다 — 아래첨자 ℓ 은층 ·level 인덱스이고, 괄호를 받는 $\ell(\cdot; \cdot)$ 은 inner loss 다. 괄호 유무로 구분한다. ℓ_2 노름은 관용 표기 그대로쓴다.

91.6 용어 — 프레임

sleep-time compute — 질의 q 가 도착하기 전, 유휴시간에 예산 B_s 만큼 소비하는 계산으로서, 그 산출물이 이후 질의의 wake-time 비용 B_t 를 줄이거나 정확도를 높이는 것. 이 조작적 정의는**갱신 대상 층을 지정하지 않으며**, 그 때문에 같은 이름 아래 서로다른 세 절차가 들어온다. **정의 소유 장: ch01 § 01.4.** 표기는원어를 유지한다 — “수면 계산” · “슬립타임 컴퓨트” 로 쓰지않는다.

wake — 질의를 받아 답을 만드는 시간. sleep 의 대립항이며, 어떤절차가 L_w 를 늘린다면 그것은 sleep 이 아니라 wake-time 계산이다. **ch01 § 01.6** 의 마지막 Rosetta 행이 이 시금석을 고정한다.

sleep 라운드 — (U- Θ) 또는 (U-E) 를 한 번 도는 단위. 첨자 k 가 이것을 센다. “sleep 한다” 처럼 층을 특정하지 않는 서술은금지되며, “**sleep 라운드에서 Θ 를 갱신한다**” 처럼 층을밝힌다. **ch01 § 01.3.**

E-경로 — 질의 전 오프라인 추론의 산출물을 모델 밖에 적재하는절차. (U-E) 만 실행하고 Θ 와 W 는 불변이다. **ch01 § 01.4.**

W-경로 — 문맥을 fast weight 로 접어 넣고 원 문맥을 버리는 절차. (U-W) 를 wake 가 아니라 **오프라인에 추가로 실행한다.** **ch01 § 01.4.**

Θ-경로 — 경험을 slow weights로 통합하는 절차. (U-Θ)를 실행하며 **배포 아티팩트가 바뀐다**. 이것이 나머지 둘과의 결정적 차이이다. **ch01 §01.4**. 세 경로를 parametric/non-parametric의 이분법으로 부르지 않는다 — 그 이분법은 부정확하다.

상태 3층 — $\Theta \cdot W \cdot E$ 의 3분할. 새로운 이론이 아니라 **회계 단위의 선택**이다. 나누지 않으면 evict 가능성·batching·열화의 정체·롤백 단위 네 질문을 물을 수조차 없다. **ch01 §01.1 – §01.2**.

Rosetta 사전 — 독자가 이미 아는 inference-serving 어휘를 이 라인의 어휘에 접속하는 대응표. 대응의 성격을 **동일·유비·차이가 논점** 셋으로 구분하는 것이 핵심이며, 구분하지 않으면 잘못된 직관이 이식된다. **ch01 §01.6**이 E 층과 오프라인 시계에 해당하는 행을 소유하고, W 층과 decode 내부의 행은 Neural Memory가 소유한다.

상각 — sleep 비용을 N_q 로 나누어 질의당 비용으로 환산하는 것. 식 (A). 상각 논증은 전부 조건부다 — “ N_q 가 충분히 크면”이라는 전제를 명시하지 않은 상각 주장은 이 책에서 미완성으로 취급한다. **ch01 §01.5(도구)**, **ch25(판정)**.

통일 비용 회계 — 논문마다 단위가 다른 비용 보고를 하나의 축에 재배치하는 이 책의 회계. **ch25**가 소유한다.

상태 용량 회계·정보 밀도 — 세 경로의 상태량을 바이트로 환산해 자릿수를 비교하고, dtype이 정하는 bytes/elem이 곧 정보 밀도임을 고정하는 회계. 도입은 **ch03 §03.6**(delta 아티팩트의 dtype), 확장과 경로 간 비교는 **ch26**이 소유한다.

읽 계층 delta 스테이징 — 이 책이 memory-centric 논증을 쓸 수 있다고 판단한 **유일한 지점**. Θ-경로의 사용자별 delta가 만드는 저장 계층 문제를 가리킨다. **ch29**가 소유한다. 근거가 있는 이 지점 밖에서 memory-device 연결을 만드는 것은 이 책에서 금지된다.

91.7 용어 — 판별과 분류

이 절의 용어는 전부 **ch11**이 소유한다. ch01은 판별식을 진술만하고 적용을 유예했으며, 적용 절차·분류·침묵지도는 ch11이 낸다.

판별식 — 절차 P가 sleep-time compute 인지 판정하는 네 조건. (1) **시계** — 상태를 바꾸는 연산이 질의 도착 전에 실행되는가. (2) **예산** — 그 연산이 $B_s > 0$ 을 소비하는가. (3) **대상** — 그 연산이 $\Theta \cdot W \cdot E$ 중 어느 한 층의 상태를 실제로 바꾸는가. (4) **소비자** — 바뀐 상태가 그 연산을 유발한 질의가 아닌 이후 질의에 쓰이는가. 넷을 모두 만족해야 한다. 진술은 **ch01 §01.4**, **정밀화와 적용**은 **ch11 §11.2**. 판별식은 논문의 자기 규정도 성능도 묻지 않는다 — 입력은 **갱신식과 시계의 쌍**뿐이다.

적용 절차 — 판정을 재현 가능하게 만드는 다섯 단계. 갱신연산자를 찾고 → 좌변을 세 층에 배정하고 → 시계를 읽고 → 예산을 확인하고 → 소비자를 확인한다. 1-2 단계는 논문의 수식만, 3-5 단계는 실험 설정만 본다. **저자의 서술은 어느 단계에도 입력되지 않는다**. **ch11 §11.2.2**.

A 군·B 군·C 군 — 전수 분류의 세 군. A 군은 네 조건을 모두 만족하는 9편, B 군은 이 이름 아래 자주 함께 거론되나 어느 조건에선가 탈락하는 9편, C 군은 이 범주를 자처하지 않는 배경·제약 문헌 10편으로 판별식의 판정대상이 아니다. 여기에 경계 사례 1편 (MemGPT)이 더해져 corpus 29편이 되고, 이 책의 모든 집계는 **통과 9 + 경계 1 + 불충족 19**로 쓴다. **ch11 §11.3**. 전수표는 부록 A.

시계 부재 — 무엇을 어디에 어떻게 쓰는지는 완전히 규정하지만 언제 쓰는지를 규정하지 않는 논문의 탈락 사유. LoRA·ROME·MEMIT가 해당하며, 이들에게는 쓰기 **primitive**라는 별도의 지위를 준다. **ch11 §11.2.4**.

쓰기 primitive — 시계를 갖지 않은 채 “무엇을 어떻게 쓰는가”만 규정하는 도구. 어느 시계에 엮을지는 이것을 쓰는 논문의 선택이며, 그 선택이 곧 Θ-경로 논문들의 설계다. **ch11 §11.2.4**, **ch19**.

경계 사례 규칙 — 판별식 네 조건 중 셋을 만족하고, 통과군의 논문이 그 논문을 자기 계보의 조상으로 지목하는 경우에 한해 그 논문을 해당 경로의 **출발점으로 서술**한다. 두 의무가 붙는다 — (i) 탈락한 조건을 그 장의 본문에 명시하고, (ii) **통과군의 일원으로 세지 않는다**. MemGPT가 유일한 적용 대상이다. **ch11 §11.5**.

연대가 강제한 침묵 / 선택된 침묵 — 논문 A 가 B 를 인용하지 않는 사실의 두 종류. B 의 공개가 A 보다 늦으면 연대가 강제한 침묵이며 계보 사실일 뿐이다. B 가 앞서고 같은 문제를 다루는데도 인용이 없으면 선택된 침묵이며, 이것이 서술 대상이다. 공개 간격이 수 주 이내인 **근접 동시성은 강제 쪽으로 분류한다**. ch11 § 11.6.1.

침묵 지도 — corpus의 참고문헌을 전수 확인해 경로 간 인용의 유무와 깊이를 적은 표. 인용의 정당성을 평가한 것이 아니다. ch11 § 11.6, 표 11-4.

이름이 기제보다 넓다 — 이 책의 척추 논지. 판별식을 적용하면 이 이름 아래 모인 문헌의 상당수가 어느 조건에선가 탈락하고, 어긋남은 통과·불통과 경계에서만 아니라 **경로 배경에서도** 생긴다. 예고는 ch01 § 01.4, 증거는 ch11, 판정은 ch24.

경칩 — W -경로와 Θ -경로 사이에서 산출물의 모양과 시계가 갈리는 자리를 가리키는 이 책의 용어. Generative Adapter가 그 자리에 있다 — 산출물은 LoRA 모양이지만 갱신식은 $(U-W)$ 이고 시계는 wake다. ch16.

91.8 용어 — training 실무 (Part I)

독자는 transformer inference를 잘 알고 **training 실무는 모른다**는 것이 이 책의 독자 상정이다. Part I은 그 공백만 채운다. 아래 용어는 전부 그 목적으로 도입된다.

레짐 · training run — 하나의 checkpoint가 태어나기까지의 단위. 이 책은 pre-training · SFT · RLHF/RLVR · continual 네 레짐을 각각 “무엇이 언제 움직이는가”로 정의하고, 각 레짐이 $(U-\Theta)$ 의 어느 인자를 채우는지 대응시킨다. ch02 § 02.1 – § 02.8.

token budget · step · batch · epoch — 예산의 어휘 넷. 서로 환산되지만 같은 것이 아니다. ch02 § 02.3.

“누가 이 값을 학습하는가” — 임의의 절차를 읽을 때 던지는 판별습관. Part II의 모든 논문장이 § N.4에서 이 질문에 표로 답한다. ch02 § 02.10.

full fine-tuning — Θ 전체를 갱신하는 기준선. 그것이야기는 것 (옵티마이저 상태, delta 아티팩트의 크기)이 PEFT의 동기다. ch03 § 03.1.

adapter — 층 사이에 모듈을 삽입하는 부분 갱신. 삽입의 대가는 depth이고 depth의 대가는 decode 지연이다. ch03 § 03.2.

LoRA — 가중치 변화량에 rank r 를 거는 부분 갱신. $\Delta W = BA$ 형태이며, 어느 행렬 집합 S 에 걸 것인가와 r 을 얼마로 할 것인가가 설계 축이다. ch03 § 03.3 – § 03.4.

merge — LoRA delta를 base 가중치에 흡수해 $L_w = 0$ 으로 만드는 서빙 형태. 무엇과 맞바꾼 것인지가 논점이다 (사용자별 delta를 유지할 수 없게 된다). ch03 § 03.5.

delta 아티팩트 — Θ -경로가 만드는 상태의 물리적 형태. 그 크기는 rank · 적용 집합 · dtype의 곱으로 손으로 계산된다. ch03 § 03.6.

teacher · student — distillation에서 gradient의 출처를 바꾸는 두 역할. ch04 § 04.1.

logit distillation · temperature — teacher의 출력분포를 온도로 부드럽게 만들어 student가 맞추게 하는 형태. ch04 § 04.2.

feature distillation — 출력이 아니라 내부 표현을 맞추는 형태. ch04 § 04.3.

self-distillation — teacher가 자기 자신일 때. sleep 라운드의 $\mathcal{R}$ 이 자기생성일 때 gen이 취하는 대표 형태다. ch04 § 04.4.

upward distillation — 작은 자기에서 큰 자기로 전달하는 형태. ch04 § 04.5.

policy · rollout · advantage — policy gradient의 최소 골격. LLM 위에서 무엇이 state이고 무엇이 action이고 무엇이 reward 인지를 먼저 고정한 뒤에 쓴다. ch05 § 05.1 – § 05.2.

RLHF · reward model — 채점기를 학습해 사람을 대신세우는 파이프라인. ch05 § 05.3.

DPO — reward model을 지우고 preference를 직접 loss로 바꾸는 형태. ch05 § 05.4.

RLVR — 채점기를 프로그램으로 바꾸는 형태. ch05 § 05.5.

reward hacking — 보상이 목적을 대신할 때 일어나는 실패. ch05 § 05.6.

synthetic data — gen이 자기생성일 때 만들어지는 데이터. sleep 라운드의 앞 절반은 실무적으로 대규모 batch decode job이다. ch06.

model collapse — 모델이 자기 게보가 생성한 데이터로 반복 학습될때 라운드가 진행될수록 성능이 악화되는 현상. 이 책이 쓰는 조작적 정의는원 논문의 것으로, 반복에 대해 **오차가 유계로 남는가**를 묻는다. **ch06**. 이 개념이 Θ -경로의 반증 축 하나를 공급한다 ($\rightarrow$ ch22, ch30).

catastrophic forgetting — 새 과제를 배우면서 이전 과제의 성능이무너지는 현상. **ch07**.

replay · ω — 과거 데이터를 현재 배치에섞어 망각을 막는 대응. ω 가 그 혼합비다. **ch07**.

성능 행렬 Π · ACC · BWT · FWT — 지속학습 평가의 표준 지표. Π_{ij} 에서 셋이전부 파생된다. **ch07**. ch22·ch25·ch30 이그대로 재사용한다.

CLS(complementary learning systems) — 빠른 해마와 느린신피질이라는 두 학습계를 상정하는 이론. 이 라인이 은유를 빌려 온출처이며, **은유가 왔을 뿐 논증은 오지 않았**다는 것이 ch11 § 11.6 과 ch23 의 서술 대상이다. **ch08**.

consolidation — 원어를 유지한다. 첫 등장 시 “공고화” 병기만허용하고 단독 사용은 하지 않는다. 생물학에서 consolidation 은 시냅스에걸리는 연산이며, 인공 대응물이 무엇인지 (가중치인 가 그래프 edge 인가) 를밝히지 않고 쓰면 층이 뒤바뀐다. **ch08**(생물 쪽 정의), **ch11 § 11.4.1**(층 혼동의 사례).

기억 용량 · **bits-per-parameter** — 파라미터 하나가담을 수 있는 정보량의 상한. Θ 층 용량의 실측 · 이론이여기서 나오고, 그 값이 C_{cap} 회계의 상한을 건다. **ch09**.

E 층의 형태와 읽기 비용 — 무엇을 색인하고 무엇을 조회하며 그비용이 L_w 의 어디에 잡히는지의 분해. **ch10**.

91.9 용어 — 정직성장치

이 책이 논문의 주장과 자기 해설과 자기 평가를 섞지 않기 위해 쓰는표시다. 표시가 없는 산문은 이 책의 해설이며, 논문의 주장은 언제나 논문에귀속되고 위치가 병기된다.

표 91-9 — 주장 · 해설 · 평가를 가르는 표시

표시	무엇을 뜻하는가	언제 쓰는가
[해설] 블록	논문 내용을 독자의 어휘로 재 서술한 것	경계가 흐려질 위험이 있을 때
[평가] 블록	논문과 다른 이 책의 판단	논지를지탱하는 평가는 반드시 블록으로
[본서 관찰]	논문 표에서 직접 읽히지만 논문이 문장으로 쓰지않은 사실	문장 끝에 붙인다
[본서 산술]	논문 수치로부터 이 책이 계산한 값. 비용칸이 “논문에 없음”인 논문에서 절대치를 파생할 때는 의무	“
[본서 추론]	논문 근거에서 연역했으나 논문이 주장하지 않은 것	“
[본서 판단]	짧은 평가	논지를 지탱하면 마커가 아니라 [평가] 블록으로 올린다
“논문에 없음”	비용 4 종 표의 빈 칸에 적는 말. 추정치로 메우지않는다	비용 4 종 표
TODO-VERIFY	확신 없는 서술. 주장과 확인 방법 을 함께적는다	수치는 이 상태로 본문에 쓰지 않는다

인라인 마커는 문장 끝에 괄호 없이 붙인다. 문단 전체가 이 책의 것이면마커가 아니라 블록을 쓴다.

원 논문의 절 번호는 언제나 논문 이름과 함께 쓴다 — [SEAL §4.2], [MEMIT App. B.4]. 첨자 없는 맨 §3.1 은 **이 책의 절**을 가리키는 것으로만읽힌다.

실험 수치의 등급 — 이 책 자신의 실험에서 나온 값은 등급을 달고 인용된다. 비율·crossover·순서·bound 분류는 본문 단정문으로, 절대치는 하한또는 방향성으로만 쓴다.

91.10 확정명칭

같은 대상을 두 이름으로 부르지 않기 위한 표다. 오른쪽 열의 표현은 이 책에서 쓰지 않는다.

표 91-10 — 공식 명칭 확정표

확정 명칭	쓰지 않을 것	이유
sleep-time compute learned context ($\hat{c}$)	수면 계산, 슬립타임 컴퓨트 학습된 문맥 (단독), 요약	기술 용어는 원어유지 “요약”은 $S(\cdot; B_s)$ 가 추론을 한다는 사실을 지운다
consolidation external store (E)	공고화 (단독) 외부 메모리, long-term memory	첫 등장 시 괄호 병기만 허용 “메모리”는 HBM/DRAM 과 충돌한다
slow weights / fast weights Θ -경로 / W-경로 / E-경로	느린 가중치 / 빠른 가중치 parametric / non-parametric	NM 승계 용어 이분법이 부정확하다
$\text{ret}(E, q)$	“검색” (단독)	어느 store 를 읽는지 밝힌다

“memory”를 수식어 없이 쓰지 않는다 — W 인지 E 인지 HBM/DRAM 인지 밝힌다. “sleep”을 동사로 쓰지 않는다 — “sleep 라운드에서 Θ 를 갱신한다”처럼 층을 밝힌다. 기술 용어는 영어 원어를 유지하고 조사는 한글 발음 기준으로 직접 결합한다 (“chunk 가” , “replay 를” , “ Θ 를”).

91.11 소유 장역색인

용어에서 장을 찾는 것이 위 절들이고, 장에서 그 장이 소유한 용어를 찾는 것이 이 표다. 어떤 장을 쓸 때 재정의해서는 안 되는 것을 확인하는 용도로 쓴다.

표 91-11 — 장 $\rightarrow$ 그 장이 정의를 소유하는 용어·기호

장	이 장이 정의를 소유하는 것
ch01	$\Theta \cdot W \cdot E$ 의 v2 도입, $E \cdot \hat{c} \cdot \mathcal{R}_k \cdot \mathcal{H}_k \cdot \text{ret} \cdot \text{wr} \cdot \text{gen} \cdot S(c; B_s) \cdot \eta_{\Theta} \cdot k \cdot \tau$, 식 (R)·(U-W)·(U- Θ)·(U-E)·(A), 비용 4 중 $B_s \cdot B_t \cdot L_w \cdot C_{\text{cap}} \cdot \rho$, $N_q \cdot N_q^* \cdot \kappa$, sleep-time compute 의 조작적 정의, wake, 세 경로, 판별식의 진술, Rosetta 사전확장분, 질문 지도 Q1–Q8
ch02	레짐, training run 구성요소, token budget·step(s)·batch·epoch, pre-training·SFT·RLHF/RLVR·continual, “누가 이 값을 학습하는가”
ch03	full fine-tuning, adapter, LoRA, rank r , $\mathcal{S}$, merge, delta 아티팩트, dtype 과 정보 밀도

장	이 장이 정의를 소유하는 것
ch04	teacher-student, logit distillation, temperature, feature distillation, self-distillation, upward distillation
ch05	policy-rollout-advantage, 보상 R , RLHF-reward model, DPO, RLVR, reward hacking
ch06	synthetic data, model collapse, 표본 기호 z
ch07	catastrophic forgetting, replay, 혼합비 ω , 성능 행렬 Π , ACC-BWT-FWT
ch08	CLS, consolidation(생물 쪽 정의), sleep 단계
ch09	기억 용량, bits-per-parameter
ch10	E 층의 형태, 읽기 비용 분해, 저장 항목 수 n
ch11	판별식의 정밀화와 적용 절차, A 군 · B 군 · C 군, 시계 부재, 쓰기 primitive, 경계사례 규칙, 침묵 지도, 연대가 강제한 침묵 / 선택된 침묵
ch16	경첩
ch17	청크 인덱스 j
ch25	통일 비용 회계
ch26	상태 용량 회계, 경로 간 정보 밀도 비교
ch29	웜 계층 delta 스테이징
ch30	반증 조건

부록 A — 판별식 전수분류표

A.1 이 부록의용도

이 부록은 **조회용**이다. 논문 한 편을 손에 들고 “이것이 이 책에서어디에 속하고 어느 장이 다루는가” 를 한 줄로 확인하는 데 쓴다. **논증은 ch11 에 있다**. 판별식이 왜 네 조건이어야 하는지, 각 조건을 빼면 무엇이 잘못 들어오는지, 이름과 실체가 갈리는 다섯 사례가 각각 어떤유형인지는 전부 ch11 이 낸다. 이 부록은 그 결론만 표로 옮긴다.

두 문서가 어긋나면 **ch11 이 단일 SoT** 다. 이 부록의 행을 고쳐야한다.

A.2 판별식과 표기규약

판별식. 절차 P 가 sleep-time compute 인가? (1) **시계** — 상태를 바꾸는 연산이 질의 q 가 도착하기 **전에** 실행되는가. (2) **예산** — 그 연산이 $B_s > 0$ 의 추가 계산을 소비하는가. (3) **대상** — 그 연산이 $\Theta \cdot W \cdot E$ 중 **어느 한 층의상태를 실제로 바꾸는가**. (4) **소비자** — 바뀐 상태가 그 연산을유발한 질의가 아닌 **이후 질의**에 쓰이는가. 넷을 모두 만족해야한다. 하나라도 빠지면 인접 문헌이지 이 범주가 아니다.

표의 기호는 셋이다.

- $\bigcirc$ — 만족한다.
- $\times$ — 불만족한다. **ch11 이 탈락 조건으로 지목한 칸이다**.
- — — 이 조건을 판정할 좌표가 논문에 없다. 판정이 성립하지않는다.

하나라도 $\times$ 이면 판정은 불충족이므로, 같은 행의 $\bigcirc$ 칸들은 판정에 영향을주지 않는다. $\bigcirc$ 를 적는 것은 **어느 조건에서 걸렸는지를 정확히남기기 위해서**다 — “sleep-time compute 이 아니다” 와 “아무것도 하지않는다” 는 전혀 다른 진술이다.

A.3 표 A-1 — 전수표 (29 편)

표 A-1 — corpus 29 편에 대한 판별식 네 조건, 갱신 층, 담당 장

논문	arXiv / venue	(1) 시계	(2) 예산	(3) 대상	(4) 소 비자	판정	갱신 층	담당 장
Letta Sleep- time Compute	2504. 13171	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	통과	E	ch14
MemGPT	2310. 08560	$\times$	$\bigcirc$	$\bigcirc$	$\bigcirc$	경계 사례	E	ch13
Mem0	2504. 19413	$\bigcirc$	$\bigcirc$	$\bigcirc$	$\bigcirc$	통과	E	ch15

논문	arXiv / venue	(1) 시계	(2) 예산	(3) 대상	(4) 소 비자	판정	갱신 층	담당 장
Zep	2501.13956	○	○	○	○	통과	E	ch15
Reasoning-Bank	2509.25140	○	○	○	○	통과	E	ch15
SCM	2604.20943	○	○	○	○	통과	E	ch15
Multi-Timescale (Memini)	2605.105097 · ICML 2026 Workshop	○	○	○	○	통과	E	ch15
Do LMs Need Sleep?	2605.26099	○	○	○	○	통과	W	ch17
SEAL	2506.10943 · NeurIPS 2025	○	○	○	○	통과	Θ	ch20
LM Need Sleep	2606.03979	○	○	○	○	통과	$\Theta (\leftarrow W)$	ch21
Nested Learning	2512.24695 · NeurIPS 2025	$\times$	○	○	$\times$	불충족	없음 — 갱신 시 계가 전 부 토큰	ch18
Memory Caching	2602.24281	$\times$	$\times$	$\times$	$\times$	불충족	없음 — 읽기식 을 바 꾼다	ch18
Memory Layers	2412.09764	$\times$	○	$\times$	○	불충족	없음 — 배포 후 정지	ch09 · ch19
Generative Adapter	2411.05877	$\times$	○	○	○	불충족	W — 단 시 계가 wake	ch16
LoRA	2106.09685	$\times$ 시계 부재	○	○	—	불충족	Θ — 시계 미 규정	ch03 · ch19

논문	arXiv / venue	(1) 시계	(2) 예산	(3) 대상	(4) 소 비자	판정	갱신 층	담당 장
ROME	2202.05262 · NeurIPS 2022	$\times$ 시계 부재	○	○	—	불충족	Θ — 시계 미 규정	ch19
MEMIT	2210.07229 · ICLR 2023	$\times$ 시계 부재	○	○	—	불충족	Θ — 시계 미 규정	ch19
RAG	2005.11401	○	$\times$	○	○	불충족	E — 단 $B_s = 0$	ch10 · ch13
Mem- oriz- ing Trans- form- ers	2203.08913 · ICLR 2022	$\times$	$\times$	○	○	불충족	E — 학습 중 캐시 적재	ch12
EWC	1612.00796	—	—	—	—	대상 아님 (N1)	Θ (질 의 좌표 없음)	ch07 · ch22
Deep Gen- era- tive Re- play	1705.08690 · NIPS 2017	—	—	—	—	대상 아님 (N1)	Θ (질 의 좌표 없음)	ch07 · ch22
GEM	1706.08840 · NIPS 2017	—	—	—	—	대상 아님 (N1)	Θ (질 의 좌표 없음)	ch07 · ch22
Can a LM Learn Facts Con- tinu- ally	2607.11020	—	—	—	—	대상 아님 (N1)	Θ (질 의 좌표 없음)	ch22
model col- lapse	2404.01413	—	—	—	—	대상 아님 (N2)	해당 없음	ch06 · ch22
How much do LMs mem- orize	2505.24832	—	—	—	—	대상 아님 (N2)	해당 없음	ch09 · ch22

논문	arXiv / venue	(1) 시계	(2) 예산	(3) 대상	(4) 소 비자	판정	갱신 층	담당 장
LoRA as Knowl- edge Mem- ory Rate – Dis- tor- tion Mem- ory Com- paction	2603. 01097 · ICML 2026	—	—	—	—	대상 아님 (N2)	해당 없음	ch09 · ch19
PLOS Sleep	2607. 08032 · PLoS Com- put Biol 18(11):e1010628	—	—	—	—	대상 아님 (N3)	해당 없음	ch08 · ch23
PAD	eLife e76384	—	—	—	—	대상 아님 (N3)	해당 없음	ch08 · ch23

MemGPT는 조건 (1)을 만족하지 않는다. ch11 § 11.5의 경계 사례 규칙에 따라 A 군 표에 신
되 통과군의 일원으로 세지 않는다. 이 책의 집계는 언제나 **통과 9 편 + 경계 사례 1 편 + 불충족 19
편 (B 군 9 + C 군 10) = 29 편**이며, “통과 10 편”이라는 셈은 쓰지 않는다. 판별식을 네 조건 전부
로 정의한 이상 셋만 만족하는 논문을 통과로 세면 판별식이 무의미해지기 때문이다.

Memory Caching의 네 칸이 모두 $\times$ 인 근거는 두 곳에 나뉘어 있다. ch11 표 11-2는 대표 탈
락 조건으로 (2)·(3)을 지목하고, ch11 § 11.4.2가 (1)·(4)의 불성립을 추가로 확인한다.

C군의 판정 사유 코드 — **N1**: 지속학습 일반이며 “질의 전/ 이후 질의”라는 좌표계 자체가 없
다. **N2**: 기제가 아니라 제약·이론이며 갱신 절차를 제안하지 않는다. **N3**: 생물 실증이며 대상이 언
어모델 서빙 절차가 아니다.

A.4 표 A-2 — A 군 상세

표 A-2 — A 군: 통과 9 편과 경계 사례 1 편 (MemGPT)이 무엇을 어느 주기로 쓰는가

논문	갱신 층	시계	쓰기의 모양
Letta Sleep-time Compute	E	질의 전 1 회	wr 이 문맥 블록을 새 문자열로 파괴적으로 덮어쓴다
MemGPT	E	대화 중	함수 호출을 디코딩해 store 를 고친다. 답을 만드는 것과 같은 예산 에서 나간다

논문	갱신 층	시계	쓰기의 모양
Mem0	E	메시지쌍마다	$wr \in \{\text{ADD, UPDATE, DELETE, NOOP}\}$ 를 판단한다
Zep	E	에피소드마다 + 주기적 community refresh	이중 시간그래프. wr 이 비파괴적이다
ReasoningBank	E	과제 종료 후	wr 이 순수합집합 이다 — 정리도 감쇠도 없다
SCM	E	대화 후, 상태 의존 트리거	버퍼엔트로피 · 충돌 밀도 · 경과 시간 중 하나가 임계를넘으면 사이클이 돈다. 사이클 안에 LLM 호출이 없다
Multi-Timescale (Memini)	E	이벤트마다	저장소 전체가 선형 필터뱅크다. 유일 실험에서 LLM을 쓰지 않았다
Do LMs Need Sleep?	W	KV cache를 비우기 전 N 회 재귀	W -경로의 유일한 정면 사례 다
SEAL	Θ	두 시간척도	자기 생성 self-edit을 학습 집합으로삼는다
LM Need Sleep	$\Theta (\leftarrow W)$	sleep 라운드 k	wake의 W 를 sleep의 Θ 로 넘기는 순서 가 기제다

A 군 안에서도 “질의 전”의 주기가 **논문마다 다르다**. 이 이질성이상각식 (A)에 그대로 얹힌다 — 분모 N_q 가 세는 대상이 논문마다달라지므로, 통과군에 속한다는 사실만으로 두 논문의 상각 논증을 나란히비교하면 안 된다 ($\rightarrow$ ch11 § 11.3.1, ch25).

A.5 표 A-3 — B군상세

표 A-3 — B군 9편: 탈락 조건과 실제로 일어나는 일

논문	탈락 조건	실제로 일어나는 일	이 책이 이 논문에서 가져가는 것
Nested Learning	(1), (4)	모든 갱신 시계가 토큰이고, 문맥이 끝나면상태가 소멸한다	연속체가 표기의 다리이지 기제의 다리가아니 라는 반례
Memory Caching	(2), (3) — § 11.4.2에서 (1), (4)도	$B_s = 0$. 갱신식이아니라 읽기식을 바꾼다. 세그먼트 경계는 갱신이 아니라 체크포인트 보존이다	W 층 용량 천장 의 차수
Memory Layers	(1), (3)	pre-training 1회. 배포 후 어느 층도 바뀌지않는다	Θ 층 용량 의 실측 공급원

논문	탈락 조건	실제로 일어나는 일	이 책이 이 논문에서 가져가는 것
Generative Adapter	(1)	갱신은 (U-W) 이고 시계는 wake 다. 산출물만 Θ 모양이다	W -경로와 Θ -경로사이의 경첩
LoRA	(1) 시계 부재	쓰기 primitive 이지 시계를 가진 절차가 아니다	Θ 에 쓰는 도구와 delta 아티팩트의 크기
ROME	(1) 시계 부재	직접 편집. 저자가 실용적 방법으로의도하지 않았다고 명시한다	편집 primitive 와 그 상한
MEMIT	(1) 시계 부재	같은 편집을 대량으로 확장한 판	대량 편집의상한
RAG	(2)	$\hat{c}$ 가 항등사상이다. 저장 전에 추론이 없다	E 층의형태와 읽기 비용의 분해
Memorizing Transformers	(1), (2)	쓰기가 forward pass 중의 (k, v) 캐시 적 재다	저장되는 것이 요약이 아니라 원본 텐서일 때 의상한

시계 부재 3 편 (LoRA·ROME·MEMIT) 은 다른탈락과 성격이 다르다. 이들은 “무엇을 어디에 어떻게 쓰는가” 를 완전히규정하지만 “언제 쓰는가” 를 규정하지 않으므로 판정이 성립하지 않는다. 이책은 이들에게 쓰기 **primitive** 라는 별도의 지위를 주고, 어느시계에 엮을지의 선택이 곧 Θ -경로 논문들의 설계라고 본다 ($\rightarrow$ ch11 § 11.2.4, ch19).

A.6 표 A-4 — C 군상세

표 A-4 — C 군 10 편: 왜 판정 대상이 아니며 무엇을 공급하는가

논문	사유	왜 대상이 아닌가	무엇을 공급하는가
EWC	N1	지속학습 일반. “질의 전” 이라는 좌표계 자체가 없다	망각대응의 원형 — 정규화 계열
Deep Generative Replay	N1	“	replay 의 원형 — 생성 replay 계열
GEM	N1	“	성능 행렬 Π 와 ACC·BWT·FWT 의 출처
Can a LM Learn Facts Continually	N1	“	Θ -경로의 반증 후보
model collapse	N2	기제가 아니라 $\mathcal{R}$ 이 자기생성일 때걸리는 제약	자기생성 $\mathcal{R}$ 의 반증 축
How much do LMs memorize	N2	용량 상한 이론	Θ 층 용량의이론 상한
LoRA as Knowledge Memory	N2	저랭크 쓰기의 용량에 관한 실증 분석	delta 아티팩트가 담는 정보량의 상한
Rate – Distortion Memory Compaction	N2	무엇을 버릴 것인가의 이론	압축·망각의 이론 틀. 세 경로를 모두 읽은 유일한문헌

논문	사유	왜 대상이 아닌가	무엇을 공급하는가
PLOS Sleep	N3	생물 실증. 대상이 언어모델 서빙 절차가 아니다	은유의출처와 그 실증적 한계
PAD	N3	//	//

C 군이 판정 대상이 아니라는 것은 **쓸모없다는 뜻이 아니다**. 분류의 목적은 배제가 아니라 각 문헌을 자기가 실제로 증거가 되는 자리에 놓는 것이다 (→ ch11 § 11.8).

A.7 표 A-5 — 집계

표 A-5 — 군별 · 경로별 · 탈락 조건별 집계

구분	값
corpus 전체	29 편
A 군 (판별식 통과)	9 편
경계 사례 (조건 (1) 불만족, 통과군에 세지 않음)	1 편 — MemGPT
B 군 (불충족)	9 편
C 군 (판정 대상 아님)	10 편
통과군의 경로 분포	E 6 편 · W 1 편 · Θ 2 편. 경계 사례를 더하면 E 열만 7 편이 된다
B 군 탈락 조건 (1) 시계	8 편 — Nested Learning, Memory Caching, Memory Layers, Generative Adapter, LoRA, ROME, MEMIT, Memorizing Transformers
B 군 탈락 조건 (2) 예산	3 편 — Memory Caching, RAG, Memorizing Transformers
B 군 탈락 조건 (3) 대상	2 편 — Memory Caching, Memory Layers
B 군 탈락 조건 (4) 소비자	2 편 — Nested Learning, Memory Caching
그중 시계 부재	3 편 — LoRA, ROME, MEMIT
C 군 사유 N1 (질의 좌표 없음)	4 편
C 군 사유 N2 (제약 · 이론)	4 편
C 군 사유 N3 (생물 실증)	2 편
상태 용량을 바이트로 보고한 논문	0 편

경로 분포가 이 부록에서 가장 무거운 숫자다. W -경로는 **표본이 1** 이므로, 경로별 판정에서 W -경로에 대한 진술은 다른 두 경로와 같은강도로 쓸 수 없다 (→ ch27).

A.8 표 A-6 — 담당 장에서찾기

표 A-6 — 장 → 그 장이 정면으로 다루는 논문

장	논문	군
ch03	LoRA	B
ch06	model collapse	C
ch07	EWC, Deep Generative Replay, GEM	C

장	논문	군
ch08	PLOS Sleep, PAD	C
ch09	How much do LMs memorize, Memory Layers, LoRA as Knowledge Memory	C, B, C
ch10	RAG	B
ch12	Memorizing Transformers	B
ch13	MemGPT	A (경계)
ch14	Letta Sleep-time Compute	A
ch15	Mem0, Zep, ReasoningBank, SCM, Multi-Timescale	A
ch16	Generative Adapter	B
ch17	Do LMs Need Sleep?	A
ch18	Nested Learning, Memory Caching	B
ch19	LoRA, ROME, MEMIT	B
ch20	SEAL	A
ch21	LM Need Sleep	A
ch22	Can a LM Learn Facts Continually, Rate – Distortion Memory Compaction	C
ch23	PLOS Sleep, PAD	C

A.9 이 표의 반증조건

이 부록은 판정을 싣는 표이므로 틀릴 수 있는 방식을 명시한다. 반증불가능한 판정은 이 책에서 쓰지 않는다.

1. 네 조건을 모두 만족하는데 B 군 또는 C 군에 놓인 절차가 발견되면 그것은 이 책의 오류이고, 표 A-1의 해당 행을 A 군으로 고쳐야 한다.
2. A 군의 어느 논문에서 갱신 연산자가 실제로는 상태를 바꾸지 않거나, 그 시계가 wake 임이 드러나면 그 행은 B 군으로 내려간다. 특히 SCM은 B_s 가 극단적으로 작으므로 조건 (2)의 부등식이 어디까지 유효한지에 대한 재판정이 걸릴 수 있는 자리다.
3. 경계 사례 규칙의 적용 대상이 MemGPT 외에 나타나면 § A.3의 셈이 바뀐다. 규칙은 “네 조건 중 셋 만족 + 통과군이 조상으로 지목”의 두요건을 함께 요구하며, 탈락 조건의 개수만으로 자격이 주어지지 않는다.
4. 어떤 논문이 상태 용량을 바이트로 보고한 사실이 확인되면 표 A-5의 마지막 행이 틀린다. 이 행은 corpus 전수 확인의 결과이며, corpus 밖 문헌은 이 셈에 들어가지 않는다.

판정의 근거와 다섯 개의 대표 사례는 ch11이 낸다. 이 부록의 어느 행이 의심스러우면 먼저 ch11 § 11.4의 해당 절을 보고, 그다음 § 90.1 – § 90.3의 원문식별자로 원문을 확인한다.